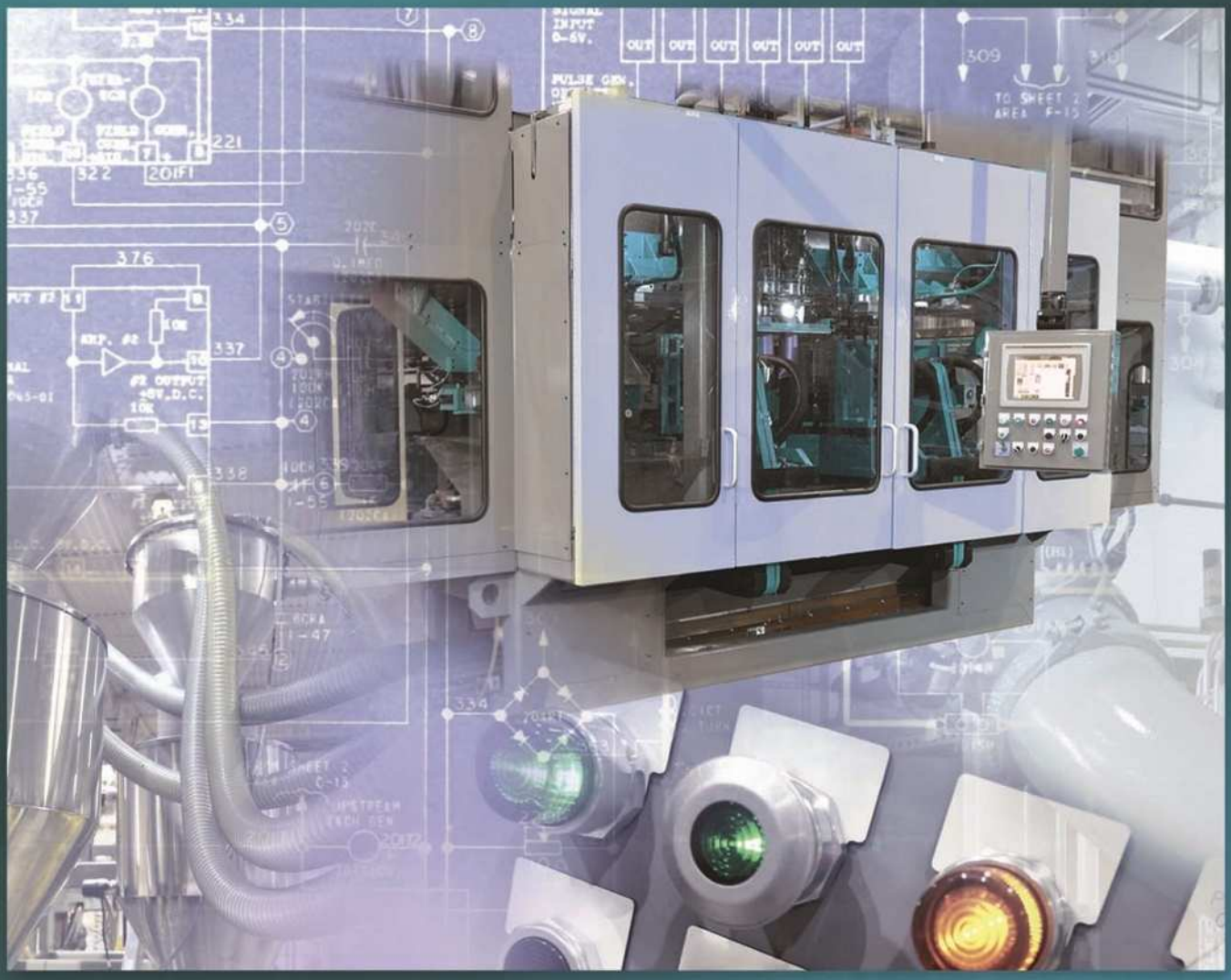
Programmable Logic Controllers

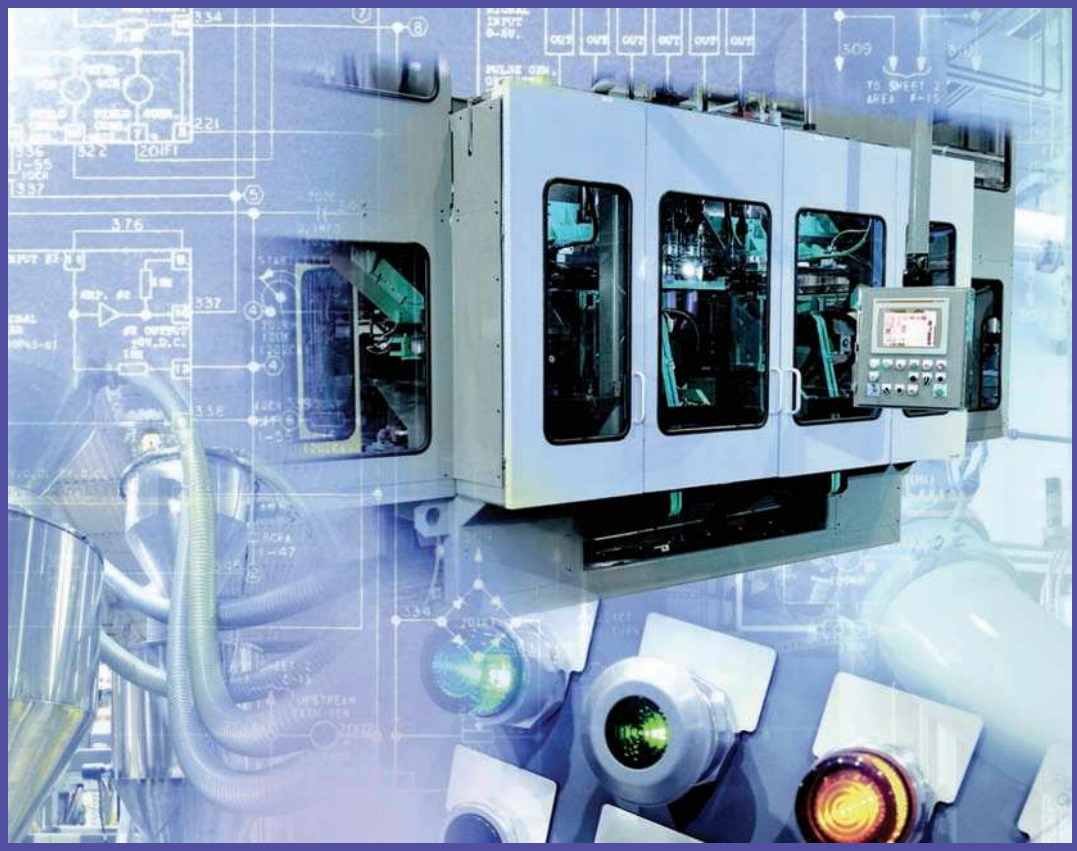
FIFTH EDITION



Frank Petruzella

Mc Graw Hill Education

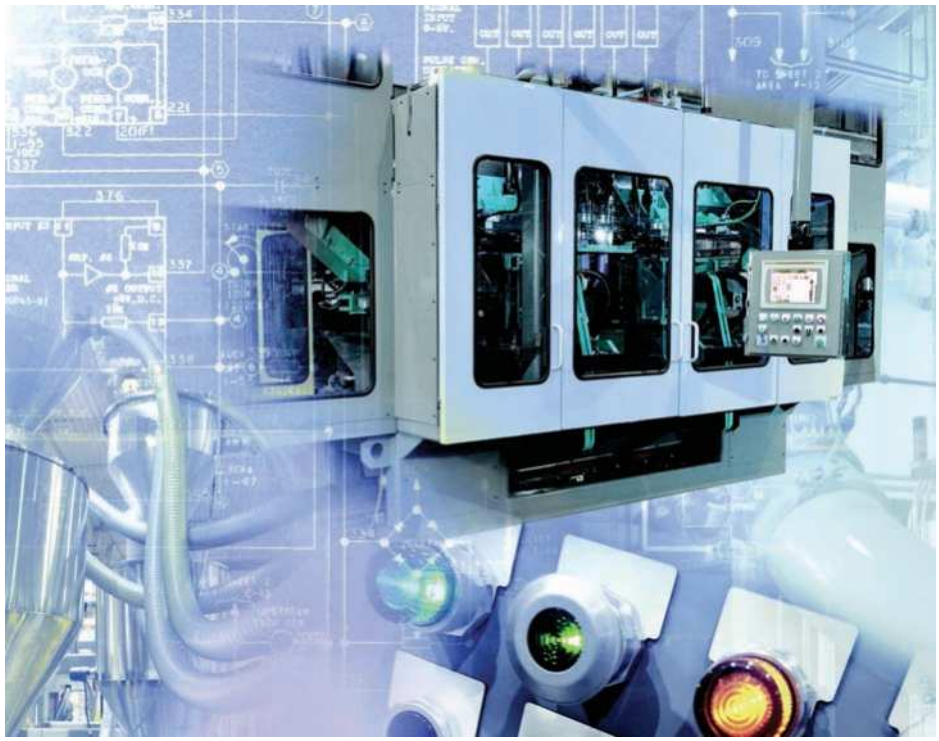
Programmable Logic Controllers



This page intentionally left blank

Programmable Logic Controllers

Fifth Edition



Frank D. Petruzella

Mc Graw Hill Education

PROGRAMMABLE LOGIC CONTROLLERS, FIFTH EDITION

Published by McGraw-Hill Education, 2 Penn Plaza, New York, NY 10121. Copyright © 2017 by McGraw-Hill Education. All rights reserved. Printed in the United States of America. Previous editions © 2011, 2005, 1998. No part of this publication may be reproduced or distributed in any form or by any means, or stored in a database or retrieval system, without the prior written consent of McGraw-Hill Education, including, but not limited to, in any network or other electronic storage or transmission, or broadcast for distance learning.

Some ancillaries, including electronic and print components, may not be available to customers outside the United States.

This book is printed on acid-free paper.

1 2 3 4 5 6 7 8 9 0 RMN/RMN 1 0 9 8 7 6

ISBN 978-0-07-337384-3
MHID 0-07-337384-2

Senior Vice President, Products & Markets: *Kurt L. Strand*
Vice President, General Manager, Products & Markets: *Marty Lange*
Vice President, Content Design & Delivery: *Kimberly Meriwether David*
Managing Director: *Thomas Timp*
Global Brand Manager: *Raghu Srinivasan*
Director, Product Development: *Rose Koos*
Product Developer: *Vincent Bradshaw*
Marketing Manager: *Nick McFadden*
Digital Product Developer: *Amy Bumbaco, Ph.D.*
Director, Content Design & Delivery: *Linda Avenarius*
Executive Program Manager: *Faye M. Herrig*
Content Project Managers: *Jessica Portz, Tammy Juran, Sandra Schnee*
Buyer: *Laura M. Fuller*
Content Licensing Specialist: *Lorraine Buczek*
Compositor: *MPS Limited*
Printer: *R. R. Donnelley*

All credits appearing on page or at the end of the book are considered to be an extension of the copyright page.

Library of Congress Cataloging-in-Publication Data

Petruzella, Frank D., author.
Programmable logic controllers / Frank D. Petruzella.—Fifth edition.
pages cm
Includes index.
ISBN 978-0-07-337384-3 (alk. paper)—ISBN 0-07-337384-2 (alk. paper) 1. Programmable controllers. I. Title.
TJ223.P76P48 2017
629.8’95—dc23

2015035302

The Internet addresses listed in the text were accurate at the time of publication. The inclusion of a website does not indicate an endorsement by the authors or McGraw-Hill Education, and McGraw-Hill Education does not guarantee the accuracy of the information presented at these sites.

mheducation.com/highered

Contents

Preface viii
Acknowledgments xi
About the Author xii

Chapter 1 Programmable Logic Controllers (PLCs): An Overview 1

1.1 Programmable Logic Controllers 2
1.2 Parts of a PLC 4
1.3 Principles of Operation 8
1.4 Modifying the Operation 11
1.5 PLCs versus Computers 11
1.6 PLC Size and Application 12
Review Questions 15
Problems 16

Chapter 2 PLC Hardware Components 17

2.1 The I/O Section 18
2.2 Discrete I/O Modules 22
2.3 Analog I/O Modules 27
2.4 Special I/O Modules 31
2.5 I/O Specifications 33
Typical Discrete I/O Module Specifications . . . *33*
Typical Analog I/O Module Specifications . . . *34*
2.6 The Central Processing Unit (CPU) 35
2.7 Memory Design 36
2.8 Memory Types 37
2.9 Programming Terminal Devices 39
2.10 Recording and Retrieving Data 39
2.11 Human Machine Interfaces (HMIs) 39
Review Questions 43
Problems 45

Chapter 3 Number Systems and Codes 46

3.1 Decimal System 47
3.2 Binary System 47
3.3 Negative Numbers 49
3.4 Octal System 49
3.5 Hexadecimal System 50
3.6 Binary Coded Decimal (BCD) System 51
3.7 Gray Code 53
3.8 ASCII Code 54
3.9 Parity Bit 54
3.10 Binary Arithmetic 55
3.11 Floating Point Arithmetic 57
Review Questions 59
Problems 60

Chapter 4 Fundamentals of Logic 61

4.1 The Binary Concept 62
4.2 AND, OR, and NOT Functions 62
The AND Function *62*
The OR Function *63*
The NOT Function *64*
The Exclusive-OR (XOR) Function *65*
4.3 Boolean Algebra 65
4.4 Developing Logic Gate Circuits from Boolean Expressions 66
4.5 Producing the Boolean Equation for a Given Logic Gate Circuit 66
4.6 Hardwired Logic versus Programmed Logic . 67
4.7 Programming Word Level Logic Instructions . 70
Review Questions 72
Problems 72

Chapter 5 Basics of PLC Programming 74

5.1 Processor Memory Organization 75
Program Files *75*
Data Files *75*
5.2 Program Scan 78
5.3 PLC Programming Languages 81
5.4 Bit-Level Logic Instructions 83
5.5 Instruction Addressing 86
5.6 Branch Instructions 87
5.7 Internal Relay Instructions 89
5.8 Programming Examine If Closed and Examine If Open Instructions 90
5.9 Entering the Ladder Diagram 91
5.10 Modes of Operation 93
5.11 Connecting with Analog Devices 93
Review Questions 95
Problems 96

Chapter 6 Developing Fundamental PLC Wiring Diagrams and Ladder Logic Programs 98

6.1 Electromagnetic Control Relays . . . 99
6.2 Contactors . . . 100
6.3 Motor Starters . . . 101
6.4 Manually Operated Switches . . . 102
6.5 Mechanically Operated Switches . . . 103
6.6 Sensors . . . 104
Proximity Sensor . . . *104*
Magnetic Reed Switch . . . *107*
Light Sensors . . . *107*
Ultrasonic Sensors . . . *109*
Strain/Weight Sensors . . . *110*
Temperature Sensors . . . *110*
Flow Measurement . . . *111*
Velocity and Position Sensors . . . *111*
6.7 Output Control Devices . . . 112
6.8 Seal-In Circuits . . . 114
6.9 Electrical Interlocking Circuits . . . 115
6.10 Latching Relays . . . 116
6.11 Converting Relay Schematics into PLC Ladder Programs . . . 121
6.12 Writing a Ladder Logic Program Directly from a Narrative Description . . . 124
6.13 Instrumentation . . . 127
Review Questions . . . 128
Problems . . . 129

Chapter 7 Programming Timers 131

7.1 Mechanical Timing Relays . . . 132
7.2 Timer Instructions . . . 134
7.3 On-Delay Timer Instruction . . . 135
7.4 Off-Delay Timer Instruction . . . 140
7.5 Retentive Timer . . . 144
7.6 Cascading Timers . . . 147
Review Questions . . . 151
Problems . . . 151

Chapter 8 Programming Counters 156

8.1 Counter Instructions . . . 157
8.2 Up-Counter . . . 159
One-Shot Instruction . . . *162*
8.3 Down-Counter . . . 166
8.4 Cascading Counters . . . 170
8.5 Incremental Encoder-Counter Applications . . . 173
8.6 Combining Counter and Timer Functions . . . 174
8.7 High-Speed Counters . . . 177
Review Questions . . . 179
Problems . . . 179

Chapter 9 Program Control Instructions 184

9.1 Program Control . . . 185
9.2 Master Control Reset Instruction . . . 185
9.3 Jump Instruction . . . 188
9.4 Subroutine Functions . . . 190
9.5 Immediate Input and Immediate Output Instructions . . . 193
9.6 Forcing External I/O Addresses . . . 195
9.7 Safety Circuitry . . . 197
9.8 Selectable Timed Interrupt . . . 200
9.9 Fault Routine . . . 201
9.10 Temporary End Instruction . . . 201
9.11 Suspend Instruction . . . 202
Review Questions . . . 203
Problems . . . 203

Chapter 10 Data Manipulation Instructions 207

10.1 Data Manipulation . . . 208
10.2 Data Transfer Operations . . . 208
10.3 Data Compare Instructions . . . 216
10.4 Data Manipulation Programs . . . 221
10.5 Numerical Data I/O Interfaces . . . 224
10.6 Closed-Loop Control . . . 226
Review Questions . . . 230
Problems . . . 231

Chapter 11 Math Instructions 234

11.1 Math Instructions . . . 235
11.2 Addition Instruction . . . 236
11.3 Subtraction Instruction . . . 238
11.4 Multiplication Instruction . . . 239
11.5 Division Instruction . . . 240
11.6 Other Word-Level Math Instructions . . . 242
11.7 File Arithmetic Operations . . . 245
Review Questions . . . 247
Problems . . . 248

Chapter 12 Sequencer and Shift Register Instructions 252

12.1 Mechanical Sequencers . . . 253
12.2 Sequencer Instructions . . . 255
12.3 Sequencer Programs . . . 259
12.4 Bit Shift Registers . . . 264
12.5 Word Shift Operations . . . 272
Review Questions . . . 277
Problems . . . 277

Chapter 13 PLC Installation Practices, Editing, and Troubleshooting 281

13.1 PLC Enclosures . . . 282
13.2 Electrical Noise . . . 284
13.3 Leaky Inputs and Outputs . . . 285
13.4 Grounding . . . 285
13.5 Voltage Variations and Surges . . . 287
13.6 Program Editing and Commissioning . . . 288
13.7 Programming and Monitoring . . . 289
13.8 Preventive Maintenance . . . 291
13.9 Troubleshooting . . . 292
Processor Module . . . *292*
Input Malfunctions . . . *292*
Output Malfunctions . . . *294*
Ladder Logic Program . . . *294*
13.10 PLC Programming Software . . . 299
Review Questions . . . 302
Problems . . . 302

Chapter 14 Process Control, Network Systems, and SCADA 305

14.1 Types of Processes . . . 306
14.2 Structure of Control Systems . . . 308
14.3 On/Off Control . . . 310
14.4 PID Control . . . 311
14.5 Motion Control . . . 315
14.6 Data Communications . . . 316
Data Highway . . . *322*
Serial Communication . . . *322*
DeviceNet . . . *322*
ControlNet . . . *325*
EtherNet/IP . . . *325*
Modbus . . . *326*
Fieldbus . . . *326*
PROFIBUS-DP . . . *326*
14.7 Supervisory Control and Data Acquisition (SCADA) . . . 328
Review Questions . . . 331
Problems . . . 332

Chapter 15 ControlLogix Controllers 333

Part 1 Memory and Project Organization . . . 334
Memory Layout . . . 334
Configuration . . . 334
Project . . . 335
Tasks . . . 336
Programs . . . 336
Routines . . . 337
Tags . . . 337
Structures . . . 340
Creating Tags . . . 341
Monitoring and Editing Tags . . . 342
Array . . . 342
Review Questions . . . 344
Part 2 Bit-Level Programming . . . 345
Program Scan . . . 345
Creating Ladder Logic . . . 346
Tag-Based Addressing . . . 347
Adding Ladder Logic to the Main Routine . . . 348
Internal Relay Instructions . . . 350
Latch and Unlatch Instructions . . . 352
One-Shot Instruction . . . 353
Review Questions . . . 356
Problems . . . 356
Part 3 Programming Timers . . . 358
Timer Predefined Structure . . . 358
On-Delay Timer (TON) . . . 359
Off-Delay Timer (TOF) . . . 362
Retentive Timer On (RTO) . . . 364
Cascading of Timers . . . 365
Review Questions . . . 367
Problems . . . 367
Part 4 Programming Counters . . . 368
Counters . . . 368
Count-Up (CTU) Counter . . . 369
Count-Down (CTD) Counter . . . 371
Combining Counter and Timer Functions . . . 372
Review Questions . . . 373
Problems . . . 373
Part 5 Math, Comparison, and Move Instructions . . . 374
Math Instructions . . . 374
Comparison Instructions . . . 376
Move Instructions . . . 379
Combining Math, Comparison, and Move Instructions . . . 380
Review Questions . . . 383
Problems . . . 383
Part 6 Function Block Programming . . . 384
Function Block Diagram (FBD) . . . 384
FBD Programming . . . 388
Review Questions . . . 394
Problems . . . 394

Glossary . . . 395
Index . . . 407

Preface

Programmable logic controllers (PLCs) continue to evolve as new technologies are added to their capabilities. As PLC technology has advanced, so have programming languages and communications capabilities. Today's PLCs offer faster scan times, space efficient high-density input/output systems, and special interfaces to allow non-traditional devices to be attached directly to the PLC.

Now in its Fifth Edition, changes made to the content of the text have been made **solely** based on reviews from current instructors and include:

- material that should be added or deleted from chapters
- topics requiring more in-depth coverage
- increased integration of the ControlLogix platform of controllers
- chapter modifications require to meet current curriculum needs

The primary source of information for a particular PLC is always the accompanying user manuals provided by the manufacturer. This textbook is not intended to replace the vendor's reference material, but rather to complement, clarify, and expand on this information. The text covers the basics of programmable logic controllers in a manner that complements instruction with a SLC-500 or ControlLogix platform. The underlying PLC principles and concepts covered in the text are common to most manufacturers. They serve to maximize the knowledge gained through on-the-job training and programs offered by different vendors.

The text is written in an easy-to-read style that is designed for students with no prior PLC experience. For example, when the operation of a program is called for, a bulleted list is used to summarize its execution. The bulled list replaces a lengthy paragraph and is especially helpful when covering the different steps related to the execution of a program.

Each chapter begins with a brief introduction outlining chapter coverage and learning objectives. When applicable, the relay equivalent of the virtual programmed instruction is explained first, followed by the appropriate PLC instruction. Chapters conclude with a set of review questions and problems. The review questions are closely related to the chapter objectives and require students to recall and apply information covered in the chapter. The problems range from easy to difficult, thus challenging students at various levels of competence.

Features new to the Fifth Edition include:

- **Key concepts** and terms are **highlighted** in bold the first time they appear.
- New/updated **photos** and **line art** for every **chapter**.
- **New topics** for every **chapter** as requested by reviewers.
- Addition review **questions** for new topics.
- Updated instructor **PowerPoint** lessons.
- More than **175** SLC-500 and ControlLogix program simulation **videos tied directly** to the programs studied in the text

  In addition, students who are using McGraw-Hill's Connect can watch simulated, step-by-step execution of numerous ladder logic programming examples. They're **guided** by an audio commentary that explains what to look for as the program is executed. The videos are part of the Student Resources section of Connect.

Chapter changes in this edition include:

Chapter 1

- Testing of field devices.
- Extended coverage of scan cycle sequence.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 2

- ControlLogix Base and Alias addressing.
- Extended coverage of DC module Sinking and Sourcing.
- Analog module input sensor 2-, 3-, and 4-wire connections.
- Scaling of PLC analog inputs and outputs.
- Extended coverage of Human Machine Interfaces (HMIs)
- Additional chapter review questions.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 3

- 16 bit 2's complement.
- Floating point arithmetic.
- Additional chapter problems.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 4

- Modification to hardwired programming examples
- Additional test bank questions.
- Additional chapter review questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 5

- Electrical versus logical continuity.
- Evaluating XIO and XIC bit instructions.
- Rack-based versus tag-based addressing.
- Connecting with analog devices.
- Additional test bank questions.
- Additional chapter review questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 6

- Magnetic reed float switch.
- Resistance temperature detectors (RTDs).
- Electrical interlocking circuits.
- Process instrumentation.
- Additional test bank questions.
- Additional chapter review questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 7

- Extended coverage of timer instructions.
- ControlLogix timer instruction.
- Reciprocating timers.
- TON timer bit table.
- TOF timer bit table.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 8

- ControlLogix counter instruction.
- Extended coverage of CTD instruction.
- Additional information on incremental encoders.
- New section on High-Speed Counter instruction.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 9

- Extended coverage of MCR instruction.
- Extended coverage of Jump instruction.
- Extended coverage of Immediate Input and Output instructions.
- ControlLogix Immediate Output instruction.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 10

- Extended coverage of the Masked Move instruction.
- New example of a copy instruction program.
- New example of a data compare program.
- ControlLogix Limit Comparison instruction and program.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 11

- Extended coverage of basic math instruction.
- New example of a compute instruction program.
- New coverage Modulo (MOD) instruction.
- New scale analog input using the SCP instruction.
- New scale analog output using the SCP instruction.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 12

- Extended coverage of Sequencer Output (SQO) instruction.
- ControlLogix Sequencer Output (SQO) instruction and program.
- ControlLogix shift registers instruction and program.
- ControlLogix FIFO instruction and program.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 13

- Extended coverage of communications using RSLinx and RSWho.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 14

- SERCOS standard communication for motion control.
- HART communication protocol.
- SCADA alarm monitoring.
- FactoryTalk services platform.
- Additional test bank questions.
- Program video simulations.
- New and modified line diagrams and photos.

Chapter 15

Part 1

- Extended coverage of tag types.
- Program video simulations.
- New and modified line diagrams and photos.

Part 2

- Reversing conveyor motor program and operation.
- Motor pilot light internal relay program and operation.
- Latch/unlatch car wash program and operation.
- One-shot program instructions used in conjunction with math operations.
- Program video simulations.
- New and modified line diagrams and photos.

Part 3

- Cascading TON timers for timed event-driven routines program and operation
- Program video simulations.
- New and modified line diagrams and photos.

Part 4

- Combining Counter and Timer Functions program and operation.
- Program video simulations.
- New and modified line diagrams and photos.

Part 5

- Monitoring the setting of a thumbwheel switch program and operation.
- PLC program for three-speed control of a conveyor system program and operation.
- Conveyor parts tracking program and operation.
- Program video simulations.
- New and modified line diagrams and photos.
- Part 6 Function block parameters tab.
- Program video simulations.
- New and modified line diagrams and photos.

Acknowledgments

I would like to thank the following reviewers for their comments and suggestions:

Noureddine Bekhouche
Jacksonville State University

Mark Bohnet
Northwest Iowa Community College

Michael Buck
Dakota County Technical College

Wayne Buroker
Waukesha County Technical College

Jerry Clark
Northwest Mississippi Community College

Chris Haley
North Georgia Technical College

Garrett Hunter
Western Illinois University

Wael Ibrahim
ECPI University

Ahmed Kamal
Tennessee Tech University

Gholam H. Massiha
University of Louisiana at Lafayette

Randy Owens
Henderson Community College

James Schabowski
Waukesha County Technical College

Jenifer Shannon
Penn State University, Berks Campus

Accounties Lashan Smith
Tri-County Technical College

Kenneth E. Swayne
Pellissippi State Community College

John Veitch
SUNY Adirondack

William Walker
Truckee Meadows Community College

Robert Permenter
Albany Technical College

A special thanks to Don Pelster of Nashville State Community College, for his outstanding work on performing a technical edit of the text and providing us with detailed feedback, suggestions and recommendations.

Frank D. Petruzella

About the Author

Frank D. Petruzella has extensive practical experience in the electrical control field, as well as many years of experience teaching and authoring textbooks. Before becoming a full time educator, he was employed as an apprentice and electrician in areas of electrical installation and maintenance. He holds a Master of Science degree from Niagara University, a Bachelor of Science degree from the State University of New York College–Buffalo, as well as diplomas in Electrical Power and Electronics from the Erie County Technical Institute.

Programmable Logic Controllers **makes it easy to learn PLCs from the ground up! Up-to-the-minute revisions include all the newest developments in programming, installing, and maintaining processes. Clearly developed chapters deliver the organizing objectives, explanatory content with helpful diagrams and illustrations, and closing review problems that evaluate retention of the chapter objectives.**

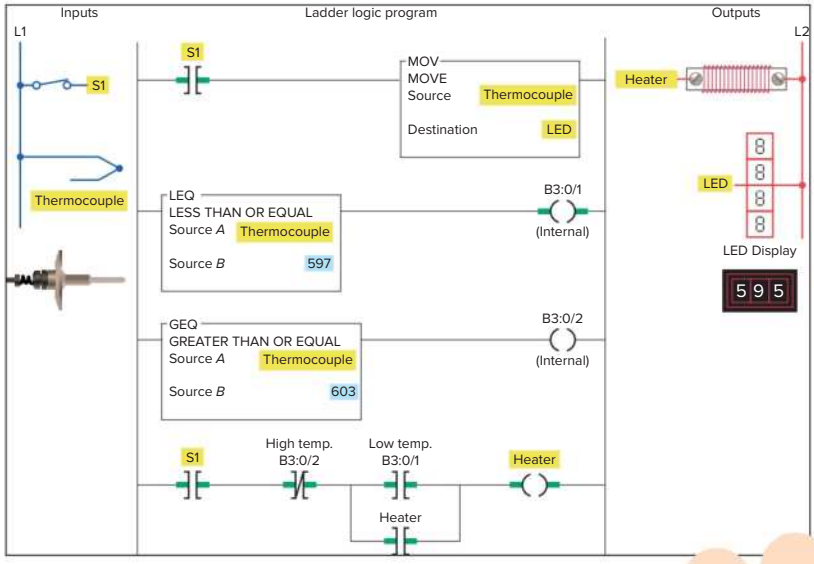
CHAPTER OBJECTIVES overview the chapter, letting students and instructors focus on the main points to better grasp concepts and retain information.

Chapter Objectives

After completing this chapter, you will be able to:

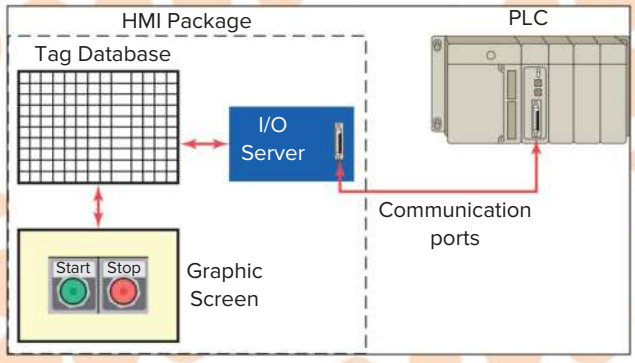
- Describe the operation of pneumatic on-delay and off-delay timers
- Describe PLC timer instruction and differentiate between a nonretentive and retentive timer
- Convert fundamental timer relay schematic diagrams to PLC ladder logic programs
- Analyze and interpret typical PLC timer ladder logic programs
- Program the control of outputs using the timer instruction control bits

Chapter content includes rich illustrative detail and extensive visual aids, allowing students to grasp concepts more quickly and understand practical applications

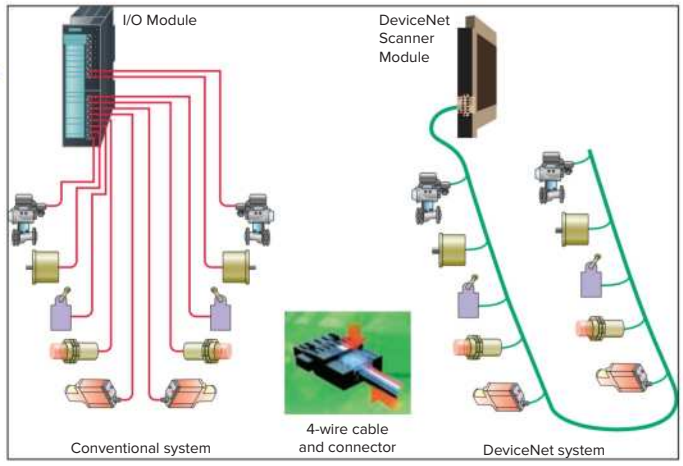


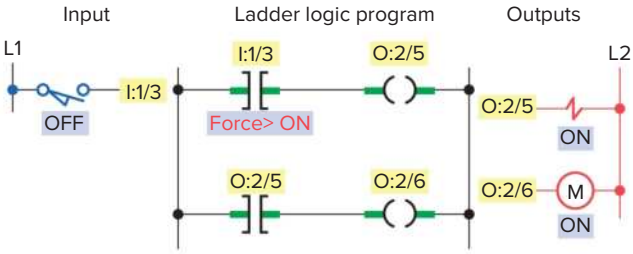
Here, drawings and photos of real-world input and output devices have been included

In Chapter 02, students not only read about but can also see how HMIs fit into an overall PLC system, giving them a practical introduction to the topics



Coverage of communications and control networks utilizes clear graphics to demonstrate how things work





- The processor ignores the actual state of input limit switch I:1/3.
- Although limit switch I:1/3 is off (0 or false) the processor considers it as being in the on (1 or true) state.
- The program scan records this, and the program is executed with this forced status.
- In other words, the program is executed as if the limit switch were actually closed.

BULLETED LISTS break down processes to helpfully summarize execution of tasks

Diagrams, such as this one illustrating an overview of the function block programming language, help students put the pieces together

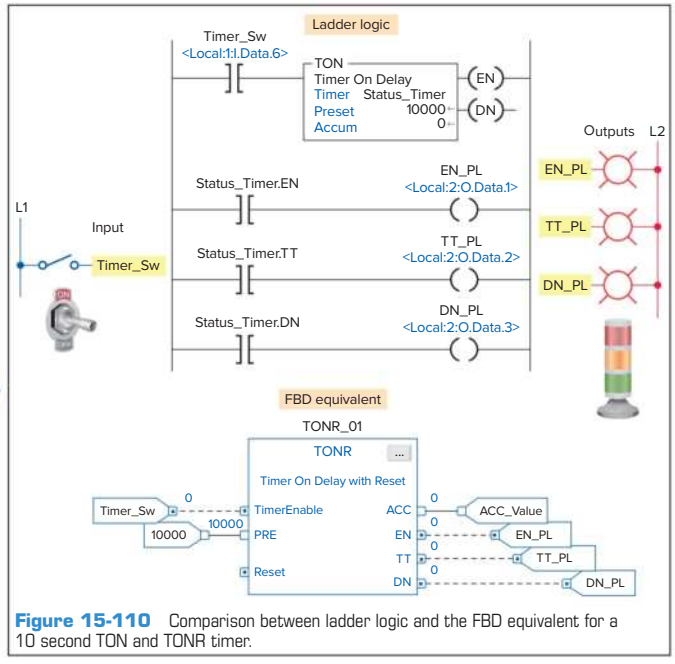
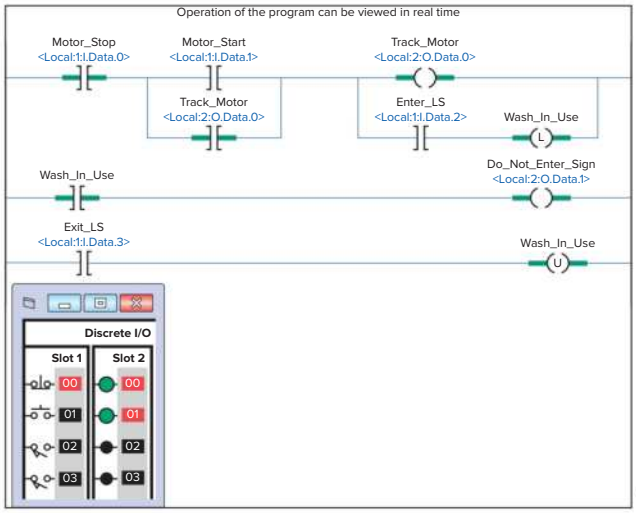


Figure 15-110 Comparison between ladder logic and the FBD equivalent for a 10 second TON and TONR timer.



More than **175** SLC-500 and ControlLogix program simulation **videos tied directly** to the programs studied in the text

END-OF-CHAPTER REVIEWS are structured to reinforce chapter objectives

CHAPTER 6 REVIEW QUESTIONS

1. Explain the basic operating principle of an electromagnetic control relay.
2. What is the operating difference between a normally open and a normally closed relay contact?
3. In what ways are control relay coils and contacts rated?
4. How do contactors differ from relays?
5. What is the main difference between a contactor and a magnetic motor starter?
6. **a.** Draw the schematic for an across-the-line AC magnetic motor starter.
   b. With reference to this schematic, explain the function of each of the following parts:
   - **i.** Main contact M
   - **ii.** Control contact M
   - **iii.** Starter coil M
   - **iv.** OL relay coils
   - **v.** OL relay contact
7. The current requirement for the control circuit of a magnetic starter is normally much smaller than that required by the power circuit. Why?
8. Compare the method of operation of each of the following types of switches:
   a. Manually operated switch
   b. Mechanically operated switch
   c. Proximity switch
15. Compare the operation of the reflective-type and through-beam photoelectric sensors.
16. Give an explanation of how a scanner and a decoder act in conjunction with each other to read a bar code.
17. How does an ultrasonic sensor operate?
18. Explain the principle of operation of a strain gauge.
19. Explain the principle of operation of a thermocouple.
20. What is the most common approach taken with regard to the measurement of fluid flow?
21. Explain how a tachometer is used to measure rotational speed.
22. How does an optical encoder work?
23. Draw an electrical symbol used to represent each of the following PLC control devices:
   a. Pilot light
   b. Relay
   c. Motor starter coil
   d. OL relay contact
   e. Alarm
   f. Heater
   g. Solenoid
   h. Solenoid valve
   i. Motor
   j. Horn
24. Explain the function of each of the following actuators:
   a. Solenoid
   b. Solenoid valve

CHAPTER 6 PROBLEMS

1. Design and draw the schematic for a conventional hardwired relay circuit that will perform each of the following circuit functions when a normally closed pushbutton is pressed:
   - Switch a pilot light on
   - De-energize a solenoid
   - Start a motor running
   - Sound a horn
2. Design and draw the schematic for a conventional hardwired circuit that will perform the following circuit functions using two break-make pushbuttons:
   - Turn on light L1 when pushbutton PB1 is pressed.
   - Turn on light L2 when pushbutton PB2 is pressed.
   - Electrically interlock the pushbuttons so that L1 and L2 cannot both be turned on at the same time.
3. Study the ladder logic program in Figure 6-77, and answer the questions that follow:
   a. Under what condition will the latch rung 1 be true?
   b. Under what conditions will the unlatch rung 2 be true?
   c. Under what condition will rung 3 be true?
   d. When PL1 is on, the relay is in what state (latched or unlatched)?
   e. When PL2 is on, the relay is in what state (latched or unlatched)?
   f. If AC power is removed and then restored to the circuit, what pilot light will automatically come on when the power is restored?
   g. Assume the relay is in its latched state and all three inputs are false. What input change(s) must occur

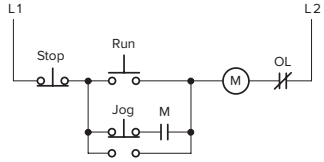
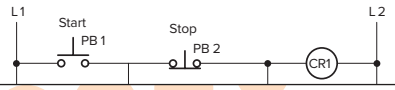


Figure 6-78 Hardwired control circuit for Problem 4.

will correctly execute the hardwired control circuit in Figure 6-78.
Assume: Stop pushbutton used is an NO type.
Run pushbutton used is an NO type.
Jog pushbutton used has one set of NO contacts.
OL contact is hardwired.

5. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program that will correctly execute the hardwired control circuit in Figure 6-79.
Assume: PB1 pushbutton used is an NO type.
PB2 pushbutton used is an NC type.
PS1 pressure switch used is an NO type.
LS1 limit switch used has only one set of NC contacts.

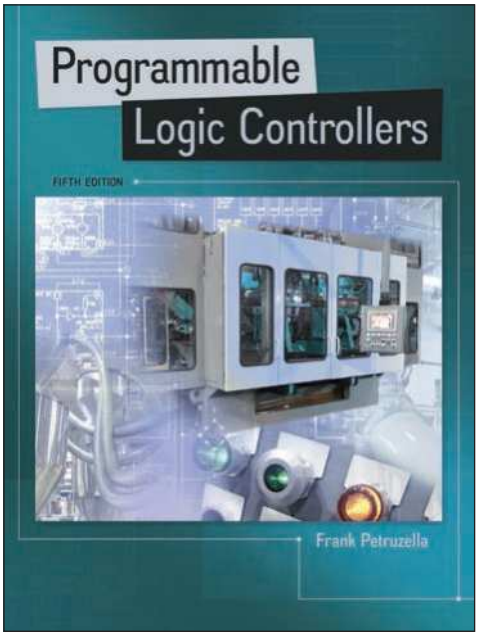


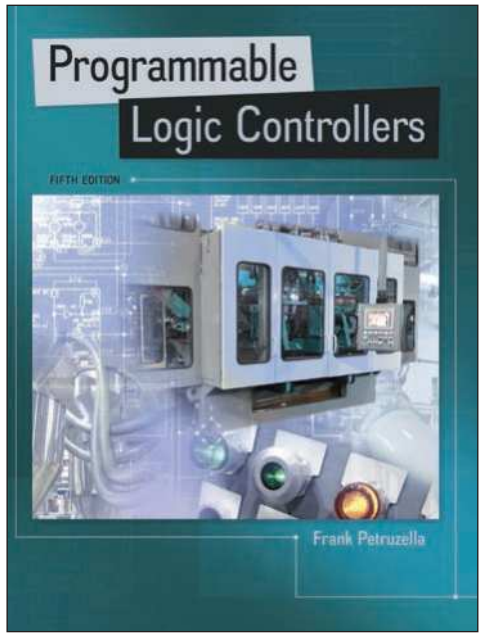
EXAMPLE PROBLEMS help bring home the applicability of chapter concepts

ANCILLARIES THAT WORK

Expanded on and updated from the previous edition, this new edition includes an outstanding instructor support package:

- ExamView and EZ Test question test banks for each chapter.
- PowerPoint lessons with animations that help visualize the actual process.
- Activity Manual contains true/false, completion, matching, and multiple-choice tests for every chapter in the text. So that students get a better understanding of programmable logic controllers, the manual also includes a wide range of programming assignments and additional practice exercises.
- Answers to the questions and problems in the textbook, Activities Manual, and LogixPro Manual. Available on the Instructor Resources section of Connect.





In addition, for students, this edition also has available:

- *LogixPro PLC Lab Manual for use with Programmable Logic Controllers* Fifth Edition, with LogixPro PLC Simulator. This manual contains:
- McGraw-Hill's Connect and Smartbook.
- LogixPro simulations with audio and video for those using Connect.
  - Over 250 LogixPro student lab exercises sequenced to support material covered in the text.

Mc Graw Hill Education

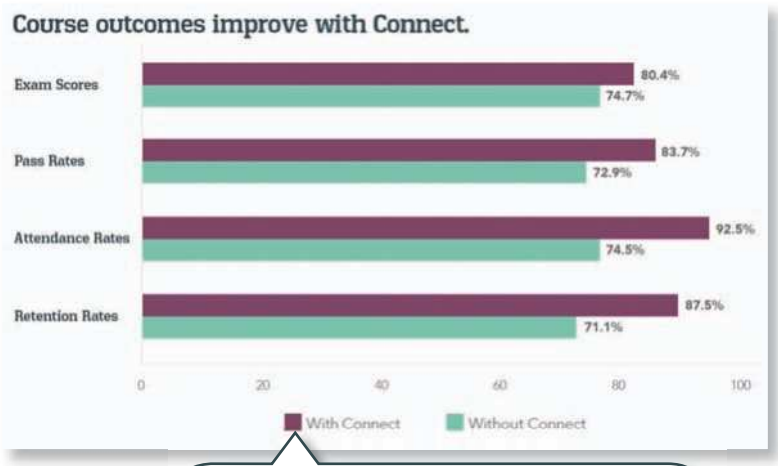
connect®

Required=Results

McGraw-Hill Connect®
Learn Without Limits

Connect is a teaching and learning platform that is proven to deliver better results for students and instructors.

Connect empowers students by continually adapting to deliver precisely what they need, when they need it, and how they need it, so your class time is more engaging and effective.

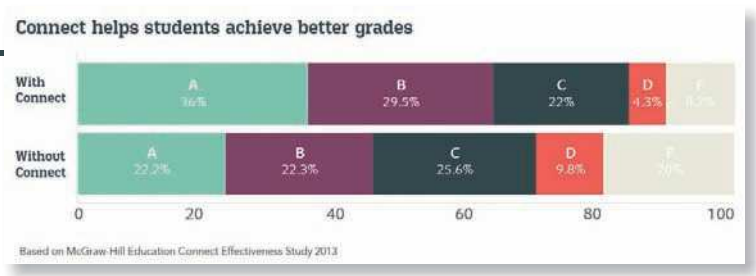


88% of instructors who use **Connect** require it; instructor satisfaction **increases** by 38% when **Connect** is required.

Using **Connect** improves passing rates by **10.8%** and retention by **16.4%.**

Analytics

Connect Insight®



Connect Insight is Connect's new one-of-a-kind visual analytics dashboard—now available for both instructors and students—that provides at-a-glance information regarding student performance, which is immediately actionable. By presenting assignment, assessment, and topical performance results together with a time metric that is easily visible for aggregate or individual results, Connect Insight gives the user the ability to take a just-in-time approach to teaching and learning, which was never before available. Connect Insight presents data that empowers students and helps instructors improve class performance in a way that is efficient and effective.

Students can view their results for any **Connect** course.

Mobile

Connect's new, intuitive mobile interface gives students and instructors flexible and convenient, anytime–anywhere access to all components of the Connect platform.

Adaptive

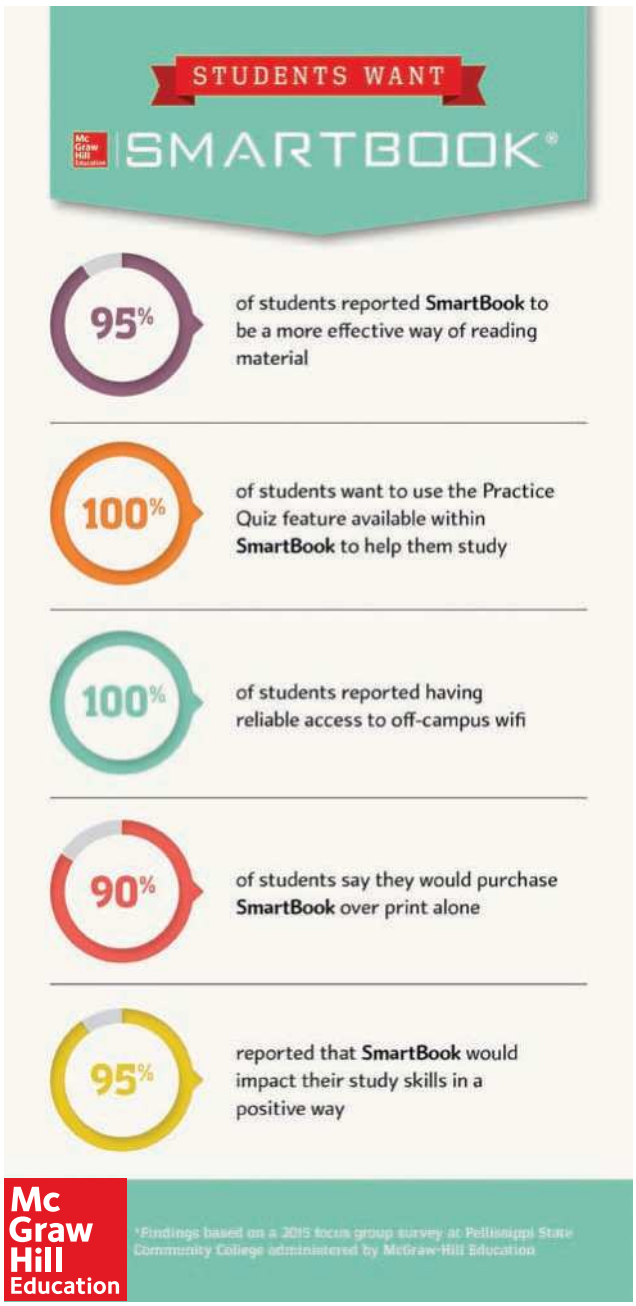
THE FIRST AND ONLY **ADAPTIVE READING EXPERIENCE** DESIGNED TO TRANSFORM THE WAY STUDENTS READ

More students earn **A's** and **B's** when they use McGraw-Hill Education **Adaptive** products.

SmartBook®

Proven to help students improve grades and study more efficiently, SmartBook contains the same content within the print book, but actively tailors that content to the needs of the individual. SmartBook's adaptive technology provides precise, personalized instruction on what the student should do next, guiding the student to master and remember key concepts, targeting gaps in knowledge and offering customized feedback, and driving the student toward comprehension and retention of the subject matter. Available on smartphones and tablets, SmartBook puts learning at the student's fingertips—anywhere, anytime.

Over **4 billion questions** have been answered, making McGraw-Hill Education products more intelligent, reliable, and precise.



1

Programmable Logic Controllers (PLCs)
An Overview

Image Courtesy of Rockwell Automation, Inc.

Chapter Objectives

After completing this chapter, you will be able to:

- Define what a programmable logic controller (PLC) is and list its advantages over relay systems
- Identify the main parts of a PLC and describe their functions
- Outline the basic sequence of operation for a PLC
- Identify the general classifications of PLCs

This chapter gives a brief history of the evolution of the programmable logic controller, or PLC. The reasons for changing from relay control systems to PLCs are discussed. You will learn the basic parts of a PLC, how a PLC is used to control a process, and the different kinds of PLCs and their applications. The ladder logic language, which was developed to simplify the task of programming PLCs, is introduced.

1.1 Programmable Logic Controllers

Programmable logic controllers (Figure 1-1) are now the most widely used industrial process control technology. A **programmable logic controller (PLC)** is an industrial grade computer that is capable of being programmed to perform control functions. The programmable controller has eliminated much of the hardwiring associated with conventional relay control circuits. Other benefits include fast response, easy programming and installation, high control speed, network compatibility, troubleshooting and testing convenience, and high reliability.

The PLC is designed for multiple input and output arrangements, extended temperature ranges, immunity to electrical noise, and resistance to vibration and impact. Programs for the control and operation of manufacturing process equipment and machinery are typically stored in battery-backed or nonvolatile memory. A PLC is an example of a **real-time system** since the output of the system controlled by the PLC depends on the input conditions.

The PLC is, then, basically a digital computer designed for use in machine control. Unlike a personal computer, it has been designed to operate in the industrial environment and is equipped with special input/output interfaces and a control programming language. The common abbreviation used in industry for these devices, PC, can be confusing because it is also the abbreviation for "personal computer." Therefore, most manufacturers refer to their programmable controller as a PLC, which stands for "programmable logic controller."

Initially the PLC was used to replace **relay logic**, but its ever-increasing range of functions means that it is found in many and more complex applications. Because the structure of a PLC is based on the same principles as those employed in computer architecture, it is capable not only of performing relay switching tasks but also of performing other applications such as timing, counting, calculating, comparing, and the processing of analog signals.

Programmable controllers offer several advantages over a conventional relay type of control. Relays have to be hardwired to perform a specific function. When the system requirements change, the relay wiring has to be changed or modified. In extreme cases, such as in the auto industry, complete control panels had to be replaced since it was not economically feasible to rewire the old panels with each model changeover. The programmable controller has eliminated much of the hardwiring associated with conventional relay control circuits (Figure 1-2). It is small and inexpensive compared to equivalent relay-based process control systems. Modern control systems still include relays, but these are rarely used for logic.

PLCs provide many other benefits including:

- ***Increased Reliability.*** Once a program has been written and tested, it can be easily downloaded to other PLCs. Since all the logic is contained in the PLC's memory, there is no chance of making a logic wiring error (Figure 1-3). The program takes the place of much of the external wiring that would normally be required for control of a process. Hardwiring, though still required to connect field devices, is less intensive. PLCs also offer the reliability associated with solid-state components.
- ***More Flexibility.*** It is easier to create and change a program in a PLC than to wire and rewire a circuit. With a PLC the relationships between the inputs and outputs are determined by the user program instead of the manner in which they are interconnected (Figure 1-4). Original equipment manufacturers can provide system updates by simply sending out a new program. End users can modify the program in the field, or if desired, security can be provided by hardware features such as key locks and by software passwords.
- ***Lower Cost.*** PLCs were originally designed to replace relay control logic, and the cost savings have been so significant that relay control is becoming

(*a*) (*b*)

Figure 1-1 Programmable logic controller.
Source: (*a–b*) Courtesy GE Intelligent Platforms.

(*a*)

(*b*)

Figure 1-2 Relay- and PLC-based control panels. (*a*) Relay-based control panel. (*b*) PLC-based control panel.
Source: (*a*) Courtesy Mid-Illini Technical Group, Inc.; (*b*) Photo courtesy Ramco Electric, Ltd.

obsolete except for power applications. Generally, if an application has more than about a half-dozen control relays, it will probably be less expensive to install a PLC.

- ***Communications Capability.*** A PLC can communicate with other controllers or computer equipment to perform such functions as supervisory control, data gathering, monitoring devices and process parameters, and download and upload of programs (Figure 1-5).

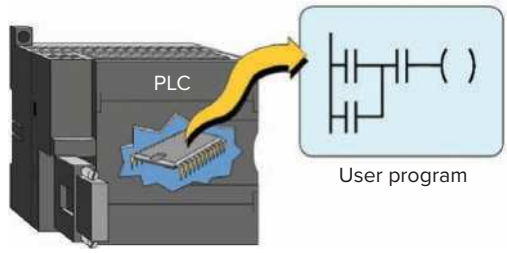


Figure 1-3 All the logic is contained in the PLC's memory.

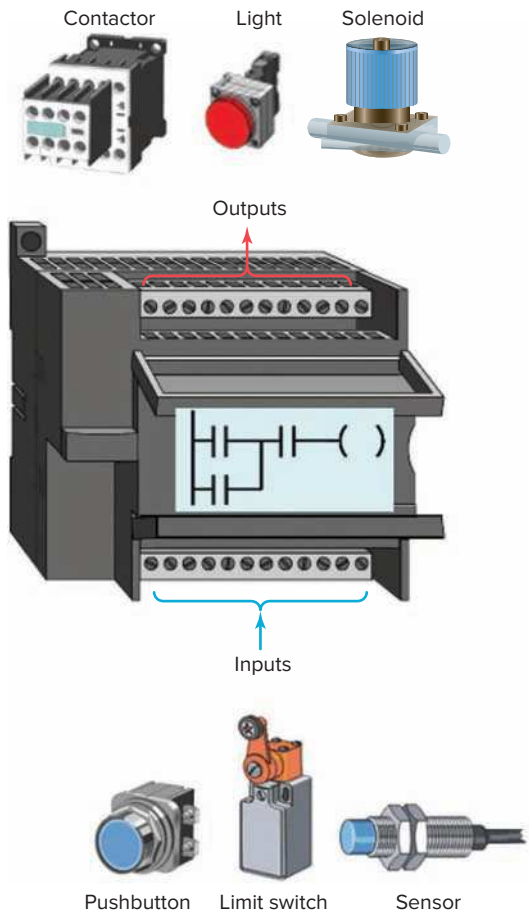


Figure 1-4 Relationships between the inputs and outputs are determined by the user program.

- ***Faster Response Time.*** PLCs are designed for high-speed and real-time applications (Figure 1-6). The programmable controller operates in real time, which means that an event taking place in the field will result in the execution of an operation or output. Machines that process thousands of items per second and objects that spend only a fraction of a second in front of a sensor require the PLC's quick-response capability.
- ***Easier to Troubleshoot.*** PLCs have resident diagnostics and override functions that allow users to easily trace and correct software and hardware

Figure 1-5 PLC communication module.
Source: Photo courtesy Automation Direct, **www.automationdirect.com**.

Figure 1-6 High-speed counting.
Source: Courtesy Banner Engineering Corp.

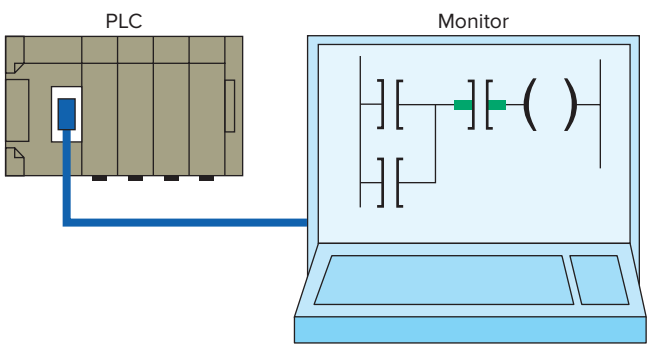


Figure 1-7 Control program can be displayed on a monitor in real time.

problems. To find and fix problems, users can display the control program on a monitor and watch it in real time as it executes (Figure 1-7)·

- **Easier to Test Field Devices.** A PLC control panel has the ability to check field devices at a common point. For example, a control system consisting of hundreds of input and output field devices may be contained within a very large manufacturing area. Thus, it would take a considerable amount of time to check each device at its location. By having each device wired back to a common point on a PLC module, each device could be checked for operation fairly quickly.

1.2 Parts of a PLC

A typical PLC can be divided into parts, as illustrated in Figure 1-8. These are the *central processing unit (CPU)*, the *input/output (I/O)* section, the *power supply,* and the *programming device.* The term *architecture* can refer to PLC hardware, to PLC software, or to a combination of both. An *open* architecture design allows the system to be connected easily to devices and programs made by other manufacturers. Open architectures use off-the-shelf components that conform to approved standards. A system with a *closed* architecture is one whose design is proprietary, making it more difficult to connect to other systems. Most PLC systems are in fact **proprietary**, so you must be sure that any generic hardware or software you may use is compatible with your particular PLC. Also, although the principal concepts are the same in all methods of programming, there might be slight differences in addressing, memory allocation, retrieval, and data handling for different models. Consequently, PLC programs cannot be interchanged among different PLC manufacturers.

There are two ways in which I/Os (Inputs/Outputs) are incorporated into the PLC: fixed and modular. ***Fixed I/O*** (Figure 1-9) is typical of small PLCs that come in one package with no separate, removable units. The processor and I/O are packaged together, and the I/O terminals will have a fixed number of connections built in for inputs and outputs. The main advantage of this type of packaging is lower cost. The number of available I/O points varies and usually can be expanded by buying additional units of fixed I/O. One disadvantage of fixed I/O is its lack of flexibility; you are limited in what you can get in the quantities and types dictated by the packaging. Also, for some models, if any part in the unit fails, the whole unit has to be replaced.

Modular I/O (Figure 1-10) is divided by compartments into which separate modules can be plugged. This feature greatly increases your options and the unit's flexibility. You can choose from the modules available from the manufacturer and mix them any way you desire. The basic modular controller consists of a rack, power supply, processor module (CPU), input/output (I/O modules), and an operator interface for programming and

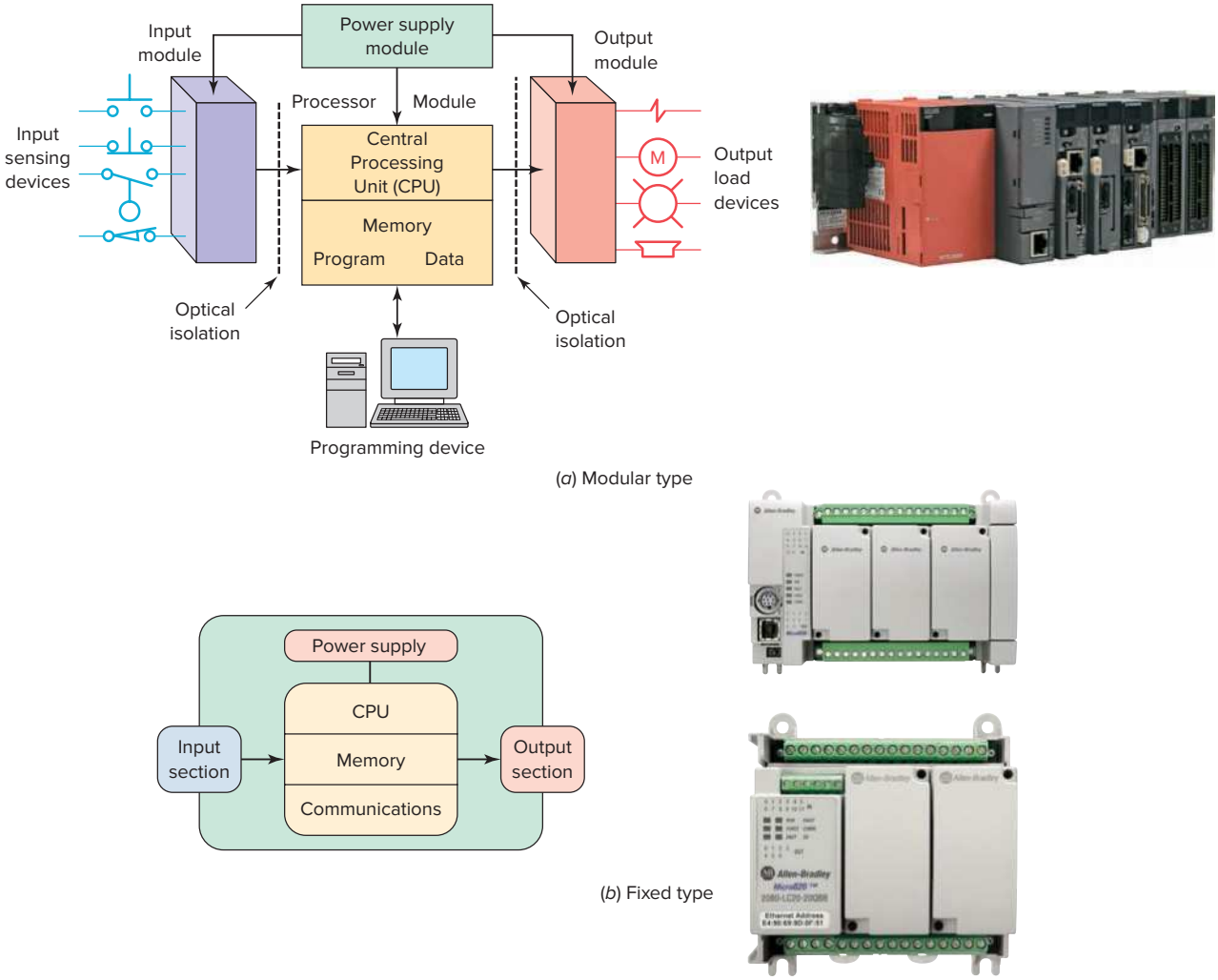


Figure 1-8 Typical parts of a programmable logic controller.
Source: (*a*) Courtesy Mitsubishi Automation; (*b*) Images Courtesy of Rockwell Automation, Inc.

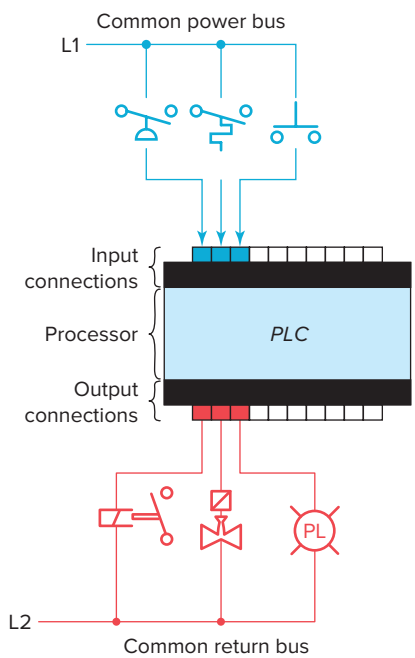


Figure 1-9 Fixed I/O configuration.

monitoring. The modules plug into a rack. When a module is slid into the rack, it makes an electrical connection with a series of contacts called the backplane, located at the rear of the rack. The PLC processor is also connected to the backplane and can communicate with all the modules in the rack.

The ***power supply*** supplies DC power to other modules that plug into the rack (Figure 1-11). For large PLC systems, this power supply does not normally supply power to the field devices. With larger systems, power to field devices is provided by external alternating current (AC) or direct current (DC) supplies. For some small micro PLC systems, the power supply may be used to power field devices.

The ***processor*** **(CPU)** is the "brain" of the PLC. A typical processor (Figure 1-12) usually consists of a microprocessor for implementing the logic and controlling the communications among the modules. The processor requires memory for storing user program instructions, numerical values, and I/O devices status.

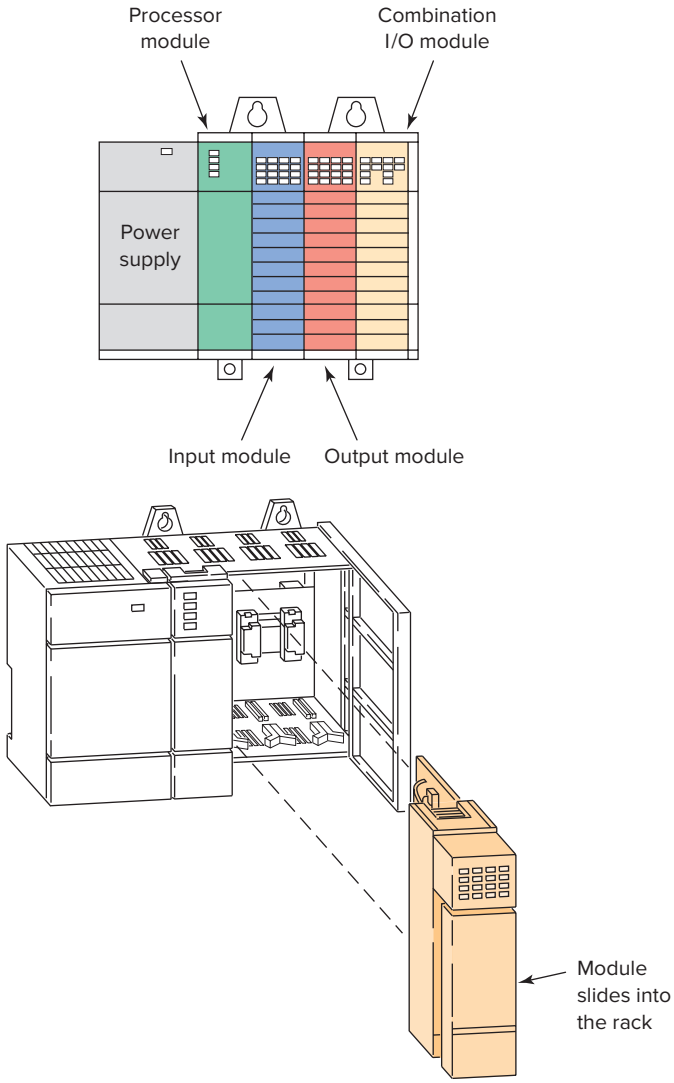


Figure 1-10 Modular I/O configuration.

Figure 1-11 The power supply supplies DC power to other modules that plug into the rack.
Source: Photo of PLC Modicon M340 © Schneider Electric, 2010. **www.schneider-electric.com**.

Figure 1-12 Typical PLC processor modules.
Source: Image Courtesy of Rockwell Automation, Inc.

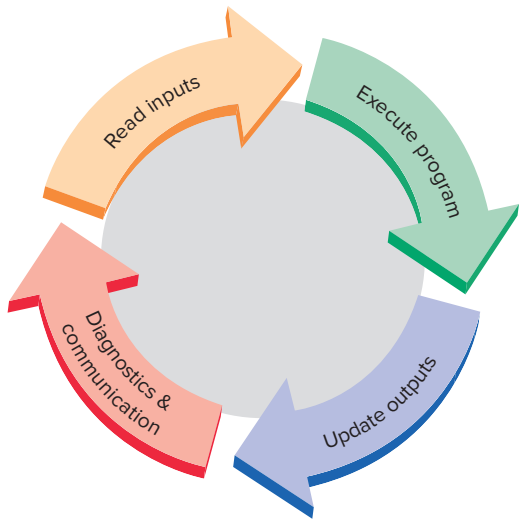


Figure 1-13 Typical PLC scan cycle.

The CPU controls all PLC activity and is designed so that the user can enter the desired program in relay ladder logic. The PLC program is executed as part of a repetitive process referred to as a scan (Figure 1-13). A typical PLC scan starts with the CPU reading the status of inputs. Then, the application program is executed. Once the program execution is completed, the status of all outputs is updated. Next, the CPU performs internal diagnostic and communication tasks. This process is repeated continuously as long as the PLC is in the run mode.

The ***I/O system*** forms the interface by which field devices are connected to the controller (Figure 1-14). The purpose of this interface is to condition the various signals received from or sent to external field devices. Input devices such as pushbuttons, limit switches, and sensors

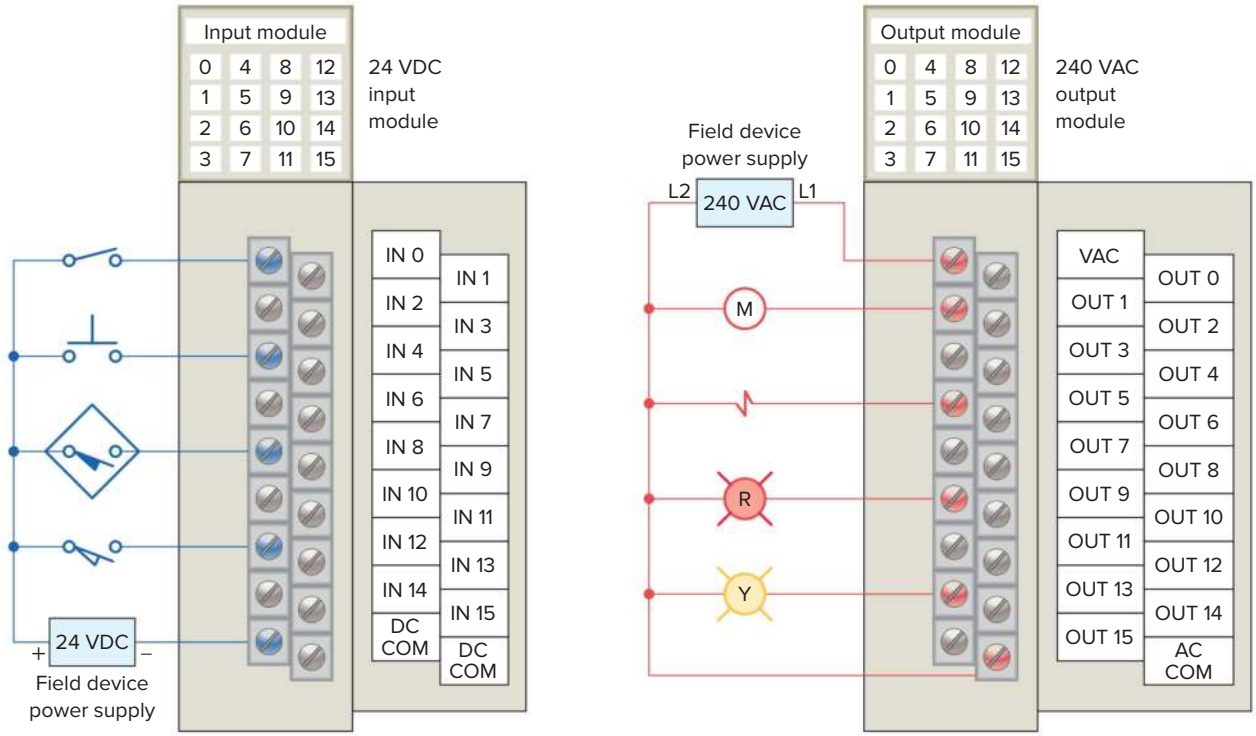


Figure 1-14 Typical PLC input/output (I/O) system connections.

are hardwired to the input terminals. Output devices such as small motors, motor starters, solenoid valves, and indicator lights are hardwired to the output terminals. To electrically isolate the internal components from the input and output terminals, PLCs commonly employ an optical isolator, which uses light to couple the circuits together. The external devices are also referred to as "field" or "real-world" inputs and outputs. The terms ***field*** or ***real world*** are used to distinguish actual external devices that exist and must be physically wired from the internal user program that duplicates the function of relays, timers, and counters.

A ***programming device*** is used to enter the desired program into the memory of the processor. The program can be entered using relay ladder logic, which is one of the most popular programming languages. Instead of words, ladder logic programming language uses graphic symbols that show their intended outcome. A program in ladder logic is similar to a schematic for a relay control circuit. It is a special language written to make it easy for people familiar with relay logic control to program the PLC. Hand-held programming devices are sometimes used to program small PLCs because they are inexpensive and easy to use. Once plugged into the PLC, they can be used to enter and monitor programs. Both compact hand-held units and laptop computers are frequently used on the factory floor for troubleshooting equipment, modifying programs, and transferring programs to multiple machines.

A personal computer (PC) is the most commonly used programming device. Most brands of PLCs have software available so that a PC can be used as the programming device. This software allows users to create, edit, document, store, and troubleshoot ladder logic programs (Figure 1-15). The computer monitor is able to display more logic on the screen than can hand-held types, thus simplifying the interpretation of the program. The personal computer communicates with the PLC processor via a serial or parallel data communications link, or Ethernet. If the programming unit is not in use, it may be unplugged and removed. Removing the programming unit will not affect the operation of the user program.

A ***program*** is a user-developed series of instructions that directs the PLC to execute actions. A *programming language* provides rules for combining the instructions so that they produce the desired actions. ***Relay ladder logic (RLL)*** is the standard programming language used with PLCs. Its origin is based on electromechanical relay control. The relay ladder logic program graphically represents rungs of contacts, coils, and special instruction blocks. RLL was originally designed for easy use and understanding for its users and has been modified to keep up with the increasing demands of industry's control needs.

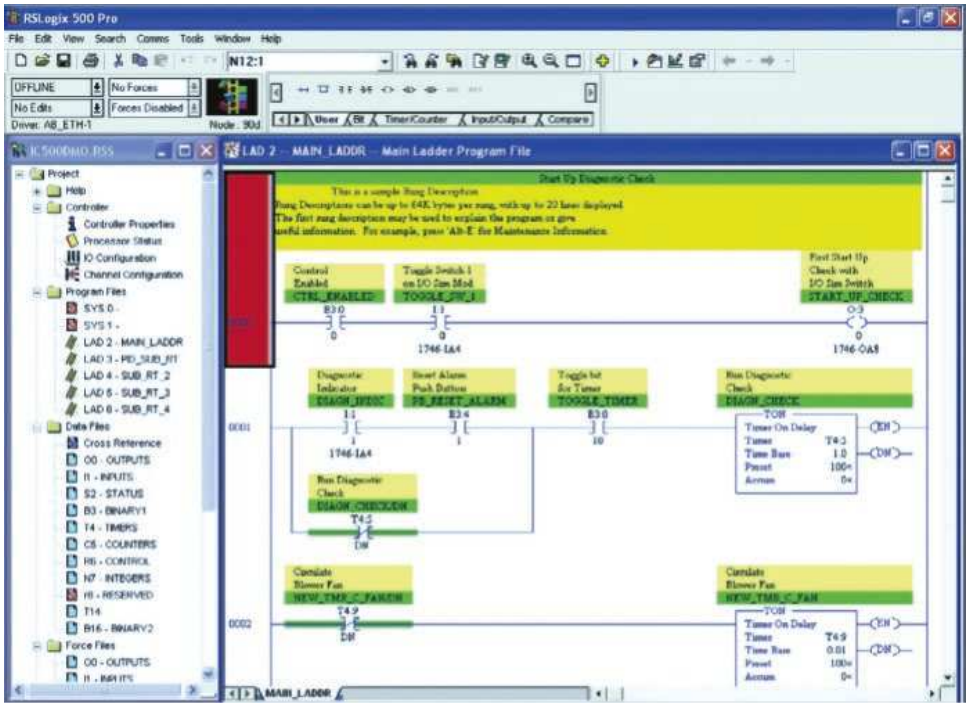


Figure 1-15 Typical PC software used to create a ladder logic program.
Source: Image Courtesy of Rockwell Automation, Inc.

1.3 Principles of Operation

To get an idea of how a PLC operates, consider the simple process control problem illustrated in Figure 1-16. Here a mixer motor is to be used to automatically stir the liquid in a vat when the temperature and pressure reach preset values. In addition, direct manual operation of the motor is provided by means of a separate pushbutton station. The process is monitored with temperature and pressure sensor switches that close their respective contacts when conditions reach their preset values.

This control problem can be solved using the relay method for motor control shown in the relay ladder diagram of Figure 1-17. The motor starter coil (M) is energized when both the pressure and temperature switches are closed or when the manual pushbutton is pressed.

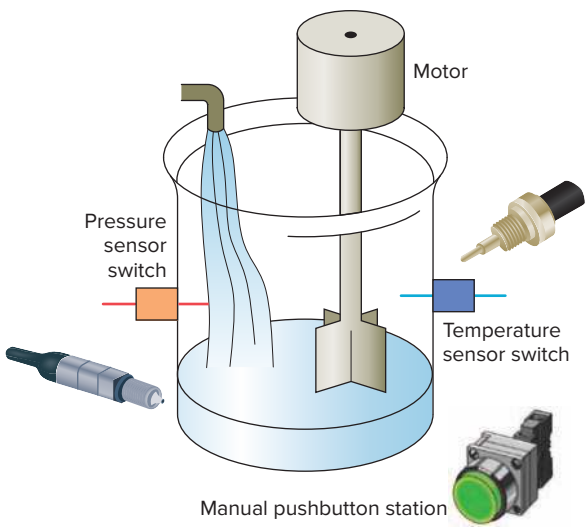


Figure 1-16 Mixer process control problem.

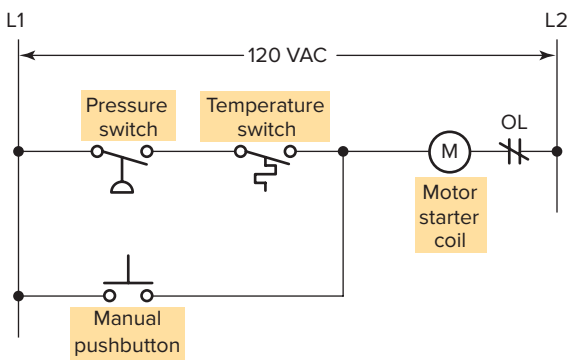


Figure 1-17 Process control relay ladder diagram.

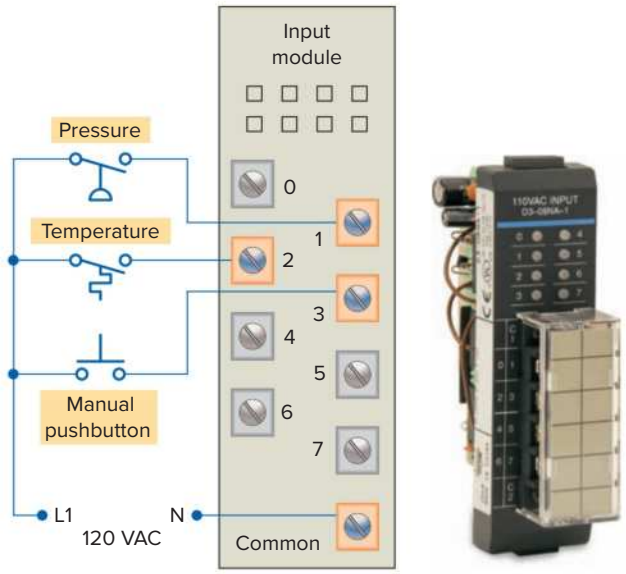


Figure 1-18 Typical wiring connections for a 120 VAC modular configured input module.
Source: Photo courtesy Automation Direct, **www.automationdirect.com**.

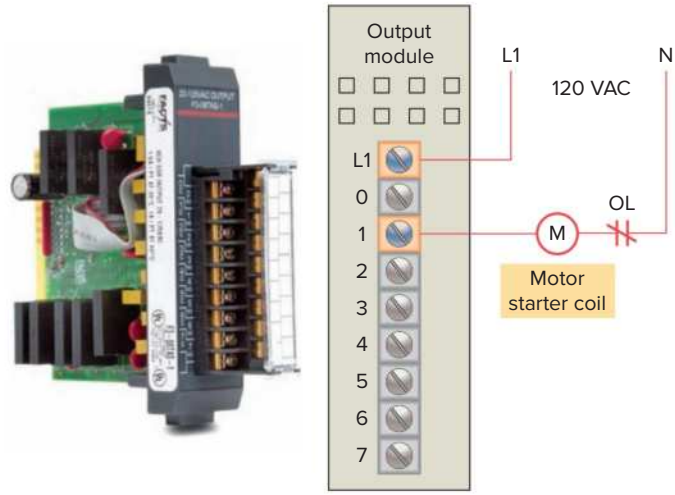


Figure 1-19 Typical wiring connections for a 120 VAC modular configured output module.
Source: Photo courtesy Automation Direct, **www.automationdirect.com**.

Now let's look at how a programmable logic controller might be used for this application. The same input field devices (pressure switch, temperature switch, and push-button) are used. These devices would be hardwired to an appropriate input module according to the manufacturer's addressing location scheme. Typical wiring connections for a 120 VAC modular configured input module are shown in Figure 1-18.

The same output field device (motor starter coil) would also be used. This device would be hardwired to an appropriate output module according to the manufacturer's addressing location scheme. Typical wiring connections for a 120 VAC modular configured output module are shown in Figure 1-19.

Next, the PLC ladder logic program would be constructed and entered into the memory of the CPU. A typical ladder logic program for this process is shown in Figure 1-20. The format used is similar to the layout of

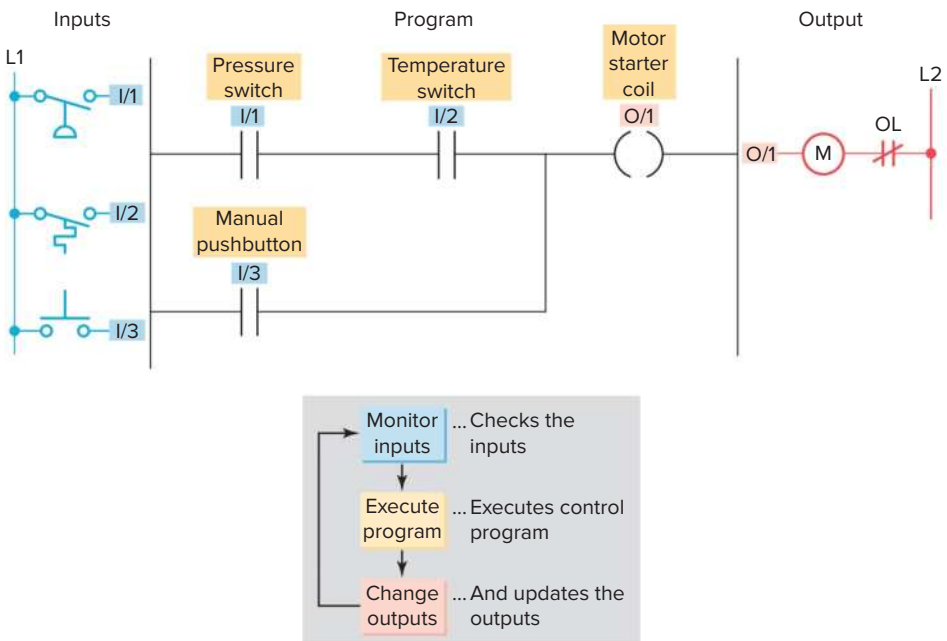


Figure 1-20 Process control PLC ladder logic program with typical addressing scheme.

the hardwired relay ladder circuit. The individual symbols represent instructions, whereas the numbers represent the instruction location addresses. To program the controller, you enter these instructions one by one into the processor memory from the programming device. Each input and output device is given an address, which lets the PLC know where it is physically connected. Note that the I/O address format will differ, depending on the PLC model and manufacturer. Instructions are stored in the user program portion of the processor memory. During the program scan the controller monitors the inputs, executes the control program, and changes the output accordingly.

For the program to operate, the controller is placed in the RUN mode, or operating cycle. During the program scan, the controller monitors the inputs, executes the control program, and changes the output accordingly. Each ⫴ symbol (looks like a normally open contact) is an instruction. The ( ) symbol is considered to represent a coil that, when energized, will energize the device that is wired to the respective output. In the ladder logic program of Figure 1-20, the coil O/1 is energized when contacts I/1 and I/2 are closed or when contact I/3 is closed. Either of these conditions provides a continuous logic path from left to right across the rung that includes the coil.

A programmable logic controller operates in real time in that an event taking place in the field will result in an operation or output taking place. The RUN operation for the process control scheme can be described by the following sequence of events:

- First, the pressure switch, temperature switch, and pushbutton inputs are examined and their status is recorded in the controller's memory.
- A closed contact is recorded in memory as logic 1 and an open contact as logic 0.
- Next the ladder diagram is evaluated, with each internal contact given an OPEN or CLOSED status according to its recorded 1 or 0 state.
- When the states of the input contacts provide logic continuity from left to right across the rung, the output coil memory location is given a logic 1 value and the output module interface contacts will close.
- When there is no logic continuity of the program rung, the output coil memory location is set to logic 0 and the output module interface contacts will be open.
- The completion of one cycle of this sequence by the controller is called a *scan.* The scan time, the time required for one full cycle, provides a measure of the speed of response of the PLC.
- Generally, the output memory location is updated during the scan but the actual output is not updated until the end of the program scan during the I/O scan.

Figure 1-21 shows the typical wiring required to implement the process control scheme using a fixed PLC

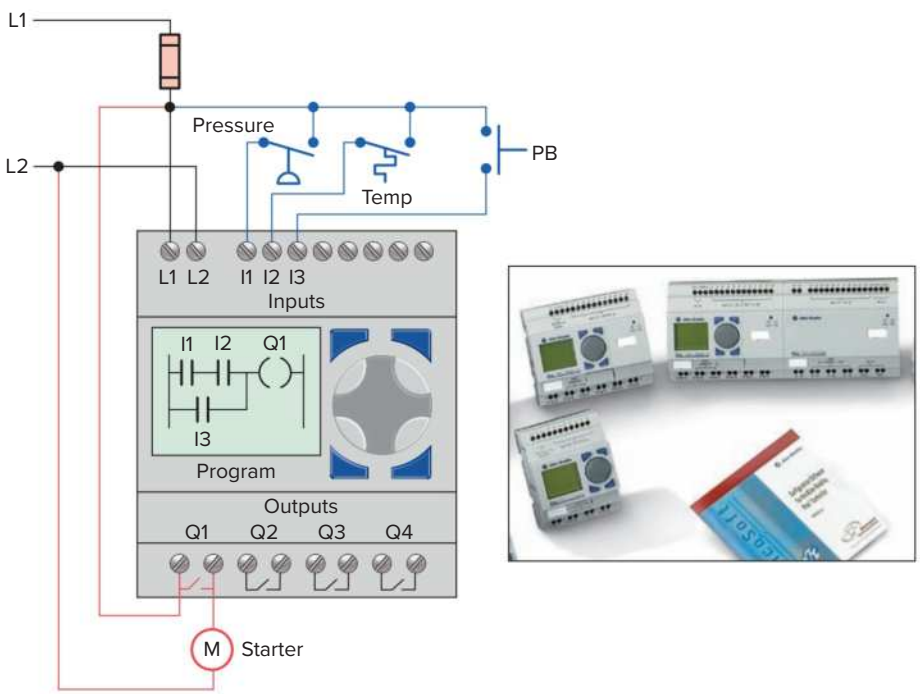


Figure 1-21 Typical wiring required to implement the process control scheme using a fixed PLC controller.
Source: Image Courtesy of Rockwell Automation, Inc.

controller. In this example, the Allen-Bradley Pico controller equipped with 8 inputs and 4 outputs is used to control and monitor the process. Installation can be summarized as follows:

- Fused power lines, of the specified voltage type and level, are connected to the controller's L1 and L2 terminals.
- The pressure switch, temperature switch, and pushbutton field input devices are hardwired between L1 and controller input terminals I1, I2, and I3, respectively.
- The motor starter coil connects directly to L2 and in series with Q1 relay output contacts to L1.
- The ladder logic program is entered using the front keypad and LCD display.
- Pico programming software is also available that allows you to create as well as test your program using a personal computer.

1.4 Modifying the Operation

One of the important features of a PLC is the ease with which the program can be changed. For example, assume that the original process control circuit for the mixing operation must be modified as shown in the relay ladder diagram of Figure 1-22. The change requires that the manual pushbutton control be permitted to operate at any pressure, but not unless the specified temperature setting has been reached.

If a relay system were used, it would require some rewiring of the circuit shown in Figure 1-22 to achieve the desired change. However, if a PLC system were used, no rewiring would be necessary. The inputs and outputs are still the same. All that is required is to change the PLC ladder logic program as shown in Figure 1-23.

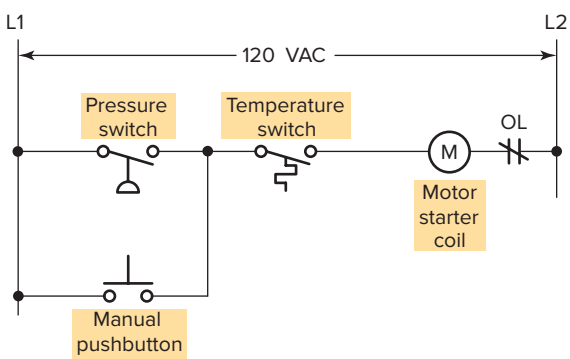


Figure 1-22 Relay ladder diagram for the modified process.

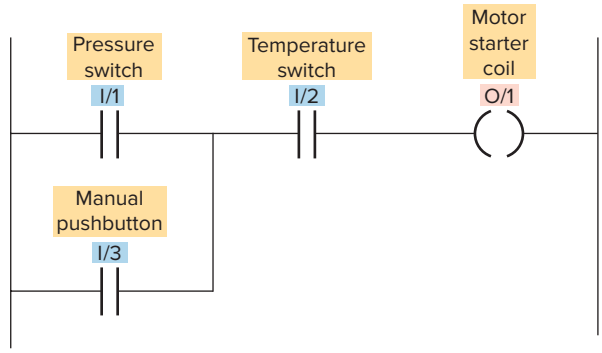


Figure 1-23 PLC ladder logic program for the modified process.

1.5 PLCs versus Computers

The architecture of a PLC is basically the same as that of a personal computer. A personal computer (PC) can be made to operate as a programmable logic controller if you provide some way for the computer to receive information from devices such as pushbuttons or switches. You also need a program to process the inputs and some way to turn devices on and off.

However, some important characteristics distinguish PLCs from personal computers. First, unlike PCs, the PLC is designed to operate in the industrial environment with wide ranges of ambient temperature and humidity. A well-designed industrial PLC installation, such as that shown in Figure 1-24, is not usually affected by the electrical noise inherent in most industrial locations.

Unlike the personal computer, the PLC is programmed in relay ladder logic or other easily learned languages. The PLC comes with its program language built into its memory and has no permanently attached keyboard, CD drive, or monitor. Instead, PLCs come equipped with

Figure 1-24 PLC installed in an industrial environment. Source: Courtesy of Softac Systems, Ltd.

terminals for input and output field devices as well as communication ports.

Computers are complex computing machines capable of executing several programs or tasks simultaneously and in any order. Most PLCs, on the other hand, execute a single program in an orderly and sequential fashion from first to last instruction.

PLC control systems have been designed to be easily installed and maintained. Troubleshooting is simplified by the use of fault indicators and messaging displayed on the programmer screen. Input/output modules for connecting the field devices are easily connected and replaced.

Software associated with a PLC but written and run on a personal computer falls into the following two broad categories:

- PLC software that allows the user to program and document gives the user the tools to write a PLC program—using ladder logic or another programming language—and document or explain the program in as much detail as is necessary.
- PLC software that allows the user to monitor and control the process is also called a ***human machine interface (HMI)***. It enables the user to view a process—or a graphical representation of a process—on a monitor, determine how the system is running, trend values, and receive alarm conditions (Figure 1-25). Many operator interfaces do not use PLC software. PLCs can be integrated with HMIs but the same software does not program both devices.

Figure 1-25 Human Machine Interface (HMI)
Source: Image Courtesy of Rockwell Automation, Inc.

Figure 1-26 Programmable automation controller (PAC).
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

Most recently automation manufacturers have responded to the increased requirements of industrial control systems by blending the advantages of PLC-style control with that of PC-based systems. Such a device has been termed a programmable automation controller, or PAC (Figure 1-26). Programmable automation controllers combine PLC ruggedness with PC functionality. Using PACs, you can build advanced systems incorporating software capabilities such as advanced control, communication, data logging, and signal processing with rugged hardware performing logic, motion, process control, and vision.

1.6 PLC Size and Application

The criteria used in categorizing PLCs include functionality, number of inputs and outputs, cost, and physical size (Figure 1-27). Of these, the ***I/O count*** is the most

Figure 1-27 Typical range of sizes of programmable controllers.
Source: Courtesy Siemens.

Figure 1-28 Single-ended PLC application.
Source: Courtesy Rogers Machinery Company, Inc.

important factor. In general, the nano is the smallest size with less than 15 I/O points. This is followed by micro types (15 to 128 I/O points), medium types (128 to 512 I/O points), and large types (over 512 I/O points).

Matching the PLC with the application is a key factor in the selection process. In general it is not advisable to buy a PLC system that is larger than current needs dictate. However, future conditions should be anticipated to ensure that the system is the proper size to fill the current and possibly future requirements of an application.

There are three major types of PLC application: single-ended, multitask, and control management. A ***single-ended*** or **stand-alone** PLC application involves one PLC controlling one process (Figure 1-28). This would be a stand-alone unit and would not be used for communicating with other computers or PLCs. The size and sophistication of the process being controlled are obvious factors in determining which PLC to select. The applications could dictate a large processor, but usually this category requires a small PLC.

A ***multitask*** PLC application involves one PLC controlling several processes. Adequate I/O capacity is a significant factor in this type of installation. In addition, if the PLC would be a subsystem of a larger process and would have to communicate with a central PLC or computer, provisions for a data communications network are also required.

A ***control management*** PLC application involves one PLC controlling several others (Figure 1-29). This kind of application requires a large PLC processor designed to communicate with other PLCs and possibly with a computer. The control management PLC supervises several PLCs by downloading programs that tell the other PLCs what has to be done. It must be capable of connection to all PLCs so that by proper addressing it can communicate with any one it wishes to.

Memory is the part of a PLC that stores data, instructions, and the control program. Memory size is usually expressed in K values: 1 K, 6 K, 12 K, and so on. The measurement kilo, abbreviated K, normally refers to 1000 units. When dealing with computer or PLC memory, however, 1 K means 1024, because this measurement is based on the binary number system ($2^{10} = 1024$). Depending on memory type, 1 K can mean 1024 bits, 1024 bytes, or 1024 words.

Although it is common for us to measure the memory capacity of PLCs in words, we need to know the number of bits in each word before memory size can be accurately compared. Modern computers usually have a word size of 16, 32, or 64 bits. For example, a PLC that uses 8-bit words has 49,152 bits of storage with a 6 K word capacity ($8 \times 6 \times 1024 = 49{,}152$), whereas a PLC using 32-bit words has 196,608 bits of storage with the same 6 K memory ($32 \times 6 \times 1024 = 196{,}608$). The amount of memory required depends on the application. Factors affecting the memory size needed for a particular PLC installation include:

- Number of I/O points used
- Size of control program
- Data-collecting requirements
- Supervisory functions required
- Future expansion

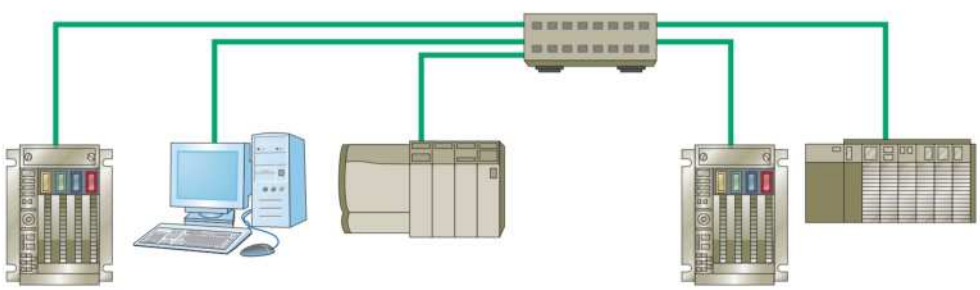


Figure 1-29 Control management PLC application.

Table 1-1 Typical PLC Instructions

Instruction	Operation
XIC (Examine ON)	Examine a bit for an ON condition
XIO (Examine OFF)	Examine a bit for an OFF condition
OTE (Output Energize)	Turn ON a bit (nonretentive)
OTL (Output Latch)	Latch a bit (retentive)
OTU (Output Unlatch)	Unlatch a bit (retentive)
TOF (Timer Off-Delay)	Turn an output ON or OFF after its rung has been OFF for a preset time interval
TON (Timer On-Delay)	Turn an output ON or OFF after its rung has been ON for a preset time interval
CTD (Count Down)	Use a software counter to count down from a specified value
CTU (Count Up)	Use a software counter to count up to a specified value

The ***instruction set*** for a particular PLC lists the different types of instructions supported. Typically, this ranges from 15 instructions on smaller units up to 100 instructions on larger, more powerful units (see Table 1-1).

CHAPTER 1 REVIEW QUESTIONS

1. What is a programmable logic controller (PLC)?
2. Identify four tasks in addition to relay switching operations that PLCs are capable of performing.
3. List six distinct advantages that PLCs offer over conventional relay-based control systems.
4. Explain the differences between open and proprietary PLC architecture.
5. State two ways in which I/O is incorporated into the PLC.
6. Describe how the I/O modules connect to the processor in a modular-type PLC configuration.
7. Explain the main function of each of the following major components of a PLC:
   a. Processor module (CPU)
   b. I/O modules
   c. Programming device
   d. Power supply module
8. What are the two most common types of PLC programming devices?
9. Explain the terms *program* and *programming language* as they apply to a PLC.
10. What is the standard programming language used with PLCs?
11. Answer the following with reference to the process control relay ladder diagram of Figure 1-17 of this chapter:
    a. When do the pressure switch contacts close?
    b. When do the temperature switch contacts close?
    c. How are the pressure and temperature switches connected with respect to each other?
    d. Describe the two conditions under which the motor starter coil will become energized.
    e. What is the approximate value of the voltage drop across each of the following when their contacts are open?
    (1) Pressure switch
    (2) Temperature switch
    (3) Manual pushbutton
12. The programmable controller operates in real time. What does this mean?
13. Answer the following with reference to the process control PLC ladder logic diagram of Figure 1-20 of this chapter:
    a. What do the individual symbols represent?
    b. What do the numbers represent?
    c. What field device is the number I/2 identified with?
    d. What field device is the number O/1 identified with?
    e. What two conditions will provide a continuous path from left to right across the rung?
    f. Describe the sequence of operation of the controller for one scan of the program.
14. Compare the method by which the process control operation is changed in a relay-based system to the method used for a PLC-based system.
15. Compare the PLC and PC with regard to:
    a. Physical hardware differences
    b. Operating environment
    c. Method of programming
    d. Execution of program
16. What two categories of software written and run on PCs are used in conjunction with PLCs?
17. What is a programmable automation controller (PAC)?
18. List four criteria by which PLCs are categorized.
19. Compare the single-ended, multitask, and control management types of PLC applications.
20. What is the memory capacity, expressed in bits, for a PLC that uses 16-bit words and has an 8 K word capacity?
21. List five factors affecting the memory size needed for a particular PLC installation.
22. What does the instruction set for a particular PLC refer to?

CHAPTER 1 PROBLEMS

1. Given two single-pole switches, write a program that will turn on an output when both switch *A* and switch *B* are closed.
2. Given two single-pole switches, write a program that will turn on an output when either switch *A* or switch *B* is closed.
3. Given four NO (Normally Open) pushbuttons (*A-B-C-D*), write a program that will turn a lamp on if pushbuttons *A* and *B* or *C* and *D* are closed.
4. Write a program for the relay ladder diagram shown in Figure 1-30.

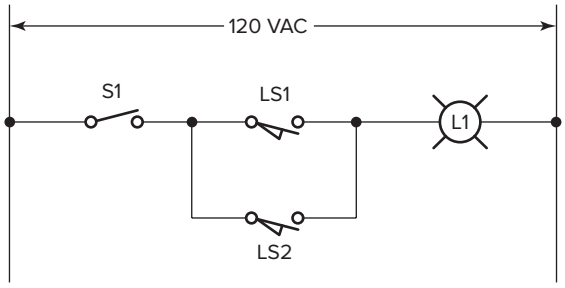


Figure 1-30 Circuit for Problem 4.

5. Write a program for the relay ladder diagram shown in Figure 1-31.

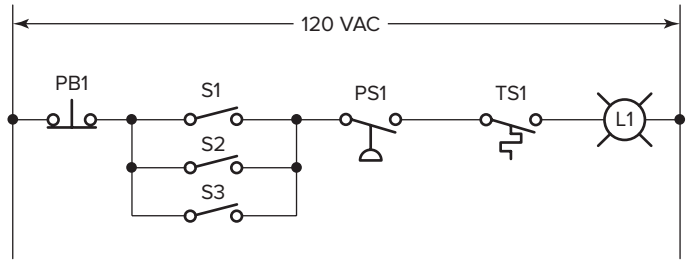


Figure 1-31 Circuit for Problem 5.

2

PLC Hardware Components

Courtesy of Nercon

Chapter Objectives

After completing this chapter, you will be able to:

- List and describe the function of the hardware components used in PLC systems
- Describe the basic circuitry and applications for discrete and analog I/O modules, and interpret typical I/O and CPU specifications
- Explain I/O addressing
- Describe the general classes and types of PLC memory devices
- List and describe the different types of PLC peripheral support devices available

This chapter exposes you to the details of PLC hardware and modules that make up a PLC control system. The chapter's illustrations show the various parts of a PLC as well as general connection paths. In this chapter we discuss the CPU and memory hardware components, including the various types of memory that are available, and we describe the hardware of the input/output section, including the difference between the discrete and analog types of modules.

2.1 The I/O Section

The **input/output (I/O)** section of a PLC is the section to which all field devices are connected and provides the interface between them and the CPU. Input/output arrangements are built into a fixed PLC while modular types use external I/O modules that plug into the PLC.

Figure 2-1 illustrates a rack-based I/O section made up of individual I/O modules. Input interface modules accept signals from the machine or process devices and convert them into signals that can be used by the controller. Output interface modules convert controller signals into external signals used to control the machine or process. A typical PLC has room for several I/O modules, allowing it to be customized for a particular application by selecting the appropriate modules. Each slot in the rack is capable of accommodating any type of I/O module.

The I/O system provides an interface between the hardwired components in the field and the CPU. The input interface allows *status information* regarding processes to be communicated to the CPU, and thus allows the CPU to communicate *operating signals* through the output interface to the process devices under its control.

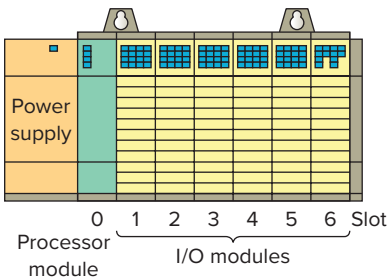


Figure 2-1 Rack-based I/O section.

One benefit of a PLC system is the ability to locate the I/O modules near the field devices, as illustrated in Figure 2-2, in order to minimize the amount of wiring required. The processor receives signals from the remote input modules and sends signals back to their output modules via the communication module.

A rack is referred to as a ***remote*** rack when it is located away from the processor module. To communicate with the processor, the remote rack uses a special communications network. Each remote rack requires a unique station number to distinguish one from another. The remote racks are linked to the local rack through a ***communications module.*** Cables connect the modules with each other. If fiber optic cable is used between the CPU and I/O rack, it is possible to operate I/O points from distances greater than 20 miles with no voltage drop. Coaxial cable will allow remote I/O to be installed at distances greater than two miles. Fiber optic cable will not pick up noise caused by adjacent high power lines or equipment normally found in an industrial environment. Coaxial cable is more susceptible to this type of noise.

The PLC's memory system stores information about the status of all the inputs and outputs. To keep track of all this information, it uses a system called *addressing*. An **address** is a label or number that indicates where a certain piece of information is located in a PLC's memory. Just as your home address tells where you live in your city, a device's or a piece of data's address tells where

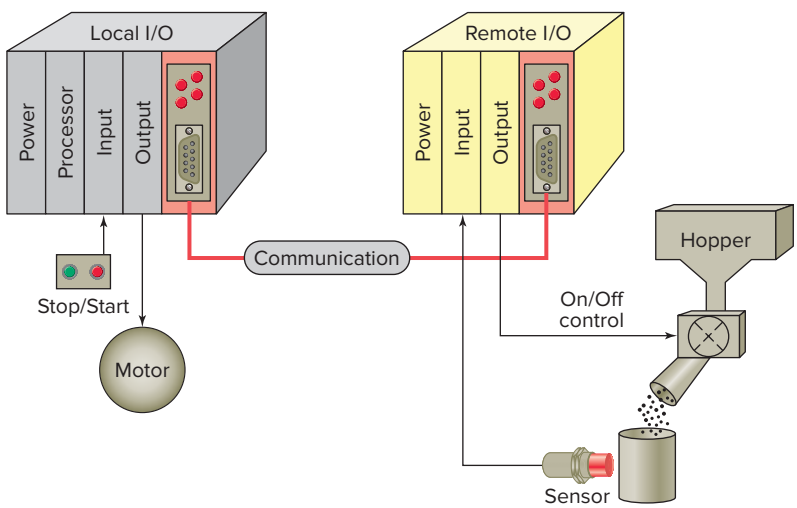


Figure 2-2 Remote I/O rack.

information about it resides in the PLC's memory. That way, if a PLC wants to find out information about a field device, it knows to look in its corresponding address location. Examples of addressing schemes include ***rack/slot-based***, versions of which are used in Allen-Bradley SLC 500 controllers, ***tag-based*** used in Allen-Bradley ControlLogix controllers, and PC-based control used in soft PLCs.

In general, rack/slot-based addressing elements include:

Type—The type determines if an input or output is being addressed.

Slot—The slot number is the physical location of the I/O module. This may be a combination of the rack number and the slot number when using expansion racks.

Word and Slot—The word and slot are used to identify the actual terminal connection in a particular I/O module. A discrete module usually uses only one word, and each connection corresponds to a different bit that makes up the word.

With a rack/slot address system the location of a module within a rack and the terminal number of a module to which an input or output device is connected will determine the device's address.

Figure 2-3 illustrates the Allen-Bradley **SLC 500** controller rack/slot addressing format. The address is used by the processor to identify where the device is located to monitor or control it. In addition, there is some means of connecting field wiring on the I/O module housing. Connecting the field wiring to the I/O housing allows easier disconnection and reconnection of the wiring to change modules. Lights are also added to each module to indicate the ON or OFF status of each I/O circuit. Most output modules also have blown fuse indicators. The following are typical examples of SLC 500 real-world general input and output addresses:

O:4/15	**Output module in slot 4, terminal 15**
I:3/8	**Input module in slot 3, terminal 8**
O:6.0	**Output module, slot 6**
I:5.0	**Input module, slot 5**

Every input and output device connected to a discrete I/O module is addressed to a specific ***bit*** in the PLC's memory. A bit is a binary digit that can be either 1 or 0. Analog I/O modules use a ***word*** addressing format, which allows the entire words to be addressed. The bit part of the address is usually not used; however, bits of the digital representation of the analog value can be addressed by the programmer if necessary. Figure 2-4 illustrates bit level and word level addressing as it applies to an SLC 500 controller.

Tag-based memory structures are the newest type of PLC memory addressing. Figure 2-5 illustrates the Allen-Bradley ControlLogix and CompactLogix tag-based addressing format. Memory locations are defined by using base and alias tags. A **base tag** defines a memory location where data are stored. An **alias tag** is used to create an alternate name (alias) for a tag. The alias tag is often used to create a tag name to represent a real world input or output.

Figure 2-6 shows a comparison between rack/slot-based addressing and tag-based addressing. Input and output modules, when configured, automatically create their own tags like Local:1:I.Data.1. Tag names are descriptive to the data being stored in them. The alias tag lets you use names that are more meaningful for the application. In this example:

- Pressure_switch is used instead of I:1/1
- Temperature_switch is used instead of I:1/2
- Manual_pushbutton is used instead of I:1/3
- Mixer_motor is used instead of O:2/1

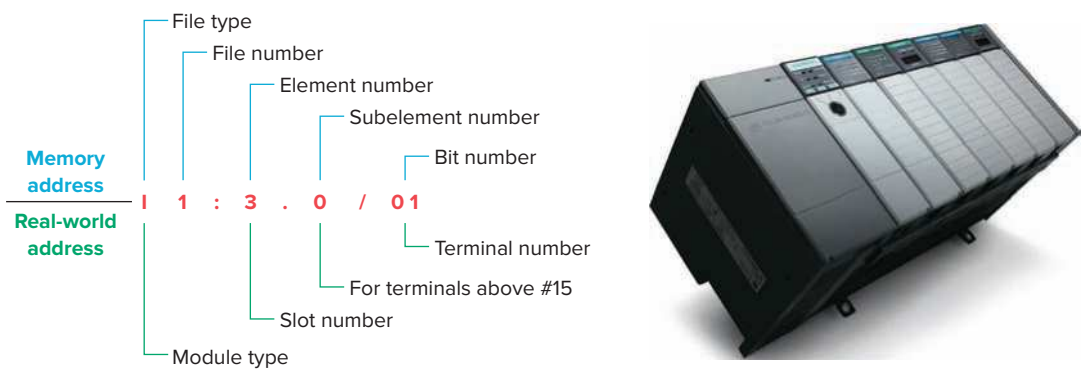


Figure 2-3 Allen-Bradley SLC 500 rack/slot-based addressing format.
Source: Image Courtesy of Rockwell Automation, Inc.

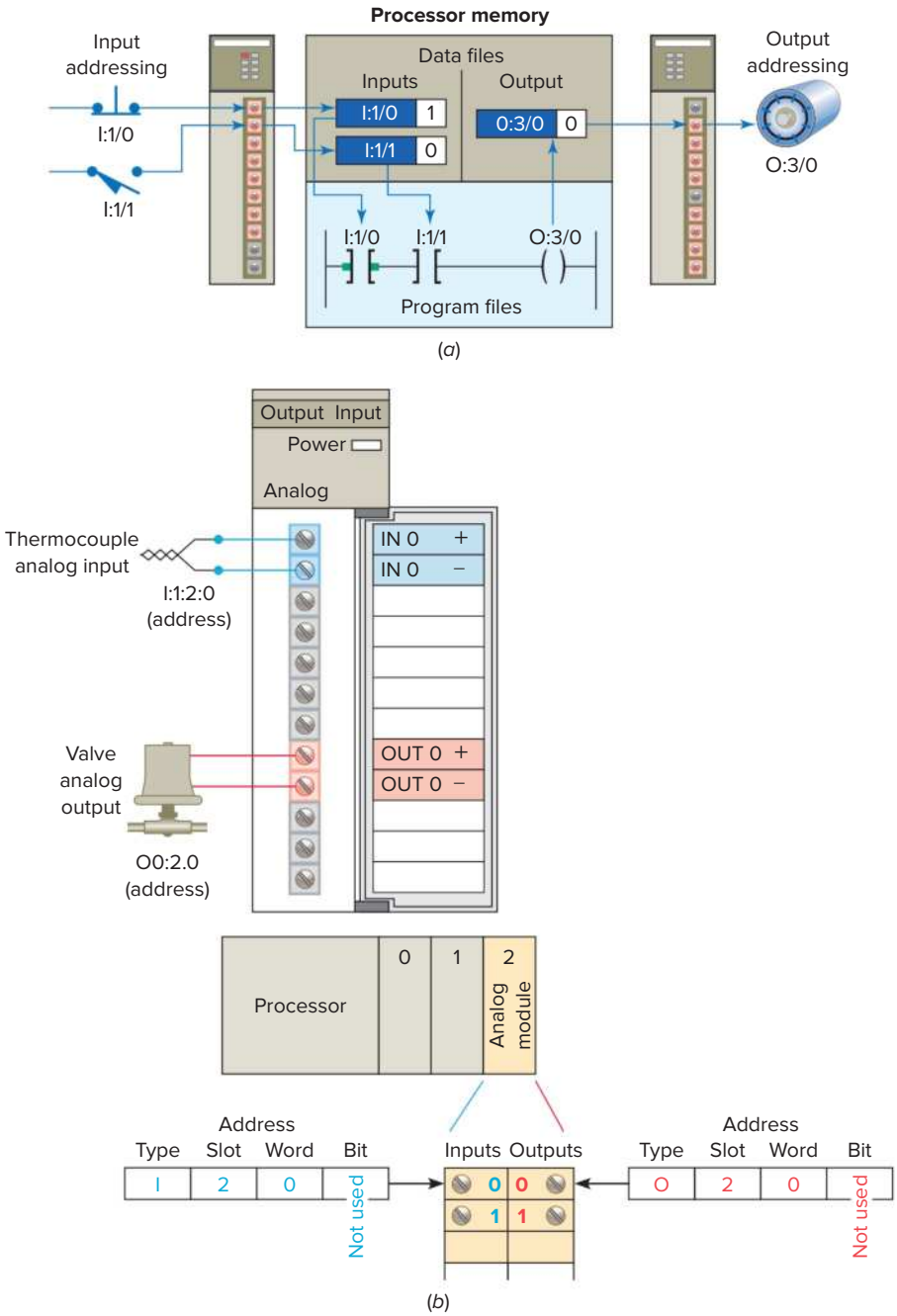
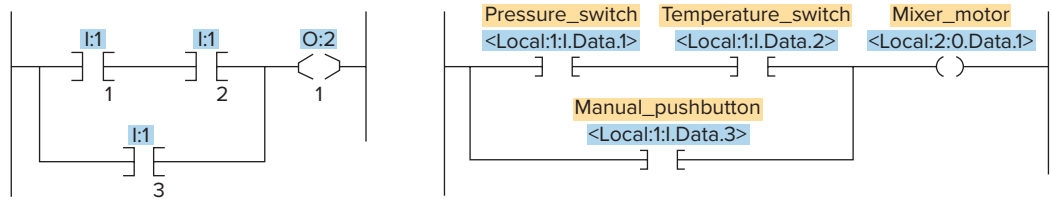


Figure 2-4 SLC 500 bit level and word level addressing. (*a*) Bit level addressing. (*b*) Word level addressing.

Figure 2-5 Allen-Bradley ControlLogix tag-based addressing format.
Source: Image Courtesy of Rockwell Automation, Inc.



(*a*) SLC 500 rack/slot-based addressing (*b*) Equivalent ControlLogix 5000 tag-base addressing

Figure 2-6 Rack/slot-based versus tag-based addressing.

PC-based control runs on personal or industrial hardened computers. Also known as soft PLCs, they simulate the functions of a PLC on a PC, allowing open architecture systems to replace proprietary PLCs. This implementation uses an input/output card (Figure 2-7) in conjunction with the PC as an interface for the field devices.

Combination I/O modules can have both input and output connections in the same physical module as illustrated in Figure 2-8. A module is made up of a printed circuit board and a terminal assembly. The printed circuit board contains the electronic circuitry used to interface the circuit of the processor with that of the input or output device. Modules are designed to plug into a slot or connector in the I/O rack or directly into the processor. The terminal assembly, which is attached to the front edge of the printed circuit board, is used for making field-wiring connections. Modules contain terminals for each input and output connection, status lights for each of the inputs and outputs, and connections to the power supply used to

Figure 2-7 Typical PC interface card.
Source: Photo © Beckhoff Automation GmbH & Co. KG.

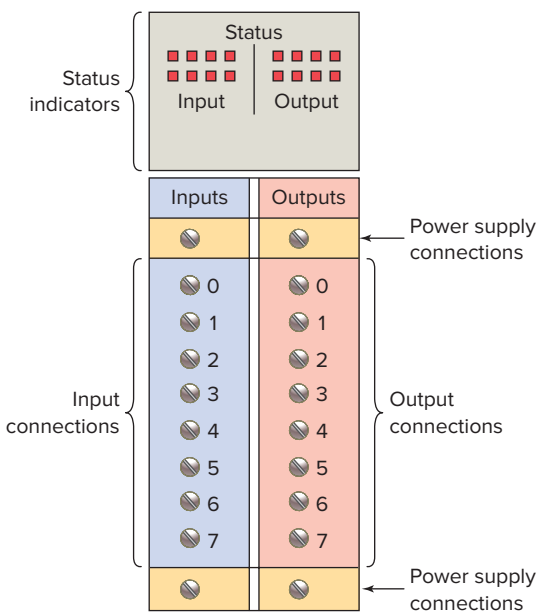


Figure 2-8 Typical combination I/O module.
Source: Image Courtesy of Rockwell Automation, Inc.

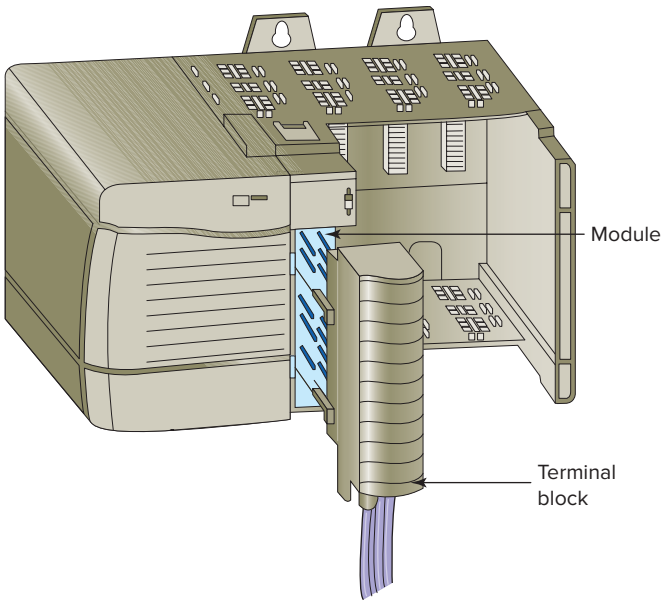


Figure 2-9 Plug-in terminal block.

power the inputs and outputs. Terminal and status light arrangements vary with different manufacturers.

Most PLC modules have plug-in wiring terminal strips. The terminal block is plugged into the actual module as illustrated in Figure 2-9. If there is a problem with a module, the entire strip is removed, a new module is inserted, and the terminal strip is plugged into the new module. Unless otherwise specified, never install or remove I/O modules or terminal blocks while the PLC is powered. A module inserted into the wrong slot could be damaged by improper voltages connected through the wiring arm. Most faceplates and I/O modules are keyed to prevent putting the wrong faceplate on the wrong module. In other words, an output module cannot be placed in the slot where an input module was originally located.

Input and output modules can be placed anywhere in a rack, but they are normally grouped together for ease of wiring. I/O modules can be 8, 16, 32, or 64 point cards (Figure 2-10). The number refers to the number of inputs or outputs available. The standard I/O module has eight inputs or outputs. A ***high-density*** module may have up to 64 inputs or outputs. The advantage with the high-density module is that it is possible to install up to 64 inputs or outputs in one slot for greater space savings. The only disadvantage is that the high-density output modules cannot handle as much current per output.

2.2 Discrete I/O Modules

The most common type of I/O interface module is the ***discrete*** type (Figure 2-11). This type of interface connects field input devices of the ON/OFF nature such as selector switches, pushbuttons, and limit switches. Likewise, output control is limited to devices such as lights, relays, solenoids, and motor starters that require simple ON/OFF switching. The classification of discrete I/O covers ***bit-oriented*** inputs and outputs. In this type of input or output, each bit represents a complete information element in itself and provides the status of some external contact or advises of the presence or absence of power in a process circuit.

Each discrete I/O module is powered by some *field-supplied* voltage source. Since these voltages can be of different magnitude or type, I/O modules are available at various AC and DC voltage ratings, as listed in Table 2-1.

The modules themselves receive their voltage and current for proper operation from the backplane of the rack enclosure into which they are inserted, as illustrated in Figure 2-12. **Backplane** power is provided by the PLC module power supply and is used to power the electronics that reside on the I/O module circuit board. The relatively higher currents required by the loads of an output module are normally provided by user-supplied power. Module power supplies typically may be rated for 3 A, 4 A, 12 A, or 16 A depending on the type and number of modules used.

Figure 2-10 16, 32, and 64 point I/O modules.
Source: (*all*) Photos courtesy Omron Industrial Automation, **www.ia.omron.com.**

Figure 2-11 Discrete input and output devices.

Table 2-1 Common Ratings for Discrete I/O Interface Modules

Input Interfaces	Output Interfaces
12 V **AC/DC** /24 V **AC/DC**	12–48 V **AC**
48 V **AC/DC**	120 V **AC**
120 V **AC/DC**	230 V **AC**
230 V **AC/DC**	120 V **DC**
5 V **DC** (TTL level)	230 V **DC**
	5 V **DC** (TTL level)
	24 V **DC**

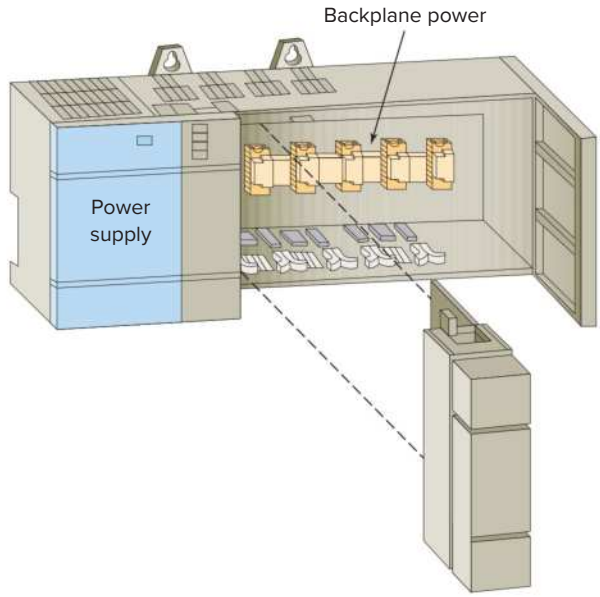


Figure 2-12 Modules receive their voltage and current from the backplane.

Figure 2-13 shows the block diagrams for one input of a typical alternating current (AC) *discrete input module*. The input circuit is composed of two basic sections: the ***power*** section and the ***logic*** section. An optical isolator is used to provide electrical isolation between the field wiring and the PLC backplane internal circuitry. The input LED turns on or off, indicating the status of the input device. Logic circuits process the digital signal to the processor. Internal PLC control circuitry typically operates at 5 VDC or less volts.

A simplified diagram for a single input of a discrete AC input module is shown in Figure 2-14. The operation of the circuit can be summarized as follows:

- The input noise filter consisting of the capacitor and resistors R1 and R2 removes false signals that are due to contact bounce or electrical interference.
- When the pushbutton is closed, 120 VAC is applied to the bridge rectifier input.
- This results in a low-level DC output voltage that is applied across the LED of the optical isolator.

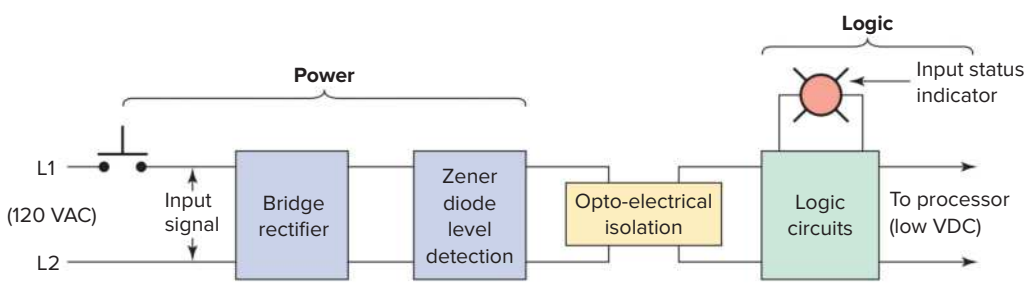


Figure 2-13 Discrete AC input module block diagram.

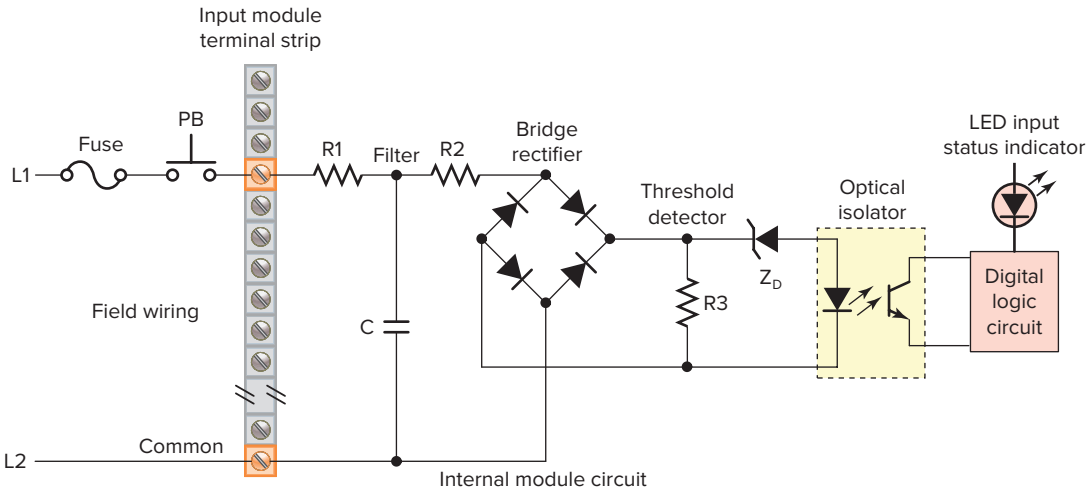


Figure 2-14 Simplified diagram for a single input of a discrete AC input module.

- The zener diode (Z_D) voltage rating sets the minimum threshold level of voltage that can be detected.
- When light from the LED strikes the phototransistor, it switches into conduction and the status of the pushbutton is communicated in logic to the processor.
- The optical isolator not only separates the higher AC input voltage from the logic circuits but also prevents damage to the processor due to line voltage transients. In addition, this isolation also helps reduce the effects of electrical noise, common in the industrial environment, which can cause erratic operation of the processor.
- For fault diagnosis, an input state LED indicator is on when the input pushbutton is closed. This indicator may be wired on either side of the optical isolator.
- An AC/DC type of input module is used for both AC and DC inputs as the input polarity does not matter.
- A PLC input module will have either all inputs isolated from each other with no common input connections or groups of inputs that share a common connection.

Discrete input modules perform four tasks in the PLC control system. They:

- Sense when a signal is received from a field device.
- Convert the input signal to the correct voltage level for the particular PLC.
- Isolate the PLC from fluctuations in the input signal's voltage or current.
- Send a signal to the processor indicating which sensor originated the signal.

Figure 2-15 shows the block diagram for one output of a typical discrete output module. Like the input module, it is composed of two basic sections: the power section and the logic section, coupled by an isolation circuit. The output interface can be thought of as an electronic switch that turns the output load device on and off. Logic circuits determine the output status. An output LED indicates the status of the output signal.

A simplified diagram for a single output of a discrete AC output module is shown in Figure 2-16. The operation of the circuits ….set can be summarized as follows:

- As part of its normal operation, the digital logic circuits ….set of the processor sets the output status according to the program.

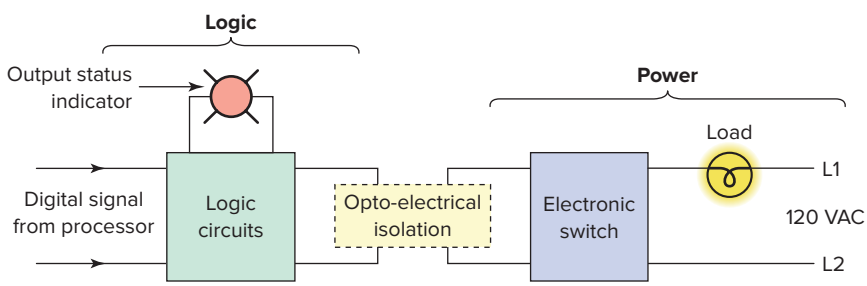


Figure 2-15 Discrete AC output module block diagram.

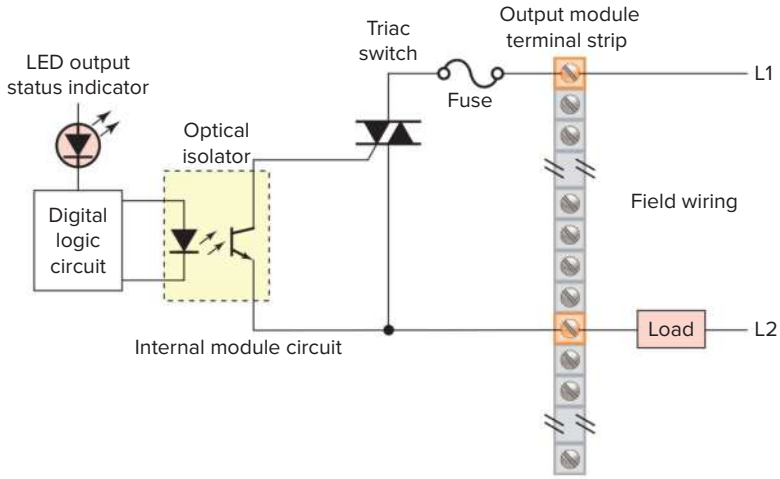


Figure 2-16 Simplified diagram for a single output of a discrete AC output module.

- When the processor calls for an output load to be energized, a voltage is applied across the LED of the opto-isolator.
- The LED then emits light, which switches the phototransistor into conduction.
- This in turn triggers the triac AC semiconductor switch into conduction, allowing current to flow to the output load.
- Since the triac conducts in either direction, the output to the load is alternating current.
- The triac, rather than having ON and OFF status, actually has LOW and HIGH resistance levels, respectively. In its OFF state (HIGH resistance), a small leakage current of a few milliamperes still flows through the triac.
- As with input circuits, the output interface is usually provided with LEDs that indicate the status of each output.
- Fuses are normally required for the output module, and they are provided on a per circuit basis, thus allowing for each circuit to be protected and operated separately. Some modules also provide visual indicators for fuse status.
- The triac cannot be used to switch a DC load.
- For fault diagnosis, the LED output status indicator is on whenever the PLC is commanding that the output load be switched on.

Individual AC outputs are usually limited by the size of the triac to 1 A or 2 A. The maximum current load for any one module is also specified. To protect the output module circuits, specified current ratings should not be exceeded. For controlling larger loads, such as large motors, a standard control relay is connected to the output module. The contacts of the relay can then be used to control a larger load or motor starter, as shown in Figure 2-17. When a control relay is used in this manner, it is called an ***interposing*** **relay.**

Discrete output modules are used to turn field output devices either on or off. These modules can be used to control any two-state device, and they are available in AC and DC versions and in various voltage ranges and current ratings. Output modules can be purchased with *transistor, triac,* or *relay* output as illustrated in Figure 2-18. Triac outputs can be used only for control of AC devices,

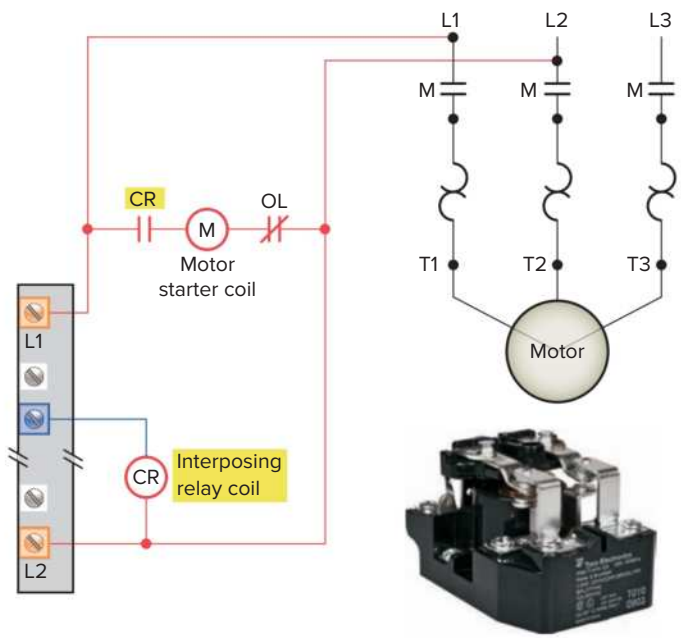


Figure 2-17 Interposing relay connection.
Source: Courtesy Tyco Electronics Ltd.

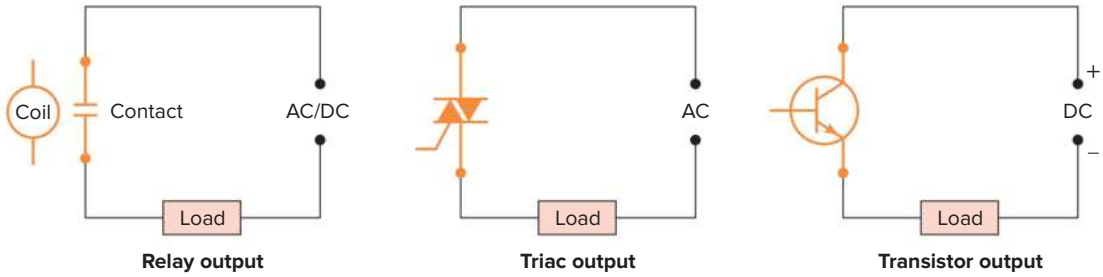


Figure 2-18 Relay, transistor, and triac switching elements.

whereas transistor outputs can be used only for control of DC devices. The discrete relay contact output module uses electromechanical as the switching element. These relay outputs can be used with AC or DC devices, but they have a much slower switching time compared to solid-state outputs. Allen-Bradley modules are color-coded for identification as follows:

Color	Type of I/O
Red	AC inputs/outputs
Blue	DC inputs/outputs
Orange	Relay outputs
Green	Specialty modules
Black	I/O wiring; terminal blocks are not removable

Certain DC I/O modules specify whether the module is designed for interfacing with current-source or current-sink devices. If the module is a current-sourcing module, then the input or output device must be a current-sinking device. Conversely, if the module is specified as current-sinking, then the connected device must be current-sourcing. Some modules allow the user to select whether the module will act as current sinking or current sourcing, thereby allowing it to be set to whatever the field devices require. Sinking and sourcing terminology applies only to DC input and output circuits.

Allen-Bradley delineates between the various digital DC modules by sorting them into two categories: **Sinking** and **Sourcing.** These terms are used to describe a current signal flow relationship between field input and output devices. If a device provides current when it is ON, it is said to be sourcing current. Conversely, if a device receives current when it is ON, it is said to be sinking current.

Figures 2-19 and 2-20 show device connections for both sourcing and sinking configurations:

- Conventional current (+ to −) is assumed.
- In sinking devices, current flows into the device's terminal from the module (the module provides, or sources the current).
- In sourcing devices, current flows out of the device's terminal into the module (the module receives, or sinks, the current).
- A sourcing I/O device or I/O module will always have a connection directly to the positive side of the DC power supply.
- A sinking I/O device or I/O module will always have a connection directly to the negative side of the DC power supply.

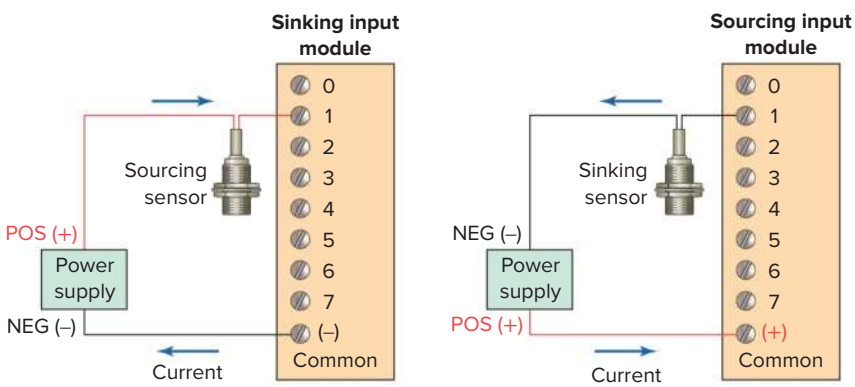


Figure 2-19 Sinking and sourcing inputs.

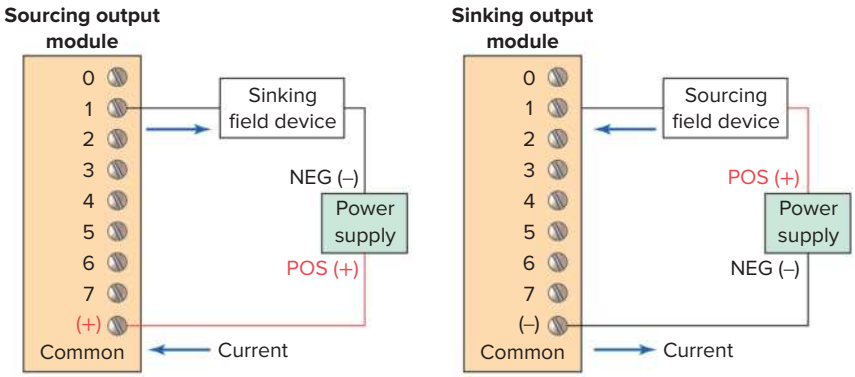


Figure 2-20 Sinking and sourcing outputs.

- Input and output points that are sinking or sourcing only can conduct current in only one direction. Therefore, it is possible to connect the external supply and field device to the I/O point with current trying to flow in the wrong direction, and the circuit **will not operate.**

2.3 Analog I/O Modules

Earlier PLCs were limited to discrete or digital I/O interfaces, which allowed only on/off-type devices to be connected. This limitation meant that the PLC could have only partial control of many process applications. Today, however, a complete range of both discrete and analog interfaces are available that will allow controllers to be applied to practically any type of control process.

Discrete devices are inputs and outputs that have only two states: on and off. In comparison, **analog** devices represent physical quantities that can have an infinite number of values. Typical analog inputs and outputs vary from 0 to 20 mA, 4 to 20 mA, or 0 to 10 V. Figure 2-21 illustrates how PLC analog input and output modules are used in measuring and displaying the level of fluid in a tank. The analog input interface module contains the circuitry necessary to accept an analog voltage or current signal from the level transmitter field device. This input is converted from an analog to a digital value for use by the processor. The circuitry of the analog output

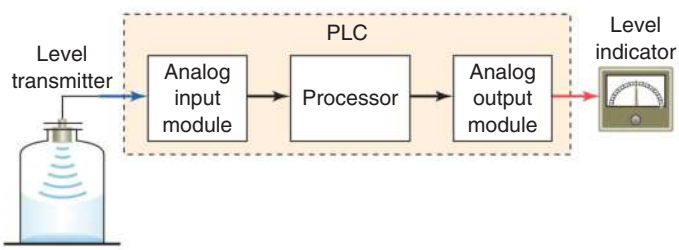


Figure 2-21 Analog input and output to a PLC.

module accepts the digital value from the processor and converts it back to an analog signal that drives the field tank level meter.

Analog input modules normally have multiple input channels that allow 4, 8, or 16 devices to be interface to the PLC. The two basic types of analog input modules are ***voltage*** **sensing** and ***current*** **sensing.** Input modules have user-selectable dip switch settings to choose whether each input will be a current or voltage input. Analog sensors measure a varying physical quantity over a specific range and generate a corresponding voltage or current signal. Common physical quantities measured by a PLC analog module include temperature, speed, level, flow, weight, pressure, and position. For example, a sensor may measure temperature over a range of 0 to 500°C, and output a corresponding voltage signal that varies between 0 and 50 mV.

Figure 2-22 illustrates an example of a voltage sensing input analog module used to measure temperature. The connection diagram applies to an Allen-Bradley MicroLogic 4-channel analog thermocouple input module. A varying DC voltage in the low millivolt range, proportional to the temperature being monitored, is produced by the thermocouple. This voltage is amplified and digitized by the analog input module and then sent to the processor on command from a program instruction. Because of the low voltage level of the input signal, a twisted shielded pair cable is used in wiring the circuit to reduce unwanted electrical noise signals that can be induced in the conductors from other wiring. When using an ungrounded thermocouple, the shield must be connected to ground at the module end. To obtain accurate readings from each of the channels, the temperature between the thermocouple wire and the input channel must be compensated for. A cold junction compensating (CJC) thermistor is integrated in the terminal block for this purpose.

The transition of an analog signal to digital values is accomplished by an analog-to-digital (A/D) converter, the main element of the analog input module. Analog voltage input modules are available in two types: unipolar

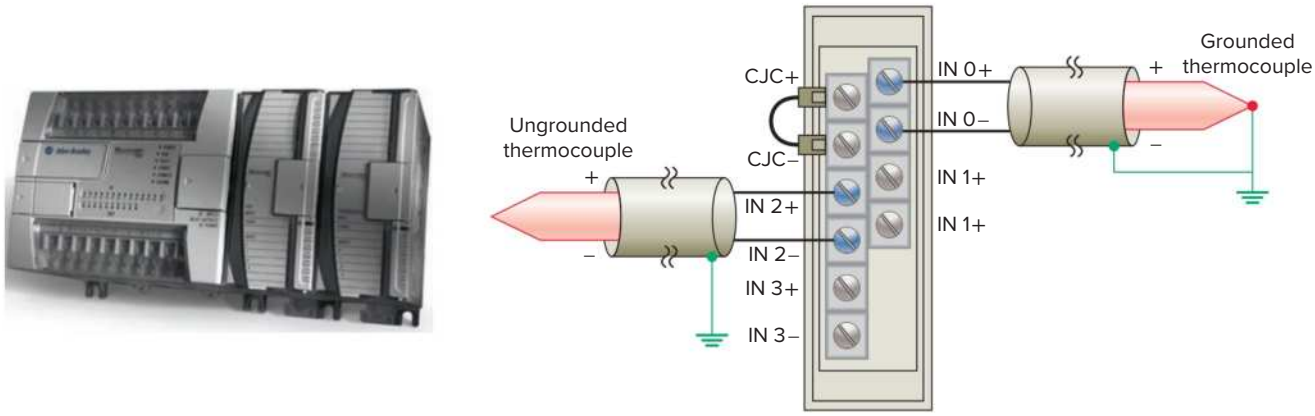


Figure 2-22 MicroLogix 4-channel analog thermocouple input module.
Source: Image Courtesy of Rockwell Automation, Inc.

and bipolar. ***Unipolar*** modules can accept an input signal that varies in the positive direction only. For example, if the field device outputs 0 to +10 V, then the unipolar modules would be used. Bipolar signals swing between a maximum negative value and a maximum positive value. For example, if the field device outputs −10 to +10 V a bipolar module would be used. The ***resolution*** of an analog input channel refers to the smallest change in input signal value that can be sensed and is based on the number of bits used in the digital representation. Analog input modules must produce a range of digital values between a maximum and minimum value to represent the analog signal over its entire span. Typical specifications are as follows:

Span of analog input	Bipolar	10 V	−10 to +10 V
		5 V	−5 to +5 V
	Unipolar	10 V	0 to +10 V
		5 V	0 to +5 V
Resolution			0.3 mV

When connecting voltage sensing inputs, close adherence to specified requirements regarding wire length is important to minimize signal degrading and the effects of electromagnetic noise interference induced along the connecting conductors. Current input signals, which are not as sensitive to noise as voltage signals, are typically not distance limited. Current sensing input modules typically accept analog data over the range of 4 to 20 mA, but can accommodate signal ranges of –20 to +20 mA. The loop power may be supplied by the sensor or may be provided by the analog output module as illustrated in Figure 2-23. Shielded twisted pair cable is normally recommended for connecting any type of analog input signal.

Field devices that provide an analog output as their signal are usually connected to transmitters, which in turn send the analog signal to the module, as illustrated in Figure 2-24. A **transducer** converts a field device's variable (e.g., pressure, temperature etc.) into a very low-level electric signal (current or voltage) that can be amplified by a **transmitter** and then input into the analog module.

The method user to wire two-, three-, and four-wire sensors to an analog input module is illustrated in Figure 2-25. The module does not provide loop power for analog inputs. A separate power that matches the transmitter specifications is used. All analog common

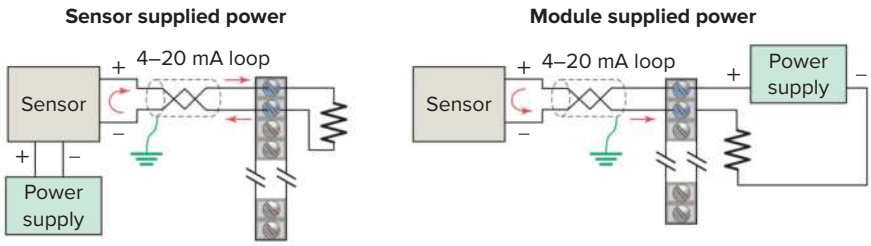


Figure 2-23 Sensor and analog module supplied power.

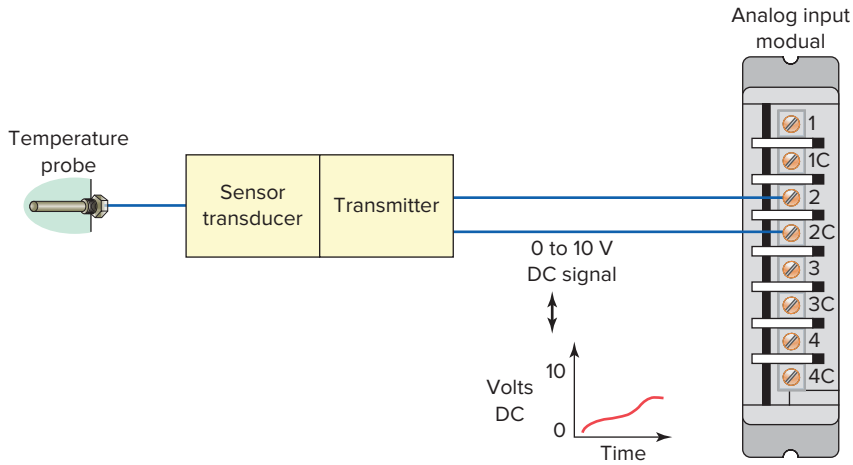


Figure 2-24 Analog input module circuit.

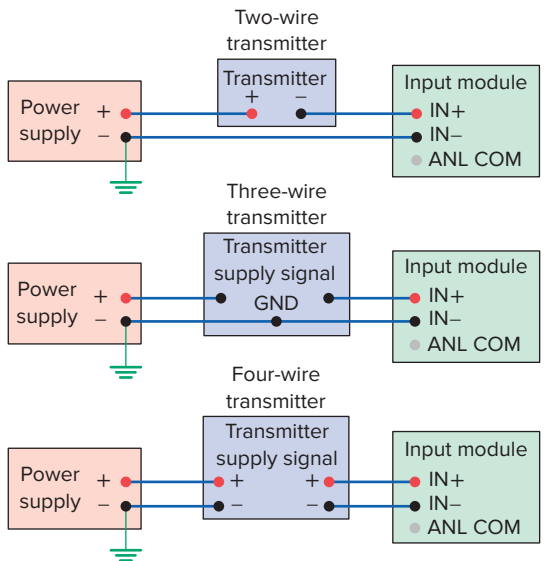


Figure 2-25 Wiring two-, three-, and four-wire sensors to an analog input module.

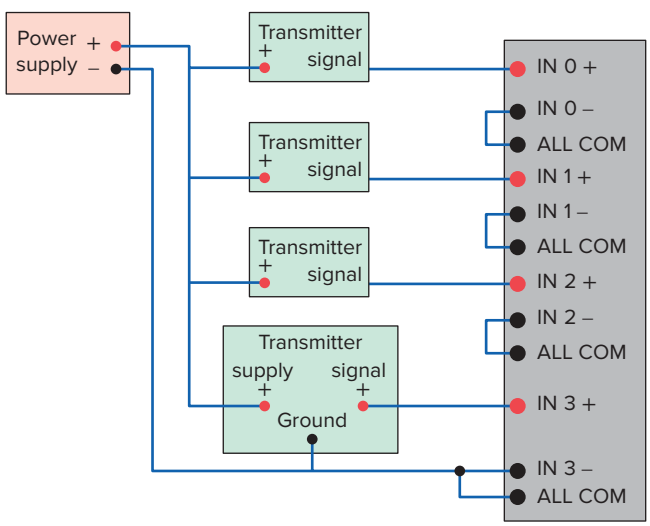


Figure 2-26 Wiring single-ended analog input devices.

(ANL COM) points are electrically connected together inside the module but not to earth ground. When wiring single-ended analog input devices to the analog input card, the number of total wires necessary can be limited by using the ANALOG COMMON terminal, as shown in Figure 2-26. Note that differential inputs are more immune to noise than single-ended inputs.

The ***analog output interface module*** receives from the processor digital data, which are converted into a proportional voltage or current to control an analog field device. The transition of a digital signal to analog values is accomplished by a **digital-to-analog (D/A)** converter, the main element of the analog output module. An analog output signal is a continuous and changing signal that is varied under the control of the PLC program. Common devices controlled by a PLC analog output module include instruments, control valves, chart recorder, electronic drives, and other types of control devices that respond to analog signals. They employ standard analog output ranges such as ±5 V, ±10 V, 0 to 5 V, 0 to 10 V, 4 to 20 mA, or 0 to 20 mA.

Figure 2-27 illustrates the use of analog I/O modules in a typical PLC control system. In this application the

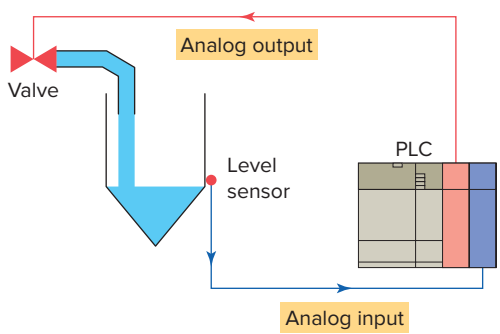


Figure 2-27 Typical analog I/O control system.

PLC controls the amount of fluid placed in a holding tank by adjusting the percentage of the valve opening. The analog output from the PLC is used to control the flow by controlling the amount of the valve opening. The valve is initially open 100%. As the fluid level in the tank approaches the preset point, the processor modifies the output, which adjusts the valve to maintain a set point.

Transducers produce either voltage or current proportional to some engineering units such as temperature (°C or °F), pressure (lb/in^2), distance (cm), etc. **Scaling** refers to changing a quantity from one notation to another and involves:

Engineering units: The units a human uses and understands

Transducer units: Either a voltage or current

Binary, raw, or machine units: The units the processor requires

The SCP (Scale with Parameters) instruction in RSLogix 500 is used to produce a scaled output value that has a linear relationship between the input and scaled values. It allows you to take an analog input from a sensor and scale it to the output units you require. Figure 2-28 illustrates a typical application involving temperature measurement. Setting up the SCP instruction to calculate the scaled temperature value in degrees Celsius can be summarized as follows:

- The *Input* parameter is the value to be scaled (in this case analog input I:1.1)
- The *Input Min* parameter is the value that is read by the analog card when the input is − 10V (in this case −32768)
- The *Input Max* parameter is the value that is read by the analog card when the input is 10V (in this case 32767)
- The *Scaled Min* parameter is the lowest value you want the SCP to calculate (in this case −100)
- The *Scaled Max* parameter is the highest value you want the SCP to calculate (in this case 200)
- The *Scaled Output* parameter is the address where you want to store the result of the SCP (in this case N7:60)

The SCP instruction in Figure 2-29 is used to scale the analog output to a proportional valve. The instruction directs the analog output to provide a 4 to 20mA signal, which is scaled to the valve position based on a percentage between 0 and 100. The module is scaled to represent 4 mA as the low signal and 20 mA as the high signal.

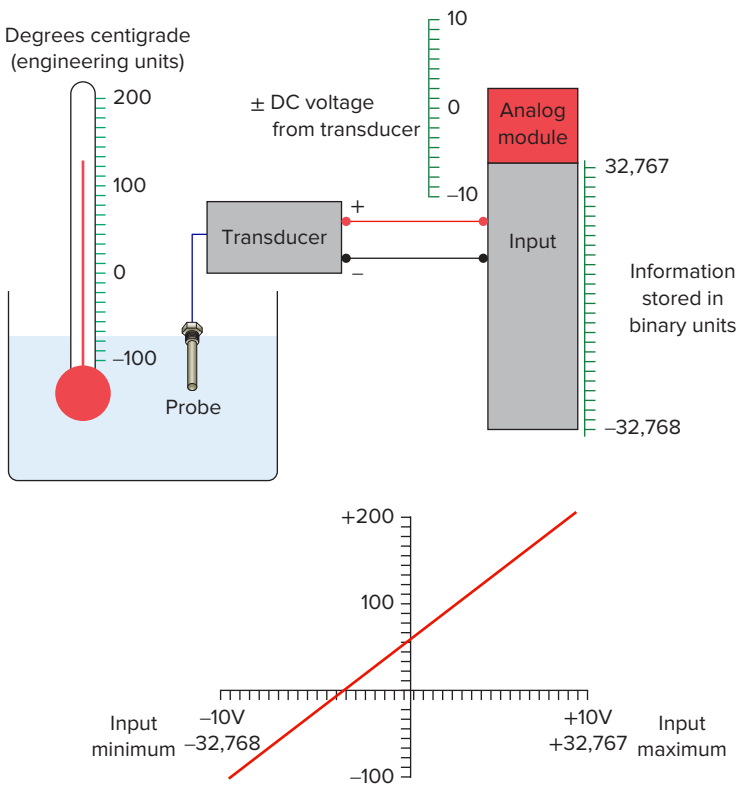


Figure 2-28 Measuring temperature.

SCP
Scale with parameters
SCP

Input	I:1.1
Input_Min	–32768
Input_Max	32767
Scaled_Min	–100
Scaled_Max	200
Output	N7:60

SCP
Scale with parameters
SCP

Input	N7:21
Input_Min	0
Input_Max	100
Scaled_Min	6242
Scaled_Max	31208
Output	O:1.0

Figure 2-29 Scaling the analog output to a proportional valve.

Scaling allows you to configure the module so that 4 mA returns a value of 0% in engineering units and 20 mA returns a value of 100% in engineering units. The execution of the instruction can be summarized as follows:

- The proportional valve is connected to the PLC output O:1.0.
- A 4 to 20 mA signal varies in magnitude to operate the valve from closed to 100% open.
- The percent of the valve open can be found in location N7:21.
- The PLC analog module provides a 4 to 20 mA output signal for a number from 6,242 to 31,208.

2.4 Special I/O Modules

Many different types of I/O modules have been developed to meet special needs. These include:

HIGH-SPEED COUNTER MODULE

The high-speed counter module is used to provide an interface for applications requiring counter speeds that surpass the capability of the PLC ladder program. High-speed counter modules are used to count pulses (Figure 2-30) from sensors, encoders, and switches that operate at very high speeds. They have the electronics needed to count independently of the processor. A typical count rate available is 0 to 100 kHz, which means the module would be able to count 100,000 pulses per second.

THUMBWHEEL MODULE

The thumbwheel module allows the use of thumbwheel switches (Figure 2-31) for feeding information to the PLC to be used in the control program.

Figure 2-31 Thumbwheel switch.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com.**

TTL MODULE

The TTL module allows the transmitting and receiving of TTL (Transistor-Transistor-Logic) signals. This module allows devices that produce TTL-level signals to communicate with the PLC's processor.

ENCODER-COUNTER MODULE

An encoder-counter module allows the user to read the signal from an encoder (Figure 2-32) on a real-time basis and stores this information so it can be read later by the processor.

BASIC OR ASCII MODULE

The BASIC or ASCII module runs user-written BASIC and C programs. These programs are independent of the PLC processor and provide an easy, fast interface between remote foreign devices and the PLC processor. Typical applications include interfaces to bar code readers, robots, printers, and displays.

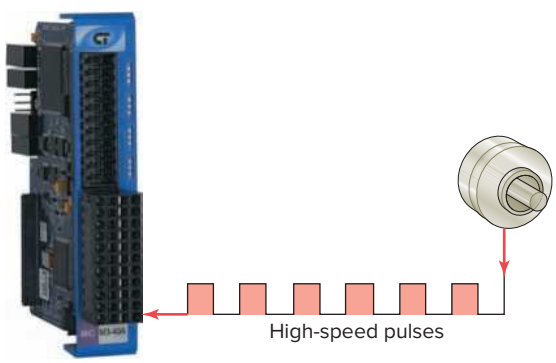


Figure 2-30 High-speed counter module.
Source: Courtesy Control Technology Corporation.

Figure 2-32 Encoder.
Source: Photo courtesy of Allied Motion Technologies, Inc.

Figure 2-33 Stepper-motor.
Source: Courtesy Sherline Products.

STEPPER-MOTOR MODULE

The stepper-motor module provides pulse trains to a stepper-motor translator, which enables control of a stepper motor (Figure 2-33). The commands for the module are determined by the control program in the PLC.

BCD-OUTPUT MODULE

The BCD-output module enables a PLC to operate devices that require BCD-coded signals such as seven-segment displays (Figure 2-34).

Some special modules are referred to as *intelligent I/O* because they have their own microprocessors on board that can function in parallel with the PLC. These include:

PID MODULE

The proportional-integral-derivative (PID) module (Figure 2-35) is used in process control applications that incorporate PID algorithms. An algorithm is a complex program based on mathematical calculations. A PID module allows process control to take place outside the CPU. This arrangement prevents the CPU from being burdened with complex calculations. The basic function of this module is to provide the control action required to maintain a process variable such as temperature, flow, level, or speed within set limits of a specified set point.

Figure 2-35 PID module.
Source: Courtesy Red Lion Controls.

MOTION AND POSITION CONTROL MODULE

Motion and position control modules are used in applications involving accurate high-speed machining and packaging operations. Intelligent position and motion control modules permit PLCs to control stepper and servo motors. These systems require a drive, which contains the power electronics that translate the signals from the PLC module into signals required by the motor (Figure 2-36).

COMMUNICATION MODULES

Serial communications modules (Figure 2-37) are used to establish point-to-point connections with other intelligent devices for the exchange of data. Such connections are normally established with computers, operator stations, process control systems, and other PLCs. Communication modules allow the user to connect the PLC to high-speed local networks that may be different from the network communication provided with the PLC.

Figure 2-34 Seven-segment display.
Source: Courtesy Red Lion Controls.

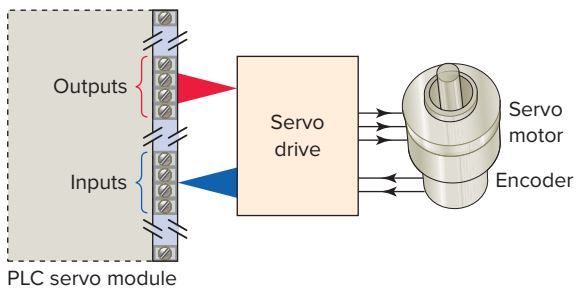


Figure 2-36 PLC servo module.

Figure 2-37 Serial communications module.
Source: Photo courtesy Automation Direct, **www.automationdirect.com.**

2.5 I/O Specifications

Manufacturers' specifications provide information about how an interface device is correctly and safely used. These specifications place certain limitations not only on the I/O module but also on the field equipment that it can operate. Some PLC systems support *hot swappable* I/O modules designed to be changed with the power on and the PLC operating. The following is a list of some typical manufacturers' I/O specifications, along with a short description of what is specified.

Typical Discrete I/O Module Specifications

NOMINAL INPUT VOLTAGE

This discrete input module voltage value specifies the magnitude (e.g., 5, 24, 230 V) and type (AC or DC) of user-supplied voltage that a module is designed to accept. Input modules are typically designed to operate correctly without damage within a range of plus or minus 10% of the input voltage rating. With DC input modules, the input voltage may also be expressed as an operating range (e.g., 24 to 60 V DC) over which the module will operate.

INPUT THRESHOLD VOLTAGES

This discrete input module specification specifies two values: a minimum ON-state voltage that is the minimum voltage at which logic 1 is recognized as absolutely ON; and a maximum OFF-state voltage which is the voltage at which logic 0 is recognized as absolutely OFF.

NOMINAL CURRENT PER INPUT

This value specifies the minimum input current that the discrete input devices must be capable of driving to operate the input circuit. This input current value, in conjunction with the input voltage, functions as a threshold to protect against detecting noise or leakage currents as valid signals.

AMBIENT TEMPERATURE RATING

This value specifies what the maximum temperature of the air surrounding the I/O modules should be for best operating conditions.

INPUT ON/OFF DELAY

Also known as *response time,* this value specifies the maximum time duration required by an input module's circuitry to recognize that a field device has switched ON (input ON-delay) or switched OFF (input OFF-delay). This delay is a result of filtering circuitry provided to protect against contact bounce and voltage transients. This input delay is typically in the 9 to 25 ms range.

OUTPUT VOLTAGE

This AC or DC value specifies the magnitude (e.g., 5 V, 115 V, 230 V) and type (AC or DC) of user-supplied voltage at which a discrete output module is designed to operate. The output field device that the module interfaces to the PLC must be matched to this specification. Output modules are typically designed to operate within a range of plus or minus 10% of the nominal output voltage rating.

OUTPUT CURRENT

These values specify the maximum current that a single output and the module as a whole can safely carry under load (at rated voltage). This rating is a function of the module's components and heat dissipation characteristics. A device drawing more than the rated output current results in overloading, causing the output fuse to blow. As an example, the specification may give each output a current limit of 1 A. The overall rating of the module current will normally be less than the total of the individuals. The overall rating might be 6 A because each of the eight devices would not normally draw their 1 A at the same time. Other names for the output current rating are *maximum continuous current* and *maximum load current.*

INRUSH CURRENT

An inrush current is a momentary surge of current that an AC or DC output circuit encounters when energizing inductive, capacitive, or filament loads. This value specifies the maximum inrush current and duration (e.g., 20 A for 0.1 s) for which an output circuit can exceed its maximum continuous current rating.

SHORT CIRCUIT PROTECTION

Short circuit protection is provided for AC and DC output modules by either fuses or some other current-limiting circuitry. This specification will designate whether the particular module's design has individual protection for each circuit or if fuse protection is provided for groups (e.g., 4 or 8) of outputs.

LEAKAGE CURRENT

This value specifies the amount of current still conducting through an output circuit even after the output has been turned off. Leakage current is a characteristic exhibited by solid-state switching devices such as transistors and triacs and is normally 1 to 2 mA. Leakage current is normally not large enough to falsely trigger an output device but must be taken into consideration when switching very low current sensitive devices.

ELECTRICAL ISOLATION

Recall that I/O module circuitry is electrically isolated to protect the low-level internal circuitry of the PLC from high voltages that can be encountered from field device connections. The specification for electrical isolation, typically 1500 or 2500 V AC, rates the module's capacity for sustaining an excessive voltage at its input or output terminals.

POINTS PER MODULE

This specification defines the number of field inputs or outputs that can be connected to a single module. Most commonly, a discrete module will have 8, 16, or 32 circuits; however, low-end controllers may have only 2 or 4 circuits. Modules with 32 or 64 input or output bits are referred to as *high-density* modules. Some modules provide more than one common terminal, which allows the user to use different voltage ranges on the same card as well as to distribute the current more effectively.

BACKPLANE CURRENT DRAW

This value indicates the amount of current the module requires from the backplane. The sum of the backplane current drawn for all modules in a chassis is used to select the appropriate chassis power supply rating.

Typical Analog I/O Module Specifications

CHANNELS PER MODULE

Whereas individual circuits on discrete I/O modules are referred to as points, circuits on analog I/O modules are often referred to as channels. These modules normally have 4, 8, or 16 channels. Analog modules may allow for either single-ended or differential connections. *Single-ended* connections use a single ground terminal for all channels or for groups of channels. *Differential* connections use a separate positive and negative terminal for each channel. If the module normally allows 16 single-ended connections, it will generally allow only 8 differential connections. Single-ended connections are more susceptible to electrical noise.

INPUT CURRENT/VOLTAGE RANGE(S)

These are the voltage or current signal ranges that an analog input module is designed to accept. The input ranges must be matched accordingly to the varying current or voltage signals generated by the analog sensors.

OUTPUT CURRENT/VOLTAGE RANGE(S)

This specification defines the current or voltage signal ranges that a particular analog output module is designed to output under program control. The output ranges must be matched according to the varying voltage or current signals that will be required to drive the analog output devices.

INPUT PROTECTION

Analog input circuits are usually protected against accidentally connecting a voltage that exceeds the specified input voltage range.

RESOLUTION

The resolution of an analog I/O module specifies how accurately an analog value can be represented digitally. This specification determines the smallest measurable unit of current or voltage. The higher the resolution (typically specified in bits or mV), the more accurately an analog value can be represented.

INPUT IMPEDANCE AND CAPACITANCE

For analog I/Os, these values must be matched to the external device connected to the module. Typical ratings are in Megohm (MΩ) and picofarads (pF).

COMMON-MODE REJECTION

Noise is generally caused by electromagnetic interference, radio frequency interference, and ground loops. Common-mode noise rejection applies only to differential inputs and

refers to an analog module's ability to prevent noise from interfering with data integrity on a single channel and from channel to channel on the module. Noise that is picked up equally in parallel wires is rejected because the difference is zero. Twisted pair wires are used to ensure that this type of noise is equal on both wires. Common-mode rejection is normally expressed in decibels or as a ratio.

2.6 The Central Processing Unit (CPU)

The central processing unit (CPU) is built into single-unit fixed PLCs while modular rack types typically use a plug-in module. CPU, controller, and processor are all terms used by different manufacturers to denote the same module that performs basically the same functions. Processors vary in processing speed and memory options. A processor module can be divided into two sections: the **CPU section** and the **memory section** (Figure 2-38). The CPU section executes the program and makes the decisions needed by the PLC to operate and communicate with other modules. The memory section electronically stores the PLC program along with other retrievable digital information.

The PLC power supply provides the necessary power (typically 5 VDC) to the processor and I/O modules plugged into the backplane of the rack (Figure 2-39). Power supplies are available for most voltage sources encountered. The power supply converts 115 VAC or 230 VAC into the usable DC voltage required by the CPU, memory, and I/O electronic circuitry. PLC power supplies are normally designed to withstand momentary losses of power without affecting the operation of the PLC. *Hold-up time,* which is the length of time a PLC can tolerate a power loss, typically ranges from 10 ms to 3 s.

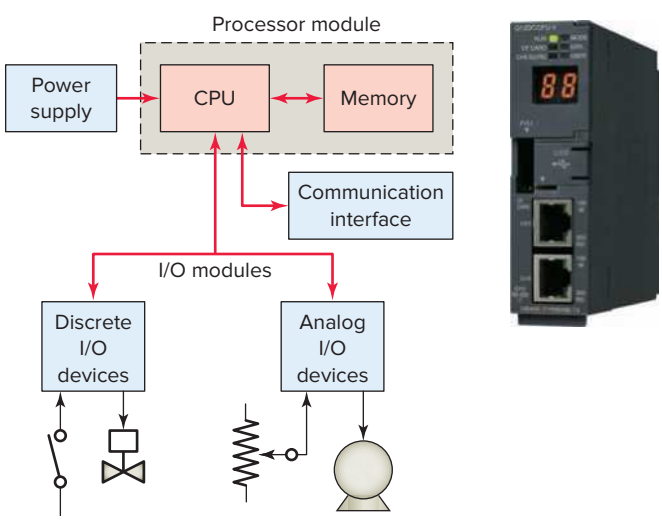


Figure 2-38 Sections of a PLC processor module.
Source: Courtesy Mitsubishi Automation.

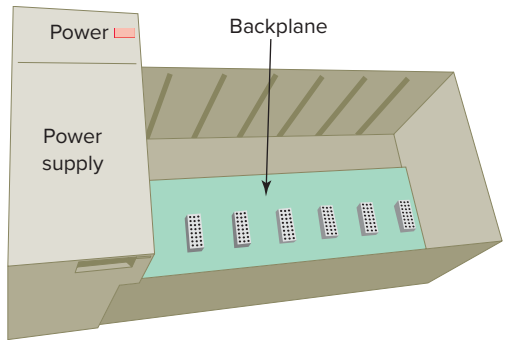


Figure 2-39 PLC power supply.

The CPU contains the similar type of microprocessor found in a personal computer. The difference is that the program used with the microprocessor is designed to facilitate industrial control rather than provide general-purpose computing. The CPU executes the operating system, manages memory, monitors inputs, evaluates the user logic (ladder program), and turns on the appropriate outputs.

The CPU of a PLC system may contain more than one processor. One advantage of using multiprocessing is that the overall operating speed is improved. Each processor has its own memory and programs, which operate simultaneously and independently. In such configurations the scan of each processor is parallel and independent thus reducing the total response time. Fault-tolerant PLC systems support dual processors for critical processes. These systems allow the user to configure the system with ***redundant*** (two) processors, which allows transfer of control to the second processor in the event of a processor fault.

Associated with the processor unit will be a number of status LED indicators to provide system diagnostic information to the operator (Figure 2-40). Also, a keyswitch may be provided that allows you to select one of the following three modes of operation: RUN, PROG, and REM.

RUN Position

- Places the processor in the Run mode
- Executes the ladder program and energizes output devices
- Prevents you from performing online program editing in this position
- Prevents you from using a programmer/operator interface device to change the processor mode

PROG Position

- Places the processor in the Program mode
- Prevents the processor from scanning or executing the ladder program, and the controller outputs are de-energized

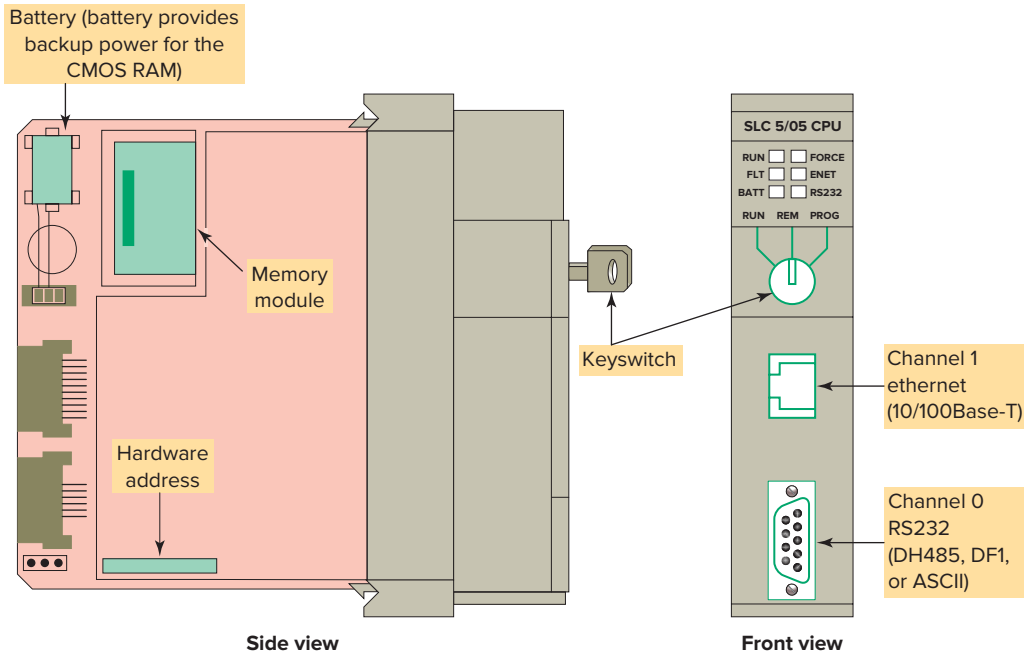


Figure 2-40 Typical processor module.

- Allows you to perform program entry and editing
- Prevents you from using a programmer/operator interface device to change the processor mode

REM Position

- Places the processor in the Remote mode: either the REMote Run, REMote Program, or REMote Test mode
- Allows you to change the processor mode from a programmer/operator interface device
- Allows you to perform online program editing

The processor module also contains circuitry to communicate with the programming device. Somewhere on the module you will find a connector that allows the PLC to be connected to an external programming device. The decision-making capabilities of PLC processors go far beyond simple logic processing. The processor performs other functions such as timing, counting, latching, comparing, motion control and complex math functions.

PLC processors have changed constantly due to advancements in computer technology and greater demand from applications. Today, processors are faster and have additional instructions added as new models are introduced. Because PLCs are microprocessor based, they can be made to perform tasks that a computer can do. In addition to their control functions, PLCs can be networked to do supervisory control and data acquisition (SCADA).

Many electronic components found in processors and other types of PLC modules are sensitive to *electrostatic* voltages that can degrade their performance or damage them. The following static control procedures should be followed when handling and working with static-sensitive devices and modules:

- Ground yourself by touching a conductive surface before handling static-sensitive components.
- Wear a wrist strap that provides a path to bleed off any charge that may build up during work.
- Be careful not to touch the backplane connector or connector pins of the PLC system (always handle the circuit cards by the edge if possible).
- Be careful not to touch other circuit components in a module when you configure or replace its internal components.
- When not in use, store module in its static-shield bag.
- If available, use a static-safe work station.

2.7 Memory Design

Memory is the element that stores information, programs, and data in a PLC. The user memory of a PLC includes space for the user program as well as addressable memory locations for storage of data. Data are stored in memory locations by a process called *writing*. Data are retrieved from memory by what is referred to as *reading*.

The complexity of the program determines the amount of memory required. Memory elements store individual pieces of information called *bits* (for *binary digits*). The amount of memory capacity is specified in increments of

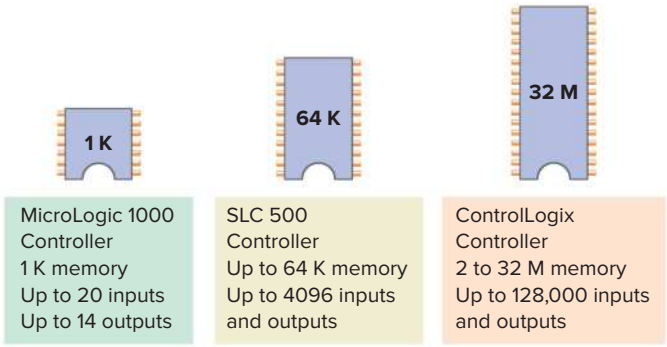


Figure 2-41 Typical PLC memory sizes.

1000 or in "K" increments, where 1 K is 1024 bytes of memory storage (a byte is 8 bits).

The program is stored in the memory as 1s and 0s, which are typically assembled in the form of 16-bit words. Memory sizes are commonly expressed in thousands of words that can be stored in the system; thus 2 K is a memory of 2000 words, and 64 K is a memory of 64,000 words. The memory size varies from as small as 1 K for small systems to 32 MB for very large systems (Figure 2-41). Memory capacity is an important prerequisite for determining whether a particular processor will handle the requirements of the specific application.

Memory *location* refers to an address in the CPU's memory where a binary word can be stored. A word usually consists of 16 bits. Each binary piece of data is a bit and eight bits make up one byte (Figure 2-42). Memory

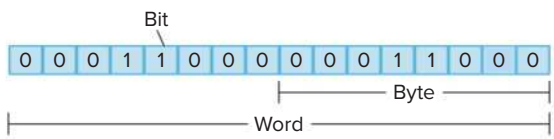


Figure 2-42 Memory bit, byte, and word.

utilization refers to the number of memory locations required to store each type of instruction. A rule of thumb for memory locations is one location per coil or contact. One K of memory would then allow a program containing 1000 coils and contacts to be stored in memory.

The memory of a PLC may be broken into sections that have specific functions. Sections of memory used to store the status of inputs and outputs are called input status files or tables and output status files or tables (Figure 2-43). These terms simply refer to a location where the status of an input or output device is stored. Each bit is either a 1 or 0, depending on whether the input is open or closed. A closed contact would have a binary 1 stored in its respective location in the input table, whereas an open contact would have a 0 stored. A lamp that is ON would have a 1 stored in its respective location in the output table, whereas a lamp that is OFF would have a 0 stored. Input and output image tables are constantly being revised by the CPU. Each time a memory location is examined, the table changes if the contact or coil has changed state.

PLCs execute memory-checking routines to be sure that the PLC memory has not been corrupted. This memory checking is undertaken for safety reasons. It helps ensure that the PLC will not execute if memory is corrupted.

2.8 Memory Types

Memory can be placed into two general categories: volatile and nonvolatile. Volatile memory will lose its stored information if all operating power is lost or removed. Volatile memory is easily altered and is quite suitable for most applications when supported by battery backup.

Nonvolatile memory has the ability to retain stored information when power is removed accidentally or intentionally. As the name implies, programmable logic controllers have programmable memory that allows users

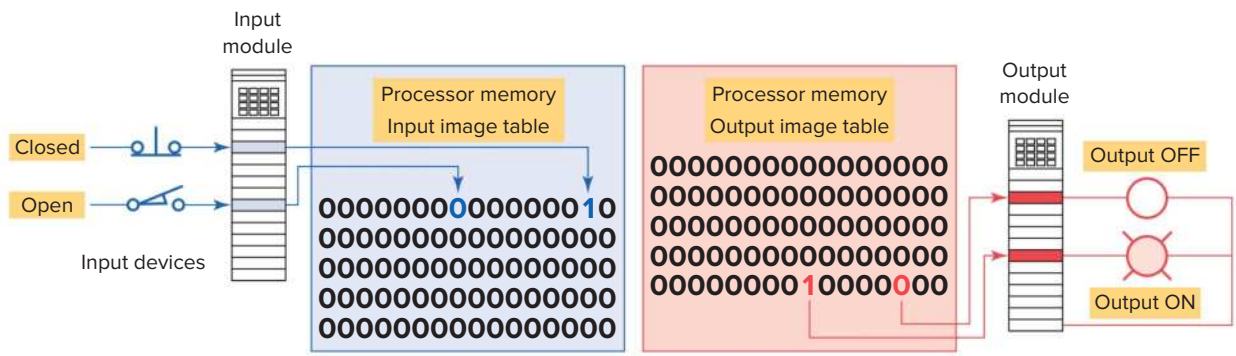


Figure 2-43 Input and output tables.

to develop and modify control programs. This memory is made nonvolatile so that if power is lost, the PLC holds its programming.

Read Only Memory (ROM) stores programs, and data cannot be changed after the memory chip has been manufactured. ROM is normally used to store the programs and data that define the capabilities of the PLC. ROM memory is nonvolatile, meaning that its contents will not be lost if power is lost. ROM is used by the PLC for the operating system. The operating system is burned into ROM by the PLC manufacturer and controls the system software that the user uses to program the PLC. When Allen Bradley burns the operating system into memory it is called PROM (programmable read-only memory).

Random Access Memory (RAM), sometimes referred to as *read-write (R/W) memory,* is designed so that information can be written into or read from the memory. RAM is used as a temporary storage area of data that may need to be quickly changed. RAM is volatile, meaning that the data stored in RAM will be lost if power is lost. A battery backup is required to avoid losing data in the event of a power loss (Figure 2-44). Most PLCs use CMOS-RAM technology for user memory. CMOS-RAM chips have very low current draw and can maintain memory with a lithium battery for an extended time, two to five years in many cases. Some processors have a capacitor that provides at least 30 minutes of battery backup when the battery is disconnected and power is OFF.

Erasable Programmable Read-Only Memory (EPROM) provides some level of security against unauthorized or unwanted changes in a program. EPROMs are designed so that data stored in them can be read, but not easily altered without special equipment. For example, UV EPROMs (ultraviolet erasable programmable read-only memory) can only be erased with an ultraviolet light. EPROM memory is used to back up, store, or transfer PLC programs.

Electrically erasable programmable read-only memory (EEPROM) is a nonvolatile memory that offers the same programming flexibility as does RAM. The EEPROM can be electrically overwritten with new data instead of being erased with ultraviolet light. Because the EEPROM is nonvolatile memory, it does not require battery backup. It provides permanent storage of the program and can be changed easily using standard programming devices. Typically, an EEPROM memory module is used to store, back up, or transfer PLC programs (Figure 2-45).

Flash EEPROMs are similar to EEPROMs in that they can only be used for backup storage. The main difference comes in the flash memory: they are extremely fast at saving and retrieving files. In addition, they do not need to be physically removed from the processor for reprogramming; this can be done using the circuitry within the processor module in which they reside. Flash memory is also sometimes built into the processor module (Figure 2-46), where it automatically backs up parts of RAM. If power fails while a PLC with flash memory is running, the PLC will resume running without having lost any working data after power is restored.

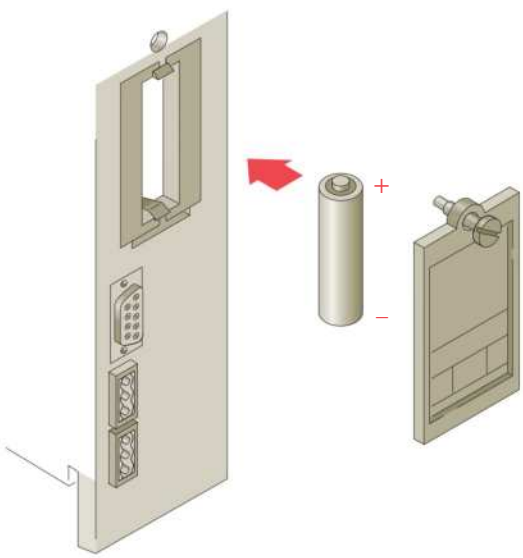


Figure 2-44 Battery used to back up processor RAM.

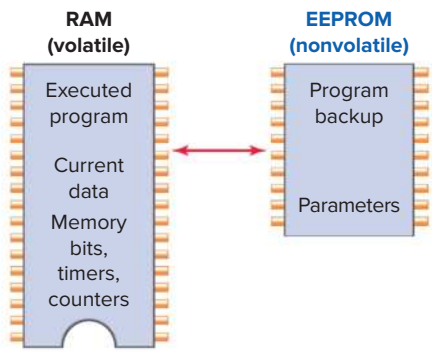


Figure 2-45 EEPROM memory module is used to store, back up, or transfer PLC programs.

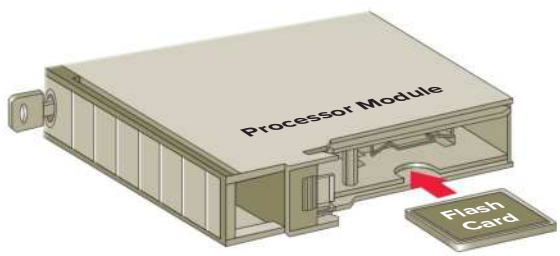


Figure 2-46 Flash memory card installed in a socket on the processor.

2.9 Programming Terminal Devices

A programming terminal device is needed to enter, modify, and troubleshoot the PLC program. PLC manufacturers use various types of programming devices. The simplest type is the hand-held type programmer shown in Figure 2-47. This proprietary programming device has a connecting cable so that it can be plugged into a PLC's programming port. Certain controllers use a plug-in panel rather than a hand-held device.

Hand-held programmers are compact, inexpensive, and easy to use. These units contain multifunction keys and a liquid-crystal display (LCD) or light-emitting diode (LED) window. There are usually keys for instruction entering and editing, and navigation keys for moving around the program. Hand-held programmers have limited display capabilities. Some units will display only the last instruction that has been programmed, whereas other units will display from two to four rungs of ladder logic. So-called intelligent hand-held programmers are designed to support a certain family of PLCs from a specific manufacturer.

The most popular method of PLC programming is to use a personal computer (PC) in conjunction with the manufacturer's programming software (Figure 2-48). Typical capabilities of the programming software include online and offline program editing, online program monitoring, program documentation, diagnosing malfunctions in the PLC, and troubleshooting the controlled system. Hard-copy reports generated in the software can be printed on the computer's printer. Most software packages will not allow you to develop programs on another manufacturer's PLC. In some cases, a single manufacturer will have multiple PLC families, each requiring its own software to program.

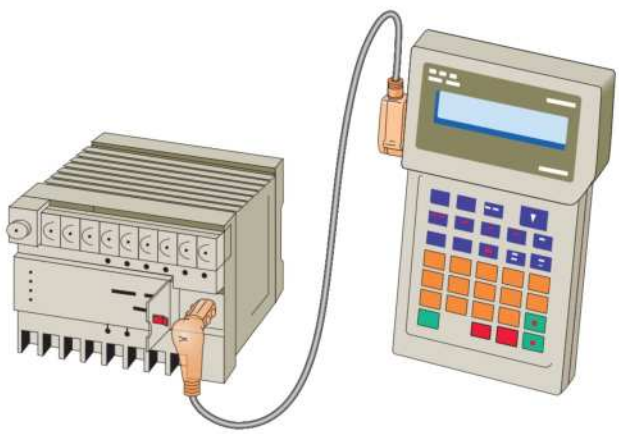


Figure 2-47 Hand-held programming terminal.

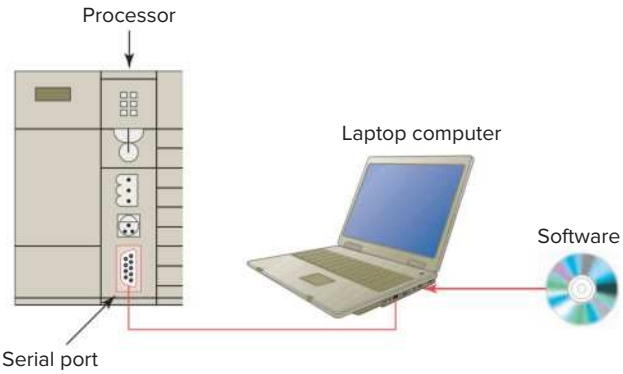


Figure 2-48 Personal computer used as the programming device.

2.10 Recording and Retrieving Data

Printers are used to provide hard-copy printouts of the processor's memory in ladder program format. Lengthy ladder programs cannot be shown completely on a screen. Typically, a screen shows a maximum of five rungs at a time. A printout can show programs of any length and analyze the complete program.

The PLC can have only one program in memory at a time. To change the program in the PLC, it is necessary either to enter a new program directly from the keyboard or to download one from the computer hard drive. Some CPUs support the use of a memory cartridge that provides portable EEPROM storage for the user program (Figure 2-49). The cartridge can be used to copy a program from one PLC to another similar type PLC.

2.11 Human Machine Interfaces (HMIs)

In the past, the typical user interface to a control system consisted of a panel with switches, pushbuttons, pilot lights, gauges, analog meters, and the like. With the advent of

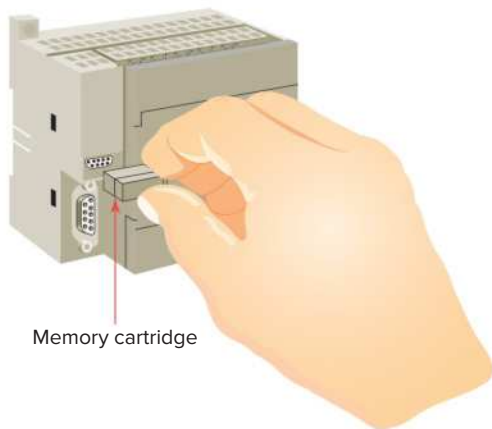


Figure 2-49 Memory cartridge provides portable storage for user program.

Figure 2-50 Human Machine Interface (HMI).
Source: Courtesy of Nercon.

digital control systems, larger hard-wired panels have been replaced by a computer screen with process graphics and operator commands entered via a keyboard (Figure 2-50).

Human machine interfaces give the ability to the operator and to management to view the operation in real time. Through personal computer–based setup software, you can configure display screens to:

- Replace hardwired pushbuttons and pilot lights with realistic-looking icons. The machine operator need only touch the display panel to activate the pushbuttons.
- Show operations in graphic format for easier viewing.
- Allow the operator to change timer and counter presets by touching the numeric keypad graphic on the touch screen.
- Show alarms, complete with time of occurrence and location.
- Display variables as they change over time.

The Allen-Bradley Pico GFX-70 controller, shown in Figure 2-51, serves as a controller with HMI capabilities. This device consists of three modular parts: an HMI, processor/power supply, and I/O modules.

The display/keypad can be used as an operator interface or can be linked to control operations to provide real-time feedback. It has the ability to show text, date and time, as well as custom messages and bitmap graphics,

Figure 2-51 Allen-Bradley Pico GFX-70 controller.
Source: Image Courtesy of Rockwell Automation, Inc.

allowing operators to acknowledge fault messages, enter values, and initiate actions. Users can create both the control program and HMI functionality using a personal computer with PicoSoft Pro software installed or the controller's on-board display buttons.

Human Machine Interfaces (HMIs), are also referred to as User Interface, Operator Panel, or Terminal and provide a means of *controlling*, *monitoring, managing*, and/or visualizing device processes. They can be located on the machine or in centralized control rooms. The general structure of an HMI package is shown in Figure 2-52. The tag database variables are programmed to interact with the graphic screen objects and communicate with the PLC through the I/O server.

The design of the HMI application plays a critical role in determining the operator's ability to effectively manage the operation, particularly in response to abnormal situations. The major tasks in the development of an HMI application are:

- *Set up the communication with the PLC*. This involves configuring all necessary software and hardware components.

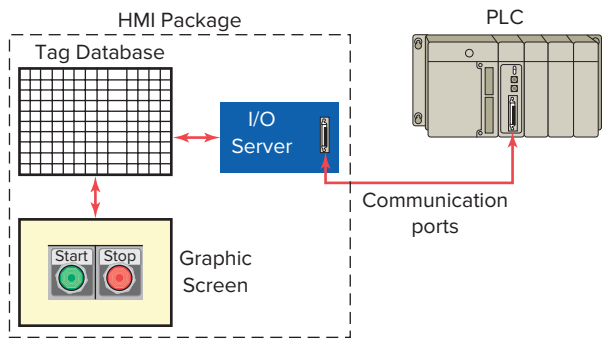


Figure 2-52 General structure of a HMI package.

- *Create the tag database*. Most HMI packages provide a way to import tags from the PLC programming software.
- *Insert the graphical objects on the screen*. Graphics are drawn or imported from a library of common objects.
- *Animate the objects*. There are two basic types of animation: user input and display. User input types allow an operator to change tag values. A display animation allows a value to be displayed and also allows an object to change shape, position, and color.

Many different types of HMI hardware and software features are available. These include:

HMI MONITOR AND ENCLOSURE

HMI operator panels typically contain monochrome or 256 color display screens. These systems often communicate directly with the PLC to read or write memory locations.

- A monochrome monitor uses one color for the background and another to display text or images on the screen.
- Color displays enable clearer process representation and in general brighten up their systems. The color convention for status and alarms should follow the same convention as their hardwired equivalents, namely:
  - *Red*—for alarm, danger, and stop
  - *Yellow*—for caution and risk of danger
  - *Green*—for ready, running, and safe condition
- Screen resolution is expressed as width × height, with the units in pixels.
- Screen memory is expressed in Megabytes (MB).
- The environmental certification refers to the type of electrical enclosure used to protect their contents from troublesome operating conditions such as dust, liquids, and extreme variations in temperature (Figure 2-53).
- The screen may or may not be touch-sensitive. The touch-sensitive screen allows for more devices and data to be displayed in a smaller area. Detailed information about an object can be accessed by touching the object.

ALARMS

Alarms are messages which indicate that a fault condition is present (Figure 2-54). An alarm summary can present a complete list of timestamped active alarms. Typically an alarm can exist in the following states:

Figure 2-53 HMI installed in an industrial environment.
Source: Photo Courtesy PC Enclosures, **http://www.pcenclosures.net.**

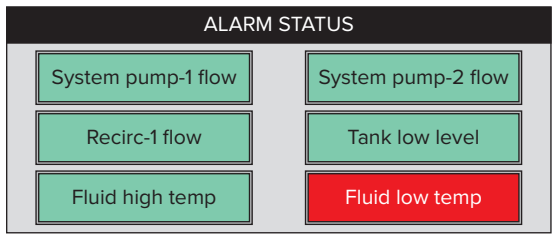


Figure 2-54 Typical alarm status screen.

- *Inactive*—The condition being monitored does not have any faults present, and there is no associated alarm message waiting to be acknowledged.
- *Active*—A fault condition is present, and the alarm message has not been acknowledged by the operator.
- *Acknowledged*—The fault condition is present, and the operator has acknowledged the alarm message.
- *OK* - The fault condition is no longer present, but the operator has not acknowledged the alarm message yet.

EVENT HISTORY

An event history presents a time-stamped list of all significant events that have occurred in the process. Many problems within the plant or equipment may occur when no one is monitoring the system, and intermittent problems may be difficult to diagnose without a history of previous issues.

TREND

Values of important process variables, such as flow, temperature, and production rate, over a period of time are shown by this type of display. This type of display provides the ability to chart the progress of the process in real time, providing the same function as a strip chart

recorder. For example, suppose you are monitoring pressure of a Pounds per Square Inch Gauge (psig) as shown in Figure 2-55. According to the table, you can see that it's OK right now, but that's all you know. This trend shows the pressure oscillating around a known good level. We may want to check on the cause of oscillation, but there appears to be no immediate problem.

Current Pressure	Alarm Level	Shutdown Level
235.2 psig	250 psig	300 psig

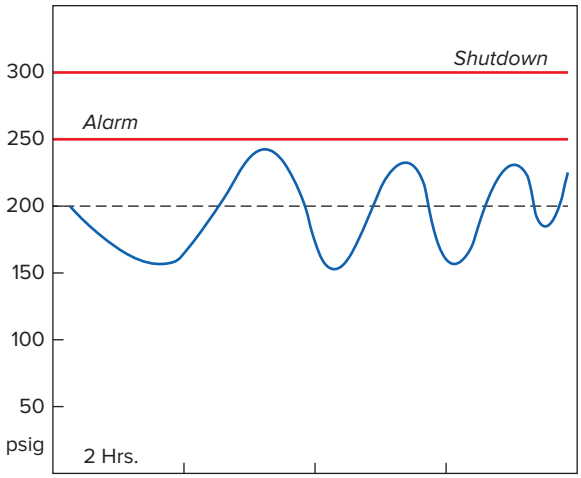


Figure 2-55 Trend monitoring of a pressure gauge.

GRAPHICS LIBRARY

The graphics library contained within an HMI development package provides buttons, lights, switches, sliders, meters, fills, and other graphic objects (Figure 2-56). It saves design time by providing graphics and faceplates for numerous industrial control devices that would otherwise have to be created manually. Librarian applications may include easy-to-use features for resizing, changing color scheme, and orientation of objects, as well as building your own graphics into the library.

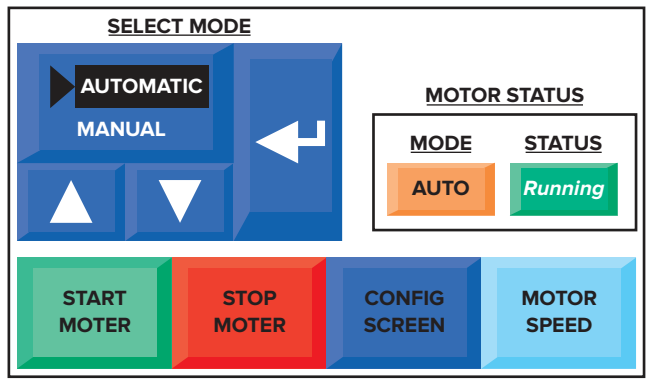


Figure 2-56 Typical motor control graphics.

CHAPTER 2 REVIEW QUESTIONS

1. What is the function of a PLC input interface module?
2. What is the function of a PLC output interface module?
3. With reference to a PLC rack:
   a. What is a *remote rack?*
   b. Why are remote racks used?
4. How does the processor identify the location of a specific input or output device?
5. List the three basic elements of rack/slot-based addressing.
6. Compare bit level and word level addressing.
7. In what way does tag-based addressing differ from rack/slot-based addressing?
8. What do PC-based control systems use to interface with field devices?
9. What type of I/O modules have both inputs and outputs connected to them?
10. In addition to field devices, what other connections are made to a PLC module?
11. Most PLC modules use plug-in wiring terminal strips. Why?
12. What are the advantage and the disadvantage of using high-density modules?
13. With reference to PLC discrete input modules:
    a. What types of field input devices are suitable for use with them?
    b. List three examples of discrete input devices.
14. With reference to PLC discrete output modules:
    a. What types of field output devices are suitable for use with them?
    b. List three examples of discrete output devices.
15. Explain the function of the backplane of a PLC rack.
16. What is the function of the optical isolator circuit used in discrete I/O module circuits?
17. Name the two distinct sections of an I/O module.
18. List four tasks performed by a discrete input module.
19. What electronic element can be used as the switching device for a 120 VAC discrete output interface module?
20. With reference to discrete output module current ratings:
    a. What is the maximum current rating for a typical 120 VAC output module?
    b. Explain one method of handling outputs with larger current requirements.
21. What electronic element can be used as the switching device for DC discrete output modules?
22. A discrete relay type output module can be used to switch either AC or DC load devices. Why?
23. With reference to sourcing and sinking I/O modules:
    a. What current relationship are the terms *sourcing* and *sinking* used to describe?
    b. If an I/O module is specified as a current-sinking type, then which type of field device (sinking or sourcing) it is electrically compatible with?
24. Compare discrete and analog I/O modules with respect to the type of input or output devices with which they can be used.
25. Explain the function of the analog-to-digital (A/D) converter circuit used in analog input modules.
26. Explain the function of the digital-to-analog (D/A) converter circuit used in analog output modules.
27. Name the two general sensing classifications for analog input modules.
28. List five common physical quantities measured by a PLC analog input module.
29. What type of cable is used when connecting a thermocouple to a voltage sensing analog input module? Why?
30. Explain the difference between a unipolar and bipolar analog input module.
31. The resolution of an analog input channel is specified as 0.3 mV. What does this tell you?
32. In what two ways can the loop power for current sensing input modules be supplied?
33. List three field devices that are commonly controlled by a PLC analog output module.
34. State one application for each of the following special I/O modules:
    a. High-speed counter module
    b. Thumbwheel module
    c. TTL module
    d. Encoder-counter module
    e. BASIC or ASCII module
    f. Stepper-motor module
    g. BCD-output module

35. List one application for each of the following intelligent I/O modules:
- **a.** PID module
- **b.** Motion and position control module
- **c.** Communication module

36. Write a short explanation for each of the following discrete I/O module specifications:
- **a.** Nominal input voltage
- **b.** Input threshold voltages
- **c.** Nominal current per input
- **d.** Ambient temperature rating
- **e.** Input ON/OFF delay
- **f.** Output voltage
- **g.** Output current
- **h.** Inrush current
- **i.** Short circuit protection
- **j.** Leakage current
- **k.** Electrical isolation
- **l.** Points per module
- **m.** Backplane current draw

37. Write a short explanation for each of the following analog I/O module specifications:
- **a.** Channels per module
- **b.** Input current/voltage range(s)
- **c.** Output current/voltage range(s)
- **d.** Input protection
- **e.** Resolution
- **f.** Input impedance and capacitance
- **g.** Common-mode rejection

38. Compare the function of the CPU and memory sections of a PLC processor.

39. With reference to the PLC chassis power supply:
- **a.** What conversion of power takes place within the power supply circuit?
- **b.** Explain the term *hold-up time* as it applies to the power supply.

40. Explain the purpose of a redundant PLC processor.

41. Describe three typical modes of operation that can be selected by the keyswitch of a processor.

42. State five other functions, in addition to simple logic processing, that PLC processors are capable of performing.

43. List five important procedures to follow when handling static-sensitive PLC components.

44. Define each of the following terms as they apply to the memory element of a PLC:
- **a.** writing
- **b.** reading
- **c.** bits
- **d.** location
- **e.** utilization

45. With reference to the I/O image tables:
- **a.** What information is stored in PLC input and output tables?
- **b.** What is the input status of a closed switch stored as?
- **c.** What is the input status of an open switch stored as?
- **d.** What is the status of an output that is ON stored as?
- **e.** What is the status of an output that is OFF stored as?

46. Why do PLCs execute memory-checking routines?

47. Compare the memory storage characteristics of volatile and nonvolatile memory elements.

48. What information is normally stored in the ROM memory of a PLC?

49. What information is normally stored in the RAM memory of a PLC?

50. What information is normally stored in an EEPROM memory module?

51. What are the advantages of a processor that utilizes a flash memory card?

52. List three functions of a PLC programming terminal device.

53. Give one advantage and one limitation to the use of hand-held programming devices.

54. What is required for a personal computer to be used as a PLC programming terminal?

55. List four important capabilities of PLC programming software.

56. How many programs can a PLC have stored in memory at any one time?

57. Outline four functions that an HMI interface screen can be configured to perform.

58. List the four major tasks in the development of an HMI application.

59. What information does an HMI trend display convey?

60. Define the term scaling as it applies to PLC inputs and outputs.

61. What is the function of a transducer?

62. In a tag based PLC memory structure, what is the function of a base tag and an alias tag?

CHAPTER 2 PROBLEMS

1. A discrete 120 VAC output module is to be used to control a 230 VDC solenoid valve. Draw a diagram showing how this could be accomplished using an interposing relay.
2. Assume a thermocouple, which supplies the input to an analog input module, generates a linear voltage of from 20 to 50 mV when the temperature changes from 750 to 1250°F. How much voltage will be generated when the temperature of the thermocouple is at 1000°F?
3. With reference to I/O module specifications:
   a. If the ON-delay time of a given discrete input module is specified as 12 ms, how much is this expressed in seconds?
   b. If the output leakage current of a discrete output module is specified as 950 μA, how much is this expressed in amperes?
   c. If the ambient temperature rating for an I/O module is specified as 60°C, how much is this expressed in degrees Fahrenheit?
4. Create a five-digit code using the SLC 500 rack/slot-based addressing format for each of the following:
   a. A pushbutton connected to terminal 5 of module group 2 located on rack 1.
   b. A lamp connected to terminal 3 of module group 0 located on rack 2.
5. Assume the triac of an AC discrete output module fails in the shorted state. How would this affect the device connected to this output?
6. A personal computer is to be used to program several different PLCs from different manufacturers. What would be required?

3

Number Systems and Codes

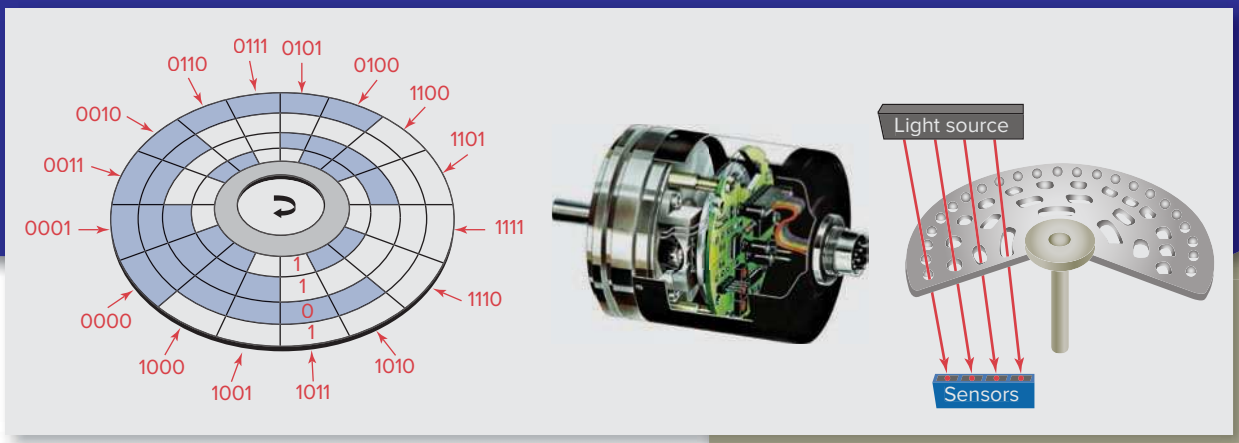


Photo Courtesy Baumer Electric.

Using PLCs requires us to become familiar with other number systems besides decimal. Some PLC models and individual PLC functions use other numbering systems. This chapter deals with some of these numbering systems, including binary, octal, hexadecimal, BCD, Gray, and ASCII codes. The basics of each system, as well as conversion from one system to another, are explained.

Chapter Objectives

After completing this chapter, you will be able to:

- Define the decimal, binary, octal, and hexadecimal numbering systems and be able to convert from one numbering or coding system to another
- Explain the BCD, Gray, and ASCII code systems
- Define the terms *bit, byte, word, least significant bit (LSB),* and *most significant bit (MSB)* as they apply to binary memory locations
- Add, subtract, multiply, and divide binary numbers

3.1 Decimal System

Knowledge of different number systems and digital codes is quite useful when working with PLCs or with almost any type of digital computer. This is true because a basic requirement of these devices is to represent, store, and operate on numbers. In general, PLCs work on binary numbers in one form or another; these are used to represent various codes or quantities.

The ***decimal system,*** which is most common to us, has a **base of 10.** The radix or base of a number system determines the total number of different symbols or digits used by that system. For instance, in the decimal system, 10 unique numbers or digits—i.e., the digits 0 through 9—are used: the total number of symbols is the same as the base, and the symbol with the largest value is 1 less than the base.

The value of a decimal number depends on the digits that make up the number and the place value of each digit. A place (weight) value is assigned to each position that a digit would hold from right to left. In the decimal system the first position, starting from the rightmost position, is 0; the second is 1; the third is 2; and so on up to the last position. The weighted value of each position can be expressed as the base (10 in this case) raised to the power of the position. For the decimal system then, the position weights are 1, 10, 100, 1000, and so on. Figure 3-1 illustrates how the value of a decimal number can be calculated by multiplying each digit by the weight of its position and summing the results.

3.2 Binary System

The ***binary system*** uses the number **2 as the base.** The only allowable digits are 0 and 1. With digital circuits it is easy to distinguish between two voltage levels (i.e., +5 V and 0 V), which can be related to the binary digits 1 and 0 (Figure 3-2). Therefore, the binary system can be applied quite easily to PLCs and computer systems.

Since the binary system uses only two digits, each position of a binary number can go through only two

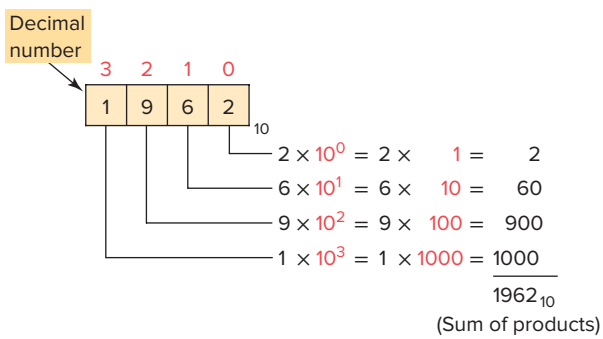


Figure 3-1 Weighted value in the decimal system.

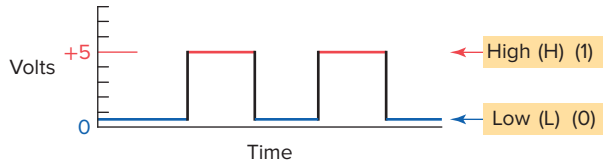


Figure 3-2 Digital signal waveform.

changes, and then a 1 is carried to the immediate left position. Table 3-1 shows a comparison among four common number systems: decimal (base 10), octal (base 8), hexadecimal (base 16), and binary (base 2). Note that all numbering systems start at *zero.*

The decimal equivalent of a binary number can be determined in a manner similar to that used for a decimal number. This time the weighted values of the positions are 1, 2, 4, 8, 16, 32, 64, and so on. The weighted value, instead of being 10 raised to the power of the position, is 2 raised to the power of the position. Figure 3-3 illustrates how the binary number 10101101 is converted to its decimal equivalent: 173.

Each digit of a binary number is known as a *bit.* In a PLC the processor-memory element consists of hundreds or thousands of locations. These locations, or registers,

Table 3-1 Number System Comparisons

Decimal	Octal	Hexadecimal	Binary
0	0	0	0
1	1	1	1
2	2	2	10
3	3	3	11
4	4	4	100
5	5	5	101
6	6	6	110
7	7	7	111
8	10	8	1000
9	11	9	1001
10	12	A	1010
11	13	B	1011
12	14	C	1100
13	15	D	1101
14	16	E	1110
15	17	F	1111
16	20	10	10000
17	21	11	10001
18	22	12	10010
19	23	13	10011
20	24	14	10100

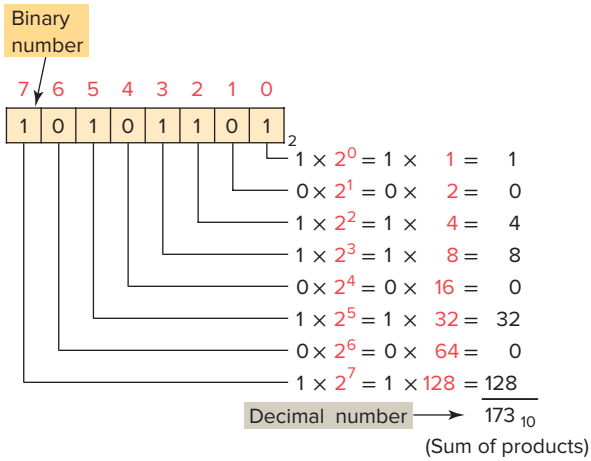


Figure 3-3 Converting a binary number to a decimal number.

are referred to as words. Each ***word*** is capable of storing data in the form of binary digits, or bits. The number of bits that a word can store depends on the type of PLC system used. Sixteen-bit and 32-bit words are the most common. Bits can also be grouped within a word into bytes. A group of 8 bits is a byte, and a group of 2 or more bytes is a word. Figure 3-4 illustrates a 16-bit word made up of 2 bytes. The least significant bit (LSB) is the digit that represents the smallest value, and the most significant bit (MSB) is the digit that represents the largest value. A bit within the word can exist only in two states: a logical 1 (or ON) condition, or a logical 0 (or OFF) condition.

PLC memory is organized using bytes, single words, or double words. Older PLCs use 8-bit or 16-bit memory words while newer systems, such as the ControlLogix platform from Allen-Bradley, use 32-bit double words. The size of the programmable controller memory relates to the amount of user program that can be stored. If the memory size is 1 K word (Figure 3-5), it can store 1024 words or 16,384 (1024 × 16) bits of information using 16-bit words, or 32,768 (1024 × 32) bits using 32-bit words.

To convert a decimal number to its binary equivalent, we must perform a series of divisions by 2. Figure 3-6 illustrates the conversion of the decimal number 47 to binary. We start by dividing the decimal number by 2. If

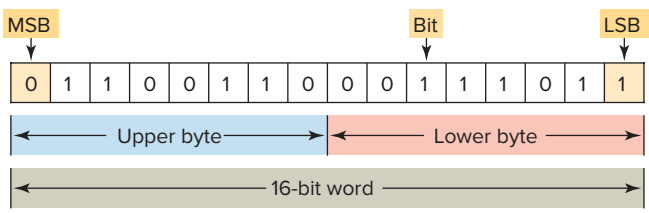


Figure 3-4 A 16-bit word.

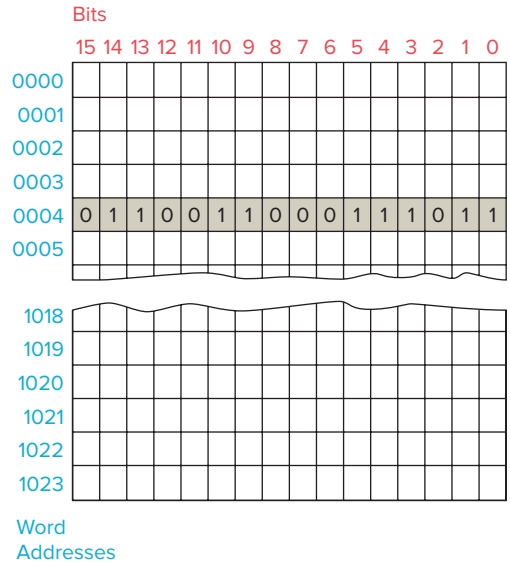


Figure 3-5 1-K word memory.

there is a remainder, it is placed in the LSB of the binary number. If there is no remainder, a 0 is placed in the LSB. The result of the division is brought down and the process is repeated until the result of successive divisions has been reduced to 0.

Even though the binary system has only two digits, it can be used to represent any quantity that can be represented in the decimal system. All PLCs work internally in the binary system. The processor, being a digital device, understands only 0s and 1s, or binary.

Computer memory is, then, a series of binary 1s and 0s. Figure 3-7 shows the output status file for an Allen-Bradley SLC 500 modular chassis, which is made up of single bits grouped into 16-bit words. One or more 16-bit output file word is reserved for each slot in the chassis. Each bit represents the ON or OFF state of one output point.

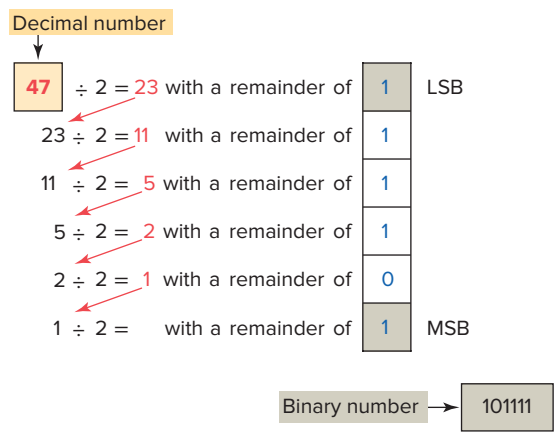


Figure 3-6 Converting a decimal number to a binary number.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Address
1	1	0	0	0	0	1	0	1	1	1	1	0	0	0	1	O:1
0	0	1	1	0	1	0	0	0	0	0	0	1	1	1	1	O:2
1	0	1	0	1	1	0	0	1	1	1	0	0	0	0	1	O:3
0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	O:4
1	1	1	0	1	0	0	1	1	1	0	0	1	1	0	1	O:5

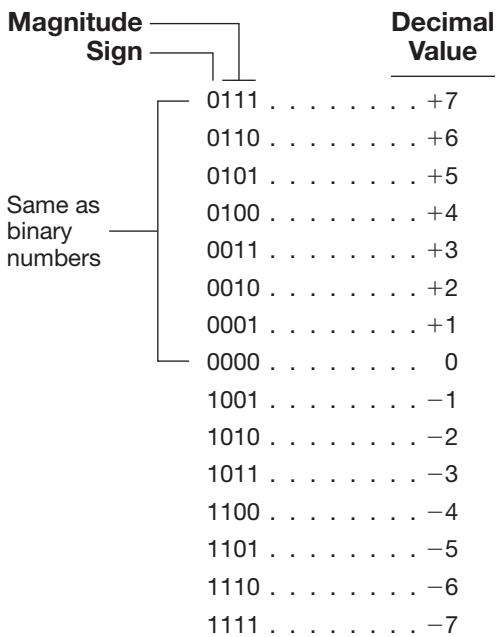
Figure 3-7 SLC 500 output status file.

These points are numbered 0 through 15 across the top row from right to left. The column on the far right lists the output module address. Although the table in Figure 3-7 illustrates sequentially addressed output status file words, in reality a word is created in the table only if the processor finds an output module residing in a particular slot. If the slot is empty, no word will be created.

3.3 Negative Numbers

If a decimal number is positive, it has a plus sign; if a number is negative, it has a minus sign. In binary number systems, such as used in a PLC, it is not possible to use positive and negative symbols to represent the polarity of a number. One method of representing a binary number as either a positive or negative value is to use an extra digit, or ***sign bit,*** at the MSB side of the number. In the sign bit position, a 0 indicates that the number is positive, and a 1 indicates a negative number (Table 3-2).

Table 3-2 Signed Binary Numbers



Another method of expressing a negative number in a digital system is by using the complement of a binary number. To complement a binary number, change all the 1s to 0s and all the 0s to 1s. This is known as the 1's complement form of a binary number. For example, the 1's complement of 1001 is 0110.

The most common way to express a negative binary number is to show it as a **2's complement** number. The 2's complement is the binary number that results when 1 is added to the 1's complement. This system is shown in Table 3-3. A zero sign bit means a positive number, whereas a 1 sign bit means a negative number.

Using the 2's complement makes it easier for the PLC to perform mathematical operations. The correct sign bit is generated by forming the 2's complement. The PLC knows that a number retrieved from memory is a negative number if the MSB is 1. Whenever a negative number is entered from a keyboard, the PLC stores it as a 2's complement. What follows is the original number in true binary followed by its 1's complement, its 2's complement, and finally, its decimal equivalent.

3.4 Octal System

To express the number in the binary system requires many more digits than in the decimal system. Too many binary digits can become cumbersome to read or write. To solve this problem, other related numbering systems are used.

The ***octal numbering system,*** a **base 8** system, is used because 8 data bits make up a byte of information that can be addressed. Octal is a convenient means of handling large binary numbers. As shown in Table 3-4, one octal digit can be used to express three binary digits. As in all other numbering systems, each digit in an octal number has a weighted decimal value according to its position. Figure 3-8 illustrates how the octal number 462 is converted to its decimal equivalent: 306.

Octal converts easily to binary equivalents. For example, the octal number 462 is converted to its binary equivalent by assembling the 3-bit groups, as illustrated in Figure 3-9.

Table 3-3 1's and 2's Complement Representation of Positive and Negative Numbers

Signed Decimal	1's Complement	2's Complement	2's Complement-16 bit
+7	0111	0111	0000 0000 0000 0111
+6	0110	0110	0000 0000 0000 0110
+5	0101	0101	0000 0000 0000 0101
+4	0100	0100	0000 0000 0000 0100
+3	0011	0011	0000 0000 0000 0011
+2	0010	0010	0000 0000 0000 0010
+1	0001	0001	0000 0000 0000 0001
0	0000	0000	0000 0000 0000 0000
−1	1110	1111	1111 1111 1111 1111
−2	1101	1110	1111 1111 1111 1110
−3	1100	1101	1111 1111 1111 1101
−4	1011	1100	1111 1111 1111 1100
−5	1010	1011	1111 1111 1111 1011
−6	1001	1010	1111 1111 1111 1010
−7	1000	1001	1111 1111 1111 1001

Same as binary numbers

Table 3-4 Binary and Related Octal Code

Binary	Octal
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

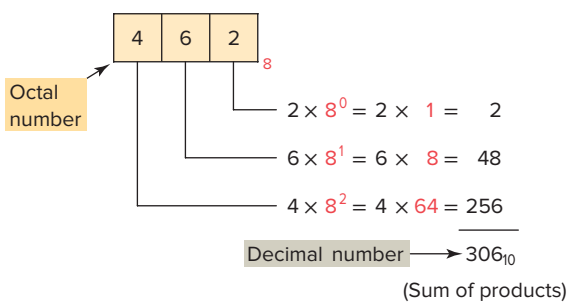


Figure 3-8 Converting an octal number to a decimal number.

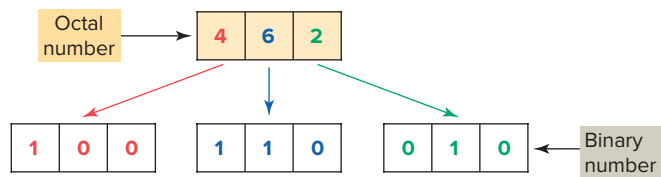


Figure 3-9 Converting an octal number to a binary number.

Notice the simplicity of the notation: the octal 462 is much easier to read and write than its binary equivalent is.

3.5 Hexadecimal System

The **hexadecimal (hex)** numbering system is used in programmable controllers because a word of data consists of 16 data bits, or two 8-bit bytes. The hexadecimal system is a **base 16** system, with A to F used to represent decimal numbers 10 to 15 (Table 3-5). The hexadecimal numbering system allows the status of a large number of binary bits to be represented in a small space, such as on a computer screen or PLC programming device display.

The techniques used when converting hexadecimal to decimal and decimal to hexadecimal are the same as those used for binary and octal. To convert a hexadecimal

Table 3-5 Hexadecimal Numbering System

Hexadecimal	Binary	Decimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15

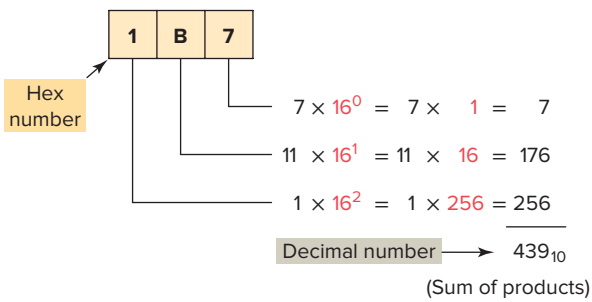


Figure 3-10 Converting a hexadecimal number to a decimal number.

number to its decimal equivalent, the hexadecimal digits in the columns are multiplied by the base 16 weight, depending on digit significance. Figure 3-10 illustrates how the conversion would be done for the hex number 1B7.

Hexadecimal numbers can easily be converted to binary numbers. Conversion is accomplished by writing the 4-bit binary equivalent of the hex digit for each position, as illustrated in Figure 3-11.

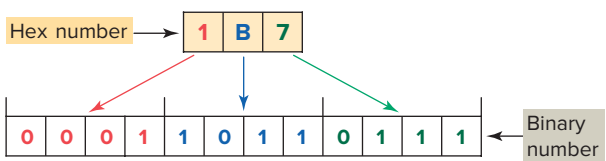


Figure 3-11 Converting a hexadecimal number to a binary number.

3.6 Binary Coded Decimal (BCD) System

The **binary coded decimal (BCD)** system provides a convenient way of handling large numbers that need to be input to or output from a PLC. As you can see from looking at the various number systems, there is no easy way to go from binary to decimal and back. The BCD system provides a means of converting a code readily handled by humans (decimal) to a code readily handled by the equipment (binary). PLC thumbwheel switches and LED displays are examples of PLC devices that make use of the BCD number system. Table 3-6 shows examples of numeric values in decimal, binary, BCD, and hexadecimal representation.

The BCD system uses 4 bits to represent each decimal digit. The 4 bits used are the binary equivalents of the numbers from 0 to 9. In the BCD system, the largest decimal number that can be displayed by any four digits is 9.

The BCD representation of a decimal number is obtained by replacing each decimal digit by its BCD equivalent. To distinguish the BCD numbering system from a binary system, a BCD designation is placed to the right of the units digit. The BCD representation of the decimal number 7863 is shown in Figure 3-12.

A thumbwheel switch is one example of an input device that uses BCD. Figure 3-13 shows a single-digit BCD thumbwheel. The circuit board attached to the thumbwheel has one connection for each bit's weight plus a common connection. The operator dials in a decimal digit between 0 and 9, and the thumbwheel switch outputs the equivalent 4 bits of BCD data. In this example, the number eight is dialed to produce the input bit pattern of 1000. A four-digit thumbwheel switch, similar to the one shown, would control a total of 16 (4 × 4) PLC inputs.

Scientific calculators are available to convert numbers back and forth between decimal, binary, octal, and hexadecimal. In addition, PLCs contain number conversion functions such as illustrated in Figure 3-14. BCD-to-binary conversion is required for the input while binary-to-BCD conversion is required for the output. The Convert-to-BCD instruction will convert the binary bit pattern at the source address, N7:23, into a BCD bit pattern of the same decimal value and store it at the destination address, O:20. The instruction executes every time it is scanned, and the instruction is true.

Many PLCs allow you to change the format of the data that the data monitor displays. For example, the change radix function found on Allen-Bradley controllers allows

Table 3-6 Numeric Values in Decimal, Binary, BCD, and Hexadecimal Representation

Decimal	Binary	BCD	Hexadecimal
0	0	0000	0
1	1	0001	1
2	10	0010	2
3	11	0011	3
4	100	0100	4
5	101	0101	5
6	110	0110	6
7	111	0111	7
8	1000	1000	8
9	1001	1001	9
10	1010	0001 0000	A
11	1011	0001 0001	B
12	1100	0001 0010	C
13	1101	0001 0011	D
14	1110	0001 0100	E
15	1111	0001 0101	F
16	1 0000	0001 0110	10
17	1 0001	0001 0111	11
18	1 0010	0001 1000	12
19	1 0011	0001 1001	13
20	1 0100	0010 0000	14
126	111 1110	0001 0010 0110	7E
127	111 1111	0001 0010 0111	7F
128	1000 0000	0001 0010 1000	80
510	1 1111 1110	0101 0001 0000	1FE
511	1 1111 1111	0101 0001 0001	1FF
512	10 0000 0000	0101 0001 0010	200

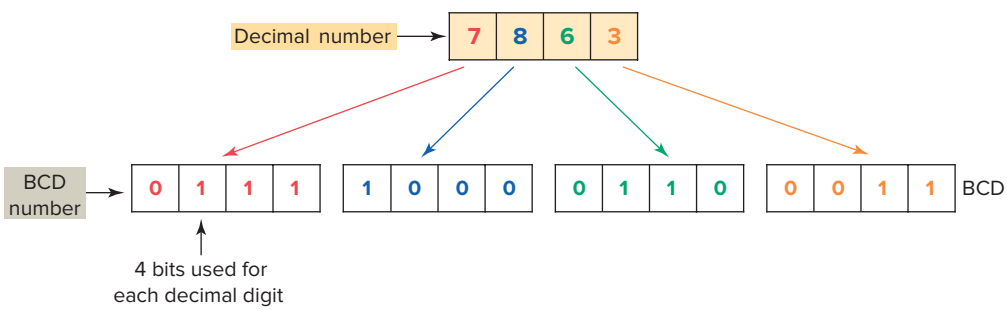


Figure 3-12 The BCD representation of a decimal number.

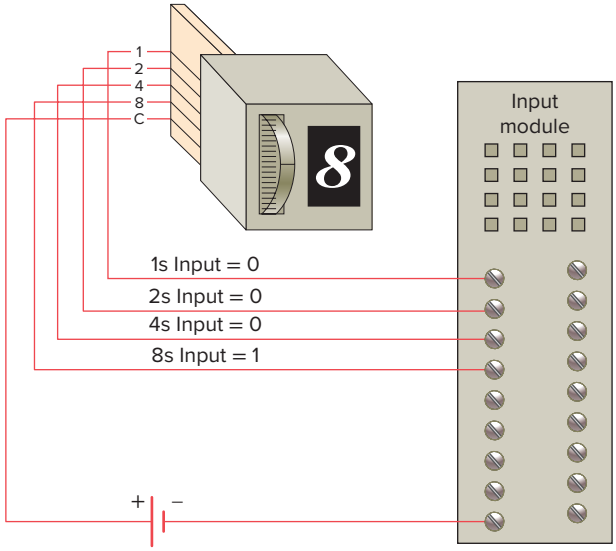


Figure 3-13 BCD thumbwheel switch interfaced to a PLC.

you to change the display format of data to binary, octal, decimal, hexadecimal, or ASCII.

3.7 Gray Code

The **Gray code** is a special type of binary code that does not use position weighting. In other words, each position does not have a definite weight. The Gray code is set up so that as we progress from one number to the next, only one bit changes. This can be quite confusing for counting circuits, but it is ideal for encoder circuits. For example, absolute encoders are position transducers that use the Gray code to determine angular position. The Gray code has the advantage that for each "count" (each transition from one number to the next) only one digit changes. Table 3-7 shows the Gray code and the binary equivalent for comparison.

Table 3-7 Gray Code and Binary Equivalent

Gray Code	Binary
0000	0000
0001	0001
0011	0010
0010	0011
0110	0100
0111	0101
0101	0110
0100	0111
1100	1000
1101	1001
1111	1010
1110	1011
1010	1100
1011	1101
1001	1110
1000	1111

In binary, as many as four digits could change for a single "count." For example, the transition from binary 0111 to 1000 (decimal 7 to 8) involves a change in all four digits. This kind of change increases the possibility for error in certain digital circuits. For this reason, the Gray code is considered to be an error-minimizing code.

Gray codes are used with position encoders for accurate control of the motion of robots, machine tools, and servomechanisms. Figure 3-15 shows an optical encoder disk that uses a 4-bit Gray code to detect changes in

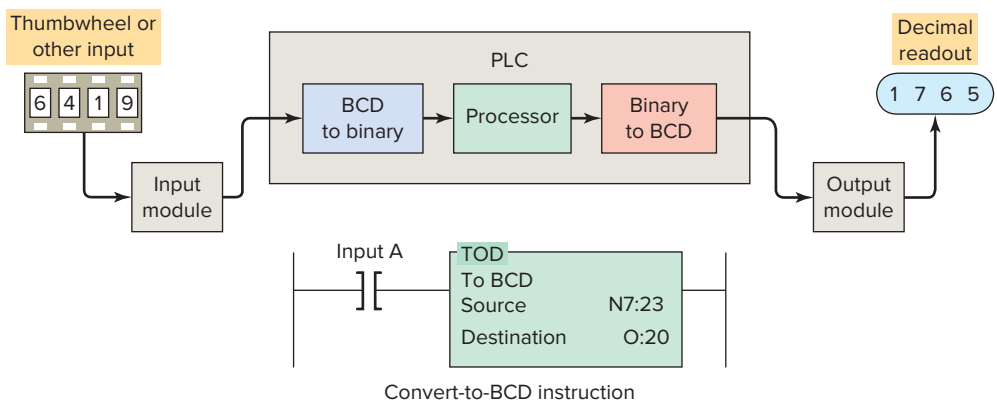


Figure 3-14 PLC number conversion.

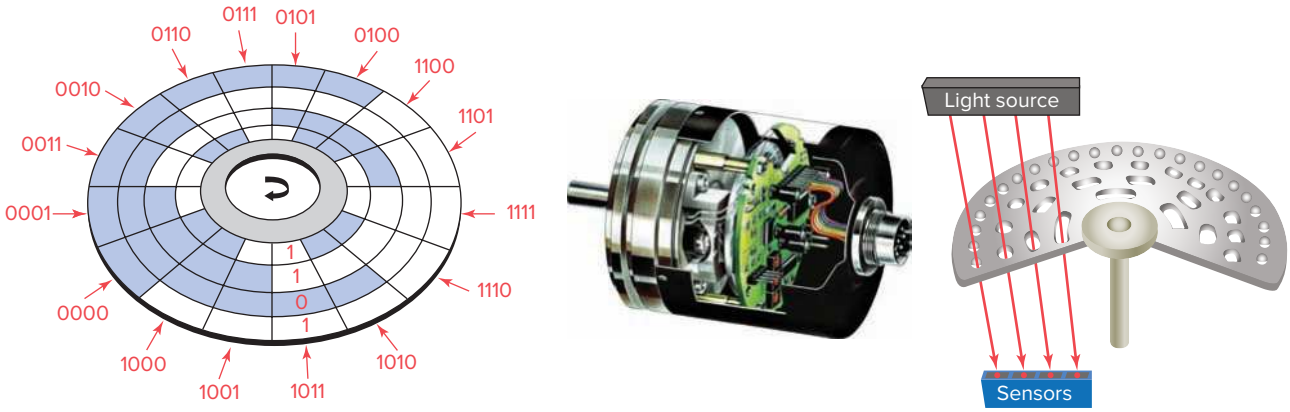


Figure 3-15 Optical encoder disk.
Source: Photo courtesy Baumer Electric.

angular position. In this example, the encoder disk is attached to a rotating shaft and outputs a digital Gray code signal that is used to determine the position of the shaft. A fixed array of photo diodes senses the reflected light from each of the cells across a row of the encoder path. Depending on the amount of light reflected, each cell will output a voltage corresponding to a binary 1 or 0. Thus, a different 4-bit word is generated for each row of the disk.

3.8 ASCII Code

ASCII stands for American Standard Code for Information Interchange. It is an alphanumeric code because it includes letters as well as numbers. The characters accessed by the ASCII code include 10 numeric digits; 26 lowercase and 26 uppercase letters of the alphabet; and about 25 special characters, including those found on a standard typewriter. Table 3-8 shows a partial listing of the ASCII code. It is used to interface the PLC CPU with alphanumeric keyboards and printers.

The ASCII code is a seven-bit code in which the decimal digits are represented by the 8-4-2-1 BCD code preceded by 011. Uppercase letters are preceded by 100 or 101. Lowercase letters are preceded by 110 or 111. Character symbols are preceded by 010, 011, 101, and 111. This seven-bit code provides all possible combinations of characters used when communicating with peripherals or interfaces in a PLC system.

The keystrokes on the keyboard of a computer are converted directly into ASCII for processing by the computer. Each time you press a key on a computer keyboard, a 7- or 8-bit word is stored in computer memory to represent the alphanumeric, function, or control data represented by the specific keyboard key that was depressed. ASCII input modules convert ASCII code input information from an external device to alphanumeric information that the PLC can process. The communication interfacing is done through either an RS-232 or RS-422 protocol. Modules are available that will transmit and receive ASCII files and that can be used to create an operator interface. The user writes a program in the BASIC language that operates in conjunction with the ladder logic as the program runs.

3.9 Parity Bit

Some PLC communication systems use a binary bit to check the accuracy of data transmission. For example, when data are transferred between PLCs, one of the binary digits may be accidentally changed from a 1 to a 0. This can happen because of a transient or a noise or because of a failure in some portion of the transmission network. A parity bit is used to detect errors that may occur while a word is moved.

Parity is a system in which each character transmitted contains one additional bit. That bit is known as a parity bit. The bit may be a binary 0 or binary 1, depending on the number of 1s and 0s in the character itself. Two systems of parity are normally used: odd and even. Odd parity means that the total number of binary 1 bits in the character, including the parity bit, is odd. Even parity means that the number of binary 1 bits in the character, including the parity bit, is even. Examples of odd and even parity are shown in Table 3-9.

Table 3-8 Partial Listing of ASCII Code

Character	7-Bit ASCII	Character	7-Bit ASCII
A	100 0001	X	101 1000
B	100 0010	Y	101 1001
C	100 0011	Z	101 1010
D	100 0100	0	011 0000
E	100 0101	1	011 0001
F	100 0110	2	011 0010
G	100 0111	3	011 0011
H	100 1000	4	011 0100
I	100 1001	5	011 0101
J	100 1010	6	011 0110
K	100 1011	7	011 0111
L	100 1100	8	011 1000
M	100 1101	9	011 1001
N	100 1110	blank	010 0000
O	100 1111	.	010 1110
P	101 0000	,	010 1100
Q	101 0001	+	010 1011
R	101 0010	−	010 1101
S	101 0011	#	010 0011
T	101 0100	(	010 1000
U	101 0101	%	010 0101
V	101 0110	=	011 1101
W	101 0111		

Table 3-9 Odd and Even Parity

Character	Even Parity Bit	Odd Parity Bit
0000	0	1
0001	1	0
0010	1	0
0011	0	1
0100	1	0
0101	0	1
0110	0	1
0111	1	0
1000	1	0
1001	0	1

3.10 Binary Arithmetic

Arithmetic circuit units form a part of the CPU. Mathematical operations include addition, subtraction, multiplication, and division. Binary addition follows rules similar to decimal addition. When adding with binary numbers, there are only four conditions that can occur:

$$\begin{array}{cccc} 0 & 1 & 0 & 1 \\ +0 & +0 & +1 & +1 \\ \hline 0 & 1 & 1 & 0 \text{ carry } 1 \end{array}$$

The first three conditions are easy because they are like adding decimals, but the last condition is slightly different. In decimal, $1 + 1 = 2$. In binary, a 2 is written 10. Therefore, in binary, $1 + 1 = 0$, with a carry of 1 to the next most significant place value. When adding larger binary numbers, the resulting 1s are carried

into higher-order columns, as shown in the following examples.

Decimal	Equivalent binary
5	101
+2	+ 10
7	111

	carry	
	1	
10	10	10
+ 3	+	11
13	11	01

	carry	carry	
	1	1	
26		1	1010
+12	+		1100
38	1	0	0110

In arithmetic functions, the initial numeric quantities that are to be combined by subtraction are the minuend and subtrahend. The result of the subtraction process is called the difference, represented as:

A (minuend)
−B (subtrahend)
C (difference)

To subtract from larger binary numbers, subtract column by column, borrowing from the adjacent column when necessary. Remember that when borrowing from the adjacent column, there are now two digits, i.e., 0 borrow 1 gives 10.

EXAMPLE 10-1

Subtract 1001 from 1101.

```
 1101
−1001
 0100
```

Subtract 0111 from 1011.

```
 1011
−0111
 0100
```

Binary numbers can also be negative. The procedure for this calculation is identical to that of decimal numbers because the smaller value is subtracted from the larger value and a negative sign is placed in front of the result.

EXAMPLE 10-2

Subtract 111 from 100.

```
  111
−100
−011
```

Subtract 11011 from 10111.

```
 11011
−10111
−00100
```

There are other methods available for doing subtraction:

1's complement
2's complement

The procedure for subtracting numbers using the 1's complement is as follows:

Step 1 Change the subtrahend to 1's complement.
Step 2 Add the two numbers.
Step 3 Remove the last carry and add it to the number (end-around carry).

Decimal	Binary		
10	1010	1's complement →	1010
− 6	−0110		+1001
4	100		10011
		End-around carry →	+1
			100

When there is a carry at the end of the result, the result is positive. When there is no carry, then the result is negative and a minus sign has to be placed in front of it.

EXAMPLE 10-3

Subtract 11011 from 01101.

```
 01101
+ ∅00100   The 1's complement
 10001     There is no carry,
           so we take the 1's
           complement and add
           the minus sign:
           −01110
```

For subtraction using the 2's complement, the 2's complement is added instead of subtracting the numbers.

In the result, if the carry is a 1, then the result is positive; if the carry is a 0, then the result is negative and requires a minus sign.

EXAMPLE 10-4

Subtract 101 from 111.

111	
+ ∅011	The 2's complement
1010	The first 1 indicates that the result is positive, so it is disregarded:
	010

EXAMPLE 10-5

Subtract 11011 from 01101.

01101	
+ ∅00101	The 2's complement
10010	There is no carry, so the result is negative; therefore a 1 has to be subtracted and the 1's complement taken to give the result:
subtract 1	10010 − 1 = 10001
1's complement	−01110

Binary numbers are multiplied in the same manner as decimal numbers. When multiplying binary numbers, there are only four conditions that can occur:

$$0 \times 0 = 0$$
$$0 \times 1 = 0$$
$$1 \times 0 = 0$$
$$1 \times 1 = 1$$

To multiply numbers with more than one digit, form partial products and add them together, as shown in the following example.

Decimal	Equivalent binary
5	101
×6	×110
30	000
	101
	101
	11110

The process for dividing one binary number by another is the same for both binary and decimal numbers, as shown in the following example.

Decimal	Equivalent binary
7	111
2)14	10)1110
	10
	11
	10
	10
	10
	00

The basic function of a comparator is to compare the relative magnitude of two quantities. PLC data comparison instructions are used to compare the data stored in two words (or registers). At times, devices may need to be controlled when they are less than, equal to, or greater than other data values or set points used in the application, such as timer and counter values. The basic compare instructions are as follows:

$A = B$ (A equals B)
$A > B$ (A is greater than B)
$A < B$ (A is less than B)

3.11 Floating Point Arithmetic

Certain PLC-related computations are performed using **floating point** arithmetic. The term *floating point* refers to the fact that the decimal point can float or be placed anywhere relative to the significant digits of the number. The main features of floating-point representation are:

- Floating point can support a much wider range of values. It can represent numbers that are very small or numbers that are very large.
- Floating point provides an easy method of dealing with fractions. Without floating point, a PLC word can only represent an integer or whole number.

An example of a floating point number system is shown in Figure 3-16. The representations shown in this example

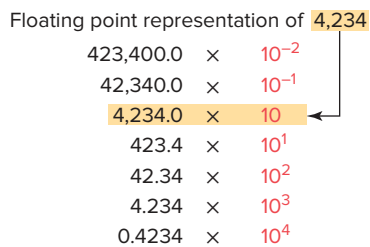


Figure 3-16 Example of a floating point number system.

vary based on the position of the decimal point, which floats to the left or right, with a corresponding change in the exponent value.

- The numbers shown on the left side represent the *significand*, or *mantissa*, and the numbers shown on the right indicate the exponent value.
- A significand contains the number's digits and an exponent indicates where the decimal (or binary) point is located relative to the beginning of the significand.
- Decimal floating-point numbers usually take the form of scientific notation with a decimal point always between the 1st and 2nd digits.

Floating point numbers have three basic components: the sign, the exponent, and the mantissa, as shown in Figure 3-17.

- The **sign** of a binary floating-point number is represented by a single bit. A 1 bit indicates a negative number, and a 0 bit indicates a positive number.
- The **mantissa,** always a positive number, holds the significant digits of the floating-point number.
- The **exponent** indicates the positive or negative power of the radix that the mantissa and sign should be multiplied by.

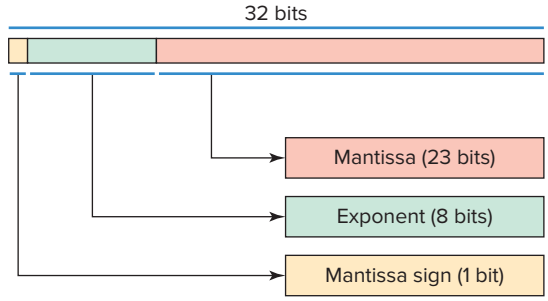


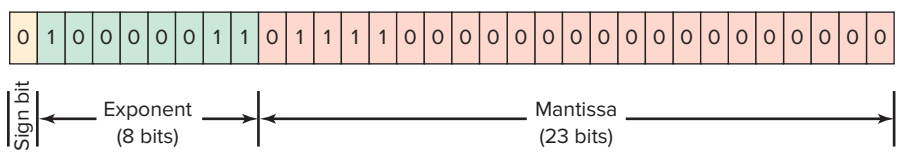
Figure 3-17 Basic components of a floating point number.

Floating point numbers are also known as real numbers. The IEEE 754 Standard is the most commonly used standard for representing these numbers and includes:

- **Single precision:** 32 bits, consisting of
  - Sign bit (1 bit)
  - Exponent (8 bits)
  - Mantissa (23 bits)
- **Double precision:** 64 bits, consisting of
  - Sign bit (1 bit)
  - Exponent (11 bits)
  - Mantissa (52 bits)

Figure 3-18 is an example of how the decimal number 23.5 would be represented in a single precision 32-bit floating point binary format.

Figure 3-18 Decimal number 23.5 represented in a single precision 32-bit floating point binary format.



CHAPTER 3 REVIEW QUESTIONS

1. Convert each of the following binary numbers to decimal numbers:
   a. 10
   b. 100
   c. 111
   d. 1011
   e. 1100
   f. 10010
   g. 10101
   h. 11111
   i. 11001101
   j. 11100011
2. Convert each of the following decimal numbers to binary numbers:
   a. 7
   b. 19
   c. 28
   d. 46
   e. 57
   f. 86
   g. 94
   h. 112
   i. 148
   j. 230
3. Convert each of the following octal numbers to decimal numbers:
   a. 36
   b. 104
   c. 120
   d. 216
   e. 360
   f. 1516
4. Convert each of the following octal numbers to binary numbers:
   a. 74
   b. 130
   c. 250
   d. 1510
   e. 2551
   f. 2634
5. Convert each of the following hexadecimal numbers to decimal numbers:
   a. 5A
   b. C7
   c. 9B5
   d. 1A6
6. Convert each of the following hexadecimal numbers to binary numbers:
   a. 4C
   b. E8
   c. 6D2
   d. 31B
7. Convert each of the following decimal numbers to BCD:
   a. 146
   b. 389
   c. 1678
   d. 2502
8. What is the most important characteristic of the Gray code?
9. What makes the binary system so applicable to computer circuits?
10. Define the following as they apply to the binary memory locations or registers:
    a. Bit
    b. Byte
    c. Word
    d. LSB
    e. MSB
11. State the base used for each of the following number systems:
    a. Octal
    b. Decimal
    c. Binary
    d. Hexadecimal
12. Define the term *sign bit*.
13. Explain the difference between the 1's complement of a number and the 2's complement.
14. What is ASCII code?
15. Why are parity bits used?
16. Add the following binary numbers:
    a. 110 + 111
    b. 101 + 011
    c. 1100 + 1011
17. Subtract the following binary numbers:
    a. 1101 − 101
    b. 1001 − 110
    c. 10111 − 10010

18. Multiply the following binary numbers:
a. 110 × 110
b. 010 × 101
c. 101 × 11

19. Divide the following unsigned binary numbers:
a. 1010 ÷ 10
b. 1100 ÷ 11
c. 110110 ÷ 10

CHAPTER 3 PROBLEMS

1. The following binary PLC coded information is to be programmed using the hexadecimal code. Convert each piece of binary information to the appropriate hexadecimal code for entry into the PLC from the keyboard.
a. 0001 1111
b. 0010 0101
c. 0100 1110
d. 0011 1001

2. The encoder circuit shown in Figure 3-19 is used to convert the decimal digits on the keyboard to a binary code. State the output status (HIGH/LOW) of A-B-C-D when decimal number
a. 2 is pressed.
b. 5 is pressed.
c. 7 is pressed.
d. 8 is pressed.

3. If the bits of a 16-bit word or register are numbered according to the octal numbering system, beginning with 00, what consecutive numbers would be used to represent each of the bits?

4. Express the decimal number 18 in each of the following number codes:
a. Binary
b. Octal
c. Hexadecimal
d. BCD

5. Add the binary number 110 to the negative binary number −101.

6. Subtract 10112 − 1102.

7. Multiply the following unsigned binary numbers: 101.1 and 10.11.

8. Divide the following unsigned binary numbers: 1110 by 10.

9. State two instances that might call for the use of a floating-point numbering system.

10. What are the three basic components of a floating-point number?

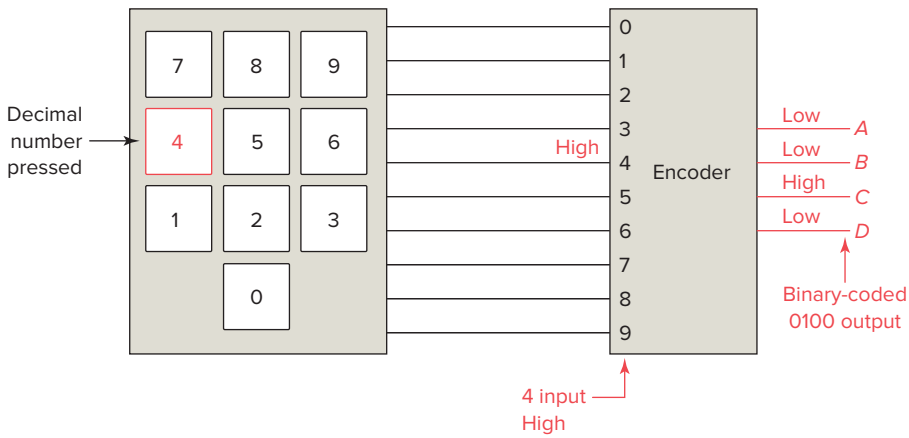
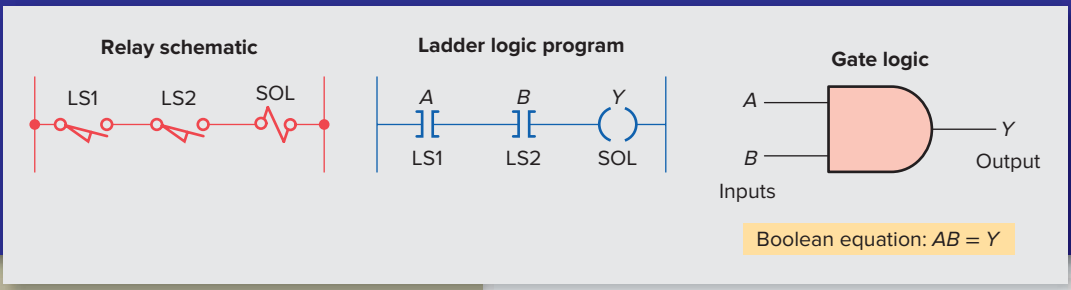


Figure 3-19 Diagram for Problem 2.

4

Fundamentals of Logic



Chapter Objectives

After completing this chapter, you will be able to:

- Describe the binary concept and the functions of gates
- Draw the logic symbol, construct a truth table, and state the Boolean equation for the AND, OR, and NOT functions
- Construct circuits from Boolean expressions and derive Boolean equations for given logic circuits
- Convert relay ladder schematics to ladder logic programs
- Develop elementary programs based on logic gate functions
- Program instructions that perform logical operations

This chapter gives an overview of digital logic gates and illustrates how to duplicate this type of control on a PLC. Boolean algebra, which is a shorthand way of writing digital gate diagrams, is discussed briefly. Some small hand-held programmers have digital logic keys, such as AND, OR, and NOT, and are programmed using Boolean expressions.

4.1 The Binary Concept

The PLC, like all digital equipment, operates on the binary principle. The term ***binary principle*** refers to the idea that many things can be thought of as existing in only one of two states. These states are 1 and 0. The 1 and 0 can represent ON or OFF, open or closed, true or false, high or low, or any other two conditions. The key to the speed and accuracy with which binary information can be processed is that there are only two states, each of which is distinctly different. There is no in-between state so when information is processed the outcome is either yes or no.

A ***logic gate*** is a circuit with several inputs but only one output that is activated by particular combinations of input conditions. The two-state binary concept, applied to gates, can be the basis for making decisions. The high beam automobile lighting circuit of Figure 4-1 is an example of a logical AND decision. For this application, the high beam light can be turned on only when the light switch AND the high beam switch are closed.

The dome light automobile circuit of Figure 4-2 is an example of a logical OR decision. For this application, the dome light will be turned on whenever the passenger door switch OR the driver door switch is activated.

Logic is the ability to make decisions when one or more different factors must be taken into account before an action is taken. This is the basis for the operation of the PLC, where it is required for a device to operate when certain conditions have been met.

4.2 AND, OR, and NOT Functions

The operations performed by digital equipment are based on three fundamental logic functions: AND, OR, and NOT. Each function has a rule that will determine the outcome and a *symbol* that represents the operation. For the purpose of this discussion, the outcome or output is called Y and the signal inputs are called *A, B, C,* and so on. Also, binary 1 represents the presence of a signal or the occurrence of some event, and binary 0 represents the absence of the signal or nonoccurrence of the event.

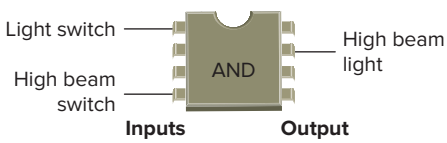


Figure 4-1 The logical AND.

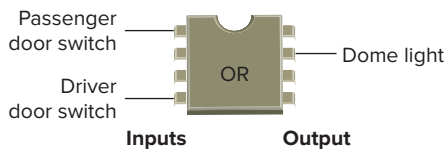


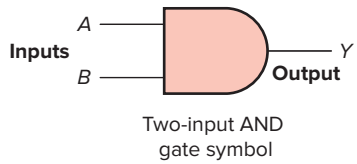
Figure 4-2 The logical OR.

The AND Function

The symbol drawn in Figure 4-3 is that of an AND gate. An **AND gate** is a device with two or more inputs and one output. The AND gate output is 1 only if all inputs are 1. The AND truth table in Figure 4-3 shows the resulting output from each of the possible input combinations.

Logic gate ***truth tables*** show each possible input to the gate or circuit and the resultant output depending upon the combination of the input(s).

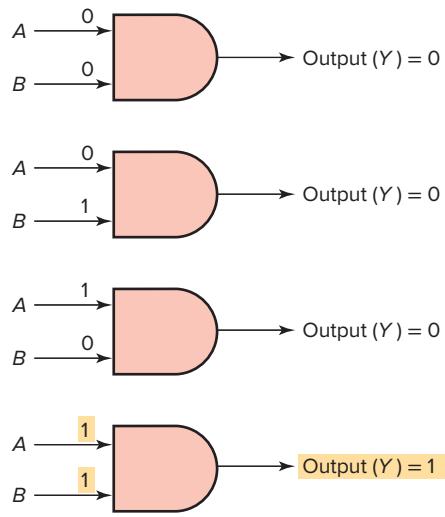
Since logic gates are digital ICs (Integrated Circuits) their input and output signals can be in only one of two possible digital states, i.e., logic 0 or logic 1. Thus, the logic state of the output of a logic gate depends on the logic states of each of its individual inputs. Figure 4-4



AND truth table

Inputs		Output
A	*B*	*Y*
0	0	0
0	1	0
1	0	0
1	1	1

Figure 4-3 AND gate.



Truth table

A	*B*	*Y*
0	0	0
0	1	0
1	0	0
1	1	1

Figure 4-4 AND logic gate digital signal states.

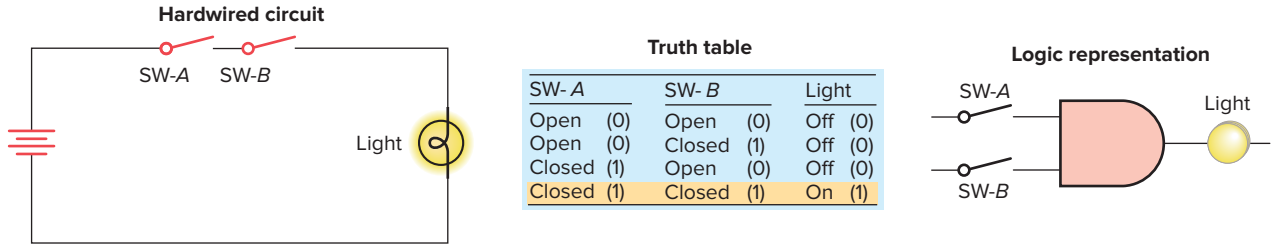


Figure 4-5 AND logic gate operates similarly to control devices connected in series.

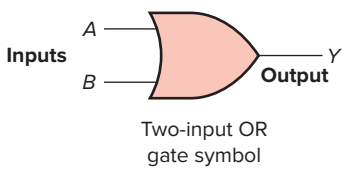
illustrates the four possible combinations of inputs for a 2-input AND gate. The basic rules that apply to an AND gate are:

- If all inputs are 1, the output will be 1.
- If any input is 0, the output will be 0.

The AND logic gate operates similarly to control devices connected in ***series***, as illustrated in Figure 4-5. The light will be on only when both switch A and switch B are closed.

The OR Function

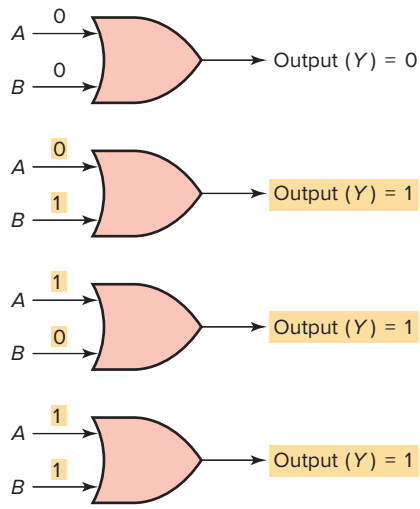
The symbol drawn in Figure 4-6 is that of an OR gate. An **OR gate** can have any number of inputs but only one output. The OR gate output is 1 if one or more inputs are 1. The truth table in Figure 4-6 shows the resulting output Y from each possible input combination.



OR truth table

Inputs		Output
A	*B*	*Y*
0	0	0
0	1	1
1	0	1
1	1	1

Figure 4-6 OR gate.



Truth table

Inputs		Output
A	*B*	*Y*
0	0	0
0	1	1
1	0	1
1	1	1

Figure 4-7 OR logic gate digital signal states.

Figure 4-7 illustrates the four possible combinations of inputs for a 2-input OR gate. The basic rules that apply to an OR gate are:

- If one or more inputs are 1, the output is 1.
- If all inputs are 0, the output will be 0.

The OR logic gate operates similarly to control devices connected in ***parallel***, as illustrated in Figure 4-8. The light will be on if switch A or switch B or both are closed.

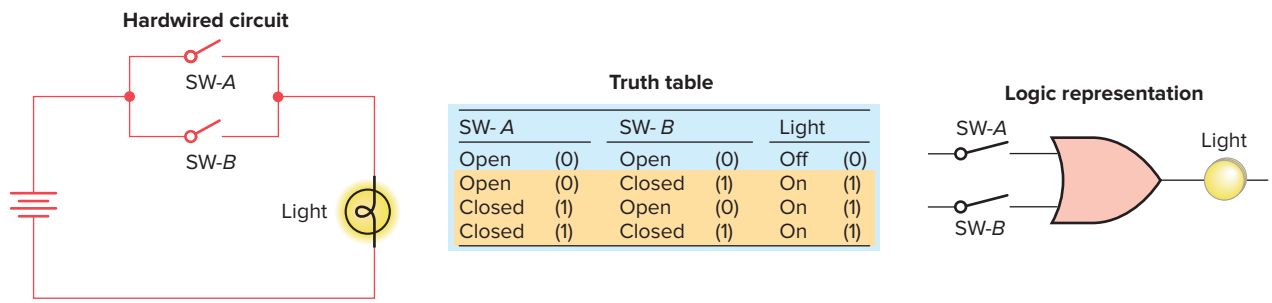
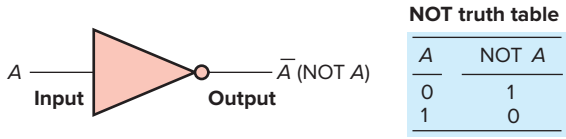


Figure 4-8 OR logic gate operates similarly to control devices connected in parallel.



NOT truth table

A	NOT A
0	1
1	0

Figure 4-9 NOT function.

The NOT Function

The symbol drawn in Figure 4-9 is that of a NOT function. Unlike the AND and OR functions, the **NOT function** can have *only one* input. The NOT output is 1 if the input is 0. The output is 0 if the input is 1. The result of the NOT operation is always the inverse of the input, and the NOT function is, therefore, called an ***inverter***. The NOT function is often depicted by using a bar across the top of the letter, indicating an inverted output. The small circle at the output of the inverter is referred to as a bubble and indicates that an inversion of the logical function has taken place.

The logical NOT function can be performed on a contact input simply by using a normally closed instead of a normally open contact. Figure 4-10 shows an example of the NOT function constructed using a normally closed pushbutton in series with a lamp. When the input pushbutton is *not* actuated, the output lamp is ON. When the input pushbutton is actuated, the output lamp switches OFF.

The NOT function is most often used in conjunction with the AND or the OR gate. Figure 4-11 shows the NOT function connected to one input of an AND gate for a low-pressure indicator circuit. If the power is on (1) and the pressure switch is not closed (0), the warning light will be on (1).

The NOT symbol placed at the output of an AND gate would invert the normal output result. An AND gate with an inverted output is called a ***NAND*** gate. The NAND gate symbol and truth table are shown in Figure 4-12. The NAND function is often used in integrated circuit logic arrays and can be used in programmable controllers to solve complex logic.

The same rule about inverting the normal output result applies if a NOT symbol is placed at the output of the OR gate. The normal output is inverted, and the function is referred to as a ***NOR*** gate. The NOR gate symbol and truth table are shown in Figure 4-13.

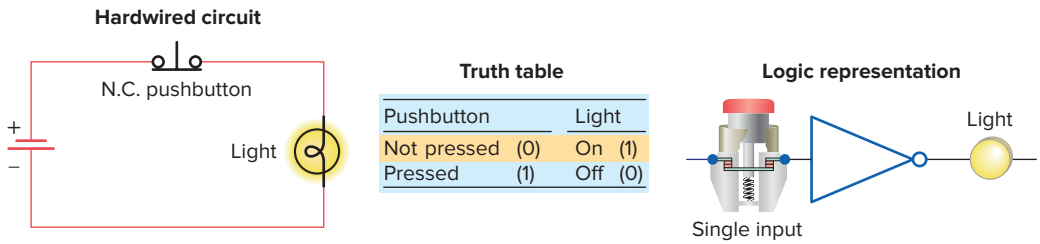


Figure 4-10 NOT function constructed using a normally closed pushbutton.

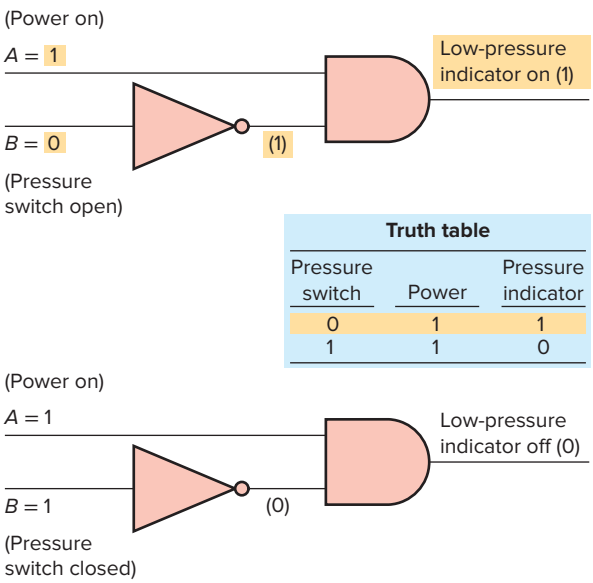
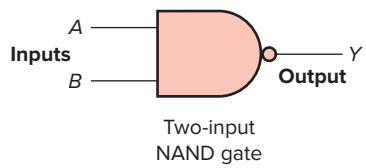


Figure 4-11 NOT function is most often used in conjunction with an AND gate.

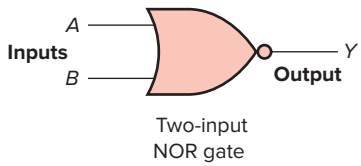


Two-input NAND gate

NAND truth table

Inputs		Output
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Figure 4-12 NAND gate symbol and truth table.

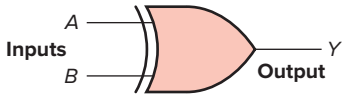


Two-input NOR gate

NOR truth table

Inputs		Output
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Figure 4-13 NOR gate symbol and truth table.



Truth table

Inputs		Output
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Figure 4.14 The XOR gate symbol and truth table.

The Exclusive-OR (XOR) Function

An often-used combination of gates is the exclusive-OR (XOR) function. The ***XOR*** gate symbol and truth table are shown in Figure 4-14. The output of this circuit is HIGH only when one input or the other is HIGH, but not both. The exclusive-OR gate is commonly used for the *comparison* of two binary numbers.

4.3 Boolean Algebra

The mathematical study of the binary number system and logic is called ***Boolean algebra.*** The purpose of this algebra is to provide a simple way of writing complicated combinations of logic statements. There are many applications where Boolean algebra could be applied to solving PLC programming problems.

Figure 4-15 summarizes the basic operators of Boolean algebra as they relate to the basic AND, OR, and NOT functions. Inputs are represented by capital letters A, B, C, and so on, and the output by a capital Y. The dot (•), or no symbol, represents the AND operation, an addition sign (+) represents the OR operation, the circle with an addition sign ⊕ represents the exclusive-OR operation, and a bar over the letter $\overline{A}$ represents

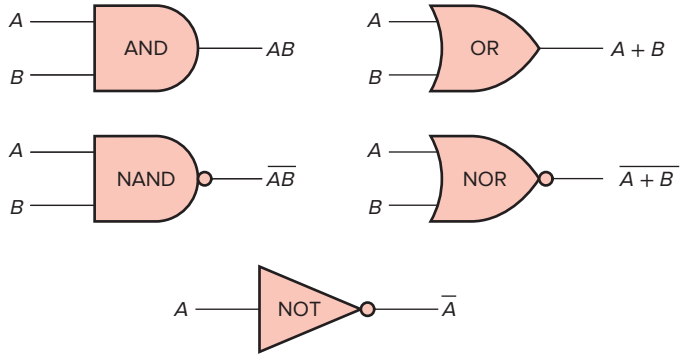


Figure 4-16 Logic operators used singly to form logical statements.

the NOT operation. The Boolean equations are used to express the mathematical function of the logic gate.

PLC digital systems may be designed using Boolean algebra. Circuit functions are represented by Boolean equations. Figure 4-16 illustrates how logic operators AND, NAND, OR, NOR, and NOT are used singly to form logical statements. Figure 4-17 illustrates how basic logic operators are used in combination to form Boolean equations.

An understanding of the technique of writing simplified Boolean equations for complex logical statements is a useful tool when creating PLC control programs. Some laws of Boolean algebra are different from those of ordinary algebra. These three basic laws illustrate the close comparison between Boolean algebra and ordinary algebra, as well as one major difference between the two:

COMMUTATIVE LAW

$A + B = B + A$

$A \cdot B = B \cdot A$

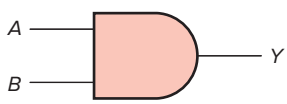
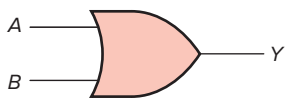
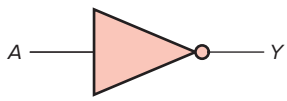
Logic symbol	Logic statement	Boolean equation	Boolean notations
	Y is 1 if A and B are 1	$Y = A \cdot B$ or $Y = AB$	Symbol / Meaning: • and; + or; – not; ° invert; = result in
	Y is 1 if A or B is 1	$Y = A + B$	
	Y is 1 if A is 0 Y is 0 if A is 1	$Y = \overline{A}$	

Figure 4-15 Boolean algebra as related to AND, OR, and NOT functions.

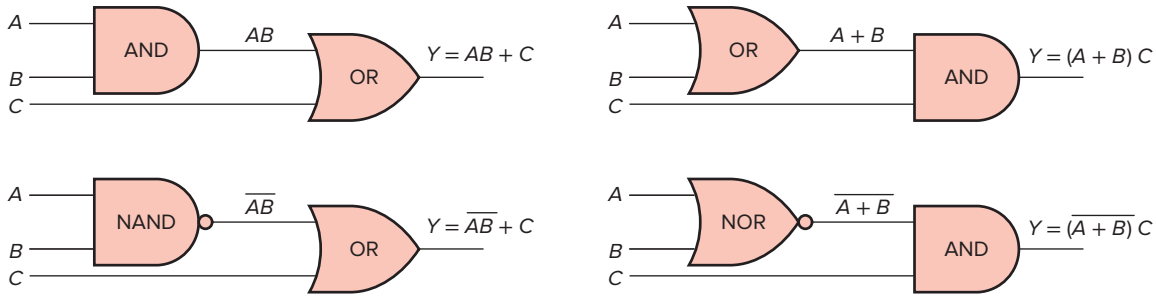


Figure 4-17 Logic operators used in combination to form Boolean equations.

ASSOCIATIVE LAW

$(A + B) + C = A + (B + C)$

$(A \cdot B) \cdot C = A \cdot (B \cdot C)$

DISTRIBUTIVE LAW

$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$

$A + (B \cdot C) = (A + B) \cdot (A + C)$

This law holds true only in Boolean algebra.

4.4 Developing Logic Gate Circuits from Boolean Expressions

As logic gate circuits become more complex, the need to express these circuits in Boolean form becomes greater. Figure 4-18 shows a logic gate circuit developed from the Boolean expression $Y = AB + C$. The procedure is as follows:

Boolean expression: $Y = AB + C$

Gates required: (by inspection)

1 - AND gate with input A and B

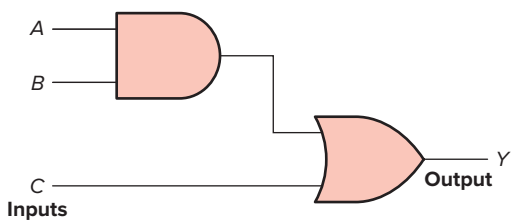


Figure 4-18 Logic gate circuit developed from the Boolean expression $Y = AB + C$.

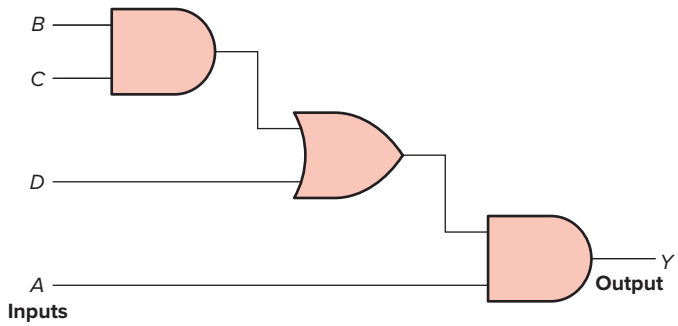


Figure 4-19 Logic gate circuit developed from the Boolean expression $Y = A(BC + D)$.

1 - OR gate with input C and output from previous AND gate

Figure 4-19 shows a logic gate circuit developed from the Boolean expression $Y = A(BC + D)$. The procedure is as follows:

Boolean expression: $Y = A(BC + D)$

Gates required: (by inspection)

1 - AND gate with inputs B and C

1 - OR gate with inputs BC and D

1 - AND gate with input A and the output from the OR gate

4.5 Producing the Boolean Equation for a Given Logic Gate Circuit

A simple logic gate is quite straightforward in its operation. However, by grouping these gates into combinations, it becomes more difficult to determine which combinations of inputs will produce an output. The Boolean equation for the logic circuit of Figure 4-20 is determined as follows:

- The output of the OR gate is $A + B$
- The output of the inverter is $\overline{D}$
- Based on the input combination applied to the AND gate the Boolean equation for the circuit is $Y = C\overline{D}(A + B)$

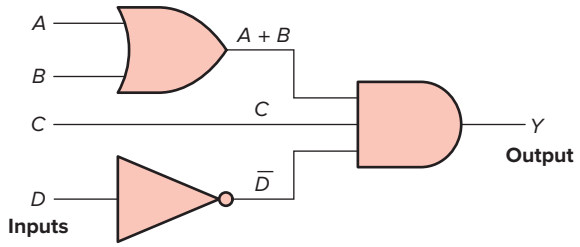


Figure 4-20 Determining the Boolean equation for a logic circuit.

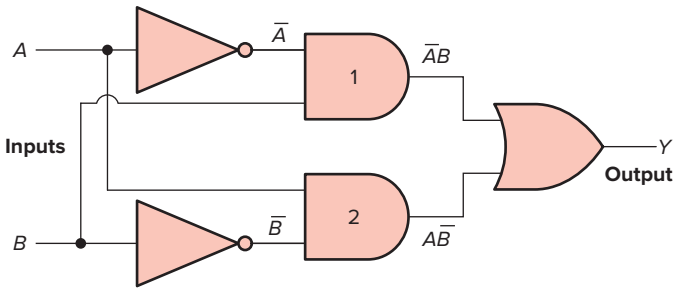


Figure 4-21 Determining the Boolean equation for a logic circuit.

The Boolean equation for the logic circuit of Figure 4-21 is determined as follows:

- The output of AND gate 1 is $\overline{A}B$
- The output of AND gate 2 is $A\overline{B}$
- Based on the combination of inputs applied to the OR gate the Boolean equation for the circuit is $Y = \overline{A}B + A\overline{B}$

4.6 Hardwired Logic versus Programmed Logic

The term ***hardwired logic*** refers to logic control functions that are determined by the way devices are electrically interconnected. Hardwired logic can be implemented using relays and relay ladder schematics. Relay ladder schematics are universally used and understood in industry. Figure 4-22 shows a typical relay ladder schematic of a motor stop/start control station with pilot lights. The control scheme is drawn between two vertical supply lines. All the components are placed between these two lines, called rails or legs, connecting the two power lines with what look like rungs of a ladder—thus the name, relay ladder schematic.

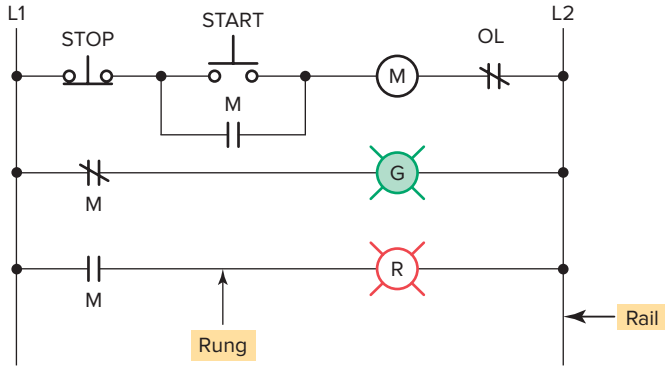


Figure 4-22 Motor stop/start relay ladder schematic.

Hardwired logic is fixed; it is changeable only by altering the way devices are electrically interconnected. In contrast, programmable control is based on the basic logic functions, which are programmable and easily changed. These functions (AND, OR, NOT) are used either singly or in combinations to form instructions that will determine if a device is to be switched on or off. The form in which these instructions are implemented to convey commands to the PLC is called the ***language***. The most common PLC language is *ladder logic*. Figure 4-23 shows a typical ***ladder logic program*** for the motor start/stop circuit. The instructions used are the relay equivalent of normally open (NO) and normally closed (NC) contacts and coils.

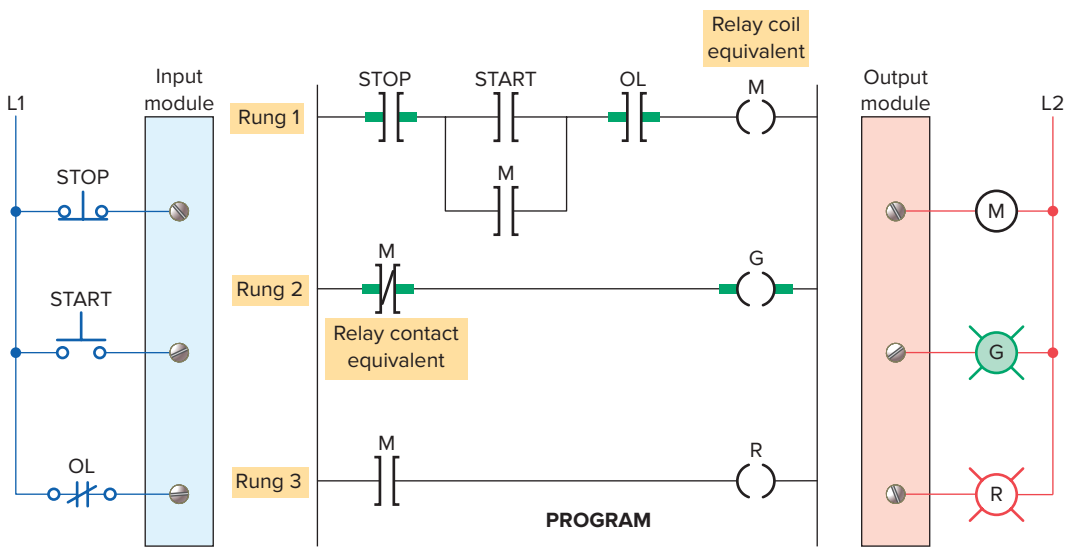
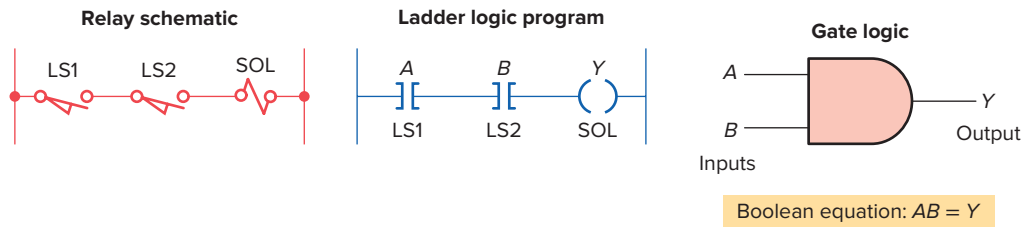


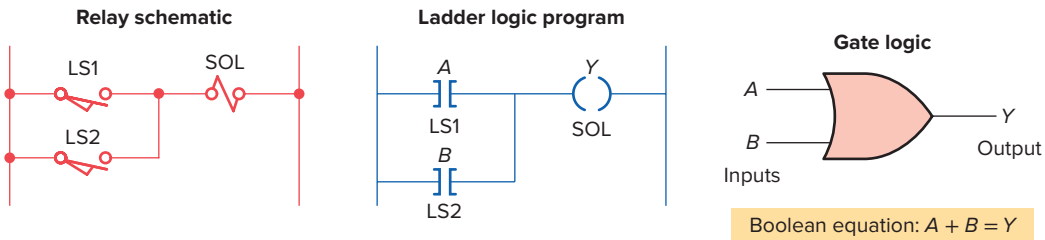
Figure 4-23 Motor stop/start ladder logic program.

PLC contact symbolism is a simple way of expressing the control logic in terms of symbols. These symbols are basically the same as those used for representing hard-wired relay control circuits. A **rung** is the contact symbolism required to control an output. Some PLCs allow a rung to have multiple outputs while others allow only one output per rung. A complete ladder logic program then consists of several rungs, each of which controls an output. In programmed logic all mechanical switch contacts are represented by a software contact symbol and all electromagnetic coils are represented by a software coil symbol.

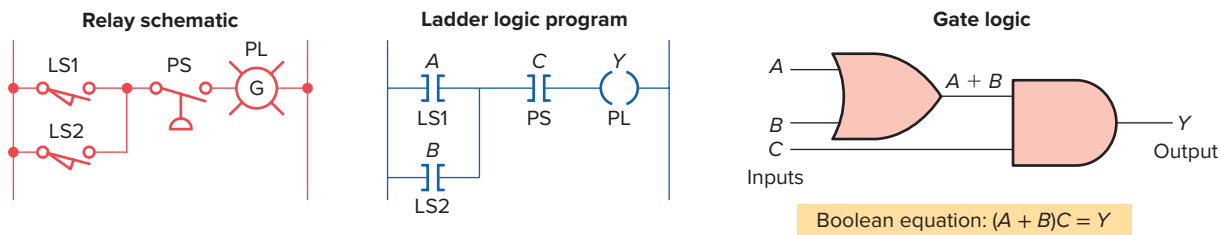
Because the PLC uses ladder logic diagrams, the conversion from any existing relay logic to programmed logic is simplified. Each rung is a combination of **input** conditions (symbols) connected from left to right, with the symbol that represents the **output** at the far right. The symbols that represent the inputs are connected in series, parallel, or some combination of the two to obtain the desired logic. The following group of examples illustrates the relationship between the relay ladder schematic, the ladder logic program, and the equivalent logic gate circuit.



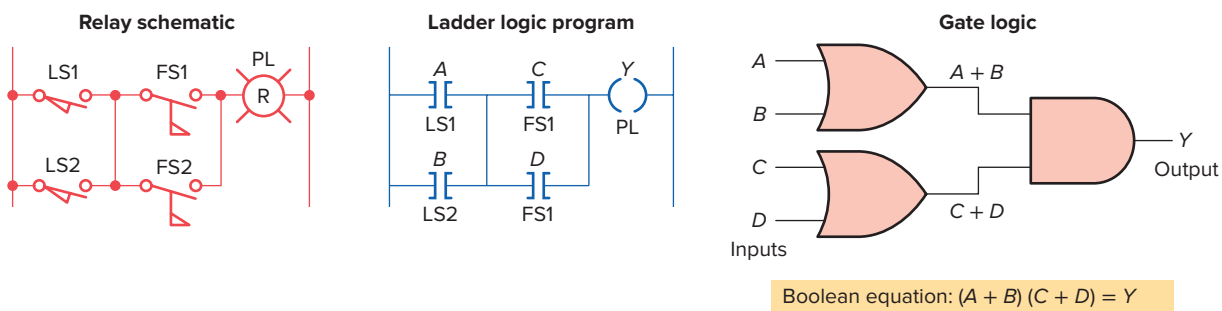
Example 4-1 Two limit switches connected in series and used to control a solenoid valve.



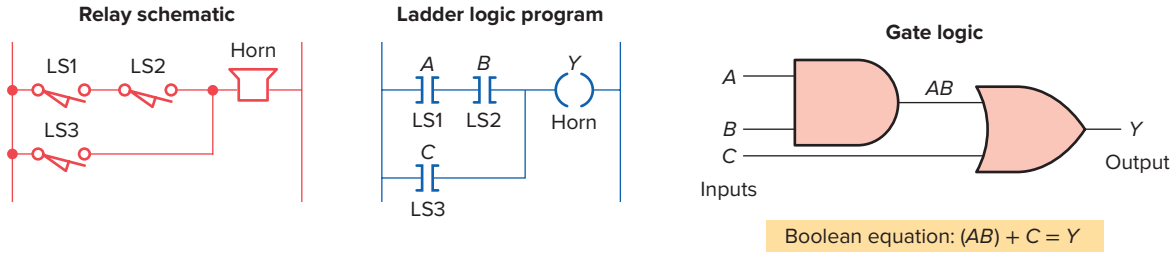
Example 4-2 Two limit switches connected in parallel and used to control a solenoid valve.



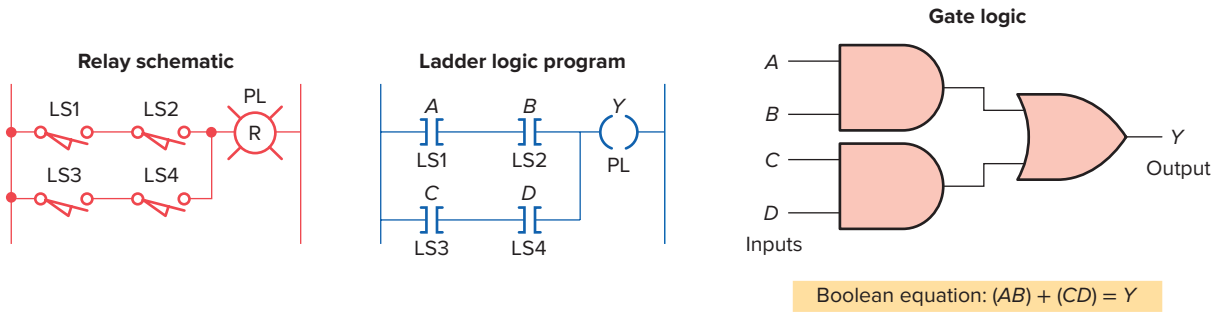
Example 4-3 Two limit switches connected in parallel with each other and in series with a pressure switch.



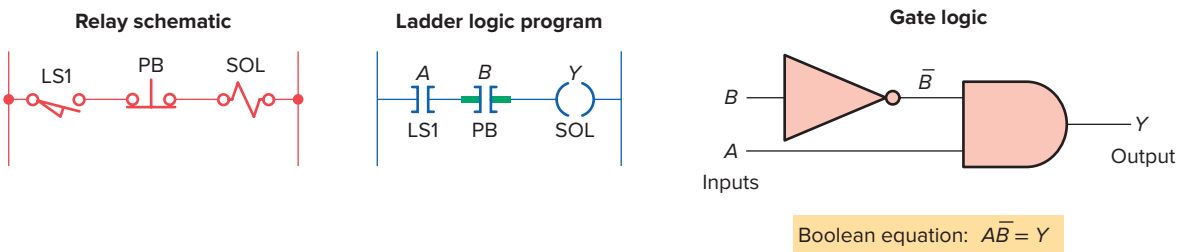
Example 4-4 Two limit switches connected in parallel with each other and in series with two sets of flow switches (that are connected in parallel with each other), and used to control a pilot light.



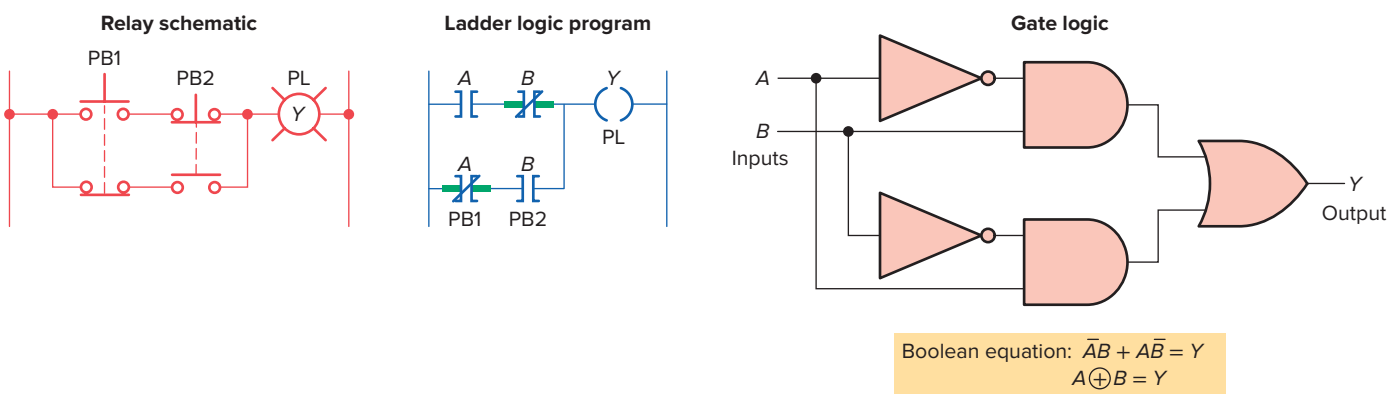
Example 4-5 Two limit switches connected in series with each other and in parallel with a third limit switch, and used to control a warning horn.



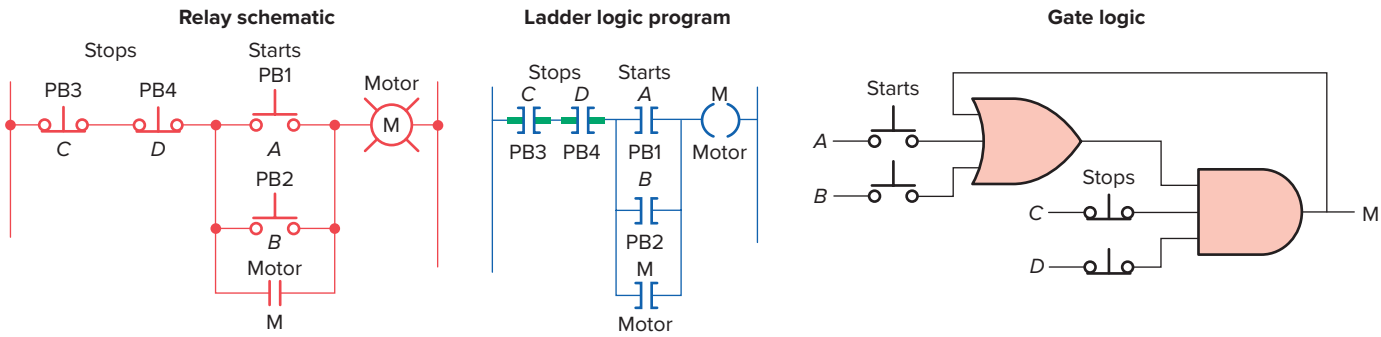
Example 4-6 Two limit switches connected in series with each other and in parallel with two other limit switches (that are connected in series with each other), and used to control a pilot light.



Example 4-7 One limit switch connected in series with a normally closed pushbutton and used to control a solenoid valve. This circuit is programmed so that the output solenoid will be turned on when the limit switch is closed and the pushbutton is *not pushed.*



Example 4-8 Exclusive-OR circuit. The output lamp of this circuit is ON only when pushbutton A or B is pressed, but not both. This circuit has been programmed using only the normally open A and B pushbutton contacts as the inputs to the program.



Example 4-9 A motor control circuit with two start/stop buttons. When either start button is depressed, the motor runs. By use of a seal-in contact, it continues to run when the start button is released. Either stop button stops the motor when it is depressed.

4.7 Programming Word Level Logic Instructions

Most PLCs provide **word-level** logic instructions as part of their instruction set. Table 4-1 shows how to select the correct word logic instruction for different situations.

Figure 4-24 illustrates the operation of the AND instruction to perform a word-level AND operation using the bits in the two source addresses. This instruction tells the processor to perform an AND operation on B3:5 and B3:7 and to store the result in destination B3:10 when input device A is true. The destination bits are a result of the logical AND operation.

Figure 4-25 illustrates the operation of a word-level OR instruction, which ORs the data in Source A, bit by bit, with the data in Source B and stores the result at the destination address. The address of Source A is B3:1, the address of Source B is B3:2, and the destination address is B3:20. The instruction may be programmed conditionally, with input instruction(s) preceding it, or unconditionally, as shown, without any input instructions preceding it.

Figure 4-26 illustrates the operation of a word-level XOR instruction. In this example, data from input I:1.0 are compared, bit by bit, with data from input I:3.0.

Table 4-1 Selecting Logic Instructions

If you want to . . .	. . . use this instruction.
Know when matching bits in two different words are both ON	AND
Know when one or both matching bits in two different words are ON	OR
Know when one or the other bit of matching bits in two different words is ON	XOR
Reverse the state of bits in a word	NOT

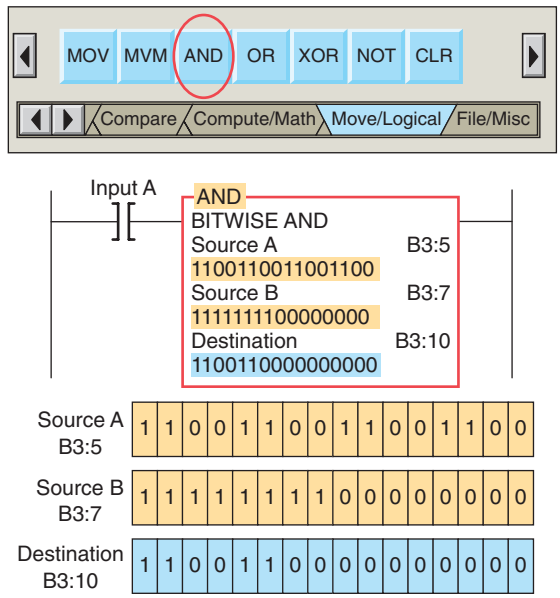


Figure 4-24 Word-level AND instruction.

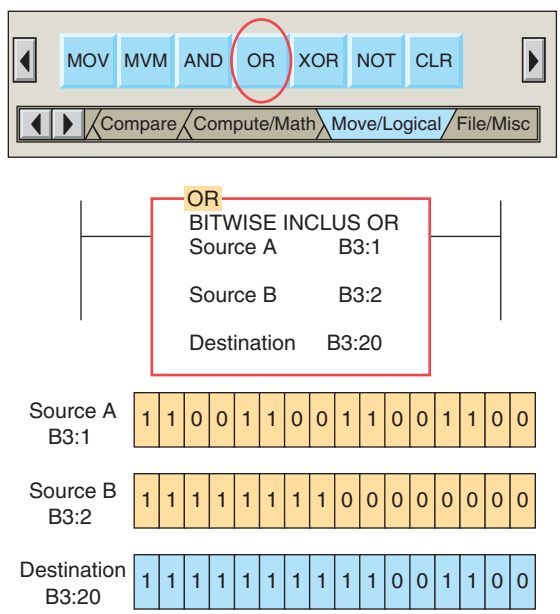


Figure 4-25 Word-level OR instruction.

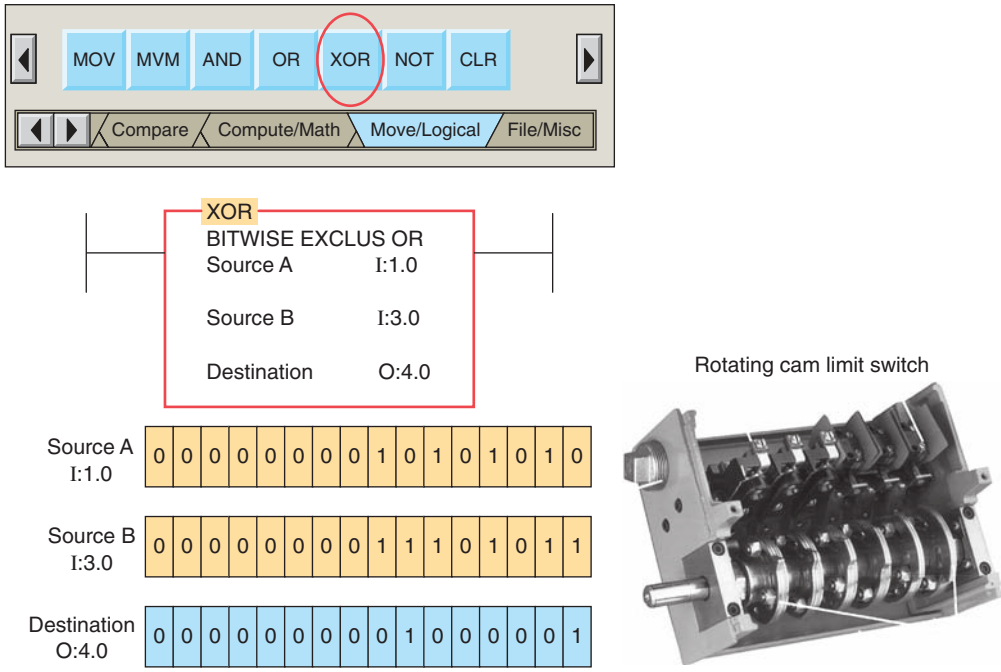


Figure 4-26 Word-level XOR instruction.
Source: Image Courtesy of Rockwell Automation, Inc.

Any mismatches energize the corresponding bit in word O:4.0. As you can see, there is a 1 in every bit location in the destination corresponding to the bit locations where Source A and Source B are different, and a 0 in the destination where Source A and Source B are the same. The XOR is often used in diagnostics, where real-world inputs, such as rotary cam limit switches, are compared with their desired states.

Figure 4-27 illustrates the operation of a word-level NOT instruction. This instruction inverts the bits from the source word to the destination word. The bit pattern in B3:10 is the result of the instruction being true and is the inverse of the bit pattern in B3:9.

For 32-bit PLCs, such as the Allen-Bradley ControlLogix controller, the source and destination may be a SINT (one-byte integer), INT (two-byte integer), or DINT (four-byte integer).

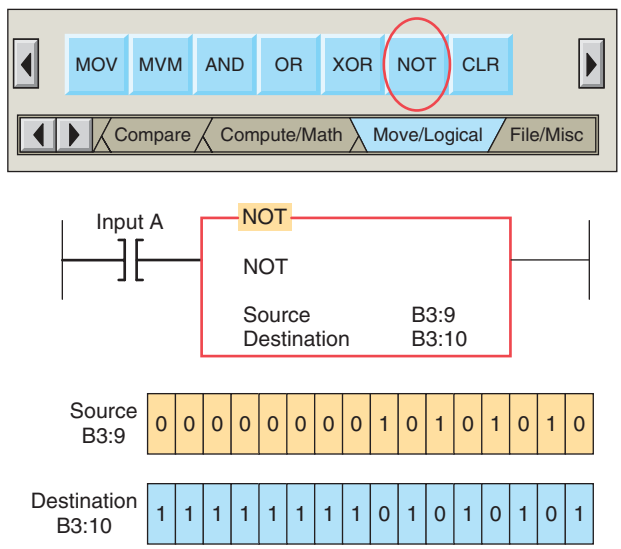


Figure 4-27 Word-level NOT operation.

CHAPTER 4 REVIEW QUESTIONS

1. Explain the binary principle.
2. What is a logic gate?
3. Draw the logic symbol, construct a truth table, and state the Boolean equation for each of the following:
   a. Two-input AND gate
   b. NOT function
   c. Three-input OR gate
   d. XOR function
4. Express each of the following equations as a ladder logic program:
   a. $Y = (A + B)CD$
   b. $Y = A\overline{B}C + \overline{D} + E$
   c. $Y = [(\overline{A} + \overline{B})C] + DE$
   d. $Y = (\overline{A}B\overline{C}) + (D\overline{E}F)$
5. Write the ladder logic program, draw the logic gate circuit, and state the Boolean equation for the two relay ladder diagrams in Figure 4-28.
6. Develop a logic gate circuit for each of the following Boolean expressions using AND, OR, and NOT gates:
   a. $Y = ABC + D$
   b. $Y = AB + CD$
   c. $Y = (A + B)(\overline{C} + D)$
   d. $Y = \overline{A}(B + CD)$
   e. $Y = \overline{A}B + C$
   f. $Y = (ABC + D)(E\overline{F})$
7. State the logic instruction you would use when you want to:
   a. Know when one or both matching bits in two different words are 1.
   b. Reverse the state of bits in a word.
   c. Know when matching bits in two different words are both 1.
   d. Know when one or the other bit of matching bits, but not both, in two different words is 1.
8. For the logic gate circuit shown in Figure 4-29:
   a. Determine the Boolean equation.
   b. Draw an equivalent ladder logic program for the gate circuit.
9. For the logic gate circuit shown in Figure 4-30:
   a. Determine the Boolean equation.
   b. Draw an equivalent ladder logic program for the gate circuit.

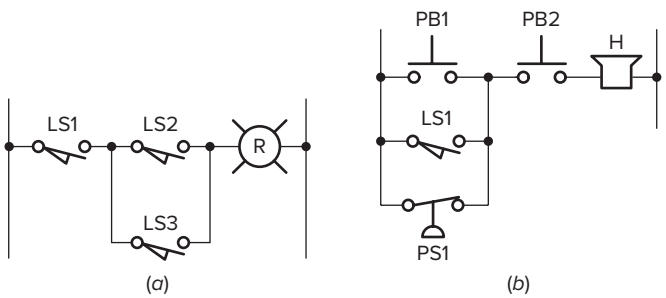


Figure 4-28 Question 5 relay ladder diagrams.

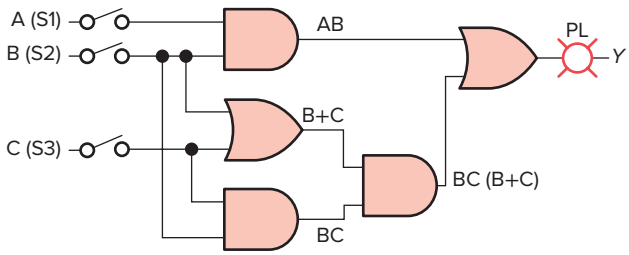


Figure 4-29

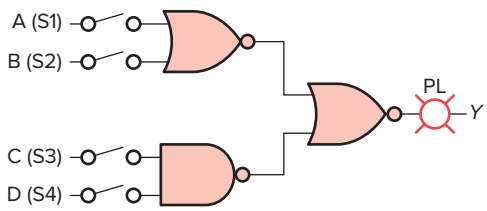
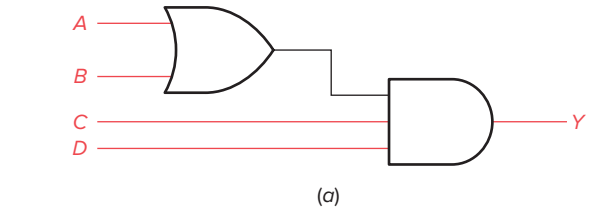


Figure 4-30

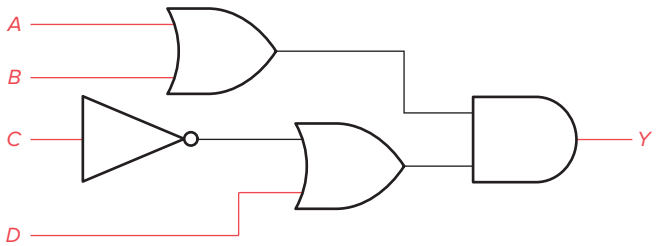
CHAPTER 4 PROBLEMS

1. It is required to have a pilot light come on when all of the following circuit requirements are met:
   - All four circuit pressure switches must be closed.
   - At least two out of three circuit limit switches must be closed.
   - The reset switch must not be closed.

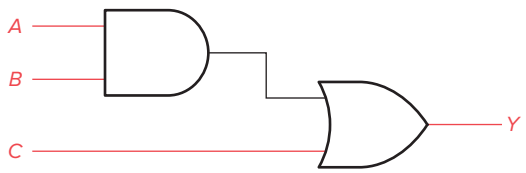
   Using AND, OR, and NOT gates, design a logic circuit that will solve this hypothetical problem.
2. Write the Boolean equation for each of the logic gate circuits in Figure 4-31a–f.



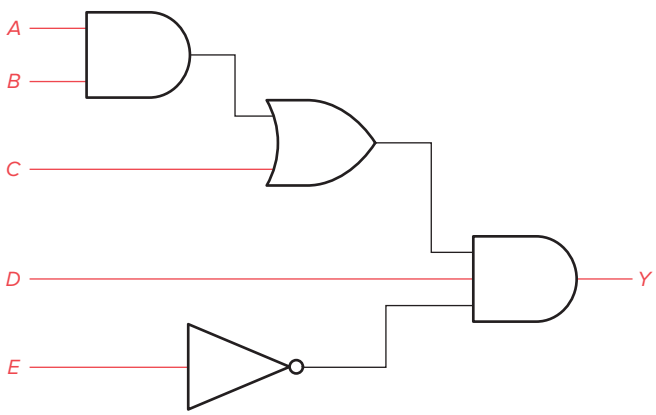
(a)



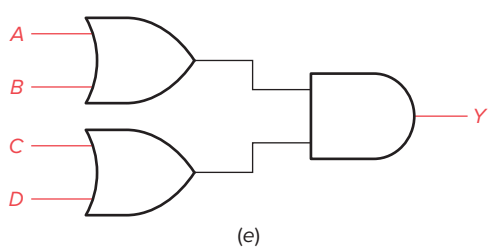
(b)



(c)

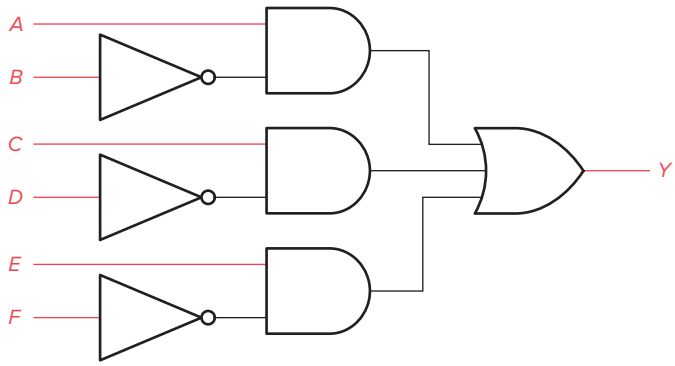


(d)



(e)

Figure 4-31 Logic gate circuits for Problem 2.



(f)

Figure 4-31 (*Continued*)

3. The logic circuit of Figure 4-32 is used to activate an alarm when its output Y is logic HIGH or 1. Draw a truth table for the circuit showing the resulting output for all 16 of the possible input conditions.
4. What will be the data stored in the destination address of Figure 4-33 for each of the following logical operations?
   a. AND operation
   b. OR operation
   c. XOR operation
5. Write the Boolean expression and draw the gate logic diagram and typical PLC ladder logic diagram for a control system wherein a fan is to run only when all of the following conditions are met:
   - Input A is OFF
   - Input B is ON or input C is ON, or both B and C are ON
   - Inputs D and E are both ON
   - One or more of inputs F, G, or H are ON

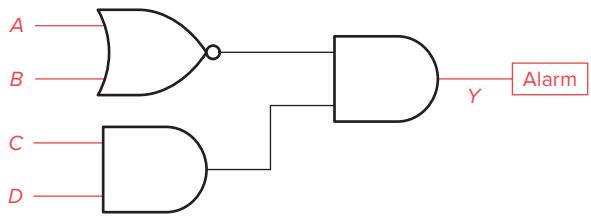


Figure 4-32 Logic circuit for Problem 3.

Source A	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	0
Source B	0	0	0	0	0	0	0	0	1	1	1	0	1	0	1	1
Destination																

Figure 4-33 Data for Problem 4.

5

Basics of PLC Programming

Symbol	Name	Bit status	Instruction status
	XIC	0	FALSE
		1	TRUE
	XIO	0	TRUE
		1	FALSE

Each input and output PLC module terminal is identified by a unique address. In PLCs, the internal symbol for any input is a contact. Similarly, in most cases, the internal PLC symbol for all outputs is a coil. This chapter shows how these contact/coil functions are used to program a PLC for circuit operation. This chapter covers only the basic set of instructions that perform functions similar to relay functions. You will also learn more about the program scan cycle and the scan time of a PLC.

Chapter Objectives

After completing this chapter, you will be able to:

- Define and identify the functions of a PLC memory map
- Describe input and output image table files and types of data files
- Describe the PLC program scan sequence
- Understand how ladder diagram language, Boolean language, and function chart programming language are used to communicate information to the PLC
- Define and identify the function of internal relay instructions
- Identify the common operating modes found in PLCs
- Write and enter ladder logic programs

5.1 Processor Memory Organization

While the fundamental concepts of PLC programming are common to all manufacturers, differences in memory organization, I/O addressing, and instruction set mean that PLC programs are never perfectly interchangeable among different makers. Even within the same product line of a single manufacturer, different models may not be directly compatible.

The **memory** map or structure for a PLC processor consists of several areas, some of these having specific roles. Allen-Bradley PLCs have two different memory structures identified by the terms *rack-based* systems and *tag-based* systems. The SLC 500 family of controllers uses a **rack-based** fixed memory structure. The I/O addresses are derived using the slot location of the input and output modules within the PLC rack. In comparison, the ControlLogix 5000 series of controllers uses a **tag-based** memory structure for assigning and referencing memory locations. A tag is a friendly name for a memory location. In tag-based memory structures there are no fixed areas of memory allocated for I/O addresses or other types of data. The memory organization for rack-based systems will be covered in detail in this chapter and that for tag-based systems in Chapter 15.

Memory organization takes into account the way a PLC divides the available memory into different sections. The memory space can be divided into two broad categories: *program files* and *data files*. Individual sections, their order, and the sections' length will vary and may be fixed or variable, depending on the manufacturer and model.

Program files are the part of the processor memory that stores the user ladder logic program. The program accounts for most of the total memory of a given PLC system. It contains the ladder logic that controls the machine operation. This logic consists of instructions that are programmed in a ladder logic format. Most instructions require one word of memory.

The **data files** store the information needed to carry out the user program. This includes information such as the status of input and output devices, timer and counter values, data storage, and so on. Contents of the data table can be divided into two categories: status data and numbers or codes. Status is ON/OFF type of information represented by 1s and 0s, stored in unique bit locations. Number or code information is represented by groups of bits that are stored in unique byte or word locations.

Figure 5-1 shows the program and data file organization for the SLC 500 controller. The contents of each file are as follows.

Program Files

Program files (Figure 5-2) are the areas of processor memory where ladder logic programming is stored. They may include:

- **System functions (file 0)**—This file is always included and contains various system-related information and user-programmed information such as processor type, I/O configuration, processor file name, and password.
- **Reserved (file 1)**—This file is reserved by the processor and is not accessible to the user.
- **Main ladder program (file 2)**—This file is always included and contains user-programmed instructions that define how the controller is to operate.
- **Subroutine ladder program (files 3–255)**—These files are user-created and are activated according to subroutine instructions residing in the main ladder program file.

Data Files

The data file portion (Figure 5-3) of the processor's memory stores input and output status, processor status, the status of various bits, and numerical data. All this information is accessed via the ladder logic program. These files are organized by the type of data they contain and may include:

- **Output (file 0)**—This file stores the state of the output terminals for the controller.
- **Input (file 1)**—This file stores the status of the input terminals for the controller.
- **Status (file 2)**—This file stores controller operation information and is useful for troubleshooting controller and program operation.
- **Bit (file 3)**—This file is used for internal relay logic storage.
- **Timer (file 4)**—This file stores the timer accumulated and preset values and status bits.
- **Counter (file 5)**—This file stores the counter accumulated and preset values and status bits.
- **Control (file 6)**—This file stores the length, pointer position, and status bit for specific instructions such as shift registers and sequencers.
- **Integer (file 7)**—This file is used to store whole number values or bit information.
- **Float (file 8)**—The floating point file is used to store fractional numerical data or numerical values greater than 32,767. This file applies to selected PLC processors.

The I/O address format for the SLC family of PLCs is shown in Figure 5-4. The format consists of the following three parts:

Part 1: I for input, and a colon to separate the module type from the slot.

O for output and a colon to separate the module type from the slot.

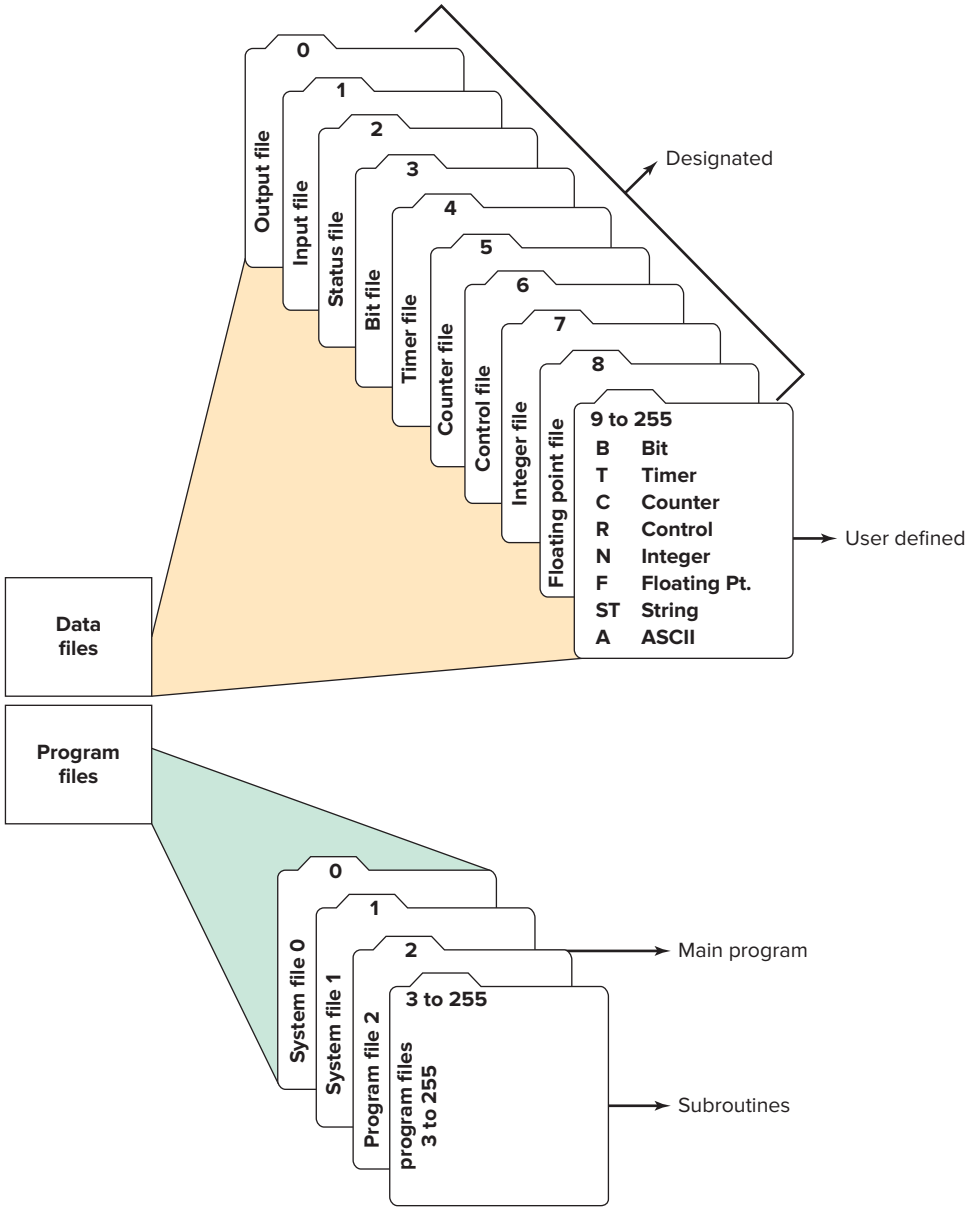


Figure 5-1 Program and data file organization for the SLC 500 controller.

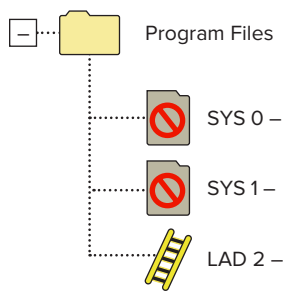


Figure 5-2 Program file tree.

Part 2: The module slot number and a forward slash to separate the slot from the terminal screw.

Part 3: The screw terminal number.

The SLC 500 stores data in data tables that are based on 16-bit words. The ***input image table file*** is that part of the program memory allocated to storing the on/off status of connected discrete inputs. Figure 5-5 shows the connection of an open and closed switch to the input image table file through the input module. Its operation can be summarized as follows.

- For the switch that is closed, the processor detects a voltage at the input terminal and records that information by storing a binary 1 in its bit location.
- For the switch that is open, the processor detects no voltage at the input terminal and records that information by storing a binary 0 in its bit location.
- Each connected input has a bit in the input image table file that corresponds exactly to the terminal to which the input is connected.
- The input image table file is changed to reflect the current status of the switch during the I/O scan phase of operation.
- If the input is on (switch closed), its corresponding bit in the table is set to 1.
- If the input is off (switch open), the corresponding bit is cleared, or reset to 0.

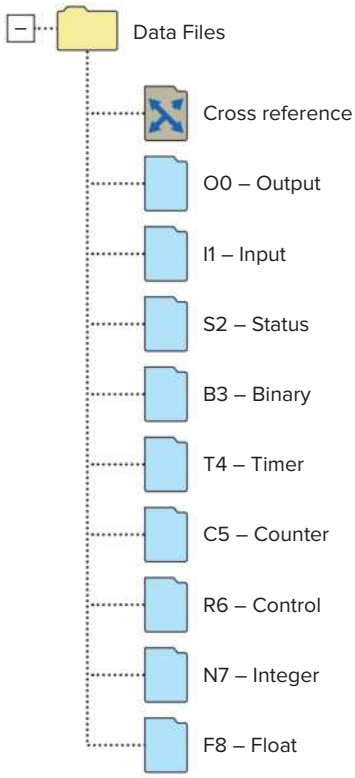


Figure 5-3 Data file tree.

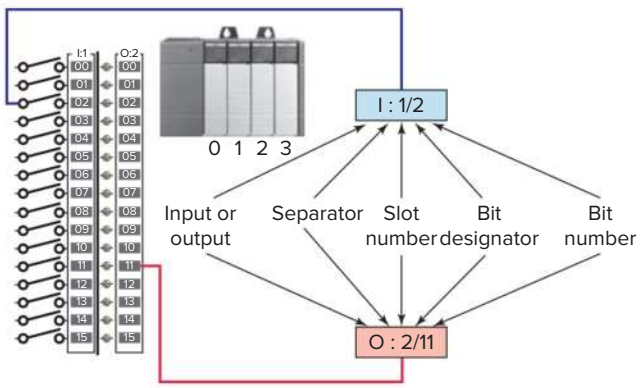


Figure 5-4 I/O address format for the SLC family of PLCs.
Source: Image Courtesy of Rockwell Automation, Inc.

- The processor continually reads the current input status and updates the input image table file.

The ***output image table file*** is that part of the program memory allocated to storing the actual on/off status of connected discrete outputs. Figure 5-6 shows a typical connection of two pilot lights to the output image table file through the output module. Its operation can be summarized as follows.

- The status of each light (ON/OFF) is controlled by the user program and is indicated by the presence of 1 (ON) and 0 (OFF).
- Each connected output has a bit in the output image table file that corresponds exactly to the terminal to which the output is connected.

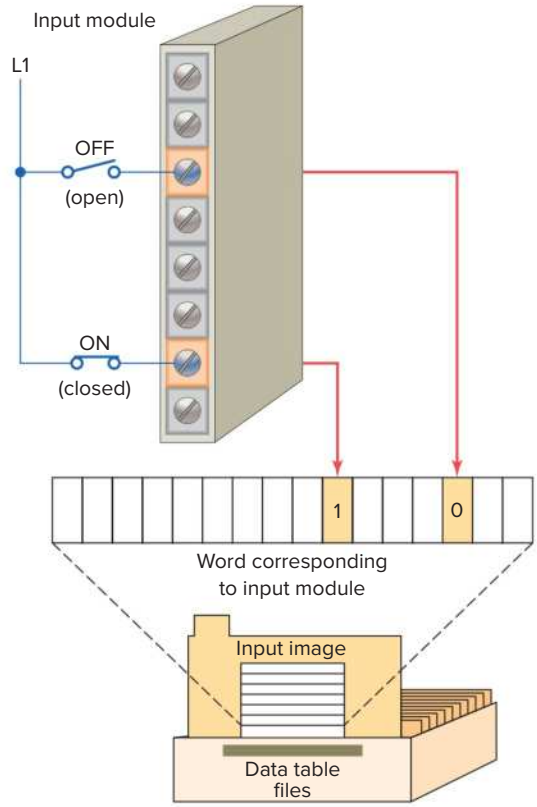


Figure 5-5 Connection of an open and closed switch to the input image table file through the input module.

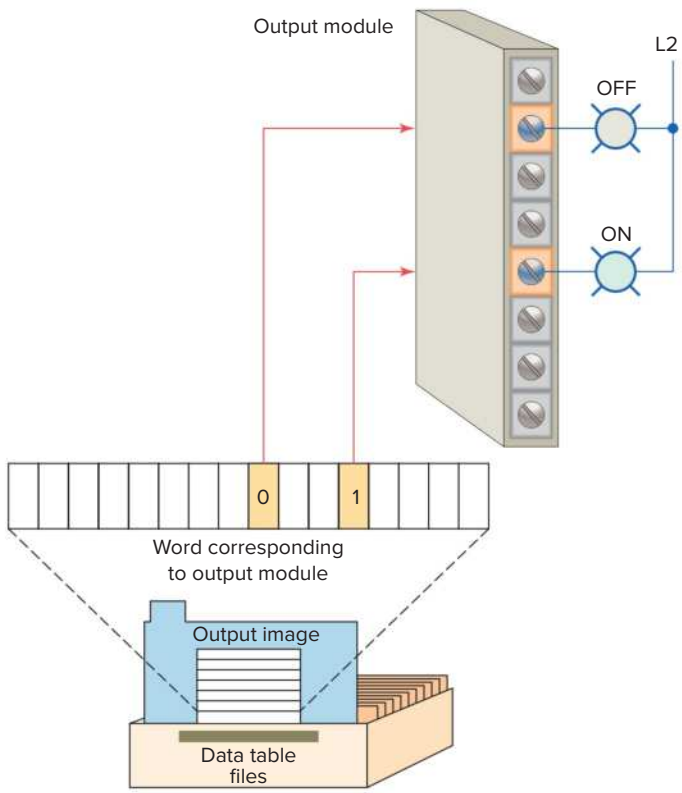


Figure 5-6 Connections of pilot lights to the output image table file through the output module.

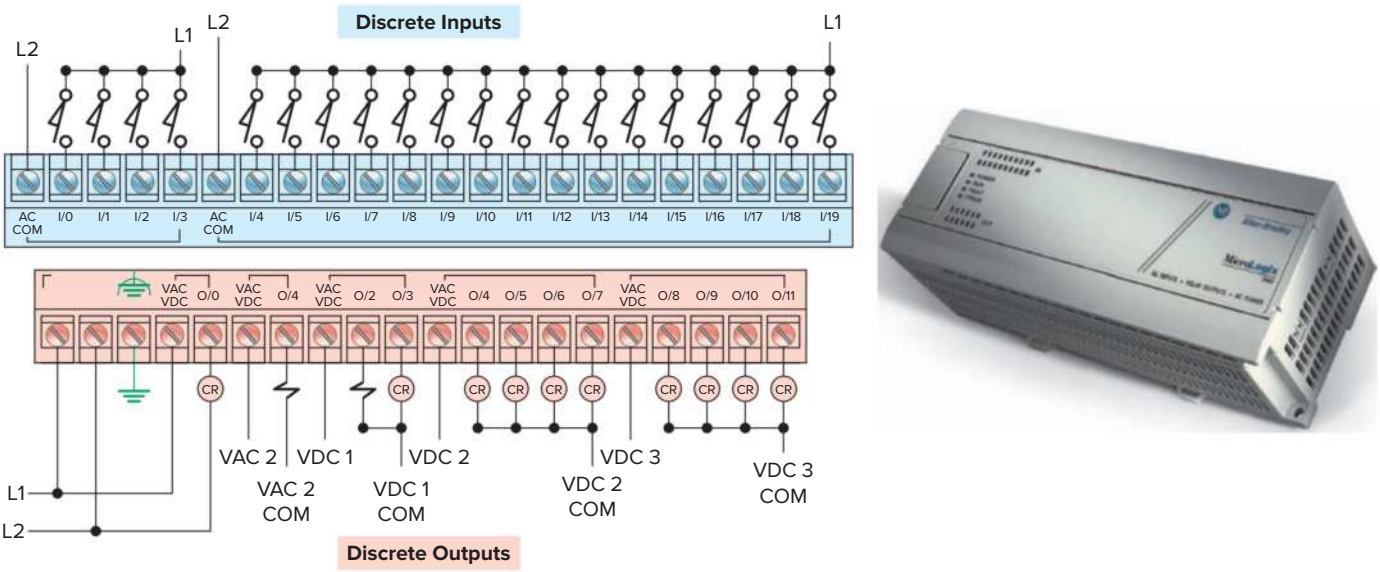


Figure 5-7 Typical micro PLC with predefined addresses.
Source: Image Courtesy of Rockwell Automation, Inc.

- If the program calls for a specific output to be ON, its corresponding bit in the table is set to 1.
- If the program calls for the output to be OFF, its corresponding bit in the table is set to 0.
- The processor continually activates or deactivates the output status according to the output table file status.

Typically, micro PLCs have a fixed number of inputs and outputs. Figure 5-7 shows the MicroLogix controller from the Allen-Bradley MicroLogix 1000 family of controllers. The controller has 20 discrete inputs with predefined addresses I/0 through I/19 and 12 discrete outputs with predefined addresses O/0 through O/11. Some units also contain analog inputs and outputs embedded into the base unit or available through add-on modules.

5.2 Program Scan

When a PLC executes a program, it must know—in real time—when external devices controlling a process are changing. During each operating cycle, the processor reads all the inputs, takes these values, and energizes or de-energizes the outputs according to the user program. This process is known as a ***program scan cycle.*** Figure 5-8 illustrates a single PLC operating cycle consisting of the *input scan, program scan, output scan,* and housekeeping duties. Because the inputs can change at any time, it constantly repeats this cycle as long as the PLC is in the RUN mode.

The time it takes to complete a scan cycle is called the **scan cycle time** and indicates how fast the controller can react to changes in inputs. The time required to make a

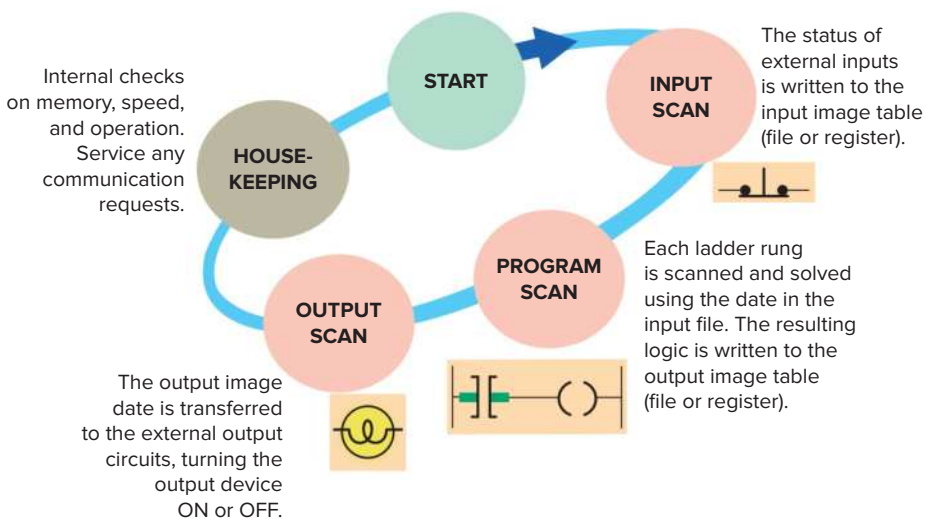


Figure 5-8 PLC program scan cycle.

single scan can vary from about 1 to 20 ms. If a controller has to react to an input signal that changes states twice during the scan time, it is possible that the PLC will never be able to detect this change. For example, if it takes 8 ms for the CPU to scan a program, and an input contact is opening and closing every 4 ms, the program may not respond to the contact changing state. The CPU will detect a change if it occurs during the update of the input image table file, but the CPU will not respond to every change. The scan time is a function of the following:

- The speed of the processor module
- The length of the ladder program
- The type of instructions executed
- The actual ladder true/false conditions

The actual scan time is calculated and stored in the PLC's memory. The PLC computes the scan time each time the END instruction is executed. Scan time data can be monitored via the PLC programming. Typical scan time data include the maximum scan time and the last scan time.

The scan is normally a continuous and sequential process of reading the status of inputs, evaluating the control logic, and updating the outputs. Figure 5-9 shows an overview of the data flow during the scan process. For each rung executed, the PLC processor will:

- Examine the status of the input image table bits.
- Solve the ladder logic in order to determine logical continuity.
- Update the appropriate output image table bits, if necessary.
- Copy the output image table status to all of the output terminals. Power is applied to the output device if the output image table bit has been previously set to a 1.
- Copy the status of all of the input terminals to the input image table. If an input is active (i.e., there is electrical continuity), the corresponding bit in the input image table will be set to a 1.

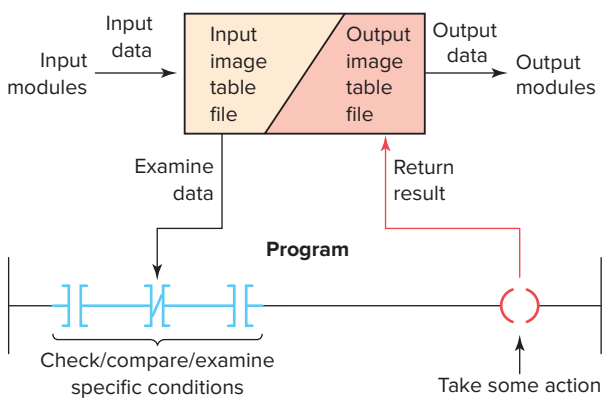


Figure 5-9 Overview of the data flow during the scan process.

In a rung of any hardwired circuit there must be **electrical continuity** in order for the load to energize. The rung has electrical continuity only when the current flow is established in a path from one side of the power rail to the other. There is no electrical continuity in the PLC ladder logic program. Instead, the rung must be evaluated in terms of **logical continuity** rather than electrical continuity. When there is a continuous path of true conditional instructions in a rung, logical continuity exists; accordingly the output instruction is true and the status bit will be set to a 1 (ON).

The controller evaluates ladder logic rung instructions based on the rung condition preceding the instruction (rung-condition-in), as illustrated in Figure 5-10.

- If the rung-condition-in to an input instruction is true, the controller evaluates the instruction and sets the rung-condition-out to match the results of the evaluation.
- If the instruction evaluates to true, the rung-condition-out is true.
- If the instruction evaluates to false, the rung-condition-out is false.
- If the rung-condition-in to an output instruction is true, the rung-condition-out is set to true.
- If the rung-condition-in to an output instruction is false, the rung-condition-out is set to false.

Figure 5-11 illustrates the scan process applied to a simple single rung program. The operation of the scan process can be summarized as follows:

- If the input device connected to address I:3/6 is closed, the input module circuitry senses voltage at the input terminal and a 1 (ON) condition is entered into the input image table bit I:3/6.
- During the program scan, the processor examines bit I:3/6 for a 1 (ON) condition.
- In this case, because input I:3/6 is 1, the rung is said to be TRUE or have *logic continuity.*

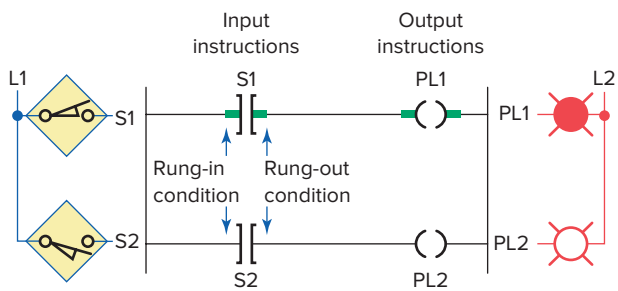


Figure 5-10 Evaluating ladder logic rung conditions.

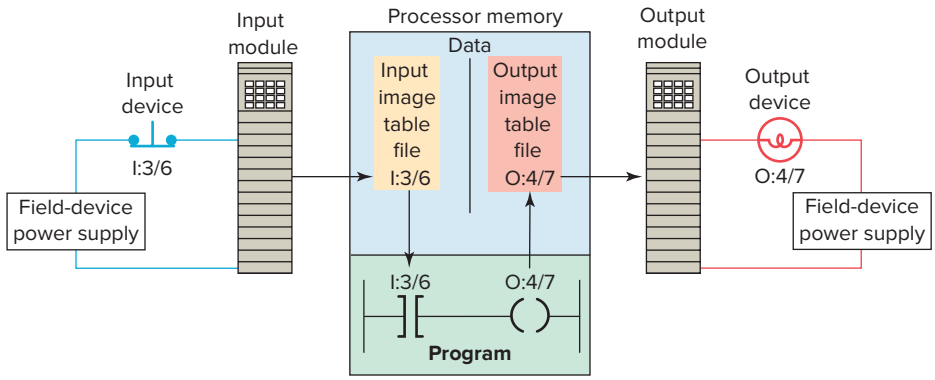


Figure 5-11 Scan process applied to a single rung program.

- The processor then sets the output image table bit O:4/7 to 1.
- The processor turns on output O:4/7 during the next I/O scan, and the output device (light) wired to this terminal becomes energized.
- This process is repeated as long as the processor is in the RUN mode.
- If the input device opens, electrical continuity is lost, and a 0 would be placed in the input image table. As a result, the rung is said to be FALSE due to loss of logic continuity.
- The processor would then set the output image table bit O:4/7 to 0, causing the output device to turn off.

Ladder programs process inputs at the beginning of a scan and outputs at the end of a scan, as illustrated in Figure 5-12. For each rung executed, the PLC processor will:

Step 1 Update the input image table by sensing the voltage of the input terminals. Based on the absence or presence of a voltage, a 0 or a 1 is stored into the memory bit location designated for a particular input terminal.

Step 2 Solve the ladder logic in order to determine logical continuity. The processor scans the ladder program and evaluates the logical continuity of each rung by referring to the input image table to see if the input conditions are met. If the conditions controlling an output are met, the processor immediately writes a 1 in its memory location, indicating that the output will be turned ON; conversely, if the conditions are not met a 0 indicating that the device will be turned OFF is written into its memory location.

Step 3 The final step of the scan process is to update the actual states of the output devices by transferring the output table results to the output module, thereby switching the connected output devices ON (1) or OFF (0). If the status of any input devices changes when the processor is in step 2 or 3, the output condition will not react to them until the next processor scan.

Each instruction entered into a program requires a certain amount of time for the instruction to be executed. The amount of time required depends on the instruction. For example, it takes less time for a processor to read the status of an input contact than it does to read the accumulated value of a timer or counter. The time taken to scan the user program is also dependent on the clock frequency of the microprocessor system. The higher the clock frequency, the faster is the scan rate. Typical processor clock frequencies range between 1 to 10 MHz.

There are two basic scan patterns that different PLC manufacturers use to accomplish the scan function

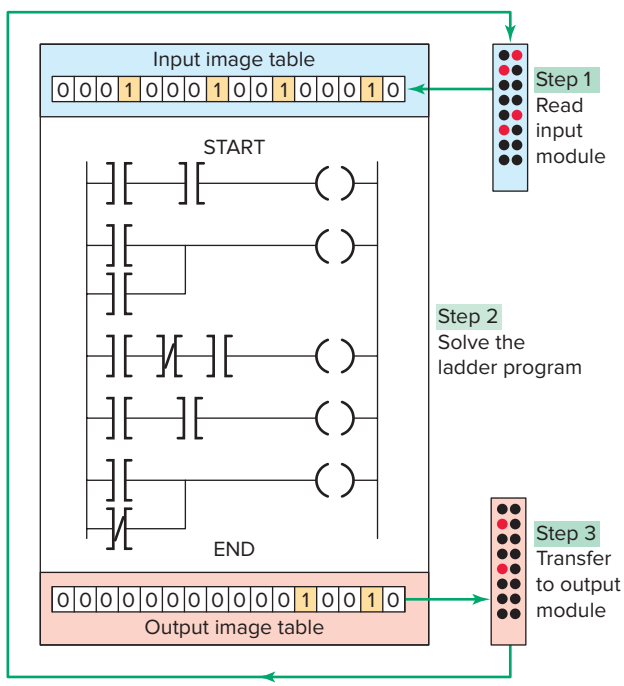


Figure 5-12 Scan process applied to a multiple rung program.

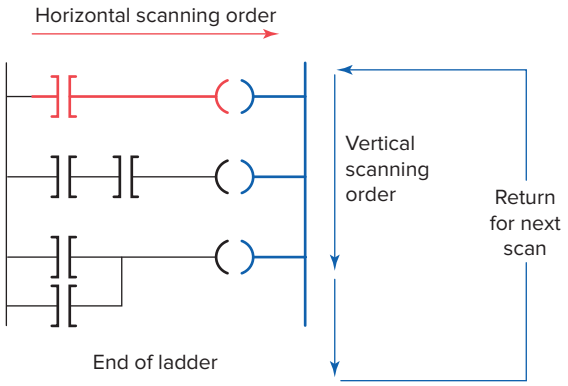


Figure 5-13 Scanning can be vertical or horizontal.

(Figure 5-13). Allen-Bradley PLCs use the ***horizontal*** scan by rung method. In this system, the processor examines input and output instructions from the first command, top left in the program, horizontally, rung by rung. Modicon PLCs use the ***vertical*** scan by column method. In this system, the processor examines input and output instructions from the top left command entered in the ladder diagram, vertically, column by column and page by page. Pages are executed in sequence. Both methods are appropriate; however, misunderstanding the way the PLC scans a program can cause programming bugs.

5.3 PLC Programming Languages

The term ***PLC programming language*** refers to the method by which the user communicates information to the PLC. The standard IEC 61131 (Figure 5-14) was established to standardize the multiple languages associated with PLC programming by defining the following five standard languages:

- **Ladder Diagram (LD)**—a symbolic depiction of instructions arranged in rungs similar to ladder-formatted schematic diagrams.
- **Function Block Diagram (FBD)**—a graphical depiction of process flow using simple and complex interconnecting blocks.
- **Sequential Function Chart (SFC)**—a graphical depiction of interconnecting steps, actions, and transitions.
- **Instruction List (IL)**—a low-level, text-based language that uses mnemonic instructions.
- **Structured Text (ST)**—a high-level, text-based language such as BASIC, C, or PASCAL specifically developed for industrial control applications.

Ladder diagram language is the most commonly used PLC language and is designed to mimic relay logic. The ladder diagram is popular for those who prefer to define control actions in terms of relay contacts and coils, and other functions as block instructions. Figure 5-15 shows a comparison of ladder diagram programming and instruction list programming. Figure 5-15a shows the original relay hardwired control circuit. Figure 5-15b shows the equivalent logic ladder diagram programmed into a controller. Note how closely the ladder diagram program resembles the hardwired relay circuit. The input/output addressing is generally different for each PLC

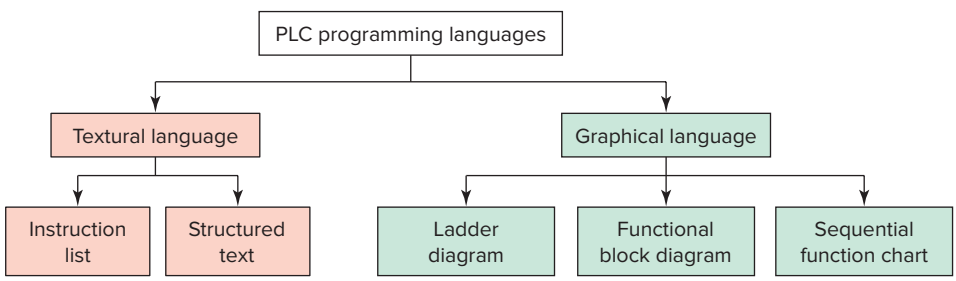
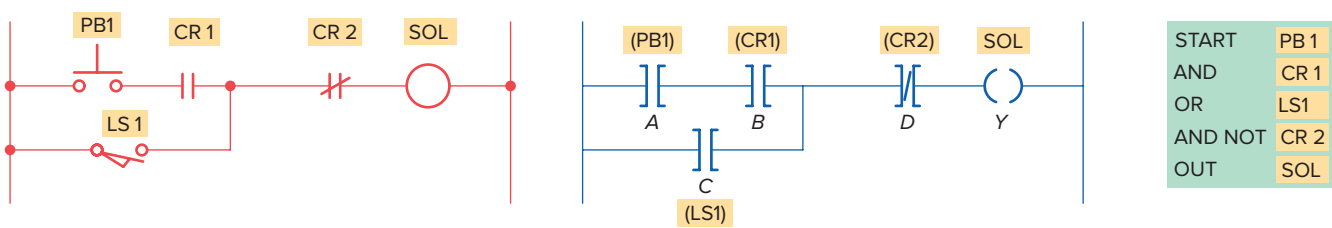


Figure 5-14 Standard IEC 61131 languages associated with PLC programming.



(*a*) Hardwired relay control circuit

(*b*) Equivalent ladder diagram (LD) program

(*c*) Equivalent instruction list (IL) program

Figure 5-15 Comparison of ladder diagram and instruction list programming.

manufacturer. Figure 5-15c shows how the original hard-wired circuit could be programmed using the instruction list programming language. Note that the instructional list consists of a series of instructions that refer to the basic AND, OR, and NOT logic gate functions.

Functional block diagram programming uses instructions that are programmed as blocks wired together on screen to accomplish certain functions. Typical types of function blocks include logic, timers, and counters. Functional block diagrams are similar in layout to electrical/electronic block diagrams used to simplify complex systems by showing blocks of functionality. The primary concept behind a functional block diagram is data flow. Function blocks are linked together to complete a circuit that satisfies a control requirement. Data flow on a path from inputs, through function blocks or instructions, and then to outputs.

The use of function blocks for programming of programmable logic controllers (PLCs) is gaining wider acceptance. Rather than the classic contact and coil representation of ladder diagram or relay ladder logic programming, function blocks present a graphical image to the programmer with underlying algorithms already defined. The programmer simply completes needed information within the block to complete that phase of the program. Figure 5-16 shows function block diagram equivalents to ladder logic contacts.

Figure 5-17 illustrates how ladder diagram and functional block diagram programming could be used to produce the same logical output. For this application, the objective is to turn on caution pilot light PL 1 whenever both sensor switch 1 and sensor switch 2 are closed. The ladder logic consists of a single rung across the power rails. This rung contains the two input sensor instructions programmed in series with the pilot light output instruction.

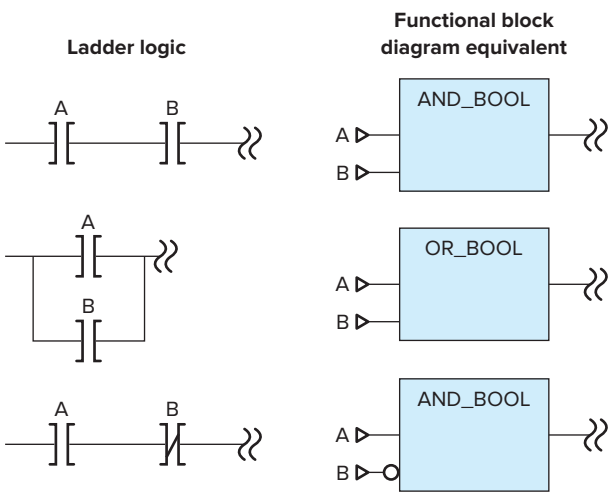


Figure 5-16 Function block diagram equivalents to ladder logic contacts.

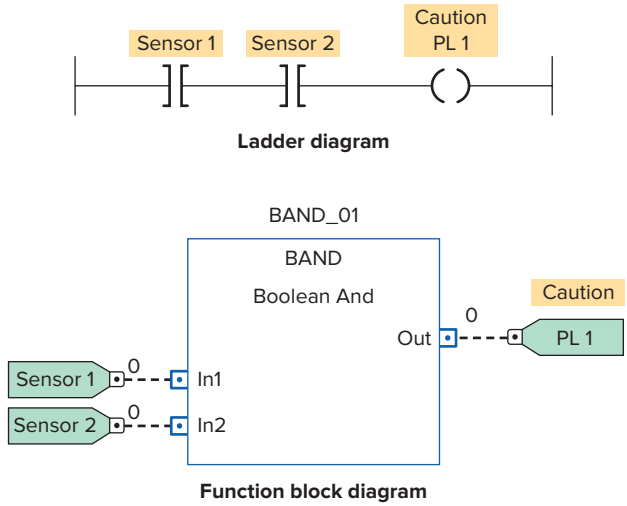


Figure 5-17 PLC ladder and equivalent function block diagram.

The function block solution consists of a logic *Boolean And* function block with two input references tags for the sensors and a single output reference tag for the pilot light. Note there are no power rails in the function block diagram.

Sequential function chart programming language is similar to a flowchart of your process. SFC programming is designed to accommodate the programming of more advanced processes. This type of program can be split into steps with multiple operations happening in parallel branches. The basic elements of a sequential function chart program are shown in Figure 5-18.

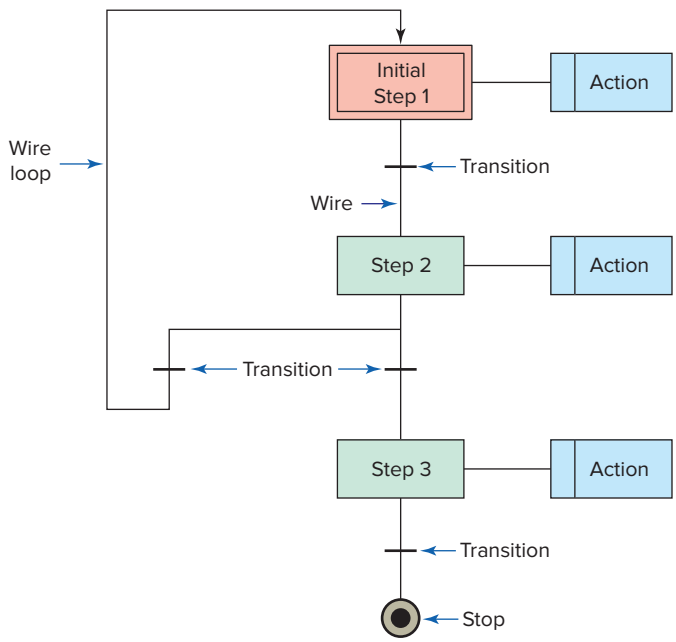
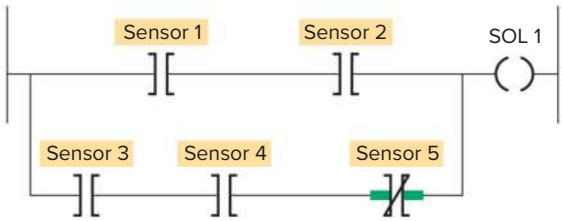


Figure 5-18 Major elements of a sequential function chart program.



Ladder diagram (LD) program

```
IF Sensor_1 AND Sensor_2 THEN
        SOL_1 := 1;
ELSEIF Sensor_3 AND Sensor_4 AND NOT Sensor_5 THEN
        SOL_1 := 1;
END_IF;
```

Structured text (ST) program

Figure 5-19 PLC ladder and equivalent structured text program.

Structured text is a high-level text language primarily used to implement complex procedures that cannot be easily expressed with graphical languages. Structured text uses statements to define what to execute. Figure 5-19 illustrates how structured text and ladder diagram programming could be used to produce the same logical output. For this application, the objective is to energize SOL 1 whenever either one of the two following circuit conditions exists:

- Sensor 1 and Sensor 2 switches are both closed.
- Sensor 3 and Sensor 4 switches are both closed and Sensor 5 switch is open.

5.4 Bit-Level Logic Instructions

The ladder diagram language is basically a ***symbolic*** set of instructions used to create the controller program. Bit-level symbolic instructions fall into two separate categories: instructions that examine data and instructions that control data. Each symbolic instruction is a command to perform a specific operation. These ladder instruction symbols are arranged to obtain the desired control logic that is to be entered into the memory of the PLC.

Representations of contacts and coils are the basic symbols of the logic ladder diagram instruction set. The three fundamental symbols that are used to translate relay control logic to contact symbolic logic are Examine If Closed (XIC), Examine If Open (XIO), and Output Energize (OTE). Each of these instructions relates to a **single bit** of PLC memory that is specified by the instruction's address. While the XIO and XIC are represented by symbols that resemble a normally-open and normally-closed relay contact, they **do not** operate like

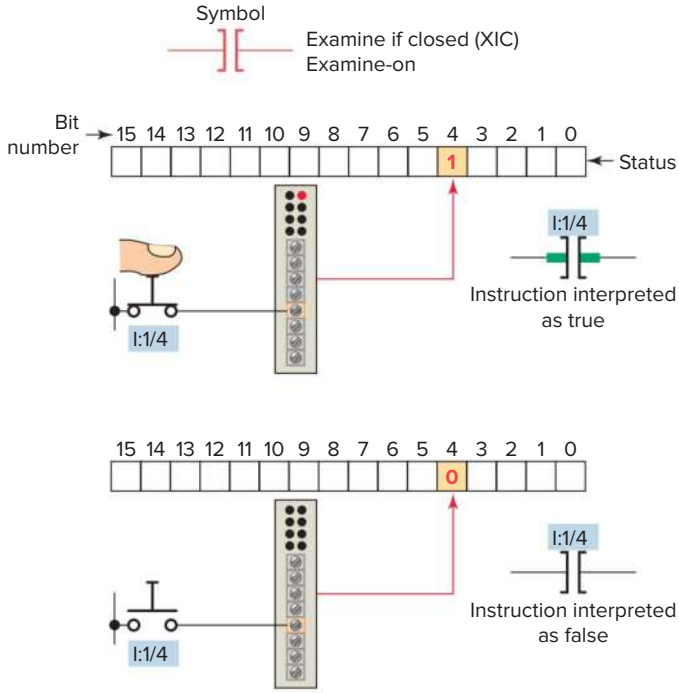


Figure 5-20 Examine If Closed (XIC) instruction.

relay contacts. Instead, they operate as commands that examine the value (0 or 1) of a bit of data to determine its true or false logical condition.

The symbol for the ***Examine If Closed (XIC)*** instruction is shown in Figure 5-20. The XIC instruction is also called the Examine-on instruction. Associated with each XIC instruction is a memory bit linked to the status of an input device or an internal logical condition in a rung. This instruction asks the PLC's processor to examine if the contact is *closed.* It does this by examining the bit at the memory location specified by the address in the following manner:

- The memory bit is set to 1 or 0 depending on the status of the input (physical) device or internal (logical) relay address associated with that bit.
- A 1 corresponds to a true status or on condition.
- A 0 corresponds to a false status or off condition.
- When the Examine-on instruction is associated with a physical input, the instruction will be set to 1 when a physical input is present (voltage is applied to the input terminal), and 0 when there is no physical input present (no voltage applied to the input terminal).
- When the Examine-on instruction is associated by address with an internal relay, then the status of the bit is dependent on the logical status of the internal bit with the same address as the instruction.

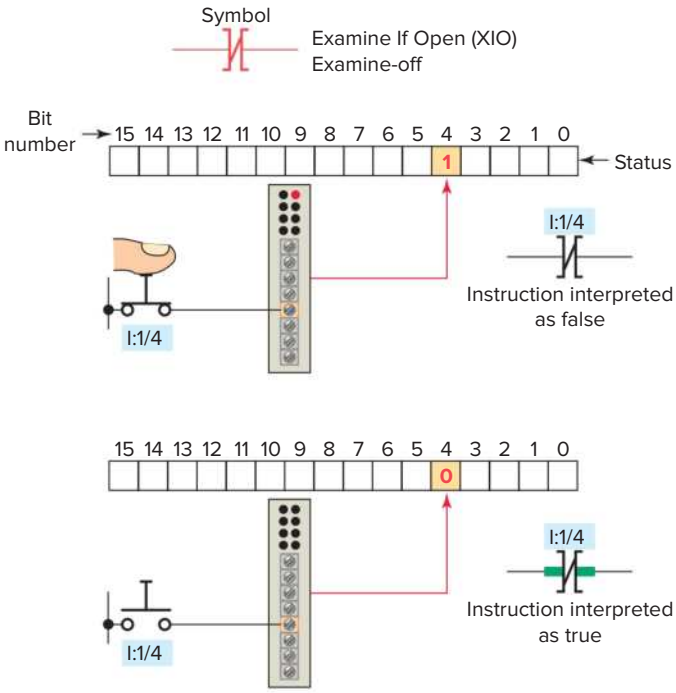


Figure 5-21 Examine If Open (XIO) instruction.

- If the instruction memory bit is a 1 (true) this instruction will allow rung continuity through itself, like a closed relay contact.
- If the instruction memory bit is a 0 (false) this instruction will not allow rung continuity through itself and will assume a normally open state just like an open relay contact.

The symbol for the ***Examine If Open (XIO)*** instruction is shown in Figure 5-21. The XIO instruction, which is also called the Examine-off instruction, looks and operates like a normally closed relay contact. Associated with each XIO instruction is a memory bit linked to the status of an input device or an internal logical condition in a rung. This instruction asks the PLC's processor to examine if the contact is *open.* It does this by examining the bit at the memory location specified by the address in the following manner:

- As with any other input the memory bit is set to 1 or 0 depending on the status of the input (physical) device or internal (logical) relay address associated with that bit.
- A 1 corresponds to a true status or on condition.
- A 0 corresponds to a false status or off condition.
- When the Examine-off instruction is used to examine a physical input, then the instruction will be interpreted as false when there is a physical input (voltage) present (the bit is 1) and will be interpreted as true when there is no physical input present (the bit is 0).
- If the Examine-off instruction were associated by address with an internal relay, then the status of the bit would be dependent on the logical status of the internal bit with the same address as the instruction.
- Like the Examine-on instruction, the status of the instruction (true or false) determines if the instruction will allow rung continuity through itself, like a closed relay contact.
- The memory bit always follows the status (true = 1 or false = 0) of the input address or internal address assigned to it. The interpretation of that bit, however, is determined by which instruction is used to examine it.
- Examine-on instructions always interpret a 1 status as true and a 0 status as false, while Examine-off instructions interpret a 1 status as false and a 0 status as true, as illustrated in Figure 5-22.

Symbol	Name	Bit status	Instruction status
	XIC	0	FALSE
		1	TRUE
	XIO	0	TRUE
		1	FALSE

Figure 5-22 Interpreting Examine-on and Examine-off instructions.

The symbol for the ***Output Energize (OTE)*** instruction is shown in Figure 5-23. The OTE instruction looks and operates like a relay coil and is associated with a memory bit. This instruction signals the PLC to energize (switch on) or de-energize (switch off) the output. The processor makes this instruction true (analogous to energizing a coil) when there is a logical path of true XIC and XIO instructions in the rung. The operation of the Output Energize instruction can be summarized as follows:

- The status bit of the addressed Output Energize instruction is set to 1 to energize the output and to 0 to de-energize the output.
- If a true logic path is established with the input instructions in the rung, the OTE instruction is energized and the output device wired to its terminal is energized.
- If a true logic path cannot be established or rung conditions go false, the OTE instruction is de-energized and the output device wired to it is switched off.

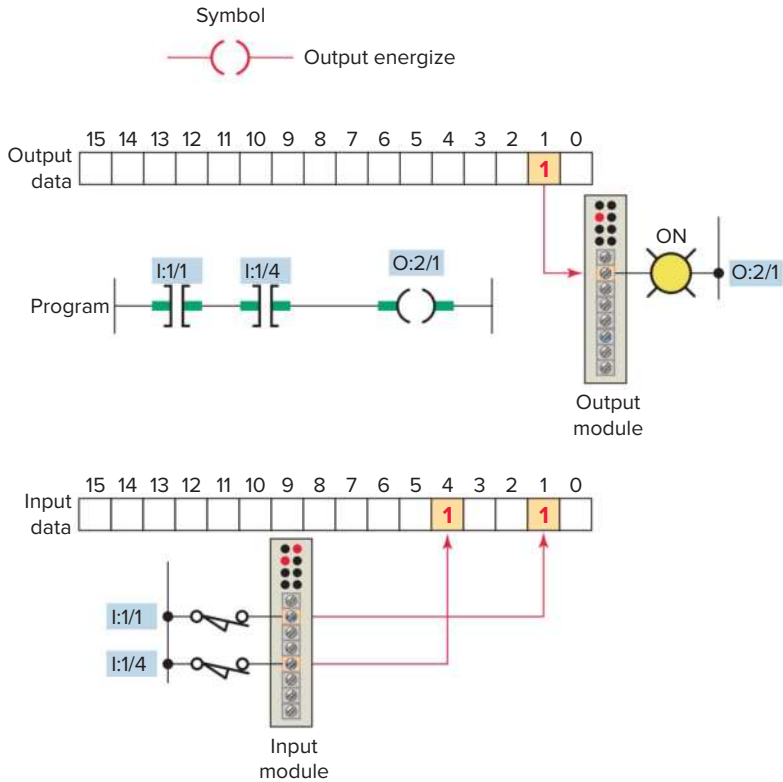


Figure 5-23 Output Energize (OTE) instruction.

Sometimes beginner programmers who are used to thinking in terms of hardwired relay control circuits tend to use the same type of contact (NO or NC) in the ladder logic program that corresponds to the type of field switch wired to the discrete input. While this is true in many instances, it is not the best way to think of the concept. A better approach is to separate the action of the field device from the action of the PLC bits as illustrated in Figure 5-24. A signal present makes the NO bit (1) true; a signal absent makes the NO bit (0) false. The reverse is true for an NC bit. A signal present makes the NC bit (1) false; a signal absent makes the NC bit (0) true.

The main function of the ladder logic diagram program is to control outputs based on input conditions, as illustrated in Figure 5-25. This control is accomplished through the use of what is referred to as a ladder rung. In general, a rung consists of a set of input conditions, represented by contact instructions, and an output instruction at the end of the rung, represented by the coil symbol.

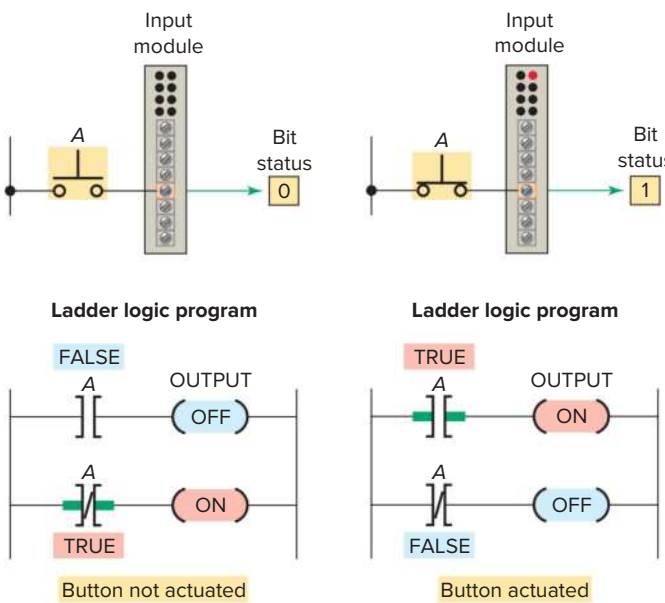


Figure 5-24 Separating the action of the field device and PLC bit.

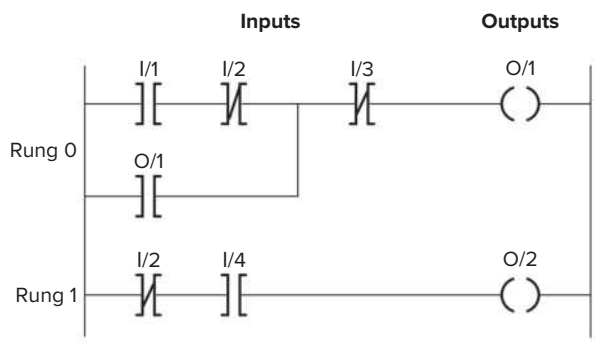


Figure 5-25 Ladder logic diagram rungs.

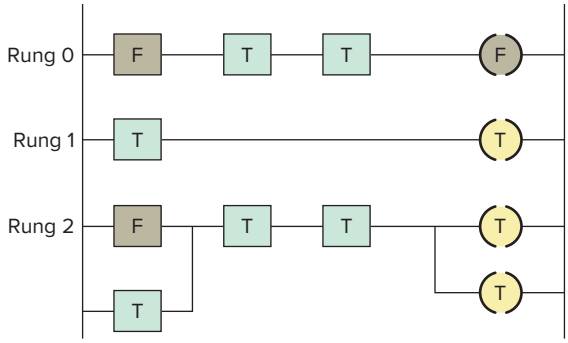


Figure 5-26 Logical continuity.

Each contact or coil symbol is referenced with an address that identifies what is being evaluated and what is being controlled. The same contact instruction can be used throughout the program whenever that condition needs to be evaluated. While this is true for the XIO and XIC contact instructions, the same **cannot** be said for the **OTE coil** instruction. A common mistake for the novice programmer is to place the same addressed OTE instruction on multiple rungs within the same program. This practice is to be avoided since it will lead to unpredictable program outcomes. The number of ladder logic relays and input and output instructions is limited only by memory size. Most PLCs allow more than one output per rung.

For an output to be activated or energized, at least one left-to-right true logical path must exist, as illustrated in Figure 5-26. A complete closed path is referred to as having logical continuity. When logical continuity exists in at least one path, the rung condition and Output Energize instruction are said to be true. The rung condition and OTE instruction are false if no logical continuity path has been established. During controller operation, the processor evaluates the rung logic and changes the state of the outputs according to the logical continuity of rungs.

5.5 Instruction Addressing

To complete the entry of a relay-type instruction, you must assign an **address** to each instruction. This address indicates what PLC input is connected to what input device and what PLC output will drive what output device.

The addressing of real inputs and outputs, as well as internals, depends on the PLC model used. Addressing formats can vary from one PLC family to another as well as for different manufacturers. These addresses can be represented in decimal, octal, or hexadecimal depending on the number system used by the PLC. The address identifies the function of an instruction and links it to a particular bit in the data table portion of the memory. Figure 5-27 shows the addressing format for an Allen-Bradley SLC 500 controller. Addresses contain the slot number of the module where input or output devices are connected. Addresses are formatted as file type, file number, slot number, and bit.

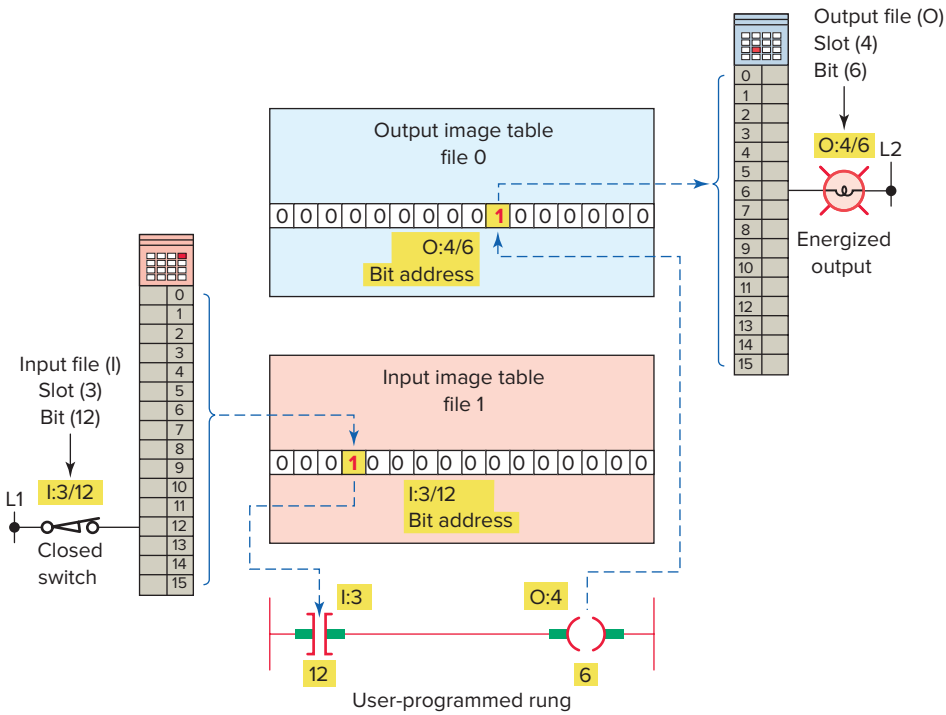


Figure 5-27 Addressing format for an Allen-Bradley SLC 500 controller.

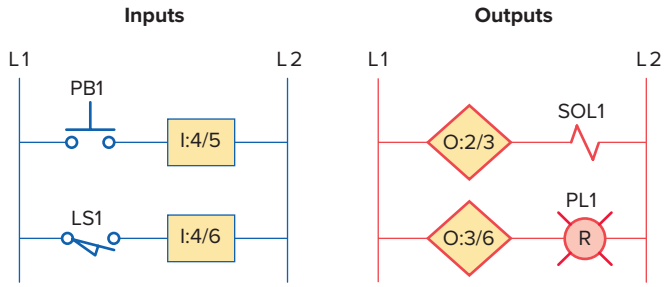


Figure 5-28 I/O connection diagram.

Allen-Bradley Logix 5000 controllers offer a more flexible method of addressing memory space. Instead of a fixed device with a fixed address space, **tags** are used for assigning and referencing memory spaces. Tags are a pure text based addressing scheme and a departure from the more conventional ways of programming PLCs.

The assignment of an I/O address can be included in the I/O connection diagram, as shown in Figure 5-28. Inputs and outputs are typically represented by squares and diamonds, respectively.

5.6 Branch Instructions

Branch instructions are used to create parallel paths of input condition instructions. This allows more than one combination of input conditions (OR logic) to establish logic continuity in a rung. Figure 5-29 illustrates a typical branch instruction. The rung will be true if either instruction *A* or *B* is true.

Input branching by formation of parallel branches can be used in your application program to allow more than one combination of input conditions. If at least one of these parallel branches forms a true logic path, the rung logic is true and the output will be energized. If none of the parallel branches complete a logical path, logic rung continuity is not established and the output will be de-energized. In the example shown in Figure 5-30, either *A* and *B,* or *C* provides logical continuity and energizes output *D.*

On most PLC models, branches can be established at both input and output portions of a rung. With **output branching,** you can program parallel outputs on a rung to allow a true logic path to control multiple outputs, as

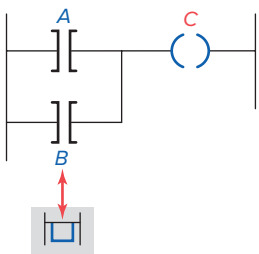


Figure 5-29 Typical branch instruction.

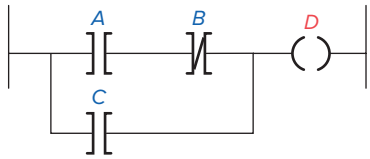


Figure 5-30 Parallel input branches.

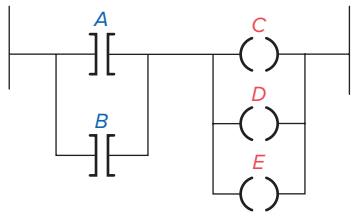


Figure 5-31 Parallel output branches.

illustrated in Figure 5-31. When there is a true logic rung path, all parallel outputs become true. In the example shown, either *A* or *B* provides a true logical path to all three output instructions: *C, D,* and *E.*

Additional input logic instructions (conditions) can be programmed in the output branches to enhance conditional control of the outputs. When there is a true logic path, including extra input conditions on an output branch, that branch becomes true. In the example shown in Figure 5-32, either *A* and *D* or *B* and *D* provide a true logic path to *E.*

Input and output branches can be ***nested*** to avoid redundant instructions and to speed up processor scan time. Figure 5-33 illustrates nested input and output branches. A nested branch starts or ends within another branch.

In some PLC models, the programming of a branch circuit within a branch circuit or a *nested* branch cannot be done directly. It is possible, however, to program

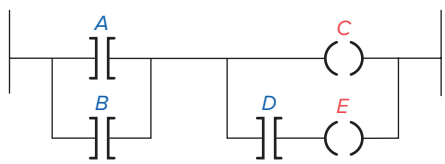


Figure 5-32 Parallel output branching with conditions.

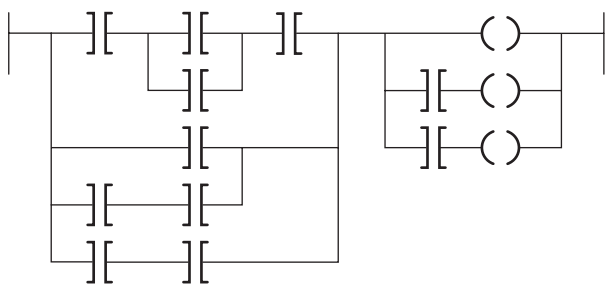


Figure 5-33 Nested input and output branches.

a logically equivalent branching condition. Figure 5-34 shows an example of a circuit that contains a nested contact *D*. To obtain the required logic, the circuit would be programmed as shown in Figure 5-35. The duplication of contact *C* eliminates the nested contact *D*. Nested branching can be converted into non-nested branches by repeating instructions to make parallel equivalents.

Some PLC manufacturers have virtually no limitations on allowable series elements, parallel branches, or outputs. For others, there may be limitations to the number of series contact instructions that can be included in one rung of a ladder diagram as well as limitations to the number of parallel branches. Also, there is an additional limitation with some PLCs: only one output per rung and the output must be located at the end of the rung. The only limitation on the number of rungs is memory size. Figure 5-36 shows the matrix limitation diagram for a typical PLC. A maximum of seven parallel lines and 10 series contacts per rung is possible.

Another limitation to branch circuit programming is that the PLC will not allow for programming of vertical contacts. A typical example of this limitation is contact *C* of the user program drawn in Figure 5-37. To obtain the required logic, the circuit would be reprogrammed as shown in Figure 5-38.

The processor examines the ladder logic rung for logic continuity from left to right *only*. The processor never allows for flow from right to left. This situation presents a

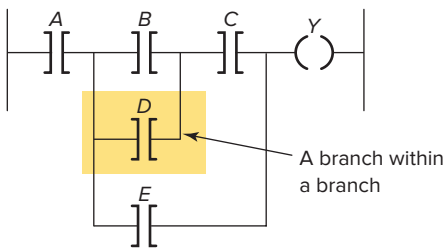


Figure 5-34 Nested contact program.

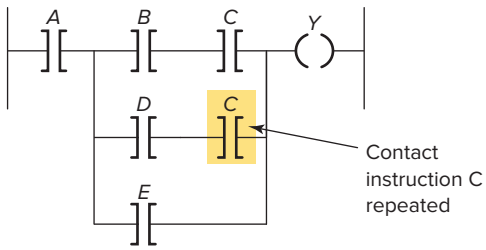
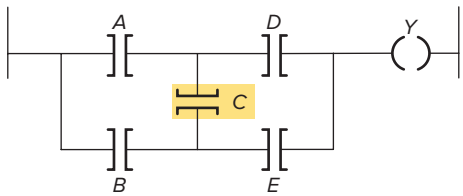


Figure 5-35 Program required to eliminate nested contact.



Boolean equation: $Y = (AD) + (BCD) + (BE) + (ACE)$

Figure 5-37 Program with vertical contact.

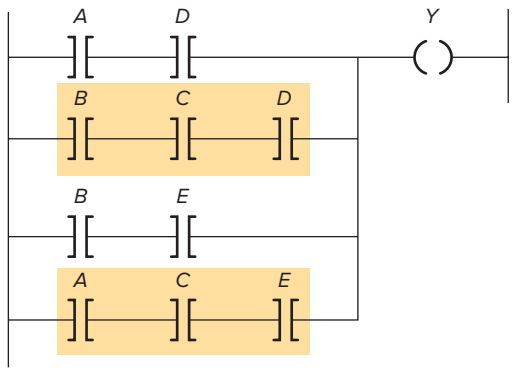


Figure 5-38 Reprogrammed to eliminate vertical contact.

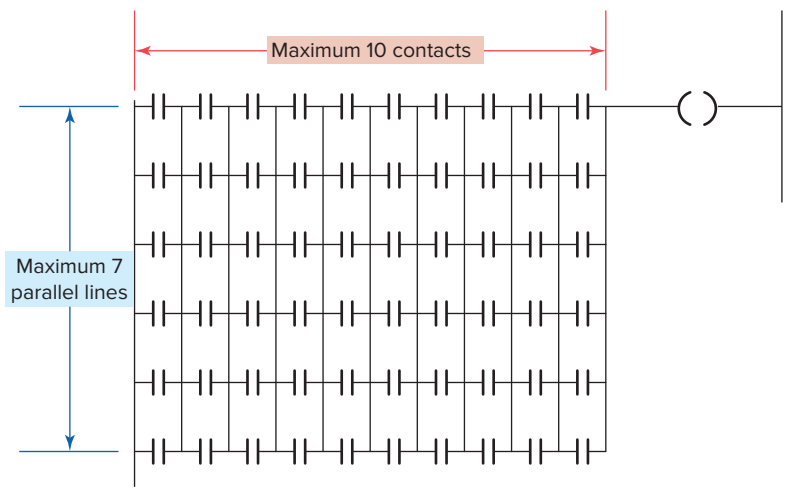
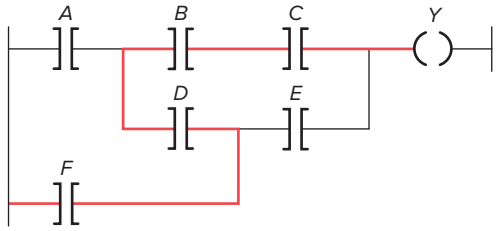


Figure 5-36 PLC matrix limitation diagram.



Boolean equation: $Y = (ABC) + (ADE) + (FE) + (FDBC)$

Figure 5-39 Original circuit.

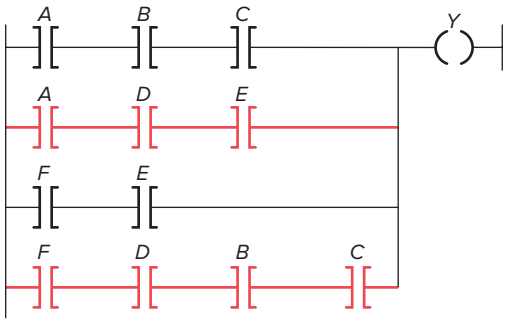


Figure 5-40 Reprogrammed circuit.

problem for user program circuits similar to that shown in Figure 5-39. If programmed as shown, contact combination *FDBC* would be ignored. To obtain the required logic, the circuit would be reprogrammed as shown in Figure 5-40.

5.7 Internal Relay Instructions

Most PLCs have an area of the memory allocated for what are known as internal storage bits. These storage bits are also called ***internal outputs, internal coils, internal control relays,*** or simply ***internal bits.*** Internal outputs are on/off signals generated by programmed logic. Unlike a discrete output, an internal output does not directly control an output field device. The internal output operates just like any output that is controlled by programmed logic; however, the output is used strictly for internal purposes.

The advantage of using internal outputs is that there are many situations in which an output instruction is required in a program but no physical connection to a field device is needed. If there are no physical outputs wired to a bit address, the address can be used as an internal storage point. Internal storage bits or points can be programmed by the user to perform relay functions without occupying a physical output. In this way internal outputs can minimize output module point requirements whenever practical.

Internal outputs are single-bit storage locations in memory and are addressed as such. SLC 500 controllers use bit file B3 for storage and addressing of internal output bits. The addressing for bit B3:1/3 illustrated in Figure 5-41 consists of the file number followed by word and bit numbers.

An internal control relay can be used when a program requires more series contacts than the rung allows. Figure 5-42 shows a circuit that allows for only 7 series contacts when 12 are actually required for the programmed logic. To solve this problem, the contacts are split into two rungs. Rung 1 contains seven of the required contacts and is programmed to control internal relay coil B3:1/3. The address of the first programmed contact on Rung 2 is B3:1/3 followed by the remaining five contacts and the discrete output. When the logic controlling the internal output is true, the referenced bit B3:1/3 is turned on or set to 1. The advantage of an internal storage bit in this manner is that it saves an output bit from being used.

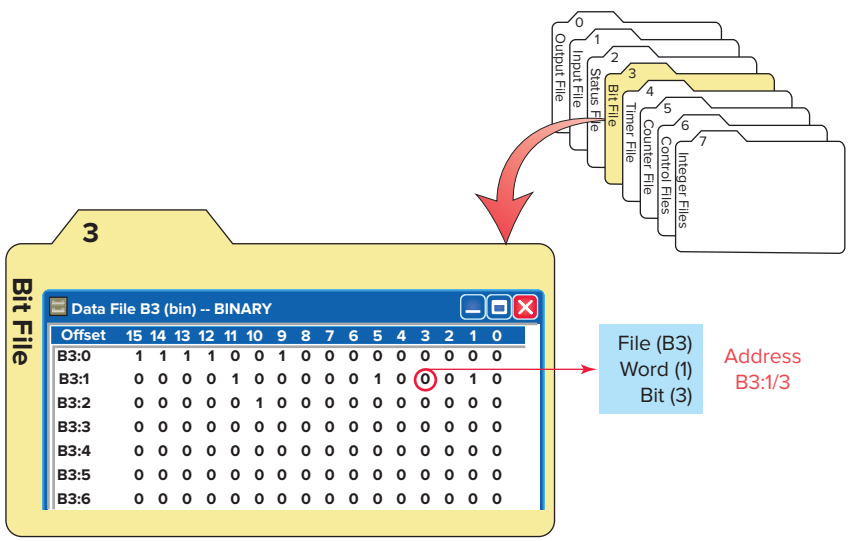


Figure 5-41 SLC 500 controllers use bit file B3 for internal bit addressing.

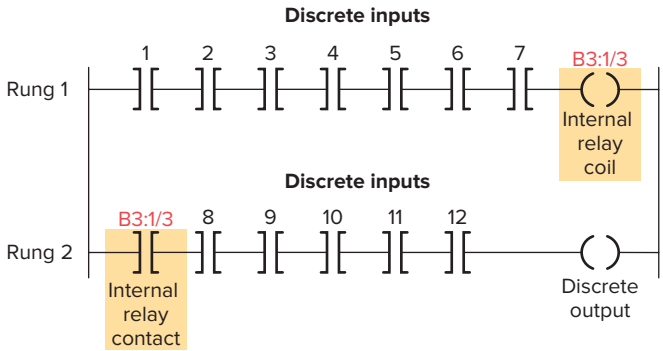


Figure 5-42 Programmed internal relay control.

5.8 Programming Examine If Closed and Examine If Open Instructions

A simple program using the Examine If Closed (XIC) instruction is shown in Figure 5-43. This figure shows a hardwired circuit and a user program that provides the same results. You will note that *both the NO and the NC* pushbuttons are represented by the Examine If Closed symbol. This is because the normal state of an input (NO or NC) does not matter to the controller. What does matter is that if contacts need to close to energize the output, then the Examine If Closed instruction is used. Since both PB1 and PB2 must be closed to energize the pilot light, the Examine If Closed instruction is used for both.

A simple program using the Examine If Open (XIO) instruction is shown in Figure 5-44. Both the hardwired circuit and user program are shown. In the hardwired circuit, when the pushbutton is *open* relay coil CR is de-energized and its NO contact closes to switch the pilot light on. When the pushbutton is *closed,* relay coil CR is energized and its NC contact opens to switch the pilot light off. The pushbutton is represented in the user program by an Examine If Open instruction. This is because

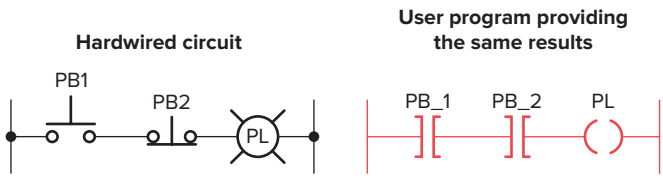


Figure 5-43 Simple program that uses the Examine If Closed (XIC) instruction.

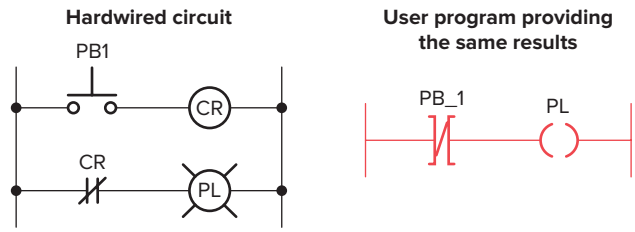


Figure 5-44 Simple program that uses the Examine If Open (XIO) instruction.

If the data table bit is	The status of the instruction is		
	XIC EXAMINE IF CLOSED	XIO EXAMINE IF OPEN	OTE OUTPUT ENERGIZE
Logic 0	False	True	False
Logic 1	True	False	False

Time	Instruction outcome		
	XIC	XIO	OTE
t_1 (initial)	False	True	False
t_2	True	True	Goes true
t_3	True	False	Goes false
t_4	False	False	Remains false

Input bit status		
XIC	XIO	OTE
0	0	0
1	0	1
1	1	0
0	1	0

Figure 5-45 Simple program using both the XIC and XIO instructions.

the rung must be true when the external pushbutton is open and false when the pushbutton is closed. Using an Examine If Open instruction to represent the pushbutton satisfies these requirements. The NO or NC mechanical action of the pushbutton is not a consideration. It is important to remember that the user program is not an electrical circuit but a *logic* circuit. In effect, we are interested in logic continuity when establishing an output.

Figure 5-45 shows a simple program using both the XIC and XIO instructions. The logic states (0 or 1) indicate whether an instruction is true or false and is the basis of controller operation. The figure summarizes the on/off state of the output as determined by the changing states of the inputs in the rung. The time aspect relates to the repeated scans of the program, wherein the input table is updated with the most current status bits.

5.9 Entering the Ladder Diagram

Most of today's PLC programming packages operate in the **Windows environment**. For example, Allen-Bradley's RSLogix software packages are Windows programming packages used to develop ladder logic programs. This software, in various versions, can be used to program the SLC 500, ControlLogix, and MicroLogic family of processors.

Entering the ladder diagram, or actual programming, is usually accomplished with a computer keyboard or hand-held programming device. Because hardware and programming techniques vary with each manufacturer, it is necessary to refer to the programming manual for a specific PLC to determine how the instructions are entered.

One method of entering a program is through a hand-held keyboard. Keyboards usually have relay symbol and special function keys along with numeric keys for addressing. Some also have alphanumeric keys (letters and numbers) for other special programming functions. In hand-held units, the keyboard is small and the keys have multiple functions. Multiple-function keys work like second-function keys on calculators.

A personal computer is most often used today as the programmer. The computer is adapted to the particular PLC model through the use of the relevant programmable controller software.

Figure 5-46 shows the RSLogix SLC 500 main window. Different screens, toolbars, and dialog boxes are used to navigate through the Windows environment. It is important that you understand the purpose of the various screens, toolbars, and windows to make the most effective use of the software. This information is available from the software reference manual for the particular PLC family and will become more familiar to you as you develop programs using the software.

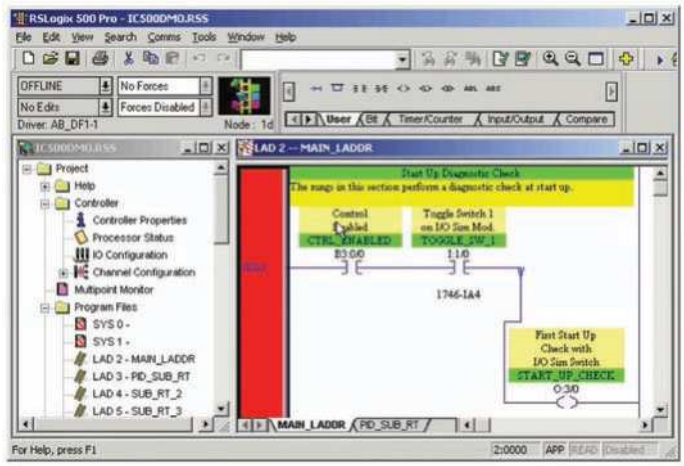


Figure 5-46 RSLogix SLC 500 main window.
Source: Image Courtesy of Rockwell Automation, Inc.

Figure 5-47 Typical instruction toolbar with bit instructions selected.

Figure 5-47 shows a typical instruction toolbar with bit instructions selected. To place an instruction on a rung, click its icon on the toolbar and simply drag the instruction straight off the toolbar onto the rung of the ladder. Drop points are shown on the ladder to help position the instruction. In addition, instructions can also be dragged from other rungs in the project. There are several different methods that you can use to address instructions. You can enter an address by manually typing it in or by dragging the address from data files or other instructions.

Some of the windows you will need to use when working with RSLogix 500 software include:

- **Main Window**—This window opens each time you create a new project or open an existing one. Some of the features associated with this window include the following:
  - Window Title Bar—The title bar is located at the topmost strip of the window and displays the name of the program as well as that of the opened file.
  - Menu Bar—The menu bar is located below the title bar. The menu contains key words associated with menus that are opened by clicking on the key word.
  - Windows Toolbar—The Windows toolbar buttons execute standard Windows commands when you click on them.
  - Program/Processor Status Toolbar—This toolbar contains four drop-down lists that identify the current processor operating mode, current online

edit status, and whether forces are present and enabled.

- Project Window—This window displays the file folders listed in the project tree.
- Project Tree—The project tree is a visual representation of all folders and their associated files contained in the current project. From the project tree, you can open files, create files, modify file parameters, copy files, hide or unhide files, delete files, and rename files.
- Result Window—This window displays the results of either a search or a verify operation. The verify operation is used to check the ladder program for errors.
- Active Tab—This tab identifies which program is currently active.
- Status Bar—This bar contains information relevant to the current file.
- Split Bar—The split bar is used to split the ladder window to display two different program files or groups of ladder rungs.
- Tabbed Instruction Toolbar—This toolbar displays the instruction set as a group of tabbed categories.
- Instruction Palette—This tool contains all the available instructions displayed in one table to make the selection of instructions easier.
- Ladder Window—This window displays the currently open ladder program file and is used to develop and edit ladder programs.
- Ladder Window Properties—This window allows you to change the display of your ladder program and its associated addressing and documentation.

- **Select Processor Type**—The programming software needs to know what processor is being used in conjunction with the user program. The Select Processor Type screen (Figure 5-48) contains a list of the different processors that the RSLogix software can program. You simply scroll down the list until you find the processor you are using and select it.

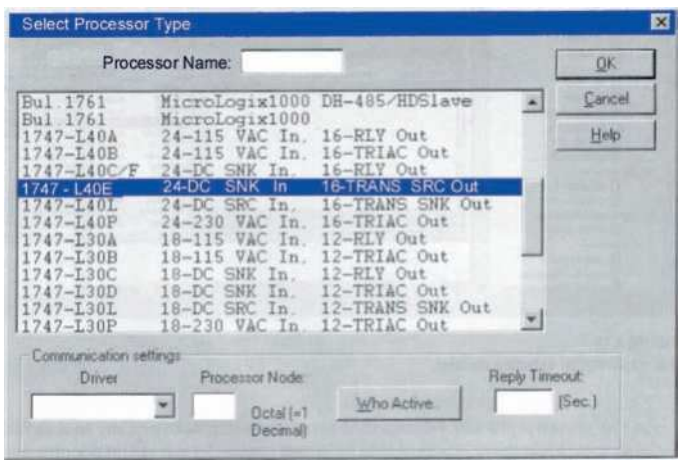


Figure 5-48 Select processor type screen.
Source: Image Courtesy of Rockwell Automation, Inc.

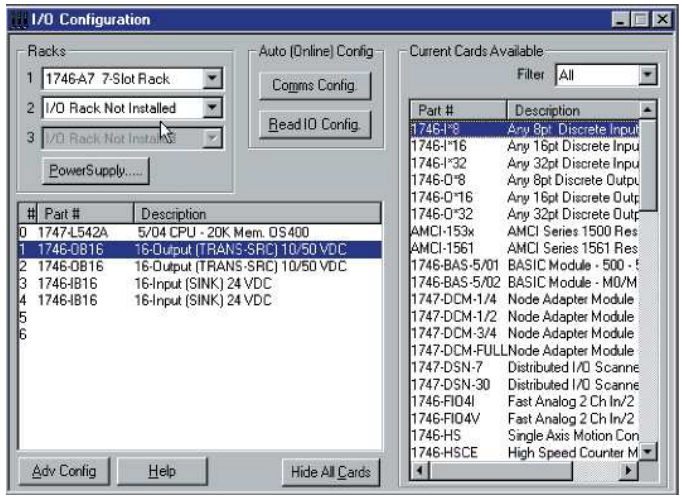


Figure 5-49 I/O configuration screen.
Source: Image Courtesy of Rockwell Automation, Inc.

- **I/O Configuration**—The I/O Configuration screen (Figure 5-49) lets you click or drag-and-drop a module from an all-inclusive list to assign it to a slot in your configuration.
- **Data Files**—Data File screens contain data that are used in conjunction with ladder program instructions and include input and output files as well as timer, counter, integer, and bit files. Figure 5-50 shows an example of the bit file B3, which is used for internal relays. Note that all the addresses from this file start with B3.

Relay ladder logic is a graphical programming language designed to closely represent the appearance of a wired relay system. It offers considerable advantages

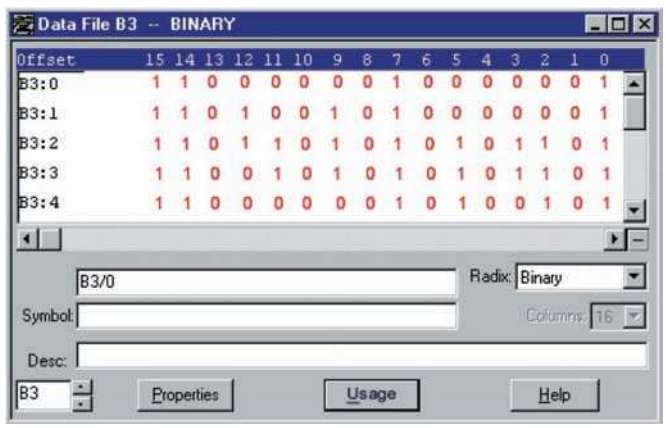


Figure 5-50 Data bit file B3 screen.
Source: Courtesy of TheLearningPit.

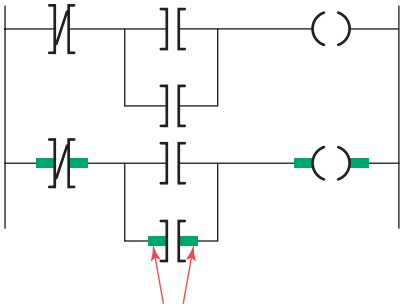


Figure 5-51 Monitoring a ladder logic program.

for PLC control. Not only is it reasonably intuitive, especially for users with relay experience, but it is also particularly effective in an online mode when the PLC is actually performing control. Operation of the logic is apparent from the highlighting of rungs of the various instructions on-screen, which identifies the logic state of contacts in real time (Figure 5-51) and which rungs have logic continuity.

For most PLC systems, each Examine If Closed and Examine If Open contact, each output, and each branch Start/End instruction requires one word of user memory. You can refer to the SLC 500 Controller Properties to see the number of instruction words used and the number left as the program is being developed.

5.10 Modes of Operation

A processor has basically two modes of operation: the *program mode* and some variation of the *run mode.* The number of different operating modes and the method of accessing them varies with the manufacturer. Figure 5-52 shows a typical three-position keyswitch used to select different processor modes of operation.

Some common operating modes are explained in the following paragraphs.

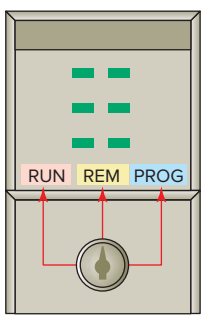


Figure 5-52 Three-position keyswitch used to select different processor modes of operation.

Program Mode The program mode is used to enter a new program, edit or update an existing program, upload files, download files, document (print out) programs, or change any software configuration file in the program. When the PLC is switched into the program mode, all outputs from the PLC are forced off regardless of their rung logic status, and the ladder I/O scan sequence is halted.

Run Mode The run mode is used to execute the user program. Input devices are monitored and output devices are energized accordingly. After all instructions have been entered in a new program or all changes made to an existing program, the processor is put in the run mode.

Test Mode The test mode is used to operate or monitor the user program without energizing any outputs. The processor still reads inputs, executes the ladder program, and updates the output status table files, but without energizing the output circuits. This feature is often used after developing or editing a program to test the program execution before allowing the PLC to operate real-world outputs. Variations of the test mode can include the *single-step test mode,* which directs the processor to execute a selected single rung or group of rungs; the *single-scan test mode,* which executes a single processor operating scan or cycle; and the *continuous-scan test mode,* which directs the processor to continuously run the program for checking or troubleshooting.

Remote Mode Some processors have a three-position switch to change the processor operating mode. In the Run position, all logic is solved and the I/O is enabled. In the Program position, all logic solving is stopped and the I/O is disabled. The Remote position allows the PLC to be remotely changed between program and run mode by a personal computer connected to the PLC processor. The remote mode may be beneficial when the controller is in a location that is not easily accessible.

5.11 Connecting with Analog Devices

Electrical devices and signals can be divided into two categories: analog and digital. **Digital** devices operate using discrete ON or OFF signals that have only two possible values. **Analog** signals can take any shape and represent an infinite number of possible values, as illustrated in Figure 5-53. Analog circuits are usually much more susceptible to noise (small, undesired variations in voltage). Small changes in the voltage level of an analog signal

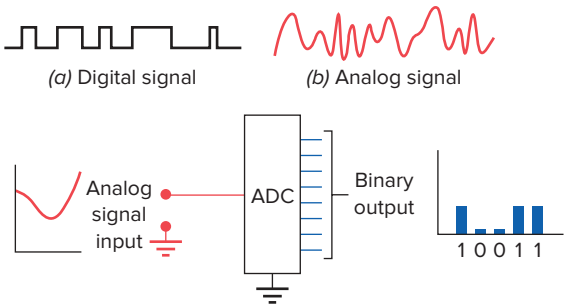


Figure 5-53 Digital and analog signals.

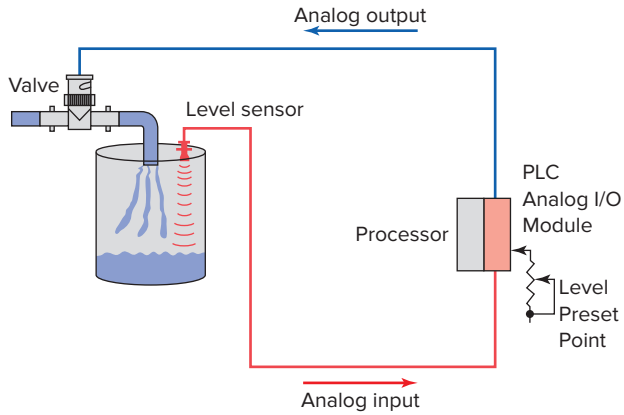


Figure 5-54 Analog control for a tank-filling process.

may produce significant errors as the signal is processed. Analog signals must be coded into digital signals before they can be processed by the PLC. An analog-to-digital converter (ADC) converts analog input signals to digital signals. A digital-to-analog converter (DAC) converts digital output signals to analog signals.

Analog applications are present in many forms. Figure 5-54 shows a typical use of analog control for a tank-filling process. The operation of the circuit can be summarized as follows:

- The processor controls the amount of fluid placed in a holding tank by adjusting the percentage of the valve opening.
- The valve is initially open 100%.
- As the fluid level in the tank approaches the level preset point, the processor modifies the output to degrade, closing the valve to 90%, 80%, etc., adjusting the valve to maintain a set point.

CHAPTER 5 REVIEW QUESTIONS

1. What does the memory map for a typical PLC processor consist of?
2. Compare the function of the PLC program and data files.
3. In what manner are data files organized?
4. List eight different types of data files used by an SLC 500 controller.
5. **a.** What information is stored in the input image table file?
   b. In what form is this information stored?
6. **a.** What information is stored in the output image table file?
   b. In what form is this information stored?
7. Outline the sequence of events involved in a PLC scan cycle.
8. List four factors that enter into the length of the scan time.
9. Compare the way horizontal and vertical scan patterns examine input and output instructions.
10. List the five standard PLC languages as defined by the International Standard for Programmable Controllers, and give a brief description of each.
11. Draw the symbol and state the equivalent instruction for each of the following: NO contact, NC contact, and coil.
12. Answer the following with regard to the Examine If Closed instruction:
    a. What is another common name for this instruction?
    b. What is this instruction asking the processor to examine?
    c. Under what condition is the status bit associated with this instruction 0?
    d. Under what condition is the status bit associated with this instruction 1?
    e. Under what condition is this instruction logically true?
    f. What state does this instruction assume when it is false?
13. Answer the following with regard to the Examine If Open instruction:
    a. What is another common name for this instruction?
    b. What is this instruction asking the processor to examine?
    c. Under what condition is the status bit associated with this instruction 0?
    d. Under what condition is the status bit associated with this instruction 1?
    e. Under what condition is this instruction logically true?
    f. What state does this instruction assume when it is false?
14. Answer the following with regard to the Output Energize instruction:
    a. What part of an electromagnetic relay does this instruction look and act like?
    b. What is this instruction asking the processor to do?
    c. Under what condition is the status bit associated with this instruction 0?
    d. Under what condition is the status bit associated with this instruction 1?
15. A normally closed pushbutton is connected to a PLC discrete input. Does this mean it must be represented by a normally closed contact in the ladder logic program? Explain why or why not.
16. Answer the following with regard to a ladder logic rung:
    a. Describe the basic makeup of a ladder logic rung.
    b. How are the contacts and coil of a rung identified?
    c. When is the ladder rung considered as having logic continuity?
17. What does the address assigned to an instruction indicate?
18. When are input branch instructions used as part of a ladder logic program?
19. Identify two matrix limitations that may apply to certain PLCs.
20. In what way does an internal output differ from a discrete output.
21. A normally open limit switch is to be programmed to control a solenoid. What determines whether an Examine-on or Examine-off contact instruction is used?
22. Explain the purpose of Windows based programming software such as RSLogix.
23. Briefly describe each of the following PLC modes of operation:
    a. Program
    b. Test
    c. Run

24. Under what condition is a ladder logic rung said to have logic continuity?

25. Electrical devices and signals can be divided into two categories: analog and digital. What is the major difference between these two categories?

26. Compare the way memory space is assigned and referenced in rack-based and tag-based PLCs.

CHAPTER 5 PROBLEMS

1. Assign each of the following discrete input and output addresses based on the SLC 500 format.
- **a.** Limit switch connected to terminal screw 4 of the module in slot 1 of the chassis.
- **b.** Pressure switch connected to terminal screw 2 of the module in slot 3 of the chassis.
- **c.** Pushbutton connected to terminal screw 0 of the module in slot 6 of the chassis.
- **d.** Pilot light connected to terminal screw 13 of the module in slot 2 of the chassis.
- **e.** Motor starter coil connected to terminal screw 6 of the module in slot 4 of the chassis.
- **f.** Solenoid connected to terminal screw 8 of the module in slot 5 of the chassis.

2. Redraw the program shown in Figure 5-55 corrected to solve the problem of a nested contact.

3. Redraw the program shown in Figure 5-56 corrected to solve the problem of a nested vertical programmed contact.

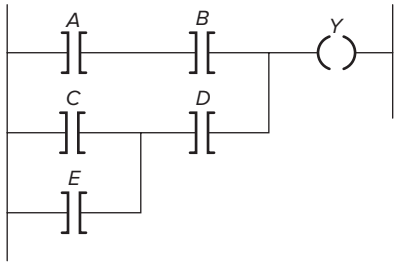


Figure 5-55 Program for Problem 2.

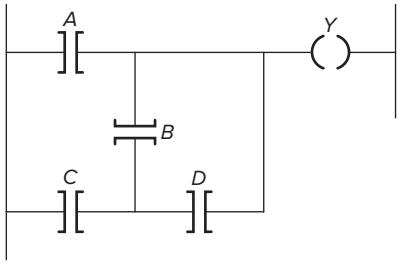


Figure 5-56 Program for Problem 3.

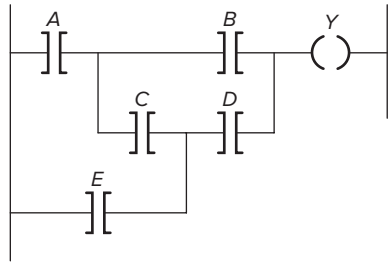


Figure 5-57 Program for Problem 4.

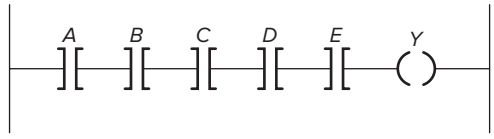


Figure 5-58 Program for Problem 5.

4. Redraw the program shown in Figure 5-57 corrected to solve the problem of some logic ignored.

5. Redraw the program shown in Figure 5-58 corrected to solve the problem of too many series contacts (only four allowed).

6. Draw the equivalent ladder logic program used to implement the hardwired circuit drawn in Figure 5-59, wired using:
- **a.** A limit switch with a single NO contact connected to the PLC discrete input module
- **b.** A limit switch with a single NC contact connected to the PLC discrete input module

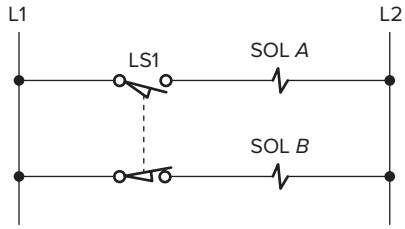


Figure 5-59 Hardwired circuit for Problem 6.

7. Assuming the hardwired circuit drawn in Figure 5-60 is to be implemented using a PLC program, identify
- **a.** All input field devices
- **b.** All output field devices
- **c.** All devices that could be programmed using internal relay instructions

8. What instruction would you select for each of the following discrete input field devices to accomplish the desired task? (State the reason for your answer.)
- **a.** Turn on a light when a conveyor motor is running in reverse. The input field device is a set of contacts on the conveyor start relay that close when the motor is running forward and open when it is running in reverse.
- **b.** When a pushbutton is pressed, it operates a solenoid. The input field device is a normally open pushbutton.
- **c.** Stop a motor from running when a pushbutton is pressed. The input field device is a normally closed pushbutton.
- **d.** When a limit switch is closed, it triggers an instruction ON. The input field device is a limit switch that stores a 1 in a data table bit when closed.

9. Write the ladder logic program needed to implement each of the following (assume inputs *A, B,* and *C* are all normally open toggle switches):
- **a.** When input *A* is closed, turn ON and hold ON outputs *X* and *Y* until *A* opens.
- **b.** When input *A* is closed and either input *B* or *C* is open, turn ON output *Y*; otherwise, it should be OFF.

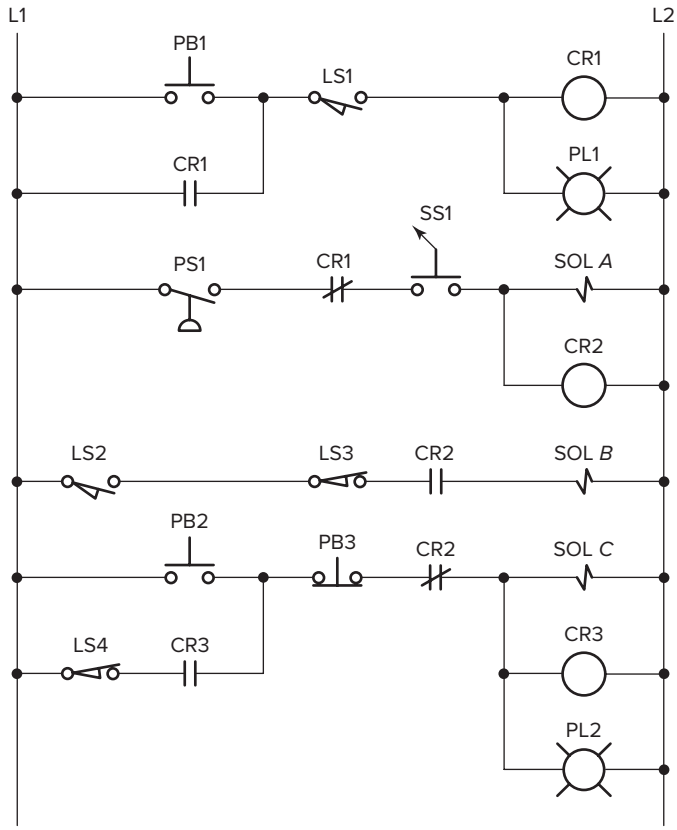
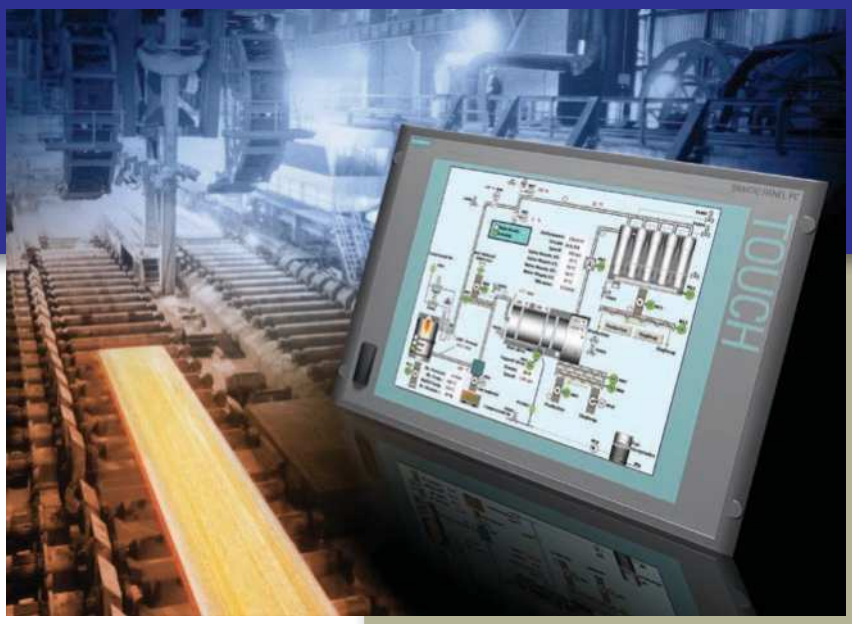


Figure 5-60 Hardwired circuit for Problem 7.

- **c.** When input *A* is closed or open, turn ON output *Y*.
- **d.** When input *A* is closed, turn ON output *X* and turn OFF output *Y*.

6

Developing Fundamental PLC Wiring Diagrams and Ladder Logic Programs



Courtesy of Siemens

For ease of understanding, ladder logic programs can be compared to relay schematics. This chapter gives examples of how traditional relay schematics are converted into PLC ladder logic programs. You will learn more about the wide variety of field devices commonly used in connection with the I/O modules.

Chapter Objectives

After completing this chapter, you will be able to:

- Identify the functions of electromagnetic control relays, contactors, and motor starters
- Identify switches commonly found in PLC installations
- Explain the operation of sensors commonly found in PLC installations
- Explain the operation of output control devices commonly found in PLC installations
- Describe the operation of an electromagnetic latching relay and the PLC-programmed LATCH/UNLATCH instruction
- Compare sequential and combination control processes
- Convert fundamental relay ladder diagrams to PLC ladder logic programs
- Write PLC programs directly from a narrative description

6.1 Electromagnetic Control Relays

The PLC's original purpose was the replacement of **electromagnetic relays** with a solid-state switching system that could be programmed. Although the PLC has replaced much of the relay control logic, electromagnetic relays are still used as auxiliary devices to switch I/O field devices. The programmable controller is designed to replace the physically small control relays that make logic decisions but are not designed to handle heavy current or high voltage (Figure 6-1). In addition, an understanding of electromagnetic relay operation and terminology is important for correctly converting relay schematic diagrams to ladder logic programs.

An electrical relay is a magnetic switch. It uses electromagnetism to switch contacts. A relay will usually have only one coil but may have any number of different contacts. Figure 6-2 illustrates the operation of a typical control relay. With no current flow through the coil (de-energized), the armature is held away from the core of the coil by spring tension. When the coil is energized, it produces an electromagnetic field. Action of this field, in turn, causes the physical movement of the armature. Movement of the armature causes the contact points of the relay to open or close. The coil and contacts are insulated from each other; therefore, under normal conditions, no electric circuit will exist between them.

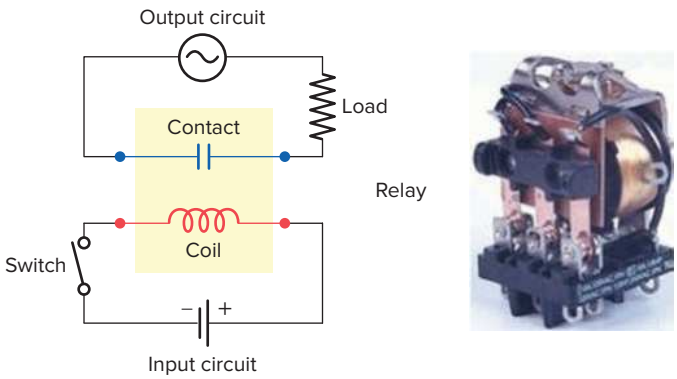


Figure 6-1 Electromechanical control relay.
Source: Courtesy Tyco Electronics Ltd.

The symbol used to represent a control relay is shown in Figure 6-3. The contacts are represented by a pair of short parallel lines and are identified with the coil by means of the letters. The letter M frequently indicates a motor starter, while CR is used for control relays. ***Normally open (NO) contacts*** are defined as those contacts that are open when no current flows through the coil but that *close* as soon as the coil conducts a current or is energized. ***Normally closed (NC) contacts are closed*** when the coil is de-energized and open when the coil is energized. Each contact is usually drawn as it would appear with the coil de-energized.

A typical control relay used to control two pilot lights is shown in Figure 6-4. The operation of the circuit can be summarized as follows:

- With the switch open, coil CR is de-energized.
- The circuit to the green pilot light is completed through the normally closed contact, so this light will be on.
- At the same time, the circuit to the red pilot light is opened through the normally open contact, so this light will be off.

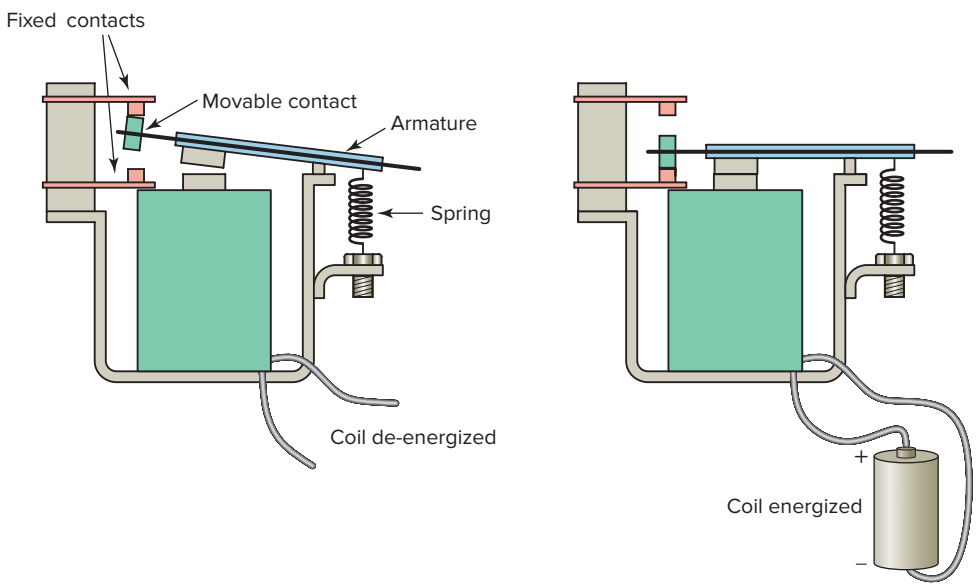


Figure 6-2 Relay operation.

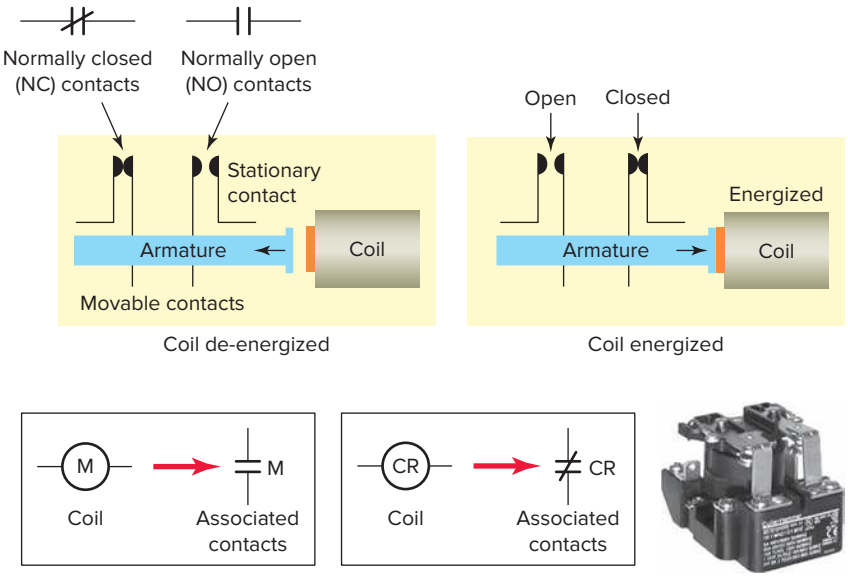


Figure 6-3 Relay normally open and normally closed contacts.
Source: Photo courtesy Eaton Corporation, **www.eaton.com**.

- With the switch closed, the coil is energized.
- The normally open contact closes to switch the red pilot light on.
- At the same time, the normally closed contact opens to switch the green pilot light off.

Single-pole double-throw relay

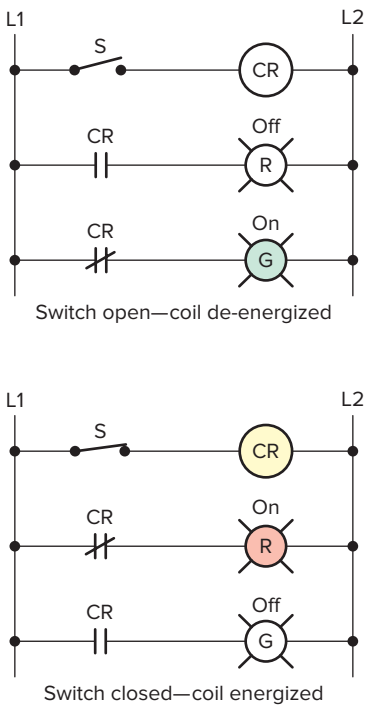


Figure 6-4 Control relay used to control two pilot lights.
Source: Photo courtesy Digi-Key Corporation, **www.digikey.com**.

Control relay coils and contacts have separate ratings. Coils are rated for the type of operating current (DC or AC) and normal operating voltage. Contacts are rated in terms of the maximum amount of current the contacts are capable of handling at a specified voltage level and type (AC or DC). Control relay contacts generally are not designed to carry heavy currents or high voltages. The contacts are usually rated between 5 and 10 Amp, with the most common rating for the coil voltage being 120 VAC.

6.2 Contactors

A **contactor** is a special type of relay designed to handle heavy power loads that are beyond the capability of control relays. Figure 6-5 shows a three-pole magnetic contactor. Unlike relays, contactors are designed to make and break higher powered circuits without being damaged. Such loads include lights, heaters, transformers, capacitors, and electric motors for which overload protection is provided separately or not required.

Programmable controllers normally have an output capacity capable of operating a contactor coil, but not that needed to operate heavy power loads directly. Figure 6-6 illustrates the application of a PLC used in conjunction with a contactor to switch power on and off to a pump. The output module is connected in series with the coil to form a low-current switching circuit. The contacts of the contactor are connected in series with the pump motor to form a high-current switching circuit.

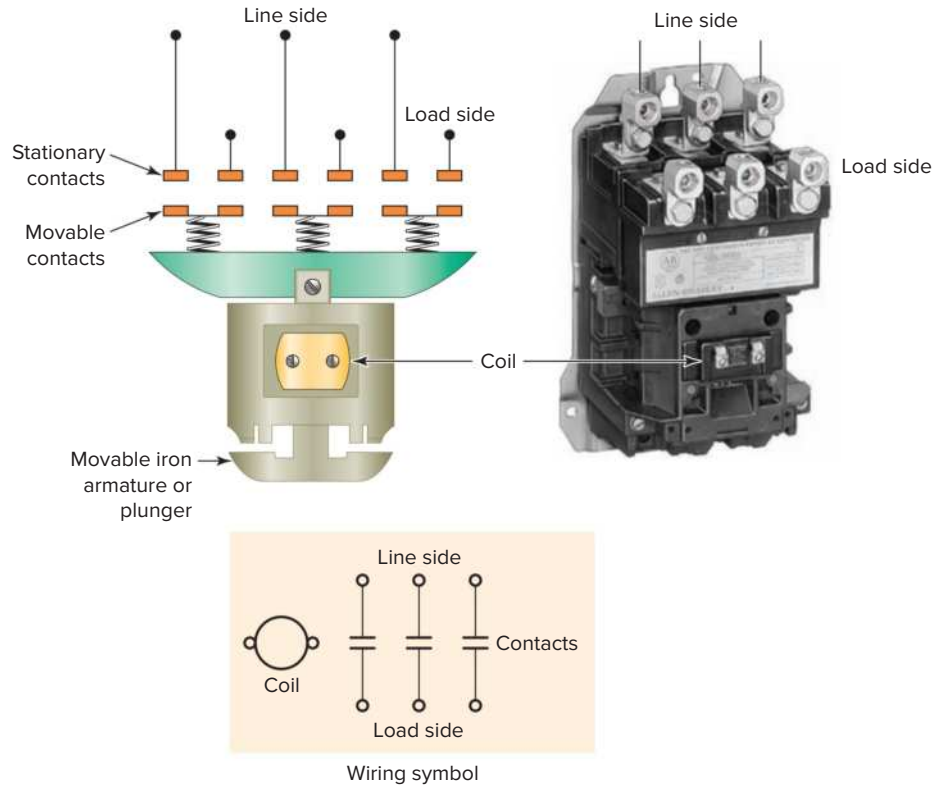


Figure 6-5 Three-pole magnetic contactor.
Source: Image Courtesy of Rockwell Automation, Inc.

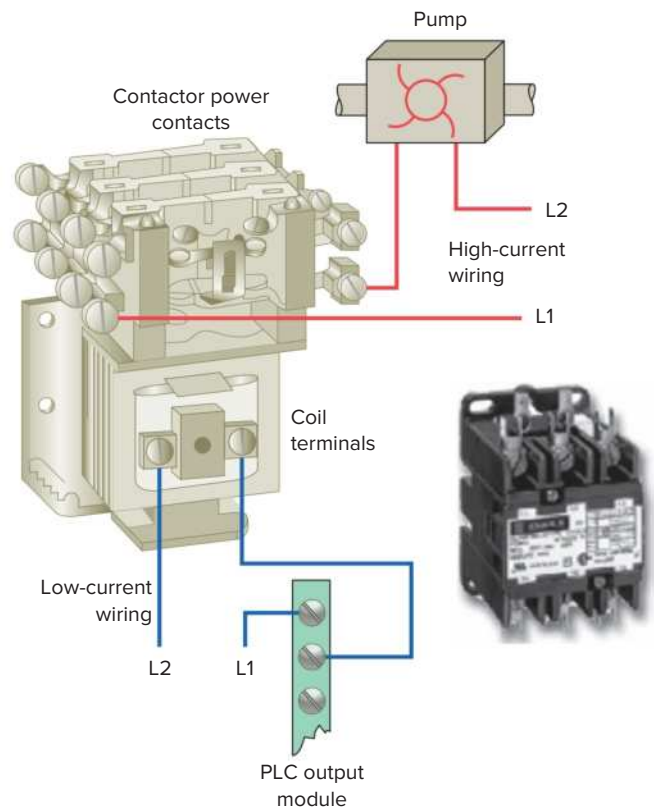


Figure 6-6 Contactor used in conjunction with a PLC output.
Source: This material and associated copyrights are proprietary to, and used with the permission of Schneider Electric.

6.3 Motor Starters

A ***motor starter*** is designed to provide power to motors. The motor starter is made up of a contactor with an ***overload relay*** attached physically and electrically to it as illustrated in Figure 6-7. The function of the overload relay can be summarized as follows:

- Overload relays are designed to meet the special protective needs of motor control circuits.
- They allow harmless temporary overloads that occur when a motor starts.
- The overload relay will trip and disconnect power to the motor if an overload condition persists.
- Overload relays can be reset after the overload condition has been corrected.

Figure 6-8 shows the diagram for a typical three-phase, magnetic motor starter. The operation of the circuit can be summarized as follows:

- When the START button is pressed coil M is energized closing all normally open M contacts.
- The M contacts in series with the motor close to complete the current path to the motor. These contacts are part of the ***power*** circuit and must be designed to handle the full load current of the motor.

Figure 6-7 Motor starter is a contactor with an attached overload relay.
Source: Image Courtesy of Rockwell Automation, Inc.

- Control contact M (across START button) closes to seal in the coil circuit when the START button is released. This contact is part of the ***control*** circuit and, as such, is only required to handle the small amount of current needed to energize the coil.
- An overload (OL) relay is provided to protect the motor against current overloads. The normally closed relay contact OL opens automatically when an overload current is sensed to de-energize the M coil and stop the motor.

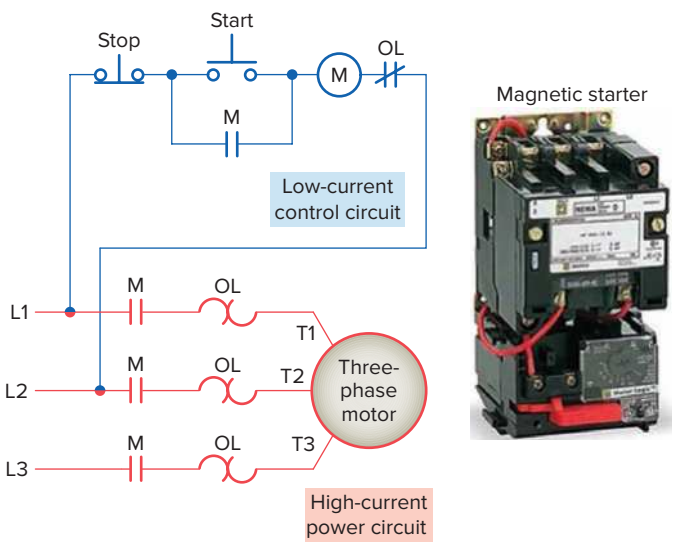


Figure 6-8 Three-phase magnetic motor starter.
Source: This material and associated copyrights are proprietary to, and used with the permission of Schneider Electric.

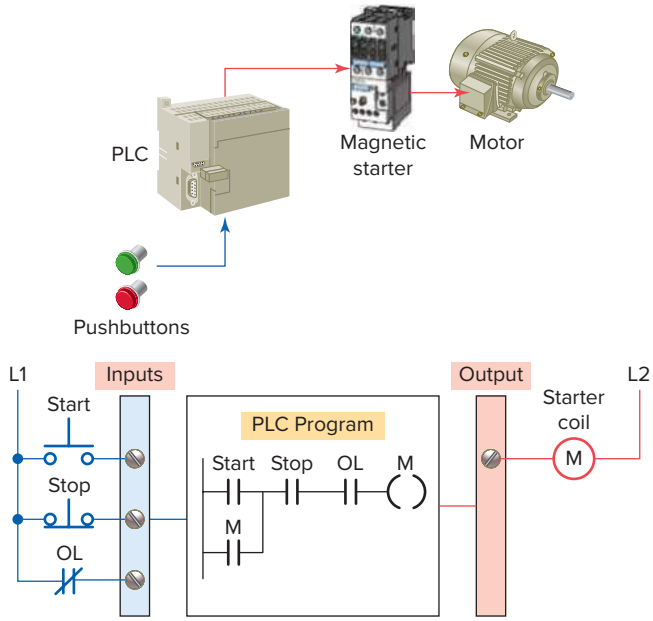


Figure 6-9 PLC control of a motor.

Motor starters are available in various standard National Electric Manufacturers Association (NEMA) sizes and ratings. When a PLC needs to control a large motor, it must work in conjunction with a starter as illustrated in Figure 6-9. The power requirements for the starter coil must be within the power rating of the output module of the PLC. Note that the control logic is determined and executed by the program within the PLC and not by the hardwired arrangement of the input control devices.

6.4 Manually Operated Switches

Manually operated switches are controlled by hand. These include toggle switches, pushbutton switches, knife switches, and selector switches.

Pushbutton switches are the most common form of manual control. A pushbutton operates by opening or closing contacts when pressed. Figure 6-10 shows commonly used types of pushbutton switches, which include:

- *Normally open (NO) pushbutton,* which makes a circuit when it is pressed and returns to its open position when the button is released.
- *Normally closed (NC) pushbutton,* which opens the circuit when it is pressed and returns to the closed position when the button is released.
- *Break-before-make pushbutton* in which the top section contacts are NC and the bottom section contacts are NO. When the button is pressed, the top contacts open before the bottom contacts are closed.

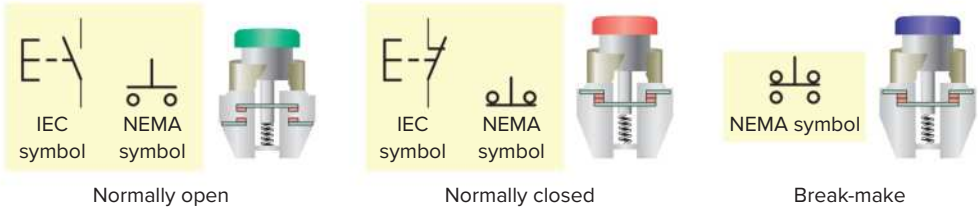
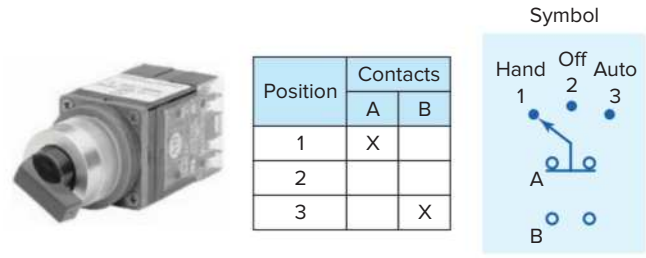


Figure 6-10 Commonly used types of pushbutton switches.



Position	Contacts	
	A	B
1	X	
2		
3		X

Figure 6-11 Three-position selector switch.
Source: Image Courtesy of Rockwell Automation, Inc.

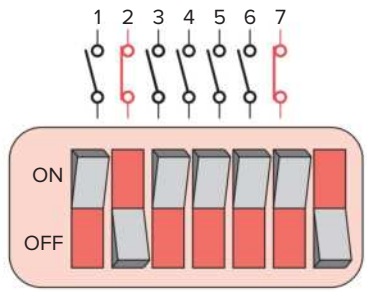


Figure 6-12 DIP switch.

The ***selector switch*** is another common manually operated switch. The main difference between a pushbutton and selector switch is the operator mechanism. A selector switch operator is rotated (instead of pushed) to open and close contacts of the attached contact block. Figure 6-11 shows a three-position selector switch. Switch positions are established by turning the operator knob right or left. Selector switches may have two or more selector positions, with either maintained contact position or spring return to give momentary contact operation.

Dual in-line package (DIP) switches are small switch assemblies designed for mounting on printed circuit board modules (Figure 6-12). The pins or terminals on the bottom of the DIP switch are the same size and spacing as an integrated circuit (IC) chip. The individual switches may be of the toggle, rocker, or slide kind. DIP switches use binary (on/off) settings to set the parameters for a particular module. For example, the input voltage range on a particular input module may be selected by means of DIP switches located on the back of the module.

6.5 Mechanically Operated Switches

A ***mechanically operated switch*** is controlled automatically by factors such as pressure, position, or temperature. The ***limit switch,*** shown in Figure 6-13, is a very common industrial control device. Limit switches are designed to operate only when a predetermined limit is reached, and they are usually actuated by contact with an object such as a cam. These devices take the place of a human operator. They are often used in the control circuits of machine processes to govern the starting, stopping, or reversal of motors.

The ***temperature switch,*** or ***thermostat,*** shown in Figure 6-14 is used to sense temperature changes. Although there are many types available, they are all actuated by some specific environmental temperature change.

Figure 6-13 Mechanically operated limit switch.
Source: Photo courtesy Eaton Corporation.

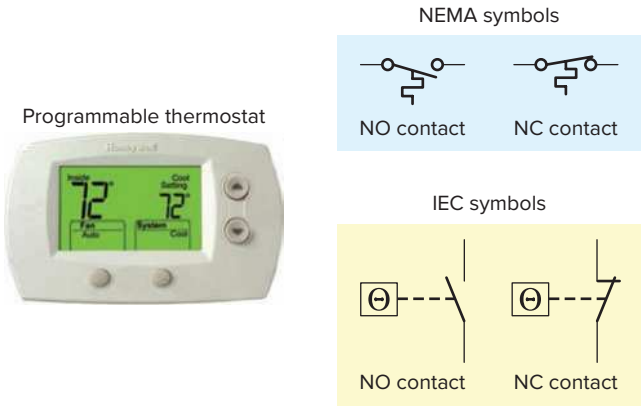


Figure 6-14 Temperature switch.
Source: Photo courtesy Honeywell, **www.honeywell.com**.

Temperature switches open or close when a designated temperature is reached. Industrial applications for these devices include maintaining the desired temperature range of air, gases, liquids, or solids.

Pressure switches, such as that shown in Figure 6-15, are used to control the pressure of liquids and gases. Although many different types are available, they are all basically designed to actuate (open or close) their contacts when a specified pressure is reached. Pressure switches can be pneumatically (air) or hydraulically (liquid) operated switches. Generally, bellows or a diaphragm presses up against a small microswitch and causes it to open or close.

Level switches are used to sense liquid levels in vessels and provide automatic control for motors that transfer liquids from sumps or into tanks. They are also used to open or close piping solenoid valves to control fluids. The float switch shown in Figure 6-16 is a type of level switch. This switch is weighted so that as the liquid rises the switch floats and turns upside down, actuating its internal contacts.

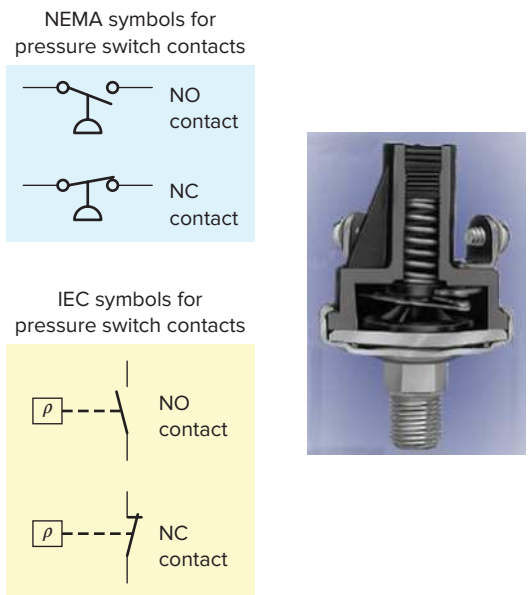


Figure 6-15 Pressure switch.
Source: Photo courtesy Honeywell, **www.honeywell.com**.

Figure 6-16 Float type level switch.
Source: Courtesy Dwyer Instruments.

6.6 Sensors

Sensors are used for detecting, and often measuring, the magnitude of something. They convert mechanical, magnetic, thermal, optical, and chemical variations into electric voltages and currents. Sensors are usually categorized by what they measure, and they play an important role in modern manufacturing process control.

Proximity Sensor

Pilot control devices have limited current handling capacity and are used to control current to a secondary device, such as a contactor coil, which in turn can be used to switch heavier load currents. **Proximity sensors** or switches, such as that shown in Figure 6-17, are pilot devices that detect the presence of an object (usually called the target) *without physical contact.* These solid-state electronic devices are completely encapsulated to protect against excessive vibration, liquids, chemicals, and corrosive agents found in the industrial environment. Proximity sensors are used when:

- The object being detected is too small, lightweight, or soft to operate a mechanical switch.
- Rapid response and high switching rates are required, as in counting or ejection control applications.
- An object has to be sensed through nonmetallic barriers such as glass, plastic, and paper cartons.

Figure 6-17 Proximity sensor.
Source: Photo courtesy Turck, Inc., **www.turck.com**.

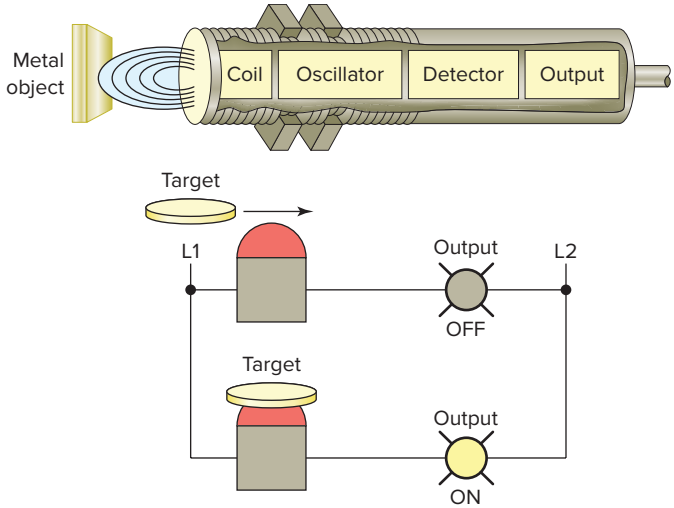


Figure 6-18 Inductive proximity sensor.

- Hostile environments prevent proper operation of mechanical switches and demand improved sealing properties.
- Long life and reliable service are required.
- A fast electronic control system requires a bounce-free input signal.

Proximity sensors operate on different principles, depending on the type of matter being detected. When an application calls for noncontact metallic target sensing, an ***inductive-type proximity sensor*** is used. Inductive proximity sensors are used to detect both ferrous metals (containing iron) and nonferrous metals (such as copper, aluminum, and brass).

Inductive proximity sensors operate under the electrical principle of inductance, where a fluctuating current induces an electromotive force (emf) in a target object. The block diagram for an inductive proximity sensor is shown in Figure 6-18 and its operation can be summarized as follows:

- The oscillator circuit generates a high-frequency electromagnetic field that radiates from the end of the sensor.
- When a metal object enters the field, eddy currents are induced in the surface of the object.
- The eddy currents on the object absorb some of the radiated energy from the sensor, resulting in a loss of energy and change of strength of the oscillator.
- The sensor's detection circuit monitors the oscillator's strength and triggers a solid-state output at a specific level.
- Once the metal object leaves the sensing area, the oscillator returns to its initial value.

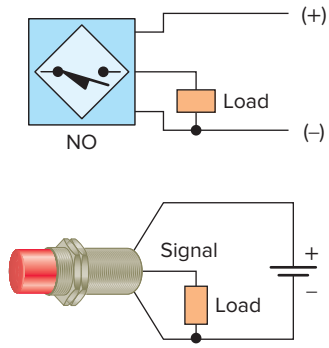


Figure 6-19 Typical three-wire DC sensor connection.

Most sensor applications operate either at 24V DC or at 120V AC. The method of connecting a proximity sensor varies with the type of sensor and its application. Figure 6-19 shows a typical three-wire DC sensor connection. The three-wire DC proximity sensor has the positive and negative line leads connected directly to it. When the sensor is actuated, the circuit will connect the signal wire to the positive side of the line if operating normally open. If operating normally closed, the circuit will disconnect the signal wire from the positive side of the line.

Figure 6-20 shows a typical two-wire proximity sensor connection intended to be connected in series with the load. They are manufactured for either AC or DC supply voltages. In the off state, enough current must flow through the circuit to keep the sensor active. This off state current is called leakage current and typically may range from 1 to 2 mA. When the switch is actuated, it will conduct the normal load circuit current.

Figure 6-21 shows the proximity sensor sensing range. Hysteresis is the distance between the operating point when the target approaches the proximity sensor face and the release point when the target is moving away from the sensor face. The object must be closer to turn the sensor on rather than to turn it off. If the target is moving toward the sensor, it will have to move to a closer point. Once the sensor turns on, it will remain on until the target moves to

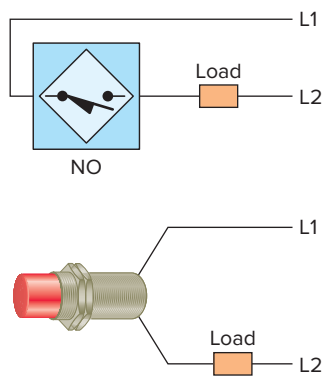


Figure 6-20 Typical two-wire proximity sensor connection.

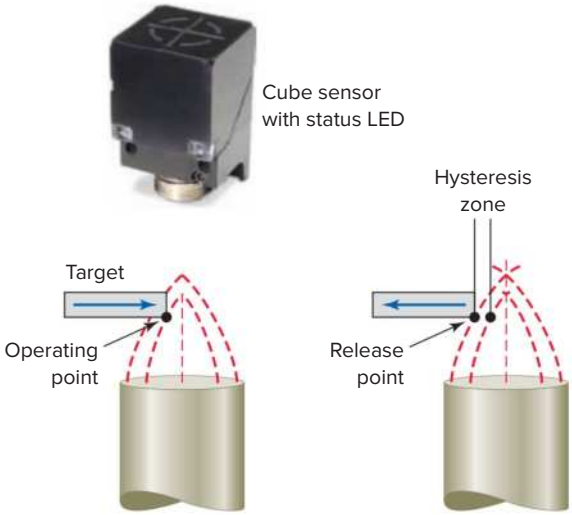


Figure 6-21 Proximity sensor sensing range.
Source: Photo courtesy Eaton Corporation, **www.eaton.com**.

the release point. Hysteresis is needed to keep proximity sensors from chattering when subjected to shock and vibration, slow-moving targets, or minor disturbances such as electrical noise and temperature drift. Most proximity sensors come equipped with an LED status indicator to verify the output switching action.

As a result of solid-state switching of the output, a small leakage current flows through the sensor even when the output is turned off. Similarly, when the sensor is on, a small voltage drop is lost across its output terminals. To operate properly, a proximity sensor should be powered continuously. Figure 6-22 illustrates the use of a bleeder resistor connected to allow enough current for the sensor to operate but not enough to turn on the input of the PLC.

Capacitive proximity sensors are similar to inductive proximity sensors. The main differences between the two types are that capacitive proximity sensors produce an

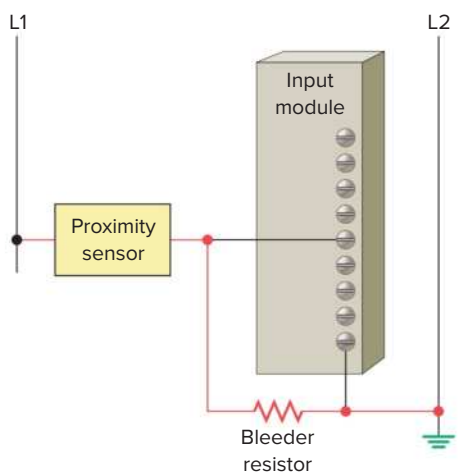


Figure 6-22 Bleeder resistor connected to continuously power a proximity sensor.

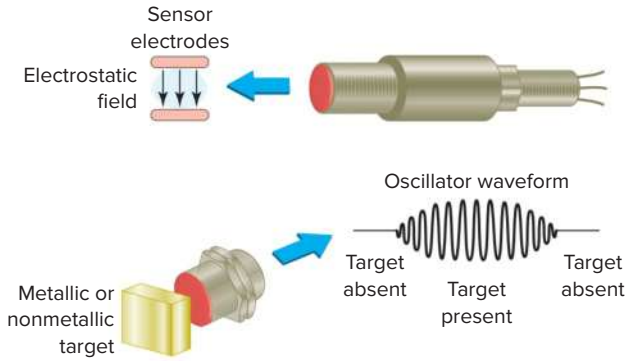


Figure 6-23 Capacitive proximity sensor.

electrostatic field instead of an electromagnetic field and are actuated by both conductive and nonconductive materials.

Figure 6-23 illustrates the operation of a capacitive sensor. A capacitive sensor contains a high-frequency oscillator along with a sensing surface formed by two metal electrodes. When the target nears the sensing surface, it enters the electrostatic field of the electrodes and changes the capacitance of the oscillator. As a result, the oscillator circuit begins oscillating and changes the output state of the sensor when it reaches a certain amplitude. As the target moves away from the sensor, the oscillator's amplitude decreases, switching the sensor back to its original state.

Capacitive proximity sensors will sense metal objects as well as nonmetallic materials such as paper, glass, liquids, and cloth. They typically have a short sensing range of about 1 inch, regardless of the type of material being sensed. The larger the dielectric constant of a target, the easier it is for the capacitive sensor to detect. This makes possible the detection of materials inside nonmetallic containers as illustrated in Figure 6-24. In this example, the liquid has a much higher dielectric constant than the cardboard container, which gives the sensor the ability to see through the container and detect the liquid. In the process shown, detected empty containers are automatically diverted via the push rod.

Inductive proximity switches may be actuated only by a metal and are insensitive to humidity, dust, dirt, and the like. Capacitive proximity switches, however, can be actuated

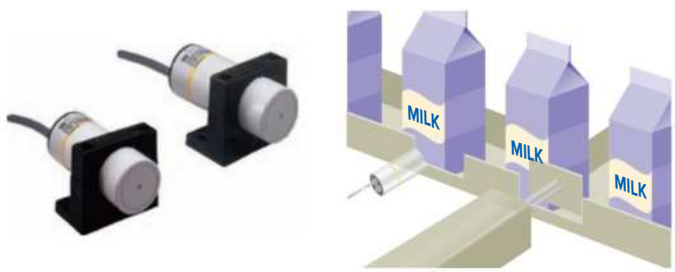


Figure 6-24 Capacitive proximity sensor liquid detection.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

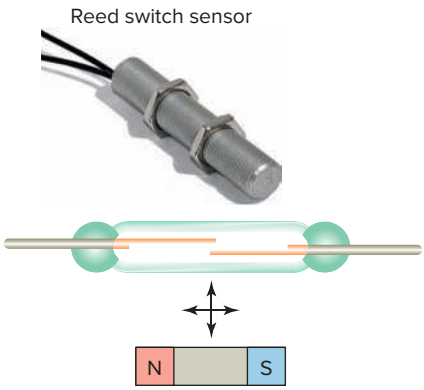


Figure 6-25 Magnetic reed switch.
Source: Courtesy of Reed Switch Developments Corp., used with permission.

by any dirt in their environment. For general applications, the capacitive proximity switches are not really an alternative but a supplement to the inductive proximity switches. They are a supplement when there is no metal available for the actuation (e.g., for woodworking machines and for determining the exact level of liquids or powders).

Magnetic Reed Switch

A ***magnetic reed switch*** is composed of two flat contact tabs that are hermetically sealed (airtight) in a glass tube filled with protective gas, as illustrated in Figure 6-25. When a magnetic force is generated parallel to the reed switch, the reeds become flux carriers in the magnetic circuit. The overlapping ends of the reeds become opposite magnetic poles, which attract each other. If the magnetic force between the poles is strong enough to overcome the restoring force of the reeds, the reeds will be drawn together to actuate the switch. Because the contacts are sealed, they are unaffected by dust, humidity, and fumes; thus, their life expectancy is quite high.

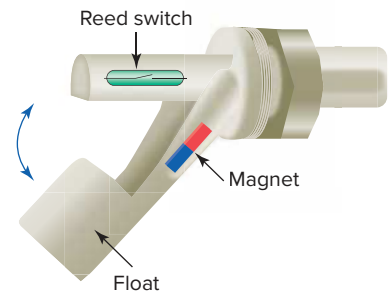


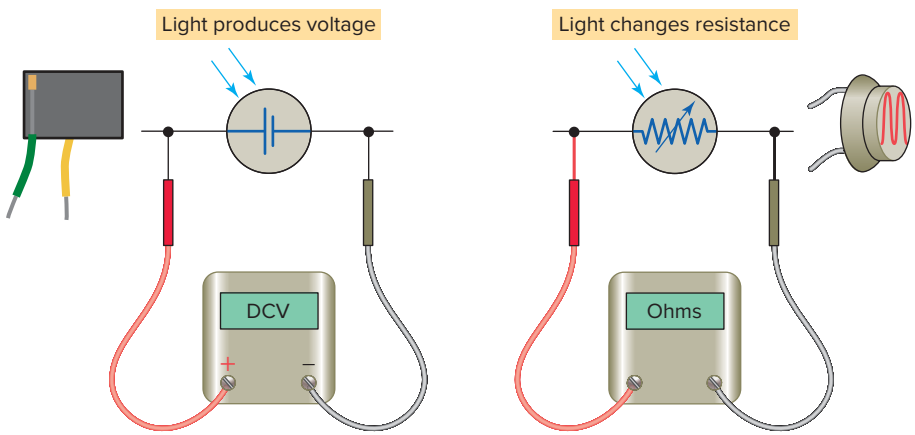
Figure 6-26 Magnetic reed float switch.

One practical application for a magnetic reed switch is its use in a float switch, illustrated in Figure 6-26. The reed switch opens or closes a circuit as the level of a liquid rises or falls. The switch assembly is made up of a permanent magnet installed within the movable float arm and a magnetic reed switch installed within the fixed housing. The movement of the float, due to the changing liquid level, will cause the reed switch to open or close a circuit at a particular level.

Light Sensors

The photovoltaic cell and the photoconductive cell, illustrated in Figure 6-27, are two examples of light sensors. The ***photovoltaic* or *solar cell*** reacts to light by converting the light energy directly into electric energy. The ***photoconductive cell*** (also called a *photoresistive cell*) reacts to light by change in the resistance of the cell.

A ***photoelectric sensor*** is an optical control device that operates by detecting a visible or invisible beam of light and responding to a change in the received light intensity. Photoelectric sensors are composed of two basic components: a transmitter (light source) and a receiver (sensor), as shown in Figure 6-28. These two components may or may not be housed in separate units. The



(*a*) Photovoltaic solar cell (*b*) Photoconductive cell

Figure 6-27 Photovoltaic and photoconductive light cells.

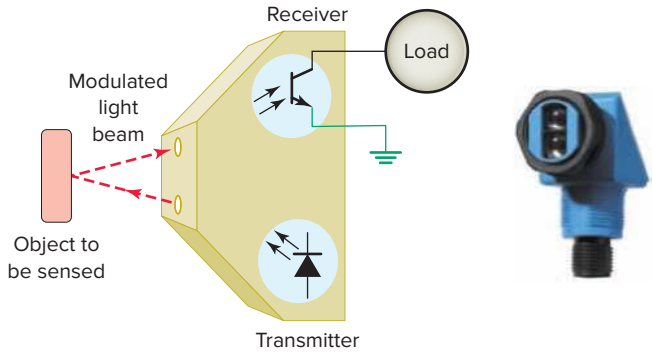


Figure 6-28 Photoelectric sensor.
Source: Photo courtesy SICK, Inc., **www.sick.com**.

basic operation of a photoelectric sensor can be summarized as follows:

- The transmitter contains a light source, usually an LED along with an oscillator.
- The oscillator modulates or turns the LED on and off at a high rate of speed.
- The transmitter sends this modulated light beam to the receiver.
- The receiver decodes the light beam and switches the output device, which interfaces with the load.
- The receiver is tuned to its emitter's modulation frequency and will amplify only the light signal that pulses at the specific frequency.
- Most sensors allow adjustment of how much light will cause the output of the sensor to change state.
- Response time is related to the frequency of the light pulses. Response times may become important when an application calls for the detection of very small objects, objects moving at a high rate of speed, or both.

The **scan technique** refers to the method used by photoelectric sensors to detect an object. The *through-beam* scan technique (also called direct scan) places the transmitter and receiver in direct line with each other, as illustrated in Figure 6-29. Because the light beam travels in only one direction, through-beam scanning provides long-range sensing. Quite often, a garage door opener has a through-beam photoelectric sensor mounted near the floor, across the width of the door. For this application the sensor senses that nothing is in the path of the door when it is closing.

In a *retroreflective scan,* the transmitter and receiver are housed in the same enclosure. This arrangement requires the use of a separate reflector or reflective tape mounted across from the sensor to return light back to the receiver. The retroreflective scan is designed to respond to objects that interrupt the beam normally maintained between the transmitter and receiver, as illustrated in Figure 6-30. In contrast to a through-beam application, retroreflective sensors are used for medium-range applications.

Fiber optics is not a scan technique, but another method for transmitting light. ***Fiber optic sensors*** use a flexible cable containing tiny fibers that channel light from emitter to receiver, as illustrated in Figure 6-31. Fiber optic sensor systems are completely immune to all forms of electrical interference. The fact that an optical fiber does not contain any moving parts and carries only light means that there is no possibility of a spark. This means that it can be safely used even in the most hazardous sensing environments such as a refinery for producing gases, grain bins, mining, pharmaceutical manufacturing, and chemical processing.

Bar code technology is widely implemented in industry to enter data quickly and accurately. ***Bar code scanners*** are the eyes of the data collection system. A light source within the scanner illuminates the bar code symbol; those bars absorb light, and spaces reflect light. A photodetector collects this light in the form of an electronic-signal pattern representing the printed symbol. The decoder receives the signal from the scanner and converts these data into the character data representation

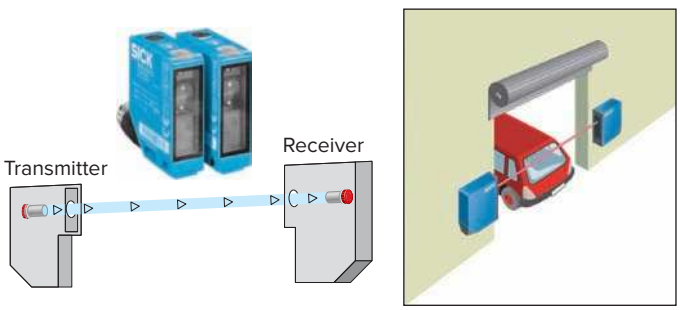


Figure 6-29 Through-beam scan.
Source: Photo courtesy SICK, Inc., **www.sick.com**.

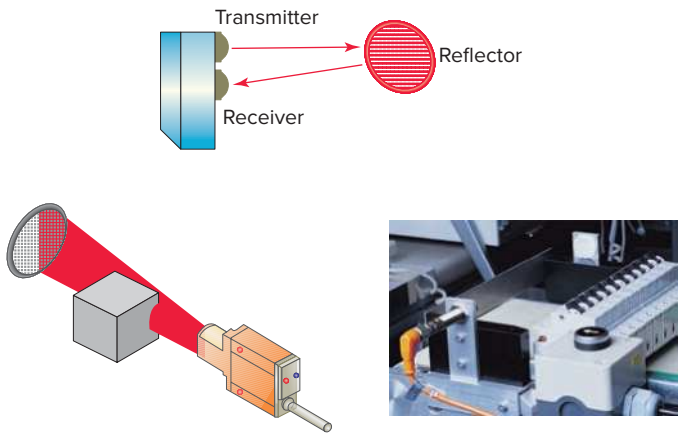


Figure 6-30 Retroreflective scan.
Source: Photo courtesy ifm efector, **www.ifm.com/us**.

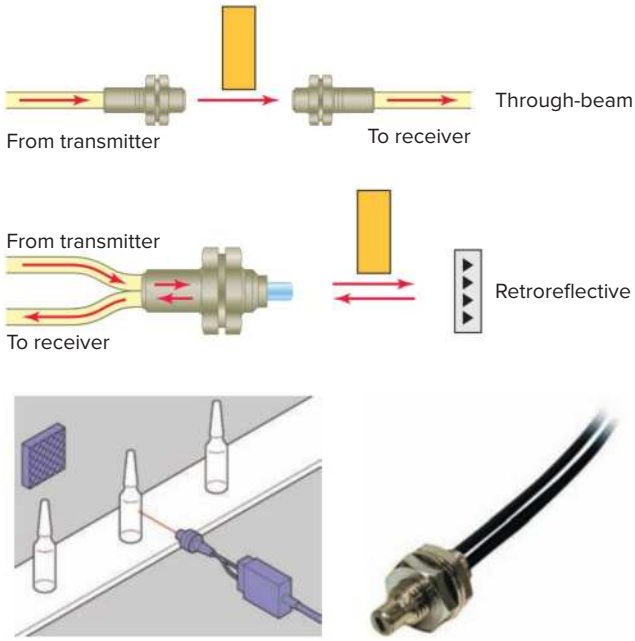


Figure 6-31 Fiber optic sensors.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

of the symbol's code. Figure 6-32 illustrates a typical PLC application which involves a bar code module reading the bar code on boxes as they move along a conveyor line. The PLC is then used to divert the boxes to the appropriate product lines according to the data read from the bar code.

Ultrasonic Sensors

An ***ultrasonic sensor*** operates by sending high-frequency sound waves toward the target and measuring the time it takes for the pulses to bounce back. The time taken for this echo to return to the sensor is directly proportional to the distance or height of the object because sound has a constant velocity.

Figure 6-33 illustrates a practical application in which the returning echo signal is electronically converted to a 4- to 20-mA output, which supplies a monitored flow rate

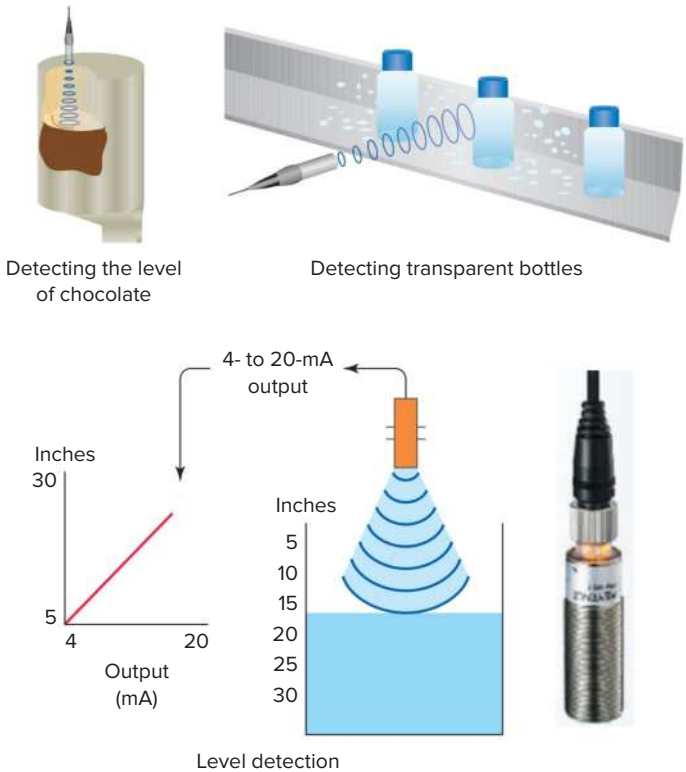


Figure 6-33 Ultrasonic sensor.
Source: Courtesy Keyence Canada, Inc.

to external control devices. The operation of this process can be summarized as follows:

- The 4- to 20-mA mA represents the sensor's measurement span.
- The 4-mA set point is typically placed near the bottom of the empty tank, or the greatest measurement distance from the sensor.
- The 20-mA set point is typically placed near the top of the full tank, or the shortest measurement distance from the sensor.
- The sensor will proportionately generate a 4-mA signal when the tank is empty and a 20-mA signal when the tank is full.

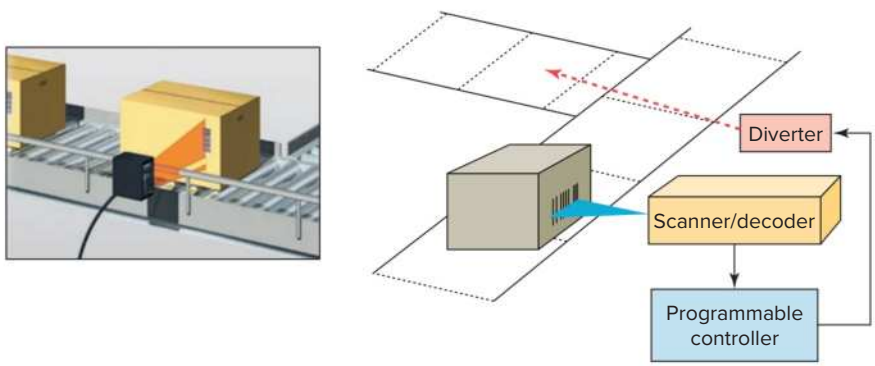


Figure 6-32 PLC bar code application.
Source: Courtesy Keyence Canada, Inc.

- Ultrasonic sensors can detect solids, fluids, granular objects, and textiles. In addition, they enable the detection of different objects irrespective of color and transparency and therefore are ideal for monitoring transparent objects.

Strain/Weight Sensors

A ***strain gauge*** converts a mechanical strain into an electric signal. Strain gauges are based on the principle that the resistance of a conductor varies with length and cross-sectional area. The force applied to the gauge causes the gauge to bend. This bending action also distorts the physical size of the gauge, which in turn changes its *resistance.* This resistance change is fed to a bridge circuit that detects small changes in the gauge's resistance. *Strain gauge load cells* are usually made with steel and sensitive strain gauges. As the load cell is loaded, the metal elongates or compresses very slightly. The strain gauge detects this movement and translates it to a varying voltage signal. Many sizes and shapes of load cells are available, and they range in sensitivity from grams to millions of pounds. Strain gauge–based load cells are used extensively for industrial weighing applications similar to the one illustrated in Figure 6-34.

Temperature Sensors

The thermocouple is the most widely used temperature sensor. Thermocouples operate on the principle that when two dissimilar metals are joined, a predictable DC voltage will be generated that relates to the difference in temperature between the hot junction and the cold junction (Figure 6-35). The hot junction (measuring junction) is the joined end of a thermocouple that is exposed to the process where the temperature measurement is desired. The cold junction (reference

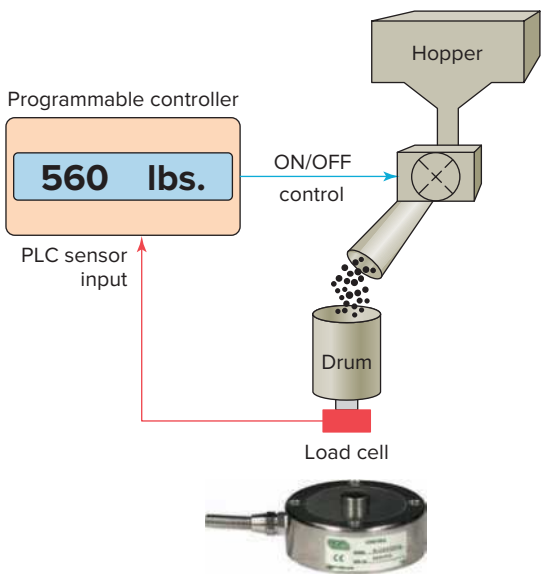


Figure 6-34 Strain gauge load cell.
Source: Courtesy RDP Group.

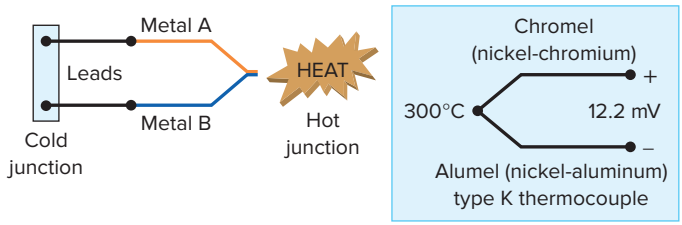


Figure 6-35 Thermocouple temperature sensor.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

junction) is the end of a thermocouple that is kept at a constant temperature to provide a reference point. For example, a K-type thermocouple, when heated to a temperature of 300°C at the hot junction, will produce 12.2 mV at the cold junction. Because of their ruggedness and wide temperature range, thermocouples are used in industry to monitor and control oven and furnace temperatures. Thermocouples produce a relative low output signal that is nonlinear. As a result, accurate thermocouple measurements need signal conditioning modules with outputs, which are linearly scaled to temperature.

Resistance temperature detectors (RTDs) are wire-wound temperature-sensing devices that operate on the principle of the positive temperature coefficient (PTC) of metals. That means the electrical resistance of metals is directly proportional to temperature. The hotter they become, the larger or higher the value of their electrical resistance. This proportional variation is precise and repeatable, and therefore allows the consistent measurement of temperature through electrical resistance detection. Platinum is the material most often used in RTDs because of its superiority regarding temperature limit, linearity, and stability.

RTDs are among the most precise temperature sensors available and are normally found encapsulated in probes for external temperature sensing and measurement or enclosed inside devices where they measure temperature as a part of the device's function. Figure 6-36 illustrates how

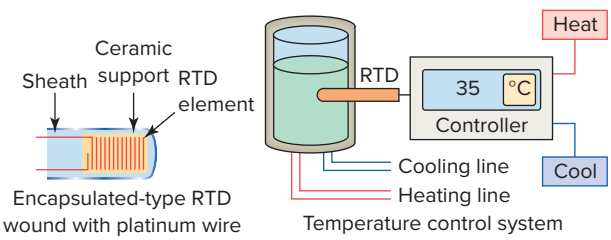


Figure 6-36 Resistance temperature detector (RTD).

an RTD is used as part of a temperature control system. A controller uses the signal from the RTD sensor to monitor the temperature of the liquid in the tank and thereby control heating and cooling lines.

Flow Measurement

Many industrial processes depend on accurate measurement of fluid flow. Although there are several ways to measure fluid flow, the usual approach is to convert the kinetic energy that the fluid has into some other measurable form.

Turbine-type flowmeters are a popular means of measurement and control of liquid products in industrial, chemical, and petroleum operations. Turbine flowmeters, like windmills, utilize their angular velocity (rotation speed) to indicate the flow velocity. The operation of a turbine flowmeter is illustrated in Figure 6-37. Its basic construction consists of a bladed turbine rotor installed in a flow tube. The bladed rotor rotates on its axis in proportion to the rate of the liquid flow through the tube. A magnetic pickup sensor is positioned as close to the rotor as practical. Fluid passing through the flow tube causes the rotor to rotate, which generates pulses in the pickup coil. The frequency of the pulses is then transmitted to readout electronics and displayed as gallons per minute.

Velocity and Position Sensors

Tachometer generators provide a convenient means of converting rotational speed into an analog voltage signal that can be used for motor speed indication and control applications. A tachometer generator is a small AC or DC generator that develops an output voltage (proportional to its rpm) whose phase or polarity depends on the rotor's direction of rotation. The DC tachometer generator usually has permanent magnetic field excitation. The AC tachometer generator field is excited by a constant AC supply. In either case, the rotor of the tachometer is mechanically connected, directly or indirectly, to the load.

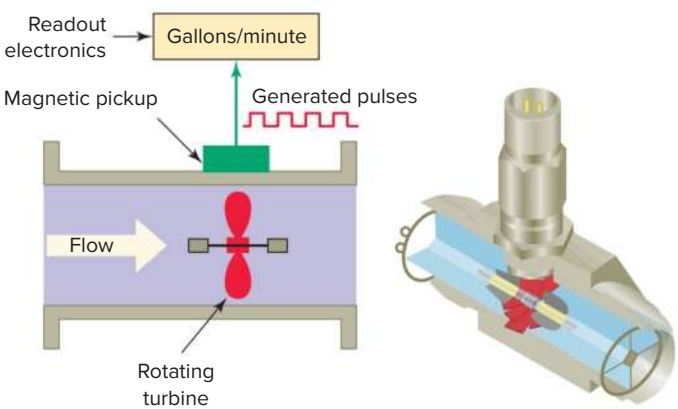


Figure 6-37 Turbine type flowmeter.

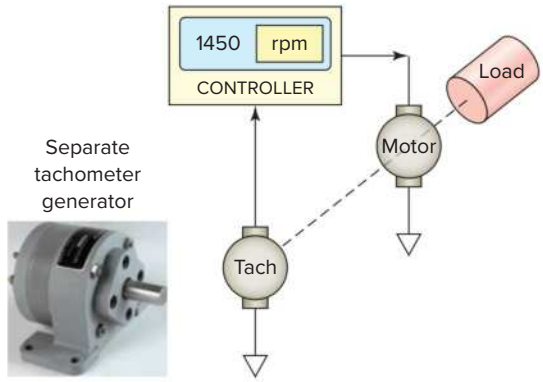


Figure 6-38 Tachometer generator feedback.
Source: Courtesy ATC Digitec.

Figure 6-38 illustrates motor speed control applications in which a tachometer generator is used to provide a feedback voltage to the motor controller that is proportional to motor speed. The control motor and tachometer generator may be contained in the same or separate housings.

An ***encoder*** is used to convert linear or rotary motion into a binary digital signal. Encoders are used in applications where positions have to be precisely determined. The optical encoder illustrated in Figure 6-39 uses a light source shining on an optical disk with lines or slots that interrupt the beam of light to an optical sensor. An electronic circuit counts the interruptions of the beam and generates the encoder's digital output pulses.

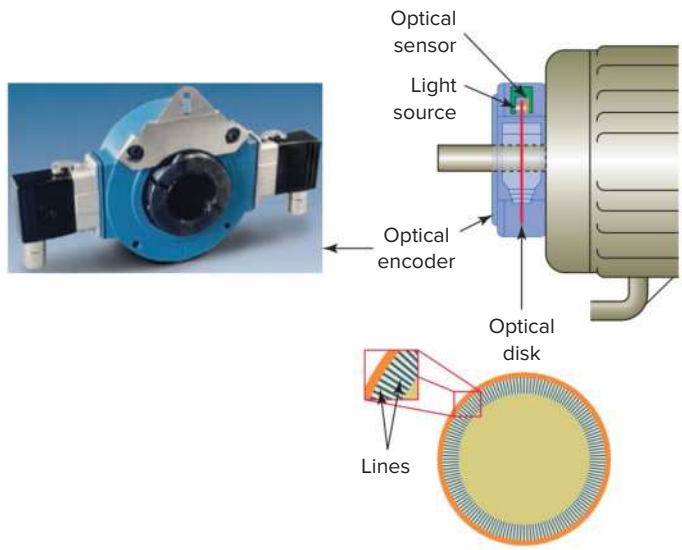


Figure 6-39 Optical encoder.
Source: Photo courtesy Avtron, **www.avtron.com**.

6.7 Output Control Devices

A variety of output control devices can be operated by the PLC output to control traditional industrial processes. These devices include pilot lights, control relays, motor starters, alarms, heaters, solenoids, solenoid valves, small motors, and horns. Similar electrical symbols are used to represent these devices both on relay schematics and PLC output connection diagrams. Figure 6-40 shows common electrical symbols used for various output devices. Although these symbols are generally acceptable, some differences among manufacturers do exist.

An ***actuator***, in the electrical sense, is any device that converts an electrical signal into mechanical movement. An electromechanical solenoid is an actuator that uses electrical energy to magnetically cause mechanical control action. A **solenoid** consists of a coil, frame, and plunger (or armature, as it is sometimes called). Figure 6-41 shows the basic construction and operation of a solenoid. Its operation can be summarized as follows:

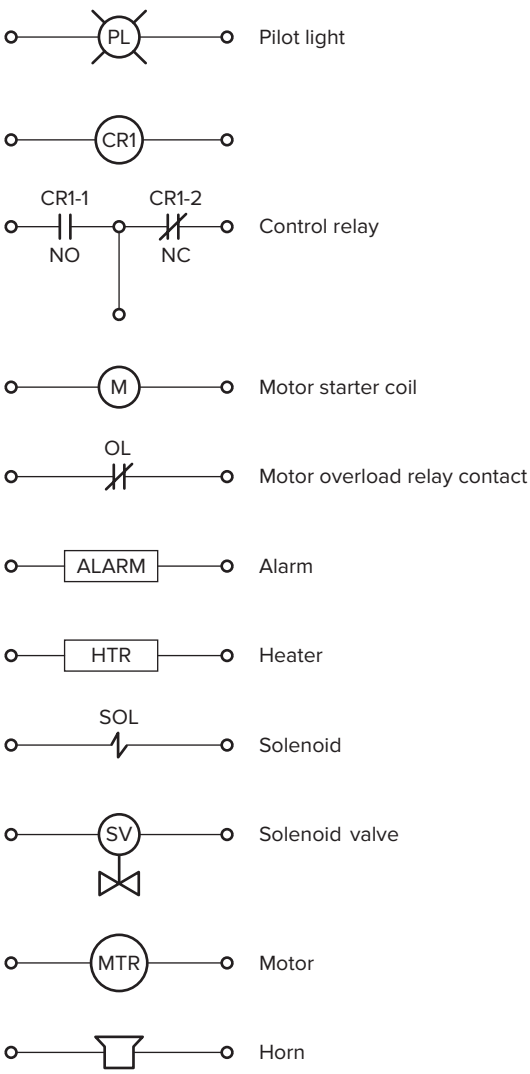


Figure 6-40 Symbols for output control devices.

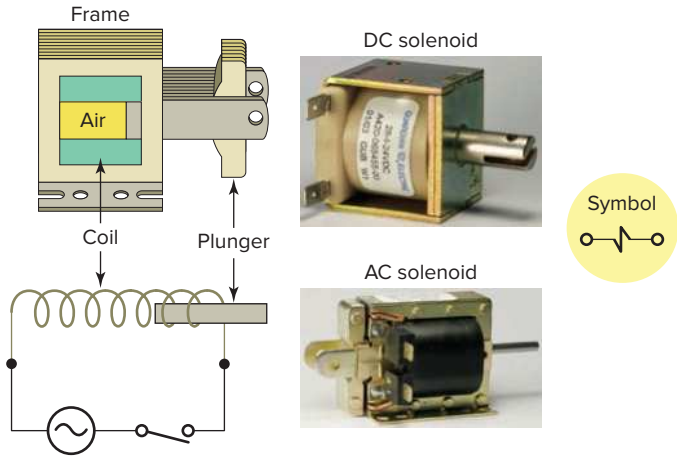


Figure 6-41 Solenoid construction and operation.
Source: Photos courtesy Guardian Electric, **www.guardian-electric.com**.

- The coil and frame form the fixed part.
- When the coil is energized, it produces a magnetic field that attracts the plunger, pulling it into the frame and thus creating mechanical motion.
- When the coil is de-energized the plunger returns to its normal position through gravity or assistance from spring assemblies within the solenoid.
- The frame and plunger of an AC-operated solenoid are constructed with laminated pieces instead of a solid piece of iron to limit eddy currents induced by the magnetic field.

Solenoid valves are electromechanical devices that work by passing an electrical current through a solenoid, thereby changing the state of the valve. Normally, there is a mechanical element, which is often a spring, that holds the valve in its default position. A solenoid valve is a combination of a solenoid coil operator and valve, which controls the flow of liquids, gases, steam, and other media. When electrically energized, they open, shut off, or direct the flow of media.

Figure 6-42 illustrates the construction and principle of operation of a typical fluid solenoid valve. Its operation can be summarized as follows:

- The valve body contains an orifice in which a disk or plug is positioned to restrict or allow flow.
- Flow through the orifice is either restricted or allowed depending on whether the solenoid coil is energized or de-energized.
- When the coil is energized, the core is drawn into the solenoid coil to open the valve.
- The spring returns the valve to its original closed position when the coil is de-energized.

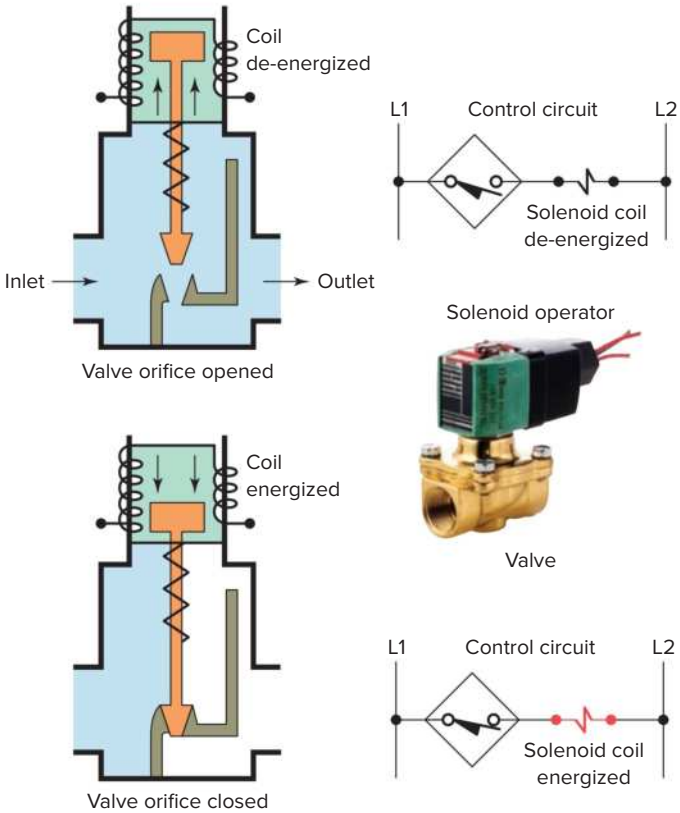


Figure 6-42 Solenoid valve construction and operation.
Source: Photo courtesy ASCO Valve Inc., **www.ascovalve.com**.

- A valve must be installed with direction of flow in accordance with the arrow cast on the side of the valve body.

Stepper motors operate differently than standard types, which rotate continuously when voltage is applied to their terminals. The shaft of a stepper motor rotates in discrete increments when electrical command pulses are applied to it in the proper sequence. Every revolution is divided into a number of steps, and the motor must be sent a voltage pulse for each step. The amount of rotation is directly proportional to the number of pulses, and the speed of rotation is relative to the frequency of those pulses. A 1-degree-per-step

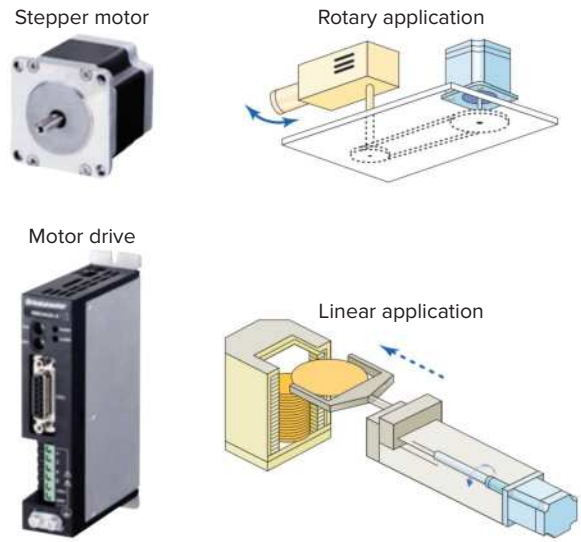


Figure 6-43 Stepper motor/drive unit.
Source: Photos courtesy Oriental Motor, **www.orientalmotor.com**.

motor will require 360 pulses to move through one revolution; the degrees per step are known as the ***resolution.*** When stopped, a stepper motor inherently holds its position. Stepper systems are used most often in "open-loop" control systems, where the controller tells the motor only how many steps to move and how fast to move, but does not have any way of knowing what position the motor is at.

The movement created by each pulse is precise and repeatable, which is why stepper motors are so effective for load-positioning applications. Conversion of rotary to linear motion inside a linear actuator is accomplished through a threaded nut and lead screw. Generally, stepper motors produce less than 1 hp and are therefore frequently used in low-power position control applications. Figure 6-43 shows a stepper motor/drive unit along with typical rotary and linear applications.

All ***servo motors*** operate in closed-loop mode, whereas most stepper motors operate in open-loop mode. Closed-loop and open-loop control schemes are illustrated in Figure 6-44. ***Open loop*** is control

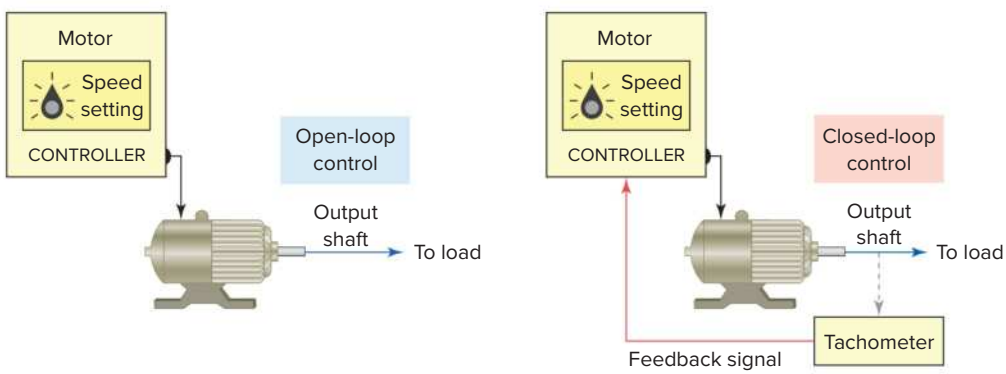


Figure 6-44 Open- and closed-loop motor control systems.

without feedback, for example, when the controller tells the stepper motor how many steps to move and how fast to move, but does not verify where the motor is. ***Closed-loop*** control compares speed or position feedback with the commanded speed or position and generates a modified command to make the error smaller. The error is the difference between the required speed or position and the actual speed or position.

Figure 6-45 illustrates a closed-loop servo motor system. The motor controller directs operation of the servo motor by sending speed or position command signals to the amplifier, which drives the servo motor. A feedback device such as an encoder for position and a tachometer for speed are either incorporated within the servo motor or are remotely mounted, often on the load itself. These provide the servo motor's position and speed feedback information that the controller compares to its programmed motion profile and uses to alter its position or speed.

Motor/controller

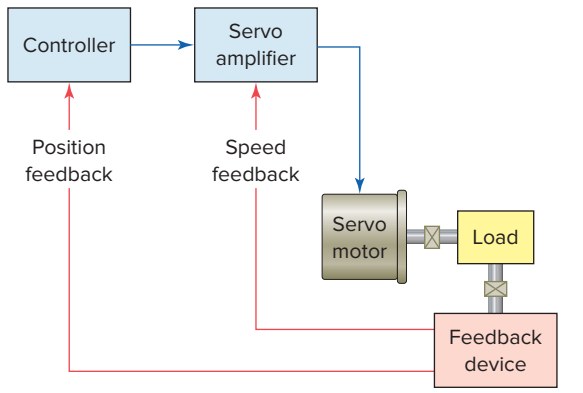
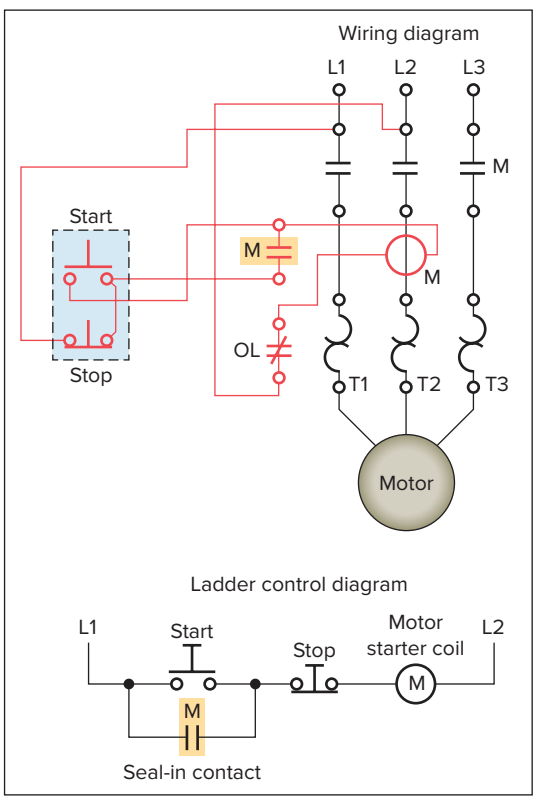


Figure 6-45 Closed-loop servo motor system.
Source: Photos courtesy Omron Industrial Automation, **www.ia.omron.com**.

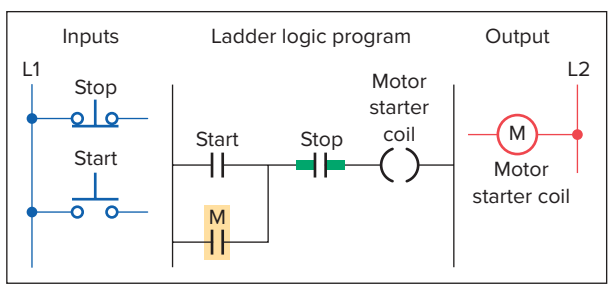
6.8 Seal-In Circuits

Seal-in, or ***holding,*** circuits are very common in both relay logic and PLC logic. Essentially, a seal-in circuit is a method of maintaining current flow after a momentary switch has been pressed and released. In these types of circuits, the seal-in contact is usually in parallel with the momentary device.

The motor stop/start circuit shown in Figure 6-46 is a typical example of a seal-in circuit. The hardwired circuit consists of a normally closed stop button in series with a normally open start button. The seal-in auxiliary contact of the starter is connected in parallel with the start button to keep the starter coil energized when the start button is



Hardwired



Programmed

Figure 6-46 Hardwired and programmed seal-in circuit.

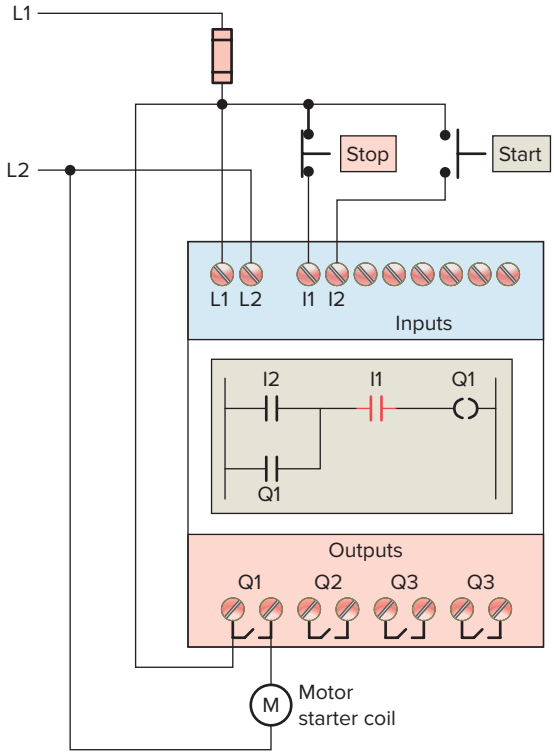


Figure 6-47 Motor seal-in circuit implemented using an Allen-Bradley Pico controller.

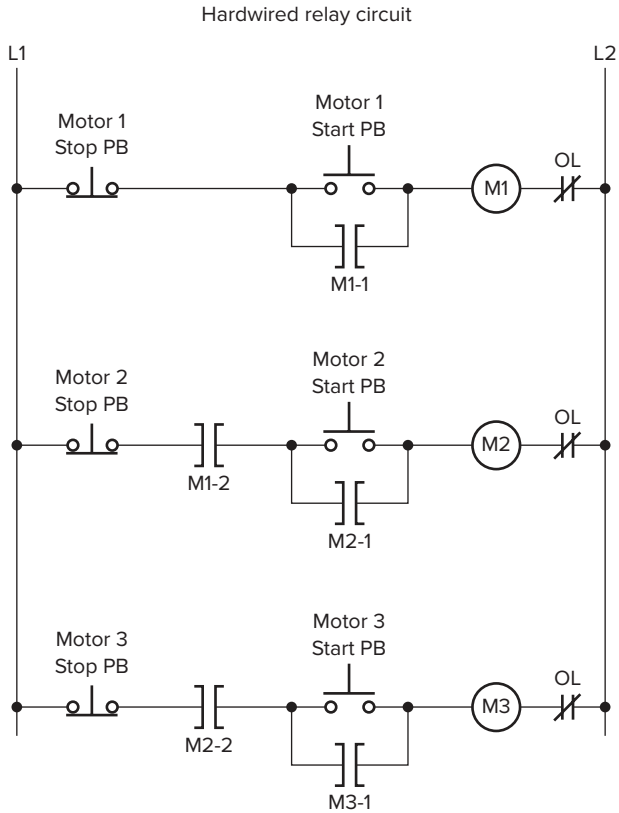


Figure 6-48 Sequential hardwired three motor relay control circuit.

released. When this circuit is programmed into a PLC, both the start and stop buttons are examined for a closed condition because both buttons must be closed to cause the motor starter to operate.

Figure 6-47 shows a PLC wiring diagram of the motor seal-in circuit implemented using an Allen-Bradley Pico controller. The controller is programmed using ladder logic. Each programming element can be entered directly via the Pico display. This controller also lets you program the circuit from a personal computer using PicoSoft programming software.

6.9 Electrical Interlocking Circuits

An electrical interlocking circuit is used to prevent a piece of equipment from operating under certain potentially hazardous or undesirable conditions. Figure 6-48 shows a three motor hardwired relay control circuit electrically interlocked to prevent the motors from accidently operating in an order other than their proper sequence. The interlocking feature of the circuit can be summarized as follows:

- Motor 1 has to be operating before Motor 2 can be started.
- The NO auxiliary interlocking contact M1-2 is used for this purpose.
- Motor 2 has to be operating before Motor 3 can be started.
- The NO auxiliary interlocking contact M2-2 is used for this purpose.

Figure 6-49 shows a PLC program equivalent of the hardwired circuit.

Pushbutton interlocking is one of the methods of preventing two loads from being energized simultaneously. The hardwired pushbutton interlocking circuit of Figure 6-50 is designed to prevent solenoids SOL-A and SOL-B from being energized at the same time. The interlocking feature of the circuit can be summarized as follows:

- Each pushbutton is equipped with a set of momentarily normally open (NO) and normally closed (NC) contacts mechanically connected together.
- The NC contact of SOL-A pushbutton is connected in series with the NO contact of SOL-B pushbutton.
- The NO contact of SOL-A pushbutton is connected in series with the NC contact of SOL-B pushbutton.

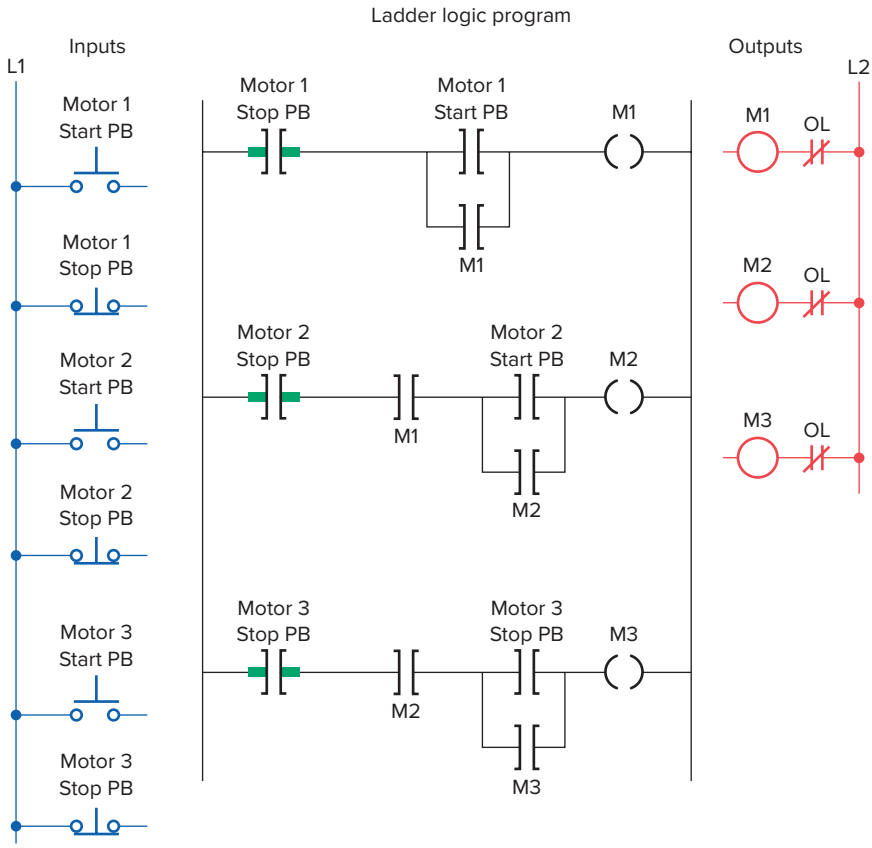


Figure 6-49 PLC program equivalent of the hardwired sequential motor control circuit.

- When SOL-A pushbutton is pressed its NO contact completes the circuit to SOL-A and its NC contacts opens the current path to SOL-B.
- When SOL-B pushbutton is pressed its NO contact completes the circuit to SOL-B and its NC contacts opens the current path to SOL-A.
- When both buttons are pushed, neither solenoid will be energized.

Figure 6-51 shows a PLC program equivalent of the hardwired circuit implemented using two NO pushbuttons.

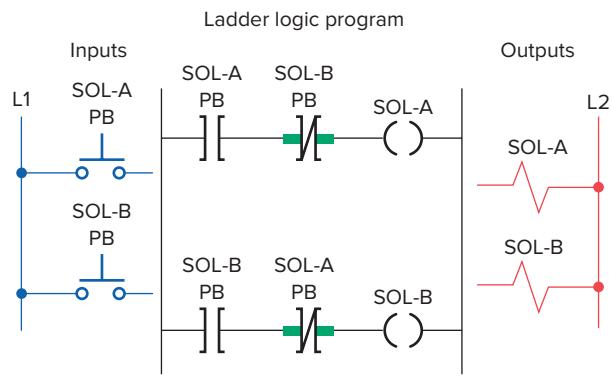


Figure 6-51 PLC program equivalent of the hardwired pushbutton interlocking circuit.

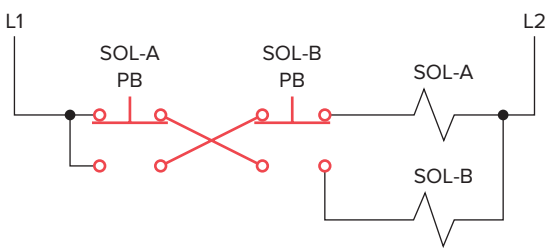


Figure 6-50 Hardwired pushbutton interlocking circuit.

6.10 Latching Relays

Electromagnetic **latching relays** are designed to hold the relay closed after power has been removed from the coil. Latching relays are used where it is necessary for contacts to stay open and/or closed even though the coil is energized only momentarily. Figure 6-52 shows a latching relay that uses two coils. The ***latch*** coil is momentarily energized to set the latch and hold the relay in the latched

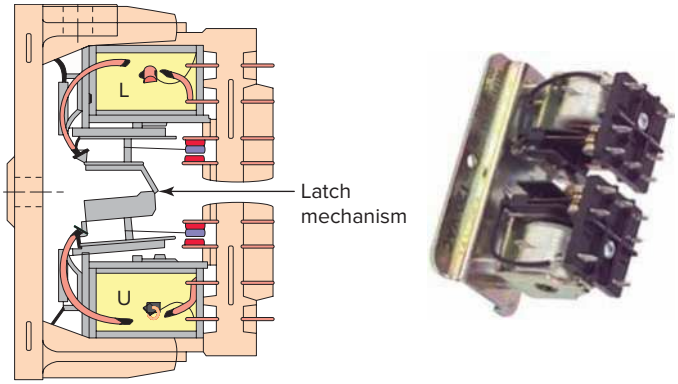


Figure 6-52 Two-coil mechanical latching relay.
Source: Courtesy Relay Service Company.

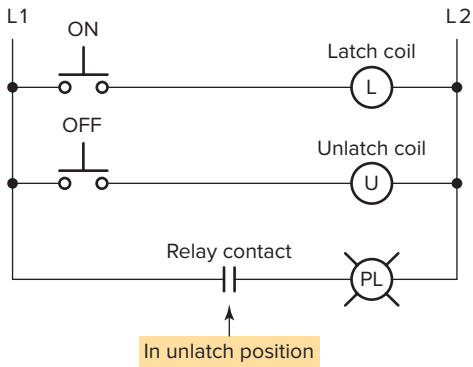


Figure 6-53 Hardwired control circuit for an electromagnetic latching relay.

position. The ***unlatch*** or release coil is momentarily energized to disengage the mechanical latch and return the relay to the unlatched position.

Figure 6-53 shows a hardwired control circuit for an electromagnetic latching relay. The operation of the circuit can be summarized as follows:

- The contact is shown with the relay in the *unlatched* position.
- In this state the circuit to the pilot light is open and so the light is off.
- When the ON button is *momentarily* actuated, the latch coil is energized to set the relay to its latched position.
- The contacts close, completing the circuit to the pilot light, and so the light is switched on.
- The relay coil does *not* have to be continuously energized to hold the contacts closed and keep the light on.
- The only way to switch the lamp off is to actuate the OFF button, which will energize the unlatch coil and return the contacts to their open, unlatched state.
- In cases of power loss, the relay will remain in its original latched or unlatched state when power is restored.

An electromagnetic latching relay function can be programmed on a PLC to work like its real-world counterparts. The instruction set for the SLC 500 includes a set of output instructions that duplicates the operation of the mechanical latch. A description of the output latch (**OTL**) and output unlatch (**OTU**) instruction is given in Figure 6-54. The OTL and OTU instructions differ from the OTE instruction in that they must be used together. Both the latch and unlatch outputs must have the same address. The OTL (latch) instruction can only turn a bit on and the OTU (unlatch) instruction can only turn a bit off.

The operation of the output latch and output unlatch coil instruction is illustrated in the ladder program of Figure 6-55. The operation of the program can be summarized as follows:

- Both the latch (L) and the unlatch (U) coil have the *same* address (O:2/5).
- When the on pushbutton (I:1/0) is momentarily actuated, the latch rung becomes true and the latch status bit (O:2/5) is set to 1, and so the light output is switched on.

Command	Name	Symbol	Description
OTL	Output latch	(L)	OTL sets the bit to "1" when the rung becomes true and retains its state when the rung loses continuity or a power cycle occurs.
OTU	Output unlatch	(U)	OTU resets the bit to "0" when the rung becomes true and retains it.

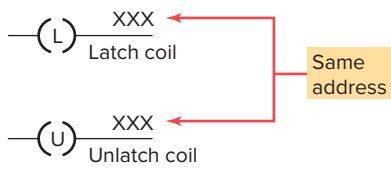


Figure 6-54 Output latch and output unlatch instruction.

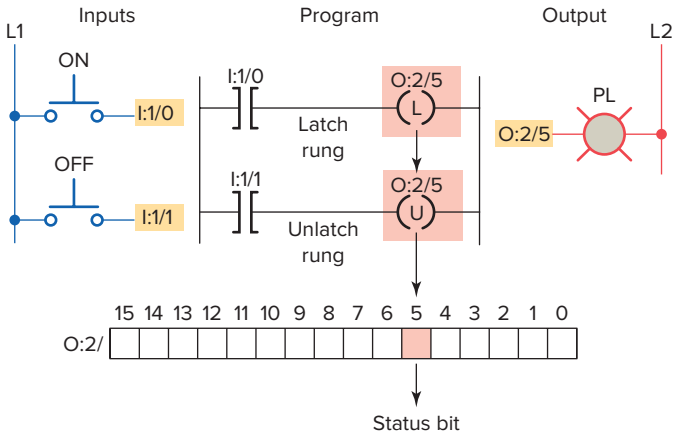


Figure 6-55 Output latch and output unlatch operation.

- The status bit *will remain set to 1* when the pushbutton is released and logical continuity of the latch rung is lost.
- When the off pushbutton (I:1/1) is momentarily actuated, the unlatch rung becomes true and the latch status bit (O:2/5) is reset back to 0 and so the light is switched off.
- The status bit *will remain reset to 0* when the pushbutton is released and logical continuity of the latch rung is lost.

Output latch is an output instruction with a bit-level address. When the instruction is true, it sets a bit in the output image file. It is a retentive instruction because the bit remains set when the latch instruction goes false. In most applications it is used with an unlatch instruction. The output unlatch instruction is also an output instruction with a bit-level address. When the instruction is true, it resets a bit in the output image file. It, too, is a retentive instruction because the bit remains reset when the instruction goes false.

The process shown in Figure 6-56 is to be used to control the level of water in a storage tank by turning a discharge pump on or off. The modes of operation are to be programmed as follows:

OFF Position—The water pump will *stop* if it is running and will *not* start if it is stopped.

Manual Mode—The pump will start if the water in the tank is at any level except low.

Automatic Mode—If the level of water in the tank *reaches a high point,* the water pump will *start* so that water can be removed from the tank, thus lowering the level.

- When the water level *reaches a low point,* the pump will *stop*.

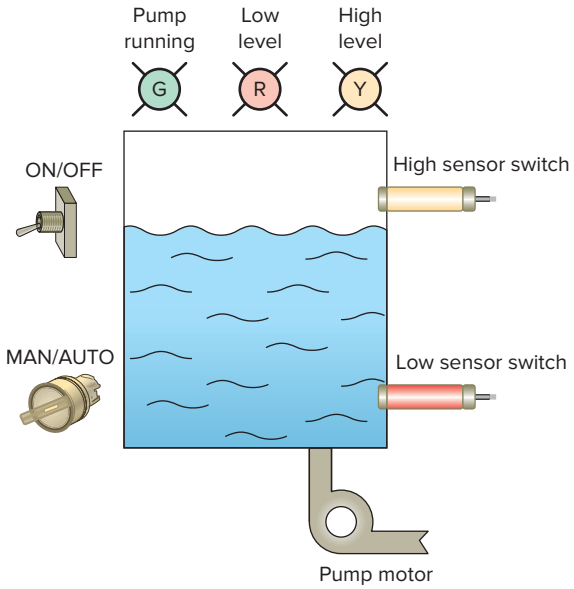


Figure 6-56 Process used to control the level of water in a storage tank.

Status Indicating Lights—Water pump running light (green)

- Low water level status light (red)
- High water level status light (yellow)

Figure 6-57 shows a program that can be used to implement control of the water level in the storage tank. The latch and unlatch instructions form part of the program. The operation of the program can be summarized as follows:

- An internal storage bit is used for the latch and address rather than an actual discrete output address. Both have the same addresses.
- The rung 1 Examine-on instruction addressed to the off/on switch prevents the pump motor from starting under any condition when in the off (open) state.
- In the MAN mode, the rung 1 Examine-on instruction addressed to the low sensor switch allows the pump motor to operate only when the low level sensor switch is closed.
- In the AUTO mode, whenever the high sensor switch is momentarily closed the Examine-on instruction of rung 1 addressed to it will energize the latch coil. The pump will begin running and continue to operate until the unlatch coil is energized by the rung 3 Examine-off instruction addressed to the low sensor switch.
- The pump running status light is controlled by the rung 4 Examine-on instruction addressed to the motor output.

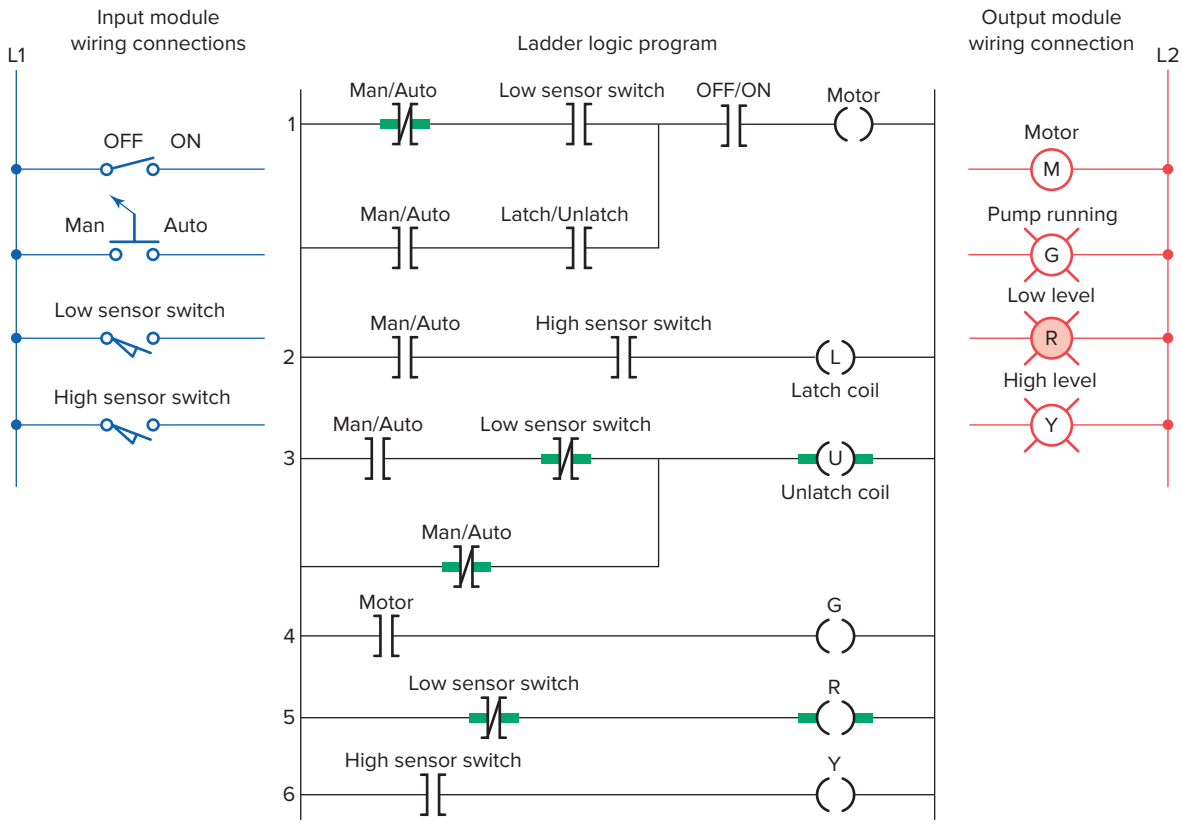


Figure 6-57 Program used to implement control of the water level in the storage tank.

- The low-level status light is controlled by the rung 5 Examine-off instruction addressed to the low sensor switch.
- The high-level status light is controlled by the rung 6 Examine-on instruction addressed to the high sensor switch.

Figure 6-58 shows a typical I/O module wiring diagram and addressing format for the water level control program implemented using an Allen-Bradley modular SLC 500 controller. The chassis power supply has a relatively small power rating and is used to supply DC power to all devices physically mounted in the backplane of the PLC rack. In this application a 24 VDC field power supply is used for the input devices and a 120 VAC field power supply for the output devices. This allows a low-voltage 24-volt control signal to control 240-volt output devices. SLC 500 controllers use a rack/slot-based address system where the slot location of the I/O modules in the rack establishes the PLC address. The addresses for the field devices of this particular application are shown below:

FIELD DEVICE	ADDRESS	Signifies
OFF/ON Switch	**I:2/0**	The input module in slot 2 and screw terminal 0
MAN/AUTO Switch	**I:2/4**	The input module in slot 2 and screw terminal 4
LOW SENSOR Switch	**I:2/8**	The input module in slot 2 and screw terminal 8
HIGH SENSOR Switch	**I:2/12**	The input module in slot 2 and screw terminal 12
MOTOR	**O:3/1**	The output module in slot 3 and screw terminal 1
PUMP RUNNING Light	**O:3/5**	The output module in slot 3 and screw terminal 5
LOW LEVEL Light	**O:3/9**	The output module in slot 3 and screw terminal 9
HIGH LEVEL Light	**O:3/13**	The output module in slot 3 and screw terminal 13
	B3:0/0	Internal retentive bit instruction that does not drive a real-word device

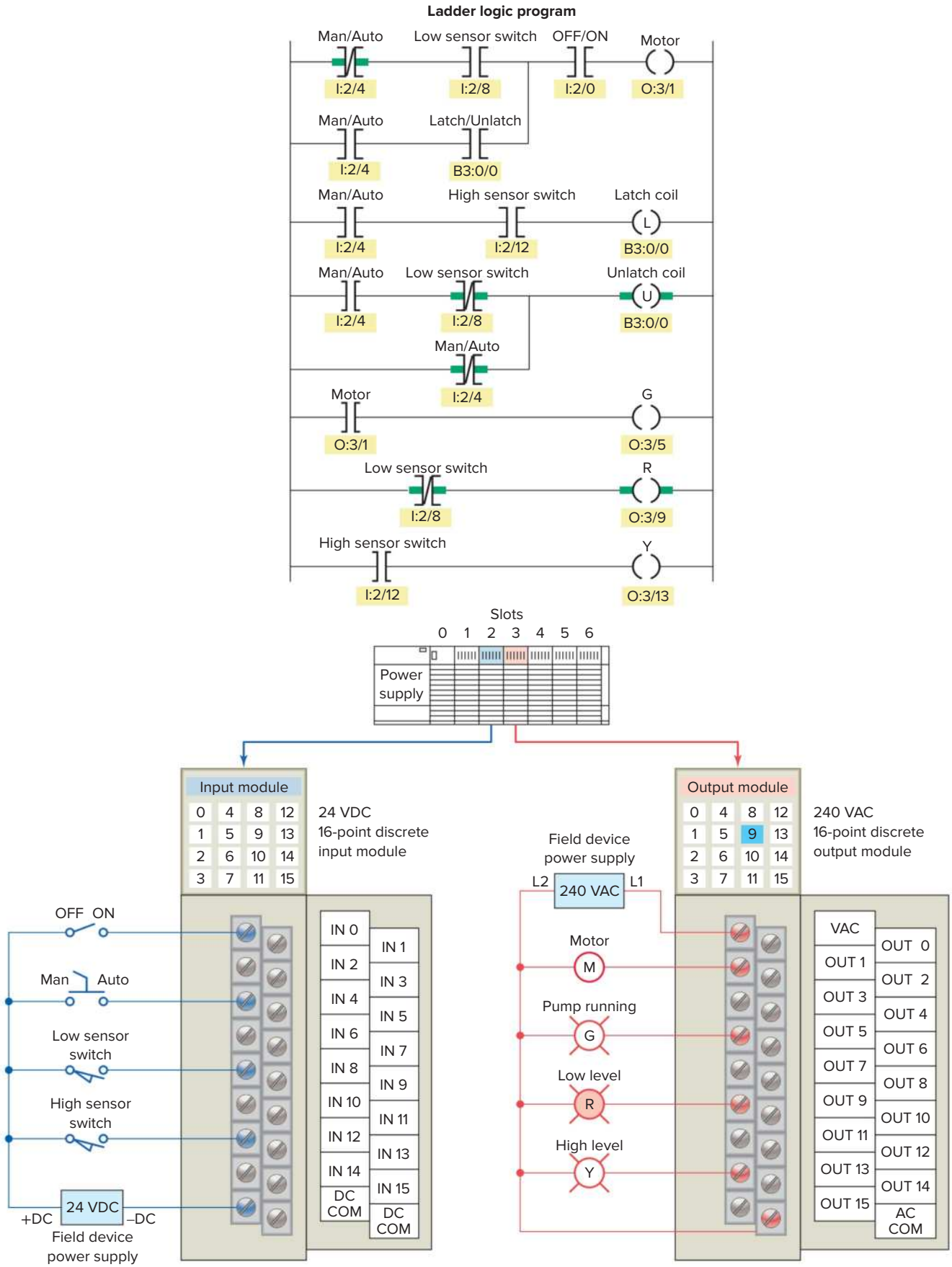


Figure 6-58 Water-level control program implemented using an Allen-Bradley modular SLC 500 controller.

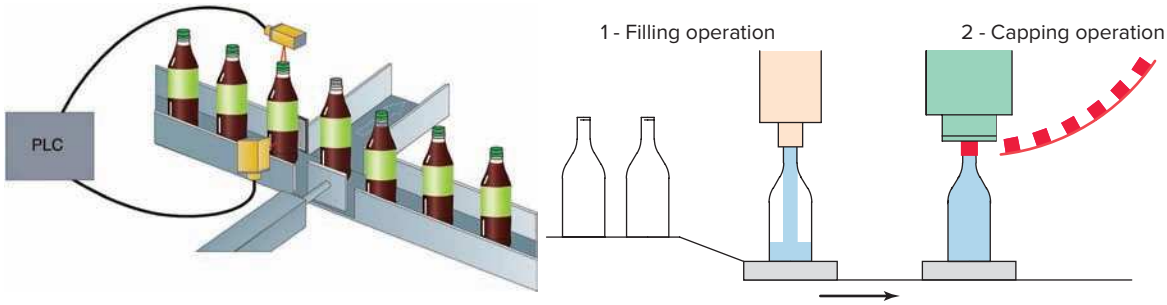


Figure 6-59 Sequential control process.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

6.11 Converting Relay Schematics into PLC Ladder Programs

The best approach to developing a PLC program from a relay schematic is to understand first the operation of each relay ladder rung. As each relay ladder rung is understood, an equivalent PLC rung can be generated. This process will require access to the relay schematic, documentation of the various input and output devices used, and possibly a process flow diagram of the operation.

Most control processes require the completion of several operations to produce the required output. Manufacturing, machining, assembling, packaging, finishing, or transporting of products requires the precise coordination of tasks.

A ***sequential*** control process is required for processes that demand that certain operations be performed in a specific order. Figure 6-59 illustrates part of a bottle filling process. In the filling and capping operations, the tasks are (1) fill bottle and (2) press on cap. These tasks must be performed in the proper order. Obviously we could not fill the bottle after the cap is pressed on. This process, therefore, requires sequential control.

Combination controls require that certain operations be performed without regard to the order in which they are performed. Figure 6-60 illustrates another part of the same bottle filling process. Here, the tasks are (1) place label 1 on bottle and (2) place label 2 on bottle. The order in which the tasks are performed does not really matter. In fact, however, many industrial processes that are not inherently sequential in nature are performed in a sequential manner for the most efficient order of operations.

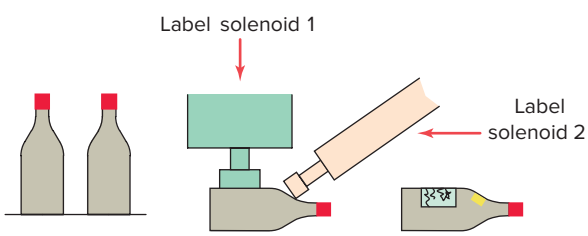


Figure 6-60 Combination control process.

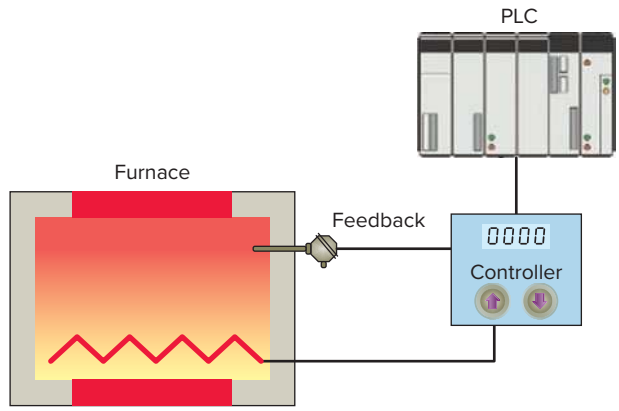


Figure 6-61 Automatic control process.

Automatic control involves maintaining a desired set point at an output. One example is maintaining a certain set-point temperature in a furnace as illustrated in Figure 6-61. If there is deviation from that set point, an error is determined by comparing the output against the set point and using this error to make a correction. This requires feedback from the output to the control for the input.

The converting of a simple sequential process can be examined with reference to the process flow diagram illustrated in Figure 6-62. The sequential task is as follows:

1. Start button is pressed.
2. Table motor is started.

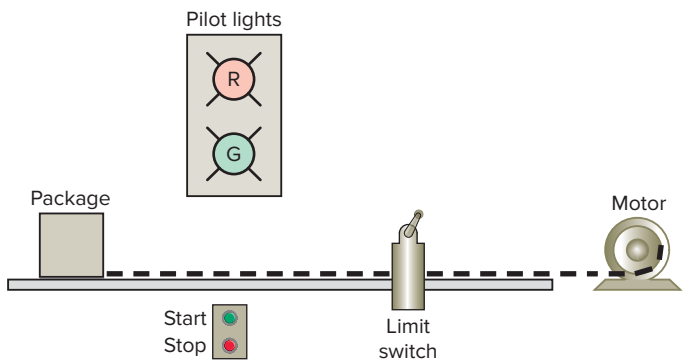


Figure 6-62 Sequential process flow diagram.

3. Package moves to the position of the limit switch and automatically stops.

Other auxiliary features include:

- a stop button that will stop the table, for any reason, before the package reaches the limit switch position;
- a red pilot light to indicate the table is stopped; and
- a green pilot light to indicate the table is running.

A relay schematic for the sequential process is shown in Figure 6-63. The operation of this hardwired circuit can be summarized as follows:

- Start button is actuated; CR is energized if stop button and limit switch are not actuated.
- Contact CR-1 closes, sealing in CR when the start button is released.
- Contact CR-2 opens, switching the red pilot light from on to off.
- Contact CR-3 closes, switching the green pilot light from off to on.
- Contact CR-4 closes to energize the motor starter coil, starting the motor and moving the package toward the limit switch.
- Limit switch is actuated, de-energizing relay coil CR.
- Contact CR-1 opens, opening the seal-in circuit.
- Contact CR-2 closes, switching the red pilot light from off to on.
- Contact CR-3 opens, switching the green pilot light from on to off.
- Contact CR-4 opens, de-energizing the motor starter coil to stop the motor and end the sequence.

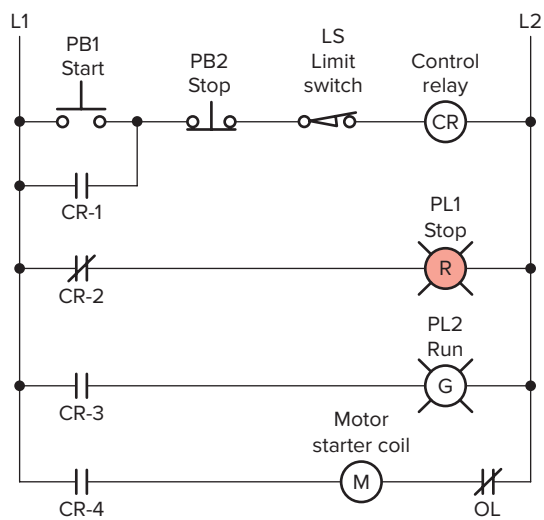


Figure 6-63 Relay schematic for the sequential process.

Figure 6-64 shows an I/O connection diagram for a programmed version of the sequential process. Each input and output device is represented by its symbol and associated address. These addresses will indicate what PLC input is connected to what input device and what PLC output will drive what output device. The address code, of course, will depend on the PLC model used. This example uses SLC 500 addressing for the process. Note that the electromagnetic control relay CR is *not* needed because its function is replaced by an *internal* PLC control relay.

The hardwired relay schematic for the sequential process can be converted to the PLC ladder logic program shown in Figure 6-65. In converting the process to a program the operation of each rung must be understood. The pushbuttons PB1, PB2 as well as limit switch LS are all programmed using the examine-closed (–] [–) instruction to produce the desired logic control. Also, internal relay B3:1/0 is used to replace control relay CR. To obtain the desired control logic, all internal relay contacts are programmed using the PLC contact instruction that matches the coil de-energized state. The internal relay implemented in software requires one coil address the contacts of which can be examined for an ON or OFF condition as many times as you like.

There is more than one method to correctly design the ladder logic program for a given control process. In some cases one arrangement may be more efficient in terms of the amount of memory used and the time required to scan the program. Figure 6-66 illustrates an example of an arrangement of series instructions of a rung programmed for optimum scan time. The series instructions are programmed from the most likely to be *false* (far left) to the least likely to be *false* (far right). Once the processor sees a false input instruction in series, the processor stops checking the rung at the false condition and sets the output false.

Figure 6-67 illustrates an example of an arrangement of parallel instructions of a rung programmed for optimum scan time. The parallel path that is most often *true* is

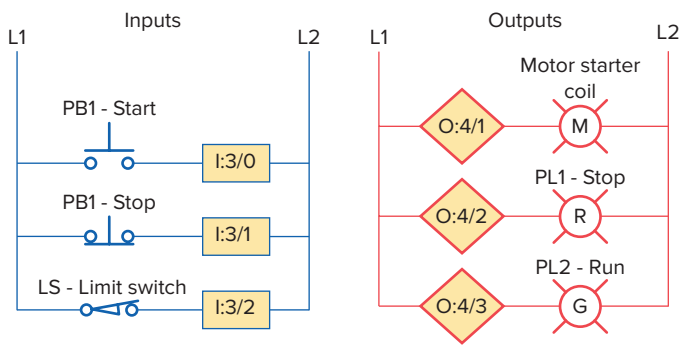


Figure 6-64 I/O connection diagram.

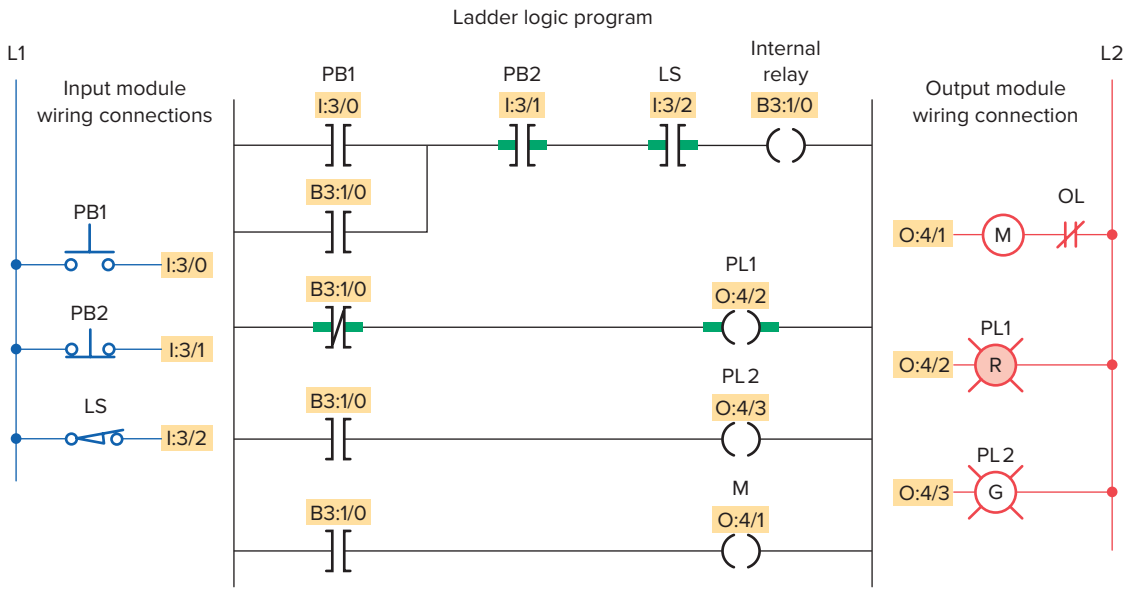


Figure 6-65 Sequential process PLC ladder logic program.

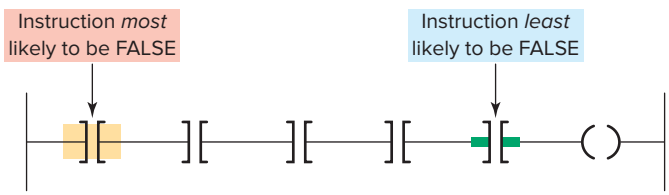


Figure 6-66 Series instructions programmed for optimum scan time.

placed on the top of the rung. The processor will not look at the others unless the top path is *false*.

Figure 6-68 shows a hardwired jog control circuit that incorporates a jog control relay. The operation of the circuit can be summarized as follows:

- Pressing the start pushbutton completes a circuit for the CR coil, closing the CR1 and CR2 contacts.
- The CR1 contact completes the circuit for the M coil, starting the motor.

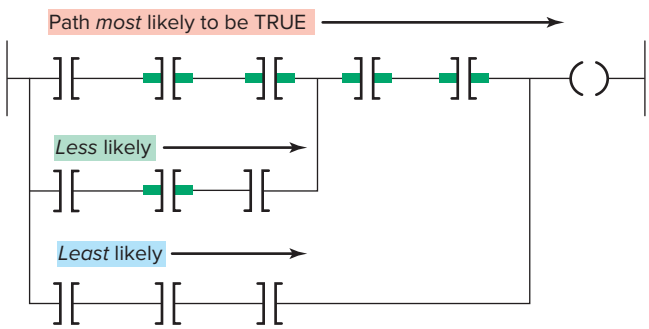
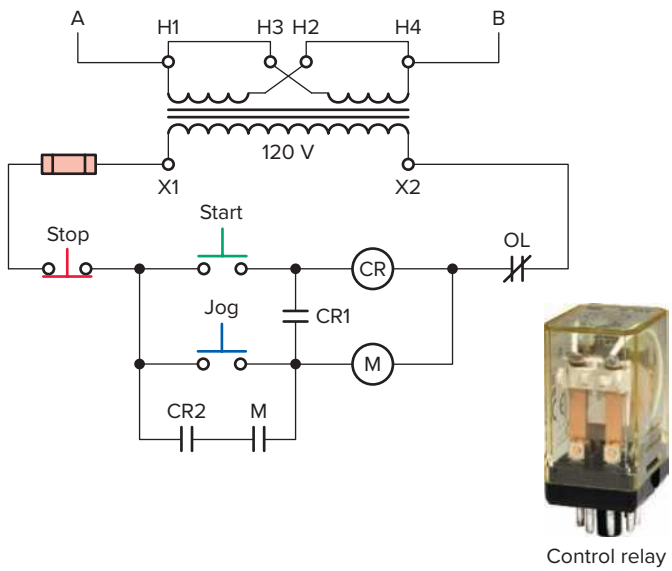


Figure 6-67 Parallel instructions programmed for optimum scan time.



Control relay

Figure 6-68 Jog circuit with control relay.
Source: Photo courtesy IDEC Corporation, **www.IDEC.com/usa**, RR Relay.

- The M maintaining contact closes; this maintains the circuit for the M coil.
- Pressing the jog button energizes the M coil only, starting the motor. Both CR contacts remain open, and the CR coil is de-energized. The M coil will not remain energized when the jog push button is released.

Figure 6-69 shows a PLC program equivalent of the hardwired relay jog circuit. Note that the function of the control relay is now accomplished using an internal PLC instruction (B3:1/0).

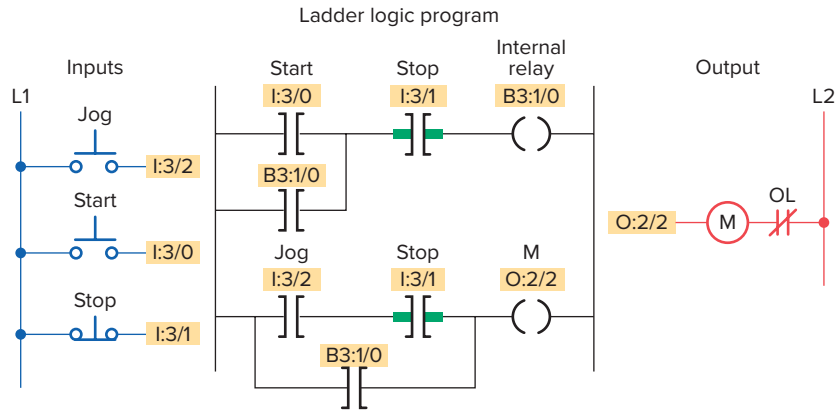


Figure 6-69 PLC program equivalent of the hardwired relay jog circuit.

6.12 Writing a Ladder Logic Program Directly from a Narrative Description

In most cases, it is possible to prepare a ladder logic program directly from the narrative description of a control process. Some of the steps in planning a program are as follows:

- Define the process to be controlled.
- Draw a sketch of the process, including all sensors and manual controls needed to carry out the control sequence.
- List the sequence of operational steps in as much detail as possible.
- Write the ladder logic program to be used as a basis for the PLC program.
- Consider different scenarios where the process sequence may go astray and make adjustments as needed.
- Consider the safety of operating personnel and make adjustments as needed.

The following are examples of ladder logic programs derived from narrative descriptions of control processes.

EXAMPLE 6-1

Figure 6-70 shows the sketch of a drilling process that requires the drill press to turn on only if there is a part present and the operator has one hand on each of the start switches. This precaution will ensure that the operator's hands are not in the way of the drill.

The sequence of operation requires that switches 1 and 2 and the part sensor all be activated to make the drill motor operate. Figure 6-71 shows the ladder logic program required for the process implemented using an SLC 500 controller.

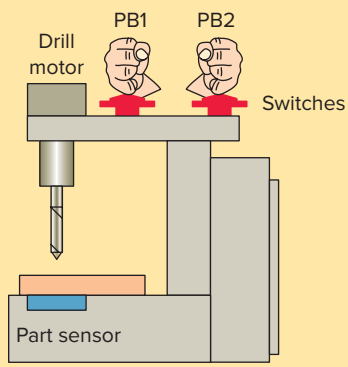


Figure 6-70 Sketch of the drilling process.

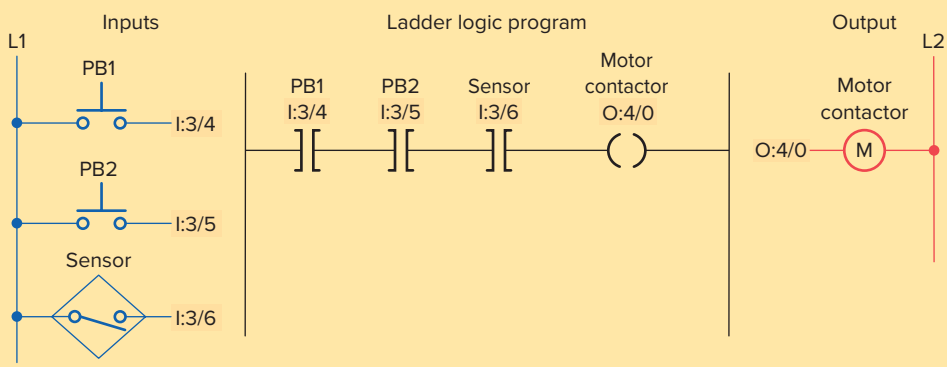


Figure 6-71 Drilling process PLC program.

EXAMPLE 6-2

A motorized overhead garage door is to be operated automatically to preset open and closed positions. The field devices include one of each of the following:

- Reversing *motor contactor* for the up and down directions.
- Normally open *down limit switch* to sense when the door is fully closed.
- Normally open-held closed *up limit switch* to sense when the door is fully opened.
- Normally open *door up button* for the up direction.
- Normally open *door down button* for the down direction.
- Normally closed *door stop button* for stopping the door.
- Red *door ajar light* to signal when the door is partially open.
- Green *door open light* to signal when the door is fully open.
- Yellow *door closed light* to signal when the door is fully closed.

The sequence of operation requires that:

- When the up button is pushed, the up motor contactor energizes and the door travels upward until the up limit switch is actuated.
- When the down button is pushed, the down motor contactor energizes and the door travels down until the down limit switch is actuated.
- When the stop button is pushed, the motor stops. The motor must be stopped before it can change direction.

Figure 6-72 shows the ladder logic program required for the operation implemented using an SLC 500 controller.

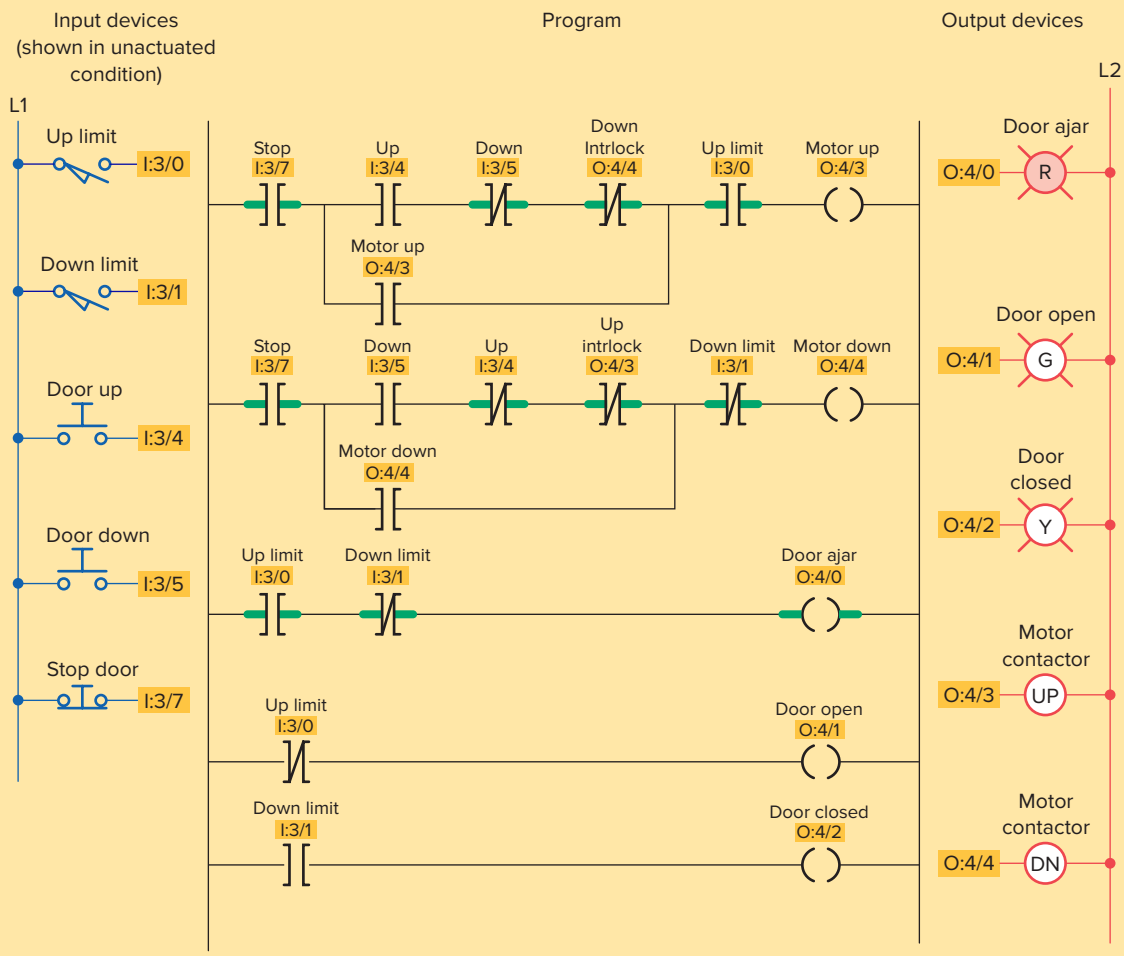


Figure 6-72 Motorized overhead garage door PLC program.

EXAMPLE 6-3

Figure 6-73 shows the sketch of a continuous filling operation. This process requires that boxes moving on a conveyor be automatically positioned and filled.

The sequence of operation for the continuous filling operation is as follows:

- Start the conveyor when the start button is momentarily pressed.
- Stop the conveyor when the stop button is momentarily pressed.
- Energize the run status light when the process is operating.
- Energize the standby status light when the process is stopped.
- Stop the conveyor when the right edge of the box is first sensed by the photosensor.
- With the box in position and the conveyor stopped, open the solenoid valve and allow the box to fill. Filling should stop when the level sensor goes true.
- Energize the full light when the box is full. The full light should remain energized until the box is moved clear of the photosensor.

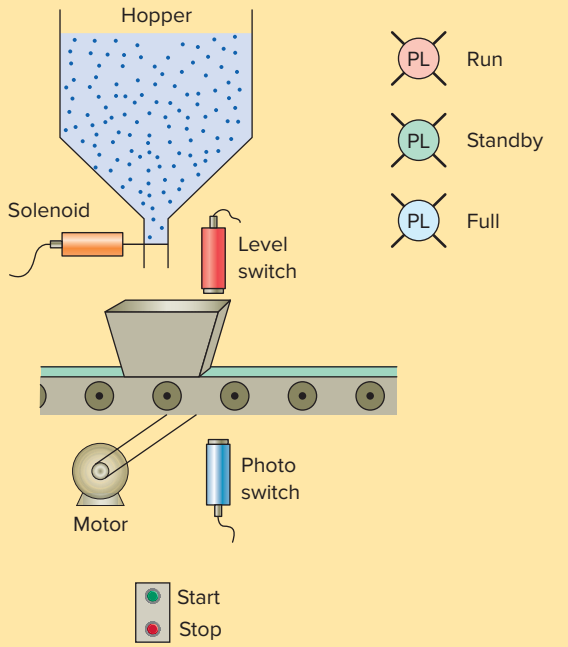


Figure 6-73 Sketch of the continuous filling operation.

Figure 6-74 shows the ladder logic program required for the operation.

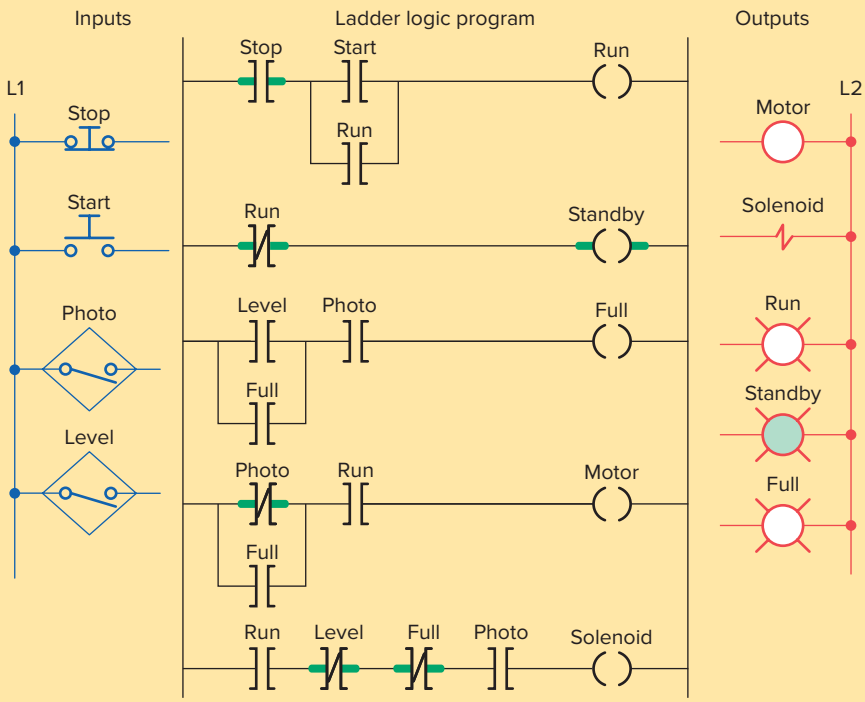


Figure 6-74 Continuous filling operation PLC program.

6.13 Instrumentation

Instrumentation is the use of measuring instruments to monitor and control a process. It involves the design and calibration of systems used to measure, record, and control industrial process variables. These variables may include pressure, temperature, flow rate, weight, and chemical consistency. An **instrument** is a device that measures and/or acts to control any kind of physical process and may include flow devices, level devices, thermocouples, and pressure switches.

Every instrument has at least one input and one output.

- For a pressure sensor, the input could be some fluid pressure and the output a 4- to 20-mA current signal.
- For a loop indicator, the input could be a 4- to 20-mA current signal and the output an electronic display.
- For a variable-speed motor drive, the input could be an electronic signal and the output electric power to the motor.

To **calibrate** an instrument means to check, and if necessary adjust, its response so the output accurately corresponds to its input throughout a specified range. Instrument calibration involves exposure of the instrument to an actual input stimulus of precisely known quantity. For a pressure gauge, this would mean subjecting the pressure instrument to known fluid pressures and comparing the instrument response against those known pressure quantities. **Smart instruments** (Figure 6-75) that contain microprocessors have built-in diagnostic ability, greater accuracy, and the ability to communicate digitally with host devices for reporting of various parameters.

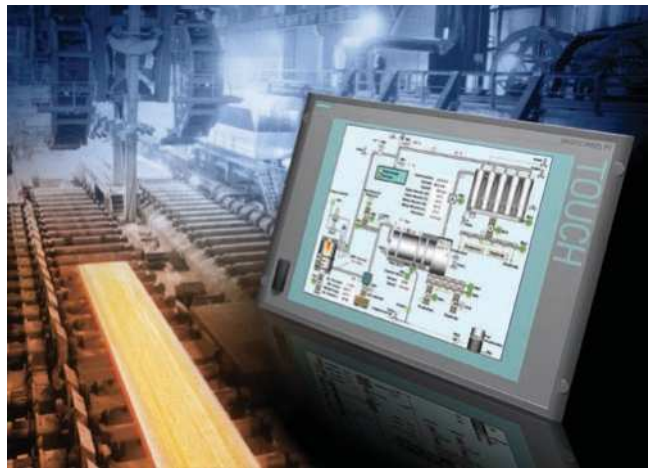


Figure 6-76 Process parameters display.
Source: Courtesy of Siemens.

The PLC's role as part of an industrial instrumentation system is to receive, process, and send signals from input and to output devices. With the use of programming software the PLC can control, monitor, and display all the parameters associated with a given process (Figure 6-76).

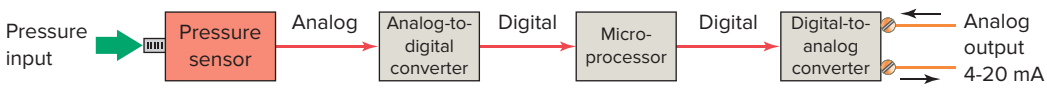


Figure 6-75 Smart instruments.
Source: Photo courtesy Emerson.

CHAPTER 6 REVIEW QUESTIONS

1. Explain the basic operating principle of an electromagnetic control relay.
2. What is the operating difference between a normally open and a normally closed relay contact?
3. In what ways are control relay coils and contacts rated?
4. How do contactors differ from relays?
5. What is the main difference between a contactor and a magnetic motor starter?
6. **a.** Draw the schematic for an across-the-line AC magnetic motor starter.
   b. With reference to this schematic, explain the function of each of the following parts:
   i. Main contact M
   ii. Control contact M
   iii. Starter coil M
   iv. OL relay coils
   v. OL relay contact
7. The current requirement for the control circuit of a magnetic starter is normally much smaller than that required by the power circuit. Why?
8. Compare the method of operation of each of the following types of switches:
   a. Manually operated switch
   b. Mechanically operated switch
   c. Proximity switch
9. What do the abbreviations NO and NC represent when used to describe switch contacts?
10. Draw the electrical symbol used to represent each of the following switches:
    a. NO pushbutton switch
    b. NC pushbutton switch
    c. Break-make pushbutton switch
    d. Three-position selector switch
    e. NO limit switch
    f. NC temperature switch
    g. NO pressure switch
    h. NC level switch
    i. NO proximity switch
11. Outline the method used to actuate inductive and capacitive proximity sensors.
12. How are reed switch sensors actuated?
13. Compare the operation of a photovoltaic solar cell with that of a photoconductive cell.
14. What are the two basic components of a photoelectric sensor?
15. Compare the operation of the reflective-type and through-beam photoelectric sensors.
16. Give an explanation of how a scanner and a decoder act in conjunction with each other to read a bar code.
17. How does an ultrasonic sensor operate?
18. Explain the principle of operation of a strain gauge.
19. Explain the principle of operation of a thermocouple.
20. What is the most common approach taken with regard to the measurement of fluid flow?
21. Explain how a tachometer is used to measure rotational speed.
22. How does an optical encoder work?
23. Draw an electrical symbol used to represent each of the following PLC control devices:
    a. Pilot light
    b. Relay
    c. Motor starter coil
    d. OL relay contact
    e. Alarm
    f. Heater
    g. Solenoid
    h. Solenoid valve
    i. Motor
    j. Horn
24. Explain the function of each of the following actuators:
    a. Solenoid
    b. Solenoid valve
    c. Stepper motor
25. Compare the operation of open-loop and closed-loop control.
26. What is a seal-in circuit?
27. In what way is the construction and operation of an electromechanical latching relay different from a standard relay?
28. Give a short description of each of the following control processes:
    a. Sequential
    b. Combination
    c. Automatic
29. Compare the type of sensor signal obtained from a thermocouple with that from an RTD.
30. Explain how a magnetic reed float switch works.
31. What is the function of an electrical interlocking circuit?
32. What is the role of instrumentation in an industrial process?
33. You have been assigned the task of calibrating an instrument. How would you proceed?

CHAPTER 6 PROBLEMS

1. Design and draw the schematic for a conventional hardwired relay circuit that will perform each of the following circuit functions when a normally closed pushbutton is pressed:
   - Switch a pilot light on
   - De-energize a solenoid
   - Start a motor running
   - Sound a horn
2. Design and draw the schematic for a conventional hardwired circuit that will perform the following circuit functions using two break-make pushbuttons:
   - Turn on light L1 when pushbutton PB1 is pressed.
   - Turn on light L2 when pushbutton PB2 is pressed.
   - Electrically interlock the pushbuttons so that L1 and L2 cannot both be turned on at the same time.
3. Study the ladder logic program in Figure 6-77, and answer the questions that follow:
   a. Under what condition will the latch rung 1 be true?
   b. Under what conditions will the unlatch rung 2 be true?
   c. Under what condition will rung 3 be true?
   d. When PL1 is on, the relay is in what state (latched or unlatched)?
   e. When PL2 is on, the relay is in what state (latched or unlatched)?
   f. If AC power is removed and then restored to the circuit, what pilot light will automatically come on when the power is restored?
   g. Assume the relay is in its latched state and all three inputs are false. What input change(s) must occur for the relay to switch into its unlatched state?
   h. If the examine if closed instructions at addresses I/1, I/2, and I/3 are all true, what state will the relay remain in (latched or unlatched)?
4. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program that will correctly execute the hardwired control circuit in Figure 6-78.
   Assume: Stop pushbutton used is an NO type.
   Run pushbutton used is an NO type.
   Jog pushbutton used has one set of NO contacts.
   OL contact is hardwired.
5. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program that will correctly execute the hardwired control circuit in Figure 6-79.
   Assume: PB1 pushbutton used is an NO type.
   PB2 pushbutton used is an NC type.
   PS1 pressure switch used is an NO type.
   LS1 limit switch used has only one set of NC contacts.

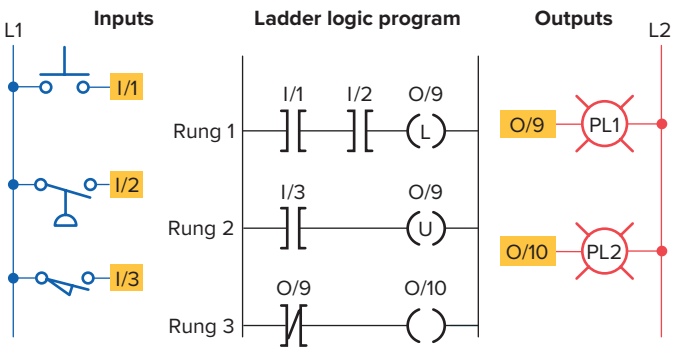


Figure 6-77 Ladder logic program for Problem 3.

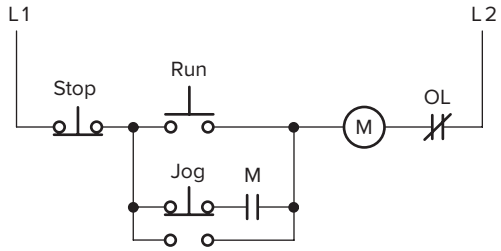


Figure 6-78 Hardwired control circuit for Problem 4.

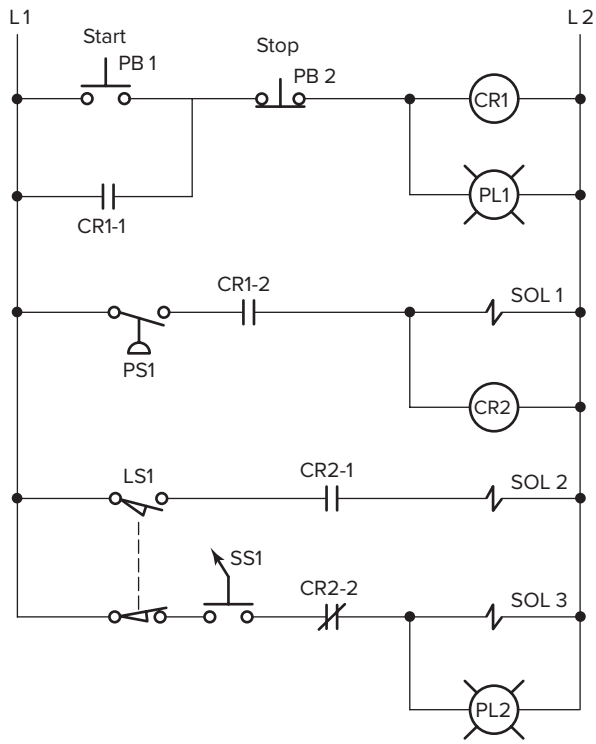


Figure 6-79 Hardwired control circuit for Problem 5.

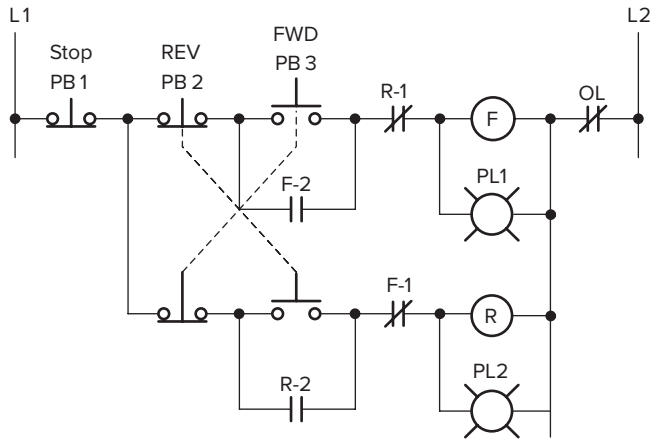


Figure 6-80 Hardwired control circuit for Problem 6.

6. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program that will correctly execute the hardwired control circuit in Figure 6-80.
Assume: PB1 pushbutton used is an NC type.
PB2 and PB3 are each wired using one set of NO contacts.
OL contact is hardwired.

7. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program for the following motor control specifications:
- A motor must be started and stopped from any one of three start/stop pushbutton stations.
- Each start/stop station contains one NO start pushbutton and one NC stop pushbutton.
- Motor OL contacts are to be hardwired.

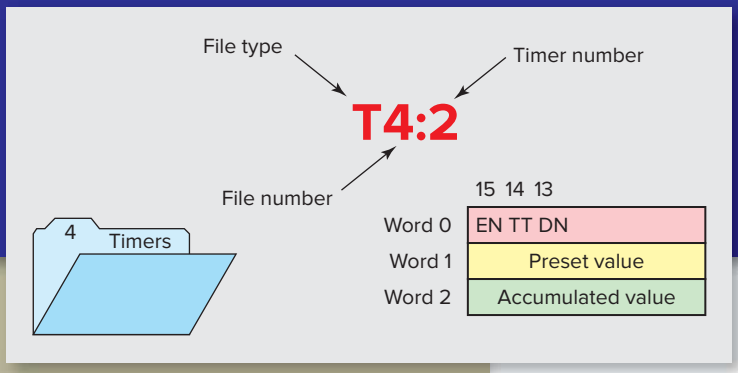
8. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program for the following motor control specifications:
- Three starters are to be wired so that each starter is operated from its own start/stop pushbutton station.
- A master stop station is to be included that will trip out all starters when pushed.
- Overload relay contacts are to be programmed so that an overload on any one of the starters will automatically drop all of the starters.
- *All* pushbuttons are to be wired using one set of NO contacts.

9. A temperature control system consists of four thermostats controlling three heating units. The thermostat contacts are set to close at 50°, 60°, 70°, and 80°F, respectively. The PLC ladder logic program is to be designed so that at a temperature below 50°F, three heaters are to be ON. From 50° to 60°F, two heaters are to be ON. For 60° to 70°F, one heater is to be ON. Above 80°F, there is a safety shutoff for all three heaters in case one stays on because of a malfunction. A master switch is to be used to turn the system ON and OFF. Prepare a typical PLC program for this control process.

10. A pump is to be used to fill two storage tanks. The pump is manually started by the operator from a start/stop station. When the first tank is full, the control logic must be able to automatically stop flow to the first tank and direct flow to the second tank through the use of sensors and electric solenoid valves. When the second tank is full, the pump must shut down automatically. Indicator lamps are to be included to signal when each tank is full.
a. Draw a sketch of the process.
b. Prepare a typical PLC program for this control process.

11. Write the optimum ladder logic rung for each of the following scenarios, and arrange the instructions for optimum performance:
a. If limit switches LS1 or LS2 or LS3 are on, or if LS5 and LS7 are on, turn on; otherwise, turn off. (Commonly, if LS5 and LS7 are on, the other conditions rarely occur.)
b. Turn on an output when switches SW6, SW7, and SW8 are all on, or when SW55 is on. (SW55 is an indication of an alarm state, so it is rarely on; SW7 is on most often, then SW8, then SW6.)

7
Programming Timers



Chapter Objectives

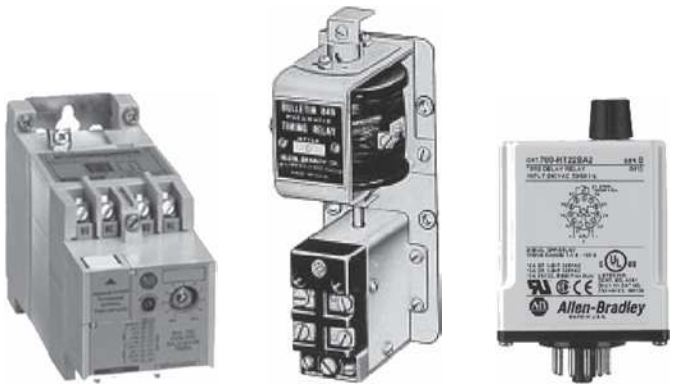
After completing this chapter, you will be able to:

- Describe the operation of pneumatic on-delay and off-delay timers
- Describe PLC timer instruction and differentiate between a nonretentive and retentive timer
- Convert fundamental timer relay schematic diagrams to PLC ladder logic programs
- Analyze and interpret typical PLC timer ladder logic programs
- Program the control of outputs using the timer instruction control bits

The most commonly used PLC instruction, after coils and contacts, is the timer. This chapter deals with how timers time intervals and the way in which they can control outputs. We discuss the basic PLC on-delay timer function, as well as other timing functions derived from it, and typical industrial timing tasks.

7.1 Mechanical Timing Relays

There are very few industrial control systems that do not need at least one or two timed functions. **Mechanical timing relays** are used to delay the opening or closing of contacts for circuit control. The operation of a mechanical timing relay is similar to that of a control relay, except that certain of its contacts are designed to operate at a preset time interval, after the coil is energized or de-energized. Typical types of mechanical and electronic timing relays are shown in Figure 7-1. Timers allow a multitude of operations in a control circuit to be automatically started and stopped at different time intervals.



Solid-state timing relay Pneumatic timing relay Plug-in timing relay

Figure 7-1 Timing relays.
Source: Image Courtesy of Rockwell Automation, Inc.

Figure 7-2 shows the construction of an on-delay pneumatic (air) timer. The time-delay function depends on the transfer of air through a restricted orifice. The time-delay period is adjusted by positioning the needle valve to vary the amount of orifice restriction. When the coil is energized, the timed contacts are delayed from opening or closing. However, when the coil is de-energized, the timed contacts return instantaneously to their normal state. This particular pneumatic timer has **instantaneous contacts** in addition to timed contacts. The instantaneous contacts change state as soon as the timer coil is powered while the delayed contacts change state at the end of the time delay. Instantaneous contacts are often used as holding or sealing contacts in a control circuit.

Mechanical timing relays provide time delay through two arrangements. The first arrangement, *on delay,* provides time delay when the relay coil is *energized.* The second arrangement, *off delay,* provides time delay when the relay coil is *de-energized.* Figure 7-3 illustrates the different relay symbols used for timed contacts.

The **on-delay timer** is sometimes referred to as DOE, which stands for delay on energize. The time delay of the contacts begins once the timer is switched on; hence the term *on-delay timing.* Figure 7-4 shows an on-delay timer circuit that uses a normally open, timed closed (NOTC) contact. The operation of the circuit can be summarized as follows:

- With S1 initially open, TD coil is de-energized so TD1 contacts are open and light L1 will be off.

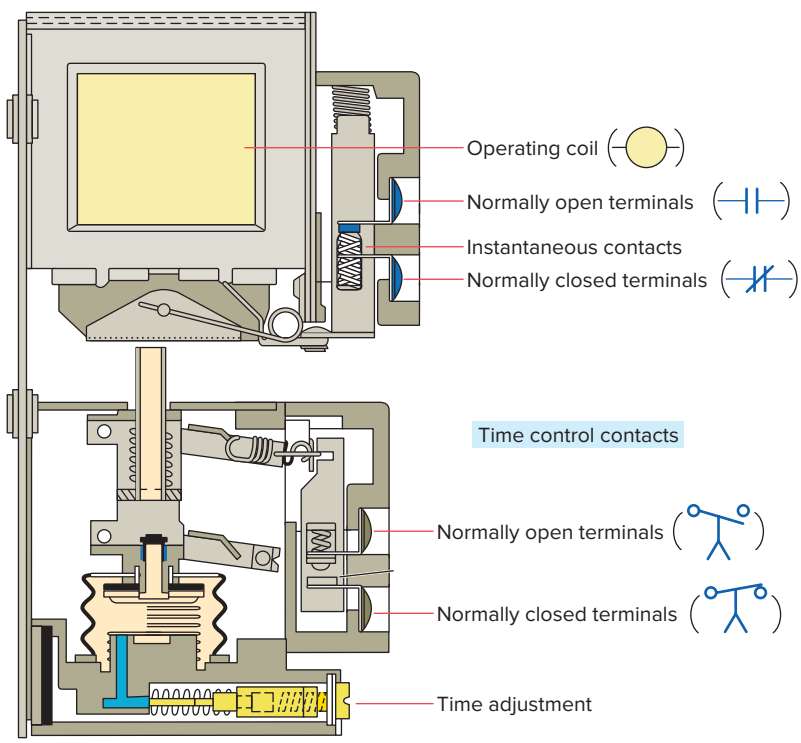
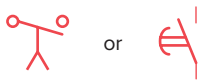
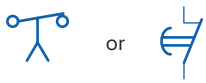


Figure 7-2 Pneumatic on-delay timer.

On-delay symbols	
	
Normally open, timed closed contact (NOTC). Contact is open when relay coil is de-energized. When relay is energized, there is a time delay in closing.	Normally closed, timed open contact (NCTO). Contact is closed when relay coil is de-energized. When relay is energized, there is a time delay in opening.

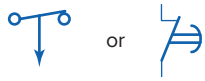
Off-delay symbols	
	
Normally open, timed open contact (NOTO). Contact is normally open when relay coil is de-energized. When relay coil is energized, contact closes instantly. When relay coil is de-energized, there is a time delay before the contact opens.	Normally closed, timed closed contact (NCTC). Contact is normally closed when relay coil is de-energized. When relay coil is energized, contact opens instantly. When relay coil is de-energized, there is a time delay before the contact closes.

Figure 7-3 Timed contact symbols.

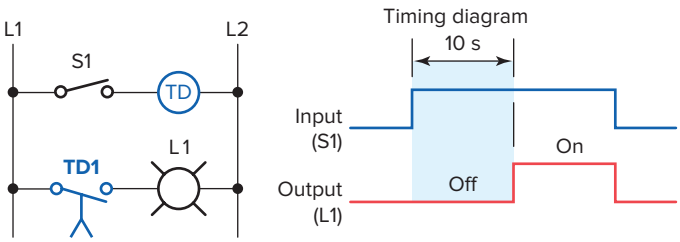


Figure 7-4 On-delay timer circuit that uses a normally open, timed closed (NOTC) contact.

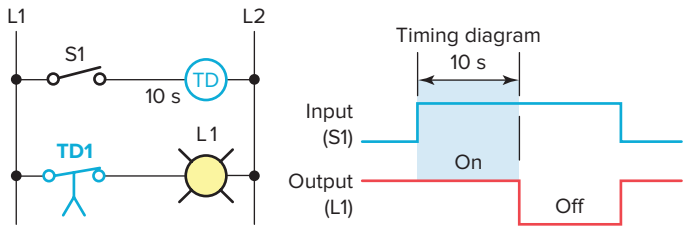


Figure 7-5 On-delay timer circuit that uses a normally closed, timed open (NCTO) contact.

- When S1 is closed, TD coil is energized and the timing period starts. TD1 contacts are delayed from closing so L1 remains off.
- After the 10 s time-delay period has elapsed, TD1 contacts close and L1 is switched on.
- When S1 is opened, TD coil is de-energized and TD1 contacts open instantly to switch L1 off.

Figure 7-5 shows an on-delay timer circuit that uses a normally closed, timed open (NCTO) contact. The operation of the circuit can be summarized as follows:

- With S1 initially open, TD coil is de-energized so TD1 contacts are closed and light L1 will be on.
- When S1 is closed, TD coil is energized and the timing period starts. TD1 contacts are delayed from opening so L1 remains on.
- After the 10 s time-delay period has elapsed, TD1 contacts open and L1 is switched off.
- When S1 is opened, TD coil is de-energized and TD1 contacts close instantly to switch L1 on.

Figure 7-6 shows an **off-delay timer** circuit that uses a normally open, timed open (NOTO) contact. The operation of the circuit can be summarized as follows:

- With S1 initially open, TD coil is de-energized so TD1 contacts are open and light L1 will be off.
- When S1 is closed, TD coil is energized and TD1 contacts close instantly to switch light L1 on.
- When S1 is opened, TD coil is de-energized and the timing period starts.
- After the 10 s time-delay period has elapsed, TD1 contacts open to switch the light off.

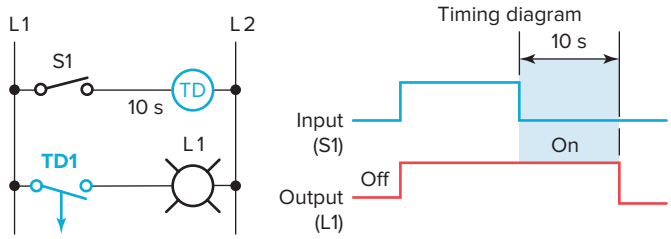


Figure 7-6 Off-delay timer circuit that uses a normally open, timed open (NOTO) contact.

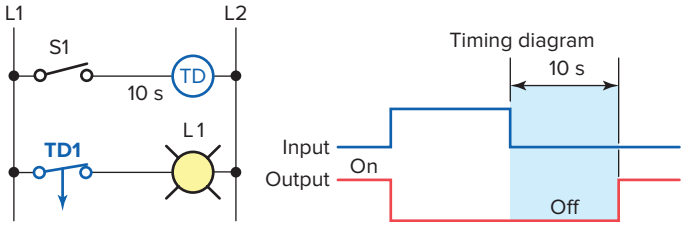


Figure 7-7 Off-delay timer circuit that uses a normally closed, timed closed (NCTC) contact.

Figure 7-7 shows an off-delay timer circuit that uses a normally closed, timed closed (NCTC) contact. The operation of the circuit can be summarized as follows:

- With S1 initially open, TD coil is de-energized so TD1 contacts are closed and light L1 will be on.
- When S1 is closed, TD coil is energized and TD1 contacts open instantly to switch light L1 off.
- When S1 is opened, TD coil is de-energized and the timing period starts. TD1 contacts are delayed from closing so L1 remains off.
- After the 10 s time-delay period has elapsed, TD1 contacts close to switch the light on.

7.2 Timer Instructions

PLC timers are instructions that provide the same functions as on-delay and off-delay mechanical and electronic timing relays. All PLC timers are output instructions. PLC timers offer several advantages over their mechanical and electronic counterparts. These include the fact that:

- The entire timing function occurs inside the controller.
- Time settings can be easily changed.
- The number of timers used in a circuit can be increased or decreased through the use of programming changes rather than wiring changes.
- Timer accuracy and repeatability are extremely high because time delays are generated in the PLC processor.

In general, there are three different PLC timer types: the *on-delay timer (**TON**), off-delay timer (**TOF**),* and *retentive timer on (**RTO**).* The most common is the on-delay timer, which is the basic function. There are also many other timing configurations, all of which can be derived from one or more of the basic time-delay functions. Figure 7-8 shows the timer selection toolbar for the Allen-Bradley SLC 500 PLC and its associated RSLogix software. These timer commands can be summarized as follows:

TON (Timer On Delay)—Counts time-based intervals when the instruction is true.

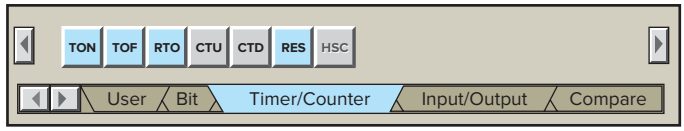


Figure 7-8 Timer selection toolbar.

TOF (Timer Off Delay)—Counts time-based intervals when the instruction transitions from a true to false condition.

RTO (Retentive Timer On)—Counts time-based intervals when the instruction is true and retains the accumulated value when the instruction goes false or when power cycle occurs.

RES (Reset)—Resets a retentive timer's accumulated value to zero.

Several quantities are associated with the timer instruction:

- **Time Base** The time base of a timer is the unit of time used by a timer to time an event. A timer instruction times an event by counting the number of times the time base has occurred. Depending on the manufacturer and type of PLC, time base values can be in 1 ms (0.001 s), 10 ms (0.01 s), 100 ms (0.1 s), or 1 second intervals. For example, if a timer has a time base of 1 second and it is timing something that is 5 seconds long, the PLC will wait until the time base has occurred 5 times before the timer times out. Conversely, if the PLC's time base setting is 0.01 seconds, it will wait until the time base has occurred 500 times before timing out. The smaller the time base, the better the accuracy of the timer.
- **Preset Value** The preset value of a timer represents the time duration for the timing circuit. Total timing interval = the preset value x time base. For example, for a timer with a preset value of 100 and a time base of 0.1s the time duration for the timer is:
  Total timing interval = 100×0.1 s
  = 10 seconds
- **Accumulated Value** The accumulated value of the timer represents the amount of time that has elapsed from the moment the timing started. It keeps track of how many times the time base has occurred since the timer instruction was initiated.

Although each manufacturer may represent timers differently on the ladder logic program, most timers operate in a similar manner. One of the first methods used depicts the timer instruction as a relay coil similar to that of a mechanical timing relay. Figure 7-9 shows a **coil-formatted timer** instruction.

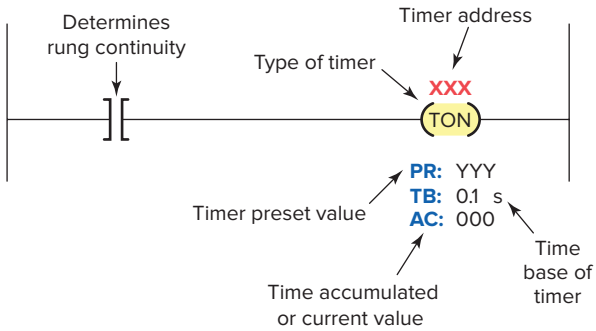


Figure 7-9 Coil-formatted timer instruction.

Timers are most often represented by boxes in ladder logic. Figure 7-10 illustrates a generic **block format** for a retentive timer that requires two input lines. Its operation can be summarized as follows:

- The timer block has two input conditions associated with it, namely, the *control* and *reset.*
- The control line controls the actual timing operation of the timer. Whenever this line is true or power is supplied to this input, the timer will time. Removal of power from the control line input halts the further timing of the timer.
- The reset line resets the timer's accumulated value to zero.
- Some manufacturers require that *both* the control and reset lines be true for the timer to time; removal of power from the reset input resets the timer to zero.
- Other manufacturers' PLCs require power flow for the control input only and no power flow on the reset input for the timer to operate. For this type of timer operation, the timer is reset whenever the reset input is true.
- The timer instruction block contains information pertaining to the operation of the timer, including the preset time, the time base of the timer, and the current or accumulated time.

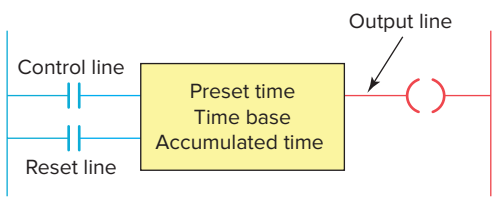


Figure 7-10 Block-formatted timer instruction.

- All block-formatted timers provide at least one output signal from the timer. The timer continuously compares its current time with its preset time, and its output is false (logic 0) as long as the current time is less than the preset time. When the current time equals the preset time, the output changes to true (logic 1).

7.3 On-Delay Timer Instruction

Most timers are output instructions that are conditioned by input instructions. An ***on-delay timer*** is used when you want to program a time delay before an instruction becomes true. Figure 7-11 illustrates the principle of operation of an on-delay timer. Its operation can be summarized as follows:

- The on-delay timer operates such that when the rung containing the timer is true, the timer time-out period commences.
- The timed output becomes true sometime after the timer rung becomes true; hence, the timer is said to have an on-delay.
- The length of the time delay can be adjusted by changing the preset value.
- In addition, some PLCs allow the option of changing the time base, or resolution, of the timer. As the time base you select becomes smaller, the accuracy of the timer increases.

The Allen-Bradley SLC 500 timer file is file 4 (Figure 7-12). Each timer is composed of three 16-bit words, collectively called a timer element. There can be

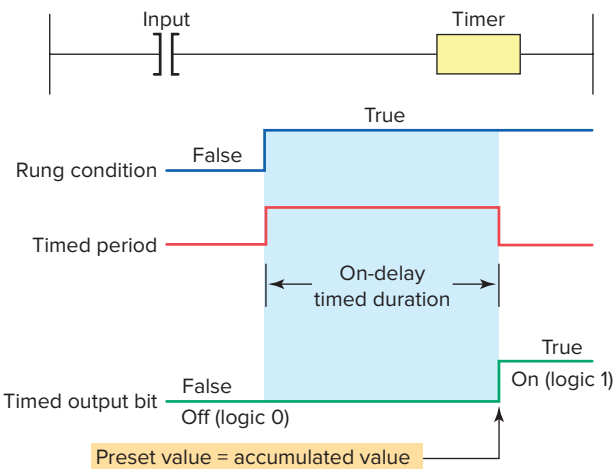


Figure 7-11 Principle of operation of an on-delay timer.

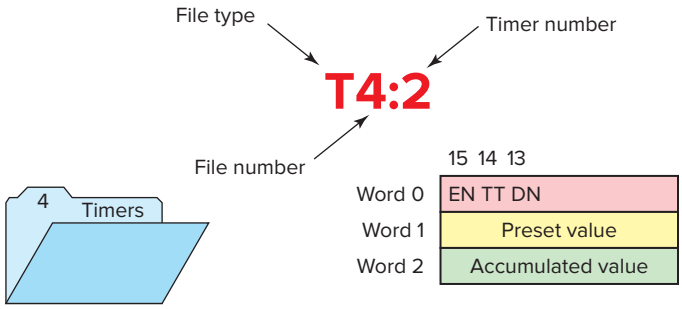


Figure 7-12 SLC 500 timer file.

up to 256 timer elements. Addresses for timer file 4, timer element number 2 (T4:2), are listed below.

T4 = timer file 4

:2 = timer element number 2 (0–255 timer elements per file)

T4:2/DN is the address for the done bit of the timer.

T4:2/TT is the address for the timer-timing bit of the timer.

T4:2/EN is the address for the enable bit of the timer.

The *control word* uses the following three control bits:

Enable (EN) bit—The enable bit is true (has a status of 1) whenever the timer instruction is true. When the timer instruction is false, the enable bit is false (has a status of 0).

Timer-timing (TT) bit—The timer-timing bit is true whenever the accumulated value of the timer is changing, which means the timer is timing. When the timer is not timing, the accumulated value is not changing, so the timer-timing bit is false.

Done (DN) bit—The done bit changes state whenever the accumulated value reaches the preset value. Its state depends on the type of timer being used.

The *preset value (PRE) word* is the set point of the timer, that is, the value up to which the timer will time. The preset word has a range of 0 through 32,767 and is stored in binary form. The preset will not store a negative number.

The *accumulated value (ACC) word* is the value that increments as the timer is timing. The accumulated value will stop incrementing when its value reaches the preset value.

The timer instruction also requires that you enter a *time base,* which is either 1.0 or 0.01 s. The actual preset time interval is the time base multiplied by the value stored in the timer's preset word. The actual accumulated time interval is the time base multiplied by the value stored in the timer's accumulated word.

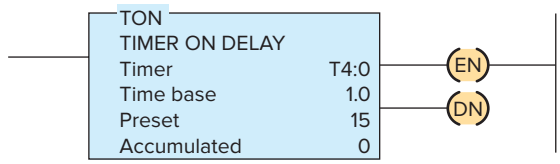


Figure 7-13 On-delay timer instruction.

Figure 7-13 shows an example of the on-delay timer instruction used as part of the Allen-Bradley SLC 500 controller instruction sets. The information to be entered includes:

Timer number—This number must come from the timer file. In the example shown, the timer number is T4:0, which represents timer file 4, timer 0 in that file. The timer address must be unique for this timer and may not be used for any other timer.

Time base—The time base (which is always expressed in seconds) may be either 1.0 or 0.01 s. In the example shown, the time base is 1.0 s.

Preset value—In the example shown, the preset value is 15. The timer preset value can range from 0 through 32,767.

Accumulated value—In the example shown, the accumulated value is 0. The timer's accumulated value normally is entered as 0, although it is possible to enter a value from 0 through 32,767. Regardless of the value that is preloaded, the timer value will become 0 whenever the timer is reset.

The timer instruction for the SLC 500 and ControlLogix 5000 processors operates in exactly the same manner. Figure 7-14 shows the timer selection toolbar, on-delay timer instruction, and expanded timer structure for a ControlLogix controller.

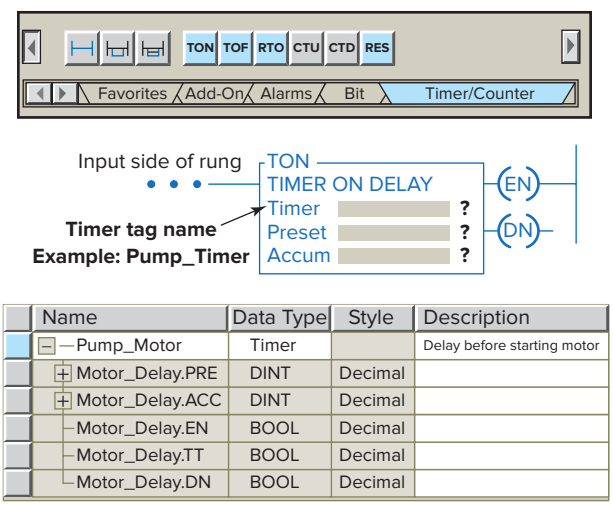


Figure 7-14 ControlLogix timer instruction.

- Logix processors use a tag name, such as Pump_ Motor, instead of a timer number.
- This descriptive tag name makes it easier to know what function the timer serves in the control system.
- The time base is fixed at 0.001 s (1 ms). Therefore there is no parameter field.
- The associated timer data (PRE, ACC, EN, TT, DN) are found within the expanded timer structure.

The on-delay timer (TON) is the most commonly used timer. Figure 7-15 shows a PLC program that uses an on-delay timer. The operation of the program can be summarized as follows:

- The timer is activated by input switch A.

Timer ON DELAY (TON)		
TON Instruction OFF	Enable Bit (EN)	0
	Timer Timing Bit (TT)	0
	Done Bit (DN)	0
	Accumulating	NO
TON Instruction ON	Enable Bit (EN)	1
	Timer Timing Bit (TT)	1
	Done Bit (DN)	0
	Accumulating	YES
Timed Out Accum = Preset	Enable Bit (EN)	1
	Timer Timing Bit (TT)	0
	Done Bit (DN)	1
	Accumulating	NO
Instruction OFF after timed out	Enable Bit (EN)	0
	Timer Timing Bit (TT)	0
	Done Bit (DN)	0
	Accumulating	Reset

Table showing how each bit is effected during the program operation.

- When input switch A is closed (true or set to 1), the processor starts timer T4:0 timing and sets the EN and TT bits to true or 1.
- This turns ON outputs B and C
- The accumulated value increases in one-second time base intervals.
- When the accumulated time equals the preset time (10 s), the DN bit is set to 1, output D is turned ON, the TT bit is reset to 0 and output C is turned OFF.
- As long as input switch A remains closed the EN bit is set to 1 and output B will be ON.
- If input switch A is opened at any time before or after the timer has timed out, the accumulated time is automatically reset to 0 and output B is turned OFF.
- This timer configuration is termed *nonretentive* because any loss of continuity to the timer causes the timer instruction to reset.
- This timing operation is that of an on-delay timer because output D is switched on 10 s after the switch has been actuated from the off to the on position.

Figure 7-16 shows the timing diagram for the on-delay timer's control bits. The sequence of operation is as follows:

- The first true period of the timer rung shows the timer timing to 4 s and then going false.
- The timer resets, and both the timer-timing bit and the enable bit go false. The accumulated value also resets to 0.
- For the second true period input *A* remains true in excess of 10 s.

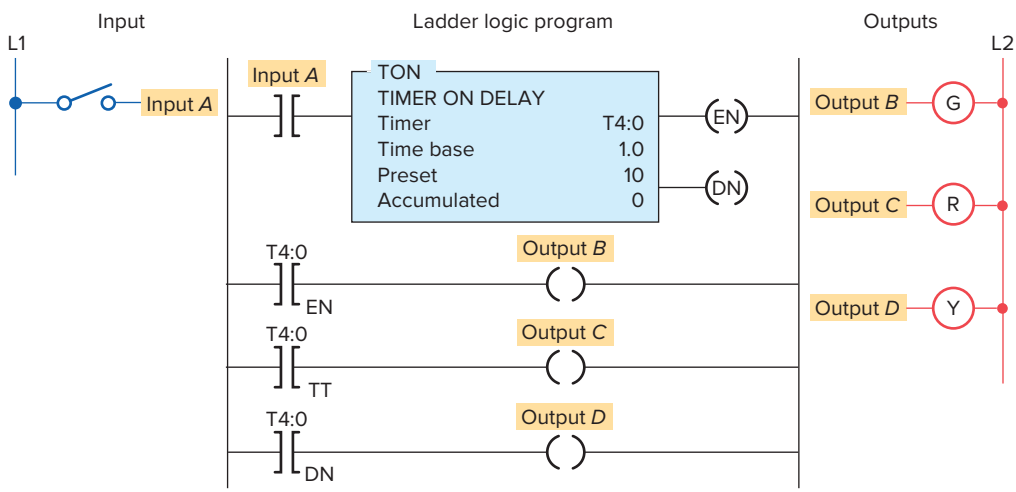


Figure 7-15 PLC on-delay timer program.

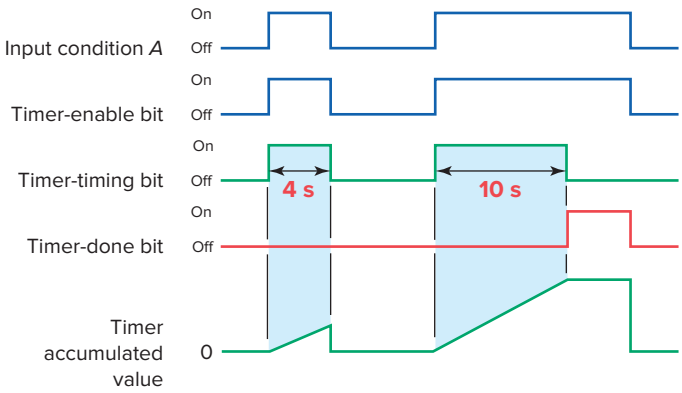


Figure 7-16 Timing diagram for an on-delay timer.

- When the accumulated value reaches 10 s, the done bit (DN) goes from false to true and the timer-timing bit (TT) goes from true to false.
- When input *A* goes false, the timer instruction goes false and also resets, at which time the control bits are all reset and the accumulated value resets to 0.

The timer table for an Allen-Bradley SLC 500 is shown in Figure 7-17. Addressing is done at three different levels: the element level, the word level, and the bit level. The timer uses three words per element. Each element consists of a control word, a preset word, and an accumulated word. Each word has 16 bits, which are numbered from 0 to 15. When addressing to the bit level, the address always refers to the bit within the word:

EN = Bit 15 enable
TT = Bit 14 timer timing
DN = Bit 13 done

Timer Table

	/EN	/TT	/DN	.PRE	.ACC
T4:0	0	0	0	10	0
T4:1	0	0	0	0	0
T4:2	0	0	0	0	0
T4:3	0	0	0	0	0
T4:4	0	0	0	0	0
T4:5	0	0	0	0	0

Address: T4:0 Table: T4: Timer

Figure 7-17 SLC 500 timer table.

Timers may or may not have an instantaneous output (also known as the enable bit) signal associated with them. If an instantaneous output signal is required from a timer and it is not provided as part of the timer instruction, an equivalent instantaneous contact instruction can be programmed using an internally referenced relay coil. Figure 7-18 shows an application of this technique. The operation of the program can be summarized as follows:

- According to the hardwired relay circuit diagram, coil M is to be energized 5 s after the start pushbutton is pressed.
- Contact TD-1 is the instantaneous contact, and contact TD-2 is the timed contact.
- The ladder logic program shows that a contact instruction referenced to an internal relay is now used to operate the timer.
- The instantaneous contact is referenced to the internal relay coil, whereas the time-delay contact is referenced to the timer output coil.

Figure 7-19 shows an application for an on-delay timer that uses an NCTO contact. This circuit is used as a warning signal when moving equipment, such as a conveyor motor, is about to be started. The operation of the circuit can be summarized as follows:

- According to the hardwired relay circuit diagram, coil CR is energized when the start pushbutton PB1 is momentarily actuated.
- As a result, contact CR-1 closes to seal in CR coil, contact CR-2 closes to energize timer coil TD, and contact CR-3 closes to sound the horn.
- After a 10-s time-delay period, timer contact TD-1 opens to automatically switch the horn off.
- The ladder logic program shows how an equivalent circuit could be programmed using a PLC.
- The logic on the last rung is the same as the timer-timing bit and as such can be used with timers that do not have a timer-timing output.

Timers are often used as part of **automatic sequential control** systems. Figure 7-20 shows how a series of motors can be started automatically with only one start/stop control station. The operation of the circuit can be summarized as follows:

- According to the relay ladder schematic, lube-oil pump motor starter coil M1 is energized when the start pushbutton PB2 is momentarily actuated.
- As a result, M1-1 control contact closes to seal in M1, and the lube-oil pump motor starts.

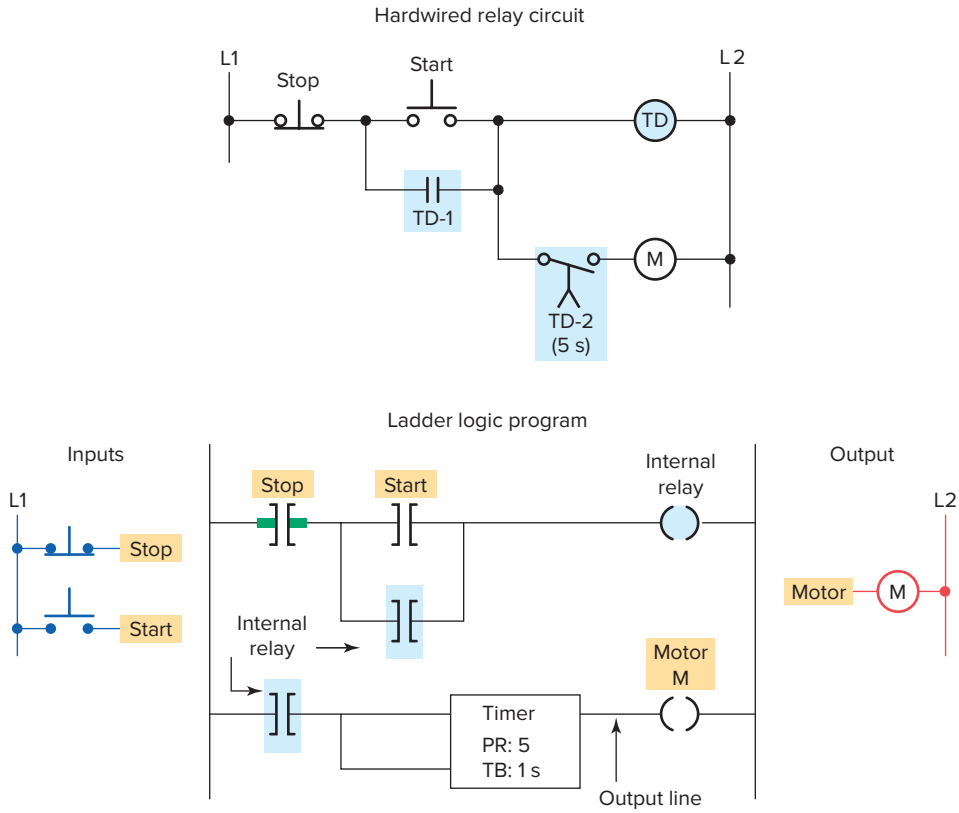


Figure 7-18 Instantaneous contact instruction can be programmed using an internally referenced relay coil.

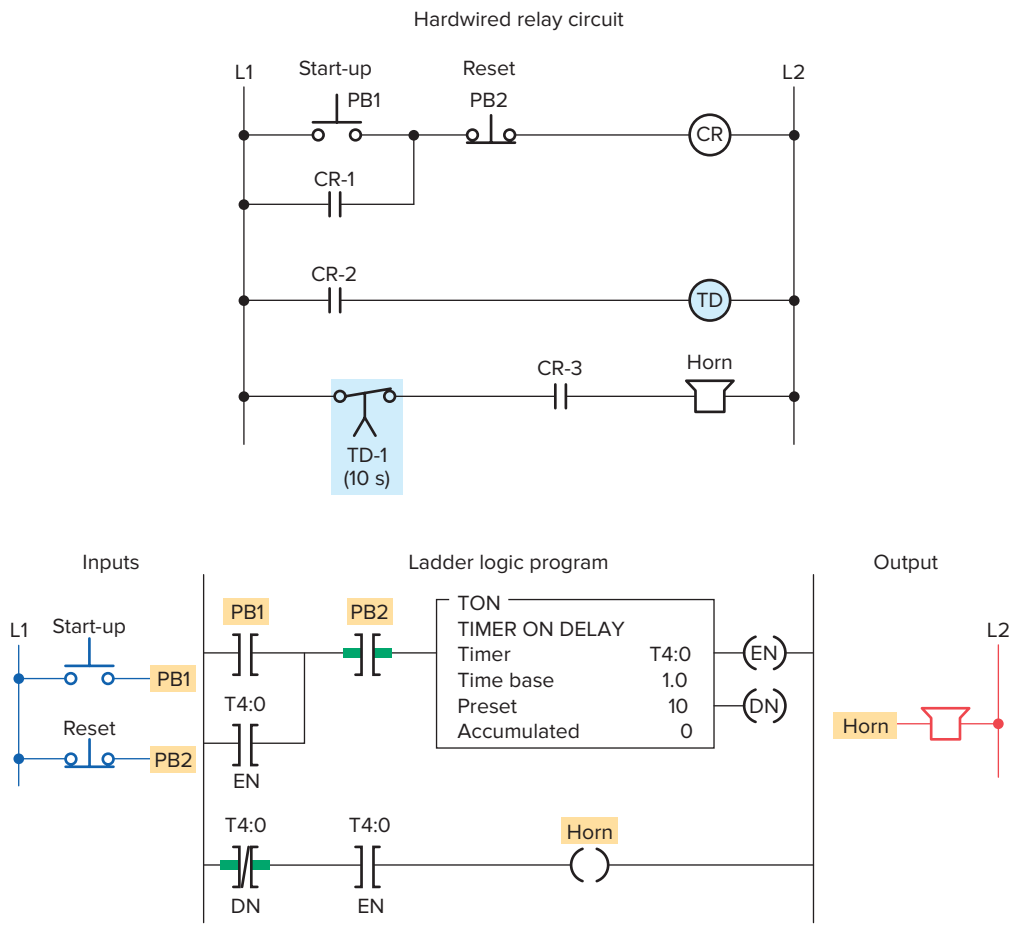


Figure 7-19 Conveyor warning signal circuit.

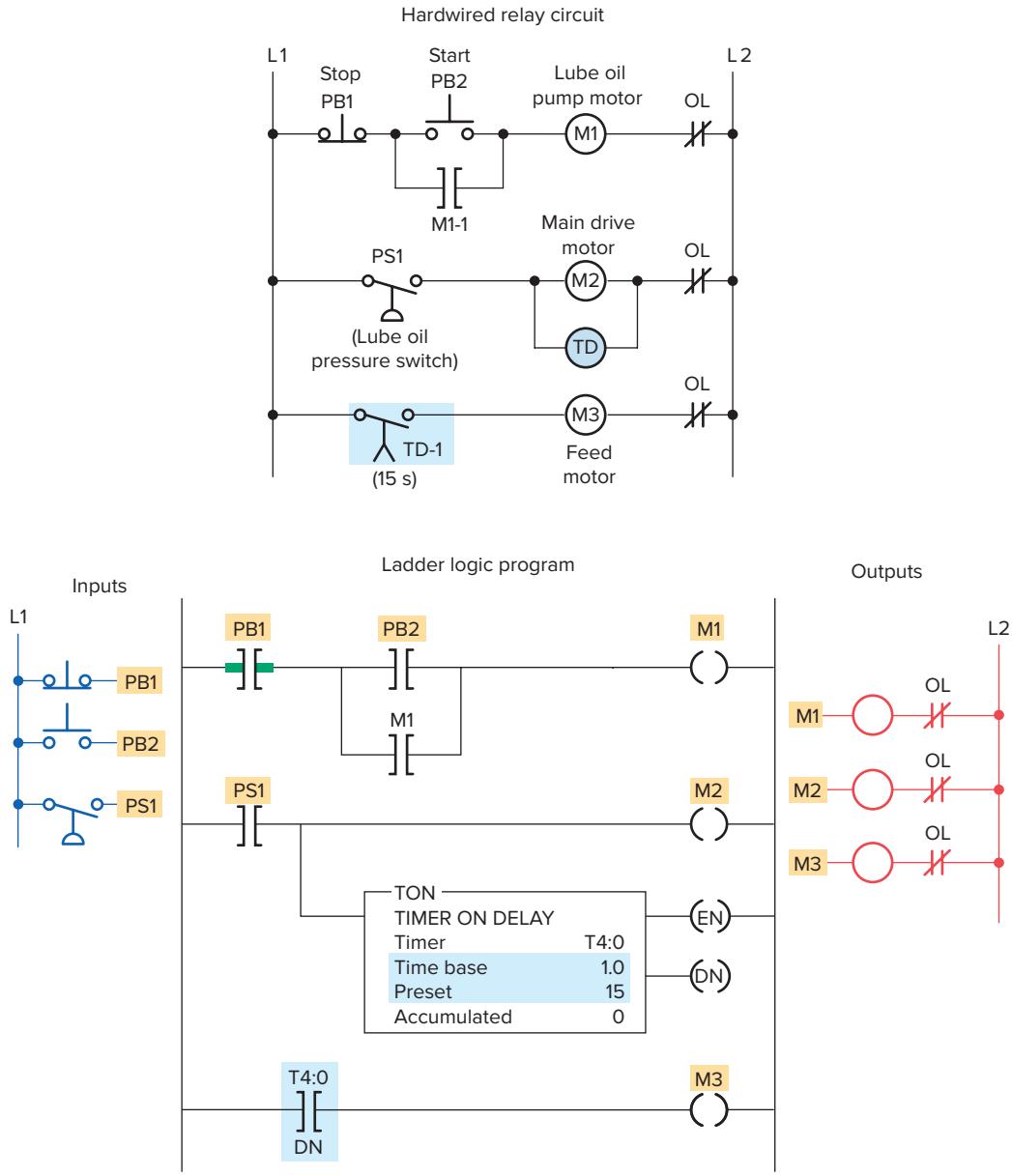


Figure 7-20 Automatic sequential control system.

- When the lube-oil pump builds up sufficient oil pressure, the lube-oil pressure switch PS1 closes.
- This in turn energizes coil M2 to start the main drive motor and energizes coil TD to begin the time-delay period.
- After the preset time-delay period of 15 s, TD-1 contact closes to energize coil M3 and start the feed motor.
- The ladder logic program shows how an equivalent circuit could be programmed using a PLC. The enable bit is used to seal in the timer so it continues to time until its preset value equals the accumulated value. The program sequence is reset by actuating the reset button.

7.4 Off-Delay Timer Instruction

The ***off-delay timer (TOF)*** operation will keep the output energized for a time period after the rung containing the timer has gone false. Figure 7-21 illustrates the programming of an off-delay timer that uses the SLC 500 TOF timer instruction. TOF starts timing when

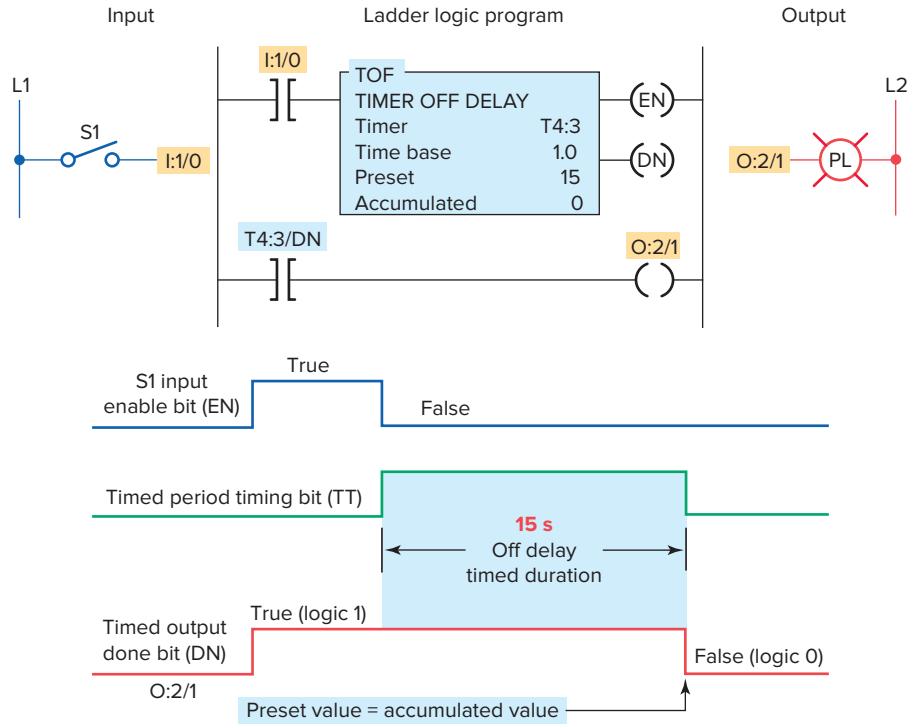


Figure 7-21 Off-delay programmed timer.

the instruction goes from ON to OFF or from true to false. The operation of the circuit can be summarized as follows:

- When the switch connected to input I:1/0 is first closed, timed output O:2/1 is set to 1 immediately and the lamp is switched on.
- If this switch is now opened, logic continuity is lost and the timer begins counting.
- After 15 s, when the accumulated time equals the preset time, the output is reset to 0 and the lamp switches off.
- If logic continuity is gained before the timer is timed out, the accumulated time is reset to 0. For this reason, this timer is also classified as nonretentive.

Figure 7-22 illustrates the use of an off-delay timer instruction used to switch motors *off* sequentially at 5 second intervals. The operation of the program can be summarized as follows:

- Timer preset values for T4:1, T4:2, and T4:3 are set for 5, 10, and 15 s, respectively.
- Closing the input switch SW immediately sets the done bit of each of the three off-delay timers to 1, immediately turning on motors M1, M2, and M3.
- If SW is then opened, logic continuity to all three timers is lost and each timer begins counting.
- Timer T4:1 times out after 5 s resetting its done bit to zero to de-energize motor M1.

Timer OFF DELAY (TOF)		
TOF Instruction ON	Enable Bit (EN)	1
	Timer Timing Bit (TT)	0
	Done Bit (DN)	1
	Accumulating	NO
TOF Instruction OFF	Enable Bit (EN)	0
	Timer Timing Bit (TT)	1
	Done Bit (DN)	1
	Accumulating	YES
Timed Out Accum = Preset	Enable Bit (EN)	0
	Timer Timing Bit (TT)	0
	Done Bit (DN)	0
	Accumulating	NO
Instruction OFF after timed out	Enable Bit (EN)	1
	Timer Timing Bit (TT)	0
	Done Bit (DN)	1
	Accumulating	Reset

Table showing how each bit is effected during the program operation.

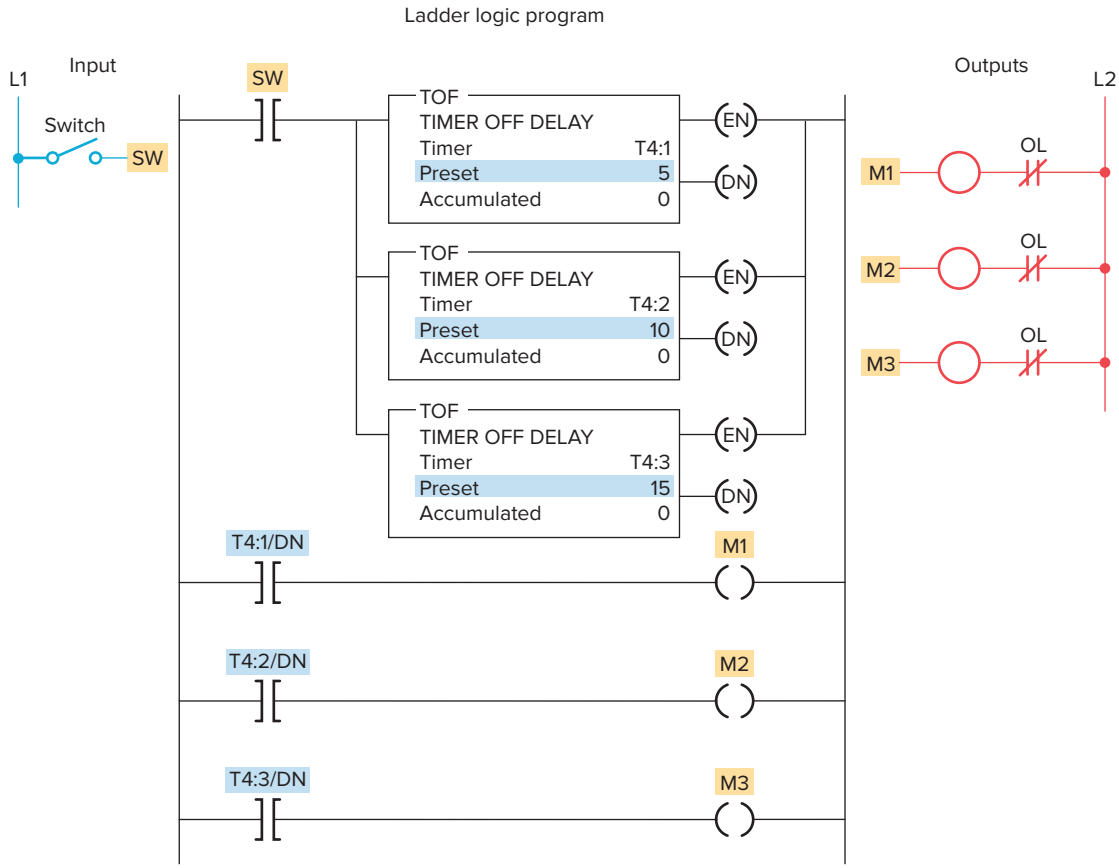


Figure 7-22 Program for switching motors off at 5 s intervals.

- Timer T4:2 times out 5 s later resetting its done bit to zero to de-energize motor M2.
- Timer T4:3 times out 5 s later resetting its done bit to zero to de-energize motor M3.

Figure 7-23 shows a hardwired off-delay timer relay circuit with both instantaneous and timed contacts. The operation of the circuit can be summarized as follows:

- When power is first applied (limit switch LS open), motor starter coil M1 is energized and the green pilot light is on.
- At the same time, motor starter coil M2 is de-energized, and the red pilot light is off.
- When limit switch LS closes, off-delay timer coil TD energizes.
- As a result, timed contact TD-1 opens to de-energize motor starter coil M1, timed contact TD-2 closes to energize motor starter coil M2, instantaneous contact TD-3 opens to switch the green light off, and instantaneous contact TD-4 closes to switch the red light on. The circuit remains in this state as long as limit switch LS1 is closed.
- When limit switch LS1 is opened, the off-delay timer coil TD de-energizes and the time-delay period is started.

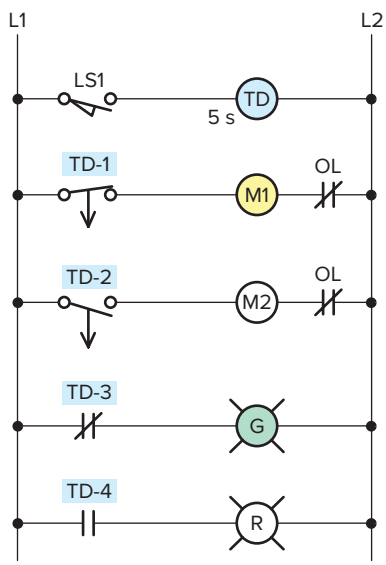


Figure 7-23 Hardwired off-delay timer relay circuit with both instantaneous and timed contacts.

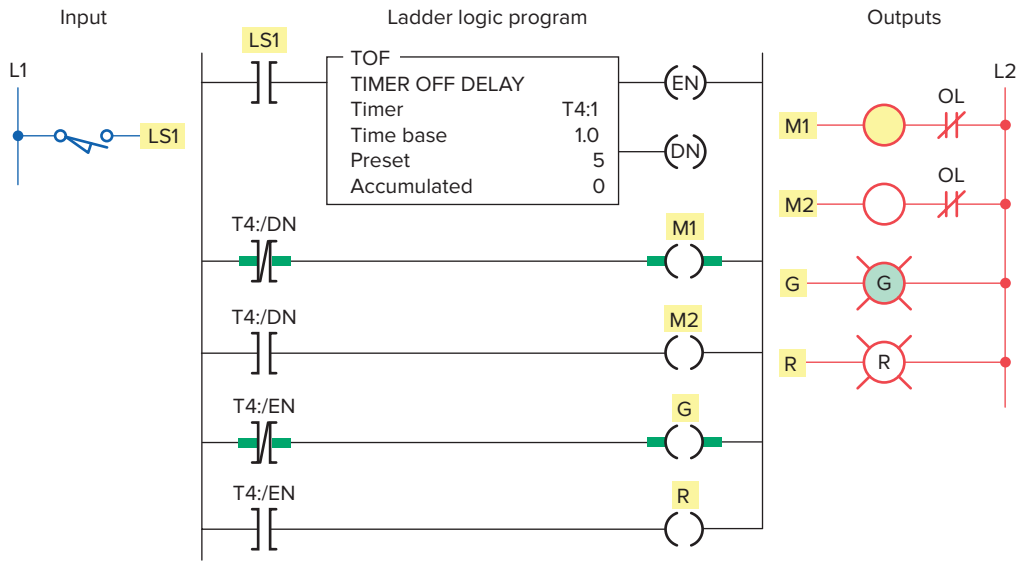


Figure 7-24 Equivalent PLC program of the hardwired off-delay timer relay circuit containing both instantaneous and timed contacts.

- Instantaneous contact TD-3 closes to switch the green light on, and instantaneous contact TD-4 opens to switch the red light off.
- After a 5-s time-delay period, timed contact TD-1 closes to energize motor starter M1, and timed contact TD-2 opens to de-energize motor starter M2.

Figure 7-24 shows an equivalent PLC program of the hardwired off-delay timer relay circuit containing both instantaneous and timed contacts. The timer instruction carries out all of the functions of the original physical timer.

Figure 7-25 shows a program that uses both the on-delay and the off-delay timer instruction. The process

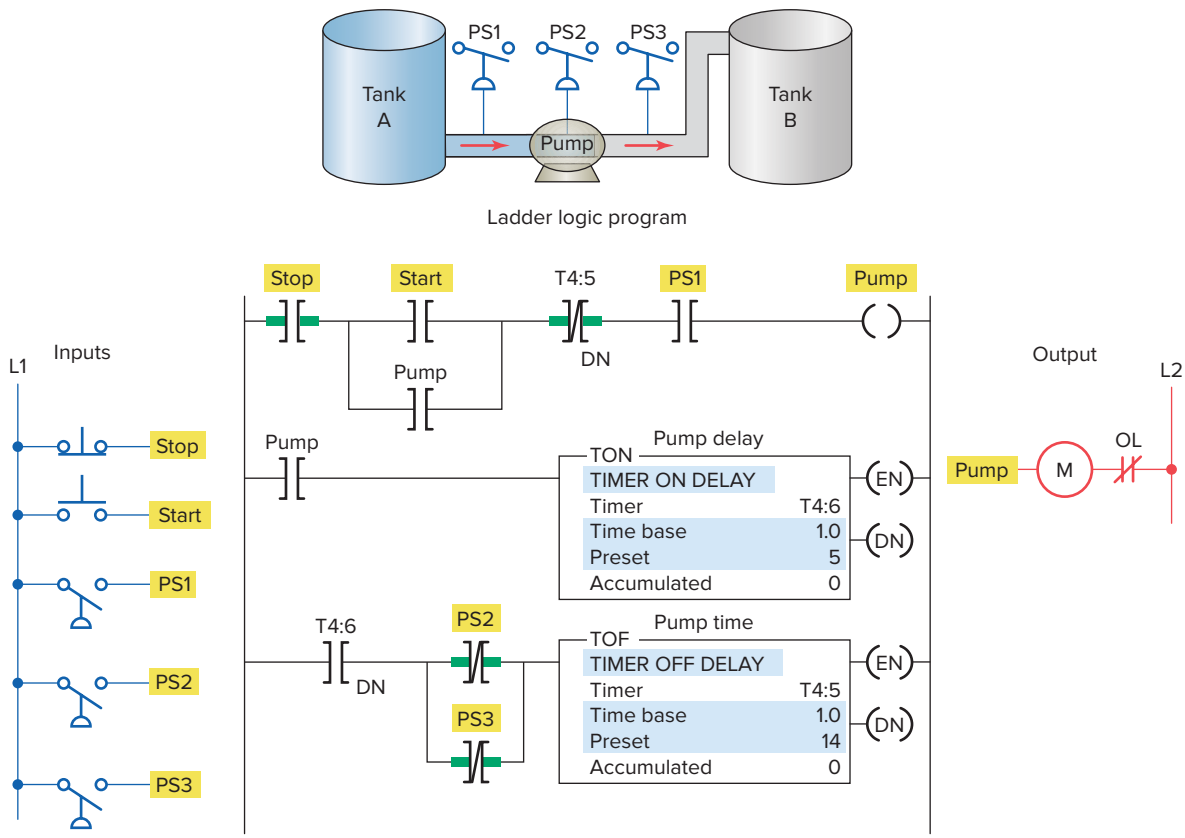


Figure 7-25 Fluid pumping process.

involves pumping fluid from tank A to tank B. The operation of the process can be summarized as follows:

- Before starting, PS1 must be closed.
- When the start button is pushed, the pump starts. The button can then be released and the pump continues to operate.
- When the stop button is pushed, the pump stops.
- PS2 and PS3 must be closed 5 s after the pump starts. If either PS2 or PS3 opens, the pump will shut off and will not be able to start again for another 14 s.

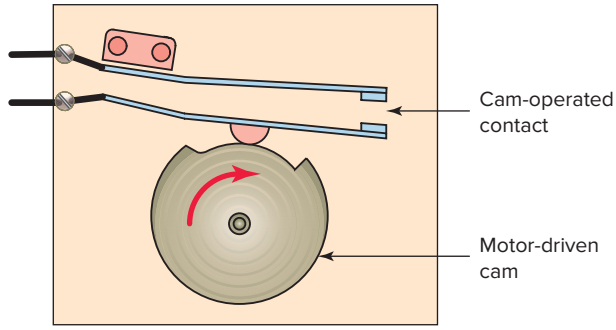


Figure 7-26 Electromechanical retentive timer.

7.5 Retentive Timer

A ***retentive timer*** accumulates time whenever the device receives power, and it maintains the current time should power be removed from the device. When the timer accumulates time equal to its preset value, the contacts of the device change state. Loss of power to the timer after reaching its preset value does not affect the state of the contacts. The retentive timer must be intentionally reset with a separate signal for the accumulated time to be reset and for the contacts of the device to return to its nonenergized state.

Figure 7-26 illustrates the action of a motor-driven, electromechanical retentive timer used in some appliances. The shaft-mounted cam is driven by a motor. Once power is applied, the motor starts turning the shaft and cam. The positioning of the lobes of the cam and the gear reduction of the motor determine the time it takes for the motor to turn the cam far enough to activate the contacts. If power is removed from the motor, the shaft stops but *does not reset.*

A PLC retentive timer is used when you want to retain accumulated time values through power loss or the change in the rung state from true to false. The PLC-programmed retentive on-delay timer (RTO) is programmed in a manner similar to the nonretentive on-delay timer (TON), with one major exception—a retentive timer reset (RES) instruction. Unlike the TON, the RTO will hold its accumulated value when the timer rung goes false and will continue timing where it left off when the timer rung goes true again. This timer must be accompanied by a timer reset instruction to reset the accumulated value of the timer to 0. The RES instruction is the only automatic means of resetting the accumulated value of a retentive timer. The RES instruction has the same address as the timer it is to reset. Whenever the RES instruction is true, both the timer accumulated value and the timer done bit (DN) are reset to 0. Figure 7-27 shows a PLC program for a retentive on-delay timer. The operation of the program can be summarized as follows:

- The timer will start to time when time pushbutton PB1 is closed.
- If the pushbutton is closed for 3 s and then opened for 3 s, the timer accumulated value will remain at 3 s.
- When the time pushbutton is closed again, the timer picks up the time at 3 s and continues timing.

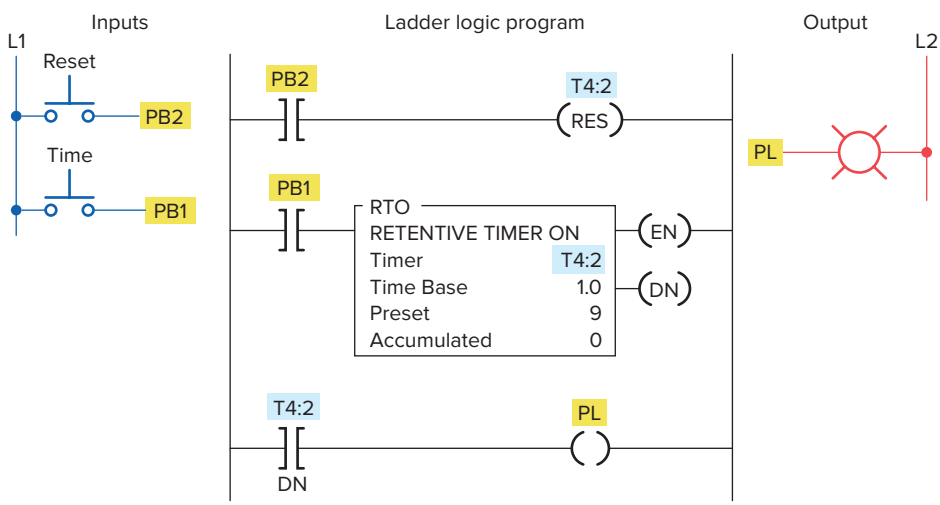


Figure 7-27 Retentive on-delay timer program.

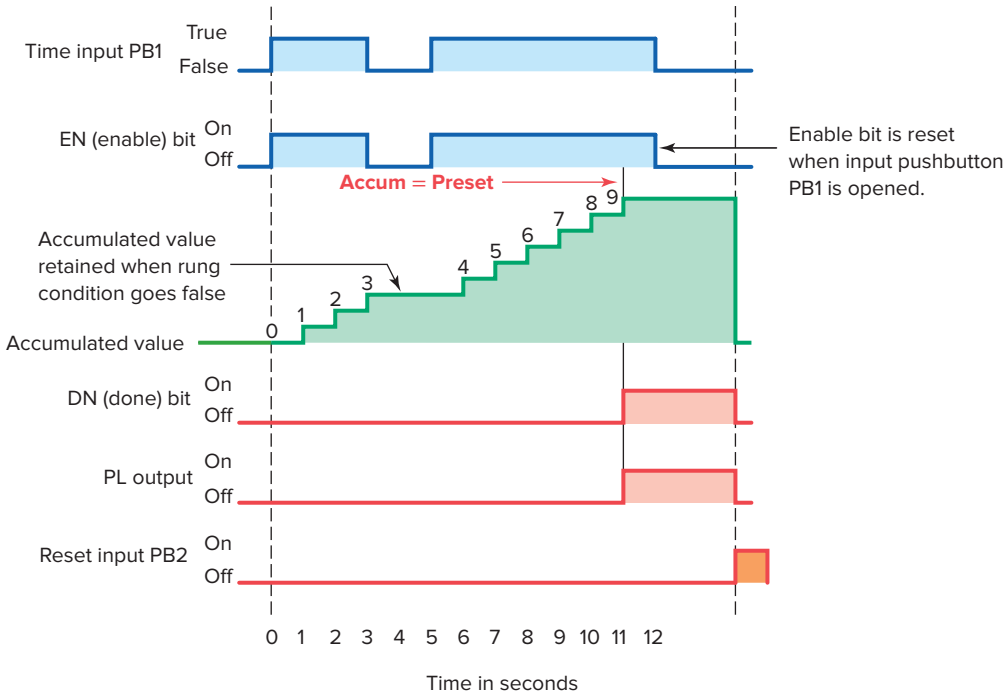


Figure 7-28 Retentive on-delay timer timing chart.

- When the accumulated value (9) equals the preset value (9), the timer done bit T4:2/DN is set to 1 and the pilot light output PL is switched on.
- Whenever the momentary reset pushbutton is closed, the timer accumulated value is reset to 0.

Figure 7-28 shows a timing chart for the retentive on-delay timer program. The timing operation can be summarized as follows:

- When the timing rung is true (PB1 closed), the timer will commence timing.
- If the timing rung goes false, the timer will stop timing but will recommence timing for the stored accumulated value each time the rung goes true.
- When the reset PB2 is closed, the T4:2/DN bit is reset to 0 and turns the pilot light output off. The accumulated value is also reset and held at zero until the reset pushbutton is opened.

The program drawn in Figure 7-29 illustrates a practical application for an RTO. The purpose of the RTO timer is to detect whenever a piping system has sustained

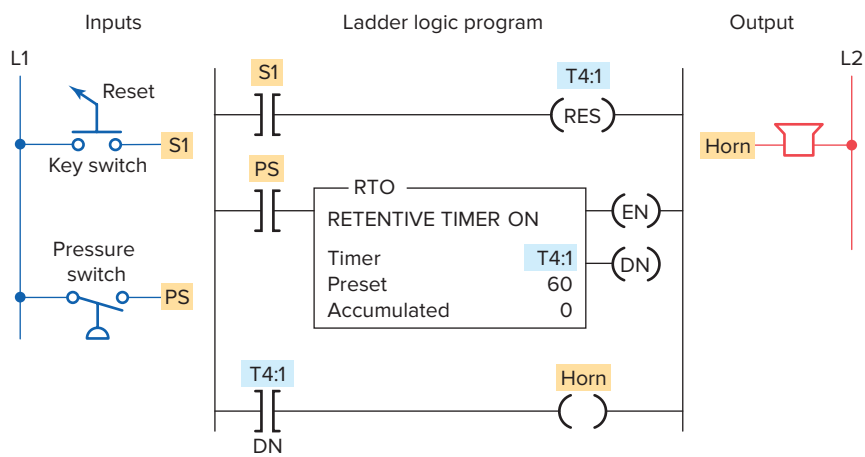


Figure 7-29 Retentive on-delay timer alarm program.

a *cumulative* overpressure condition for 60 s. At that point, a horn is sounded automatically to call attention to the malfunction. When they are alerted, maintenance personnel can silence the alarm by switching the key switch S1 to the reset (contact closed) position. After the problem has been corrected, the alarm system can be reactivated by switching the key switch to open contact position.

Figure 7-30 shows a practical application that uses the on-delay, off-delay, and retentive on-delay instructions in the same program. In this industrial application, there is a machine with a large steel shaft supported by babbitted bearings. This shaft is coupled to a large electric motor. The bearings need lubrication, which is supplied by an oil pump driven by a small electric motor. The operation of the program can be summarized as follows:

- To start the machine, the operator turns SW on.
- Before the *motor* shaft starts to turn, the bearings are supplied with oil by the *pump* for 10 seconds.
- The bearings also receive oil when the machine is running.
- When the operator turns SW off to stop the machine, the oil pump continues to supply oil for 15 s.
- A retentive timer is used to track the total running time of the pump. When the total running time is 3 hours, the motor is shut down and a pilot light is turned on to indicate that the filter and oil need to be changed.
- A reset button is provided to reset the process after the filter and oil have been changed.

Retentive timers do not have to be timed out completely to be reset. Rather, such a timer can be reset at any time during its operation. Note that the reset input to the timer will override the control input of the timer even though the control input to the timer has logic continuity.

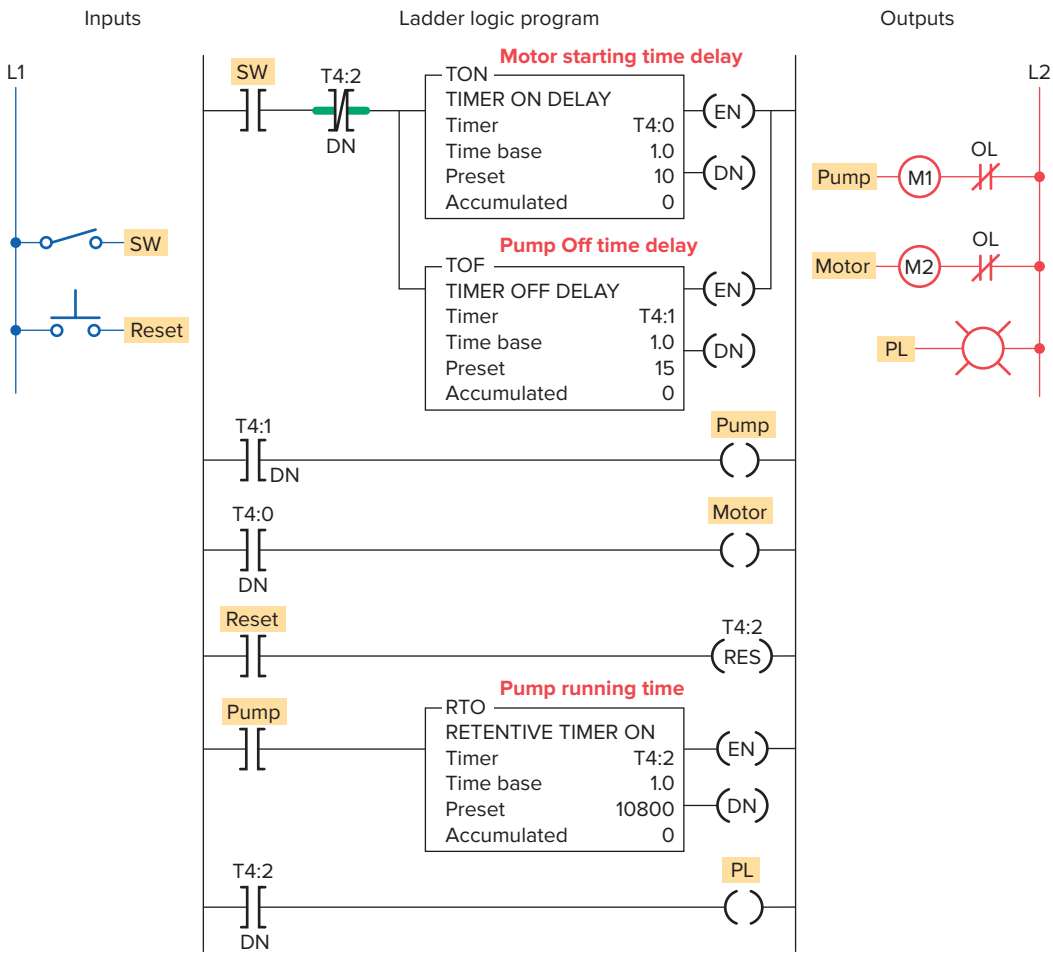


Figure 7-30 Bearing lubrication program.

7.6 Cascading Timers

When one timer's output triggers another timer's input, those timers are referred to as **cascaded**. Timers can be interconnected, or cascaded, to satisfy a number of logic control functions.

Figure 7-31 shows how three motors can be started automatically in sequence with a 20 s time delay between each using two hardwired on-delay timers. The operation of the circuit can be summarized as follows:

- Motor starter coil M1 is energized when the momentary start pushbutton PB2 is actuated.
- As a result, motor 1 starts, contact M1-1 closes to seal in M1, and timer coil TD1 is energized to begin the first time-delay period.
- After the preset time period of 20 s, TD1-1 contact closes to energize motor starter coil M2.
- As a result, motor 2 starts and timer coil TD2 is energized to begin the second time-delay period.
- After the preset time period of 20 s, TD2-1 contact closes to energize motor starter coil M3, and so motor 3 starts.

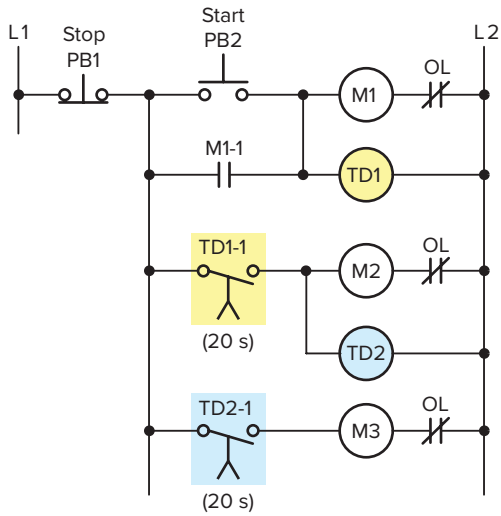


Figure 7-31 Hardwired sequential time-delayed motor-starting circuit.

Figure 7-32 shows an equivalent PLC program of the hardwired sequential time-delayed motor-starting circuit. Two programmed on-delay timers are cascaded together to obtain the same logic as the original hardwired timer relay circuit. Note that the output of timer T4:1 is used to control the input logic to timer T4:2.

Reciprocating timers are defined as timing functions where the output of one timer is used to reset the input of a second timer, each resetting the other. These types of timers are used in situations where a constant cycling

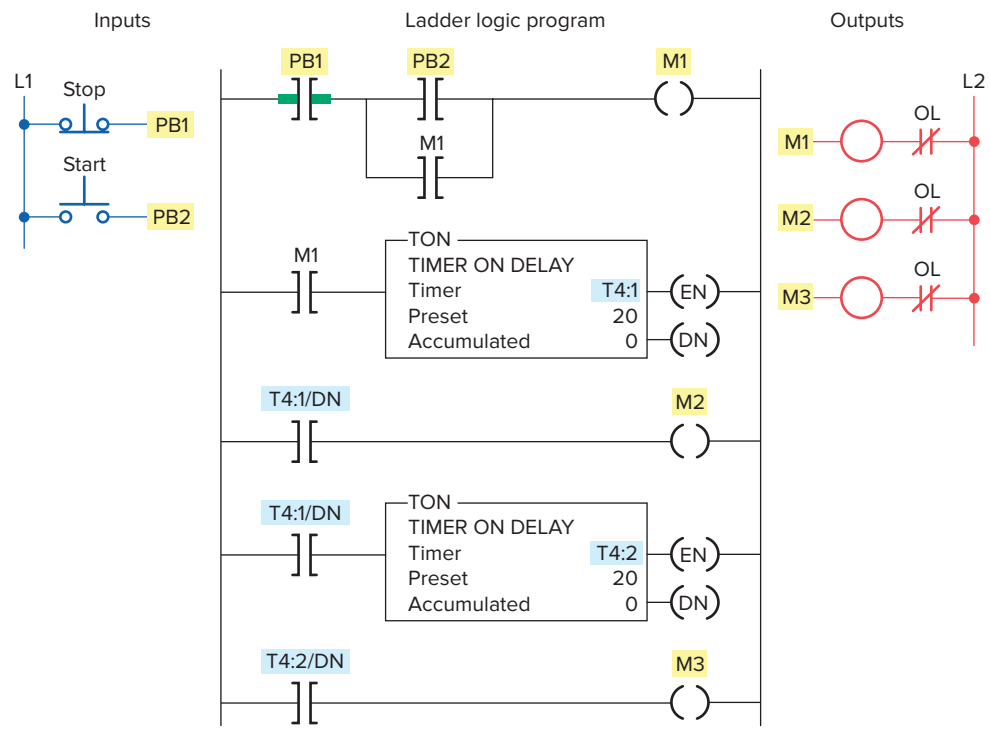


Figure 7-32 Equivalent PLC program of the sequential time-delayed motor-starting circuit.

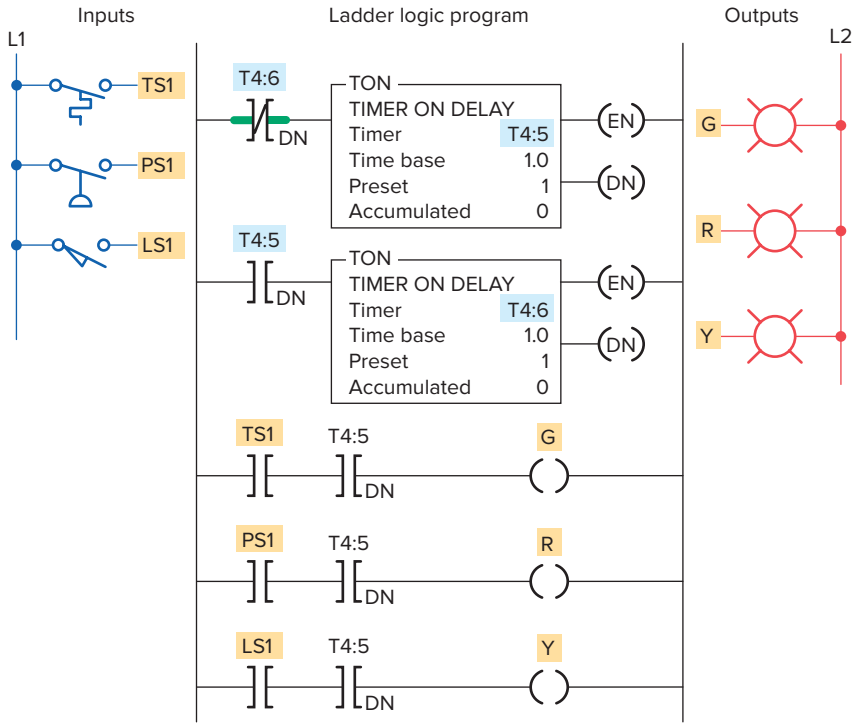


Figure 7-33 Annunciator flasher program.

of an output is required. For example, if a flashing light is required in the event of a control system failure, a program with reciprocating timers could be used to create the flashing output function.

Two timers can be interconnected to form an oscillator or reciprocating circuit. The oscillator logic is basically a timing circuit programmed to generate periodic output pulses of any duration. Figure 7-33 shows the program for an annunciator flasher circuit. Two internal timers form the oscillator circuit, which generates a timed, pulsed output. The oscillator circuit output is programmed in series with the alarm condition. If the alarm condition (temperature, pressure, or limit switch) is true, the appropriate output indicating light will flash. Note that any number of alarm conditions could be programmed using the same flasher circuit.

At times you may require a time-delay period longer than the maximum preset time allowed for the single timer instruction of the PLC being used. When this is the case, the problem can be solved by simply cascading timers, as illustrated in Figure 7-34. The operation of the program can be summarized as follows:

- The total time-delay period required is 42,000 s.
- The first timer, T4:1, is programmed for a preset time of 30,000 s and begins timing when input SW is closed.
- When T4:1 completes its time-delay period 30,000 s later, the T4:1/DN bit will be set to 1.
- This in turn activates the second timer, T4:2, which is preset for the remaining 12,000 s of the total 42,000-s time delay.
- Once T4:2 reaches its preset time, the T4:2/DN bit will be set to 1, which switches on the output PL, the pilot light, to indicate the completion of the full 42,000-s time delay.
- Opening input SW at any time will reset both timers and switch output PL off.

A typical application for a PLC circuit combining both cascading and reciprocating functions would be the control of traffic signals. The ladder logic circuit of Figure 7-35 illustrates a control of a set of traffic lights in one direction. The operation of the program can be summarized as follows:

- Transition from red light to green light to amber light is accomplished by the interconnection of the three TON timer instructions.

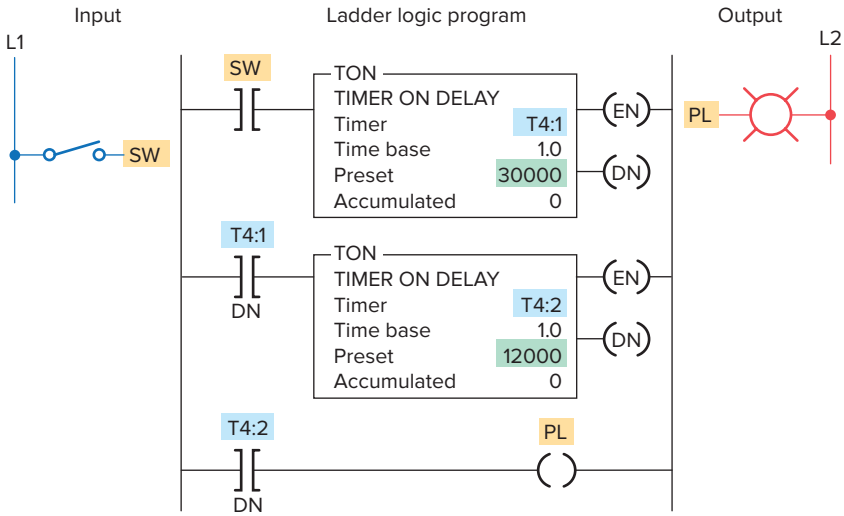


Figure 7-34 Cascading of timers for longer time delays.

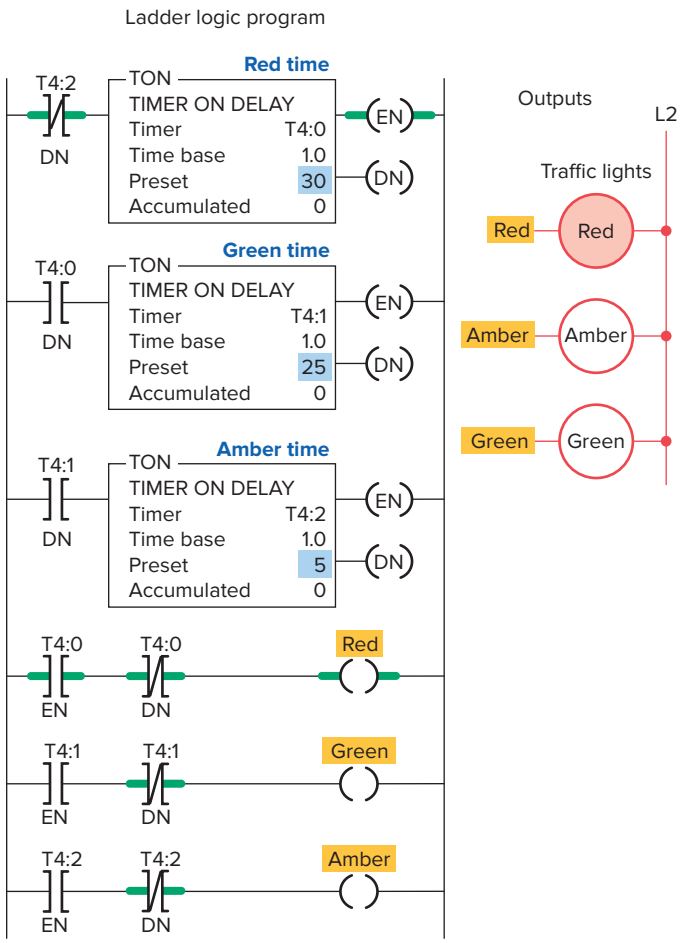


Figure 7-35 Control of traffic lights in one direction.

- The input to timer T4:0 is controlled by the T4:2 done bit.
- The input to timer T4:1 is controlled by the T4:0 done bit.
- The input rung to timer T4:2 is controlled by the T4:1 done bit.
- The timed sequence of the lights is:
  Red—30 s on
  Green—25 s on
  Amber—5 s on
- The sequence then repeats itself.

The chart shown in Figure 7-36 shows the timed sequence of the lights for two-directional control of traffic lights.

Figure 7-37 shows the original traffic light program modified to include three more lights that control traffic flow in two directions.

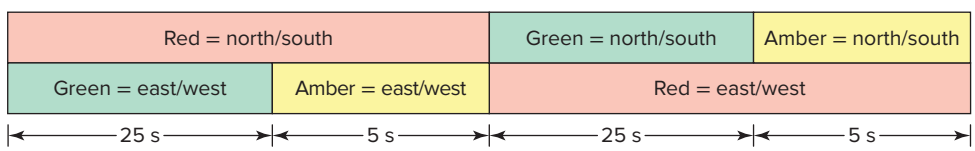


Figure 7-36 Timing chart for two-directional control of traffic lights.

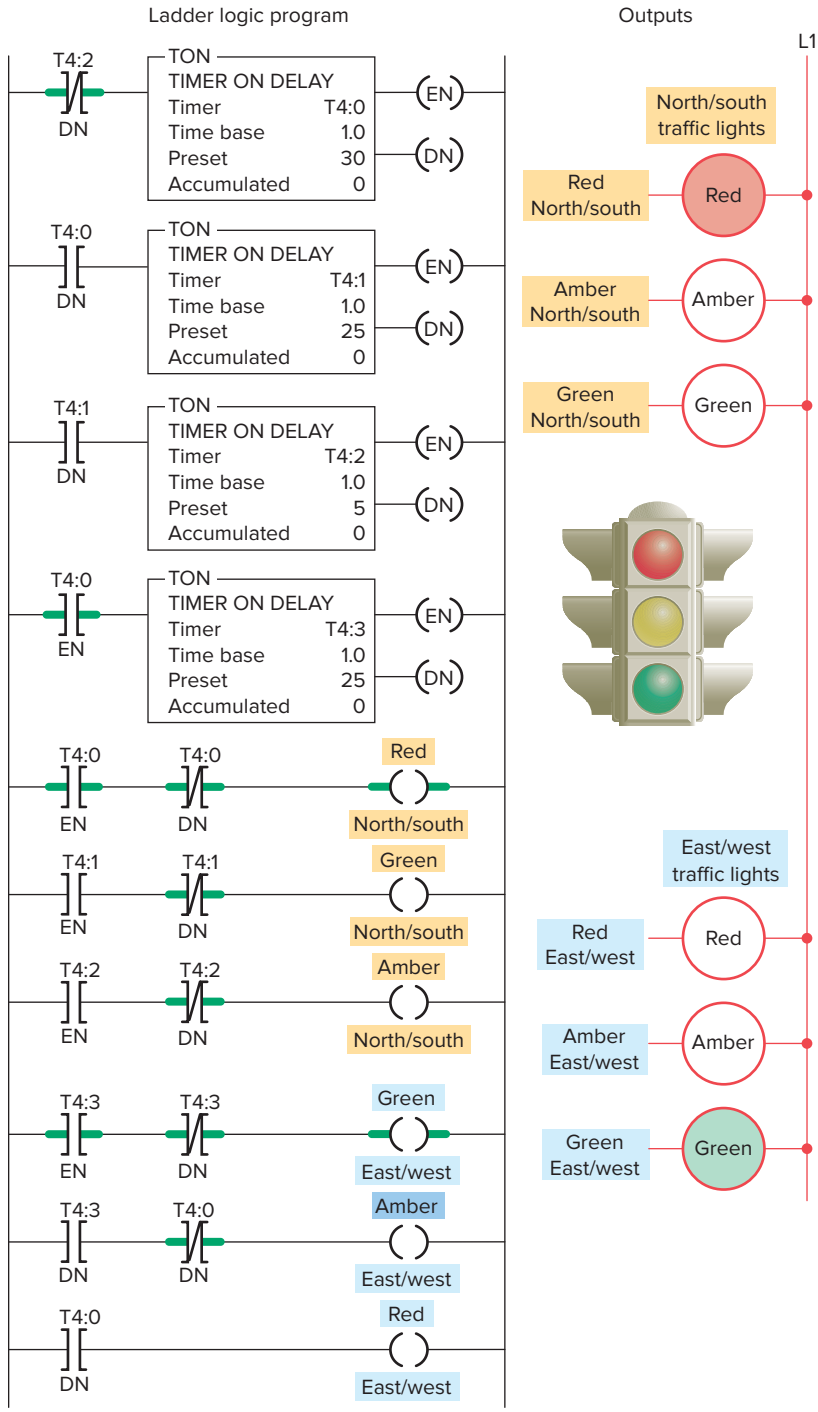


Figure 7-37 Control of traffic lights in two directions.

CHAPTER 7 REVIEW QUESTIONS

1. Explain the difference between the timed and instantaneous contacts of a mechanical timing relay.
2. Draw the symbol and explain the operation of each of the following timed contacts of a mechanical timing relay:
   a. On-delay timer—NOTC contact
   b. On-delay timer—NCTO contact
   c. Off-delay timer—NOTO contact
   d. Off-delay timer—NCTC contact
3. Name five pieces of information usually associated with a PLC timer instruction.
4. When is the output of a programmed timer energized?
5. **a.** What are the two methods commonly used to represent a timer instruction within a PLC's ladder logic program?
   b. Which method is preferred? Why?
6. **a.** Explain the difference between the operation of a nonretentive timer and that of a retentive timer.
   b. Explain how the accumulated count of programmed retentive and nonretentive timers is reset to zero.
7. State three advantages of using programmed PLC timers over mechanical timing relays.
8. For a TON timer:
   a. When is the enable bit of a timer instruction true?
   b. When is the timer-timing bit of a timer instruction true?
   c. When does the done bit of a timer change state?
9. For a TOF timer:
   a. When is the enable bit of a timer instruction true?
   b. When is the timer-timing bit of a timer instruction true?
   c. When does the done bit of a timer change state?
10. Explain what each of the following quantities associated with a PLC timer instruction represents:
    a. Preset time
    b. Accumulated time
    c. Time base
11. State the method used to reset the accumulated time of each of the following:
    a. TON timer
    b. TOF timer
    c. RTO timer

CHAPTER 7 PROBLEMS

1. **a.** With reference to the relay schematic diagram in Figure 7-38, state the status of each light (on or off) after each of the following sequential events:
   I. Power is first applied and switch S1 is open.
   II. Switch S1 has just closed.
   III. Switch S1 has been closed for 5 s.
   IV. Switch S1 has just opened.
   V. Switch S1 has been opened for 5 s.
   b. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program that will execute this hardwired control circuit correctly.
2. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program that will correctly execute the hardwired relay control circuit shown in Figure 7-39.
3. Study the ladder logic program in Figure 7-40 and answer the questions that follow:
   a. What type of timer has been programmed?
   b. What is the length of the time-delay period?

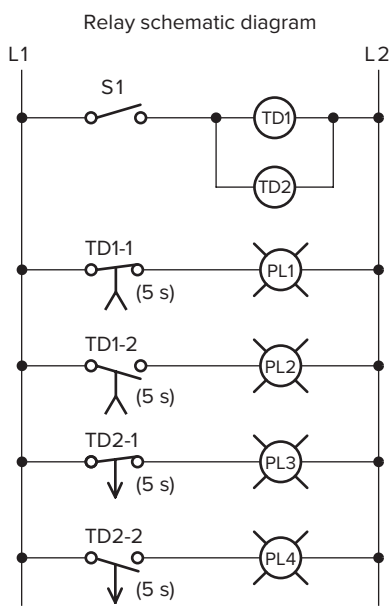


Figure 7-38 Relay schematic diagram for Problem 1.

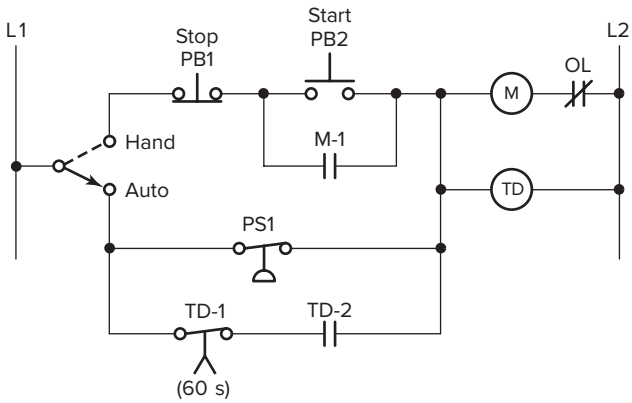


Figure 7-39 Hardwired relay control circuit for Problem 2.

c. What is the value of the accumulated time when power is first applied?
d. When does the timer start timing?
e. When does the timer stop timing and reset itself?
f. When input LS1 is first closed, which rungs are true and which are false?
g. When input LS1 is first closed, state the status (on or off) of each output.
h. When the timer's accumulated value equals the preset value, which rungs are true and which are false?
i. When the timer's accumulated value equals the preset value, state the status (on or off) of each output.
j. Suppose that rung 1 is true for 5 s and then power is lost. What will the accumulated value of the counter be when power is restored?

4. Study the ladder logic program in Figure 7-41 and answer the questions that follow:
a. What type of timer has been programmed?
b. What is the length of the time-delay period?
c. What is the value of the accumulated time when power is first applied?
d. When does the timer start timing?
e. When does the timer stop timing and reset itself?
f. When input LS1 is first closed, which rungs are true and which are false?
g. When input LS1 is first closed, state the status (on or off) of each output.
h. When the timer's accumulated value equals the preset value, which rungs are true and which are false?
i. When the timer's accumulated value equals the preset value, state the status (on or off) of each output.
j. Suppose that rung 1 is true for 5 s and then power is lost. What will the accumulated value of the counter be when power is restored?

5. Study the ladder logic program in Figure 7-42, and answer the questions that follow:
a. What type of timer has been programmed?
b. What is the length of the time-delay period?
c. When does the timer start timing?

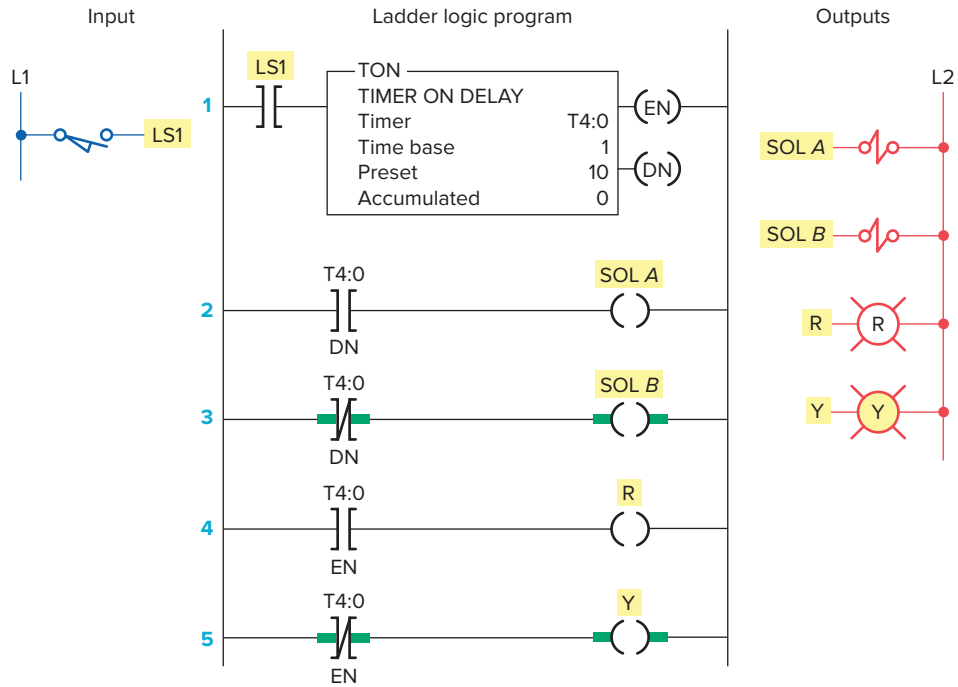


Figure 7-40 Ladder logic program for Problem 3.

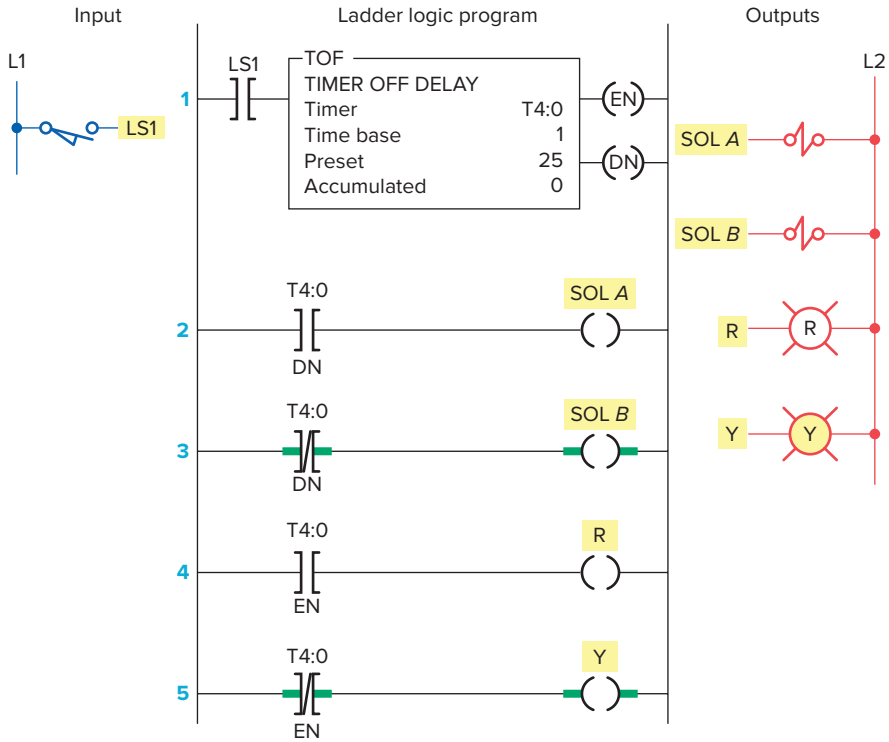


Figure 7-41 Ladder logic program for Problem 4.

d. When is the timer reset?
e. When will rung 3 be true?
f. When will rung 5 be true?
g. When will output PL4 be energized?
h. Assume that your accumulated time value is up to 020 and power to your system is lost. What will your accumulated time value be when power is restored?

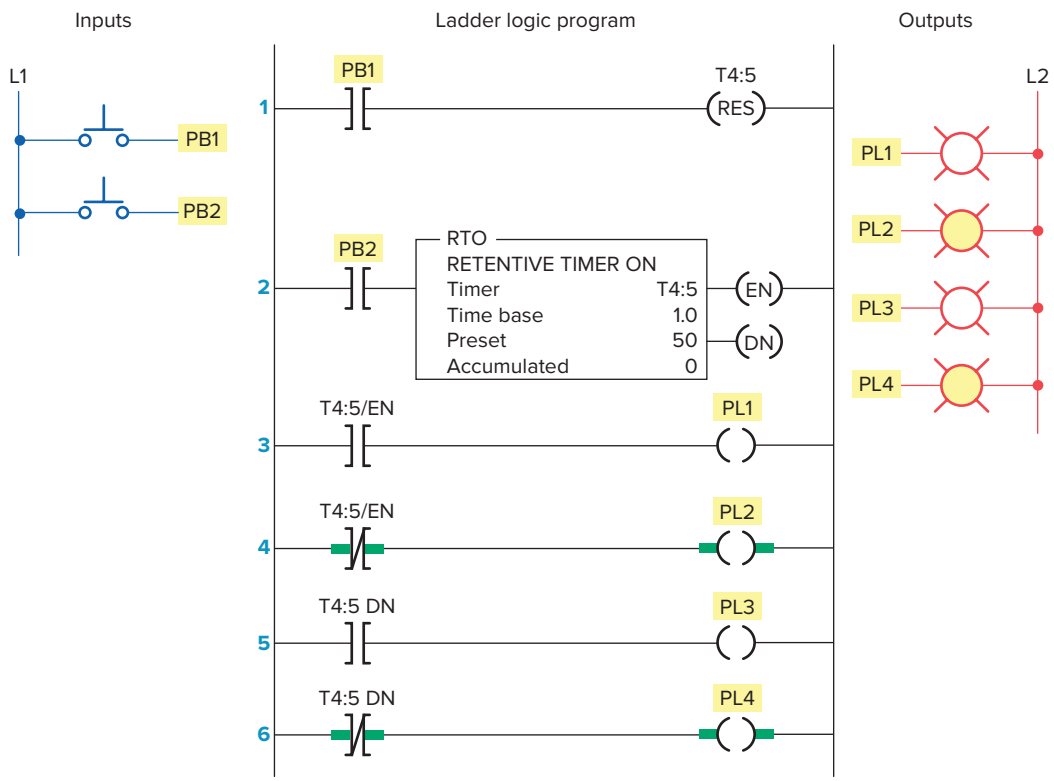


Figure 7-42 Ladder logic program for Problem 5.

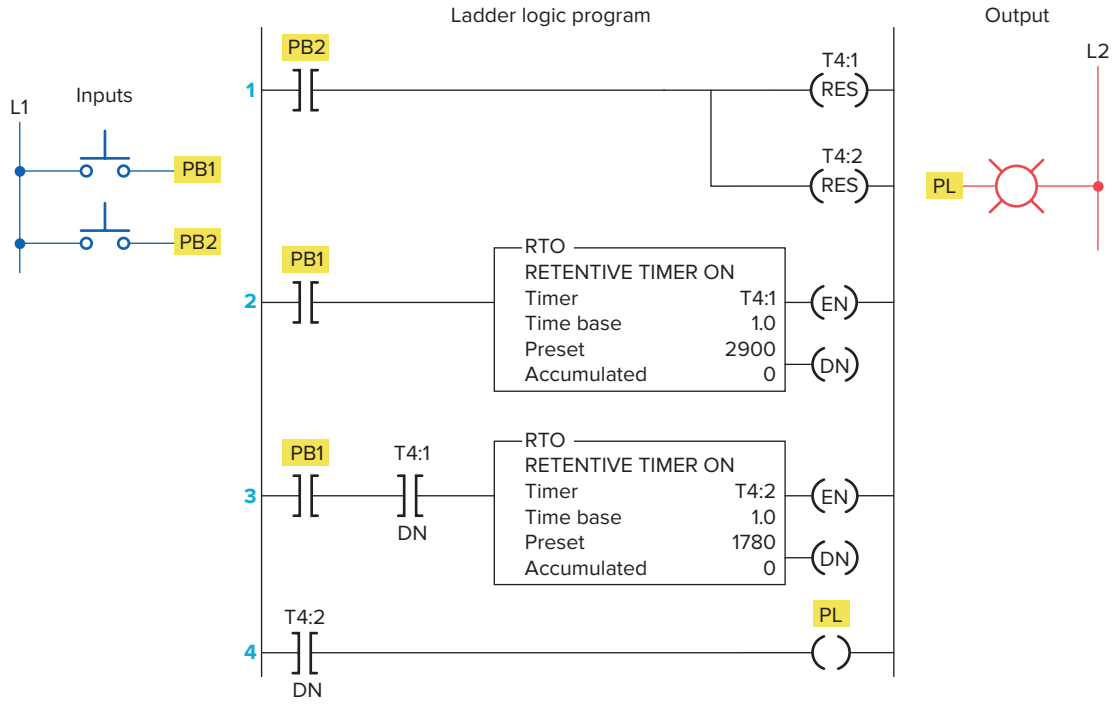


Figure 7-43 Ladder logic program for Problem 6.

i. What happens if inputs PB1 and PB2 are both true at the same time?

6. Study the ladder logic program in Figure 7-43 and answer the questions that follow:

a. What is the purpose of interconnecting the two timers?

b. How much time must elapse before output PL is energized?

c. What two conditions must be satisfied for timer T4:2 to start timing?

d. Assume that output PL is on and power to the system is lost. When power is restored, what will the status of this output be?

e. When input PB2 is on, what will happen?

f. When input PB1 is on, how much accumulated time must elapse before rung 3 will be true?

7. You have a machine that cycles on and off during its operation. You need to keep a record of its total run time for maintenance purposes. Which timer would accomplish this?

8. Write a ladder logic program that will turn on a light, PL, 15 s after switch S1 has been turned on.

9. Study the on-delay timer ladder logic program in Figure 7-44, and from each of the conditions stated, determine whether the timer is reset, timing, or timed out or if the conditions stated are not possible.

a. The input is true, and EN is 1, TT is 1, and DN is 0.

b. The input is true, and EN is 1, TT is 1, and DN is 1.

c. The input is false, and EN is 0, TT is 0, and DN is 0.

d. The input is true, and EN is 1, TT is 0, and DN is 1.

10. Study the off-delay timer ladder logic program in Figure 7-45, and from each of the conditions stated, determine whether the timer is reset, timing, or timed out or if the conditions stated are not possible.

a. The input is true, and EN is 0, TT is 0, and DN is 1.

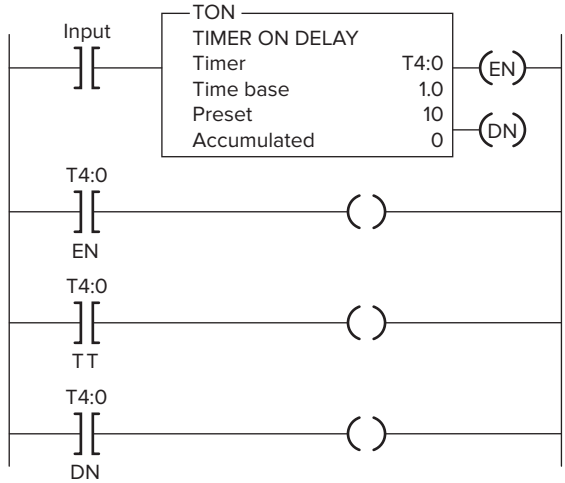


Figure 7-44 On-delay timer ladder logic program for Problem 9.

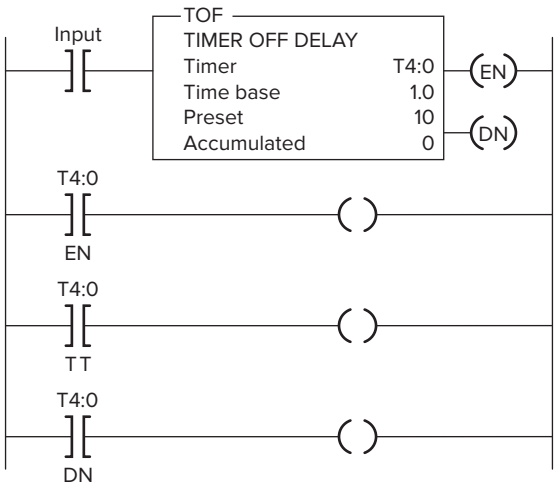


Figure 7-45 Off-delay timer ladder logic program for Problem 10.

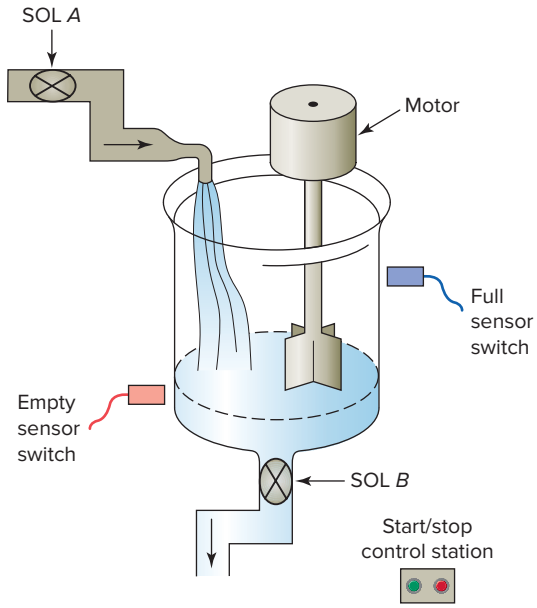


Figure 7-46 Process for Problem 13.

b. The input is true, and EN is 1, TT is 1, and DN is 1.

c. The input is true, and EN is 1, TT is 0, and DN is 1.

d. The input is false, and EN is 0, TT is 1, and DN is 1.

e. The input is false, and EN is 0, TT is 0, and DN is 0.

11. Write a program for an "anti–tie down circuit" that will disallow a punch press solenoid from operating unless both hands are on the two palm start buttons. Both buttons must be pressed at the same time within 0.5 s. The circuit also will not allow the operator to tie down one of the buttons and operate the press with just one button. (Hint: Once either of the buttons is pressed, begin timing 0.5 s. Then, if both buttons are not pressed, prevent the press solenoid from operating.)

12. Modify the program for the control of traffic lights in two directions so that there is a 3-s period when both directions will have their red lights illuminated.

13. Write a program to implement the process illustrated in Figure 7-46. The sequence of operation is to be as follows:

- Normally open start and normally closed stop pushbuttons are used to start and stop the process.
- When the start button is pressed, solenoid *A* energizes to start filling the tank.
- As the tank fills, the empty level sensor switch closes.
- When the tank is full, the full level sensor switch closes.
- Solenoid *A* is de-energized.
- The agitate motor starts automatically and runs for 3 min to mix the liquid.
- When the agitate motor stops, solenoid *B* is energized to empty the tank.
- When the tank is completely empty, the empty sensor switch opens to de-energize solenoid *B*.
- The start button is pressed to repeat the sequence.

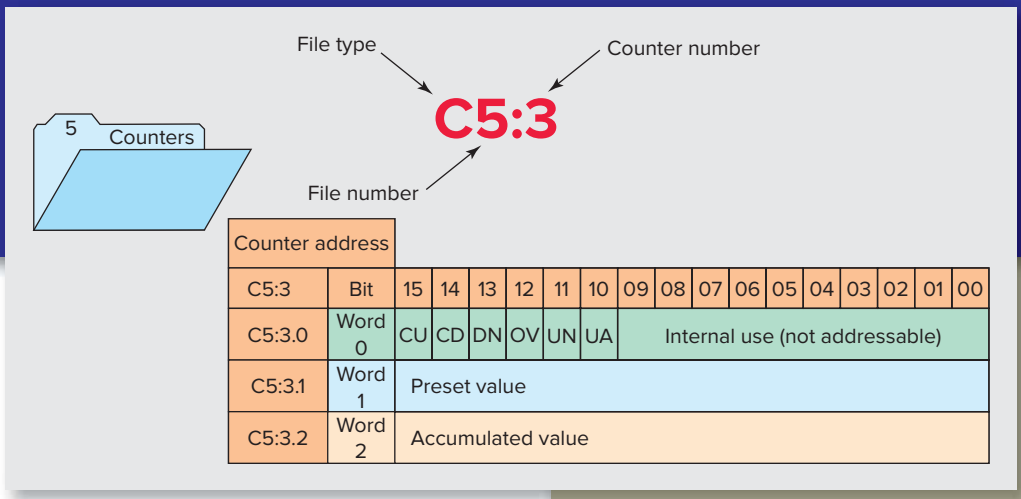
14. When the lights are turned off in a building, an exit door light is to remain on for an additional 2 min, and the parking lot lights are to remain on for an additional 3 min after the door light goes out. Write a program to implement this process.

15. Write a program to simulate the operation of a sequential taillight system. The light system consists of three separate lights on each side of the car. Each set of lights will be activated separately, by either the left or right turn signal switch. There is to be a 1-s delay between the activation of each light, and a 1-s period when all the lights are off. Ensure that when both switches are on, the system will not operate. Use the least number of timers possible. The sequence of operation should be as follows:

- The switch is operated.
- Light 1 is illuminated.
- Light 2 is illuminated 1 s later.
- Light 3 is illuminated 1 s later.
- Light 3 is illuminated for 1 s.
- All lights are off for 1 s.
- The system repeats while the switch is on.

8

Programming Counters



Counter address																	
C5:3	Bit	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
C5:3.0	Word 0	CU	CD	DN	OV	UN	UA	Internal use (not addressable)									
C5:3.1	Word 1	Preset value															
C5:3.2	Word 2	Accumulated value															

All PLCs include both up-counters and down-counters. Counter instructions and their function in ladder logic are explained in this chapter. Typical examples of PLC counters include the following: straight counting in a process, two counters used to give the sum of two counts, and two counters used to give the difference between two counts.

Chapter Objectives

After completing this chapter, you will be able to:

- List and describe the functions of PLC counter instructions
- Describe the operating principle of a transitional, or one-shot, contact
- Analyze and interpret typical PLC counter ladder logic programs
- Apply the PLC counter function and associated circuitry to control systems
- Apply combinations of counters and timers to control systems

8.1 Counter Instructions

Programmed counters can serve the same function as mechanical counters. Figure 8-1 shows the construction of a simple **mechanical counter.** Every time the actuating lever is moved over, the counter adds one number; the actuating lever then returns automatically to its original position. Resetting to zero is done with a pushbutton located on the side of the unit.

Electronic counters, such as those shown in Figure 8-2, can count up, count down, or be combined to count up and down. Although the majority of counters used in industry are up-counters, numerous applications require the implementation of down-counters or of combination up/down-counters.

All PLC manufacturers offer some form of counter instruction as part of their instruction set. One common counter application is keeping track of the number of items moving past a given point as illustrated in Figure 8-3.

Counters are similar to timers except that they do not operate on an internal clock but are dependent on external or program sources for counting. The two methods used to represent a counter within a PLC's ladder logic program are the coil format and the block format. Figure 8-4 shows a typical coil-formatted up-counter instruction. The up-counter increments its accumulated value by 1 each time the counter rung makes a false-to-true transition. When the accumulated count equals the preset count the counter output is energized or set to 1. Shown as part of the instruction are the:

Counter type
Counter address
Counter preset value
Accumulated count

The counter reset instruction must be used in conjunction with the counter instruction. Up-counters are always reset to zero. Down-counters may be reset to zero or to some preset value. Some manufacturers include the reset function as a part of the general counter instruction, whereas others dedicate a separate instruction for resetting the counter. Figure 8-5 shows a **coil-formatted** counter instruction with a separate instruction for resetting the counter. When programmed, the counter reset coil (CTR) is given the same reference address as the

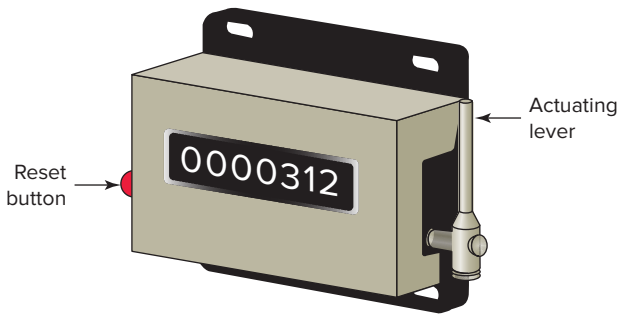


Figure 8-1 Mechanical counter.

Figure 8-2 Electronic counters.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

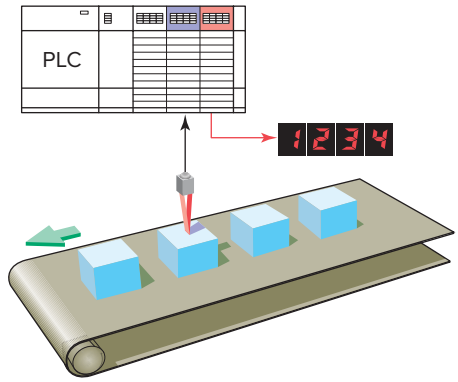


Figure 8-3 Counter application.

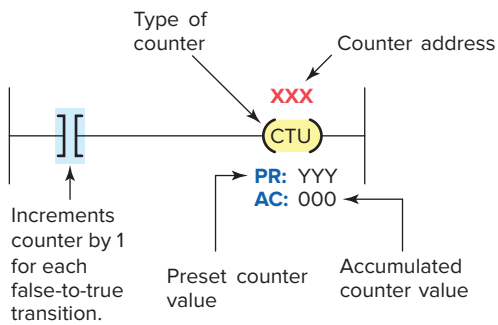


Figure 8-4 Coil-formatted up-counter instruction.

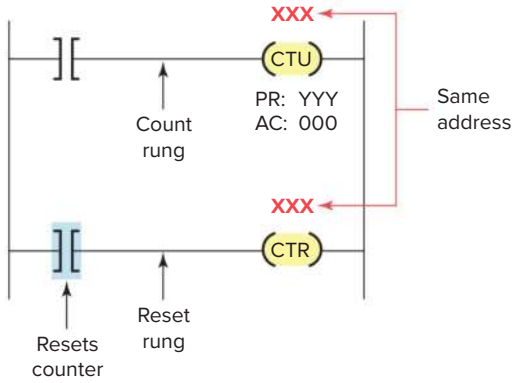


Figure 8-5 Coil-formatted counter and reset instructions.

counter (CTU) that it is to reset. In this example the reset instruction is activated whenever the CTR rung condition is true.

Figure 8-6 shows a ***block-formatted*** counter. The instruction block indicates the type of counter (up or down), along with the counter's preset value and accumulated or current value. The counter has two input conditions associated with it, namely, the count and reset. All PLC counters operate, or count, on the leading edge of the input signal. The counter will either **increment** or **decrement** whenever the count input transfers from an off state to an on state. The counter will *not* operate on the trailing edge, or on-to-off transition, of the input condition.

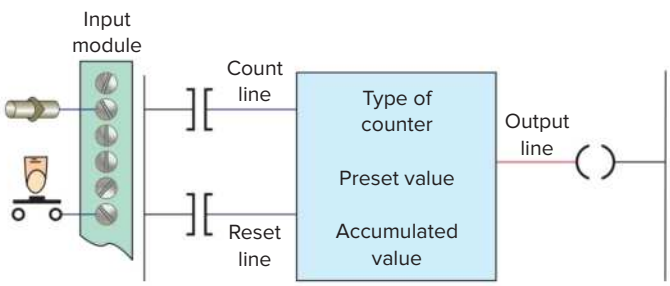


Figure 8-6 Block-formatted counter instruction.

Some manufacturers require the reset rung or line to be true to reset the counter, whereas others require it to be false to reset the counter. For this reason, it is wise to consult the PLC's operations manual before attempting any programming of counter circuits.

PLC counters are normally retentive; that is, whatever count was contained in the counter at the time of a processor shutdown will be restored to the counter on power-up. The counter may be reset, however, if the reset condition is activated at the time of power restoration.

PLC counters can be designed to count up to a preset value or to count down to a preset value. The ***up-counter*** is incremented by 1 each time the rung containing the counter goes from false to true. The ***down-counter*** decrements by 1 each time the rung containing the counter is energized. These rung transitions can result from events occurring in the program, such as parts traveling past a sensor or actuating a limit switch. The preset value of a programmable controller counter can be set by the operator or can be loaded into a memory location as a result of a program decision.

Figure 8-7 illustrates the counting sequence of an up-counter and a down-counter. The value indicated by the counter is termed the ***accumulated value.*** The counter will increment or decrement, depending on the type of counter, until the accumulated value of the counter is equal to or greater than the preset value, at which time an output will be produced. A counter reset is always provided to cause the counter accumulated value to be reset to a predetermined value.

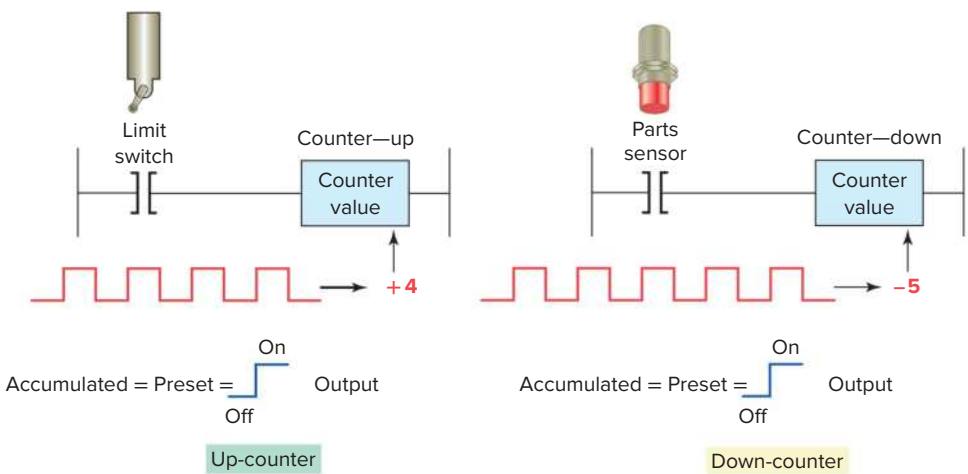


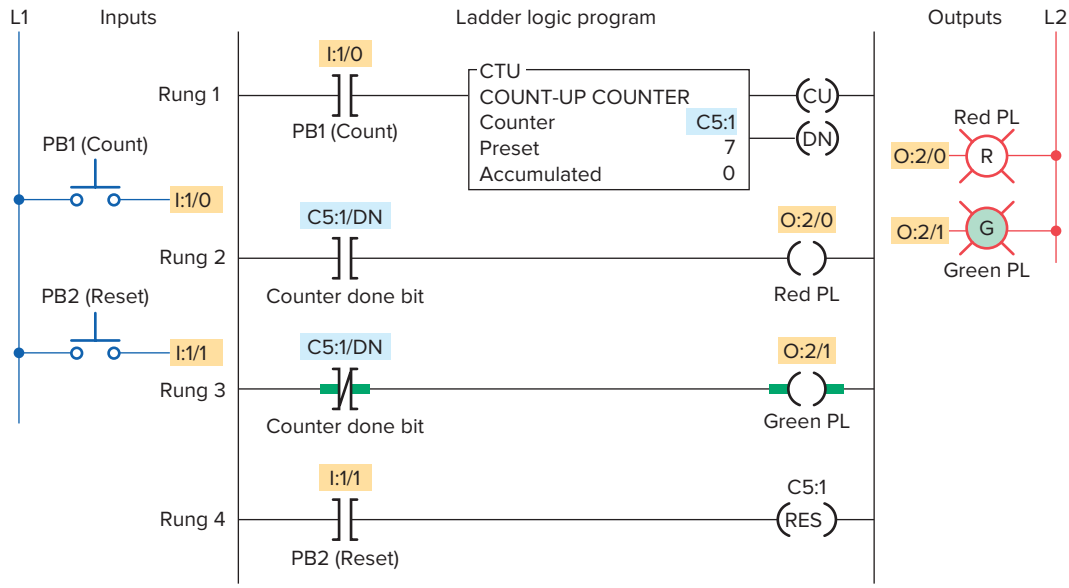
Figure 8-7 Counter counting sequence.

8.2 Up-Counter

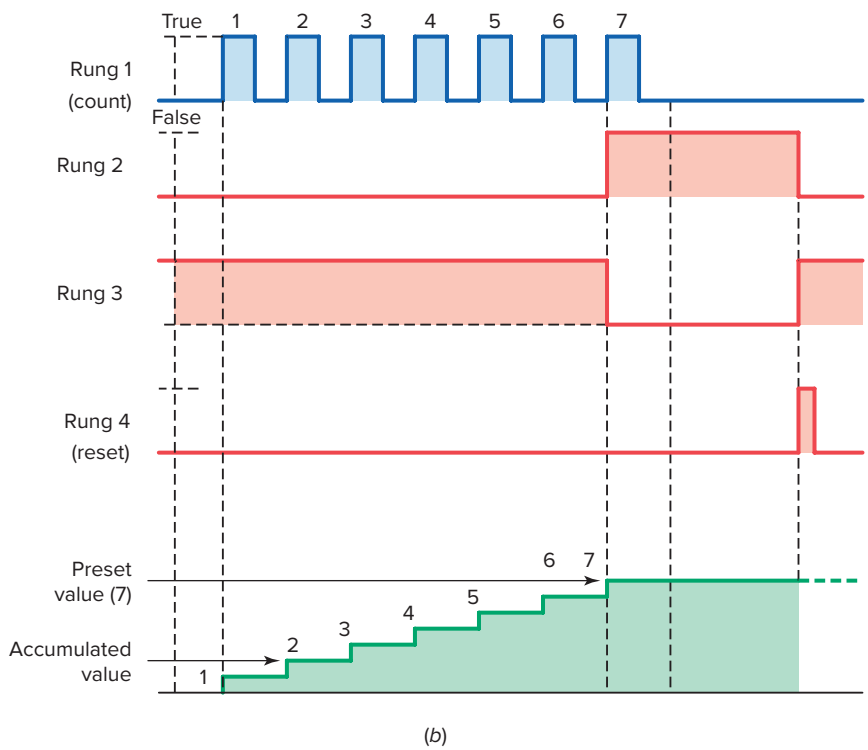
The up-counter is an output instruction whose function is to increment its accumulated value on false-to-true transitions of its instruction. It thus can be used to count false-to-true transitions of an input instruction and then trigger an event after a required number of counts or transitions. The up-counter output instruction will increment by 1 each time the counted event occurs.

Figure 8-8 shows the program and timing diagram for an SLC 500 **Count-Up Counter.** This control application is designed to turn the red pilot light on and the green pilot light off after an accumulated count of 7. The operation of the program can be summarized as follows:

- Operating pushbutton PB1 provides the off-to-on transition pulses that are counted by the counter.
- The preset value of the counter is set for 7.



(*a*)



(*b*)

Figure 8-8 Simple up-counter program. (a) Program. (b) Timing diagram.

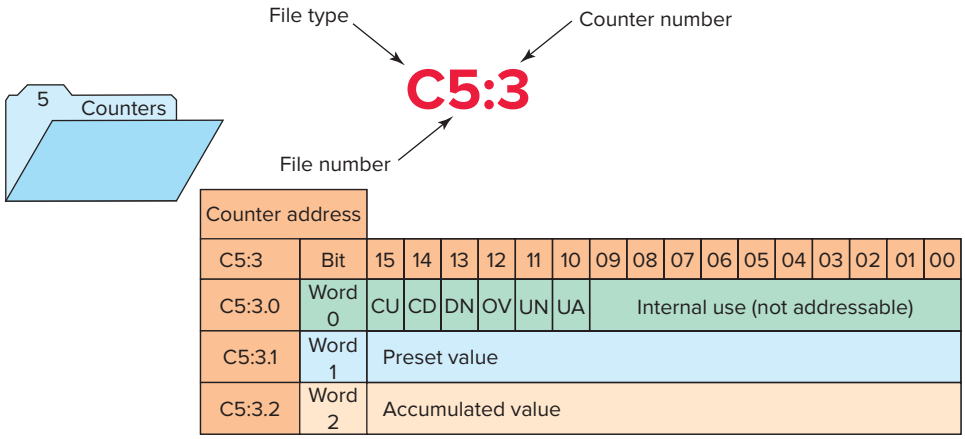


Figure 8-9 SLC 500 counter file.

- Each false-to-true transition of rung 1 increases the counter's accumulated value by 1.
- Output O:2/1 is energized as long as the accumulated value is less than 7.
- After 7 pulses, or counts, when the preset counter value equals the accumulated counter value, output DN is energized.
- As a result, rung 2 becomes true and energizes output O:2/0 to switch the red pilot light on.
- At the same time, rung 3 becomes false and de-energizes output O:2/1 to switch the green pilot light off.
- The counter is reset by closing pushbutton PB2, which makes rung 4 true and resets the accumulated count to zero.
- Counting can resume when rung 4 goes false again.

The Allen-Bradley SLC 500 counter file is file 5 (Figure 8-9). Each counter is composed of three 16-bit words, collectively called a counter element. These three data words are the control word, preset word, and accumulated word. Each of the three data words shares the same base address, which is the address of the counter itself. There can be up to 256 counter elements. Addresses for counter file 5, counter element 3 (C5:3), are listed below.

C5 = counter file 5

:3 = counter element 3 (0–255 counter elements per file)

C5:3/DN is the address for the done bit of the counter.

C5:3/CU is the address for the count-up enable bit of the counter.

C5:3/CD is the address for the count-down enable bit of the counter.

C5:3/OV is the address for the overflow bit of the counter.

C5:3/UN is the address for the underflow bit of the counter.

C5:3/UA is the address for the update accumulator bit of the counter. This instruction is only used with the High-Speed Counter (HSC) instruction.

Figure 8-10 shows the counter table for the Allen-Bradley SLC 500 controller. The ***control word*** uses status control bits consisting of the following:

Count-Up (CU) Enable Bit—The count-up enable bit is used with the count-up counter and is true whenever the count-up counter instruction is true. If the count-up counter instruction is false, the CU bit is false.

Count-Down (CD) Enable Bit—The count-down enable bit is used with the count-down counter and is true whenever the count-down counter instruction is true. If the count-down counter instruction is false, the CD bit is false.

Done (DN) Bit—The done bit is true whenever the accumulated value is equal to or greater than the preset value of the counter, for either the count-up or the count-down counter.

Overflow (OV) Bit—The overflow bit is true whenever the counter counts past its maximum value, which is 32,767. On the next count, the counter will wrap around to –32,768 and will continue counting

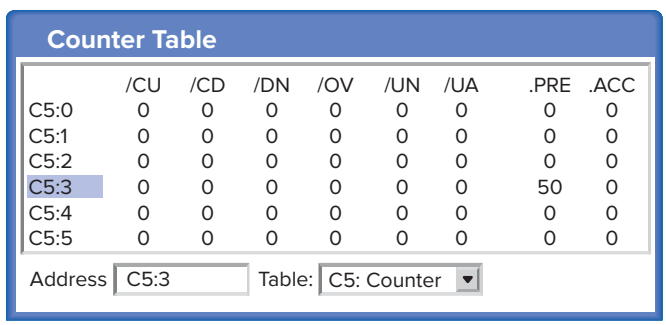


Figure 8-10 SLC 500 counter table.

from there toward 0 on successive false-to-true transitions of the count-up counter.

Underflow (UN) Bit—The underflow bit will go true when the counter counts below −32,768. The counter will wrap around to +32,767 and continue counting down toward 0 on successive false-to-true rung transitions of the count-down counter.

Update Accumulator (UA) Bit—The update accumulator bit is used only in conjunction with an external HSC (high-speed counter).

The ***preset value (PRE)*** *word* specifies the value that the counter must count to before it changes the state of the done bit. The preset value is the set point of the counter and ranges from −32,768 to +32,767. The number is stored in binary form, with any negative numbers being stored in 2's complement binary.

The ***accumulated value (ACC)*** *word* is the current count based on the number of times the rung goes from false to true. The accumulated value either increments with a false-to-true transition of the count-up counter instruction or decrements with a false-to-true transition of the count-down counter instruction. It has the same range as the preset: −32,768 to +32,767. The accumulated value will continue to count past the preset value instead of stopping at the preset like a timer does.

Figure 8-11 shows an example of the count-up counter and its status bits used in the SLC 500 controller

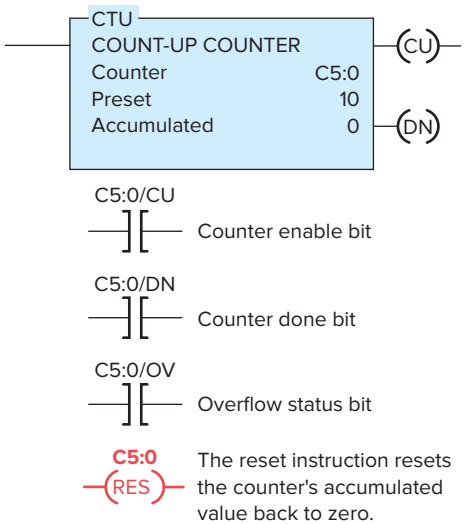


Figure 8-11 Count-up counter instruction.

instruction set. The address for counters begins at C5:0 and continues through C5:255. The information to be entered includes:

Counter Number—This number must come from the counter file. In the example shown, the counter number is C5:0, which represents counter file 5, counter 0 in that file. The address for this counter should not be used for any other count-up counter.

Preset Value—The preset value can range from −32,768 to +32,767. In the example shown, the preset value is 10.

Accumulated Value—The accumulated value can also range from −32,768 to +32,767. Typically, as in this example, the value entered in the accumulated word is 0. Regardless of what value is entered, the reset instruction will reset the accumulated value to 0.

Figure 8-12 shows the timer/counter menu tab from the RSLogix toolbar. Several timer and counter instructions appear when this tab is selected. The first three are timer instructions that are covered in Chapter 7. The next two instructions from the left are the up-counter (CTU) and down-counter (CTD) instructions. To the right of the CTU and CTD instructions is the reset (RES) instruction, which is used by both counters and timers. The counter commands can be summarized as follows:

CTU (Count-Up)—Increments the accumulated value at each false-to-true transition and retains the accumulated value when an off/on power cycle occurs.

CTD (Count-Down)—Decrements the accumulated value at each false-to-true transition and retains the accumulated value when an on/off power cycle occurs.

HSC (High-Speed Counter)—Counts high-speed pulses from a high-speed input.

Figure 8-13 shows a PLC counter program used to stop a motor from running after 10 operations. The operation of the program can be summarized as follows:

- Up-counter C5:0 counts the number of on/off times the motor starts.
- The preset value of the counter is set to 10.

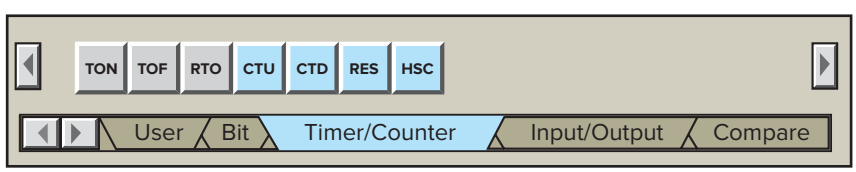


Figure 8-12 Counter selection toolbar.

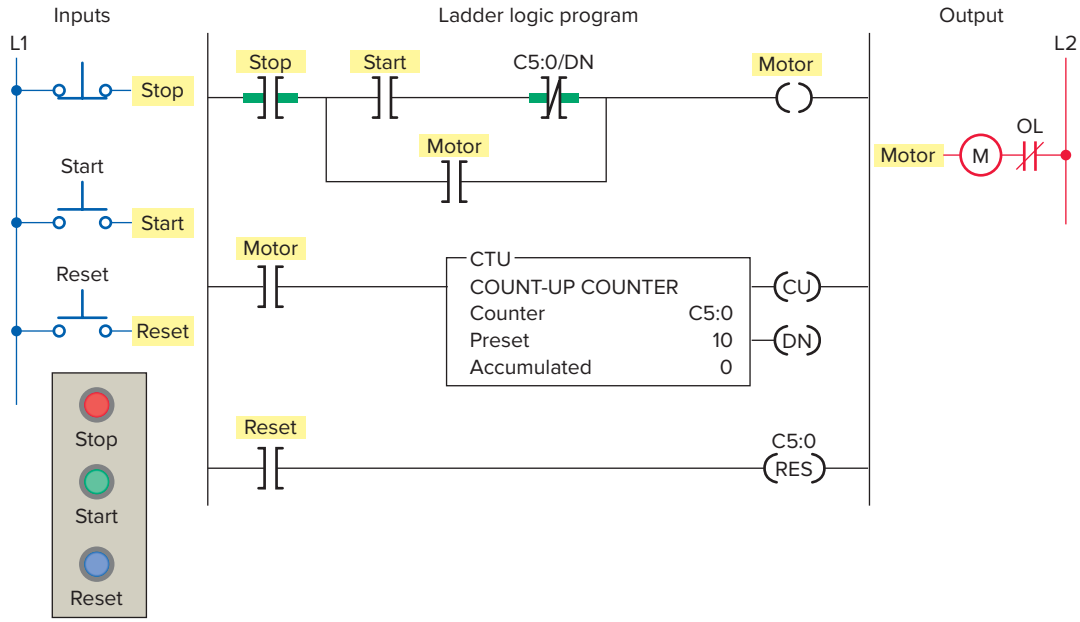


Figure 8-13 PLC counter program used to stop a motor from running after 10 operations.

- A counter done bit examine-off instruction is programmed in series with the motor output instruction.
- A motor output examine-on instruction is used to increment the accumulated value of the counter for each off/on operation.
- After the count of 10 is reached the counter done bit examine-off instruction goes false preventing the motor from being started.
- Closure of the reset pushbutton resets the accumulated count to zero.

Figure 8-14 shows a PLC can-counting program that uses three up-counters. The operation of the program can be summarized as follows:

- Counter C5:2 counts the total number of cans coming off an assembly line for final packaging.
- Each package must contain 10 parts.
- When 10 cans are detected, counter C5:1 sets bit B3:0/1 to initiate the box closing sequence.
- Counter C5:3 counts the total number of packages filled in a day. (The maximum number of packages per day is 300.)
- A pushbutton is used to restart the total part and package count from zero daily.

The counter instructions for the SLC 500 and ControlLogix 5000 processors operate in exactly the same manner. Figure 8-15 shows the counter selection toolbar, CTU counter instruction, and program tags dialog box for a ControlLogix controller.

- Logix processors use a tag name, such as Package_Count, instead of a counter number.
- This descriptive tag name makes it easier to know what function the counter serves in the control system.
- The associated counter data (PRE, ACC, CU, CD, DN, OV, UN) are found within the program tags dialog box.

One-Shot Instruction

Figure 8-16 shows the program for a ***one-shot, or transitional,*** *contact circuit* that is often used to automatically clear or reset a counter. The program is designed to generate an output pulse that, when triggered, goes on for the duration of one program scan and then goes off. The one-shot can be triggered from a momentary signal or from a signal that comes on and stays on for some time. Whichever signal is used, the one-shot is triggered by the leading-edge (off-to-on) transition of the input signal. It stays on for one scan and goes off. It stays off until the trigger goes off, and then comes on again. The one-shot is perfect for resetting both counters and timers since it stays on for one scan only.

Some PLCs provide transitional contacts or one-shot instructions in addition to the standard NO and NC contact instructions. The *off-to-on* transitional contact instruction, shown in Figure 8-17a, is programmed to provide a one-shot pulse when the referenced trigger signal makes a positive (off-to-on) transition. This contact will close for exactly one program scan whenever the trigger signal

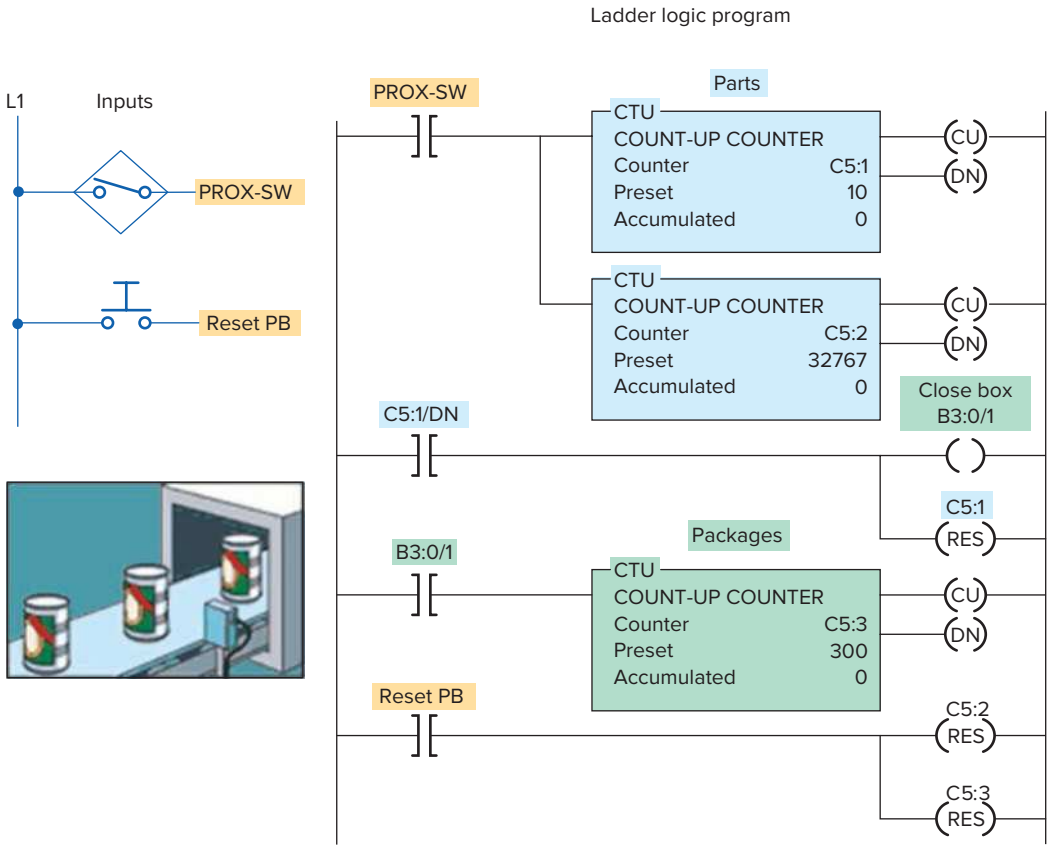
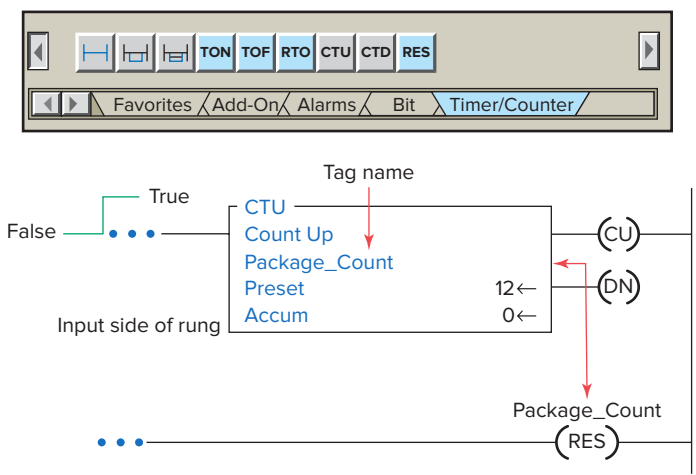


Figure 8-14 Can-counting program.



	Tag Name	Alias For	Base Tag	Type	Style	Description
	Package_count			COUNTER	Decimal	12 can counter
	Package_count.PRE			DINT	Decimal	
	Package_count.ACC			DINT	Decimal	
	Package_count.CU			BOOL	Decimal	
	Package_count.CD			BOOL	Decimal	
	Package_count.DN			BOOL	Decimal	
	Package_count.OV			BOOL	Decimal	
	Package_count.UN			BOOL	Decimal	

Figure 8-15 ControlLogix counter instruction.

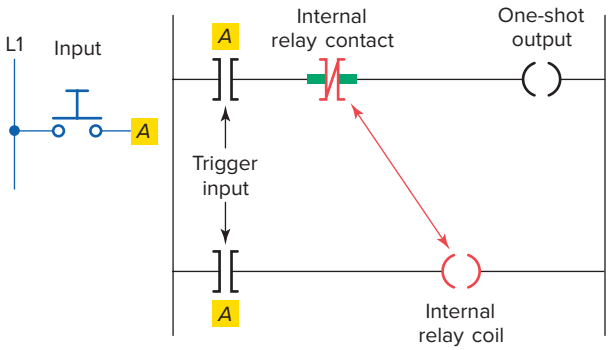
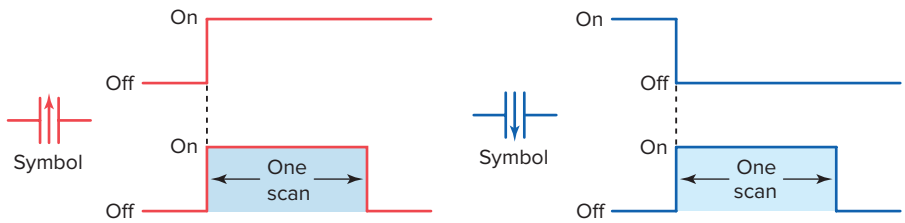


Figure 8-16 One-shot, or transitional, contact program.

goes from off to on. The contact will allow logic continuity for one scan and then open, even though the triggering signal may stay on. The *on-to-off* transitional contact, shown in Figure 8-17b, provides the same operation as the off-to-on transitional contact instruction, except that it allows logic continuity for a single scan whenever the trigger signal goes from an on to an off state.

The conveyor motor PLC program of Figure 8-18 illustrates the application of an up-counter along with a programmed one-shot (OSR) transitional contact instruction. The counter counts the number of cases coming off the conveyor. When the total number of cases reaches 50, the conveyor motor stops automatically. The trucks being loaded will take a total of only 50 cases of this particular product; however, the count can be changed for different product lines. The operation of the program can be summarized as follows:

- The momentary start button is pressed to start the conveyor motor M1.
- The passage of cases is sensed by the proximity switch.
- Cases move past the proximity switch and increment the counter's accumulated value with each false-to-true transition of the switch.



(*a*) Off-to-on transitional contact (*b*) On-to-off-transitional contact

Figure 8-17 Transitional contact instructions.

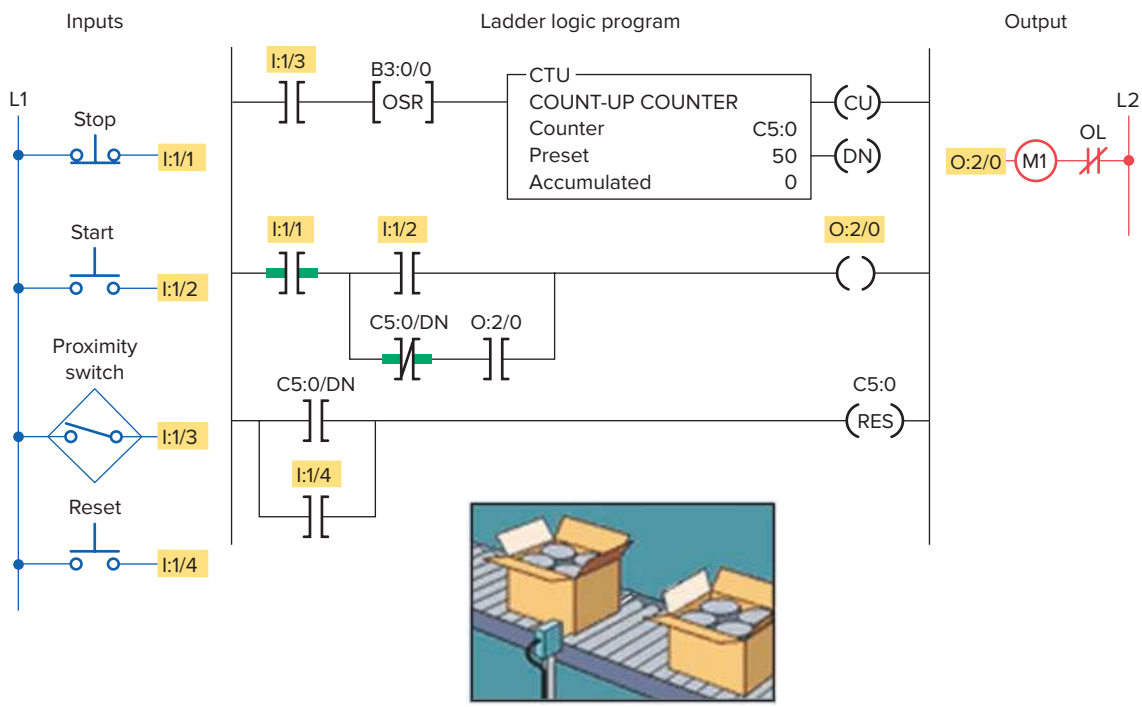


Figure 8-18 Case-counting program.

- The retentive OSR instruction is true for only one scan and then false again, even if the triggering proximity switch signal stays true. This may be required for the count pulse to operate properly.
- After a count of 50, the done bit of the counter changes state to stop the conveyor motor automatically and reset the counter's accumulated value to zero.
- The conveyor motor can be stopped and started manually at any time without loss of the accumulated count.
- The accumulated count of the counter can be reset manually at any time by means of the count reset button.

The Allen-Bradley SLC 500 **one-shot rising (OSR)** instruction is an input instruction that triggers an event to occur one time. The OSR instruction is placed in the ladder logic before the output instruction. When the rung conditions preceding the OSR instructions go from false-to-true, the OSR instruction goes true also but for only one scan. Figure 8-19 illustrates the operation of an OSR rung which can be summarized as follows:

- The OSR, one-shot rising instruction is used to make the counter reset instruction (RES) true for one scan when limit switch input LS1 goes from false to true.
- The OSR is assigned a Boolean bit (B3:0/0) that is not used anywhere else in the program.
- The OSR instruction must immediately precede the output instruction.
- When the limit switch closes the LS1 and OSR, input instructions go from false to true. The OSR instruction conditions the rung so that the counter C5:1 reset output instruction goes true for one program scan.
- The output reset instruction goes false and remains false for successive scans until the input makes another false-to-true transition.
- The OSR bit is set to 1 as long as the limit switch remains closed.
- The OSR bit is reset to 0 when the limit switch is opened.

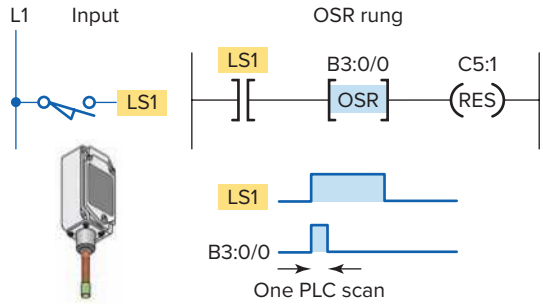


Figure 8-19 One-shot rising (OSR) instruction.

Applications for the OSR instruction include freezing rapidly displayed LED values. Figure 8-20 shows a one-shot instruction used to send data to an output LED display. The one-shot allows the rapidly changing accumulated time from the timer to be frozen to ensure a readable, stable display. The operation of the program is summarized as follows:

- The accumulated value of timer T4:1 is converted to Binary Coded Decimal (BCD) and moved to output word O:6 where an LED display is connected.

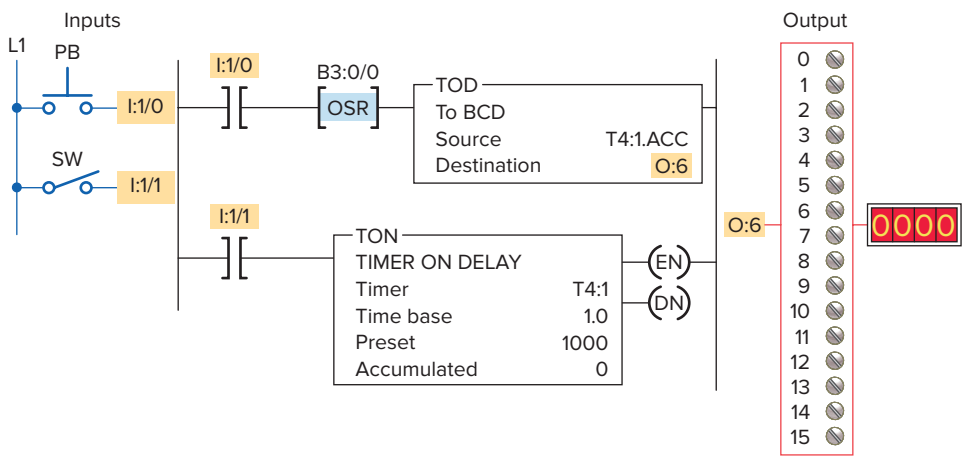


Figure 8-20 OSR instruction used to freeze rapidly displayed LED values.

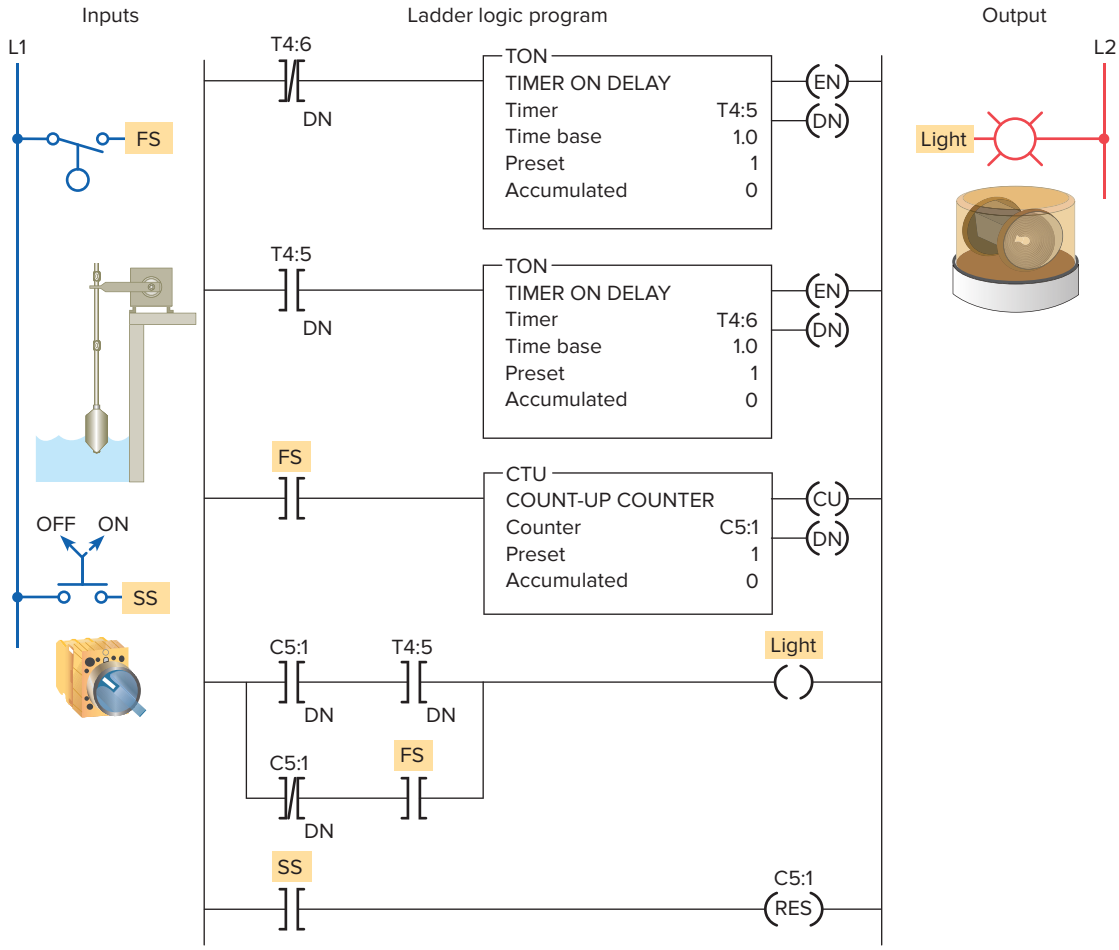


Figure 8-21 Alarm monitor program.

- When the timer is running, SW (I:1/1) closed, the accumulated value changes rapidly.
- Closing the momentary pushbutton PB (I:1/0) will freeze and display the value at that point in time.

The alarm monitor PLC program of Figure 8-21 illustrates the application of an up-counter used in conjunction with the programmed timed oscillator circuit studied in Chapter 7. The operation of the program can be summarized as follows:

- The alarm is triggered by the closing of float switch FS.
- The light will flash whenever the alarm condition is triggered and has not been acknowledged, even if the alarm condition clears in the meantime.
- The alarm is acknowledged by closing selector switch SS.
- The light will operate in the steady on mode when the alarm trigger condition still exists but has been acknowledged.

8.3 Down-Counter

The down-counter instruction will count down or decrement by 1 each time the counted event occurs. Each time the down-count event occurs, the accumulated value is decremented. Normally the down-counter is used in conjunction with the up-counter to form an up/down-counter.

Figure 8-22 shows the program and timing diagram for a generic, block-formatted up/down-counter. The operation of the program can be summarized as follows:

- Separate count-up and count-down inputs are provided.
- Assuming the preset value of the counter is 3 and the accumulated count is 0, pulsing the count-up input (PB1) three times will switch the output light from off to on.
- This particular PLC counter keeps track of the number of counts received above the preset value.

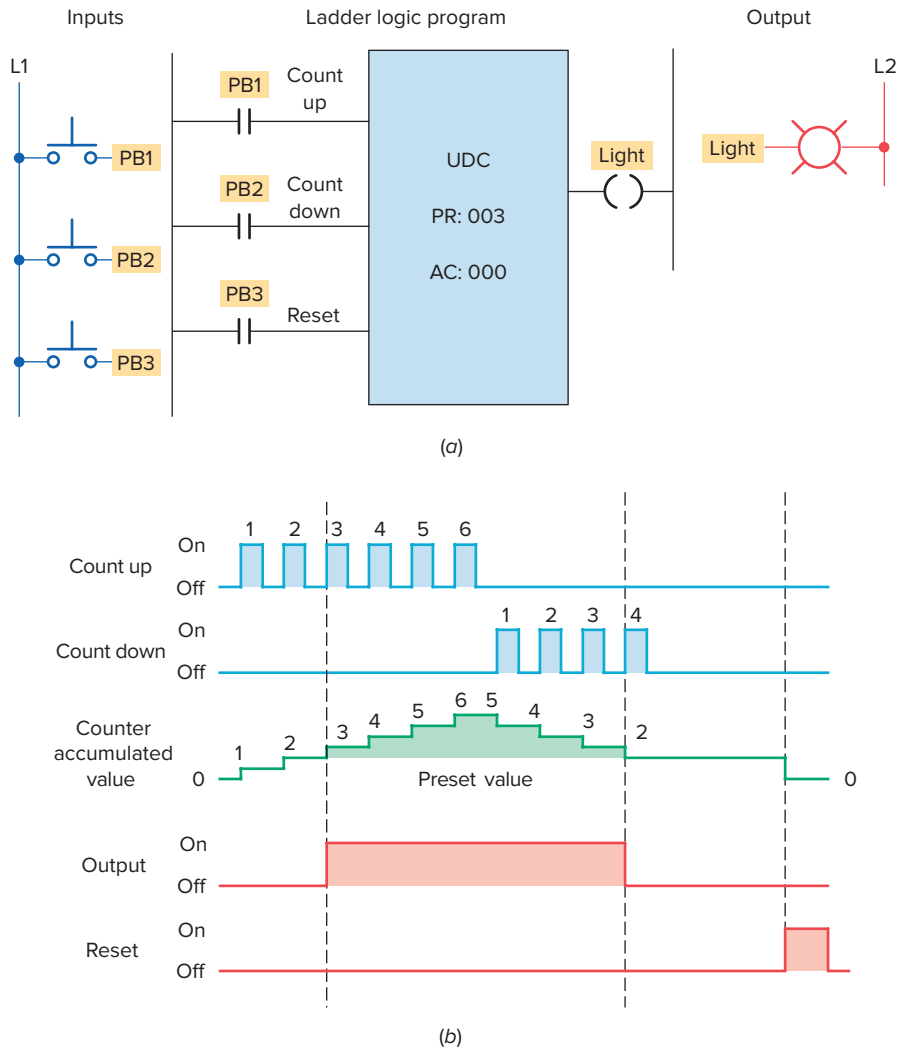


Figure 8-22 Generic up/down-counter program. (a) Program. (b) Counting diagram.

As a result, three additional pulses of the count-up input (PB1) produce an accumulated value of 6 but no change in the output.

- If the count-down input (PB2) is now pulsed four times, the accumulated count is reduced to 2 (6 − 4). As a result, the accumulated count drops below the preset count and the output light switches from on to off.
- Pulsing the reset input (PB3) at any time will reset the accumulated count to 0 and turn the output light off.

Not all counter instructions count in the same manner. Some up-counters count only to their preset values, and additional counts are ignored. Other up-counters keep track of the number of counts received above the counter's preset value. Conversely, some down-counters will simply count down to zero and no further. Other down-counters may count below zero and begin counting down from the largest preset value that can be set for the PLC's counter instruction. For example, a PLC up/down-counter that has a maximum counter preset limit of 999 may count up as follows: 997, 998, 999, 000, 001, 002, and so on. The same counter would count down in the following manner: 002, 001, 000, 999, 998, 997, and so on.

One application for an up/down-counter is to keep count of the cars that enter and leave a parking garage. Figure 8-23 shows a typical PLC program that could be used to implement this. The operation of the program can be summarized as follows:

- As a car enters, the enter switch triggers the up-counter output instruction and increments the accumulated count by 1.

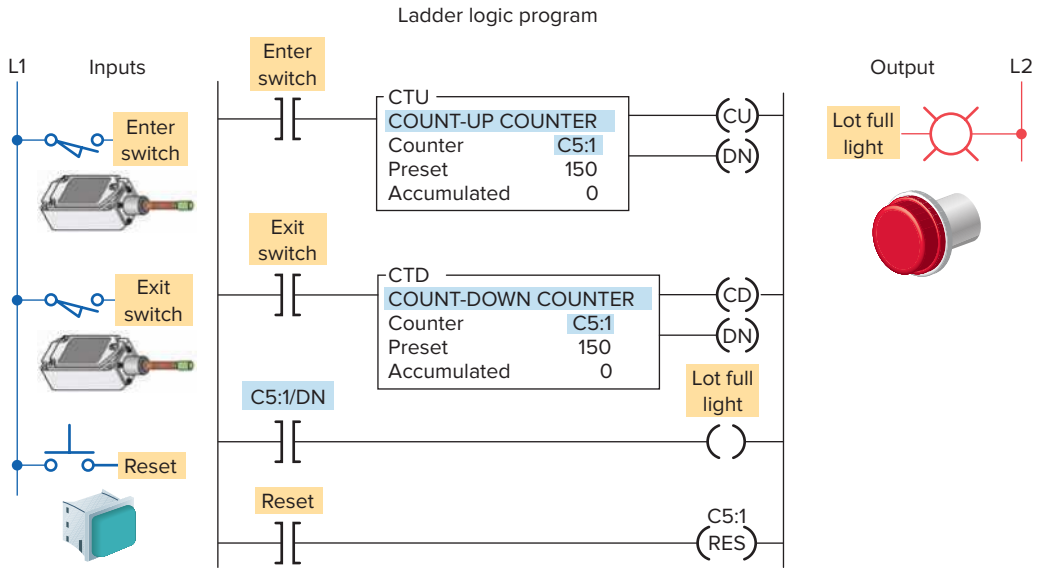


Figure 8-23 Parking garage counter.

- As a car leaves, the exit switch triggers the down-counter output instruction and decrements the accumulated count by 1.
- Because both the up- and down-counters have the same address, C5:1, the accumulated value will be the same in both instructions as well as the preset.
- Whenever the accumulated value of 150 equals the preset value of 150, the counter output is energized by the done bit to light up the Lot Full sign.
- A reset button has been provided to reset the accumulated count.

Figure 8-24 shows an example of the count-down counter instruction used as part of the Allen-Bradley SLC 500 controller instruction set. The information to be entered into the instruction is the same as for the count-up counter instruction.

The CTD instruction decrements its accumulated value by 1 every time it is transitioned. It sets its done bit when the accumulated value is equal to or greater than the preset value. The program of Figure 8-24 contains a count-down counter instruction, the operation of which can be summarized as follows:

- With the program in the state shown, the CTD done bit will be set (1) and output 0:2/0 will be energized because the accumulated value of 4 is greater than the preset value of 2.
- When the CTD instruction rung makes a false-to-true transition, the accumulated value decreases by one count to 3.

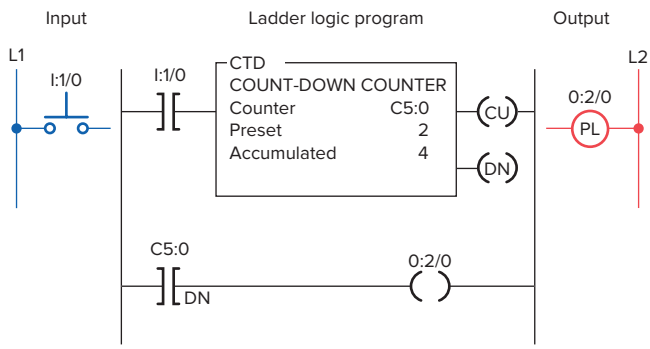


Figure 8-24 Count-down counter instruction.

- When the input rung condition makes another false-to-true transition, the accumulated value will decrease to 2.
- When the input makes one more false-to true transition, the accumulated value will drop to 1.
- At this point the accumulated value of 1 is less than the preset value of 2 so the done bit will be reset (0) de-energizing output O:2/0.

Figure 8-25 shows an up/down-counter program that will increase the counter's accumulated value when pushbutton PB1 is pressed and will decrease the counter's accumulated value when pushbutton PB2 is pressed. Note that the same address is given to the ***up-counter*** instruction, the **down-counter** instruction, and the **reset** instruction. All three instructions will be looking at the ***same address*** in the counter file. When input *A* goes from false to true, one count is added to the accumulated value. When input *B* goes from false

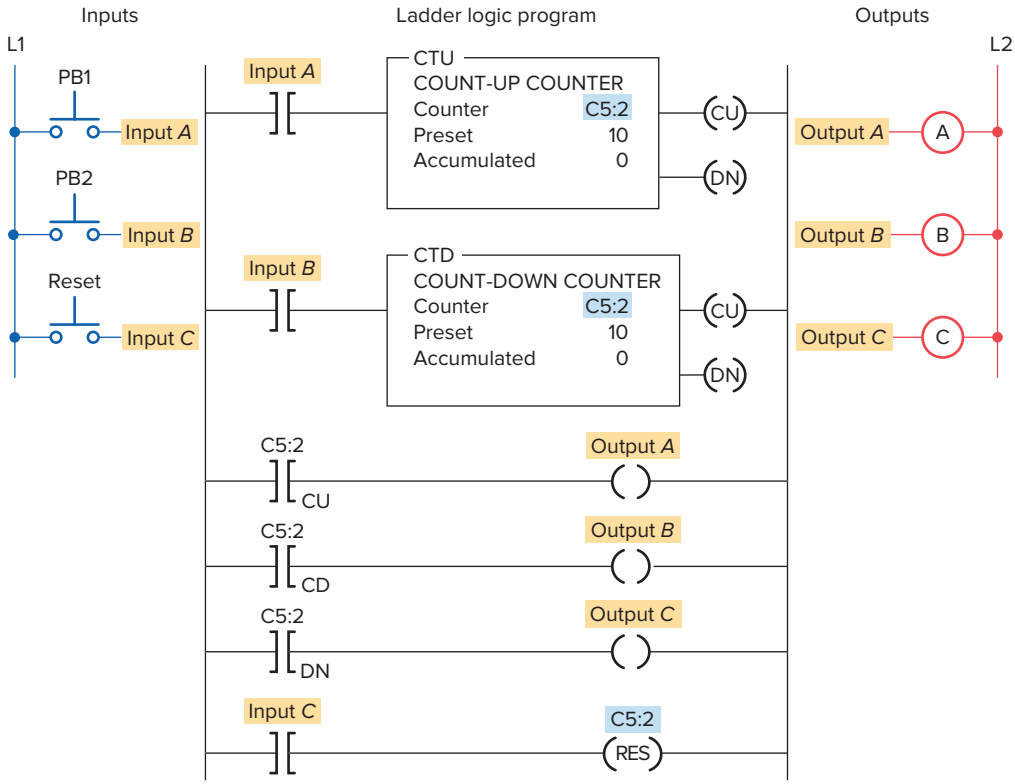


Figure 8-25 Up/down-counter program.

to true, one count is subtracted from the accumulated value. The operation of the program can be summarized as follows:

- When the CTU instruction is true, C5:2/CU will be true, causing output *A* to be true.
- When the CTD instruction is true, C5:2/CD will be true, causing output *B* to be true.
- When the accumulated value is greater than or equal to the preset value, C5:2/DN will be true, causing output *C* to be true.
- Input *C* going true will cause both counter instructions to reset. When *reset* by the **RES instruction,** the accumulated value will be reset to 0 and the done bit will be reset.

Figure 8-26 illustrates the operation of the up/down-counter program used to provide continuous monitoring of items in process. An in-feed photoelectric sensor counts raw parts going into the system, and an out-feed photoelectric sensor counts finished parts leaving the machine. The number of parts between the in-feed and out-feed is indicated by the accumulated count of the counter. Counts applied to the up-input are added, and counts applied to the down-input are subtracted. The operation of the program can be summarized as follows:

- Before start-up, the system is completely empty of parts, and the counter is reset manually to 0.
- When the operation begins, raw parts move through the in-feed sensor, with each part generating an up count.
- After processing, finished parts appearing at the out-feed sensor generate down counts, so the accumulated count of the counter continuously indicates the number of in-process parts.
- The counter preset value is irrelevant in this application. It does not matter whether the counter outputs are on or off. The output on-off logic is not used. We have arbitrarily set the counter's preset values to 50.

The maximum speed of transitions that you can count is determined by your program's scan time. For a reliable count, your counter input signal must be fixed for one scan time. If the input changes faster than one scan period, the count value will become unreliable because counts will be missed. When this situation occurs, you need to use a high-speed counter input or a separate counter I/O module designed for high-speed applications.

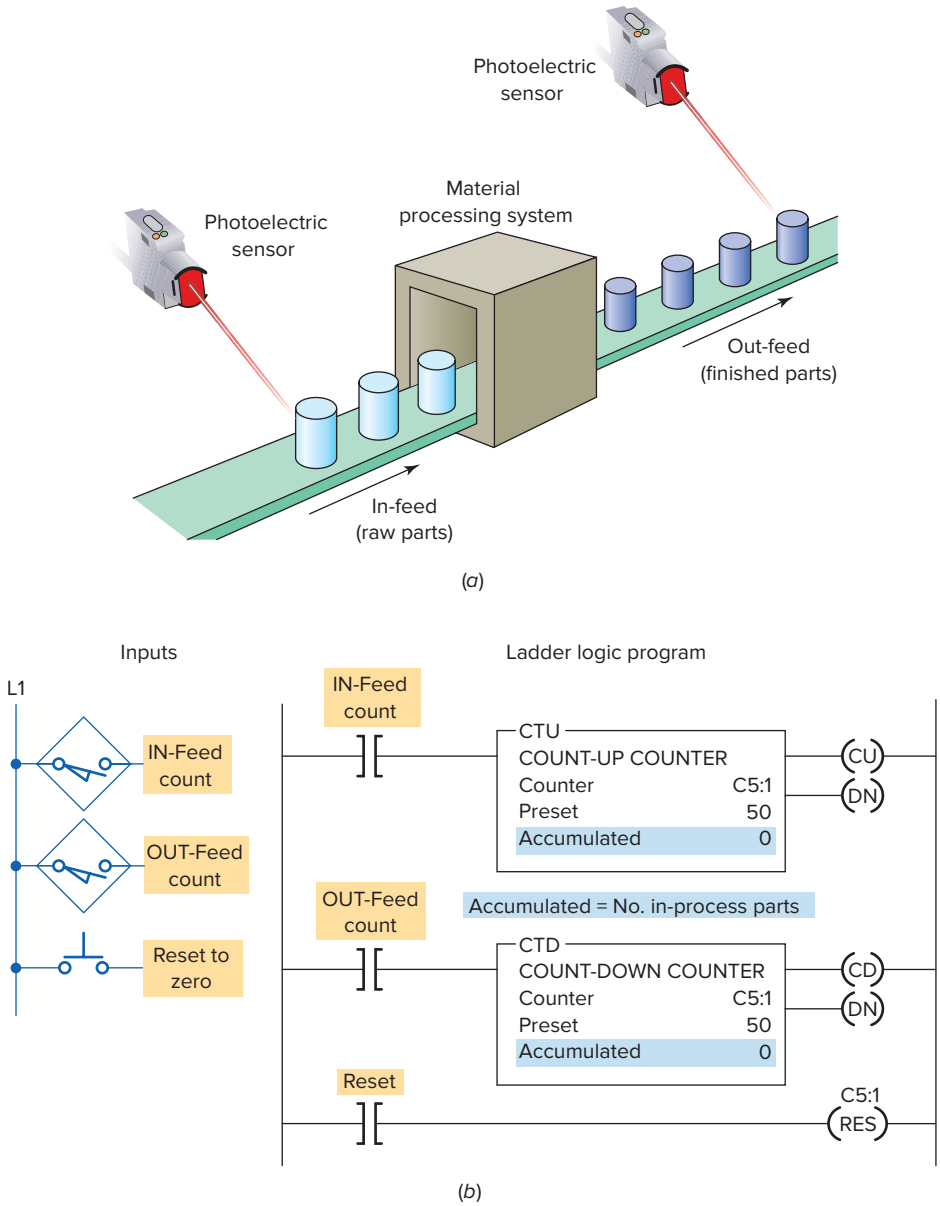


Figure 8-26 In-process monitoring program. (a) Process. (b) Program.

8.4 Cascading Counters

Depending on the application, it may be necessary to count events that exceed the maximum number allowable per counter instruction. One way of accomplishing this count is by interconnecting, or **cascading,** two counters. The program of Figure 8-27 illustrates the application of the technique. The operation of the program can be summarized as follows:

- The output of the first counter is programmed into the input of the second counter.
- When the accumulated value of the second counter is equal to its preset, the DN bit will be true, which allows the first counter to count.
- The status bits of both counters are programmed in series to produce an output.
- These two counters allow twice as many counts to be measured.
- A CTU instruction that is reset while the counter logic remains true will result in an accumulated value of 1 instead of 0. Using the OSR instruction in the counter enabling logic prevents this from happening.

Another method of cascading counters is sometimes used when an extremely large number of counts must be stored. For example, if you require a counter to count up to 250,000, it is possible to achieve this by using only two counters. Figure 8-28 shows how the

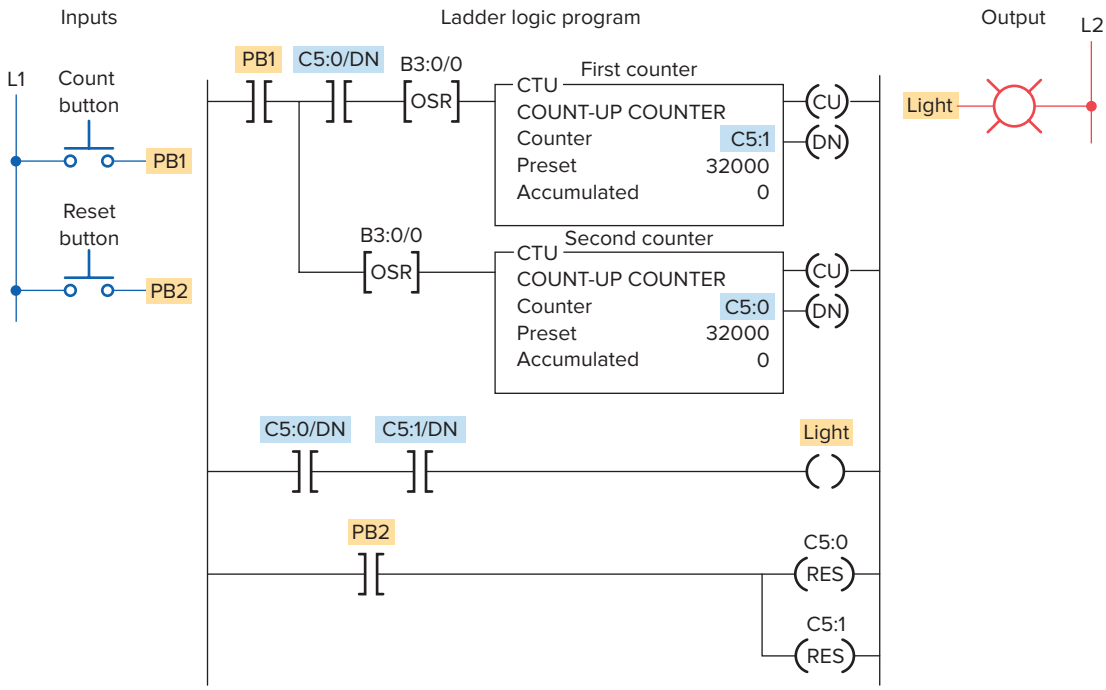


Figure 8-27 Counting beyond the maximum count.

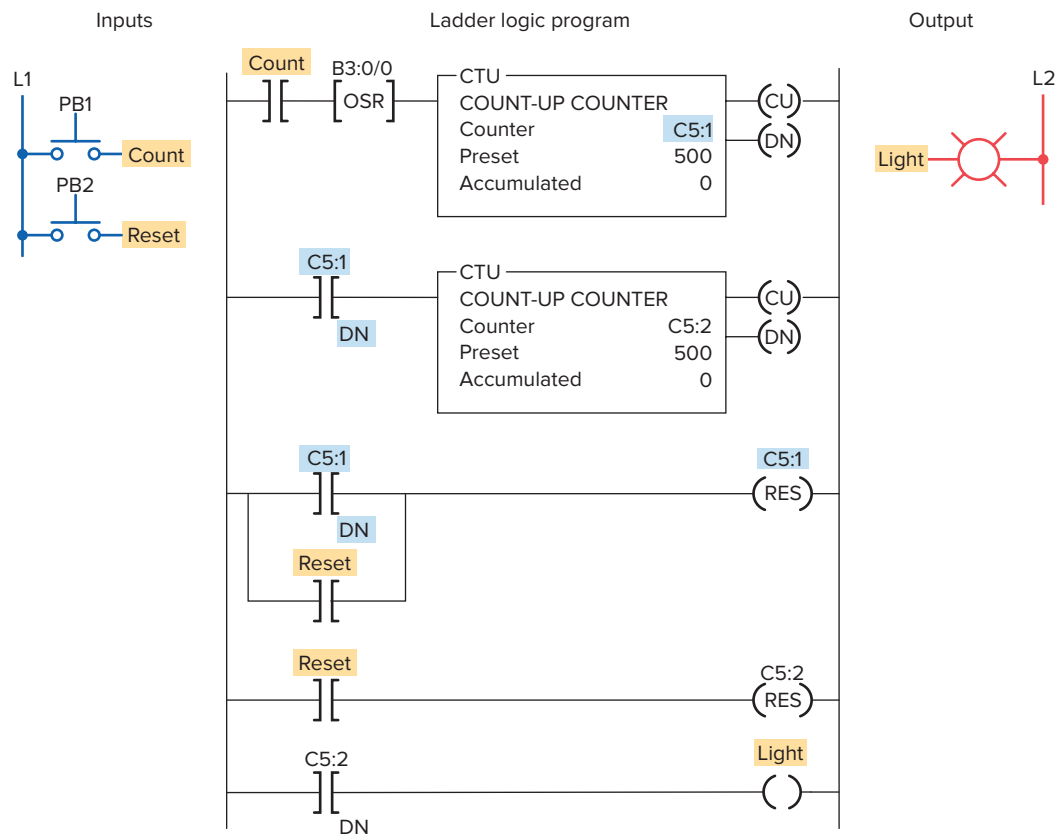


Figure 8-28 Cascading counters for extremely large counts.

two counters would be programmed for this purpose. The operation of the program can be summarized as follows:

- Counter C5:1 has a preset value of 500 and counter C5:2 has a preset value of 500.
- Whenever counter C5:1 reaches 500, its done bit resets counter C5:1 and increments counter C5:2 by 1.
- When the done bit of counter C5:1 has turned on and off 500 times, the output light becomes energized. Therefore, the output light turns on after 500 × 500, or 250,000, transitions of the count input.

Some PLCs include a real-time clock as part of their instruction set. A real-time clock allows you to display the time of day or to log data pertaining to the operation of the process. The logic used to implement a clock as part of a PLC's program is straightforward and simple to accomplish. A single timer instruction and counter instructions are all you need.

Figure 8-29 illustrates a timer-counter program that produces a time-of-day clock measuring time in hours and minutes. The operation of the program can be summarized as follows:

- An RTO timer instruction (T4:0) is programmed first with a preset value of 60 seconds.
- The T4:0 timer times for a 60-second period, after which its done bit is set.
- This, in turn, causes the up-counter (C5:0) of rung 001 to increment 1 count.
- On the next processor scan, the timer is reset and begins timing again.
- The C5:0 counter is preset to 60 counts, and each time the timer completes its time-delay period, its count is incremented.
- When the C5:0 counter reaches its preset value of 60, its done bit is set.
- This, in turn, causes the up-counter (C5:1) of rung 002, which is preset for 24 counts, to increment 1 count.
- Whenever the C5:1 counter reaches its preset value of 24, its done bit is set to reset itself.
- The time of day is generated by examining the current, or accumulated, count or time for each counter and the timer.
- Counter C5:1 indicates the hour of the day in 24-h military format, while the current minutes are represented by the accumulated count value of counter C5:0.
- The timer displays the seconds of a minute as its current, or accumulated, time value.

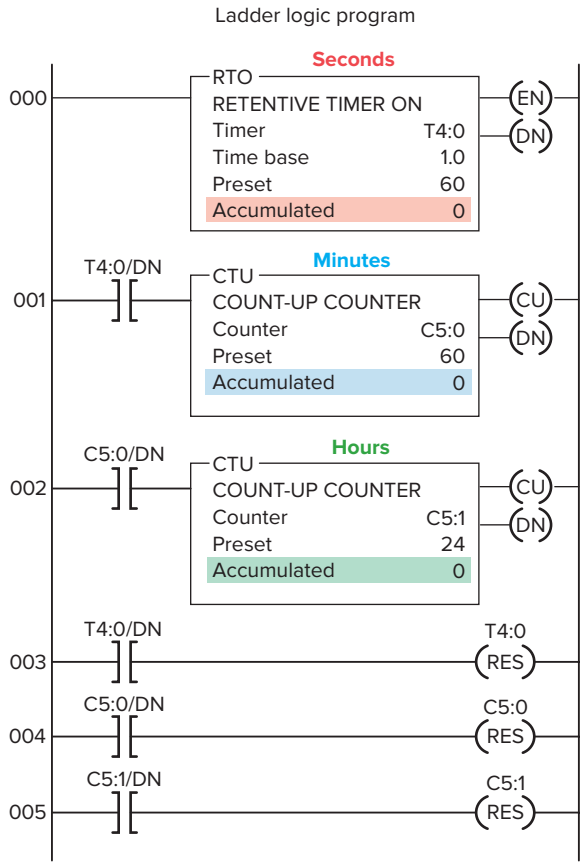


Figure 8-29 24-hour clock program.

The 24-hour clock can be used to record the time of an event. Figure 8-30 illustrates the principle of this technique. In this application the time of the opening of a pressure switch is to be recorded. The operation of the program can be summarized as follows:

- The circuit is set into operation by pressing the reset button and setting the clock for the time of day.
- This starts the 24-hour clock and switches the set indicating light on.
- Should the pressure switch open at any time, the clock will automatically stop and the trip indicating light will switch on.
- The clock can then be read to determine the time of opening of the pressure switch.

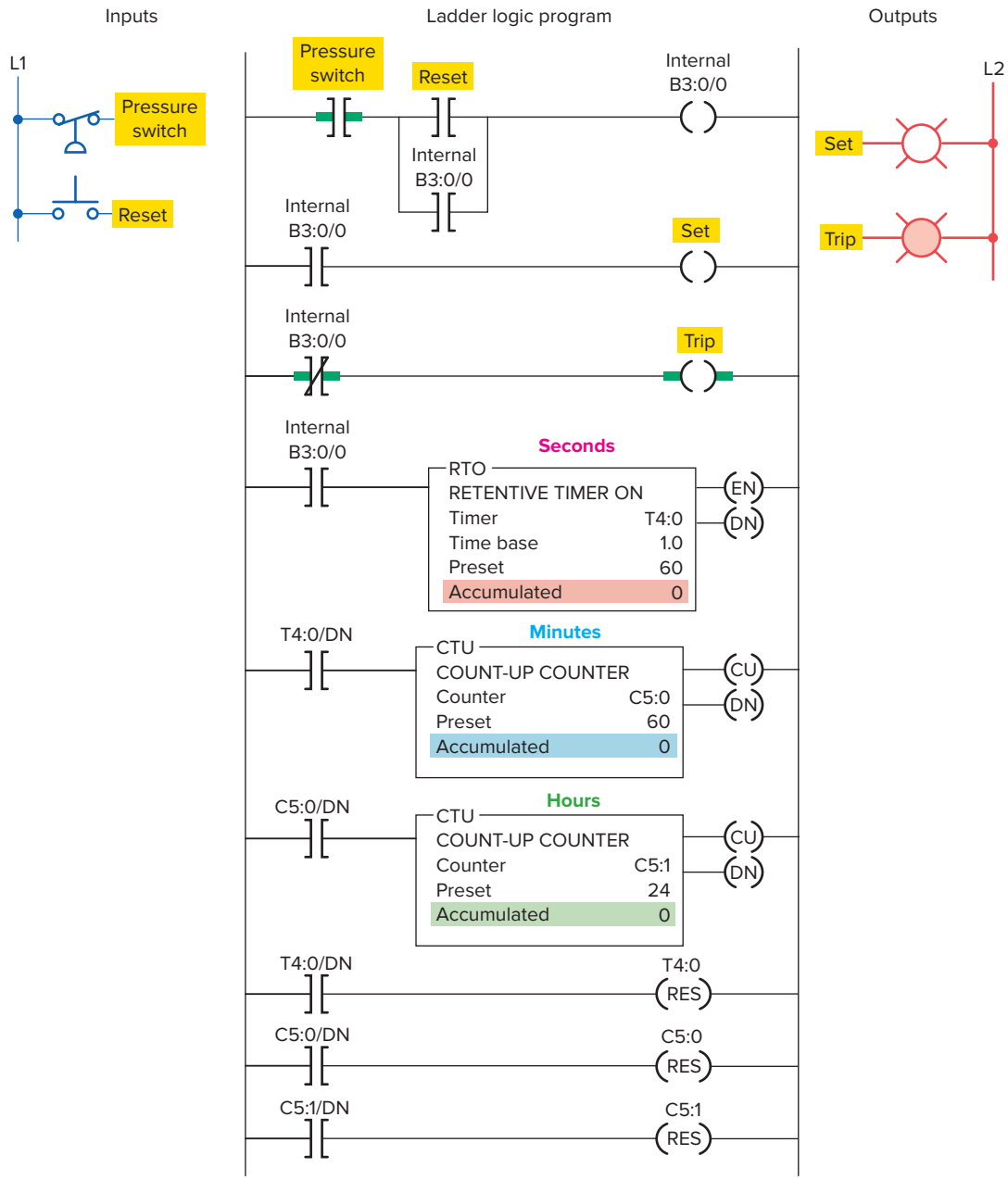


Figure 8-30 Monitoring the time of an event.

8.5 Incremental Encoder-Counter Applications

Incremental encoders are used to track motion. They provide a specific number of equally spaced pulses per revolution or per inch or millimeter of linear motion. Incremental encoders output pulses each time and only when the shaft is turned. The **incremental optical encoder** shown in Figure 8-31creates a series of square waves as its shaft is rotated. The encoder disk interrupts the light as the encoder shaft is rotated to produce the square wave output waveform.

The number of square waves obtained from the output of the encoder can be made to correspond to the mechanical movement required. For example, to divide a shaft revolution into 100 parts, an encoder could be selected to supply 100 square wave-cycles per revolution. By using a counter to count those cycles, we could tell how far the shaft had rotated.

Figure 8-32 illustrates an example of cutting objects to a specified length. The object is advanced for a specified distance and measured by encoder pulses to determine the correct length for cutting.

Figure 8-33 shows a counter program used for length measurement. This system accumulates the total length

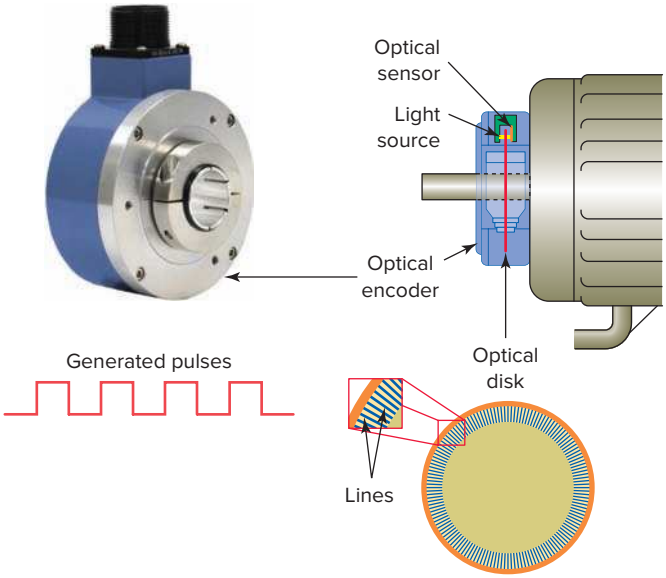


Figure 8-31 Optical incremental encoder.
Source: Courtesy of Nidec Avtron Automation.

of random pieces of bar stock moved on a conveyor. The operation of the program can be summarized as follows:

- Count input pulses are generated by the magnetic sensor, which detects passing teeth on a conveyor drive sprocket.
- If 10 teeth per foot of conveyor motion pass the sensor, the accumulated count of the counter would indicate feet in tenths.

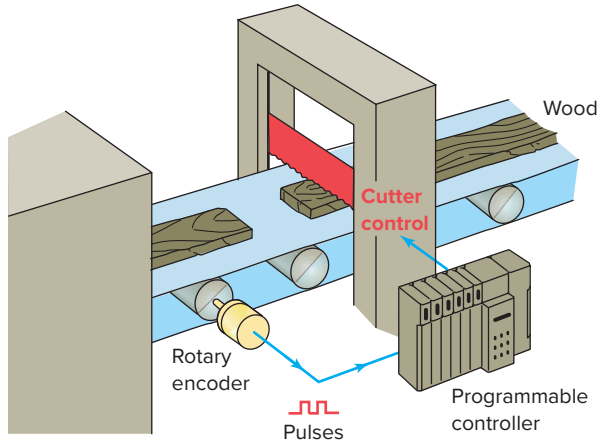


Figure 8-32 Cutting objects to a specified length.

- The photoelectric sensor monitors a reference point on the conveyor. When activated, it prevents the unit from counting, thus permitting the counter to accumulate counts only when bar stock is moving.
- The counter is reset by closing the reset button.

8.6 Combining Counter and Timer Functions

Many PLC applications use both the counter function and the timer function. Figure 8-34 illustrates an automatic stacking program that requires both a timer and counter.

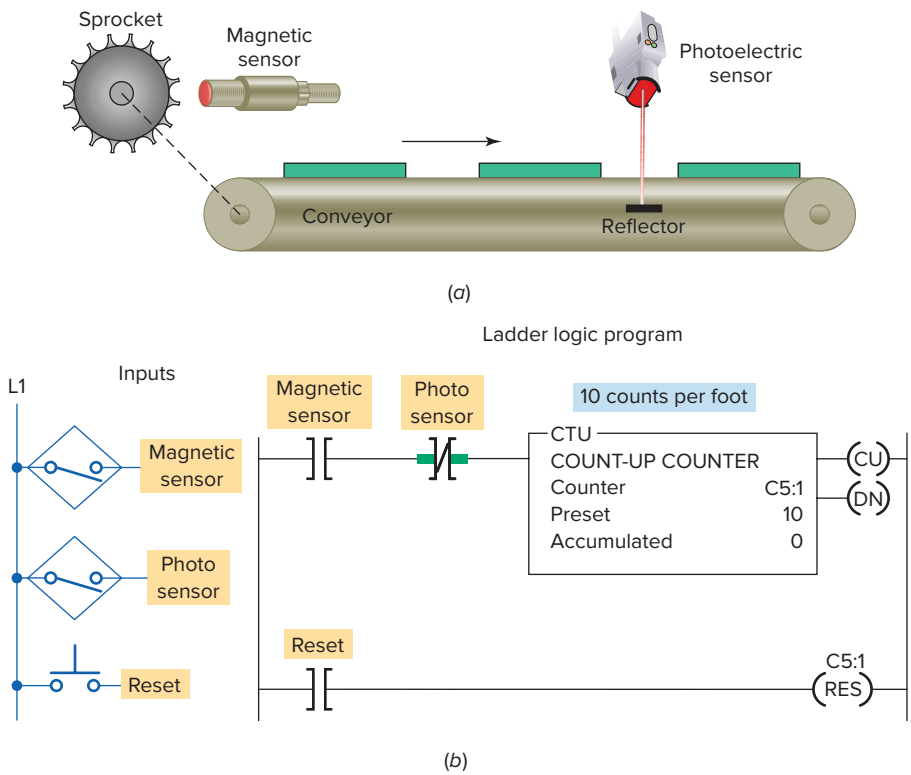


Figure 8-33 Counter used for length measurement. (a) Process. (b) Program.

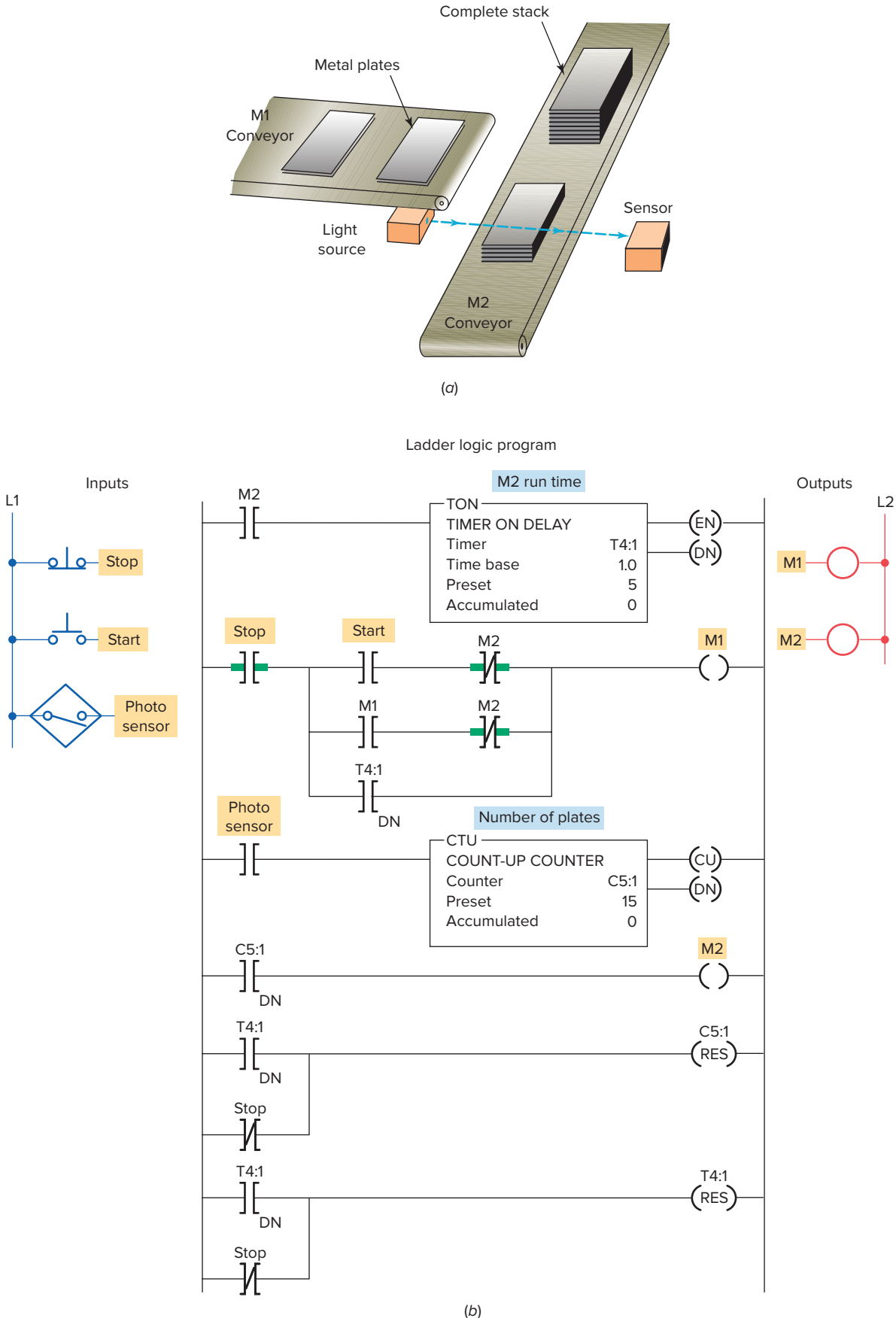


Figure 8-34 Automatic stacking program. (a) Process. (b) Program.

In this process, conveyor M1 is used to stack metal plates onto conveyor M2. The photoelectric sensor provides an input pulse to the PLC counter each time a metal plate drops from conveyor M1 to M2. When 15 plates have been stacked, conveyor M2 is activated for 5 s by the PLC timer. The operation of the program can be summarized as follows:

- When the start button is pressed, conveyor M1 begins running.
- After 15 plates have been stacked, conveyor M1 stops and conveyor M2 begins running.
- After conveyor M2 has been operated for 5 s, it stops and the sequence is repeated automatically.
- The done bit of the timer resets the timer and the counter and provides a momentary pulse to automatically restart conveyor M1.

Figure 8-35 shows a motor lock-out program. This program is designed to prevent a machine operator from starting a motor that has tripped off more than 5 times in an hour. The operation of the program can be summarized as follows:

- The normally open overload (OL) relay contact momentarily closes each time an overload current is sensed.

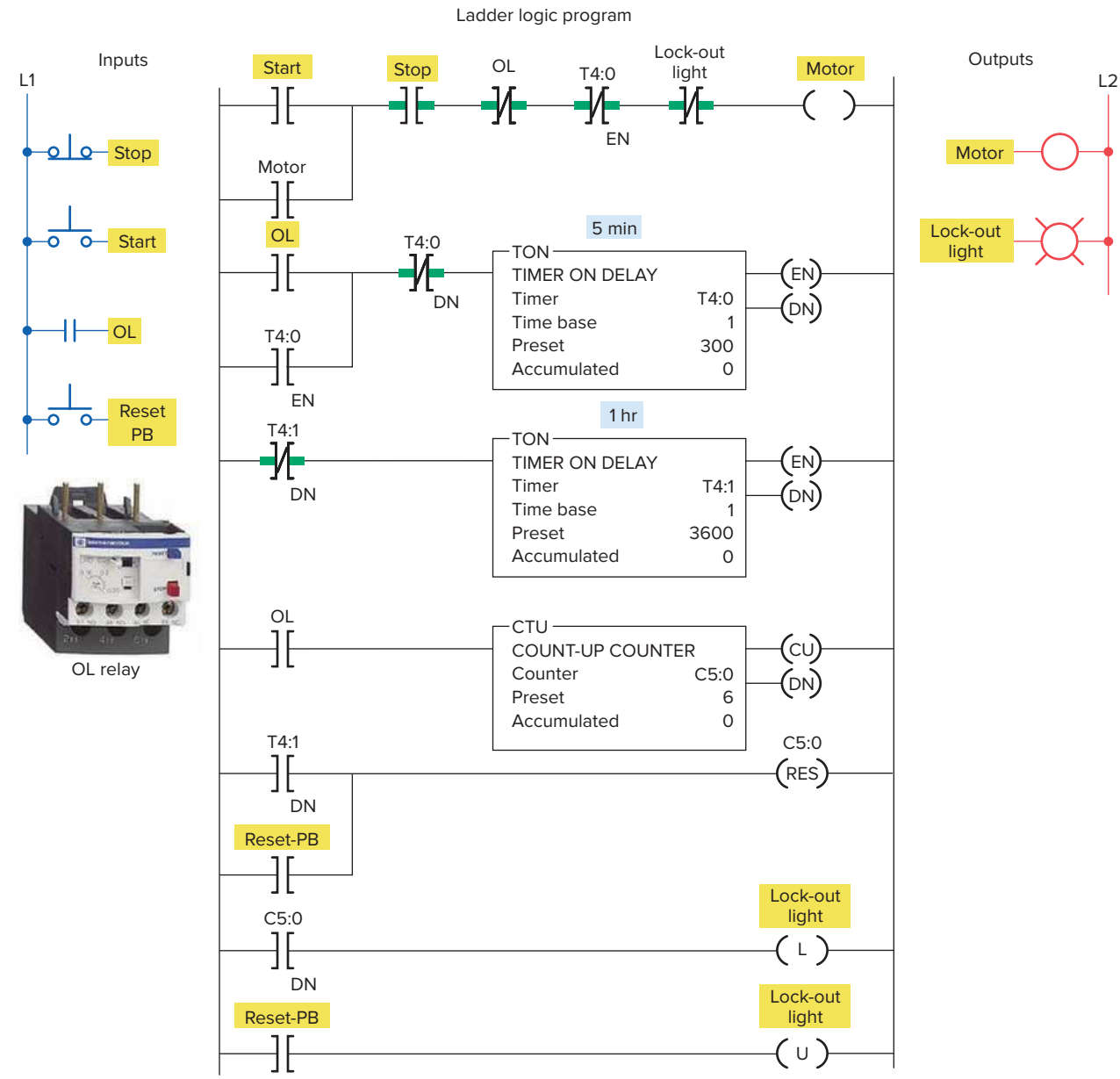


Figure 8-35 Motor lock-out program.

Source: This material and associated copyrights are proprietary to, and used with the permission of Schneider Electric.

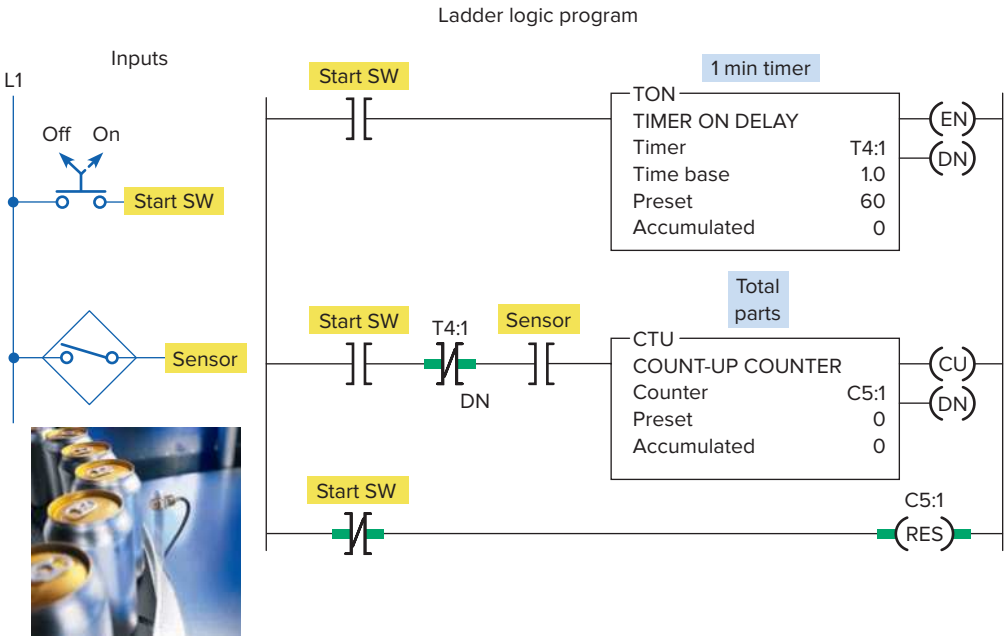


Figure 8-36 Product flow rate program.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

- Every time the motor stops due to an overload condition, the motor start circuit is locked out for 5 min.
- If the motor trips off more than 5 times in an hour, the motor start circuit is permanently locked out and cannot be started until the reset button is actuated.
- The lock-out pilot light is switched on whenever a permanent lock-out condition exists.

Figure 8-36 shows a product part flow rate program. This program is designed to indicate how many parts pass a given process point per minute. The operation of the program can be summarized as follows:

- When the start switch is closed, both the timer and counter are enabled.
- The counter is pulsed for each part that passes the parts sensor.
- The counting begins and the timer starts timing through its 1-minute time interval.
- At the end of 1 minute, the timer done bit causes the counter rung to go false.
- Sensor pulses continue but do not affect the PLC counter.
- The number of parts for the past minute is represented by the accumulated value of the counter.
- The sequence is reset by momentarily opening and closing the start switch.

A timer is sometimes used to drive a counter when an extremely long time-delay period is required. For example, if you require a timer to time to 1,000,000 s, you can achieve this by using a single timer and counter. Figure 8-37 shows how the timer and counter would be programmed for such a purpose. The operation of the program can be summarized as follows:

- Timer T4:0 has a preset value of 10,000, and counter C5:0 has a preset value of 100.
- Each time the timer T4:0 input contact closes for 10,000 s, its done bit resets timer T4:0 and increments counter C5:0 by 1.
- When the done bit of timer T4:0 has turned on and off 100 times, the output light becomes energized.
- Therefore, the output light turns on after 10,000 × 100, or 1,000,000, seconds after the timer input contact closes.

8-7 High-Speed Counters

The maximum counting frequency of a traditional PLC's counter is limited by the scan time of the processor. When the frequency of the input signal is higher than that of the scan time, it is necessary to utilize **a high-speed counter (HSC),** to avoid errors. For example, using an incremental encoder in a length-measuring application generally requires the use of a high-speed counter. The

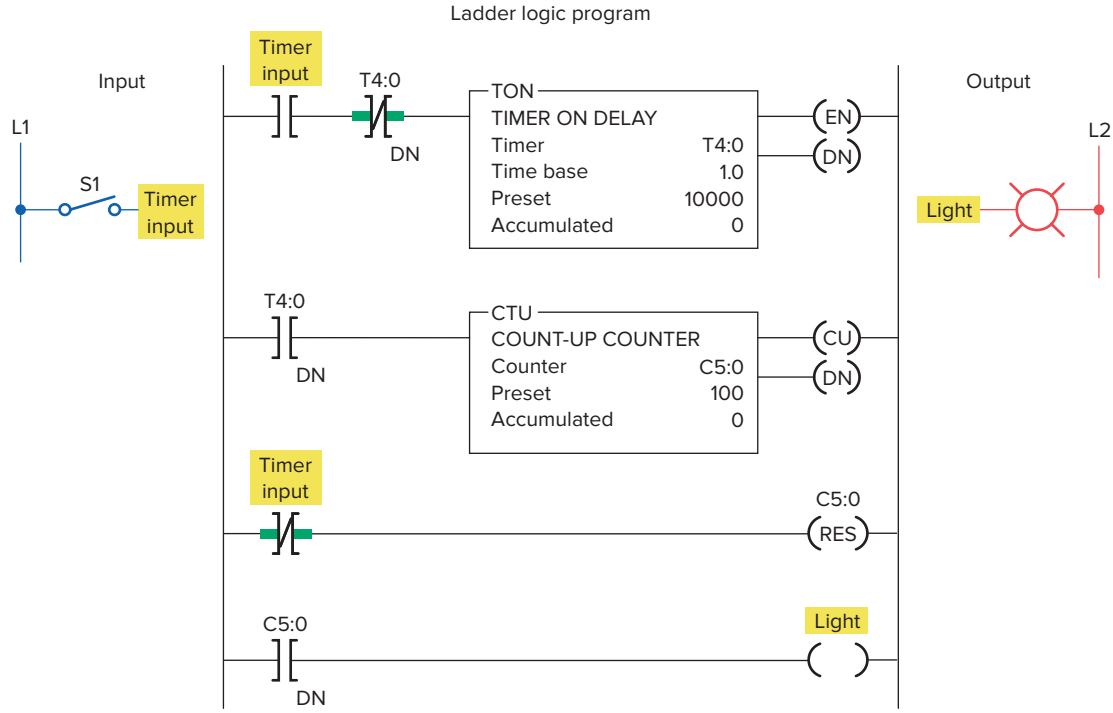


Figure 8-37 Timer driving a counter to produce an extremely long time-delay period.

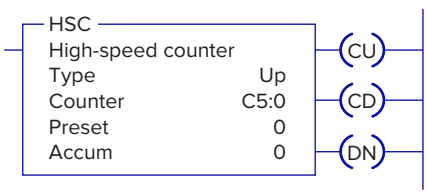


Figure 8-38 Program for Problem 1.

HSC instruction may be imbedded in the CPU, or fixed hardware, or a separate module.

Figure 8-38 shows a high-speed up-counter instruction for an Allen-Bradley MicroLogix controller. This particular controller has an imbedded high-speed counter that is able to perform counts of events between the scan of the program. Then, when the program actually scans through it can see the count value that the counter has reached.

- The controller has one 20 KHz high-speed counter, which means it would be able to count **20,000 pulses per second.**
- The high-speed counter operates independently of the controller scan.
- The HSC instruction is used to configure, control, and monitor the controller's internal hardware counter. Only one HSC instruction can be used in a program.
- The high-speed counter instruction address is fixed at C5:0.
- This counter instruction can be programmed as either an up-counter or bidirectional (Up/Down) counter.
- The hardware counter's accumulator increments or decrements in response to external input signals.
- The input filter response time is the time from the external input voltage reaching an on or off state to the micro controller recognizing that change of state. The higher you set the response time, the longer it takes for the input state change to reach the micro controller. However, setting higher response times also provides better filtering of high frequency noise.
- When the high-speed counter is enabled, data table counter C5:0 is used by the ladder program for monitoring the high-speed counter accumulator and status.

CHAPTER 8 REVIEW QUESTIONS

1. Name the three forms of PLC counter instructions, and explain the basic operation of each.
2. State four pieces of information usually associated with a PLC counter instruction.
3. In a PLC counter instruction, what rule applies to the addressing of the counter and reset instructions?
4. When is the output of a PLC counter energized?
5. When does the PLC counter instruction increment or decrement its current count?
6. The counter instructions of PLCs are normally retentive. Explain what this means.
7. **a.** Compare the operation of a standard Examine-on contact instruction with that of an off-to-on transitional contact.
   b. What is the normal function of a transitional contact used in conjunction with a counter?
8. Explain how an OSR (one-shot rising) instruction can be used to freeze rapidly changing data.
9. Identify the type of counter you would choose for each of the following situations:
   a. Count the total number of parts made during each shift.
   b. Keep track of the current number of parts in a stage of a process as they enter and exit.
   c. There are 10 parts in a full hopper. As parts leave, keep track of the number of parts remaining in the hopper
10. Describe the basic programming process involved in the cascading of two counters.
11. **a.** When is the overflow bit of an up-counter set?
    b. When is the underflow bit of a down-counter set?
12. Describe two common applications for counters.
13. What determines the maximum speed of transitions that a PLC counter can count? Why?

CHAPTER 8 PROBLEMS

1. Study the ladder logic program in Figure 8-39, and answer the questions that follow:
   a. What type of counter has been programmed?
   b. When would output O:2/0 be energized?
   c. When would output O:2/1 be energized?
   d. Suppose your accumulated value is 24 and you lose ac line power to the controller. When power is restored to your controller, what will your accumulated value be?
   e. Rung 4 goes true and, while it is true, rung 1 goes through five false-to-true transitions of rung conditions. What is the accumulated value of the counter after this sequence of events?
   f. When will the count be incremented?
   g. When will the count be reset?

Ladder logic program

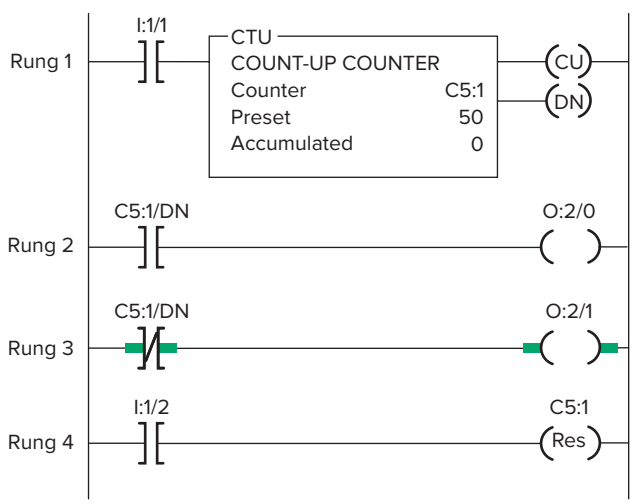


Figure 8-39 Program for Problem 1.

2. Study the ladder logic program in Figure 8-40, and answer the questions that follow:
   a. Suppose the input pushbutton is actuated from off to on and remains held on. How will the status of output B3:0/9 be affected?
   b. Suppose the input pushbutton is now released to the normally off position and remains off. How will the status of output B3:0/9 be affected?
3. Study the ladder logic program in Figure 8-41, and answer the questions that follow:
   a. What type of counter has been programmed?
   b. What input address will cause the counter to increment?

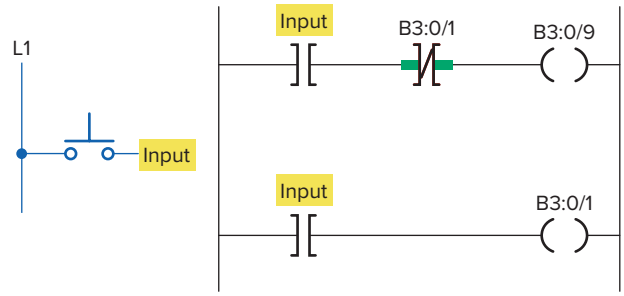


Figure 8-40 Program for Problem 2.

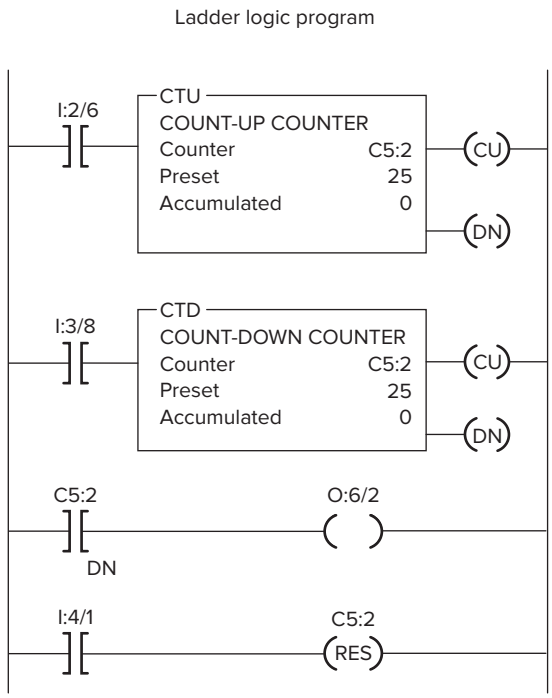


Figure 8-41 Program for Problem 3.

c. What input address will cause the counter to decrement?
d. What input address will reset the counter to a count of zero?
e. When would output O:6/2 be energized?
f. Suppose the counter is first reset, and then input I:2/6 is actuated 15 times and input I:3/8 is actuated 5 times. What is the accumulated count value?

4. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program for the following counter specifications:
- Counts the number of times a pushbutton is closed.
- Decrements the accumulated value of the counter each time a second pushbutton is closed.
- Turns on a light anytime the accumulated value of the counter is less than 20.
- Turns on a second light when the accumulated value of the counter is equal to or greater than 20.
- Resets the counter to 0 when a selector switch is closed.

5. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program that will execute the following control circuit correctly:
- Turns on a nonretentive timer when a switch is closed (preset value of timer is 10 s).
- Resets timer automatically through a programmed transitional contact when it times out.
- Counts the number of times the timer goes to 10 s.
- Resets counter automatically through a second programmed transitional contact at a count of 5.
- Latches on a light at the count of 5.
- Resets light to off and counter to 0 when a selector switch is closed.

6. Design a PLC program and prepare a typical I/O connection diagram and ladder logic program that will correctly execute the industrial control process in Figure 8-42. The sequence of operation is as follows:
- Product in position (limit switch LS1 contacts close).
- The start button is pressed and the conveyor motor starts to move the product forward toward position *A* (limit switch LS1 contacts open when the actuating arm returns to its normal position).
- The conveyor moves the product forward to position *A* and stops (position detected by 8 off-to-on output pulses from the encoder, which are counted by an up-counter).
- A time delay of 10 s occurs, after which the conveyor starts to move the product to limit switch LS2 and stops (LS2 contacts close when the actuating arm is hit by the product).

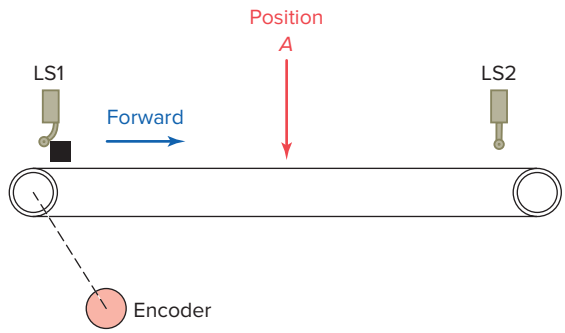


Figure 8-42 Control process for Problem 6.

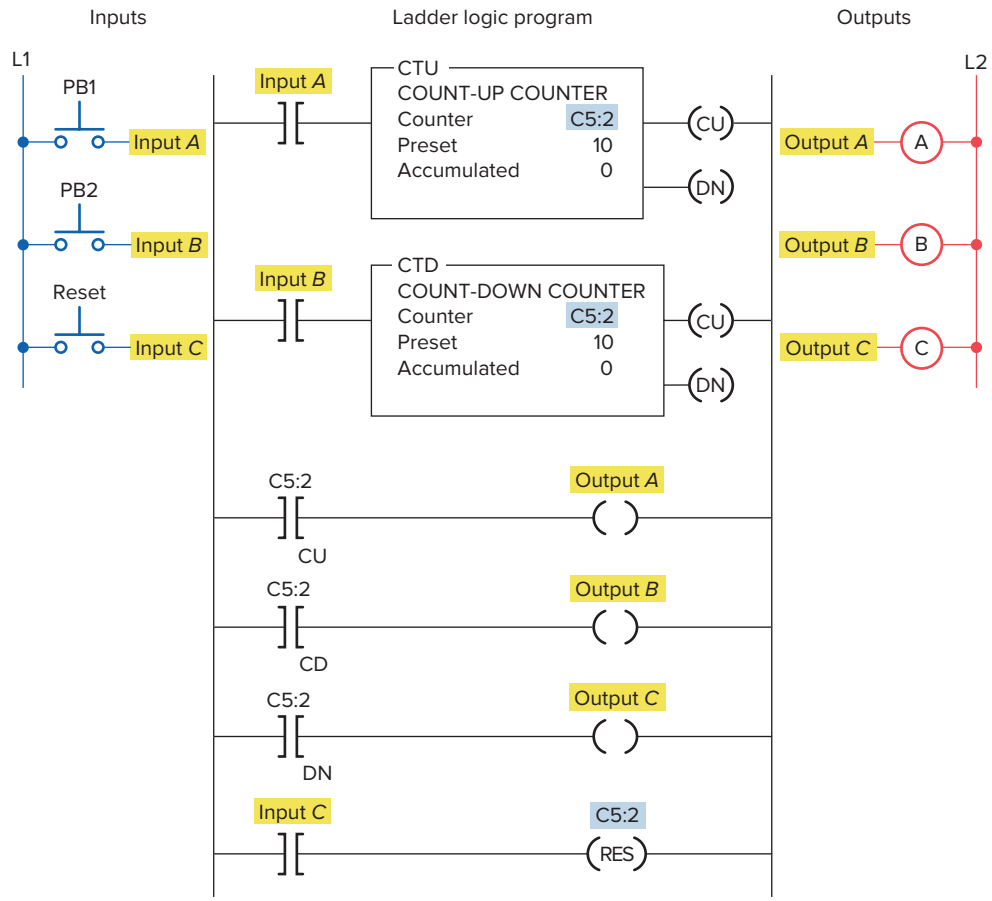


Figure 8-43 Program for Problem 7.

- An emergency stop button is used to stop the process at any time.
- If the sequence is interrupted by an emergency stop, counter and timer are reset automatically.

7. Answer the following questions with reference to the up/down-counter program shown in Figure 8-43. Assume that the following sequence of events occurs:

- Input *C* is momentarily closed.
- 20 on/off transitions of input *A* occur.
- 5 on/off transitions of input *B* occur.

As a result:

a. What is the accumulated count of counter CTU?
b. What is the accumulated count of counter CTD?
c. What is the state of output *A*?
d. What is the state of output *B*?
e. What is the state of output *C*?

8. Write a program to implement the process illustrated in Figure 8-44. An up-counter must be programmed as part of a batch-counting operation to sort parts automatically for quality control. The counter is installed to divert 1 part out of every

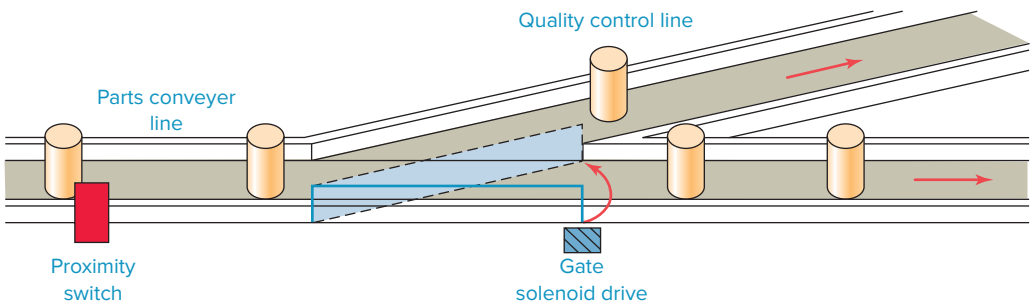


Figure 8-44 Control process for Problem 8.

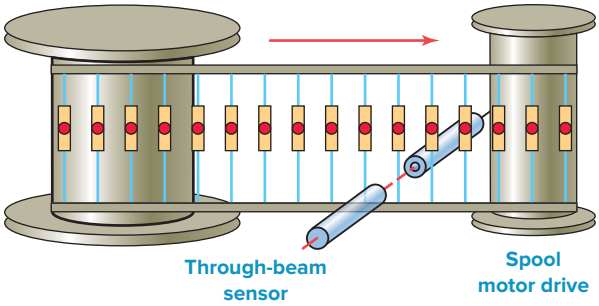


Figure 8-45 Control process for Problem 10.

1000 for quality control or inspection purposes. The circuit operates as follows:

- A start/stop pushbutton station is used to turn the conveyor motor on and off.
- A proximity sensor counts the parts as they pass by on the conveyor.
- When a count of 1000 is reached, the counter's output activates the gate solenoid, diverting the part to the inspection line.
- The gate solenoid is energized for 2 s, which allows enough time for the part to continue to the quality control line.
- The gate returns to its normal position when the 2-s time period ends.
- The counter resets to 0 and continues to accumulate counts.
- A reset pushbutton is provided to reset the counter manually.

9. Write a program that will increment a counter's accumulated value 1 count every 60 s. A second counter's accumulated value will increment 1 count every time the first counter's accumulated value reaches 60. The first counter will reset when its accumulated value reaches 60, and the second counter will reset when its accumulated value reaches 12.

10. Write a program to implement the process illustrated in Figure 8-45. A company that makes electronic assembly kits needs a counter to count and control the number of resistors placed into each kit. The controller must stop the take-up spool at a predetermined amount of resistors (100). A worker on the floor will then cut the resistor strip and place it in the kit. The circuit operates as follows:

- A start/stop pushbutton station is used to turn the spool motor drive on and off manually.
- A through-beam sensor counts the resistors as they pass by.
- A counter preset for 100 (the amount of resistors in each kit) will automatically stop the take-up spool when the accumulated count reaches 100.
- A second counter is provided to count the grand total used.
- Manual reset buttons are provided for each counter.

11. Write a program that will latch on a light 20 s after an input switch has been turned on. The timer will continue to cycle up to 20 s and reset itself until the input switch has been turned off. After the third time the timer has timed to 20 s, the light will be unlatched.

12. Write a program that will turn a light on when a count reaches 20. The light is then to go off when a count of 30 is reached.

13. Write a program to implement the box-stacking process illustrated in Figure 8-46. This application requires the control of a conveyor belt that feeds a mechanical stacker. The stacker can stack various numbers of cartons of ceiling tile onto each pallet (depending on the pallet size and the preset value of the counter). When the required number of cartons has been stacked, the conveyor is stopped until the loaded pallet is removed and an empty pallet is placed onto the loading area. A photoelectric sensor will be used to provide count pulses to the counter after each carton passes by. In addition to a conveyor motor start/stop station, a remote reset button is provided to allow the operator to reset the system from the forklift after an empty pallet is placed onto the loading area.

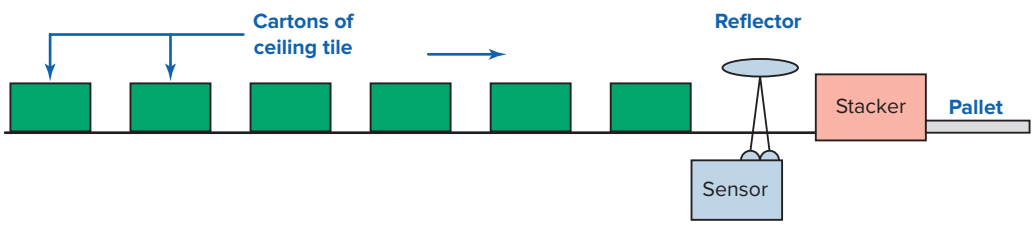


Figure 8-46 Control process for Problem 13.

The operation of this system can be summarized as follows:

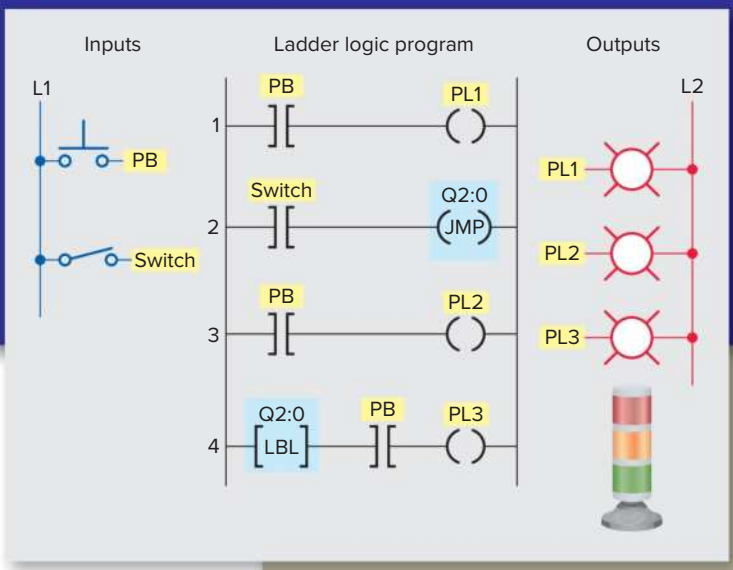
- The conveyor is started by pressing the start button.
- As each box passes the photoelectric sensor, a count is registered.
- When the preset value is reached (in this case 12), the conveyor belt turns off.
- The forklift operator removes the loaded pallet.
- After the empty pallet is in position, the forklift operator presses the remote reset button, which then starts the whole cycle over again.

14. Write a program to operate a light according to the following sequence:

- A momentary pushbutton is pressed to start the sequence.
- The light is switched on and remains on for 2 s.
- The light is then switched off and remains off for 2 s.
- A counter is incremented by 1 after this sequence.
- The sequence then repeats for a total of 4 counts.
- After the fourth count, the sequence will stop and the counter will be reset to zero.

9

Program Control Instructions



The program control instructions covered in this chapter are used to alter the program scan from its normal sequence. The use of program control instructions can shorten the time required to complete a program scan. Portions of the program not being utilized at any particular time can be jumped over, and outputs in specific zones in the program can be left in their desired states. Typical industrial program control applications are explained.

Chapter Objectives

After completing this chapter, you will be able to:

- State the purpose of program control instructions
- Describe the operation of the master control reset instruction and develop an elementary program illustrating its use
- Describe the operation of the jump instruction and the label instruction
- Explain the function of subroutines
- Describe the immediate input and output instructions function
- Describe the forcing capability of the PLC
- Describe safety considerations built into PLCs and programmed into a PLC installation
- Explain the differences between standard and safety PLCs
- Describe the function of the selectable timed interrupt and fault routine files
- Explain how the temporary end instruction can be used to troubleshoot a program

9.1 Program Control

Several output-type instructions, which are often referred to as ***override*** instructions, provide a means of executing sections of the control logic if certain conditions are met. These program control instructions allow for greater program flexibility and greater efficiency in the program scan. Portions of the program not being utilized at any particular time can be jumped over, and outputs in specific zones in the program can be left in their desired states.

Program control instructions are used to enable or disable a block of logic program or to move execution of a program from one place to another place. Figure 9-1 shows the *Program Control* menu tab for the Allen-Bradley SLC 500 PLC and its associated RSLogix software. The program control commands can be summarized as follows:

JMP (Jump to Label)—Jump forward/backward to a corresponding label instruction.

LBL (Label)—Specifies label location.

JSR (Jump to Subroutine)—Jump to a designated subroutine instruction.

RET (Return from Subroutine)—Exits current subroutine and returns to previous condition.

SBR (Subroutine)—Identifies the subroutine program.

TND (Temporary End)—Makes a temporary end that halts program execution.

MCR (Master Control Reset)—Clears all set nonretentive output rungs between the paired MCR instructions.

SUS (Suspend)—Identifies conditions for debugging and system troubleshooting.

9.2 Master Control Reset Instruction

Hardwired master control relays are used in relay control circuitry to provide input/output power shutdown of an entire circuit. Figure 9-2 shows a typical **hardwired master control relay circuit.** In this circuit, unless the master control relay coil is energized, there is no power flow to the load side of the MCR contacts.

The equivalent PLC instruction to a Master Control Relay is the **Master Control Reset (MCR)** instruction. This instruction functions in a similar manner to the hardwired master control relay; that is, when the instruction is true, the circuit functions normally, and when the instruction is false, nonretentive outputs are switched off.

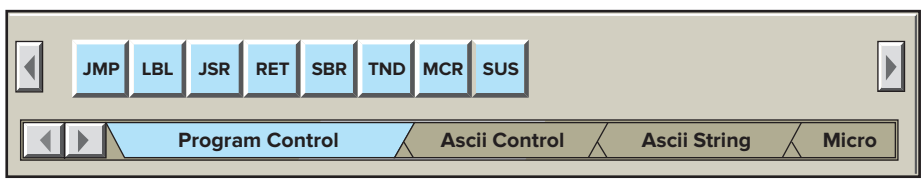


Figure 9-1 Program Control menu tab.

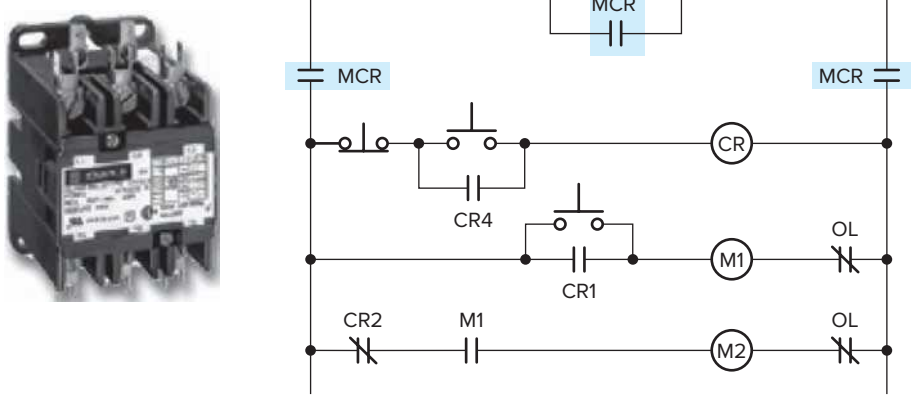


Figure 9-2 Hardwired master control relay.

Source: This material and associated copyrights are proprietary to, and used with the permission of Schneider Electric.

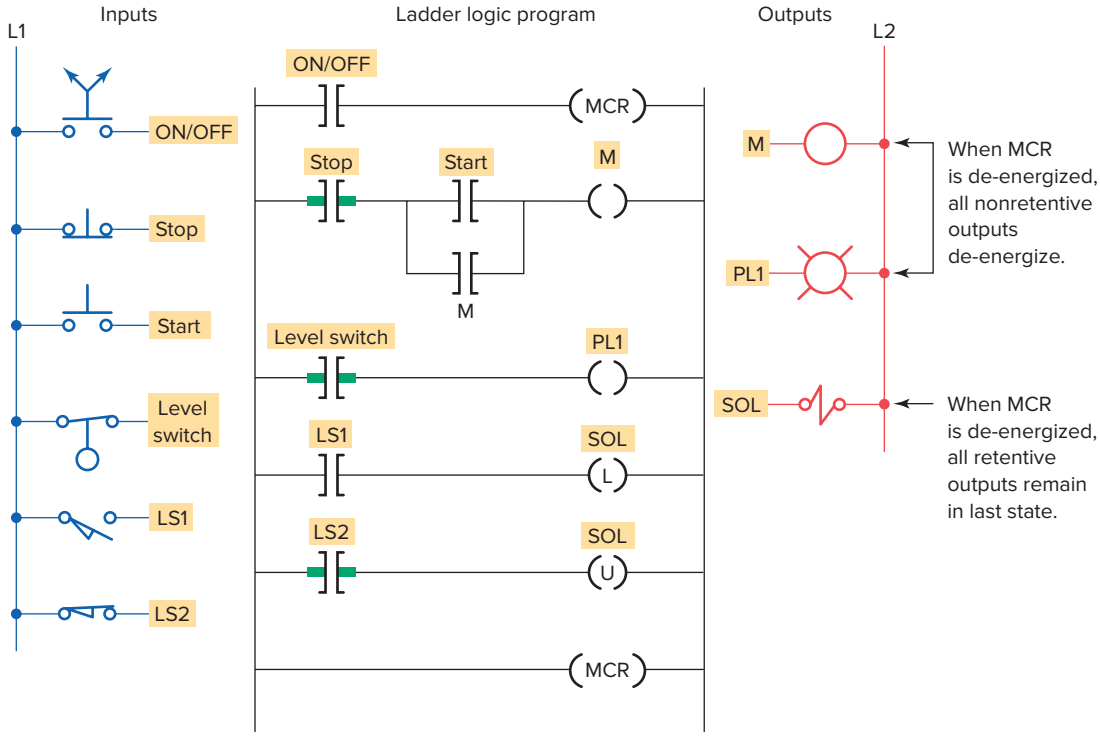


Figure 9-3 Master Control Reset (MCR) instruction.

The programmed MCR instruction is not a substitute for a hardwired Master Control Relay. It is highly recommended that all PLC systems include a hardwired MCR and Emergency Stop switches to provide safe, effective shutdown of I/O power.

A *Master Control Reset (MCR)* instruction is an output coil instruction that functions like a master control relay. MCR coil instructions are used in pairs and can be programmed to control an entire circuit or to control only selected rungs of a circuit. In the program of Figure 9-3, the MCR is programmed to control an entire circuit. The operation of the program can be summarized as follows:

- The section or zone being controlled begins with the first MCR instruction and ends with the second MCR.
- When the first MCR instruction is false, or disabled, all nonretentive rungs below it, in this case, outputs M and PL1, will be de-energized even if the programmed logic for each rung is true.
- All retentive rungs, in this case SOL, will remain in their last state.
- Assume the motor M is running and the MCR instruction becomes disabled. The motor will immediately become de-energized and stop operating. When the MCR instruction then becomes enabled, the motor will not revert back to its previous running state but will have to be restarted via the start pushbutton.
- Assume the level switch is closed and the MCR instruction becomes disabled. Pilot light PL1 will immediately become de-energized even though the level switch instruction is true and the rung appears to have logic continuity. When the MCR instruction then becomes enabled, PL1 will automatically be energized, provided the level switch has remained closed.
- Assume solenoid SOL has been latched energized, both limit switches LS1 and LS2 are open, and the MCR instruction becomes disabled. Solenoid SOL will remain energized. When the MCR instruction then becomes enabled, the SOL will remain energized, provided both LS1 and LS2 remained open.
- Assume solenoid SOL has been latched de-energized, both limit switches LS1 and LS2 are open, and the MCR instruction becomes disabled. Solenoid SOL will remain de-energized. When the MCR instruction then becomes enabled, the SOL will remain de-energized, provided both LS1 and LS2 remained open.
- Retentive instructions should not normally be placed within an MCR zone because the MCR zone maintains retentive instructions in the state last active when the instruction disabled.

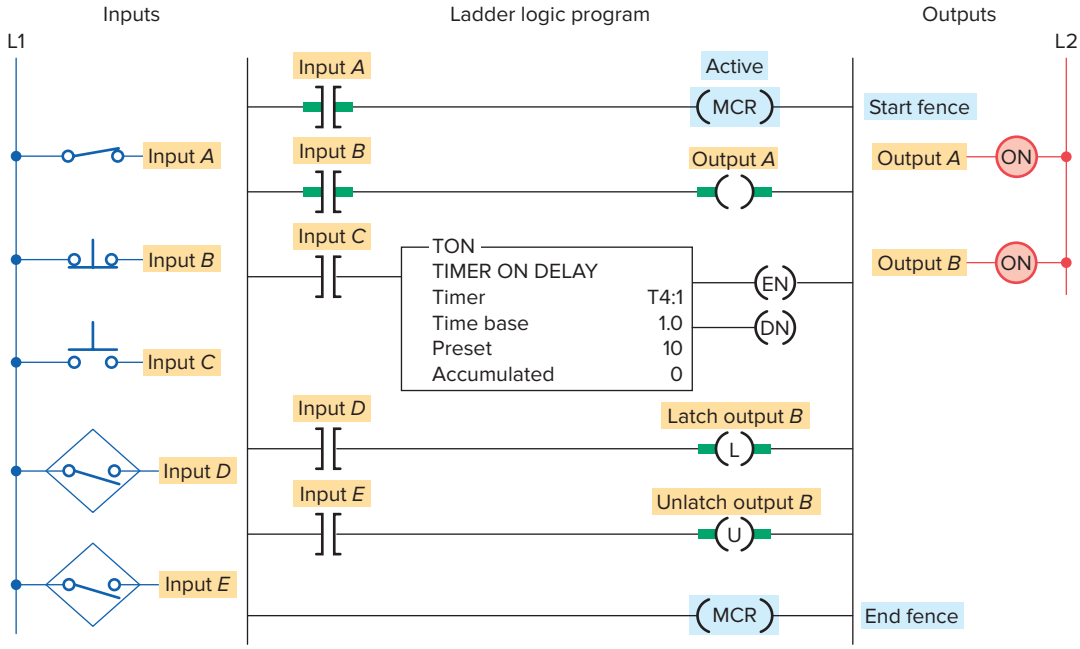


Figure 9-4 MCR fenced zone with the zone true.

Allen-Bradley SLC 500 controllers use the master control reset instruction to set up single or multiple zones within a program. The MCR instruction is used in pairs to disable or enable a zone within a ladder program, and it has ***no address.*** Figure 9-4 shows the programming of an MCR fenced zone with the zone true. The operation of the program can be summarized as follows:

- The MCR zone is enclosed by a ***start fence,*** which is a rung with a conditional MCR, and an ***end fence,*** which is a rung with an unconditional MCR.
- Input *A* of the start rung is true so all outputs act according to their rung logic as if the zone did not exist.

Figure 9-5 shows the programmed MCR fenced zone with the zone false. The operation of the program can be summarized as follows:

- When the MCR in the start fence is false, all rungs within the zone are treated as false. The scan

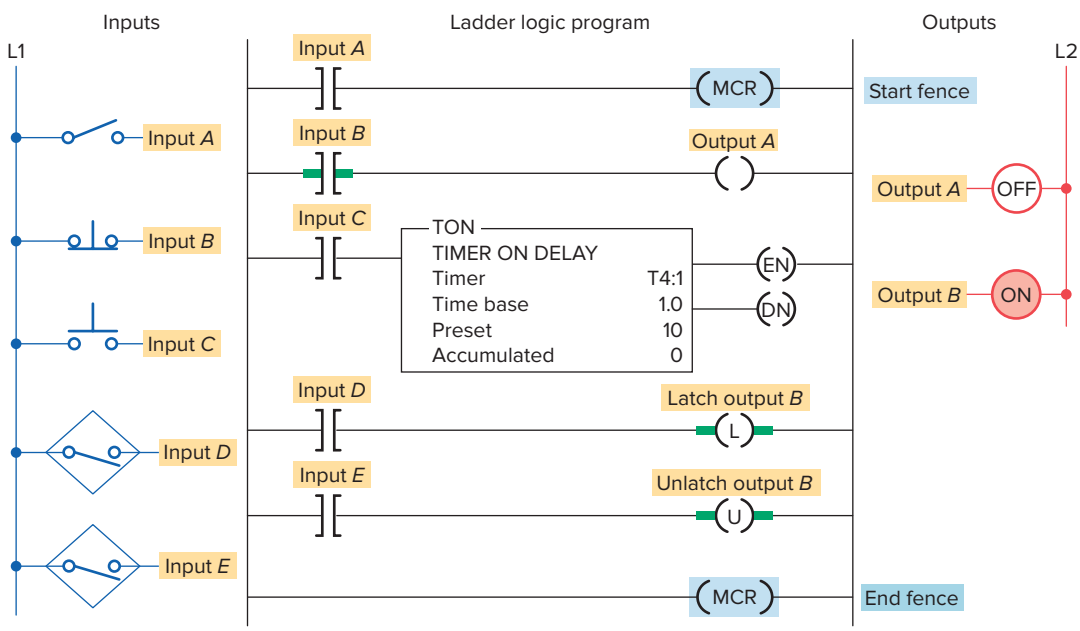


Figure 9-5 MCR fenced zone with the zone false.

ignores the inputs and de-energizes all nonretentive outputs (that is, the output energize instruction, the on-delay timer, and the off-delay timer).

- All retentive devices, such as latches, retentive timers, and counters, remain in their last state. TOF timers will start timing when the MCR goes false.
- Input *A* of the start rung is false so output *A* and T4:1 will be false and output *B* will remain in its last state.
- The input conditions in each rung will have no effect on the output conditions.

A common application of an MCR zone control involves examining one or more fault bits as part of the start fence and enclosing the portion of the program you want de-energized in case of a fault in the MCR zone. In case of a detected fault condition, the outputs in that zone would be de-energized automatically.

If you start instructions such as timers or counters in an MCR zone, instruction operation ceases when the zone is disabled. The TOF timer will activate when placed inside a false MCR zone. When troubleshooting a program that contains an MCR zone, you need to be aware of which rungs are within zones in order to correctly edit the circuit.

MCR-controlled areas must contain only two MCR instructions—one to define the start and one to define the end. Never overlap or nest MCR zones. Any additional MCR instructions, or a jump instruction programmed to jump to an MCR zone, could produce unexpected and damaging results to your program and to machine operation.

In addition to controlling power to an entire system, MCRs are also used when only a portion of a program is required to be isolated. For example:

- Inhibiting zones of the program while loading recipes.
- Monitoring emergency stops.
- Establishing preconditions to synchronize a machine on start-up.

9.3 Jump Instruction

In PLC programming it is sometimes desirable to be able to jump over certain program instructions when certain conditions exist. The ***jump (JMP)*** instruction is an output instruction used for this purpose. When the jump instruction is used, the PLC will not execute the instructions of a rung that is jumped. The jump instruction is often used to jump over instructions not pertinent to the machine's operation at that instant. In addition, sections of a program may be programmed to be jumped should a production fault occur.

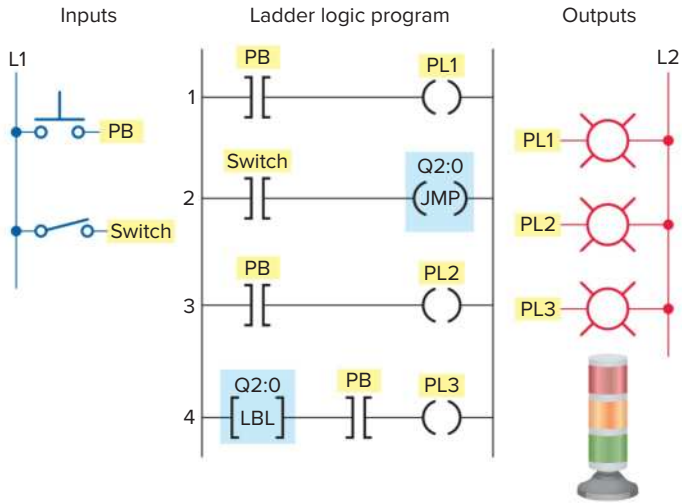


Figure 9-6 Jump (JMP) operation.

Some manufacturers provide a skip instruction, which is essentially the same as the jump instruction.

The program of Figure 9-6 illustrates the use of a jump instruction in conjunction with Allen-Bradley SLC 500 programmable controllers. In this example, Addresses Q2:0 through Q2:255 are the addresses used for the jump (JMP) instructions. The Q2 is internal and provided by the software as you program the JMP instruction. The Q2 simply identifies this as ladder file 2. A JMP instruction in ladder file 3 would be Q3. The ***label (LBL)*** instruction is a target for the jump instruction.

- The jump instruction with its associated label instruction (LBL) must have the same address.
- The area of the program that the processor jumps over is defined by the locations of the jump and label instructions in the program.
- When the jump instruction is true, all logic between the jump and label instructions is bypassed and the processor continues scanning after the LBL instruction.
- The label instruction must be programmed as the first instruction on the rung where it resides.
- The label instruction is always true, and the remaining instructions on the rung must make up a verifiable rung.
- The instructions to the right of the LBL on the label rung are outside the jump zone and as such are not affected by the jump.

The operation of the program can be summarized as follows:

- When the switch is open the jump instruction is not activated.

- With the switch open, closing PB turns on all three pilot lights.
- When the switch is closed the jump (JMP) instruction will activate.
- With the switch closed, pressing PB turns on pilot lights PL1 and PL3 only.
- Rung 3 is skipped over during the PLC program scan so PL2 will remain in its last state before the JMP was enabled.

Figure 9-7 illustrates the effect on input and output instructions of jumped rungs in a program. The label instruction is used to identify the ladder rung that is the target destination but does not contribute to logic continuity. For practical purposes the label instruction is always considered to be logically true. The operation of the program can be summarized as follows:

- Rungs 1, 2, 3, 8, 9, 10 are programmed outside of the jumped section and will always be executed as normal rungs.
- If rung 4, which contains the JMP instruction, is **false,** the Jump instruction is false and the jump is not executed.
- Rungs 5, 6, and 7 are executed as normal and the label instruction on rung 8 is transparent.

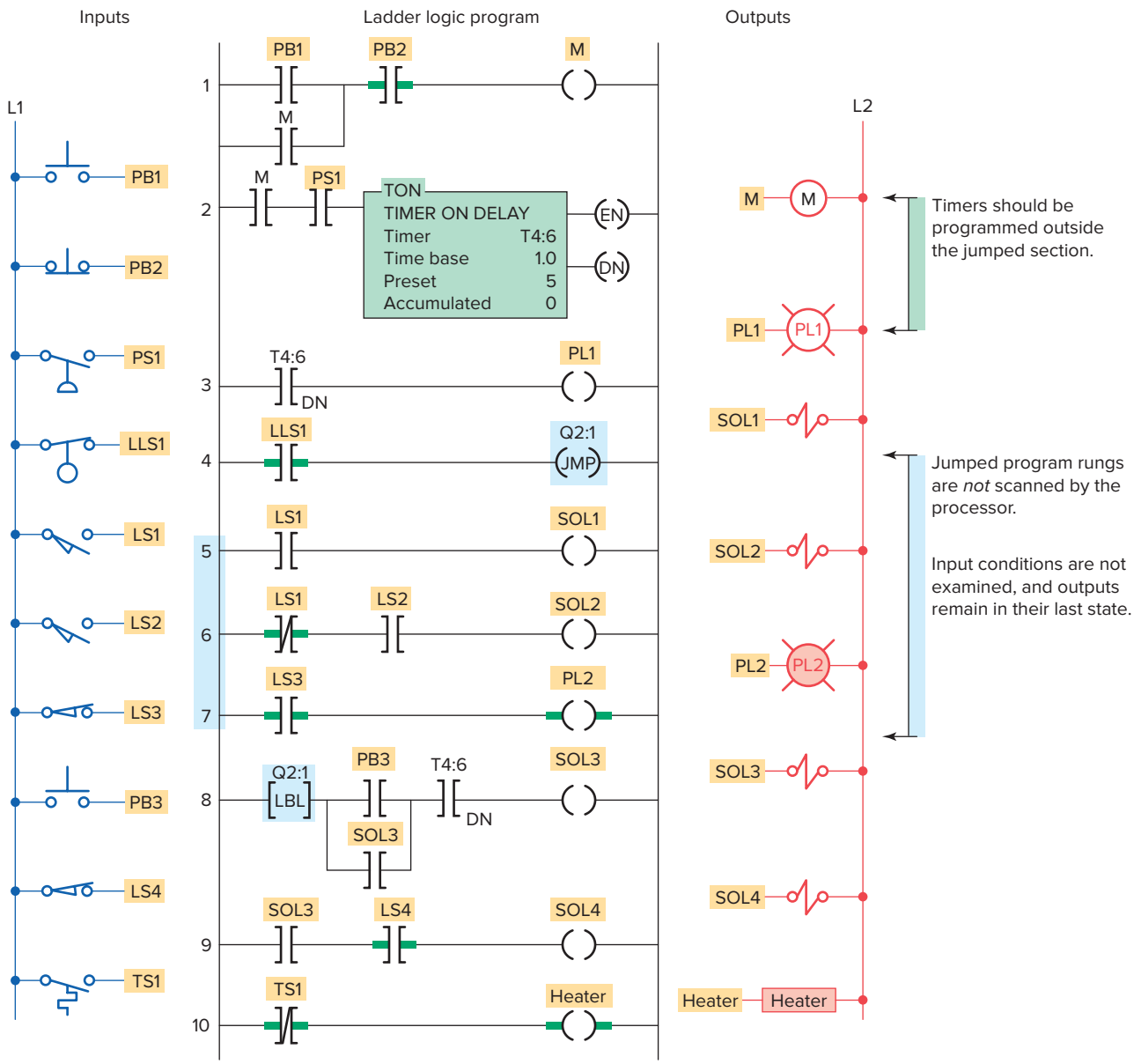


Figure 9-7 Effect on input and output instructions of jumped rungs.

- When rung 4, containing the JMP instruction, is **true,** the processor is instructed to jump to the LBL target in rung 8 and continue to execute the main program from that point.
- Instructions to the right of the LBL are out of the jump zone and are executed as a normal rung.
- Jumped rungs 5, 6, and 7 are not scanned by the processor.
- Input conditions for the jumped rungs are not examined and outputs controlled by these rungs remain in their last state.
- Any timers or counters programmed within the jump area cease to function and will not update themselves during this period. For this reason they are usually programmed outside the jumped section in the main program zone.
- This is called a forward jump, as we are jumping forward in the program.

You can jump to the same label from multiple jump locations, as illustrated in the program of Figure 9-8. In this example, there are two jump instructions addressed Q2:5. There is a single label instruction addressed Q2:5. The scan can then jump from either jump instruction to label Q2:5, depending on whether input *A* or input *D* is true.

It is possible to jump backward in the program, but this should not be done an excessive number of times. Care must be taken that the scan does not remain in a loop too long. The processor has a watchdog timer that sets the maximum allowable time for a total program scan. If this time is exceeded, the processor will indicate a fault and shut down.

The forward jump is similar to an MCR instruction in that both permit an input logic condition to skip over a block of PLC ladder logic. The main difference between the two is in how the outputs are handled when the instructions are executed. The MCR instruction sets all nonretentive outputs to the false state and keeps the retentive outputs in their last state. The JMP instruction leaves all outputs in their last state. You should never jump into a Master Control Reset zone. If you do, instructions that are programmed within the MCR zone starting at the LBL instruction and ending at the end MCR instruction will always be evaluated as though the MCR zone is true, without consideration to the state of the start MCR instruction.

9.4 Subroutine Functions

In addition to the main ladder logic program, PLC programs may also contain additional program files known as *subroutines.* A subroutine is a short program that is used by the main program to perform a specific function. Large programs are often broken into subroutine program files, which are called and executed from the main program. In the SLC 500 series PLCs, the main ladder logic program is in program file two (shown as LAD 2). Ladder logic programs for subroutines can be placed in file number three (LAD 3) through file number 255 (LAD 255).

Use of subroutines is a valuable tool in PLC programming. At times it is better to construct programs that consist of several subroutines than a lengthy single program. When programs are written with subroutines, each subroutine can be tested individually for functionality. These subroutines can then be called from the main program as illustrated in Figure 9-9.

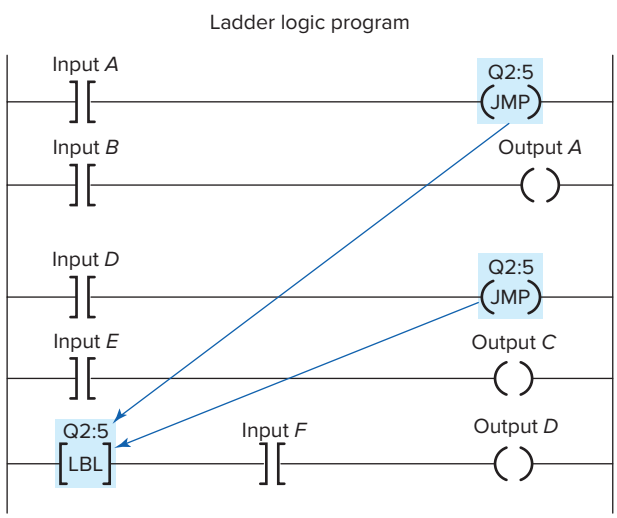


Figure 9-8 Jump-to-label from two locations.

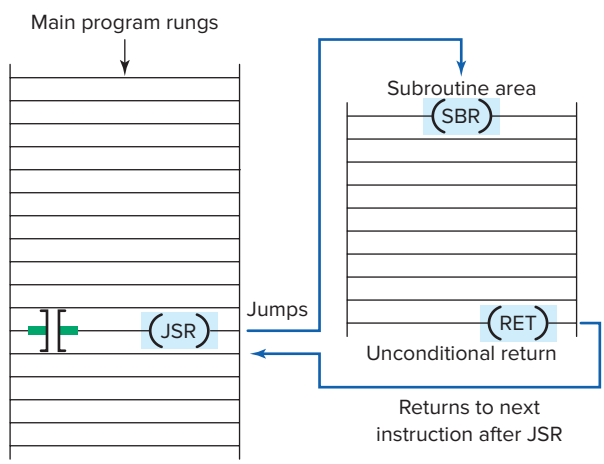


Figure 9-9 Main program with a call from a subroutine.

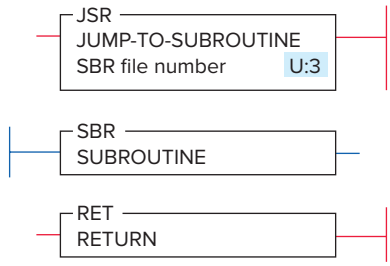


Figure 9-10 Allen-Bradley subroutine-related instructions.

When a subroutine is called from the main program, the program is able to escape from the main program and *go to* a program *subroutine* to perform certain functions and *then return* to the main program. In situations in which a machine has a portion of its cycle that must be repeated several times during one machine cycle, the subroutine can save a great deal of duplicate programming. The sequence of rungs could be programmed one time into a subroutine and just called when needed.

The subroutine concept is the same for all programmable controllers, but the method used to call and return from a subroutine uses different commands, depending on the PLC manufacturer. The subroutine-related instructions used in the Allen-Bradley PLCs shown in Figure 9-10 are the jump to subroutine (JSR) output instruction, the subroutine (SBR) input instruction, and the return (RET) output instruction.

The subroutine instructions can be summarized as follows:

Jump to Subroutine (JSR)—The JSR instruction redirects logic execution from the current ladder file to the specific subroutine file. When rung conditions are true for this output instruction, it causes the processor to jump to the targeted subroutine file. Each subroutine must have a unique file number (decimal 3–255).

Subroutine (SBR)—The SBR instruction is the first input instruction on the first rung in the subroutine file. It serves as an identifier that the program file is a subroutine. This file number is used in the JSR instruction to identify the target to which the program should jump. It is always true, and although its use is optional, it is still recommended.

Return (RET)—The RET instruction is an output instruction that marks the end of the subroutine file. It causes the scan to return to the main program at the instruction following the JSR instruction where it exited the program. The scan returns from the end of the file if there is no RET instruction. The rung containing the RET instruction may be conditional if this rung precedes the end of the subroutine. In this way, the processor omits the balance of a subroutine only if its rung condition is true.

The jump to subroutine (JSR), subroutine (SBR), and return (RET) instructions are used to direct the controller to execute a subroutine file. Figure 9-11 shows a materials conveyor system with a flashing pilot light as a subroutine. The operation of the program can be summarized as follows:

- If the weight on the conveyor exceeds a preset value, the solenoid is de-energized and pilot light PL1will begin flashing.
- When the weight sensor switch closes, the JSR is activated and directs the processor scan to jump to the subroutine U:3.
- The subroutine program is scanned and pilot light PL1 begins flashing.
- When the weight sensor switch opens, the processor will no longer scan the subroutine area and pilot light PL1 will return to its normal on state.

The Allen-Bradley SLC 500 controller main program is located in program file 2 whereas subroutines are assigned to program file numbers 3 to 255. Each subroutine must be programmed in its own program file by assigning it a unique file number. Figure 9-12 illustrates the procedure for setting up a subroutine and can be summarized as follows:

- Note each ladder location where a subroutine should be called.
- Create a subroutine file for each location. Each subroutine file should begin with an SBR instruction.
- At each ladder location where a subroutine is called, program a JSR instruction specifying the subroutine file number.
- The RET instruction is optional.
  - The end of a subroutine program will cause a return to the main program.
  - If you want to end a subroutine program before it executes to the end of program file, a conditional return (RET) instruction may be used.

Nesting subroutines allows you to direct program flow from the main program to a subroutine and then to another subroutine, as illustrated in Figure 9-13. Nested subroutines make complex programming easier and program operation faster because the programmer does not have to continually return from one subroutine to enter another.

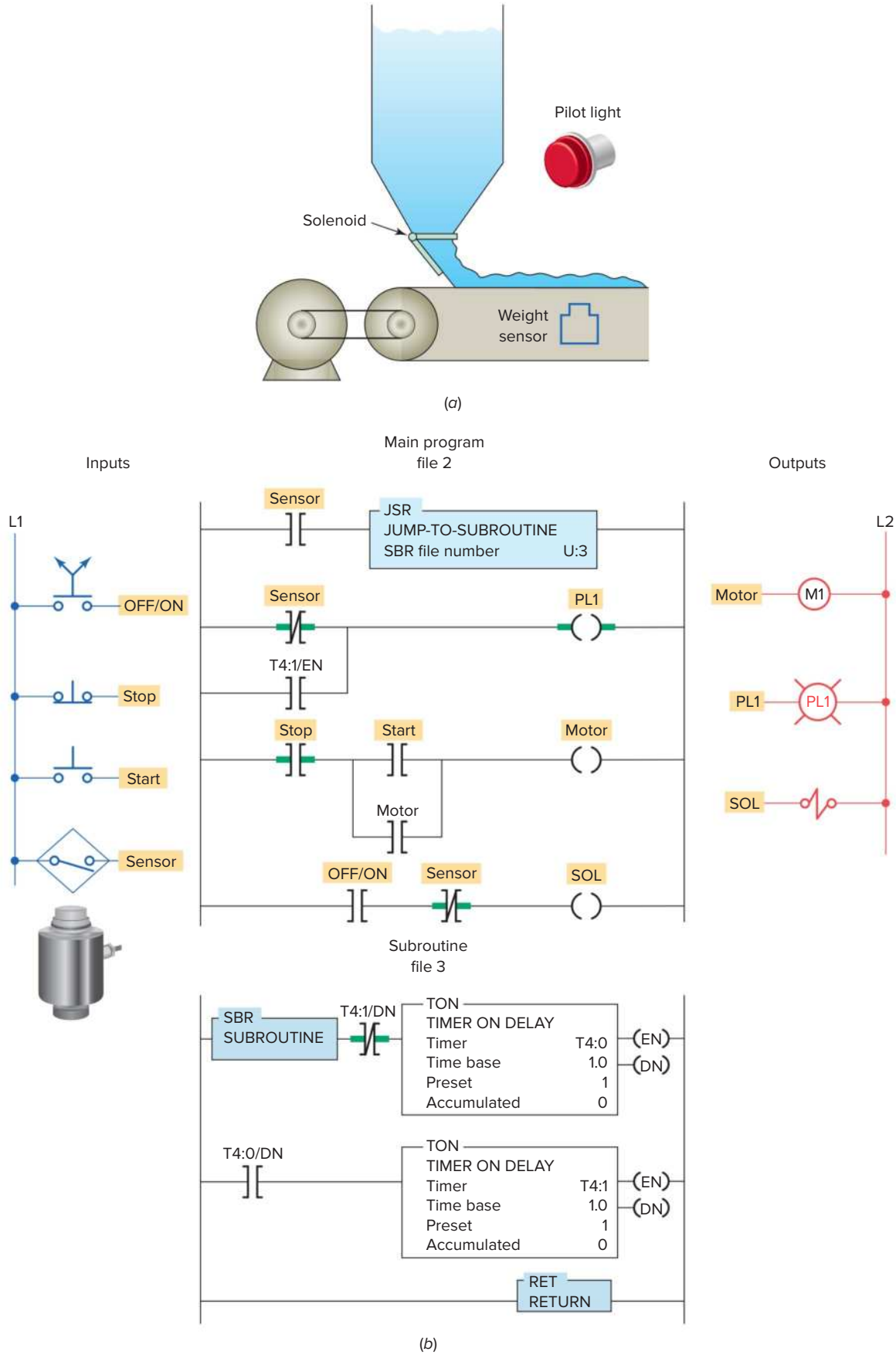


Figure 9-11 Flashing pilot light subroutine. (a) Process. (b) Program.

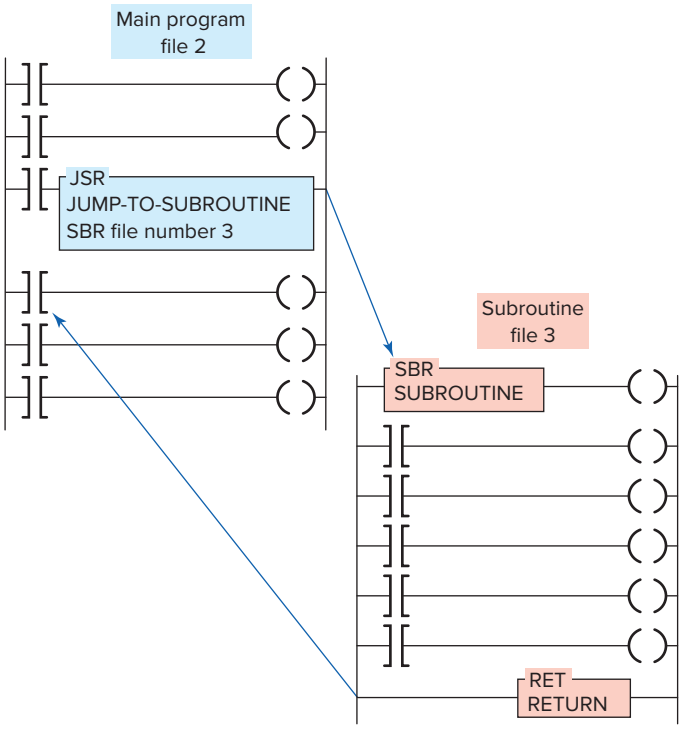


Figure 9-12 Setting up a subroutine file.

Programming nested subroutines may cause scan time problems because while the subroutine is being scanned, the main program is not. Excessive delays in scanning the main program may cause the outputs to operate later than required. This situation may be avoided by updating critical I/O using immediate input and/or immediate output instructions.

9.5 Immediate Input and Immediate Output Instructions

The PLC input scan normally records the inputs before the program scan, and the output scan normally updates the outputs after the program scan. **Immediate I/O** instructions allow you to update data prior to the normal input scan as illustrated in Figure 9-14.

Immediate I/O instructions interrupt the normal program scan to update the input image table file with current input data or to update an output module group with the current output image table file data. Allen-Bradley SLC 500 PLC's immediate I/O instructions are called **immediate input with mask (IIM)** and **immediate output with mask (IOM).**

- Masking is a means of selectively screening out data.
- Masking allows the programmer to specify which of the 16 bits are to be copied from an input module to the input image data table (or from the output image table to an output module).
- The other bits in the input image table or output module are not affected by these instructions.

The immediate input with mask (IIM) instruction is shown in Figure 9-15. The **IIM instruction** operates on the inputs assigned to a particular word of a slot. When the IIM rung is true, the program scan is interrupted, and data from a specific input slot are transferred through the mask to the input data file. These data are then available

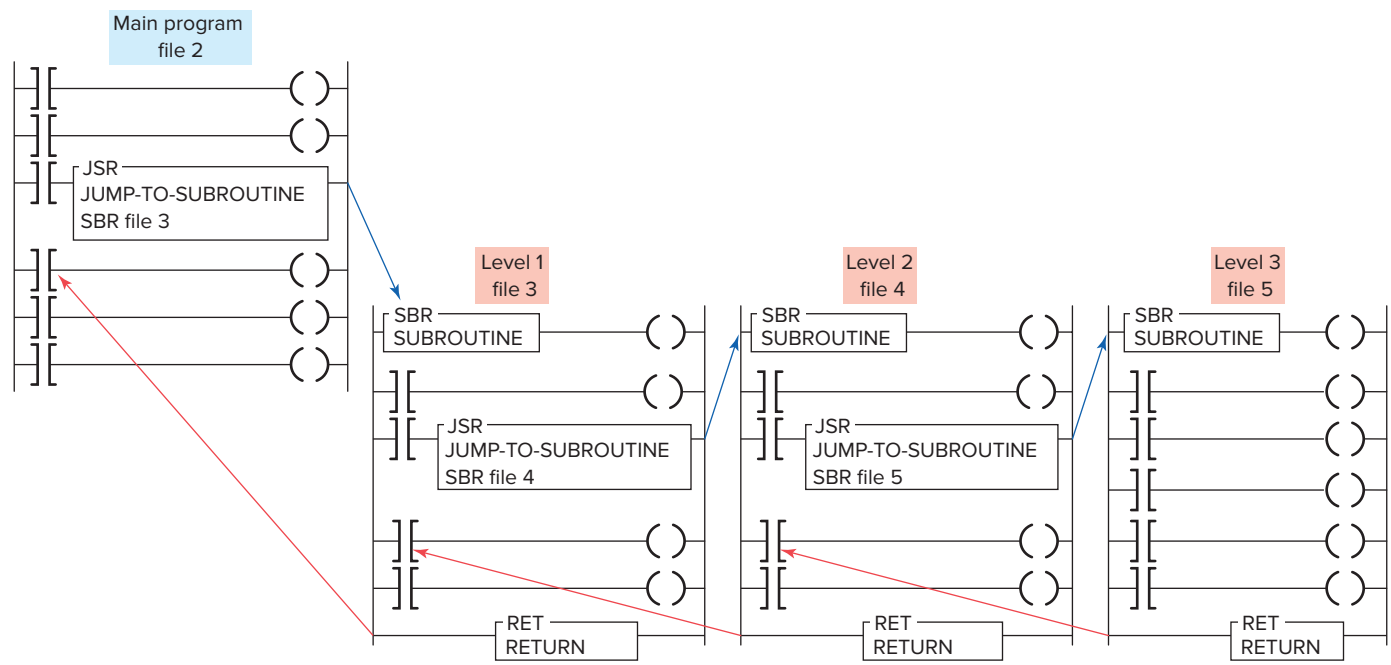


Figure 9-13 Nested subroutines.

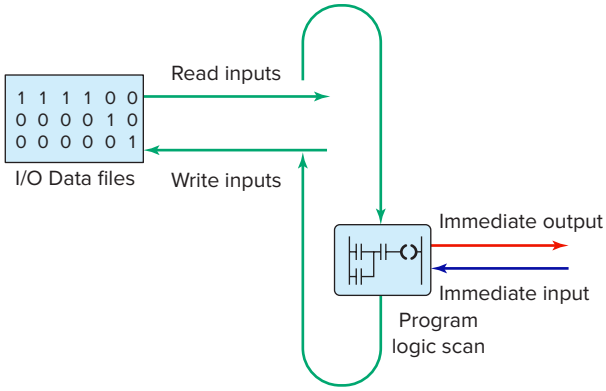


Figure 9-14 Immediate I/O instructions.

to the commands in the ladder following the IIM instruction. The following parameters are entered in the instruction:

Slot Specifies the slot and word that contain the data to be updated. For example, I:3.0 means the input of slot 3, word 0.

Mask Specifies either a hex constant or a register address. For the mask, a 1 in the bit position passes data from the source to the destination. A 0 inhibits or blocks bits from passing from the source to the destination.

Length Used to transfer more than one word per slot.

The program operation of the instruction is summarized as follows:

- The IIM instruction retrieves data from I:1.0 and passes it through the mask.
- The mask permits only the four least significant bits to be moved to the input register I:1.0.
- This allows the programmer to update only sections of the inputs to be used throughout the rest of the program.

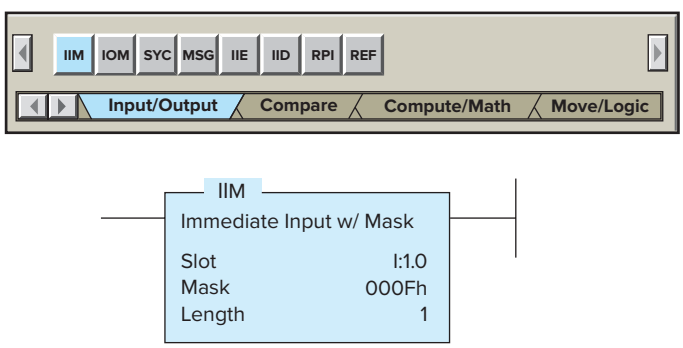


Figure 9-15 Immediate input with mask (IIM) instruction.

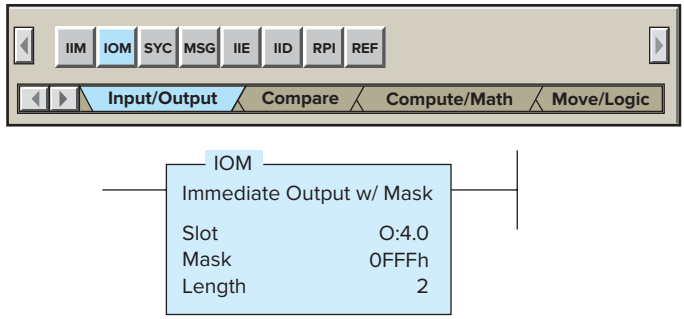


Figure 9-16 Immediate output with mask (IOM) instruction.

The immediate output with mask **(IOM) instruction** is shown in Figure 9-16. The IOM operates on the physical outputs assigned to a particular word of a slot. When the IOM rung is true, the program scan is interrupted to update output data to the module located in the slot specified in the instruction. These data are then available to the commands in the ladder following the IOM instruction. The parameters entered are basically the same as those entered for the IIM instruction.

Processor communication with the local chassis is many times faster than communication with the remote chassis. This is due to the fact that local I/O scan is synchronous with the program scan and communication is in *parallel* with the processor, whereas the remote I/O scan is asynchronous with the program scan and communication with remote I/O is *serial.* For this reason, fast-acting devices should be wired into the local chassis.

ControlLogix PLCs have no immediate input instruction as they use asynchronous I/O control compared to the SLC 500 controllers which use synchronous I/O control. ControlLogix controllers do have an immediate output (IOT) instruction, which operates the same as the immediate output instruction for the SLC 500. Figure 9-17 shows an example of the IOT instruction. In this example, when the IOT instruction executes, it immediately updates the entire output module Local:3:0. When you use the instruction to update the output card, address the entire card (Local:3:0), and not the individual outputs (Local:3:0. Data 0).

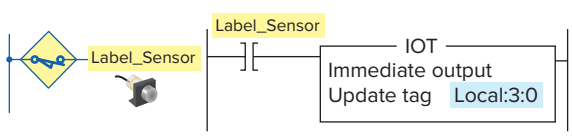


Figure 9-17 ControlLogix immediate output instruction.

9.6 Forcing External I/O Addresses

The **force function** is essentially a manual override control function. Forcing allows the PLC user to turn an external input or output on or off from the keyboard of the programming device. This is accomplished regardless of the actual state of the field device. The forcing capability allows a machine or process to continue operation until a faulty field device can be repaired. It is also valuable during start-up and troubleshooting of a machine or process to simulate the action of portions of the program that have not yet been implemented.

Forcing inputs manipulates the input image table file bits and thus affects *all* areas of the program that use those bits. The forcing of inputs is done just after the input scan. When we force an input address, we are forcing the status bit of the instruction at the I/O address to an on or off state. Figure 9-18 illustrates how an input is forced on. The operation of the program can be summarized as follows:

- The processor ignores the actual state of input limit switch I:1/3.
- Although limit switch I:1/3 is off (0 or false) the processor considers it as being in the on (1 or true) state.
- The program scan records this, and the program is executed with this forced status.
- In other words, the program is executed as if the limit switch were actually closed.

Forcing outputs affects *only* the addressed output terminal. Therefore, since the output image table file bits are unaffected, your program will be unaffected. When we force an output address, we are forcing only the output terminal to an on or off state. The status bit of the output instruction

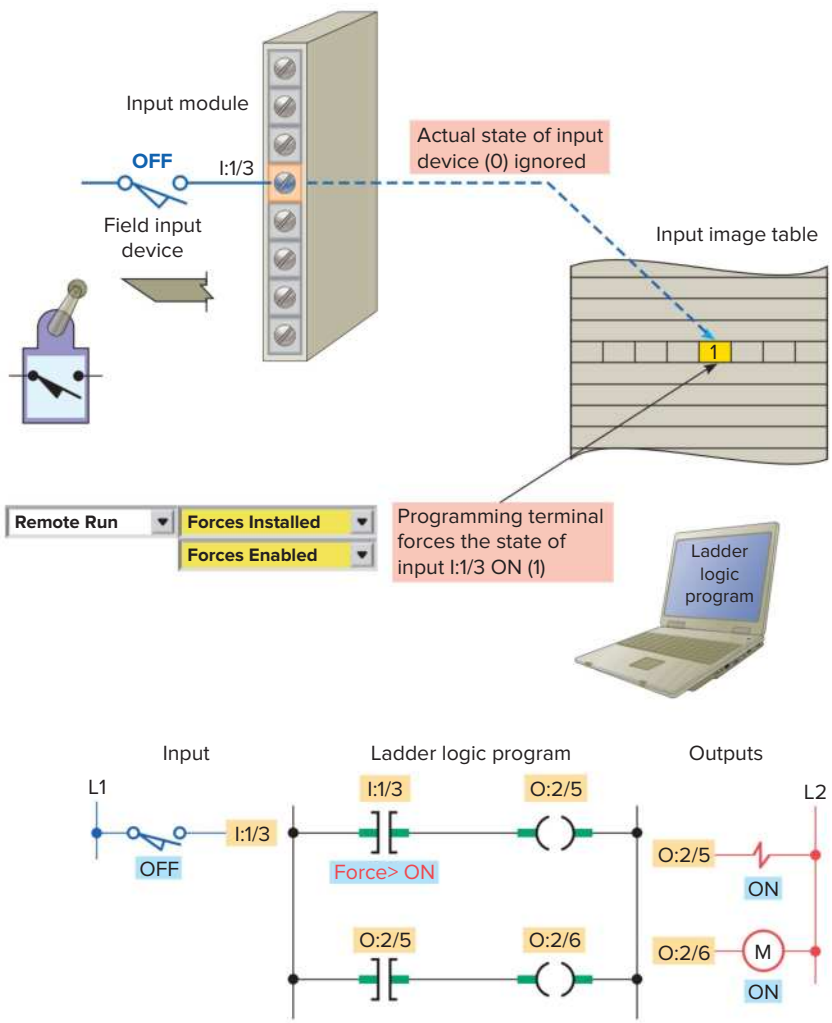


Figure 9-18 Forcing an input on.

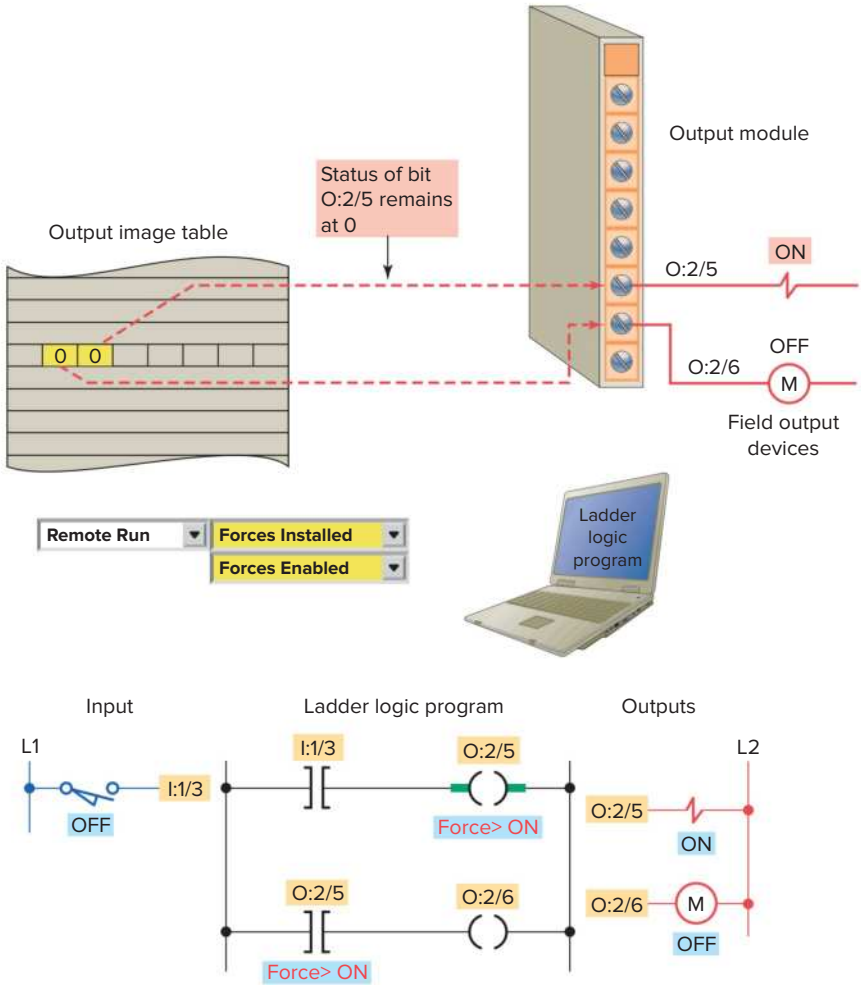


Figure 9-19 Forcing an output on.

at the address is usually not affected. Figure 9-19 illustrates how an output is forced on. The operation of the program can be summarized as follows:

- The processor ignores the actual state of solenoid output O:2/5.
- The programming device sets the force state in the output force data file and the PLC implements the force to turn solenoid output O:2/5 on even though the output image table file indicates that the user logic is setting the point to off.
- M output O:2/6 remains off because the status bit of output O:2/5 is not affected by the force instruction.
- Not all brands of PLCs operate this way. For example, forcing an output with a GE Fanuc controller will cause the contacts that have the same address as the output to also change to the appropriate state.

Overriding of physical inputs on conventional relay control systems can be accomplished by installing hardwire jumpers. With PLC control, hardwire jumpers are not necessary because the input data table values can be forced to an on or off state. The force function allows you to override the actual status of external input circuits by forcing external data bits on or off. Similarly, you can override the processor logic and status of output data file bits by forcing output bits on or off. By forcing outputs off, you can prevent the controller from energizing those outputs even though the ladder logic, which normally controls them, may be true. In other instances, outputs may be forced on even though logic for the rungs controlling those outputs may be false.

Figure 9-20 shows the forces version of the data table with bit I:1/3 forced on. You can enter and enable or disable forces while you are monitoring your file offline, or in any processor mode while monitoring your file online. With RSLogix 500 software, the steps are as follows:

1. Open the program file in which you want to force the logic on or off.

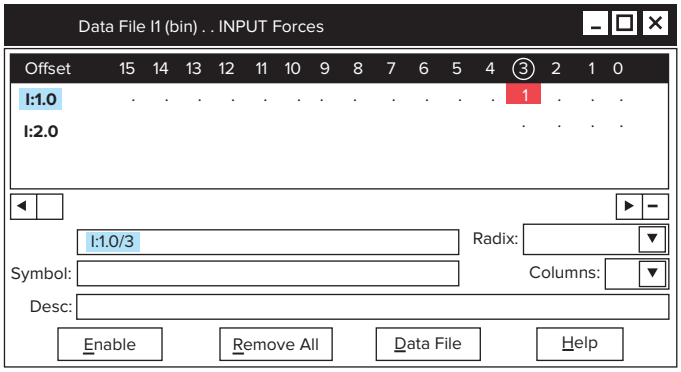


Figure 9-20 Forces version of the data table with bit I:1/3 forced on.

2. With the right mouse button, click the I/O bit you want to force.
3. From the menu that appears, select Go to Data Table or select Force On or Force Off.
4. From the associated data table that appears, click on the Forces button.
5. The Forces version of the data table appears with the selected bit highlighted. Click on this bit with the right mouse button.
6. From the menu that appears, you can force the selected bit on or off.

Exercise care when you use forcing functions. **If used incorrectly, force functions can cause injuries to persons working around a system, and/or equipment damage.** For this reason, forcing functions should be used only by personnel who completely understand the circuit and the process machinery or driven equipment (Figure 9-21). You must understand the potential effect that forcing given inputs or outputs will have on machine operation in order to avoid possible personal injury and equipment damage. Before using a force function, check whether the force acts on the I/O point only or whether it acts on the user logic as well as on the I/O point. Most programming terminals and PLC CPUs provide some visible means of alerting the user that a force is in effect.

In situations in which rotating equipment is involved, the force instruction can be extremely dangerous. For example, if maintenance personnel are performing routine maintenance on a de-energized motor, the machine may suddenly become energized by someone forcing the motor to turn on. This is why a hardwired master control circuit is required for the I/O rack. The hardwired circuit will provide a method of physically removing power to the I/O system, thereby ensuring that it is impossible to energize any inputs or outputs when the master control is off.

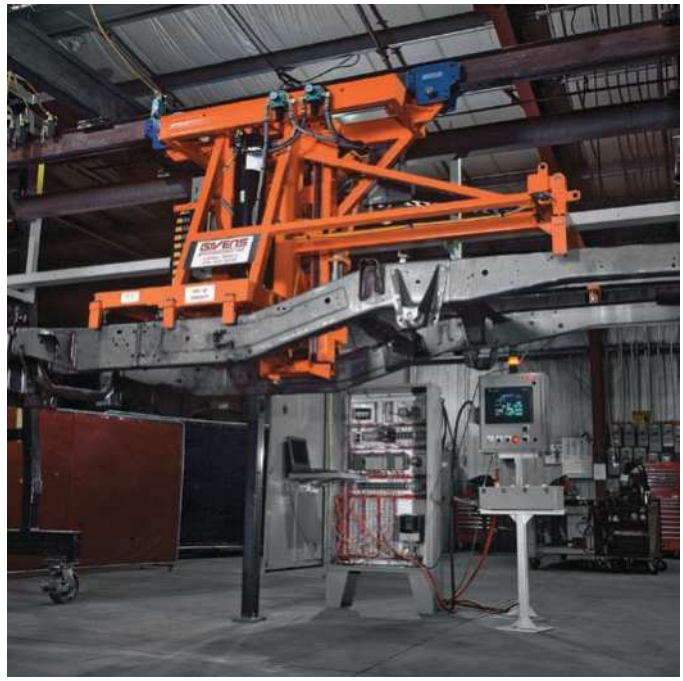


Figure 9-21 Exercise care when you use forcing functions.
Source: Courtesy Givens Engineering Inc.

9.7 Safety Circuitry

Sufficient emergency circuits must be provided to stop either partially or totally the operation of the controller or the controlled machine or process. These circuits should be **hardwired outside the controller** so that in the event of total controller failure, independent and rapid shutdown is available.

Figure 9-22 shows typical safety wiring requirements for a PLC installation. The safety requirements of this installation can be summarized as follows:

- A main disconnect switch is installed on the incoming power lines as a means of removing power from the entire programmable controller system.
- The main power disconnect switch should be located where operators and maintenance personnel have quick and easy access to it. Ideally, the disconnect switch is mounted on the outside of the PLC enclosure so that it can be accessed without opening the enclosure.
- In addition to disconnecting electrical power, you should de-energize, lock out, and tag all other sources of power (pneumatic and hydraulic) before you work on a machine or process controlled by the controller.
- An isolation transformer is used to isolate the controller from the main power distribution system and step the voltage down to 120 VAC.

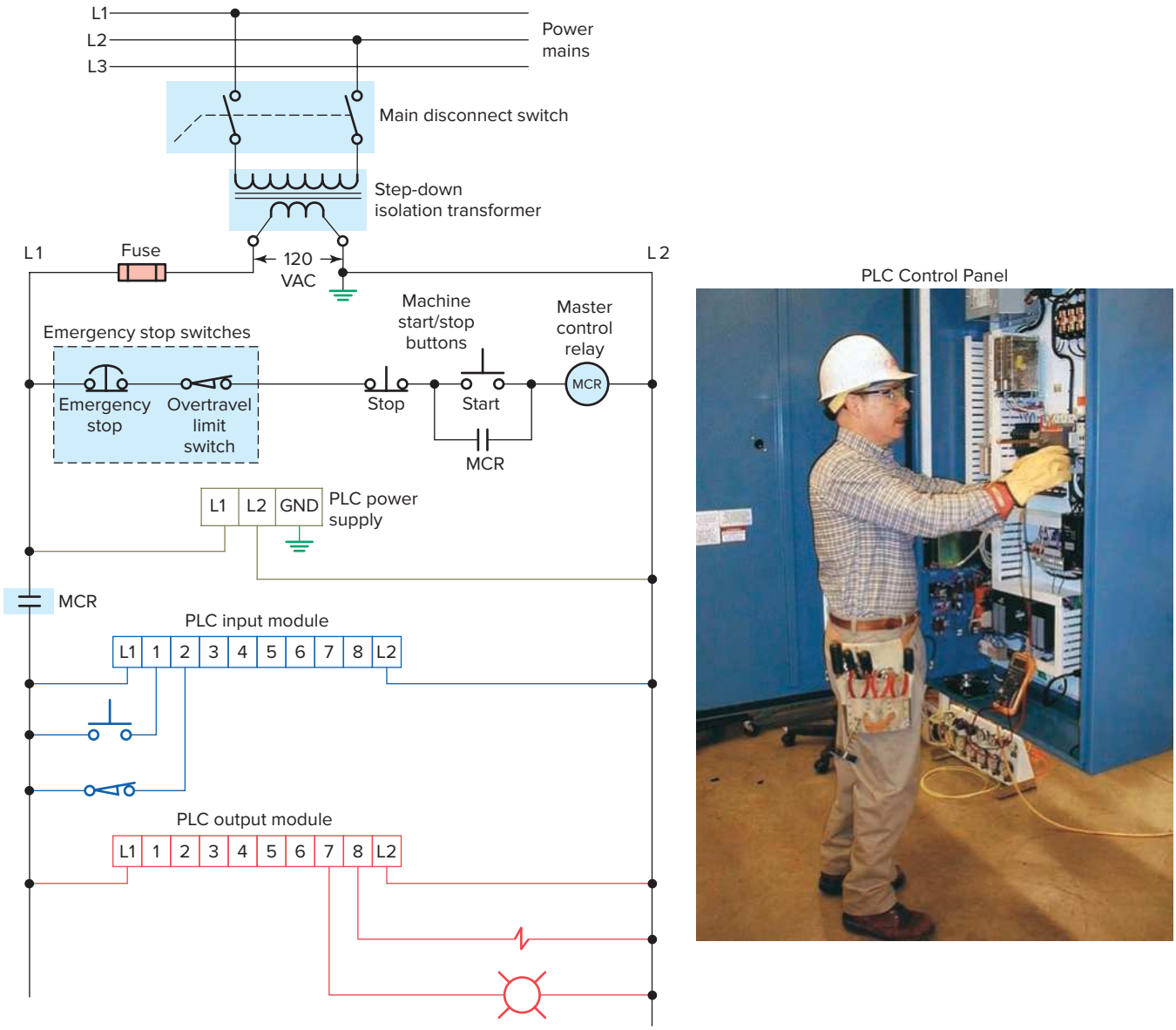
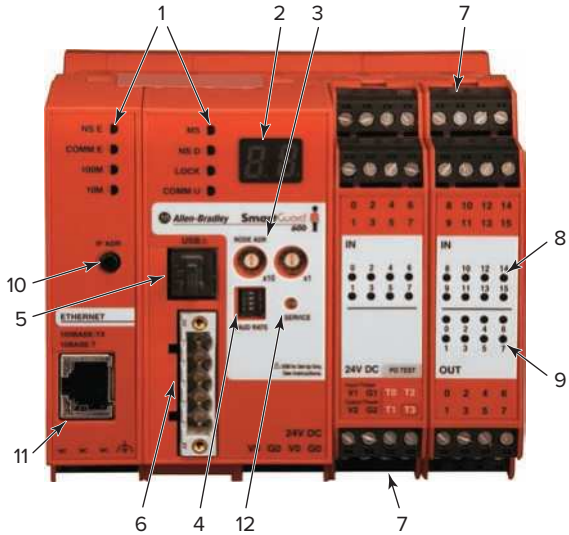


Figure 9-22 Safety wiring requirements for a PLC installation.
Source: Courtesy Minarik Automation & Control.

- A hardwired master control relay is included to provide a convenient means for emergency controller shutdown. Because the master control relay allows the placement of several emergency-stop switches in different locations, its installation is important from a safety standpoint.
- Overtravel limit switches or mushroom head emergency stop pushbuttons are wired in series so that when one of them opens, the master control is de-energized.
- This removes power to input and output device circuits. Power continues to be supplied to the controller power supply so that any diagnostic indicators on the processor module can still be observed.
- Note that the master control relay is not a substitute for a disconnect switch. When you are replacing any module, replacing output fuses, or working on equipment, the main disconnect switch should be pulled and locked out.

The ***master control relay*** must be able to inhibit all machine motion by removing power to the machine I/O devices when the relay is de-energized. This hardwired electromechanical component must not be dependent on electronic components (hardware or software). Any part can fail, including the switches in a master control



Number	Feature
1	Module status indicators
2	Alphanumeric display
3	Node address switches
4	Baud rate switches
5	USB port
6	DeviceNet communication connector
7	Terminal connectors
8	Input status indicators
9	Output status indicators
10	IP address display switch
11	Ethernet connector
12	Service switch

Figure 9-23 Safety PLC.
Source: Image Courtesy of Rockwell Automation, Inc.

relay circuit. The failure of one of these switches would most likely cause an open circuit, which would be a safe power-off failure. However, if one of these switches shorts out, it no longer provides any safety protection. These switches should be tested periodically to ensure that they will stop machine motion when needed. Never alter these circuits to defeat their function. Serious injury or machine damage could result.

Safety PLCs, such as the one shown in Figure 9-23, are now available for applications that require more advanced safety functionality. A safety PLC is typically certified by third parties to meet rigid safety and reliability requirements of international standards. Both standard and safety PLCs have the ability to perform control functions but a standard PLC was not initially designed to be fault tolerant and fail-safe. That is the fundamental difference.

Some of the differences between standard and safety PLCs include the following:

- A standard PLC has one microprocessor that executes the program, Flash memory area that stores the program, RAM for making calculations, ports for communications, and I/O for detection and control of the machine. In contrast, a safety PLC has redundant microprocessors, Flash and RAM that are continuously monitored by a watchdog circuit, and a synchronous detection circuit. *Redundancy* is duplication. The probability of hazards arising from one malfunction in an electrical circuit can be minimized by creating partial or complete redundancy (duplication).
- Standard PLC inputs provide no internal means for testing the functionality of the input circuitry. By contrast, safety PLCs have an internal output circuit associated with each input for the purpose of testing the input circuitry. Inputs are driven both high and low for very short cycles during runtime to verify their functionality.
- Safety PLCs use power supplies designed specifically for use in safety control systems and redundant backplane circuitry between the controller and I/O modules.

Safety considerations should be developed as part of the PLC program. A PLC program for any application will be only as safe as the time and thought spent on both personnel and hardware considerations make it. One such consideration involves the use of a motor starter ***auxiliary seal-in contact,*** shown in Figure 9-24, in place of the programmed contact referenced to the output coil instruction. The use of the field-generated starter auxiliary contact status in the program is more costly in terms of field wiring and hardware, but it is *safer* because it provides positive feedback to the processor about the exact status of the motor. Assume, for example, that the OL contact of the starter opens under an overload condition. The motor, of course, would stop operating because power would be lost to the starter coil. If the program was written using an examine-on contact instruction referenced to the output coil instruction as the seal-in for the circuit, the processor would never know that power had been lost to the motor. When the OL was reset, the motor would restart instantly, creating a potentially unsafe operating condition.

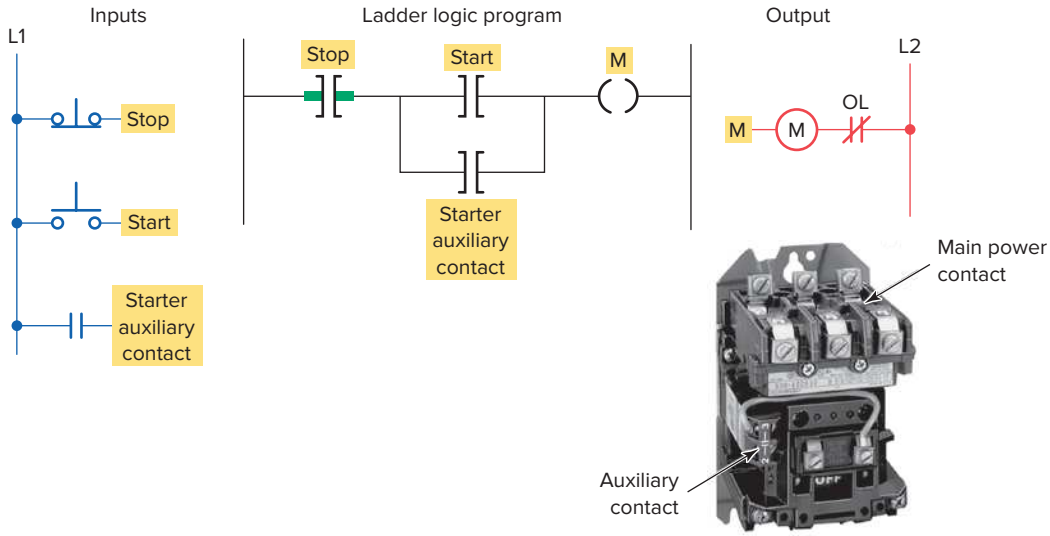


Figure 9-24 Motor starter programmed using the starter auxiliary seal-in contact.
Source: Image Courtesy of Rockwell Automation, Inc.

Another safety consideration concerns the ***wiring of stop buttons.*** A stop button is generally considered a safety function as well as an operating function. As such, ***all stop buttons should be wired using a normally closed contact programmed to examine for an on condition*** (Figure 9-25). Using a normally open contact programmed to examine for an off condition will produce the same logic but is not considered to be as safe. Assume that the latter configuration is used. If, by some chain of events, the circuit between the button and the input point were to be broken, the stop button could be depressed forever, but the PLC logic could never react to the stop command because the input would never be true. The same holds true if power were lost to the stop button control circuit. If the normally closed wiring configuration is used, the input point receives power continuously unless the stop function is desired. Any faults occurring with the stop circuit wiring, or a loss of circuit power, would effectively be equivalent to an intentional stop.

9.8 Selectable Timed Interrupt

The ***selectable timed interrupt (STI)*** instruction is used to interrupt the scan of the main program file automatically, on a time basis, to scan a specified subroutine file. For Allen-Bradley SLC 500 controllers, the time base at which the program file is executed and the program file assigned as the selectable timed interrupt file are determined by the values stored in words S:30 and S:31 of the status section of the data files. The value in S:30 stores the time base, which may be from 1 through 32,767, at 10 millisecond increments. Word S:31 stores the program file assigned as the selectable interrupt file, which may be any program file from 3 through 999. Entering a 0 in the time-base word disables the selectable timed interrupt.

Programming the selectable timed interrupt is done when a section of program needs to be executed on a *time basis* rather than on an *event basis.* For example, a

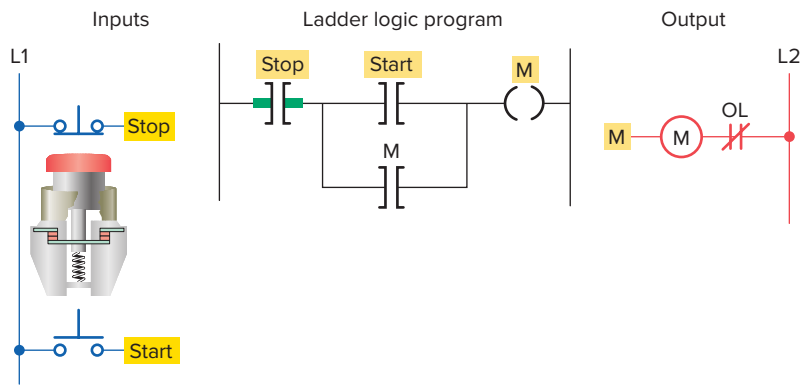


Figure 9-25 Wiring of stop buttons.

program may require certain calculations to be executed at a repeatable time interval for accuracy. These calculations can be accomplished by placing this programming in the selectable timed-interrupt file. This instruction can also be used for process applications that require periodic lubrication.

The immediate input and immediate output instructions are often located in a selectable timed interrupt file, so that a particular section of program is updated on a timed basis. This process could be done on a high-speed line, when items on the line are being examined and the rate at which they pass the sensor is faster than the scan time of the program. In this way, the item can be scanned multiple times during the program scan, and the appropriate action may be taken before the end of the scan.

The ***selectable timed disable (STD)*** instruction is generally paired with the ***selectable timed enable (STE)*** instruction to create zones in which STI interrupts cannot occur. Figure 9-26 illustrates the use of the STD and STE instructions and can be summarized as follows:

- In this program, STI is assumed to be in effect.
- The STD and STE instructions in rungs 6 and 12 are included in the ladder program to avoid having STI subroutine execution at any point in rungs 7 through 11.
- The STD instruction (rung 6) resets the STI enable bit, and the STE instruction (rung 12) sets the enable bit again.
- The SELECTABLE TIMED ENABLE instruction of rung 0 is triggered by the first pass bit status file S:1/15. The first pass bit, S:1/15, will only be true for the first scan through ladder file 3 when the PLC processor goes into the run mode. On subsequent scans, S:1/15 will not be true. This ensures that the STI function is initialized after each power cycle.

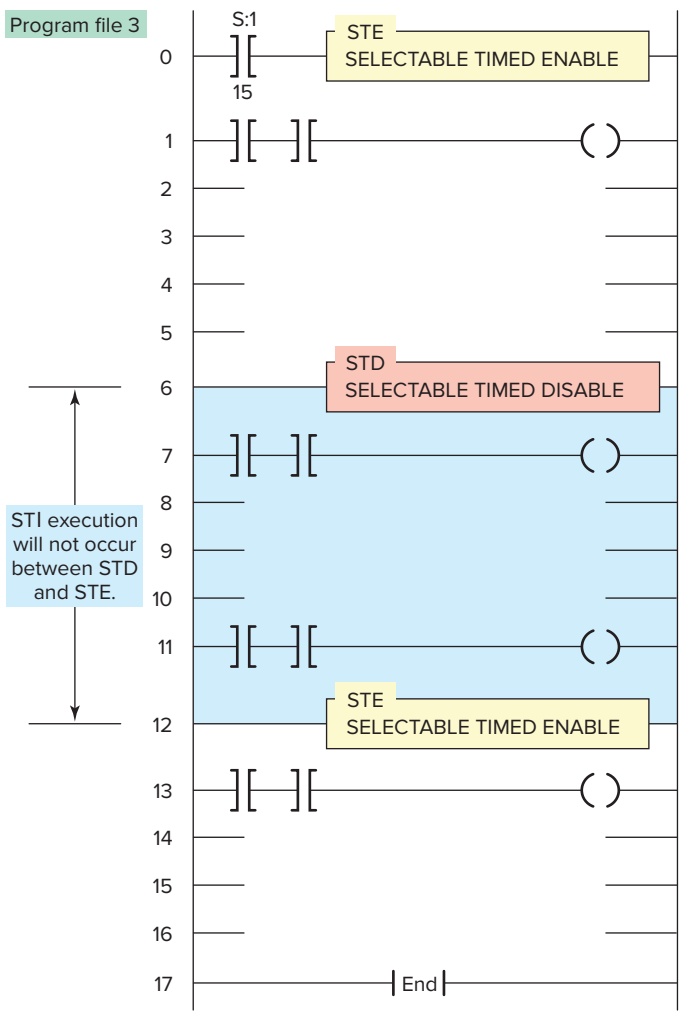


Figure 9-26 Selectable timed disable (STD) and selectable timed enable (STE) instructions.

9.9 Fault Routine

Allen-Bradley SLC 500 controllers allow you to designate a subroutine file as a **fault routine.** If used, it determines how the processor responds to a programming error. The program file assigned as the fault routine is determined by the value stored in word S:29 of the status file. Entering a 0 in word S:29 disables the fault routine.

There are two kinds of major faults that result in a processor fault: recoverable and nonrecoverable faults. When the processor detects a major fault, it looks for a fault routine. If a fault routine exists, it is executed; if one does not exist, the processor shuts down. When there is a fault routine, and the fault is *recoverable,* the fault routine is executed. If the fault is *nonrecoverable,* the fault routine is scanned once and shuts down. Either way, the fault routine allows for an orderly shutdown.

9.10 Temporary End Instruction

The ***temporary end (TND)*** instruction is an output instruction used to progressively debug a program or conditionally omit the balance of your current program file or subroutines. When rung conditions are true, this instruction stops the program scan, updates the I/O, and resumes scanning at rung 0 of the main program file.

Figure 9-27 illustrates the use of the TND instruction in troubleshooting a program. The TND instruction lets your program run only up to this instruction. You can move it progressively through your program as you debug each new section. You can program the TND instruction unconditionally, or you can condition its rung according to your debugging needs.

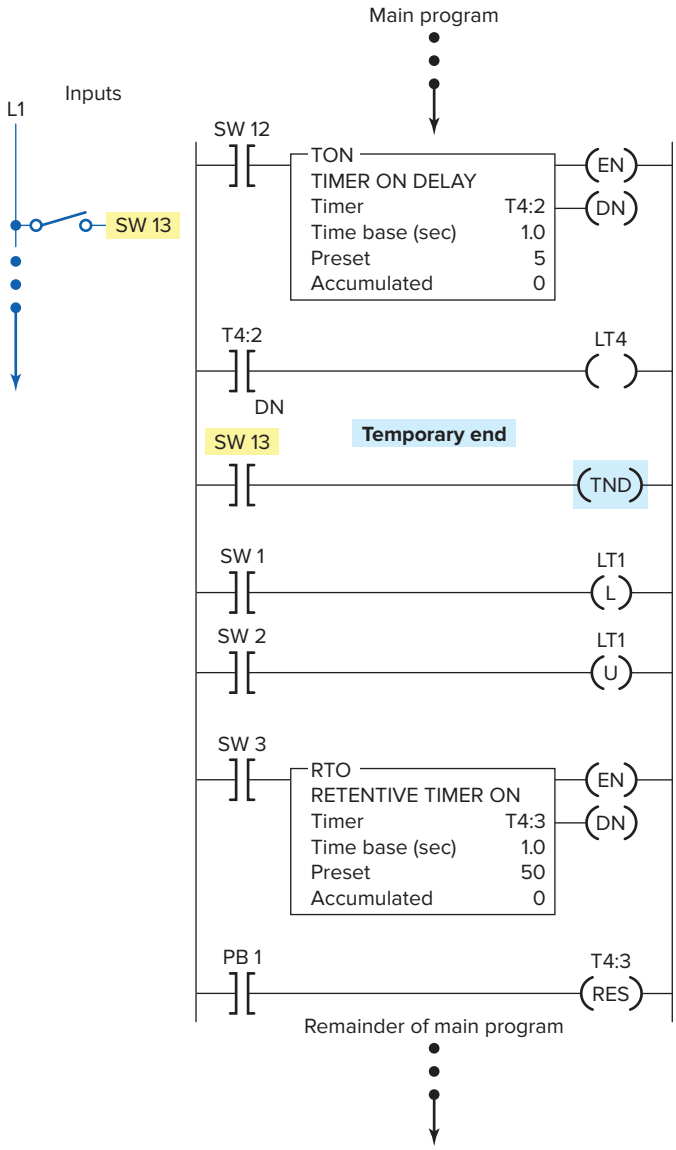


Figure 9-27 Temporary end (TND) instruction.

9.11 Suspend Instruction

The ***suspend (SUS)*** instruction is used to trap and identify specific conditions during system troubleshooting and program debugging. Figure 9-28 shows a suspend instruction in a ladder logic rung. The execution of the instruction can be summarized as follows:

- When you program the SUS instruction, you must enter a suspend ID number (number 100 is used in this example).
- When the rung is true, the SUS output instruction places the controller in the suspend mode and the PLC immediately terminates scan cycling.
- All ladder logic outputs are de-energized, but other status files have the data present when the suspend instruction is executed.
- The SUS instruction writes the suspend ID number (100) to S:7 as it executes.
- You can include several SUS instructions in a program, each with a different suspend ID and read S:7 to determine which SUS instruction caused the PLC to halt.
- Status file S:8 will contain the number of the program file that was executing when the SUS instruction executed.

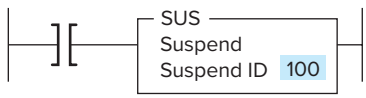


Figure 9-28 Suspend (SUS) instruction.

CHAPTER 9 REVIEW QUESTIONS

1. **a.** Two MCR output instructions are to be programmed to control a section of a program. Explain the programming procedure to be followed.
   b. State how the status of the output devices within the fenced zone will be affected when the MCR instruction makes a false-to-true transition.
   c. State how the status of the output devices within the fenced zone will be affected when the MCR instruction makes a true-to-false transition.
2. What is the main advantage of the jump instruction?
3. What types of instructions are not normally included inside the jumped section of a program? Why?
4. **a.** What is the purpose of the label instruction in the jump-to-label instruction pair?
   b. When the jump-to-label instruction is executed, in what way are the jumped rungs affected?
5. **a.** Explain what the jump-to-subroutine instruction allows the program to do.
   b. In what type of machine operation can this instruction save a great deal of duplicate programming?
6. What advantage is there to the nesting of subroutines?
7. **a.** When are the immediate input and immediate output instructions used?
   b. Why is it of little benefit to program an immediate input or immediate output instruction near the beginning of a program?
8. **a.** What does the forcing capability of a PLC allow the user to do?
   b. Outline two practical uses for forcing functions.
   c. Why should extreme care be exercised when using forcing functions?
9. Why should emergency stop circuits be hardwired instead of programmed?
10. State the function of each of the following in the basic safety wiring for a PLC installation:
    a. Main disconnect switch
    b. Isolation transformer
    c. Emergency stops
    d. Master control relay
11. Compare standard and safety PLCs with regard to:
    a. Processors
    b. Input circuitry
    c. Output circuitry
    d. Power supplies
12. When programming a motor starter circuit, why is it safer to use the starter seal-in auxiliary contact in place of a programmed contact referenced to the output coil instruction?
13. When programming stop buttons, why is it safer to use an NC pushbutton programmed to examine for an on condition than an NO pushbutton programmed to examine for an off condition?
14. Explain the selectable timed interrupt function.
15. Explain the function of the fault routine file.
16. How is the temporary end instruction used to troubleshoot a program?

CHAPTER 9 PROBLEMS

1. Answer the questions, in sequence, for the MCR program in Figure 9-29, assuming the program has just been entered and the PLC is placed in the RUN mode with all switches turned off.
   a. Switches S2 and S3 are turned on. Will outputs PL1 and PL2 come on? Why?
   b. With switches S2 and S3 still on, switch S1 is turned on. Will output PL1 or PL2 or both come on? Why?
   c. With switches S2 and S3 still on, switch S1 is turned off. Will both outputs PL1 and PL2 de-energize? Why?
   d. With all other switches off, switch S6 is turned on. Will the timer time? Why?
   e. With switch S6 still on, switch S5 is turned on. Will the timer time? Why?
   f. With switch S6 still on, switch S5 is turned off. What happens to the timer? If the timer was an

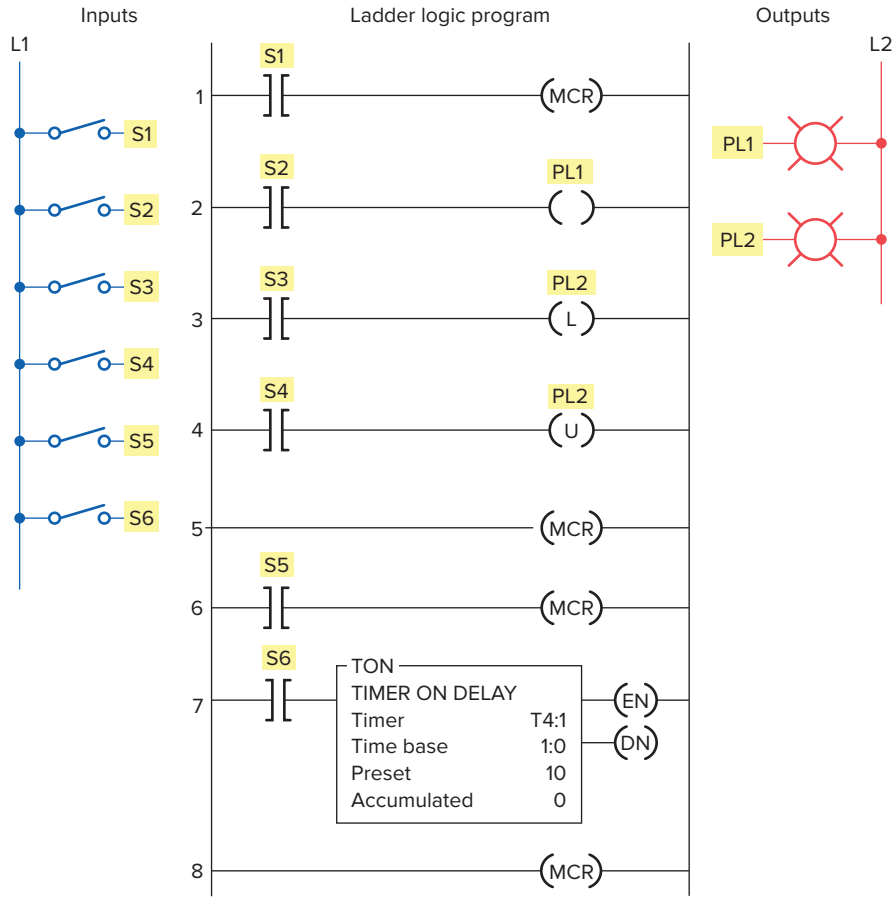


Figure 9-29 Program for Problem 1.

RTO type instead of a TON, what would happen to the accumulated value?

2. Answer the questions, in sequence, for the jump-to-label program in Figure 9-30. Assume all switches are turned *off after each operation.*

a. Switch S3 is turned on. Will output PL1 be energized? Why?

b. Switch S2 is turned on *first,* then switch S5 is turned on. Will output PL4 be energized? Why?

c. Switch S3 is turned on and output PL1 is energized. Next, switch S2 is turned on. Will output PL1 be energized or de-energized after turning on switch S2? Why?

d. All switches are turned on in order according to the following sequence: S1, S2, S3, S5, S4. Which pilot lights will turn on?

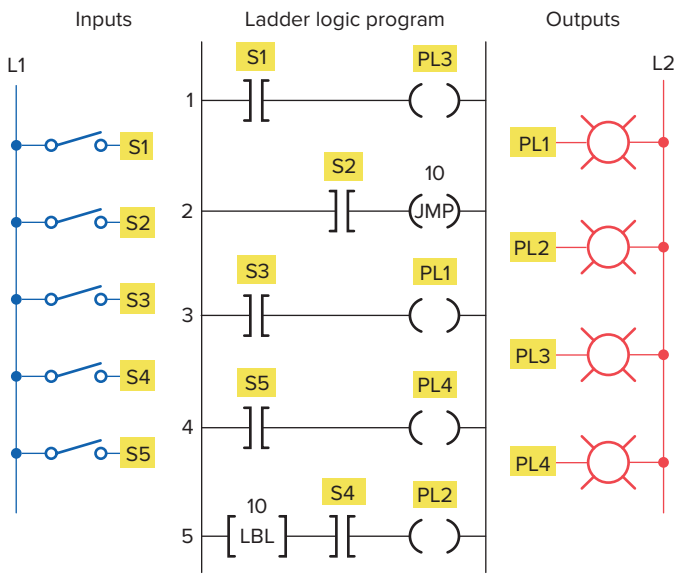


Figure 9-30 Program for Problem 2.

3. Answer the questions, in sequence, for the jump-to-subroutine and return program in Figure 9-31. Assume all switches are *turned off after each operation.*

a. Switches S1, S3, S4, and S5 are all turned on. Which pilot light will *not* be turned on? Why?

b. Switch S2 is turned on and then switch S4 is turned on. Will output PL3 be energized? Why?

c. To what rung does the RET instruction return the program scan?

4. Answer the questions, in sequence, for Figure 9-32. Assume all switches are *turned off after each operation.*

a. Switches S2, S12, and S5 are turned on in order. Will output PL5 be energized? Why?

b. All switches except S7 are turned off. Will RTO start timing? Why?

c. Switches S3 and S8 are turned on in order. Will pilot light PL2 come on? Why?

d. When will timer TON function?

e. Assume all switches are turned on. In what order will the rungs be scanned?

f. Assume all switches are turned off. In what order will the rungs be scanned?

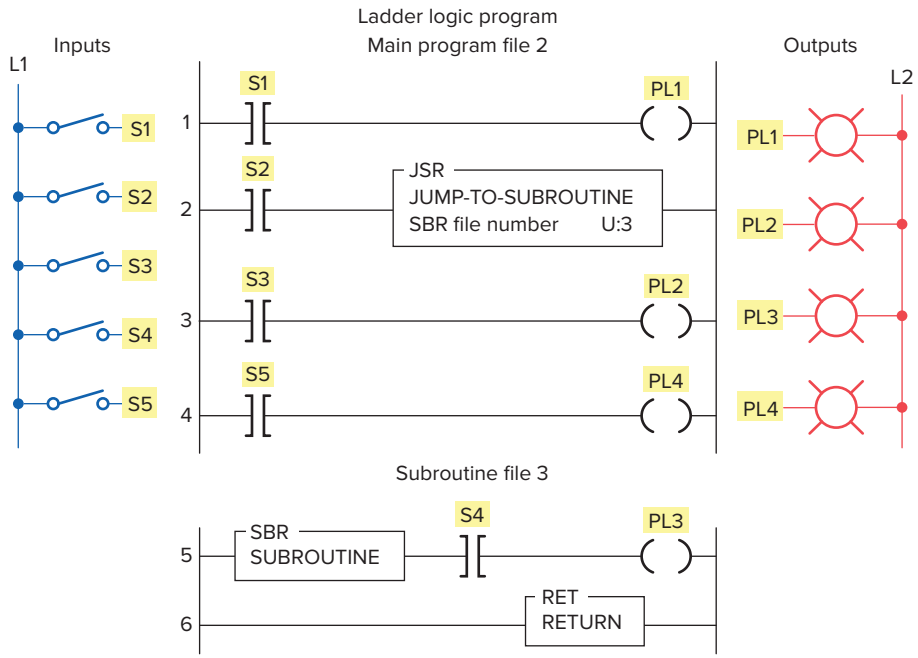


Figure 9-31 Program for Problem 3.

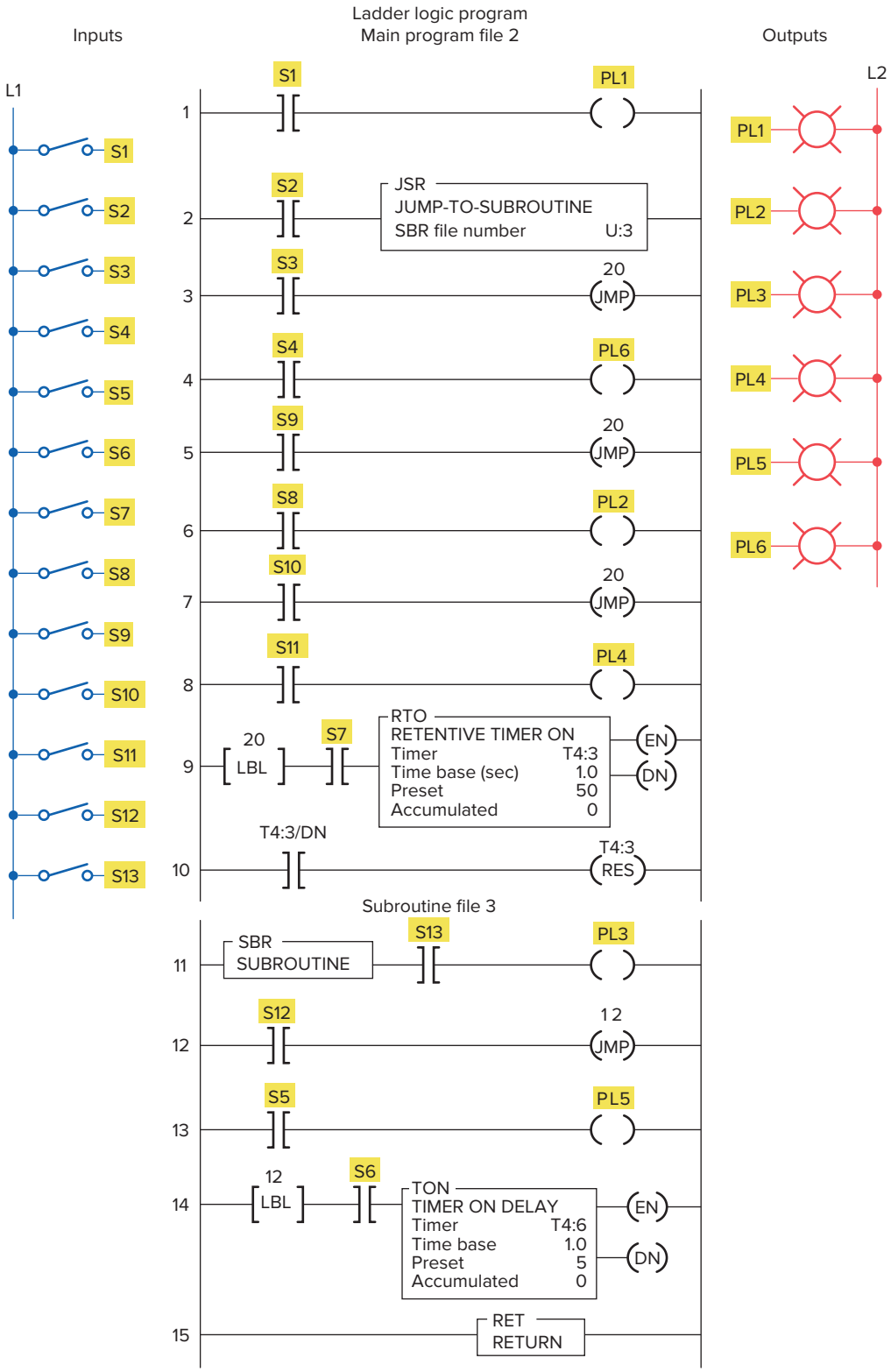
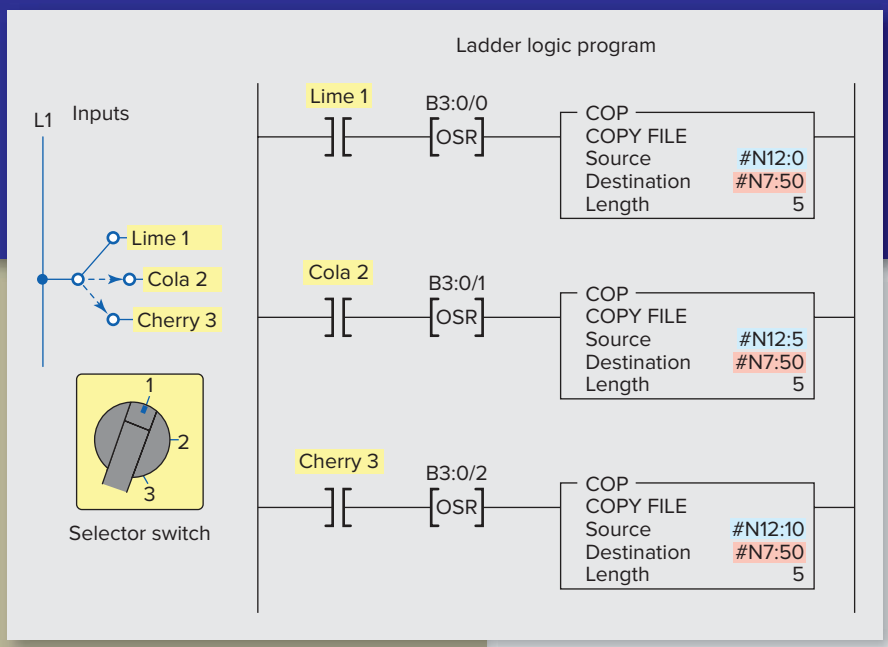


Figure 9-32 Program for Problem 4.

10

Data Manipulation Instructions



Chapter Objectives

After completing this chapter, you will be able to:

- Execute data transfer of word and file level instructions from one memory location to another
- Interpret data transfer and data compare instructions as they apply to a PLC program
- Compare the operation of discrete I/Os with that of multibit and analog types
- Understand the basic operation of PLC closed-loop control systems

Data manipulation involves transferring data and operating on data with math functions, data conversions, data comparison, and logical operations. This chapter covers both data manipulation instructions that operate on word data and those that operate on file data, which involve multiple words. Data manipulations are performed internally in a manner similar to that used in microcomputers. Examples of processes that need these operations on a fast and continuous basis are studied.

10.1 Data Manipulation

Data manipulation instructions allow numerical data stored in the controller's memory to be operated on within the control program. It includes operations involving moving or transferring numeric information stored in one memory word location to another word in a different location, and carrying out simple operations such as converting from one data format to another.

The use of data manipulation extends a controller's capability from that of simple on/off control based on binary logic, to quantitative decision making involving data comparisons, arithmetic, and conversions—which in turn can be applied to analog and positioning control.

There are two basic classes of instructions to accomplish data manipulation: instructions that operate on word data and those that operate on file, or block, data, which involve multiple words.

Each data manipulation instruction requires words of data memory for operation. The words of data memory in singular form may be referred to either as ***registers*** or as ***words,*** depending on the manufacturer. The terms *table* or *file* are generally used when a *consecutive* group of related data memory words is referenced. Figure 10-1 illustrates the difference between a word and a file. The data contained in files and words will be in the form of binary *bits* represented as series of 1s and 0s. A group of consecutive elements or words in an Allen-Bradley SLC 500 are referred to as a file.

The data manipulation instructions allow the movement, manipulation, or storage of data in either single- or multiple-word groups from one data memory area of the PLC to another. Use of these PLC instructions in applications that require the generation and manipulation of large quantities of data greatly reduces the complexity and quantity of the programming required. Data manipulation

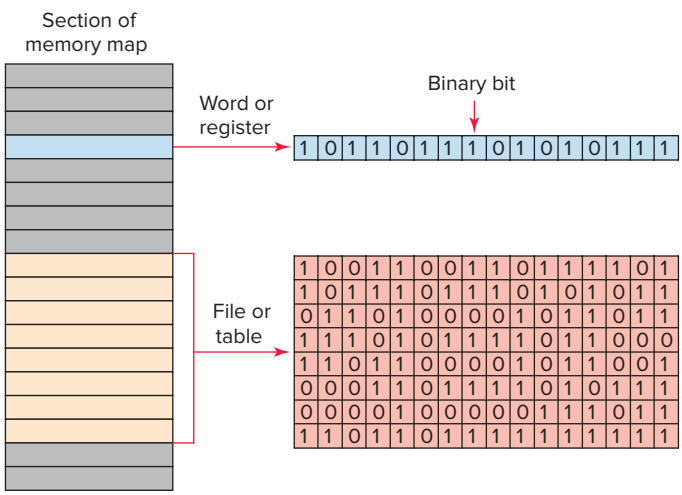


Figure 10-1 Data files, words, and bits.

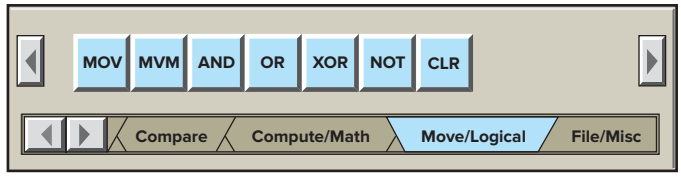


Figure 10-2 Move/Logical menu tab.

can be placed in two broad categories: *data transfer* and *data comparison.*

The manipulation of entire words is an important feature of a programmable controller. This feature enables PLCs to handle inputs and outputs containing multiple bit configurations such as analog inputs and outputs. Arithmetic functions also require data within the programmable controller to be handled in word or register format. To simplify the explanation of the various data manipulation instructions available, the instruction protocol for the Allen-Bradley SLC 500 families of PLCs will be used. Again, even though the format and instructions vary with each manufacturer, the concepts of data manipulation remain the same.

Figure 10-2 shows the **Move/Logical** menu tab for the SLC 500 PLC and its associated RSLogix software. The commands can be summarized as follows:

MOV (Move)—Moves the source value to the destination.

MVM (Masked Move)—Moves data from a source location to a selected portion of the destination.

AND (And)—Performs a bitwise AND operation.

OR (Or)—Performs a bitwise OR operation.

XOR (Exclusive Or)—Performs a bitwise XOR operation.

NOT (Not)—Performs a bitwise NOT operation.

CLR (Clear)—Sets all bits of a word to zero.

10.2 Data Transfer Operations

Data transfer instructions simply involve the transfer of the contents from one word or register to another. Figure 10-3*a* and *b* illustrate the concept of moving numerical binary data from one memory location to another. Figure 10-3*a* shows the original data are in register N7:30 and N7:20. Figure 10-3*b* shows that after the data transfer has occurred register N7:20 now holds a duplicate of the information that is in register N7:30. The previously existing data stored in register N7:20 have been replaced with those of N7:30. This process is referred to as ***writing over the existing data.***

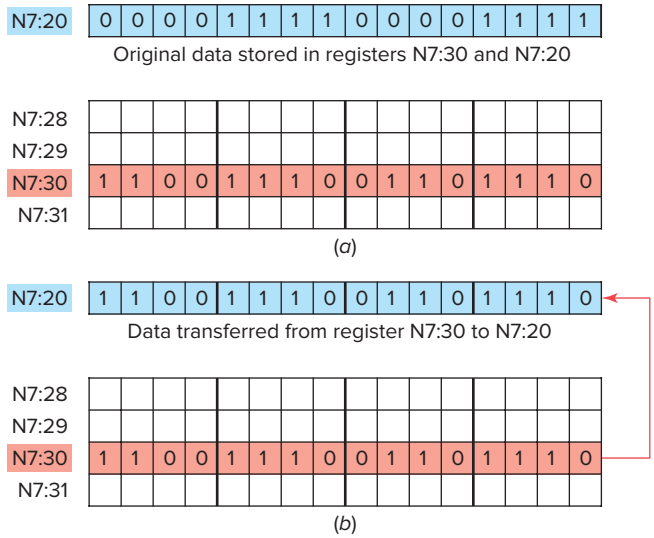


Figure 10-3 Data transfer concept.

Data transfer instructions can address almost any location in the memory. Prestored values can be automatically retrieved and placed in any new location. That location may be the preset register for a timer or counter or even an output register that controls a seven-segment display.

SLC 500 controllers use a block-formatted ***move (MOV)*** instruction to accomplish data moves. The MOV instruction is used to copy the value in one register or word to another. This instruction copies data from a *source* register to a *destination* register. Figure 10-4 shows an example of the MOV instruction. The operation of the program can be summarized as follows:

- When the rung is true, input switch *A* closed, the value stored at the source address, N7:30, is copied into the destination address, N7:20.
- When the rung goes false, input switch *A* opened, the destination address will retain the value unless it is changed elsewhere in the program.
- The source value remains unchanged and no data conversion occurs.

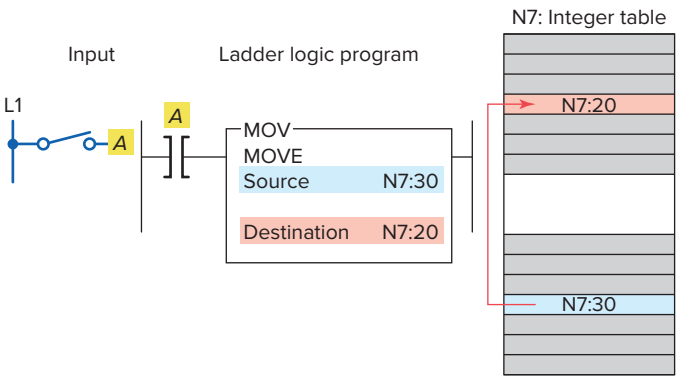


Figure 10-4 SLC 500 block-formatted move instruction.

- The instruction may be programmed with input conditions preceding it, or it may be programmed unconditionally.

A **Masked Move (MVM)** instruction functions like a standard move (MOV) instruction, except that a masked move allows data to be filtered out that is not required to be moved. The mask parameter specified in the instruction block is what performs this filtering process. This mask parameter can be either a word address location or a numerical constant. The operation of the mask parameter can be summarized as follows:

- The pattern of characters in the mask determines which source bits will be passed through to the destination address.
- The bits in the mask that are set to zero (0) do not pass data.
- Only the bits in the mask that are set to one (1) will pass the source data through to the destination.
- Bits in the destination are not affected when the corresponding bits in the mask are zero.
- The MVM instruction is used to copy the desired part of a 16-bit word by masking the rest of the value.

Figure 10-5 shows an example of a **mask move (MVM)** instruction. This instruction transfers data through the mask from the source address, B3:0, to the destination address, B3:4. The operation of the program can be summarized as follows:

- The mask may be entered as an address or in hexadecimal format, and its value will be displayed in hexadecimal.

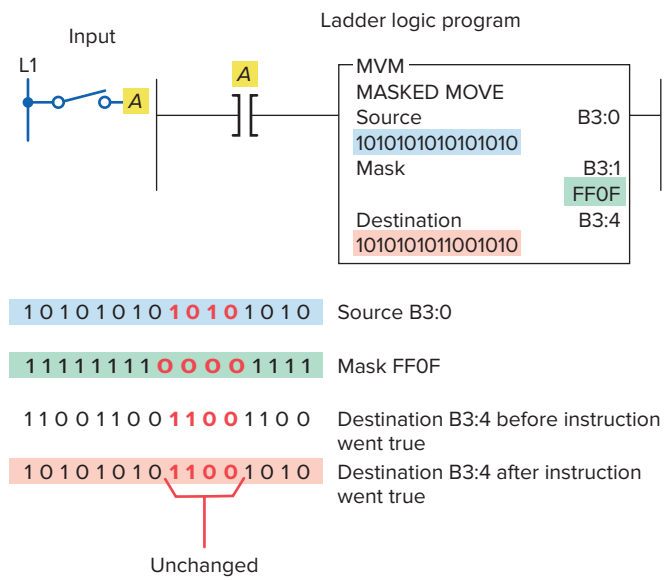


Figure 10-5 Masked move (MVM) instruction.

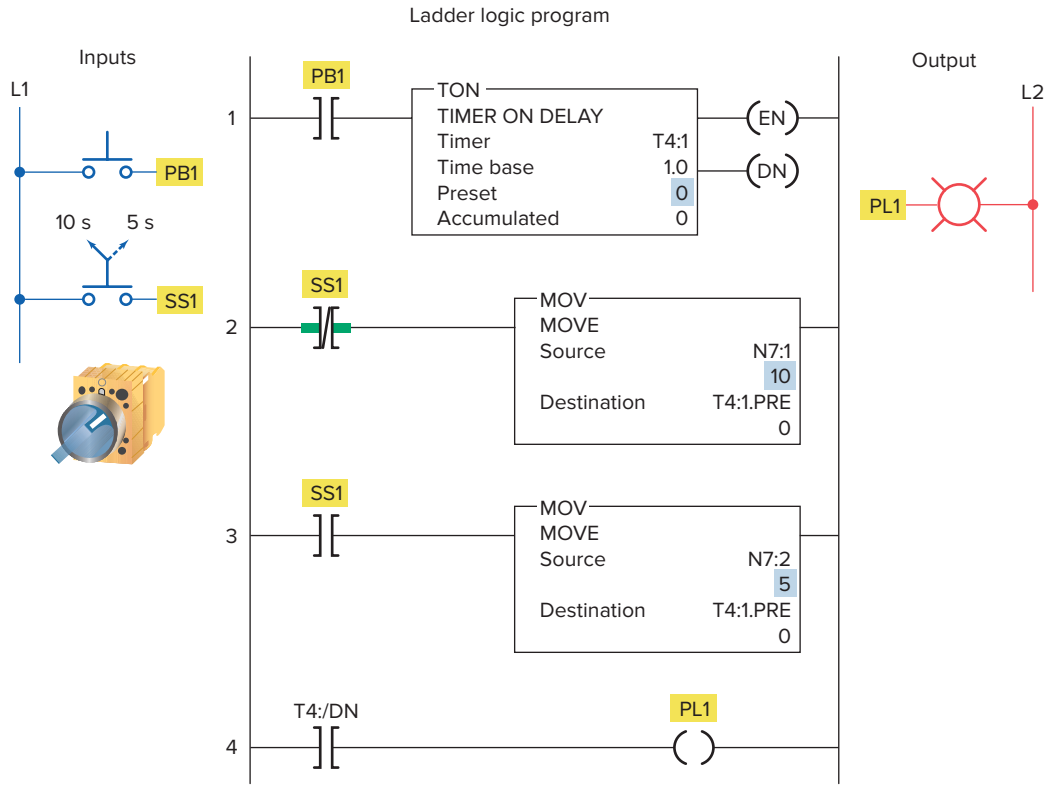


Figure 10-6 Move instruction used to change the preset time of a timer.

- Where there is a 1 in the mask, data will pass from the source to the destination.
- Where there is a 0 in the mask, data in the destination will remain in their last state.
- Status in bits 4–7 are unchanged due to zeroes in the mask (remained in their last state).
- Status in bits 0–3 and 8–15 were copied from the source to destination when the MVM instruction went true.
- The mask must be the same word size as the source and destination.

The program of Figure 10-6 illustrates how the move (MOV) instruction can be used to create variable preset timer values. A two-position selector switch is operated to select one of two preset timer values. Operation of the program can be summarized as follows:

- When the selector switch is in the open 10 s position, rung 2 has logic continuity and rung 3 does not.
- As a result, the value 10 stored at the source address, N7:1, is copied into the destination address, T4:1.PRE.
- Therefore, the preset value of timer T4:1 will change from 0 to 10.
- When pushbutton PB1 is closed, there will be a 10 s delay period before the pilot light is energized.
- When the selector switch is in the closed 5 s position, rung 3 has logic continuity and rung 2 does not.
- As a result, the value 5 stored at the source address, N7:2, is copied into the destination address, T4:1. PRE.
- Closing pushbutton PB1 will now result in a 5 s time-delay period before the pilot light is energized.

The program of Figure 10-7 illustrates how the move (MOV) instruction can be used to create variable preset counter values. The operation of the program can be summarized as follows:

- Limit switch LSI is programmed to the input of up-counter C5:1 and counts the number of parts coming off a conveyor line onto a storage rack.
- Three different types of products are run on this line.
- The storage rack has room for only 300 boxes of product *A* or 175 boxes of product *B* or 50 boxes of product *C*.

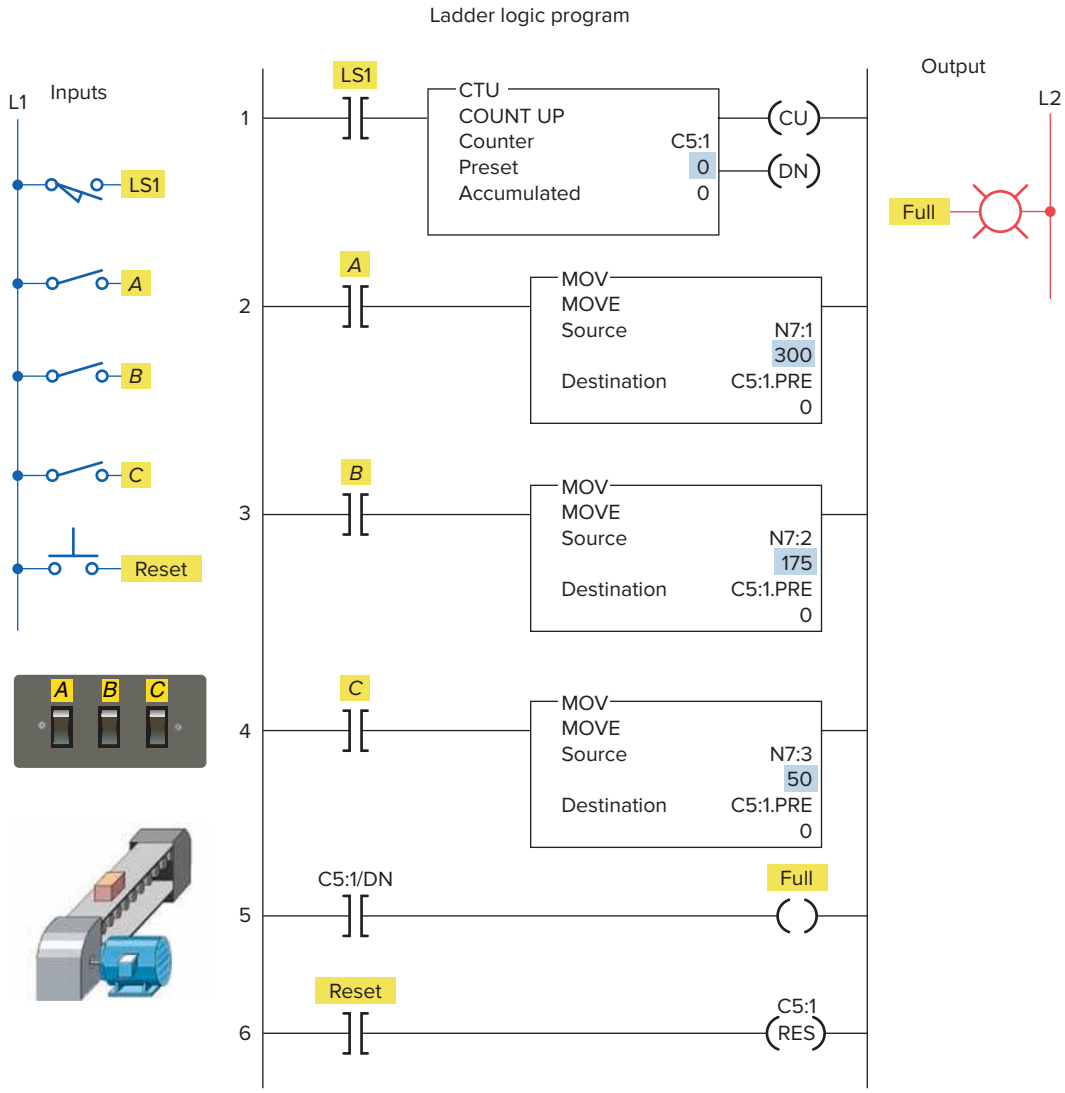


Figure 10-7 Move instruction used to change the preset count of a counter.

- Three momentary switches are used to select the desired preset counter value depending on the product line (*A, B,* or *C*) being manufactured.
- A reset button is provided to reset the accumulated count to 0.
- A pilot lamp is switched on to indicate when the storage rack is full.
- The program has been constructed so that normally only one of the three switches will be closed at any one time. If more than one of the preset counter switches is closed, the *last* value is selected.

A ***file*** is a group of related consecutive words in the data table that have a defined start and end and are used to store information. For example, a batch process program may contain several separate recipes in different files that can be selected by an operator.

In some instances it may be necessary to shift complete files from one location to another within the programmable controller memory. Such data shifts are termed ***file-to-file shifts.*** File-to-file shifts are used when the data in one file represent a set of conditions that must interact with the programmable controller program several times and, therefore, must remain *intact* after each operation. Because the data within this file must also be changed by the program action, a second file is used to handle the data changes, and the information within that file is allowed to be altered by the program. The data in the first file, however, remain constant and therefore can be used many times. Other types of data manipulation used with file instructions include word-to-file and file-to-word moves, as illustrated in Figure 10-8.

Files allow large amounts of data to be scanned quickly and are useful in programs requiring the transfer,

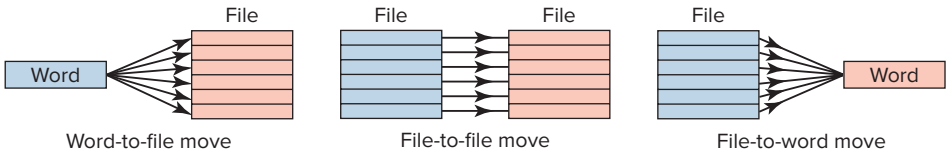


Figure 10-8 Moving data using file instructions.

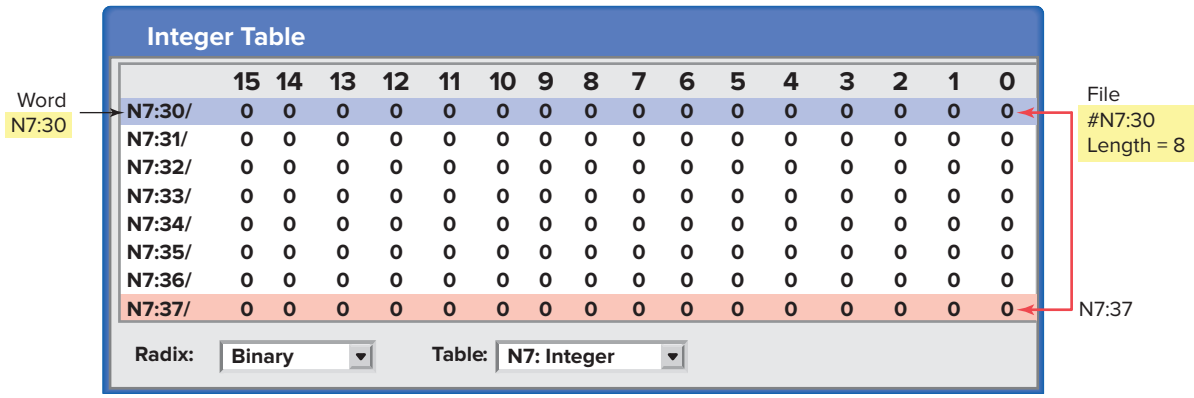


Figure 10-9 SLC 500 word and file address.

comparison, or conversion of data. Most PLC manufacturers display file instructions in block format on the programming terminal screen. Figure 10-9 compares the SLC 500 controller word and file addressing. The addressing formats can be summarized as follows:

- The address that defines the beginning of a file or group of words starts with the pound sign #.
- The # prefix is omitted in a single word or element address.
- Address N7:30 is a word address that represents a single word: word number 30 in integer file 7.
- Address #N7:30 represents the starting address of a group of consecutive words in integer file 7. The length is eight words, which is determined by the instruction where the file address is used.

The ***file arithmetic and logic (FAL)*** instruction is used to copy data from one file to another and to do file math and file logic. This instruction is available on Allen-Bradley PLC-5 and ControlLogix platforms. An example of the FAL instruction is shown in Figure 10-10.

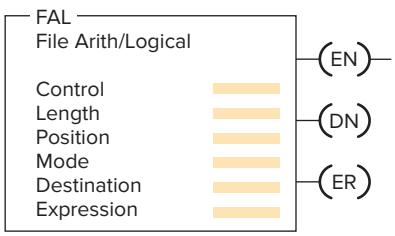


Figure 10-10 File Arithmetic/Logical (FAL) instruction.

The basic operation of the FAL instruction is similar in all functions and requires the following parameters and PLC-5 addresses to be entered in the instruction:

Control

- Is the first entry and the address of the control structure in the control area (R) of processor memory.
- The processor uses this information to run the instruction.
- The default file for the control file is data file 6.
- The control element for the FAL instruction must be unique for that instruction and may not be used to control any other instruction.
- The control element is made up of three words.
- The control word uses four control bits: bit 15 (enable bit), bit 13 (done bit), bit 11 (error bit), and bit 10 (unload bit).
- For ControlLogix the control address would be a tag such as control_1 with a data type of control.

Length

- Is the second entry and represents the file length.
- This entry will be in words, except for the floating-point file, for which the length is in elements. (A floating-point element consists of two words.)
- The maximum length possible is 1000 elements. Enter any decimal number from 1 to 1000.
- For ControlLogix the number would be a double integer (DINT).

Position

- Is the third entry and represents the current location in the data block that the processor is accessing.
- It points to the word being operated on.
- The position starts with 0 and indexes to 1 less than the file length.
- You generally enter a 0 to start at the beginning of a file. You may also enter another position at which you want the FAL to start its operation.
- When the instruction resets, however, it will reset the position to 0.
- You can manipulate the position from the program.

Mode

- Is the fourth entry and represents the number of file elements operated on per program scan. There are three choices: all mode, numeric mode, and incremental mode.

All Mode

- For this mode you enter the letter *A*.
- In the all mode, the instruction will transfer the complete file of data in *one* scan.
- The enable (EN) bit will go true when the instruction goes true and will follow the rung condition.
- When all of the data have been transferred, the done (DN) bit will go true. This change will occur on the same scan during which the instruction goes true.
- If the instruction does not go to completion due to an error in the transfer of data (such as trying to store too large or too small a number for the data-table type), the instruction will stop at that point and set the error (ER) bit. The scan will continue, but the instruction will not continue until the error bit is reset.
- If the instruction goes to completion, the enable bit and the done bit will remain set until the instruction goes false, at which point the position, the enable bit, and the done bit will all be reset to 0.

Numeric Mode

- For this mode you enter a decimal number (1–1000).
- In the numeric mode, the file operation is distributed over a number of program scans.
- The value you enter sets the number of elements to be transferred per scan.
- The numeric mode can decrease the time it takes to complete a program scan. Instead of waiting for the total file length to be transferred in one scan, the numeric mode breaks up the transfer of the file data into multiple scans, thereby cutting down on the instruction execution time per scan.

Incremental mode

- For this mode you enter the letter I.
- In the incremental mode, one element of data is operated on for every false-to-true transition of the instruction.
- The first time the instruction sees a false-to-true transition and the position is at 0, the data in the first element of the file are operated on. The position will remain at 0 and the UL bit will be set. The EN bit will follow the instruction's condition.
- On the second false-to-true transition, the position will index to 1, and data in the second word of the file will be operated on.
- The UL bit controls whether the instruction will operate just on data in the current position, or whether it will index the position and then transfer data. If the UL bit is reset, the instruction—on a false-to-true transition of the instruction—will operate on the data in the current position and set the UL bit. If the UL bit is set, the instruction—on a false-to-true transition of the instruction—will index the position by 1 and operate on the data in their new position.

Destination

- Is the fifth entry and is the address at which the processor stores the result of the operation.
- The instruction converts to the data type specified by the destination address.
- It may be either a file address or an element address.

Expression

- Is the last entry and contains addresses, program constants, and operators that specify the source of data and the operations to be performed.
- The expression entered determines the function of the FAL instruction.
- The expression may consist of file addresses, element addresses, or a constant and may contain only one function because the FAL instruction may perform only one function.

Figure 10-11 shows an example of a file-to-file copy function using the FAL instruction. The operation of the program can be summarized as follows:

- When input *A* goes true, data from the expression file #N7:20 will be copied into the destination file #N7:50.

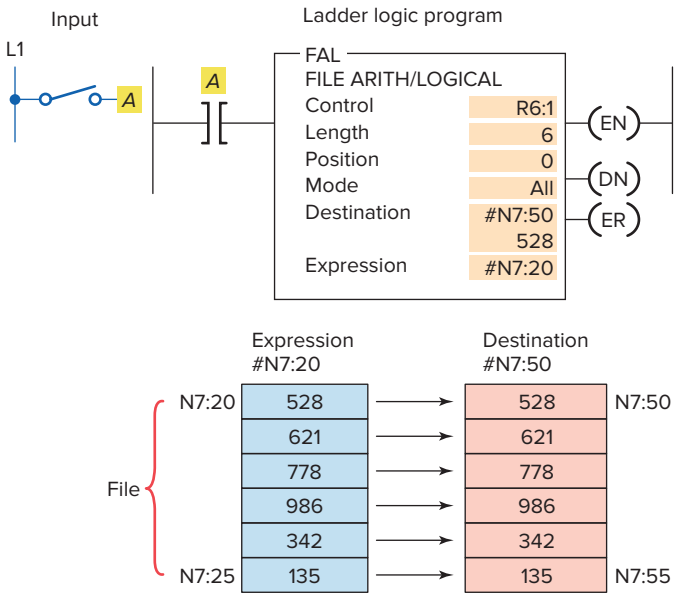


Figure 10-11 File-to-file copy function using the FAL instruction.

- The length of the two files is set by the value entered in the control element word R6:1.LEN.
- In this instruction, we have also used the ALL mode, which means all of the data will be transferred in the first scan in which the FAL instruction sees a false-to-true transition.
- The DN bit will also come on in that scan unless an error occurs in the transfer of data, in which case the ER bit will be set, the instruction will stop operation at that position, and then the scan will continue at the next instruction.

Figure 10-12 shows an example of a file-to-word copy function using the FAL instruction. The operation of the program can be summarized as follows:

- With each false-to-true rung transition of input *A*, the processor reads one word of integer file N29.
- The processor starts reading at word 0, and writes the image into word 5 of integer file N29.
- The instruction writes over any data in the destination.

Figure 10-13 shows an example of a word-to-file copy function using the FAL instruction. It is similar to the file-to-word copy function except that the instruction copies data from a word address into a file. The operation of the program can be summarized as follows:

- The expression is a word address (N7:100) and the destination is a file address (#N7:101).

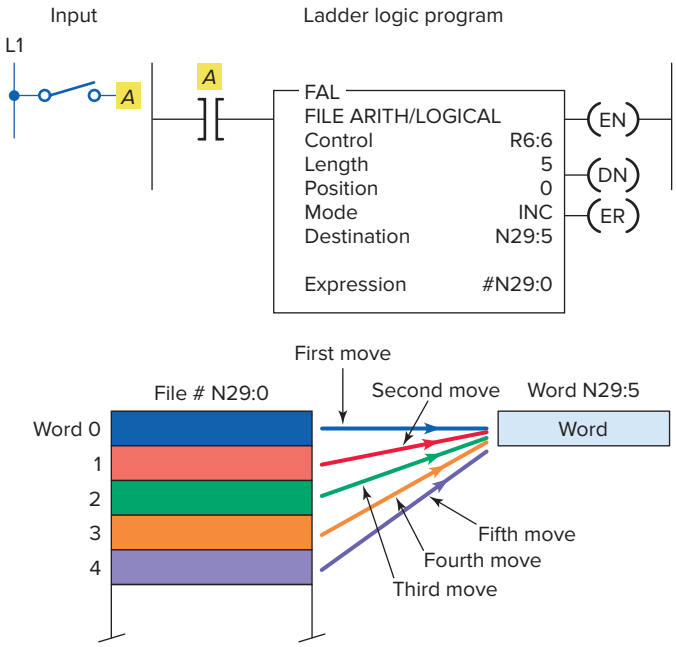


Figure 10-12 File-to-word copy function using the FAL instruction.

- If we start with position 0, the data from N7:100 will be copied into N7:101 on the first false-to-true transition of input *A*.
- The second false-to-true transition of input *A* will copy the data from N7:100 into N7:102.
- On successive false-to-true transitions of the instruction, the data will be copied into the next position in the file until the end of the file, N7:106, is reached.

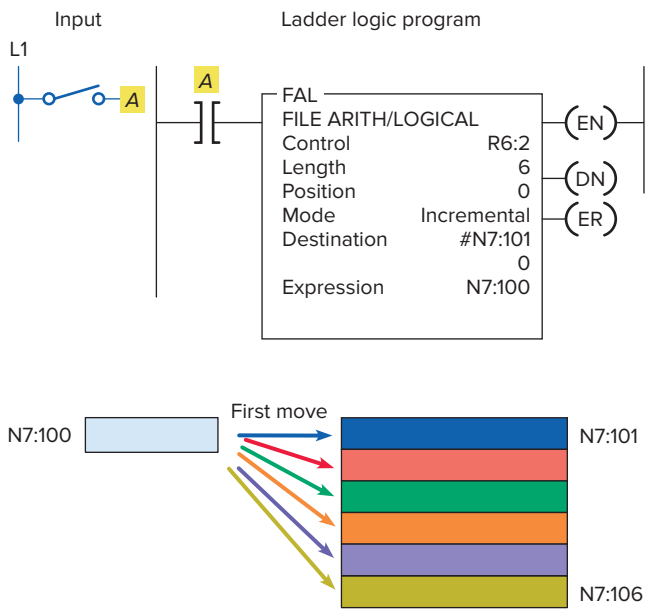


Figure 10-13 Word-to-file copy function using the FAL instruction.

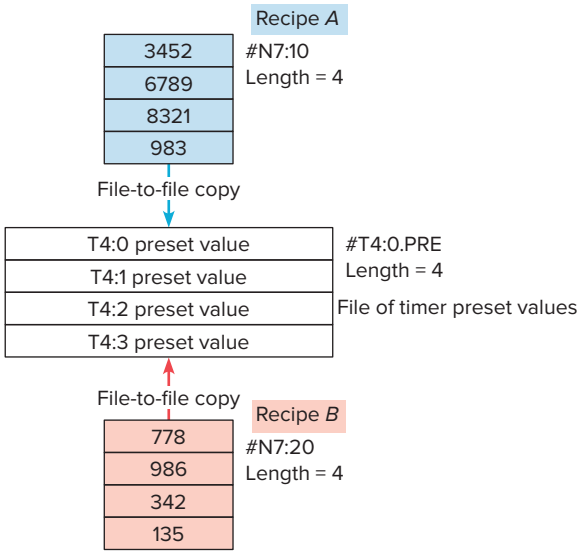


Figure 10-14 Copying recipes and storing values for timer presets.

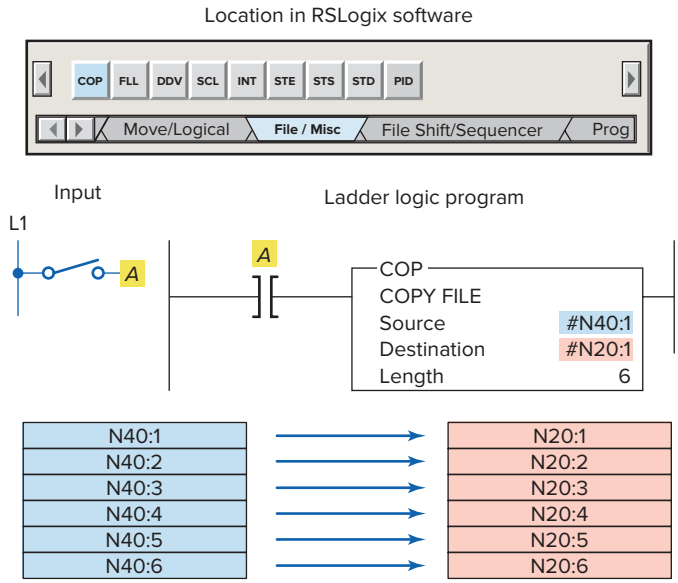


Figure 10-15 File copy (COP) instruction.

The exceptions to the rule that file addresses must take consecutive words in the data table are in the *timer, counter,* and *control data* files for the FAL instruction. In these three data files, if you designate a file address, the FAL instruction will take every third word in that file and make a file of preset, accumulated, length, or position data within the corresponding file type. This might be done, for example, so that recipes storing values for timer presets can be moved into the timer presets, as illustrated in Figure 10-14.

The ***file copy (COP)*** instruction and the ***fill file (FLL)*** instruction are high-speed instructions that operate more quickly than the same operation with the FAL instruction. Unlike the FAL instruction, there is no control element to monitor or manipulate. Data conversion does not take place, so the source and destination should be the same file types. An example of the file COP instruction is shown in Figure 10-15. The operation of the program can be summarized as follows:

- Both the source and destination are file addresses.
- When input *A* goes true, the values in file N40 are copied to file N20.
- The instruction copies the entire file length for each scan during which the instruction is true.

An example of the fill file (FLL) instruction is shown in Figure 10-16. It operates in a manner similar to the FAL instruction that performs the word-to-file copy in the ALL mode. The operation of the program can be summarized as follows:

- When input *A* goes true, the value in N15:5 is copied into N20:1 through N20:6.

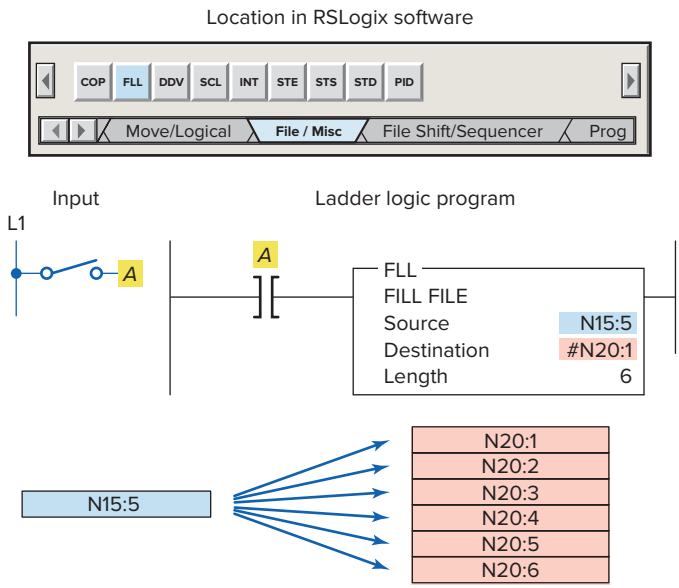


Figure 10-16 Fill file (FLL) instruction.

- Because the instruction transfers to the end of the file, the file will be filled with the same data value in each word.

The FLL instruction is frequently used to zero all of the data in a file, as illustrated in the program of Figure 10-17. The operation of the program can be summarized as follows:

- Momentarily pressing pushbutton PB1 copies the contents of file #N10:0 into file #N12:0.
- Momentarily pressing pushbutton PB2 then clears file #N12:0.
- Note that 0 is entered for the source value.

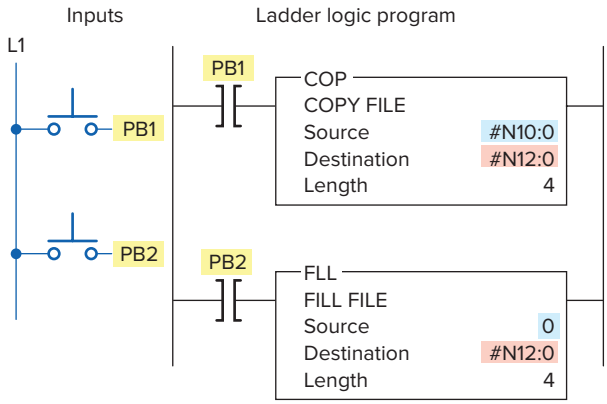


Figure 10-17 Using the FLL instruction to change all the data in a file to zero.

Figure 10-18 is an example of the copy (COP) instruction used as part of a PLC drink-manufacturing program. The operation of the program can be summarized as follows:

- A three-position selector switch is used for drink selection.
- Each selector switch position is electrically isolated so that only one input circuit can be energized at any one time.
- Each of the three selector switch inputs is wired to its corresponding input module address.
- Each recipe uses 5 memory words.
- Depending on the type of drink selected, the recipe is copied to the common working register #N7:50.
- The OSR instruction ensures that the copy instruction is executed only once on a false-to-true transition of the selector switch. In this way, if making the same recipe over a long period of time, the recipe needs to be copied only one time, not at every scan.

10.3 Data Compare Instructions

Data transfer operations are all **output** instructions, whereas ***data compare*** instructions are ***input*** instructions. Data compare instructions are used to compare numerical values. These instructions compare the data stored in two or more words (or registers) and make decisions based on the program instructions. Numeric values in two words of memory can be compared for each of the basic data compare instructions shown in Figure 10-19, depending on the PLC.

Data comparison concepts have already been used with the timer and counter instructions. In both these instructions,

Name	Symbol
Equal to	(=)
Not equal to	(≠)
Less than	(<)
Greater than	(>)
Less than or equal to	(≤)
Greater than or equal to	(≥)

Figure 10-19 Basic PLC data compare instructions.

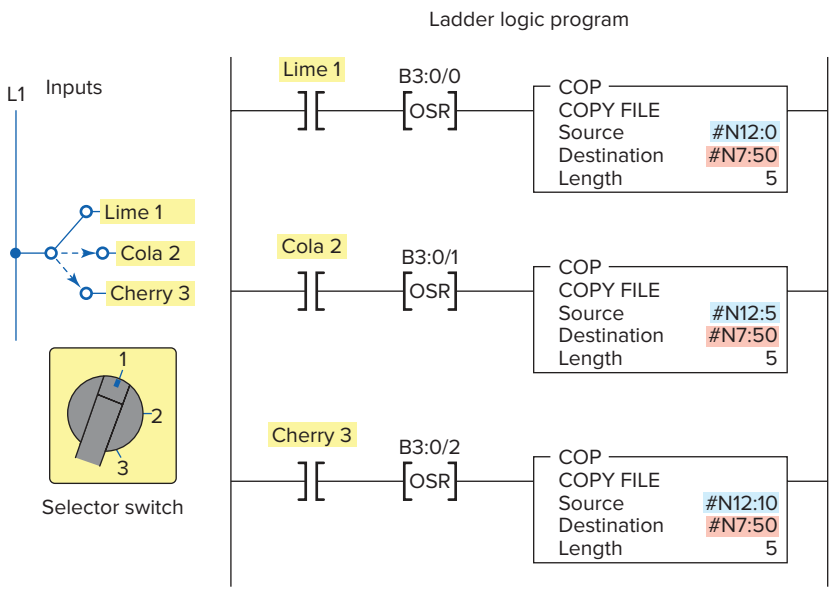


Figure 10-18 The copy (COP) instruction used as part of a PLC drink-manufacturing program.

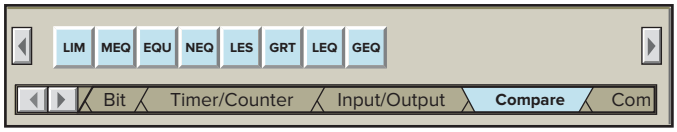


Figure 10-20 Compare menu tab.

an output was turned on or off when the accumulated value of the timer or counter equaled its preset value. What actually occurred was that the accumulated numeric data in one memory word was compared to the preset value of another memory word on each scan of the processor. When the processor saw that the accumulated value was equal to the preset value, it switched the output on or off.

Comparison instructions are used to test pairs of values to determine if a rung is true. Figure 10-20 shows the Compare menu tab for the Allen-Bradley SLC 500 PLC and its associated RSLogix software. The compare instructions can be summarized as follows:

LIM (Limit test)—Tests whether one value is within the limit range of two other values.

MEQ (Masked Comparison for Equal)—Tests portions of two values to see whether they are equal. Compares 16-bit data of a source address to 16-bit data at a reference address through a mask.

EQU (Equal)—Tests whether the value of Source A is equal to the value of Source B

NEQ (Not Equal)—Tests whether the value of Source A is not equal to the value of Source B

LES (Less Than)—Tests whether the value of Source A is less than the value of Source B

GRT (Greater Than)—Tests whether the value of Source A is greater than the value of Source B

LEQ (Less Than or Equal)—Tests whether the value of Source A is less than or equal to the value of Source B.

GEQ (Greater Than or Equal)—Tests whether the value of Source A is greater than or equal to the value of Source B

The ***equal (EQU)*** instruction is an input instruction that compares source *A* to source *B:* when source *A* is equal to source *B,* the instruction is logically true; otherwise it is logically false. Figure 10-21 shows an example

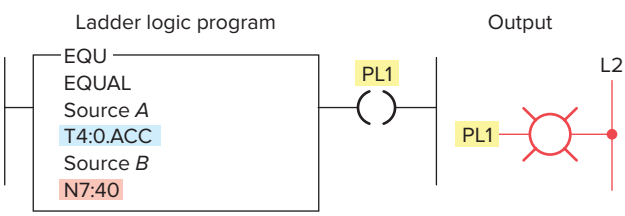


Figure 10-21 EQU logic rung.

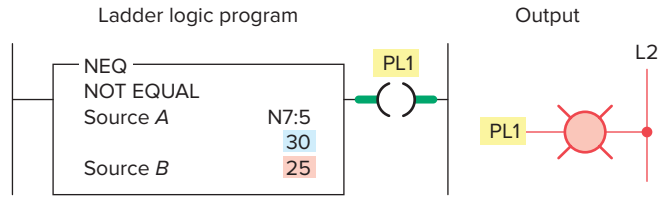


Figure 10-22 NEQ logic rung.

of an EQU logic rung. The operation of the rung can be summarized as follows:

- When the accumulated value of counter T4:0 stored in source *A*'s address equals the value in source *B*'s address, N7:40, the instruction is true and the output is energized.
- Source *A* may be a word address or a floating-point address.
- Source *B* may be a word address, a floating-point address, or a constant value.
- With the equal instruction, the floating-point data is not recommended because of the exactness required. One of the other comparison instructions, such as the limit test, is preferred.

The ***not equal (NEQ)*** instruction is an input instruction that compares source *A* to source *B:* when source *A* is not equal to source *B,* the instruction is logically true; otherwise it is logically false. Figure 10-22 shows an example of an NEQ logic rung. The operation of the rung can be summarized as follows:

- When the value stored at source *A*'s address, N7:5, is not equal to 25, the output will be true; otherwise, the output will be false.
- The value stored at Source *A* is 30.
- The value stored at Source *B* is 25.
- Since the two values are not the same the output will be true or on.
- In all input-comparison instructions, Source *A* must be an address and Source *B* can be an address or a constant.

The ***greater than (GRT)*** instruction is an input instruction that compares source *A* to source *B:* when source *A* is greater than source *B,* the instruction is logically true; otherwise it is logically false. Figure 10-23 shows an

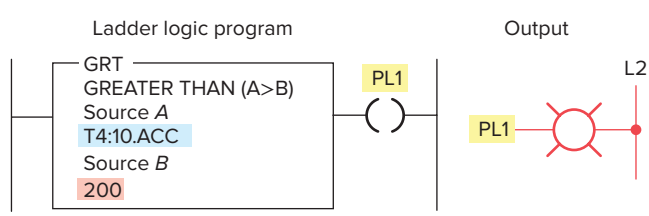


Figure 10-23 GRT logic rung.

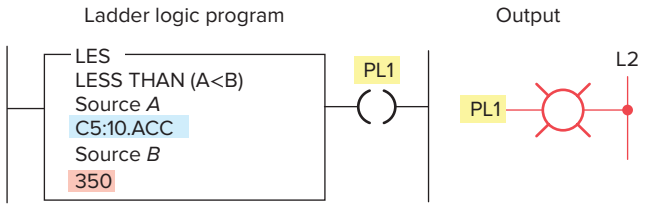


Figure 10-24 LES logic rung.

example of a GRT logic rung. The operation of the rung can be summarized as follows:

- The instruction is either true or false, depending on the values being compared.
- When the accumulated value of timer T4:10, stored at the address of source A, is greater than the constant 200 of source B, the output will be on; otherwise the output will be off.

The ***less than (LES)*** instruction is an input instruction that compares source *A* to source *B:* when source *A* is less than source *B,* the instruction is logically true; otherwise it is logically false. Figure 10-24 shows an example of an LES logic rung. The operation of the rung can be summarized as follows:

- The instruction is either true or false, depending on the values being compared.
- When the accumulated value of counter C5:10, stored at the address of source *A,* is less than the constant 350 of source *B,* the output will be on; otherwise, it will be off.

The ***greater than or equal (GEQ)*** instruction is an input instruction that compares source *A* to source *B:* when source *A* is greater than or equal to source *B,* the instruction is logically true; otherwise it is logically false. Figure 10-25 shows an example of a GEQ logic rung. The operation of the rung can be summarized as follows:

- When the value stored at the address of source *A,* N7:55, is greater than or equal to the value stored at the address of source *B,* N7:12, the output will be true; otherwise, it will be false.
- The value stored at source *A* is 100.

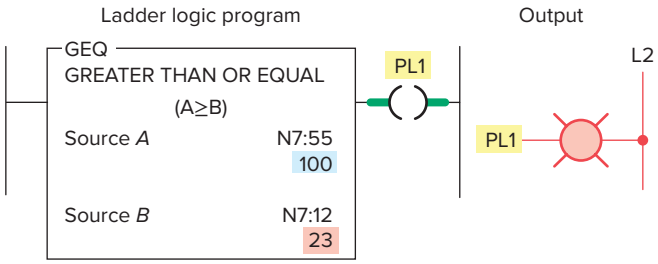


Figure 10-25 GEQ logic rung.

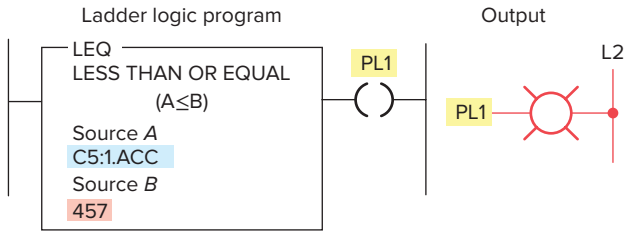


Figure 10-26 LEQ logic rung.

- The value stored at source *B* is 23.
- Therefore the output will be true or on.

The ***less than or equal (LEQ)*** instruction is an input instruction that compares source *A* to source *B:* when source *A* is less than or equal to source *B,* the instruction is logically true; otherwise it is logically false. Figure 10-26 shows an example of an LEQ logic rung. The operation of the rung can be summarized as follows:

- When the accumulated count of counter C5:1 is less than or equal to 457, the pilot light will turn on.
- The accumulated value of the counter is less than 457.
- Therefore the output will be false or off.

Figure 10-27 shows an example of an up-counter used in conjunction with the LES, EQU, and GRT compare instructions to trigger outputs based on different values of the counter's accumulated count. The operation of the program can be summarized as follows:

- A Less Than (LES) 10 comparison is made with the counter's accumulative value. As a result, the LESS Output will be energized anytime the accumulated count is 9 or less.
- An Equal (EQU) to 20 comparison is made with the counter's accumulative value. As a result, the EQU Output will be energized only when the accumulated count is 20.
- A Greater Than (GRT) 30 comparison is made with the counter's accumulative value. As a result, the GRT Output will be energized anytime the accumulated count is 31 or more.

The ***limit test (LIM)*** instruction is used to test whether values are within or outside the specified range. Applications in which the limit test instruction is used include allowing a process to operate as long as the temperature is within or outside a specified range.

Programming the LIM instruction consists of entering three parameters: low limit, test, and high limit. The limit test instruction functions in the following two ways:

- ***The instruction is true if***—The lower limit is equal to or less than the higher limit, and the test

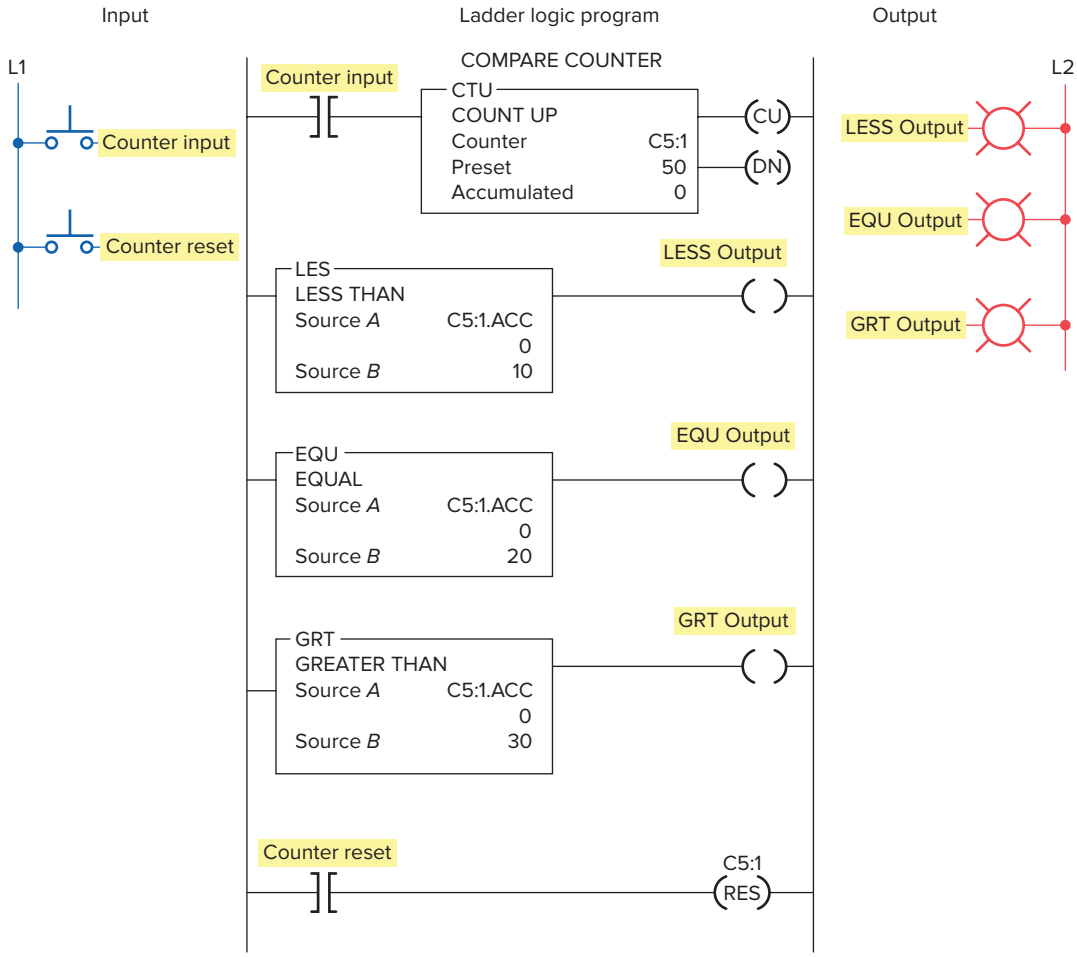


Figure 10-27 Triggering outputs based on the accumulated value of a counter.

parameter value is equal to or inside the limits. Otherwise the instruction is false.

- ***The instruction is true if***—The lower limit has a value greater than the higher limit, and the instruction is equal to or outside the limits. Otherwise the instruction is false.

The limit test instruction is said to be circular because it can function in either of two ways. Figure 10-28 shows an example of an LIM instruction where the low limit value is less than the high limit value. The operation of the logic rung can be summarized as follows:

- The high limit has a value of 50, and the low limit has a value of 25.
- Instruction is true for values of the test from 25 through 50.
- Instruction is false for test values less than 25 or greater than 50.
- Instruction is true because the test value is 48.

Figure 10-29 shows an example of an LIM instruction where the low limit value is greater than the high limit

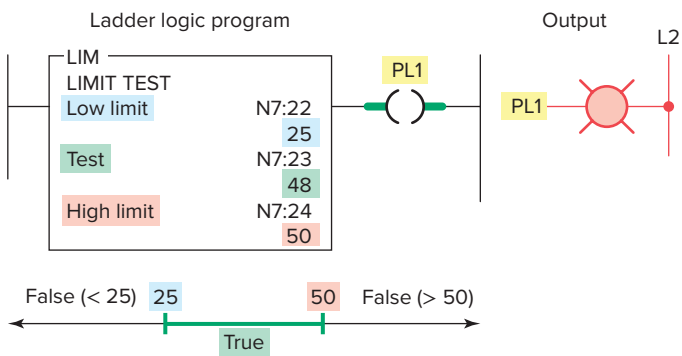


Figure 10-28 LIM instruction where the low limit value is less than the high limit value.

value. The operation of the logic rung can be summarized as follows:

- The high limit has a value of 50, and the low limit has a value of 100.
- Instruction is true for test values of 50 and less than 50 and for test values of 100 and greater than 100.

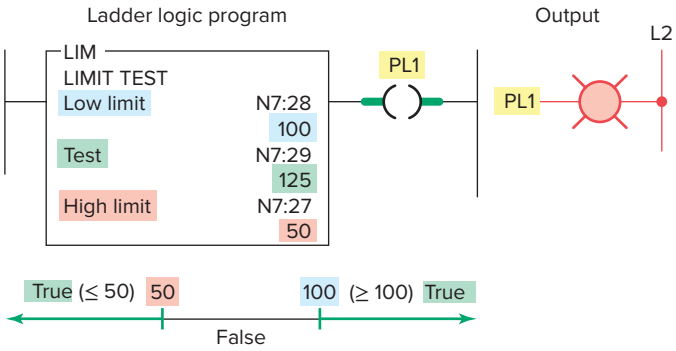


Figure 10-29 LIM instruction where the low limit value is greater than the high limit value.

- Instruction is false for test values greater than 50 and less than 100.
- Instruction is true because the test value is 125.

The program of Figure 10-30 shows a practical example of the ControlLogix program with the Limit Test (LIM) instruction. In this program, the LIM instruction will energize the Count_Within_Range output when the counter is within the range of 6 to 12 counts. Note that the range includes the values set as the low and high limits.

The *masked comparison for equal (MEQ)* instruction compares a value from a source address with data at a compare address and allows portions of the data to be masked. One application for the MEQ instruction is to compare the correct position of up to 16 limit switches when the source contains the limit switch address and the compare stores their desired states. The mask can block out the switches you don't want to compare (Figure 10-31).

Figure 10-31 MEQ instruction can be used to monitor the state of limit switches.
Source: Courtesy Jayashree Electrodevices.

Figure 10-32 shows an example of an MEQ instruction. The operation of the logic rung can be summarized as follows:

- When the data at the source address match the data at the compare address bit-by-bit (less masked bits), the instruction is true.
- The instruction goes false as soon as it detects a mismatch.
- A mask passes data when the mask bits are set (1); a mask blocks data when the mask bits are reset (0).

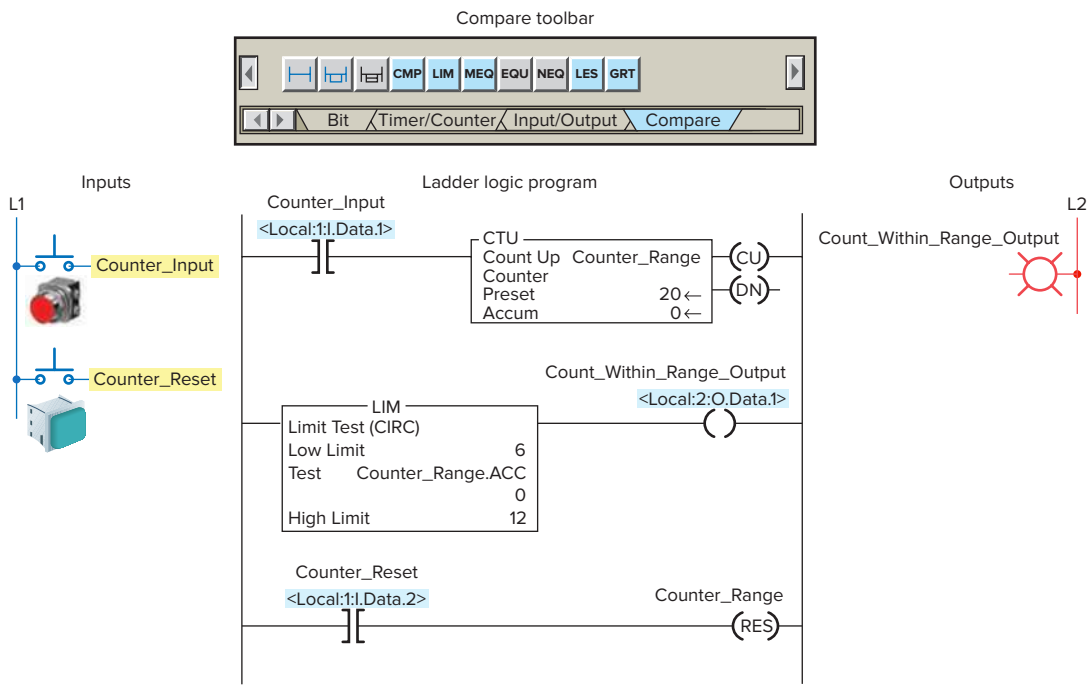


Figure 10-30 ControlLogix program with the Limit Test (LIM) instruction.

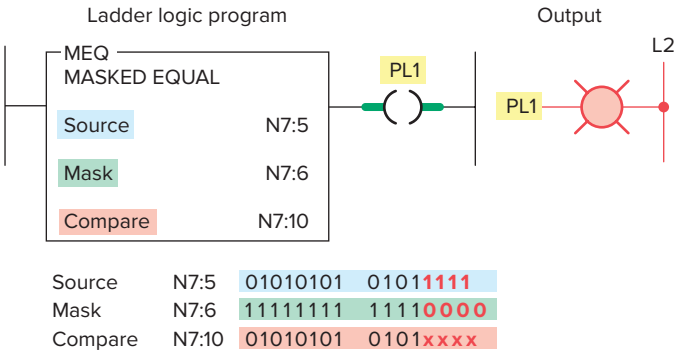


Figure 10-32 Masked comparison for equal (MEQ) logic rung.

- The mask must be the same element size (16 bits) as the source and compare addresses.
- You must set mask bits to 1 to compare data. Bits in the compare address that correspond to 0s in the mask are not compared.
- If you want the ladder program to change mask value, store the mask at a data address. Otherwise, enter a hexadecimal value for a constant mask value.

10.4 Data Manipulation Programs

Data manipulation instructions give new dimension and flexibility to the programming of control circuits. For example, consider the hardwired relay-operated, time-delay circuit in Figure 10-33. This circuit uses three electromechanical time-delay relays to control four solenoid valves.

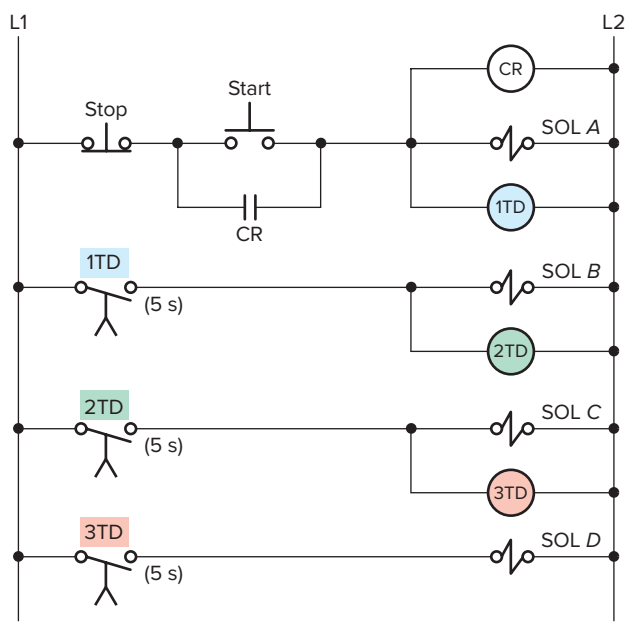


Figure 10-33 Three electromechanical time-delay relays used to control four solenoid valves.

The operation of the hardwired circuit can be summarized as follows:

- When the momentary start pushbutton is pressed solenoid *A* is energized immediately.
- Solenoid *B* is energized 5 s later than solenoid *A*.
- Solenoid *C* is energized 10 s later than solenoid *A*.
- Solenoid *D* is energized 15 s later than solenoid *A*.

The hardwired time-delay circuit could be implemented using a conventional PLC program and three internal timers. However, the same circuit can be programmed using only *one* internal timer along with data compare instructions. Figure 10-34 shows the program required to implement the circuit using only one internal timer. The operation of the program can be summarized as follows:

- The momentary stop button is closed.
- When the momentary start button is pressed, SOL *A* output energizes immediately to switch on solenoid *A*.
- SOL *A* examine-on contact becomes true to seal in output SOL *A* and to start on-delay timer T4:1 timing.
- The timer preset time is set to 15 seconds.
- Output SOL *B* will energize after a total time delay of 5 seconds, when the accumulated time becomes equal to and then greater than 5 seconds. This, in turn, will energize solenoid *B*.
- Output SOL *C* will energize after a total time delay of 10 seconds, when the accumulated time becomes equal to and then greater than 10 seconds. This, in turn, will energize solenoid *C*.
- Output SOL *D* will energize (through the timer done bit T4:1/DN) after a total time delay of 15 seconds to energize solenoid *D*.

Figure 10-35 shows an application of an on-delay timer program implemented using the EQU instruction. The operation of the program can be summarized as follows:

- When the switch (S1) is closed, timer T4:1 will begin timing.
- Both EQU instructions' source *A*s are addressed to get the accumulated value from the timer while it is running.
- The EQU instruction of rung 2 has the value of 5 stored in source *B*.
- When the accumulated value of the timer reaches 5, the EQU instruction of rung 2 will become logic true for 1 second.
- As a result, the latch output will energize to switch the pilot light PL1 on.

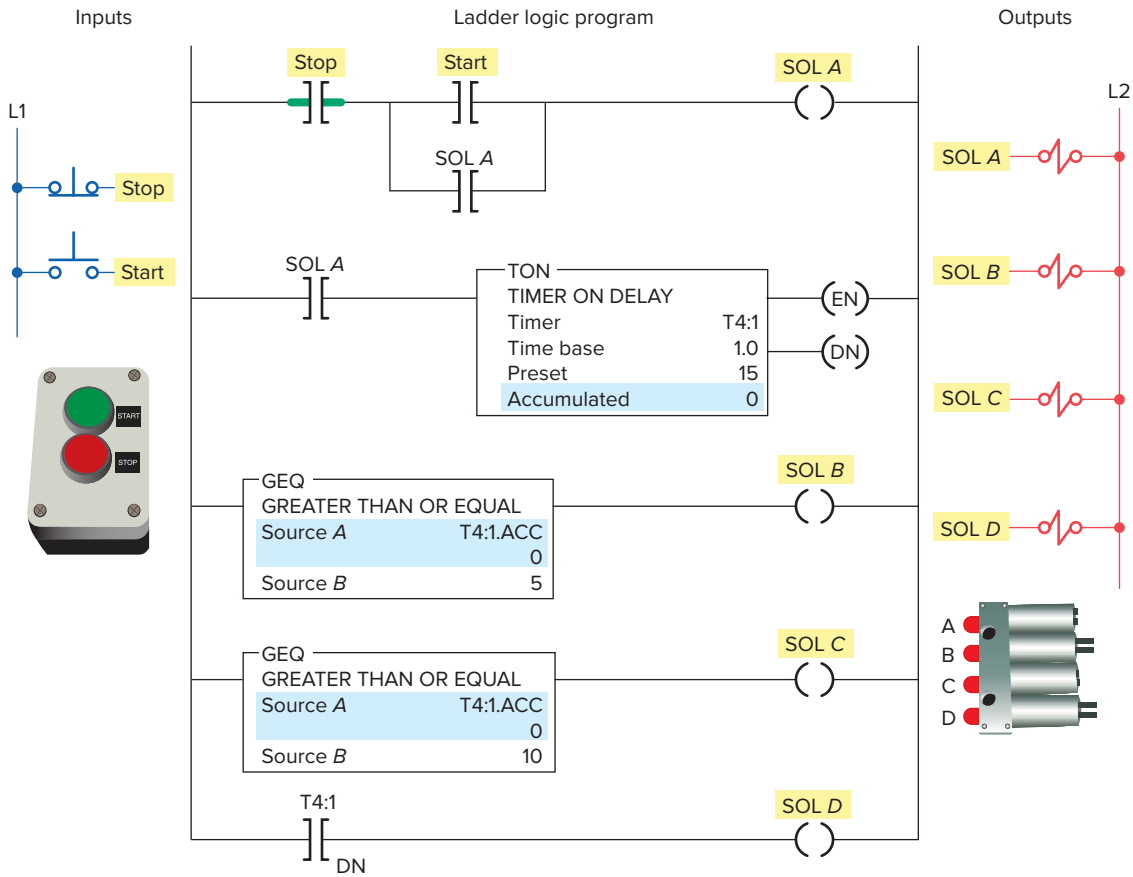


Figure 10-34 Controlling multiple loads using one timer and the GEQ instruction.

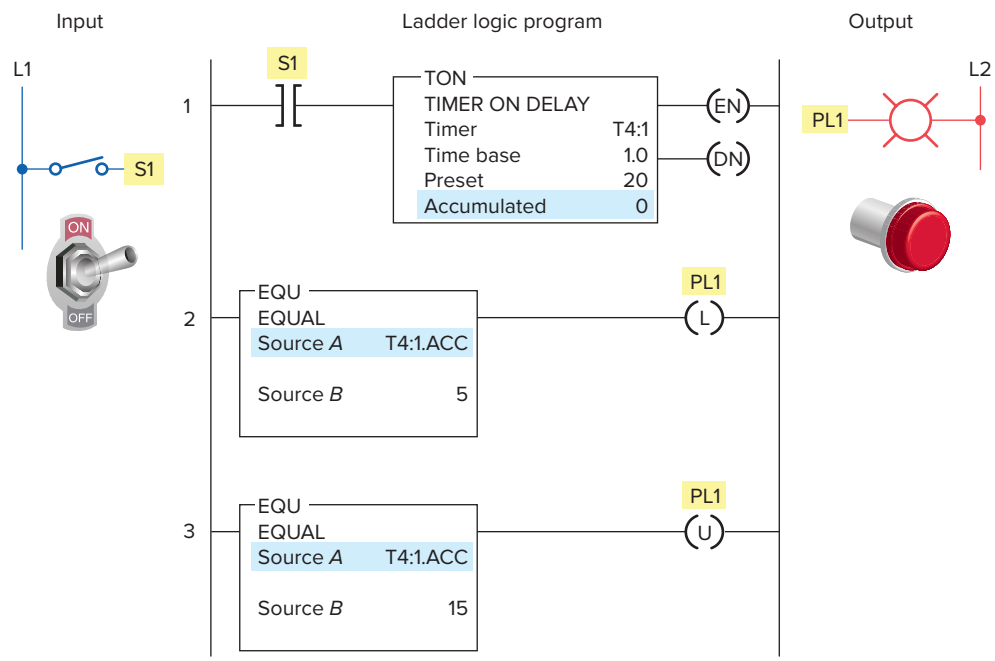


Figure 10-35 Timer program implemented using the EQU instruction.

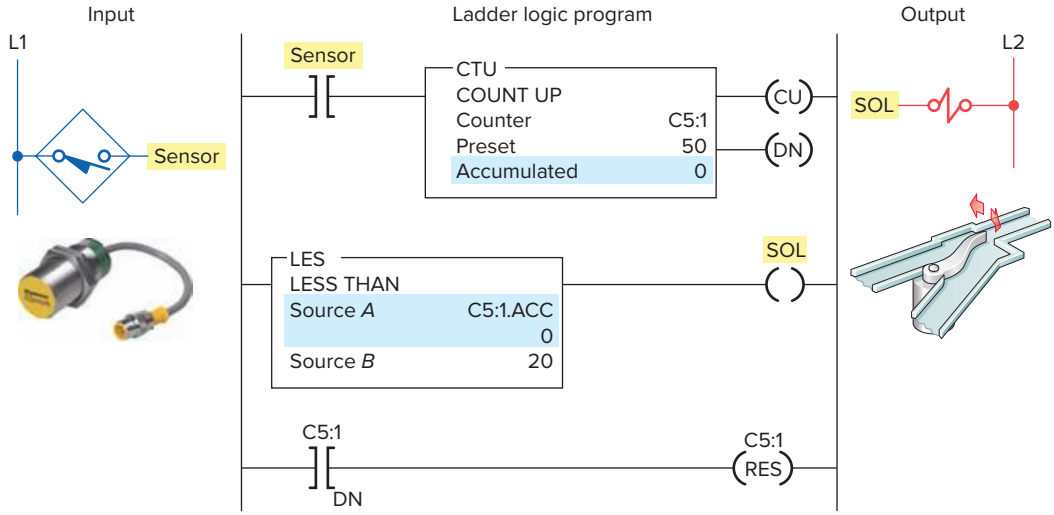


Figure 10-36 Counter program implemented using the LES instruction.
Source: Photo courtesy Turck, Inc., **www.turck.com**.

- When the accumulated value of the timer reaches 15, the EQU instruction of rung 3 will be true for 1 second.
- As a result, the unlatch output will energize to switch the pilot light PL1 off.
- Therefore, when the switch is closed, the pilot light will come on after 5 seconds, stay on for 10 seconds, and then turn off.

Figure 10-36 shows an application of an up-counter program implemented using the LES instruction. The operation of the program can be summarized as follows:

- Up-counter C5:1 will increment by 1 for every false-to-true transition of the proximity sensor switch.
- Source *A* of the LES instruction is addressed to the accumulated value of the counter and source *B* has a constant value of 20.
- The LES instruction will be true as the long as the value contained in source *A* is *less than* that of source *B*.
- Therefore, output solenoid SOL will be energized when the accumulated value of the counter is between 0 and 19.
- When the counter's accumulated value reaches 20, the LES instruction will go false, de-energizing output solenoid SOL.
- When the counter's accumulated value reaches its preset value of 50, the counter reset will be energized through the counter done bit (C5:1/DN) to reset the accumulated count to 0.

The use of comparison instructions is generally straightforward. However, one precaution involves the use of these instructions in PLC programs used to control

Figure 10-37 Vessel filling operation.
Source: Courtesy Feige Filling.

the flow in vessel filling operations (Figure 10-37). This control scenario can be summarized as follows:

- The receiving vessel has its weight monitored continuously by the PLC program as it fills.

- When the weight reaches a preset value, the flow is cut off.
- While the vessel fills, the PLC performs a comparison between the vessel's current weight and a predetermined constant programmed in the processor.
- If the programmer uses only the equal instruction, problems may result.
- As the vessel fills, the comparison for equality will be false. At the instant the vessel weight reaches the desired preset value of the equal instruction, the instruction becomes true and the flow is stopped.
- However, should the supply system leak additional material into the vessel, the total weight of the material could rise *above* the preset value, causing the instruction to go false and the vessel to overfill.
- The simplest solution to this problem is to program the comparison instruction as a greater than or equal to instruction. This way, any excess material entering the vessel will not affect the filling operation.
- It may be necessary, however, to include additional programming to indicate a serious overfill condition.

10.5 Numerical Data I/O Interfaces

The expanding data manipulation processing capabilities of PLCs led to the development of I/O interfaces known as numerical data I/O interfaces. In general, numerical data I/O interfaces can be divided into two groups: those that provide interface to *multibit digital* devices and those that provide interface to *analog* devices.

The multibit digital devices are like the discrete I/O because processed signals are discrete (on/off). The difference is that, with the discrete I/O, only a *single* bit is required to read an input or control an output. Multibit interfaces allow a group of bits to be input or output *as a unit.* They can be used to accommodate devices that require BCD inputs or outputs.

The *thumbwheel switches (TWS),* shown in Figure 10-38, are typical BCD input devices. Each one of the four switches provides four binary digits at its output that correspond to the decimal number selected on the switch. The conversion from a single decimal digit to four binary digits is performed by the TWS device. The BCD input module allows the processor to accept the 4-bit digital codes and input their data into specific register or word locations in memory to be used by the control program. Data manipulation instructions can be used to access the data from the input module allowing a person to change set points, timer, or counter presets *externally* without modifying the control program.

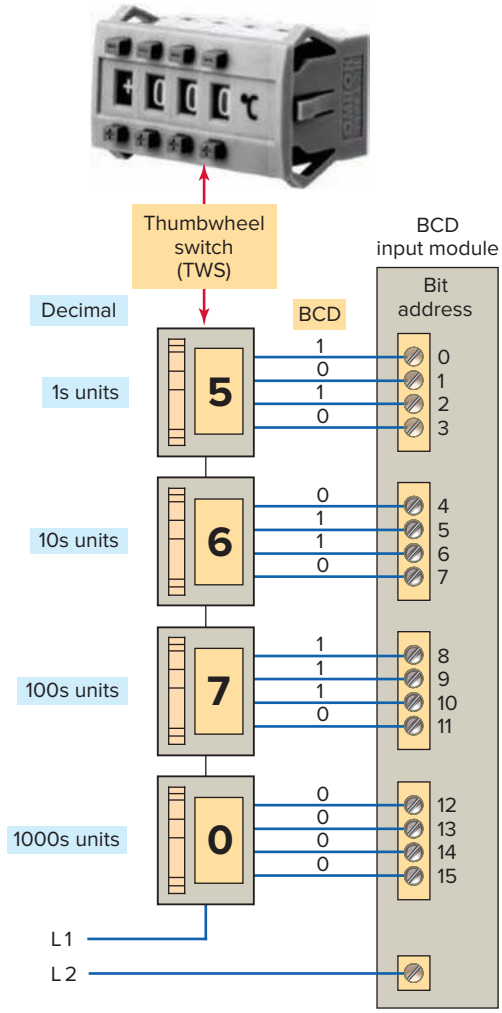


Figure 10-38 BCD input interface module connected to a thumbwheel switch.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

The *seven-segment LED* display board, shown in Figure 10-39, is a typical Binary Coded Decimal (BCD) output device. It displays a decimal number that corresponds to the BCD value it receives at its input. Conversion of the four binary bits to a single decimal digit on the display is performed by the LED display device. The BCD output module is used to output data from a specific register or word location in memory. This type of output module enables a PLC to operate devices that require BCD coded signals.

Figure 10-40 shows a PLC program that uses a BCD input interface module connected to a thumbwheel switch and a BCD output interface module connected to an LED display board. The program is designed so that the LEDs display the setting of the thumbwheel switch. Both the MOV and EQU instructions form part of the program. The operation of the program can be summarized as follows:

- The LED display board monitors the decimal setting of the thumbwheel switch.

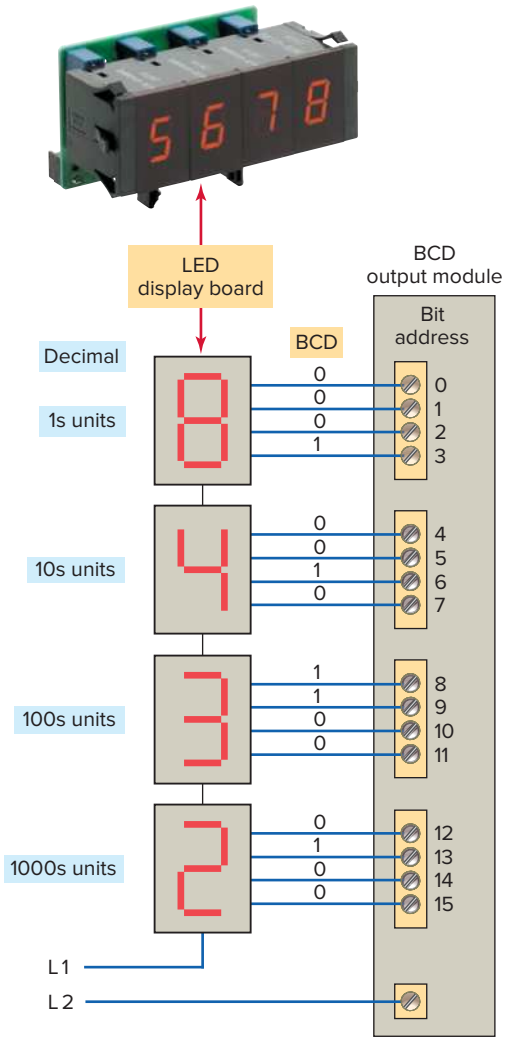


Figure 10-39 BCD output interface module connected to a seven-segment LED display board.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

- The MOV instruction is used to move the data from the thumbwheel switch input to the LED display output.
- Setting of the thumbwheel switch is compared to the reference number 1208 stored in source *B* by the EQU instruction.
- Pilot light output PL is energized whenever the input switch S1 is true (closed) and the value of the thumbwheel switch is equal to 1208.

Input and output modules can be addressed either at the bit level or at the word level. Analog modules convert analog signals to 16-bit digital signals (input) or 16-bit digital signals to analog values (output). An analog I/O will allow monitoring and control of analog voltages and currents. Figure 10-41 illustrates how an analog input interface operates. The operation of this input module can be summarized as follows:

- The analog input module contains the circuitry necessary to accept analog voltage or current signals from field devices.
- The input signal is converted from an analog to a digital value by an analog-to-digital (A/D) converter circuit.
- The conversion value, which is proportional to the analog signal, is passed through the controller's data bus and stored in a specific register or word location in memory for later use by the control program.

An analog output interface module receives numerical data from the processor; these data are then translated into a proportional voltage or current to control an analog field

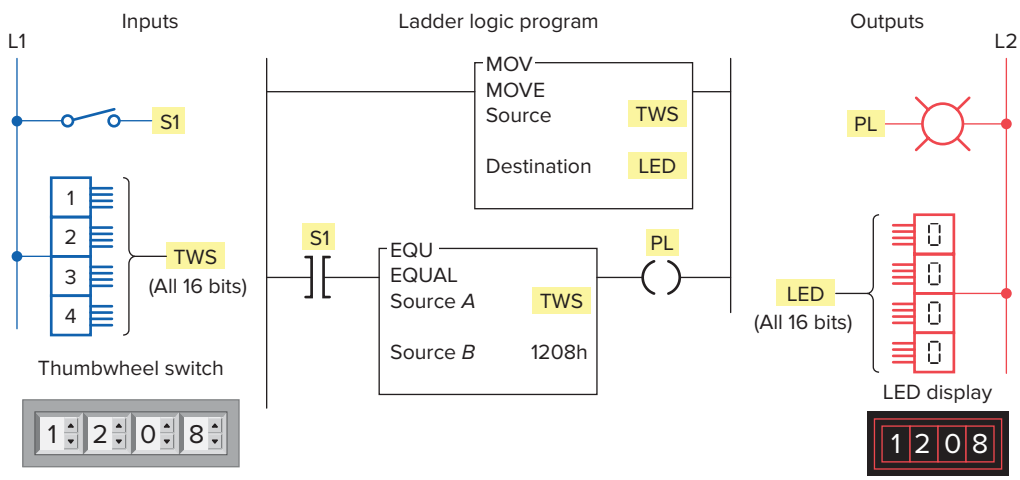


Figure 10-40 Monitoring the setting of a thumbwheel switch.

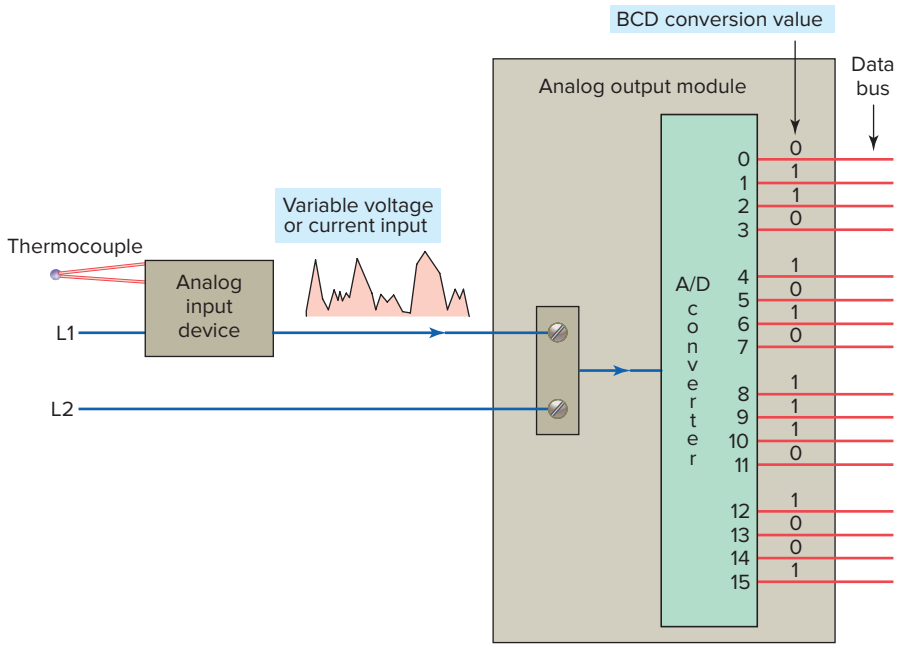


Figure 10-41 Analog input interface module.

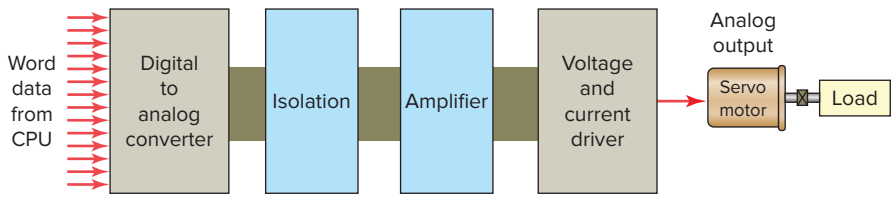


Figure 10-42 Analog output interface module.

device. Figure 10-42 illustrates how an analog output interface operates. The operation of this output module can be summarized as follows:

- The function of the analog output module is to accept a range of numeric values output from the PLC program and to produce a varying current or voltage signal required to control a connected analog output device.
- Data from a specific register or word location in the CPU memory are passed through the controller's data bus to the digital-to-analog (D/A) converter.
- The analog output from the D/A converter is then used to control the analog output device.
- The level of the analog signal output is based on the digital value of the data word supplied by the CPU and manipulated by the control program.
- These output interfaces normally require an external power supply that meets certain current and voltage requirements.

10.6 Closed-Loop Control

In open-loop control, no feedback loop is employed and system variations which cause the output to deviate from the desired value are not detected or corrected. A closed-loop system utilizes feedback to measure the actual system operating parameter being controlled such as temperature, pressure, flow, level, or speed. This feedback signal is sent back to the PLC where it is compared with the desired system set-point. The controller develops an error signal that initiates corrective action and drives the final output device to the desired value.

PLC *set-point control* in its simplest form compares an input value, such as analog or thumbwheel inputs, to a set-point value. A discrete output signal is provided if the input value is less than, equal to, or greater than the set-point value. The temperature control program of Figure 10-43 is one example of set-point control. In this application, a PLC is to provide for simple off/on control of the electric heating elements of an oven.

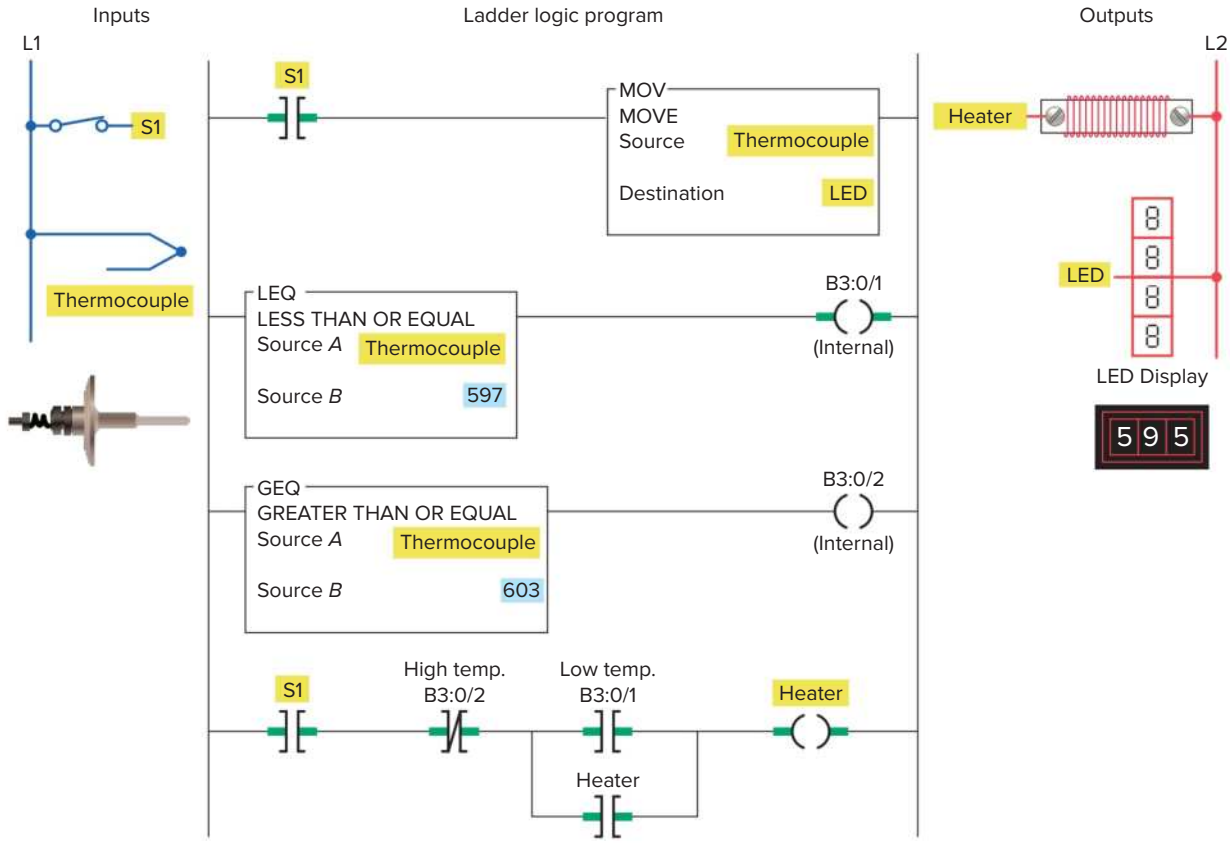


Figure 10-43 Set-point control program.

The operation of the program can be summarized as follows:

- Oven is to maintain an average set-point temperature of 600°F with a variation of about 1 percent between the off and on cycles.
- The electric heaters are turned on when the temperature of the oven is 597°F or less and will stay on until the temperature rises to 603°F or more.
- The electric heaters stay off until the temperature drops to 597°F, at which time the cycle repeats itself.
- Whenever the less than or equal (LEQ) instruction is true, a low-temperature condition exists and the program switches on the heater.
- Whenever the greater than or equal (GEQ) instruction is true, a high-temperature condition exists and the program switches off the heater.
- For the program as shown the temperature is 595°F so the LEQ instruction and B3:0/1 will both be true and the heater output will be switched on and sealed-in through the heater examine-on instruction.
- Once the temperature increases to 598°F the LEQ instruction goes false but the heater output remains on until the temperature rises to 603°F.
- At the 603°F point the GEQ instruction and B3:0/2 will both be true and the heater will be switched off.

Several set-point control schemes can be performed by different PLC models. These include on/off control, proportional (P) control, proportional-integral (PI) control, and proportional-integral-derivative (PID) control. Each involves the use of some form of closed-loop control to maintain a process characteristic, such as a temperature, pressure, flow, or level, at a desired value. When a control system is designed such that it receives operating information from the machine and makes adjustments to the machine based on this operating information, the system is said to be a closed-loop system.

The block diagram of a closed-loop control system is shown in Figure 10-44. A measurement is made of the variable to be controlled. This measurement is then compared to a reference point, or set-point. If a difference (error) exists between the actual and desired levels, the PLC control program will take the necessary corrective

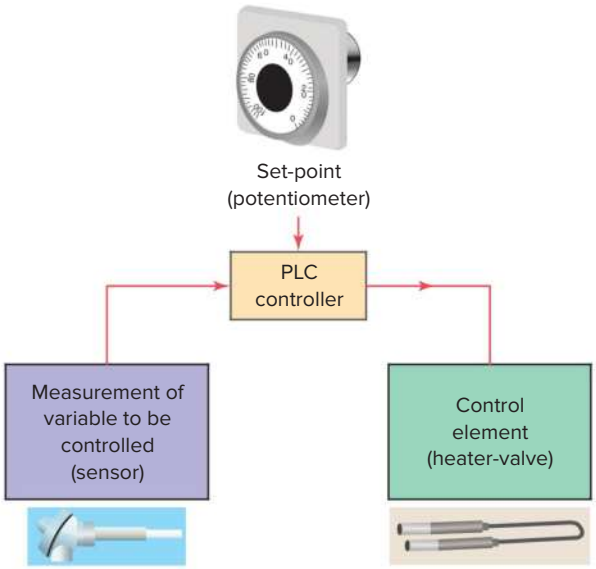


Figure 10-44 Closed-loop control system.

action. Adjustments are made continuously by the PLC until the difference between the desired and actual output is as small as is practical.

With on/off PLC control (also known as *two-position* and *bang-bang control*), the output or final control element is either on or off—one for the occasion when the value of the measured variable is above the set-point and the other for the occasion when the value is below the set-point. The controller will never keep the final control element in an intermediate position. Most residential thermostats are on/off type controllers.

On/off control is inexpensive but not accurate enough for most process and machine control applications. On/off control almost always means overshoot and resultant system cycling. For this reason a *deadband* usually exists around the set-point. The deadband or hysteresis of the control loop is the difference between the on and off operating points.

Proportional controls are designed to eliminate the hunting or cycling associated with on/off control. They allow the final control element to take intermediate positions between on and off. This permits *analog control* of the final control element to vary the amount of energy to the process, depending on how much the value of the measured variable has shifted from the desired value.

The process illustrated in Figure 10-45 is an example of a proportional control process. The PLC analog output module controls the amount of fluid placed in the holding tank by adjusting the percentage of valve opening. The valve is initially open 100 percent. As the fluid level in the tank approaches the preset point, the processor modifies

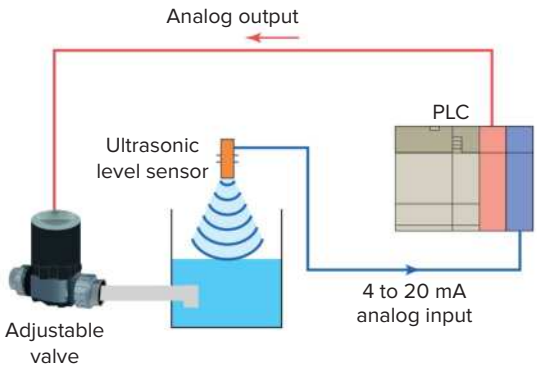


Figure 10-45 Proportional control process.

the output to degrade closing the valve by different percentages, adjusting the valve to maintain a set-point.

Proportional-integral-derivative (PID) control is the most sophisticated and widely used type of process control. PID operations are more complex and are mathematically based. PID controllers produce outputs that depend on the *magnitude, duration,* and *rate of change* of the system error signal. Sudden system disturbances are met with an aggressive attempt to correct the condition. A PID controller can reduce the system error to 0 faster than any other controller.

A typical PID control loop is illustrated in Figure 10-46. The loop measures the process, compares it to a set-point, and then manipulates the output in the direction which should move the process toward the set-point. The terminology used in conjunction with a PID loop can be summarized as follows:

- Operating information that the controller receives from the machine is called the *process variable (PV)* or *feedback.*
- Input from the operator that tells the controller the desired operating point is called the *set-point (SP).*
- When operating, the controller determines whether the machine needs adjustment by comparing (by subtraction) the set-point and the process variable

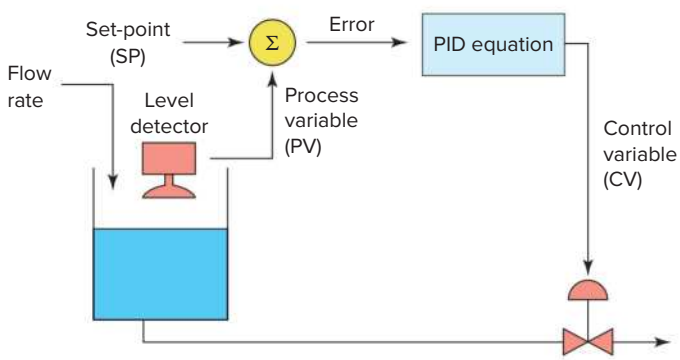


Figure 10-46 Typical PID control loop.

to produce a difference (the difference is called the *error*).

- Output from the loop is called the *control variable (CV),* which is connected to the controlling part of the process.
- The PID loop takes appropriate action to modify the process operating point until the control variable and the set-point are very nearly equal.

Programmable controllers are either equipped with PID I/O modules that produce PID control or have sufficient mathematical functions of their own to allow PID control to be carried out. Figure 10-47 shows an SLC 500 PID instruction with typical addresses for the parameters entered. The PID instruction normally controls a closed loop using inputs from an analog input module and provides an output to an analog output module. Explanation

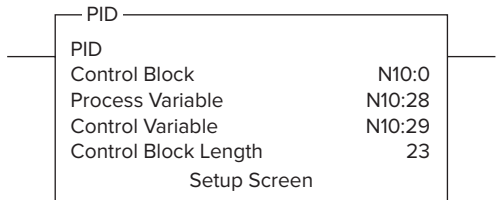


Figure 10-47 SLC 500 PID instruction.

of the PID instruction parameters can be summarized as follows:

- Control Block is the file that stores the data required to operate the instruction.
- Process Variable (PV) is an element address that stores the process input value.
- Control Variable (CV) is an element address that stores the output of the PID instruction.

CHAPTER 10 REVIEW QUESTIONS

1. In general, what do data manipulation instructions allow the PLC to do?
2. Explain the difference between a register or word and a table or file.
3. Into what two broad categories can data manipulation instructions be placed?
4. What takes place with regard to a data transfer instruction?
5. The MOV instruction is to be used to copy the information stored in word N7:20 to N7:35. What address is entered into the source and the destination?
6. What is the purpose of the mask word in the MVM instruction?
7. List three types of data shifts used with file instructions.
8. List the six parameters and addresses that must be entered into the file arithmetic and logic (FAL) instruction.
9. Assume the ALL mode has been entered as part of a FAL instruction. How will this affect the transfer of data?
10. What is the advantage of using the file copy (COP) or fill file (FLL) instruction rather than the FAL instruction for the transfer of data?
11. What are data compare instructions used for?
12. Name and draw the symbols for the six different types of data compare instructions.
13. Explain what each of the logic rungs in Figure 10-48 is instructing the processor to do.
14. What does the limit test (LIM) instruction test values for?
15. How are multibit I/O interfaces different from the discrete type?
16. Assume that a thumbwheel switch is set for the decimal number 3286.
    a. What is the equivalent BCD value for this setting?
    b. What is the equivalent binary value for this setting?
17. Assume that a thermocouple is connected to an analog input module. Explain how the temperature of the thermocouple is communicated to the processor.

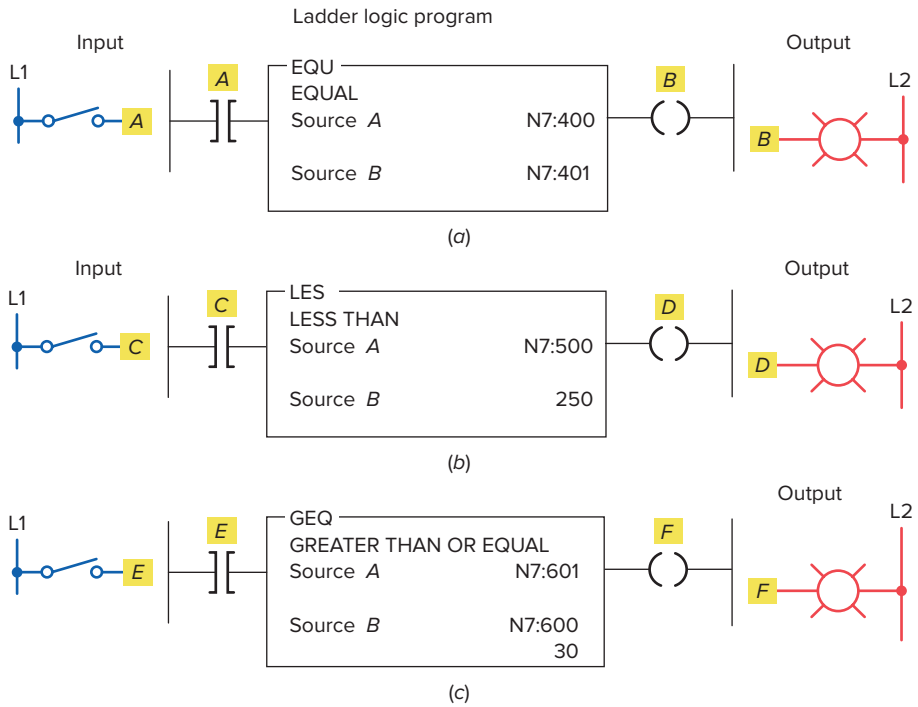


Figure 10-48 Logic rungs for Question 13.

18. Outline the process by which an analog output interface module operates the field device connected to it.
19. Compare the operation of open-loop and closed-loop PLC systems.
20. Outline the control process involved with simple PLC set-point control.
21. Compare the operation of the final control element in on/off and proportional control systems.
22. Explain the meaning of the following terms as they apply to a PID control:
    a. Process variable
    b. Set-point
    c. Error
    d. Control variable

CHAPTER 10 PROBLEMS

1. Study the data transfer program of Figure 10-49 and answer the following questions:
   a. When S1 is open, what decimal number will be stored in integer word address N7:13 of the MOV instruction?
   b. When S1 is on, what decimal number will be stored in integer word address N7:112 of the MOV instruction?
   c. When S1 is on, what decimal number will appear in the LED display?
   d. What is required for the decimal number 216 to appear in the LED display?
2. Study the data transfer counter program of Figure 10-50 and answer the following questions:
   a. What determines the preset value of the counter?
   b. Outline the steps to follow to operate the program so that the PL1 output is energized after 25 off-to-on transitions of the count PB input.
3. Construct a nonretentive timer program that will turn on a pilot light after a time-delay period. Use a thumbwheel switch to vary the preset time-delay value of the timer.
4. Study the data compare program of Figure 10-51 and answer the following questions:
   a. Will the pilot light PL1 come on whenever switch S1 is closed? Why?
   b. Must switch S1 be closed to change the number stored in source *A* of the EQU instruction?
   c. What number or numbers need to be set on the thumbwheel in order to turn on the pilot light?

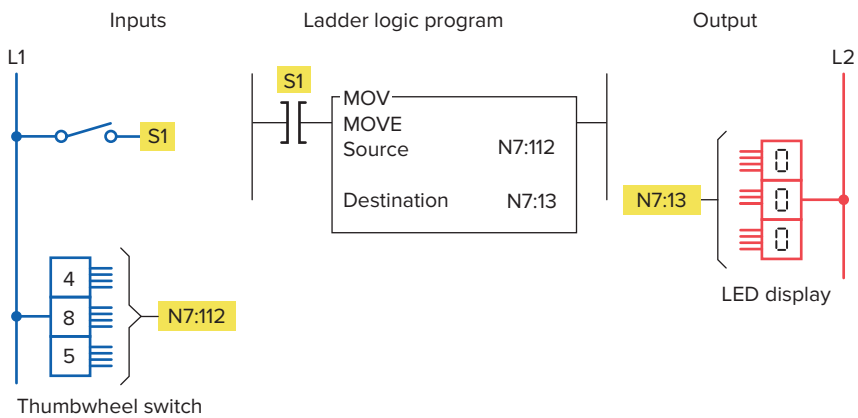


Figure 10-49 Program for Problem 1.

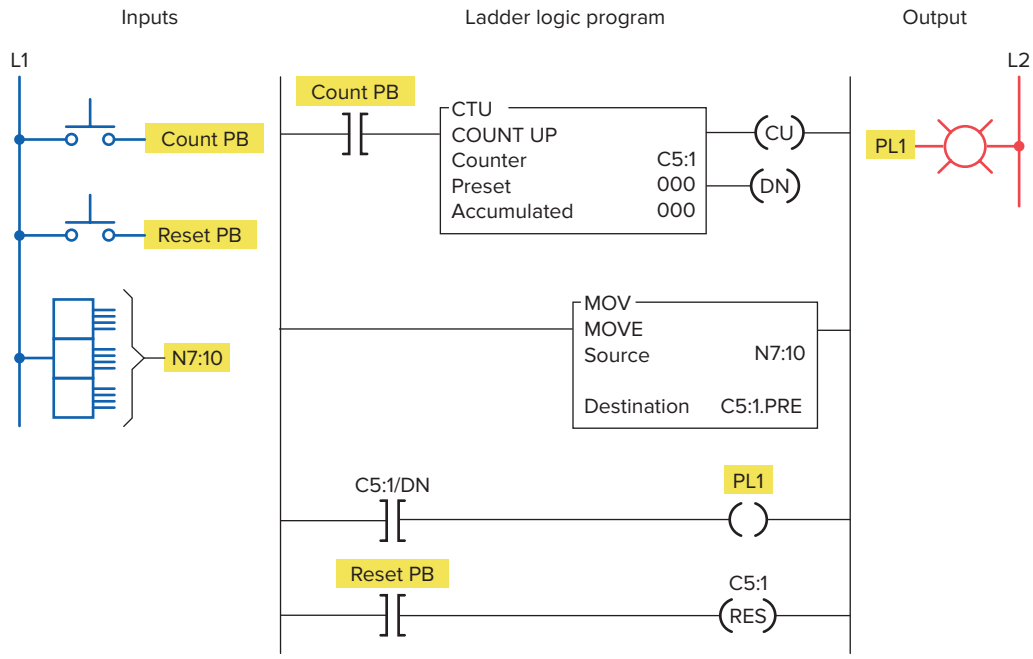


Figure 10-50 Program for Problem 2.

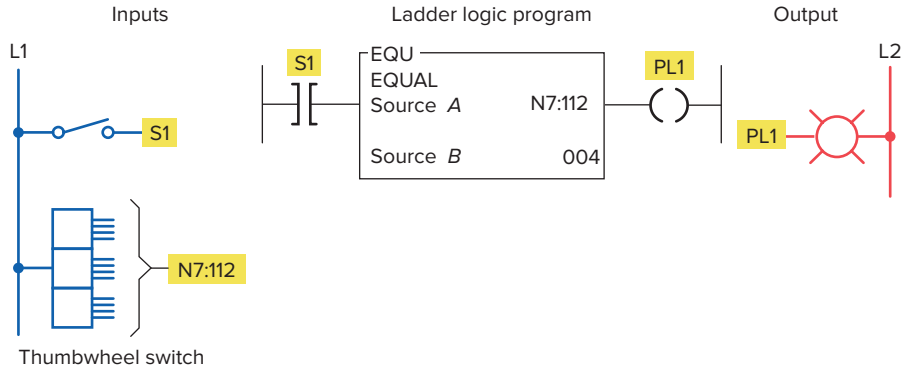


Figure 10-51 Program for Problem 4.

5. Study the data compare program in Figure 10-52 and answer the following questions:
- **a.** List the values for the thumbwheel switch that would allow the pilot light to turn on.
- **b.** If the value in the word N7:112 is 003 and switch S1 is open, will the pilot light turn on? Why?
- **c.** Assume that source *B* is addressed to the accumulated count of an up-counter. With S1 closed, what setting of the thumbwheel switch would be required to turn the pilot light off when the accumulated count reaches 150?

6. Write a program to perform the following:
- **a.** Turn on pilot light 1 (PL1) if the thumbwheel switch value is less than 4.
- **b.** Turn on pilot light 2 (PL2) if the thumbwheel switch value is equal to 4.
- **c.** Turn on pilot light 3 (PL3) if the thumbwheel switch value is greater than 4.
- **d.** Turn on pilot light 4 (PL4) if the thumbwheel switch value is less than or equal to 4.
- **e.** Turn on pilot light 5 (PL5) if the thumbwheel switch value is greater than or equal to 4.

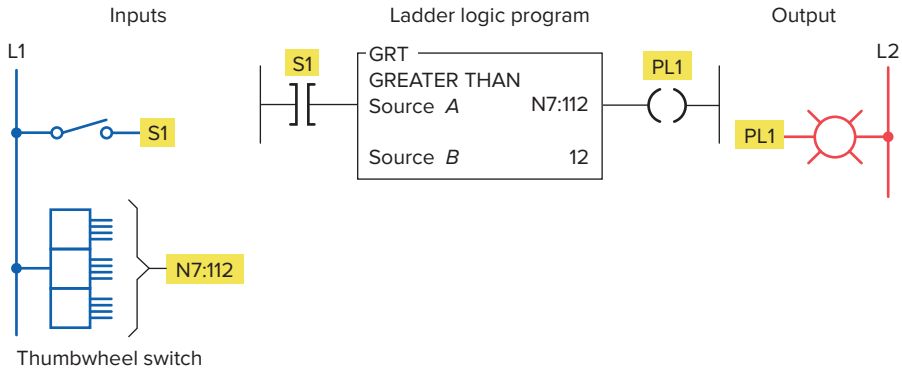
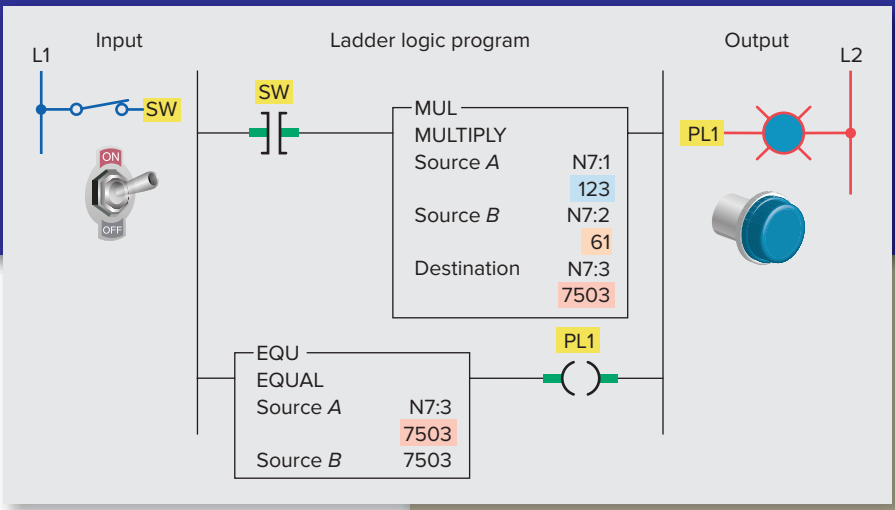


Figure 10-52 Program for Problem 5.

7. Write a program that will copy the value stored at address N7:56 into address N7:60.
8. Write a program that uses the mask move instruction to move only the upper 8 bits of the value stored at address I:2.0 to address O:2.1 and to ignore the lower 8 bits.
9. Write a program that uses the FAL instruction to copy 20 words of data from the integer data file, starting with N7:40, into the integer data file, starting with N7:80.
10. Write a program that uses the COP instruction to copy 128 bits of data from the memory area, starting at B3:0, to the memory area, starting at B3:8.
11. Write a program that will cause a light to come on only if a PLC counter has a value of 6 or 10.
12. Write a program that will cause a light to come on if a PLC counter value is less than 10 or more than 30.
13. Write a program for the following: The temperature reading from a thermocouple is to be read and stored in a memory location every 5 minutes for 4 hours. The temperature reading is brought in continuously and stored in address N7:150. File #7:200 is to contain the data from the last full 4-hour period.

11

Math Instructions



Most PLCs have arithmetic function capabilities. Basic PLC math instructions include add, subtract, multiply, and divide to calculate the sum, difference, product, and quotient of the content of word registers. The PLC is capable of doing many arithmetic operations per scan period for fast updating of data. This chapter covers the basic mathematical instructions performed by PLCs and their applications.

Chapter Objectives

After completing this chapter, you will be able to:

- Analyze and interpret math instructions as they apply to a PLC program
- Create PLC programs involving math instructions
- Apply combinations of PLC arithmetic functions to processes

11.1 Math Instructions

Math instructions, like data manipulation instructions, enable the programmable controller to take on more of the qualities of a conventional computer. The PLC's math functions capability allows it to perform arithmetic functions on values stored in memory words or registers. For example, assume you are using a counter to keep track of the number of parts manufactured, and you would like to display how many more parts must be produced in order to reach a certain quota. This display would require the data in the accumulated value of the counter to be subtracted from the quota required. Other applications include combining parts counted, subtracting detected defects, and calculating run rates.

Depending on what type of processor is used, various math instructions can be programmed. The basic four mathematical functions performed by PLCs are:

Addition—The capability to add one piece of data to another.

Subtraction—The capability to subtract one piece of data from another.

Multiplication—The capability to multiply one piece of data by another.

Division—The capability to divide one piece of data by another.

Math instructions use the contents of two words or registers and perform the desired function. The PLC instructions for data manipulation (data transfer and data compare) are used with the math symbols to perform math functions. Math instructions are all output instructions. These instructions can be conditional or unconditional. With conditional arithmetic instructions the input logic determines when the instruction executes. Unconditional arithmetic instructions execute with each scan.

Figure 11-1 shows the Compute/Math menu tab for the SLC 500 PLC and its associated RSLogix software. The commands can be summarized as follows:

CPT (Compute)—Evaluates an expression and stores the result in the destination.

ADD (Add)—Adds source *A* to source *B* and stores the result in the destination.

SUB (Subtract)—Subtracts source *B* from source *A* and stores the result in the destination.

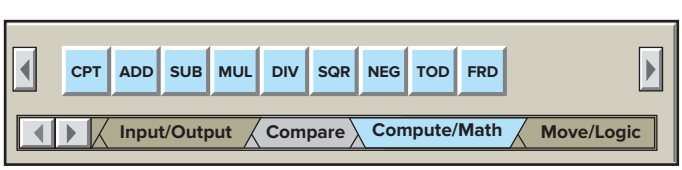


Figure 11-1 Compute/Math menu tab.

MUL (Multiply)—Multiplies source *A* by source *B* and stores the result in the destination.

DIV (Divide)—Divides source *A* by source *B* and stores the result in the math register and destination.

SQR (Square Root)—Calculates the square root of the source and places the integer result in the destination.

NEG (Negate)—Changes the sign of the source and places it in the destination.

TOD (To BCD)—Converts a 16-bit integer source value to BCD and stores it in the math register or the destination.

FRD (From BCD)—Converts a BCD value in the math register or the source to an integer and stores it in the destination.

The basic math instructions are ADD, SUB, MUL, and DIV. Each of these instructions has three parameter fields. Namely, Source A, Source B and Destination fields.

- The **Source A** and **Source B** fields can be an input rack location, file address, instruction field, or a fixed value. For example:

Input Location	I:1
File Address	N7:5
Instruction Field	C5:2.ACC
Fixed Value	30

- The **Destination** fields can be an output location, file address, or an instruction field. For example:

Output location	O:2
File Address	N7:8
Instruction Field	T4:1.PRE

Figure 11-2 shows the ***CPT (compute)*** instruction used with SLC 500 controllers. When CPT instruction is executed, then copy, arithmetic, logical, or conversion operation residing in the expression field of this instruction is performed and the result is sent to the destination. The execution time of a CPT instruction is longer than that of a single arithmetic operation and uses more instruction words.

The main advantage of the compute instruction is that it allows you to enter quite complex expressions in one instruction. Figure 11-3 shows a ladder rung used to convert a Fahrenheit temperature to a Celsius temperature using a single RSLogix 5000 compute instruction. The CPT

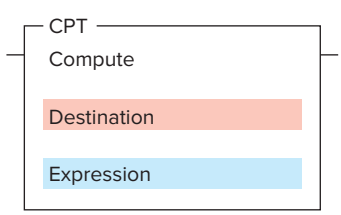


Figure 11-2 SLC 500 CPT (compute) instruction.

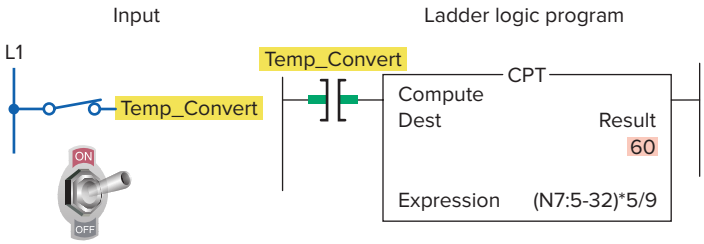


Figure 11-3 Compute instruction used to convert from Fahrenheit to Celsius.

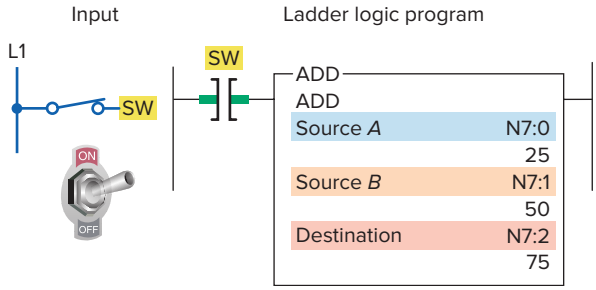


Figure 11-4 SLC 500 ADD instruction.

instruction for the SLC 500 and Logix 5000 processors operate in the same manner. This operation of the rung can be summarized as follows:

- The arithmetic operation [°C = (°F − 32) × (5/9)] is defined in the Expression.
- The compute operation is performed whenever the Temp_Convert input tag is true.
- When the CPT instruction is executed the result of the equation is put into the DEST tag name Result.
- In this example a temperature of 140°F is programmed into N7:5 of the Expression and the computed value of 60°C appears in the Result.
- The CPT has its own instruction set consisting of commands, or operators, that can be embedded in a mathematical expression. These commands range from add and subtract to BCD conversion and absolute values.
- The operations you write into the expression are performed by the instruction in a prescribed order, not necessarily the order you write them. The first order is parentheses so you can override the order of operation by grouping terms within parentheses. This forces the instruction to perform an operation within the parentheses ahead of other operations. In this case parentheses were used to assure that the subtraction was done before the multiplication and division.

11.2 Addition Instruction

Most math instructions take two input values, perform the specified arithmetic function, and output the result to an assigned memory location. For example, the ***ADD*** instruction performs the addition of two values stored in the referenced memory locations. How these values are accessed depends on the controller. Figure 11-4 shows the ADD instruction used with the SLC 500 controllers. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the rung will be true.
- The value stored at the source *A* address, N7:0 (25), is added to the value stored at the source *B* address, N7:1 (50).
- The answer (75) is stored at the destination address N7:2.
- Source *A* and source *B* can be either values or addresses that contain values, but *A* and *B* cannot both be constants.

The program of Figure 11-5 illustrates how the ADD instruction can be used to add the accumulated counts of two up-counters. This application requires a pilot light to come on when the sum of the counts from the two counters is equal to or greater than 350. The operation of the program can be summarized as follows:

- Source *A* of the ADD instruction is addressed to the accumulated value of counter C5:0.
- Source *B* of the ADD instruction is addressed to the accumulated value of counter C5:1.
- The value at source *A* is added to the value at source *B,* and the result (answer) is stored at destination address N7:1.
- Source *A* of the GEQ (greater than or equal) instruction is addressed to the value of the destination address N7:1.
- Source *B* of the GEQ instruction contains the constant value of 350.
- The GEQ instruction and PL1 output will be true whenever the accumulated sum of the values in the two counters is equal to or greater than the constant value 350.
- A reset button is provided to reset the accumulated count of both counters to zero.

When performing math functions, care must be taken to ensure that values remain in the range that the data table or file can store; otherwise, the overflow bit will be set. The arithmetic status bits for the SLC 500 controller are found in word 0, bits 0 to 3 of the processor status file S2 (Figure 11-6). After an instruction is executed, the

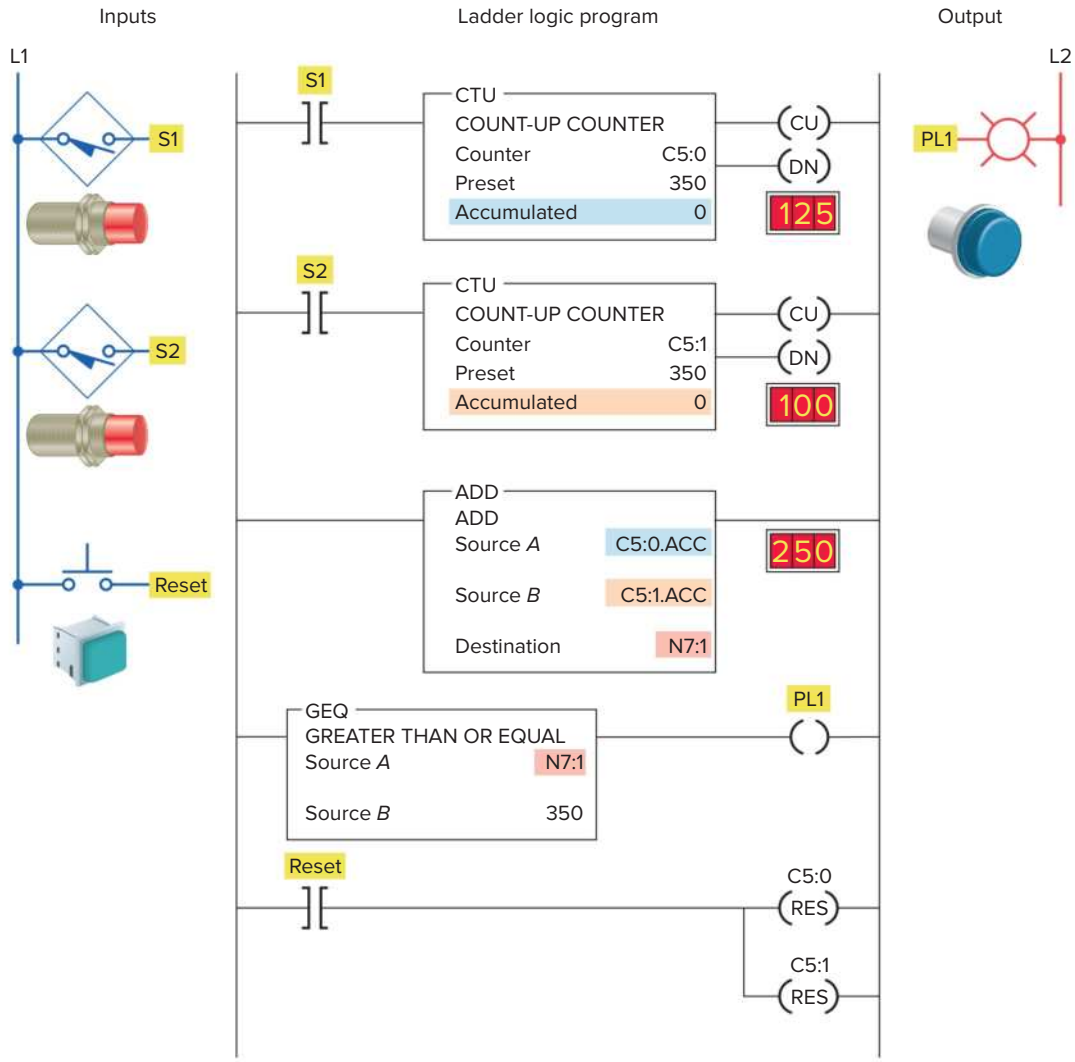


Figure 11-5 Counter program that uses the ADD instruction.

arithmetic status bits in the status file are updated. The description of each bit can be summarized as follows:

Carry (C)—Address S2:0/0, is set to 1 when there is a carry in the ADD instruction or a borrow in the SUB instruction.

Overflow (O)—Address S2:0/1, is set to 1 when the result is too large to fit in the destination register.

Zero Bit (Z)—Address S2:0/2, is set to 1 when the result of the subtract instruction is zero.

Status Table

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S2:0/	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
S2:1/	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
S2:2/	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
S2:3/	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
S2:4/	0	1	0	0	0	0	0	1	1	0	0	0	0	0	0	1
S2:5/	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Address S2:0 Table: S2:Status

Figure 11-6 Processor status file S2.

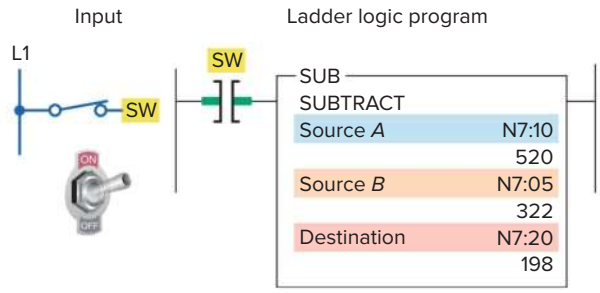


Figure 11-7 SLC 500 SUB (subtract) instruction.

Sign Bit (S)—Address S2:0/3, is set to 1 when the result is a negative number.

11.3 Subtraction Instruction

The ***SUB (subtract)*** instruction is an output instruction that subtracts one value from another and stores the result in the destination address. When rung conditions are true, the subtract instruction subtracts source *B* from source *A* and stores the result in the destination. Figure 11-7 shows the SUB instruction used with the SLC 500 controllers. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the rung will be true.
- The value stored at the source *B* address, N7:05 (322), is subtracted from the value stored at the source *A* address, N7:10 (520).
- The answer (198) is stored at the destination address, N7:20.
- Source *A* and source *B* can be either values or addresses that contain values, but *A* and *B* cannot both be constants.

The program of Figure 11-8 shows how the SUB function can be used to indicate a vessel overfill condition. This application requires an alarm to sound when a supply system leaks 5 lb or more of raw material into the vessel after a preset weight of 500 lb has been reached. The operation of the program can be summarized as follows:

- When the start button is pressed, the fill solenoid (rung 1) and filling indicating light (rung 2) are

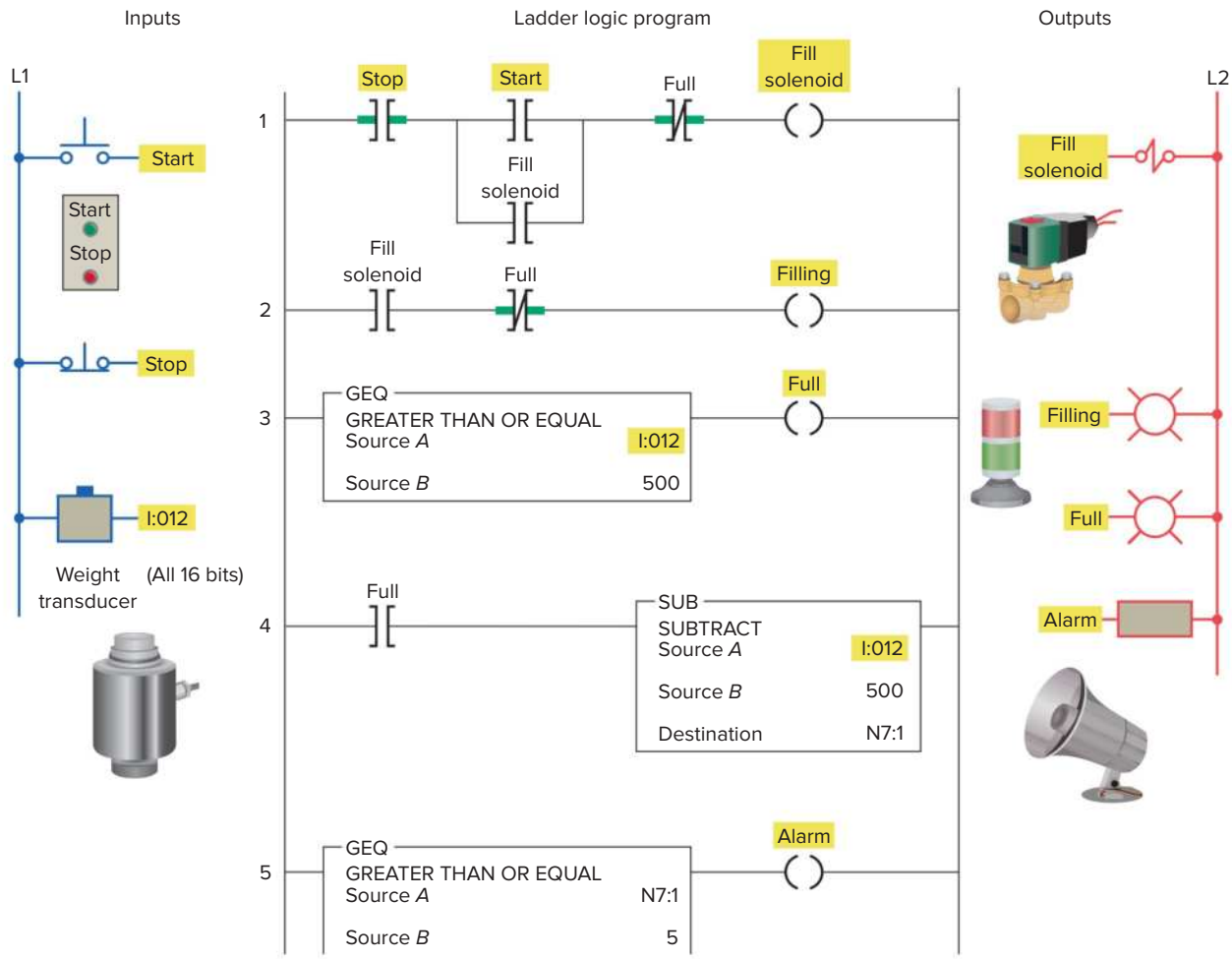


Figure 11-8 Vessel overfill alarm program.

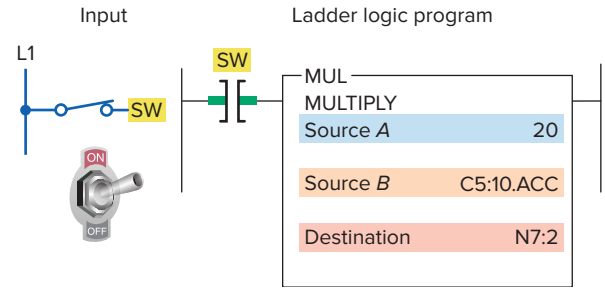


Figure 11-9 SLC 500 MUL (multiply) instruction.

turned on and raw material is allowed to flow into the vessel.

- The vessel has its weight monitored continuously by the PLC program (rung 3) as it fills.
- When the weight reaches 500 lb, the fill solenoid is de-energized and the flow is cut off.
- At the same time, the filling pilot light indicator is turned off and the full pilot light indicator (rung 3) is turned on.
- Should the fill solenoid leak 5 lb or more of raw material into the vessel, the alarm (rung 5) will energize and stay energized until the overflow level is reduced below the 5-lb overflow limit.

11.4 Multiplication Instruction

The ***multiply (MUL)*** instruction is an output instruction that multiplies two values and stores the result in the destination address. Figure 11-9 shows the MUL instruction used with the SLC 500 controllers. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the rung will be true.
- The data in source *A* (constant 20) will be multiplied by the data in source *B* (accumulated value of counter C5:10).
- The resultant answer is placed in the destination N7:2.
- Similar to previous math instructions, source *A* and *B* in multiplication instructions can be values (constants) or addresses that contain values, but *A* and *B* cannot both be constants.

The program of Figure 11-10 is an example of how MUL instruction calculates the product of two sources. The operation of the program can be summarized as follows:

- When input switch SW is closed the MUL instruction is executed.
- The value stored in source *A*, address N7:1 (123), is then multiplied by the value stored in source *B*, address N7:2 (61).
- The product (7503) is placed into destination word N7:3.
- As a result, the equal instruction becomes true, turning output PL1 on.

The program of Figure 11-11 is an example of how the MUL instruction is used as part of an oven temperature control program. The operation of the program can be summarized as follows:

- The PLC calculates the upper and lower deadband, or off/on limits, about the set-point.
- Upper and lower temperature limits are set automatically at ±1 percent regardless of the set-point value.
- Set-point temperature is adjusted by means of the thumbwheel switch.
- The analog thermocouple interface module is used to monitor the current temperature of the oven.

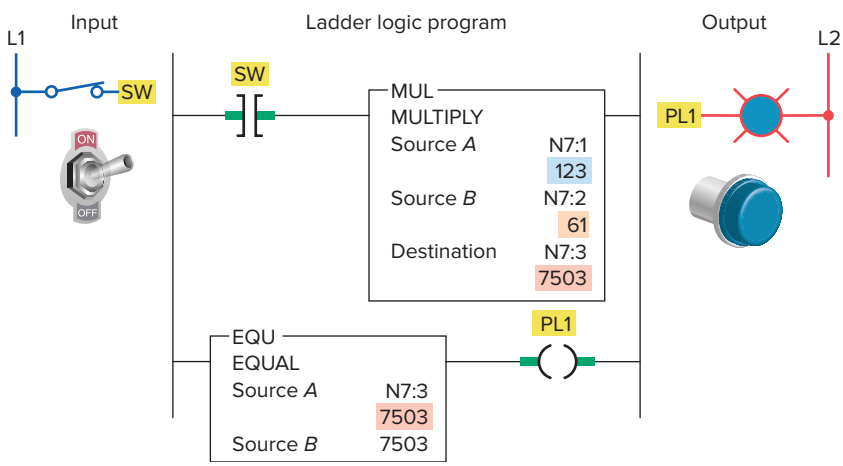


Figure 11-10 MUL instruction used to calculate the product of two sources.

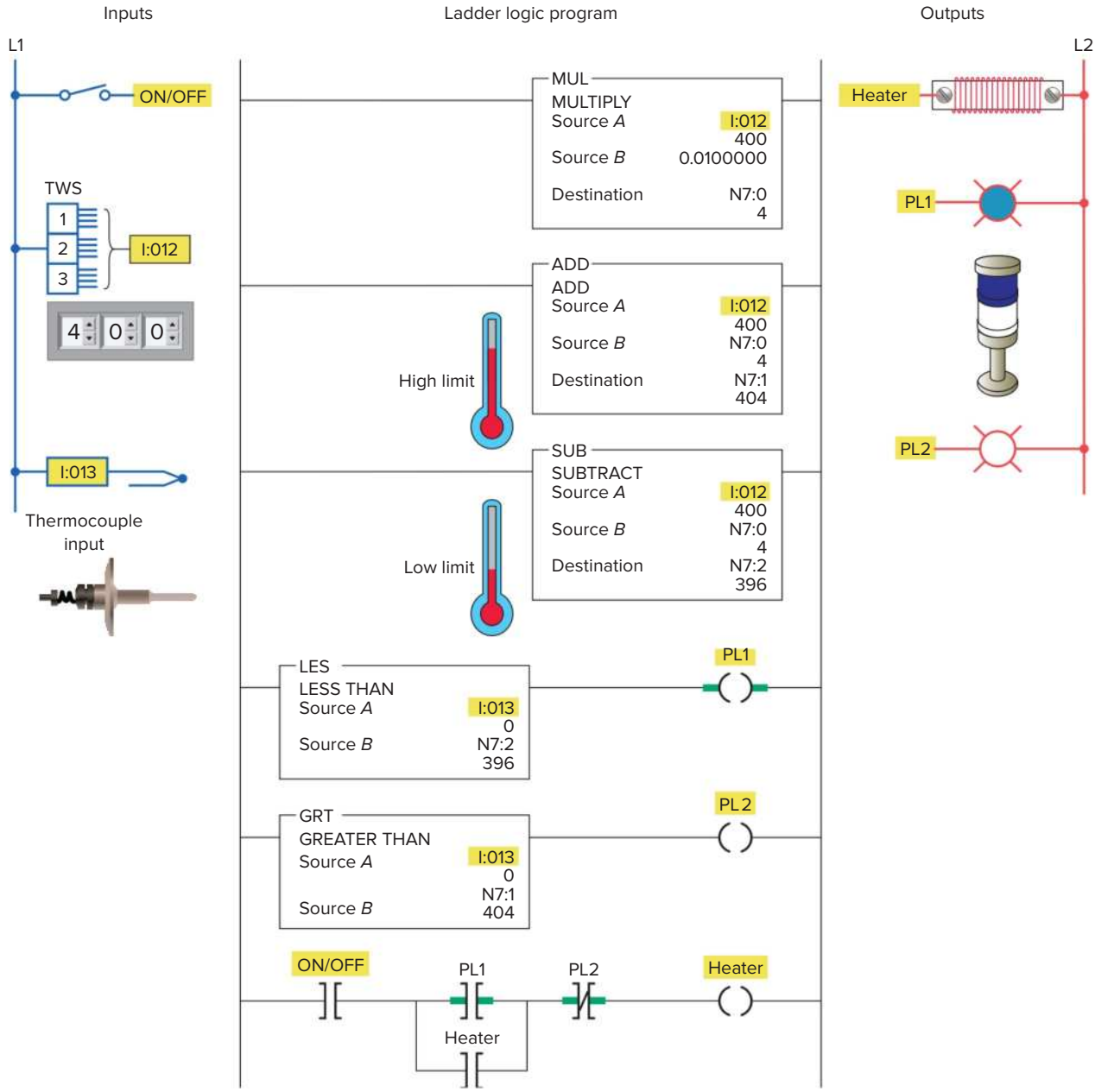


Figure 11-11 The MUL instruction used as part of a temperature control program.

- In this example, the set-point temperature is 400°F.
- Therefore, the electric heaters will be turned on when the temperature of the oven drops to less than 396°F and stay on until the temperature rises above 404°F.
- If the set-point is changed to 100°F, the deadband remains at ±1 percent, with the lower limit being 99°F and the upper limit being 101°F.
- The number stored in word N7:1 represents the upper temperature limit, and the number stored in word N7:2 represents the lower limit.

11.5 Division Instruction

The ***divide (DIV)*** instruction divides the value in source *A* by the value in source *B* and stores the result in the destination and math register. Figure 11-12 shows an example of the DIV instruction. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the rung will be true.
- The data in source *A* (the accumulated value of counter C5:10) is then divided by the data in source *B* (the constant 2).

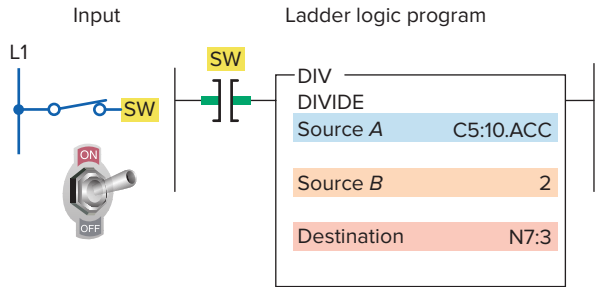


Figure 11-12 SLC 500 DIV (divide) instruction.

- The result is placed in the destination N7:3.
- If the remainder is 0.5 or greater, a roundup occurs in the integer destination.
- The value stored in the math register consists of the unrounded quotient (placed in the most significant word) and the remainder (placed in the least significant word).
- Some PLCs support the use of floating point numbers as well as integer (whole number) values. As an example, 10 divided by 3 may be expressed as 3.333333 (floating-point notation) or 3 with a remainder of 1.
- A minor fault bit is set upon detection of a division by zero.

The program of Figure 11-13 is an example of how the DIV instruction calculates the integer value that results from dividing source *A* by source *B*. The operation of the program can be summarized as follows:

- When input switch SW is closed the DIV instruction is executed.
- The value stored in source *A*, address N7:0 (120), is then divided by the value stored in source *B*, address N7:1 (4).
- The answer, 30, is placed in the destination address N7:5.
- As a result, the equal instruction becomes true, turning output PL1 on.

The program of Figure 11-14 is an example of how the DIV function is used as part of a program to convert Celsius temperature to Fahrenheit. The operation of the program can be summarized as follows:

- The thumbwheel switch connected to the input module indicates Celsius temperature.
- The program is designed to convert the recorded Celsius temperature in the data table to Fahrenheit values for display.
- The following conversion formula forms the basis for the program:

$$F = \left(\frac{9}{5} \times C\right) + 32$$

- In this example, a current temperature reading of 60°C is assumed.
- The PLC processor carries out its internal operations using binary numbers and the FRD instruction is used to convert the 16-bit integer values from the thumbwheel switch into BCD values.
- The MUL instruction multiplies the temperature (60°C) by 9 and stores the product (540) in address N7:0.

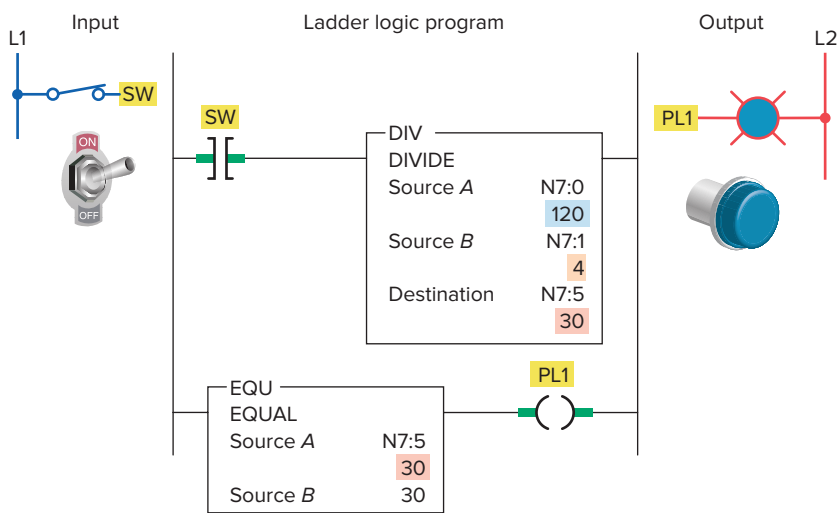


Figure 11-13 DIV instruction used to calculate the value that results from dividing source *A* by source *B*.

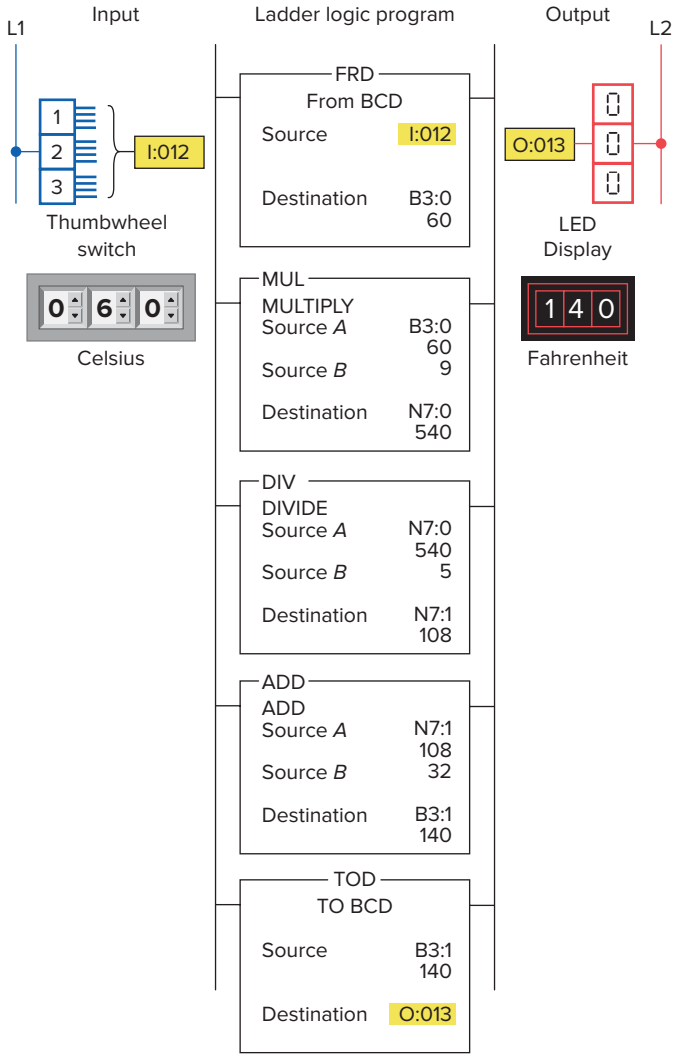


Figure 11-14 Program for converting Celsius temperature to Fahrenheit.

- The DIV instruction divides 5 into the 540 and stores the answer (108) in address N7:1.
- The ADD instruction adds 32 to the value of 108 and stores the sum (140) in address B3:1.
- Finally the TOD instruction is used to convert BCD values into integers. to interface with the LED display.
- Thus 60°C is displayed as 140°F.

The **Modulo (MOD)** instruction, shown in Figure 11-15, is part of the ControlLogix Compute/Math instruction set. This instruction is used to calculate the **remainder** after the value stored in Source A is divided by the value stored in Source B. In this unconditional rung example:

$$\frac{Source\ A}{Source\ B} = Dest\ (remainder)$$

$$\frac{Thumb_Value}{2} = Actual_Value$$

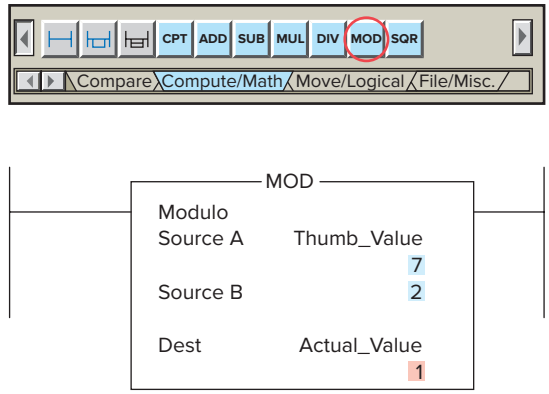


Figure 11-15 ControlLogix Modulo (MOD) instruction.

$$Thumb_Value = 7$$

$$\frac{7}{2} = 1$$

11.6 Other Word-Level Math Instructions

The program of Figure 11-16 is an example of the ***square root (SQR)*** instruction. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the SQR instruction is executed.
- The number whose square root we want to determine (144) is placed in the source.
- The function calculates the square root and places it (12) in the destination.
- If the value of the source is negative, the instruction will store the square root of the absolute (positive) value of the source at the destination.

The program of Figure 11-17 is an example of the ***negate (NEG)*** instruction. This math function changes the sign of the source value from positive to negative. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the NEG instruction is executed.

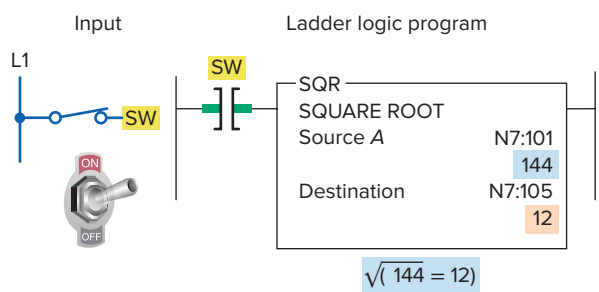


Figure 11-16 SLC 500 SQR (square root) instruction.

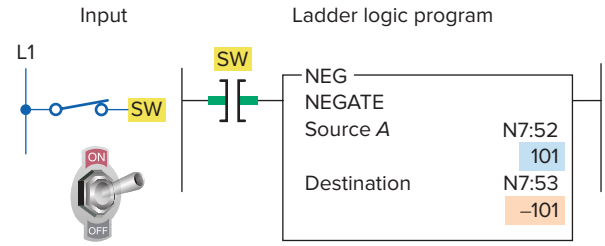


Figure 11-17 SLC 500 NEG (negate) instruction.

- The positive value 101 stored at the source address N7:52 is negated to −101 and stored in destination address N7:53.
- Positive numbers will be stored in straight binary format, and negative numbers will be stored as 2's complement.

The program of Figure 11-18 is an example of the ***clear (CLR)*** instruction. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the CLR instruction is executed.
- Upon execution it sets all bits of a word to zero.
- In this example it changes the value of all bits stored in the destination address N7:22 to 0.

The ***convert to BCD (TOD)*** instruction is used to convert 16-bit integers into *binary-coded decimal (BCD)* values. This instruction could be used when transferring data from the processor (which stores data in binary format) to an external device, such as an LED display, that functions in BCD format. The program of Figure 11-19 is an example of the TOD instruction. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the TOD instruction is executed.
- The binary bit pattern at the source address N7:23 is converted into a BCD bit pattern of the same decimal value at the destination address O:20.

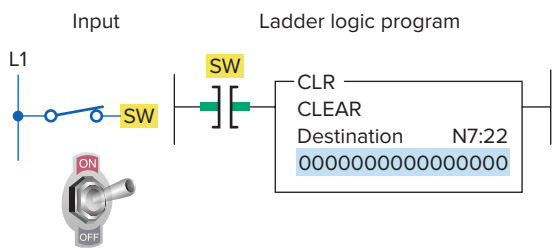


Figure 11-18 SLC 500 CLR (clear) instruction.

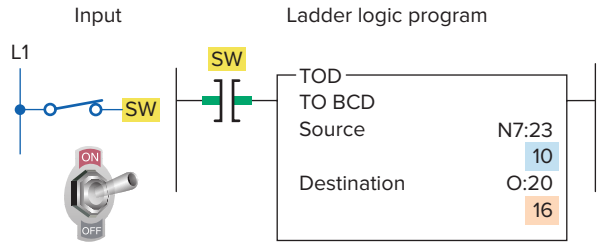


Figure 11-19 SLC 500 TOD (convert to BCD) instruction.

- The source displays the value 10, which is the correct decimal value; however, the destination displays the value 16.
- The processor interprets all bit patterns as binary; therefore the value 16 is the binary interpretation of the BCD bit pattern.
- The bit pattern for 10 BCD is the same as the bit pattern for 16 binary.

The ***convert from BCD (FRD)*** instruction is used to convert binary-coded decimal (BCD) values to integer values. This instruction could be used to convert data from a BCD external source, such as a BCD thumbwheel switch, to the binary format in which the processor operates. The program of Figure 11-20 is an example of the FRD instruction. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the FDR instruction is executed.
- The BCD bit pattern stored at the source address I:30 is converted into a binary bit pattern of the same decimal value at the destination address, N7:24.

At times it is necessary to make conversions to analog input and output values to ensure correct interpretation and processing. As a result, data must be **scaled**, or resized, before it can be used by a PLC control algorithm or output to a field device. The SLC 500 Scale data (**SCL**) and Scale with Parameters (**SCP**) instructions are used to

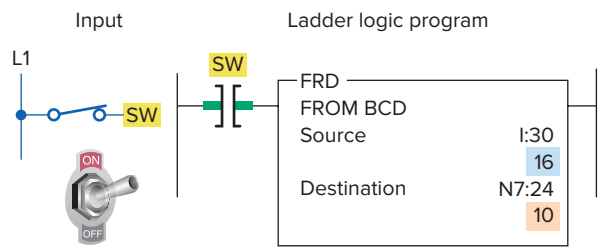


Figure 11-20 SLC 500 FRD (convert from BCD) instruction.

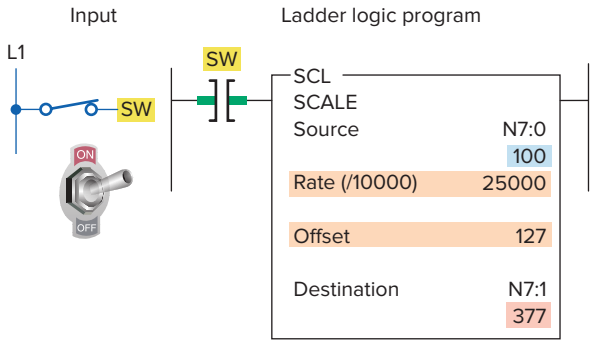


Figure 11-21 SLC 500 SCL (scale) instruction.

perform this task. Both instructions use the same formula to perform the scaling function, which is:

$$y = mx + b$$

Where: y is the output
m is the scaling rate
x is the input
b is the offset

Scaling rate (m) = (scaled Max − scaled Min) / (input Max − input Min) Offset (b) = (scaled Min) − (input Min x m)

The ladder rung of Figure 11-21 is an example of the use of the SCL instruction. When rung conditions are true, this instruction multiplies the source by a specified rate. The rounded result is then added to an offset value and placed in the destination. The execution of the instruction can be summarized as follows:

- When input switch SW is closed the SCL instruction is executed.
- The number 100 stored at the source address, N7:0, is multiplied by the rate 25,000, divided by 10,000, and added to 127.
- The result, 377, is placed in the destination address, N7:1.

Figure 11-22 shows an example of an analog input to a PLC, and the SCP instruction used to scale its data. The execution of the instruction can be summarized as follows:

- A strain-gage pressure transducer is connected to input I:1.0.
- The gage measures pressure from 0 to 1000 psi and provides an analog output of 0 to 10V.
- The unscaled range is 0 to 32,767, and the output is loaded into N7:20.
- When executed, the SCP instruction places a number between 0 and 1000 into N7:20 (destination) based on the input signal (0 to 10V) coming from the transducer into the analog input module.

Figure 11-23 shows an example of an analog output from a PLC, and the SCP instruction used to scale its data. The execution of the instruction can be summarized as follows:

- A proportional control valve is connected to the PLC output O:1.0.
- A 4 to 20 mA signal operates the valve from closed to 100% open.
- The percent open is in location N7:21.
- The PLC analog module provides a 4 to 20 mA output signal for a number between from 6,242 to 31,208.
- The SCP directs analog output O:1.0 to provide a 4 to 20 mA signal, which is scaled to the valve position based on a number between 0 and 100.

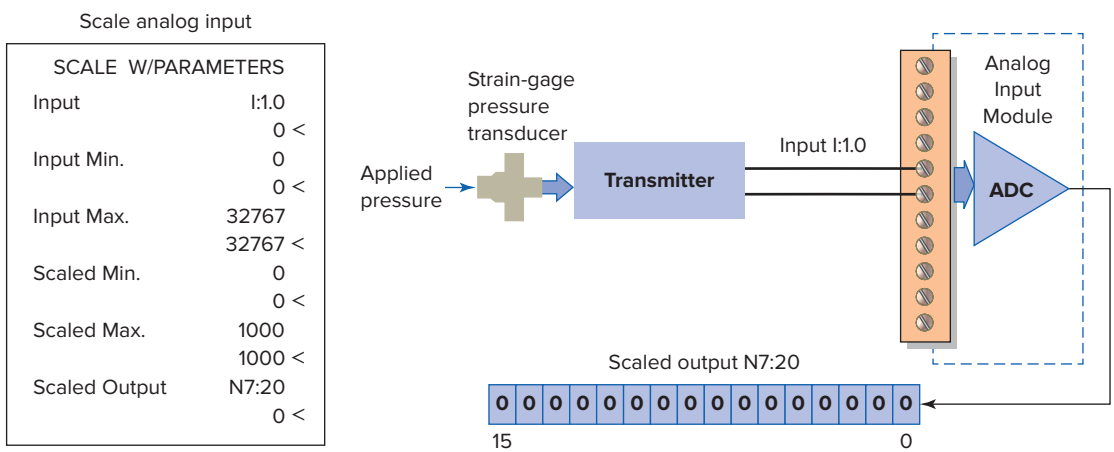


Figure 11-22 Scale analog input using the SCP instruction.

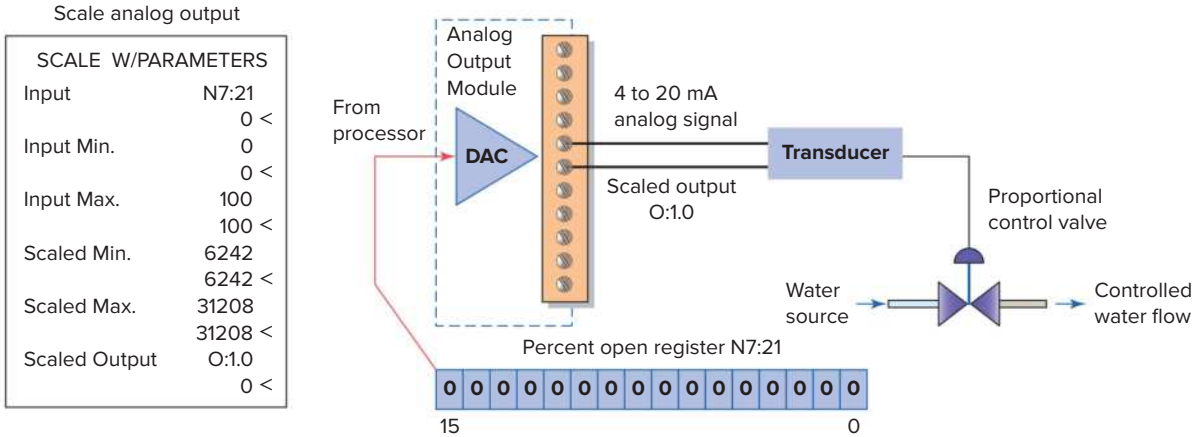


Figure 11-23 Scale analog output using the SCP instruction.

11.7 File Arithmetic Operations

File arithmetic functions include file add, file subtract, file multiply, file divide, file square root, file convert from BCD, and file convert to BCD. The *file arithmetic and logic (FAL)* instruction can combine an arithmetic operation with file transfer. The arithmetic operations that can be implemented with the FAL are ADD, SUB, MULT, DIV, and SQR.

The *file add* function of the FAL instruction can be used to perform addition operations on multiple words. The program of Figure 11-24 is an example of the file add function of the FAL instruction. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the rung goes true and the expression tells the processor to add the data in file address N7:25 to the data stored in file address N7:50 and store the result in file address N7:100.
- The rate per scan is set at All, so the instruction goes to completion in one scan.

The program of Figure 11-25 is an example of the *file subtract* function of the FAL instruction. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the rung goes true and the processor subtracts a program constant (255) from each word of file address N10:0 and stores the result at the destination file address, N7:255.
- The rate per scan is set at 2, so it will take 2 scans from the moment the instruction goes true to complete its operation.

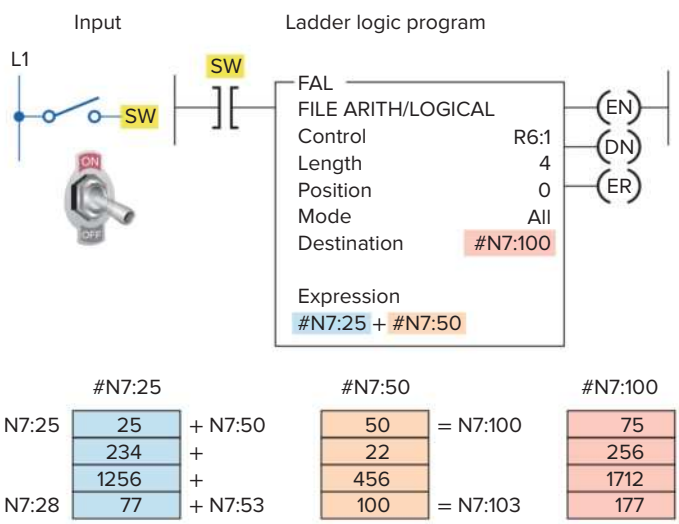


Figure 11-24 SLC 500 file add function of the FAL instruction.

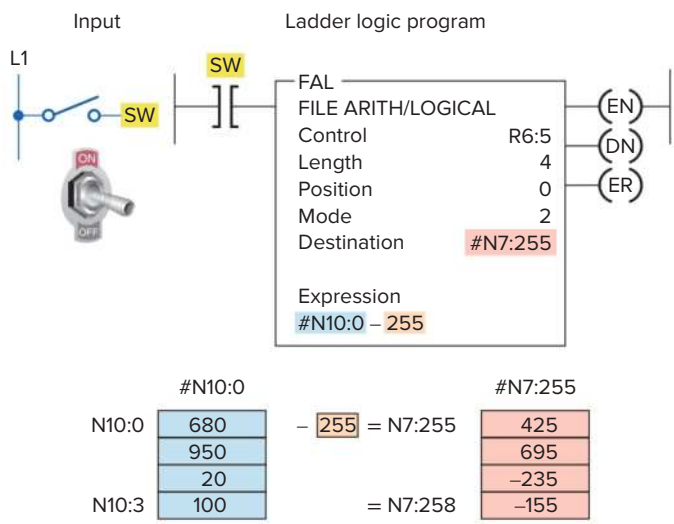


Figure 11-25 SLC 500 file subtract function of the FAL instruction.

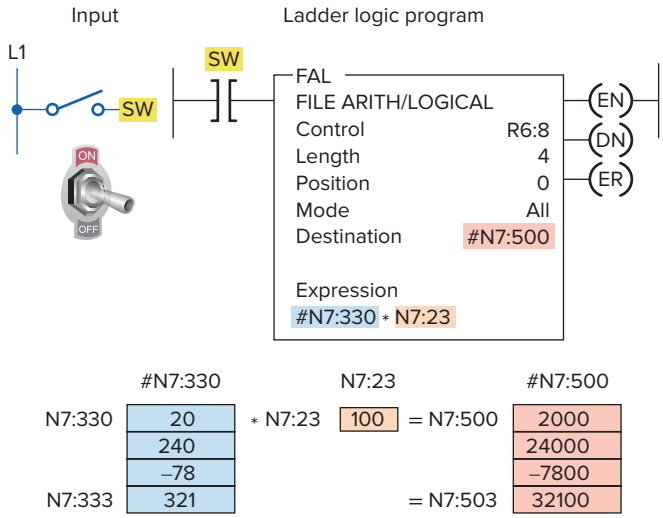


Figure 11-26 SLC 500 file multiply function of the FAL instruction.

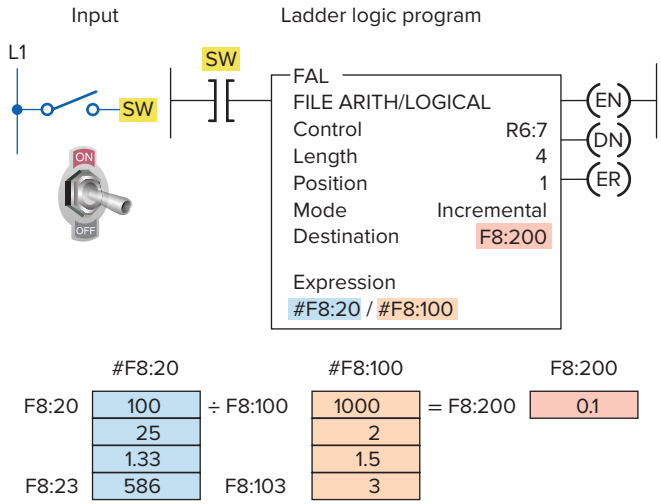


Figure 11-27 SLC 500 file divide function of the FAL instruction.

The program of Figure 11-26 is an example of the ***file multiply*** function of the FAL instruction. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the rung goes true and the data in file address N7:330 is multiplied by the data in element address N7:23, with the result stored at the destination file address N7:500.
- The rate per scan is set at All, so the instruction goes to completion in one scan.

The program of Figure 11-27 is an example of the ***file divide*** function of the FAL instruction. The operation of the logic rung can be summarized as follows:

- When input switch SW is closed the rung goes true and the data in file address F8:20 is divided by the data in file address F8:100, with the result stored in element address F8:200.
- The mode is Incremental, so the instruction operates on one set of elements for each false-to-true transition of the instruction.

CHAPTER 11 REVIEW QUESTIONS

1. Explain the function of math instructions as applied to the PLC.
2. Name the four basic math functions performed by PLCs.
3. What standard format is used for PLC math instructions?
4. Would math instructions be classified as input or output instructions?
5. With reference to the instruction of Figure 11-28, what is the value of the number stored at source *B* if N7:3 contains a value of 60 and N7:20 contains a value of 80?
6. With reference to the instruction of Figure 11-29, what is the value of the number stored at the destination if N7:3 contains a value of 500?
7. With reference to the instruction of Figure 11-30, what is the value of the number stored at the destination if N7:3 contains a value of 40 and N7:4 contains a value of 3?
8. With reference to the instruction of Figure 11-31, what is the value of the number stored at the destination if N7:3 contains a value of 15 and N7:4 contains a value of 4?
9. With reference to the instruction of Figure 11-32, what is the value of the number stored at N7:20 if N7:3 contains a value of 2345?
10. With reference to the instruction of Figure 11-33, what will be the value of each of the bits in word B3:3 when the rung goes true?
11. With reference to the instruction of Figure 11-34, what is the value of the number stored at N7:101?
12. With reference to the instruction of Figure 11-35, list the values that will be stored in file #N7:10 when the rung goes true.

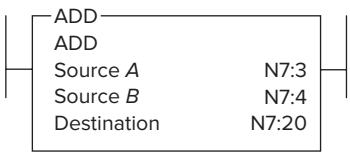


Figure 11-28 Instruction for Question 5.

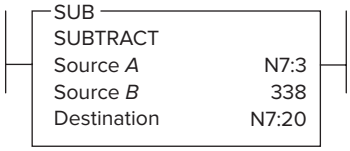


Figure 11-29 Instruction for Question 6.

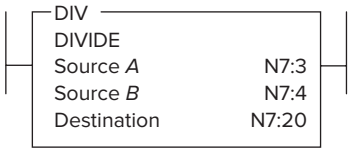


Figure 11-30 Instruction for Question 7.

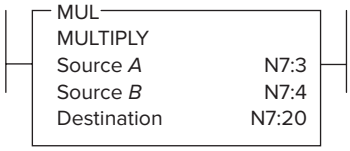


Figure 11-31 Instruction for Question 8.

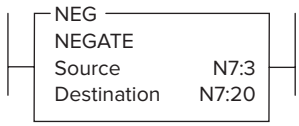


Figure 11-32 Instruction for Question 9.

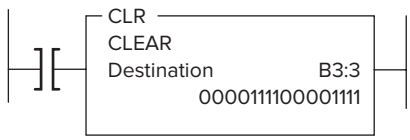


Figure 11-33 Instruction for Question 10.

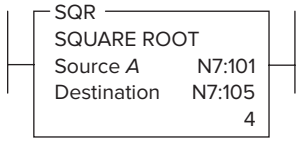
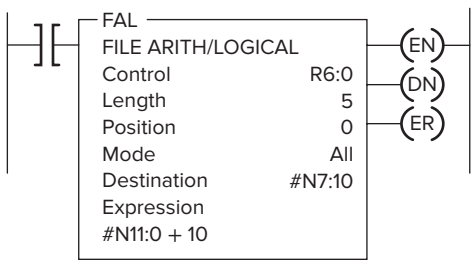


Figure 11-34 Instruction for Question 11.



File #N11:0

328
150
10
32
0

Figure 11-35 Instruction for Question 12.

CHAPTER 11 PROBLEMS

1. Answer each of the following with reference to the counter program shown in Figure 11-36.

a. Assume the accumulated count of counters C5:0 and C5:1 to be 148 and 36, respectively. State the value of the number stored in each of the following words at this point:
(1) C5:0.ACC
(2) C5:1.ACC
(3) N7:1
(4) Source *B* of the GEQ instruction

b. Will output PL1 be energized at this point? Why?

c. Assume the accumulated count of counters C5:0 and C5:1 to be 250 and 175, respectively. State the value of the number stored in each of the following words at this point:
(1) C5:0.ACC
(2) C5:1.ACC
(3) N7:1
(4) Source *B* of the GEQ instruction

d. Will output PL1 be energized at this point? Why?

2. Answer each of the following with reference to the overfill alarm program shown in Figure 11-37.

a. Assume that the vessel is filling and has reached the 300-lb point. State the status of each of the logic rungs (true or false) at this point.

b. Assume that the vessel is filling and has reached the 480-lb point. State the value of the number stored in each of the following words at this point:
(1) I:012
(2) N7:1

c. Assume that the vessel is filled to a weight of 502 lb. State the status of each of the logic rungs (true or false) for this condition.

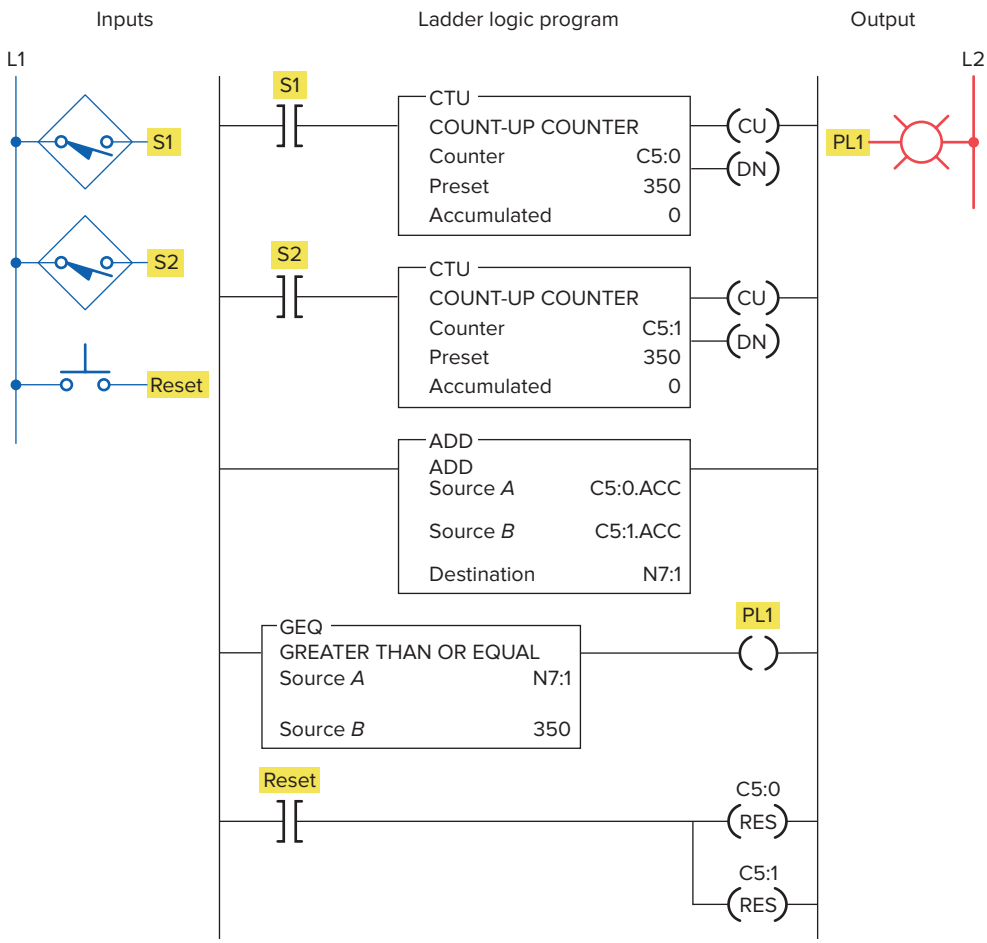


Figure 11-36 Program for Problem 1.

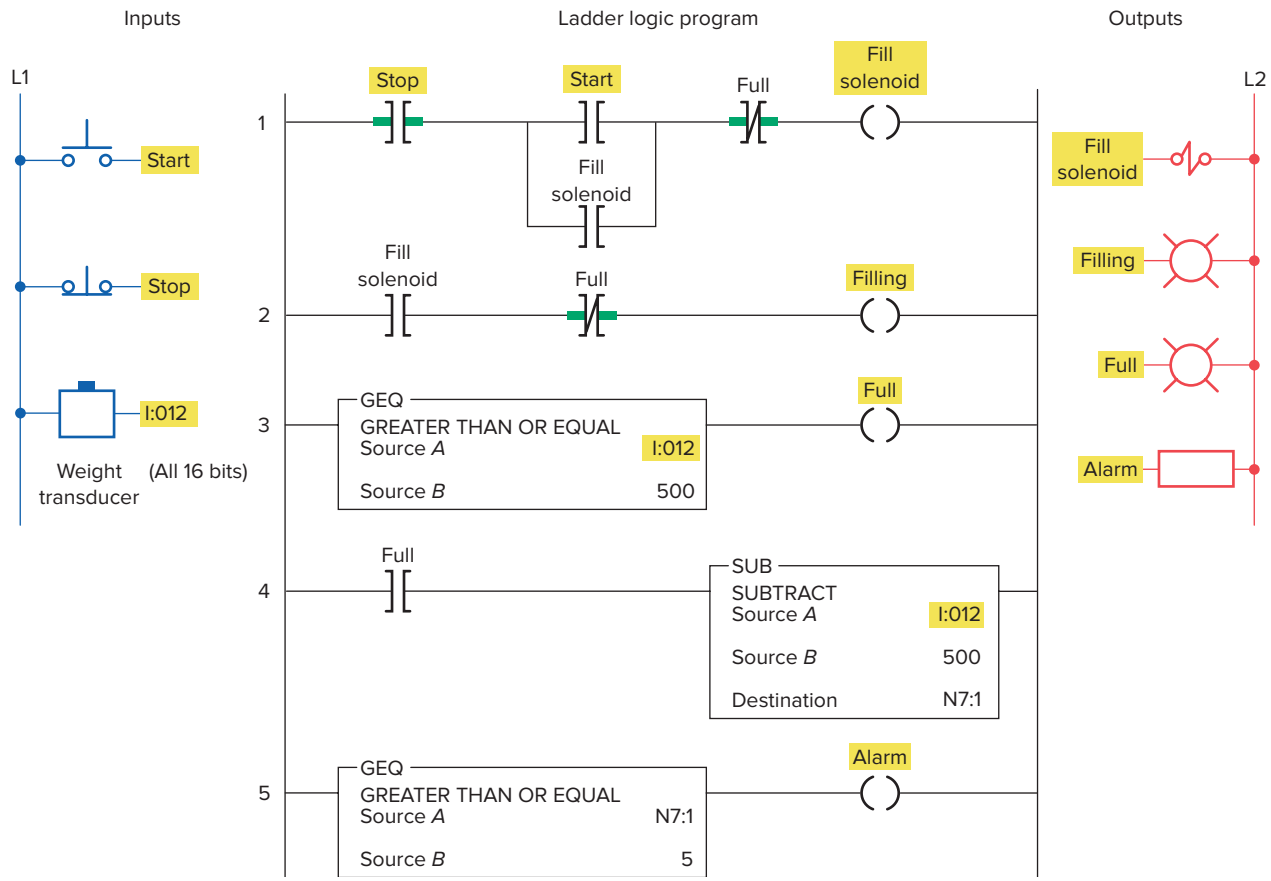


Figure 11-37 Program for Problem 2.

d. Assume that the vessel is filled to a weight of 510 lb. State the value of the number stored in each of the following words for this condition:
(1) I:012
(2) N7:1

e. With the vessel filled to a weight of 510 lb, state the status of each of the logic rungs (true or false).

3. Answer the following with reference to the temperature control program shown in Figure 11-38.

a. Assume that the set-point temperature is 600°F. At what temperature will the electric heaters be turned on and off?

b. Assume that the set-point temperature is 600°F and the thermocouple input module indicates a temperature of 590°F. What is the value of the number stored in each of the following words at this point?
(1) I:012
(2) I:013
(3) N7:0
(4) N7:1
(5) N7:2

c. Assume that the set-point temperature is 600°F and the thermocouple input module indicates a temperature of 608°F. What is the status (energized or not energized) of each of the following outputs?
(1) PL1
(2) PL2
(3) Heater

4. With reference to the Celsius to Fahrenheit conversion program shown in Figure 11-39, state the value of the number stored in each of the following words for a thumbwheel setting of 035:

a. I:012
b. N7:0
c. N7:1
d. O:013

5. Design a program that will add the values stored at N7:23 and N7:24 and store the result in N7:30 whenever input *A* is true, and then, when input *B* is true, will copy the data from N7:30 to N7:31.

6. Design a program that will take the accumulated value from TON timer T4:1 and display it on a 4-digit, BCD format set of LEDs. Use address

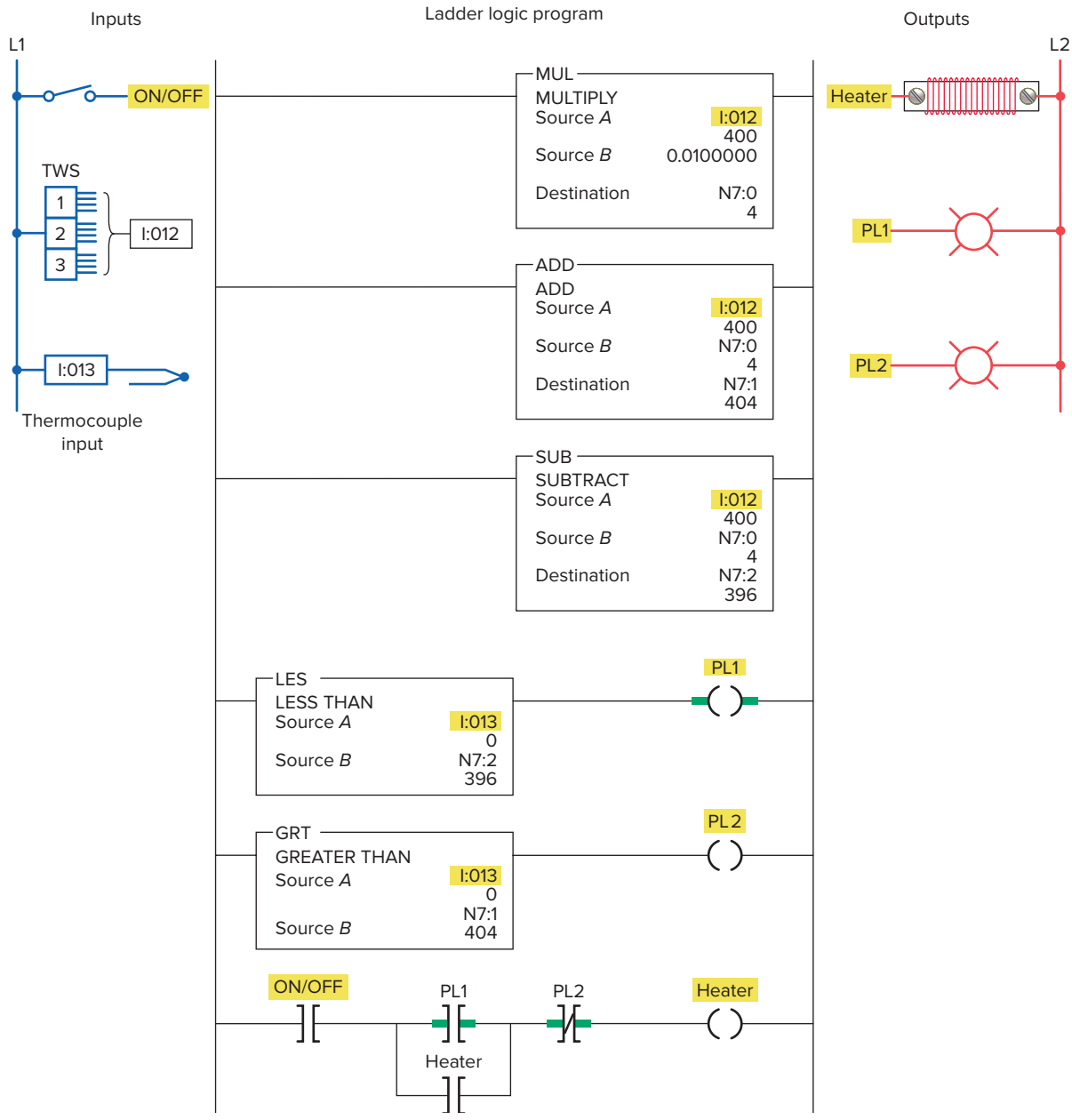


Figure 11-38 Program for Problem 3.

O:023 for the LEDs. Include the provision to change the preset value of the timer from a set of 4-digit BCD thumbwheels when input *A* is true. Use address I:012 for the thumbwheels.

7. Design a program that will implement the following arithmetic operation:
- Use an MOV instruction and place the value 45 in N7:0 and 286 in N7:1.
- Add the values together and store the result in N7:2.
- Subtract the value in N7:2 from 785 and store the result in N7:3.
- Multiply the value in N7:3 by 25 and store the result in N7:4.
- Divide the value in N7:4 by 35 and store the result in F8:0.

8. **a.** There are three part conveyor lines (1-2-3) feeding a main conveyor. Each of the three conveyor lines has its own counter. Construct a PLC program to obtain the total count of parts on the main conveyor.

b. Add a timer to the program that will update the total count every 30 s.

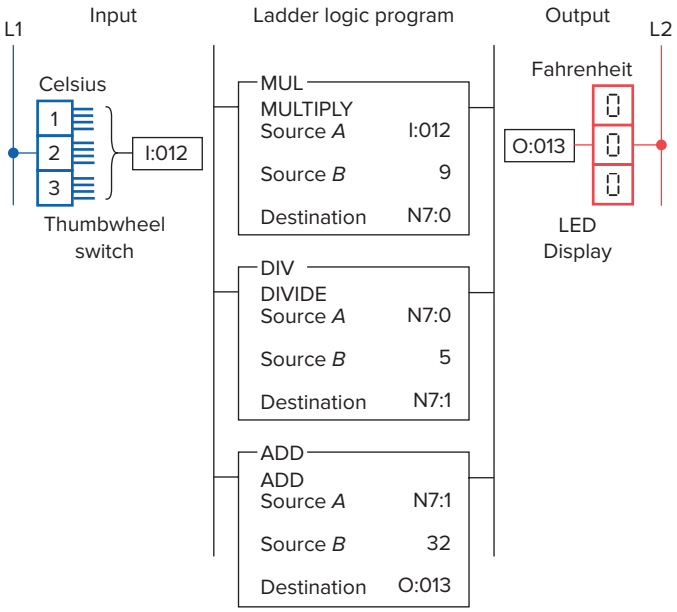


Figure 11-39 Program for Problem 4.

9. With reference to math instruction program shown in Figure 11-40, when the input goes true, what value will be stored at each of the following?
a. N7:3
b. N7:5
c. F8:1

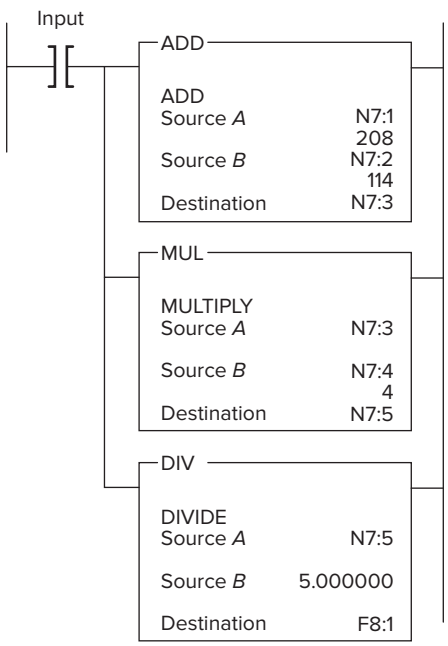


Figure 11-40 Program for Problem 9.

10. With reference to the math instruction program shown in Figure 11-41, when the input goes true, what value will be stored at each of the following?
a. N7:3
b. N7:4
c. N7:5
d. N7:6

11. Two part conveyor lines, A and B, feed a main conveyor line, M. A third conveyor line, R, removes rejected parts a short distance away from the main conveyor. Conveyors A, B, and R have parts counters connected to them. Construct a PLC program to obtain the total parts output of main conveyor M.

12. A main conveyor has two conveyors, A and B, feeding it. Feeder conveyor A puts six-packs of canned soda on the main conveyor. Feeder conveyor B puts eight-packs of canned soda on the main conveyor. Both feeder conveyors have counters that count the number of *packs* leaving them. Construct a PLC program to give a *total can* count on the main conveyor.

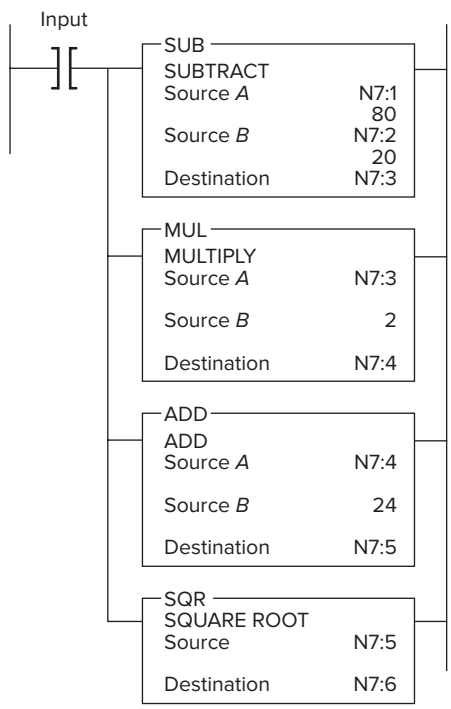


Figure 11-41 Program for Problem 10.

12

Sequencer and Shift Register Instructions

Image Courtesy Banner Engineering Corp.

This chapter explains how the PLC sequencer and shift register functions operate and how they can be applied to control problems. The sequencer instruction evolved from the mechanical drum switch, and it can handle complex sequencing control problems more easily than does the drum switch. Shift registers are often used to track parts on automated manufacturing lines by shifting either status or values through data files.

Chapter Objectives

After completing this chapter, you will be able to:

- Identify and describe the various forms of mechanical sequencers and explain the basic operation of each
- Interpret and explain information associated with PLC sequencer output, compare, and load instructions
- Compare the operation of an event-driven and a time-driven sequencer
- Describe the operation of bit and word shift registers
- Interpret and develop programs that use shift registers

12.1 Mechanical Sequencers

Sequencer instructions are designed to operate much like the mechanical rotating cam limit switch shown in Figure 12-1. These mechanical type sequencers are often referred to as drum switches, rotary switches, stepper switches, or cam switches. They are often used to control machinery that has a repetitive cycle of operation.

Figure 12-2 illustrates the operation of a cam-operated **sequencer switch.** An electric motor is used to drive the cams. A series of leaf-spring mounted contacts interacts with the cam so that in different degrees of rotation of the cam, various contacts are closed and opened to energize and de-energize various electrical devices. As the cams rotate, load devices connected to the contacts can change from an on to an off state, from an off to an on state, or remain at the same state.

Figure 12-3 illustrates a typical mechanical drum-operated sequencer switch. The switch consists of a series of normally open contact blocks that are operated by pegs located on the motor-driven drum. The operation of this sequencer can be summarized as follows:

- Pegs are placed at specific locations around the circumference of the drum to operate the contact blocks.
- When the drum is rotated, contacts that align with the pegs will close, whereas the contacts where there are no pegs will remain open.
- The presence of a peg can be interpreted as logic 1, or on, and the absence of a peg as logic 0, or off.
- The equivalent sequencer data table illustrates the logic state for the first four steps of the drum cylinder.

Figure 12-1 Rotating cam limit switch.
Source: Images Courtesy of Rockwell Automation, Inc.

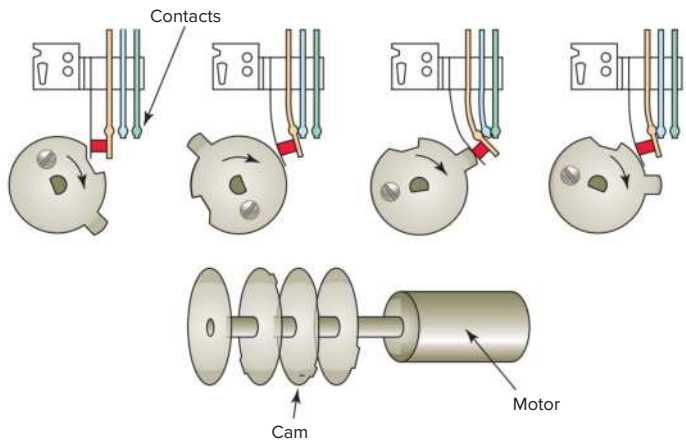


Figure 12-2 Mechanical cam-operated sequencer.

- Each location where there was a peg is represented by a 1 (on), and the positions where there were no pegs are each represented by a 0 (off).

Sequencer switches are useful whenever a repeatable operating pattern is required. One example is the timed

Equivalent sequencer data table

0	1	0	1	0	1	1	0	0	0	1	0	1	0	1	0	4
1	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	3
0	1	1	1	0	0	1	0	1	0	1	0	1	0	1	0	2
1	1	1	1	0	0	1	1	0	0	1	0	1	0	1	1	1

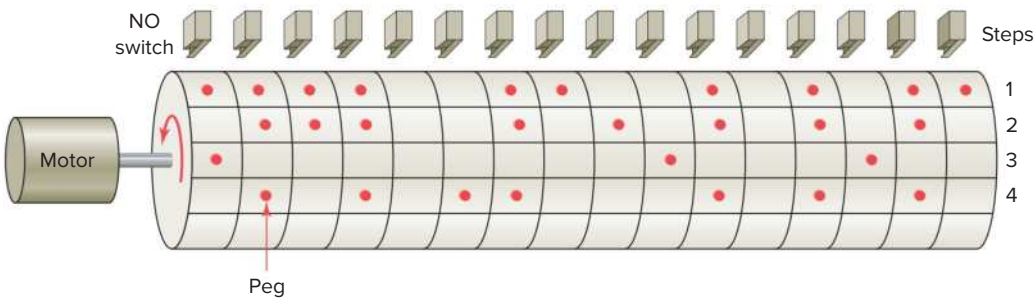


Figure 12-3 Mechanical drum-operated sequencer switch.

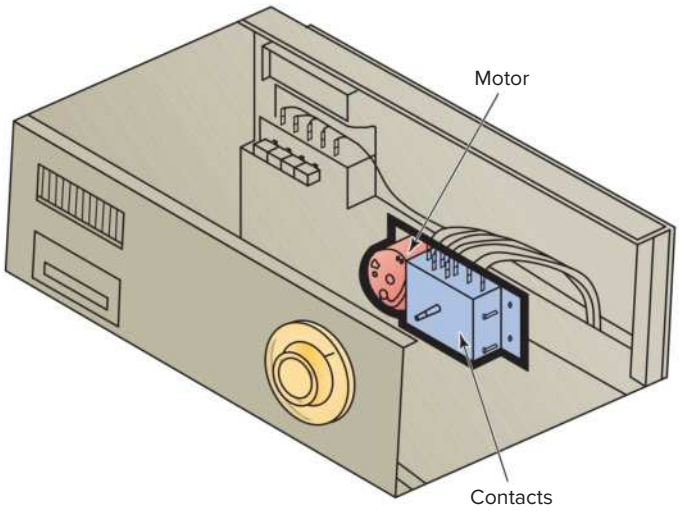
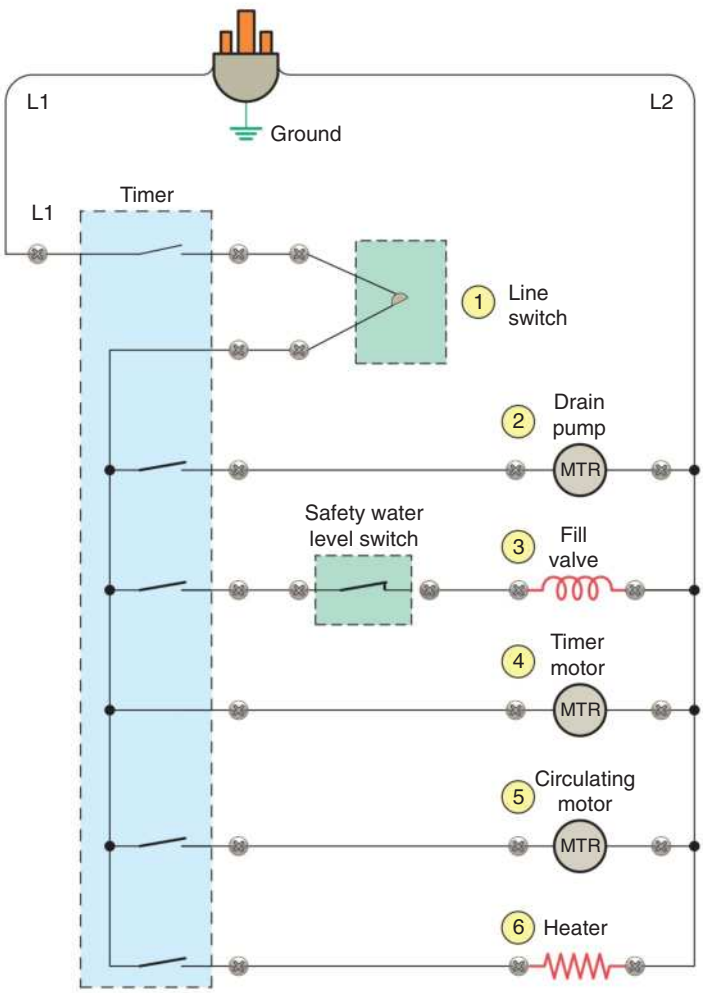


Figure 12-4 Dishwasher timed sequencer switch.

sequencer switch used in dishwashers to pilot the machinery through a wash cycle (Figure 12-4). The cycle is always the same with a fixed routine of actions at each step for a specific time to complete its specified task. The domestic washing machine is another example of the use of a sequencer, as are dryers and similar time-clock controlled devices.

An example of the wiring and timing chart for a dishwasher that uses a cam-operated sequencer, commonly known as the timer, is shown in Figure 12-5. A synchronous motor drives a mechanical train that, in turn, drives a series of cam wheels. The operation of this sequencer can be summarized as follows:

- The timer motor operates continuously throughout the cycle of operation.
- The cam advances in time increments of 45 seconds in duration.
- The data timing chart shows the sequence of operation of the timer.
- A total of sixty 45 second steps are used to complete the 45 minute operating cycle.
- Numbers in the active devices column refer to control devices active during each step of the cycle.



Machine function		Timer increment	Active devices
Off		0–1	
First prerinse	Drain	2	1 2 4
	Fill	3	1 3 4 5
	Rinse	4–5	1 4 5 6
	Drain	6	1 2 4 5
Prewash	Fill	7	1 3 4 5
	Wash	8–10	1 4 5 6
	Drain	11	1 2 4 5
Second prerinse	Fill	12	1 3 4 5
	Rinse	13–15	1 4 5 6
	Drain	16	1 2 4
Wash	Fill	17	1 3 4
	Wash	18–30	1 4 5 6
	Drain	31	1 2 4 5
First rinse	Fill	32	1 3 4 5
	Rinse	33–34	1 4 5 6
	Drain	35	1 2 4 5
Second rinse	Fill	36	1 3 4 5
	Rinse	37–41	1 4 5 6
	Drain	42	1 2 4 5
Dry	Dry	43–58	1 4 6
	Drain	59	1 2 4 6
	Dry	60	1 4 6

Figure 12-5 Dishwasher wiring diagram and timing chart.

12.2 Sequencer Instructions

PLC **sequencer instructions** replace the mechanical drum sequencer that is used to control machines that have a stepped sequence of repeatable operations. Programmed sequencers can perform the same specific on or off patterns of outputs that are continuously repeated with a drum switch, but with much more flexibility. Sequencer instructions simplify your ladder program by allowing you to use a single instruction or pair of instructions to perform complex operations. For example, the on/off operation of 16 discrete outputs can be controlled, using a sequencer instruction, with only one ladder rung. By contrast, the equivalent contact-coil ladder control arrangement would need 16 rungs in the program.

Depending on the PLC manufacturer, various sequencer instructions can be programmed. Figure 12-6 shows the **Sequencer** menu tab for the Allen-Bradley SLC 500 PLC and its associated RSLogix software. For the Allen-Bradley line of controllers, sequencer commands may include the following:

SQO (Sequencer Output)—Is an output instruction that uses a file to control various output devices.

SQI (Sequencer Input)—Is an input instruction that compares bits from an input file to corresponding bits from a source address. The instruction is true if all pairs of bits are the same.

SQC (Sequencer Compare)—Is an output instruction that compares bits from an input source file to corresponding bits from data words in a sequence file. If all pairs of bits are the same, then a bit in the control register is set to 1.

SQL (Sequencer Load)—Is an output instruction used to capture reference conditions by manually stepping the machine through its operating sequences. It transfers data from the input source module to the sequencer file. The instruction functions much like a file-to-word transfer instruction.

Figure 12-7 shows an example of an **SQO (Sequencer Output)** instruction. Its execution is summarized as follows:

- The SQO instruction is placed on the right side of the rung as an output.

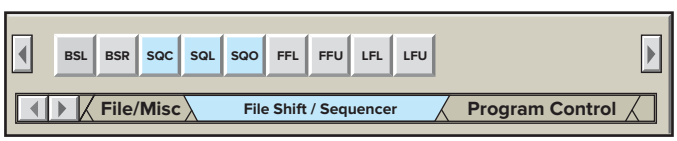


Figure 12-6 Sequencer menu tab.

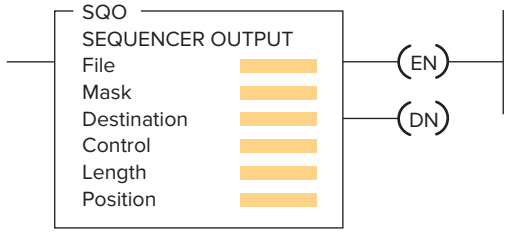


Figure 12-7 SQO (Sequencer Output) instruction.

- Each time the rung makes a false-to-true transition the position is incremented to the next step in the sequencer File.
- Data from the sequencer File are then transferred through the Mask into the specified Destination address.
- The data are updated during each scan where the rung remains true.
- When the last word in the sequencer file is transferred the done bit is set.
- Then, on the next false-to-true transition, the Destination data are cleared and the Position pointer is reset to step one.
- At start-up, when the processor is switched from program mode to the run mode the operation depends on the rung being true or false on the initial scan.
  - If true, the instruction transfers the value in step zero.
  - If false, the first rung transition from false-to-true transfers the value in step one of the instruction.
- The Mask bits must be set in order to change the value in the Destination word. The bits mask data when reset to 0 and pass data when set to 1.

The Sequencer Output (SQO) instruction is used to control output devices sequentially. The desired sequence operation is stored in a data file or array. As the sequencer advances through its steps, the stored data are transferred one word at a time through a Mask to the Destination. Parameters that may be required to be entered in sequencer instructions can be summarized as follows:

File—Is the starting address for the registers in the sequencer file and you must use the indexed file indicator (#) for this address. The file contains the data that will be transferred to the destination address when the instruction undergoes a false-to-true transition. Each word in the file represents a position, starting with position 0 and continuing to the file length.

Mask—Is the bit pattern through which the sequencer instruction moves source data to the destination address.

Recall that in the mask bit pattern, a 1 passes values while a 0 blocks the data flow but the existing bit value remains the same. You use a mask register or file name when you want to change the mask pattern under program control. An ***h*** is placed behind the parameter to indicate that the mask is a hexadecimal number or a ***B*** is placed to indicate binary notation. Decimal notation is entered without any indicator.

Source—Is the address of the input word or file from which the SQC and SQL instruction obtains data for comparison or input to its sequencer file.

Destination—Is the address of the output word or file to which the SQO moves the data from its sequencer file.

Control—Is the address that contains the parameters with control information for the instruction. The control register stores the status byte of the instruction, the length of the sequencer file, and the instantaneous position in the file as follows:

- The **enable bit (EN;** bit 15) is set by a false-to-true rung transition and indicates that the instruction is enabled. It follows the rung condition.
- The **done bit (DN;** bit 13) is set after the last word in the sequencer file is transferred. On the next false-to-true transition of the rung with the done bit set, the position pointer is reset to 1.
- The **error bit (ER;** bit 11) is set when the processor detects a negative position value, or a negative or zero length value.

Length—Is the number of steps of the sequencer file starting at position 1. Position 0 is the start-up position. The instruction resets (wraps) to position 1 at each cycle completion. The actual file length will be 1 plus the file length entered in the instruction.

Position—Indicates the step that is desired to start the sequencer instruction. The position is the word location or step in the sequencer file from which the instruction moves data. Any value up to the file length may be entered, but the instruction will always reset to 1 on the true-to-false transition after the instruction has operated on the last position. Before we start the sequence, we need a starting point at which the sequencer is in a neutral position. The start position is all zeros, representing this neutral position; thus, all outputs will be off in position 0.

To program a sequencer, binary information is first entered into the sequencer file or register made up of a series of consecutive memory words. The sequencer file is typically a bit file that contains one bit file word representing the output action required for each step of the sequence. Data are entered for each sequencer step according to the requirements of the control application. As the sequencer advances through the steps, binary information is transferred from the sequencer file to the output word.

To illustrate the purpose and function of the sequencer file, we will examine the operation of the four-step sequence process shown in Figure 12-8. This sequencer is to be used to control traffic in two directions. The operation of the process can be summarized as follows:

- Six outputs are to be energized from one 16-point output module.
- Each light is controlled by one bit address of output word O:2.
- The first 6 bits are programmed to execute the following sequence of light outputs:
  - **Step 1:** Outputs O:2/0 (red) and O:2/5 (green) lights will be energized.
  - **Step 2:** Outputs O:2/0 (red) and O:2/4 (yellow) will be energized.
  - **Step 3:** Outputs O:2/2 (green) and O:2/3 (red) will be energized.
  - **Step 4:** Outputs O:2/1 (yellow) and O:2/3 (red) will be energized.
- Words B3:0, B3:1, B3:2, B3:3, and B3:4 make up the sequencer file.
- Binary information (1s and 0s) that reflects the desired on or off light status for each of the four steps is entered into each word of the sequencer file.
- Before starting the sequence, you need a starting point where the sequencer is in a neutral position. This is provided by the start position which is all zeros.

Due to the way in which the sequencer instruction operates, all output points must be on a single output module. When a sequencer operates on an entire output word, there may be outputs associated with the word that do *not* need to be controlled by the sequencer. In our example, bits 6 through 15 of output word O:2 are not used by the sequencer but could be used elsewhere in the program. To prevent the sequencer from controlling these bits of the output word, a mask word is used. The use of a mask word is illustrated in Figure 12-9. The operation of the mask can be summarized as follows:

- The **mask word** selectively screens out data from the sequencer word file to the output word.
- The hex number 003Fh is entered as the mask parameter.
- For each bit of output word O:2 that the sequencer is to control, the corresponding bit of the mask word must be set to 1.

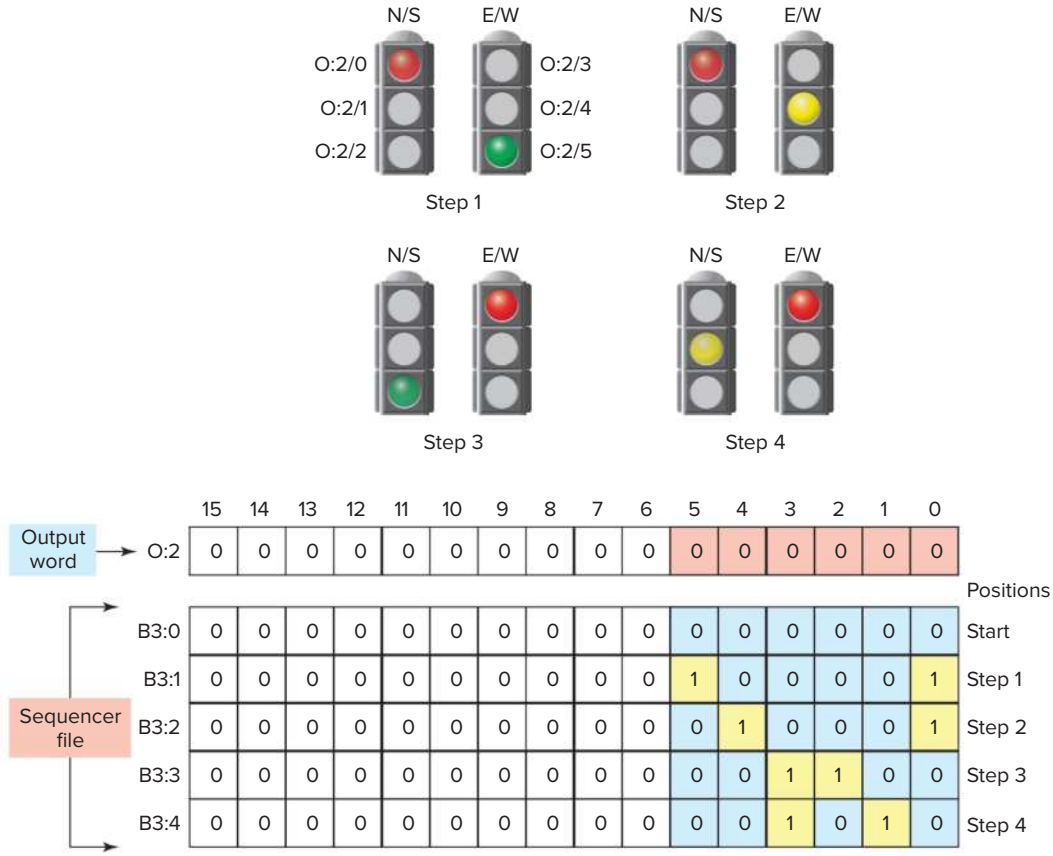


Figure 12-8 Four-step sequencer.

- The arrows in the figure indicate the unmasked bits that are passed through the mask and into the destination address.
- The dashes in the bits of the designation address indicate that those bits remain unchanged in the designation location during the sequencing.
- These unchanged bits therefore can be used independently of the sequencer.

The sequencer output instruction requires preceding logic on the rung where it is located. When this logic goes from false to true, it triggers the sequencer to perform its functions. Only when the logic preceding the sequencer instruction makes the transition from false to true will it go through its functions of reading the data file, applying the mask, and transferring the masked data file to the output destination. After this cycle, it waits for another false-to-true occurrence of the preceding logic to increment to the next step.

Figure 12-10 illustrates how the sequencer moves data from a file to an output. The operation of the logic rung can be summarized as follows:

- Pushbutton PB is used to send false-to-true trigger signals to the sequencer output instruction.
- The position of the sequencer instruction is incremented by one for each false-to-true transition of the sequencer rung.
- Whenever PB is momentarily closed, the sequencer is both enabled and advanced to the next position.
- When the sequencer is at step 1, the binary information in word B3:1 (100001) of the sequencer file is transferred into word O:2 of the output.
- As a result output O:2/0 and O:2/5 will be on and all the rest will be off.
- Advancing the sequencer to step 2 will transfer the data from word B3:2 (010001) into word O:2.
- As a result output O:2/0 and O:2/4 will be on and all the rest will be off.
- Advancing the sequencer to step 3 will transfer the data from word B3:3 (001100) into word O:2.
- As a result output O:2/2 and O:2/3 will be on and all the rest will be off.
- Advancing the sequencer to step 4 will transfer the data from word B3:4 (001010) into word O:2.
- As a result output O:2/1 and O:2/3 will be on and all the rest will be off.

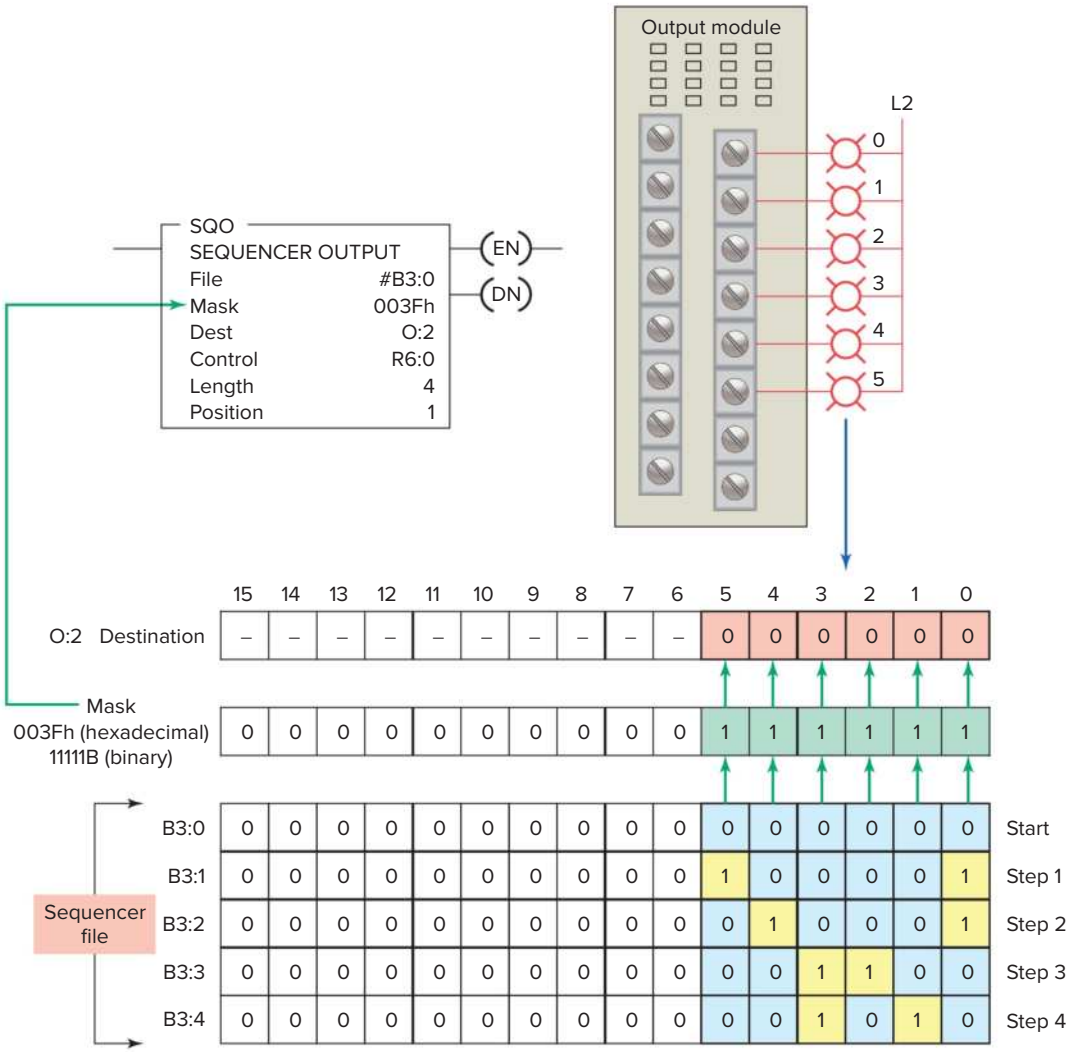


Figure 12-9 Sequencer moving data through a mask word.

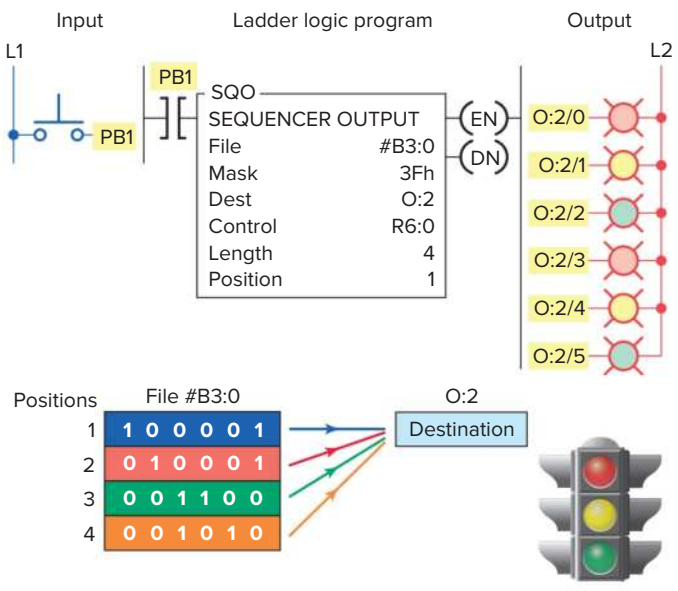


Figure 12-10 Sequencer moving data from a file to an output.

- When the position parameter reaches 4 (the value in the length parameter), all words would have been moved so the DN (done bit) in the instruction is set to 1.
- On the next false-to-true transition of the rung, with done bit set, the position pointer is automatically reset to 1.

Sequencer instructions are usually retentive, and there can be an upper limit to the number of external outputs and steps that can be operated on by a single instruction. Many sequencer instructions reset the sequencer automatically to step 1 on completion of the last sequence step. Other instructions provide an individual reset control line or a combination of both.

The ControlLogix Sequencer Output (SQO) instruction, shown in Figure 12-11, operates similarly to that of

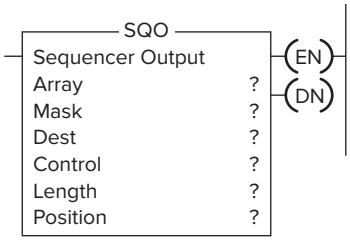


Figure 12-11 ControlLogix Sequencer Output (SQO) instruction.

the SLC 500 Sequencer Output instruction. The six parameters entered into this instruction are:

- **Array**—An array tag of the type DINT is the first entry you need to make. This is a word-level tag that defines the starting word of the sequencer data array. The desired output conditions for each step are manually entered into the array in the tag editor. The ControlLogix processor puts the radix and the # sign in front of the value to indicate the radix of the displayed number. For example, 16# (hexadecimal) or 2# (binary).
- **Mask**—The Mask works exactly like the mask in the MVM instruction. It can be a word level tag or a hexadecimal program constant. When the SQO transfers 32-bits of data to an output word, there might be outputs associated with the word that do not need to be controlled by the SQO. By masking these bits the SQO will not control them and they could be used for other purposes in the program.
- **DEST**—This is the word level data type DINT tag where the data from the instruction will be sent.
- **Control**—The tag of data type Control contains the control structure for the instruction. The Control tag has several bits that can be used: enable (EN), done (DN), error (ER), and so on. The control element also contains the Length of the sequencer (number of steps) and the current Position (step in the sequence).
- **Length**—The length (LEN) parameter stores the value that defines the number of steps the sequencer should make. It also defines the number of words required in the sequencer array. Position 0 is the start-up position. The first time the SQO instruction is enabled it moves from position 0 to position 1 when the instruction is toggled. The instruction resets to position 1 at the end of the last step. The array size must be at least one element larger than the size of the length.
- **Position**—The Position (POS) parameter stores the current step of the sequencer. Steps are numbered starting at zero.

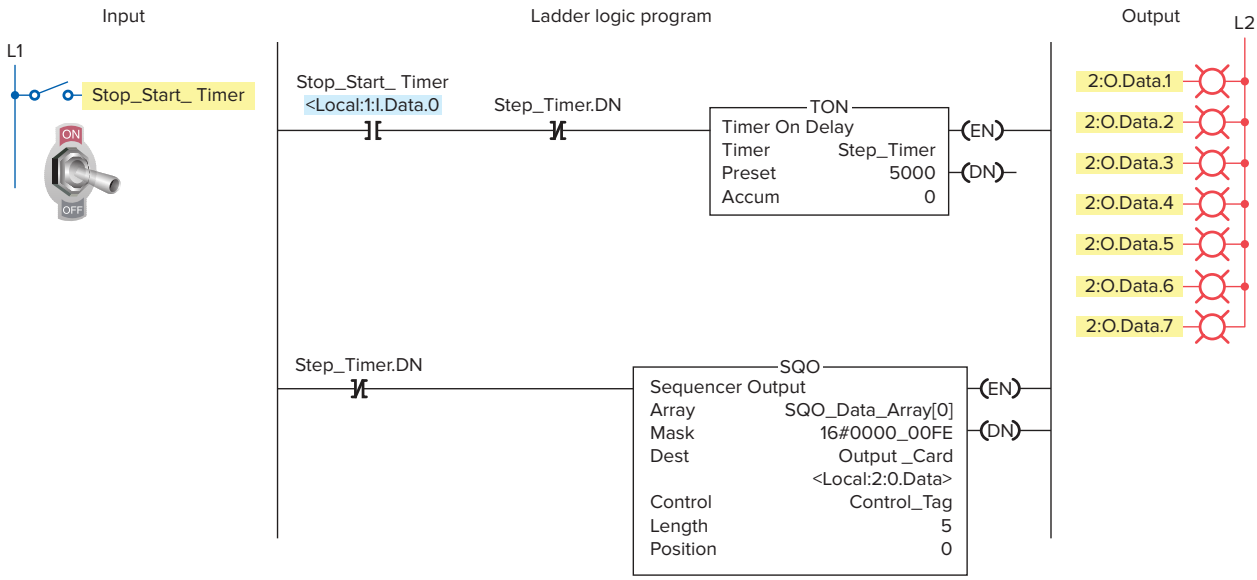
Figure 12-12 shows an example of a ControlLogix time-driven sequencer output program. The program is designed to execute a sequential process involving five steps which control several outputs. The outputs which are on for each step are shown in the table. The operation of the program can be summarized as follows:

- The array parameter SQO_Data_Array contains the desired output states for each step.
- The output states for each step are entered at the starting location SQO_Data_Array[0].
- In this application none of the outputs are energized in step 0. When the SQO instruction executes, it will be in step 0 on initial start-up.
- When the SQO instruction advances to step 5, it will return to step 1 and continue from there.
- The mask has a constant hexadecimal value of 0000_00FF which is the same as 0000 0000 0000 0000 0000 0000 1111 1111 binary bits. Each bit corresponds to one output in the SQO instruction.
- In this example the two hexadecimal Fs represent 8 binary 1s in memory. A value of 1 in the mask allows the output state from the step to be sent to the Dest.
- The DN (done) bit of the 5-second TON Step_Timer is used to trigger the SQO instruction.
- Every time the timer ACC value reaches 5 seconds, the timer DN bit changes state causing the SQO instruction to increment the Position number of the Control tag and move to the next step.
- Note that the timer DN bit also resets the ACC value of the timer to 0 and the timer starts timing to 5 seconds again.

12.3 Sequencer Programs

A sequencer program can be *event-driven* or *time-driven.* An event-driven sequencer operates similarly to a mechanical stepper switch that increments by one step for each pulse applied to it. A **time-driven** sequencer operates similarly to a mechanical drum switch that increments automatically after a preset time period.

A sequencer chart, such as the one shown in Figure 12-13, is a table that lists the sequence of operation of the outputs controlled by the sequencer instruction. These tables use a ***matrix-style*** chart format. A matrix is a two-dimensional, rectangular array of quantities. A time-driven sequencer chart usually indicates outputs on its horizontal axis and the time duration on its vertical axis. An event-driven sequencer indicates outputs on its horizontal axis and the input, or event, on its vertical axis.



Output states entered into sequencer data array

[–] SQO_Data_Array	{. . .}
[+] SQO_Data_Array[0]	2#0000_0000_0000_0000_0000_0000_0000_0000
[+] SQO_Data_Array[1]	2#0000_0000_0000_0000_0000_0000_0000_0010
[+] SQO_Data_Array[2]	2#0000_0000_0000_0000_0000_0000_0000_0110
[+] SQO_Data_Array[3]	2#0000_0000_0000_0000_0000_0000_0001_0000
[+] SQO_Data_Array[4]	2#0000_0000_0000_0000_0000_0000_1100_0000
[+] SQO_Data_Array[5]	2#0000_0000_0000_0000_0000_0000_1111_0000

Outputs which are On for each step

Outputs								Step
7	6	5	4	3	2	1	0	
								0
						On		1
					On	On		2
			On					3
On	On			On				4
On	On	On	On					5

Figure 12-12 ControlLogix time-driven sequencer output program.

An example of a time-driven sequencer with timed steps that are not all the same is shown in Figure 12-14. This sequencer program is used for automatic traffic light control at a four-way intersection. Output lights operate in a sequential fashion with variably timed steps. The system requires two SQO instructions: one for the light outputs and the other for the timed steps. Both SQOs have R6:0 for the control and 4 for the length. The first position is on for 25 seconds, the second for 5 seconds, the third for 25 seconds, and the fourth for 5 seconds.

The operation of the time-driven sequencer program can be summarized as follows:

- The bits controlling the traffic light outputs are stored in integer file #N7:0 of the first SQO instruction. The settings for the output bits for each position are entered and stored in binary table format as shown in Figure 12-15. Each word of the #N7:0 file is moved from the file by the program to the destination output word O:2 as previously described.
- The second SQO instruction sequencer file, #N7:10, contains the stored preset timer values 25, 5, 25, 5 seconds. These settings are stored in words N7:11, N7:12, N7:13, and N7:15 as illustrated in Figure 12-16. Each word of the #N7:10 file is moved by the program to the destination address T4:1.PRE, which is the preset value for the timer. The program moves information from this file to timer T4:1's preset. The mask allows the proper data to pass and blocks the unnecessary data.
- The timer cycles the two SQO instructions through their four states.
- Since both of the SQO instructions have R6:0 for control and 4 for length, they are stepped in unison to provide a sequentially timed output.

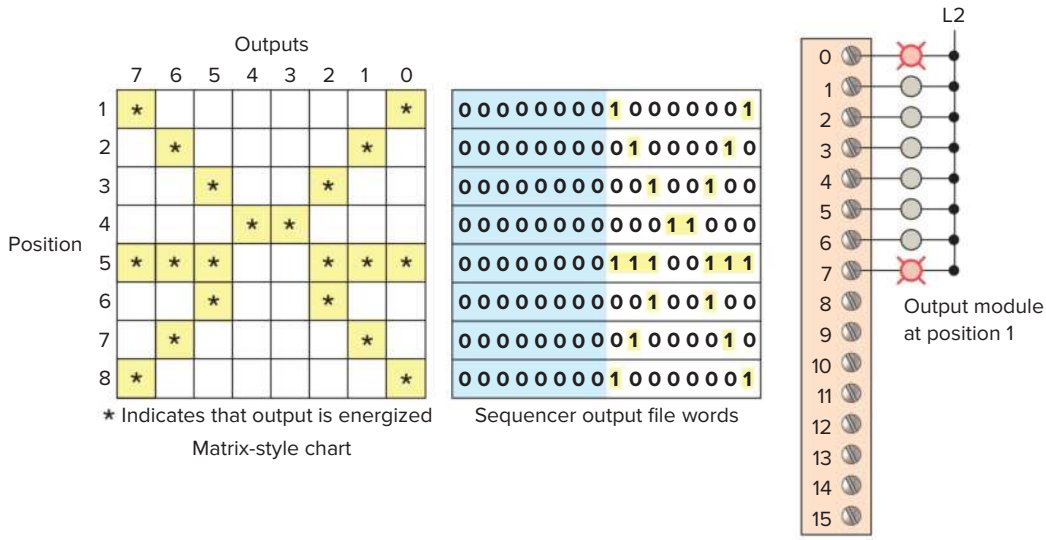


Figure 12-13 Sequencer chart.

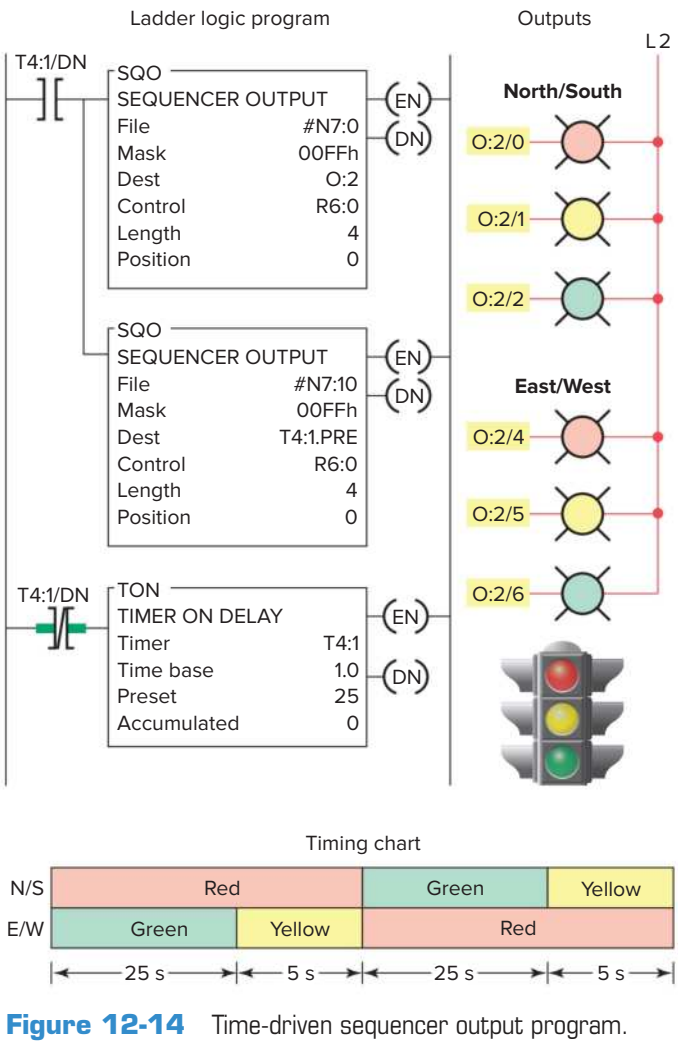


Figure 12-14 Time-driven sequencer output program.

An example of a time-driven sequencer program in which the time interval between sequencer steps is always a constant set value is shown in Figure 12-17. The operation of the program can be summarized as follows:

- The preset time of timer T4:0 is set for 3 seconds.
- The settings of the output bits for each sequencer position are entered and stored in bit file #B3:0.
- The timer is started by the closing switch SW and 3 seconds later the timer done bit is set to 1.
- As a result the timer done bit increments the SQO instruction to the next position and resets the timer.
- The destination is O:2 and all 16 bits of this word are used for outputs.
- The mask is FFFF hexadecimal or 1111111111111111 binary, which allows all 16 bits to pass through.
- As long as input SW is closed the program continues operating with 3 seconds between sequencer steps.

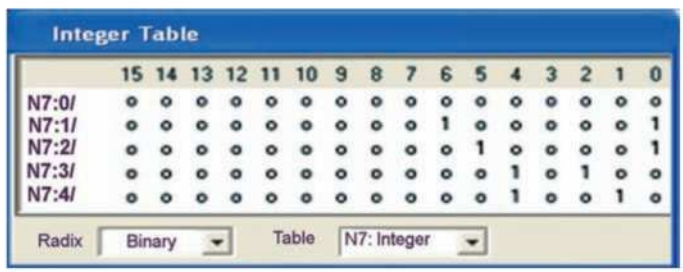


Figure 12-15 Sequencer file #N7:0 light cycle settings.
Source: Courtesy of TheLearningPit

Figure 12-16 Sequencer file #N7:10 timer settings.
Source: Courtesy of TheLearningPit

With an event-driven sequencer, the SQO instruction advances to the next step by an external pulsed input event rather than a preset time. An example of an event-driven sequencer is shown in Figure 12-18. The operation of the program can be summarized as follows:

- The sequencer SQO instruction uses two OR configured sensor switches (S1 and S2).
- Any one of the two parallel paths can make the SQO rung true.
- As each event occurs, that OR branch makes a false-to-true transition advancing the sequencer position.
- Data are copied from file #B3:0 at the bit locations through mask word, F0FF hex or 1111000011111111 binary, to the destination O:2. Mask bits are set to 1 to pass data and reset to 0 to mask data.
- Once the position reaches the last position on the true-to-false transition of the instruction the position will reset to 1.
- Note that the data in O:2 match the data in position 2 in the file, except for the data in bits 8 through 11.
- Bits 8 through 11 may be controlled from elsewhere in the program; they are not affected by the sequencer instruction because of the 0 in these bit positions in the mask.

The **sequencer compare (SQC)** instruction is an output instruction used to compare bits from an input source file to corresponding bits from data words in a sequencer file. When the pairs of bits are the same, then the found (FD) bit in the control register is set to 1. This instruction can be used to compare the status of a machine's input devices with what is required for normal operation. When the status of the input devices on the machine (on or off)

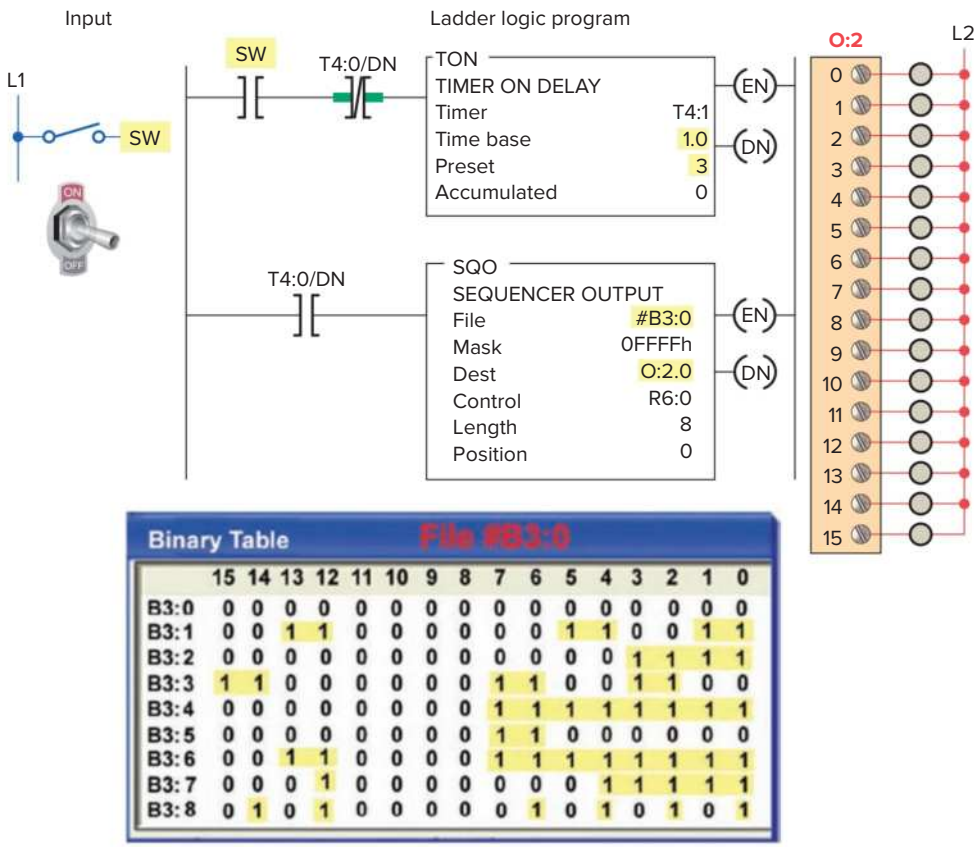


Figure 12-17 Time-driven sequencer with constant time interval between steps.
Source: Courtesy of TheLearningPit

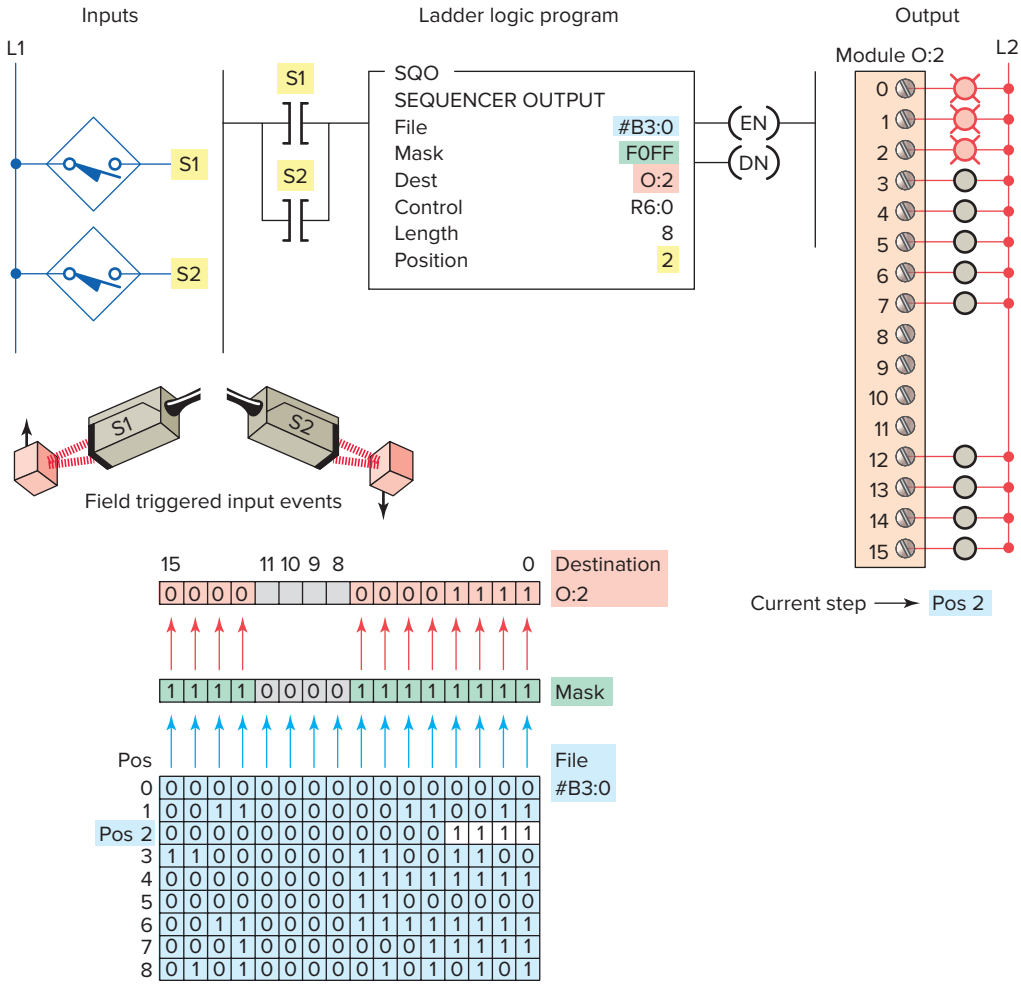


Figure 12-18 Event-driven sequencer output program.

are identical to the data stored in the sequencer file, the control found bit is set to 1.

An example of an SLC 500 sequencer compare (SQC) instruction program is shown in Figure 12-19. The operation of the program can be summarized as follows:

- The data in the highest 4 bits of the source (I:1) are compared to the data in file #B3:22.
- In this example, the highest 4 bits in I:1 match the status of the highest 4 bits in B3:25 at step position 3.
- If the pushbutton input I:1/0 is true at this point, the found (FD) bit is set, which turns output PL1 on.
- Whenever the combination of opened and closed switches connected to I:1/12, I:1/13, I:1/14, and I:1/15 is equal to the combination of 1s and 0s on a step in the sequencer reference file and the input I:1/0 is true, the PL1 output will become energized.
- The mask (F000h) allows unused bits of the sequencer instruction to be used independently. In this example, unused bit I:1/0 is used for the conditional input of the sequencer compare rung.

The *sequencer load (SQL)* instruction is used to read the PLC input module and store the input data in the sequencer file. Loading input conditions for a large number of process steps is prone to errors. In such instances the sequencer load instruction can be used to load data into a sequencer file one step at a time. For example, a robot may be jogged manually through its sequence of operation, with its input devices read at each step. At each step, the status of the input devices is written to the data file in the sequencer compare instruction. As a result, the file is loaded with the desired input status at each step, and these data are then used for comparison with the input devices when the machine is run in automatic mode.

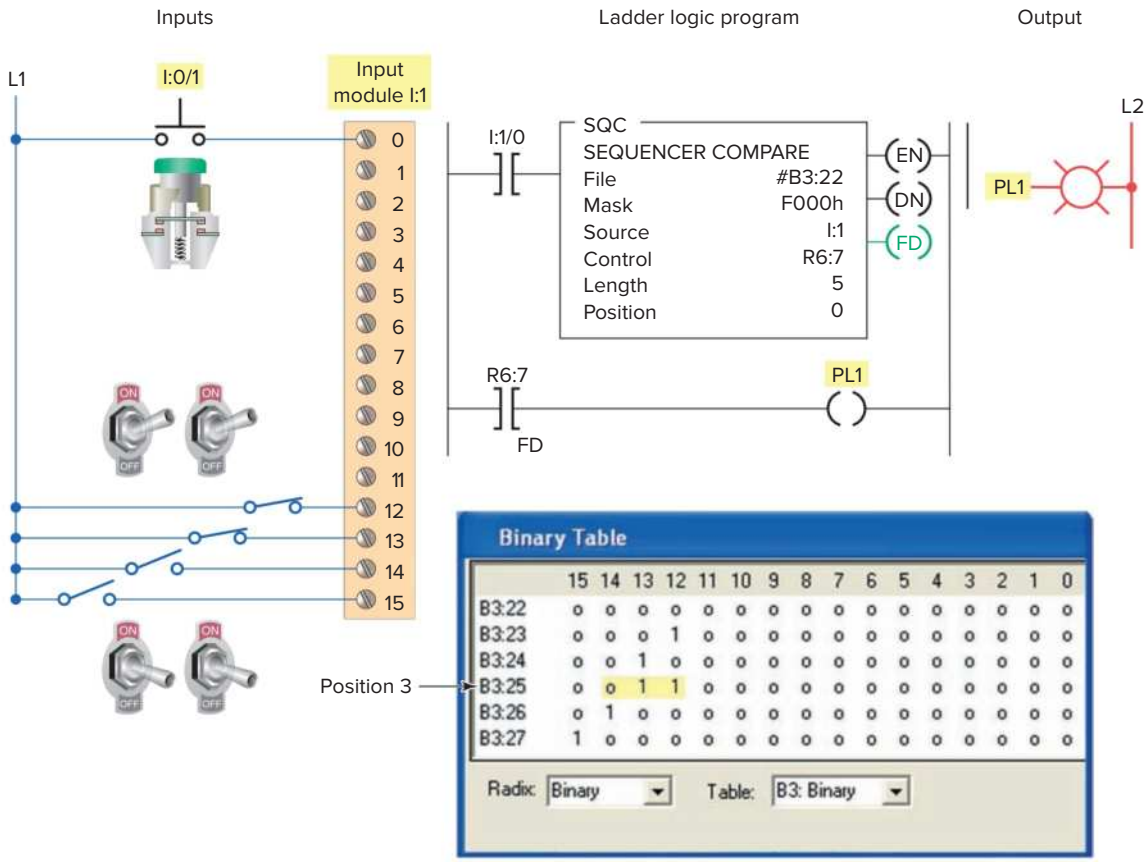


Figure 12-19 Sequencer compare (SQC) instruction program.
Source: Courtesy of TheLearningPit

An example of an SLC 500 sequencer load (SQL) instruction program is shown in Figure 12-20. The operation of the program can be summarized as follows:

- The sequencer load instruction is used to load the file and does *not* function during the machine's normal operation.
- It replaces the manual loading of data into the file with the programming terminal.
- The sequencer load instruction *does not* use a mask. It copies data directly from the source address to the sequencer file.
- When the instruction goes from false to true, the instruction indexes to the next position and copies the data.
- When the instruction has operated on the last position and has a true-to-false transition, it resets to position 1.
- It transfers data in position 0 only if it is at position 0 and the instruction is true and the processor goes from the program to run mode.
- By manually jogging the machine through its cycle, the switches connected to input I:2 of the source can be read at each position and written into the file by momentarily pressing PB1. Otherwise, the data would have to be entered into the file manually.

12.4 Bit Shift Registers

The PLC not only uses a fixed pattern of register (word) bits, but also can easily manipulate and change individual bits. A bit ***shift register*** is a register that allows the shifting of bits through a single register or group of registers. The bit shift register shifts bits serially (from bit to bit) through an array in an orderly fashion.

A shift register can be used to simulate the movement, or *track* the flow, of parts and information. We use the shift register whenever we need to store the status of an event so that we can act on it at a later time. Shift registers can shift either status or values through data files. Common applications for shift registers include the following:

- Tracking of parts through an assembly line
- Controlling of machine or process operations
- Inventory control
- System diagnostics

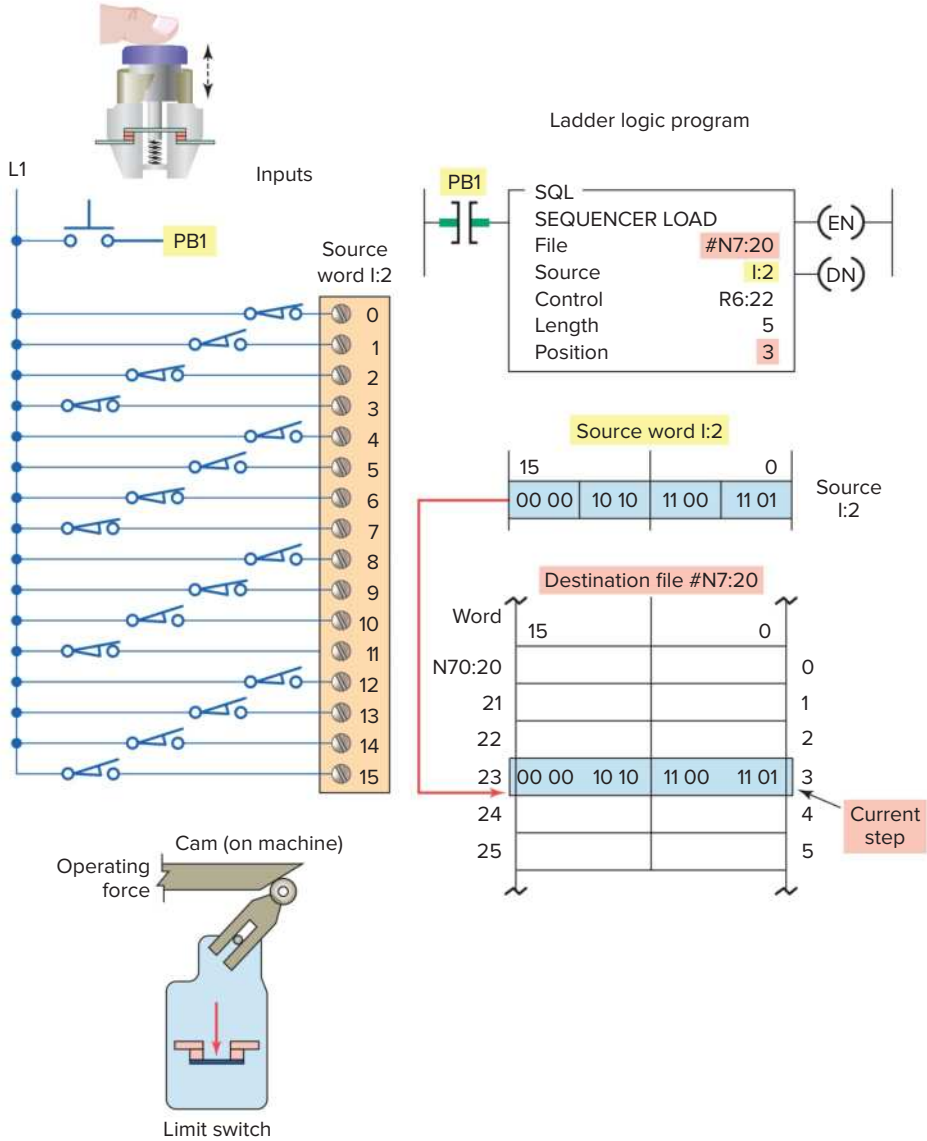


Figure 12-20 Sequencer load (SQL) instruction program.

Figure 12-21 illustrates the basic concept of a shift register. A shift pulse or clock causes each bit in the shift register to move 1 position to the right. At some point, the number of data bits fed into the shift register will exceed the register's storage capacity. When this happens, the first data bits fed into the shift register by the shift pulse are lost at the end of the shift register. Typically, data in the shift register could represent the following:

- Part types, quality, and size
- The presence or absence of parts
- The order in which events occur
- Identification numbers or locations
- A fault condition that caused a shutdown

You can program a shift register to shift status data either right or left, as illustrated in Figure 12-22, by shifting either status or values through data files. When you want to track parts on a status basis, use bit shift registers. Bit shift instructions will shift bit status from a source bit address, through a data file, and out to an unload bit, one bit at a time. There are two bit shift instructions: ***bit shift left (BSL),*** which shifts bit status from a lower address number to a higher address number through a data file, and ***bit shift right (BSR),*** which shifts data from a higher address number to a lower address number through a data file. Some PLCs provide a ***circulating shift register*** function, which allows you to repeat a pattern again and again.

When working with a bit shift register, you can identify each bit by its position in the register. Therefore,

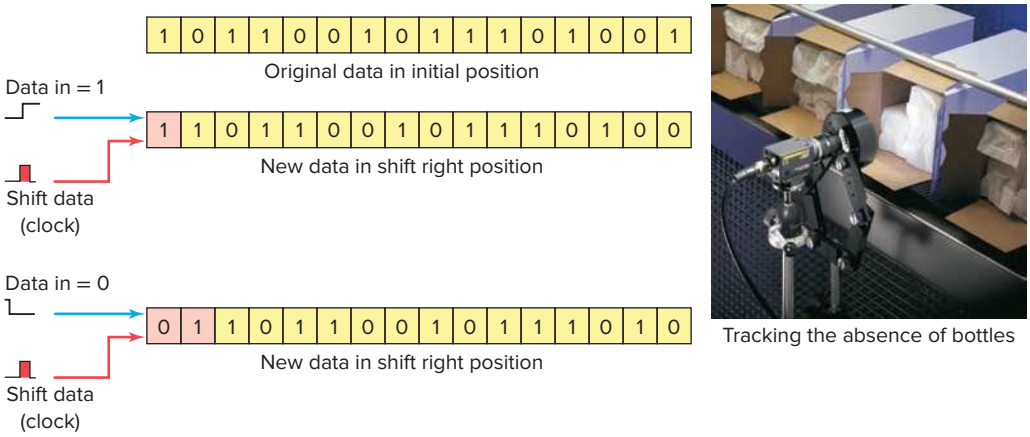


Figure 12-21 Basic concept of a shift register.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

working with any bit in the register becomes a matter of identifying the position it occupies rather than the conventional word number/bit number addressing scheme.

Figure 12-23 shows the **File Shift** menu tab and BSL and BSR instruction blocks that are part of the instruction set for the Allen-Bradley SLC 500 controllers. The commands can be summarized as follows:

BSL (Bit Shift Left)—Loads a bit of data into a bit array, shifts the pattern of data through the array to the left, and unloads the last bit of data in the array.

BSR (Bit Shift Right)—Loads a bit of data into a bit array, shifts the pattern of data through the array to the right, and unloads the last bit of data in the array.

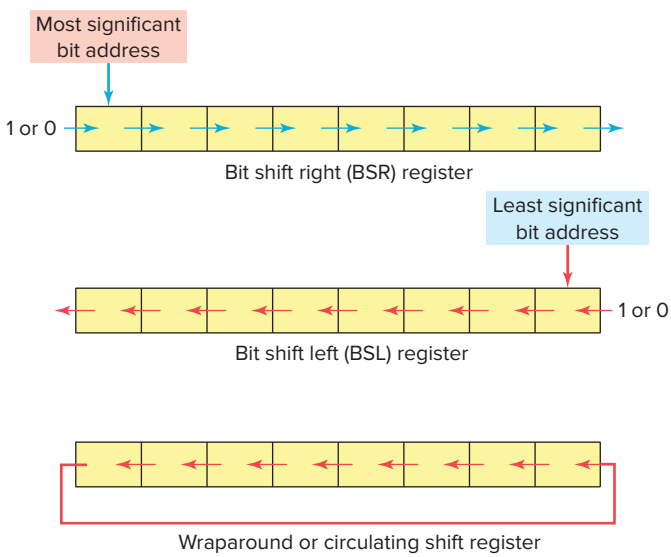


Figure 12-22 Types of shift registers.

Shift registers are useful for tracking the status or identification of a part as it moves down an assembly line. The data file used for a shift register usually is the bit file because its data are displayed in binary format, making it easier to read. BSL and BSR are output instructions that load data into a bit array one bit at a time. The data are shifted through the array, then unloaded one bit at a time.

The BSL instruction has the same operands as the BSR instruction. The difference is the direction in which the bits are indexed. A bit shift instruction will execute when its input control logic goes from false to true. To program a bit shift instruction, you need to provide the processor with the following information:

File—The address of the bit array you want to manipulate. The address must start with the # sign and at bit 0 of the first word or element. Any remaining bits in the last word of the array cannot be used elsewhere in the program because the instruction invalidates them.

Control—R data-table type. The address is unique to the instruction and cannot be used to control any other instruction. It is a three-word element that consists of the status word, the length, and the position.

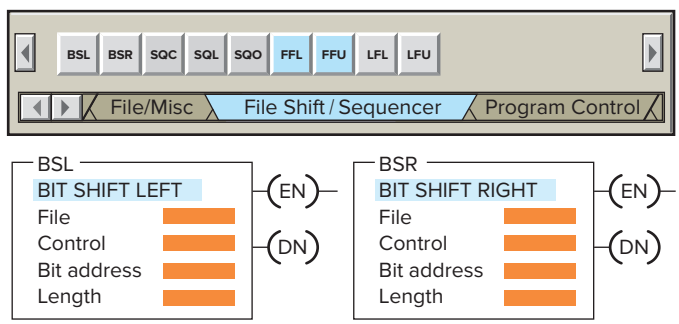


Figure 12-23 Bit shift left and bit shift right instructions.

Bit address—Is the address of the source bit. The instruction inserts the status of this bit in either the first (lowest) bit position (for the BSL instruction) or the last (highest) bit position (for the BSR instruction) in the array.

Length—Indicates the number of bits to be shifted, or the file length, in bits. The status bits of the control word are the enable, done, error, and unload bits. Their functions can be summarized as follows:

- **Enable Bit (EN)**—The enable bit follows the instructions status and is set to 1 when the instruction is true.
- **Done Bit (DN)**—The done bit is set to 1 when the instruction has shifted all bits in the file one position. It resets to 0 when the instruction goes false.
- **Error Bit (ER)**—The error bit is set to 1 when the instruction has detected an error, which can happen when a negative number is entered in the length.
- **Unload Bit (UL)**—The unload bit's status is controlled by shifting of the last bit of the file into the unload bit when the instruction is executed. It is the bit location into which the status from the last bit in the file shifts when the instruction goes from false to true. When the next shift occurs, these data are lost, unless additional programming is done to retain the data.

An example of a bit shift left (BSL) instruction program is shown in Figure 12-24. The operation of the program can be summarized as follows:

- Momentary actuation of limit switch LS causes the BSL instruction to execute.
- When the rung goes from false to true, the enable bit is set and the data block is shifted to the left (to a higher bit number) one bit position.
- The specified bit, at sensor bit address I:1/1, is shifted into the first bit position, B3:10/0.
- The last bit is shifted out of the array and stored in the unload bit, R6:0/UL.
- The status that was previously in the unload bit is lost.
- All the bits in the unused portion of the last word of the file are invalid and should not be used elsewhere in the program.
- For wraparound operation, set the position of the bit address to the last bit of the array or to the UL bit, whichever applies.

An example of a bit shift right (BSR) instruction program is shown in Figure 12-25. The operation of the program can be summarized as follows:

- Before the rung goes from false to true, the status of bits in words B3:50 and B3:51 is as shown.
- The status of the bit address, I:3/5, is a 0, and the status of the unload bit, R6:1/UL, is a 1.

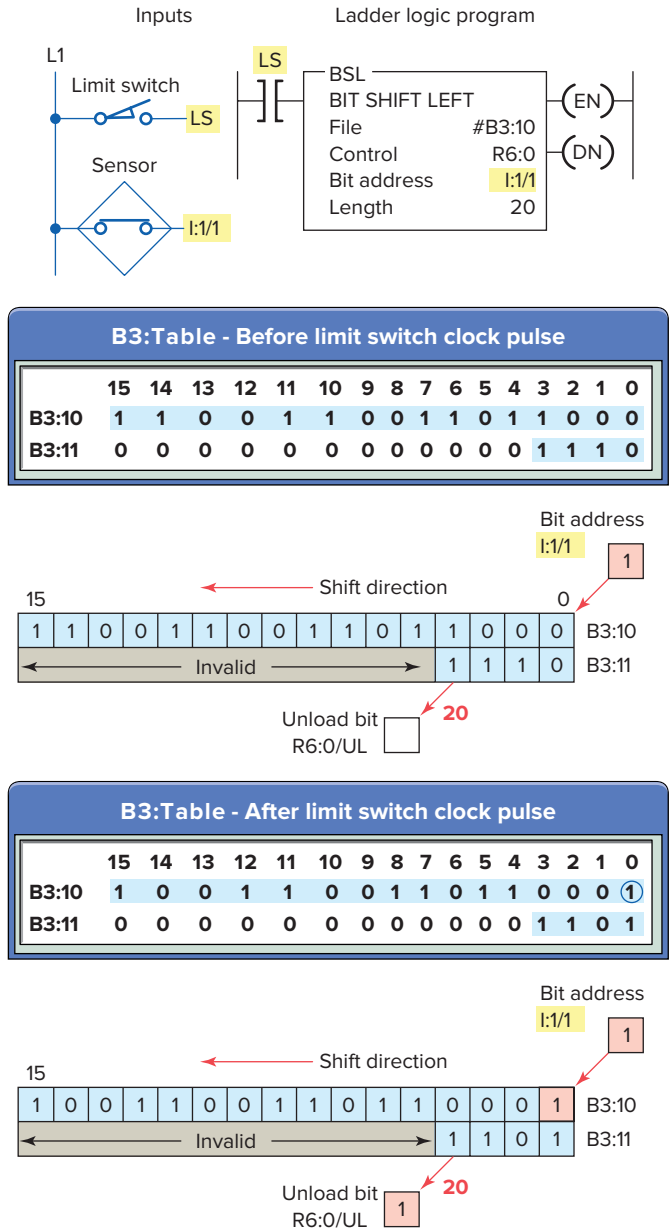


Figure 12-24 Bit shift left (BSL) instruction program.

- When limit switch LS closes, the status of the bit address, I:3/5, is shifted into B3:51/7, which is the 24th bit in the file.
- The status of all the bits in the file is shifted one position to the right, through the length of 24 bits.
- The status of B3:50/0 is shifted to the unload bit, R6:1/UL. The status that was previously in the unload bit is lost.

An example of a bit BSL instruction program with wraparound operation is shown in Figure 12-26. The clock pulse input is a fixed regular 3 second pulse–generated

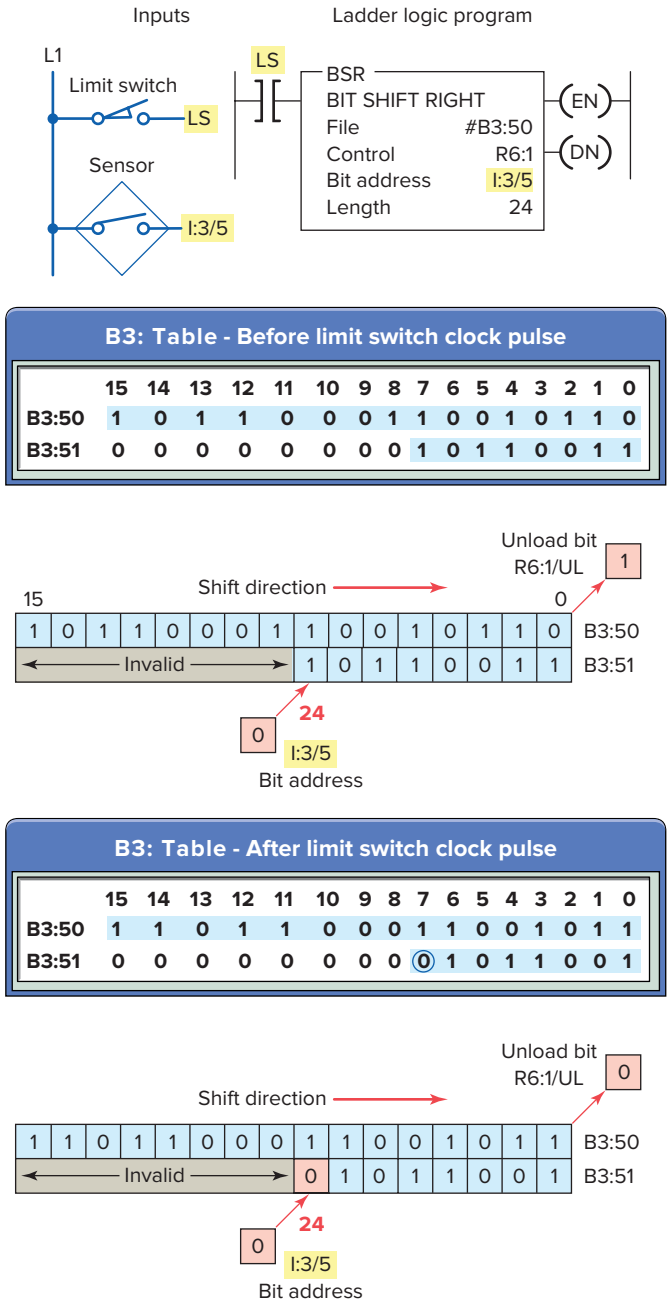


Figure 12-25 Bit shift right (BSR) instruction program.

on-delay timer T4:0. The operation of the program can be summarized as follows:

- Go to the data tables and set bit addresses B3:0/0, B3:0/1, B3:0/2 to logic 0 and bit address R6:0/UL to logic 1.
- When the PLC is then placed in run, bit B3:0/0 is set to logic 1 causing PL1 to turn on.
- Closing input switch SW starts timer T4:0 timing.
- After 3 seconds, the timer done bit is set to reset the timer accumulated time to zero and shift the logic bit 1 to the left to B3:0/1.
- This causes PL1 to turn off and PL2 to turn on.
- After another 3 seconds, the timer done bit is set once again.
- The BSL instruction shifts the bits to the left once more and causes PL2 to turn off and PL3 to turn on.
- The process continues with each of the pilot lights turned on in sequence for 3 seconds.

A shift register is often used in material handling processes where some form of binary information must be synchronized with a moving part on a conveyor. The binary information refers to any two conditions that can be assigned to the moving product—for example, the presence or absence of a part. As the part moves along the conveyer, some form of sensing device will determine which of these two categories the passing product falls into. Figure 12-27 illustrates cartons traveling on a conveyor being detected by a photoelectric sensor. The sensor that drives the data line on a shift register is fixed such that the beam detects the presence or absence of a carton. A logic 1 sensor condition state can indicate the presence of a carton, and a 0 the absence.

The process of Figure 12-28 illustrates a spray-painting operation controlled by a shift left register. As the parts pass along the production line, the shift register bit patterns represent the items on the conveyor hangers to be painted. Each file bit location represents a station on the line, and the status of the bit indicates whether or not a part is present at that station.

The program for the spray-painting operation is shown in Figure 12-29. Its operation can be summarized as follows:

- Limit switch LS1 is used to detect the hanger and limit switch LS2 the part.
- The pulse generated by the hanger-operated limit switch LS1 shifts the status of the data provided by part-detection limit switch LS2.
- The logic of this operation is such that when a part to be painted and a part hanger occur together at station 1 (indicated by simultaneous closing of LS2 and by LS1), logic 1 is input into the shift register at B3:0/0.
- This causes the SOL 1 rung to be true and the undercoat spray gun to energize.
- At station 5, a 1 appears in bit B3:0/5 of the shift register to make the SOL 2 rung true and topcoat spray gun energize.

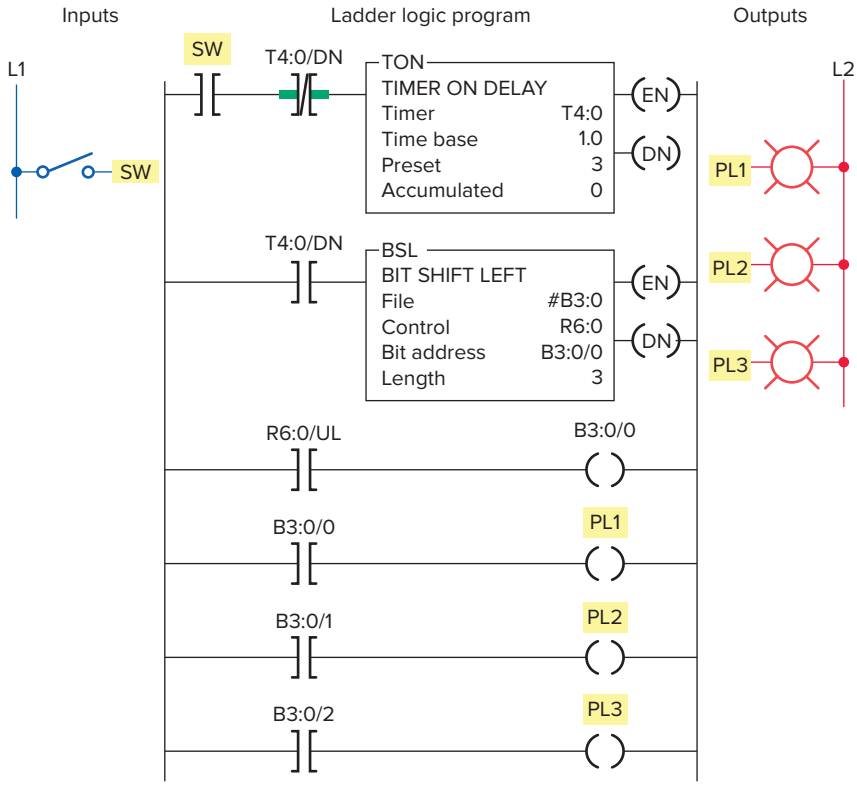


Figure 12-26 BSL instruction with a wraparound operation.

- Logic 0 in the shift register indicates that the conveyor has no parts on it to be sprayed, and it therefore inhibits the operation of the spray guns.
- Counter C5:1 counts the parts as they enter the process and counter C5:2 as they exit.

Figure 12-27 Cartons traveling on a conveyor being detected by a photoelectric sensor.
Source: Courtesy Banner Engineering Corp.

- The count obtained by the two counters should be equal when no parts are being painted.
- Whenever the two counts are equal in value the equal instruction executes to turn on pilot light PL1. This is an indication that the parts commencing the spray-painting run equal the parts that have completed it.

An example for a bit shift program used to keep track of carriers flowing through a 16-station machine is shown in Figure 12-30. The operation of the program can be summarized as follows:

- Proximity switch 1 senses a carrier, and proximity switch 2 senses a part on the carrier.
- Clock pulse generated by carrier proximity switch I:1/1 shifts the status of the data provided by part detection proximity switch I:1/2.
- When a part and carrier are sensed together, indicated by simultaneous closing of I:1/2 and I:1/1, logic 1 is input into the shift register at output O:4/0 to energize the pilot light connected to it.
- Remaining pilot lights connected to output module O:4 turn on in sequence as carriers with parts move through each station.
- They turn off or remain off as empty carriers move through.

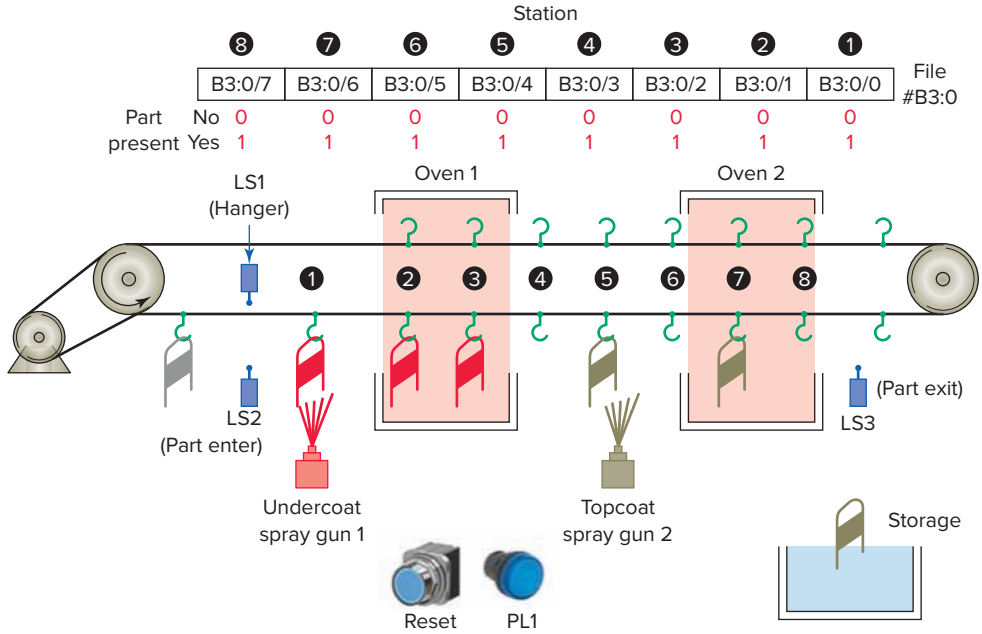


Figure 12-28 Spray-painting operation controlled by a shift left register.

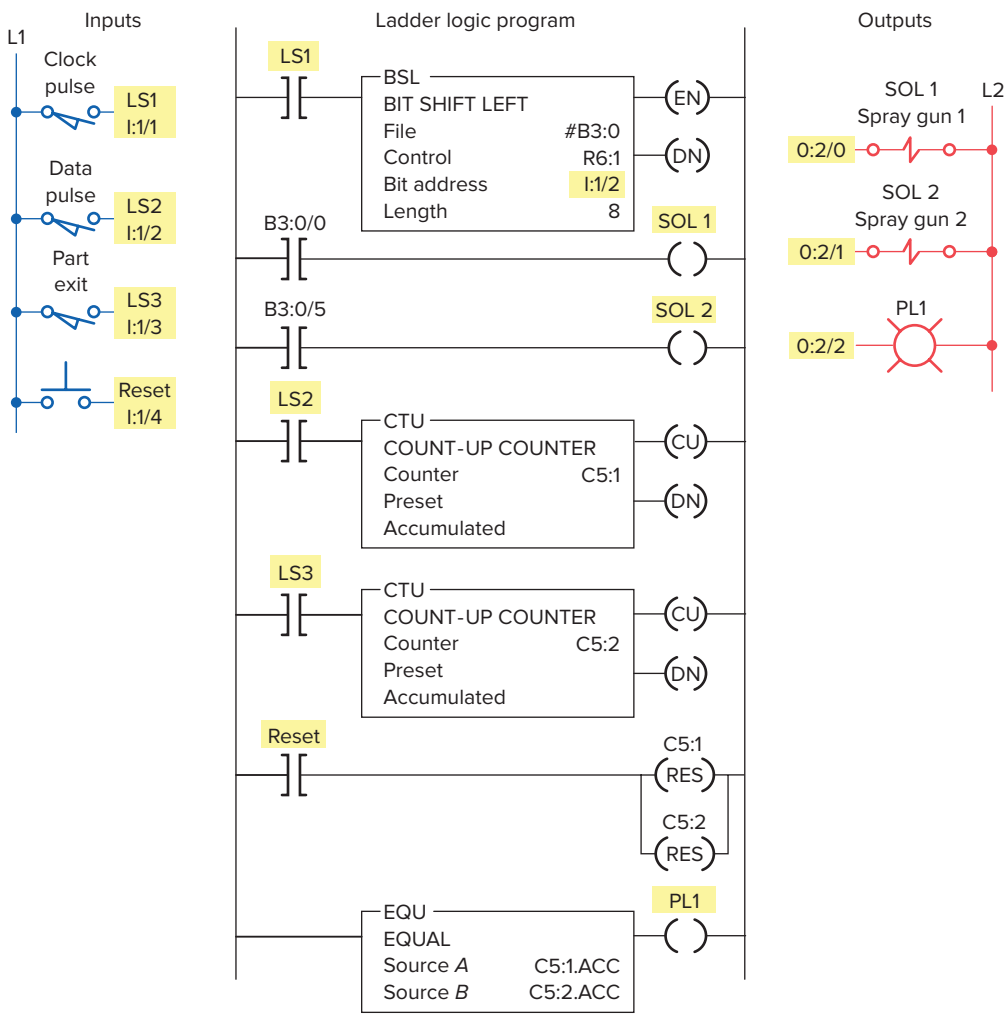


Figure 12-29 Spray-painting operation program.

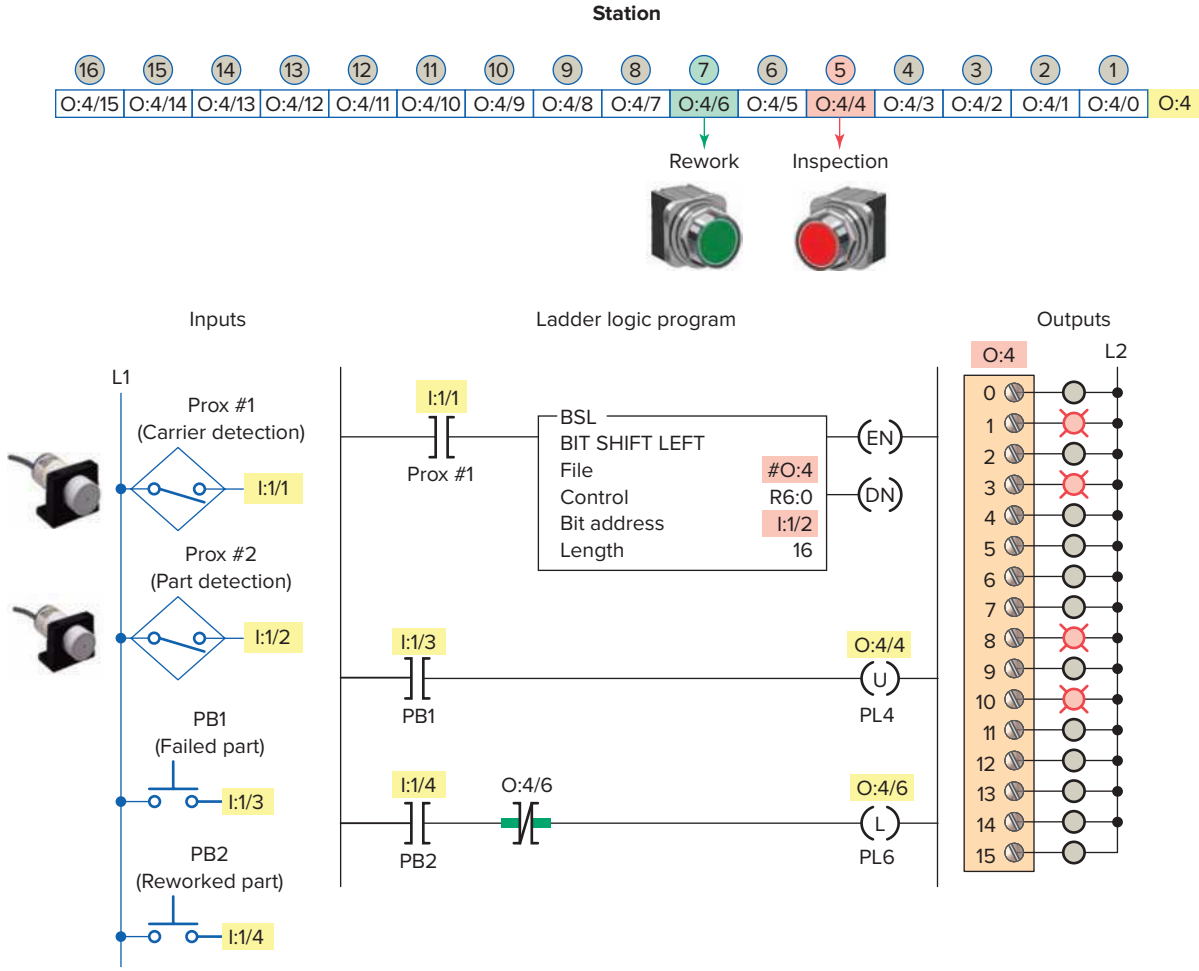


Figure 12-30 Program for tracking of carriers flowing through a 16-station machine.
Source: Photos courtesy Omron Industrial Automation, **www.ia.omron.com**.

- Station 5 is an inspection station where parts are examined.
- If the part fails, the inspectors push PB1 as they remove the part from the system, which turns output O:4/4 off.
- Rework parts can be added back into the system at station 7.
- When the operator puts a part on an empty carrier, he or she pushes PB2, turning output O:4/6 on to resume tracking.

Allen Bradley ControlLogix has two shift register instructions: Bit Shift Left (BSL) and Bit Shift Right (BSR). The BSL instruction is shown in Figure 12-31. Data are loaded into, shifted through, and unloaded from a bit array, one bit at a time when the instruction transitions from false to true. Bit shift instructions contain the following parameters.

Array—A one-dimensional array of type DINT with enough bits to accommodate the length.

Control—A tag of data type Control that stores the status bits and the size (number of bits) of the bit array.

Source Bit—A tag of type BOOL, the status of which the instruction inserts into the first bit position of the bit array on every false-to-true rung transition.

Length—The size (number of bits) of the bit array.

Figure 12-32 illustrates an application for the ControlLogix Bit Shift Left (BSL) instruction. A pass/fail inspection station is used in a conveyor system to keep track of defective products as they move down the conveyor at an equal distance. A partially completed product is moved down a conveyor to an inspector before proceeding to the final stages of manufacture. If a product is deemed defective, it is pointless to continue building it, so the part is identified as a reject. Before the part arrives at the reject bin, it must pass through three more zones of manufacture. The operation of the program can be summarized as follows:

- At the zone 1 pass/fail station an inspector examines the product before it proceeds to the final 3 stages of manufacture.

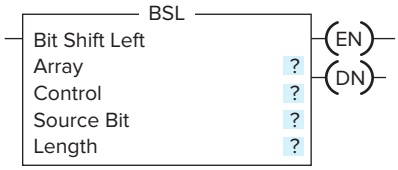
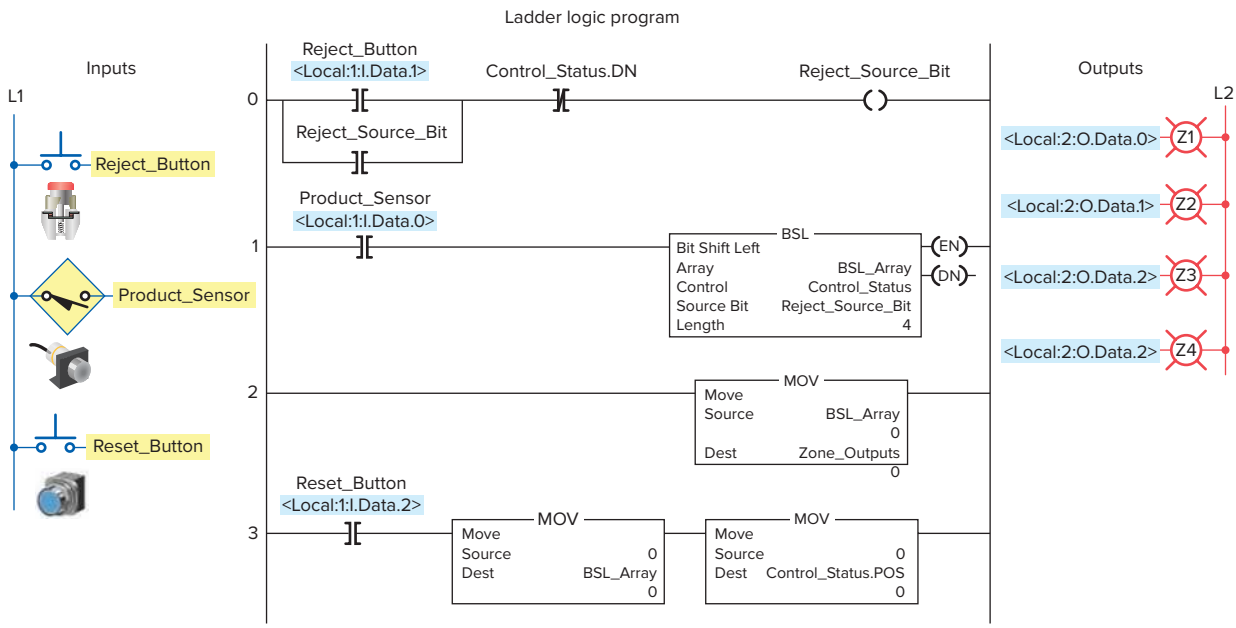


Figure 12-31 ControlLogix Bit Shift Left (BSL) instruction.

- If the product is deemed defective the inspector actuates the Reject_Button energizing the Reject_Source_Bit through its seal circuit.
- The rejected product passes by the Productor_Sensor and as a result the Reject_Source_Bit (1) is loaded into the first step of the BSL_Array which energizes the Z1 reject pilot light.
- At the same time the Control_Status.DN bit momentarily changes state to open the seal-in circuit to the Reject_Source_Bit, readying it for the next product reject.
- Each of the remaining 3 zones is equipped with a reject pilot light to warn the assembler to ignore the product if the lamp is on.
- Each successive product activating the Product_Sensor will advance the data bit one position to the left in the register, energizing the 3 remaining reject lamps in turn.
- The MOV instruction of rung 2 posts the BSL_Array contents to the four Zone_Outputs.
- Activating the Reset_Button of rung 3 at any time energizes both rung MOV instructions to reset the BSL_Array and Control_Status to 0.

12.5 Word Shift Operations

The ***first in, first out (FIFO)*** instructions are word shift operations that are similar to bit shift operations. Word shifting provides a simpler method of loading and unloading data into a file, usually called the *stack*. It is often



Tag Name	Value	Style	Data Type
⊞BSL_Array	0	Decimal	DINT
⊞Control_Status	{...}		CONTROL
⊟Zone_Outputs	0	Decimal	DINT
Zone_Outputs.0	0	Decimal	BOOL
Zone_Outputs.1	0	Decimal	BOOL
Zone_Outputs.2	0	Decimal	BOOL
Zone_Outputs.3	0	Decimal	BOOL
Product_Sensor	0	Decimal	BOOL
Reject_Button	0	Decimal	BOOL
Reject_Source_Bit	0	Decimal	BOOL
Reset_Button	0	Decimal	BOOL

Figure 12-32 ControlLogix pass/fail inspection program.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

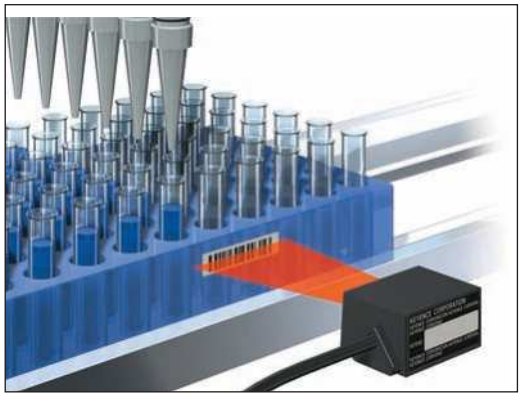


Figure 12-33 Barcode reader.
Source: Courtesy Keyence Canada Inc.

used for tracking parts through an assembly line, where parts are represented by values that have a part number or an assembly code. Figure 12-33 shows a barcode reader used for reading printed barcode data on boxes.

A bit shift register operates ***synchronously;*** because for every bit shifted in, one is shifted out. Data entered in a bit shift register must be shifted the length of the register (one position per shift pulse) before they are available to shift out.

A FIFO function operates ***asynchronously.*** Rather than shifting bits of information within a word it shifts the data from a complete word into a file or stack. Unlike the bit shift register, two separate shift pulses are required: one to shift data into the file (load) and one to shift data out of the file (unload). These two shift pulses operate independently (asynchronously) of each other. Data loaded in a FIFO can be immediately available for unload, regardless of length.

The FFL and FFU instruction are used in pairs. The FFL *loads* logic words into a user-created file called a FIFO stack. The FFU instruction is used to *unload* the words from the FIFO stack, in the same order as the words were entered. The first word entered is the first word out.

The SLC 500 FIFO load (FFL) instruction is shown in Figure 12-34. The parameters that are required to be entered in the instruction block are summarized as follows:

Source—Word address from which the data are entered into the FIFO file.

FIFO—Address of the file in which the data are entered. The address must start with a # sign.

Control—R data-table type and is the file address of the control structure. The status bits, stack length, and position are stored in this element.

Length—File length in words. Specifies the maximum number of words in the stack.

Position—Is the next available location where the instruction loads data into the stack. The first address in the stack is position 0. As each word is entered into the stack, the position counter, on both the FFL and FFU, will increment up by one. The stack is considered full when the position value equals the length. The status bits of the control word are the enable (EN), the done (DN), and the empty (EM) bits. Their functions can be summarized as follows:

- **Enable Bit (EN)**—The enable bit follows the instructions status and is set to 1 when the instruction is true.
- **Done Bit (DN)**—The done bit is set to 1when the instruction's position equals the length. When the done bit is set, the FIFO is full and does not accept any more data. Also the data in the FIFO file are not overwritten when the instruction goes from false to true.
- **Empty Bit (EM)**—The empty bit is set to 1 when all the data have been unloaded from the FIFO file.

Figure 12-35 shows the SLC 500 FIFO unload (FFU) instruction. The following parameters need to be entered in the SLC 500 FFU instruction:

FIFO—Address of the file in which the data are entered. The address must start with a # sign. When *paired* with an FFL instruction, this address is the same as the address for the FFL.

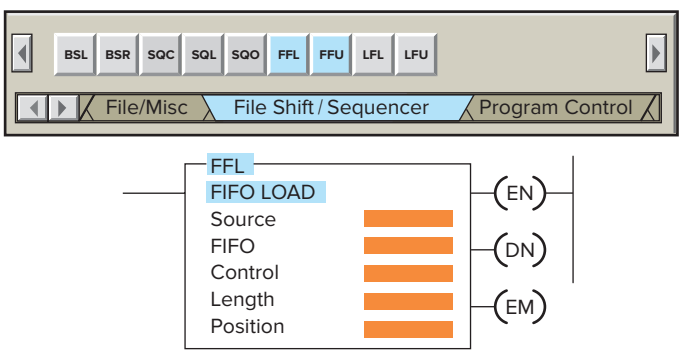


Figure 12-34 SLC 500 FIFO load (FFL) instruction.

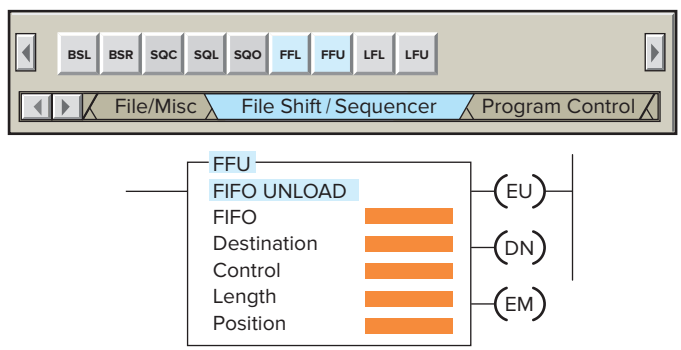


Figure 12-35 SLC 500 FIFO unload (FFU) instruction.

Destination—Address to which the FFU unloads data.

Control—R data-table type. It is a three-word element that consists of the status word, the length, and the position. When it is paired with the FFL, the control addresses are the same.

Length—File length in words. Specifies the maximum number of words in the stack.

Position—Next location from which data are unloaded when the instruction goes from false to true.

The status bits of the control word are the enable (EN), the done (DN), and the empty (EM) bits. The enable bit follows the instruction's status, the done bit is set when the instruction's position equals the length, and the empty bit is set when all the data have been unloaded from the FIFO file.

The program of Figure 12-36 is an example of how data are indexed in and out of a FIFO file using the FFL and FFU instruction pair. The operation of the program can be summarized as follows:

- The FIFO load and FIFO unload instructions share the same control element, R6:0, which may not be used to control any other instructions.
- FIFO, #N7:12, is the address of the stack. The same address is programmed for the FFL and FFU instructions.
- Data enter the FIFO file from the source address, N7:10, on a false-to-true transition of input *A*.
- Data are placed at the position indicated in the instruction on a false-to-true transition of the FFL instruction, after which the position indicates the current number of data entries in the FIFO file.

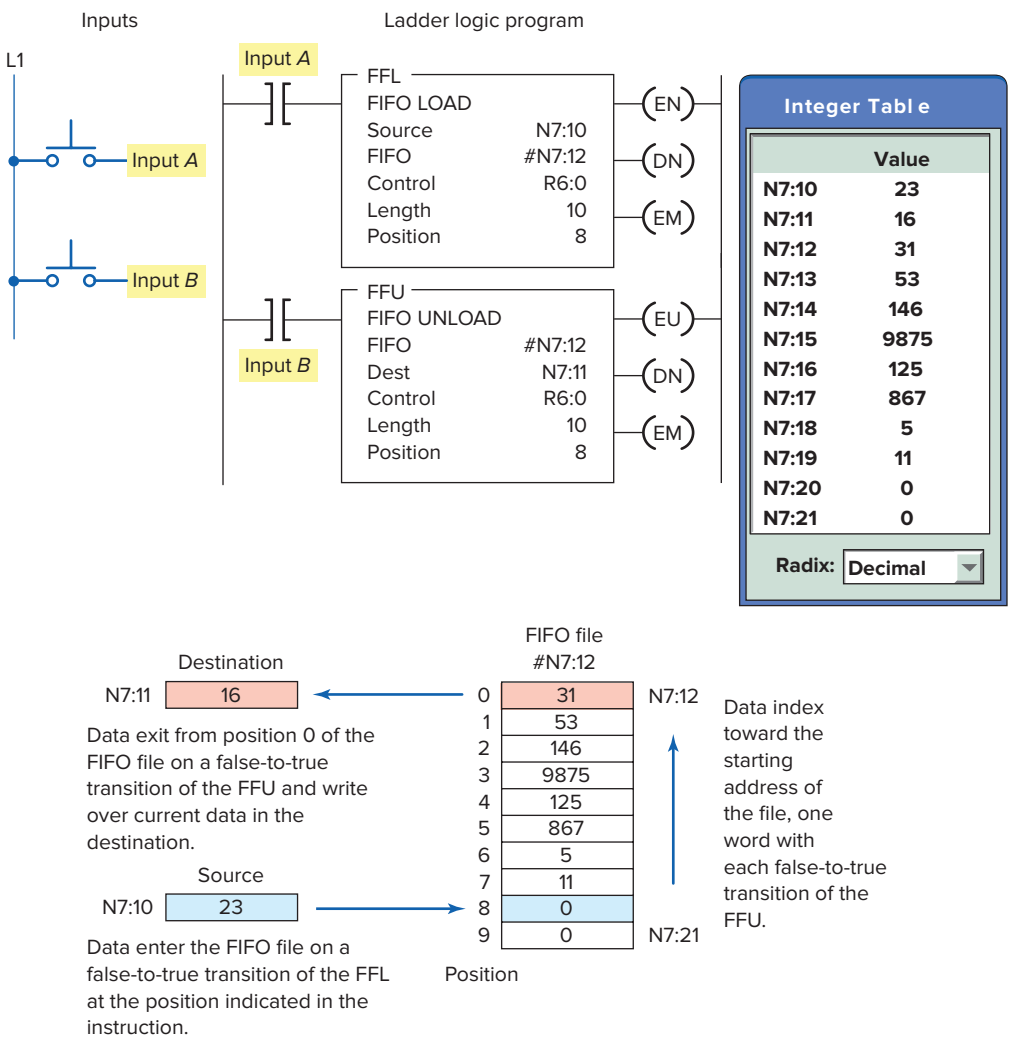


Figure 12-36 How data are indexed in and out of a FIFO file.

- The FIFO file fills from the beginning address of the FIFO file and indexes to one higher address for each false-to-true transition of input *A*.
- A false-to-true transition of input *B* causes all data in the FIFO file to shift one position toward the starting address of the file, with the data from the starting address of the file shifting to the destination address, N7:11.

The FIFO instruction is often used for inventory control. One example is where different parts need to be removed from inventory to be used in production. Each part is assigned a unique code, which is loaded into a FIFO stack, and parts are removed in the order prescribed by the stack. This type of control ensures that the oldest part in the inventory is used first as the first part entered is the first part removed.

The opposite principle—where the last data to be stored are the first to be retrieved—is known as *LIFO (Last In, First Out)*. The LIFO instruction inverts the order of the data it receives by outputting the last data received first and the first data received last. A useful analogy is a pile of work on your desk. As new work arrives you drop it on the top of the stack. If your stack is LIFO, you pick your next job from the top of the pile. If your stack is FIFO, you pick your work from the bottom of the pile. Figure 12-37 shows how the FIFO and LIFO operations work for container stacking operations.

The difference between FIFO and LIFO stack operation is that the LIFO instruction removes data in the reverse of the order they are loaded (last in, first out). An example of

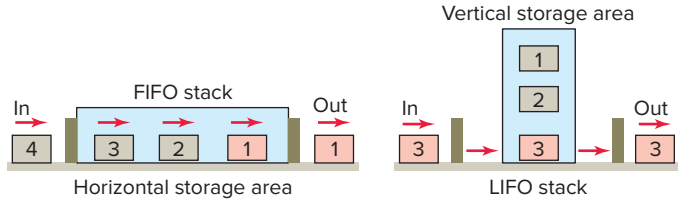


Figure 12-37 FIFO and LIFO container stacking operations.

the LIFO instruction pair is shown in Figure 12-38 and the operation of this function can be summarized as follows:

- The load and unload of the LIFO stack operates similarly to that of the FIFO stack, except that the last word in the LIFO stack is the first word that is unloaded from the stack.
- Words can be added to the LIFO stack without disturbing the words already loaded on the stack.
- Otherwise, LIFO instructions operate the same as FIFO instructions.

Allen Bradley ControlLogix programming with FIFO and LIFO functions operates similarly to the SLC 500 instructions except that tags and arrays are used in the parameter definitions. The program of Figure 12-39 is an example of the use of a ControlLogix FIFO instruction pair as part of a data stack operation. The operation of the program can be summarized as follows.

- The thumbwheel switch input is used to set the decimal number.

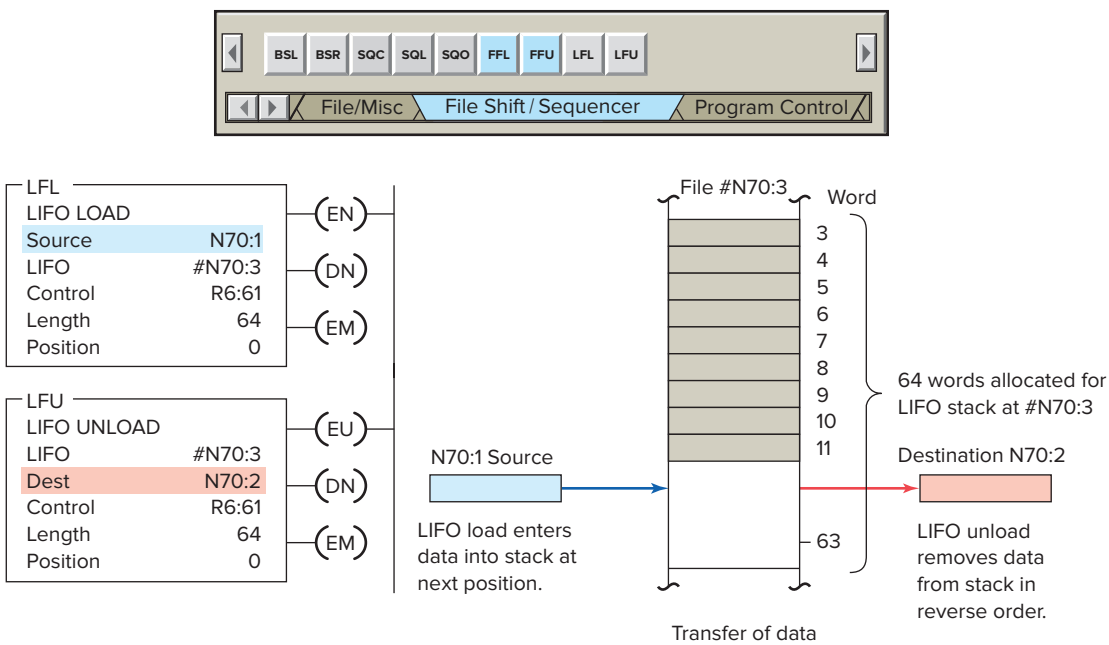


Figure 12-38 LIFO instruction pair.

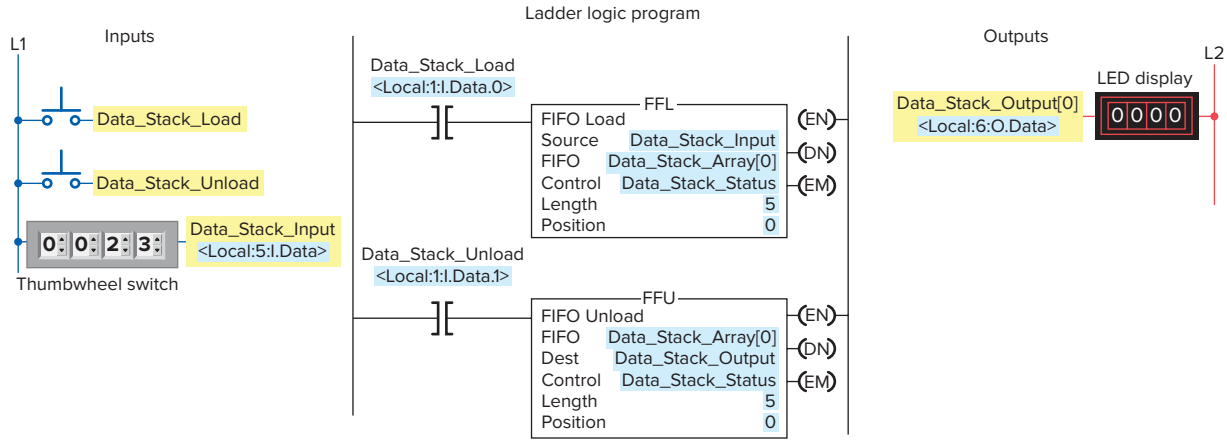


Figure 12-39 ControlLogix FIFO instruction pair.

- The data stack is capable of containing 5 words.
- It takes values from the input thumbwheel switch and stores them in the data stack array.
- The values can be pulled from the stack in a FIFO order and sent to the stack output location.
- The Data_Stack_Load input pushbutton is used to load the decimal numbers into the array.
- The output module is used to display the decimal numbers and represents the destination address.
- The Data_Stack_Unload input pushbutton is used to trigger the FIFO unload operation.

CHAPTER 12 REVIEW QUESTIONS

1. Describe the operation of a drum switch.
2. What type of operations are sequencers most suitable for?
3. Why are PLC sequencers easier to program than PLC discrete outputs?
4. Answer the following with regard to an SLC 500 PLC sequencer output instruction:
   a. Where is the information for each sequencer step entered?
   b. What is the function of the output word?
   c. Explain the transfer of data that occurs as the sequencer is advanced through its various steps.
5. What is the function of the file of a sequencer?
6. What is the function of the mask in the sequencer instruction?
7. What is the relationship between the length and the position in a sequencer instruction?
8. What output and step programming limits may be placed on sequencer instructions?
9. Sequencer instructions are usually retentive. Explain what this means.
10. Compare the operation of an event-driven and a time-driven sequencer.
11. Explain the function of a sequencer compare instruction.
12. What is the primary application in which an SQL instruction is used?
13. Explain the function of a sequencer load instruction.
14. How does a bit shift register manipulate individual bits?
15. List four common applications for bit shift registers.
16. When using a sensor as the input to the bit address of a BSL instruction, what is its function?
17. Compare the operation of the BSL and BSR bit shift instructions.
18. A bit shift register is said to operate in a synchronous manner. Explain what this means.
19. What is the function of the unload bit in a BSL instruction?
20. What is the function of the unload bit in a BSR instruction?
21. A first in, first out word shift register operates in an asynchronous manner. Explain what this means.
22. Why are both FFL and FFU instructions needed to perform a FIFO function?
23. Compare the operation of a FIFO register and a LIFO register.

CHAPTER 12 PROBLEMS

1. Construct an equivalent sequencer data table for the four steps of the mechanical drum-operated sequencer drawn in Figure 12-40.
2. Answer the following with reference to the sequencer file #B3:0 shown in Figure 12-41:
   a. Assume that output bit addresses O:2/0 through O:2/15 are controlling associated output pilot

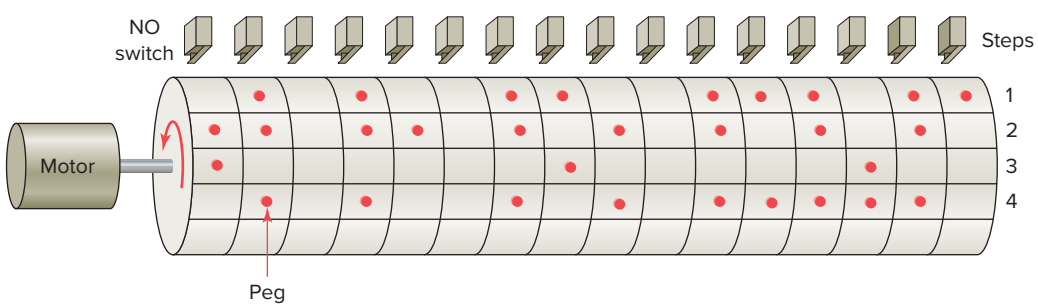


Figure 12-40 Drum-operated sequencer for Problem 1.

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Output O:2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
																	Positions
B3:0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Start
B3:1	1	1	0	1	1	0	1	1	0	1	1	0	0	0	1	1	Step 1
B3:2	0	0	1	0	0	1	0	0	1	0	0	1	1	1	0	0	Step 2
B3:3	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	Step 3
B3:4	1	1	1	1	0	0	0	0	1	1	1	1	0	0	0	0	Step 4

Figure 12-41 Sequencer file for Problem 2.

lights PL1 through PL16. State the status of each light for steps 1 through 4.

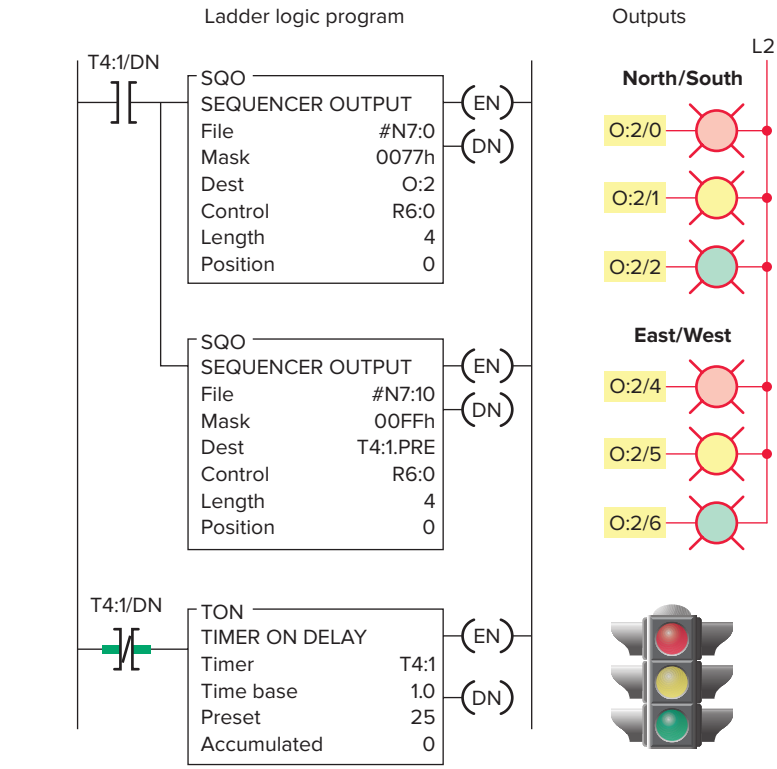
b. Which output bit addresses could be masked and which could not? Why or why not?

c. State the status of each bit of output word O:2 for step 3 of the sequencer cycle.

3. Answer each of the following with reference to the timer-driven sequencer program shown in Figure 12-42:

a. How many bit outputs are controlled by this sequencer?

b. What is the address of the word that controls the outputs?



Integer Table

	Value
N7:10	0
N7:11	25
N7:12	5
N7:13	25
N7:14	5

Radix: Decimal

Integer Table

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
N7:0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
N7:1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1
N7:2	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1
N7:3	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0
N7:4	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0

Radix: Binary Table: N7:Integer

Figure 12-42 Timer-driven sequencer program for Problem 3.

c. What is the address of the sequencer file that sets the states for the outputs?
d. What is the address of the sequencer file that contains the preset timer values?
e. For what length of time is the red light programmed to be on?
f. For what length of time is the green light programmed to be on?
g. For what length of time is the yellow light programmed to be on?
h. What is the time required for one complete cycle of the sequencer?
i. Assume that the decimal value stored in N7:13 is changed to 35. Outline the changes that this new value will have on the timing of the traffic lights.

4. Answer each of the following with reference to the event-driven sequencer program shown in Figure 12-43:
a. When does the sequencer advance to the next step?
b. Assume that the sequencer is at position 2, as shown; what bit outputs will be on?
c. Assume that the sequencer is stepped to position 8; what bit outputs will be on?
d. Assume that the sequencer is at position 8 and a true-to-false transition of one of the inputs occurs. What happens as a result?

5. Using whatever PLC sequencer output instruction you are most familiar with, develop a program that will operate the cylinders in the desired sequence. The time between each step is to be 3 seconds. The desired sequence of operation will be as follows:
- All cylinders to retract.
- Cylinder 1 advance.
- Cylinder 1 retract and cylinder 3 advance.
- Cylinder 2 advance and cylinder 5 advance.
- Cylinder 4 advance and cylinder 2 retract.
- Cylinder 3 retract and cylinder 5 retract.
- Cylinder 6 advance and cylinder 4 retract.
- Cylinder 6 retract.
- Sequence to repeat.

6. Using whatever PLC sequencer output instruction you are most familiar with, develop a program to implement an automatic car-wash process. The process is to be event-driven by the vehicle, which activates various limit switches (LS1 through LS6) as it is pulled by a conveyor chain through the car-wash bay. Design the program to operate the car wash in the following manner:
- The vehicle is connected to the conveyor chain and pulled inside the car-wash bay.
- LS1 turns on the water input valve.

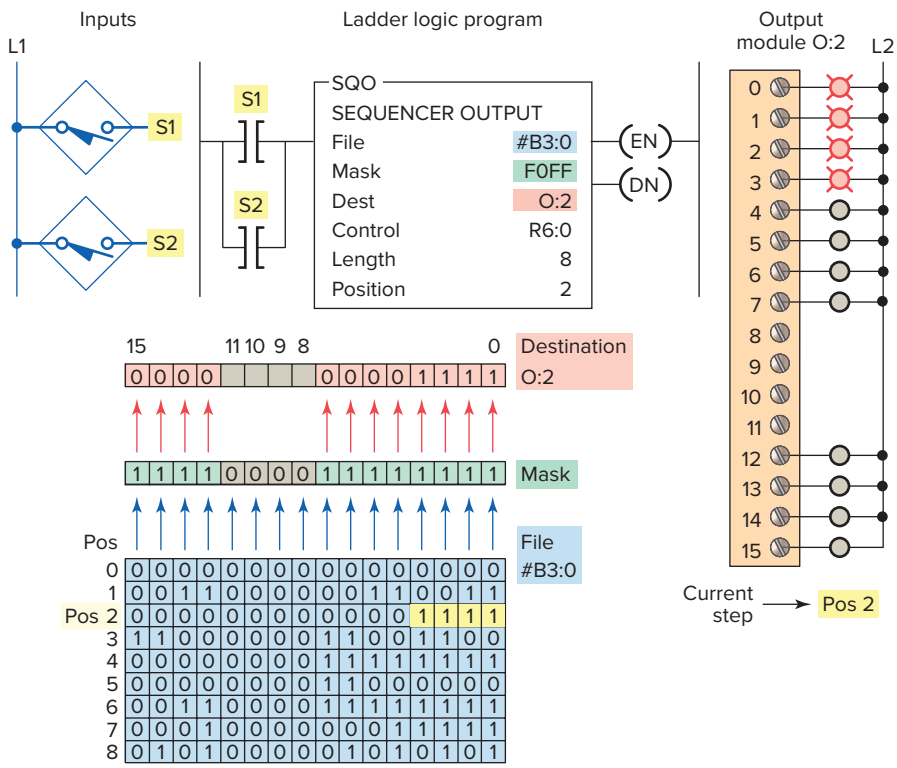


Figure 12-43 Event-driven sequencer program for Problem 4.

- LS2 turns on the soap release valve, which mixes with the water input valve to provide a wash spray.
- LS3 shuts off the soap valve, and the water input valve remains on to rinse the vehicle.
- LS4 shuts off the water input valve and activates the hot wax valve, if selected.
- LS5 shuts off the hot wax valve and starts the air-blower motor.
- LS6 shuts off the air blower. The vehicle exits the car wash.

7. A product moves continuously down an assembly line that has four stations, as shown in Figure 12-44.

- The product enters the inspection zone, where its presence is sensed by the proximity switch.
- The inspector examines it and activates a reject button if the product fails inspection.
- If the product is defective, reject status lights come on at stations 1, 2, and 3 to tell the assembler to ignore the part.

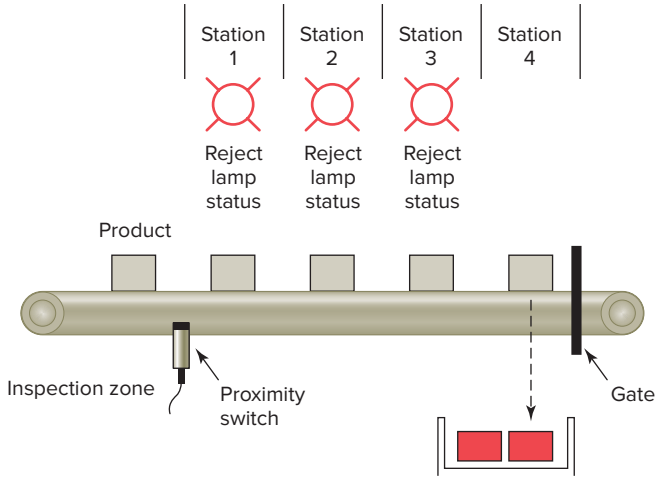
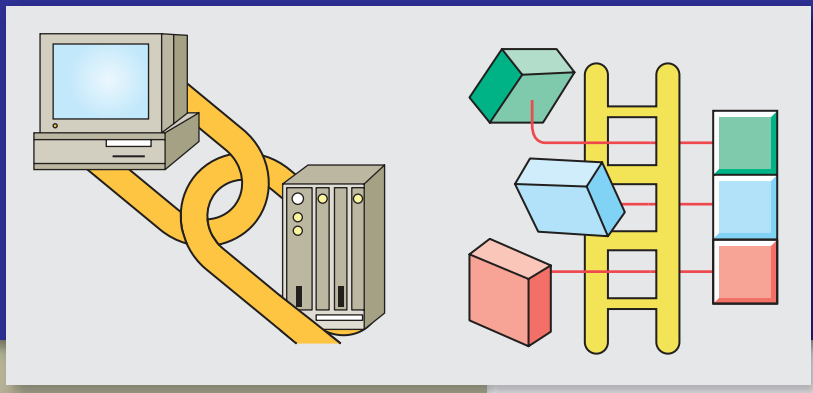


Figure 12-44 Assembly line program for Problem 7.

- When a defective part reaches station 4, a diverter gate is activated to direct that part to a reject bin.
- Using whatever PLC bit shift register you are most familiar with, develop a program to implement this process.

13

PLC Installation Practices, Editing, and Troubleshooting



Chapter Objectives

After completing this chapter, you will be able to:

- Outline and describe requirements for a PLC enclosure
- Identify and describe noise reduction techniques
- Describe proper grounding practices and preventive maintenance tasks associated with PLC systems
- List and describe specific PLC troubleshooting procedures

This chapter discusses guidelines for the installation, maintenance, and troubleshooting of a PLC-controlled system. The chapter gives you information on proper grounding that ensures personal safety as well as correct operation of equipment. Unique troubleshooting procedures that apply specifically to PLCs are listed and explained.

13.1 PLC Enclosures

A PLC system, if installed properly, should give years of trouble-free service. The design of PLCs includes a number of rugged features that allow them to be installed in almost any industrial environment. However, problems can occur if the system is not installed properly.

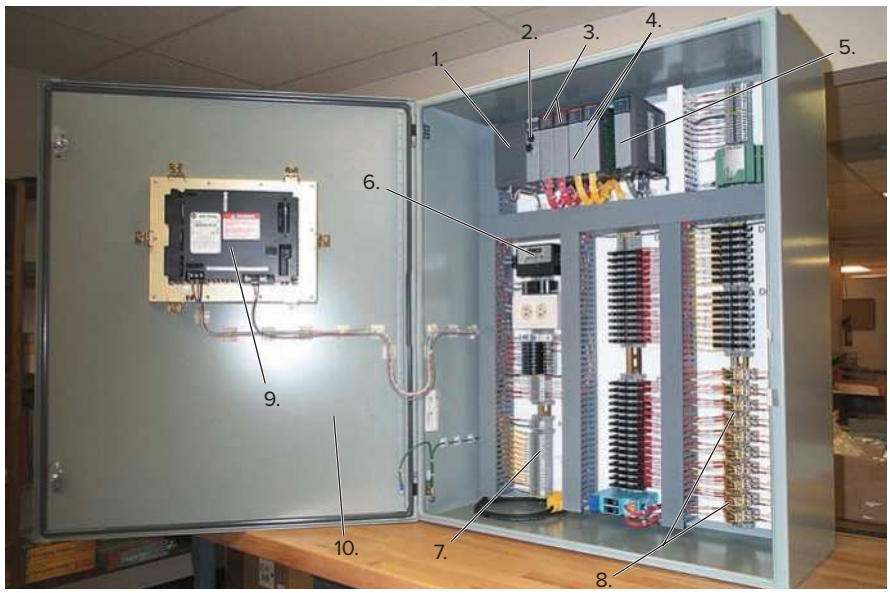
Programmable logic controllers (PLCs) require protection against temperature extremes, humidity, dust, shock, and vibration or corrosive environments. For these reasons, PLCs are generally mounted within a machine or in a separate ***enclosure*** as shown in Figure 13-1.

An enclosure is the chief protection from atmospheric conditions. The National Electrical Manufacturers Association (NEMA) has defined enclosure types, based on the degree of protection an enclosure will provide. For most solid-state control devices, a NEMA 12 enclosure is recommended. This type of enclosure is for general-purpose areas and is designed to be dust-tight. Typically, metal enclosures are used because metal enclosures provide shielding that helps minimize the effects of electromagnetic radiation that may be generated by surrounding equipment.

Every PLC installation will dissipate heat from its power supplies, local I/O racks, and processor. This heat accumulates in the enclosure and must be dissipated from it into the surrounding air. Excessive heat can cause erratic operation of the PLC or PLC failure. For many applications, normal convection cooling will keep the controller components within the specified temperature operating range. Proper spacing of components that provides adequate room within the enclosure is usually sufficient for heat dissipation. The temperature inside the enclosure must not exceed the maximum operating temperature of the controller (typically 60°C maximum). Additional cooling provisions, such as a fan or blower, may be required where high internal or ambient temperatures are encountered. PLCs are always mounted horizontally with the name of the manufacturer facing out and right-side up, as illustrated in Figure 13-2. Vertical mounting is not recommended due to thermal considerations.

Figure 13-2 PLCs are always mounted horizontally.
Source: Courtesy Rogers Machinery Company, Inc.

A hardwired electromechanical ***master control relay (MCR)*** is normally included as part of the wiring for a



1. Power supply
2. PLC (programmable logic controller)
3. Digital input cards
4. Digital output cards
5. Analog input cards
6. Transient surge protectors
7. Circuit breakers
8. Relay switches
9. Operator interface terminal
10. NEMA 12 enclosure

Figure 13-1 Typical PLC control panel enclosure.
Source: Courtesy Aaron Associates.

PLC system. The master control relay provides a means of de-energizing the entire circuit that *is not* dependent on software. The internally programmed MCR of a PLC is not sufficient to meet safety requirements. The hardwired MCR is connected to interrupt power to the I/O rack in the event of an emergency, but still allow power to be maintained at the processor. Figure 13-3 shows the typical wiring for an AC power distribution with a master control

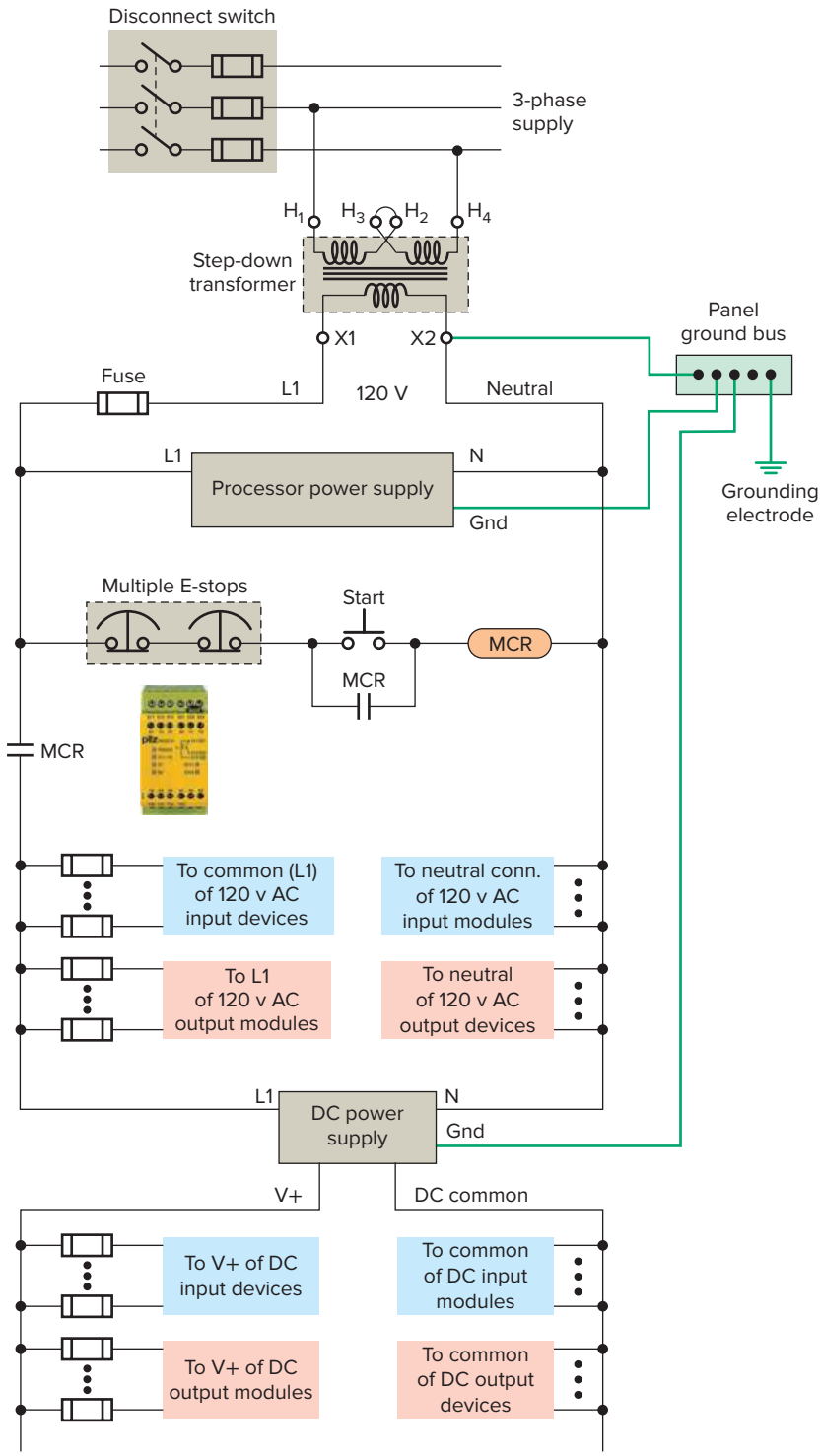


Figure 13-3 Typical wiring for an AC power distribution with a master control relay.
Source: Courtesy Pilz GmbH & Co. KG.

relay. The operation of the circuit can be summarized as follows:

- A power disconnect switch is provided so that, when required, the PLC can be serviced with the power off.
- The step-down transformer provides isolation from the main power distribution system and decreases the voltage to the 120 V required for the controller power supplies and DC power supplies.
- The momentary start button is pressed to energize the master control relay.
- Pressing any one of the emergency-stop switches de-energizes the master control relay and thus de-energizes the I/O devices.
- Power to the processor of the PLC remains on so status LEDs can continue to provide up-to-date information.
- Emergency stop buttons use normally closed contacts wired in series for fail-safe operation. In the event a wire is broken or comes off a terminal, the MCR relay is de-energized and power is removed.

13.2 Electrical Noise

Electrical noise, also called **electromagnetic interference, or EMI,** is unwanted electrical signals that produce undesirable effects and otherwise disrupt the control system circuits. EMI may be either radiated or conducted. *Radiated* noise originates from a source and travels through the air while *conducted* noise travels on an actual conductor, such as a power line.

When the PLC is operated in a noise-polluted industrial environment, special consideration should be given to possible electrical interference. To increase the operating noise margin, the controller should be located away from noise-generating devices such as large AC motors and high-frequency welders. Malfunctions resulting from noise are temporary occurrences of operating errors that can result in hazardous machine operation in certain applications. Noise usually enters through input, output, and power supply lines. Noise may be coupled into these lines by an electrostatic field or through electromagnetic induction. The following reduce the effect of electrical interference:

- Manufacturer design features
- Proper mounting of the controller within an enclosure
- Proper equipment grounding

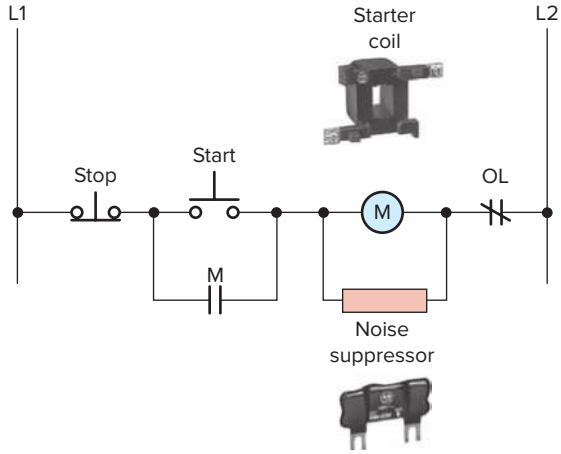


Figure 13-4 Motor starter noise suppression.
Source: Images Courtesy of Rockwell Automation, Inc.

- Proper routing of wiring
- Proper suppression added to noise-generating devices

Noise suppression is normally needed for **inductive loads** such as relays, solenoids, and motor starters when operated by hard contact devices such as pushbuttons or selector switches. When inductive loads are switched off, high transient voltages are generated that if not suppressed can reach several thousand volts. Figure 13-4 illustrates a typical noise suppression circuit that is used to suppress the high voltage spikes generated when a motor starter coil is de-energized.

Lack of surge suppression on inductive loads may contribute to processor faults and sporadic operation. RAM can be corrupted (lost), and I/O modules can appear faulty or can reset themselves. When inductive devices are energized or de-energized, they can cause an electrical pulse to be back-fed into the PLC system. The back-fed pulse, when entering the PLC system, can be mistaken by the PLC for a computer pulse. It takes only one false pulse to create a malfunction of the orderly flow of PLC operational sequences.

Proper routing of field power and signal wiring to the PLC enclosure as well as inside the enclosure helps to cut down on electrical noise (also known as cross-talk interference). The following are some general guidelines for PLC wire routing:

- Use the shortest possible wire runs for I/O signals.
- When possible, conductors that are run from the PLC enclosure to another location should be in a metal conduit as the metal can serve as a shield against EMI.
- *Never* run signal wiring and power wiring in the same conduit.
- Segregate I/O wiring by signal type. Route AC and DC I/O signal wires in separate wireways.

Figure 13-5 Heat-shrinkable wire identification sleeves.
Source: Courtesy Tyco Electronics Ltd.

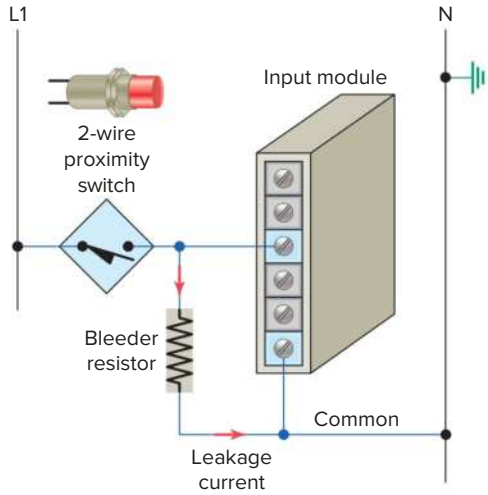


Figure 13-6 Bleeder resistor connection for input sensors.

- Low-level signal conductors such as thermocouples and serial communications should be run as shielded twisted pair and routed separately.
- A fiber optic system, which is totally immune to all kinds of electrical interference, can also be used for signal wiring.

An important part of a PLC installation is clearly **identifying** each wire to be connected and the terminal to which it is connected. A reliable labeling method, such as the heat-shrinkable wire identification sleeves shown in Figure 13-5, should be used to label each wire. Wiring connectors for input/output modules usually include spaces for labels used for identifying each I/O address and device connected. Proper wire and terminal identification will simplify the installation and aid in troubleshooting and maintenance.

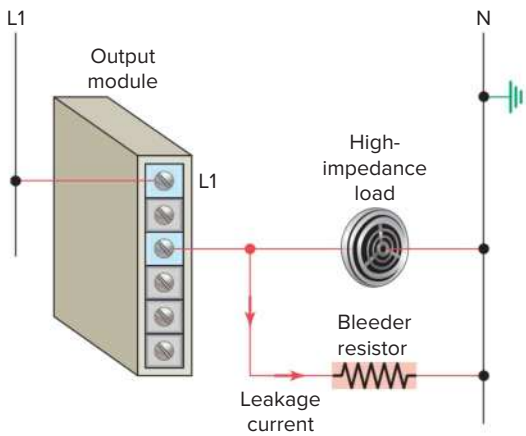


Figure 13-7 Bleeder resistor connection for a high-impedance output.

13.3 Leaky Inputs and Outputs

Many electronic devices with transistor or triac outputs exhibit a small **leakage current** even when in the off state that may need to be considered when they are connected to PLC input modules. This so-called leakage is typically exhibited by two-wire proximity, photoelectric, and other such sensors. Often, the leaky input will only cause the module's input indicator to flicker. However, a large enough leakage current can activate the input circuit, creating a false input signal.

A common solution to the problem of leaky input current is to connect a **bleeder resistor** across or in parallel with the input, as shown in Figure 13-6. The bleeder resistor acts as an additional lower resistance load, which allows the leakage current to flow through the lower resistance path. Typically a 10 to 20 kΩ resistor is used to solve the problem.

Leakage current may also occur with the solid-state switch used in many output modules. Problems similar to that encountered with input modules can be created when a high-impedance load device is used with these modules. For example, a PLC output might supply a sound alert device as illustrated in Figure 13-7. In this case the leakage current could cause continuous false or intermittent operation. A resistor can be connected as shown to bleed off this current. An isolation relay could also be used to solve this type of problem.

13.4 Grounding

Proper **grounding** is an important safety measure in all electrical installations. The authoritative source on grounding requirements for a PLC installation is the **National Electrical Code.** The NEC specifies the types

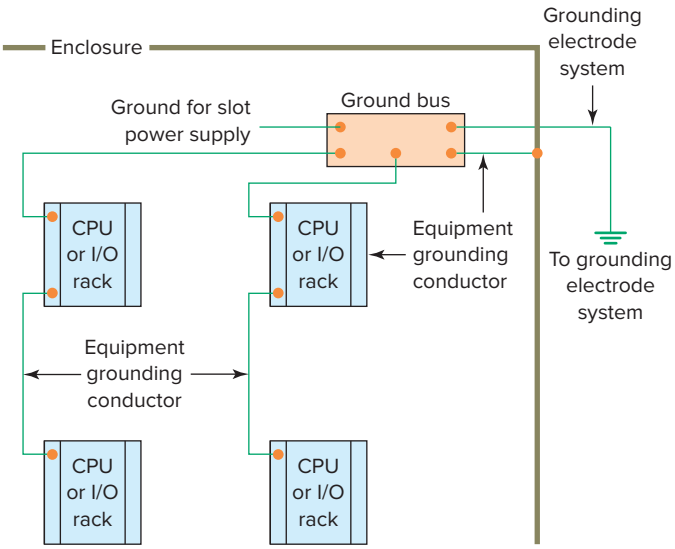


Figure 13-8 PLC grounding system.

of conductors, color codes, and connections necessary for safe grounding of electrical components. In addition, most manufacturers provide detailed information on the proper grounding methods to use in an enclosure.

Figure 13-8 illustrates a PLC grounding system. A properly installed grounding system will provide a low-impedance path to earth ground. The complete PLC installation, including enclosures, CPU and I/O chassis, and power supplies are all connected to a single low-impedance ground. These connections should exhibit low DC resistance and low high-frequency impedance. A central ground bus bar is provided as a single point of reference inside the enclosure to which all chassis and power supply equipment grounding conductors are connected. The ground bus is then connected to the building's earth ground.

In the event of a high value of ground current, the temperature of the conductor could cause the solder to melt, resulting in interruption of the ground connection. Therefore the grounding path must be permanent (no solder), continuous, and able to conduct safely the ground-fault current in the system with minimal impedance. Paint or other nonconductive material should be scraped away from the area where a chassis makes contact with the enclosure. The minimum ground wire size should be No. 12 AWG stranded copper for PLC equipment grounds and No. 8 AWG stranded copper for enclosure backplane grounds. Ground connections should be made with a star washer between the grounding wire and lug and metal enclosure surface, as illustrated in Figure 13-9.

- Any protective ground wires must have a resistance value of less than 0.1 Ω.
- The resistance from the system ground to the earth ground must have a value of less than 0.1 Ω.

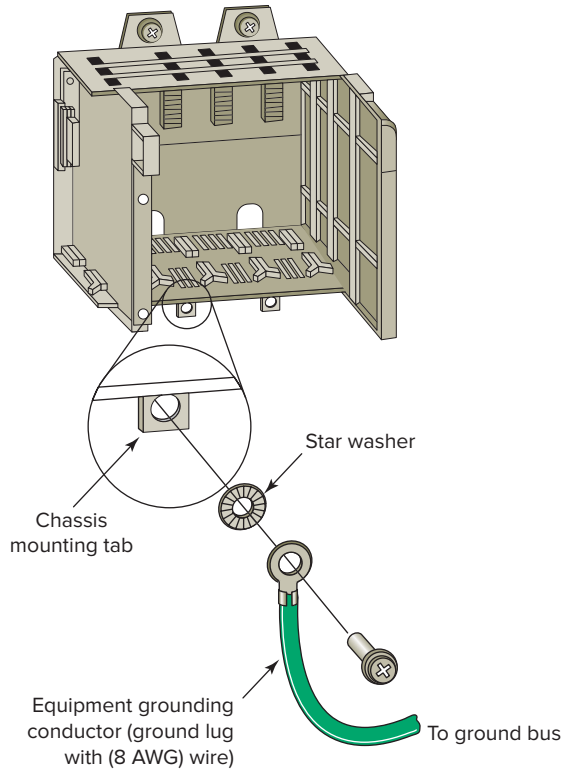


Figure 13-9 Make ground connections using a star washer.

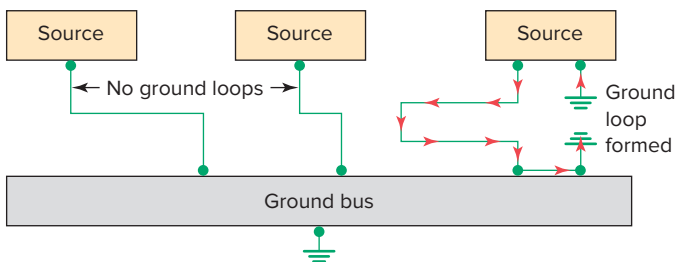


Figure 13-10 Formation of ground loops.

Ground loops can cause problems by adding or subtracting current or voltage from input signal devices. A ground loop circuit can develop when each device's ground is tied to a different earth potential thereby allowing current to flow between the grounds, as illustrated in Figure 13-10. If a varying magnetic field passes through one of these ground loops, a voltage is produced and current flows in the loop. The receiving device is unable to differentiate between the wanted and unwanted signals and, thus, can't accurately reflect actual process conditions. Certain connections require shielded cables to help reduce the effects of electrical noise coupling. Each shield should be grounded at one end only, as a shield grounded at both ends forms a ground loop.

13.5 Voltage Variations and Surges

The power supply section of the PLC system is built to sustain line fluctuations and still allow the system to function within its operating range. If voltage fluctuations exceed this range, then a system shutdown will occur. In areas where excessive line voltage variation or extended brownouts are anticipated, installing a ***constant voltage (CV)*** transformer may be required to minimize nuisance shutdowns of the PLC.

Isolation transformers are used in some PLC systems to isolate the PLC from electrical disturbances generated by other equipment connected to the distribution system. Although the PLC is designed to operate in harsh environments, other equipment may generate considerable amounts of interference that may result in intermittent disturbances in normal operation. A normal practice is to place the PLC power supply and I/O devices on a separate transformer that may also serve as a step-down transformer to reduce the incoming voltage to the desired level.

When current in an inductive load is interrupted or turned off, a very high voltage spike is generated. This high voltage can be reduced or eliminated through suppression techniques which absorb the inductive induced voltage. Generally, output modules designed to drive inductive loads include suppression networks built in as part of the module circuit.

An additional external **suppression device** is recommended if an output module is used to control devices such as relays, solenoids, motor starters, or motors. The suppression device is wired in parallel (directly across) and as close as possible to the load device. The suppression components must be rated appropriately to suppress the switching transient characteristic of the particular inductive device. Figure 13-11 illustrates how a diode is connected to suppress DC inductive loads. The operation of the circuit can be summarized as follows:

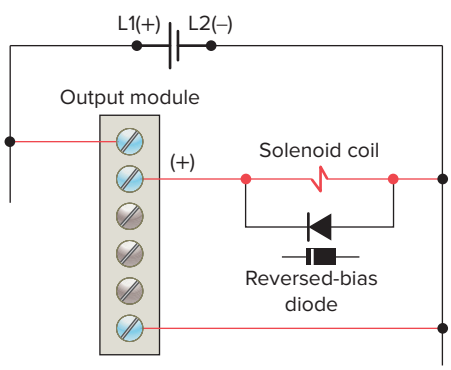


Figure 13-11 Diode connected to suppress DC inductive loads.

- The diode is connected in reverse-bias across the solenoid load.
- In normal operation, the electric current can't flow through the diode, so it flows through the solenoid coil.
- When voltage to the solenoid is switched off a voltage opposite in polarity to the original applied voltage is generated by the collapsing magnetic field.
- The induced voltage creates a current flow through the diode bleeding off the high-voltage spike.

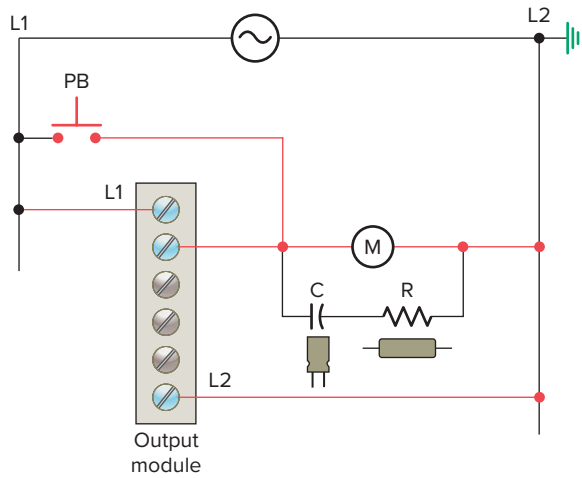


Figure 13-12 RC snubber circuit connected to suppress AC loads.

Figure 13-12 illustrates how an RC (resistor/capacitor) snubber circuit is connected for suppressing AC load devices. The operation of the circuit can be summarized as follows:

- The voltage peak, which occurs at the instant the current path to the coil is opened, is safely short-circuited by the RC network.
- The resistor and capacitor connected in series slows the rate of rise of the transient voltage.
- The voltage across the capacitor cannot change instantaneously, so a decreasing transient current will flow through it for a small fraction of a second, allowing the voltage to increase more slowly when the circuit is opened.

The ***metal oxide varistor (MOV)*** surge suppressor, shown in Figure 13-13, is the most popular surge protection device. It functions in a manner similar to two zener

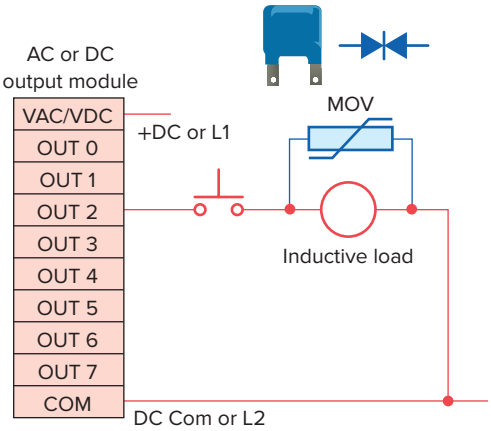


Figure 13-13 Metal oxide varistor (MOV) surge suppressor.

diodes connected back-to-back. The operation of an MOV can be summarized as follows:

- The device acts as an open circuit until the voltage across it in either direction exceeds its rated value.
- Any greater voltage peak instantly makes the device act like a short circuit that bypasses this voltage away from the rest of the circuit.

13.6 Program Editing and Commissioning

After you have entered the rungs for your program, you may need to modify them. ***Editing*** is simply the ability to make changes to an existing program through a variety of editing functions. Using the editing function, instructions and rungs can be added or deleted; addresses, data, and bits can be changed. Again, the editing format varies with different manufacturers and PLC models.

Today, most PLC programming software is Microsoft Windows based, so if you are familiar with Windows and know how to point and click with a mouse, you should have no problem editing a program. In general, both instructions and rungs are selected simply by clicking on them with the left mouse button. Double clicking with the left mouse button allows you to edit an instruction's address, whereas right clicking displays a pop-up menu of related editing commands. If you want to include additional explanation of a symbol or address, you can place an address description on your ladder rung directly above the symbol. To add a page or rung comment, right click on the rung number to which you wish to add the page or rung comment.

Preparing a control process for start-up, also called ***commissioning,*** involves a series of tests to ensure that the PLC, the ladder logic program, the I/O devices, and all associated wiring operate according to specifications. Before commissioning any control system, you should have a good understanding of how the control system operates and how the various components interact. The following are general steps to be followed when commissioning a PLC system:

- Before applying power to the PLC or the input devices, disconnect or otherwise isolate any output device that could potentially cause damage or injury. Typically this precaution would pertain to outputs that cause movement such as starting a motor or operating a valve.
- Apply power to the PLC and input devices. Measure the voltage to verify that rated voltage is being applied.
- Examine the PLC's status indicator lights. If power is properly applied, the power indicator should be on, and there should be no fault indication. If the PLC does not power up properly, it may be faulty. PLCs rarely fail, but if they do fail, it usually happens immediately upon powering up.
- Verify that you have communication with the PLC via the programming device that is running the PLC programming software.
- Place the PLC in a mode that prevents it from energizing its output circuits. Depending on the make of the PLC, this mode may be called *disable, continuous test,* or *single-scan* mode. This mode will allow you to monitor input devices, execute the program, and update the output image file while keeping the output circuits de-energized.
- Manually activate each input device, one at a time, to verify that the PLC's input status lights turn on and off as expected. Monitor the associated condition instruction to verify that the input device corresponds to the correct program address and that the instruction turns true or false as expected.
- Manually test each output. One way you can do this is by applying power to the terminal where the output device is wired. This test will check the output field device and its associated wiring.
- After verifying all inputs, outputs, and program addresses, verify all preset values for counters, timers, and so on.
- Reconnect any output devices that may have been disconnected and place the PLC in the run mode. Test the operation of all emergency stop buttons and the total system operation.

13.7 Programming and Monitoring

When you program a PLC, several instruction entry modes are available, depending on the manufacturer and the model of the unit. A personal computer, with appropriate software, is generally used to program and monitor the program in the PLC. Additionally, it makes possible ***offline programming,*** which involves writing and storing the program in the personal computer without its being connected to the PLC and later downloading it to the PLC. Figure 13-14 illustrates how programs are downloaded and uploaded from and to the computer.

With ***online programming*** the program can be modified, the modifications can be tested, and finally they can be accepted or rejected while the PLC is running. However, offline programming is the safest manner in which to edit a program because additions, changes, and deletions do not affect the operation of the system until downloaded to the PLC.

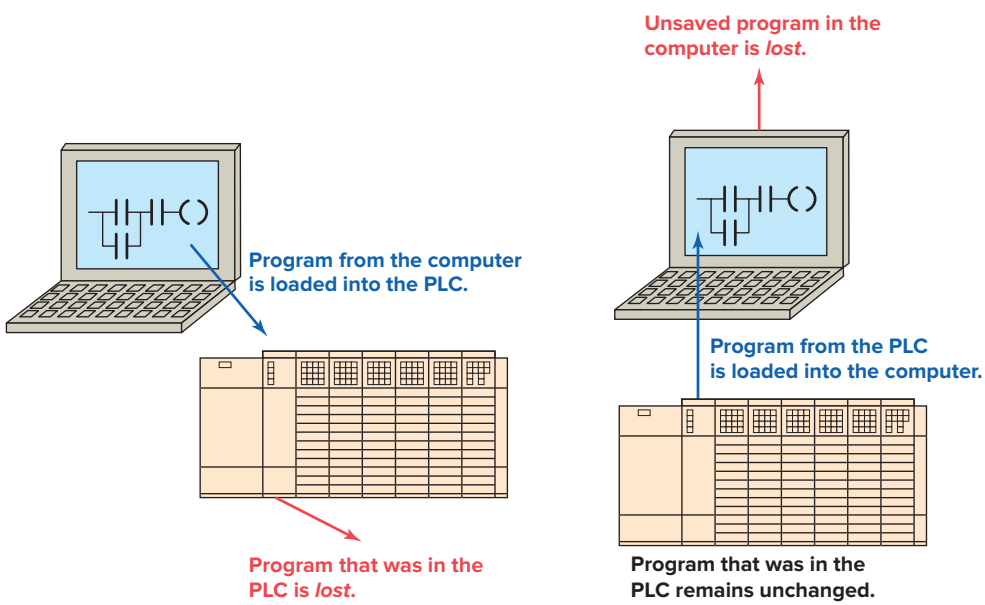
Many manufacturers provide a ***continuous test mode*** that causes the processor to operate from the user program without energizing any outputs. This mode allows the control program to be executed and debugged while the outputs are disabled. A check of each rung can be done by monitoring the corresponding output rung on the programming device. A ***single-scan*** test mode may also be available for debugging the control logic. This mode causes the processor to complete a single scan of the user program each time the single-scan key is pressed with no outputs being energized.

An online programming mode permits the user to change the program during machine operation. As the PLC controls its equipment or process, the user can add, change, or delete control instructions and data values as desired. Any modification made is executed immediately on entry of the instruction. Therefore, the user should assess in advance all possible sequences of machine operation that will result from the change. Online programming should be done only by experienced personnel who understand fully the operation of the PLC they are dealing with and the machinery being controlled. If at all possible, changes should be made offline to provide a safe transition from existing programming to new programming.

Two useful monitoring tools provided with PLC programming packages are data monitor and cross reference. ***Data monitoring*** functions allow you to monitor and/or modify specified program variables. The ***cross reference*** function allows you to search each instance of a particular address.

The data monitor feature allows you to display data from any place in the data table. Depending on the PLC, the data monitor function can be used to do the following:

- View data within an instruction
- Store data or values for an instruction prior to use
- Set or reset values and/or bits during a debug operation for control purposes
- Change the radix or data format



(*a*) Downloading a program (*b*) Uploading a program

Figure 13-14 Downloading and uploading PLC program.

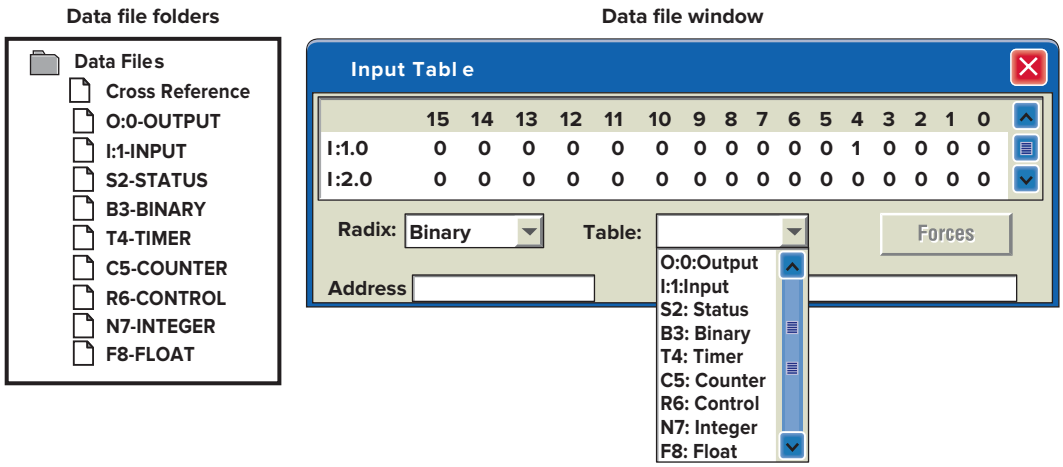


Figure 13-15 Data file folder and window.
Source: Courtesy of TheLearningPit.

Figure 13-15 shows the data file folder and window for the Allen-Bradley SLC 500 PLC and its associated RSLogix software. The data file folder allows the user to determine the status of I/O files as well as the status file (S2), binary file (B3), timer file (T4), counter file (C5), control file (R6), integer file (N7), and the floating-point file (F8). Always be careful when manipulating data using the data monitor function. Changing data could affect the program and turn output devices on or off.

When troubleshooting a PLC, it may be necessary to locate each instance of a particular address in the ladder program. The cross reference function searches all program files to locate each instance of the selected address. A user can then trace the operation by finding all the places where a particular output coil or contact with the same address is used in the program. Figure 13-16 shows a sample cross reference report for the Allen-Bradley SLC 500 PLC and its associated RSLogix software. Its contents can be summarized as follows:

- The report contains all the addresses used in the program.
- Addresses are displayed in the same order as the data table files.
- The address that the search was performed for (O:2/1) is highlighted.
- The description for each address is displayed.
- Listing includes the instruction type, program file, and rung number for each address.
- Each occurrence of the address is displayed, starting with program file 2 and rung 0.

The ***contact histogram*** function allows you to view the transition history (the on and off states) of a data table value. The status of the bit(s) (on or off) and the length of time the bit(s) remained on or off (in hours, minutes, seconds, and hundredths of a second) are displayed. In a contact histogram file, the accumulated time indicates the total time that the histogram function was running. The delta time of the contact histogram indicates the elapsed time between the changes in states. Contact histograms are extremely

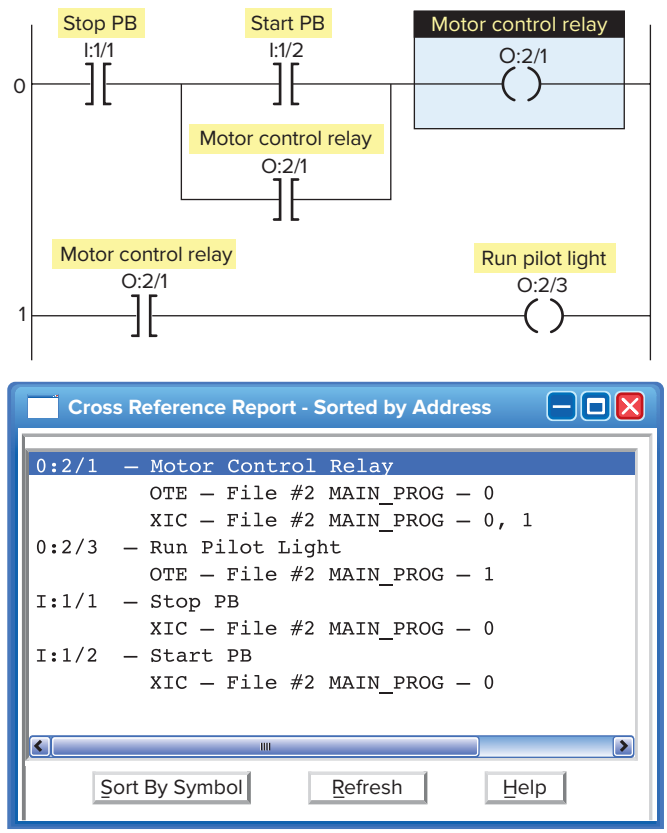


Figure 13-16 Sample cross reference report.
Source: Courtesy of TheLearningPit.

useful for detecting intermittent problems, either hardware- or logic-related. By tracking the status and time between status changes, you can detect different types of problems.

13.8 Preventive Maintenance

The biggest deterrent to PLC system faults is a proper **preventive maintenance** program. Although PLCs have been designed to minimize maintenance and provide trouble-free operation, there are several preventive measures that should be looked at regularly.

Many control systems operate processes that must be shut down for short periods for product changes. The following preventive maintenance tasks should be carried out during these short shutdown periods:

- Any filters that have been installed in enclosures should be cleaned or replaced to ensure that clear air circulation is present inside the enclosure.
- Dust or dirt accumulated on PLC circuit boards should be cleaned. If dust is allowed to build up on heat sinks and electronic circuitry, an obstruction of heat dissipation could occur and cause circuit malfunction. Furthermore, if conductive dust reaches the electronic boards, a short circuit could result and cause permanent damage to the circuit board. Ensuring that the enclosure door is kept closed will prevent the rapid buildup of these contaminants.
- Connections to the I/O modules should be checked for tightness to ensure that all plugs, sockets, terminal strips, and module connections are making connections and that the module is installed securely. Loose connections may result not only in improper function of the controller but also in damage to the components of the system.
- All field I/O devices should be inspected to ensure that they are adjusted properly. Circuit boards dealing with process control analogs should be calibrated every 6 months. Other devices, such as sensors, should be serviced on a monthly basis. Field devices in the environment, which have to translate mechanical signals into electrical, may gum up, get dirty, crack, or break—and then they will no longer trip at the correct setting.
- Care should be taken to ensure that heavy noise- or heat-generating equipment is not moved too close to the PLC.
- Check the condition of the battery that backs up the RAM memory in the CPU (Figure 13-17). Most CPUs have a status indicator that shows whether the battery's voltage is sufficient to back up the memory stored in the PLC. If a battery module is to be replaced, it must be replaced with exactly the same type of battery module.
- Stock commonly needed spare parts. Input and output modules are the PLC components that fail most often.
- Keep a master copy of operating programs used.

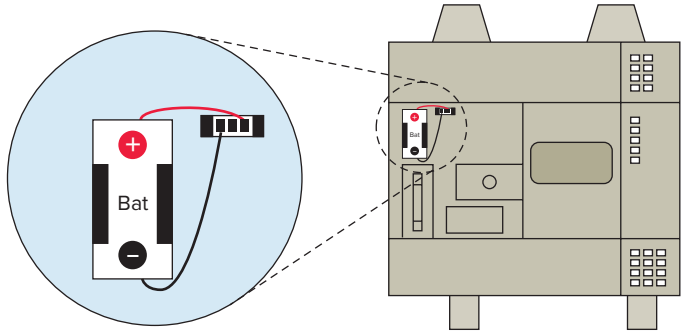


Figure 13-17 CPU backup memory battery.

To avoid injury to personnel and to prevent equipment damage, connections should always be checked with power removed from the system. In addition to disconnecting electrical power, all other sources of power (pneumatic and hydraulic) should be de-energized before someone works on a machine or process controlled by a PLC. Most companies use **lockout and tagout procedures,** shown in Figure 13-18, to make sure that equipment does not operate while maintenance and repairs are conducted. A personnel protection tag is placed on the power source for the equipment and the PLC, and it can be removed only by the person who originally placed the tag. In addition to the tag, a lock is also attached so that equipment cannot be energized.

Figure 13-18 Lockout/tagout devices.
Source: Photo courtesy Panduit Corporation, **www.panduit.com**.

13.9 Troubleshooting

In the event of a PLC fault, you should employ a careful and systematic approach to troubleshoot the system to resolve the problem. PLCs are relatively easy to troubleshoot because the control program can be displayed on a monitor and watched in real time as it executes. If a control system has been operating, you can be fairly confident of the accuracy of the program logic. For a system that has never worked or is just being commissioned, programming errors should be considered.

When a problem occurs, the first step in the troubleshooting procedure is to identify the problem and its source. The source of a problem can generally be narrowed down to the processor module, I/O hardware, wiring, machine inputs or outputs, or ladder logic program. Once a problem is recognized, it is usually quite simple to deal with. The following sections will deal with troubleshooting these potential problem areas.

Processor Module

The processor is responsible for the ***self-detection*** of potential problems. It performs error checks during its operation and sends status information to indicators that are normally located on the front of the processor module. You can diagnose processor faults or obtain more detailed information about the processor by accessing the processor status through programming software. Figure 13-19 shows sample diagnostics LEDs found on a processor module. What they indicate can be summarized as follows:

RUN (Green)

- On steady indicates that the process is in the RUN mode.
- Flashing during operation indicates that the process is transferring a program from RAM to the memory module.
- Off indicates that processor is in a mode other than RUN.

FLT (Red)

- Flashing at power-up indicates that the processor has not been configured.
- Flashing during operation indicates a major error either in the processor, chassis, or memory.
- On steady indicates that a fatal error is present (no communications).
- Off indicates there are no errors.

BATT (Red)

- On steady indicates the battery voltage has fallen below a threshold level, or the battery is missing or not connected.
- Off indicates that the battery is functional.

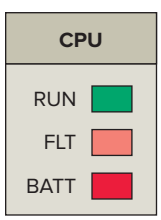


Figure 13-19 Processor diagnostics LEDs.

The processor then monitors itself continually for any problems that might cause the controller to execute the user program improperly. Depending on the controller, a set of fault relay contacts may be available. The fault relay is controlled by the processor and is activated when one or more specific fault conditions occur. The fault relay contacts are used to disable the outputs and signal a failure.

Most PLCs incorporate a ***watchdog timer*** to monitor the scan process of the system. The watchdog timer is usually a separate timing circuit that must be set and reset by the processor within a predetermined period. The watchdog timer circuit monitors how long it takes the CPU to complete a scan. If the CPU scan takes too long, a watchdog major error will be declared. PLC user manuals will show how to apply this function.

The PLC processor hardware is not likely to fail because today's microprocessors and microcomputer hardware are very reliable when operated within the stated limits of temperature, moisture, and so on. The PLC processor chassis is typically designed to withstand harsh environments.

Input Malfunctions

If the controller is operating in the RUN mode but output devices do not operate as programmed, the faults could be associated with any of the following:

- Input and output wiring between field devices and modules
- Field device or module power supplies
- Input sensing devices
- Output actuators
- PLC I/O modules
- PLC processor

Narrowing down the problem source can usually be accomplished by comparing the actual status of the suspect I/O with controller status indicators. Usually each input or output device has at least two status indicators. One of these indicators is on the I/O module; the other indicator is provided by the programming device monitor.

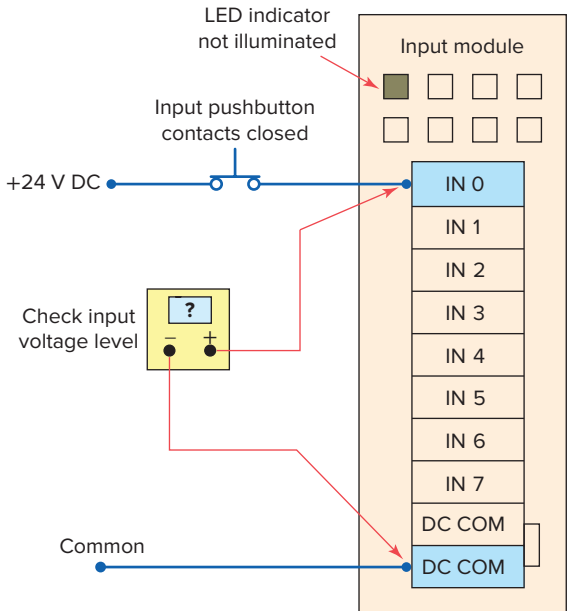


Figure 13-20 Checking for input malfunctions.

The circuit of Figure 13-20 illustrates how to check for discrete input malfunctions. The steps taken can be summarized as follows:

- When input hardware is suspected to be the source of a problem, the first check is to see if the status indicator on the input module illuminates when it is receiving power from its corresponding input device (e.g., pushbutton, limit switch).
- If the status indicator on the input module does *not* illuminate when the input device is on, take a voltage measurement across the input terminal to check for the proper voltage level.
- If the voltage level is correct, then the input module should be replaced.
- If the voltage level is not correct, power supply, wiring, or input device may be faulty.

If the programming device monitor does not show the correct status indication for a condition instruction, the input module may not be converting the input signal properly to the logic level voltage required by the processor module. In this case, the input module should be replaced. If a replacement module does not eliminate the problem and wiring is assumed to be correct, then the I/O rack, communication cable, or processor should be suspected. Figure 13-21 shows a typical input device troubleshooting guide. This guide reviews condition instructions and how their true/false status relates to external input devices.

Input device troubleshooting guide

Input device condition	Input module status indicator	Monitor display status indicator		Possible fault(s)
Closed — ON 24 V DC input	ON	True	False	None - correct indications
Open — OFF 0 V DC input	OFF	False	True	None - correct indications
Closed — ON 24 V DC input	ON	False	True	Sensor condition, input voltage, status indicator are correct. Ladder instructions have incorrect indications. Input module or processor fault.
Closed — ON 0 V DC input	OFF	False	True	Status indicator and instructions agree but not with the sensor condition. Open field device or wiring.
Open — OFF 0 V DC input	OFF	True	False	Sensor condition, input voltage, status indicator are correct. Ladder instructions have incorrect indications. Input module or processor fault.
Open — OFF 24 V DC input	ON	True	False	Input voltage, status indicator, and ladder instructions agree but not with sensor condition. Short circuit in the field device or wiring.

Figure 13-21 Input troubleshooting guide.

Output Malfunctions

In addition to the logic indicator, some output modules incorporate either a blown fuse indicator or a power indicator or both. A blown fuse indicator indicates the status of the protective fuse in the output circuit, while a power indicator shows that power is being applied to the load.

Electronic protection, as shown in Figure 13-22, is also used to provide protection for the modules from short-circuit and overload current conditions. The protection is based on a thermal cut-out principle. In the event of a short-circuit or overload current condition on an output channel, that channel will limit current within milliseconds after its thermal cut-out temperature has been reached. All other channels continue to operate as directed by the processor.

When an output does not energize as expected, first check the output module blown fuse indicator. Many output modules have each output fused. This indicator will normally illuminate only when the output circuit corresponding to the blown fuse is energized. If this indicator is illuminated, correct the cause of the malfunction and replace the blown fuse in the module.

Figure 13-23 shows a typical discrete output module troubleshooting guide. In general, the following items should be noted when troubleshooting discrete output modules:

- If the blown fuse indicator is not illuminated (fuse OK), then check to see if the output device is responding to the LED status indicator.
- An output module's logic status indicator functions similarly to an input module's status indicator. When it is on, the status LED indicates that the module's logic circuitry has recognized a command from the processor to turn on.
- If an output rung is energized, the module status indicator is on, and the output device is not responding, then the wiring to the output device or the output device itself should be suspected.

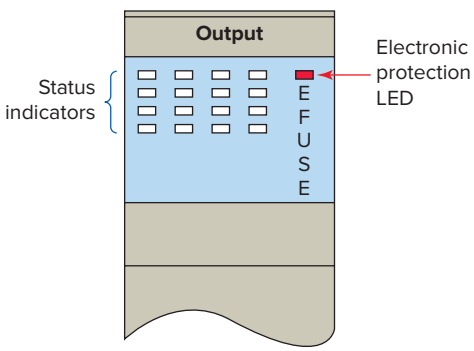


Figure 13-22 Electronic output module protection.

- If, according to the programming device monitor, an output device is commanded to turn on but the status indicator is off, then the output module or processors may be at fault.
- Check voltage at output; if incorrect, power supply, wiring, or output device may be faulty.

Ladder Logic Program

Many PLC software programs offer various software checks used to verify program logic. Figure 13-24 shows a sample of verifying program errors using RSLogix 500 software. Selecting **edit** then **verify project** will check the program for errors. The sample shows what the error message might look like.

The ladder logic program itself is not likely to fail, assuming that the program was at one time working correctly. A hardware fault in the memory IC that holds the ladder logic program could alter the program, but this is a PLC hardware failure. If all other possible sources of trouble have been eliminated, the ladder logic program should be reloaded into the PLC from the master copy of the program. Make sure the master copy of the program is up to date before you download it to the PLC.

Start program troubleshooting by identifying which outputs operate properly and which outputs do not. Then trace back from the output on the nonfunctioning rung and examine the logic to determine what may be preventing the output from energizing. Common logic errors include:

- Programming an examine if closed instruction instead of an examine if open (or vice versa)
- Using an incorrect address in the program

Although the ladder logic program is not likely to fail, the process may be in a state that was unaccounted for in the original program and thus is not controlled properly. In this case, the program needs to be modified to include this new state. A careful examination of the description of the control system and the ladder logic program can help identify this type of fault.

The force on and force off instructions allow you to turn specific bits on or off for testing purposes. Figure 13-25 illustrates how forces are identified as being enabled or disabled in RSLogix 500 software. Forcing lets you simulate operation or control an output device. For example, forcing a solenoid valve on will tell you immediately whether the solenoid is functional when the program is bypassed. If it is, the problem must be related to the software and not the hardware. If the output fails to respond when forced, either the actual output module is causing the problem or the solenoid itself is malfunctioning. ***Take all necessary precautions to protect personnel and equipment during forcing.***

Output device troubleshooting guide			
Output device condition	Output module status indicator	Monitor display status indicator	Fault(s)
Energized — ON	ON	True	None - correct indication
De-energized — OFF	OFF	False	None - correct indication
De-energized — OFF	ON	True	Output instruction and status indicator agree but the field device does not. Open field device or wiring. Module circuit or fuse.
De-energized — OFF	OFF	True	Field device status and status indicator agree but the output condition does not. Module circuit or fuse.

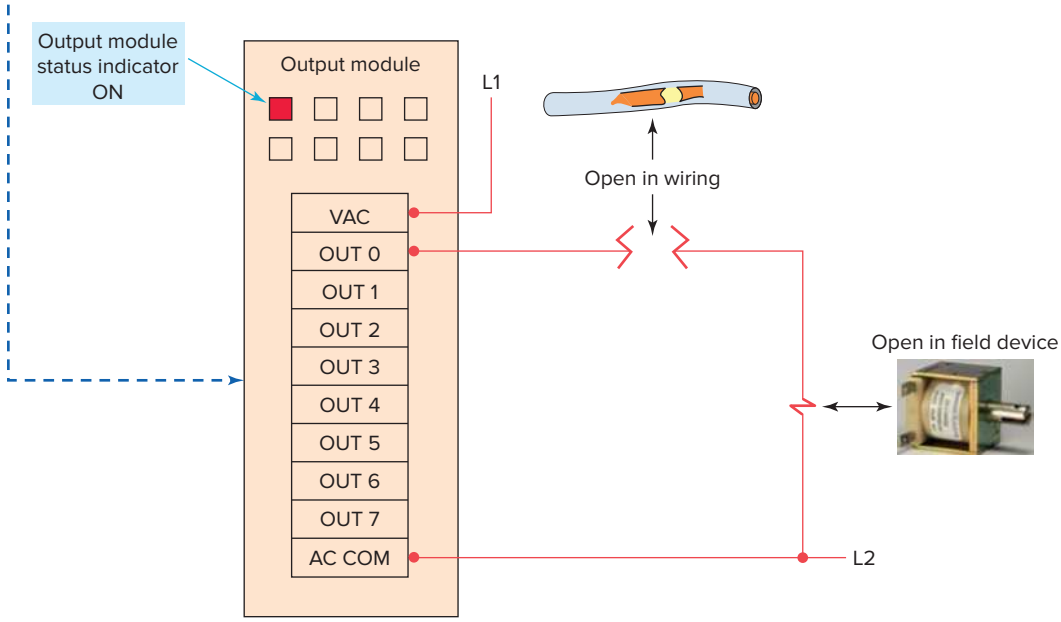


Figure 13-23 Output troubleshooting guide.
Source: Photo courtesy Guardian Electric, **www.guardian-electric.com**.

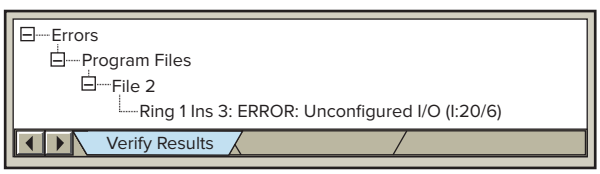


Figure 13-24 Sample of verifying program errors.

Figure 13-25 Indication of enabled forces.
Source: Courtesy of TheLearningPit.

Certain diagnostic instructions may be included as part of a PLC's instruction set for troubleshooting purposes. The ***temporary end (TND)*** instruction, shown in Figure 13-26, is used when you want to change the amount of logic scanned to progressively debug your program. The

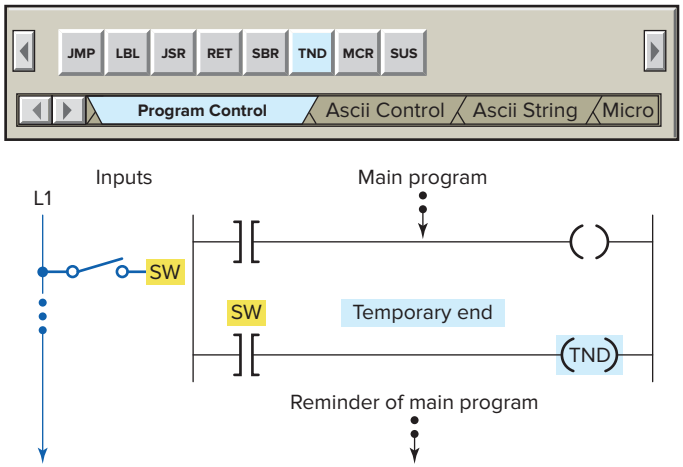


Figure 13-26 TND (temporary end) diagnostic instruction.

operation of this output instruction can be summarized as follows:

- The instruction operates only when its rung conditions are true and stops the processor from scanning any logic beyond the TND instruction.
- When the processor encounters a true TND rung, it resets the watchdog timer (to 0), performs an I/O update, and begins running the ladder program at the first instruction in the main program.
- If the TND rung is false, the processor continues the scan until the next TND instruction or the END statement.
- By inserting the TND instruction at different locations in the program you can test parts of the program sequentially until the entire program has been tested.
- Once the troubleshooting process has been completed, any remaining TND instructions are removed from the program.

The ***suspend (SUS)*** instruction, shown in Figure 13-27, is used to trap and identify specific conditions for program debugging and system troubleshooting. The operation of this output instruction can be summarized as follows:

- When the rung is true, this instruction places the controller in the *suspend* or *idle* mode.
- The suspend ID, in this case 100, must be selected by the programmer and entered in the instruction.
- When the SUS instruction executes, the ID number 100 is written in word 7 (S:7) of the status file.
- If multiple suspend instructions are present, then this will indicate which SUS instruction was active.
- The suspend file (program or subroutine number identifying where the executed SUS instruction resides) is placed in word 8 (S:8) of the status file.

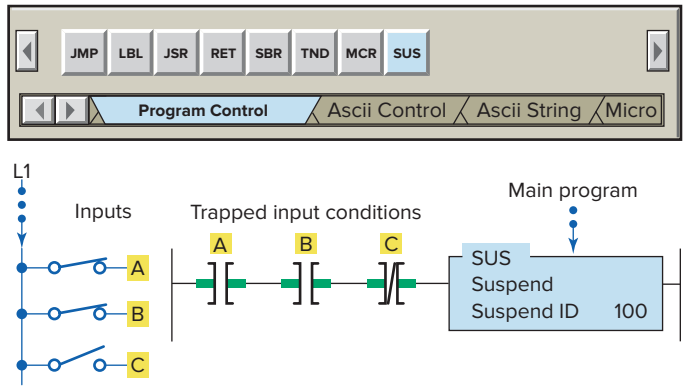


Figure 13-27 SUS (suspend) diagnostic instruction.

- All ladder logic outputs are de-energized, but other status files have the data present when the suspend instruction was executed.

Most PLC system faults occur in the field wiring and devices. The wiring between the field devices and the terminals of the I/O modules is a likely place for problems to occur. Faulty wiring and mechanical connection problems can interrupt or short the signals sent to and from the I/O modules.

The sensors and actuators connected to the I/O of the process can also fail. Mechanical switches can wear out or be damaged during normal operation. Motors, heaters, lights, and sensors can also fail. Input and output field devices must be compatible with the I/O module to ensure proper operation.

When an instruction does not seem to be working correctly, the problem may be an addressing conflict caused by the *same address* being used for two or more coil instructions in the same program. As a result, multiple rung conditions can control the same output coil, making troubleshooting more difficult. In the case of duplicate outputs, the monitored rung may be true; but if a rung farther down in the ladder diagram is false, the PLC will keep the output off. The program of Figure 13-28 illustrates what

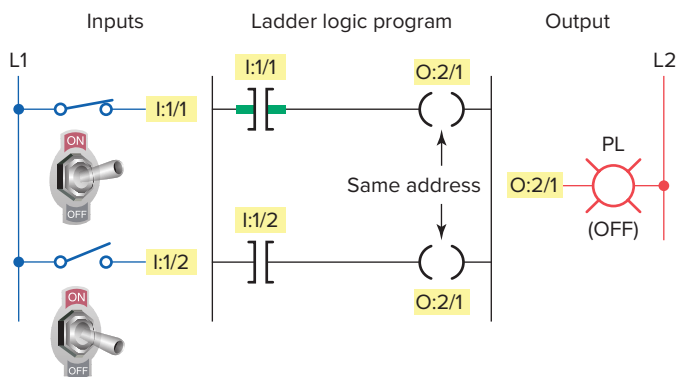


Figure 13-28 Program with the same address used for two coils.

happens when the same address is used for two coils. The resulting problem scenario can be summarized as follows:

- The problem is turning input switch I:1/1 on *will not* turn on PL output O:2/1 as it appears to be programmed.
- The root of the problem lies in the fact that the PLC scans the program from left to right and top to bottom.
- Whenever input switch I:1/1 is true (closed) and input switch I:1/2 is false (open) output O:2/1 will be off.
- This is because when the PLC updates the outputs it does so based on the status of input I:1/2.
- Regardless of whether input I:1/1 is open or closed the output reacts only to the status of input switch I:1/2.

When a problem occurs, the best way to proceed is to try to logically identify the devices or connections that could be causing the problem rather than arbitrarily checking every connection, switch, motor, sensor, I/O module, and so on. First, observe the system in operation and try to describe the problem. Using these observations and the description of the control system, you should identify the possible sources of trouble. Compare the logic status of the hardwired inputs and outputs to their actual state, as illustrated in Figure 13-29. Any disagreements indicate malfunctions as well as their approximate location.

Some of your troubleshooting can be accomplished by interpreting the status indicators on the I/O modules. The key is to know whether the status indicators are telling you that there is a fault or that the system is normal. Often PLC manufacturers supply a troubleshooting guide, map, or tree that presents a list of observed problems and their possible sources. Figure 13-30 shows a sample troubleshooting tree for a discrete output module. Figures 13-31 and 13-32 are samples of input and output troubleshooting guides.

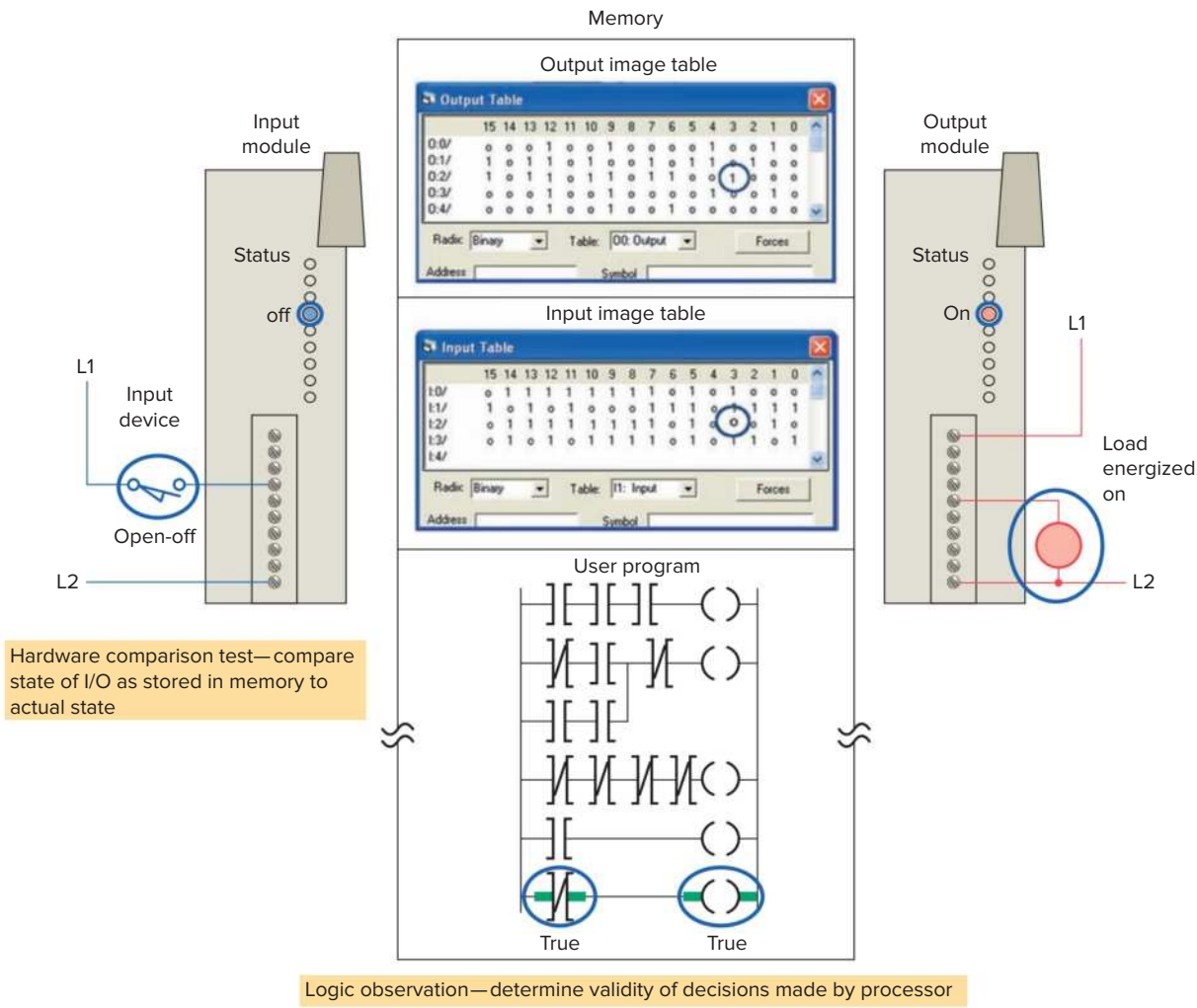


Figure 13-29 General methods of troubleshooting.
Source: Courtesy of TheLearningPit.

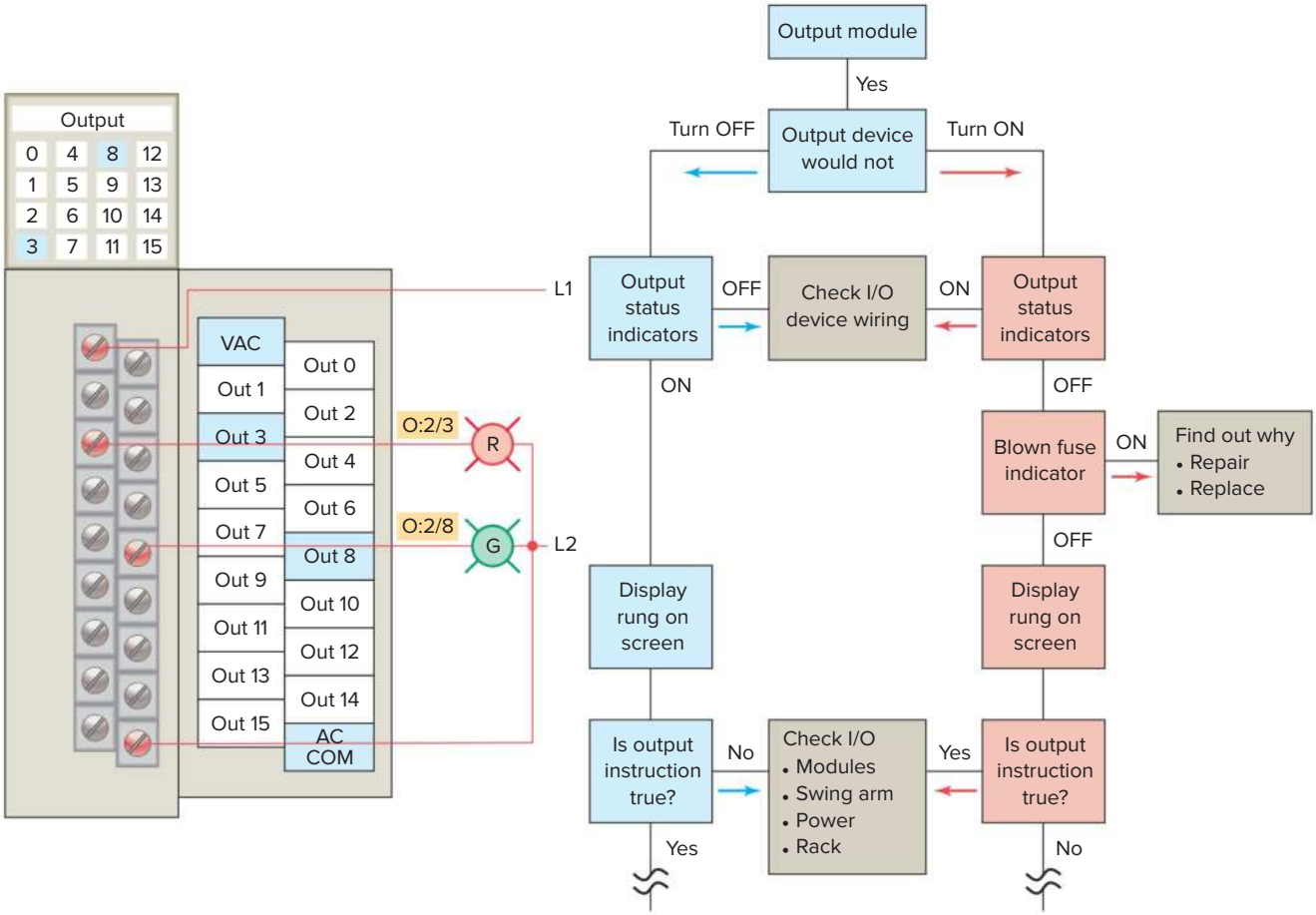


Figure 13-30 Troubleshooting tree for a discrete output module.

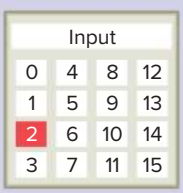
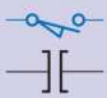
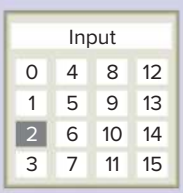
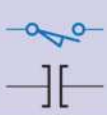
If Your Input Circuit LED Is . . .	And Your Input Device Is . . .	And	Probable Cause
ON 	On/Closed/Activated	Your input device will not turn off.	Device is shorted or damaged.
		Your program operates as though it is off.	Input circuit wiring or module.
			Input is forced off in program.
	Off/Open/Deactivated 	Your program operates as though it is on and/or the input circuit will not turn off.	Input device off-state leakage current exceeds input circuit specification.
			Input device is shorted or damaged.
			Input circuit wiring or module.
OFF 	On/Closed/Activated	Your program operates as though it is off and/or the input circuit will not turn on.	Input circuit is incompatible.
			Low voltage across the input.
			Input circuit wiring or module.
			Input signal turn-on time too fast for input circuit.
	Off/Open/Deactivated 	Your input device will not turn on.	Input device is shorted or damaged.
		Your program operates as though it is on.	Input is forced on in program.
			Input circuit wiring or module.

Figure 13-31 Input troubleshooting guide.

If Your Output Circuit LED Is . . .	And Your Output Device Is . . .	And	Probable Cause
ON Output 0 4 8 12 1 5 9 13 2 6 10 14 3 7 11 15	On/Energized	Your program indicates that the output circuit is off or the output circuit will not turn off.	Programming problem: - Check for duplicate outputs and addresses. - If using subroutines, outputs are left in their last state when not executing subroutines. - Use the force function to force output off. If this does not force the output off, output circuit is damaged. If the output does force off, then check again for logic/programming problem.
			Output is forced on in program.
			Output circuit wiring or module.
	Off/De-energized	Your output device will not turn on and the program indicates that it is on.	Low or no voltage across the load.
			Output device is incompatible: check specifications and sink/source compatibility (if dc output).
			Output circuit wiring or module.
OFF Output 0 4 8 12 1 5 9 13 2 6 10 14 3 7 11 15	On/Energized	Your output device will not turn off and the program indicates that it is off.	Output device is incompatible.
			Output circuit off-state leakage current may exceed output device specification.
			Output circuit wiring or module.
			Output device is shorted or damaged.
	Off/De-energized	Your program indicates that the output circuit is on or the output circuit will not turn on.	Programming problem: - Check for duplicate outputs and addresses. - If using subroutines, outputs are left in their last state when not executing subroutines. - Use the force function to force output on. If this does not force the output on, output circuit is damaged. If the output does force on, then check again for logic/programming problem.
			Output is forced off in program.
			Output circuit wiring or module.

Figure 13-32 Output troubleshooting guide.

13.10 PLC Programming Software

You must establish a way for your personal computer (PC) software to communicate with the programmable logic controller (PLC) processor. Making this connection is known as ***configuring* the communications.** The method used to configure the communications varies with each brand of controller. In Allen-Bradley controllers, ***RSLogix*** software, Figure 13-33, is required to develop and edit ladder programs. A second software package, ***RSLinx,*** is needed to set up the communication path between the PLC processor and the personal computer

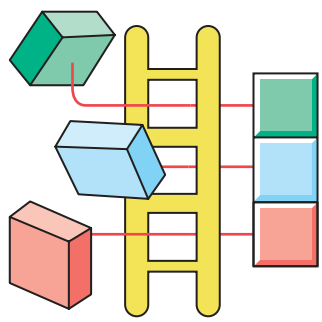


Figure 13-33 RSLogix software.

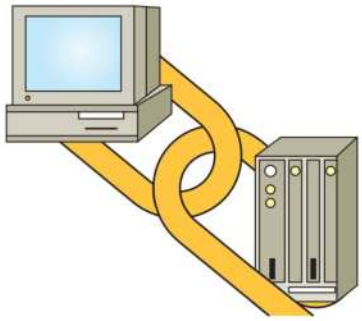


Figure 13-34 RSLinx software.

(PC). You cannot download multiple projects to the PLC and then run them when required. The PLC will accept only one program at a time, but the program can consist of multiple subroutine files which can be conditionally called from the main program.

RSLinx software, Figure 13-34, is available in multiple packages to meet the demand for a variety of cost and functionality requirements. This software package is used as the driver between your PC and PLC processor. A ***driver*** is a computer program that controls a device. For example, you must have the correct printer driver installed in your PC in order to be able to print a word-processing document created on your PC. RSLinx works much like the printer driver for RSLogix software. The RSLinx program must be opened and drivers configured before communications can be established between a PC and a PLC that is using RSLogix software.

RSLinx allows RSLogix to communicate through an interface cable to the PLC processor. The simplest connection between a PC and a PLC is a point-to-point direct connection through the computer serial port, as illustrated in Figure 13-35. A serial cable is used to connect to your PC's COM 1 or COM 2 port and to the PLC processor's serial communications port. With RSLinx software you can auto-configure the serial connection and thus automatically find the proper serial port configuration.

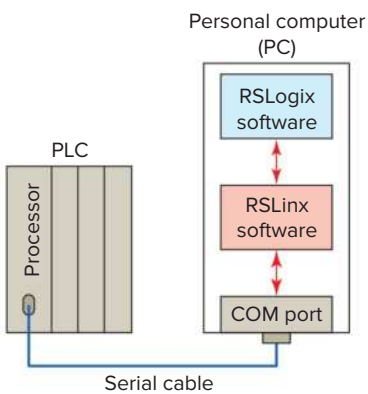


Figure 13-35 Direct PC-to-PLC software connection.

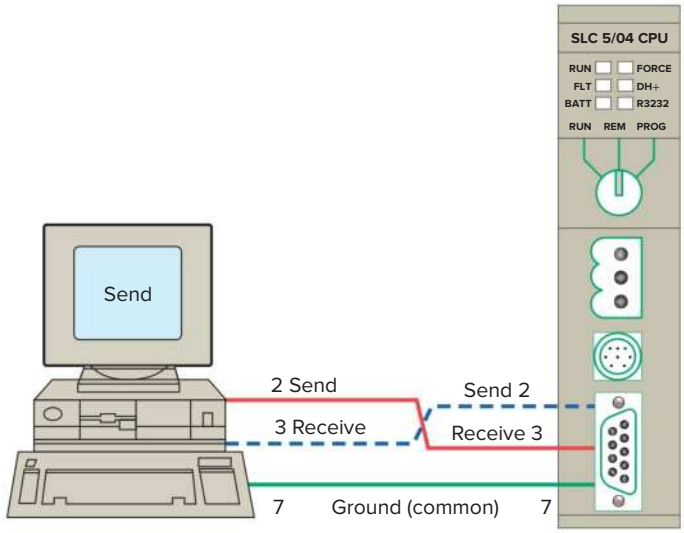


Figure 13-36 Serial wiring connection.

Two important aspects of the communication link must be considered, namely, the RS-232 standard and the communications protocol. The ***RS-232 standard*** specifies a function for each of the wires inside the standard communications cable and their associated pins. ***Communications protocol*** is a standardized method for transmitting data and/or establishing communications between different devices.

Minimum configuration for two-way communications requires the use of only three connected wires, as shown in Figure 13-36. For ease of connection, the RS-232 standard specifies that computer devices have male connectors and that peripheral equipment have female connectors. Direct communication between two computers, such as a PC and a PLC, does not involve intermediate peripheral equipment. Therefore, a serial null-modem type cable must be used for the connection because both the PC and the PLC processor use pin 2 for data output and pin 3 for data input.

RSLinx is a Windows based communication software package developed by Rockwell Software to interface to all of the Rockwell and A-B industrial control and automation hardware. To setup RSLinx:

- Open RSLinx.
- Click on Communications and select configure drivers from the dialog box.
- Select RS-232 from the available driver types.
- Click on add new and select your communications port from the dialog box.
- Click on auto configure to automatically set the baud rate and parity.
- Minimize the RSLinx window, but leave the program running.

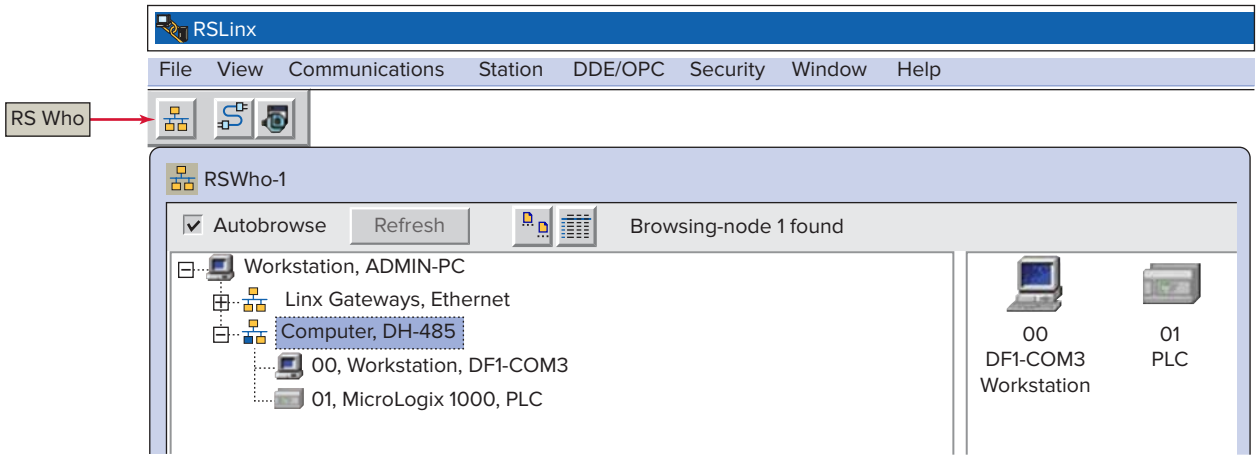


Figure 13-37 The who active or RSWho window.
Source: Image Courtesy of Rockwell Automation, Inc.

RSWho is RSLinx's network browser interface. The who active or RSWho function, Figure 13-37, shows you what stations are connected on your PLC network. The left pane of RSWho is the tree control, which shows networks and devices. The right pane is the list control, which shows all members of a collection. A collection is a network, or a device that is a bridge. A device that appears with a red X through it indicates a communication status error, such as loss of power to a device or a disconnected communication cable.

CHAPTER 13 REVIEW QUESTIONS

1. Why are PLCs installed within an enclosure?
2. What methods are used to keep enclosure temperatures within allowable limits?
3. State two ways in which electrical noise may be coupled into a PLC control system.
4. List three potential noise-generating inductive devices.
5. Describe four ways in which careful wire routing can help cut down on electrical noise.
6. **a.** What type of input field devices and output modules are most likely to have a small leakage current flow when they are in the off state? Why?
   b. Explain how an input bleeder resistor reduces leakage current.
7. Summarize the basic grounding requirements for a PLC system.
8. Under what condition can a ground loop circuit be developed?
9. When line voltage variations to the PLC power supply are excessive, what can be done to solve the problem?
10. What operating state will cause an inductive load to generate a very high voltage spike?
11. Explain how a diode is connected to suppress a DC inductive load.
12. Explain how an MOV suppresses an AC inductive load.
13. What is the purpose of PLC editing functions?
14. What is involved with commissioning a PLC system?
15. **a.** Compare offline and online programming.
    b. Which method is safer? Why?
16. List four uses for the data monitor function.
17. What information is provided by the cross reference function?
18. What information is provided by the contact histogram function?
19. List five preventive maintenance tasks that should be carried out on the PLC installation regularly.
20. Outline the general procedure followed to lock out and tag a PLC installation.
21. Typically, what does each of the following processor diagnostic light states indicate?
    a. RUN light is off.
    b. Fault light is off.
    c. BATTERY light is on.
22. Explain the function of a watchdog timer circuit.
23. A PLC is operating in the RUN mode but output devices do not operate as programmed. List five faults that could be responsible for this condition.
24. What is the verify results function used for?
25. A fast-acting solenoid-operated gate is suspected of not functioning properly when energized and de-energized by the PLC program. Explain how you would use the force function to check its operation.
26. What happens when the processor encounters a temporary end instruction?
27. Explain the function of the suspend instruction.
28. In what negative ways can faulty wiring and connections affect signals sent to and from the I/O modules?
29. The same address is used for two coil instructions within the same PLC program. What will happen as a consequence of this?
30. Compare the uses for RSLogix and RSLinx programming software.

CHAPTER 13 PROBLEMS

1. The enclosure door of a PLC installation is not kept closed. What potential problem could this create?
2. A fuse is blown in an output module. Suggest two possible reasons why the fuse blew.
3. Whenever a crane located over a PLC installation is started from a standstill, temporary malfunction of the PLC system occurs. What is one likely cause of the problem?

4. During the static checkout of a PLC system, a specific output is forced on by the programming device. If an indicator other than the expected one turns on, what is the probable problem?
5. The input device to a module is activated, but the LED status indicator does not come on. A check of the voltage to the input module indicates that no voltage is present. Suggest two possible causes of the problem.
6. An output is forced on. The module logic light comes on, but the field device does not work. A check of the voltage on the output module indicates the proper voltage level. Suggest two possible causes of the problem.
7. A specific output is forced on, but the LED module indicator does not come on. A check of the voltage at the output module indicates a voltage far below the normal on level. What is the first thing to check?
8. An electronic-based input sensor is wired to a high-impedance PLC input and is falsely activating the input. How can this problem be corrected?
9. An LED logic indicator is illuminated, and according to the programming device monitor, the processor is not recognizing the input. If a replacement module does not eliminate the problem, what two other items should be suspected?
10. **a.** A normally open field limit switch examined for an on state normally cycles from on to off five times during one machine cycle. How could you tell by observing the LED status light that the limit switch is functioning properly?
    b. How could you tell by observing the programming device monitor that the limit switch is functioning properly?
    c. How could you tell by observing the LED status light whether the limit switch was stuck open?
    d. How could you tell by observing the programming device monitor whether the limit switch was stuck open?
    e. How could you tell by observing the LED status light if the limit switch was stuck closed?
    f. How could you tell by observing the programming device monitor if the limit switch was stuck closed?
11. Assume that prior to putting a PLC system into operation, you want to verify that each *input device* is connected to the correct input terminal and that the input module or point is functioning properly. Outline a method of carrying out this test.
12. Assume that prior to putting a PLC system into operation, you want to verify that each *output device* is connected to the correct output terminal and that the output module or point is functioning properly. Outline a method of carrying out this test.
13. With reference to the ladder logic program of Figure 13-38, add instructions to modify the program to ensure that the second pump_2 does not run while pump_1 is running. If this condition occurs, the program should suspend operation and enter code identification number 100 into S2:7.
14. The program of Figure 13-39 is supposed to execute to sequentially turn PL1 off for 5 seconds and on for 10 seconds whenever input *A* is closed.
    a. Examine the ladder logic and describe how the circuit would operate as programmed.
    b. Troubleshoot the program and identify what needs to be changed to have it operate properly.

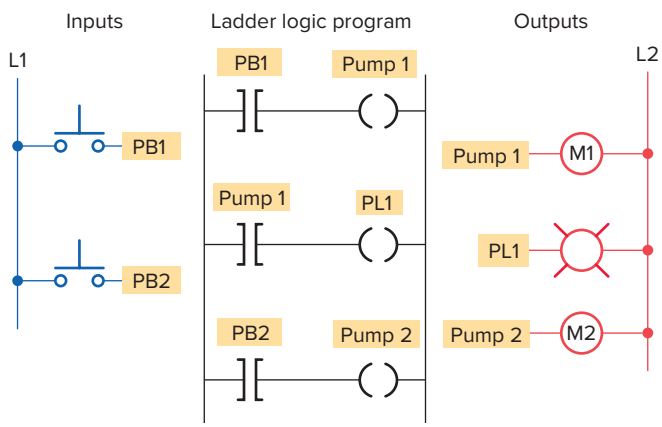


Figure 13-38 Program for Problem 13.

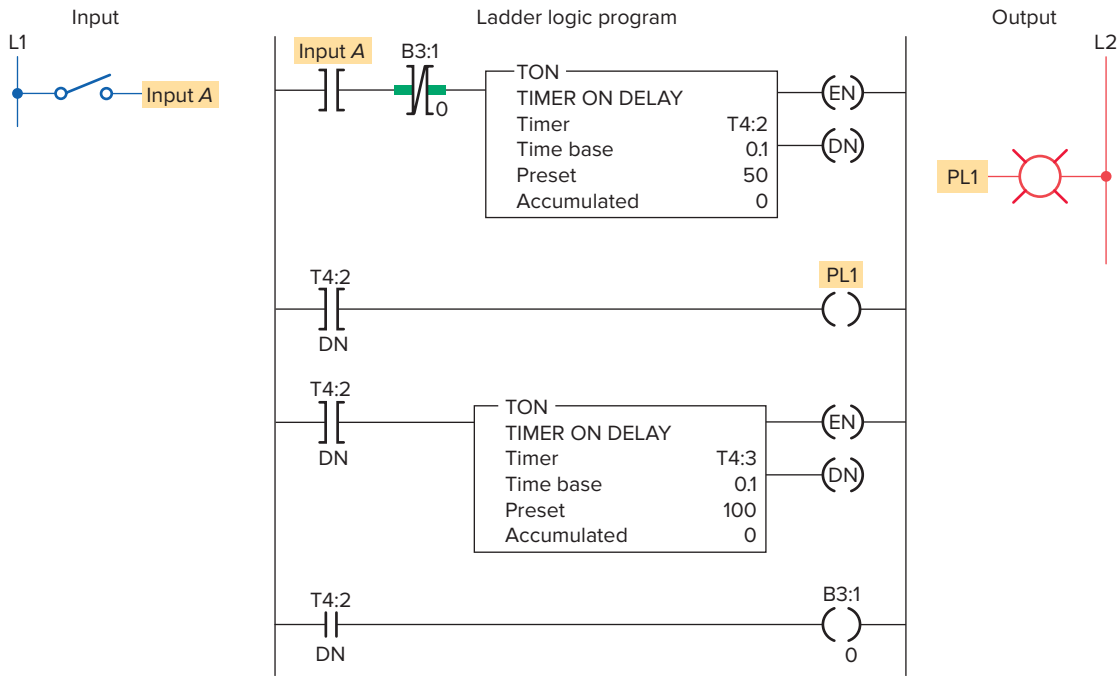
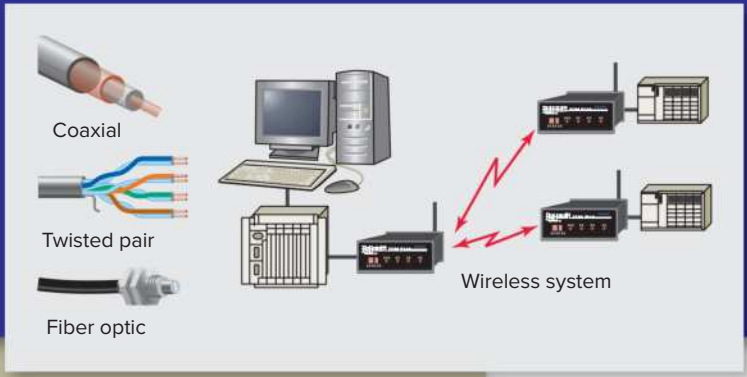


Figure 13-39 Program for Problem 14.

14

Process Control, Network Systems, and SCADA



Chapter Objectives

After completing this chapter, you will be able to:

- Discuss the operation of continuous process, batch production, and discrete manufacturing processes
- Compare individual, centralized, and distributive control systems
- Explain the functions of the major components of a process control system
- Describe the various functions of electronic HMI screens
- Recognize and explain the functions of the control elements of a closed-loop control system
- Explain how on/off control works
- Explain how PID control works
- Identify the levels of an industrial network
- Discuss the different types of network architecture and protocols
- Describe a typical SCADA application

This chapter introduces the kinds of industrial processes that can be PLC controlled. SCADA is such a process. Different types of control systems are used for complex processes. These control systems may be PLCs, but other controllers include robots, data terminals, and computers. For these controllers to work together, they must communicate. This chapter will discuss the different kinds of industrial processes and the means by which they communicate.

14.1 Types of Processes

Process control is the automated control of a process. Such systems typically deal with analog signals from sensors. The ability of a PLC to perform math functions and utilize analog signals makes it ideally suited for this type of control. Manufacturing is based on a series of processes being applied to raw materials. Typical applications of process control systems include automobile assembly, petrochemical production, oil refining, power generation, and food processing.

A ***continuous process*** is one in which raw materials enter one end of the system and the finished product comes out the other end of the system; the process itself runs continuously. Figure 14-1 shows a continuous process used in an automotive engine assembly line. Parts are mounted sequentially, in an assembly-line fashion, through a series of stations. Assembly and adjustments are carried out by both automated machine and manual operations.

In ***batch processing,*** there is no flow of product material from one section of the process to another. Instead, a set amount of each of the inputs to the process is received in a batch, and then some operation is performed on the batch to produce a product. Products produced using the batch process include food, beverages, pharmaceutical products, paint, and fertilizer. Figure 14-2 shows an example of a batch process. Three ingredients are mixed together, heated, and then stored. Recipes are the key to producing batches as each batch may have different characteristics by design.

Discrete manufacturing is characterized by individual or separate unit production. With this manufacturing process, a series of operations produces a useful output product. Discrete manufacturing systems typically deal with digital inputs to PLCs that cause motors and robotic devices to be activated. The work piece is normally a discrete part that must be handled on an individual basis. Making car interiors, as illustrated in Figure 14-3, is one example of discrete manufacturing.

Possible control configurations include individual, centralized, and distributed. ***Individual control*** is used to control a single machine. This type of control does not normally require communication with other controllers. Figure 14-4 shows an individual control application for a cut to length operation. The operator enters the feed length and batch count via the interface control panel and then presses the start button to initiate the process. Stock lengths vary, so the operator needs to select the length and the number of pieces to be cut.

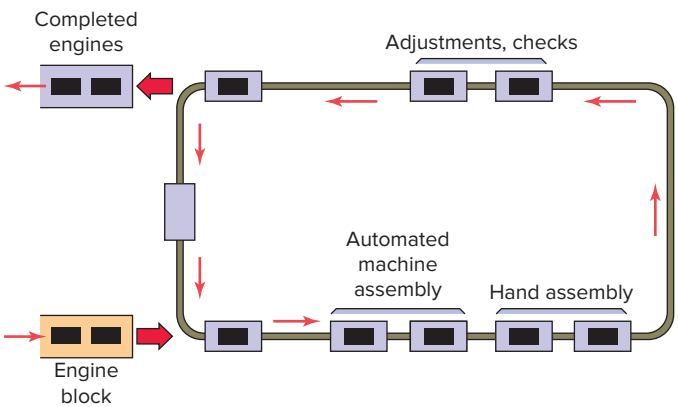


Figure 14-1 Continuous process.

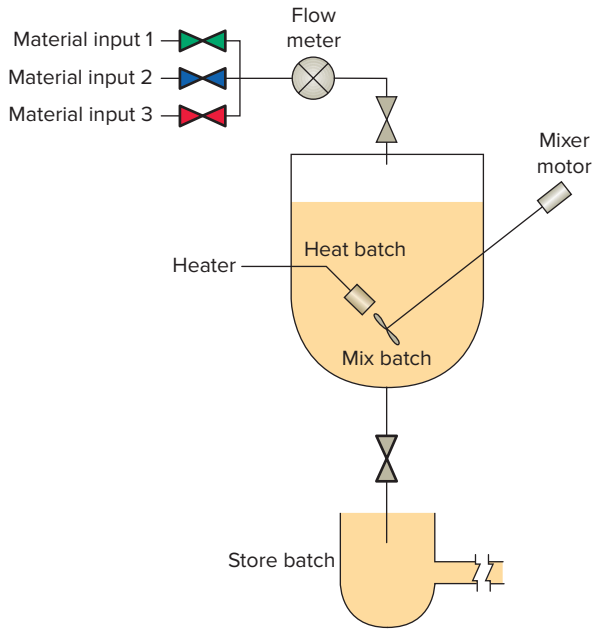


Figure 14-2 Batch process.

Figure 14-3 Discrete manufacturing.
Source: Courtesy Automation IG.

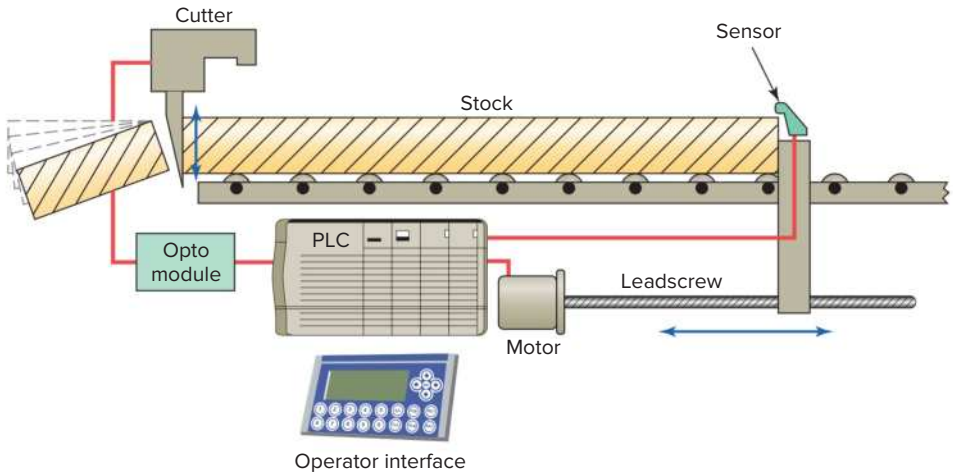


Figure 14-4 Individual control.

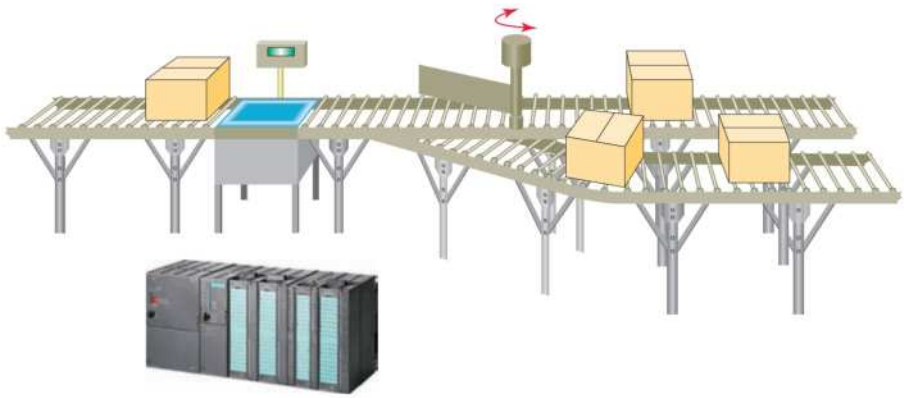


Figure 14-5 Centralized control.
Source: Courtesy Siemens.

Centralized control is used when several machines or processes are controlled by one central controller. The control layout uses a single, large control system to control many diverse manufacturing processes and operations, as illustrated in Figure 14-5. The main features of centralized control can be summarized as follows:

- All individual steps in the manufacturing process are handled by a central control system controller.
- No exchange of controller status or data is sent to other controllers.
- If the main controller fails, the whole process stops.

A ***distributive control system (DCS)*** is a network-based system. Distributive control involves two or more PLCs communicating with each other to accomplish the complete control task, as illustrated in Figure 14-6. Each PLC controls different processes locally and the PLCs are constantly exchanging information through the communications link and reporting on the status of the process.

The main features of a distributive control system can be summarized as follows:

- Distributive control permits the distribution of the processing tasks among several controllers.
- Each PLC controls its associated machine or process.
- High-speed communication among the computers is done through CAT-5 or CAT-6 twisted pair wires, single coaxial cables, fiber optics, or the Ethernet.
- Distributive control drastically reduces field wiring and heightens performance because it places the controller and I/O close to the machine process being controlled.
- Depending on the process, one PLC failure would not necessarily halt the complete process.
- DCS is supervised by a host computer that may perform monitoring/supervising functions such as report generation and storage of data.

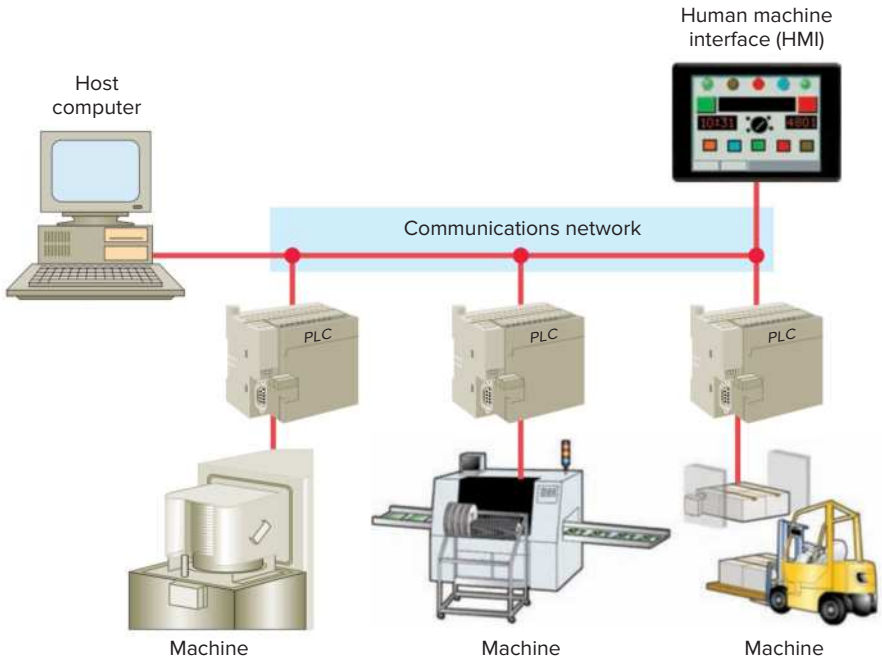


Figure 14-6 Distributive control system (DCS).

14.2 Structure of Control Systems

Process control normally applies to the manufacturing or processing of products in industry. In the case of a programmable controller, the process or machine is operated and supervised under the control of the user program. The major components of a process control system include the following:

Sensors

- Provide inputs from the process and from the external environment
- Convert physical information such as pressure, temperature, flow rate, and position into electrical signals

Human Machine Interface (HMI)

- Allows human inputs through various types of programmed switches, controls, and keypads to set up the starting conditions or alter the control of a process

Signal Conditioning

- Involves converting input and output signals to a usable form
- May include signal-conditioning techniques such as amplification, attenuation, filtering, scaling, A/D and D/A converters

Actuators

- Convert system output electrical signals into physical action
- Process actuators that include flow control valves, pumps, positioning drives, variable speed drives, clutches, brakes, solenoids, stepping motors, and power relays

Controller

- Makes the system's decisions based on the input signals
- Generates output signals that operate actuators to carry out the decisions

Human machine interface (HMI) equipment provides a control and visualization interface between a human and a process (Figure 14-7). HMIs allow operators to control,

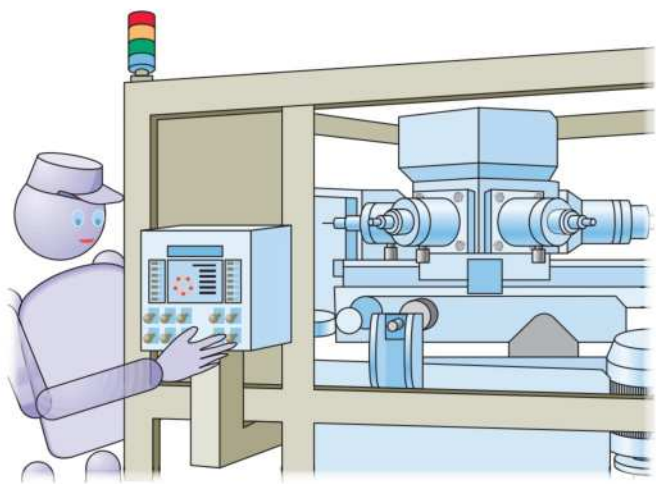


Figure 14-7 Human machine interface (HMI).

monitor, diagnose, and manage the application. Depending on the requirements and complexity of the process, the operator may be required to:

- Stop and start the process.
- Operate the controls and make the adjustments required for the process and monitor its progress.
- Detect abnormal situations and undertake corrective action.

Graphic HMI terminals offer electronic interfacing in a wide variety of sizes and configurations. They replace traditional wired panels with a touch screen with graphical representations of switches and indicators. Types of graphical display screens include the following:

Operational Summary—used to monitor the process.

Configuration/Setup—textual in nature, used to detail process parameters.

Alarm Summary—provides a list of time-stamped active alarms.

Event History—presents a time-stamped list of all significant events that have occurred in the process.

Trend Values—displays information on process variables, such as flow, temperature, and production rate, over a period of time.

Manual Control—generally available only to maintenance personnel and meant to bypass parts of the automatic control system.

Diagnostics—used by maintenance personnel to diagnose equipment failures.

Graphic terminals come fully packaged with hardware, software, and communications. Figure 14-8 shows the Allen-Bradley family of PanelView graphic terminals. The setup varies with the vendor. In general, the tasks required to develop an HMI application include:

- Establish a communication link with the PLCs.
- Create the tag addresses database.

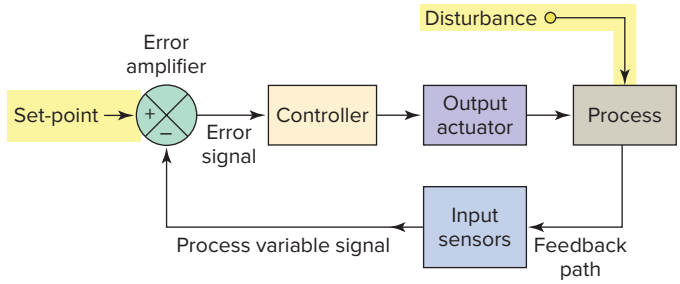


Figure 14-9 Closed-loop control system.

- Edit and create graphical objects on the screens.
- Animate the objects.

Most control systems are closed loop in that they utilize feedback in which the output of a process affects the input control signal. A closed-loop system measures the actual output of the process and compares it to the desired output. Adjustments are made continuously by the control system until the difference between the desired and actual output falls within a predetermined tolerance.

Figure 14-9 illustrates an example of a closed-loop control system. The actual output is sensed and fed back to be subtracted from the set-point input that indicates what output is desired. If a difference occurs, a signal to the controller causes it to take action to change the actual output until the difference is 0. The operation of the component parts are as follows:

Set-point—The input that determines the desired operating point for the process.

Process Variable—Refers to the feedback signal that contains information about the current process status.

Error Amplifier—Determines whether the process operation matches the set-point. The magnitude and polarity of the error signal will determine how the process will be brought back under control.

Controller—Produces the appropriate corrective output signal based on the error signal input.

Figure 14-8 PanelView graphic terminals.
Source: Image Courtesy of Rockwell Automation, Inc.

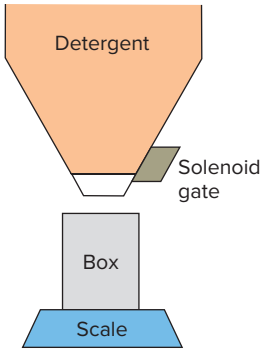


Figure 14-10 Container-filling process.

Output Actuator—The component that directly affects a process change. Examples are motors, heaters, fans, and solenoids.

The process shown in Figure 14-10 is an example of a closed-loop continuous control process used to automatically fill box containers to a specified weight of detergent. An empty box is moved into position and filling begins. The weight of the box and contents is monitored. When the actual weight equals the desired weight, filling is halted.

Operation and block diagrams for the container-filling process are shown in Figure 14-11. The operation of the process can be summarized as follows:

- A sensor attached to the scale weighing the container generates the voltage signal or digital code that represents the weight of the container and contents.
- The sensor signal is subtracted from the voltage signal or digital code that has been input to represent the desired weight.
- As long as the difference between the input signal and feedback signal is greater than 0, the controller keeps the solenoid gate open.
- When the difference becomes 0, the controller outputs a signal that closes the gate.

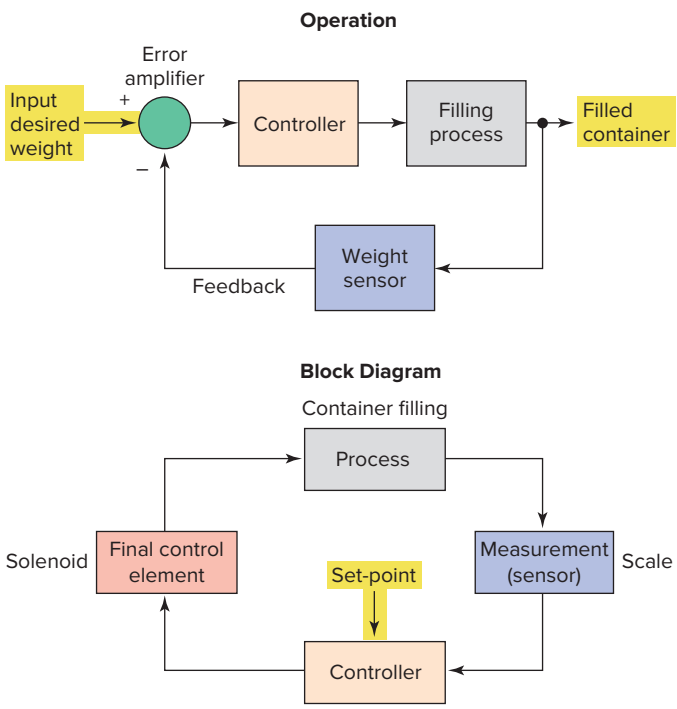


Figure 14-11 Operation and block diagrams for the container-filling process.

Virtually all feedback controllers determine their output by observing the error between the set-point and a measurement of the process variable. Errors can occur when an operator changes the set-point or when a disturbance or a load on the process changes the process variable. The controller's role is to eliminate the error automatically.

14.3 On/Off Control

With ***on/off controllers*** the final control element is either on or off—one for the occasion when the value of the measured variable is above the set-point and the other for the occasion when the value is below the set-point. The controller will never keep the final control element in an intermediate position. Controlling activity is achieved by the period of on-off cycling action.

Figure 14-12 shows a system using on/off control in which a liquid is heated by steam. The operation of the process can be summarized as follows:

- If the liquid temperature goes below the set-point, the steam valve opens and the steam is turned on.
- When the liquid temperature goes above the set-point, the steam valve closes and the steam is shut off.
- The on/off cycle will continue as long as the system is operating.

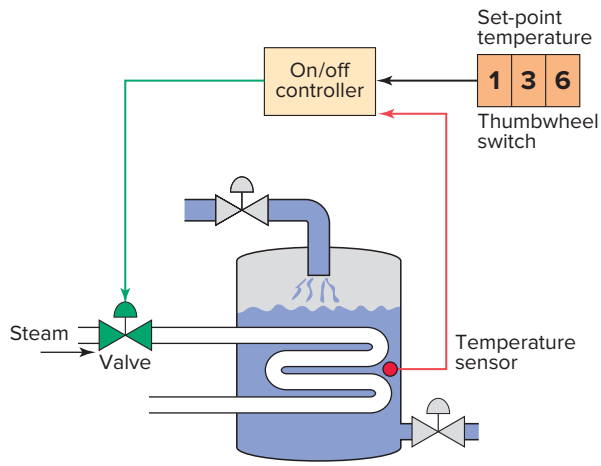


Figure 14-12 On/off controlled liquid heating system.

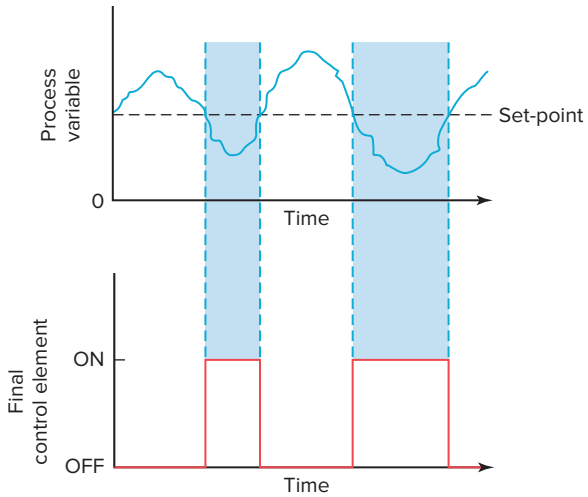


Figure 14-13 On/off control response.

Figure 14-13 illustrates the control response for an on/off temperature controller. The action of the control response can be summarized as follows:

- The output turns on when the temperature falls below the set-point and turns off when the temperature reaches the set-point.
- Control is simple, but overshoot and cycling about the set-point can be disadvantageous in some processes.
- The measured variable will oscillate around the set-point at an amplitude and frequency that depend on the capacity and time response of the process.
- Oscillations may be reduced in amplitude by increasing the sensitivity of the controller. This increase will cause the controller to turn on and off more often, a possibly undesirable result.
- On/off control is used when a more precise control is unnecessary.

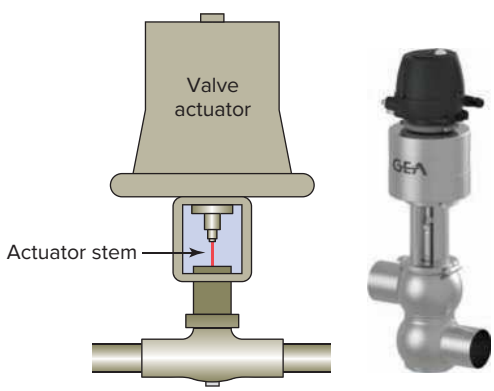
A ***deadband*** is usually established around the set-point. The deadband of the controller is usually a selectable value that determines the error range above and below the set-point that will not produce an output as long as the process variable is within the set limits. The inclusion of deadband eliminates any hunting by the control device around the set-point. **Hunting** occurs when minor adjustments of the controlled position are continually made due to minor fluctuations.

14.4 PID Control

Proportional controllers are designed to eliminate the hunting or cycling associated with on/off control. They allow the final control element to take intermediate positions between on and off. Proportioning action permits ***analog control*** of the final control element to vary the amount of energy to the process, depending on how much the value of the measured variable has shifted from the desired value.

A proportional controller allows tighter control of the process variable because its output can take on any value between fully on and fully off, depending on the magnitude of the error signal. Figure 14-14 shows an example of a motor-driven analog proportional control valve used as a final control element. The action of the control valve actuator can be summarized as follows:

- The actuator receives an input current between 4 and 20 mA from the controller.
- In response, it provides linear control of the valve.
- A value of 4 mA corresponds to a minimum value opening (often 0) and 20 mA corresponds to a maximum value opening (full scale).
- The 4 mA lower limit allows the system to detect opens. If the circuit is open, 0 mA would result, and the system can issue an alarm.
- Because the signal is a current, it is unaffected by reasonable variations in connecting wire resistance and is less susceptible to noise pickup from other signals than is a voltage signal.



Actuator current (mA)	Valve response (% open)
4	0
6	12.5
8	25
10	37.5
12	50
14	62.5
16	75
18	87.5
20	100

Figure 14-14 Motor-driven analog proportional control valve.
Source: Photo Courtesy Gea Group.

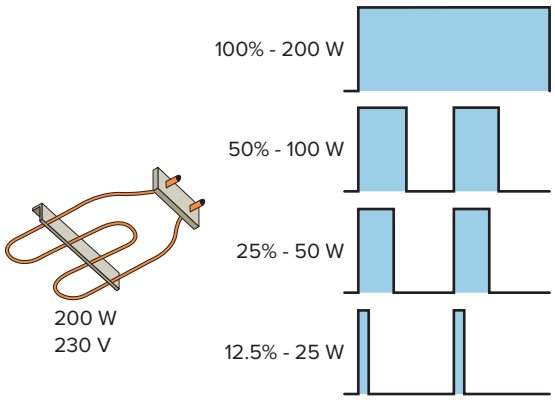


Figure 14-15 Time proportioning of a heater element.

Proportioning action can also be accomplished by turning the final control element on and off for short intervals. This ***time proportioning*** (also known as ***pulse width modulation***) varies the ratio of on time to off. Figure 14-15 shows an example of time proportioning used to produce varying wattage from a 200 watt heater element as follows:

- To produce 100 W the heater must be on 50% of the time.
- To produce 50 W the heater must be on 25% of the time.
- To produce 25 W the heater must be on 12.5% of the time.

Proportioning action occurs within a proportional band around the set-point. The table of Figure 14-16 is an example of the proportional band for a heating application with a set-point of 500°F and a proportional band of 80°F (±40°F). Proportioning action can be summarized as follows:

- Proportional controllers have analog input and output values that vary over the range necessary for control of the process.

Time proportional				4–20 mA proportional	
Percent on	On time (seconds)	Off time (seconds)	Temp. (°F)	Output level	Percent output
0.0	0.0	20.0	over 540	4 mA	0.0
0.0	0.0	20.0	540.0	4 mA	0.0
12.5	2.5	17.5	530.0	6 mA	12.5
25.0	5.0	15.0	520.0	8 mA	25.0
37.5	7.5	12.5	510.0	10 mA	37.5
50.0	10.0	10.0	500.0	12 mA	50.0
62.5	12.5	7.5	490.0	14 mA	62.5
75.0	15.0	5.0	480.0	16 mA	75.0
87.5	17.5	2.5	470.0	18 mA	87.5
100.0	20.0	0.0	460.0	20 mA	100.0
100.0	20.0	0.0	under 460	20 mA	100.0

Figure 14-16 Proportional band for a heating application.

- The current temperature is called the Process Variable (PV), while the desired temperature is known as the Set-Point (SP).
- The proportional controller changes the controller output in proportion to the difference between the SP and PV. The greater the difference, the greater the corrective action applied.
- At the set-point (the midpoint of the proportional band), the output on:off ratio is 1:1; that is, the on time and off time are equal.
- If the temperature is further from the set-point, the on and off times vary in proportion to the temperature difference.
- If the temperature is below the set-point, the output will be on longer; if the temperature is too high, the output will be off longer.

In theory, a proportional controller should be all that is needed for process control. Any change in system output is corrected by an appropriate change in controller output. Unfortunately, the operation of a proportional controller leads to a steady-state error known as ***offset,*** or ***droop.*** This steady-state error is the difference between the attained value of the controller and the required value that results in an offset signal that is slightly lower than the set-point value, as illustrated in Figure 14-17. Depending on the PLC application, this offset may or may not be acceptable.

Proportional control is often used in conjunction with integral control and/or derivative control.

- The ***integral action,*** sometimes termed reset action, responds to the size and time duration of the error signal. An error signal exists when there is a difference between the process variable and the set-point, so the integral action will cause the output to change and continue to change until the error no longer exists. Integral action eliminates steady-state error. The amount of integral action is measured as minutes per repeat or repeats per minute, which is the relationship between changes and time.

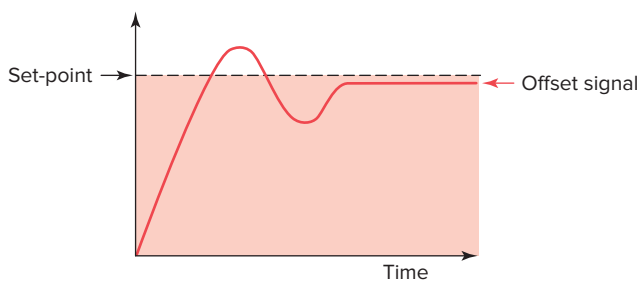


Figure 14-17 Proportional control steady-state error.

- The ***derivative action*** responds to the speed at which the error signal is changing—that is, the greater the error change, the greater the correcting output. The derivative action is measured in terms of time.

Proportional plus integral (PI) control combines the characteristics of both types of control. A step change in the set-point causes the controller to respond proportionally, followed by the integral response, which is added to the proportional response. Because the integral mode determines the output change as a function of time, the more integral action found in the control, the faster the output changes. This action can be summarized as follows:

- To eliminate the offset error, the controller needs to change its output until the process variable error is zero.
- Reset integral control action changes the controller output by the amount needed to drive the process variable back to the set-point value.
- After the reset integral control action a new equilibrium point is established.
- Since the proportional controller must always operate on its proportional band, the proportional band must be shifted to include the new equilibrium point.
- A controller with reset integral control does this automatically.

Rate action (derivative control) acts on the error signal just like reset does, but rate action is a function of the rate of change rather than the magnitude of error. Rate action is applied as a change in output for a selectable time interval, usually stated in minutes. Rate-induced change in controller output is calculated from the derivative of the error. Input change, rather than proportional control error change, is used to improve response. Rate action quickly positions the output, whereas proportional action alone would eventually position the output. In effect, rate action puts the brakes on any offset or error by quickly shifting the proportional band. ***Proportional plus derivative (PD) control*** is used in process control systems with errors that change very rapidly. By adding derivative control to proportional control, we obtain a controller output that responds to the error's rate of change as well as to its magnitude.

PID control is a feedback control method that combines proportional, integral, and derivative actions. The proportional action provides smooth control without hunting. The integral action automatically corrects offset. The derivative action responds quickly to large external disturbances. The PID controller is the most widely used type of process controller. When combined into a single control loop the proportional, integral, and derivative modes complement each other to reduce the system error to zero faster than any other controller. Figure 14-18 shows the block diagram of a PID control loop, the operation of which can be summarized as follows:

- During setup, the set-point, proportional band, reset (integral), rate (derivative), and output limits are specified.
- All these can be changed during operation to tune the process.
- The integral term improves accuracy, and the derivative reduces overshoot for transient upsets.
- The output can be used to control valve positions, temperature, flow metering equipment, and so on.
- PID control allows the output power level to be varied.
- As an example, assume that a furnace is set at 50°C.
- The heater power will increase as the temperature falls below the 50°C set-point.
- The lower the temperature the higher the power.
- PID has the effect of gently turning the power down as the signal gets close to the set-point.

The long-term operation of any system, large or small, requires a mass-energy balance between input and output. If a process were operated at equilibrium at all times, control would be simple. Because change does occur, the critical parameter in process control is time, that is, how long it takes for a change in any input to appear in the output. System time constants can vary from fractions of a second to many hours. The PID controller has the ability to tune its control action to specific process time constants and therefore to deal with process changes over time. PID control changes the amount of output signal in a *mathematically* specified way that accounts for the amount of error and the rate of signal change.

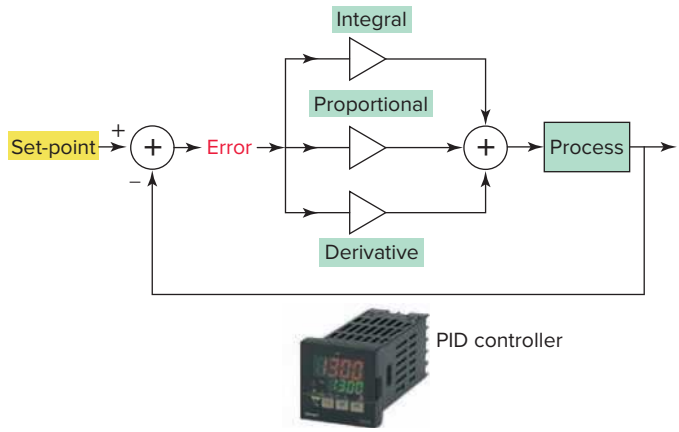


Figure 14-18 PID control loop.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

Either programmable controllers can be fitted with input/output modules that produce PID control, or they will already have sufficient mathematical functions to allow PID control to be carried out. PID is essentially an equation that the controller uses to evaluate the controlled variable. Figure 14-19 illustrates how a programmable logic controller can be used in the control of a PID loop. The operation of the PID loop can be summarized as follows:

- The process variable (pressure) is measured and feedback is generated.
- The PLC program compares the feedback to the set-point and generates an error signal.
- The error is examined by the PID loop calculation in three ways: with proportional, integral, and derivative methodology.
- The controller then issues an output to correct for any measured error by adjustment of the position of the variable flow outlet valve.

The ***response*** of a PID loop is the rate at which it compensates for error by adjusting the output. The PID loop is adjusted or tuned by changing the proportional gain, the integral gain, and/or the derivative gain. A PID loop is normally tested by making an abrupt change to the set-point and observing the controller's response rate. Adjustments can then be made as follows:

- As the proportional gain is increased, the controller responds faster.
- If the proportional gain is too high, the controller may become unstable and oscillate.
- The integral gain acts as a stabilizer.
- Integral gain also provides power, even if the error is zero (e.g., even when an oven reaches its set-point, it still needs power to stay hot).
- Without this base power, the controller will droop and hunt for the set-point.
- The derivative gain acts as an anticipator.
- Derivative gain is used to slow the controller down when change is too fast.

Basically, PID controller tuning consists of determining the appropriate values for the gain (proportional band), rate (derivative), and reset time (integral) tuning parameters (control constants) that will give the control required. Depending on the characteristics of the deviation of the process variable from the set-point, the tuning parameters interact to alter the controller's output and produce changes in the value of the process variable. In general, three methods of controller tuning are used:

Manual

- The operator estimates the tuning parameters required to give the desired controller response.
- The proportional, integral, and derivative terms must be adjusted, or tuned, individually to a particular system using a trial-and-error method.

Semiautomatic or Autotune

- The controller takes care of calculating and setting PID parameters.
  - Measures sensor output

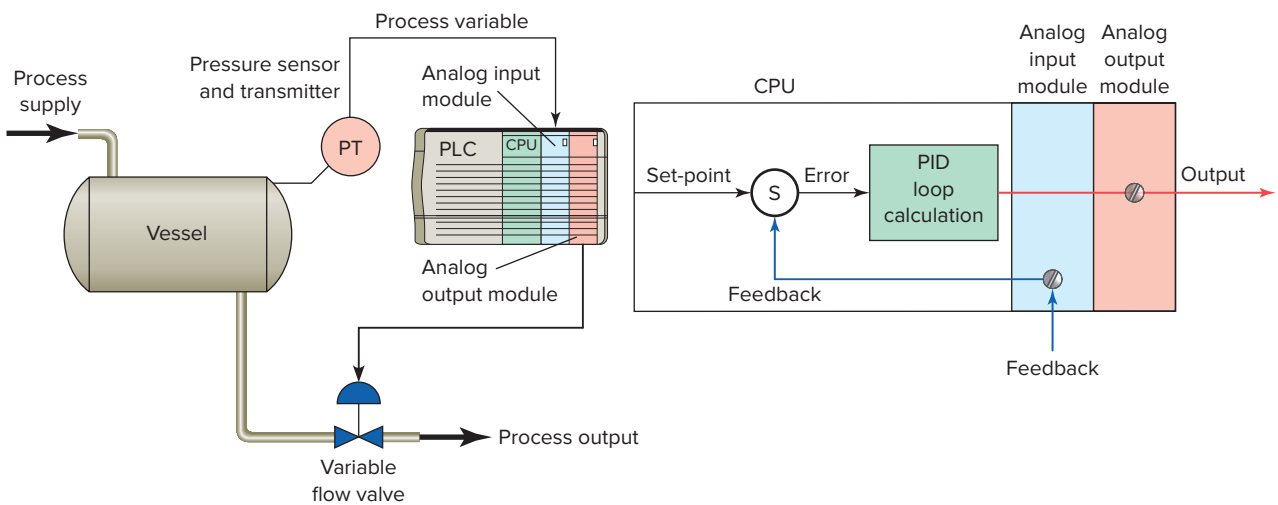


Figure 14-19 PLC control of a PID loop.

– Calculates error, sum of error, rate of change of error
– Calculates desired power with PID equations
– Updates control output

Fully Automatic or Intelligent

- This method is also known in the industry as fuzzy logic control.
- The controller uses artificial intelligence to readjust PID tuning parameters continually as necessary.
- Rather than calculating an output with a formula, the fuzzy logic controller evaluates rules. The first step is to "fuzzify" the error and change-in-error from continuous variables into linguistic variables, like "negative large" or "positive small." Simple if-then rules are evaluated to develop an output. The resulting output must be de-fuzzified into a continuous variable such as valve position.

The PID programmable controller output instruction uses closed-loop control to automatically control physical properties such as temperature, pressure, liquid level, or flow rate of process loops. Figure 14-20 shows the PID output instruction and setup screen associated with the Allen-Bradley SLC 500 instruction set. The PID instruction is straightforward: it takes one input and controls one output. Normally, the PID instruction is placed on a rung without conditional logic. The output remains at its last value when the rung goes false. A summary of the basic information that is entered into the instruction is as follows:

Control Block—File that stores the data required to operate the instruction.

Process Variable—The element address that stores the process input value.

Control Variable—The element address that stores the output of the PID instruction.

Setup Screen—Instruction on which you can double-click to bring up a display that prompts you for other parameters you must enter to fully program the PID instruction.

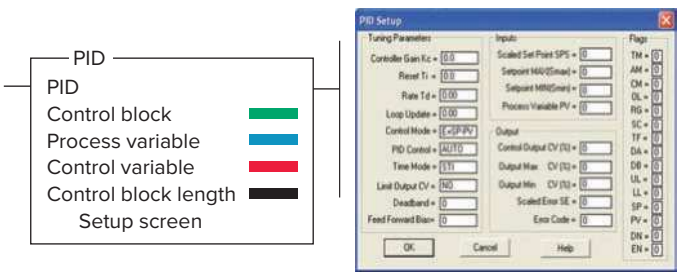


Figure 14-20 PID output instruction and setup screen.
Source: Image Courtesy of Rockwell Automation, Inc.

14.5 Motion Control

A motion control system provides precise positioning, velocity, and torque control for a wide range of motion applications. PLCs are ideally suited for both linear and rotary motion control applications. ***Pick and Place*** machines are used in the consumer products industry for a wide variety of product transfer applications. The machine takes a product from one point to another. One example is the transfer of a product to a moving conveyor belt as illustrated in Figure 14-21.

A basic PLC motion control system consists of a controller, a motion module, a servo drive, one or more motors with encoders, and the machinery being controlled. Each motor controlled in the system is referred to as an axis of motion. Figure 14-22 illustrates a bottle-filling motion control process. This application requires two axes of motion: the motor operating the bottle filler mechanism and the motor controlling the conveyor speed. The role of each control component can be summarized as follows:

Programmable Logic Controller

- The controller stores and executes the user program that controls the process.
- This program includes motion instructions that control axis movements.
- When the controller encounters a motion instruction it calculates the motion commands for the axis.
- A motion command represents the desired position, velocity, or torque of the servo motor at the particular time the calculations take place.

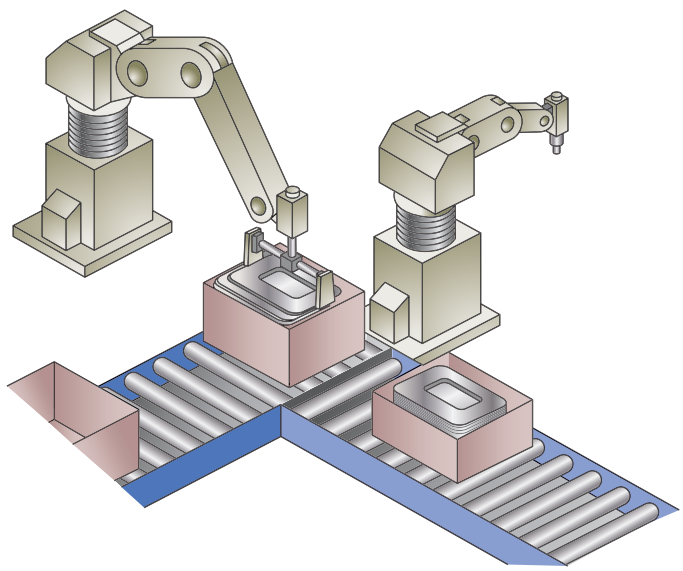


Figure 14-21 Pick and Place machine.

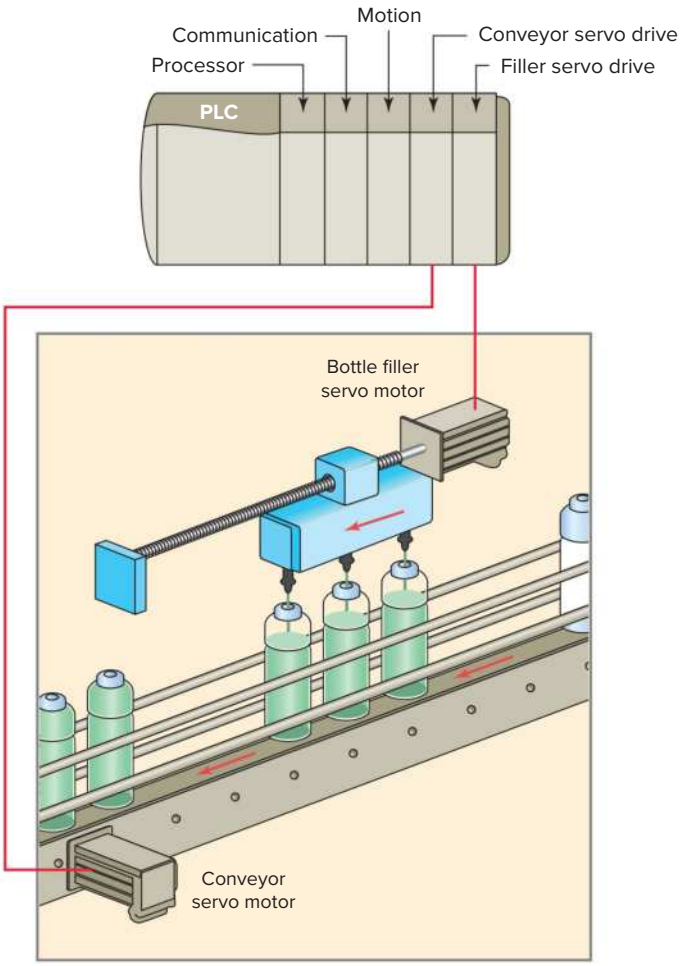


Figure 14-22 Bottle-filling motion control process.

Motion Module

- The motion module receives motion commands from the controller and transforms them into a compatible form the servo drive can understand.
- In addition it updates the controller with motor and drive information used to monitor drive and motor performance.

Servo Drive

- The servo drive receives the signal provided by the motion module and translates this signal into motor drive commands.
- These commands can include motor position, velocity, and/or torque.
- The servo drive provides power to the servo motors in response to the motion commands.
- Motor power is supplied and controlled by the servo drive.
- The servo drive monitors the motor's position and velocity by use of an encoder mounted on the motor shaft. This feedback information is used within the servo drive to ensure accurate motor motion.

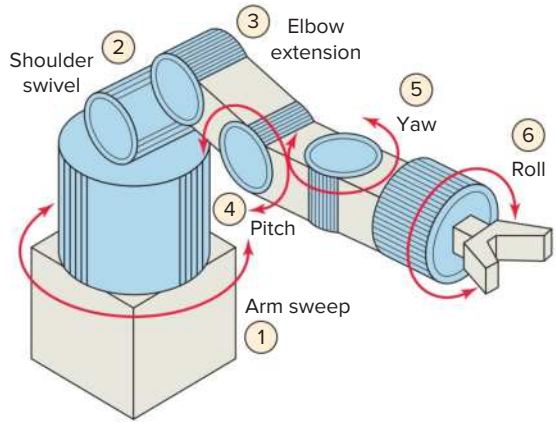


Figure 14-23 Six-axis robot arm.

Servo Motor

- The servo motors represent the axis being controlled.
- The servo motors receive electrical power from their servo drive which determines the motor shaft velocity and position.
- The filler motor must accelerate the filler mechanism in the direction the bottles are moving, match their speed, and track the bottles.
- After the bottles have been filled, the filler motor has to stop and reverse direction to return the filler mechanism to the starting position to begin the process again.

A robot is simply a series of mechanical links driven by servo motors. The basic industrial robot widely used today is an ***arm*** or ***manipulator*** that moves to perform industrial operations. Figure 14-23 illustrates the motion of a six-axis robot arm. Each axis of the robot arm is fundamentally a closed-loop servo control system. The wrist is the name usually given to the last three joints on the robot's arm. Going out along the arm, these wrist joints are known as the pitch joint, yaw joint, and roll joint. There are two types of controller setups that can be used to control an industrial robot—PLC- and PC-based systems. Depending on the difficulty of the task the robotic system will be performing, you may need a PLC or just a robot controller.

14.6 Data Communications

Data communications refers to the different ways that PLC microprocessor-based systems talk to each other and to other devices. The two general types of communications links that can be established between the PLC and other devices are point-to-point links and network links.

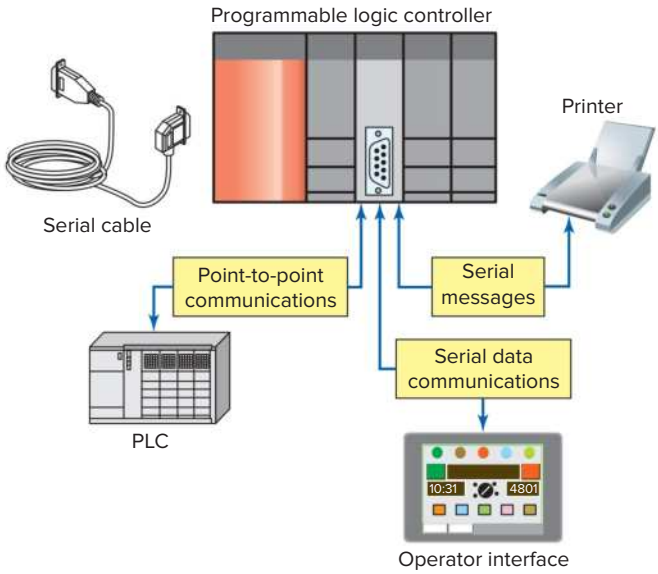


Figure 14-24 Point-to-point serial communications link.

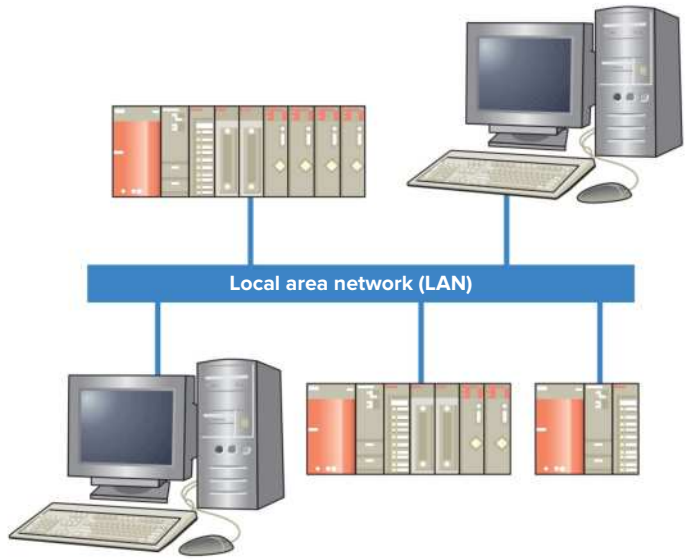


Figure 14-25 Local area network (LAN) communication link.

Figure 14-24 illustrates a ***point-to-point*** serial communications link. Serial communications is used with devices such as printers, operator workstations, motor drives, bar code readers, computers, or another PLC. Serial communications interfaces are either built into the processor module or come as separate modules. A serial module installed in each controller is normally all that is required for two PLCs of the same manufacturer to establish a point-to-point link.

As control systems become more complex, they require more effective communications schemes between the system components. A *local area network* or *LAN* is a system that interconnects data communications components within a limited geographical area, typically no more than one or two miles. Essentially, a LAN is a private, on-site communications system that allows communication between computers, PLCs, robots, and the like. The rate at which characters can be transmitted along a communications line is dependent on the number of bits of binary information that can be sent at a given time. This transfer of information is measured in bits per second, or **baud**. Figure 14-25 illustrates a LAN communication link. Network communications supports communication among multiple PLCs and other devices. PLC networks allow:

- Sharing of information such as the current state of status bits among PLCs that may determine the action of one another.
- Monitoring of information from a central location.
- Programs to be uploaded or downloaded from a central location.
- Several PLCs to operate in unison to accomplish a common goal.

Transmission media are the cable through which data and control signals flow on a network. The transmission media used in data communications systems include coaxial cable, twisted pair, or fiber optics (Figure 14-26). Each cable has different electrical capabilities and may be more or less suitable to a specific environment or network type. Not all networks transmit information through cable. Wireless Wi-Fi Ethernet networks, such as the DF1 Radio Modem, communicate through radio waves, which are transmitted through the air. Fiber optic transmission medium is becoming increasingly popular in PLC based applications. Compared with twisted pairs of wire or coaxial cables, fiber optic cables have advantages such as their small size, no electromagnetic interference, and long transmission ranges. Typically, a fiber optic cable can carry a 1,000-megabaud signal 10 km. These types of cables have very large bandwidths so they are well-suited to high-speed transmission in noisy environments.

In industrial applications, **LANs** have most often been used as the communication system for distributed control systems (DCS). Recall that a DCS system

Figure 14-26 Transmission media.

uses individual controllers to control the subsystems of a machine or process. This approach contrasts with centralized control in which a single controller governs the entire operation. A second major use of local area networks is that of supervisory control and data acquisition (SCADA). A LAN allows data collection and processing for a group of controllers to be accomplished using one host computer as the central point for collecting data.

There are three general levels of functionality of industrial networks. Figure 14-27 shows an illustration of the three levels, which can be summarized as follows:

Device Level—The device level involves various sensor and actuator devices of machines and processes. These may include devices such as sensors, switches, drives, motors, and valves.

Control Level—The control level involves the network's industrial controllers. This level may include controllers such as PLCs and robot controllers. Communications on the control level includes sharing I/O and program data between controllers.

Information Level—The information level is a plantwide network typically composed of the company's business networks and computers. This level may include scheduling, sales, management, and corporatewide information.

Each device connected on a network is known as a ***node*** or ***station***. As signals travel along a network cable, they

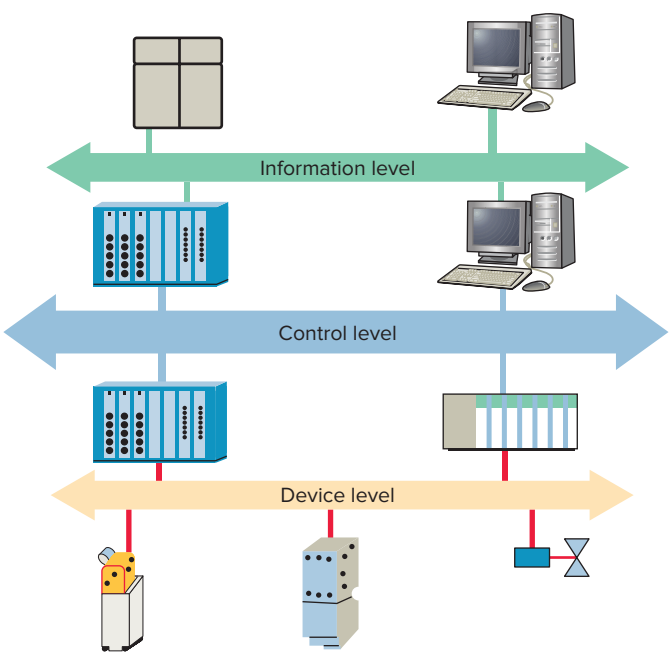


Figure 14-27 Levels of functionality of industrial networks.

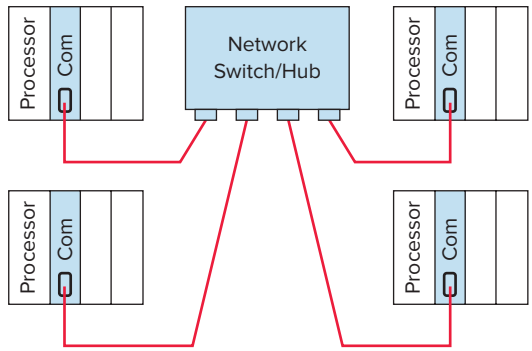


Figure 14-28 Star topology network.

degrade and become distorted in a process that is called attenuation. If a cable is long enough, the attenuation will finally make a signal unrecognizable. A ***repeater*** is a device that amplifies a signal to its original strength in order to enable its signals to travel further. Different network types will have different specifications for cable length and type without a repeater.

Network topology is the physical layout of devices on a network formed by the network cables when nodes are attached. The ***star topology*** illustrated in Figure 14-28 and its operation can be summarized as follows:

- A network switch or hub is connected to several PLC network nodes.
- Currently, most Ethernet networks use switches rather than hubs. A switch performs the same basic function as a hub but effectively increases the speed, size, and data handling capacity of the network.
- The configuration allows for bidirectional communication between switch/hub and each PLC.
- PLCs can be added to or removed from the network without disrupting the network.
- One problem with the star topology is that if the switch/hub goes down, the entire LAN is down.
- This type of system works best when information is transmitted primarily between the main controller and remote PLCs. However, if most communication is to occur between PLCs, the operation speed is affected.
- Also, the star system can use substantial amounts of communication conductors to connect all remote PLCs to one central location.

Bus topology, illustrated in Figure 14-29, is a network configuration in which all stations are connected in parallel with the communication medium and all stations receive information from every other station on the

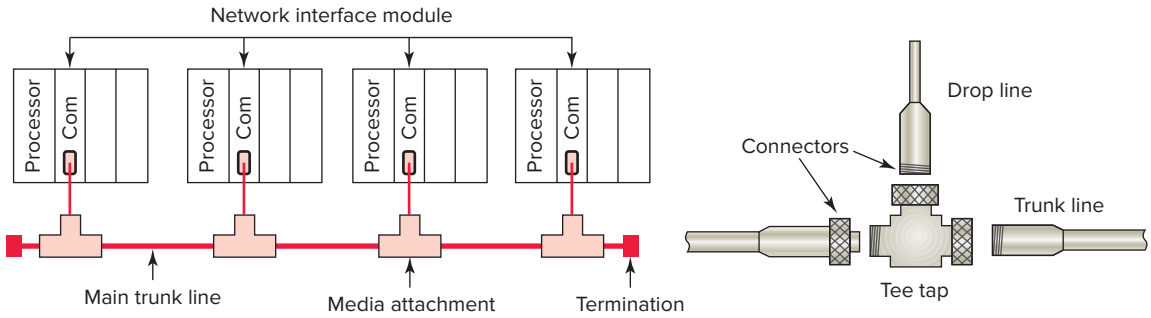


Figure 14-29 Bus topology network.

network. The operation of a bus topology network can be summarized as follows:

- Uses a single bus trunk cable to which individual PLC nodes are attached by a cable drop that taps off the main cable.
- Each PLC is interfaced to the bus using a network interface module that is attached using a drop cable or connector.
- Due to the nature of the bus technology, and the way the data are transmitted on the network, each end of the bus must be terminated with a terminating resistor.
- As the data move along the total bus, each PLC node is listening for its own node identification address and accepts only information sent to that address.
- Because of the simple linear layout, bus networks require less cable than all other topologies.
- No single station controls the network and stations can communicate freely to one another.
- Bus networks are very useful in distributive control systems, because each station or node has equal independent control capability and can exchange information at any given time.
- Another advantage of the bus network is that you can add or remove stations from the network with a minimum amount of system reconfiguration.
- This network's main disadvantage is that all the nodes rely on a common bus trunk line, and a break in that common line can affect many nodes.

I/O bus networks can be divided into two categories: device bus networks and process bus networks. ***Device bus networks*** interface with low-level information devices such as pushbuttons and limit switches that primarily transmit data relating to the on/off state of the device and its operational status. Device bus networks can be further classified as bit-wide or byte-wide buses. Device bus networks that include discrete devices as well as small analog devices are called ***byte-wide bus networks.*** These networks can transfer 50 or more bytes of data at a time. Device bus networks that interface only with discrete devices are called ***bit-wide bus networks.*** Bit-wide networks transfer less than 8 bits of information to and from simple discrete devices.

Process bus networks are capable of communicating several hundred bytes of data per transmission. The majority of devices used in process bus networks are analog, whereas most devices used in device bus networks are discrete. Process bus networks connect with high-level information devices such as smart process valves and flowmeters, which are typically used in process control applications. Process buses are slower because of their large data packet size. Most analog control devices are used in controlling such process variables as flow and temperature, which are typically slow to respond.

A ***protocol*** is a set of rules that two or more devices must follow if they are to communicate with each other. Protocols are to computers what language is to humans. This book is in English, and to understand it, you must be able to read English. Similarly, for two devices on a network to successfully communicate, they must both understand the same protocols.

A network protocol defines how data is arranged and coded for transmission on a network. In the past, communications networks were often proprietary systems designed to a specific vendor's standards; users were forced to buy all their control components from a single supplier. This is because of the different communications protocols, command sequences, error-checking schemes, and communications media used by each manufacturer. Today, the trend is toward open network systems based on international standards developed through industry associations.

Most PLCs adhere to the protocols established by the International Organization for Standardization, or ISO. The Open Systems Interconnection (**OSI model**) is a set of seven layers that define the different stages that data must go through to travel from one device to another over a network. The OSI model takes into account the

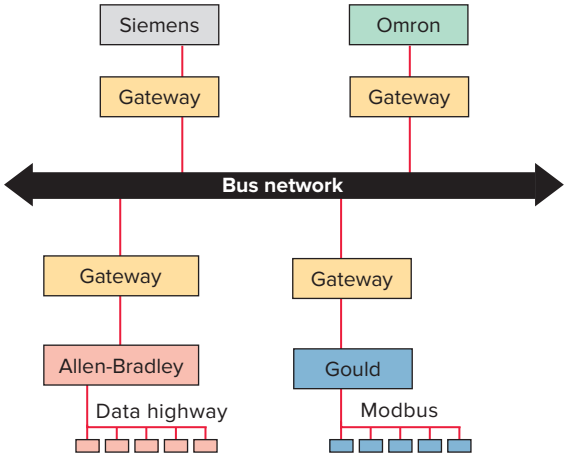


Figure 14-30 Translating from one network-access scheme to another.

hardware, software, protocols, and network architecture that are needed for any two machines to communicate. The actual connections between machines are physical, while the connections between layers in the model are logical.

Gateways (Figure 14-30) make communication possible between different architectures and protocols. They repackage and convert data going from one network to another network so that the one can understand the other's application data. Gateways can change the format of a message so that it will conform to the application program at the receiving end of the transfer. If network-access translation is their only function, the interfaces are known as ***bridges.*** If the interface also adjusts data formats or performs data transmission control, then it is called a ***gateway.***

A bus topology network requires some method of controlling a particular device's access to the bus. An ***access method*** is the manner in which a PLC accesses the network to transmit information. Network access control ensures that data are transmitted in an organized manner preventing the occurrence of more than one message on the network at a time. Although many access methods exist, the most common are token passing, collision detection, and polling.

In a ***token passing*** network, a node can transmit data on the network only when it has possession of a token. A token is simply a small packet that is passed from node to node as illustrated in Figure 14-31. When a node finishes transmitting messages, it sends a special message to the next node in the sequence, granting it the token. The token passes sequentially from node to node, allowing each an opportunity to transmit without interference. Tokens usually have a time limit to prevent a single node from tying up the token for a long period of time.

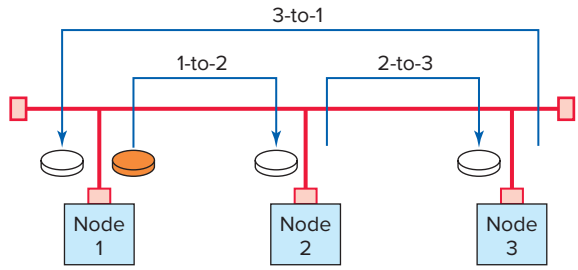


Figure 14-31 Example of token passing.

Ethernet networks use a ***collision detection*** access control scheme. With this access method, nodes listen for activity on the network and transmit only if there are no other messages on the network. On Ethernet networks there is the possibility that nodes will transmit data at the same time. When this happens a collision is detected. Each node that had sent out a message will wait a random amount of time and will resend its data if it does not detect any network activity.

The access method most often used in master/slave protocols is ***polling.*** The **master/slave network** is one in which a master controller controls all communications originating from other controllers. This configuration is illustrated in Figure 14-32 and consists of several slave controllers and one master controller. Its operation can be summarized as follows:

- The master controller sends data to the slave controllers.
- When the master needs data from a slave, it will *poll* (address) the slave and wait for a response.

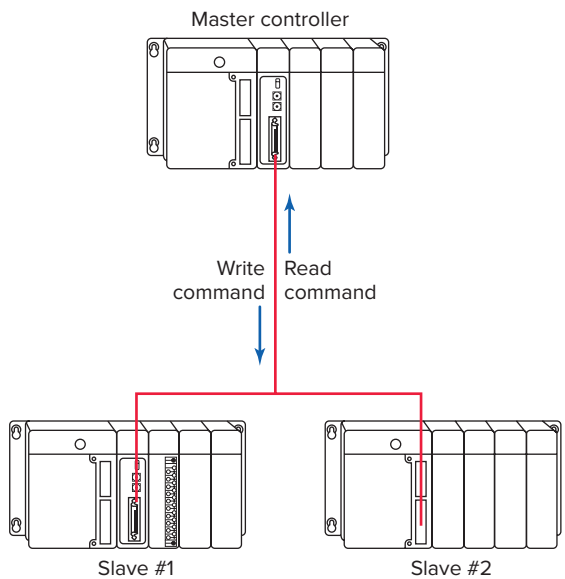


Figure 14-32 Master/slave network.

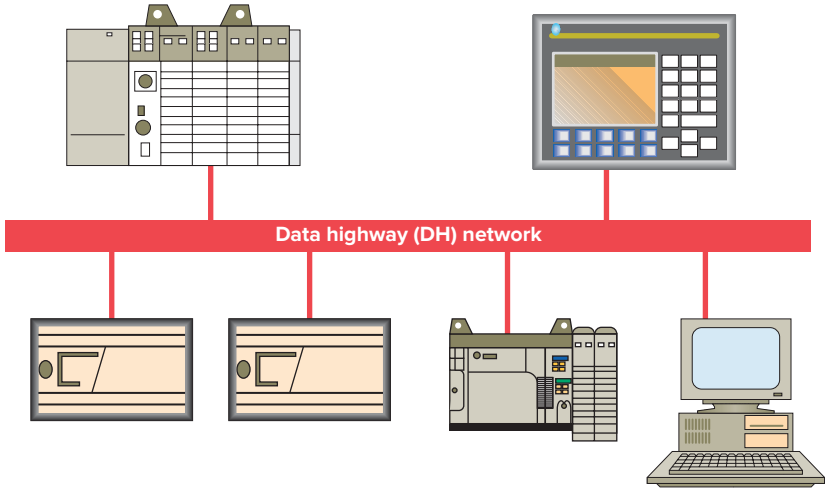


Figure 14-33 Peer-to-peer network.

- No communication takes place without the master initiating it.
- Direct communication among slave devices is not possible.
- Information to be transferred between slaves must be sent first to the network master unit, which will, in turn, retransmit the message to the designated slave device.
- Master/slave networks use two pairs of conductors. One pair of wires is used for the master to transmit data and the slave to receive them. On the other pair, the slaves transmit and the master receives.

A **peer-to-peer network** has a distributive means of control, as opposed to a master/slave network in which one node controls all communications originating from other nodes. The Allen-Bradley Data Highway, shown in Figure 14-33, is an example of a peer-to-peer network of programmable controllers and computers linked together to form a data communication system. The operation of the network can be summarized as follows:

- Peer-to-peer networks can use token passing or Ethernet collision detection.
- Each device has the ability to request use of, and then take control of, the network for the purpose of transmitting information to or requesting information from other network devices.
- Each device is identified by an address.
- When the network is operating, the token passes from one device to the next sequentially.
- The device that is transmitting the token also knows the address of the next station that will receive the token.
- Each device receives the packet information and uses it, if needed.
- Any additional information that the node has will be sent in a new packet.

There are two methods of transmitting PLC digital data: parallel and serial transmission. In ***parallel*** data transmission, all bits of the binary data are transmitted simultaneously, as illustrated in Figure 14-34. Parallel transmission of data can be summarized as follows:

- Eight transmission lines are required to transmit the 8-bit binary number.
- Each bit requires its own separate data path and all bits of a word are transmitted at the same time.
- Parallel data transmission is less common but faster than serial transmission.

In ***serial*** transmission one bit of the binary data is transferred at a time, as illustrated in Figure 14-35. Serial transmission of data can be summarized as follows:

- In serial transmission, bits are sent sequentially on the same channel (wire) which reduces costs for wire but also slows the speed of transmission.

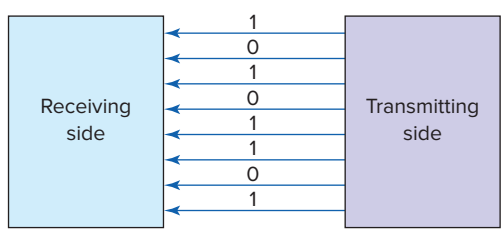


Figure 14-34 Parallel data transmission.

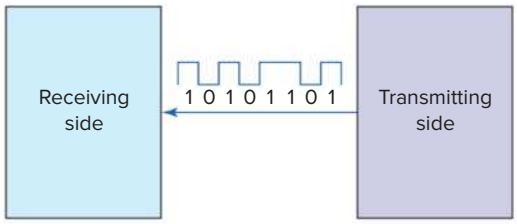


Figure 14-35 Serial data transmission.

- Serial data can be transmitted effectively over much greater distances than can parallel data.
- Each data word in the serial transmission must be denoted with a known start bit sequence followed by the data bits that contain the intelligence of the data transmission and a stop bit.
- An extra bit, termed a ***parity bit,*** may be used to provide some error-detecting ability.

A ***duplex*** communication system is a system composed of two connected devices that can communicate with one another in both directions at the same time. A *half-duplex* system provides for communication in both directions, but only one direction at a time (not simultaneously). Half-duplex transmission is used for master/slave communications. ***Full-duplex*** transmission allows the transmission of data in both directions simultaneously and can be used for peer-to-peer communications.

The different networking schemes replace traditional point-to-point hardwiring. Network control of systems minimizes the amount of wiring that needs to be done. With traditional wiring multiple wires from each device, fed through control cabinets, often result in large wire bundles running through the system. Due to the sheer volume of wires, installation time is considerable and troubleshooting is complex. If a network is used all devices can be directly connected to a single transmission media cable.

High-speed industrial networking technologies offer a variety of methods for connecting devices. PLC network configurations may be either open or proprietary (vendor-unique). Following is an overview of some of the industrial communication technologies that play a critical role in today's control systems.

Data Highway

The Allen-Bradley Data Highway networks, Data Highway Plus (DH+) and DH-485, are proprietary communications networks. They use peer-to-peer communication implementing token passing. The medium is shielded twisted pair cable. Figure 14-36 shows the DH+ network connection for an SLC 5/04 controller. The three-pin Phoenix connector is used to form the network transmission media.

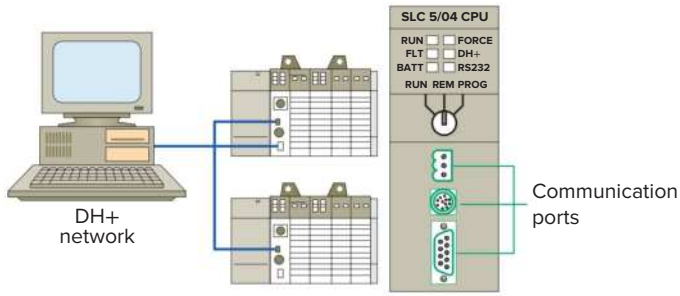


Figure 14-36 Data Highway network connection.

Serial Communication

Serial data communication is implemented using standards such as RS-232, RS-422, and RS-485. The RS in the standard's name means ***recommended standard*** that specifies the electrical, mechanical, and functional characteristics for serial communications. Serial communication interfaces are either built into the processor module or come as a separate communications interface module, as illustrated in Figure 14-37. The simplest type of connection is the RS-232 serial port. The RS interfaces are used to connect to devices such as vision systems, barcode readers, and operator terminals that must transfer quantities of data at a reasonably high rate between the remote device and the PLC. The RS-232 type of serial transmission is designed to communicate between one computer and one controller and is usually limited to lengths up to 50 feet. RS-422 and RS-485 serial transmission types are designed to communicate between one computer and multiple controllers, have a high level of noise immunity, and are usually limited to lengths of 650 feet (for RS-485) or 1650 feet (for RS-422).

DeviceNet

DeviceNet is an open device-level network. It is relatively low speed but efficient at handling the short messages

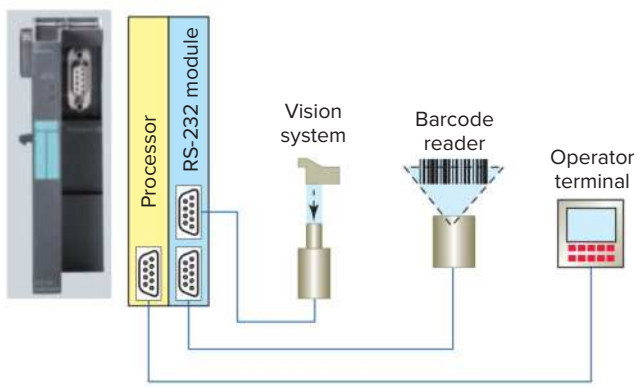


Figure 14-37 Serial communication interface.
Source: Courtesy Siemens.

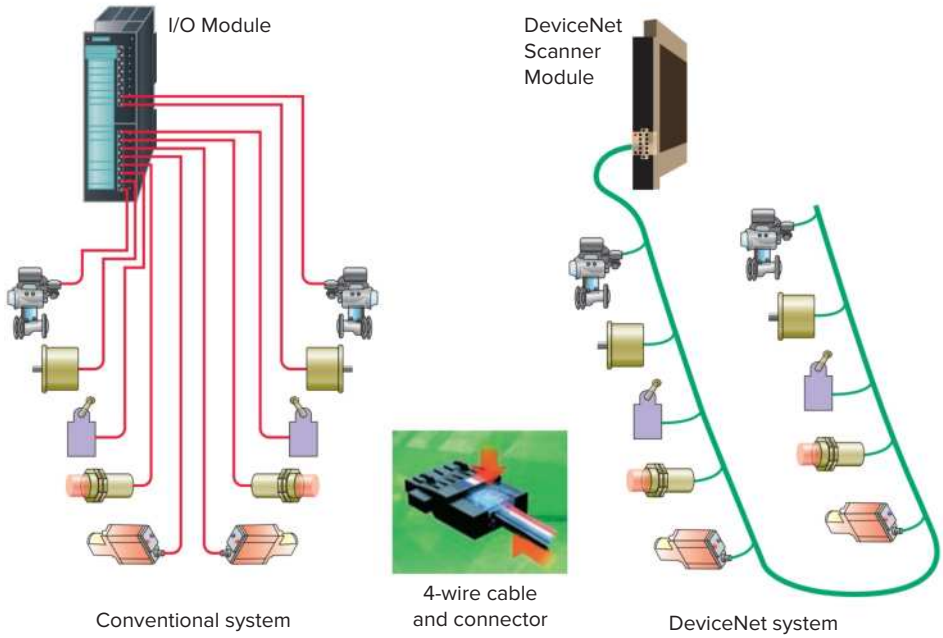


Figure 14-38 Conventional and DeviceNet I/O systems.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

to and from I/O modules. As PLCs have become more powerful, they are being required to control an increasing number of I/O field devices. Therefore, at times it may not be practical to separately wire each sensor and actuator directly into I/O modules. Figure 14-38 shows a comparison between conventional and DeviceNet I/O systems. Conventional systems have racks of inputs and outputs with each I/O device wired back to the controller. The DeviceNet protocol dramatically reduces costs by integrating all I/O devices on a 4-wire trunk network with data and power conductors in the same cable. This direct connectivity reduces costly and time-consuming wiring.

The basic function of a DeviceNet I/O bus network is to communicate information with, as well as supply power to, the field devices that are connected to the bus. The PLC drives the field devices directly with the use of a ***network scanner*** instead of I/O modules, as illustrated in Figure 14-39. The scanner module communicates with DeviceNet devices over the network to:

- Read inputs from a device.
- Write outputs to a device.
- Download configuration data.
- Monitor a device's operational status.

The scanner module communicates with the controller to exchange information which includes:

- Device I/O data
- Status information
- Configuration data

DeviceNet also has the unique feature of having power on the network. This allows devices with limited power requirements to be powered directly from the network, further reducing connection points and physical size.

DeviceNet uses the Common Industrial Protocol, called *CIP,* which is strictly object oriented. Each object has attributes (data), services (commands), and behavior

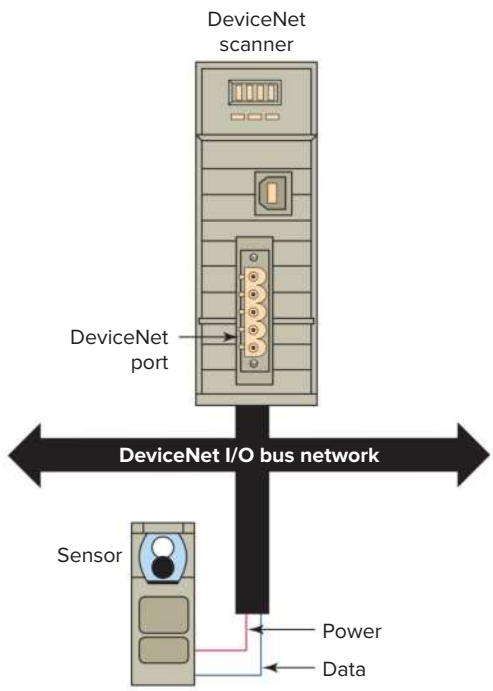


Figure 14-39 DeviceNet network scanner.

(reaction to events). Two different types of objects are defined in the CIP specification: communication objects and application-specific objects. A DeviceNet network can support up to 64 nodes and the network end-to-end distance is variable, based on network speed. Figure 14-40 shows an example of a typical layout of the trunk wiring for a DeviceNet network. Communications data is carried over two wires with a second pair of wires carrying power.

The field devices that are connected to the network contain intelligence in the form of microprocessors or other circuits. These devices can communicate not only the on/off status of field devices but also diagnostic information about their operating state. For example, you can detect via the network that a photoelectric sensor is losing reliability because of a dirty lens, and you can correct the situation before the sensor fails to

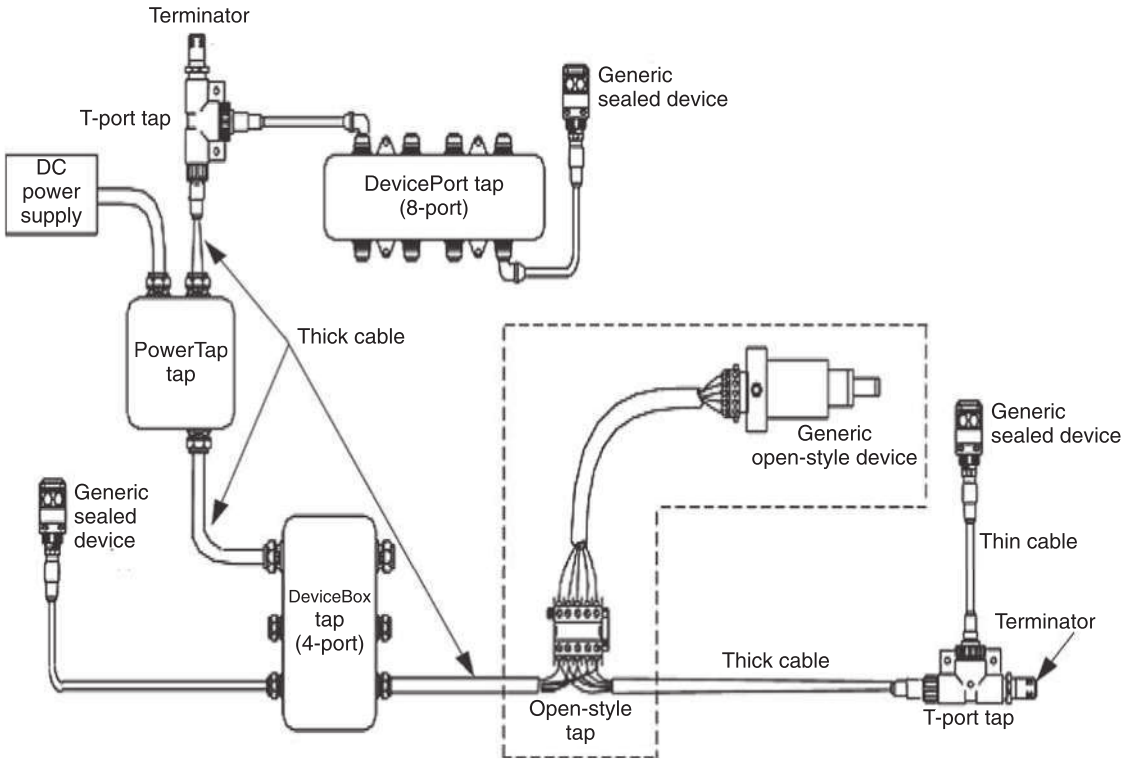


Figure 14-40 Layout of a DeviceNet network.
Source: Images Courtesy of Rockwell Automation, Inc.

detect an object. A limit switch can report the number of motions it has performed, which may be an indication that it has reached the end of its operating life and thus requires replacement.

ControlNet

ControlNet is positioned one level above DeviceNet. It uses the Common Industrial Protocol (CIP) to combine the functionality of an I/O network and a peer-to-peer network providing high-speed performance for both functions. This open high-speed network is highly deterministic and repeatable. ***Determinism*** is the ability to reliably predict when data will be delivered, and ***repeatability*** ensures that transmit times are constant and unaffected by devices connecting to, or leaving, the network. Electronic device data sheets (EDS-Files) are required for each ControlNet device. During the setup phase the ControlNet scanner must configure each device according to the EDS-Files. The ControlNet layout shown in Figure 14-41 has a ***redundant media*** option in which two separate cables are installed to guard against failures such as cut cables, loose connectors, or noise.

EtherNet/IP

EtherNet/IP (Ethernet Industrial Protocol) is an open communications protocol based on the Common Industrial Protocol (CIP) layer used in both DeviceNet and ControlNet. It allows users to link information seamlessly between devices running the EtherNet/IP protocol without custom hardware, as illustrated in Figure 14-42.

The following are some of the important features of EtherNet/IP:

- Sharing a common application layer between ControlNet, DeviceNet, and Ethernet/IP will make plug-and-play interoperability possible among complex devices from multiple vendors. ***Plug and play*** refers to the ability of a computer system to automatically configure devices. This allows you to plug in a device and play (operate) it without worrying about setting DIP switches, jumpers, and other configuration elements.
- EtherNet/IP provides standardized full-duplex operation which gives a single node, in a peer-to-peer connection, full attention and therefore maximum possible bandwidth. ***Bandwidth*** refers to the data rate supported by a network, commonly expressed in terms of bits per second. The greater the bandwidth the greater the overall performance.
- EtherNet/IP allows interoperability of industrial automation devices and control equipment on the same network used for business applications and browsing the Internet.

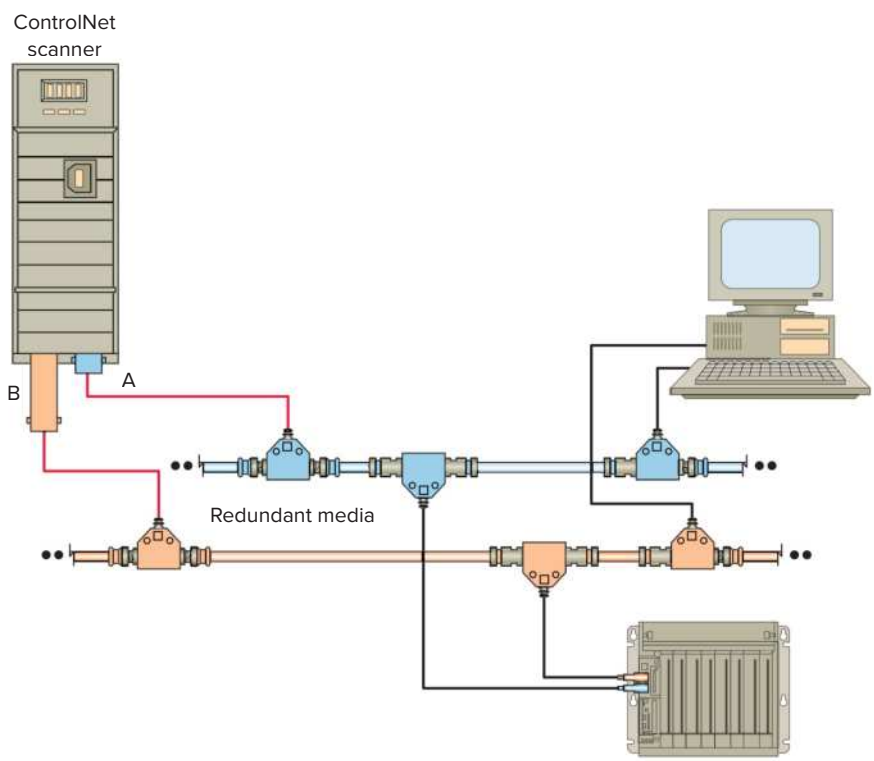


Figure 14-41 ControlNet network with redundant media installed.

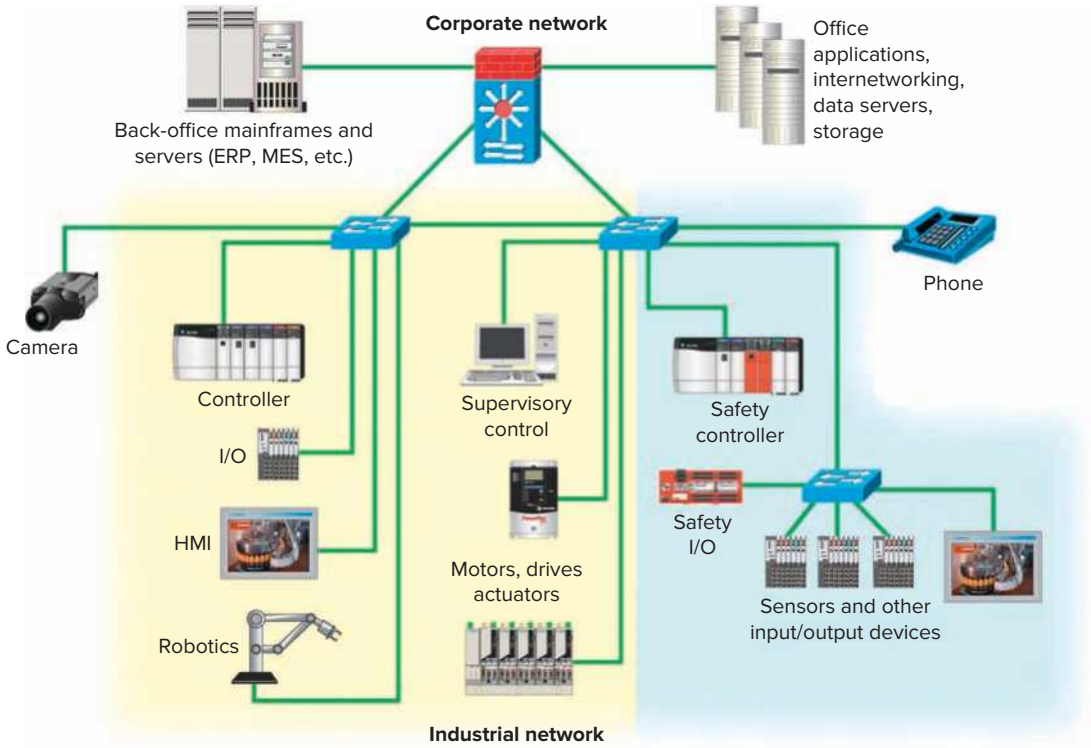


Figure 14-42 EtherNet/IP information links.
Source: Image Courtesy of Rockwell Automation, Inc.

Modbus

Modbus is a serial communication protocol originally developed by Modicon for use with its PLCs. Basically, it is a method used for transmitting information over serial lines between electronic devices. The device requesting the information is called the Modbus Master and the devices supplying information are Modbus Slaves. Modbus is an open protocol, meaning that it's free for manufacturers to build into their equipment without having to pay royalties. It has become a standard communications protocol in industry, and is one of the most commonly available means of connecting industrial electronic devices. Figure 14-43 shows an Omron PLC with Modbus-RTU network communication capabilities via RS-232C and RS-422/485 serial ports.

Fieldbus

Fieldbus is an open, serial, two-way communications system that interconnects measurement and control equipment such as sensors, actuators, and controllers. At the base level in the hierarchy of plant networks, it serves as a network for field devices used in process control applications.

There are several possible topologies for fieldbus networks. Figure 14-44 illustrates the ***daisy-chain*** topology. With this topology, the fieldbus cable is routed from device to device. Installations using this topology require connectors or wiring practices such that disconnection of a single device is possible without disrupting the continuity of the whole segment.

PROFIBUS-DP

PROFIBUS-DP (where DP stands for Decentralized Periphery) is an open, international fieldbus communication

Figure 14-43 Omron PLC with Modbus-RTU network communication capabilities.
Source: Photo courtesy Omron Industrial Automation, **www.ia.omron.com**.

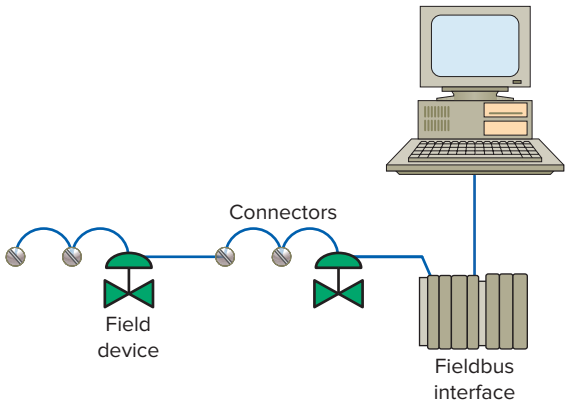


Figure 14-44 Fieldbus implemented using daisy-chain topology.

standard that supports both analog and discrete signals. It is functionally comparable to DeviceNet. The physical media are defined via the RS-485 or fiber optic transmission technologies. PROFIBUS-DP communicates at speeds up to 12 Mbps over distances up to 1200 meters. Figure 14-45 illustrates a Siemens S7-200 Micro PLC system connection to a PROFIBUS-DP network.

SERCOS

SERCOS **(Serial Real-time Communications System)** is an internationally approved communication standard for motion control. The SERCOS standard makes it possible to use devices from various manufacturers. This communication network is designed for high-speed serial communication of standardized closed-loop data, in real time, over a noise-immune fiber optic cable. The SERCOS interface modules use a single, digital fiber optic link, which eliminates as many as 18 digital wires per axis.

Figure 14-46 shows a smart belt packaging operation that uses a SERCOS module interface in conjunction with Allen-Bradley Kinetix 6000 multi-axis servo drive. The operation of the process can be summarized as follows:

- This packaging application is used to convert randomly spaced product into evenly spaced product that's properly phased to the in-feed of another machine.
- The SERCOS interface module is linked with fiber optic cable to the Allen-Bradley Kinetix 6000 servo drive.
- The randomly spaced product is fed by a conveyor driven by an induction-type motor.
- The servos that make up the smart belts follow the in-feed of the next conveyor downstream so that the smart belt's speed is matched to that machine's speed.
- The input from the sensor on each smart belt senses the product and makes position corrections based on error calculations performed in the controller.
- The corrections are performed on a percentage of the total error, based on whether the belt is performing a coarse or fine correction.
- The first belts in the process are usually coarse correction belts and typically make a large correction for as much as 70 to 100% of the total phasing error amount.
- The last belts perform fine correcting for 100% of the measured remaining phasing error even though the total error is smaller, relative to what it was at the beginning of the correcting process.

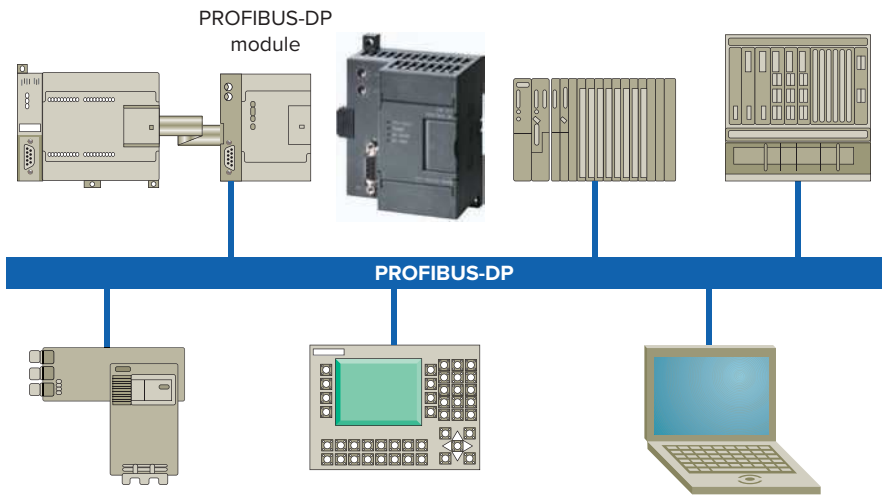


Figure 14-45 Micro PLC system connection to a PROFIBUS-DP network.
Source: Courtesy Siemens.

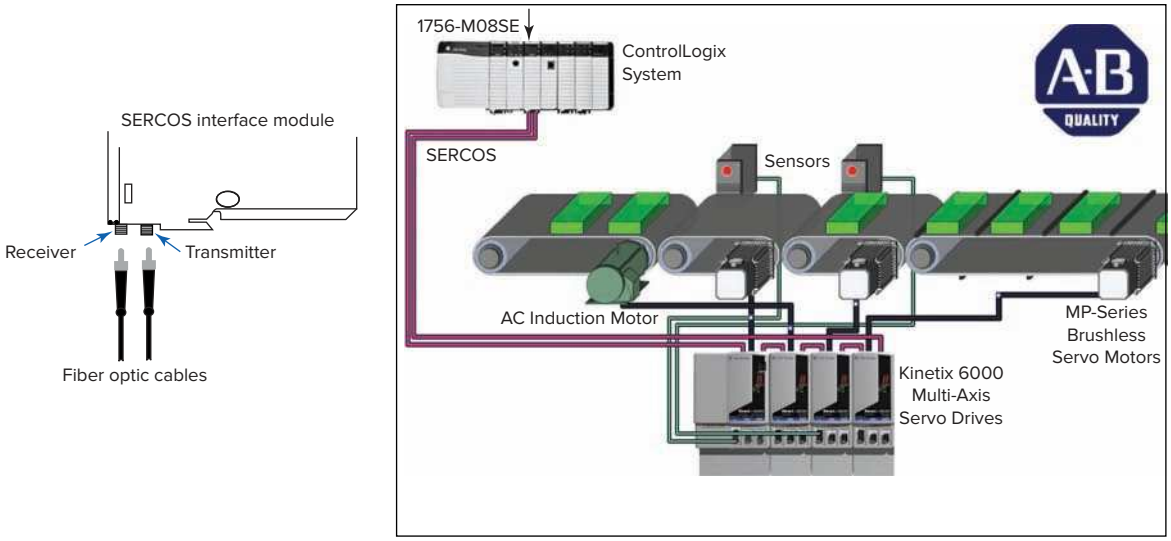


Figure 14-46 Smart belt packaging operation.
Source: Image Courtesy of Rockwell Automation, Inc.

HART

HART is an open master-slave communication protocol developed to communicate with smart field devices. Smart field devices contain more information than the traditional 4–20 mA signal. In addition, they carry out some functions that are/were originally programmed within the PLC. HART Protocol allows the simultaneous communication of the continuous 4–20 mA as well as a second digital communication path resting on top of the analog signal, but not interfering. This allows access to the data of the field device and/or exact machine-readable product description of the field devices including their data and functions. More instrumentation devices are available with the HART protocol than with any other digital communications technology.

The HART digital signal is superimposed onto the standard 4–20 mA signal, as illustrated in Figure 14-47.

- The digital signal is made up of two frequencies, 1.2 and 2.2 kHz, representing bits 1 and 0 respectively.
- Sine waves of these two frequencies are superimposed onto the analog signal cables to give simultaneous analog and digital communications.
- The DC and low-frequency current signals are modulated by the 1200 and 2200 Hz frequencies; a technique known as frequency-shift keying or FSK.
- As the average value of the FSK signal is always zero there is no effect on the 4–20 mA analogue signal.
- A minimum loop impedance of 230 ohms is required for communication.

14.7 Supervisory Control and Data Acquisition (SCADA)

In some applications, in addition to its normal control functions, the PLC is responsible for collecting data, performing the necessary processing, and structuring the data for generating reports. As an example, you could have a PLC count parts and automatically send the data to a spreadsheet on your desktop computer.

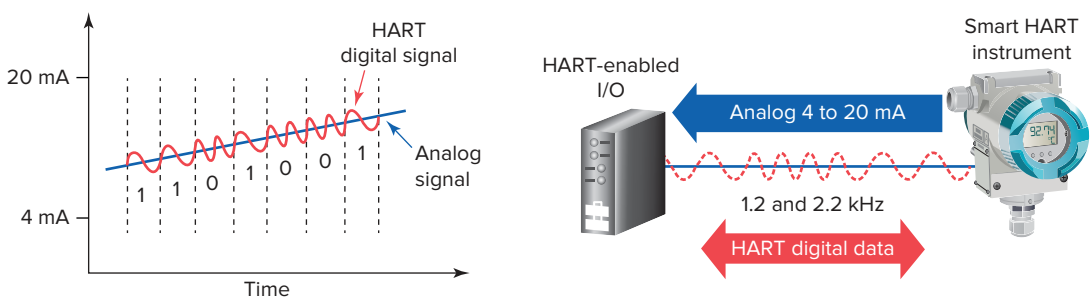


Figure 14-47 HART communication protocol.

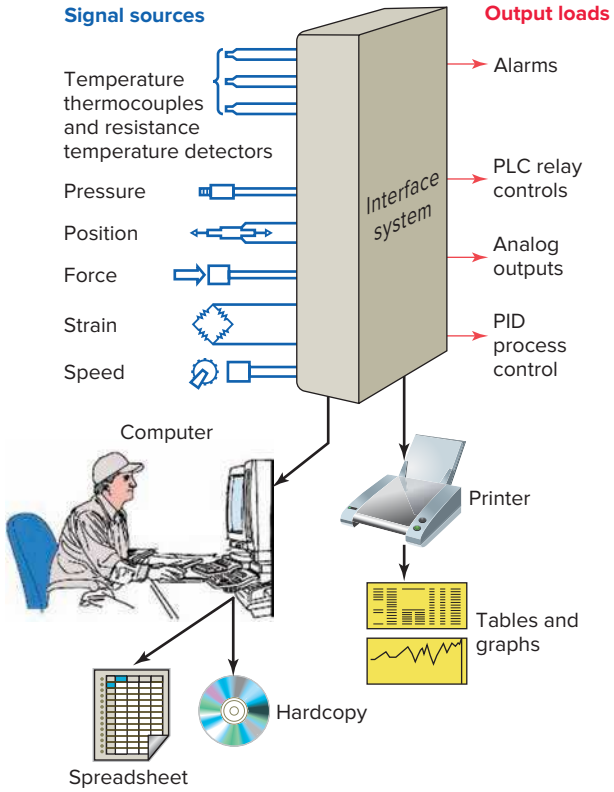


Figure 14-48 Supervisory control and data acquisition (SCADA).

Data collection is simplified by using a **SCADA (supervisory control and data acquisition)** system, shown in Figure 14-48. Exchanging data from the plant floor to a supervisory computer allows data logging, data display, trending, downloading of recipes, setting of selected parameters, and availability of general production data. The additional supervisory control output capabilities allow you to tweak your processes accurately for maximum efficiency. In general, unlike distributive control systems, a SCADA system usually refers to a system that coordinates but does not control processes in real time.

In a typical SCADA system, independent PLCs perform I/O control functions on field devices while being supervised by a SCADA/HMI software package running on a host computer, as illustrated in Figure 14-49. Process control operators monitor PLC operation on the host computer and send control commands to the PLCs if required. The great advantage of a SCADA system is that data are stored automatically in a form that can be retrieved for later analysis without error or additional work. Measurements are made under processor control and then displayed onscreen and stored to a hard drive. Accurate measurements are easy to obtain, and there are no mechanical limitations to measurement speed.

An important part of most SCADA implementations is alarm handling. An **alarm** is an announcement to the operator initiated by a process variable passing a defined limit as it approaches an undesirable or unsafe value. The announcement includes audible sounds, visual indications, and messages. Properly designed alarms will notify the operator of abnormal situations with enough time to successfully manage them. For this reason, alarms often have a several-second delay associated with them to ensure the process being monitored has stabilized before activating an alarm. Alarm management includes being able to distinguish between alarms and alerts. An **alert** provides a warning mechanism, but doesn't necessarily require immediate action.

Rockwell's **FactoryTalk** services platform is a suite of software services that include:

- **LiveData** - Provides a data link between clients and servers. For example, the connection between an HMI server and PLC.
- **Directory** - Allows products to share a common address book that finds and provides access to plant-floor resources, such as users, tags, and graphics displays.
- **Security** - Provides a range of security services that are integrated into the FactoryTalk directory.
- **Audit** - Collects messages that document changes done by users during design, management, and product operations.

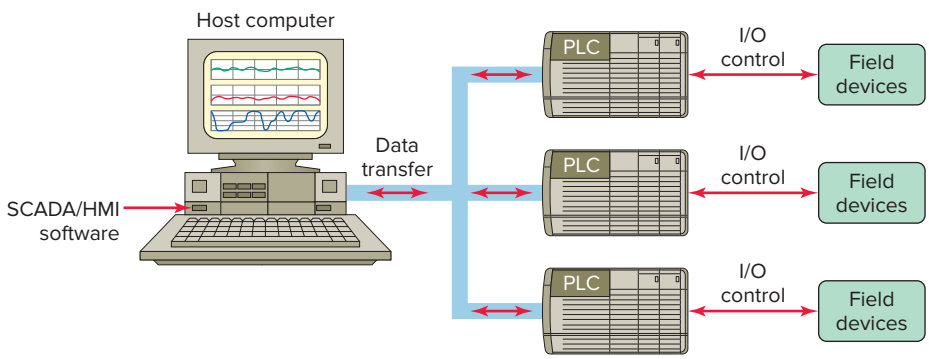


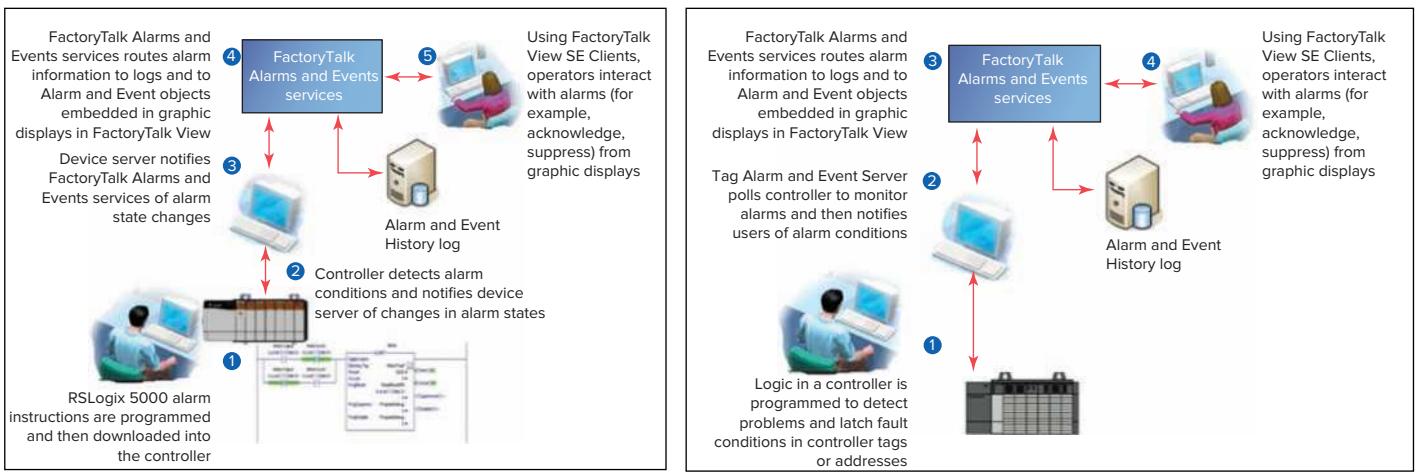
Figure 14-49 Typical SCADA system.

- **Activation** - Provides a secure, software-based system for activating Rockwell Software products.
- **Alarm and Events** - Provides both server-based and device-based process control alarm server.

FactoryTalk alarms and events support the following two types of alarm monitoring (Figure 14-50):

Device-based - Pre-built alarm instructions that are programmed in a Logix 5000 project and then downloaded into a Logix controller. The controller detects alarm conditions and publishes event information, which is routed through the system for display and logging.

Tag-based - Software-based tag servers monitor data tags for alarm conditions and publish event information for display and logging. Tag-based alarm monitoring is supported by Logix 5000 controllers and SLC 500 controllers.



(*a*) Device-based alarms

(*b*) Tag-based alarms

Figure 14-50 FactoryTalk alarm monitoring.
Source: Images Courtesy of Rockwell Automation, Inc.

CHAPTER 14 REVIEW QUESTIONS

1. Compare continuous and batch processes.
2. Compare centralized and distributive control systems.
3. State the basic function of each of the following as part of a process control system:
   a. Sensors
   b. Human-machine interface
   c. Signal conditioning
   d. Actuators
   e. Controller
4. State the purpose of each of the following types of screens associated with HMIs:
   a. Trend values
   b. Operational summary
   c. Alarm summary
5. What is the main characteristic of a closed-loop control system?
6. State the function of each of the following parts of a closed-loop control system:
   a. Set-point
   b. Process variable
   c. Error amplifier
   d. Controller
   e. Output actuator
7. Explain how on/off control works.
8. How does the proportional controller eliminate the cycling associated with on/off control?
9. Explain how a motor-driven control valve action can provide analog control.
10. How does time proportioning provide analog control?
11. What process error or deviation is produced by a proportional controller?
12. What term of a PID control is designed to eliminate offset?
13. What does the derivative action of a controller respond to?
14. List the three gain adjustments used in tuning the response of a PID control loop.
15. Compare manual, autotune, and intelligent tuning of a PID controller.
16. How many input and output values are normally referenced in a PLC PID instruction?
17. What information is contained in the process variable and control variable elements of a PID instruction?
18. State the function of each of the following elements of a PLC motion control system:
    a. Programmable controller
    b. Motion module
    c. Servo drive
    d. Servo motor
19. What does each axis of a robot arm function as?
20. List four types of communication tasks provided by local area networks.
21. Name three common types of transmission media.
22. What are the three general levels of functionality of industrial networks?
23. Define the term *node* as it applies to a network.
24. Explain the physical layout of devices on a network for each of the following network topologies:
    a. Star
    b. Bus
25. Compare device and process bus networks.
26. Define the term *protocol* as it applies to a network.
27. What is the function of a network gateway?
28. Define the term *access method* as it applies to a network.
29. Summarize the token passing network access method.
30. Summarize the collision detection network access method.
31. Summarize the polling network access method.
32. Compare parallel and serial data transmission.
33. Compare half-duplex and full-duplex data transmission.
34. Explain how networking schemes minimize the amount of wiring required.
35. What type of access control is used with DH+?
36. Compare the transmitting distances of RS-232 and RS-422/485 serial types.
37. What is DeviceNet used for?
38. List three pieces of information obtained from DeviceNet devices by the network scanner.
39. What is ControlNet used for?

40. Explain how redundant media works.

41. Define the term *bandwidth* as it applies to a network.

42. What is Ethernet/IP used for?

43. What type of protocol does Modbus use?

44. What is Fieldbus used for?

45. Summarize the two main functions of a SCADA system.

46. In what way does distributive control differ from the supervisory control of a SCADA system?

CHAPTER 14 PROBLEMS

1. Distributive control systems have to be network based. Why?

2. Assume an alarm is sounded in a control system with an electronic HMI interface. How would you proceed to identify and solve the problem?

3. How would an on/off controller respond if the deadband were too narrow?

4. In a home heating system with on/off control, what will be the effect of widening the deadband?

5. **a.** Calculate the proportional band of a temperature controller with a 5% bandwidth and a set-point of 500°F.

b. Calculate the upper and lower limits beyond which the controller functions as an on/off unit.

6. Explain the advantage of using a 4–20 mA current loop as an input signal compared to a 0–5 V input signal.

7. What does the term *deterministic* mean, and why is it important in industrial communications?

8. How might a SCADA system be applied to determine the production rate of a bottled product over a two-week period?

15
ControlLogix Controllers

Programmable logic controllers continue to evolve as new technologies are added to their capabilities. The PLC started out as a replacement for banks of relays used to turn outputs on and off as well as for timing and counting functions. Gradually, various math and logic manipulation functions were added. In order to serve today's expanding industrial control system needs, leading automation companies have created a new class of industrial controllers called ***programmable automation controllers*** or ***PACs*** (Figure 15-1). They look like PLCs in their physical appearance but incorporate advanced control of communication, data logging, and signal processing, motion, process control, and machine vision in a single programming environment.

The Allen-Bradley programmable automation controller family includes the ControlLogix system, CompactLogix system, FlexLogix system, SoftLogix 5800 controller, and DriveLogix system. *Software* is the essential difference between PACs and PLCs. Basically, the ladder logic configuration does not change but the addressing of the instructions changes. Application of the software that pertains to the Logix control platform of controllers will be covered in the various sections of this chapter. Knowledge of basic ladder logic instructions and functions (bit, timer, counter, etc.) covered in previous chapters of the text is assumed and is thus not repeated in this chapter.

Figure 15-1 Programmable automation controllers (PACs).
Source: Image Courtesy of Rockwell Automation, Inc.

Part 1 Memory and Project Organization

Part Objectives

After completing this part, you will be able to:

- Outline project organization
- Define tasks, programs, and routines
- Identify data file types
- Organize and apply the various data file types

Memory Layout

ControlLogix processors provide a flexible memory structure. There are no fixed areas of memory allocated for specific types of data or for I/O. The internal memory organization of a ControlLogix controller is configured by the user when creating a project with RSLogix 5000 software (Figure 15-2). This feature allows the program data to be constructed to meet the needs of your applications rather than requiring your application to fit a particular memory structure. A ControlLogix (CLX) system can consist of anything from a stand-alone controller and I/O modules in a single chassis, to a highly distributed system consisting of multiple chassis and networks working together.

Configuration

Configuration of a modular CLX system involves establishing a communications link between the controller and the process. The programming software needs to know what CLX hardware is being used in order to be able to send or receive data. Configuration information includes information about the type of processor and I/O modules used.

Figure 15-2 RSLogix 5000 screen.
Source: Image Courtesy of Rockwell Automation, Inc.

LogixDesigner programming software is used to set up or *configure* the memory organization of an Allen-Bradley ControlLogix controller. ***RSLinx*** communication software is used to set up a communications link between RSLogix 5000 programming software and the ControlLogix hardware as illustrated in Figure 15-3. To establish communications with a controller, a driver must be created in RSLinx software. This driver functions as the software interface to a hardware device. The ***RSWho*** is the network browse interface that provides a single window to view all configured network drivers.

Figure 15-4 shows an example of the ControlLogix's *controllers properties and modules properties* dialog boxes used as part of the configuration process. The parameters shown are typical of what general information is required. After first configuring the controller,

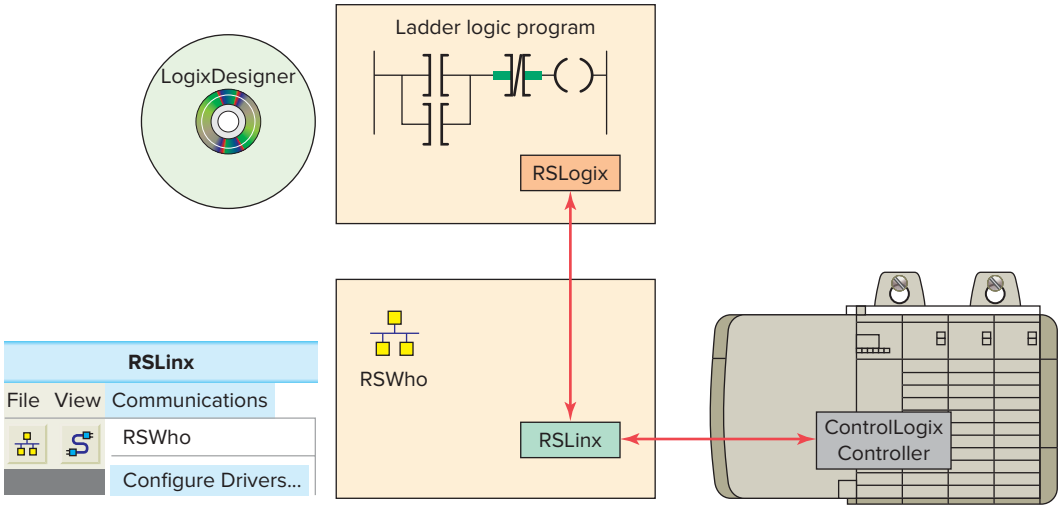


Figure 15-3 RSLinx and LogixDesigner software.

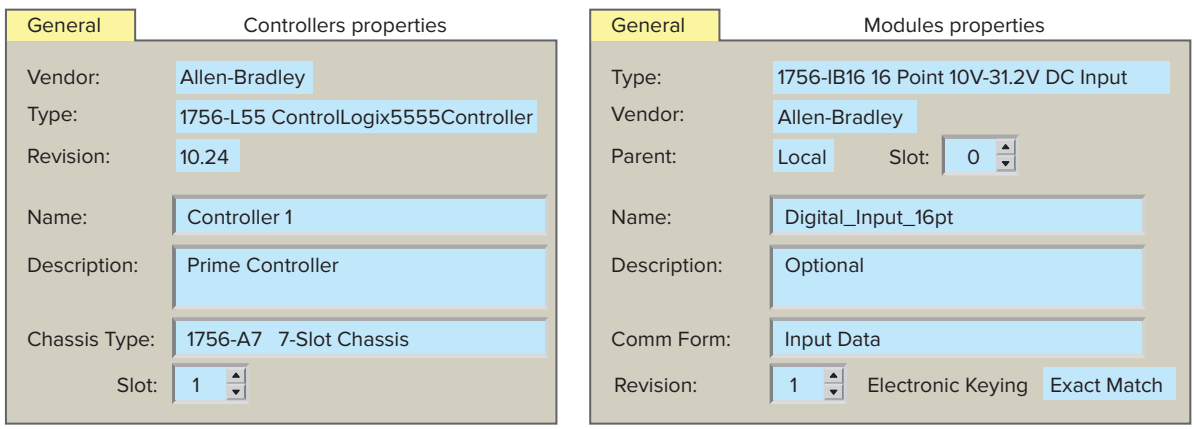


Figure 15-4 Controllers properties and modules properties dialog boxes.

the I/O modules are configured using RSLogix 5000 software. Modules will not work unless they have been properly configured. The software contains all the hardware information needed to configure any ControlLogix module.

Project

RSLogix software stores a controller's programming and configuration information in a file called a ***project***. The block diagram of the processor's project file is shown in Figure 15-5. A project file contains all information relating to the project. The main components of the project file are tasks, programs, and routines. A controller can hold and execute only one project at a time.

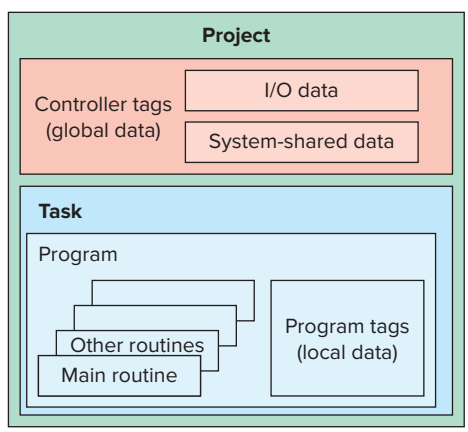


Figure 15-5 ControlLogix processor program file.

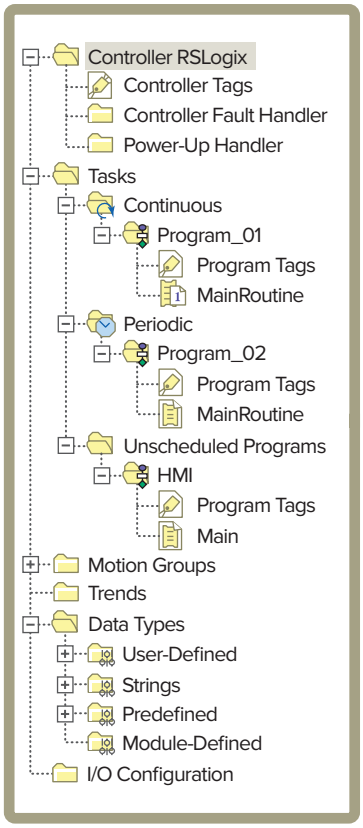


Figure 15-6 Controller organizer tree.

The RSLogix 5000 controller organizer (Figure 15-6) displays the project organization in a tree format showing tasks, programs, routines, data types, trends, I/O configuration and tags. Each folder groups common functions together. This structure simplifies the navigation and the overall view of the whole project.

In front of each folder, there is an icon containing a + sign or a − sign. The + sign indicates that the folder is closed. Click it to expand the tree display and display the files in the folder. The − sign indicates that the folder is already open and its contents are visible. Clicking on the right mouse button brings up many different, context-sensitive popup menus. Often, you find that this is a shortcut to access the property window or menu options from the menu bar.

Tasks

Tasks are the first level of scheduling within a project. A task is a collection of scheduled programs. When a task is executed, the associated programs are executed in the order listed. This list of programs is known as the program schedule. Tasks provide scheduling based on specific conditions and do not contain any executable code. Only one task may be executing at any given time. The number of tasks a controller can support depends on the specific controller. The main types of tasks (Figure 15-7) include:

- ***Continuous*** tasks execute nonstop but are always interrupted by a periodic task. Continuous tasks have the lowest priority. A ControlLogix continuous task is similar to the File 2 in the SLC 500 platform. Here the continuous task is named Main Task.
- ***Periodic*** tasks function as timed interrupts. They interrupt the continuous task and execute for a fixed length of time at specific time intervals.
- ***Event*** tasks also function as interrupts. Rather than being an interrupt on a timed basis, an event task is triggered by an event that happened or failed to happen.

Programs

Programs are the second level of scheduling within a project. The function of the folders under Main Task is to determine and specify the order in which the programs

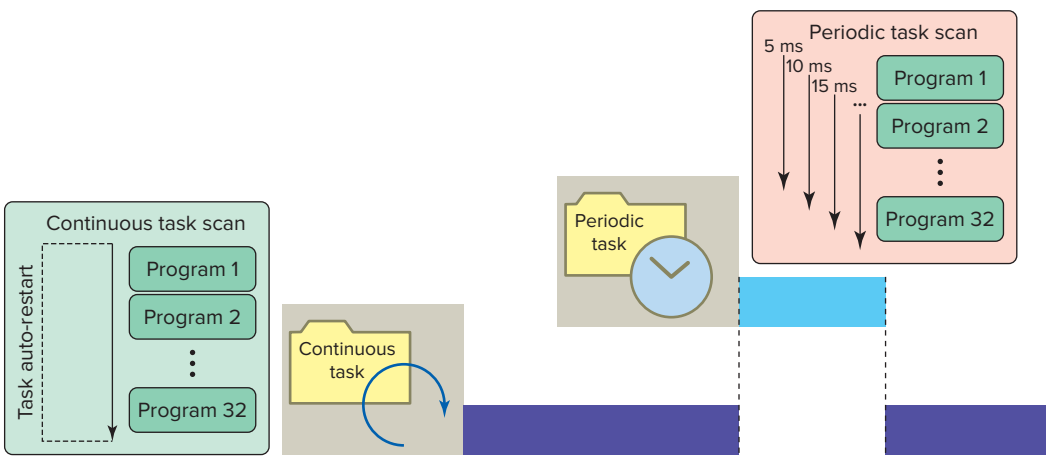


Figure 15-7 Continuous and periodic tasks.

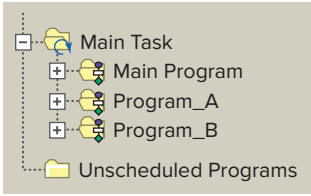


Figure 15-8 Order of execution of programs.

execute. There is no executable code within a program. Routines within programs will execute in the order listed below their associated task in the controller organizer as shown in Figure 15-8. In this example, according to the listed order, the Main Program is scheduled to execute first, Program_A second, and Progam_B third. Programs that are not assigned to a task are unscheduled. Unscheduled programs are downloaded to the controller but do not execute. These programs remain unscheduled until needed. Depending on the RSLogix 5000 software version as many as 100 programs could be scheduled within each task.

Routines

Routines are the third level of scheduling within a project and provide the executable code for the project. Each routine contains a set of logic elements for a specific programming language. When a routine is created it is specified as ladder logic, sequential function chart, function block diagram, or structured text (Figure 15-9). Any one routine must be completely in the same language. The number of routines per project is limited only by the amount of controller memory. Libraries of standard routines can be created that can be reused on multiple machines or applications. A routine can be assigned as one of the following types:

- A ***main routine*** is one configured to execute first when the program runs. Each program will have one main routine typically followed by several or many subroutines.
- A ***subroutine*** is one that is called by another routine. Subroutines are used for large or complex programming tasks or tasks that require more than one programming language.
- A ***fault* routine** is one that executes if the controller finds a program fault. Each program can have one fault routine, if desired.

Tags

Unlike conventional controllers, ControlLogix uses a *tag-based* addressing structure. Tags are meaningful names, descriptive of your application and not merely generic

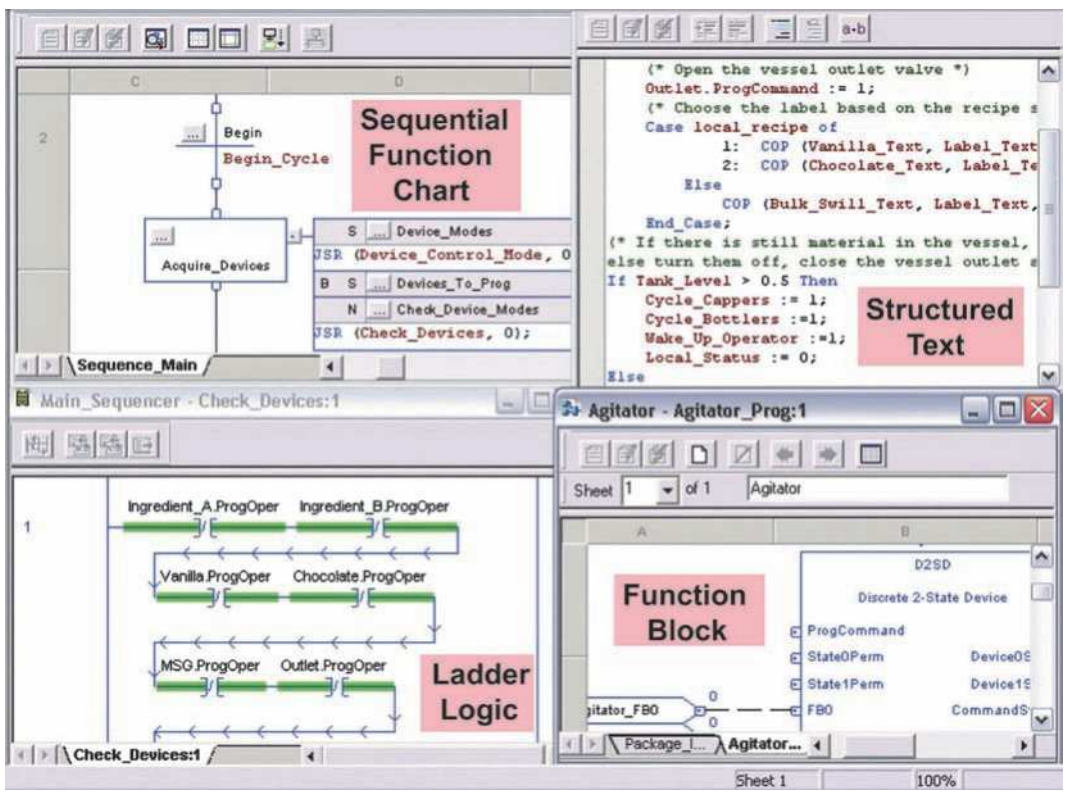


Figure 15-9 Each routine contains a set of logic elements for a specific programming language.
Source: Image Courtesy of Rockwell Automation, Inc.

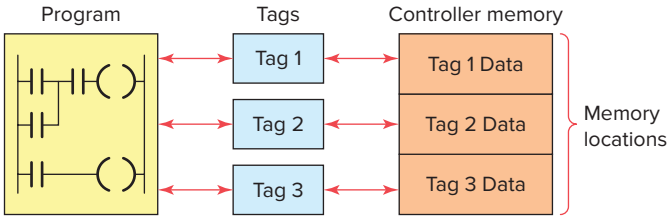


Figure 15-10 Tags used to assign memory locations.

addresses. A tag is created to represent the data and identify areas in the controller's memory where these data are stored. In applications developed using RSLogix 5000 software, there are no predefined data tables such as in an SLC 500. When you want to use or monitor data in a program you use tag names to refer to the memory locations, as illustrated in Figure 15-10. This functionality allows you to name your data specifically for their functions within the control program while providing self-documented logic. Whenever you wish to group data, you create an array, which is a grouping of tags of similar types.

Scope refers to which programs have access to a tag. The scope of a tag must be specified when you create the tag. There are two scopes for tags: program scope and controller scope. A ***program tag*** consists of data that can be accessed only by routines within a specific program (local data). The routines in other programs cannot access program scoped tags of another program. A ***controller tag*** consists of data that are accessible by all routines within a controller (global data). Figure 15-11 shows two programs, A and B, within a project. Note that each program has program scope tags with identical names (Tag_1, Tag_2, and Tag_3). Because they are program scoped, there is no relationship between them, even though they have the same name. The program scope data are accessible only to the routines within a single program. The same tag name may appear in different programs as local variables because you can select the scope in which to create the tag.

The scope of a tag must be declared when you create the tag. Figure 15-12 shows program and controller

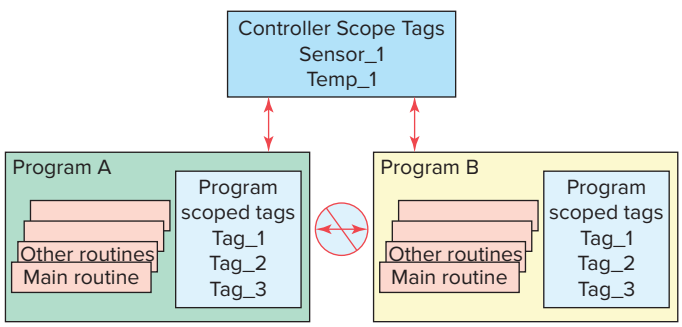


Figure 15-11 Program scoped and controller scoped tags.

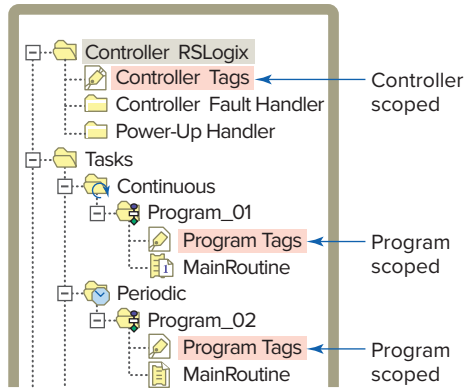


Figure 15-12 Listing of program and controller scoped tags.

scoped tags as listed in the controller organizer under the program they are assigned to. I/O tags are automatically created as controller scoped tags.

There are four different tag types: base, alias, produced, and consumed tags. The tag type defines how the tag operates within the project. A ***base tag*** stores various types of data for use by logic in the project. This tag defines a memory location where data are stored. Base tag memory use depends on the type of data the tag represents. An example of the base tag Local:2:O.Data.4 is shown in Figure 15-13 and is based on the following format:

Location	Network location LOCAL = same chassis as the controller
Slot	Slot number of I/O module in its chassis
Type	Type of data I = input O = output C = configuration S = status
Member	Specifies the type of data that the module can store. Digital (discrete) I/O modules use a DATA member. Analog I/O modules use a Channel Member (CH#)
SubMember	Specific data related to a Member.
Bit	Specific point on a digital I/O module; depends on the size of the I/O module (0-31 for a 32-point module)

An ***alias tag*** is used to create an alternate name (alias) for a tag. The alias tag is simply another name for an already named memory location. An alias tag can refer to a base, alias, consumed, or produced tag. The alias tag is often used to create a tag name to represent a real-word input or output. Figure 15-14 shows an example of the

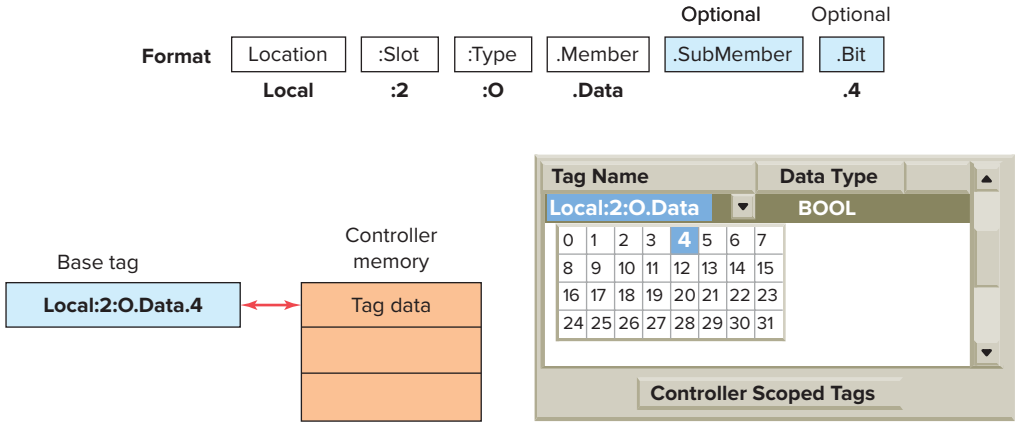


Figure 15-13 Base tag.

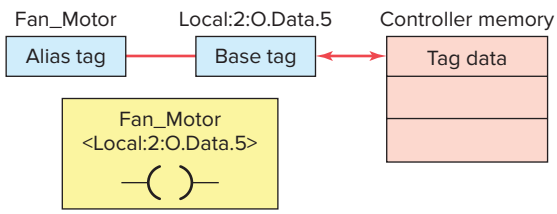


Figure 15-14 Alias tag linked to a base tag.

use of an alias tag. The alias tag (Fan_Motor) is linked to the base tag (<local:2:O.Data.5>) so that any action to the base also happens to the alias and vice versa. The alias name is easier to understand and easier to relate to the application, while the base tag contains the physical location of the output point in the ControlLogix chassis.

Produced/consumed tags are used to share tag information over a network between two or more devices. A produced tag sends data while a consumed tag receives data. Produced tags are always controller scoped. Figure 15-15 shows an example of how a controller can produce data and send them over the network to two controllers that use or consume the data. The producing controller will have a tag that is of the produced type, whereas the consuming controllers will have a tag with the exact same name that is of the consumed type.

When you design your application, you configure it to both produce globally to other controllers in the system via the backplane and to consume tags from other controllers. This feature allows you to be selective about which data are sent and received by any controller. Likewise, multiple controllers can connect to any data being produced, thereby preventing the need to send multiple messages containing the same data.

Logix controllers are based on 32-bit operations. The types of data that can be a base tag are BOOL, SINT, INT, DINT, and REAL, as illustrated in Figure 15-16 and listed below. The controller stores all data in a minimum of 4 bytes or 32 bits of data.

- A **BOOL** or Boolean base tag is 1 bit of data stored in bit 0 of a 4 byte memory location. The other bits, 1 to 31, are unused. BOOLs have a range of 0 to 1, off or on respectively.

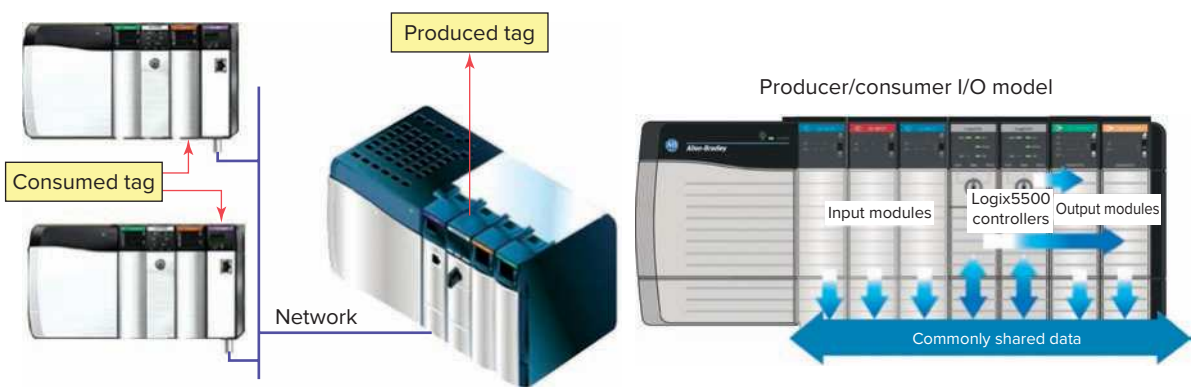


Figure 15-15 Produced/consumed tags used to share information.
Source: Image Courtesy of Rockwell Automation, Inc.

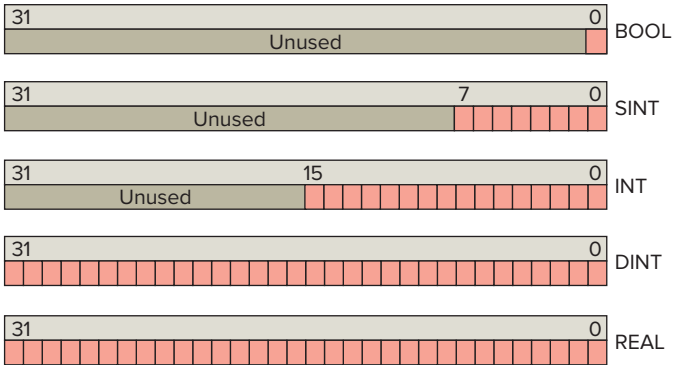


Figure 15-16 Types of base tag data.

- A **SINT** or Single Integer base tag uses 8 bits of memory and stores the data in bits 0 to 7. These bits are sometimes called the low byte. The other 3 bytes, bits 8 to 31, are unused. SINTs have a range of −128 (negative values) to 127 (positive values).
- An **INT** or Integer base tag is 16 bits, bits 0 to 15, sometimes called the lower bytes. Bits 16 to 31 are unused. INTs have a range between −32,768 and 32,767.
- A **DINT** or Double Integer base tag uses 32 bits, or all 4 bytes, and has the following range: -2^{31} to $2^{31}-1$ (−2,147,483,648 to 2,147,483,647).
- A **REAL** base tag also uses 32 bits of a memory location and has a range of values based on the IEEE Standard for Floating-Point Arithmetic.

Structures

There is another class of data types called structures. A **structure-type tag** is a grouping of different data types that function as a single unit and serve a specific purpose. An example of an RSLogix structure is shown in Figure 15-17. Each element of a structure is referred to as a member and each member of a structure can be a different data type.

	Name	Data Types	Style	Description
	PRE	DINT	Decimal	
	ACC	DINT	Decimal	
	EN	BOOL	Decimal	
	TT	BOOL		
Members	DN	BOOL	Data Types	
	FS	BOOL	Decimal	
	LS	BOOL	Decimal	
	OV	BOOL	Decimal	
	ER	BOOL		

Figure 15-17 Structure-type tag.

Data type : COUNTER

Name Counter

Description

Members **Data type size : 12 byte(s)**

	Name	Data Type	Style	Description
	PRE	DINT	Decimal	
	ACC	DINT	Decimal	
	CU	BOOL	Decimal	
	CD	BOOL	Decimal	
	DN	BOOL	Decimal	
	OV	BOOL	Decimal	
	UN	BOOL	Decimal	

Figure 15-18 Predefined structure.

There are three different types of structures in a ControlLogix controller: *predefined, module-defined,* and *user-defined.* The controller creates ***predefined* structures** for you that include timers, counters, messages and PID types. An example of a predefined counter instruction structure is shown in Figure 15-18. It is made up of the preset value, the accumulated value, and the instruction's status bits.

***Module-defined* structures** are automatically created when the I/O modules are configured for the system. When you add input or output modules a number of defined tags are automatically added to the controller tags. Figure 15-19 shows the two tags (Local:1:C and Local:1:I) created after a digital input module has been

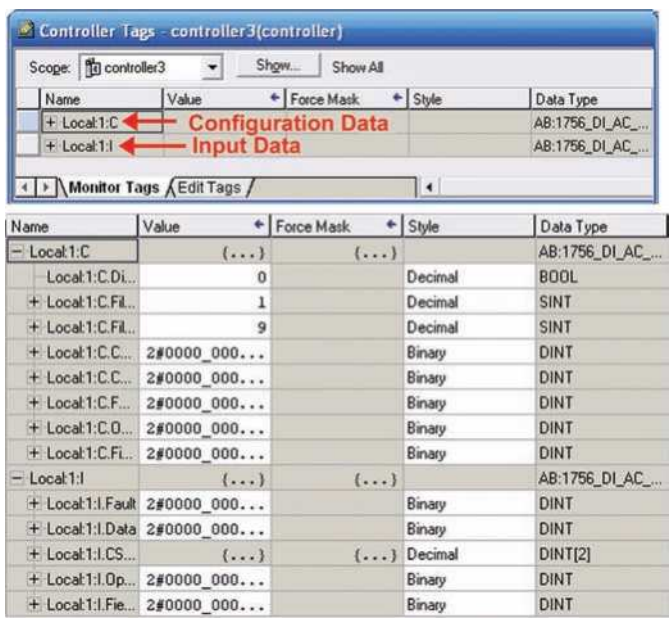


Figure 15-19 Module-defined structure for a digital input module.

Source: Image Courtesy of Rockwell Automation, Inc.

Name: **Tank** Size: 16 byte(s)

Description: Generic Storage Tank Data Type

	Name	Data Type	Style	Description
	Level	INT	Decimal	Stores the Level in Inches
	Pressure	DINT	Decimal	Stores the Pressure in PSIG
	Temp	REAL	Float	The Temperature in F
	Agitator_Speed	DINT	Decimal	Speed in RPM
*				

Figure 15-20 User-defined storage tank structure.

added. Tags of these types are created to store input, output, and configuration data for the module. Input tags labeled Data contain the actual input bits from the module. Configuration tags determine the characteristics and operation of the module. The name Local indicates that these tags are in the same rack as the processor. The 1 indicates that the module occupies slot 1 in the chassis. The letters I and C indicate whether the data are input data or configuration data.

A ***user-defined*** **structure** supplements the predefined structures by providing the ability to create custom-defined structures to store and handle data as a group. Figure 15-20 illustrates a user-defined structure that contains data for a storage tank. All data relative to the tank are stored together. In the design stage the programmer creates a generic user-defined memory structure that contains all the different aspects of the storage tank. Each member has a meaningful name and is created in the appropriate data type and style like REAL (floating point) for temperature and DINT (decimal) for agitator speed in rpm. Installation and maintenance personnel can easily locate all data associated with the operation of the tank since all the information is stored together.

Creating Tags

There is more than one way to create tags. You may create tags in the tag editor before your program is entered, enter tag names as you program, or use question marks [?] in place of tag names and assign the tags later. Figure 15-21 shows an example of a controller scope base tag created in the new tag dialog box. When defining tags, the following information has to be specified:

- A *tag name,* which must begin with an alphabetic character or an underscore (_). Names can contain only alphabetic characters, numeric characters, or underscores and may be up to 40 characters in length. They may not have consecutive or trailing underscore characters, are not case sensitive and cannot have spaces in the tag name.
- An optional *tag description,* which may be up to 120 characters in length.
- The *tag type:* base, alias, or consumed.
- The *data type,* which is obtained from the list of predefined or user-defined data types.
- The scope in which to create the tag. Your options are the controller scope or any one of the existing program scopes.
- The *display style* to be used when monitoring the tag in the programming software. The software will display the choices of available styles.
- Whether or not you want to make this tag available to other controllers and the number of other controllers that can consume the tag.

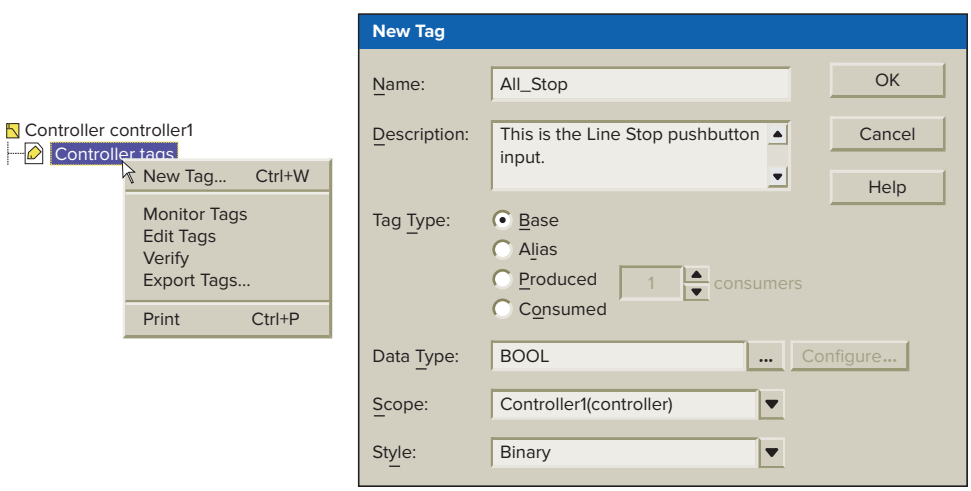


Figure 15-21 Controller scope base tag.
Source: Image Courtesy of Rockwell Automation, Inc.

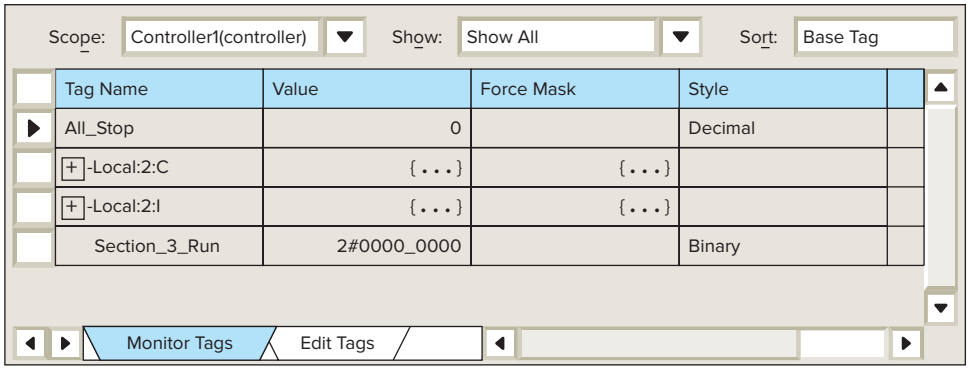


Figure 15-22 Monitor Tags window.

Monitoring and Editing Tags

After tags have been created they can be monitored using the Monitor Tags window displayed in Figure 15-22. When ***Monitor Tags*** is selected the actual value(s) for the tags will be shown. The Force Mask column is used to force inputs and outputs when troubleshooting. You can also create new tags or edit existing tags using the Edit Tags window displayed in Figure 15-23. When ***Edit Tags*** is selected new tags may be created, and existing tag properties may be modified.

Array

Many control programs require the ability to store blocks of information in memory in the form of tables that can be accessed at runtime. An ***array*** is a tag type that contains a block of multiple pieces of data. Each element

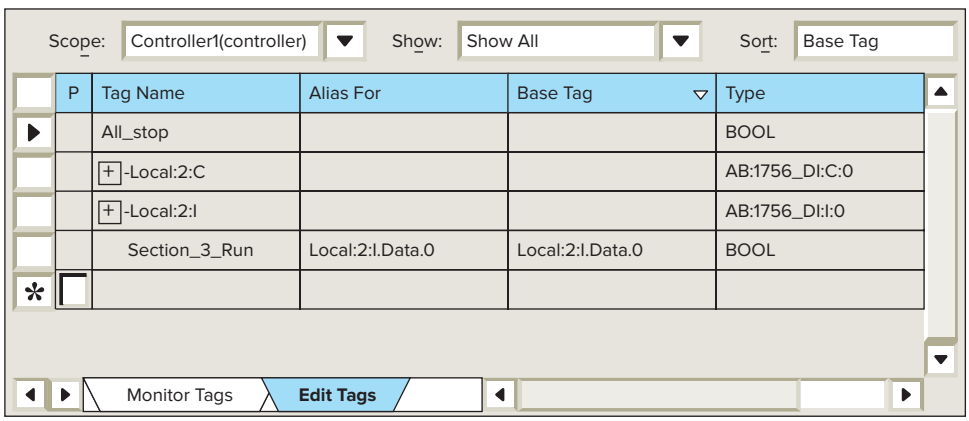


Figure 15-23 Edit Tags window.

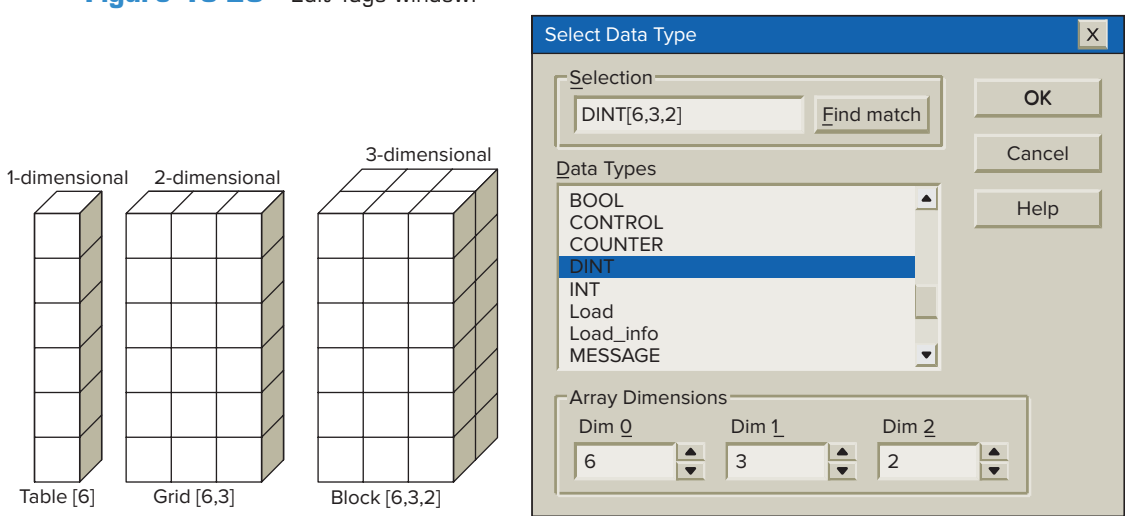


Figure 15-24 Types of arrays.
Source: Image Courtesy of Rockwell Automation, Inc.

of an array must be of the *same data type* (e.g., BOOL, SINT, or INT). An array occupies a contiguous block of controller memory. Arrays are similar to tables of values. The use of arrayed data types offers the fastest data throughput (output) from a ControlLogix processor. Because arrays are numerically sequenced tags of the same data type that occupy a contiguous memory location, large amounts of data can be retrieved efficiently. Arrays can be built using 1, 2 or 3 dimensions, as illustrated in Figure 15-24, to represent the data they are intended to contain.

A single tag within the array is one element. The element may be a basic data type or a structure. The elements start with 0 and extend to the number of elements minus 1. Figure 15-25 is an example of the memory layout for a 1-dimensional (one column of values) array created to hold five temperatures. The tag name is Temp and the array consists of 5 elements numbered 0 through 4.

Array - Temp
Data Type - INT[5]

Temp[0]	297
Temp[1]	200
Temp[2]	180
Temp[3]	120
Temp[4]	100

Figure 15-25 Memory layout for a one-dimensional array.

PART 1 REVIEW QUESTIONS

1. Compare the memory configuration of a Logix 5000 controller with that of an SLC 500 controller.
2. What does a project contain?
3. List four programming functions that can be carried out using the program organizer.
4. Explain the function of tasks within the project.
5. State the three main types of tasks.
6. What type of tasks function as timed interrupts?
7. Explain the function of programs within the project.
8. Explain the function of routines within the project.
9. Which routine is configured to execute first?
10. Name the four types of programming languages that can be used to program Logix 5000 controllers.
11. What are tags used for?
12. Compare the accessibility of program scope and controller scope tags.
13. Name the tag type used for each of the following:
    a. Create an alternate name for a tag.
    b. Share information over a network.
    c. Store various types of data.
14. What is the difference between a produced tag and a consumed tag?
15. List the five types of base tag data.
16. State the data type used for each of the following:
    a. 32-bit memory storage
    b. On/Off toggle switch
    c. 16-bit memory storage
    d. 8-bit memory storage
17. Describe the make-up of a predefined structure.
18. Describe the make-up of a module-defined structure.
19. Describe the make-up of a user-defined structure.
20. Explain two ways of creating tags.
21. When defining tags what limitations are placed on the entering of a tag name?
22. What is meant by the tag display style?
23. Write an example of an array tag used to hold 4 speeds.

Part 2 Bit-Level Programming

Part Objectives

After completing this part, you will be able to:

- Know what happens during the program scan
- Demonstrate an understanding of input, output, and internal relay addressing format for a tag-based Logix controller
- Develop ladder logic programs with input instructions and output coil combinations
- Develop ladder logic programs with latched outputs

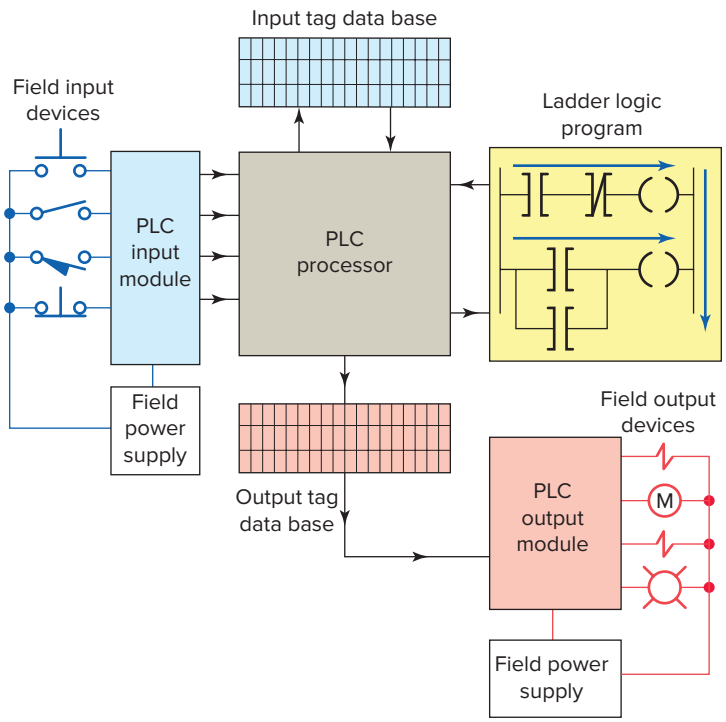


Figure 15-26 Logix controller operating cycle.

Program Scan

When a CLX controller executes a program, it must know—in real time—when external devices controlling a process are changing. During each operating cycle, the processor reads all the inputs, takes these values, and energizes or de-energizes the outputs according to the user program. This process is known as the ***program scan***.

Figure 15-26 illustrates the signal flow into and out of a Logix controller during a controller's operating cycle when ladder logic is executing. During the program scan, the controller reads rungs and branches from left to right and top to bottom as follows:

- Only one rung at a time is scanned.
- As the program is scanned, the status of inputs are checked for True (1 or ON) or False (0 or OFF) conditions.
- The status signals from the inputs are sent to the input tags where they are stored.
- As the program is scanned by the processor, inputs are checked for True or False conditions and the ladder logic is evaluated based on these values.
- The resulting ON or OFF action, as a result of evaluating each rung, is then sent to the output tags for storage.
- During the output update portion of the scan, corresponding output values are sent to the process or machine by way of the output module.

- I/O updates occur asynchronously to the scan of the logic. With a ControlLogix processor two separate 32-bit unsynchronized processes go on simultaneously—that is, asynchronously. This means that the module can update the input tag from the field and write the output tag to the field at any point (or at several points) during the processor's execution of the ladder rungs. The result is more efficiency and control over when the input field device data are updated in the input tag and when the output data resulting from the solved logic are sent to the output modules and their respective field devices.

Creating Ladder Logic

Although other programming languages are available, ladder logic is the most common programming language for PLCs. The instructions in ladder logic programming can be divided into two broad categories: input and output instructions. The most common input instruction is equivalent to a relay contact and the most common output instruction is the equivalent of a relay coil (Figure 15-27). When creating ladder I/O bit instructions, the following rules apply:

- All input instructions must be to the left of an output instruction.
- A rung cannot begin with an output instruction if it also contains an input instruction. This is because the controller tests all inputs for true or false before deciding what value the output instruction should be.
- A rung does not need to contain any input instructions, but it must contain at least one output instruction.

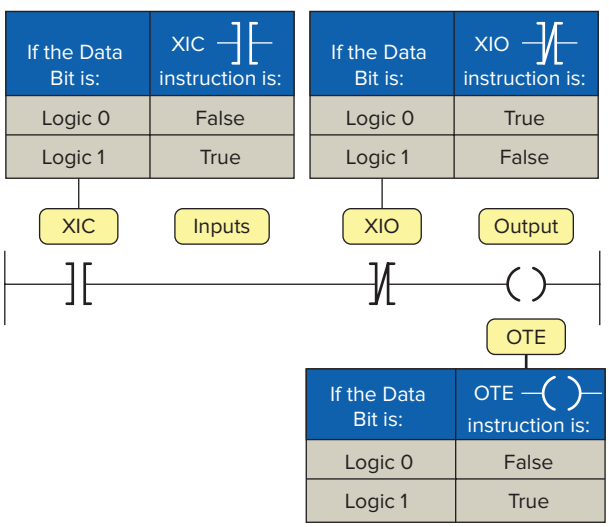


Figure 15-27 Contacts and coil instructions.

- When a rung has only one output instruction it will always be true.
- The last instruction on a rung must always be an output instruction.
- The XIC, or Examine If Closed contact instruction, checks to see if the input has a value of one. If the input is one, the XIC instruction returns a true value.
- The XIO, or Examine If Open contact instruction, checks to see if the input has a value of zero. If the input is zero, the XIO instruction returns a true value.
- The OTE or Output Energize coil instruction sets the tag associated with it to true or one when the rung has logic continuity. When true it can be used to energize an output device or simply set a value in memory to one.

ControlLogix PACs support multiple outputs on one rung. CLX controllers allow the use of serial logic that does not conform to traditional electrical hardwired circuits or ladder logic. For example, both of the rungs shown in Figure 15-28 are valid in RSLogix 5000. However the series connection of outputs would not work if wired that way in an equivalent electrical circuit or programmed that way in RSLogix 500. In both instances in RSLogix 5000, instructions tagA and tagB must be true to energize output tag1 and tag2.

In ControlLogix output instructions can be placed between input instructions as illustrated in Figure 15-29. In this example instructions tagA and tagB must be true to energize output tag1. Instructions tagA and tagB and tagC must all be true before output tag2 is set to energize.

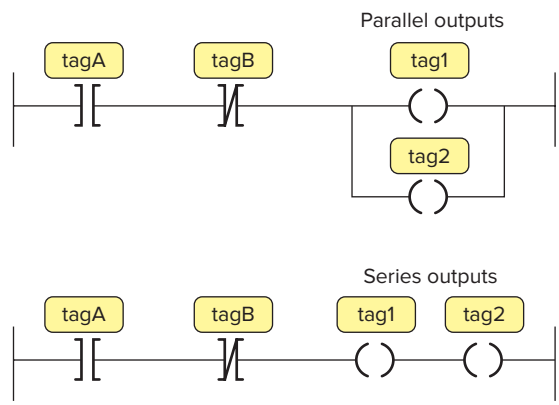


Figure 15-28 Parallel and series outputs.

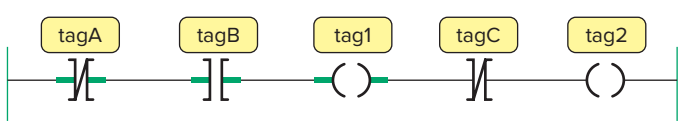


Figure 15-29 Output instruction placed between input instructions.

Tag-Based Addressing

Logix 5000 controllers use a tag-based addressing structure. A **tag** is a text-based name for an area of the controller where data is stored. An example of how a tag-based address is implemented using a ControlLogix controller is shown in Figure 15-30. Tag names use a meaningful description of the variable. In this application when the normally closed high limit switch is activated the program will switch the high limit output light on. The addressing format can be summarized as follows:

- The physical address for the tag Limit_switch is Local:1:I.Data.2(C). Local indicates that the module is in the same rack as the processor, 1 indicates that the module is in slot 1 in the rack, I indicates that the module is an input type, Data indicates that it is a digital input, 2 indicates that the limit switch is connected to terminal 2 on the module, and C indicates that it is a controller tag with global access.
- The physical address for the tag High_limit_light is Local:2:O.Data.4(C). Local indicates that the module is in the same rack as the processor, 2 indicates that the module is in slot 2 in the rack, O indicates that the module is an output type, Data indicates that it is a digital input, 4 indicates that the high limit light is connected to terminal 4 on the module, and C indicates that it is a controller tag with global access.

One advantage of the use of tag-based addressing is that the allocation of variable names for program values is not tied to specific memory locations in the memory structure, as is the case with rack/slot and rack/group type systems.

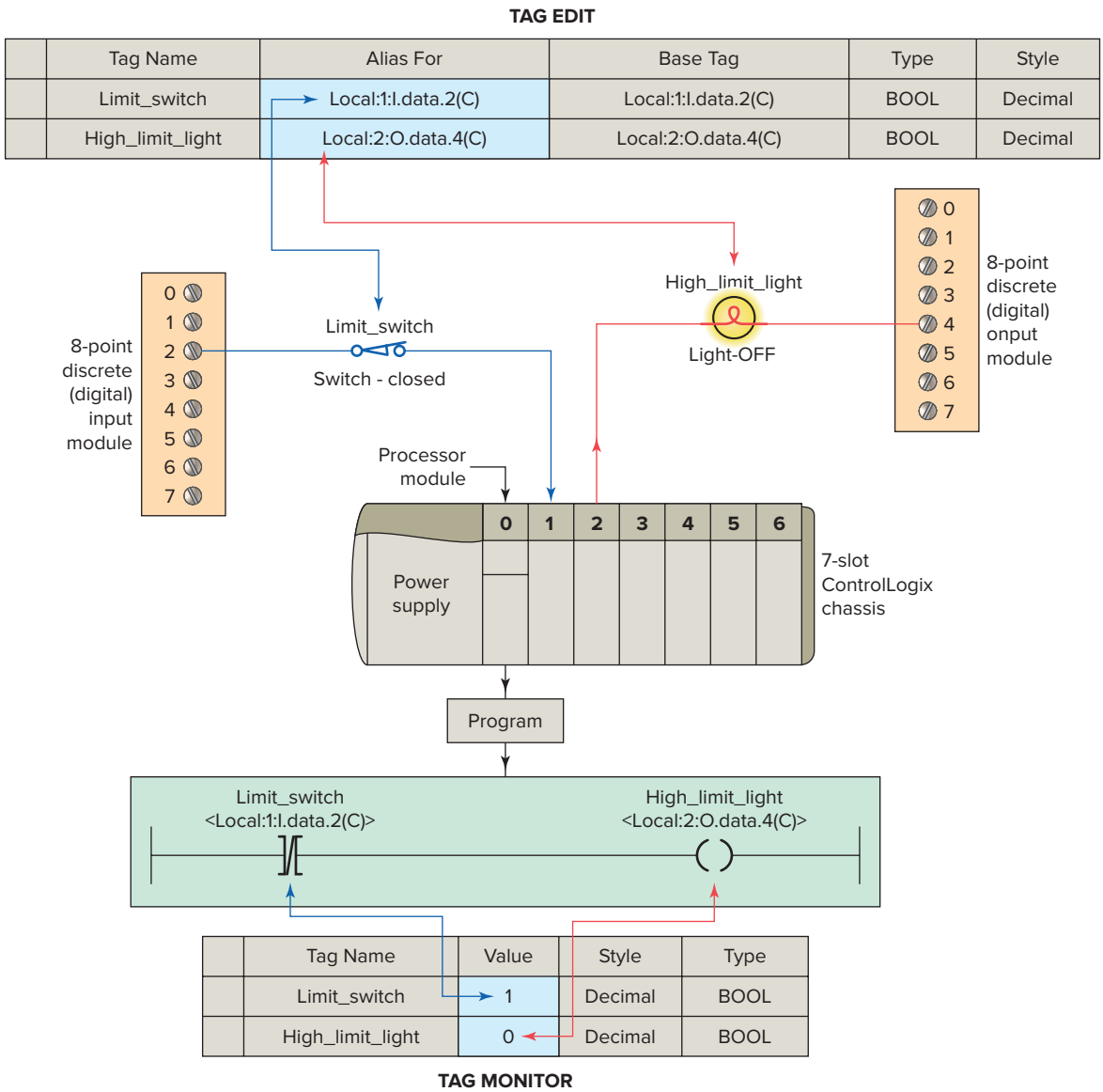


Figure 15-30 Tag-based address implementation.

Initially, all program development can proceed with just the tag names and data types assigned. Using tag aliases, programmers can write code independent of electrical connection assignments. At a later date, input and output field devices are easily matched to the pin numbers on the respective module they are connected to.

Adding Ladder Logic to the Main Routine

Figure 15-31 shows the diagram for a hardwired contactor operated motor start/stop control circuit. The normally open start button is momentarily closed to energize the contactor coil and close its main contacts to start the motor. The seal-in auxiliary contact of the contactor is connected in parallel with the start button to keep the starter coil energized when the start button is released. The normally closed stop button is momentarily opened to de-energize the contactor coil and stop the motor.

Figure 15-32 shows the ladder logic program for the motor start/stop control circuit and the RSLogix 5000 toolbar used to create it. Free form editing found in RSLogix 5000 helps speed development in that you do not have to assign addresses to instructions before adding more instructions. In this example we have chosen to use question marks [?] in place of tag names and assign the tags later. Field device wiring for the two pushbutton inputs and the single contactor coil output are as illustrated. The stop button is connected to terminal 4 and the start button to terminal 3 of the DC input module located in slot 1 of the rack. The contactor coil is connected to terminal 4 of the DC output module located in slot 2 of the rack. Both the start and stop buttons are examined for a closed condition (XIC) because both buttons must be closed to cause the motor starter to operate.

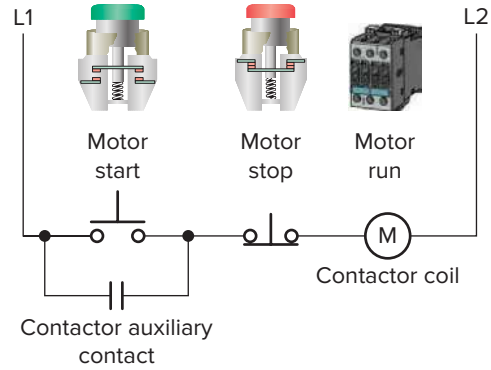


Figure 15-31 Hardwired motor start/stop control circuit.

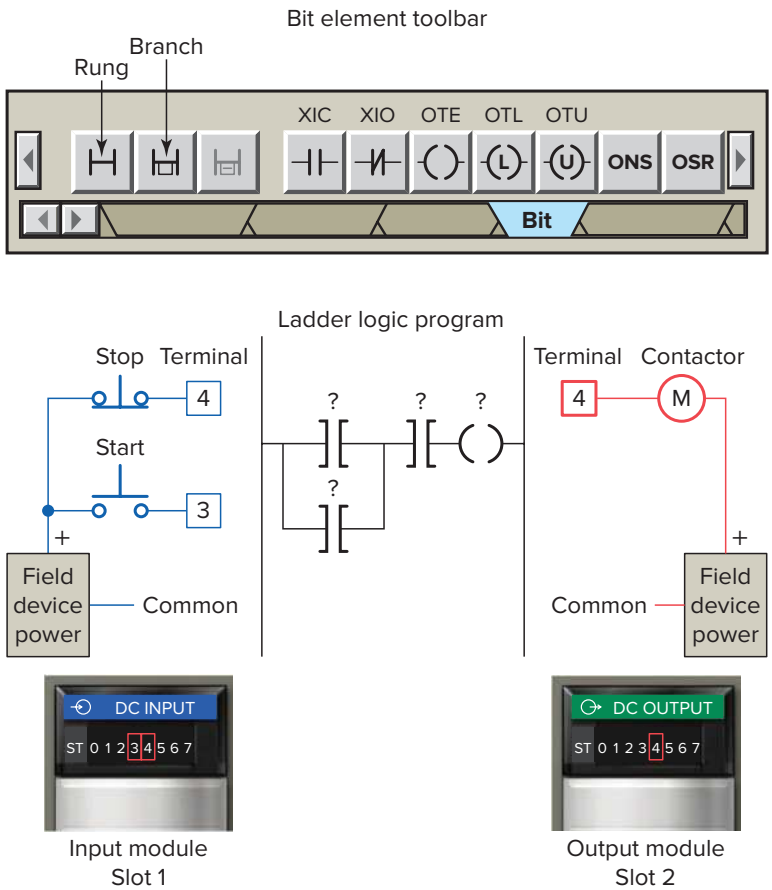


Figure 15-32 Programmed motor start/stop control circuit.

With text-based Logix systems you can use the name of the tag to document your ladder code and organize your data to mirror your application. For the programmed motor start/stop control circuit three tags, Motor_Start, Motor_Stop, and Motor_Run, are created. Figure 15-33 illustrates how the Motor_Start tag is created in the New Tag window. This window can be accessed by right clicking the ? mark above the XIC instruction in the ladder logic program. Since this tag represents a value from an input field device, a link through the module to the field device must be created. When Local:1:I.Data is selected a dialog box for all of the terminal numbers on the input module appears. The tag name (Motor_Start) used in the program is then linked to input terminal number 3 where the field device represented by the tag name is connected.

Figure 15-34 shows what the ladder logic program would look like after all three tags have been created. Users have the ability to reference data via multiple names using Aliases. This allows the flexibility to name data differently depending on their use. The tag description provides for a more meaningful description of the tag name. Tag names are downloaded and stored in the controller

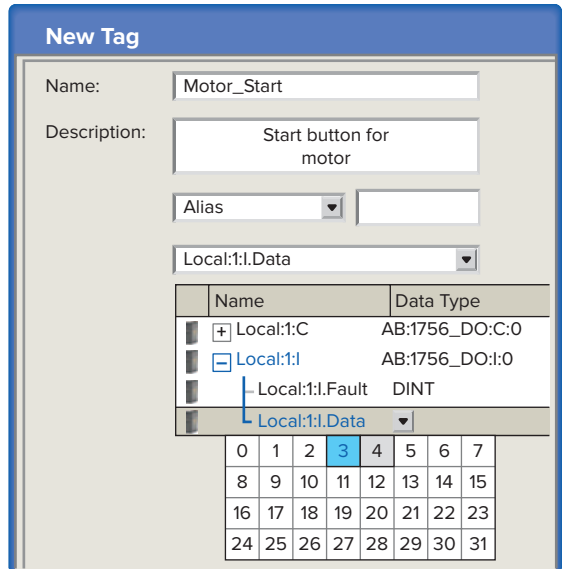


Figure 15-33 Creating the Motor_Start tag.
Source: Image Courtesy of Rockwell Automation, Inc.

but the description is not, as it is part of the documentation of the project.

Figure 15-35 shows the state of the tags created for the motor start/stop program as seen in the program and

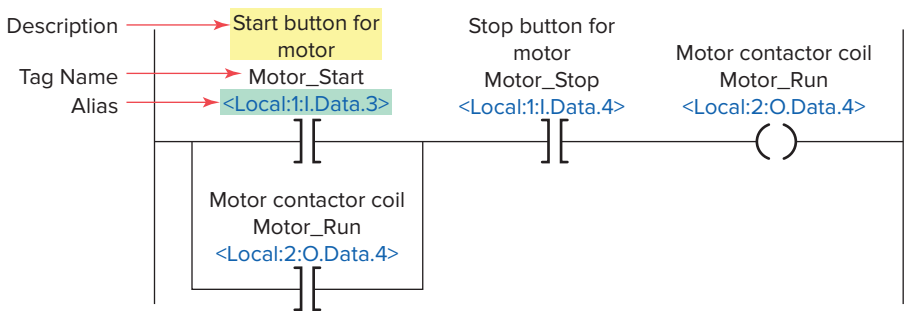
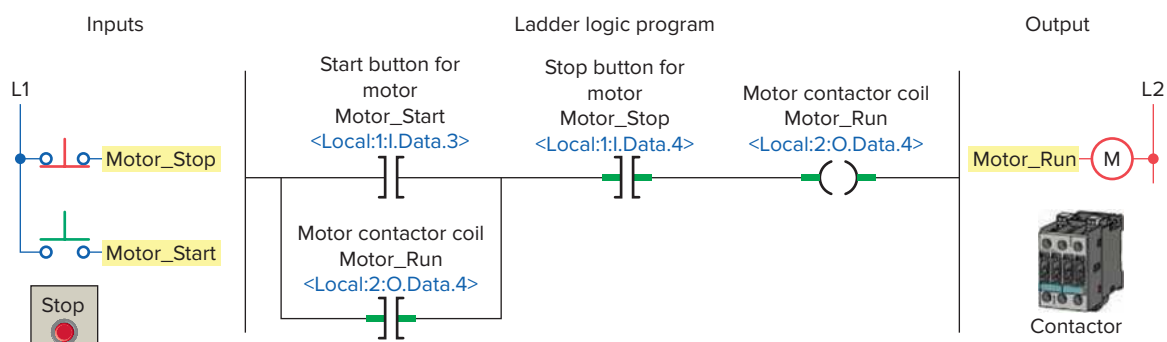


Figure 15-34 Ladder logic program after all tags have been created.



Monitor Tags Window

Tag Name	Value	Style	Data Type	Description
Motor_Start	0	Decimal	BOOL	Start button for motor
Motor_Stop	1	Decimal	BOOL	Stop button for motor
Motor_Run	1	Decimal	BOOL	Motor contactor coil

Figure 15-35 Ladder logic program and Monitor Tags window with motor operating.

Monitor Tags window, when the motor is operating. When the motor is operating:

- The XIC Motor_Start instruction is false because the NO start button is open; therefore its value is 0.
- The XIC Motor_Stop instruction is true because the NC stop button is closed; therefore its value is 1.
- The OTE Motor_Run instruction is true because the rung has logic continuity; therefore its value is 1.

A hardwired reversing conveyor motor control circuit is shown in Figure 15-36. The operation of the circuit can be summarized as follows:

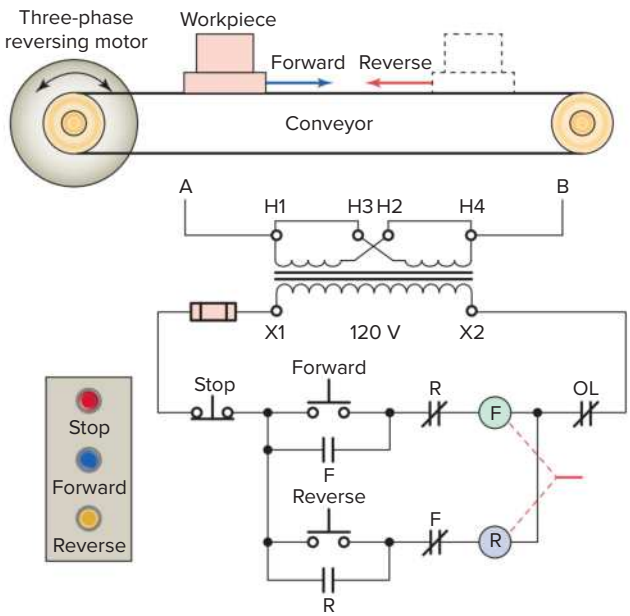


Figure 15-36 Hardwired reversing conveyor motor control circuit.

- The normally closed contact controlled by the forward contactor coil is connected in series with the reverse coil.
- The normally closed contact controlled by the reverse contactor coil is connected in series with the forward coil.
- When the forward coil is energized, the normally closed contact in series with the reverse coil is opened to prevent the reverse coil from being energized.
- When the reverse coil is energized, the normally closed contact in series with the forward coil opens to prevent the forward coil from being energized.
- To reverse the motor with this control circuit, the operator must press the stop button to de-energize the respective coil, reclosing the respective normally closed contact.
- Figure 15-37 shows the reversing starter circuit implemented using an RSLogix 5000 controller.

Internal Relay Instructions

Internal relay instructions are used when other than real-world field devices are needed as input or output reference instructions. For example, an internal relay bit is used as an output when the logical resultant of a rung is used to control other internal logic. An internal control relay is programmed in the ControlLogix system by creating a tag (either program or controller type) and assigning a Boolean type to the tag.

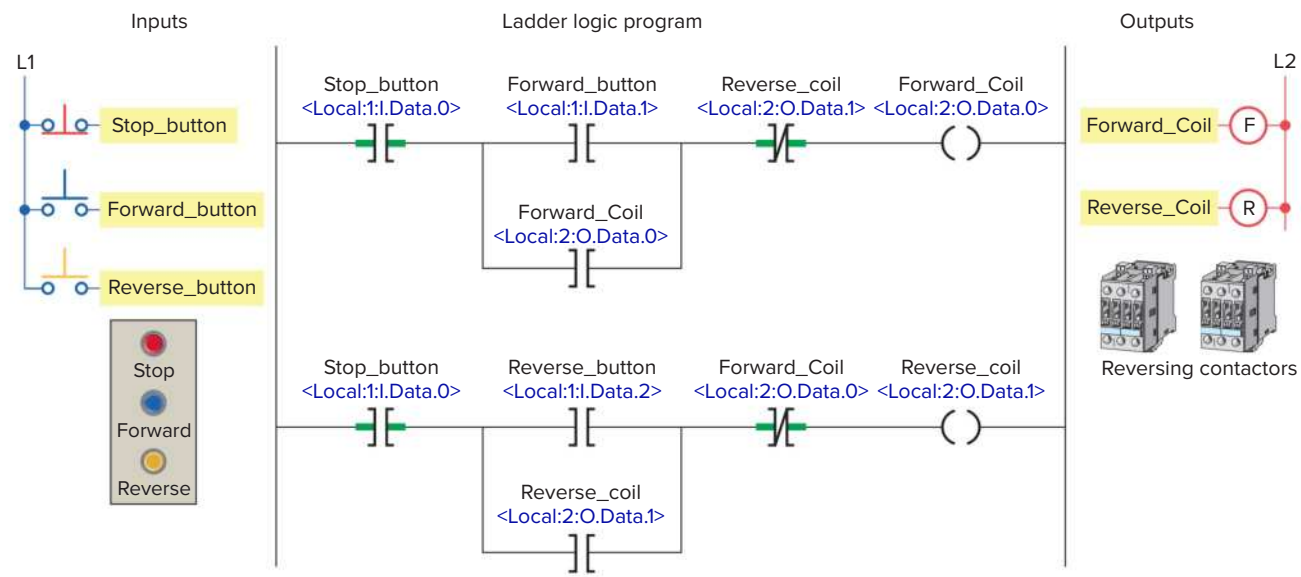
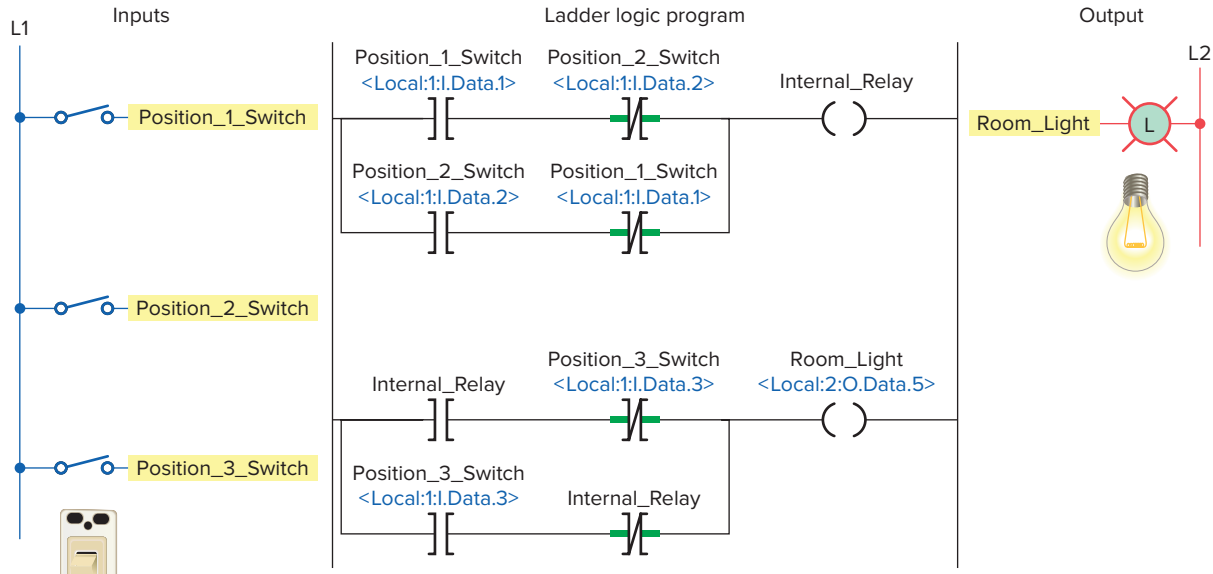


Figure 15-37 Programmed reversing conveyor motor control circuit.



Monitor Tags Window

Name	Value ←	Force Mask ←	Style	Data Type
Internal_Relay	0		Decimal	BOOL
Position_1_Sw...	0		Decimal	BOOL
Position_2_Sw...	0		Decimal	BOOL
Position_3_Sw...	0		Decimal	BOOL
Room_Light	0		Decimal	BOOL

Figure 15-38 Internal relay to implement on/off control of a room light from three different entrances.

Figure 15-38 shows a ControlLogix program that uses an internal relay to implement on/off control of a room light from three different entrances or positions. Three single pole switches are used for inputs in place of the two 3-way and one 4-way switches normally required for an equivalent hardwired control circuit. The operation of the program can be summarized as follows:

- An internal relay is used to execute the logic of the circuit without having to use a real-world output.
- The status value stored in memory for all tags, when all input switches are open, is 0 and so the room light will be off.
- Closing Position_1_Switch changes the status of its XIC instruction from false to true thereby establishing logic continuity for Rung 1.
- As a result, the status of the internal relay coil and its XIC contact change from false to true.
- This establishes logic continuity for Rung 2 and switches the room light on.
- A change in the state of any of input switches will change the current state of the light.

Internal relay output coils are used in ladder logic programs, but are not accessible at I/O racks. They are used where an output instruction is required in a program, but no physical connection to a real world device is needed. A hardwired pilot light motor control circuit is shown in Figure 15-39. The operation of the circuit can be summarized as follows:

- The Stop/Start pushbutton station controls relay coil CR.
- When CR is de-energized, the green standby pilot light is ON, the red run pilot is OFF, and the motor is not operating.

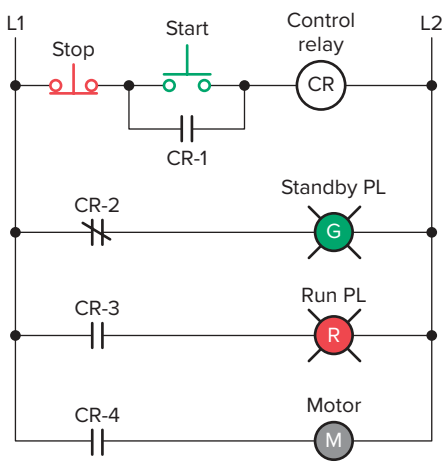


Figure 15-39 Hardwired pilot light motor control circuit

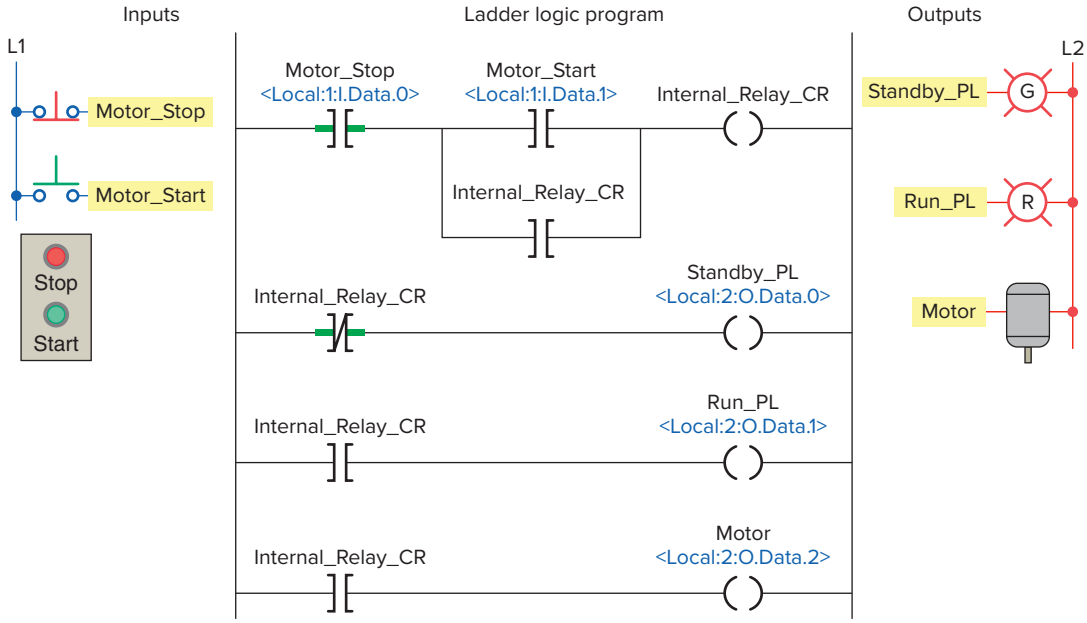


Figure 15-40 Pilot light motor control circuit implemented using an RSLogix controller.

- When CR is energized, the red run pilot light turns ON, the green standby pilot light turns OFF, and motor M starts operating.

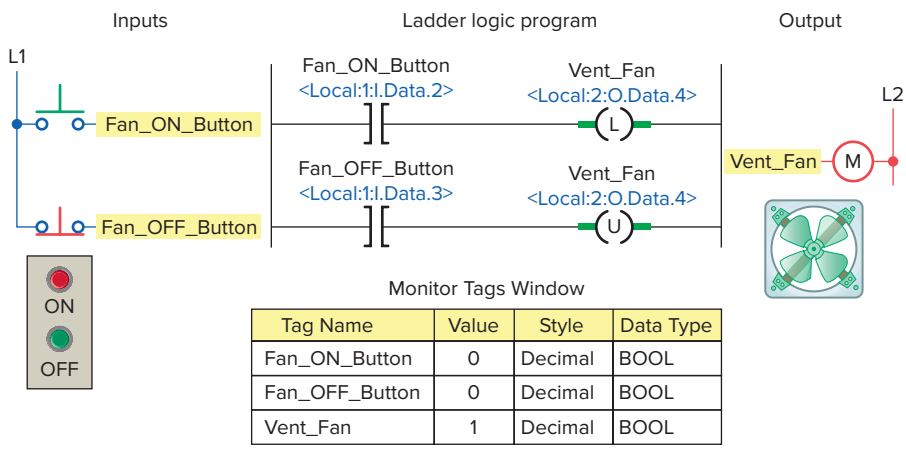
Figure 15-40 shows the pilot light motor control circuit implemented using an RSLogix controller. The control relay CR is represented by tag Internal_Relay_CR. Note that Internal_Relay_CR doesn't have an address and as such is not accessible at the I/O rack.

Latch and Unlatch Instructions

The ***output latch (OTL)*** instruction is a retentive output instruction that is used to maintain, or latch, an output. If this output is turned on, it will stay on even if the status of the input logic that caused the output to energize becomes false. The OTL instruction will remain in a latched on condition until an **unlatch instruction (OTU)** with the same referenced tag is energized. The OTL instruction is often used in programs where the value of a variable must be maintained in instances where there is a shutdown due to a power failure or system fault. Retentive memory permits the system to be restarted with memory locations holding the values that were present when the program execution was halted.

Figure 15-41 shows a ControlLogix program that uses an output latch and unlatch instruction pair to implement the control of a vent fan motor. The operation of the program can be summarized as follows:

- The OTL instruction will write a 1 to its address when true.



Monitor Tags Window

Tag Name	Value	Style	Data Type
Fan_ON_Button	0	Decimal	BOOL
Fan_OFF_Button	0	Decimal	BOOL
Vent_Fan	1	Decimal	BOOL

Figure 15-41 Output latch and unlatch instructions used to control a vent fan motor.

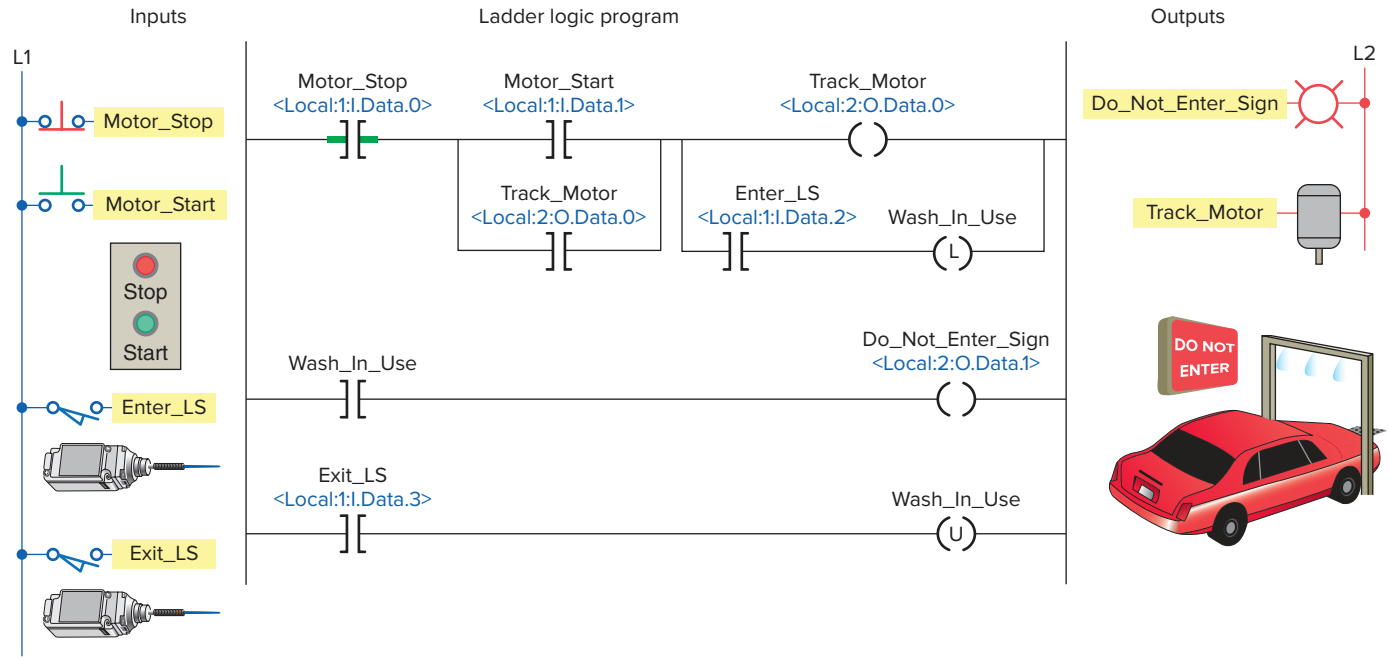


Figure 15-42 Latch/unlatch instruction used as part of a car wash program.

- When the OTL goes false, the output address will remain a 1.
- This is true even if the processor powers down and then back up.
- The output address will remain a 1 until reset to 0 by the unlatch instruction.
- If the output address is off, both the latch and unlatch instructions are not intensified, but once the bit is turned on, you will see both the latch and unlatch intensified even though both inputs are shut off.

Figure 15-42 shows an application of the latch/unlatch instruction used as part of a car wash program. The car wash uses a conveyor chain that pulls cars along a track. The operation of the program can be summarized as follows:

- The car wash only washes one car at a time.
- The operator controls the ON/OFF operation of the track motor with the track motor Stop and Start pushbuttons.
- When the Motor_Start button is closed the Track_Motor is energized and performs a seal-in function that keeps the motor operating when the Motor_Start button is released.
- As the car enters the car wash, the Enter_LS contact momentarily closes to energize the Wash_In_Use latch instruction.
- This in turn, energizes the Do_Not_Enter_Sign to indicate that the car wash is in use.
- When the car exits the car wash, the Exit_LS contact momentarily closes and energizes the Wash_In_Use unlatch instruction.
- This in turn, de-energizes the Do_Not_Enter_Sign to indicate the car wash is not in use.
- The track motor remains running ready to restart the process once another car enters.
- Once running, the track motor can be stopped at any time by operating the Motor_Stop button to de-energize the Track_Motor.

One-Shot Instruction

The CLX ***One-Shot (ONS)*** instruction is an input instruction used to turn an output on for one program scan only. The program of Figure 15-43 uses the ONS instruction with a math instruction to perform a calculation once per closing of the switch. This program is used to execute the ADD math function only once per actuation of the limit switch, no matter how long the limit switch is held closed. The operation of the program can be summarized as follows:

- On any scan for which *limit_switch_1* is cleared or *storage_1* is set, this rung has no effect.
- On any scan for which *limit_switch_1* is set and *storage_1* is cleared, the ONS instruction sets *storage_1* and the ADD instruction increments *sum* by 1.
- As long as *limit_switch_1* stays set, *sum* stays the same value. The *limit_switch_1* must go from cleared to set again for *sum* to be incremented again.

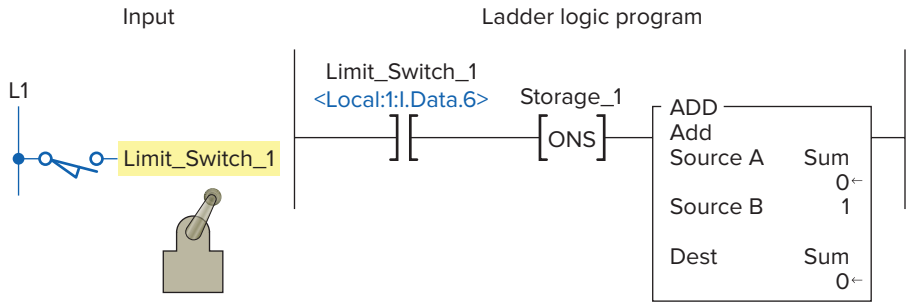


Figure 15-43 ONS instruction used to perform a calculation once per scan.

The **One-Shot Rising (OSR)** instruction shown in Figure 15-44 is an output instruction which produces an output for one program scan when the input rises from zero to one (leading edge of the input pulse). The OSR instruction sets or clears the output bit, depending on the status of the Storage Bit (SB). When enabled and the storage bit is cleared, the OSR instruction sets the output bit. When enabled and the storage bit is set or when disabled, the OSR instruction clears the output bit.

The **One-Shot Falling (OSF)** instruction shown in Figure 15-45 is an output instruction which produces an output for one program scan when the input drops from one to zero (falling edge of the input pulse). The OSF instruction sets or clears the output bit depending on the status of the storage bit. When disabled and the storage bit is set, the OSF instruction sets the output bit. When disabled and the storage bit is cleared, or when enabled, the OSF instruction clears the output bit.

One-shot instructions are very useful for math operations. The program of Figure 15-46 shows an example of a program that makes use of all three one-shot instructions. The function of the program is to add 4, 8, or 12 to a register each time the associated input sensor detects a product. The instructions evaluate the preceding conditions, and when either the ONS, OSR, or OSF input changes state, a shot is triggered. The Clear Sums input pushbutton is used to clear all three instructions back to zero.

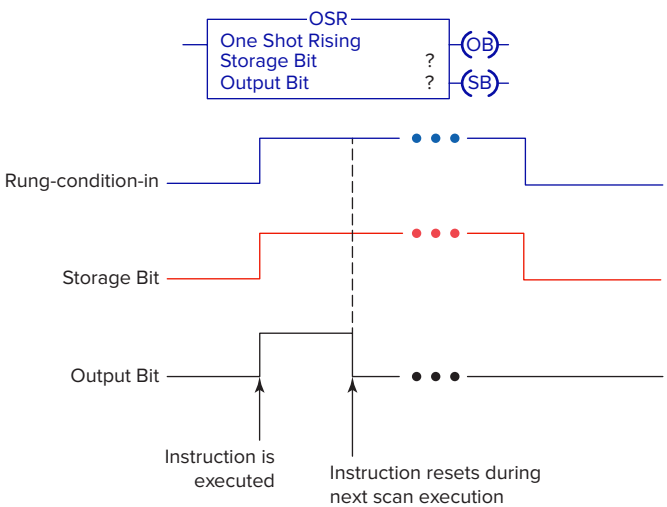


Figure 15-44 One-Shot Rising (OSR) instruction.

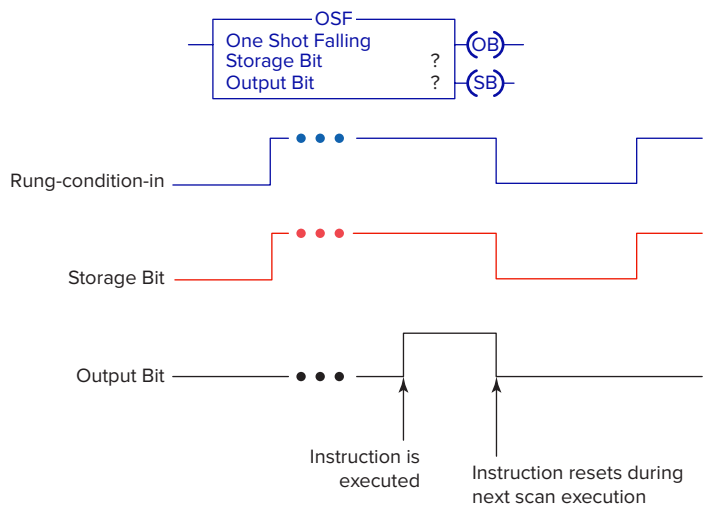


Figure 15-45 One-Shot Falling (OSF) instruction.

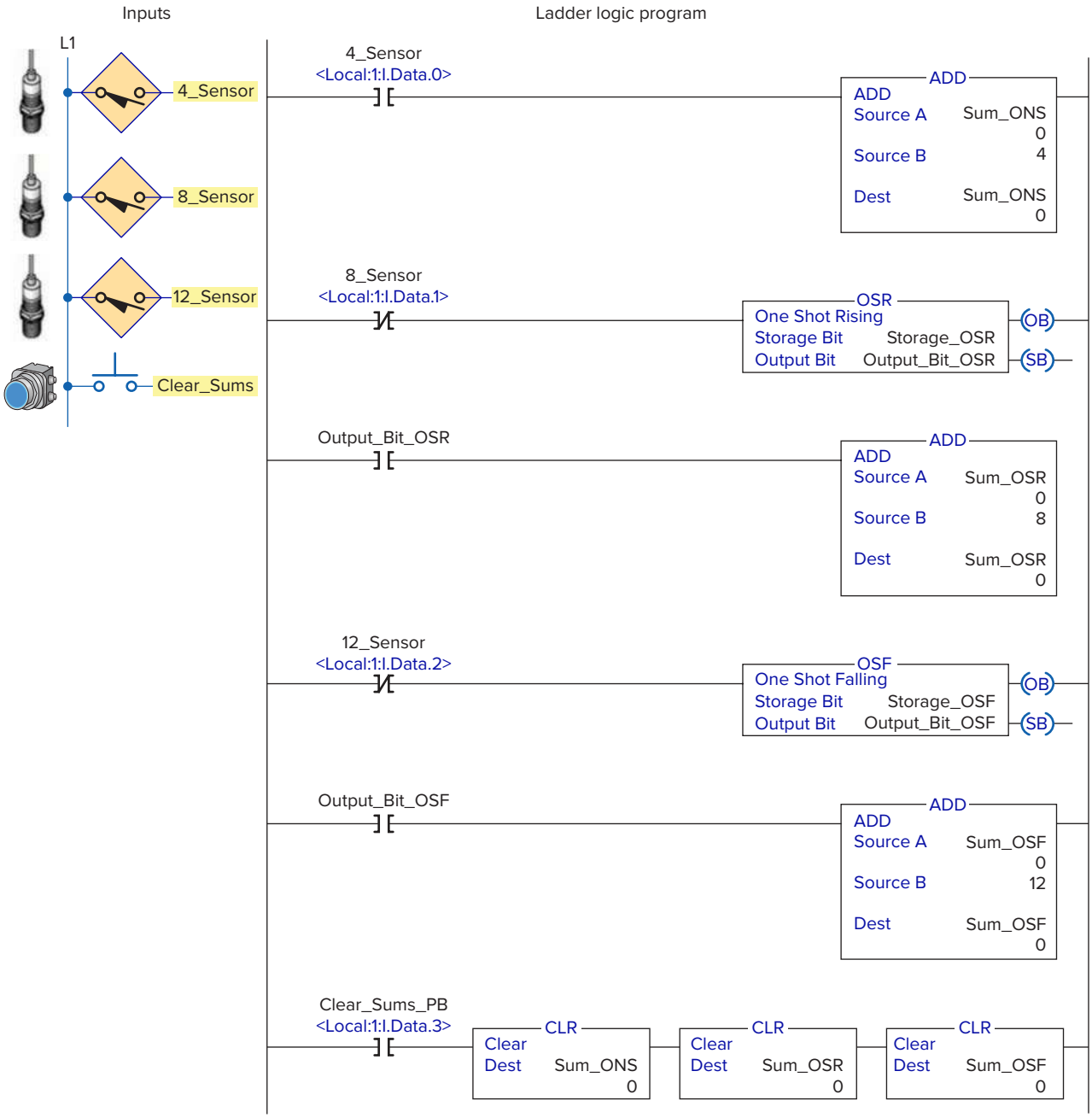


Figure 15-46 One-shot instructions used in conjunction with math operations.

PART 2 REVIEW QUESTIONS

1. What operations are performed by the processor during the program scan?
2. With a ControlLogix processor, I/O updates occur asynchronously. Explain what this means.
3. In ladder logic programming into what two broad categories can instruction types be classified?
4. A field input switch is examined using an XIC instruction.
   a. What is the value (0 or 1) stored in its memory bit when the switch is opened and closed?
   b. What is the state of the instruction (true or false) when the switch is opened and closed?
5. A field input switch is examined using an XIO instruction.
   a. What is the value (0 or 1) stored in its memory bit when the switch is opened and closed?
   b. What is the state of the instruction (true or false) when the switch is opened and closed?
6. The value of an OTE instruction as it appears in the Monitor Tags window is 1. Explain what this means as far as the status of a real-world field output and programmed XIC and XIO instructions associated with this tag are concerned.
7. Define a tag in the ControlLogix system.
8. What advantage do tag-based addressing systems have over rack/slot types?
9. How is an internal relay programmed in the ControlLogix system?
10. The output latch instruction is a retentive output instruction. Explain what retentive means.
11. The ControlLogix ONS instruction is a one-shot instruction. Explain what this means.

PART 2 PROBLEMS

1. Modify the original ControlLogix start/stop motor control program (Figure 15-34) with a second start and stop button added to the program. The additional start button is to be connected to pin 1 and the stop button to pin 2 of the digital input module.
2. Extend control of the original ControlLogix internal relay program (Figure 15-38) used to control a room light from 3 entrances to 4. The additional single-pole switch is to be connected to pin 4 of the digital input module.
3. Implement the hardwired latching relay alarm circuit of Figure 15-47 in Logix format. The alarm will be latched on anytime:
   - The normally open temperature switch closes.
   - Both normally open float switches 1 and 2 close.
   - Either normally open sensor switch 1 or 2 closes while the normally closed pressure switch is closed.
4. Implement the hardwired tank filling and emptying operation shown in Figure 15-48 in Logix format.

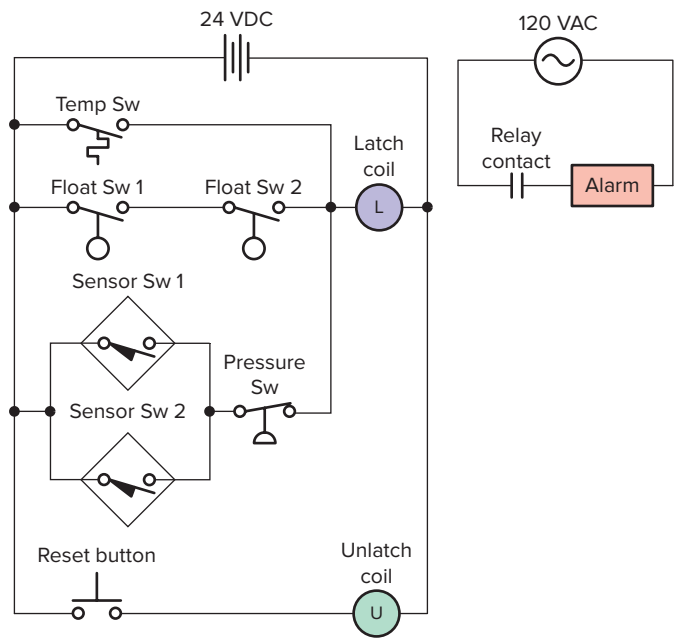


Figure 15-47 Hardwired latching relay alarm circuit for Problem 3.

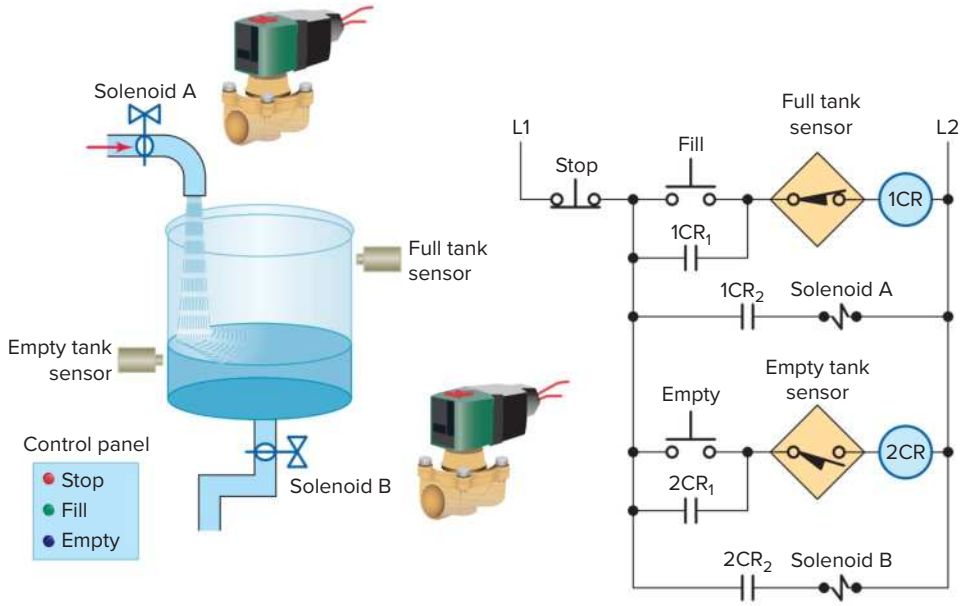


Figure 15-48 Hardwired tank filling and emptying operation for Problem 4.

The operation of the control circuit can be summarized as follows:

- Assuming the liquid level of the tank is at or below the empty level mark, momentarily pressing the FILL pushbutton will energize control relay 1CR.
- Contacts $1CR_1$ and $1CR_2$ will both close to seal in the 1CR coil and energize normally closed solenoid valve A to start filling the tank.
- As the tank fills, the normally open empty-level sensor switch closes.
- When the liquid reaches the full level, the normally closed full-level sensor switch opens to open the circuit to the 1CR relay coil and switch solenoid valve A to its de-energized closed state.
- Anytime the liquid level of the tank is above the empty-level mark, momentarily pressing the EMPTY pushbutton will energize control relay 2CR.
- Contacts $2CR_1$ and $2CR_2$ will both close to seal in the 2CR coil and energize normally closed solenoid valve B to start emptying the tank.
- When the liquid reaches the empty level, the normally open empty-level sensor switch opens to open the circuit to the 2CR relay coil and switch solenoid valve B to its de-energized closed state.
- The stop button may be pressed at any time to halt the process.

Part 3 Programming Timers

Part Objectives

After completing this part, you will be able to:

- Understand ControlLogix timer tags and their members
- Utilize status bits from timers in logic
- Develop ladder logic programs using ControlLogix timers

Timer Predefined Structure

Timers are used to turn outputs on and off after a time delay, turn outputs on or off for a set amount of time, and keep track of the time an output is on or off. The timer address in the SLC 500 controller is a data table address or symbol, whereas the timer address in the ControlLogix controller is a predefined structure of the TIMER data type. The **TIMER structure** is shown in Figure 15-49. Timer parameters and status bits include:

- **Tag Name**—User-friendly tag name for the timer (e.g., Pump_Timer). If you want to use a timer, you must create a tag of type timer.
- **Preset (PRE)**—The number of time increments that the timer must accumulate to reach the desired time delay. Specifies the value (in milliseconds) which the timer must reach before the done bit (DN) changes state. The preset value is stored as a binary number (DINT). The time base is always 1 msec. For example, for a 3 second timer, enter 3000 for the PRE value.
- **Accumulator (ACC)**—The accumulator value is the number of milliseconds the instruction has been enabled. The accumulator value stops changing when ACC value = PRE value.
- **Enable Bit (EN)**—The enable bit indicates the timing instruction is enabled. The EN bit is true when the rung input logic is true, and false when the rung input logic is false.
- **Timer Timing Bit (TT)**—The timing bit indicates that a timing operation is in process. The TT bit is true only when the accumulator is incrementing. TT remains true until the accumulator reaches the preset value.
- **Done Bit (DN)**—The done bit indicates that accumulated value (ACC) is equal to the preset (PRE)

Data Type: TIMER

Name: Pump_Timer

Description:

Members: **Data Type Size: 12 byte(s)**

	Name	Data Type	Style	Description
	PRE	DINT	Decimal	
	ACC	DINT	Decimal	
	EN	BOOL	Decimal	
	TT	BOOL	Decimal	
	DN	BOOL	Decimal	

Figure 15-49 TIMER predefined structure.
Source: Image Courtesy of Rockwell Automation, Inc.

value. The DN bit signals the end of the timing process by changing states from false-to-true or from true-to-false depending on the type of TIMER instruction used. The DN bit is the most commonly used timer status bit.

On-Delay Timer (TON)

The ***on-delay timer (TON)*** is a nonretentive output instruction used when the application requires an action to occur at some time after the rung conditions for the timer become true. The ControlLogix TON on-delay instruction and timer selection toolbar are shown in Figure 15-50. When you want to use a timer, you must create a tag of type TIMER (it is a predefined data type) and enter the preset and the accumulated value. The tag must be defined before the preset and accumulated values can be entered. A value can be entered for the accumulator while programming. When the program is downloaded this value will be in the timer for the first scan. If the TON timer is not enabled the value will be set back to zero. Normally zero will be entered for the accumulator value.

The timer tag name is declared using the new tag properties dialog box shown in Figure 15-51. Tag name, description (optional), tag type, data type, and scope are selected or typed to complete the validation. A descriptive tag name, such as Solenoid_Delay, makes it easier to know what function the timer serves in the control system.

The program of Figure 15-52 is an example of a 10000 ms (10 s) TON timer. Timers generate both word level (DINT) and bit level (BOOL) data and status. The operation of the program can be summarized with reference to the Monitor Tags window.

- The status of all instruction is shown after the timer input switch has been switched from off to on (1) and accumulated 5000 ms (5 s) of time.
- At this halfway point the EN bit is 1 since the rung is true, the TT bit is 1 since the accumulated value is changing, and the DN bit is 0 since the accumulated value does not yet equal the preset value.
- When the ACC equals PRE, the accumulated value stops incrementing, EN stays on for as long as the rung remains true, TT equals 0 since the accumulated value is not changing, and DN equals 1 since ACC = PRE.
- This will result in the DN pilot light switching on at the same time as the TT pilot light switches off.
- The EN pilot light remains on as long as the input switch is closed.
- Opening the input switch at any time causes the TON instruction to go false, resetting the counter ACC value to 0 and EN, TT, and DN bits to 0. This in turn switches off all output pilot lights.
- The TON instruction is a self-resetting timer. When the rung goes false, the timer is automatically reset. A reset instruction can be used, but usually is not.

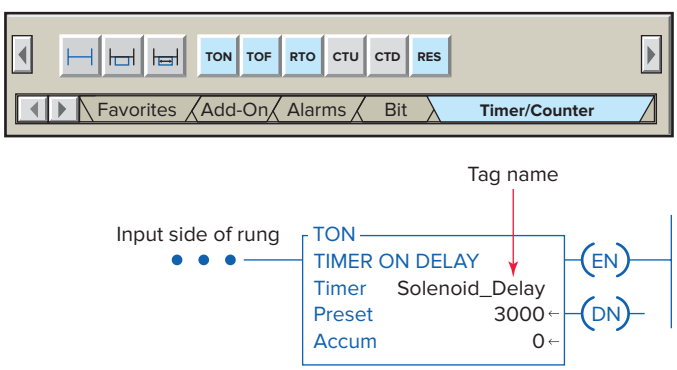


Figure 15-50 TON on-delay instruction.

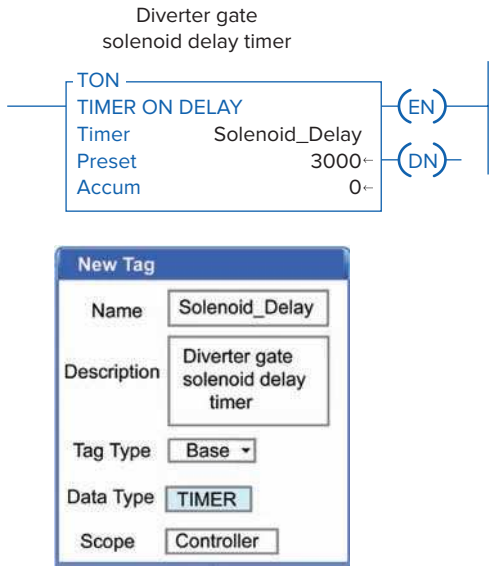
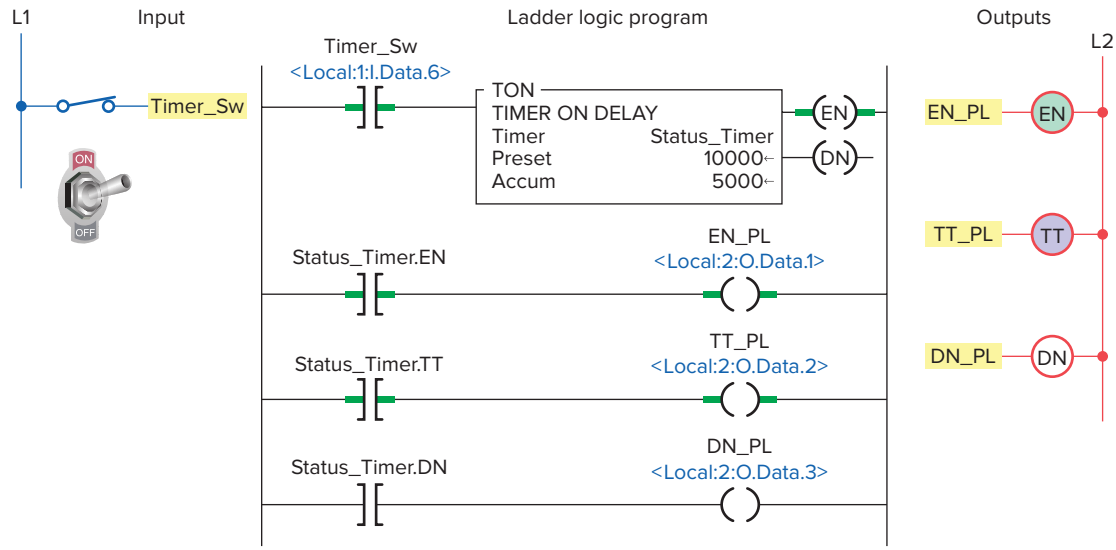


Figure 15-51 Timer tag validation.

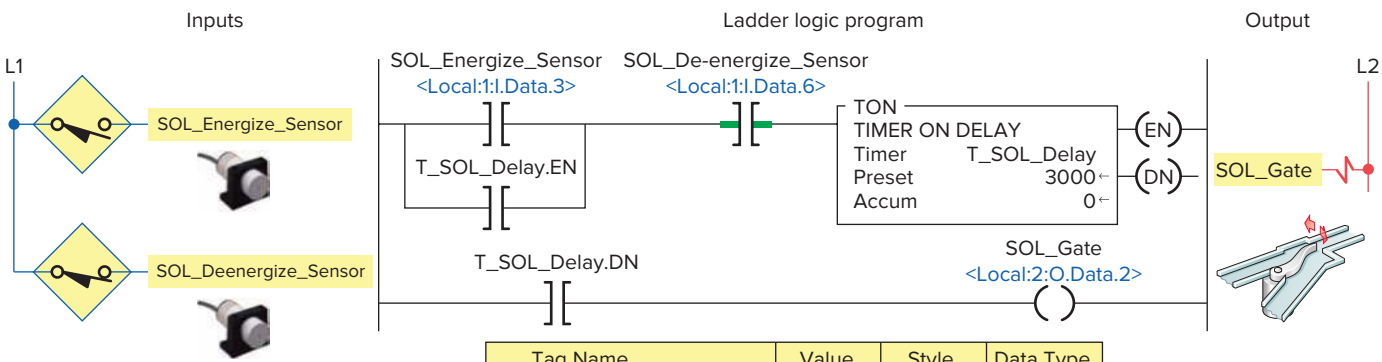
Figure 15-53 shows a TON timer used to delay the operation of a diverter gate solenoid for 3 seconds after a target has been sensed by the solenoid energize sensor. The operation of the program can be summarized as follows:

- Detection of the target causes closure of the SOL_Energize_Sensor contacts, making the timer rung true and start timing.
- With passage of the target, the SOL_Energize_Sensor contacts open but the rung remains true through the EN bit of the TON timer.
- After 3000 ms (3 s) delay time has elapsed, delay timer DN bit is set to 1 to energize the SOL_Gate.



Tag Name	Value	Style	Data Type
Timer_Sw	1	Decimal	BOOL
EN_PL	1	Decimal	BOOL
TT_PL	1	Decimal	BOOL
DN_PL	0	Decimal	BOOL
⊟ Status_Timer	{...}		TIMER
⊞ Status_Timer.**PRE**	10000	Decimal	DINT
⊞ Status_Timer.**ACC**	500	Decimal	DINT
Status_Timer.**EN**	1	Decimal	BOOL
Status_Timer.**TT**	1	Decimal	BOOL
Status_Timer.**DN**	0	Decimal	BOOL

Figure 15-52 Ten-second TON timer program.



Tag Name	Value	Style	Data Type
SOL_Energize_Sensor	0	Decimal	BOOL
SOL_Deenergize_Sensor	1	Decimal	BOOL
SOL_Gate	0	Decimal	BOOL
⊟ T_SOL_Delay	{...}		TIMER
⊞ T_SOL_Delay.PRE	3000	Decimal	DINT
⊞ T_SOL_Delay.ACC	0	Decimal	DINT
T_SOL_Delay.EN	0	Decimal	BOOL
T_SOL_Delay.TT	0	Decimal	BOOL
T_SOL_Delay.DN	0	Decimal	BOOL

Figure 15-53 TON timer used to delay the operation of a diverter gate solenoid.
Source: Photos courtesy Omron Industrial Automation, **www.ia.omron.com**.

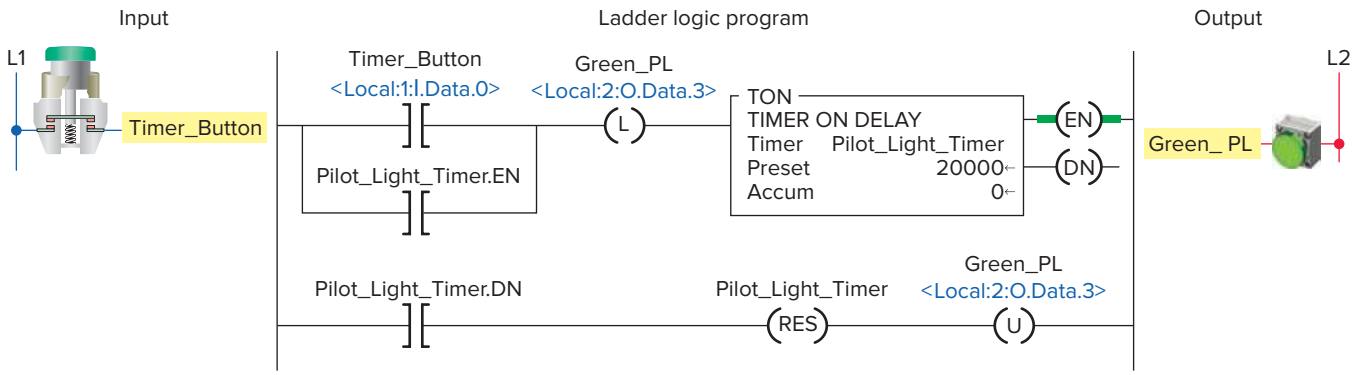


Figure 15-54 Pilot light TON timer.

- Momentary detection of the target by the SOL_ Deenergize_Sensor causes the opening of its contacts and resets the program to its original state.

Figure 15-54 shows a program that uses a TON timer to illuminate a green pilot light for 20 seconds each time a momentary button is pressed. In addition to the TON timer this program uses multiple outputs on one rung, output latch and unlatch instructions, as well as a timer reset instruction. The operation of the program can be summarized as follows:

- Initially closing the Timer_Button sets (latches) the Green_PL on and enables the Pilot_Light_Timer.
- When the button is then opened the timer rung remains true through the logic path created by the Pilot_Light_Timer.EN bit.
- After 20000 ms (20 s) have elapsed the timer DN bit is set to reset the timer to its original state and unlatch the Green_PL and switch it off.

The ControlLogix program of Figure 15-55 shows three TON timers cascaded (connected together) for traffic light control. The ladder logic used is the same as that used to program the traffic lights using the SLC 500 controller. The different tags created to fit the program are

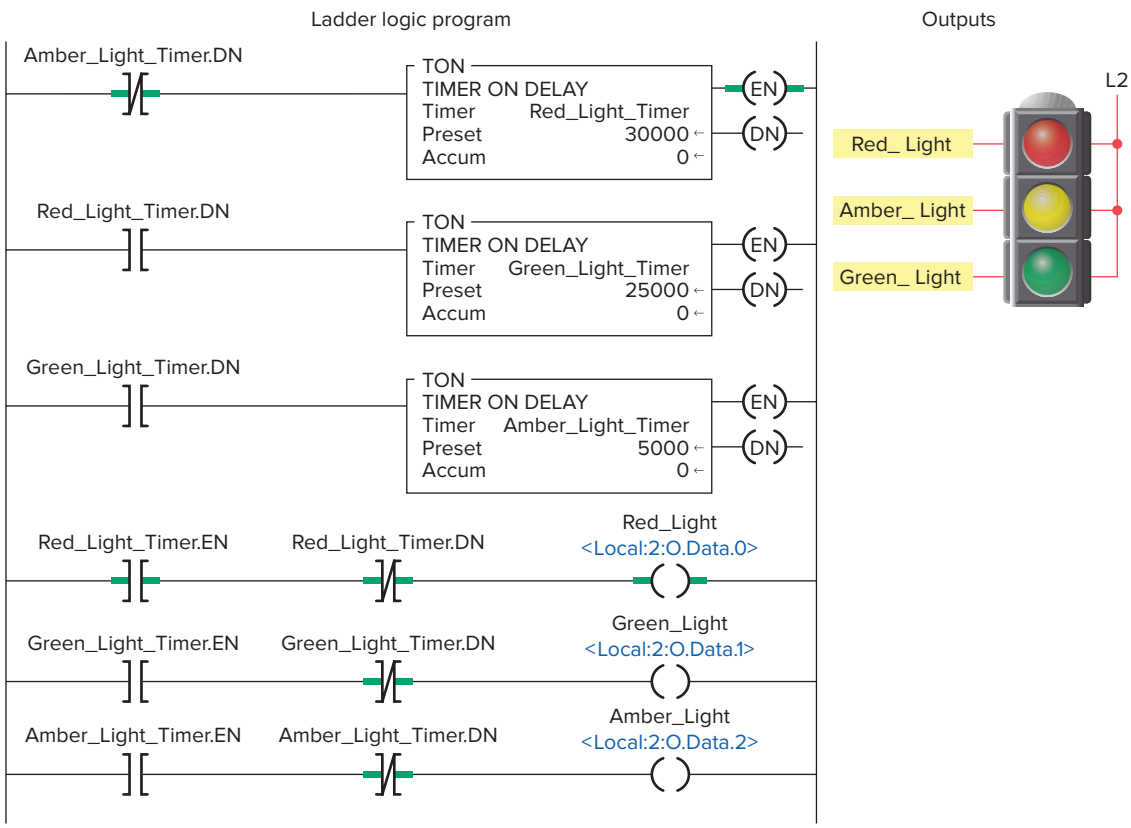


Figure 15-55 ControlLogix traffic control program.

Tag Name	Value	Style	Data Type
+-Amber_Light_Timer	{...}		TIMER
+-Green_Light_Timer	{...}		TIMER
−-Red_Light_Timer	{...}		TIMER
+-Red_Light_Timer.PRE	30000	Decimal	DINT
+-Red_Light_Timer.ACC	0	Decimal	DINT
—Red_Light_Timer.EN	1	Decimal	BOOL
—Red_Light_Timer.TT	1	Decimal	BOOL
—Red_Light_Timer.DN	0	Decimal	BOOL
Red_Light	1	Decimal	BOOL
Green_Light	0	Decimal	BOOL
Amber_Light	0	Decimal	BOOL

Figure 15-56 Tags created for traffic light program.

shown in Figure 15-56. Operation of the program can be summarized as follows:

- Transition from red light to green light to amber light is accomplished by the interconnection of the EN and DN bits of the three TON timer instructions.
- The input to the Red_Light_Timer is controlled by the Amber_Light_Timer.DN bit.
- The input to the Green_Light_Timer is controlled by the Red_Light_Timer.DN bit.
- The input to the Amber_Light_Timer is controlled by the Green_Light_Timer.DN bit.
- The timed sequence of the lights is:
  - Red—30 s on
  - Green—25 s on
  - Amber—5 s on
- The sequence then repeats itself.

Off-Delay Timer (TOF)

The ***off-delay timer (TOF)*** operates in a fashion opposite to the TON on-delay timer. An off-delay timer will turn on immediately when the rung of ladder logic is true,

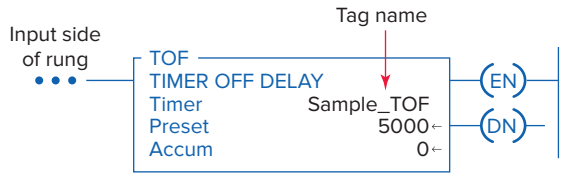


Figure 15-57 ControlLogix TOF off-delay timer instruction.

but it will delay before turning off after the rung goes false. The ControlLogix TOF off-delay timer instruction is shown in Figure 15-57. The description of the function block fields and tag references are the same as for that of a TON timer.

Figure 15-58 shows a program that uses a TOF timer to illuminate a green pilot light for 20 seconds each time a momentary button is pressed. The program code is simpler than that used to accomplish the same task using a TON timer. The operation of the program can be summarized as follows:

- When the Timer_Button is initially closed the timer rung and instruction and DN bit all become true.
- The DN bit switches on the Green_PL and the program remains in this state as long as the button is held closed.
- When the button is released the Timer_Button instruction goes false and starts the timing cycle.
- The light remains on and the timer begins accumulating time.
- When the accumulator reaches 20000 ms (20 s) the timer DN bit becomes false and the light is switched off.

The program of Figure 15-59 uses both on-delay and off-delay timers for control of a heating oven process. The different tags created to fit the program are shown

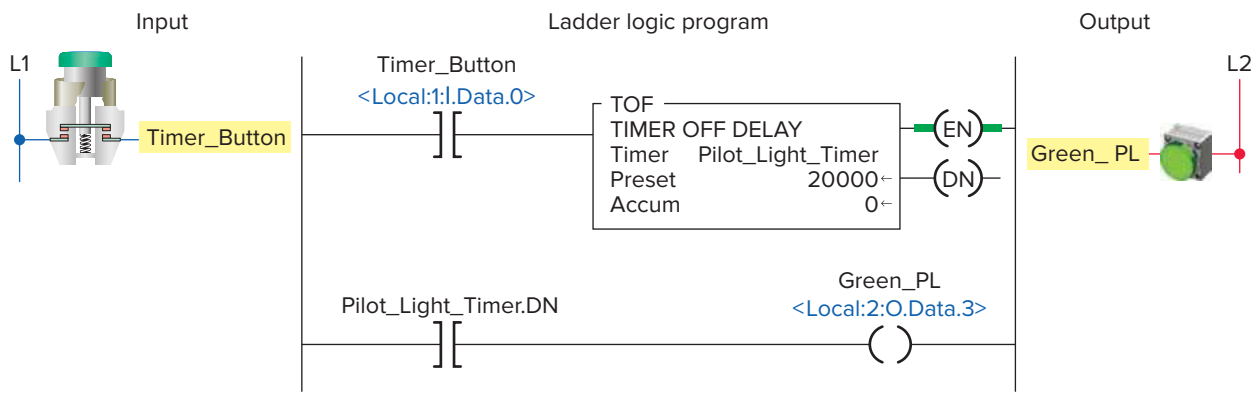


Figure 15-58 Pilot light TOF timer.

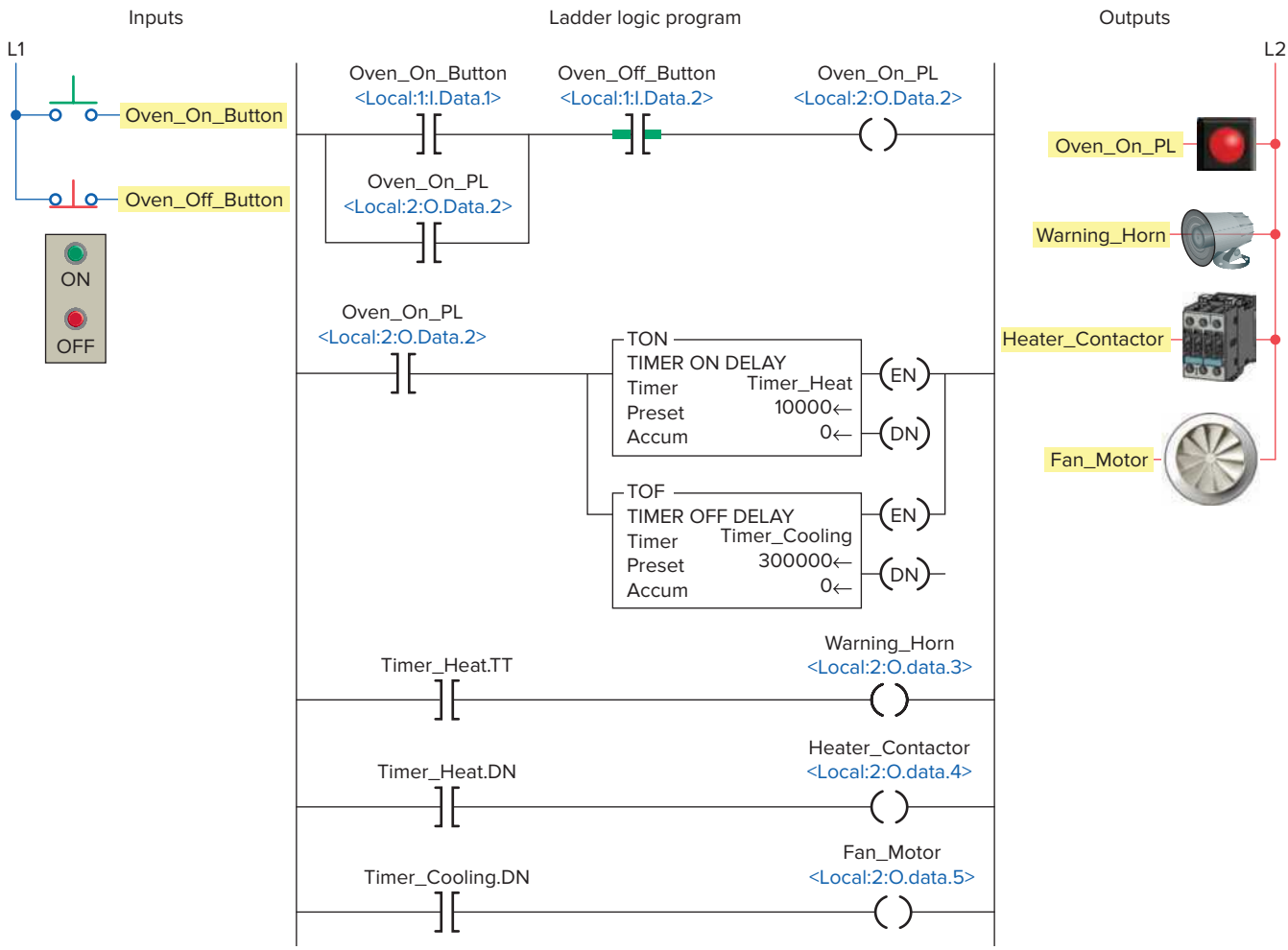


Figure 15-59 Timer control of a heating oven process.

Tag Name	Alias For	Base Tag	Data Type	Style
Warning_Horn	Local:2:O.Data.3	Local:2:O.Data.3	BOOL	Decimal
Heater_Contactor	Local:2:O.Data.4	Local:2:O.Data.4	BOOL	Decimal
Fan_Motor	Local:2:O.Data.5	Local:2:O.Data.5	BOOL	Decimal
Oven_On_PL	Local:2:O.Data.2	Local:2:O.Data.2	BOOL	Decimal
Oven_On_Button	Local:1:I.Data.1	Local:1:I.Data.1	BOOL	Decimal
Oven_Off_Button	Local:1:I.Data.2	Local:1:I.Data.2	BOOL	Decimal
+-Timer_Heat			TIMER	
+-Timer_Cooling			TIMER	

Figure 15-60 Tags created for heating oven process.

in Figure 15-60. Operation of the program can be summarized as follows:

- Pressing the Oven_On_Button energizes the Oven_On_PL output which seals itself in and enables the TON and TOF timer instructions.
- The Timer_Heat.TT bit of the TON timer becomes true which sounds the Warning_Horn to warn that the oven is about to come on.
- The Timer_Cooling.DN bit of the TOF timer becomes true which energizes the Fan_Motor.
- After 10 s (10000 ms) have elapsed the Timer_Heat.TT bit becomes false to turn off the Warning_Horn and the Timer_Heat.DN bit becomes true to energize the Heater_Contactor and turn on the heating coils.
- When the Oven_Off_Button is momentarily actuated the Oven_On_PL output goes false which turns the pilot light off and opens the continuity of its seal-in logic path.
- The Timer_Heat timer instruction and its DN bit instruction become false which de-energizes the Heater_Contactor and turns off the heating coils.
- The Timer_Cooling timer begins accumulating time and the fan continues to operate for the 5 minute (300000 ms) delay period after which the Timer_Cooling.DN bit becomes false to turn the fan off.

Retentive Timer On (RTO)

A ***retentive on-delay timer (RTO)*** operates the same as a TON timer, except that the retentive timer retains (remembers) its ACC value even if:

- The rung goes false.
- The processor is placed in the program mode.
- The processor faults.
- Power to the processor is temporarily interrupted and the processor battery is functioning properly.

The ControlLogix RTO retentive on-delay timer instruction is shown in Figure 15-61. The description of the function block fields and tag references are the same as for that of a TON timer; however, a RES reset instruction must be used to reset the accumulated value of a retentive timer. The RES instruction must have the same tag name as the timer you want to reset.

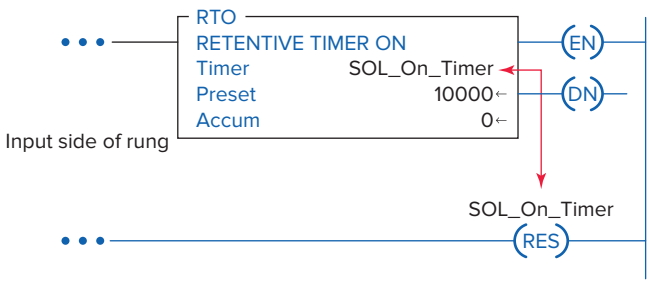


Figure 15-61 RTO retentive on-delay timer instruction.

An example application of a limit switch 2 minute (120000 ms) RTO timer program is shown in Figure 15-62. The different tags created to fit the program are shown in Figure 15-63. The operation of the program can be summarized as follows:

- The status and value of all instructions, with the timer initially reset, are as shown in the monitor tags window.
- When the Limit_Switch has been closed for 1 minute, the status and value of the instructions would be:
  - PRE – 120000
  - ACC – 60000
  - LS_Timer.EN – 1
  - LS_Timer.TT – 1
  - LS_Timer.DN – 0
  - LS_EN_PL – 1
  - LS_TT_PL – 1
  - LS_Alarm – 0
- When the Limit_Switch is opened after 1.5 minutes, the status and value of the instructions would be:
  - PRE – 120000
  - ACC – 90000
  - LS_Timer.EN – 0
  - LS_Timer.TT – 0
  - LS_Timer.DN – 0
  - LS_EN_PL – 0
  - LS_TT_PL – 0
  - LS_Alarm – 0

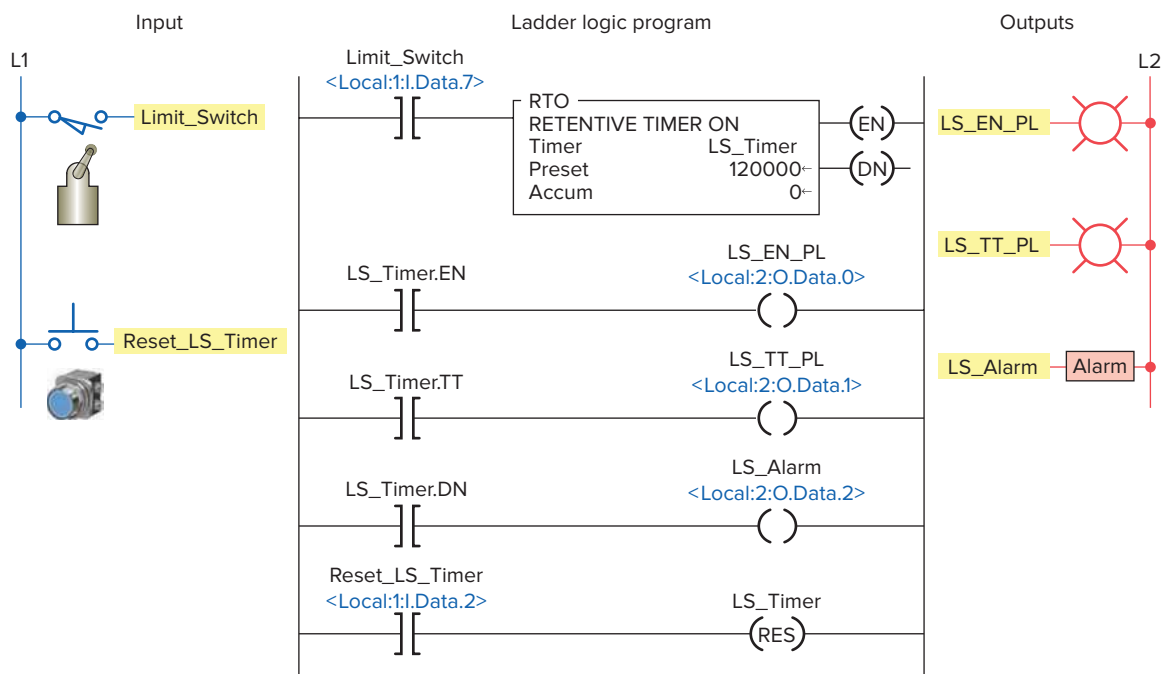


Figure 15-62 Limit switch RTO timer program.

Tag Name	Value	Style	Data Type
-LS_Timer	{...}		TIMER
-LS_Timer.PRE	120000	Decimal	DINT
-LS_Timer.ACC	0	Decimal	DINT
LS_Timer.EN	0	Decimal	BOOL
LS_Timer.TT	0	Decimal	BOOL
LS_Timer.DN	0	Decimal	BOOL
Limit_Switch	0	Decimal	BOOL
LS_EN_PL	0	Decimal	BOOL
LS_TT_PL	0	Decimal	BOOL
LS_Alarm	0	Decimal	BOOL

Figure 15-63 Tags created for the RTO retentive on-delay timer program.

- When the Limit_Switch is closed and stays closed until the timer times out, the status and value of the instructions would be:
  - PRE – 120000
  - ACC –120000
  - LS_Timer.EN – 1
  - LS_Timer.TT – 0
  - LS_Timer.DN – 1
  - LS_EN_PL – 1
  - LS_TT_PL – 0
  - LS_Alarm – 1
- When the Limit_Switch is opened after the timer times out, the status and value of the instructions would be:
  - PRE – 120000
  - ACC –120000
  - LS_Timer.EN – 0
  - LS_Timer.TT – 0
  - LS_Timer.DN – 1
  - LS_EN_PL – 0
  - LS_TT_PL – 0
  - LS_Alarm – 1
- When the Reset_LS_Timer is closed, the status and value of the instructions are reset to their original values.

Cascading of Timers

Timers can be linked together or **cascaded** to extend their control capability. The industrial control program of Figure 15-64 is an example of cascading TON timers for timed event-driven routines. In this program the timers are cascaded in such a manner that one event leads to another. The operation of the program can be summarized as follows:

- The process consists of three distinct timed events or steps.
- Each step must last for a predetermined period of time and must be performed in a specific order.
- The Start_PB, associated internal relay seal-in circuit, is activated to start the sequence.
- Event_1 has a 3 second duration, after which Event_2 begins.
- Event_2 has a 10 second duration, after which Event _3 begins.
- Event_3 has a 5 second duration, after which the entire process is completed.
- The output pilot lights turn on to signify the completion of each step in the sequence.
- Once the final step has been completed, the Process_Complete_PL is turned on.
- Activating the Stop_PB at any time will reset the process.

Reciprocating timers are timing functions where the output of one timer is used to reset the input of a second timer, each resetting the other. These types of timers are used in situations where a constant cycling of an output is required. For example, if a flashing light is required in the event of a control system failure, a program with reciprocating timers could be used to create the flashing output function, as illustrated in the PAC program of Figure 15-65. The operation of the program can be summarized as follows:

- When the contacts of the Limit_Switch close, Timer_Reciprocationg_Off beings its timing cycle.
- After two seconds, the Timer_Reciprocationg_Off. DN bit status changes to true and both the Timer_ Reciprocating_On instruction and the flashing Alarm_PL become energized.
- After two more seconds, the Timer_Reciprocating_ On.DN bit status changes to reset the sequence.

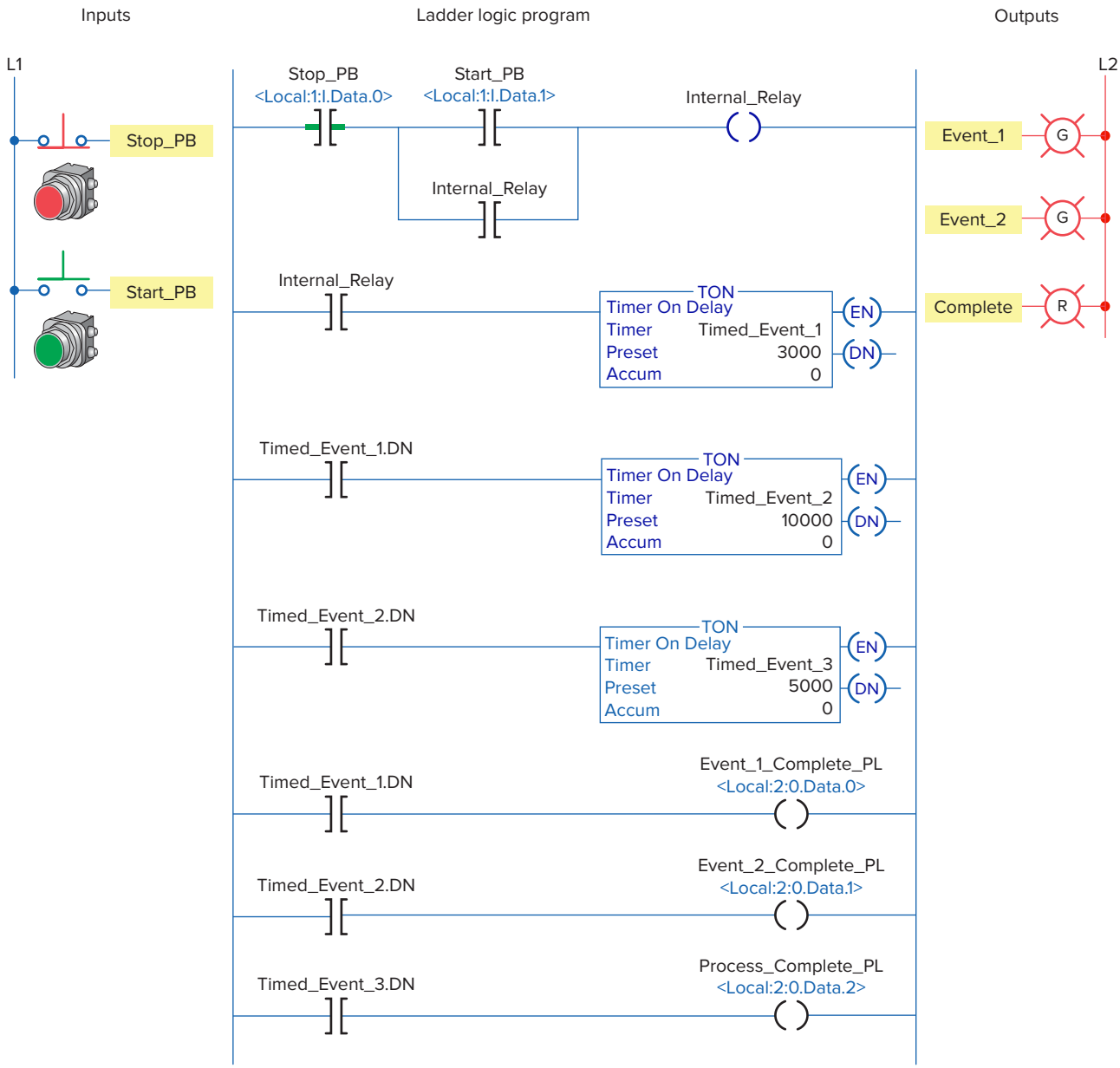


Figure 15-64 Cascading TON timers for timed event-driven routines.

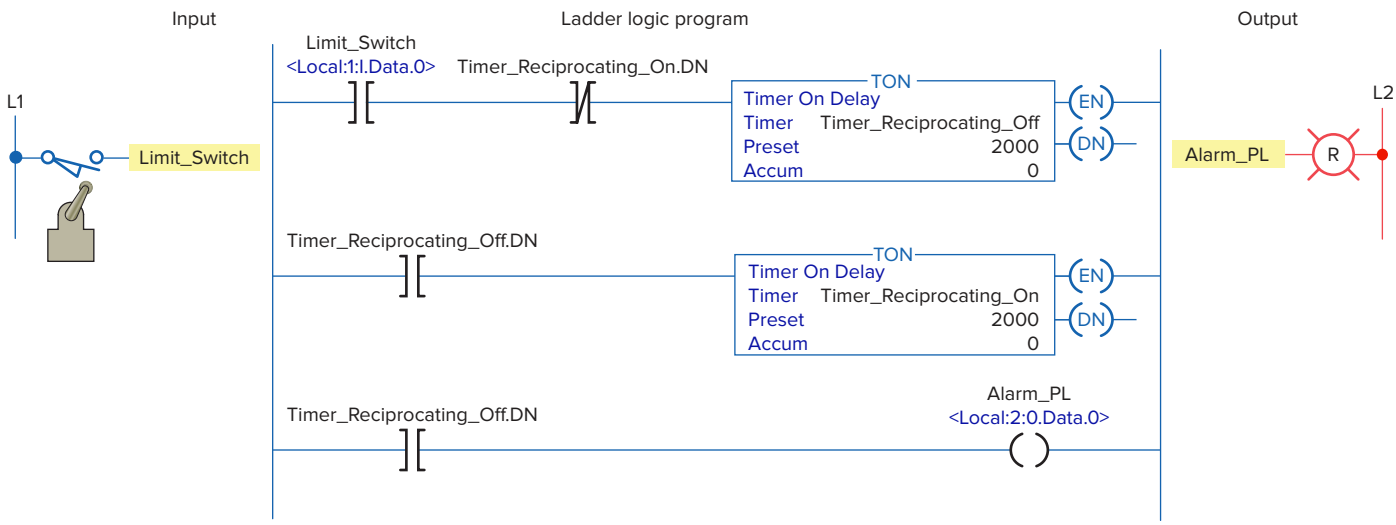


Figure 15-65 Flashing pilot light program.

PART 3 REVIEW QUESTIONS

1. Compare the methods used to address timers in an SLC 500 and a ControlLogix controller.
2. List the five different members of a TIMER structure.
3. What type of timing application may require you to use a TON on-delay timer?
4. What PRE value is used for a timer?
5. To what value is the accumulated value of a timer normally set?
6. What timer status bit is set to 1 when the TON timer times out?
7. The TON instruction is self-resetting. Explain what this means.
8. What number would be entered into the PRE value of a ControlLogix timer for a timing period of 4.5 minutes?
9. Compare the operation a TOF and a TON timer.
10. When does the rung of a TOF timer begin accumulating time?
11. The RTO timer is a retentive timer. Explain what this means.
12. How are the retentive timer and reset instruction related?

PART 3 PROBLEMS

1. Modify the original CLX ten-second TON timer program (Figure 15-52) with an additional rung added to the program that will energize a solenoid whenever the timer is enabled and timing. The solenoid is to be connected to pin 6 of the digital output module.
2. With reference to the ladder logic of the CLX diverter gate program (Figure 15-53), assume the solenoid gate fails to energize as programmed. You suspect the problem is due to an open in the solenoid coil or wiring to it. How might observation of the solenoid output status light help confirm this?
3. You are required to extend the Green light-on time of the CLX traffic control program (Figure 15-55) to 40 seconds. What changes would have to be made to the program?
4. With reference to the CLX heating oven process program (Figure 15-59), assume the oven-on pilot light burns out. In what way would the operation of the program be affected?
5. With reference to the CLX limit switch RTO program (Figure 15-62), in addition to the alarm, you are required to install a warning pilot light to indicate that the timer has timed out. How would you proceed?
6. Implement the hardwired TON alarm circuit of Figure 15-66 in Logix format.

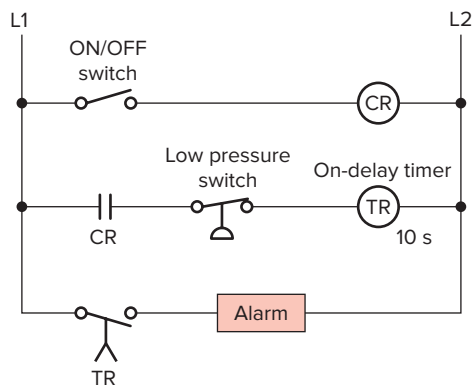


Figure 15-66 Hardwired TON alarm circuit for Problem 6.

Part 4 Programming Counters

Part Objectives

After completing this part, you will be able to:

- Understand ControlLogix counter tags and their members
- Utilize status bits from counters in logic
- Develop ladder logic programs using ControlLogix counters

Counters

Counters are similar to timers, except that a counter accumulates (counts) the changes in state of an external trigger signal whereas timers increment using an internal clock. PLC counters are generally triggered by a change in an input field device that causes a false-to-true transition of the counter ladder rung. It does not matter how long the rung stays true or false—it is only the transition that counts.

There are two basic counter types: count-up (CTU) and count-down (CTD). The ControlLogix CTU instruction and counter selection toolbar are shown in Figure 15-67. When you want to use a counter, you must create a **tag of type COUNTER** (it is a predefined data type) and enter the preset and the accumulated value. When entering the instruction, this tag must be defined before the preset and accumulated values can be entered. A RES reset instruction that has the same tag name as the counter must be used to reset the accumulated value of the counter to zero.

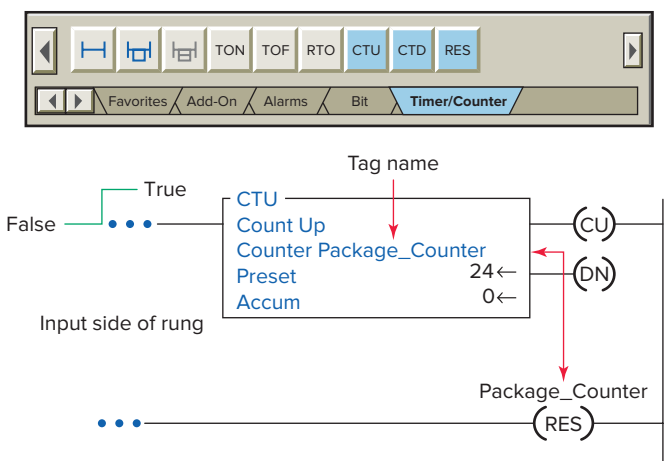


Figure 15-67 CTU count-up counter instruction.

All counters are retentive in that the accumulated value of any counter is retained, even during a power failure, until reset. The on/off status of the counter done, overflow, and underflow bits are retentive as well. ControlLogix counter parameters and status bits are shown in the edit tags window of Figure 15-68 and can be summarized as follows:

- **Preset (PRE) Value**—Specifies the value the counter must reach before the done (DN) bit turns on (1).
- **Accumulated (ACC) Value**—Is the number of false-to-true transitions of the counter rung. ACC is reset to zero when a reset (RES) instruction (of the same counter address) is executed.
- **CU (Count-Up Enable Bit)**—The count-up enable bit indicates the CTU instruction is enabled.

Tag Name	Data Type	Style
-Part_Counter	COUNTER	Decimal
-Part_Counter.PRE	DINT	Decimal
-Part_Counter.ACC	DINT	Decimal
Part_Counter.CU	BOOL	Decimal
Part_Counter.CD	BOOL	Decimal
Part_Counter.DN	BOOL	Decimal
Part_Counter.OV	BOOL	Decimal
Part_Counter.UN	BOOL	Decimal

Figure 15-68 ControlLogix counter parameters and status bits.

- **CD (Count-Down Enable Bit)**—The count-down enable bit indicates the CTD instruction is enabled.
- **DN (Count-Up Done Bit)**—The DN bit is set (1) when ACC value is equal to or greater than the PRE value and is reset by the RES instruction.
- **OV (Overflow Bit)**—The overflow bit indicates the counter exceeded the upper limit. The OV bit is set when the ACC value is greater than +2,147,483,647 and reset when the reset instruction is executed. Note that the accumulated value keeps incrementing even after the ACC value equals the PRE value.
- **UN (Underflow Bit)**—Indicates that the counter exceeded the lower limit of −2,147,483,648.

The counter tag name is declared using the new tag properties dialog box shown in Figure 15-69. Tag name, description (optional), tag type, data type (base type is used most often), and scope are selected or typed to complete the validation.

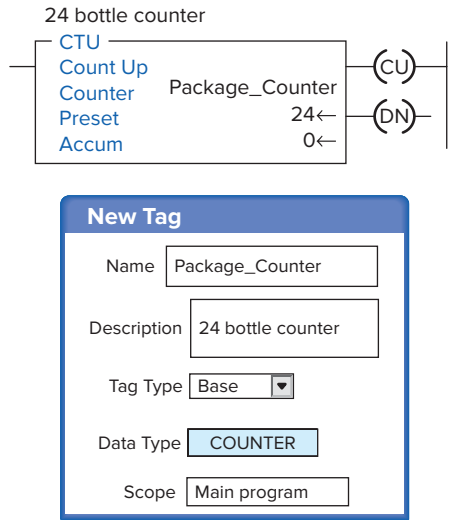


Figure 15-69 Counter tag validation.

Count-Up (CTU) Counter

Count-up (CTU) counters will cause the accumulated count to increase by 1 every time there is a false-to-true transition of the counter ladder rung. An example application of a count-up counter program used to count packets of bottles is shown in Figure 15-70. The operation of the program can be summarized as follows:

- Each open-to-close transition of the Bottle_Sensor proximity switch causes the counter to increment by 1.

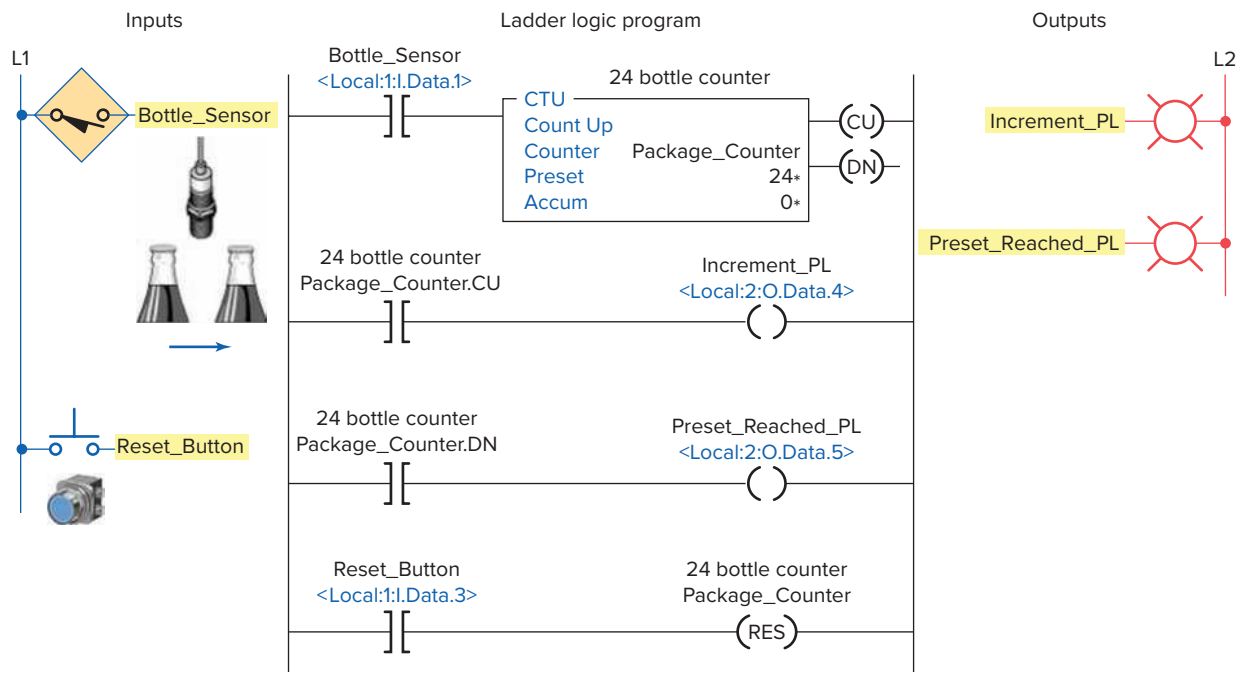


Figure 15-70 Count-up counter program used to count packets of bottles.

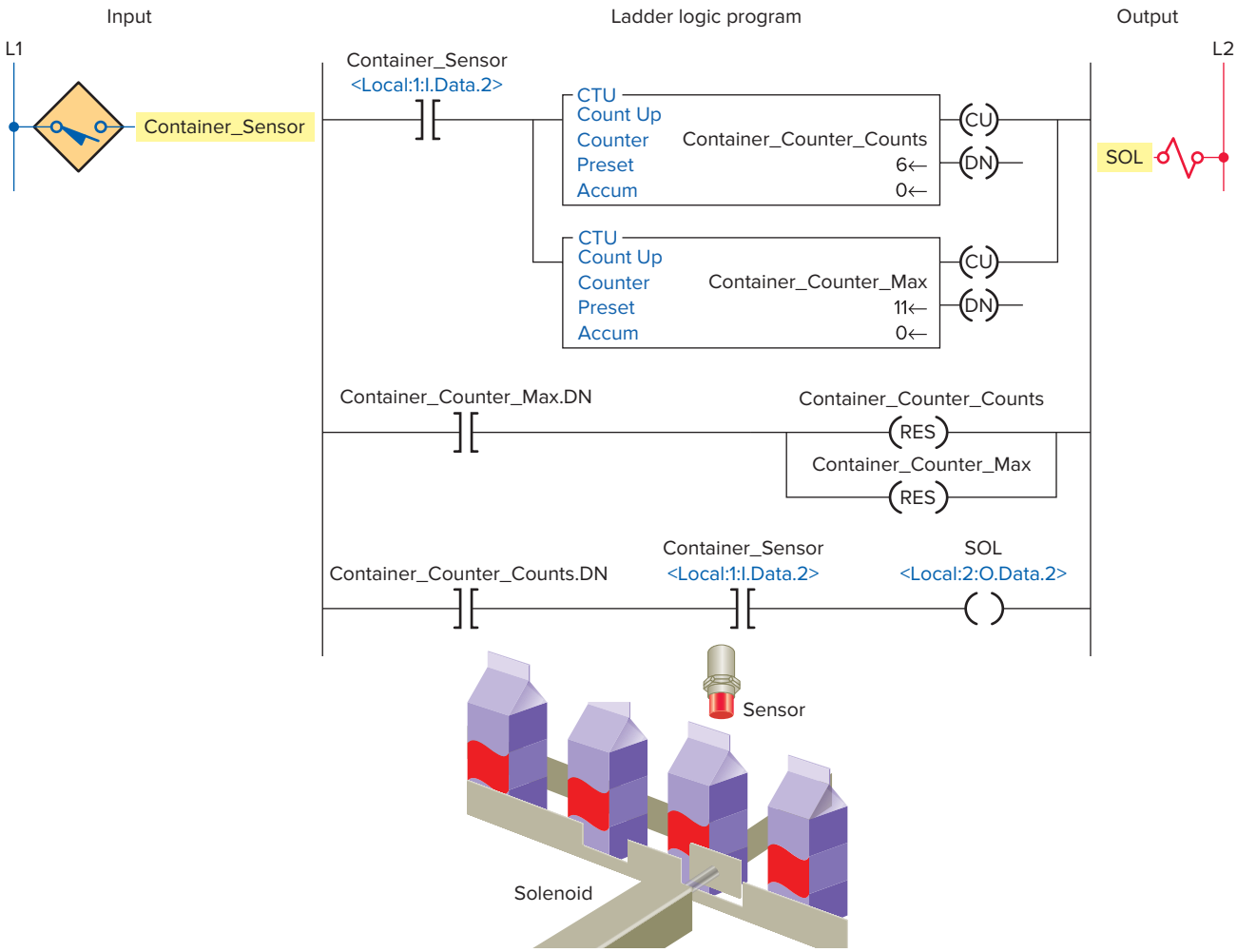


Figure 15-71 CTU program used to remove containers from a conveyor line.

- The Increment_PL controlled by the Package_ Counter.CU status bit turns on and off as each bottle passes to show that the counter is incrementing.
- When the accumulated value of the counter is 24 the DN bit of the counter is set and switches on the Preset_Reached_PL.
- The counter is reset by momentarily closing the Reset_Button.

The program shown in Figure 15-71 uses two CTU instructions as part of a program to remove 5 out of every 10 containers from a conveyor line using an electric solenoid. The different tags created to fit the program are shown in Figure 15-72. The operation of the program can be summarized as follows:

- The preset for the Container_Counter_Counts is set for 6 and that for the Container_Counter_Max is set to 11.
- When the container is detected both counters will increase their accumulated values by 1.

Tag Name	Value	Style	Data Type
Container_Counter_Counts	{...}		COUNTER
Container_Counter_Counts .PRE	6	Decimal	DINT
Container_Counter_Counts .ACC	0	Decimal	DINT
Container_Counter_Counts .CU	0	Decimal	BOOL
Container_Counter_Counts .CD	0	Decimal	BOOL
Container_Counter_Counts .DN	0	Decimal	BOOL
Container_Counter_Counts .OV	0	Decimal	BOOL
Container_Counter_Counts .UN	0	Decimal	BOOL
Container_Counter_Max	{...}		COUNTER
Container_Counter_Max .PRE	11	Decimal	DINT
Container_Counter_Max .ACC	0	Decimal	DINT
Container_Counter_Max .CU	0	Decimal	BOOL
Container_Counter_Max .CD	0	Decimal	BOOL
Container_Counter_Max .DN	0	Decimal	BOOL
Container_Counter_Max .OV	0	Decimal	BOOL
Container_Counter_Max .UN	0	Decimal	BOOL
Container_Sensor	0	Decimal	BOOL
SOL	0	Decimal	BOOL

Figure 15-72 Tags created for the CTU program used to remove containers from a conveyor line.

- When the sixth part arrives the Container_Counter_Counts counter will then be done, thereby allowing the solenoid to actuate for any container after the fifth.
- The Container_Counter_Max counter will continue until the eleventh part is detected and then both of the counters will be reset.

Count-Down (CTD) Counter

The ***count-down (CTD) counter*** operates in a fashion opposite to the count-up CTU counter. CTD counters will cause the accumulated count to decrease instead of increase by one every time there is a false-to-true transition of the counter ladder rung. The ControlLogix CTD down-counter instruction is shown in Figure 15-73. The descriptions of the function block fields and the tag references are the same as those associated with the CTU function block. The CTD instruction is typically used with a CTU instruction that references the same counter structure.

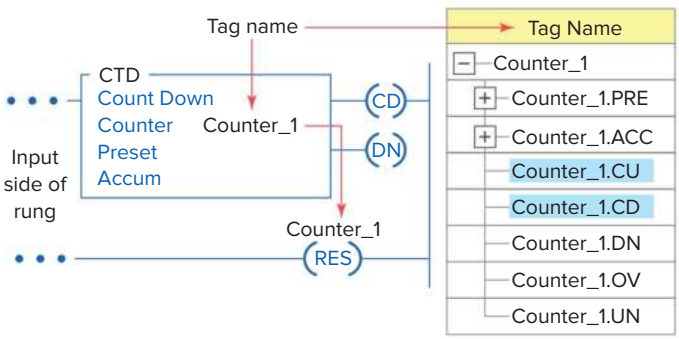


Figure 15-73 Count-down CTD counter instruction.

The application program shown in Figure 15-74 is used to limit the number of parts that can be stored in the buffer zone to a maximum of 50. A CTU counter and a CTD counter are used together *with the same address* to form an Up/Down counter. This is the most common type of application of the CTD counter. The different tags created to fit the program are shown in Figure 15-75. The operation of the program can be summarized as follows:

- The Restart_Button is momentarily actuated at any time to reset the accumulated value of the counter to zero.
- Conveyor brings parts into a buffer zone.
- Each time a part enters the buffer zone, the Enter_Limit_Sw is actuated and Counter_1 increments by 1.
- Each time a part leaves the buffer zone, the Exit_Limit_Sw is actuated and Counter_1 decrements by 1.
- When the number of parts in the buffer zone, at any one time, reaches 50, the Counter_1.DN bit is set.
- As a result the Conveyor_Contactor rung goes false to de-energize the conveyor contactor, automatically stopping the conveyor from bringing in any more parts until the accumulated count drops below 50.

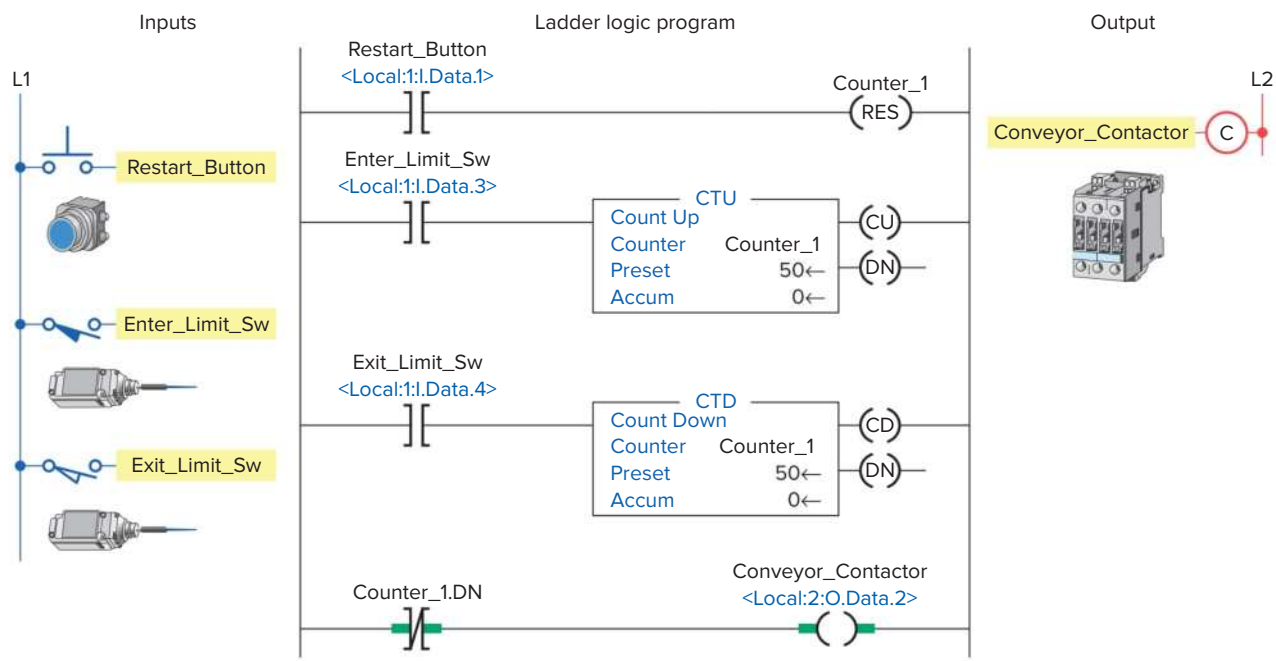


Figure 15-74 CTU counter and CTD counter used together to form an Up/Down counter.

Tag Name	Value	Style	Data Type
Counter_1	{ . . . }		COUNTER
Counter_1.PRE	50	Decimal	DINT
Counter_1.ACC	0	Decimal	DINT
Counter_1.CU	0	Decimal	BOOL
Counter_1.CD	0	Decimal	BOOL
Counter_1.DN	0	Decimal	BOOL
Counter_1.OV	0	Decimal	BOOL
Counter_1.UN	0	Decimal	BOOL
Restart_Button	0	Decimal	BOOL
Enter_Limit_Sw	0	Decimal	BOOL
Exit_Limit_Sw	0	Decimal	BOOL
Conveyor_Contactor	1	Decimal	BOOL

Figure 15-75 Tags created for the Up/Down counter program.

Combining Counter and Timer Functions

Figure 15-76 shows a bottle flow rate program implemented using a Logix controller. This program is designed to indicate how many bottles pass a given process point per minute. The operation of the program can be summarized as follows:

- When the Start_Switch is closed, both the timer and counter are enabled.
- The Total_Bottles counter is pulsed for each bottle that passes the Bottle_Sensor.
- The counting begins and the 1_Min_Timer starts timing through its one minute (60000 milliseconds) time interval.
- At the end of one minute, the timer done bit (DN) causes the counter rung to go false.
- Sensor pulses continue but do not affect the Total_Bottles counter.
- The number of bottles for the past minute is represented by the accumulated value of the Total_Bottles counter.
- The sequence is reset by momentarily opening and closing the Start_Switch.

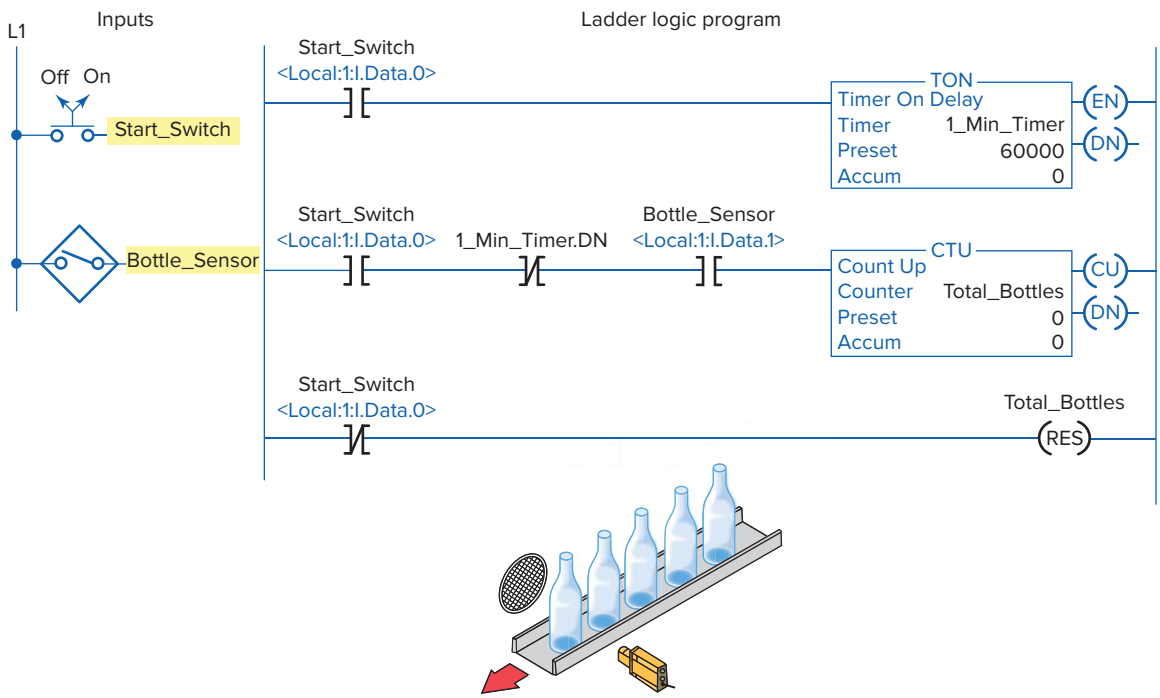


Figure 15-76 Bottle flow rate program.

PART 4 REVIEW QUESTIONS

1. In what way are timers and counters similar?
2. Outline the procedure followed to create a tag when you want to use a counter.
3. All counters are retentive. In what way does this affect their operation?
4. What is specified by the preset value of a counter?
5. When is each of the following counter bits set?
   a. CU
   b. DN
   c. CD
6. Compare the operations of a CTU and a CTD counter.
7. What is an Up/Down counter?
8. Explain how you go about creating tags for an Up/Down counter that uses a CTU and CTD instruction.

PART 4 PROBLEMS

1. With reference to the CTU packets of bottles program (Figure 15-70), what changes to the program would be required to count six bottle packets?
2. With reference to the CTU program used to remove containers from a conveyor line (Figure 15-71), assume the output solenoid coil failed open. In what way would the operation of the program be affected?
3. Modify the original Up/Down counter program (Figure 15-74) to include:
   a. A red pilot light to indicate entry of a part into the buffer zone. Light to be connected to pin 4 of the digital output module.
   b. A green pilot light to indicate exit of a part from the buffer zone. Light to be connected to pin 3 of the digital output module.
4. Write a ControlLogix program, complete with tags, for an Up/Down counter used to keep track of cars entering and exiting a parking lot. The program requirements for this application can be summarized as follows:
   - The parking lot holds 30 vehicles.
   - There is an entrance vehicle sensor and an exit vehicle sensor.
   - When the parking lot is full a Lot Full sign is illuminated.
   - Whenever a car exits the lot, a Caution Buzzer/Light is activated to warn pedestrians.

Part 5 Math, Comparison, and Move Instructions

Part Objectives

After completing this part, you will be able to:

- Utilize ControlLogix math instructions in programs
- Utilize ControlLogix comparison instructions in programs
- Utilize ControlLogix move instructions in programs
- Develop and follow the operation of programs that use math, comparison, and move instructions

Math Instructions

ControlLogix basic math instructions include addition, subtraction, multiplication, division, and square root. Figure 15-77 shows the Compute/Math toolbar for the ControlLogix controller.

The ***ADD* instruction** is used to add two numbers. This instruction adds these values from Source A and Source B. The source can be a constant value or a tag. The result of the ADD instruction is put in the destination (Dest) tag.

Figure 15-78 shows an example of an ADD instruction rung along with its Monitor Tags window. The operation of the rung can be summarized as follows:

- When the Add_Sw is closed the rung will be true.
- The ADD instruction will execute to add the number from Source A (Value_A) and the value from Source B (Value_B).
- The result will be stored in the Dest tag (Total_Value).
- In this example, the 25 was added to 50 and the result (75) was stored in Total_Value.

The ***SUB* instruction** is used to subtract two numbers. Figure 15-79 shows an example of a SUB instruction rung

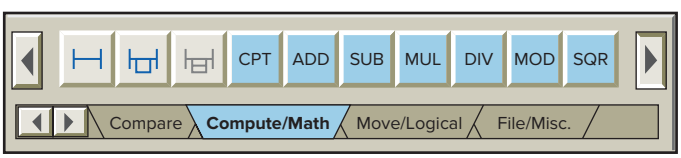
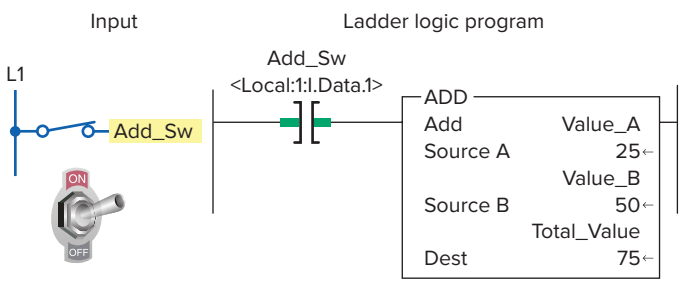


Figure 15-77 Compute/Math toolbar for the ControlLogix controller.



Tag Name	Value	Style	Data Type
Total_Value	75	Decimal	DINT
Value_A	25	Decimal	DINT
Value_B	50	Decimal	DINT
Add_Sw	1	Decimal	BOOL

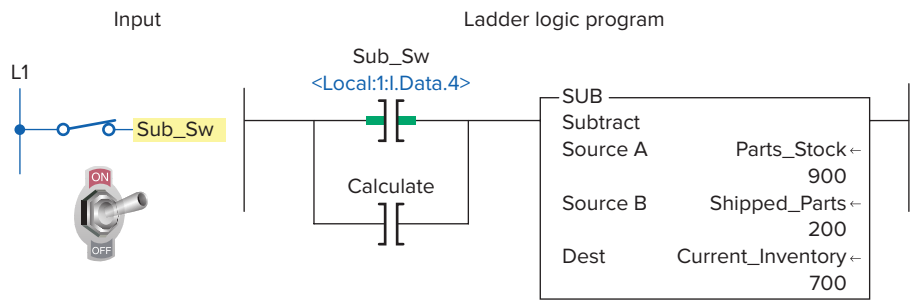
Figure 15-78 ADD instruction rung and its Monitor Tags window.

along with its Monitor Tags window. The operation of the rung can be summarized as follows:

- When the SUB_Sw or Calculate tag is true the SUB instruction is executed.
- Source B (Shipped_Parts) is subtracted from Source A (Parts_Stock) and the result is stored in the Dest tag named Current_Inventory.
- In this example, the 200 was subtracted from 900 and the result (700) was stored in Current_Inventory.
- Source A and Source B can be constants (numbers) or tags.

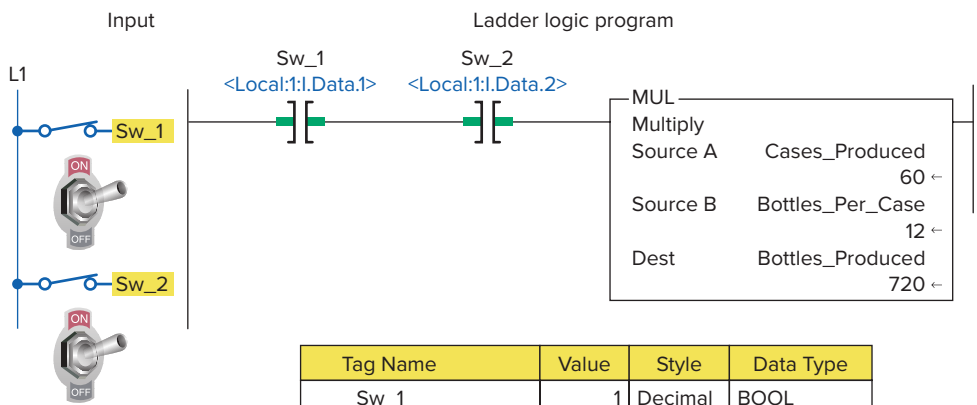
The ***MUL* instruction** is used to multiply two numbers. Figure 15-80 shows an example of a MUL instruction rung along with its Monitor Tags window. When multiple bottles are packed in cases, the number of bottles per case, the number of cases, and the multiply instruction will give you the total number of bottles. The operation of the rung can be summarized as follows:

- When the Sw_1 and Sw_2 are both true the MUL instruction is executed.
- Source A (the value in tag Cases_Produced) is multiplied by Source B (the value in tag Bottles_Per_Case) and the result is stored in the Dest tag Bottles_Produced.
- Source A and Source B can be constants (numbers) or tags.



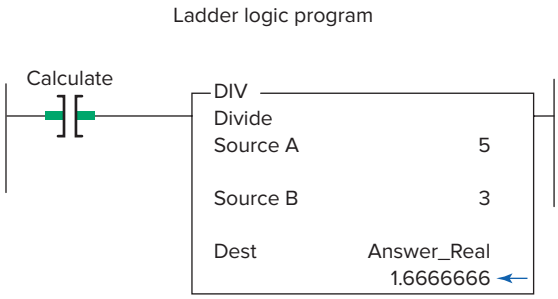
Tag Name	Value	Style	Data Type
Parts_Stock	900	Decimal	DINT
Shipped_Parts	200	Decimal	DINT
Current_Inventory	700	Decimal	DINT
Sub_Sw	1	Decimal	BOOL
Calculate	0	Decimal	BOOL

Figure 15-79 SUB instruction rung and its Monitor Tags window.



Tag Name	Value	Style	Data Type
Sw_1	1	Decimal	BOOL
Sw_2	1	Decimal	BOOL
Cases_Produced	60	Decimal	DINT
Bottles_Per_Case	12	Decimal	DINT
Bottles_Produced	720	Decimal	DINT

Figure 15-80 MUL instruction rung and its Monitor Tags window.



Tag Name	Value	Style	Data Type
Calculate	1	Decimal	BOOL
Answer_Real	1.6666666	Float	REAL

Figure 15-81 DIV instruction rung and its Monitor Tags window.

The ***DIV* instruction** is used to divide two numbers. Figure 15-81 shows an example of a DIV instruction rung along with its Monitor Tags window. The operation of the rung can be summarized as follows:

- A constant (5) is used for Source A and a constant (3) for Source B. Note that tags could have been used for Source A or Source B.
- When the Calculate tag is true the DIV instruction is executed.
- Source A (5) is divided by Source B (3) and the result (1.6666666) is stored in the Dest tag Answer_Real. Note that in this example a Real-type tag has been used for its destination.

The program of Figure 15-82 is used as part of a parts tracking system with three conveyors. The number of parts in conveyor 1 and the number of parts in conveyor 2 are added to get the number of parts on conveyor 3. The operation of the program can be summarized as follows:

- Each time Conveyor_1_Sensor is actuated the accumulated value of Counter_1_Parts is incremented by 1.
- Each time Conveyor_2_Sensor is actuated the accumulated value of Counter_2_Parts is incremented by 1.
- The addition in the ADD instruction places the sum of the accumulated values of the two counters in the Conveyor_3_Parts tag.
- When the accumulated value for either counter is equal to 150 the reset (RES) instructions for both counters are enabled to automatically reset both counter ACC values to zero.
- Both counters can also be reset manually at any time by actuation of the Manual_Conveyor_Reset button.

Comparison Instructions

Compare instructions are used to compare two values. They can be used to see if two values are equal, if one value is greater or less than the other, and so on. In ControlLogix controllers, compare instructions are input instructions that do comparisons by either using an expression or doing the comparison indicated by the specific instruction. Figure 15-83 shows the Compare toolbar for the ControlLogix controller.

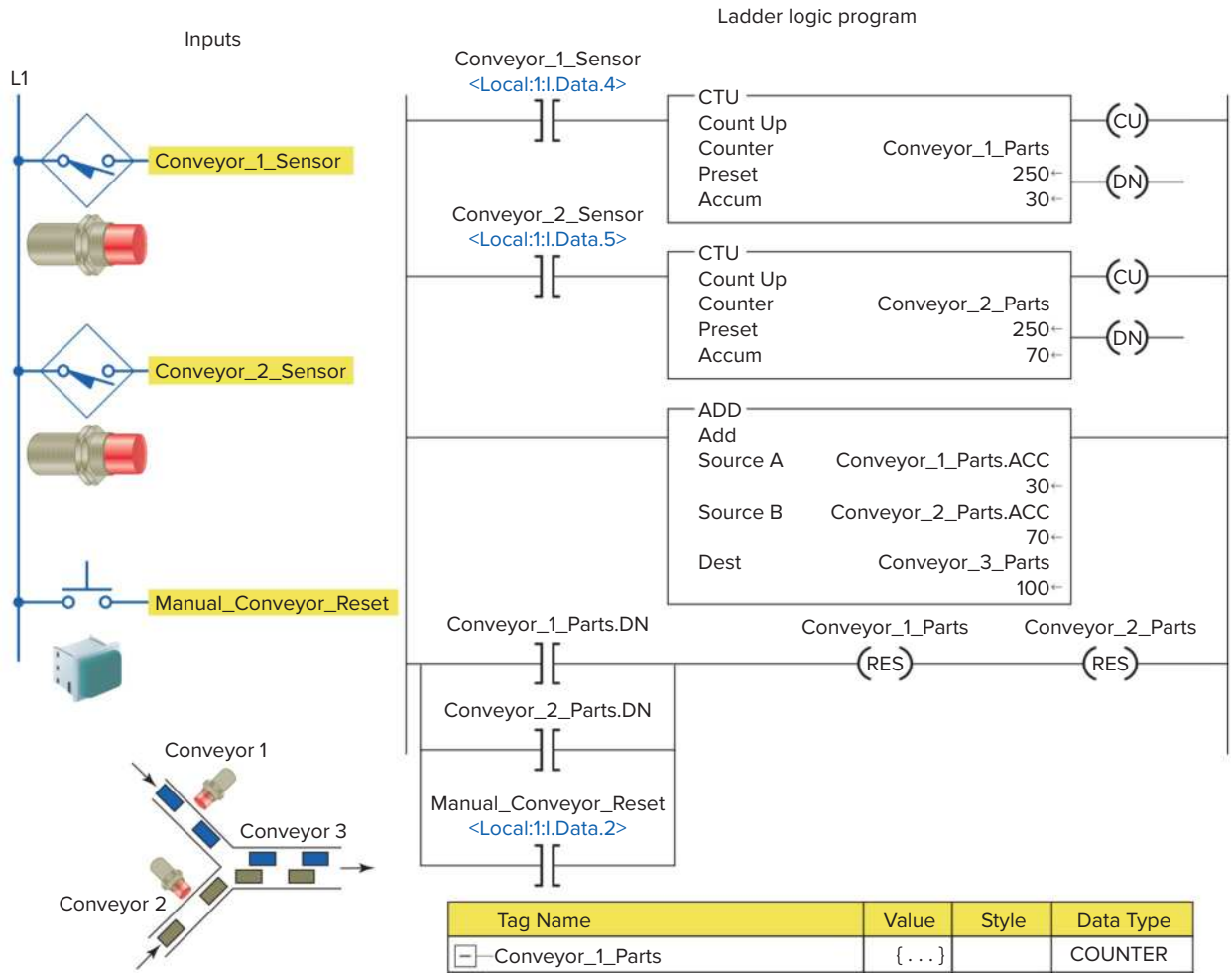
The ***equal (EQU)* instruction** is used to test if two values are equal. Values compared can be actual values or tags that contain values. Figure 15-84 shows an example of an EQU instruction rung along with its Monitor Tags window. The operation of the rung can be summarized as follows:

- The value stored at Source A is compared to the value stored at Source B.
- If the values are equal, the instruction is logically true.
- If the values are unequal, the instruction is logically false.
- In this example Source A (25) is equal to Source B (25) so the instruction is true and output Equal_PL is on.
- Source A and Source B may be SINT, INT, DINT, or REAL data types.

The ***not equal (NEQ)*** instruction is used to test two values for inequality. Figure 15-85 shows an example of an NEQ instruction rung. When Source A is not equal to Source B, the instruction is logically true; otherwise, it is logically false. In this example the two values are not equal so the Not_Equal_PL is energized.

The ***less than (LES)*** instruction is used to check if a value from one source is less than the value from a second source. Figure 15-86 shows an example of an LES instruction rung. When Source A is less than Source B, the instruction is logically true; otherwise, it is logically false. In this example Value_1 (100) is less than Value_2 (300) so the Less_Than_PL is energized.

The ***greater than (GRT)*** instruction is used to check if a value from one source is greater than the value from a second source. Figure 15-87 shows an example of a GRT instruction rung. When Source A is greater than Source B, the instruction is logically true; otherwise,



Tag Name	Value	Style	Data Type
Conveyor_1_Parts	{...}		COUNTER
Conveyor_1_Parts.PRE	250	Decimal	DINT
Conveyor_1_Parts.ACC	30	Decimal	DINT
Conveyor_1_Parts.CU	0	Decimal	BOOL
Conveyor_1_Parts.DN	0	Decimal	BOOL
Conveyor_2_Parts	{...}		COUNTER
Conveyor_2_Parts.PRE	250	Decimal	DINT
Conveyor_2_Parts.ACC	70	Decimal	DINT
Conveyor_2_Parts.CU	0	Decimal	BOOL
Conveyor_2_Parts.DN	0	Decimal	BOOL
Conveyor_1_Sensor	0	Decimal	BOOL
Conveyor_2_Sensor	0	Decimal	BOOL
Conveyor_3_Parts	100	Decimal	DINT
Manual_Conveyor_Reset	0	Decimal	BOOL

Figure 15-82 Program used as part of a parts tracking system.

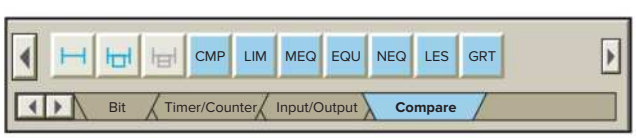
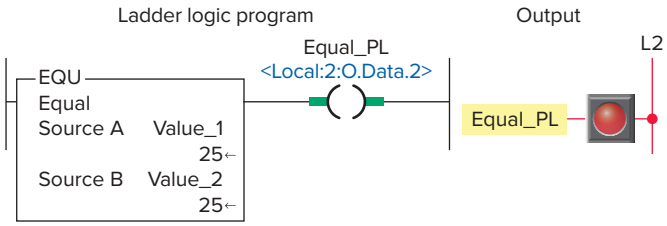


Figure 15-83 Compare toolbar for the ControlLogix controller.

it is logically false. In this example Value_1 (1420) is greater than Value_2 (1200) so the Greater_Than_PL is energized.

The ***compare (CMP)*** instruction performs a comparison on the arithmetic operations specified by the expression. The expression may contain arithmetic operators, comparison operators, and tags. The execution of a CMP instruction



Tag Name	Value	Style	Data Type
Equal_PL	1	Decimal	BOOL
+Value_1	25	Decimal	DINT
+Value_2	25	Decimal	DINT

Figure 15-84 EQU instruction rung and its Monitor Tags window.

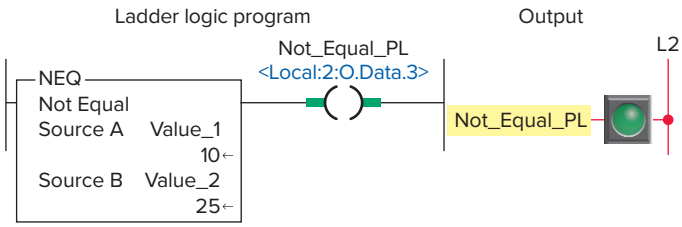


Figure 15-85 NEQ instruction rung.

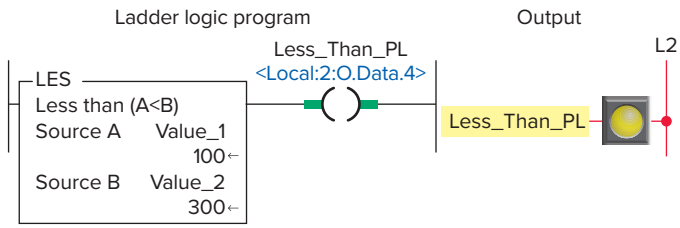


Figure 15-86 LES instruction rung.

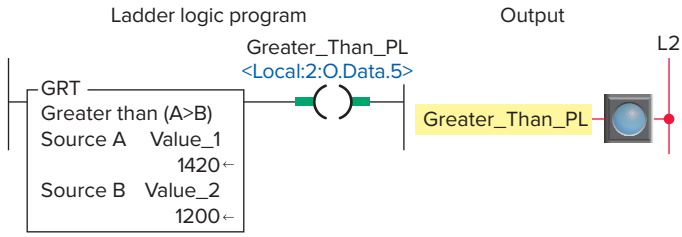
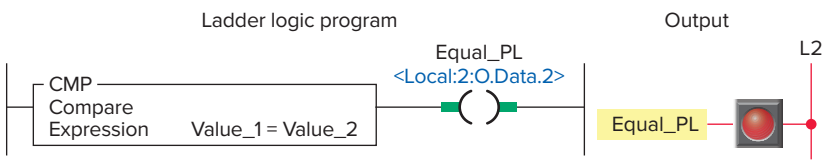


Figure 15-87 GRT instruction rung.

is slightly slower and uses more memory than the execution of the other comparison instructions. The advantage of the CMP instruction is that it allows you to enter complex expressions in one instruction. Figure 15-88 shows an example of a CMP instruction rung. In this example the comparison operator found in the expression is the equivalent of an EQU instruction. The comparison instruction is true because Value_1 (300) is equal to Value_2 (300).

The program of Figure 15-89 is an example of the use of comparison instructions used to test the accumulated value of a counter. The operation of the program can be summarized as follows:

- When the accumulated count is between 5 and 10 the GRT and LES instructions will both be logically true so the PL_1 pilot light will be on.
- When the accumulated count is equal to 15, the EQU instruction will be logically true so the PL_2 pilot light will be on.
- The PL_3 pilot light will be on at all times except when the accumulated count is 20 at which time the NEQ instruction is logically false.
- The counter is reset automatically when the accumulated count reaches 25 or manually anytime the Reset_PB is actuated.



Tag Name	Value	Style	Data Type
Equal_PL	1	Decimal	BOOL
+Value_1	300	Decimal	DINT
+Value_2	300	Decimal	DINT

Figure 15-88 CMP instruction rung.

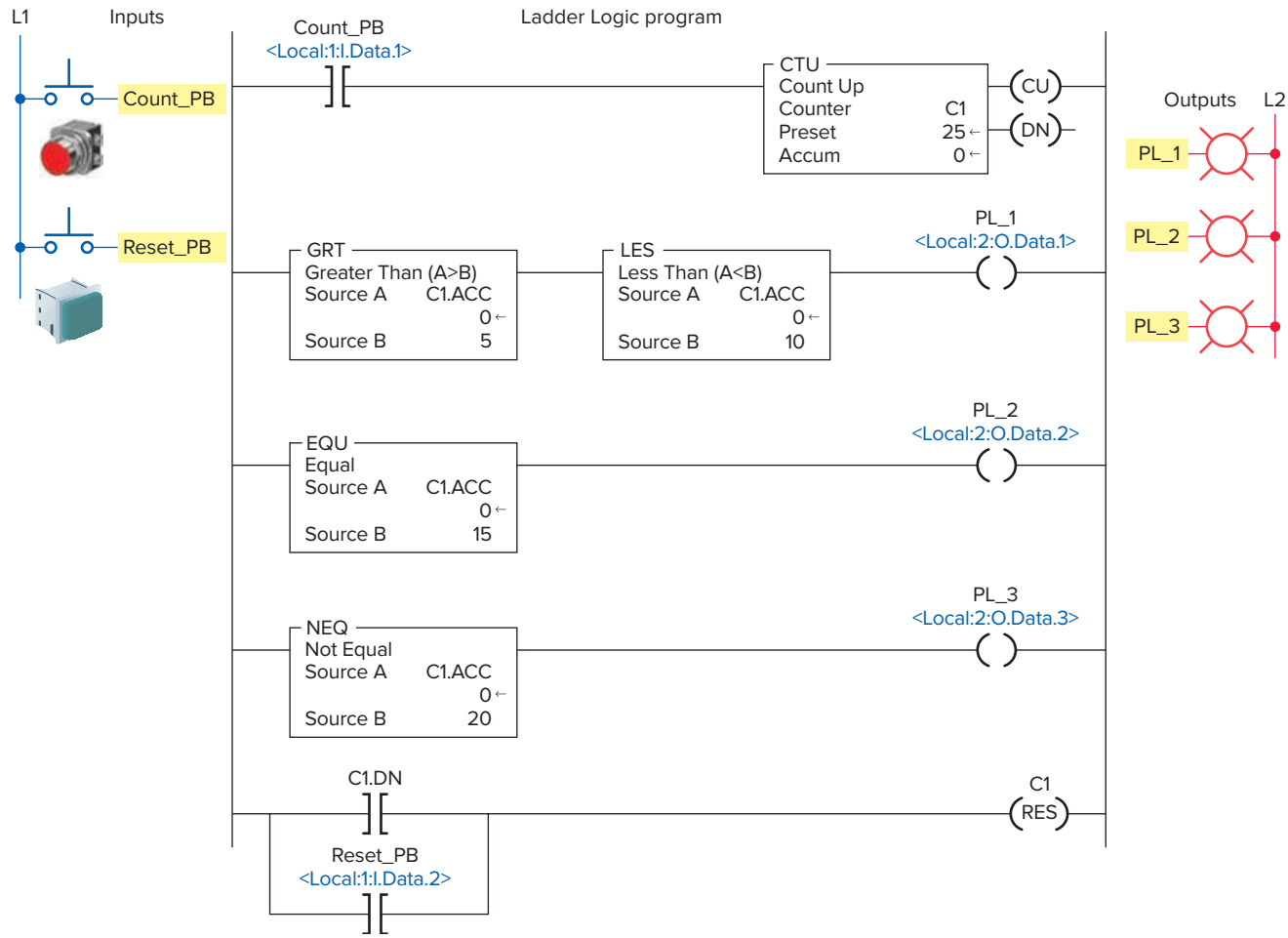


Figure 15-89 Comparison instructions used to test the accumulated value of a counter.

Move Instructions

The ***move (MOV)*** instruction is an output instruction that can move a constant or the contents of one memory location to another location. Figure 15-90 shows the Move toolbar and instruction for the ControlLogix controller. The MOV instruction is used to copy data from a source to a destination. Both the source and the destination data type of a MOV instruction may be INT, DINT, SINT, or REAL.

The program of Figure 15-91 is an example of how the MOV instruction can be used to create a variable preset timer. The operation of the program can be summarized as follows:

- Actuating the PB_10s button executes its MOV instruction to transfer 10000 to the timer preset value setting the delay period for 10 seconds.
- Actuating the PB_15s button executes its MOV instruction to transfer 15000 to the timer preset value setting the delay period for 15 seconds.
- Closing the Timer_Start switch starts the timer timing.
- While the timer is timing, the pilot light PL_1 is on for the duration of the timer preset period.
- When the timer times out, PL_1 turns off and PL_2 turns on.

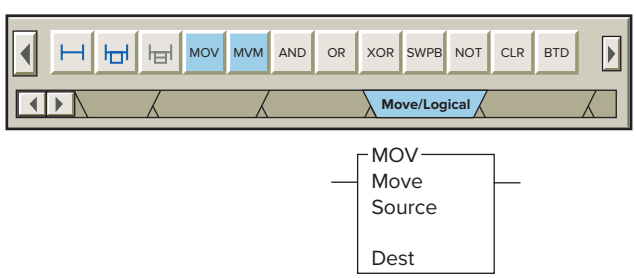


Figure 15-90 Move toolbar for the ControlLogix controller.

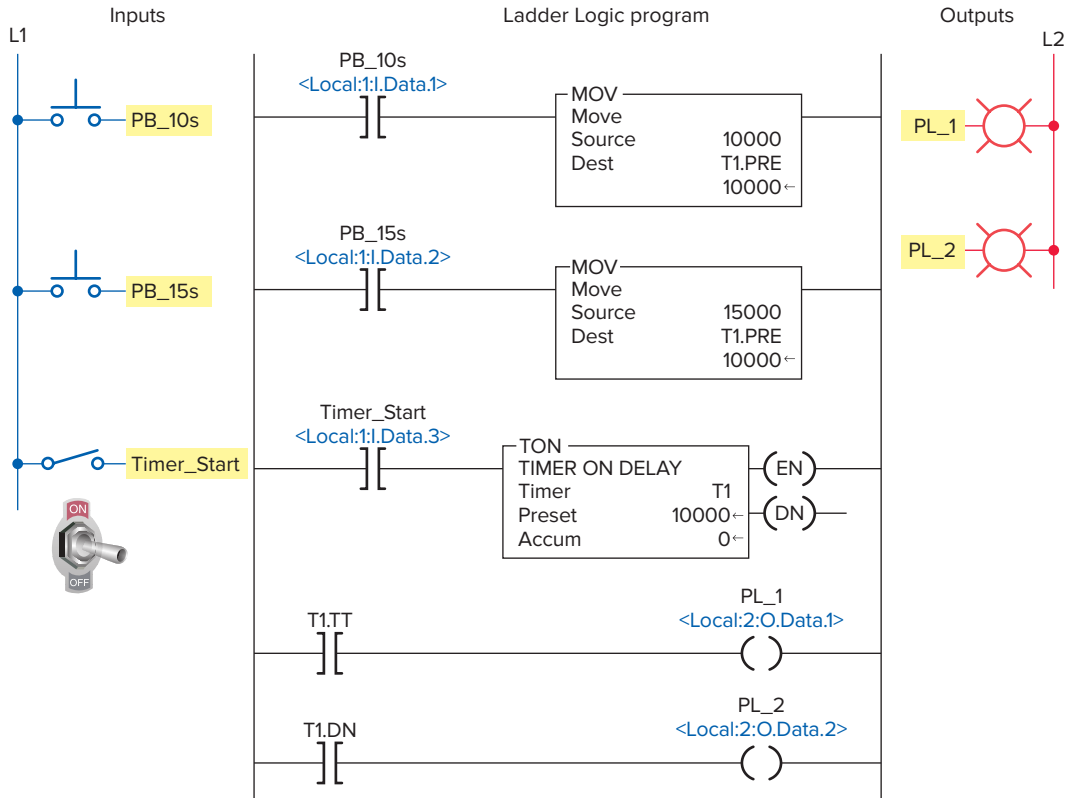


Figure 15-91 MOV instruction used to create a variable preset timer.

Combining Math, Comparison, and Move Instructions

Combining math, comparison, and move instructions provides a PLC with the ability to perform more complex operations. Figure 15-92 shows a PAC Logix program designed so that the BCD_Display displays the setting of the BCD_TWS (thumbwheel switch). Both the MOV and EQU instructions form part of the program. The operation of the program can be summarized as follows:

- The BCD_Display board monitors the decimal setting of the BCD_ TWS.
- The MOV instruction is used to move the data (in the form of BCD) from the BCD_TWS to the BCD_Display.
- The setting of BCD_TWS is compared to the decimal reference number 20 (BCD 100000) stored in Source B of the EQU instruction.
- The EQUAL_ PL output is energized whenever the PL_Equal_Sw is true (closed) and the

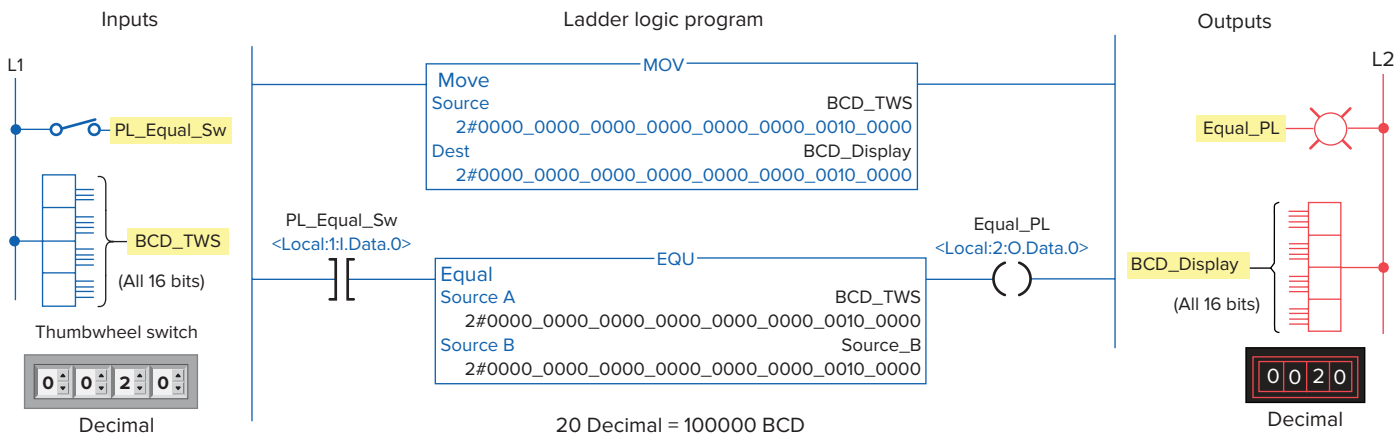


Figure 15-92 Monitoring the setting of a thumbwheel switch.

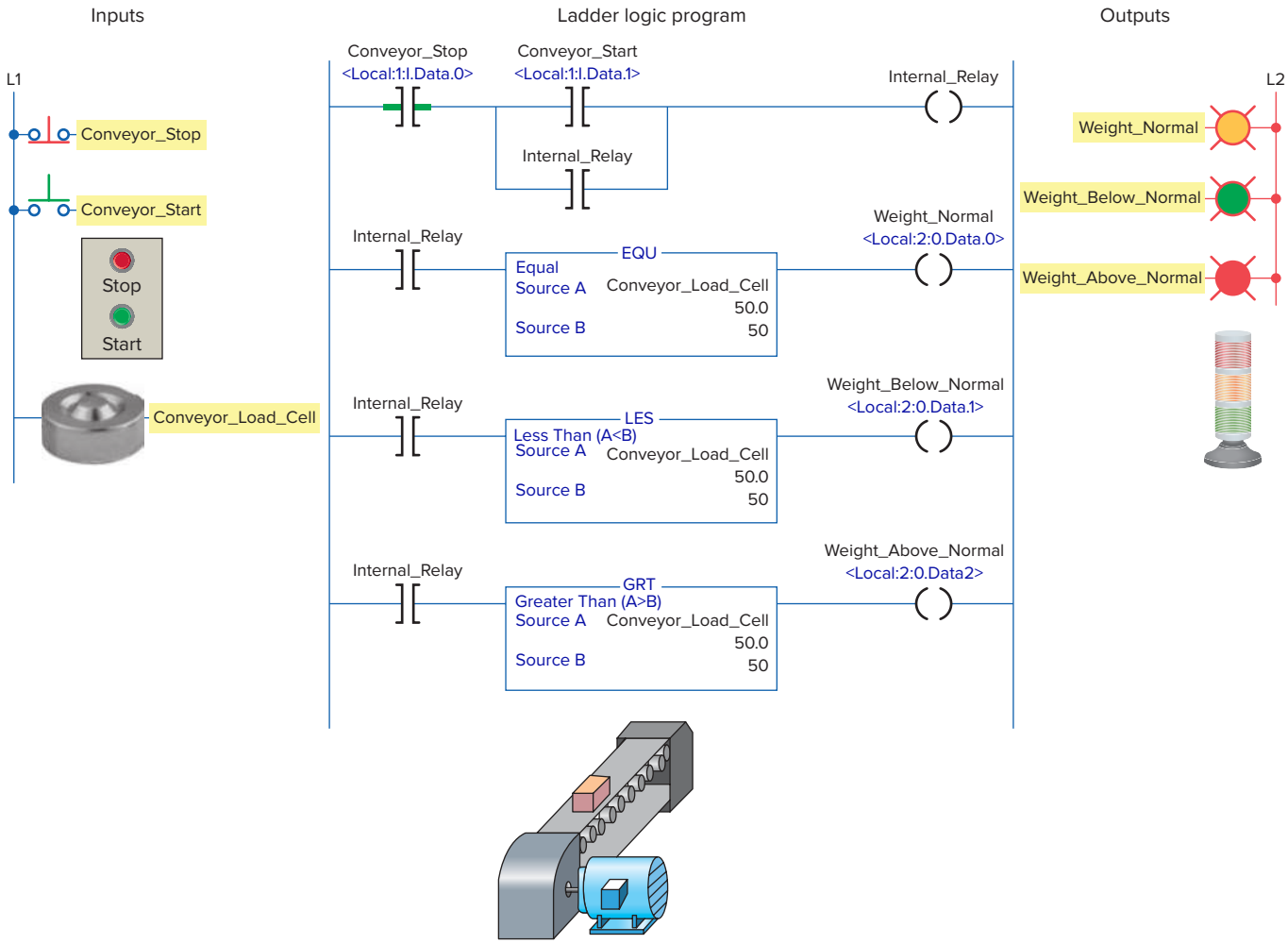


Figure 15-93 PLC program for three-speed control of a conveyor system.

value of BCD_TWS is equal to 20 decimal (BCD 100000).

Figure 15-93 shows the PLC program for speed control of a three-speed motor conveyor system. The system comes equipped with a weight detector and three-speed motor controller. It is designed to move the conveyor belt at a certain speed when a specific value of weight is on the conveyor. If the weight exceeds the preset value, the conveyor speed increases to compensate for the increase in weight. If the weight falls below the preset value, the conveyor speed is reduced to compensate for the decrease in weight. The operation of the program can be summarized as follows:

- When the Conveyor_Start button is pressed, Internal_Relay seals in, and the conveyor begins moving.
- An analog input is used as the Source A location, which stores the data obtained from the weight detector.
- The reference, or Source B value is set at 50.
- The Conveyor_Load_Cell tag stores the data from the weight detector.
- If the data in the Conveyor_Load_Cell is equal to the Source B value, the motor operates at normal speed.
- If the data in the Conveyor_Load_Cell is above the reference value, due to more weight on the conveyor, the Weight_Above_Normal output is energized and a signal is sent to the motor controller to change to a higher speed.
- If the data in the Conveyor_Load_Cell is below the reference value, due to less weight on the conveyor, the Weight_Below_Normal output is energized and a signal is sent to the motor controller to decrease its speed.

Figure 15-94 shows the parts tracking program for a conveyor system consisting of a main conveyor and rejection conveyor. If a part fails inspection, the diverter gate is energized and the part is routed onto the reject conveyor.

Each conveyor has a limit switch used to count parts as they pass by. The operation of the parts tracking program of the process can be summarized as follows:

- The Main_Conveyor_Counter.ACC contains the total number of parts that have been counted.
- The Reject_Conveyor_Counter.ACC contains the number of parts that have been rejected.
- The SUB instruction is used to provide a running count of items that have passed inspection and has as its Destination the Data_Format_Conversion tag.
- BCD_Output_Display is used to show the running count for passed items.
- Reset_PB switch is activated to reset both counters to zero and clear the BCD_ Output_Display to zero.

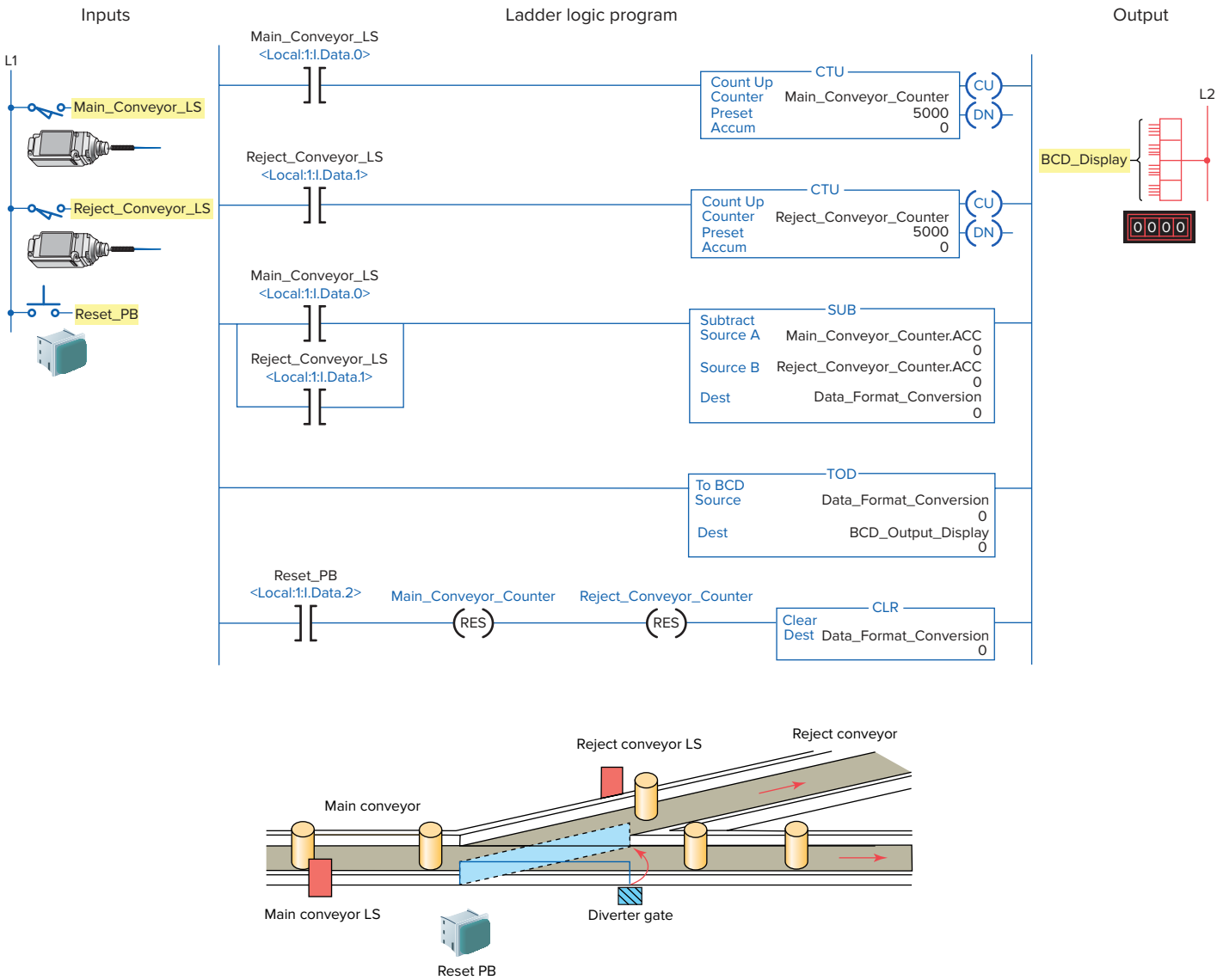


Figure 15-94 Conveyor parts tracking program.

PART 5 REVIEW QUESTIONS

1. Construct a ControlLogix ladder rung with a math instruction that executes when a toggle switch is closed to add the tag named Pressure_A (value 680) to the constant of 50 and store the answer in the tag named Result.
2. Construct a ControlLogix ladder rung with a math instruction that executes when two normally open limit switches are closed to subtract the tag named Count_1 (value 60) from the tag named Count_2 (value 460) and store the answer in the tag named Count_Total.
3. Construct a ControlLogix ladder rung with a math instruction that executes when either one of two normally open pushbuttons is closed to multiply the tag named Cases (value 10) by the constant 24 and store the answer in the tag named Cans.
4. Construct a ControlLogix ladder rung with a compare instruction that will energize a pilot light output anytime the value stored at Data_3 is 60.
5. Construct a ControlLogix ladder rung with a compare instruction that will energize a pilot light output anytime the value stored at Data_2 is not the same as that stored at Data_6.
6. Construct a ControlLogix ladder rung with compare instructions that will energize a pilot light output anytime the pressure of a system goes above 300 psi or below 100 psi.

PART 5 PROBLEMS

1. While checking the operation of the parts tracking system (Figure 15-80) with the Monitor Tags window, you note that the value of Conveyor_Sensor_1 remains at 1 with parts passing by. What can you surmise from this? Why?
2. Three conveyors are delivering the same parts in different packages. A package can hold 12, 24, or 18 parts. Proximity switches installed on each of the conveyor lines are used to advance the accumulated value of the three counters. Write a ControlLogix program that uses multiply and add instructions to calculate the sum of the parts.
3. A single pole switch is used in place of the two pushbuttons for the variable preset timer program (Figure 15-91). When this switch is closed the timer is to be set to 10 seconds and when open to 15 seconds. Make the necessary changes to the program.

Part 6 Function Block Programming

Part Objectives

After completing this part, you will be able to:

- Describe the difference between ladder logic and function block diagram programming
- Recognize the basic elements of a function block diagram
- Write and read a function block diagram

Function Block Diagram (FBD)

A ***function block diagram (FBD)*** is a graphical depiction of process flow using simple and complex interconnecting blocks. It is similar to a ladder logic diagram, except that function blocks replace the interconnection of contacts and the coils. In addition, there are no power rails.

A function block circuit is analogous to an electrical circuit where links and wires depict signal paths between components. The workplace is known as a sheet and consists of function blocks joined together with lines called wires. The structure of a function block program, or routine, is shown in Figure 15-95. A function block diagram consists of four basic elements: function block, references, wire connectors, and wires. Data flows on a wire from wire connectors or input references, through the function block, and then is passed on to an output reference. The line type of the link between function blocks indicates what type of data is present. A dash line indicates a Boolean signal path (e.g., 0 or 1) and a solid line indicates an integer or real value.

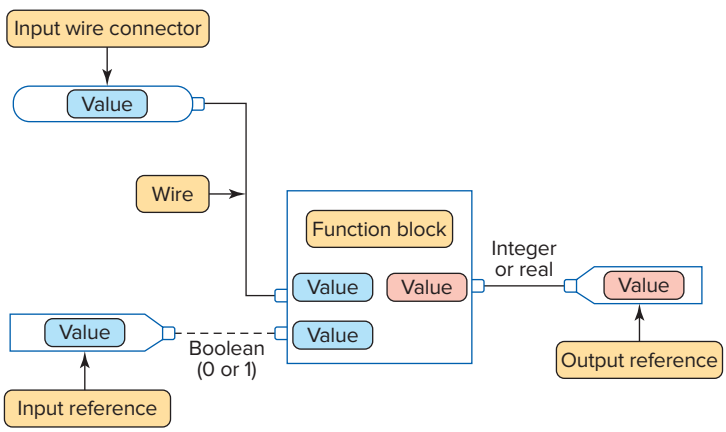
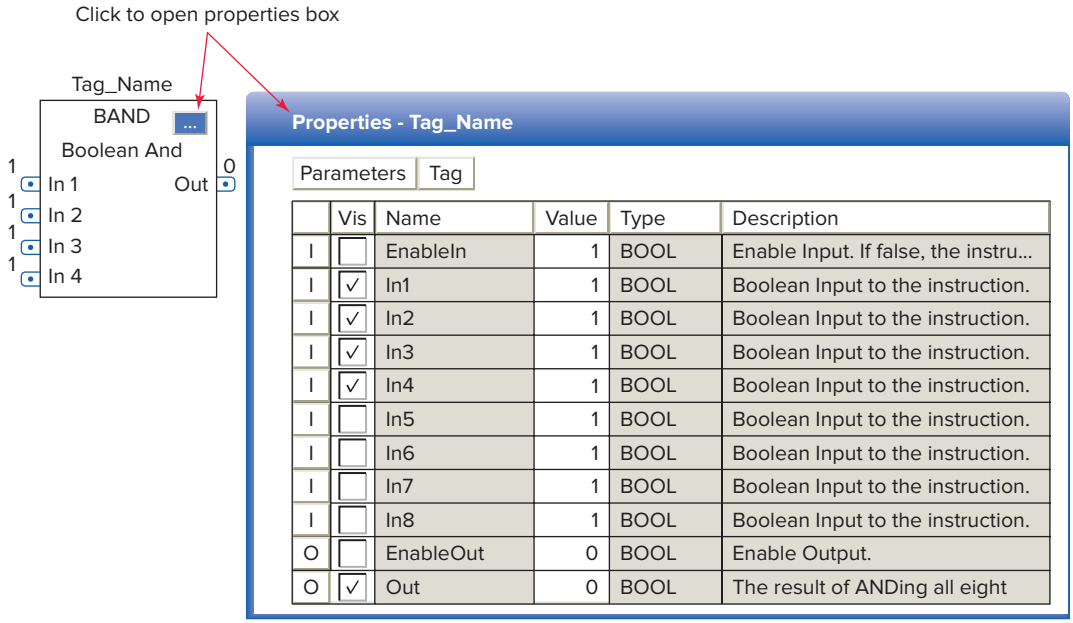


Figure 15-95 Structure of function block or routine.

Function blocks are graphical representations of executable code. A function block can take one or more inputs and make decisions or calculations and then generate one or more outputs. There are many different types of function blocks included in the programming software to perform various common tasks. In addition, customized ***Add-On*** instructions can be created by the programmer for sets of commonly used logic. Once an Add-On instruction is defined in a project, it appears on the instruction toolbar and behaves like the standard instructions.

Figure 15-96 shows an example of a BAND (Boolean AND) function block. The information associated with a function block can be summarized as follows:

- Inputs are shown entering from the left and outputs exiting on the right.
- The function block type is shown within the block.
- A tag name for the block is placed above it.



	Vis	Name	Value	Type	Description
I	☐	EnableIn	1	BOOL	Enable Input. If false, the instru...
I	✓	In1	1	BOOL	Boolean Input to the instruction.
I	✓	In2	1	BOOL	Boolean Input to the instruction.
I	✓	In3	1	BOOL	Boolean Input to the instruction.
I	✓	In4	1	BOOL	Boolean Input to the instruction.
I	☐	In5	1	BOOL	Boolean Input to the instruction.
I	☐	In6	1	BOOL	Boolean Input to the instruction.
I	☐	In7	1	BOOL	Boolean Input to the instruction.
I	☐	In8	1	BOOL	Boolean Input to the instruction.
O	☐	EnableOut	0	BOOL	Enable Output.
O	✓	Out	0	BOOL	The result of ANDing all eight

Figure 15-96 Example of a BAND (Boolean AND) function block.
Source: Image Courtesy of Rockwell Automation, Inc.

- The names of the inputs and outputs are shown within the block.
- The default view of the block has some but not all of the input and output parameters visible when the box is placed into the program.
- The properties box, used to set the option of input and output parameters, is displayed by clicking the selection button located at the upper right hand corner of the block.
- The 1 and 0 next to the inputs and outputs identify the logical state of the input and output pins for the instruction.
- The dots on the input and output pins indicate BOOL type data is required.

References represent tags that are linked to values stored in a controller's memory. The two types of references, input and output, are illustrated in Figure 15-97. An input reference, or IREF, is used to receive a value from an input device or tag. An output reference, or OREF, is used to send a value to an output device or tag. When you use an IREF or an OREF you must create a tag or assign an existing tag to the element. You may use any of the data types for an IREF or OREF.

Function blocks can be connected to other function blocks by connecting their outputs to the input of another function block using ***wires and pins*** (Figure 15-98). Wires map a signal's path and show the flow of controller execution. Each element in a function block diagram contains pins. Elements are connected by moving wires from input pins to output pins or vice versa. The pins on the left of a function block are input pins, and those on the right are output pins. To wire two elements together, click the output pin of the first element (A) and then click the input pin of the other element (B). A green dot shows a valid connection point.

Wire connectors are used to create a path without using a wire. When there are many function blocks on a sheet, or the function blocks are far apart, wire connectors used in place of wires can make the logic easier to read.

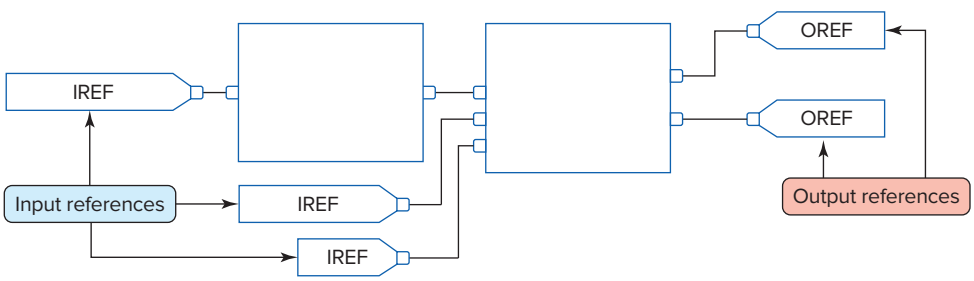


Figure 15-97 Input and output references.

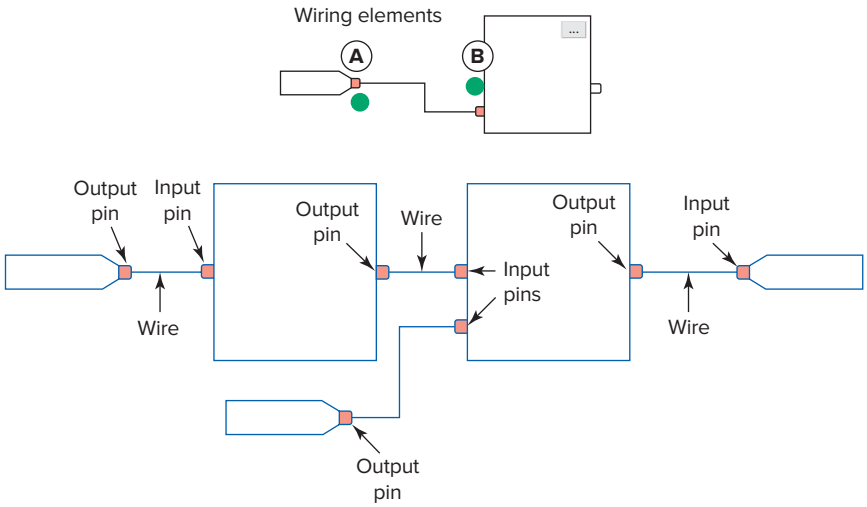


Figure 15-98 Function block diagram wire and pins.

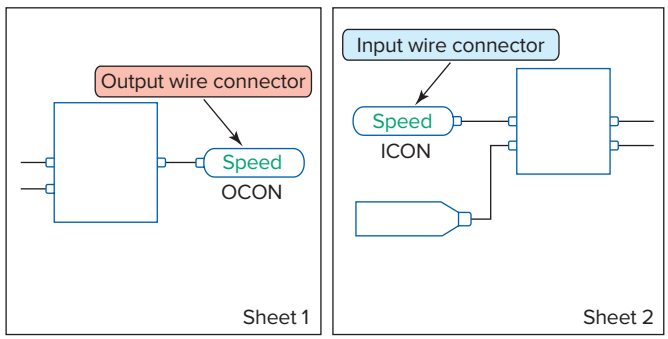


Figure 15-99 OCON and ICON wire connectors.

Wire connectors are also used to connect function blocks that are on a different sheet of the same function block routine, as illustrated in Figure 15-99. The use of wire connectors can be summarized as follows:

- An output wire connector, or **OCON**, sends a value or signal to an input wire connector, or **ICON**.
- Each output wire connector must have at least one corresponding input wire connector.
- Each output wire connector requires a unique tag name and the corresponding input connector must have the same name.
- Multiple input wire connectors can reference the same output wire connector. This lets you share data at several points in your function block diagram.

Figure 15-100 illustrates the signal flow and execution of an FBD program. The operation can be summarized as follows:

- Each program scan sets all the FBD blocks starting on the left side of the signal flow and continues to evaluate all blocks according to the signal flow until the final output is determined.
- The location of a block does not affect the order in which the blocks execute.

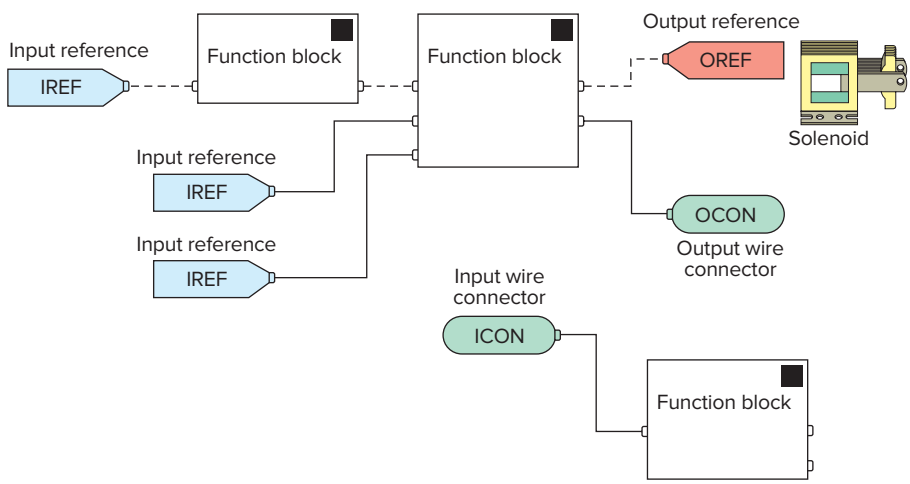


Figure 15-100 Signal flow and execution of an FBD program.

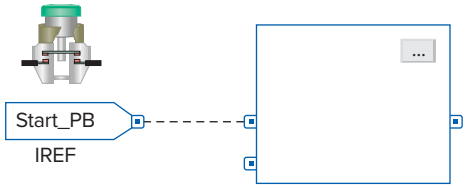


Figure 15-101 IREF is latched for the scan of the function block routine.

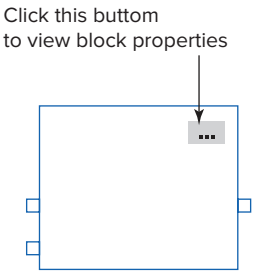
- The inputs of a block require data to be available before the controller can execute that block.
- If function blocks are not wired together, it does not matter which block executes first as there is no data flow between the blocks.
- The interconnected line between the blocks indicates what type of signal is present.

Data latching refers to how the controller verifies that the data present at the input to a function block are valid. If you use an IREF to specify input data for a function block instruction, as illustrated in Figure 15-101, the data in that IREF are latched (won't change) for the scan of the function block routine. The IREF latches data from program-scoped and controller-scoped tags. The controller updates all IREF data at the beginning of each scan. A function block routine executes in the following order:

- The controller latches all data values in IREFs.
- The controller executes the other function blocks in order.
- The controller writes outputs in OREFs.

When you add a Function Block instruction, the block appears with a set of pins for the default parameters, as illustrated in Figure 15-102. The rest of the pins are hidden. You can hide or show a pin by:

- Clicking on the Parameters tab in the Properties dialog box.
- In the Properties dialog box, on the Parameters tab, clear the Vis check box to hide the pin.



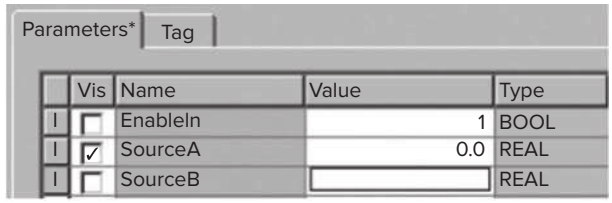


Figure 15-102 Using the Parameters tab to show or hide a pin.
Source: Image Courtesy of Rockwell Automation, Inc.

- Select the Vis check box to show the pin.
- Click OK.

To create a ***feedback loop*** around a block, wire an output pin of the block to an input pin of the same block. The input pin will receive the value of the output that was produced on the last scan of the function block. The loop contains only a single block, so execution order does not matter. Figure 15-103 shows an example of a feedback loop used to reset an on-delay timer. When the timer finishes timing its DN bit is used to reset the timer.

When a group of function blocks are in a feedback loop, the controller cannot determine which block to execute first. This problem is resolved by placing an **Assume Data Available** indicator mark at the input pin of the function block that should be executed first. In the example shown in Figure 15-104, the input for block 1 uses the data from block 3 that were produced in the previous scan. To place the indicator, click on the interconnecting wire and select the **Assume Data Available** choice.

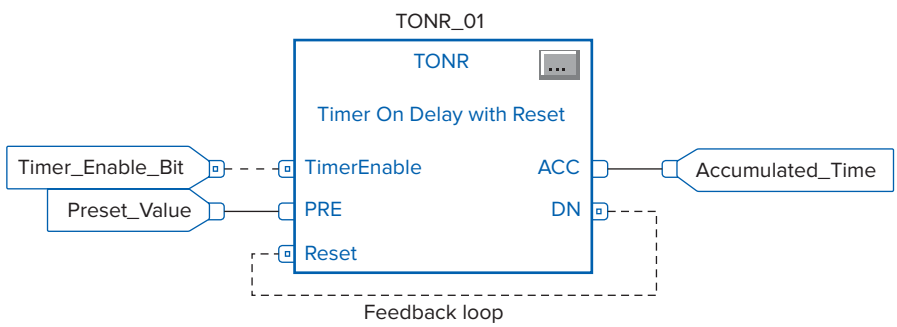


Figure 15-103 Feedback loop used to reset an on-delay timer.

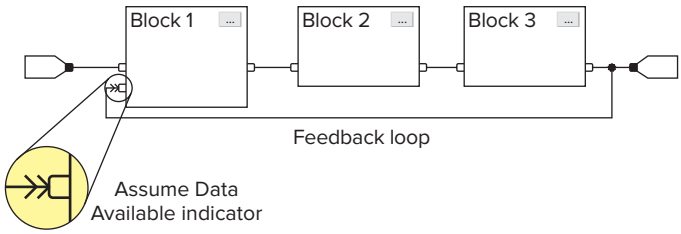


Figure 15-104 Assume Data Available indicator marker.

FBD Programming

Figure 15-105 illustrates the setup procedure for FBD programming. The steps to be followed can be summarized as follows:

- Right click on the MainProgram file and select New Routine from the pop-up menu.
- Select the Function Block diagram entry from the Type window.
- Enter a name for the Routine (e.g., FDB_Sample).
- You will now see the new program (FDB_Sample) listed under MainProgram.
- Left clicking the FBD_Sample twice opens the graphic development window.
- FBD instructions selected from the Language Element toolbar are used in the development of the program.
- Extra sheets can be added when the current sheet is full by clicking the add sheet icon. Movement between sheets is provided by left and right arrows.

The MainRoutine is always a ladder logic program in RSLogix 5000 software, and all other routines are called from the MainRoutine. Therefore, the MainRoutine will have one unconditional rung with a jump to subroutine (JSR) calling FBD_Sample. The FBD program will execute from the JSR instruction. No subroutine or return subroutine instruction in the FBD is necessary.

Function block programs are similar to ladder logic programs, except that the process is visualized in the form of function blocks instead of ladder rungs. Figure 15-106 shows a comparison between ladder logic and the FBD equivalent for a three-input AND ladder logic rung. The operation of the FBD can be summarized as follows:

- When the inputs represented by Sensor_1, Sensor_2, and Sensor_3 are true (value 1) the BAND (Boolean AND) function block will be true.
- The BAND block executes to set output Caution_PL true and switch the pilot light on.
- The 0 to the right of the input reference and out pin indicates its logic state. A 0 indicates the state of the tag is false, while a 1 signifies it is true.
- The same field input sensors and output pilot light devices and tags can be used with either program.
- The XIC and OTE contact and coil instructions have been replaced by the BAND function block.

Figure 15-107 shows a comparison between ladder logic and the FBD equivalent for a two-input OR ladder logic rung. As with ladder OR logic, if any of the two inputs is true the BOR function block will be true. In this example, with the BOR function block true, the output reference tag SOL_1 will be true, energizing the solenoid.

Figure 15-108 shows a comparison between ladder logic and the FBD equivalent for a combination of multiple inputs. The operation of the FBD can be summarized as follows:

- The alarm will be energized if either input In1 or In2 to the BOR block is true.

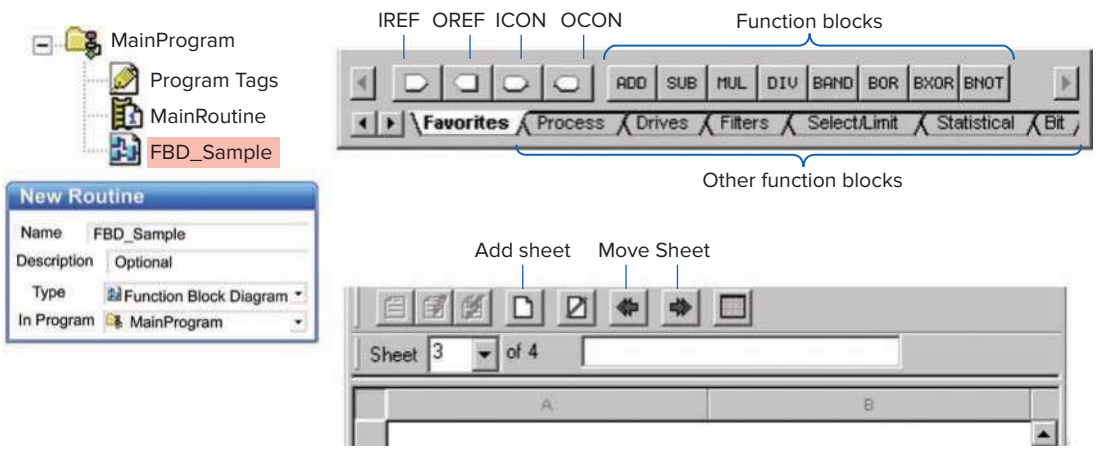


Figure 15-105 Setup procedure for FDB programming.
Source: Image Courtesy of Rockwell Automation, Inc.

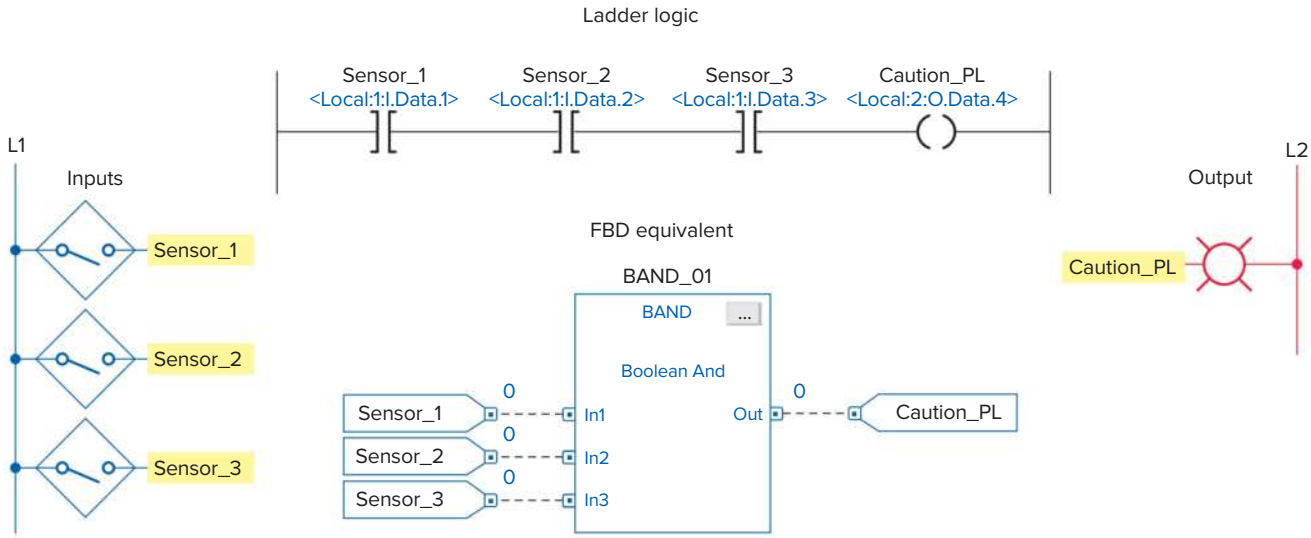


Figure 15-106 Comparison between ladder logic and the FBD equivalent for a three-input AND ladder logic rung.

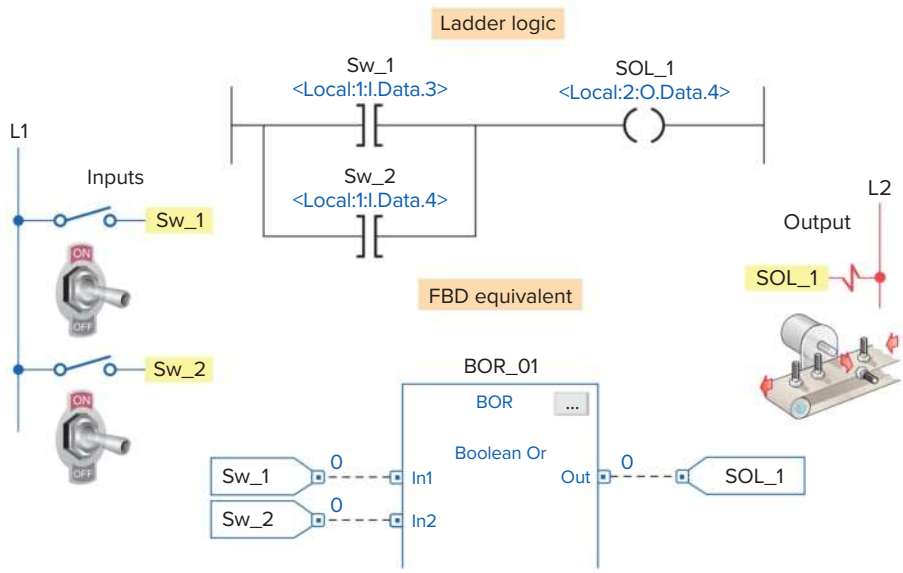


Figure 15-107 Comparison between ladder logic and the FBD equivalent for a two-input OR ladder logic rung.

- Input In2 of the BOR block will be true only when all three of the sensor switches are closed.
- Input In1 of the BOR block will be true only when the Temp_Sw is closed at the same time as the Press_Sw is open.
- The BNOT function block executes similarly to an XIO ladder logic contact instruction. When In is 0, Out is 1 and vice versa.

Figure 15-109 shows a comparison between ladder logic and the FBD equivalent for the motor start/stop control circuit. The logic sequence for starting and stopping the motor can be summarized as follows:

- When Motor_Start button is closed the BOR output will become true making the BAND output true.
- Motor_Run output energizes the contactor coil, the contacts of which close to start the motor operating.
- When the Motor_Start button is then opened the output of the BOR block remains true due to the 1 status of the feedback signal from the Motor_Run tag.

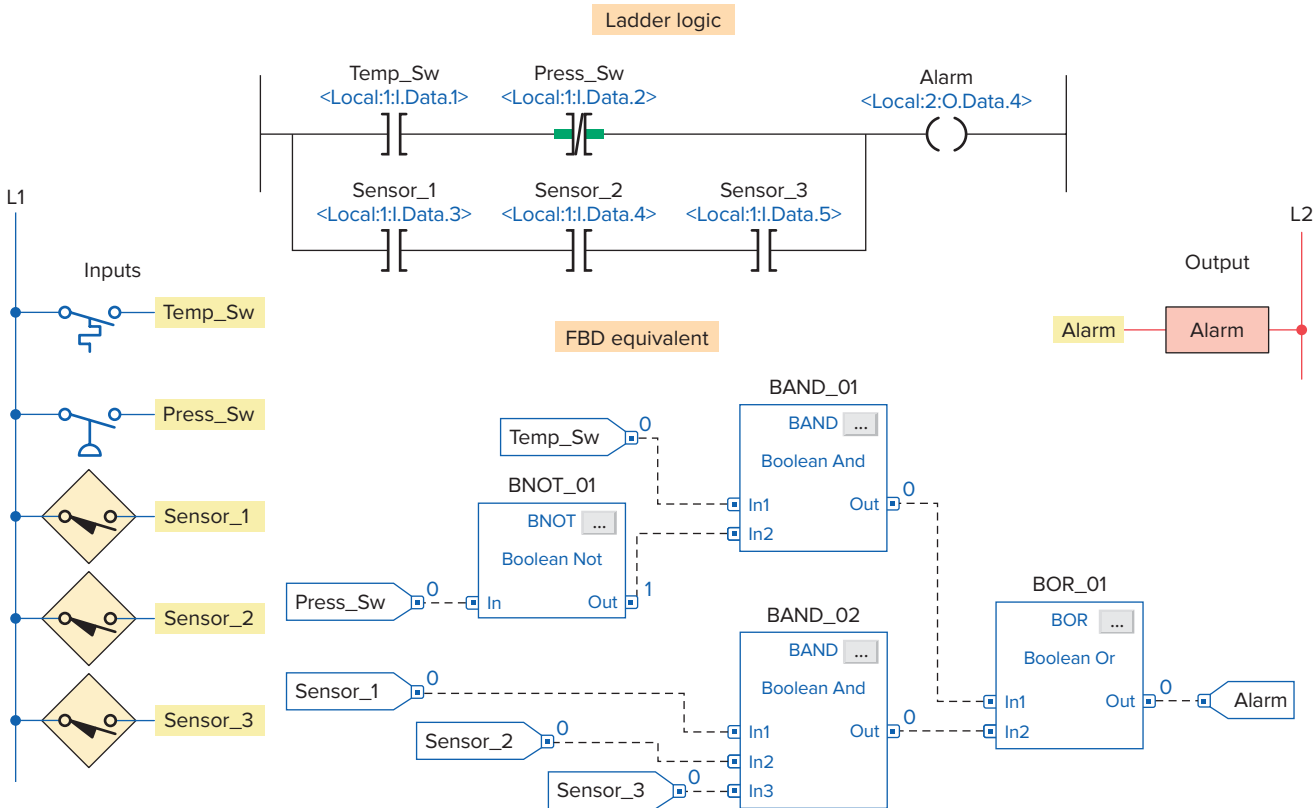


Figure 15-108 Comparison between ladder logic and the FBD equivalent for a combination of multiple inputs.

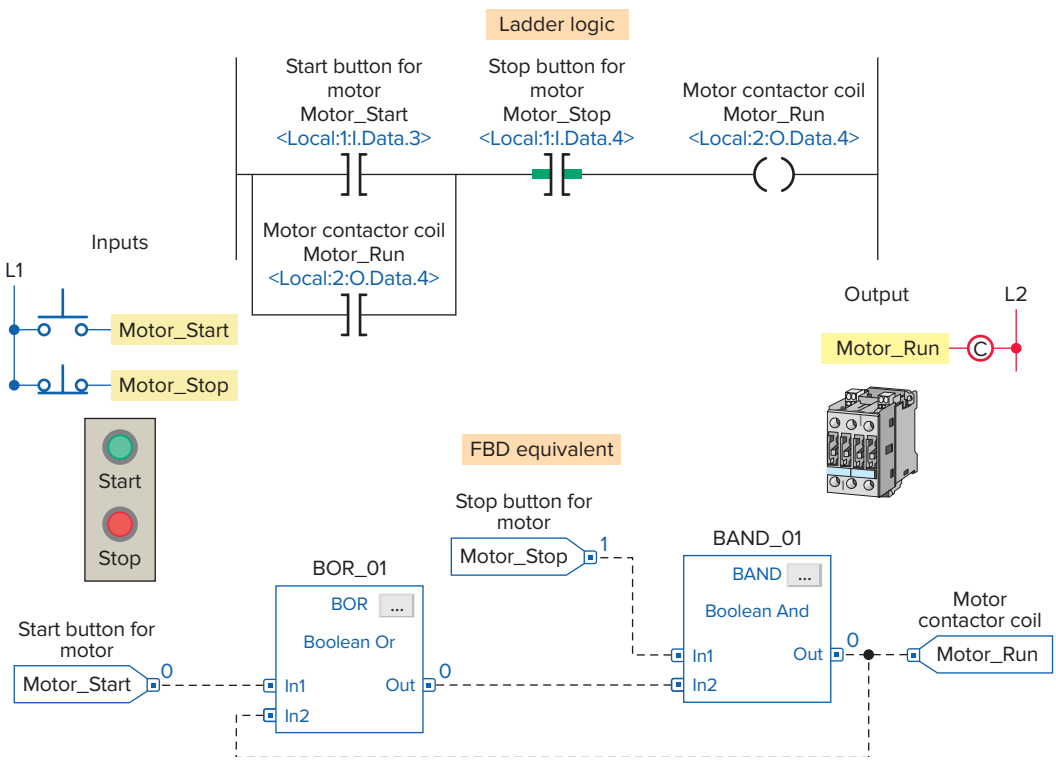


Figure 15-109 Comparison between ladder logic and the FBD equivalent for a motor start/stop control circuit.

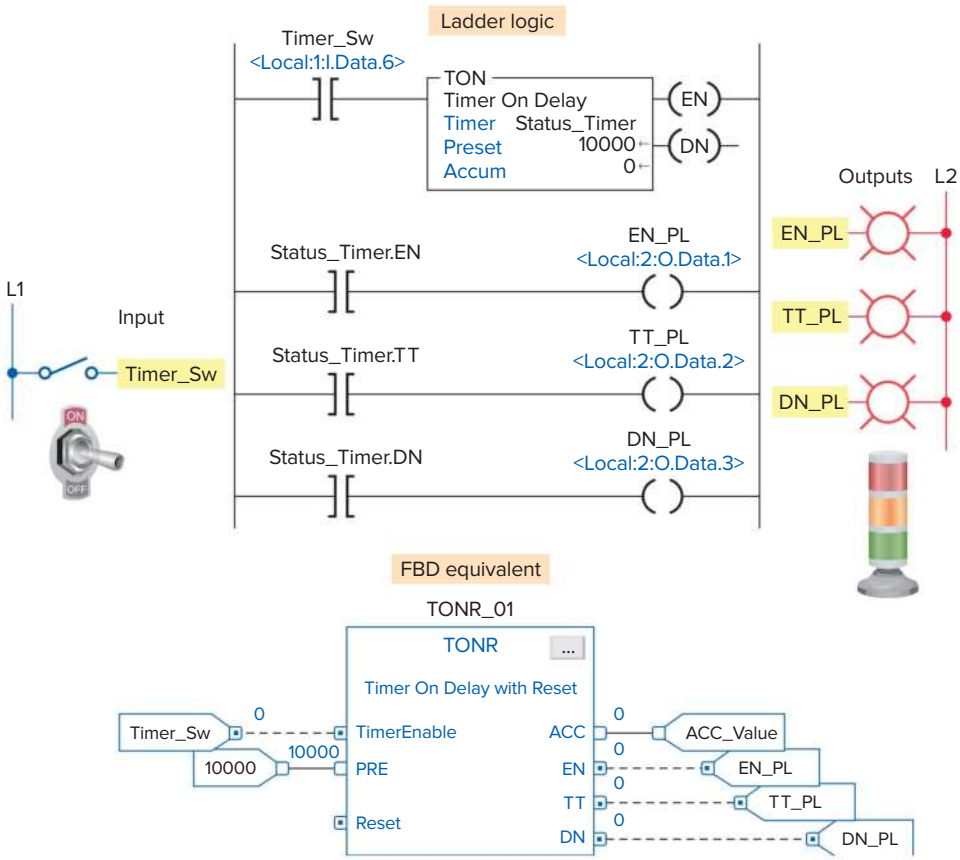


Figure 15-110 Comparison between ladder logic and the FBD equivalent for a 10 second TON and TONR timer.

- When the Motor_Stop button is opened the output of the BAND block turns false to de-energize the contactor coil and stop the motor.

Figure 15-110 shows a comparison between ladder logic and the FBD equivalent for the 10 second TON (on-delay timer) and TONR (on-delay with reset). The operation of the FBD can be summarized as follows:

- When the Timer_Sw is closed, the TONR function block timer turns true and starts accumulating time.
- The accumulated time is monitored by the output reference tag named ACC.
- The EN (enable bit) output changes to 1 to turn on the EN_PL.
- The TT (timer timing bit) output changes to 1 to turn on the TT_PL.
- The timer times out after 10 seconds to set the DN (done bit) to 1 and turn on the DN_PL and reset the TT bit to zero and turn off the TT_PL.
- The EN bit and EN_PL remain on as long as the Timer_Sw stays toggled closed.
- Opening the Timer_Sw resets all outputs as well as the accumulated value to zero.
- The timer can also be reset by way of the Reset input.

Figure 15-111 shows a comparison between ladder logic and the FBD equivalent for the Up/Down counter used to limit the number of parts stored in a buffer zone to 50. The operation of the FBD can be summarized as follows:

- The CTUD up/down counter function block accumulated value is initially reset by momentary actuation of the Restart_Button.
- The accumulated count is monitored by the output reference tag named ACC.
- Each time a part enters the buffer zone, the Enter_Limit_Sw is actuated and the CUEnable input turns true to increment the count by 1.
- Each time a part exits the buffer zone, the Exit_Limit_Sw is actuated and the CDEnable input turns true to decrement the count by 1.
- Whenever the number of parts in the buffer zone reaches 50 the DN bit is set to 1 and the output of

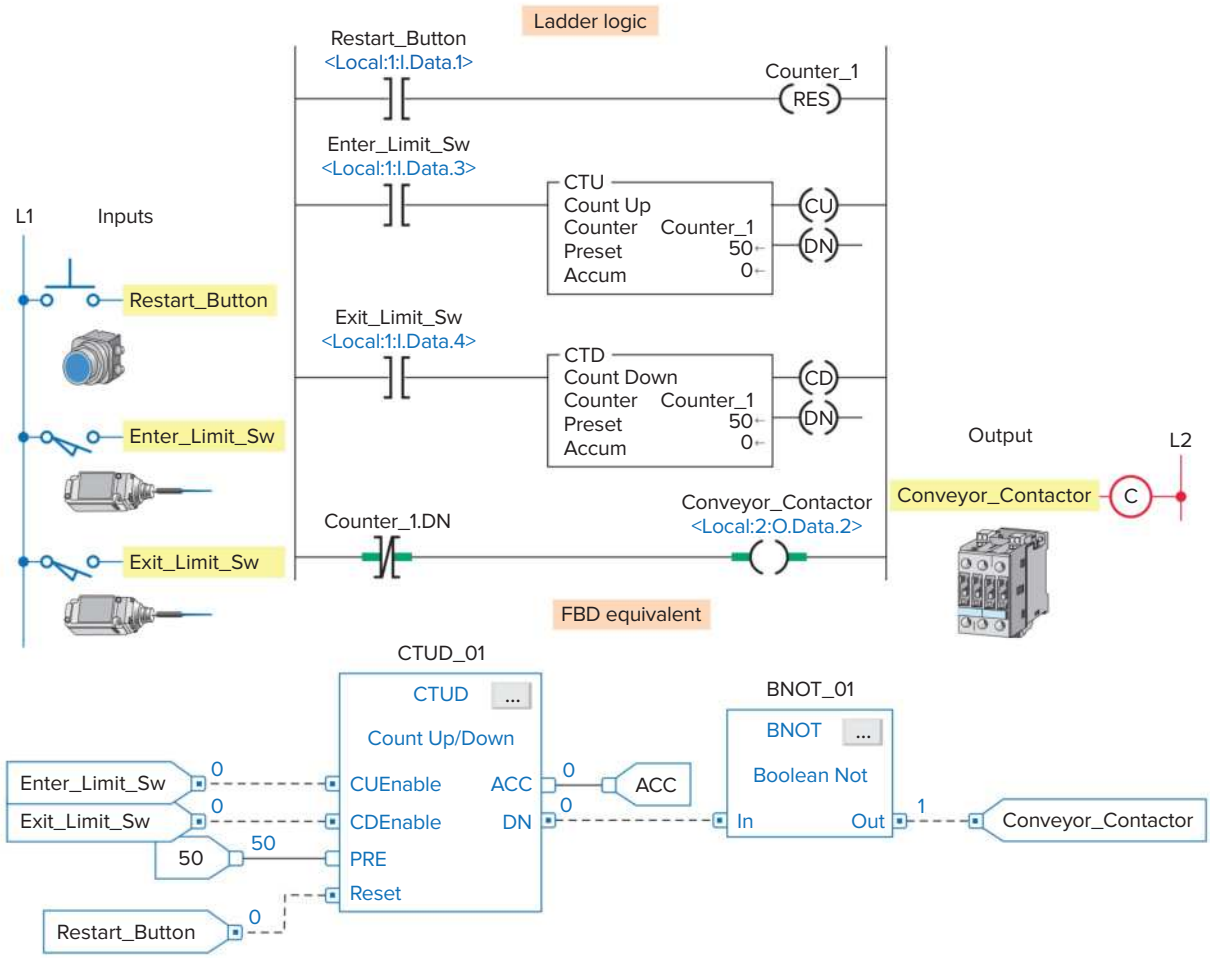


Figure 15-111 Comparison between ladder logic and the FBD equivalent for an Up/Down counter application.

the BNOT block is reset to zero. This de-energizes the Conveyor_Contactor to stop the conveyor motor from delivering more parts to the buffer zone.

Figure 15-112 shows a comparison between ladder logic and the FBD equivalent for the program used to test the accumulated value of a counter. The operation of the FBD can be summarized as follows:

- The function block routine is broken into four sheets.
- The order of the sheets does not affect the order in which the function blocks execute.
- When a function block routine executes, all sheets execute.
- Using one sheet for each device that is to be programmed helps organize your program and make it easier to understand.
- The use of the OCON and ICON named ACC enables the function blocks to be on different sheets of the same function block routine.
- The numbers and letters under the ACC output indicate the sheet number and location on the sheet where the output is used.

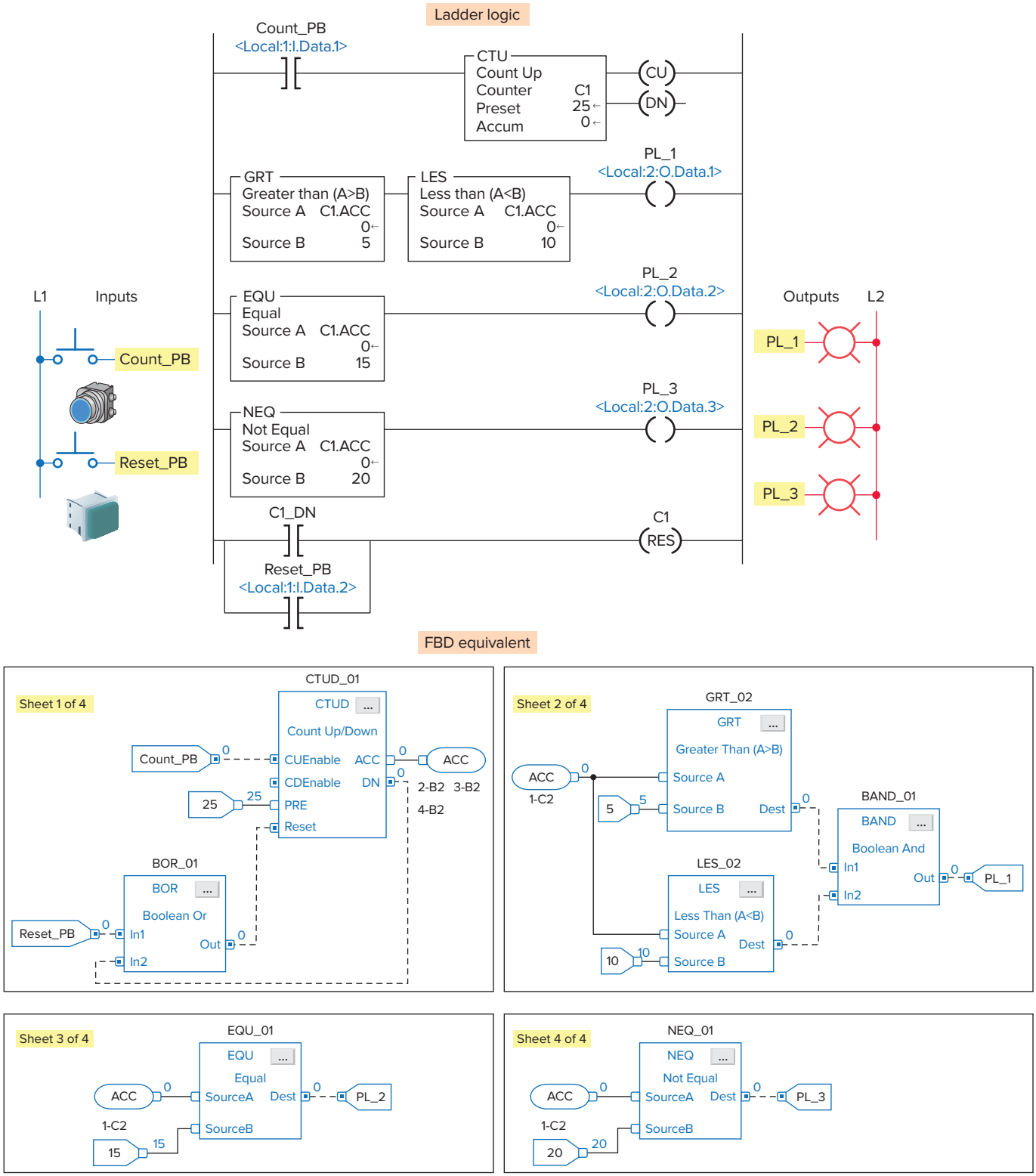


Figure 15-112 Comparison between ladder logic and the FBD equivalent for a program used to test the accumulated value of a counter.

PART 6 REVIEW QUESTIONS

1. Compare the graphical representation of a function block diagram to that of a logic ladder diagram.
2. Name the four basic elements of an FBD.
3. What do the solid and dashed interconnecting lines between FBD function blocks indicate?
4. What is an Add-On instruction?
5. How are the input and output parameter options for a function block set?
6. What does the dot on an input or output pin of a function block indicate?
7. Compare the functions of input and output reference tags.
8. Which pins of a function block are inputs and which are outputs?
9. Explain the role of input and output wire connectors.
10. How does the program scan function for an FBD program?
11. Explain data latching as it applies to function block inputs.
12. How is a function block feedback loop created?
13. What is the Assume Data Available indicator used for?
14. Outline how an FBD program is initiated.

PART 6 PROBLEMS

1. Write an FBD program that will cause the output, solenoid SOL_1, to be energized when pushbutton PB_1 is open and PB_2 is closed, and either limit switch LS_1 is open or limit switch LS_2 is closed. Assume all pushbuttons and limit switches are of the normally open type.
2. Modify the motor start/stop FBD program (Figure 15-109) to include a second start/stop pushbutton station.
3. You are required to change the on-delay time of the 10 second timer program (Figure 15-110) to 1 minute. What changes would have to be made to the FBD program?
4. Modify the Up/Down counter FBD program (Figure 15-111) to include the following pilot lights:
   - PL_1 to come on when a part enters
   - PL_2 to come on when a part exits
   - PL_3 to come on when the buffer zone is full
5. Modify the test accumulated value of a counter FBD program (Figure 15-112) as follows:
   - PL_1 to be on for an accumulated count between 0 and 5
   - PL_2 to be on for an accumulated count of 12
   - PL_3 to be on at all times except for when the accumulated count is 15.

Glossary

1's complement The system used to represent negative numbers in a personal computer and a programmable logic controller.

2's complement A numbering system used to express positive and negative binary numbers.

A

Access To locate data stored in a programmable logic controller system or in computer-related equipment.

Accumulated value The number of elapsed timed intervals or counted events.

Actuator An output device normally connected to an output module. Examples are an air valve and cylinder.

Address A code that indicates the location of data to be used by a program, or the location of additional program instructions.

Algorithm Mathematical procedure for problem solving.

Alias tag References a memory location that has been defined by another tag.

Alphanumeric Term describing character strings consisting of any combination of letters, numerals, and/or special characters (e.g., A15$) used for representing text, commands, numbers, and/or code groups.

Alternating current (AC) input module An input module that converts various alternating current signals originating at user devices to the appropriate logic-level signal for use within the processor.

Alternating current (AC) output module An output module that converts the logic-level signal of the processor to a usable output signal to control a user alternating current device.

Ambient temperature The temperature of the air surrounding a module or system.

American National Standard Code for Information Interchange (ASCII) An 8-bit (7 bits plus parity) code that represents all characters of a standard typewriter keyboard, both uppercase and lowercase, as well as a group of special characters used for control purposes.

American National Standards Institute (ANSI) A clearinghouse and coordinating agency for voluntary standards in the United States.

American wire gauge (AWG) A standard system used for designating the size of electrical conductors. Gauge numbers have an inverse relationship to size; larger numbers have a smaller diameter.

Analog device Apparatus that measures continuous information (e.g., voltage or current). The measured analog signal has an infinite number of possible values. The only limitation on resolution is the accuracy of the measuring device.

Analog input module An input circuit that employs an analog-to-digital converter to convert an analog value, measured by an analog measuring device, to a digital value that can be used by the processor.

Analog output module An output circuit that employs a digital-to-analog converter to convert a digital value, sent from the processor, to an analog value that will control a connected analog device.

Analog signal Signal having the characteristic of being continuous and changing smoothly over a given range rather than switching suddenly between certain levels, as with discrete signals.

Analog-to-digital (A/D) converter A circuit for converting a varying analog signal to a corresponding representative binary number.

AND (logic) A Boolean operation that yields a logic 1 output if all inputs are 1, and a logic 0 if any input is 0.

Arithmetic capability The ability to do addition, subtraction, multiplication, division, and other advanced math functions with the processor.

Array A group of data, of the same type, under a common name.

ASCII-input module Converts ASCII-code input information from an external peripheral into alphanumeric information a PLC can understand.

ASCII-output module Converts alphanumeric information from the PLC into ASCII code to be sent to an external peripheral.

Asynchronous Recurrent or repeated operations that occur in unrelated patterns over time.

Automatic control A process in which the output is kept at a desired level by using feedback from the output to control the input.

Auxiliary power supply A power supply not associated with the processor. Auxiliary power supplies are usually required to supply logic power to input/output racks and to other processor support hardware and are often referred to as *remote power supplies.*

B

Backplane A printed circuit board, located in the back of a chassis, that contains a data bus, power bus, and mating connectors for modules to be inserted in the chassis.

Base tag A definition of the memory location at which a data element is stored.

BASIC A computer language that uses brief English-language statements to instruct a computer or microprocessor.

Battery indicator A diagnostic aid that provides a visual indication to the user and/or an internal processor software indication that the memory power-fail support battery is in need of replacement.

Baud A unit of signaling speed equal to the number of discrete conditions or signal events per second; often defined as the number of binary digits transmitted per second.

BCD-input module Allows the processor to accept 4-bit BCD digital codes.

BCD-output module Enables a PLC to operate devices that require BCD-coded signals to operate.

Binary A number system using 2 as a base. The binary number system requires only two digits, zero (0) and one (1), to express any alphanumeric quantity desired by the user.

Binary-coded decimal (BCD) A system of numbering that expresses each individual decimal digit (0–9) of a number as a series of 4-bit

binary notations. The binary-coded decimal system is often referred to as *8421 code.*

Binary word A related group of 1s and 0s that has meaning assigned by position or by numerical value in the binary system of numbers.

Bit An abbreviated term for the words *binary digit.* The bit is the smallest unit of information in the binary numbering system. It represents a decision between one of two possible and equally likely values or states. It is often used to represent an off or on state as well as a true or false condition.

Bit manipulation instructions A family of programmable logic controller instructions that exchange, alter, move, or otherwise modify the individual bits or groups of processor data memory words.

Bit storage A user-defined data table area in which bits can be set or reset without directly affecting or controlling output devices. However, any storage bit can be monitored as necessary in the user program.

Block diagram A method of representing the major functional subdivisions, conditions, or operations of an overall system, function, or operation.

Block format Format that uses a box shape to display instructions.

Block transfer An instruction that copies the contents of one or more contiguous data memory words to a second contiguous data memory location; an instruction that transfers data between an intelligent input/output module or card and specified processor data memory locations.

BOOL A data type that stores the state of a single bit, where 0 equals off and 1 equals on.

Boolean algebra A mathematical shorthand notation that expresses logic functions, such as AND, OR, EXCLUSIVE OR, NAND, NOR, and NOT.

Branch A parallel logic path within a rung.

Buffer In software terms, a register or group of registers used for temporary storage of data; a buffer is used to compensate for transmission rate differences between the transmitter and receiving device. In hardware terms, a buffer is an isolating circuit used to avoid the reaction of one circuit with another.

Bug A system defect or error that causes a malfunction; can be caused by either software or hardware.

Burn The process by which information is entered into programmable read-only memory.

Bus A group of lines used for data transmission or control; power distribution conductors.

Bus topology A network configuration in which all stations are connected in parallel with the communication medium and all stations can receive information from any other station on the network.

Byte A group of adjacent bits usually operated on as a unit, such as when moving data to and from memory. There are 8 bits per byte.

C

Cascading In the programming of timers and counters, a technique used to extend the timing or counting range beyond what would normally be available. This technique involves the driving of one timer or counter instruction from the output of another, similar instruction.

Cell controller A specialized computer used to control a work cell through multiple paths to the various cell devices.

Central processing unit (CPU) The electronic circuitry that controls all the data activity of the PLC, performs calculations, and makes decisions with its operation controlled by a sequence of instructions. The central processing unit is also referred to as the *processor* or the *CPU.*

Channel A designated path for a signal.

Character A symbol that is one of a larger group of similar symbols and that is used to represent information on a display device. The letters of the alphabet and the decimal numbers are examples of characters used to convey information.

Chassis A rack that serves as an electrical backplane for a PLC processor and I/O modules.

Chip A tiny piece of semiconductor material on which electronic components are formed. Chips are normally made of silicon and are typically less than 1/4 in. square and 1/100 in. thick.

Clear An instruction or a sequence of instructions that removes all current information from a programmable logic controller's memory.

Clock A circuit that generates timed pulses to synchronize the timing of computer operations.

Clock rate The speed at which the microprocessor system operates.

Closed loop A control system that uses feedback from the process to maintain outputs at a desired level.

Coaxial cable A transmission line constructed such that an outer conductor forms a cylinder around a central conductor. An insulating dielectric separates the inner and outer conductors, and the complete assembly is enclosed in a protective outer sheath. Coaxial cables are not susceptible to external electric and magnetic fields and generate no electric or magnetic fields of their own.

Code A system of communications that uses arbitrary groups of symbols to represent information or instructions. A set of programmed instructions.

Coil Represents the output of a programmable logic controller. In the output devices, it is the electrical coil that, when energized, changes the status of its corresponding contacts.

Coil format Format that uses coils to display instructions.

Comment Text that is included with each PLC ladder rung and is used to help individuals understand how the program operates or how the rung interacts with the rest of the program.

Communication module Allows the user to connect the PLC to high-speed local networks that may differ from the network communication provided with the PLC.

Compare An instruction that compares the contents of two designated data memory locations of a programmable logic controller for equality or inequality.

Compatibility The ability of various specified units to replace one another with little or no reduction in capability; the ability of units to be interconnected and used without modification.

Complement A logical operation that inverts a signal or bit.

Complementary metal-oxide semiconductor (CMOS) A logic base that offers lower power consumption and high-speed operation.

Computer Any electronic device that can accept information, manipulate it according to a set of preprogrammed instructions, and supply the results of the manipulation.

Computer integrated manufacturing (CIM) A manufacturing system controlled by an easily reprogrammable computer for flexibility and speed of changeover.

Computer interface A device designed for data communication between a programmable logic controller and a computer.

Consumed tag References data that come from another controller.

Contact The current-carrying part of an electric relay or switch. The contact engages to permit power flow and disengages to interrupt power flow to a load device.

Contact bounce The uncontrollable making and breaking of a contact during the initial engaging or disengaging of the contact.

Contact histogram An instruction sequence that monitors a designated memory bit or a designated input or output point for a change of state. A listing is generated by the instruction sequence that displays how quickly the monitored point is changing state.

Contactor A special-purpose relay designed to establish and interrupt the power flow of high-current electric circuits.

Contact symbology A set of symbols used to express the control program with conventional relay symbols.

Continuous current per module The maximum current for each module. The sum of the output current for each point should not exceed this value.

Continuous current per point The maximum current each output is designed to supply continuously to a load.

Controlled variable The output variable that the automatic control adjusts to keep the process value at a set-point.

Control logic The control plan for a given system; the program.

Control loop The method of adjusting the control variable in a process control system by analyzing the process variable data and then comparing it to the set-point to determine the amount of error in the system.

ControlNet An open, high-speed, deterministic network that transfers on the same network time-critical I/O updates, controller-to-controller interlocking data, and non-time-critical data such as data monitoring and program uploads and downloads.

Control relay A relay used to control the operation of an event or a sequence of events.

Counter An electromechanical device in relay-based control systems that counts numbers of events for the purpose of controlling other devices based on the current number of counts recorded; a programmable logic controller instruction that performs the functions of its electromechanical counterpart.

Cross reference In ladder diagrams, letters or numbers to the right of coils or functions. The letters or numbers indicate where on other ladder lines contacts of the coil or function are located.

Crosstalk Undesired energy appearing in one signal path as a result of coupling from other signal paths or use of a common return line.

Current The rate of electrical electron movement, measured in amperes.

Current-carrying capacity The maximum amount of current a conductor can carry without heating beyond a predetermined safe limit.

Current sinking Refers to an output device (typically an NPN transistor) that allows current flow from the load through the output to ground.

Current sourcing Output device (typically a PNP transistor) that allows current flow from the output through the load and then to ground.

Cycle A sequence of operations repeated regularly; the time it takes for one such sequence to occur.

D

Data Information encoded in a digital form, which is stored in an assigned address of data memory for later use by the processor.

Data address A location in memory where data can be stored.

Data file A group of data memory words acted on as a group rather than singly.

Data highway A communications network that allows devices such as PLCs to communicate. They are normally proprietary, which means that only like devices of the same brand can communicate over the highway.

Data latching A technique used to read the value of the input data that will be operated on by the instructions with a function block.

Data link The equipment that makes up a data communications network.

Data manipulation The process of exchanging, altering, or moving data within a programmable logic controller or between programmable logic controllers.

Data manipulation instructions A classification of processor instructions that alter, exchange, move, or otherwise modify data memory words.

Data table The part of processor memory that contains input and output values as well as files where data are monitored, manipulated, and changed for control purposes.

Data transfer The process of moving information from one location to another, in other words, from register to register, from device to device, and so forth.

Data transmission line A medium for transferring signals over a distance.

Debouncing The act of removing intermediate noise states from a mechanical switch.

Debug The process of locating and removing mistakes from a software program or from hardware interconnections.

Decimal number system A number system that uses ten numeral digits (decimal digits): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. Each digit position has a place value of 1, 10, 100, 1000, and so on, beginning with the least significant (rightmost) digit; base 10.

Decrement The act of reducing the contents of a storage location or value in varying increments.

Determinism The ability to reliably predict when data will be delivered.

DeviceNet An open communication network designed to connect factory-floor devices together without interfacing through an I/O system. Up to 64 intelligent nodes can be connected to one DeviceNet network.

Diagnostic program A user program designed to help isolate hardware malfunctions in the programmable logic controller and the application equipment.

Diagnostics The detection and isolation of an error or malfunction.

Digital device One that processes discrete electric signals.

Digital gate A device that analyzes the digital states of its inputs and outputs and an appropriate output state.

Digital signals A system of discrete states: high or low, on or off, 1 or 0.

Digital-to-analog converter An electrical circuit that converts binary bits to a representative, continuous, analog signal.

DINT A data type that stores a 32-bit (4-byte) signed integer value.

DIP switch A group of small, in-line on-off switches. From *d*ual *i*n-line *p*ackage.

Direct addressing An addressing mode in which the memory address of the data is supplied with the instruction.

Discrete I/O A group of input and/or output modules that operate with on/off signals, as contrasted to analog modules that operate with continuously variable signals.

Disk drive The device that writes or reads data from a magnetic disk.

Diskette The flat, flexible disk on which a disk drive writes and reads.

Display The image that appears on a cathode-ray tube screen or on other image projection systems.

Display menu The list of displays from which the user selects specific information for viewing.

Distributed control A method of dividing process control into several subsystems. A PLC oversees the entire operation.

Divide A programmable logic controller instruction that performs a numerical division of one number by another.

Documentation An orderly collection of recorded hardware and software data such as tables, listings, and diagrams to provide reference information for programmable logic controller application operation and maintenance.

Done bit (DN) Bit that is set to 1 when the instruction has completed its task, such as reaching its preset value.

Double precision The system of using two addresses or registers to display a number too large for one address or register; allows the display of more significant figures because twice as many bits are used.

Down-counter A counter that starts from a specified number and increments down to zero.

Download Loading data from a master listing to a readout or another position in a computer system.

Dry-contact-output module Enables a PLC's processor to control output devices by providing a contact isolated electrically from any power source.

E

Edit The act of modifying a programmable logic controller program to eliminate mistakes and/or simplify or change system operation.

Electrically erasable programmable read-only memory (EEPROM) A type of programmable read-only memory that is programmed and erased by electrical pulses.

Electrical optical isolator A device that couples input to output using a semiconductor light source and detector in the same package.

Electromagnetic interference (EMI) A phenomenon responsible for noise in electric circuits.

Element A single instruction of a relay ladder diagram program.

Emergency stop relay A relay used to inhibit all electric power to a control system in an emergency or other event requiring that the controlled hardware be brought to an immediate halt.

Enable To permit a particular function or operation to occur under natural or preprogrammed conditions.

Enclosure A steel box with a removable cover or hinged door used to house electric equipment.

Encoder A rotary device that transmits position information; a device that transmits a fixed number of pulses for each revolution.

Energize The physical application of power to a circuit or device to activate it; the act of setting the on, true, or 1 state of a programmable logic controller's relay ladder diagram output device or instruction.

Erasable programmable read-only memory (EPROM) A programmable read-only memory that can be erased with ultraviolet light, then reprogrammed with electrical pulses.

Error signal A signal proportional to the difference between the actual output and the desired output.

Ethernet A network type or protocol that uses the Carrier Sense Multiple Access Collision Detection (CSMA/CD) network access method.

Ethernet/IP An open industrial networking standard that takes advantage of commercial off-the-shelf Ethernet communication devices and physical media; IP refers to industrial protocol.

Even parity When the sum of the number of 1s in a binary word is always even.

Examine if closed (XIC) Refers to a normally open contact instruction in a logic ladder program. An examine if closed instruction is true if its addressed bit is on (1). It is false if the bit is off (0).

Examine if open (XIO) Refers to a normally closed contact instruction in a logic ladder program. An examine if open instruction is true if its addressed bit is off (0). It is false if the bit is on (1).

Exclusive-OR gate A logic device requiring one or the other, but not both, of its inputs to be satisfied before activating its output.

Execution The performance of a specific operation accomplished through processing one instruction, a series of instructions, or a complete program.

Execution time The total time required for the execution of one specific operation.

F

False As related to programmable logic controller instructions, a disabling logic state.

Fault Any malfunction that interferes with normal operation.

Fault indicator A diagnostic aid that provides a visual indication and/or an internal processor software indication that a fault is present in the system.

Fault-routine file A special subroutine that, if assigned, executes when the processor has a major fault.

Feedback In analog systems, a correcting signal received from the output or an output monitor. The correcting signal is fed to the controller for process correction.

Fiber optic cable Transmits information via light pulses down optical fibers.

Fieldbus An open, all-digital, serial, two-way communications system that interconnects measurement and control equipment such as sensors, actuators, and controllers.

File A formatted block of data treated as a unit.

Fixed I/O Input/output terminals on a programmable logic controller that are built into the unit and are not changeable. A fixed I/O PLC has no removable modules.

Floating-point data file Used to store integers and other numerical values that cannot be stored in an integer file.

Flowchart A graphical representation for the definition, analysis, or solution of a problem. Symbols are used to represent a process or sequence of decisions and events.

Force function A mode of operation or instruction that allows an operator to override the processor to control the state of a device.

Force off function A feature that allows the user to reset an input image table file bit or de-energize an output independently of the programmable logic controller program.

Force on function A feature that allows the user to set an image table file bit or energize an output independently of the programmable logic controller program.

Full duplex A mode of data communications in which data may be transmitted and received simultaneously.

Function block Rectangular block with inputs entering from the left and outputs exiting on the right.

Function block diagram (FBD) Graphical language where the basic programming elements appear as blocks.

Function keys Keys on a personal computer, electronic operator device, or hand-held programmer keyboard that are labeled F1, F2, and so on. The operation of each of these keys is defined on many electronic operator interface devices.

G

Gate A circuit having two or more input terminals and one output terminal, where an output is present when and only when the prescribed inputs are present.

Gateway A device or pair of devices that connects two or more communication networks. This device may act as a host to each network and may transfer messages between the networks by translating their protocols.

Glitch A voltage or current spike of short duration that adversely affects the operation of a PLC.

Gray code A binary coding scheme that allows only 1 bit in the data word to change state at each increment of the code sequence.

Gray-encoder module Converts the Gray-code signal from an input device into straight binary.

Ground A conducting connection between an electric circuit or equipment chassis and the earth ground.

Ground loop A condition in which two or more electrical paths exist within a ground line.

Ground potential Zero voltage potential with respect to the ground.

H

Half-duplex A mode of data transmission that communicates in two directions but in only one direction at a time.

Handshaking The method by which two digital machines establish communication.

Hard contacts Any type of physical switch contacts.

Hard copy Any form of a printed document such as a ladder diagram program listing, paper tape, or punched cards.

Hard drive An inflexible recording disk used as a computer disk drive.

Hardware The mechanical, electric, and electronic devices that make up a programmable logic controller and its application.

Hardwired The physical interconnection of electric and electronic components with wire.

Hexadecimal A number system having a base of 16. This numbering system requires 16 elements for representation, and thus uses the decimal digits zero (0) through nine (9) and the first six letters of the alphabet, A through F.

High-speed counter encoder module A module that enables you to count and encode faster than you could with a regular control program written on a PLC in which the control program's execution is too slow.

Histogram A graphic representation of the frequency at which an event occurs.

Host computer A main computer that controls other computers, PLCs, or computer peripherals.

Human machine interface (HMI) Graphical display hardware in which machine status, alarms, messages, diagnostics, and data entry are available to the operator in graphical display format.

I

IEC 1131 programming standard The international standard for programmable controller programming languages.

Image table An area in programmable logic controller memory dedicated to input/output data. Ones and zeros (1s and 0s) represent on and off conditions, respectively. During every input/output scan, each input controls a bit in the input image table file; each output is controlled by a bit in the output image table file.

Immediate input instruction A programmable logic controller instruction that temporarily halts the user program scan so that the processor can update the input image table file with the current status of one or more user-specified input points.

Immediate output instruction A programmable logic controller instruction that temporarily halts the user program scan so that the current status of one or more user-specified output points can be updated to current output image table file status by the processor.

Impedance The total resistive and inductive opposition that an electric circuit or device offers to a varying current at a specified frequency. Impedance is measured in ohms (V) and is denoted by the symbol Z.

Increment The act of increasing the contents of a storage location or value in varying amounts.

Index A reference used to specify an element within an array.

Indirect addressing An addressing mode in which the address of the instruction serves as a reference point-instead of the actual address.

Inductance A circuit property that opposes any current change. Inductance is measured in henries and is represented by the letter H.

Industrial terminal The device used to enter and monitor the program in a PLC.

Input Information transmitted from a peripheral device to the input module and then to the data table.

Input devices Devices such as limit switches, pressure switches, pushbuttons, and analog and/or digital devices that supply data to a programmable logic controller.

Input/output (I/O) address A unique number assigned to each input and output. The address number is used when programming, monitoring, or modifying a specific input or output.

Input/output (I/O) module A plug-in assembly that contains more than one input or output circuit. A module usually contains two or more identical circuits. Normally, it contains 2, 4, 8, 16, 32, or 64 circuits.

Input/output (I/O) scan time The time required for the processor to monitor inputs and control outputs.

Input/output (I/O) update The continuous process of revising each and every bit in the input and output tables, based on the latest results

from reading the inputs and processing the outputs according to the control program.

Input scan One of three parts of the PLC scan. During the input scan, input terminals are read and the input table is updated accordingly.

Instruction A command that causes a programmable logic controller to perform one specific operation. The user enters a combination of instructions into the programmable logic controller's memory to form a unique application program.

Instruction set The set of general-purpose instructions available with a given controller. In general, different machines have different instruction sets.

INT Two-byte integer.

Integer A positive or negative whole number.

Integrated circuit (IC) A circuit in which all components are integrated on a single tiny silicon chip.

Intelligent field devices Microprocessor-based devices used to provide process-variable, performance, and diagnostic information to the PLC processor. These devices are able to execute their assigned control functions with little interaction, except communications, with their host processor.

Intelligent input/output module A microprocessor-based module that performs processing or sophisticated closed-loop application functions.

Interface A circuit that permits communication between the central processing unit and a field input or output device. Different devices require different interfaces.

Interlock A system for preventing one element or device from turning on while another device is on.

Internal coil instruction A relay coil instruction used for internal storage or buffering of an on/off logic state. An internal coil instruction differs from an output coil instruction because the on/off status of the internal coil is not passed to the input/output hardware for control of a field device.

Inversion Conversion of a high level to a low level, or vice versa.

Inverter The digital circuit that performs inversion.

I/O module A plug-in assembly, containing two or more identical input or output circuits, that contain the connections between a processor and connected devices.

Interrupt The act of redirecting a program's execution to perform a more urgent task.

IP address A specified Internet protocol address, unique and assigned by the manufacturer, for every Ethernet device.

Isolated input module A module that receives dry contacts as inputs, which the processor can recognize and change into two-state digital signals.

Isolated input/output (I/O) circuits Input and output circuits that are electrically isolated from any and all other circuits of a module. Isolated input/output circuits are designed to allow field devices that are powered from different sources to be connected to one module.

J

Jumper A short length of conduit used to make a connection between terminals around a break in a circuit.

Jump instruction An instruction that permits the bypassing of selected portions of the user program. Jump instructions are conditional whenever their operation is determined by a set of preconditions and unconditional whenever they are executed to occur every time they are programmed.

K

K $2^{10} = 1K = 1024$; used to denote size of memory and can be expressed in bits, bytes, or words; example: $2K = 2048$.

k Kilo; a prefix used with units of measurement to designate quantities 1000 times as great.

Keyboard The alphanumeric keypad on which the user types instructions to the PLC.

Keying Bands installed on backplane connectors to ensure that only one type of module can be inserted into a keyed connector.

L

Label instruction A programmable logic controller instruction that assigns an alphanumeric designation to a particular location in a program. This location is used as the target of a jump, skip, or jump to subroutine instruction.

Ladder diagram An industry standard for representing relay logic control systems. The diagram resembles a ladder because the vertical supports of the ladder appear as power feed and return buses and the horizontal rungs of the ladder appear as series and/or parallel circuits connected across the power lines.

Ladder diagram programming A method of writing a user program in a format similar to a relay ladder diagram.

Ladder matrix A rectangular array of programmed contacts that defines the number of contacts that can be programmed across a row and the number of parallel branches allowed in a single ladder rung.

Language A set of symbols and rules for representing and communicating information among people or between people and machines; the method used to instruct a programmable device to perform various operations.

Language module Enables the user to write programs in a high-level language. BASIC is the most popular language module. Other language modules available include C, Forth, and PASCAL.

Latching relay A relay that maintains a given position by mechanical or electrical means until released mechanically or electrically.

Latch instruction One-half of an instruction pair (the second instruction of the pair being the unlatch instruction) that emulates the latching action of a latching relay. The latch instruction for a programmable logic controller energizes a specified output point or internal coil until it is de-energized by a corresponding unlatch instruction.

Leakage The small amount of current that flows in a semiconductor device when it is in the off state.

Least significant bit (LSB) The bit that represents the smallest value in a byte or word.

Least significant digit (LSD) The digit that represents the smallest value in a byte or word.

Light-emitting diode (LED) A semiconductor junction that emits light when biased in the forward direction.

Light-emitting diode (LED) display A display device incorporating light-emitting diodes to form the segments of the displayed characters and numbers.

Limit switch An electric switch actuated by some part and/or motion of a machine or equipment.

Limit test A test that determines if a value is inside or outside a specified range.

Line A component part of a system used to link various subsystems located remotely from the processor; the source of power for operation; example: 120 V alternating current line.

Line-powered sensor Normally, three-wire sensors, although four-wire sensors also exist. The line-powered sensor is powered from the power supply. A separate wire (the third) is used for the output line.

Liquid-crystal display (LCD) A display device using reflected light from liquid crystals to form the segments of the displayed characters and numbers.

Load The power used by a machine or apparatus; to place data into an internal register under program control; to place a program from an external storage device into central memory under operator control.

Load-powered sensor A two-wire sensor. A small leakage current flows through the sensor even when the output is off. The current is required to operate the sensor electronics.

Local area network (LAN) A system of hardware and software designed to allow a group of intelligent devices to communicate within a fairly close proximity.

Local input/output (I/O) A programmable logic controller whose input/output distance is physically limited. The PLC must be located near the process; however, the PLC may still be mounted in a separate enclosure.

Local power supply The power supply used to provide power to the processor and a limited number of local input/output modules.

Location In reference to memory, a storage position or register identified by a unique address.

Logic A process of solving complex problems through the repeated use of simple functions that can be either true or false. The three basic logic functions are AND, OR, and NOT.

Logic diagram A diagram that represents the logic elements and their interconnections.

Logic level The voltage magnitude associated with signal pulses representing 1s and 0s in binary computation.

Loop control A control of a process or machine that uses feedback. An output status indicator modifies the input signal effect on the process control.

M

Machine language A programmable language using the binary form.

Major fault A fault condition that is severe enough for the controller to shut down, unless the condition is cleared.

Manufacturing automation protocol (MAP) Standard developed to make industrial devices communicate more easily.

Masking A means of selectively screening out data. Masking allows unused bits in a specific instruction to be used independently.

Mass storage A means of storing large amounts of data on magnetic tape, floppy disks, and so on.

Master control relay (MCR) A mandatory hardwired relay that can be de-energized by any series-connected emergency stop switch. Whenever the master control relay is de-energized, its contacts open to de-energize all application input and output devices.

Master control relay (MCR) zones User program areas in which all nonretentive outputs can be turned off simultaneously. Each master control relay zone must be delimited and controlled by master control relay fence codes (master control relay instructions).

Matrix A logic network that is an intersection of input and output connection points.

Memory That part of the programmable logic controller in which data and instructions are stored either temporarily or semi-permanently. The control program is stored in memory.

Memory map A diagram showing a system's memory addresses and what programs and data are assigned to each section of memory.

Menu A list of programming selections displayed on a programming terminal.

Metal-oxide semiconductor (MOS) A semiconductor device in which an electric field controls the conductance of a channel under a metal electrode called a *gate.*

Metal-oxide varistor (MOV) Used for suppressing electrical power surges.

Microprocessor A central processing unit manufactured on a single integrated-circuit chip (or several chips) by utilizing large-scale integration technology.

Microsecond One millionth of a second $= 1 \times 10^{-6}$ second $=$ 0.000001 second.

Millisecond One thousandth of a second $= 1 \times 10^{-3}$ second $=$ 0.001 second.

Mnemonic A term, usually an abbreviation, that is easy to remember and pronounce.

Mnemonic code A code in which information is represented by symbols or characters.

Modbus A network that uses a master/slave communication technique.

Mode A term used to refer to the selected operating method, such as automatic, manual, TEST, PROGRAM, or diagnostic.

Module An interchangeable, plug-in item containing electronic components.

Module addressing A method of identifying the input/output modules installed in a chassis.

Most significant bit (MSB) The bit representing the greatest value of a byte or word.

Most significant digit (MSD) The digit representing the greatest value of a byte or word.

Motor controller or starter A device or group of devices that serve to govern, in a predetermined manner, the electric power delivered to a motor.

Motor starter A special relay designed to provide power to motors; it has both a contactor relay and an overload relay connected in series and prewired so that, if the overload operates, the contactor is de-energized.

Move instruction A programmable logic controller instruction that moves data from one location to another. Although a move instruction typically places the data in a new location, the original data still reside in their original location.

Multiplexing The time-shared scanning of a number of data lines into a single channel, and only one data line is enabled at any time; the incorporation of two or more signals into a single wave from which the individual signals can be recovered.

Multiply instruction A programmable logic controller instruction that provides for the mathematical multiplication of two numbers.

Multiprocessing A method of applying more than one microprocessor to a specific function to speed up operation time and reduce the possibility of system failure.

N

National Electrical Code (NEC) A set of regulations developed by the National Fire Protection Association that governs the construction and installation of electric wiring and electric devices. The National Electrical Code is recognized by many governmental bodies, and compliance is mandatory in much of the United States.

National Electrical Manufacturers Association (NEMA) An organization of electric device and product manufacturers. The National Electrical Manufacturers Association issues standards relating to the design and construction of electric devices and products.

NEMA Type 12 enclosure A category of industrial enclosures intended for indoor use and designed to provide a degree of protection against dust, falling dirt, and dripping noncorrosive liquids. They do not provide protection against conditions such as internal condensation.

Nested branches A branch that begins or ends within another branch.

Network A series of stations or devices connected by some type of communications medium.

Network access control The method of accessing the network media (cable) to ensure that data are transmitted in an organized manner in order to reduce the possibilities of data corruption.

Node In hardware, a connection point on the network; in programming, the smallest possible increment in a ladder diagram.

Noise Random, unwanted electric signals, normally caused by radio waves or electric or magnetic fields generated by one conductor and picked up by another.

Noise filter or **noise suppressor** An electronic filter network used to reduce and/or eliminate any noise that may be present on the leads to an electric or electronic device.

Noise immunity A measure of insensitivity of an electronic system to noise.

Noise spike A short burst of electric noise with more magnitude than the background noise level.

Nonretentive output An output controlled continuously by a program rung. Whenever the rung changes state (true or false), the output turns on or off; contrasted with a retentive output, which remains in its last state (on or off) depending on which of its two rungs, latch or unlatch, was last true.

Nonvolatile memory A memory designed to retain its data while its power supply is turned off.

NOR The logic gate that results in zero unless both inputs are zero.

Normally closed contact (NC) A contact that is conductive when its operating coil is not energized.

Normally open contact (NO) A contact that is nonconductive when its operating coil is not energized.

NOT A logical operation that yields a logic 1 at the output if a logic 0 is entered at the input, and a logic 0 at the output if a logic 1 is entered at the input. The NOT, also called the *inverter,* is normally used in conjunction with the AND and OR functions.

O

Octal number system A base eight numbering system that uses numbers 0–7, 10–17, 20–27, and so on. There are no 8s or 9s in the octal number system.

Odd parity Condition when the sum of the number of 1s in a binary word is always odd.

Off-delay timer An electromechanical relay with contacts that change state a predetermined time period after power is removed from its coil; on re-energization of the coil, the contacts return to their shelf state immediately; also, a programmable logic controller instruction that emulates the operation of the electromechanical off-delay relay.

Offline programming and/or **offline editing** A method of programmable logic controller programming and/or editing in which the operation of the processor is stopped and all output devices are switched off. Offline programming is the safest way to develop or edit a programmable logic controller program since the entry of instructions does not affect operating hardware until the program can be verified for accuracy of entry.

On-delay timer An electromechanical relay with contacts that change state a predetermined time period after the coil is energized; the contacts return to their shelf state immediately on de-energization of the coil; also, a programmable logic controller instruction that emulates the operation of the electromechanical on-delay timer.

One-shot A programmed technique that sets a storage bit or output for only one program scan.

Online data change Allows the user to change various data table values using a peripheral device while the application is operating normally.

Online programming and/or **online editing** The ability of a processor and programming terminal to make joint user-directed additions, deletions, or changes to a user program while the processor is actively solving and executing the commands of the existing user program. Extreme care should be exercised when performing online programming to ensure that erroneous system operation does not result.

Open loop A system that has no feedback or auto correction.

Operand A number used in an arithmetic operation as an input.

Operational amplifier (op-amp) A high-gain DC amplifier used to increase signal strength for devices such as analog input modules.

Optical coupler A device that couples signals from one circuit to another by means of electromagnetic radiation, usually infrared or visible. A typical optical coupler uses a light-emitting diode to convert the electric signal of the primary circuit into light and uses a photo-transistor in the secondary circuit to reconvert the light back into an electric signal; sometimes referred to as *optical isolation*.

Optical isolation Electrical separation of two circuits with the use of an optical coupler.

OR A logical operation that yields a logic 1 output if one of any number of inputs is 1, and a logic 0 if all inputs are 0.

Output Information sent from the processor to a connected device via some interface. The information could be in the form of control data that will signal some device such as a motor to switch on or off or to vary the speed of a drive.

Output device Any connected equipment that will receive information or instructions from the central processing unit, such as control devices (e.g., motors, solenoids, alarms) or peripheral devices (e.g., line printers, disk drives, displays). Each type of output device has a unique interface to the processor.

Output image table file A portion of a processor's data memory reserved for the storage of output device statuses. A 1, on, or true state in an output image table file storage location is used to switch on the corresponding output point.

Output instruction The term applied to any programmable logic controller instruction capable of controlling the discrete or analog status of an output device connected to the programmable logic controller.

Output register or **output word** A particular word in a processor's output image table file in which numerical data are placed for transmission to a field output device.

Output scan One of three parts of the PLC scan. During the output scan, data associated with the output status table are transferred to the output terminals.

Overflow Exceeding the numerical capacity of a device such as a timer or counter. The overflow can be either a positive or negative value.

Overload A load greater than the one that a component or system is designed to handle.

Overload relay A special-purpose relay designed so that its contacts transfer whenever its current exceeds a predetermined value. Overload relays are used with electric motors to prevent motor burnout due to mechanical overload.

P

Parallel circuit A circuit in which two or more of the connected components or contact symbols in a ladder program are connected to the same pair of terminals so that current may flow through all the branches; contrasted with a series connection, in which the parts are connected end to end so that current flow has only one path.

Parallel instruction A programmable logic controller instruction used to begin and/or end a parallel branch of instructions programmed on a programming terminal.

Parallel operation A type of information transfer in which all bits, bytes, or words are handled simultaneously.

Parallel transmission A computer operation in which two or more bits of information are transmitted simultaneously.

Parity The use of a self-checking code that employs binary digits in which the total number of 1s is always even or odd.

PC Personal computer.

Peer-to-peer network A network in which nodes are given an equal chance of initiating and controlling communications.

Peripheral equipment Units that communicate with the programmable logic controller but are not part of the programmable logic controller; example: a programming device or computer.

PID Proportional-integral-derivative closed-loop control that lets the user hold a process variable at a desired set-point.

Pilot-type device Used in a circuit as a control apparatus to carry electric signals for directing performance. This device does not carry primary current.

PLC processor A computer designed specifically for programmable controllers. It supervises the action of the modules attached to it.

Polarity The directional indication of electrical flow in a circuit; the indication of charge as either positive or negative, or the indication of a magnetic pole as either north or south.

Polling A network access method where a master controller manages the communication process by interrogating each slave controller under it to determine whether the slave has any information to send.

Port A connector or terminal strip used to access a system or circuit. Generally, ports are used for the connection of peripheral equipment.

Power supply The unit that supplies the necessary voltage and current to a system's circuitry.

Preset value (PRE) The number of time intervals or events to be counted.

Pressure switch A switch activated at a specified pressure.

Process A continuous manufacturing operation.

Program A sequence of instructions to be executed by the processor to control a machine or process.

Program files The area of processor memory in which the ladder logic programming is stored.

Programmable controller A computer that has been hardened to work in an industrial environment and is equipped with special I/O and a control programming language.

Programmable read-only memory (PROM) A retentive memory used to store data. This type of memory device can be programmed only once and cannot be altered afterward.

Programming terminal A combination of keyboard and monitor used to insert, modify, and observe programs stored in a PLC.

Program scan One of three parts of the PLC scan. During the program scan, the CPU scans each rung of the user program.

Project file Contains all data associated with the PLC project. A project comprises five major pieces: help folder, controller folder, ladder folder, data folder, and data base folder.

Proportional-integral-derivative (PID) A mathematical formula that provides a closed-loop control of a process. Inputs and outputs are continuously variable and typically will be analog signals.

Protocol A formal definition of criteria for receiving and transmitting data through communications channels.

Proximity switch An input device that senses the presence or absence of a target without physical contact.

Pulse A short change in the value of a voltage or current level. A pulse has a definite rise and fall time and a finite duration.

R

Rack A housing or framework used to hold assemblies; a plastic and/or metal assembly that supports input/output modules and provides a means of supplying power and signals to each input/output module or card.

Random-access memory (RAM) A memory system that permits the random accessing of any storage location for the purpose of either storing (writing) or retrieving (reading) information. Random-access memory systems allow the data to be retrieved and stored at speeds independent of the storage locations being accessed.

Read The accessing of information from a memory system or data storage device; the gathering of information from an input device or devices or a peripheral device.

Read-only memory (ROM) A permanent memory structure in which data are placed at time of fabrication or by the user at a speed much slower than it will be read. Information entered in a read-only memory is usually not changed once it is entered.

Read/write memory Memory in which data can be stored (write mode) or accessed (read mode). The write mode replaces previously stored data with current data; the read mode does not alter stored data.

Real numbers Numbers that have both integer and fractional parts.

Real-time clock (RTC) A device that continually measures time in a system without respect to what tasks the system is performing.

Rectifier A solid-state device that converts alternating current to pulsed direct current.

Register A memory word or area for the temporary storage of data used within mathematical, logical, or transfer functions.

Relay An electrically operated device that mechanically switches electric circuits.

Relay contacts The contacts of a relay that are either opened or closed according to the condition of the relay coil. Relay contacts are designated as either normally open or normally closed in design.

Relay logic A representation of the program or other logic in a form normally used for relays.

Remote input/output (I/O) system Any input/output system that permits communication between the processor and input/output hardware over a coaxial or twin axial cable. Remote input/output systems permit the placement of input/output hardware at any distance from the processor.

Resolution The smallest distinguishable increment into which a quantity is divided.

Response time The amount of time required for a device to react to a change in its input signal or to a request.

Retentive instruction Any programmable logic controller instruction that does not need to be continuously controlled for operation. Loss of power to the instruction does not halt execution or operation of the instruction.

Retentive timer An electromechanical relay that accumulates time whenever the device receives power and maintains the current time should power be removed from the device. Loss of power to the device after reaching its preset value does not affect the state of the contacts.

Retentive timer instruction A programmable logic controller instruction that emulates the timing operation of the electromechanical retentive timer.

Retentive timer reset instruction A programmable logic controller instruction that emulates the reset operation of the electromechanical retentive timer.

Ring topology A network topology that that forms a data path in a ring.

Routine A series of instructions that perform a specific function or task.

RS-232 An Electronic Industries Association (EIA) standard for data transfer and communication for serial binary communication circuits.

Run The single, continuous execution of a program by a programmable logic controller.

Rung A group of programmable logic controller instructions that controls an output or storage bit, or performs other control functions such as file moves, arithmetic, and/or sequencer instructions. A rung is represented as one section of a ladder logic diagram.

S

SCADA An acronym for supervisory control and data acquisition.

Scan time The time required to read all inputs, execute the control program, and update local and remote input and output statuses. Scan time is, in effect, the time required to activate an output controlled by programmed logic.

Schematic A diagram of graphic symbols representing the electrical scheme of a circuit.

Search function Allows the user to display quickly any instruction in the programmable logic controller program.

Self-diagnostic The hardware and firmware within a controller that monitors its own operation and indicates any fault it can detect.

Sensor A device used to gather information by the conversion of a physical occurrence to an electric signal.

Sequencer A mechanical, electric, or electronic device that can be programmed so that a predetermined set of events occurs repeatedly.

Sequence table A table or chart indicating the sequence of operation of output devices.

Sequential control A process that dictates the correct order of events and allows one event to occur only after the completion of another.

Sequential Function Chart (SFC) Graphical language whose basic language elements are steps or states with associated actions and transitions with associated conditions used to move from the current state to the next.

Serial communication A type of information transfer in which the bits are handled sequentially; contrasted with parallel communication.

Series circuit A circuit in which the components or contact symbols are connected end to end, and all must be closed to permit current flow.

Servo module The device whose feedback is used to accomplish closed-loop control. Though programmed through a PLC, once programmed it can control a device independently without interfering with the PLC's normal operation.

Set-point The value that the process value is to be held to by the automatic control function.

Shield A barrier, usually conductive, that substantially reduces the effect of electric and/or magnetic fields.

Shift To move binary data within a shift register or other storage device.

Shift register A PLC function capable of storing and shifting binary data.

Short circuit An undesirable path of very low resistance in a circuit between two points.

Short-circuit protection Any fuse, circuit breaker, or electronic hardware used to protect a circuit or device from severe overcurrent conditions or short circuits.

Signal The event or electrical quantity that conveys information from one point to another.

Significant digit A digit that contributes to the precision of a number. The number of significant digits is counted beginning with the digit contributing the most value, called the *most significant digit* (leftmost), and ending with the digit contributing the least value, called the *least significant digit* (rightmost).

Silicon-controlled rectifier (SCR) A semiconductor device that functions as an electronic switch.

Single-scan function A supervisory instruction that causes the control program to be executed for one scan, including input/output update. This troubleshooting function allows step-by-step inspection of occurrences while the machine is stopped.

Sink mode output A mode of operation of solid-state devices in which the device controls the current from the load. For example, when the output is energized, it connects the load to the negative polarity of the supply.

SINT A data type that stores an 8-bit (1-byte) signed integer value.

Snubber A circuit generally used to suppress inductive loads; it consists of a resistor in series with a capacitor (RC snubber) and/or a MOV placed across the alternating current load.

Software Programs that control the processing of data in a system, as contrasted to the physical equipment itself (hardware).

Solid-state switch Any electronic device incorporating a transistor, silicon-controlled rectifier, or triac semiconductor switch to control the on/off flow of electric power.

Source mode output A mode of operation of solid-state output devices in which the device controls the current to the load. For example, when the output is energized, it connects the load to the positive polarity of the supply.

Star topology A network architecture in which all network nodes are connected to a central device that routes the nodes' messages.

State The logic 0 or 1 condition in programmable logic controller memory or at a circuit input or output.

Station Any programmable logic controller, computer, or data terminal connected to, and communicating by means of, a data highway.

Status indicators LEDs that indicate the on-off status of an input or output point and are visible on the outside of the PLC.

Stepper-motor module Provides pulse trains to a stepper-motor translator that enables control of a stepper motor.

STI An acronym for selectable time interrupt, a subroutine that executes on a time basis rather than an event basis.

Storage bit A bit in a data table word that can be set or reset but that is not associated with a physical input or output terminal point.

Structure Text (ST) High-level, text-based language with commands that support a highly structured program development and the ability to evaluate complex mathematical expressions.

Subroutines Program files that are scanned only when called on by logic and can be used to break the program into smaller segments.

Subtract A programmable logic controller instruction that performs the mathematical subtraction of one number from another.

Suppression device A unit that attenuates the magnitude of electrical noise.

Surge A transient wave of current or power.

Synchronous shift register A shift register in which only one change of state occurs per control pulse.

Synchronous transmission A type of serial transmission that maintains a constant time interval between successive events.

T

Tag A text-based name for an area of the controller's memory where data are stored.

Tap A device that provides mechanical and electrical connections to a trunk cable. A tap allows the signals on the trunk to be passed to a station and the signals transmitted by the stations to be passed to the trunk.

Task It holds the information necessary to schedule the program's execution and sets the execution priority for one or more programs.

Terminal address The alphanumeric address assigned to a particular input or output point. It is also related directly to a specific image table bit address.

Thermocouple A temperature-measuring device that utilizes two dissimilar metals for temperature measurement. As the junction of the two dissimilar metals is heated, a proportional voltage difference, which can be measured, is generated.

Thumbwheel switch A rotating switch used to input numeric information into a controller.

Time base A unit of time generated by a microprocessor's clock circuit and used by PLC timer instructions. Typical time bases are 0.01, 0.1, and 1.0 second.

Timed contact A normally open and/or normally closed contact that is actuated at the end of a timer's time-delay period.

Timer In relay-panel hardware, an electromechanical device that can be wired and preset to control the operating interval of other devices. In a programmable logic controller, a timer is internal to the *processor;* that is, it is controlled by a user-programmed instruction.

Toggle switch A panel-mounted switch with an extended lever; normally used for on/off switching.

Token The logical right to initiate communications in a communication network.

Token passing A technique in which tokens are circulated among nodes in a communication network.

Topology The structure of a communications network; examples are bus, ring, and star.

Transducer A device used to convert physical parameters such as temperature, pressure, and weight into electric signals.

Transformer An electric device that converts a circuit's electrical energy into a circuit or circuits with different voltages and current ratings.

Transistor A three-terminal active semiconductor device composed of silicon or germanium that is capable of switching or amplifying an electric current.

Transistor-transistor logic (TTL) A semiconductor logic family in which the basic logic element is a multiple-emitter transistor. This family of devices is characterized by high speed and medium power dissipation.

Transitional contact A contact that, depending on how it is programmed, will be on for one program scan every 0 to 1 transition, or every 1 to 0 transition, of the referenced coil.

Transmission line A system of one or more electric conductors used to transmit electric signals or power from one place to another.

Triac A solid-state component capable of switching alternating current.

True As related to programmable logic controller instructions, an on, enabled, or 1 state.

Truth table A table listing that shows the state of a given output as a function of all possible input combinations.

TTL-input module Enables devices that produce TTL-level signals to communicate with a PLC's processor.

TTL-output module Enables a PLC to operate devices requiring TTL-level signals to operate.

Twisted pair cable A pair of wires that can transmit data; the wires are twisted to provide protection against crosstalk.

U

Unlatch instruction One-half of a programmable logic controller instruction pair that emulates the unlatching action of a latching relay. The unlatch instruction de-energizes a specified output point or internal coil until re-energized by a latch instruction. The output point or internal coil remains de-energized regardless of whether or not the unlatch instruction is energized.

Up-counter An event that starts from 0 and increments up to the preset value.

V

Variable A factor that can be altered, measured, or controlled.

Variable data Numerical information that can be changed during application operation. It includes timer and counter accumulated values, thumbwheel settings, and arithmetic results.

Volatile memory A memory structure that loses its information whenever power is removed. Volatile memories require a battery backup to ensure memory retention during power outages.

W

Watchdog timer Monitors logic circuits controlling the processor. If the watchdog timer, which is reset every scan, ever times out, the processor is assumed to be faulty and is disconnected from the process.

Word A grouping or a number of bits in a sequence treated as a unit.

Word length The total number of bits that make up a word. Most programmable logic controllers use either 8, 16, or 32 bits to form a word.

Work cell A group of machines that work together to manufacture a product; normally includes one or more robots. The machines are programmed to work together in appropriate sequences. Work cells are often controlled by one or more PLCs.

Write Refers to the process of loading information into memory; can also refer to block transfer, that is, a transfer of data from the processor data table to an intelligent input/output module.

Z

Zone The portion of a PLC ladder program that can be enabled or disabled by a control function.

Index

Note: Page numbers followed by f and t denotes figures and tables respectively.

A

Abbreviations, 2
Access method, 320
Accumulated time, 134
Accumulated value
  timer, 136
  up-counter, 161
Active tab, 92
Actuators, 112
  in control system, 308
Acumulated value (ACC) word, 136
  up-counter, 161
ADC (analog-to-digital converter), 27, 94
Addition, 55–56
Addition instruction, 236–237f, 236–238
Add-On instructions, 384
Addressing, 18, 86–87, 86f
  bit level, 19, 20f
  I/O connection diagram, 87f
  rack/slot-based, 19, 19f
  rack/slot-based *vs.* tag-based, 19, 21f
  tag-based, 19, 20f
    CLX system, 347–348, 347f
  word level, 19, 20f
Addressing formats, 212, 212f
Alarm monitor program, 166f
Alarm system, SCADA, 329, 330, 330f
Alias tag, 19, 338–339, 339f
Allen-Bradley
  controllers, 51 (*see also specific types*)
  ControlLogix platform from, 48 (*see also* ControlLogix controllers)
  memory structures, 75
  MicroLogix controller, 78, 78f, 87
  RSLogix software packages, 91
  SLC 500 (*see* SLC 500 controllers)
  subroutine-related instructions, 191, 191f
Allen-Bradley Data Highway, 321, 321f, 322, 322f
Allen-Bradley Pico controller, 11, 115
  installation, 11
Allen-Bradley Pico GFX-70 controller, 40f
All Mode, FAL instruction, 213
Alternating current (AC) supply, 5
Ambient temperature rating, discrete I/O modules, 33
American Standard Code for Information Interchange (ASCII), 54
ANALOG COMMON terminal, 29, 29f
Analog control, 228, 311
Analog devices, 93–94, 224, 225
Analog input module circuit, 28, 29f
Analog I/O modules, 27–29f, 27–31
  specifications, 34–35
    channels per module, 34
    common-mode rejection, 34–35
    input current/voltage range(s), 34
    input impedance and capacitance, 34
    input protection, 34
    output current/voltage range(s), 34
    resolution, 34
Analog output interface module, 29, 225–226, 226f
Analog signals, 93–94, 94f
Analog-to-digital converter (ADC), 27, 94
AND function, 62–63, 62f, 65, 208
AND gate, 62, 62f
Annunciator flasher program, 148, 148f
Architecture, 4
Arithmetic
  binary, 55–57
  floating point, 57–58, 57–58f
Array, CLX system, 342–343, 343f
ASCII (American Standard Code for Information Interchange), 54
ASCII code, 54, 55t
ASCII module, 31
Associative law, 66
Assume Data Available indicator marker, FBD, 387, 388f
Audit, SCADA, 329
Automatic control process, 121, 121f
Automatic mode, water level control process in storage tank, 118
Automatic sequential control systems
  on-delay timer (TON), 138, 140, 140f
Automatic stacking program, 174–176, 175f
Auxiliary seal-in contact, 199, 200f

B

Backplane current draw, discrete I/O modules, 34
Backplane power, 22
BAND (Boolean AND) function block, 82, 384–385, 385f
Bandwidth, 325
Bang-bang control, 228
Barcode reader, 273, 273f
Bar code scanners, 108–109, 109f
Base tag, 19, 338, 339f, 340
Base tag data, 339–340, 340f
BASIC, 31, 81
Batch processing, 306, 306f
Battery backup, 291f
BCD. *See* Binary Coded Decimal (BCD) output device
BCD-output module, 32, 32f
BCD-to-binary conversion, 51, 53f
Bearing lubrication program, 146f
Binary arithmetic, 55–57
Binary Coded Decimal (BCD) output device, 224–225, 225f
Binary coded decimal (BCD) system, 51–53, 52–53f
  numeric values, 52t
Binary information, 268
Binary number(s)
  complement of, 49
  conversion to decimal number, 47, 48f
  decimal number conversion to, 48, 48f
  hexadecimal number conversion to, 51, 51f
  octal number conversion to, 50f
  signed, 49t
Binary principle, 62–65
  Boolean algebra and, 65–66
Binary system, 47–48f, 47–49, 47t
  binary principle, 62–65
  Boolean algebra and, 65–66
  numeric values, 52t
Binary-to-BCD conversion, 51, 53f
Bit, 47, 54, 208, 208f
Bit file, 75
Bit level addressing, 19, 20f
Bit-level logic instructions, 83–86
  Examine If Closed (XIC), 83–84, 83f
  Examine If Open (XIO), 83, 84, 84f
  Output Energize (OTE), 83, 84–85, 85f
  separating action of field device, 85, 85f
  symbolic, 83
Bit-level programming, CLX system, 345–355
  internal relay instructions, 350–352, 351–352f
  ladder logic program, 346, 346f
    addition to main routine, 348–350, 348–350f
  latch and unlatch instructions, 352–353, 352–353f
  one-shot instruction, 353–354, 354–355f
  program scan, 345–346, 345f
  tag-based addressing, 347–348, 347f
Bit-oriented I/O, 22
Bit shift instructions, 265, 266f
  bit address, 267
  carriers tracking flowing through 16-station machine, 269, 271, 271f
  control, 266
  file, 266
  length, 267
Bit shift left (BSL) instruction, 265, 266f, 267, 267f
  Allen Bradley ControlLogix, 271–272, 272f
  with wraparound operation, 267–268, 269f
Bit shift registers, 264–272
  bit shift instructions, 265–268, 266f
  File Shift menu tab, 266
Bit shift right (BSR) instruction, 265, 266f, 267–268, 268f
  Allen Bradley ControlLogix, 271
Bit-wide bus networks, 319
Bleeder resistor, 285, 285f
  connected to proximity sensor, 106, 106f
Block-formatted counter, 158, 158f
Block format timer instruction, 135, 135f
Boolean algebra, 65
  associative law, 66
  basic operators, 65f
  commutative law, 65
  distributive law, 66
  logic operators, 65f, 66f
Boolean equation, 65
  for logic gate circuits, 66–67, 67f
Boolean expression
  logic gate circuits from, 66, 66f
Bottle-filling motion control process, 315, 316f
  motion module, 316
  programmable logic controller, 315
  servo drive, 316
  servo motor, 316
Branch instructions, 87–89, 87f
  input, 87, 87f
  matrix limitation diagram, 88
  nested, 87–88, 87f
  original circuit, 89f
  output, 87, 87f
  reprogrammed circuit, 89f
  vertical contact, 88, 88f
Break-before-make pushbutton, 102
Bridges, 320
Bus topology network, 318–320, 319f
Byte, 48
Byte-wide bus networks, 319

C

Calibration, instrument, 127
Cam-operated sequencer switch, 253, 253f
Cam switches, 253
Can-counting program, 162, 163f
Capacitive proximity sensors, 106, 106f
Cascading counters, 170–172, 171f
Cascading timers, 147–148, 149f
  CLX system, 365, 366f
Centralized control, 307, 307f
Central processing unit (CPU), 4, 5–6, 6f, 35–36, 35–36f. *see also* Processor module
  backup memory battery, 291, 291f
  power supply, 35, 35f

Channels per module, analog I/O modules, 34
Circuits. *see also specific types*
  electrical interlocking, 115–116, 115–116f
  seal-in, 114–115, 114–115f
Circulating shift register function, 265
Clear (CLR) instruction, 243
Closed architecture, 4
Closed-loop control system, 226–229, 309–310, 309f
  block diagram of, 227, 228f
Closed-loop servo motor system, 114, 114f
CLR (Clear), 208
CMOS-RAM chips, 38, 38f
Coaxial cable, 317, 317f
Codes. *see* Number systems and codes
Coil-formatted counter instruction
  up-counter, 157–158, 157f, 158f
Coil-formatted timer instruction, 134–135, 135f
Cold junction compensating (CJC) thermistor, 27
Collision detection access control scheme., 320
Combination control process, 121, 121f
Combination I/O modules, 21–22, 21f
Commissioning, 288
Common Industrial Protocol (CIP), 323–324, 325
Common-mode rejection, analog I/O modules, 34–35
Communication module, 4f, 18, 32, 33f
Communications capability, 3, 4f
Communications protocol, 300
Commutative law, 65
CompactLogix system, 333
Compare menu tab, 217, 217f
Comparison instruction, CLX system, 376–378, 377–379f, 380–382, 380–382f
Compute instruction, 235–236, 235f
Compute/Math menu tab, 235, 235f
Computer memory, 48, 49f
Computers, *vs.* PLCs, 11–12, 11–12f
Configuration, CLX system, 334–335, 335f
Consecutive group, 208
Constant voltage (CV) transformer, 287
Contact histogram function, 290–291
Contact instruction, 85
Contactors, 100, 101f
Contacts
  control relays, 100
  instantaneous, 132
  normally closed (NC), 99, 100f
  normally open (NO), 99, 100f
Container-filling process, 310f
Continuous process, 306, 306f
Continuous-scan test mode, 93
Continuous tasks, 336
Continuous test mode, 289
Control, FAL instruction, 212
Control circuit, 100
Control devices, output, 112–113f, 112–114
Control file, 75
Controllers, 308–310. *see also specific types*
  deadband, 311, 311f
  proportional, 311–315, 311f
Controller tag, 338
ControlLogix 5000
  BSL instruction, 271–272, 272f
  BSR instruction, 271
  counter instructions, 162, 163f
  FIFO instruction pair, 275–276, 276f
  immediate output instruction, 194, 194f
  memory structure, 75
  Modulo (MOD) instruction, 242, 242f
  pass/fail inspection program, 271–272, 272f
  platform, 48
  program, 220, 220f
  timer instruction, 136–137, 136f
ControlLogix Sequencer Output (SQO) instruction, 258–259, 259f, 260f
  array, 259
  control, 259
  destination, 259
  length, 259
  mask, 259
  position, 259
ControlLogix (CLX) system, 334–393
  array, 342–343, 343f
  bit-level programming, 345–355
    internal relay instructions, 350–352, 351–352f
    ladder logic addition to main routine, 348–350, 348–350f
    ladder logic program, 346, 346f
    latch and unlatch instructions, 352–353, 352–353f
    one-shot instruction, 353–354, 354–355f
    program scan, 345–346, 345f
    tag-based addressing, 347–348, 347f
  comparison instructions, 376–378, 377–379f
  configuration, 334–335, 335f
  counters, 368–372, 368f
    combined with timer functions, 372, 372f
    count-down (CTD) counter, 371–372f
    count-up (CTU) counter, 369–370f, 369–371
    overview, 368–369
  FBD programming, 388–392, 388–393f
  function block diagram (FBD), 384–387, 384–387f
    add-On instructions, 384
    Assume Data Available indicator marker, 387, 388f
    BAND (Boolean AND) function block, 384–385, 385f
    data latching, 387, 387f
    feedback loop, 387, 387f
    function blocks, 384
    references, 385
    signal flow and execution, 386–387, 386f
    wire connectors, 385–386, 386f
    wires and pins, 385, 386f
  math, comparison, and move instructions, combining, 380–382, 380–382f
  math instructions, 374–376, 374–376f
  memory layout, 334, 334f
  move instructions, 379, 380f
  programs, 336–337, 337f
  project, 335–336, 335–336f
  routines, 337, 337f
  structures, 340–341, 340–341f
  tag-based addressing format, 19, 20f
  tags, 337–340, 338–340f
    creating, 341, 341f
    monitoring and editing, 342, 342f
  tasks, 336, 336f
  timers, 358–366
    cascading, 365, 366f
    off-delay timer (TOF), 362–363, 362–363f
    on-delay timer (TON), 359–362, 359–362f
    predefined structure, 358–359, 358f
    reciprocating, 365
    retentive on-delay timer (RTO), 364–365, 364–365f
Control management PLC application, 13, 13f
ControlNet, 325, 325f
Control panel
  PLC-based, 2, 3f
  relay-based, 2, 3f
Control process
  automatic, 121, 121f
  combination, 121, 121f
  sequential, 121, 121f
    flow diagram, 121–122, 121f
    I/O connection diagram, 122, 122f
    ladder logic program for, 122, 123f
    relay schematic for, 122, 122f
Control relays (CR)
  coils, 100
  contacts, 100
  electromagnetic, 99–100, 99–100f
  internal, 89
  operation, 99, 99f
  pilot lights control operation, 99–100, 100f
  symbol, 99, 100f
Control systems. *see also* Process control; Process(es)
  actuators, 308
  closed-loop, 309, 309f
  controllers, 308–310
  distributive, 307, 308f
  HMI, 308
  motion control, 315–316
  on/off control, 310–311, 310–311f
  PID control, 311–315
  sensors, 308
  signal conditioning, 308
  structure, 308–310
Control variable (CV), 229
Convert from BCD (FRD) instruction, 235, 243, 243f
Convert-to-BCD (TOD) instruction, 51, 53f, 235, 243, 243f
Conveyor warning signal circuit, 139f
COP. *See* File copy (COP) instruction
Cost, 2–3
Count-down (CTD) counter, 168, 168f
  CLX system, 371–372f
Count-down (CD) enable bit, 160
Counter file, 75
Counter function, timer function and, 174–177
Counter number, 161
Counters, 82, 156–178
  accumulated value, 158, 158f
  application, 157f
  block-formatted, 158, 158f
  cascading counters, 170–172, 171f
  CLX system, 368–372, 368f
    combined with timer functions, 372, 372f
    count-down (CTD) counter, 371–372f
    count-up (CTU) counter, 369–370f, 369–371
    overview, 368–369
  coil-formatted, 157–158, 157f, 158f
  combining with timer functions, 174–177, 175f
  decrement, 158
  down-counter, 166–169, 167f
  electronic, 157, 157f
  high-speed, 177–178, 178f
  increment, 158
  incremental encoder-counter applications, 173–174, 174f
  input conditions, 158
  instructions, 157–158
  mechanical, 157, 157f
  selection toolbar, 161f
  up-counter, 159–166, 159f
Counting, high-speed, 4f
Count-up (CTU) counter
  CLX system, 369–370f, 369–371
Count-up (CU) enable bit, 160
CPU section, processor module, 35
Cross reference function, 289, 290, 290f
Current sensing, 27
CV. *See* Control variable (CV)

D

DAC (digital-to-analog converter), 29, 94
Daisy-chain topology, 326, 327f
Data communications, 316–328. *see also* Network communications
  ControlNet, 325, 325f
  Data Highway, 322, 322f
  defined, 316
  DeviceNet, 322–325, 323–324f
  EtherNet/IP, 325, 326f
  Fieldbus, 326, 327f
  HART, 328, 328f
  industrial networks, functionality levels, 318, 318f
  LAN, 317–318, 317f
  Modbus, 326, 326f
  parallel data transmission, 321, 321f
  point-to-point serial communications link, 317, 317f
  PROFIBUS-DP, 326–327, 327f
  SERCOS, 327, 328f
  serial, 322, 322f
  serial transmission, 321–322, 321f
  transmission media, 317, 317f
Data comparison, 208, 216–221. *See also* Data manipulation
  EQU instruction, 217, 217f
  GEQ instruction, 217, 218, 218f
  GRT instruction, 217–218, 217f
  LEQ instruction, 217, 218, 218f
  LES instruction, 217, 218, 218f
  LIM instruction, 217, 218–220, 219f, 220f
  MEQ instruction, 217, 220–221, 220f, 221
  NEQ instruction, 217, 217f
  use, 216–217
  vs. data transfer, 216
Data files, 75–78, 77–78f
Data File screens, RSLogix 500 software, 92, 92f
Data flow, scan process, 79, 79f
Data Highway, 322, 322f
Data latching, FBD, 387, 387f
Data manipulation
  closed-loop control, 226–229

data comparison, 208, 216–221
data transfer, 208–216
defined, 208
FAL instruction (*See* File arithmetic and logic (FAL) instruction)
features, 208
numerical data I/O interfaces, 224–226
PID controllers and, 227, 228–229, 228f
programs, 221–224, 221f–223f
use of, 208
Data monitoring function, 289–290, 290f
Data transfer, 208–216. *See also* Data manipulation
concept, 208, 209f
defined, 208
vs. data comparison, 216
DCS (distributive control system), 307, 308f, 317–318
Deadband, 228, 311, 311f
Decimal number(s)
binary number conversion to, 47, 48f
conversion to binary number, 48, 48f
hexadecimal number conversion to, 51, 51f
octal number conversion to, 50f
Decimal system, 47, 47f, 47t
numeric values, 52t
Decrement counter, 158
Derivative action, proportional control and, 313
Destination, FAL instruction, 213
Destination register, 209
Determinism, data communication and, 325
Device bus networks, 319
DeviceNet, 322–325, 323–324f
DF1 Radio Modem, 317
Difference, 56
Digital devices, 93–94
Digital signal waveform, 47f
Digital-to-analog converter (DAC), 29, 94
DIP (dual in-line package) switches, 103, 103f
Direct current (DC) supply, 5
Directory, SCADA, 329
Direct scan technique, 108, 108f
Discrete I/O modules, 22–27, 23–25f, 24t
specifications, 33–34
ambient temperature rating, 33
backplane current draw, 34
electrical isolation, 34
input on/off delay, 33
input threshold voltages, 33
inrush current, 34
leakage current, 34
nominal current per input, 33
nominal input voltage, 33
output current, 33
output voltage, 33
points per module, 34
short circuit protection, 34
Discrete manufacturing, 306, 306f
Dishwasher
timed sequencer switch, 253–254, 254f
wiring diagram and timing chart, 254f
Distributive control system (DCS), 307, 308f, 317–318
Distributive law, 66
Division, 55, 57
Division instruction, 235, 240–242, 241f
Done (DN) bit, 136, 160
Double precision, 58
Down-counter, 158, 166–169, 167f
Drilling process, ladder logic program for (example), 124–126, 124–126f
Drink-manufacturing program, PLC, 216, 216f
DriveLogix system, 333
Driver, 300
Drum-operated sequencer switch, 253, 253f
Drum switches, 253
Dual in-line package (DIP) switches, 103, 103f
Duplex communication system, 322
Duration, PID controllers, 228

E

Editing, 288
Edit tags, CLX system, 342, 342f
Electrical continuity, 79
Electrical devices, 93–94
Electrical interlocking circuits, 115–116, 115–116f
Electrical isolation, discrete I/O modules, 34
Electrically erasable programmable read only memory (EEPROM), 38, 38f
Electrical noise, 284–285
Electromagnetic control relays, 99–100, 99–100f
Electromagnetic interference (EMI), 284–285
Electromagnetic latching relays, 116–120, 117–120f
Electromechanical retentive timer, 144f
Electronic counters, 157, 157f
Electronic output module protection, 294, 294f
Electronic timing relays, 132, 132f
Enable (EN) bit, 136
Enclosures, 282–284, 282f
HMI, 41, 41f
NEMA 12 enclosure, 282
temperature inside, 282
Encoder, 111, 111f
Encoder-counter module, 31, 31f
EQU (Equal) instruction, 217, 217f
Erasable Programmable Read Only Memory (EPROM), 38
Error amplifier, 309
Error(s), 228–229
checking, 292
verifying program, 294, 295f
EtherNet/IP, 325, 326f
Even parity bit, 54, 55t
Event-driven sequencer program, 259, 262, 263f
Event history, HMIs and, 41
Event tasks, 336
Examine If Closed (XIC) instruction, 83–84, 83f
interpretation, 84, 84f
programming, 90–91, 90f
Examine If Open (XIO) instruction, 83, 84, 84f
interpretation, 84, 84f
programming, 90–91, 90f
Exclusive-OR (XOR) function, 65, 65f, 208
Exponent, 58
Expression, FAL instruction, 213

F

FactoryTalk services platform, SCADA, 329–330, 330f
FAL. *See* File arithmetic and logic (FAL) instruction
Fault routine, 201, 337
Feedback, 228
Feedback loop, FBD, 387, 387f
Fiber optic sensors, 108, 109f
Fiber optic transmission medium, 317
Field devices, 7
Field devices testing, 4
FIFO (first in, first out) instruction, 272–276
ControlLogix programming with, 275–276, 276f
operating program, 274–275, 274f
SLC 500 FIFO load (FFL) instruction, 273, 273f
SLC 500 FIFO unload (FFU) instruction, 273–274, 273f
vs. LIFO stack operation, 275, 275f
File, 208, 208f, 211
File add function, 245, 245f
File addressing, 212, 212f
File arithmetic and logic (FAL) instruction, 245, 245f
All Mode, 213
Control, 212
Destination, 213
exceptions to rule, 215
Expression, 213
file-to-file copy function and, 213–214, 214f
file-to-word moves, 214, 214f
Incremental mode, 213
Length, 212
Mode, 213
Numeric Mode, 213
overview, 212, 212f
Position, 213
File arithmetic operations, 245–246, 245–246f
File copy (COP) instruction, 215, 215f
PLC drink-manufacturing program, 216, 216f
File divide function, 246, 246f
File multiply function, 246, 246f
File Shift menu tab, 266
File subtract function, 245, 245f
File-to-file shifts, 211, 212f
FAL instruction and, 213–214, 214f
File-to-word moves, 211
FAL instruction and, 214, 214f
Fill file (FLL), 215, 216f
First in, first out (FIFO) instruction, 272–276
ControlLogix programming with, 275–276, 276f
operating program, 274–275, 274f
SLC 500 FIFO load (FFL) instruction, 273, 273f
SLC 500 FIFO unload (FFU) instruction, 273–274, 273f
vs. LIFO stack operation, 275, 275f
Fixed I/O, 4, 5f
advantage, 4
disadvantage, 4
Flash EEPROM, 38, 38f
Flashing pilot light subroutine, 192f
Flash memory, 38, 38f
Flexibility, 2
FlexLogix system, 333
FLL. *See* Fill file (FLL)
Floating point arithmetic, 57–58
components, 58, 58f
example, 57–58, 57f, 58f
features, 57
Floating point file, 75
Flow measurement, 111, 111f
Flowmeter, turbine type, 111f
Fluid pumping process, 143–144, 143f
Force function, 195
Forcing external I/O addresses, 195–196f, 195–197
Full-duplex transmission, 322
Fully automatic/intelligent PID control, 315
Function block diagram (FBD), CLX system, 384–387
add-On instructions, 384
Assume Data Available indicator marker, 387, 388f
BAND (Boolean AND) function block, 384–385, 385f
data latching, 387, 387f
feedback loop, 387, 387f
function blocks, 384
references, 385
signal flow and execution, 386–387, 386f
wire connectors, 385–386, 386f
wires and pins, 385, 386f
Function block diagram (FBD) programming, 81, 82
CLX system, 388–392, 388–393f
concept, 82
equivalents to ladder logic contacts, 82f
ladder diagram and, 82, 82f
use of, 82
Function blocks, 384

G

Gateways, 320
GEQ. *See* Greater Than or Equal (GEQ) instruction
Graphic HMI terminals, 309
Graphics library, HMI, 42, 42f
Gray code, 53–54, 53t, 54f
Greater Than (GRT) instruction, 217–218, 217f
Greater Than or Equal (GEQ) instruction, 217, 218, 218f
Grounding, 285–286, 286f
Ground loops, 286, 286f
GRT. *See* Greater Than (GRT) instruction

H

Hand-held programming devices, 7, 39
Hand-held units, 91
Hardware, 4, 17–42
analog I/O modules, 27–29f, 27–31
BASIC or ASCII module, 31
BCD-output module, 32, 32f
communication modules, 32, 33f
CPU, 35–36, 35–36f
data recording and retrieval, 39, 39f
discrete I/O modules, 22–27, 23–25f, 24t
encoder-counter module, 31, 31f
high-speed counter module, 31, 31f
human machine interfaces (HMIs), 39–42, 40f
I/O section, 18–22, 18f
I/O specifications
analog I/O module, 34–35
discrete I/O module, 33–34
memory design, 36–37, 37f

Hardware (*continued*)
memory types, 37–38, 38f
motion and position control modules, 32, 32f
PID module, 32, 32f
programming terminal device, 39, 39f
special I/O modules, 31–32
stepper-motor module, 32, 32f
thumbwheel module, 31, 31f
TTL module, 31
Hardwired logic
defined, 67
vs. programmed logic, 67–68
Hardwired time-delay circuit, 221
Hardwired TOF relay circuit, 142–143, 142f
equivalent PLC program, 143–144, 143f
HART, 328, 328f
Heating application, proportional band for, 312, 312f
Heat-shrinkable wire identification sleeves, 285f
Hexadecimal (hex) numbering system, 47t, 50–51, 51t
conversion to binary number, 51, 51f
conversion to decimal number, 51, 51f
numeric values, 52t
High-density module, 22
High-speed counter (HSC), 160, 177–178, 178f
High-speed counter module, 31, 31f
High-speed counting, 4f
HMIs. *see* Human machine interfaces (HMIs)
Holding circuits, 114–115, 114–115f
Hold-up time, 35
Horizontal scan, 81, 81f
Human machine interfaces (HMIs), 12, 12f, 39–42, 40f, 308–309, 308f
alarms, 41, 41f
Allen-Bradley Pico GFX-70 controller, 40f
design, 40
event history, 41
graphics library, 42, 42f
graphic terminals, 309
monitor and enclosure, 41, 41f
structure, 40, 40f
tasks, 40–41
trend, 41–42, 42f
Hunting, 311
Hysteresis, 105–106

I

IEC 61131 standard
programming language, 81, 81f
IEEE 754 Standard, 58
Immediate input with mask (IIM) instruction, 193–194, 194f
Immediate I/O instructions, 193–194, 194f
Immediate output with mask (IOM) instruction, 193, 194, 194f
Incremental encoder-counter applications, 173–174, 174f
Incremental mode, FAL instruction, 213
Incremental optical encoder, 173–174, 174f
Increment counter, 158
Individual control, 306, 307f
Inductive loads, noise suppression for, 284, 284f
Inductive proximity sensors, 105, 105f
Inductive proximity switches, 106–107
Industrial networks
control level, 318
device level, 318
functionality levels, 318, 318f
information level, 318
Input branching, 87, 87f
nested, 87–88, 87f
Input current/voltage range(s), analog I/O modules, 34
Input devices, 6–7
and outputs, relationships between, 2, 3f
Input file, 75
Input image table file, 76–77, 77f
Input impedance and capacitance, analog I/O modules, 34
Input instructions, 216. *See also* Data comparison
Input malfunctions, troubleshooting, 292–293, 293f
Input on/off delay, discrete I/O modules, 33
Input/output (I/O) section, 4, 6–7, 7f, 18–22, 18f
addressing, 18–21, 19f, 20f
combination I/O modules, 21–22, 21f
field/real-world devices, 7
fixed, 4, 5f
modular, 4–5, 6f
PC interface card, 21f
plug-in terminal block, 22, 22f
rack-based, 18, 18f
remote I/O rack, 18, 18f
wiring connections, 9, 9f
Input protection, analog I/O modules, 34
Input threshold voltages, discrete I/O modules, 33
Input troubleshooting guide, 298f
Inrush current, discrete I/O modules, 34
Instantaneous contacts, 132
Instruction addressing, 86–87, 86f
Instruction list (IL) programming language, 81
vs. ladder diagram programming, 81–82, 81f
Instruction palette, 92
Instructions. *see also* Program control instructions; Sequencer instructions
counter, 157–158
override, 185
timer, 134–135, 134–135f
block format, 135, 135f
coil-formatted, 134–135, 135f
off-delay timer, 140–144, 141–143f
on-delay timer, 135–138f, 135–140
Instruction set, 14, 14t
Instruction toolbar with bit, RSLogix 500 software, 91f
Instrumentation, 127
Instrument(s), 127
calibration, 127
smart, 127, 127f
Intact, data, 211, 212f
Integer file, 75
Integral action, proportional control and, 312
Internal bits, 89
Internal coils, 89
Internal control relays, 89
Internal outputs, 89
Internal relay instructions, 89, 89f, 90f
CLX system, 350–352, 351–352f
Internal storage bits, 89
Inverter, 64
I/O address format
for SLC family of PLCs, 75–76, 77f
I/O bus networks, 319
I/O Configuration screen, RSLogix 500 software, 92, 92f
I/O connection diagram, 87f
Sequential control process, 122, 122f
I/O count, 12–13
I/O modules
analog, 27–29f, 27–31
BASIC or ASCII module, 31
BCD-output module, 32, 32f
communication modules, 32, 33f
discrete, 22–27, 23–25f, 24t
encoder-counter module, 31, 31f
high-speed counter module, 31, 31f
motion and position control modules, 32, 32f
PID module, 32, 32f
special, 31–32
stepper-motor module, 32, 32f
thumbwheel module, 31, 31f
TTL module, 31
I/O specifications
analog I/O module, 34–35
channels per module, 34
common-mode rejection, 34–35
input current/voltage range(s), 34
input impedance and capacitance, 34
input protection, 34
output current/voltage range(s), 34
resolution, 34
discrete I/O module, 33–34
ambient temperature rating, 33
backplane current draw, 34
electrical isolation, 34
input on/off delay, 33
input threshold voltages, 33
inrush current, 34
leakage current, 34
nominal current per input, 33
nominal input voltage, 33
output current, 33
output voltage, 33
points per module, 34
short circuit protection, 34
Isolation transformers, 287

J

Jog circuit with control relay, 123–124f
Jump (JMP) instruction, 188–190, 188–190f
Jump to subroutine (JSR) output instruction, 191

K

Keyboards, 91
Keyswitch, three-position, 93f
1-K word memory, 48, 48f

L

Label (LBL) instruction, 188–189
Ladder diagram (LD) language, 81
advantages, 92–93
entering program, 91–93
Data File screens, 92, 92f
instruction toolbar with bit, 91f
I/O Configuration screen, 92, 92f
main window, 91–92, 91f
Select Processor Type screen, 92, 92f
function block diagram and, 82, 82f
function of, 85–86, 85f
vs. instruction list programming, 81–82, 81f
Ladder logic contacts
function block diagram equivalents to, 82f
Ladder logic program, 7, 9–10, 9f, 67–68, 67f
CLX system, 346, 346f
addition to main routine, 348–350, 348–350f
diagram for modified process, 11f
drilling process (example), 124–126, 124–126f
electrical continuity in, 79
monitoring, 93, 93f
parallel instructions, 122–123, 123f
relay schematics conversion to, 121–123
rung conditions, evaluating, 79, 79f
for sequential control process, 122, 123f
series instructions, 122, 123f
troubleshooting, 294–299, 295–296f
writing, from narrative description, 124–126, 124–126f
Ladder rung, 85–86, 85f
Ladder window, 92
properties, 92
Language, 7, 67, 81–83, 81f
function block diagram programming, 81, 82, 82f
IEC 61131 standard, 81, 81f
instruction list programming language, 81–82, 81f
ladder diagram language, 81, 82, 82f
sequential function chart (SFC) programming language, 81, 82, 82f
structured text (ST), 81, 83, 83f
Last In, First Out (LIFO) instruction, 275
example, 275, 275f
vs. FIFO stack operation, 275, 275f
Latch coil, 116–117
Latching relays, 116–120, 117–120f
Latch instructions, CLX system, 352–353, 352–353f
Leakage current, 285
discrete I/O modules, 34
Least significant bit (LSB), 48
Length, FAL instruction, 212
Less Than (LES) instruction, 217, 218, 218f
Less Than or Equal (LEQ) instruction, 217, 218, 218f
Level switches, 104, 104f
LIFO (last in, first out) instruction, 275
example, 275, 275f
vs. FIFO stack operation, 275, 275f
Light-emitting diode (LED) window, 39
Light sensors, 107–109, 107–109f
bar code scanners, 108–109, 109f
fiber optic, 108, 109f
photoconductive cell, 107, 107f
photoelectric, 107–108, 108f
photovoltaic/solar cell, 107, 107f
scan technique, 108, 108f
LIM. *See* Limit test (LIM) instruction
Limit switch, 103, 103f
Limit test (LIM) instruction, 217, 218–220, 219f, 220f
Liquid-crystal display (LCD) windows, 39

LiveData, SCADA, 329
Local area network (LAN), 317–318, 317f
Lockout/tagout devices, 291f
Logic, 82
Logical continuity, 79, 86f
Logic gate, 62, 62f
  truth tables, 62
Logic gate circuits
  Boolean equation for, 66–67, 67f
  from Boolean expression, 66, 66f
Logic operators, 65f, 66f
Logic wiring error, 2
LogixDesigner, 334, 335f
LSB (least significant bit), 48

M

Magnetic reed switch, 107, 107f
Magnitude, PID controllers, 228
Main ladder program (file 2), 75
Main routine, CLX system, 337
  ladder logic program and, 348–350, 348–350f
Maintenance, preventive, 291, 291f
Main window, RSLogix 500 software, 91–92, 91f
Mantissa, 58
Manually operated switches, 102–103, 103f
  dual in-line package (DIP) switches, 103, 103f
  pushbutton switches, 102, 103f
  selector switch, 103, 103f
Manual mode, water level control process in storage tank, 118
Manual PID control, 314
Masked Comparison for Equal (MEQ), 217, 220–221, 220f, 221
Masked Move (MVM) instruction, 208, 209–210, 209f
Master control relay (MCR), 198–199, 282–284
  hardwired electromechanical, 282–284, 283f
Master control reset (MCR) instruction, 185–187f, 185–188
Master/slave network, 320–321, 320f
Math instructions, 234–246
  addition, 236–237f, 236–238
  clear, 243, 243f
  CLX system, 374–376, 374–376f, 380–382, 380–382f
  Compute/Math menu tab, 235, 235f
  convert from BCD (FRD), 243, 243f
  convert to BCD (TOD), 243, 243f
  division, 235, 240–242, 241f
  file arithmetic operations, 245–246, 245–246f
  multiplication, 235, 239–240, 239–240f
  negate, 242–243, 243f
  overview, 235
  scale, 243–244, 244–245f
  SLC 500 CPT (compute) instruction, 235–236, 235f
  square root, 242, 242f
  subtraction, 235, 238–239, 238f
  word-level math instructions, 242–244
Matrix limitation diagram, 88
Matrix-style sequencer chart, 259, 261f
Mechanical counters, 157, 157f
Mechanically operated switches, 103–104, 103–104f
  level switches, 104, 104f
  limit switch, 103, 103f
  pressure switches, 104, 104f
  temperature switch/thermostat, 103–104, 104f
Mechanical sequencers, 253–254, 253f
  cam-operated, 253, 253f, 254
  dishwasher timed sequencer switch., 253–254, 254f
  drum-operated, 253, 253f
  operation, 253
Mechanical timing relays, 132–134, 132f
Memory
  capacity measurement, 13
  CLX system, 334, 334f
  design, 36–37, 37f
  EEPROM, 38, 38f
  EPROM, 38
  factors, 13
  flash, 38, 38f
  location, 37
  nonvolatile, 37–38
  organization of, 48
  RAM, 38
  ROM, 38
  single bit, 83
  size, 13, 37f
  types, 37–38, 38f
  utilization, 37
  volatile, 37
Memory map, 75
Memory section, processor module, 35
Memory structures, 75
  rack-based, 75
  tag-based, 75
Menu bar, 91
MEQ. *See* Masked Comparison for Equal (MEQ)
Metal oxide varistor (MOV) surge suppressor, 287–288, 288f
MicroLogix controller, 78, 78f
  addressing, 87
  high-speed up-counter instruction, 178, 178f
Microprocessor, 35
Modbus, 326, 326f
Mode, FAL instruction, 213
Modes of operation
  program mode, 93
  remote mode, 93
  run mode, 93
  test mode, 93
  three-position keyswitch, 93f
Modular I/O, 4–5, 6f
Module-defined structures, CLX system, 340–341, 340f
Modules. *see also* I/O modules
  combination I/O, 21
Modulo (MOD) instruction, 242, 242f
Monitor, HMI, 41, 41f
Monitoring, 289–291
Monitor tags, CLX system, 342, 342f
Most significant bit (MSB), 48, 49
Motion control modules, 32, 32f
Motion control system, 315–316
  bottle-filling process, 315, 316f
Motion module, 316
Motor-driven analog proportional control valve, 311f
Motor lock-out program, 176–177, 176f
Motor seal-in circuit, 115f
Motor starters, 101–102, 102f
Move (MOV) instruction, 208, 209, 209f
  CLX system, 379–382, 380–382f
  variable preset counter values, 210–211, 211f
  variable preset timer values, 210, 210f
Move/Logical menu tab, 208, 208f
MSB (most significant bit), 48, 49
Multibit digital devices, 224
Multiple rung program, scan process and, 80, 80f
Multiplication, 55, 57
Multiplication instruction, 235, 239–240, 239–240f
Multitask PLC application, 13
MVM. *See* Masked Move (MVM) instruction

N

NAND gate symbol, 64, 64f
National Electrical Code (NEC), 285–286
National Electric Manufacturers Association (NEMA), 102, 282
Negate (NEG) instruction, 242–243, 243f
Negative numbers, 49, 49t, 50t
NEMA 12 enclosure, 282
NEQ. *See* Not equal (NEQ) instruction
Nested branching, 87–88, 87f
  contact program, 88f
  program to eliminate, 88, 88f
Nested contact program, 88f
Nested subroutines, 191, 193, 193f
Network communications. *see also* Data communications
  access method, 320
  bit-wide bus networks, 319
  bridges, 320
  bus topology, 318–320, 319f
  byte-wide bus networks, 319
  ControlNet, 325, 325f
  Data Highway, 322, 322f
  device bus networks, 319
  DeviceNet, 322–325, 323–324f
  EtherNet/IP, 325, 326f
  Fieldbus, 326, 327f
  gateways, 320
  HART, 328, 328f
  industrial networks, functionality levels, 318, 318f
  I/O bus networks, 319
  LAN, 317–318, 317f
  master/slave network, 320–321, 320f
  Modbus, 326, 326f
  OSI model, 319–320
  peer-to-peer network, 321, 321f
  process bus networks, 319
  PROFIBUS-DP, 326–327, 327f
  SERCOS, 327, 328f
  star topology, 318, 318f
  token passing network, 320, 320f
Network protocol, 319
Network scanner, 323
Node, 318
Noise, electrical, 284–285, 284f
Noise suppression, 284, 284f
Nominal current per input, discrete I/O modules, 33
Nominal input voltage, discrete I/O modules, 33
Nonretentive on-delay timer (TON), 144
Nonvolatile memory, 37–38
NOR gate symbol, 64, 64f
Normally closed, timed closed (NCTC) contact
  off-delay timer circuit, 134, 134f
Normally closed, timed open (NCTO) contact
  on-delay timer circuit, 133, 133f
Normally closed (NC) contacts, 99, 100f
Normally closed (NC) pushbutton, 102
Normally open, timed closed (NOTC) contact
  on-delay timer circuit, 132–133, 133f
Normally open, timed open (NOTO) contact
  off-delay timer circuit, 133, 133f
Normally open (NO) contacts, 99, 100f
Normally open (NO) pushbutton, 102
Not equal (NEQ) instruction, 217, 217f
NOT function, 64, 64f, 65, 208
Number systems and codes, 46–58
  ASCII code, 54, 55t
  binary arithmetic, 55–57
  binary coded decimal (BCD) system, 51–53, 52–53f, 52t
  binary system, 47–48f, 47–49
  comparisons, 47t
  decimal system, 47, 47f
  floating point arithmetic, 57–58, 57–58f
  Gray code, 53–54, 53t, 54f
  hexadecimal system, 50–51, 51t
  negative numbers, 49–50t
  octal system, 49–50, 50f, 50t
  parity bit, 54, 55t
Numerical data I/O interfaces, 224–226
  analog devices, 224
  multibit digital devices, 224
Numeric Mode, FAL instruction, 213
Numeric values, comparison, 216, 216f. *See also* Data comparison

O

Octal numbering system, 47t, 49–50, 50t
  conversion to binary number, 50f
  conversion to decimal number, 50f
  numeric values, 52t
Odd parity bit, 54, 55t
Off-delay timer (TOF), 134
  CLX system, 362–363, 362–363f
  fluid pumping process, 143–144, 143f
  hardwired TOF relay circuit, 142–143, 142f, 143f
  instruction, 140–144
  operation, 140–141, 141f
  for switching motors off, 141–142, 142f
Off-delay timer circuit
  normally closed, timed closed (NCTC) contact., 134, 134f
  normally open, timed open (NOTO) contact, 133, 133f
Offline programming, 289
OFF position, water level control process in storage tank, 118
OL relay. *see* Overload (OL) relay
On-delay timer (TON), 134
  acumulated value (ACC) word, 136
  application, 137, 138, 139f
  automatic sequential control systems, 138, 140, 140f

On-delay timer (TON) (*continued*)
CLX system, 359–362, 359–362f
control bits, 137–138, 138f
control word, 136
fluid pumping process, 143–144, 143f
instruction, 135–140
preset value (PRE) word, 136
principle of operation, 135, 135f
SLC 500 timer file, 135–136, 136f
SLC 500 timer table, 138, 138f
On-delay timer circuit
normally closed, timed open (NCTO) contact, 133, 133f
normally open, timed closed (NOTC) contact, 132–133, 133f
On-delay timer program, 221–223, 222f
One-Shot Falling (OSF) instruction
CLX system, 354, 354f
One-Shot (ONS) instruction
CLX system, 353–354, 354–355f
up-counter, 162–166, 164f
One-shot rising (OSR) instruction, 165–166, 165f
CLX system, 354, 354f
1's complement number, 49, 50t
Online programming, 289
On/off control, 310–311, 310–311f
On/off PLC control, 228
|| symbol, 10
Open architecture, 4
Open-loop motor control system, 113, 113f
Open Systems Interconnection (OSI model), 319–320
Optical encoder, 111, 111f
incremental, 173–174, 174f
Optical encoder disk, 53–54, 54f
Optical isolator, 7
OR function, 63, 63f, 65, 208
OR gate, 63, 63f
OTL instruction. *see* Output latch (OTL) instruction
OTU instruction. *see* Output unlatch (OTU) instruction
Output actuator, 310
Output branching, 87, 87f
with conditions, 87, 87f
nested, 87–88, 87f
Output control devices, 112–113f, 112–114
actuator, 112
operation, 112
servo motors, 113, 113f
stepper motors, 113, 113f
symbols for, 112f
Output current, discrete I/O modules, 33
Output current/voltage range(s), analog I/O modules, 34
Output devices, 7
inputs and, relationships between, 2, 3f
Output Energize (OTE) instruction, 83, 84–85, 85f
Output file, 75
Output image table file, 77–78, 77f
Output instructions, 216. *See also* Data transfer
Output latch (OTL) instruction, 117–118, 117–118f
CLX system, 352–353, 352–353f
Output malfunctions, troubleshooting, 294, 294f, 295f
Output troubleshooting guide, 299f
Output unlatch (OTU) instruction, 117–118, 117–118f
CLX system, 352–353, 352–353f
Output voltage, discrete I/O modules, 33
Overflow (OV) bit, 160–161
Overload (OL) relay, 102
function of, 101, 101f
Override instructions, 185

P

PanelView graphic terminals, 309f
Parallel data transmission, 321, 321f
Parity bit, 54, 55t, 322
Parking garage counter, 167–168, 168f
PASCAL, 81
PC. *see* Personal computer (PC)
PC interface card, 21f
Peer-to-peer network, 321, 321f
Periodic tasks, 336
Personal computer (PC), 7, 8f, 39
memory, 48, 49f
as programmer, 91
vs. PLCs, 11–12, 11–12f
Photoconductive cell, 107, 107f
Photoelectric sensors, 107–108, 108f
Photoresistive cell, 107
Photovoltaic cell, 107, 107f
PI. *See* Proportional-integral (PI) control
Pick and Place machines, 315, 315f
PID. *See* Proportional-integral-derivative (PID) control
PID module, 32, 32f
Pins, FBD, 385, 386f
PLC-based control panel, 2, 3f
PLC drink-manufacturing program, 216, 216f
PLC programming language. *see* Language
PLCs. *see* Programmable logic controllers (PLCs)
Plug and play, 325
Plug-in terminal block, 22, 22f
Pneumatic on-delay timer, 132f
Points per module, discrete I/O modules, 34
Point-to-point serial communications link, 317, 317f
Polling, 320
Position, FAL instruction, 213
Position control modules, 32, 32f
Position sensors, 111, 111f
Positive temperature coefficient (PTC), of metals, 110
Power circuit, 100
Power supply, 4, 5, 6f
alternating current (AC), 5
CPU, 35, 35f
direct current (DC), 5
Predefined structures, CLX system, 340, 340f
programming timers, 358–359, 358f
Preset counter values, MOV instruction and, 210–211, 211f
Preset time, 134
Preset timer values, MOV instruction and, 210, 210f
Preset value, 136
counter, 161
Preset value (PRE) word
timers, 136
up-counter, 161
Pressure switches, 104, 104f
Preventive maintenance program, 291, 291f
Principles of operation, 8–10f, 8–11
modifying, 11, 11f
Process bus networks, 319
Process control, 306. *see also* Control systems
centralized, 307, 307f
distributive control system, 307, 308f
individual, 306, 307f
Process control problem, 8, 8f
ladder logic program, 9–10, 10f
relay method for, 8–9, 8f
RUN operation, 10
wiring connections, 9, 9f, 10–11, 10f
Process(es). *see also* Control systems
batch, 306, 306f
centralized control, 307, 307f
continuous, 306, 306f
control, 306
discrete manufacturing, 306, 306f
distributive control system, 307, 308f
individual control, 306, 307f
Processor (CPU), 4, 5–6, 6f
Processor memory organization, 75–79, 76f
data files, 75–78, 77–78f
program files, 75, 76f
Processor module, 35f, 36f
CPU section, 35
memory section, 35
PROG position, 35–36
REM Position, 36
RUN position, 35
troubleshooting, 292, 292f
Process parameters display, 127f
Process variable (PV), 228, 309
Produced/consumed tags, 339, 339f
Product part flow rate program, 177, 177f
PROFIBUS-DP, 326–327, 327f
Program, 7
CLX system, 336–337, 337f
data manipulation, 221–224, 221f–223f
Program control instructions, 184–202. *see also* Instructions
fault routine, 201
forcing external I/O addresses, 195–196f, 195–197
immediate I/O instructions, 193–194, 194f
jump instruction, 188–190, 188–190f
master control reset, 185–187f, 185–188
safety circuitry, 197–200, 198–200f
selectable timed interrupt, 200–201, 201f
subroutine functions, 190–193, 190–193f
suspend instruction, 202, 202f
temporary end, 201, 202f
Program files, 75, 76f
Programmable automation controller (PAC), 12, 12f, 333, 333f
Programmable logic controllers (PLCs), 2f
abbreviations, 2
advantages, 2
applications, 2, 12–13, 13f
architecture, 4
benefits, 2–4
closed architecture, 4
communications capability, 3
defined, 2
easier to troubleshoot, 3–4
faster response time, 3
flexibility, 2
hardware, 4
industrial application, 11, 11f
instruction set, 14, 14t
I/O system, 4–5, 5f, 6–7, 6f, 7f
lower cost, 2–3
modifying operation, 11, 11f
motion control process, 315
open architecture, 4
overview, 1–14
parts of, 4–7, 5f
power supply, 5, 6f
principles of operation, 8–10f, 8–11
processor (CPU), 5–6, 6f
program, 7
programming device, 7
programming language, 7
proprietary, 4
as real-time system, 2
relay logic, 2
reliability, 2
size, 12–13, 12f
software, 4, 12
vs. computers, 11–12, 11–12f
Programmed logic, hardwired logic *vs.,* 67–68
Programming, 289–291
addressing, 86–87, 86f
analog devices, connecting with, 93–94, 94f
bit-level logic instructions, 83–86
branch instructions, 87–89, 87f
Examine If Closed (XIC) instruction, 90–91, 90f
Examine If Open (XIO) instruction, 90–91, 90f
internal relay instructions, 89, 89f, 90f
ladder diagram, 91–93, 93f
languages, 81–83, 81f
modes of operation, 93, 93f
offline, 289
online, 289
processor memory organization, 75–79, 76f
data files, 75–78, 77–78f
program files, 75, 76f
program scan cycle, 78–81, 78f
Programming counters. *see* Counters
Programming device, 4, 7
Programming language. *see* Language
Programming terminal devices, 39, 39f
Programming timers. *see* Timers
Program mode, 93
Program/processor status toolbar, 91–92
Program scan cycle, 78–81, 78f
CLX system, 345–346, 345f
data flow, 79, 79f
horizontal scan, 81, 81f
ladder logic rung conditions, evaluating, 79, 79f
multiple rung program., 80, 80f
patterns, 80–81, 81f
single rung program, 79–80, 80f
time, 78–79
vertical scan, 81, 81f
Program tag, 338
Project, CLX system, 335–336, 335–336f
Project tree, 92

Project window, 92
Proportional band, for heating application, 312, 312f
Proportional control/controllers, 228, 311–315, 311f
  derivative action and, 313
  droop, 312
  integral action and, 312
  offset, 312
  steady-state error, 312
Proportional-integral (PI) control, 227
Proportional-integral-derivative (PID) control, 227, 228–229, 228f, 311–315, 313f
  fully automatic/intelligent, 315
  manual, 314
  output instruction and setup screen, 315f
  response of, 314
  semiautomatic/autotune, 314–315
Proportional plus integral (PI) control, 313
Proportioning, time, 312, 312f
Protocol, 319
Proximity sensors, 104–106f, 104–107
  application conditions, 104–105
  bleeder resistor connected to, 106, 106f
  capacitive, 106, 106f
  inductive-type, 105, 105f
  sensing range, 105–106, 106f
  three-wire DC, 105, 105f
  two-wire, 105, 105f
Pulse width modulation, 312
Pumping process, fluid, 143–144, 143f
Pushbutton interlocking, 115, 116f
Pushbutton switches, 102, 103f
PV. *See* Process variable (PV)

R

Rack-based I/O section, 18, 18f
Rack-based memory structures, 75
Rack-based systems, 75
Rack/slot-based addressing, 19, 19f
  vs. tag-based addressing, 19, 21f
Radiated noise, 284
Random Access Memory (RAM), 38, 38f
Rate of change, PID controllers, 228
Reading, 36
Read Only Memory (ROM), 38
Real-time system, PLC as, 2
Real-world devices, 7
Reciprocating timers, 147–148
  CLX system, 365
Recording, data, 39, 39f
Redundant processors, 35
References, FBD, 385
Registers, 208
Relay-based control panel, 2, 3f
Relay ladder logic (RLL) program, 7, 92
  diagram for modified process, 11f
Relay logic, 2, 7
Relay operation, 99, 99f
Relay schematics
  conversion to PLC ladder programs, 121–123
  for sequential control process, 122, 122f
Release coil, 117
Reliability, 2, 3f
Remote I/O rack, 18, 18f
Remote mode, 93
Repeatability, data communication and, 325
Repeater, 318
Reserved (file 1), 75
Reset (RES), 134
Resistance temperature detectors (RTDs), 110–111, 110f
Resolution, analog I/O modules, 34
Response time, 3, 4f
Result window, 92
Retentive on-delay (RTO) timer, 134
  alarm program, 145–146, 145f
  application, 146
  CLX system, 364–365, 364–365f
  program, 144–145, 144f
  timing chart, 145, 145f
Retentive timer, 144–146
Retentive timer reset (RES) instruction, 144
Retrieval, data, 39, 39f
Retroreflective scan technique, 108, 108f
Return (RET) output instruction, 191
Rotary switches, 253
Rotating cam limit switch, 253f
Routines, CLX system, 337, 337f
Routing, wire, guidelines for, 284–285
RS-232, 54, 300, 322
RS-422, 54, 322
RS-485, 322
RSLinx software, 299–300, 300f, 334, 335f
RSLogix software, 91, 217, 299, 299f, 300, 334, 334f
  compute instruction, 235–236
  controller organizer tree, 336f
  counter selection tool bar, 161f
  Data File screens, 92, 92f
  instruction toolbar with bit, 91f
  I/O Configuration screen, 92, 92f
  main window, 91–92, 91f
  project, 335
  Select Processor Type screen, 92, 92f
RSWho, 301, 301f, 334
RTDs (resistance temperature detectors), 110–111, 110f
RTO timer. *see* Retentive on-delay (RTO) timer
Rung, 68
Run mode, 93

S

Safety circuitry, 197–200, 198–200f
Safety PLCs, 199, 199f
  vs. standard PLC, 199
Safety requirements, 197–198, 198f
Same address, program with, 296–297, 296f
SCADA (supervisory control and data acquisition), 36, 318, 328–330, 329–330f
  alarm, 329
  alert, 329
  FactoryTalk services platform, 329–330, 330f
SCADA/HMI software, 329
Scale instruction, 243–244, 244–245f
Scale with Parameters (SCP) instruction, 30, 243–244, 245f
Scaling, 30
Scan, 10
Scan cycle time, 78–79
Scan process, 75–79, 76f
  data flow, 79, 79f
  horizontal scan, 81, 81f
  ladder logic rung conditions, evaluating, 79, 79f
  multiple rung program., 80, 80f
  patterns, 80–81, 81f
  single rung program, 79–80, 80f
  vertical scan, 81, 81f
Scan technique, 108, 108f
Scan time, 10
Scope, defined, 338
Seal-in circuits, 114–115, 114–115f
Security, SCADA, 329
Selectable timed disable (STD) instruction, 201, 201f
Selectable timed enable (STE) instruction, 201, 201f
Selectable timed interrupt (STI) instruction, 200–201, 201f
Selection toolbar, timer, 134f
Selector switch, 103, 103f
Select Processor Type screen, RSLogix 500 software, 92, 92f
Self-detection, 292
Semiautomatic/autotune PID control, 314–315
Sensing range, proximity sensors, 105–106, 106f
Sensors, 28f, 104–111. *see also* Switches
  in control system, 308
  flow measurement, 111, 111f
  light, 107–109, 107–109f
  magnetic reed switch, 107, 107f
  position, 111, 111f
  proximity, 104–106f, 104–107
  strain/weight, 110, 110f
  temperature, 110–111, 110f
  ultrasonic, 109–110, 109f
  velocity, 111, 111f
Sequencer chart, 259, 261f
Sequencer compare (SQC) instruction, 262–263, 264f
Sequencer instructions, 255–259, 255f. *see also* Instructions
  ControlLogix Sequencer Output (SQO) instruction, 258–259, 259f
  four-step sequencer, 256–257, 257f
  mask word, 256–257
    moving data through, 256–257, 258f
  moving data from file to output, 257–258, 258f
  Sequencer menu tab, 255f
  SQC instruction, 262–263, 264f
  SQL instruction, 263–264, 265f
  SQO instruction, 255–259, 255–259f
Sequencer load (SQL) instruction, 263–264, 265f
Sequencer Output (SQO) instruction, 255–259, 255–259f
  control, 256
  destination, 256
  file, 255
  length, 256
  mask, 255–256
  position, 256
  source, 256
Sequencer programs, 259–264
  event-driven, 259, 262, 263f
  sequencer chart, 259, 261f
  SQC instruction, 262–263, 264f
  SQL instruction, 263–264, 265f
  time-driven, 259, 260–261, 261f, 262f
Sequencer switch, 253–254, 253f
  cam-operated, 253, 253f, 254
  dishwasher timed, 253–254, 254f
  drum-operated, 253, 253f
  operation, 253
Sequential control process, 121, 121f
  flow diagram, 121–122, 121f
  I/O connection diagram, 122, 122f
  ladder logic program for, 122, 123f
  relay schematic for, 122, 122f
Sequential function chart (SFC) programming language, 81, 82, 82f
Sequential time-delayed motor-starting circuit
  hardwired, 147f
  PLC program of, 147f
SERCOS (Serial Real-time Communications System), 327, 328f
Serial data communication, 317, 317f, 322, 322f
  point-to-point link, 317, 317f
Serial data transmission, 321–322, 321f
Servo drive, motion control process, 316
Servo motors, 113, 113f
  closed-loop control, 114
  motion control process, 316
  open loop control, 113–114, 113f
Set-point (SP), 228, 309
Set-point control, 226–227, 227f
Seven-segment LED display board, 224, 225f
Shift registers, 264–272
  bit, 264–272
  circulating shift register function, 265
  concept of, 265, 266f
  data in, 265
  in material handling processes, 268, 269f
  spray-painting operation, 268–269, 270f
  types of, 265, 266f
Short circuit protection, discrete I/O modules, 34
Sign, 58
Signal conditioning, in control system, 308
Signals, 93–94
Sign bit, 49
Signed binary numbers, 49t
Single bit, 83, 224
Single-ended PLC application, 13, 13f
Single precision, 58
Single rung program, scan process and, 79–80, 80f
Single-scan test mode, 93, 289
Single-step test mode, 93
Sinking inputs, 26, 26f
Six-axis robot arm, 316f
16-bit word, 48f
Size, PLCs, 12–13, 12f
  memory, 13
SLC 500 controllers, 75, 208
  addressing format, 86, 86f
  bit level and word level addressing, 19, 20f
  control word, 160–161
  counter file, 160, 160f

SLC 500 controllers (*continued*)
counter instructions, 162, 163f
counter table for, 160–161, 160f
count-up counter, 161f
CPT (compute) instruction, 235–236, 235f (*see also* Math instructions)
drilling process, ladder logic program for, 124–126, 124–126f
input image table file, 76–77, 77f
internal bit addressing, 89, 89f
jump (JMP) instruction, 188
master control reset (MCR) instruction, 187–188, 187f
on-delay timer instruction, 136, 136f
one-shot rising (OSR) instruction, 165–166, 165f
output image table file, 77–78, 77f
program and data file organization, 75, 76f
Program Control menu tab, 185
rack/slot-based addressing format, 19, 19f
Sequencer menu tab, 255f
subroutine functions, 191, 193f
timer file, 135–136, 136f
timer selection toolbar, 134f
timer table, 138, 138f
water level control program using, 119, 120f
SLC 500 controller word addressing, 212, 212f
SLC family, of PLCs. *see also* SLC 500 controllers
I/O address format, 75–76, 77f
SLC 500 FIFO load (FFL) instruction, 273, 273f
SLC 500 FIFO unload (FFU) instruction, 273–274, 273f
SLC 500 output status file, 48–49, 49f
SLC 500 Scale data (SCL) instruction, 243–244, 244f
Smart instruments, 127, 127f
SoftLogix 5800 controller, 333
Software, 4, 12, 299–300f, 299–301
Solar cell, 107, 107f
Solenoid, 112, 112f
Solenoid valve, 112
construction and operation, 112–113, 113f
Source register, 209
Sourcing inputs, 26, 26f
SP. *See* Set-point (SP)
Split bar, 92
Spray-painting operation, by shift left register, 268–269, 270f
Square root (SQR) instruction, 242, 242f
Stack, 272
Stand-alone PLC application, 13, 13f
Standard PLCs, *vs.* safety PLCs, 199
Star topology network, 318, 318f
Station, 318
Status bar, 92
Status file, 75
Stepper-motor module, 32, 32f
Stepper motors, 113, 113f
Stepper switches, 253
Stop buttons, wiring of, 200, 200f
Storage tank, water level control process in, 118, 118f
automatic mode, 118
manual mode, 118
OFF position, 118
program used for, 118–119, 119f
status indicating lights, 118
using Allen-Bradley modular SLC 500 controller, 119, 120f
Strain gauge, 110, 110f
Strain gauge load cells, 110
Structured text (ST), 81, 83, 83f
Structure-type tag, CLX system, 340, 340f
Subroutine functions, 190–193, 190–193f
Subroutine (SBR) input instruction, 191
Subroutine ladder program (files 3-255), 75
Subroutines, 337
Subtraction, 55, 56–57
Subtraction instruction, 235, 238–239, 238f
Supervisory control and data acquisition (SCADA), 36, 318, 328–330, 329–330f
alarm, 329
alert, 329
FactoryTalk services platform, 329–330, 330f
Suppression device, 287
Surges, 287–288
Suspend (SUS) instruction, 202, 202f, 296, 296f
Switches. *see also* Sensors; *specific types*
manually operated, 102–103, 103f
mechanically operated, 103–104, 103–104f
positions, 103
( ) symbol, 10
System functions (file 0), 75

T

Tabbed instruction toolbar, 92
Table, 208
Tachometer generators, 111, 111f
Tag-based addressing, 19, 20f
CLX system, 347–348, 347f
vs. rack/slot-based addressing, 19, 21f
Tag-based memory structures, 75
Tag-based systems, 75
Tags, 87. *see also specific types*
CLX system, 337–340, 338–340f, 347
creating, 341, 341f
edit tags, 342, 342f
monitor tags, 342, 342f
Tank-filling process, analog control for, 94, 94f
Tasks, CLX system, 336, 336f
continuous, 336
event, 336
periodic, 336
Temperature measurement, 30, 30f
Temperature sensors, 110–111, 110f
Temperature switch, 103–104, 104f
Temporary end (TND) instruction, 201, 202f, 295–296, 296f
Test mode, 93
Thermocouples, 110–111, 110f
Thermostat, 103–104, 104f
Three-phase magnetic motor starter, 101–102, 102f
Three-pole magnetic contactor, 101f
Three-wire DC proximity sensor, 105, 105f
Through-beam scan technique, 108, 108f
Thumbwheel module, 31, 31f
Thumbwheel switches (TWS), 51, 53f, 224, 224f, 225f
Time base, 134, 136
Timed contact symbols, 132, 133f
Time-driven sequencer program, 259, 260–261, 261f, 262f
Time proportioning, 312, 312f
Timer file, 75
Timer function, and counter function, 174–177, 372, 372f
Timer number, 136
Timers, 82, 131–150, 147–148, 147–150f
advantages, 134
cascading timers, 147–148, 149f
CLX system, 358–366
cascading, 365, 366f
off-delay timer (TOF), 362–363, 362–363f
on-delay timer (TON), 359–362, 359–362f
predefined structure, 358–359, 358f
reciprocating, 365
retentive on-delay timer (RTO), 364–365, 364–365f
instructions, 134–135, 134–135f
mechanical timing relays, 132–134, 132f
off-delay timer instruction, 140–144, 141–143f
on-delay timer instruction, 135–138f, 135–140
quantities, 134–135
reciprocating, 147–148
retentive timer, 144–145f, 144–146
selection toolbar, 134f
Timer-timing (TT) bit, 136
Timing relays, 132, 132f
Title bar, 91
TOF. *see* Off-delay timer (TOF)
Token passing network, 320, 320f
TON. *see* On-delay timer (TON)
Traffic lights, control of
in one direction, 148–149, 149f
timing chart, 149f
in two directions, 149, 150f
Transducer, 28, 29f
Transmission media, 317, 317f
Transmitter, 28, 29f
Trend monitoring, 41–42, 42f
Troubleshooting, 3–4, 4f, 12, 290, 292–299
for discrete output module, 297, 298f
general methods, 297, 297f
input guide, 298f
input malfunctions, 292–293, 293f
ladder logic program, 294–299, 295–296f
output guide, 299f
output malfunctions, 294, 294f, 295f
processor module, 292, 292f
Truth tables, 62
NAND gate symbol and, 64, 64f
NOR gate symbol and, 64, 64f
XOR gate symbol and, 65, 65f
TTL module, 31
TTL (Transistor-Transistor-Logic) signals, 31
Turbine flowmeter, 111, 111f
24-hour clock program, 172
Twisted pairs, of wire, 317
Two-position control, 228
2's complement number, 49, 50t
Two-wire proximity sensor, 105, 105f

U

Ultrasonic sensors, 109–110, 109f
Underflow (UN) bit, 161
Unipolar modules, 28
Unlatch coil, 117
Unlatch instructions
CLX system, 352–353, 352–353f
Up-counter, 158, 159–166, 159f
alarm monitor program, 166f
coil-formatted instruction, 157–158, 157f
one-shot instruction, 162–166, 164f
one-shot rising (OSR) instruction, 165–166, 165f
Up-counter program, 223, 223f
Update accumulator (UA) bit, 161
Up/down-counter, 166–169, 167–170f
User-defined structure, CLX system, 341, 341f

V

Velocity sensors, 111, 111f
Verifying program errors, 294, 295f
Vertical contact
program with, 88, 88f
reprogrammed to eliminate, 88, 88f
Vertical scan, 81, 81f
Vessel filling operation, 223–224
Vessel overfill alarm program, 238f
Volatile memory, 37
Voltage sensing, 27
Voltage variations, 287–288, 288f

W

Watchdog timer, 292
Weight sensors, 110, 110f
Windows environment, 91
Windows toolbar, 91
Wire connectors, FBD, 385–386, 386f
Wire identification sleeves, heat-shrinkable, 285f
Wireless Wi-Fi Ethernet networks, 317
Wire routing
guidelines for, 284–285
heat-shrinkable wire identification sleeves, 285, 285f
Wires/wiring
FBD, 385, 386f
of stop buttons, 200, 200f
Wiring connections
input/output (I/O) system, 9, 9f
process control scheme, 10–11, 10f
Word level addressing, 19, 20f
Word level logic instructions, 70–71, 70–71f, 70t
Word-level math instructions, 242–244. *see also* Math instructions
Words, 47–48, 208, 208f
Word shift operations, 272–276
Word-to-file moves, 211
FAL instruction and, 214, 214f
Wraparound operation
BSL instruction and, 267–268, 269f
Writing, 36
Writing over the existing data, 208

X

XOR (exclusive-OR) function, 65, 65f, 208